

IsarMathLib

Ślawomir Kołodziej, Daniel de la Concepción Sáez

June 13, 2026

Abstract

This is the proof document of the IsarMathLib project version 1.35.0. IsarMathLib is a library of formalized mathematics for Isabelle2025-1 (ZF logic).

Contents

1	Introduction to the IsarMathLib project	14
1.1	How to read IsarMathLib proofs - a tutorial	14
1.2	Overview of the project	15
2	First Order Logic	19
2.1	Notions and lemmas in FOL	19
3	ZF set theory basics	21
3.1	Lemmas in Zermelo-Fraenkel set theory	21
4	Natural numbers in IsarMathLib	29
4.1	Induction	30
4.2	Simplification rules for addition and subtraction of natural numbers	35
4.3	Intervals	35
5	Order relations - introduction	36
5.1	Definitions	37
5.2	Intervals	42
5.3	Bounded sets	43
6	More on order relations	46
6.1	Definitions and basic properties	46
6.2	Properties of (strict) total orders	47

7	Even more on order relations	48
7.1	Maximum and minimum of a set	48
7.2	Supremum and Infimum	53
7.3	Strict versions of order relations	58
8	Order on natural numbers	60
8.1	Order on natural numbers	60
9	Functions - introduction	61
9.1	Properties of functions, function spaces and (inverse) images.	61
9.2	Dependent function space	71
9.3	Functions restricted to a set	72
9.4	Constant functions	74
9.5	Injections, surjections, bijections etc.	75
9.6	Functions of two variables	79
10	Binary operations	81
10.1	Lifting operations to a function space	81
10.2	Associative and commutative operations	82
10.3	Restricting operations	83
10.4	Compositions	85
10.5	Identity function	86
10.6	Lifting to subsets	86
10.7	Distributive operations	88
11	More on functions	89
11.1	Functions and order	89
11.2	Functions in cartesian products	91
11.3	Induced relations and order isomorphisms	92
12	Finite sets - introduction	94
12.1	Definition and basic properties of finite powerset	95
13	Semilattices and Lattices	100
13.1	Semilattices	100
13.2	Lattices	103
13.3	Complete lattices	103
14	Finite sets	104
14.1	Finite powerset	104
14.2	Finite range functions	108
15	Finite sets 1	109
15.1	Finite vs. bounded sets	109

16 Finite sets and order relations	111
16.1 Finite vs. bounded sets	111
16.2 Order isomorphisms of finite sets	113
17 Cardinal numbers	115
17.1 Some new ideas on cardinals	115
17.2 Main result on cardinals (without the Axiom of Choice) . . .	116
17.3 Choice axioms	117
17.4 Finite choice	118
17.5 Pigeonhole principle	118
17.6 Countability vs enumerability	119
18 Finite choice and order relations	120
18.1 Finite choice and preorders	120
19 Equivalence relations	120
19.1 Congruent functions and projections on the quotient	121
19.2 Projecting commutative, associative and distributive operations.	124
19.3 Saturated sets	125
20 Finite sequences	127
20.1 Lists as finite sequences	127
20.2 Lists and cartesian products	135
20.3 Chains	136
21 Formal languages	137
21.1 Introduction	137
21.2 Deterministic Finite Automata	138
21.3 Operations on regular languages	142
21.4 Non-deterministic finite state automata	143
21.5 Equivalence of Non-deterministic and Deterministic Finite State Automata	145
21.6 Epsilon-NFSA languages are regular	146
21.7 Concatenation of regular languages	149
22 Inductive sequences	156
22.1 Sequences defined by induction	156
22.2 Images of inductive sequences	159
22.3 Subsets generated by a binary operation	159
22.4 Inductive sequences with changing generating function	160
22.5 The Pascal's triangle	163
22.6 Monotonic sequences	166

23 Enumerations	167
23.1 Enumerations: definition and notation	168
23.2 Properties of enumerations	168
24 Folding in ZF	169
24.1 Folding in ZF	170
25 Partitions of sets	173
25.1 Bisections	173
25.2 Partitions	174
25.3 Collections of pairwise disjoint sets	175
26 Quasigroups	176
26.1 Definitions and notation	177
27 Loops	179
27.1 Definitions and notation	179
27.2 Product of a list of loop elements	180
28 Ordered loops	181
28.1 Definition, notation and basic properties	182
28.2 Completeness vs Archimedean property	187
29 Semigroups	187
29.1 Products of sequences of semigroup elements	188
29.2 Products over sets of indices	190
29.3 Commutative semigroups	192
30 Commutative Semigroups	195
30.1 Sum of a function over a set	195
31 Monoids	197
31.1 Definition and basic properties	197
32 Summing lists in a monoid	200
32.1 Notation and basic properties of sums of lists of monoid elements	200
32.2 Multiplying monoid elements by natural numbers	202
32.3 Chains weighted in a monoid	203
33 Groups - introduction	204
33.1 Definition and basic properties of groups	204
33.2 Subgroups	212
33.3 Groups vs. loops	216
33.4 Product of a list of group elements	216

33.5	An alternative definition of groups	217
34	Groups 1	219
34.1	Translations	219
34.2	Odd functions	223
34.3	Subgroups and interval arithmetic	223
35	Abelian Group	225
35.1	Rearrangement formulae	225
36	Direct product	230
36.1	Definition	230
36.2	Associative and commutative operations	231
37	Direct product of groups	232
37.1	Neutral element and inverse of the direct product	232
37.2	The direct product of groups is a group	232
38	Groups 2	232
38.1	Lifting groups to function spaces	233
38.2	Equivalence relations on groups	235
38.3	Normal subgroups and quotient groups	237
38.4	Function spaces as monoids	240
38.5	Homomorphisms	240
39	Almost homomorphisms foundation of the real number construction	242
39.1	Group valued finite range functions	242
39.2	Almost homomorphisms	243
39.3	The classes of almost homomorphisms	248
39.4	Compositions of almost homomorphisms	249
39.5	Shifting almost homomorphisms	253
40	Ordered groups - introduction	253
40.1	Ordered groups	254
40.2	Inequalities	259
40.3	The set of positive elements	265
40.4	Intervals and bounded sets	268
41	More on ordered groups	270
41.1	Absolute value and the triangle inequality	270
41.2	Maximum absolute value of a set	276
41.3	Alternative definitions	277
41.4	Odd Extensions	279
41.5	Functions with infinite limits	280

42 Rings - introduction	281
42.1 Definition and basic properties	281
42.2 Rearrangement lemmas	287
43 Binomial theorem	288
43.1 Sums of multiplicities of powers of ring elements and binomial theorem	289
44 Ring of almost homomorphisms the ring of integers as a quotient	292
44.1 The ring of classes of almost homomorphisms	292
45 Ordered rings	294
45.1 Definition and notation	294
45.2 Absolute value for ordered rings	299
45.3 Positivity in ordered rings	301
46 Normal subgroups, simple groups and Lagrange's theorem	304
46.1 Conjugation of subgroups	305
46.2 Simple groups	306
46.3 Finite groups	306
46.4 Subgroups generated by sets	307
47 Endomorphisms of abelian groups	308
47.1 First ring of endomorphisms of an abelian group	308
48 Group homomorphisms and kernels	310
48.1 The group homomorphism locale	310
48.2 Exact sequences at a term	313
48.3 The trivial group and zero maps	314
48.4 Short exact sequences	316
48.5 The cokernel	317
49 Rings - Ideals	318
49.1 Ideals	318
49.2 Ring quotient	323
50 Rings - Ideals of quotient rings	326
50.1 Ring homomorphisms	326
50.2 Quotient ring with quotient map	332
50.3 Quotient ideals	333
50.4 Subrings	335
50.5 First isomorphism theorem	336

51 Rings - Commutative Rings	336
51.1 Commutative rings	336
51.2 Prime ideals	337
52 Rings - Direct Product and Chinese Remainder Theorem	338
52.1 Direct product of rings	338
52.2 Chinese Remainder Theorem	338
53 Fields - introduction	339
53.1 Definition and basic properties	339
53.2 Equations and identities	341
53.3 $1/0=0$	342
54 Modules	342
54.1 Definition and basic properties of modules	343
54.2 Module axioms	346
54.3 Linear Combinations on Modules	347
54.3.1 Adding linear combinations	349
54.3.2 Linear dependency	350
54.4 Submodule	350
54.4.1 Spans	351
55 Ideals as modules connecting ring and module theories	352
55.1 Ideals as Modules	352
55.2 Annihilators	353
56 Vector spaces	354
56.1 Definition and basic properties of vector spaces	354
56.2 Vector space axioms	356
57 Ordered fields	357
57.1 Definition and basic properties	357
57.2 Inequalities	360
57.3 Definition of real numbers	362
58 Integers - introduction	362
58.1 Addition and multiplication as ZF-functions.	363
58.2 Integers as an ordered group	368
58.3 Induction on integers.	377
58.4 Bounded vs. finite subsets of integers	379
58.5 Addition on integers in terms of magnitudes	380

59 Integers 1	383
59.1 Integers as a ring	383
59.2 Rearrangement lemmas	385
59.3 Integers as an ordered ring	389
59.4 Maximum and minimum of a set of integers	395
59.5 The set of nonnegative integers	397
59.6 Functions with infinite limits	401
59.7 Miscellaneous	403
60 Division on integers	403
60.1 Quotient and remainder	404
61 Integer division: three as a modulus	405
61.1 Properties of three as a modulus	405
62 Chain complexes of abelian groups	406
62.1 Chain maps	408
63 Homology groups	409
63.1 The n -th homology group	410
63.2 Chain maps induce homomorphisms on homology	411
64 Exact sequences	411
64.1 Short exact sequences of chain complexes	412
64.2 The connecting homomorphism	413
64.3 The long exact sequence as a chain complex	416
64.4 The snake lemma	418
64.5 The snake connecting homomorphism	421
65 Integers 2	422
65.1 Slopes	423
65.2 Composing slopes	432
66 Integers 3	434
66.1 Positive slopes	434
66.2 Inverting slopes	437
66.3 Completeness	440
67 Integer powers of group elements	441
67.1 Properties of natural powers of an element and its inverse . .	442
67.2 Integer powers	443
68 $\mathbb{Z}$ modules	448
68.1 Integer powers as the only integer actions on abelian groups .	448

69 Construction real numbers - the generic part	449
69.1 The definition of real numbers	450
70 Construction of real numbers	455
70.1 Definitions and notation	455
70.2 Multiplication of real numbers	458
70.3 The order on reals	460
70.4 Inverting reals	466
70.5 Completeness	468
71 Topology - introduction	475
71.1 Basic definitions and properties	475
71.2 Interior of a set	478
71.3 Closed sets, closure, boundary.	479
72 Topology 1	481
72.1 Separation axioms	482
72.2 Bases and subbases	484
72.3 Product topology	486
72.4 Hausdorff spaces	488
72.5 Compact sets are closed in Hausdorff spaces	488
73 Topology 2	489
73.1 Continuous functions.	490
73.2 Homeomorphisms	493
73.3 Topologies induced by mappings	494
73.4 Partial functions and continuity	494
73.5 Product topology and continuity	496
73.6 Pasting lemma	498
74 Topology 4	499
74.1 Nets	499
74.2 Filters	501
75 Topology and neighborhoods	504
75.1 Neighborhood systems	504
75.2 From a neighborhood system to topology	505
75.3 From a topology to a neighborhood system	505
75.4 Neighborhood systems are 1:1 with topologies	506
75.5 Set neighborhoods	507
76 Uniform spaces	509
76.1 Entourages and neighborhoods	509
76.2 Base of a uniformity	515
76.3 Uniformly continuous functions	517

77 Metric spaces	517
77.1 Pseudometric - definition and basic properties	518
77.2 Uniform structures on (pseudo-)metric spaces	522
78 Basic properties of real numbers	526
78.1 Basic notation for real numbers	526
79 Complex numbers	530
79.1 From complete ordered fields to complex numbers	530
79.2 Axioms of complex numbers	534
80 Rings - Zariski Topology	539
80.1 Spectrum	539
80.2 Zariski topology	540
80.3 Closed sets and boundary	541
80.4 Properties of Spec	542
81 Ring homomorphisms as continuous maps on Spec	543
81.1 Induced maps on spectra	543
81.2 Continuity of the induced map	543
82 Topology 3	544
82.1 The base of the product topology	545
82.2 Finite product of topologies	546
83 Relation between nets and filters	547
83.1 Relation between nets and filters	548
84 Topology - examples	550
84.1 CoCardinal Topology	550
84.2 Total set, Closed sets, Interior, Closure and Boundary	551
84.3 Excluded Set Topology	553
84.4 Total set, closed sets, interior, closure and boundary	553
84.5 Special cases and subspaces	554
84.6 Included Set Topology	554
84.7 Basic topological notions in included set topology	555
84.8 Special cases and subspaces	556
85 More examples in topology	556
85.1 New ideas using a base for a topology	557
85.2 The topology of a base	557
85.3 Dual Base for Closed Sets	558
85.4 Partition topology	558
85.5 Partition topology is a topology.	559
85.6 Total set, Closed sets, Interior, Closure and Boundary	559

85.7	Special cases and subspaces	560
85.8	Order topologies	561
85.9	Order topology is a topology	561
85.10	Total set	562
85.11	Right order and Left order topologies.	563
85.11.1	Right and Left Order topologies are topologies	563
85.11.2	Total set	564
85.12	Union of Topologies	564
86	Properties in Topology	565
86.1	Properties of compactness	565
86.2	Properties of numerability	566
86.3	Relations between numerability properties and choice principles	567
86.4	Relation between numerability and compactness	568
87	Topology 5	569
87.1	Some results for separation axioms	569
87.2	Hereditability	572
87.3	Spectrum and anti-properties	573
88	Topology 6	577
88.1	Image filter	577
88.2	Continuous at a point vs. globally continuous	577
88.3	Continuous functions and filters	578
89	Topology 7	578
89.1	Connection Properties	578
90	Topology 8	582
90.1	Definition of quotient topology	583
90.2	Quotient topologies from equivalence relations	583
91	Homeomorphisms as a group connecting topology and group theory	586
91.1	Group of homeomorphisms	586
91.2	Examples computed	587
91.3	Properties preserved by functions	588
92	Topology 10	589
92.1	Closure and closed sets in product space	589
92.2	Separation properties in product space	589
92.3	Connection properties in product space	590

93 Order topology connecting order theory and topology	590
93.1 Order topologies	591
93.2 Separation properties	591
93.3 Connectedness properties	591
93.4 Numerability axioms	593
94 Properties in topology 2	594
94.1 Local properties.	594
94.2 First examples	595
94.3 Local compactness	595
94.4 Compactification by one point	596
94.5 Hereditary properties and local properties	598
94.6 Locally-regular spaces	601
95 Properties in Topology 3	601
95.1 More anti-properties	602
95.2 First examples	602
95.3 Structural results	602
95.4 More Separation properties	604
95.5 Definitions	604
95.6 First results	604
95.7 Counter-examples	605
95.8 Other types of properties	608
95.9 Definitions	608
95.10 First examples	609
95.11 Structural results	610
96 Ultrafilters	611
97 Ultraproduct construction	612
97.1 Internal sets	613
98 Internal functions in ultraproduct construction	615
98.1 Internal functions	615
99 Hypernatural numbers	617
99.1 Auxiliary results on finite sets	617
99.2 Ultraproduct of the naturals	617
99.3 Transfer principle instances	619
99.4 Arithmetic of hypernatural numbers	620
100 Alternative definitions of uniformity	621
100.1 Uniform covers	621

101	Set of uniformities as a complete lattice	626
101.1	Uniformities on a fixed set as a partially ordered set	626
101.2	Least upper bound of a set of uniformities	627
101.3	Greatest lower bound of a set of uniformities	630
102	Real valued metric spaces	631
102.1	Real valued metric spaces: context and notation	632
102.2	Real valued metric spaces are Hausdorff as topological spaces	633
102.3	Real valued (pseudo)metric spaces as uniform spaces	634
103	Uniformity defined by a collection of pseudometrics	635
103.1	Uniformity defined by a collection of pseudometrics	635
103.2	Uniformity defined by a collection of pseudometrics - an al- ternative approach	637
103.3	Halving sequences of entourages	640
104	Topological groups - introduction	646
104.1	Topological group: definition and notation	646
104.2	Interval arithmetic, translations and inverse of set	650
104.3	Neighborhoods of zero	653
104.4	Closure in topological groups	654
104.5	Sums of sequences of elements and subsets	655
105	Topological groups 1	655
105.1	Separation properties of topological groups	656
105.2	Existence of nice neighbourhoods.	656
105.3	Rest of separation axioms	656
105.4	Local properties	657
106	Topological groups 2	657
106.1	Subgroups topologies	658
106.2	Quotient topological groups	659
107	Uniform structures on topological groups	660
107.1	Natural uniformities in topological groups: definitions and notation	660
108	Two versions of Tarski's Axiom	661
108.1	Two versions of the Tarski's axiom	661
109	Metamath introduction	662
109.1	Importing from Metamath - how is it done	662
109.2	The context for Metamath theorems	663

110	Logic and sets in Metamatah	666
110.1	Basic Metamath theorems	666
111	Complex numbers in Metamatah - introduction	716
112	Metamath examples	739
113	Metamath interface	740
113.1	MMisar0 and complex0 contexts.	741
114	Metamath sampler	741
114.1	Extended reals and order	742
114.2	Natural real numbers	742
114.3	Infimum and supremum in real numbers	743

1 Introduction to the IsarMathLib project

theory Introduction imports ZF.equalities

begin

This theory does not contain any formalized mathematics used in other theories, but is an introduction to IsarMathLib project.

1.1 How to read IsarMathLib proofs - a tutorial

Isar (the Isabelle’s formal proof language) was designed to be similar to the standard language of mathematics. Any person able to read proofs in a typical mathematical paper should be able to read and understand Isar proofs without having to learn a special proof language. However, Isar is a formal proof language and as such it does contain a couple of constructs whose meaning is hard to guess. In this tutorial we will define a notion and prove an example theorem about that notion, explaining Isar syntax along the way. This tutorial may also serve as a style guide for IsarMathLib contributors. Note that this tutorial aims to help in reading the presentation of the Isar language that is used in IsarMathLib proof document and HTML rendering on the FormalMath.org site, but does not teach how to write proofs that can be verified by Isabelle. This presentation is different than the source processed by Isabelle (the concept that the source and presentation look different should be familiar to any LaTeX user). To learn how to write Isar proofs one needs to study the source of this tutorial as well.

The first thing that mathematicians typically do is to define notions. In Isar this is done with the **definition** keyword. In our case we define a notion of two sets being disjoint. We will use the infix notation, i.e. the string **is**

`disjoint with` put between two sets to denote our notion of disjointness. The left side of the $\equiv$ symbol is the notion being defined, the right side says how we define it. In Isabelle/ZF 0 is used to denote both zero (of natural numbers) and the empty set, which is not surprising as those two things are the same in set theory.

definition

```
AreDisjoint (infix {is disjoint with} 90) where
  A {is disjoint with} B  $\equiv$   $A \cap B = 0$ 
```

We are ready to prove a theorem. Here we show that the relation of being disjoint is symmetric. We start with one of the keywords "theorem", "lemma" or "corollary". In Isar they are synonymous. Then we provide a name for the theorem. In standard mathematics theorems are numbered. In Isar we can do that too, but it is considered better to give theorems meaningful names. After the "shows" keyword we give the statement to show. The $\longleftrightarrow$ symbol denotes the equivalence in Isabelle/ZF. Here we want to show that "A is disjoint with B iff and only if B is disjoint with A". To prove this fact we show two implications - the first one that `A {is disjoint with} B` implies `B {is disjoint with} A` and then the converse one. Each of these implications is formulated as a statement to be proved and then proved in a subproof like a mini-theorem. Each subproof uses a proof block to show the implication. Proof blocks are delimited with curly brackets in Isar. Proof block is one of the constructs that does not exist in informal mathematics, so it may be confusing. When reading a proof containing a proof block I suggest to focus first on what is that we are proving in it. This can be done by looking at the first line or two of the block and then at the last statement. In our case the block starts with "assume `A {is disjoint with} B` and the last statement is "then have `B {is disjoint with} A`". It is a typical pattern when someone needs to prove an implication: one assumes the antecedent and then shows that the consequent follows from this assumption. Implications are denoted with the $\longrightarrow$ symbol in Isabelle. After we prove both implications we collect them using the "moreover" construct. The keyword "ultimately" indicates that what follows is the conclusion of the statements collected with "moreover". The "show" keyword is like "have", except that it indicates that we have arrived at the claim of the theorem (or a subproof).

theorem `disjointness_symmetric:`

```
  shows A {is disjoint with} B  $\longleftrightarrow$  B {is disjoint with} A
  <proof>
```

1.2 Overview of the project

The `Fo11`, `ZF1` and `Nat_ZF_IML` theory files contain some background material that is needed for the remaining theories.

`Order_ZF` and `Order_ZF_1a` reformulate material from standard Isabelle's

`Order` theory in terms of non-strict (less-or-equal) order relations. `Order_ZF_1` on the other hand directly continues the `Order` theory file using strict order relations (less and not equal). This is useful for translating theorems from Metamath.

In `NatOrder_ZF` we prove that the usual order on natural numbers is linear. The `func1` theory provides basic facts about functions. `func_ZF` continues this development with more advanced topics that relate to algebraic properties of binary operations, like lifting a binary operation to a function space, associative, commutative and distributive operations and properties of functions related to order relations. `func_ZF_1` is about properties of functions related to order relations.

The standard Isabelle's `Finite` theory defines the finite powerset of a set as a certain "datatype" (?) with some recursive properties. IsarMathLib's `Finite1` and `Finite_ZF_1` theories develop more facts about this notion. These two theories are obsolete now. They will be gradually replaced by an approach based on set theory rather than tools specific to Isabelle. This approach is presented in `Finite_ZF` theory file.

In `FinOrd_ZF` we talk about ordered finite sets.

The `EquivClass1` theory file is a reformulation of the material in the standard Isabelle's `EquivClass` theory in the spirit of ZF set theory.

`FiniteSeq_ZF` discusses the notion of finite sequences (a.k.a. lists).

`InductiveSeq_ZF` provides the definition and properties of (what is known in basic calculus as) sequences defined by induction, i. e. by a formula of the form $a_0 = x$, $a_{n+1} = f(a_n)$.

`Fold_ZF` shows how the familiar from functional programming notion of fold can be interpreted in set theory.

`Partitions_ZF` is about splitting a set into non-overlapping subsets. This is a common trick in proofs.

`Semigroup_ZF` treats the expressions of the form $a_0 \cdot a_1 \cdot \dots \cdot a_n$, (i.e. products of finite sequences), where $\cdot$ is an associative binary operation.

`CommutativeSemigroup_ZF` is another take on a similar subject. This time we consider the case when the operation is commutative and the result of depends only on the set of elements we are summing (additively speaking), but not the order.

The `Topology_ZF` series covers basics of general topology: interior, closure, boundary, compact sets, separation axioms and continuous functions.

`Group_ZF`, `Group_ZF_1`, `Group_ZF_1b` and `Group_ZF_2` provide basic facts of the group theory. `Group_ZF_3` considers the notion of almost homomorphisms that is needed for the real numbers construction in `Real_ZF`.

The `TopologicalGroup` connects the `Topology_ZF` and `Group_ZF` series and starts the subject of topological groups with some basic definitions and facts.

In `DirectProduct_ZF` we define direct product of groups and show some its basic properties.

The `OrderedGroup_ZF` theory treats ordered groups. This is a suprisingly large theory for such relatively obscure topic.

`Ring_ZF` defines rings. `Ring_ZF_1` covers the properties of rings that are specific to the real numbers construction in `Real_ZF`.

The `OrderedRing_ZF` theory looks at the consequences of adding a linear order to the ring algebraic structure.

`Field_ZF` and `OrderedField_ZF` contain basic facts about (you guessed it) fields and ordered fields.

`Int_ZF_IML` theory considers the integers as a monoid (multiplication) and an abelian ordered group (addition). In `Int_ZF_1` we show that integers form a commutative ring. `Int_ZF_2` contains some facts about slopes (almost homomorphisms on integers) needed for real numbers construction, used in `Real_ZF_1`.

In the `IntDiv_ZF_IML` theory we translate some properties of the integer quotient and remainder functions studied in the standard Isabelle's `IntDiv_ZF` theory to the notation used in `IsarMathLib`.

The `Real_ZF` and `Real_ZF_1` theories contain the construction of real numbers based on the paper [2] by R. D. Arthan (not Cauchy sequences, not Dedekind sections). The heavy lifting is done mostly in `Group_ZF_3`, `Ring_ZF_1` and `Int_ZF_2`. `Real_ZF` contains the part of the construction that can be done starting from generic abelian groups (rather than additive group of integers). This allows to show that real numbers form a ring. `Real_ZF_1` continues the construction using properties specific to the integers and showing that real numbers constructed this way form a complete ordered field.

`Cardinal_ZF` provides a couple of theorems about cardinals that are mostly used for studying properties of topological properties (yes, this is kind of meta). The main result (proven without AC) is that if two sets can be injectively mapped into an infinite cardinal, then so can be their union. There is also a definition of the Axiom of Choice specific for a given cardinal (so that the choice function exists for families of sets of given cardinality). Some properties are proven for such predicates, like that for finite families of sets the choice function always exists (in ZF) and that the axiom of choice for a larger cardinal implies one for a smaller cardinal.

`Group_ZF_4` considers conjugate of subgroup and defines simple groups. A nice theorem here is that endomorphisms of an abelian group form a ring. The first isomorphism theorem (a group homomorphism h induces an isomorphism between the group divided by the kernel of h and the image of h) is proven.

Turns out given a property of a topological space one can define a local version of a property in general. This is studied in the `Topology_ZF_properties_2`

theory and applied to local versions of the property of being finite or compact or Hausdorff (i.e. locally finite, locally compact, locally Hausdorff). There are a couple of nice applications, like one-point compactification that allows to show that every locally compact Hausdorff space is regular. Also there are some results on the interplay between hereditability of a property and local properties.

For a given surjection $f : X \rightarrow Y$, where X is a topological space one can consider the weakest topology on Y which makes f continuous, let's call it a quotient topology generated by f . The quotient topology generated by an equivalence relation r on X is actually a special case of this setup, where f is the natural projection of X on the quotient X/r . The properties of these two ways of getting new topologies are studied in `Topology_ZF_8` theory. The main result is that any quotient topology generated by a function is homeomorphic to a topology given by an equivalence relation, so these two approaches to quotient topologies are kind of equivalent.

As we all know, automorphisms of a topological space form a group. This fact is proven in `Topology_ZF_9` and the automorphism groups for co-cardinal, included-set, and excluded-set topologies are identified. For order topologies it is shown that order isomorphisms are homeomorphisms of the topology induced by the order. Properties preserved by continuous functions are studied and as an application it is shown for example that quotient topological spaces of compact (or connected) spaces are compact (or connected, resp.)

The `Topology_ZF_10` theory is about products of two topological spaces. It is proven that if two spaces are T_0 (or T_1 , T_2 , regular, connected) then their product is as well.

Given a total order on a set one can define a natural topology on it generated by taking the rays and intervals as the base. The `Topology_ZF_11` theory studies relations between the order and various properties of generated topology. For example one can show that if the order topology is connected, then the order is complete (in the sense that for each set bounded from above the set of upper bounds has a minimum). For a given cardinal κ we can consider generalized notion of κ -separability. Turns out κ -separability is related to (order) density of sets of cardinality κ for order topologies.

Being a topological group imposes additional structure on the topology of the group, in particular its separation properties. In `Topological_Group_ZF_1.thy` theory it is shown that if a topology is T_0 , then it must be T_3 , and that the topology in a topological group is always regular.

For a given normal subgroup of a topological group we can define a topology on the quotient group in a natural way. At the end of the `Topological_Group_ZF_2.thy` theory it is shown that such topology on the quotient group makes it a topological group.

The `Topological_Group_ZF_3.thy` theory studies the topologies on subgroups

of a topological group. A couple of nice basic properties are shown, like that the closure of a subgroup is a subgroup, closure of a normal subgroup is normal and, a bit more surprising (to me) property that every locally-compact subgroup of a T_0 group is closed.

In `Complex_ZF` we construct complex numbers starting from a complete ordered field (a model of real numbers). We also define the notation for writing about complex numbers and prove that the structure of complex numbers constructed there satisfies the axioms of complex numbers used in Metamath.

`MMI_prelude` defines the `mmisar0` context in which most theorems translated from Metamath are proven. It also contains a chapter explaining how the translation works.

In the `Metamath_interface` theory we prove a theorem that the `mmisar0` context is valid (can be used) in the `complex0` context. All theories using the translated results will import the `Metamath_interface` theory. The `Metamath_sampler` theory provides some examples of using the translated theorems in the `complex0` context.

The theories `MMI_logic_and_sets`, `MMI_Complex`, `MMI_Complex_1` and `MMI_Complex_2` contain the theorems imported from the Metamath's `set.mm` database. As the translated proofs are rather verbose these theories are not printed in this proof document. The full list of translated facts can be found in the `Metamath_theorems.txt` file included in the `IsarMathLib` distribution. The `MMI_examples` provides some theorems imported from Metamath that are printed in this proof document as examples of how translated proofs look like.

end

2 First Order Logic

theory Fol1 imports ZF.Tranc1

begin

Isabelle/ZF builds on the first order logic. Almost everything one would like to have in this area is covered in the standard Isabelle libraries. The material in this theory provides some lemmas that are missing or allow for a more readable proof style.

2.1 Notions and lemmas in FOL

This section contains mostly shortcuts and workarounds that allow to use more readable coding style.

The next lemma serves as a workaround to problems with applying the

definition of transitivity (of a relation) in our coding style (any attempt to do something like using `trans_def` puts Isabelle in an infinite loop).

```
lemma Fol1_L2: assumes
  A1:  $\forall x y z. \langle x, y \rangle \in r \wedge \langle y, z \rangle \in r \longrightarrow \langle x, z \rangle \in r$ 
  shows trans(r)
<proof>
```

Another workaround for the problem of Isabelle simplifier looping when the transitivity definition is used.

```
lemma Fol1_L3: assumes A1: trans(r) and A2:  $\langle a, b \rangle \in r \wedge \langle b, c \rangle \in r$ 
  shows  $\langle a, c \rangle \in r$ 
<proof>
```

There is a problem with application of the definition of asymmetry for relations. The next lemma is a workaround.

```
lemma Fol1_L4:
  assumes A1: antisym(r) and A2:  $\langle a, b \rangle \in r \quad \langle b, a \rangle \in r$ 
  shows a=b
<proof>
```

The definition below implements a common idiom that states that (perhaps under some assumptions) exactly one of given three statements is true.

```
definition
  Exactly_1_of_3_holds(p,q,r)  $\equiv$ 
   $(p \vee q \vee r) \wedge (p \longrightarrow \neg q \wedge \neg r) \wedge (q \longrightarrow \neg p \wedge \neg r) \wedge (r \longrightarrow \neg p \wedge \neg q)$ 
```

The next lemma allows to prove statements of the form `Exactly_1_of_3_holds(p,q,r)`.

```
lemma Fol1_L5:
  assumes p  $\vee$  q  $\vee$  r
  and p  $\longrightarrow$   $\neg$ q  $\wedge$   $\neg$ r
  and q  $\longrightarrow$   $\neg$ p  $\wedge$   $\neg$ r
  and r  $\longrightarrow$   $\neg$ p  $\wedge$   $\neg$ q
  shows Exactly_1_of_3_holds(p,q,r)
<proof>
```

If exactly one of p, q, r holds and p is not true, then q or r .

```
lemma Fol1_L6:
  assumes A1:  $\neg p$  and A2: Exactly_1_of_3_holds(p,q,r)
  shows q  $\vee$  r
<proof>
```

If exactly one of p, q, r holds and q is true, then r can not be true.

```
lemma Fol1_L7:
  assumes A1: q and A2: Exactly_1_of_3_holds(p,q,r)
  shows  $\neg r$ 
<proof>
```

The next lemma demonstrates an elegant form of the `Exactly_1_of_3_holds(p,q,r)` predicate.

lemma `Fol1_L8:`

shows `Exactly_1_of_3_holds(p,q,r)  $\longleftrightarrow$  (p $\longleftrightarrow$ q $\longleftrightarrow$ r)  $\wedge$   $\neg$ (p $\wedge$ q $\wedge$ r)`
`<proof>`

A property of the `Exactly_1_of_3_holds` predicate.

lemma `Fol1_L8A: assumes A1: Exactly_1_of_3_holds(p,q,r)`

shows `p  $\longleftrightarrow$   $\neg$ (q  $\vee$  r)`
`<proof>`

Exclusive or definition. There is one also defined in the standard Isabelle, denoted `xor`, but it relates to boolean values, which are sets. Here we define a logical functor.

definition

`Xor (infixl Xor 66) where`
`p Xor q  $\equiv$  (p $\vee$ q)  $\wedge$   $\neg$ (p  $\wedge$  q)`

The "exclusive or" is the same as negation of equivalence.

lemma `Fol1_L9: shows p Xor q  $\longleftrightarrow$   $\neg$ (p $\longleftrightarrow$ q)`

`<proof>`

Equivalence relations are symmetric.

lemma `equiv_is_sym: assumes A1: equiv(X,r) and A2:  $\langle x,y \rangle \in r$`

shows `$\langle y,x \rangle \in r$`
`<proof>`

end

3 ZF set theory basics

theory `ZF1 imports ZF.Perm`

begin

The standard Isabelle distribution contains lots of facts about basic set theory. This theory file adds some more.

3.1 Lemmas in Zermelo-Fraenkel set theory

Here we put lemmas from the set theory that we could not find in the standard Isabelle distribution or just so that they are easier to find.

In Isabelle/ZF the set difference is written with a minus sign $A - B$ because the standard backslash character is reserved for other purposes. The next

abbreviation declares that we want the set difference character $A \setminus B$ to be synonymous with the minus sign.

abbreviation set_difference (infixl $\setminus$ 65) where $A \setminus B \equiv A - B$

Complement of the complement is the set.

lemma diff_diff_eq: assumes $A \subseteq X$ shows $X \setminus (X \setminus A) = A$ *<proof>*

A set cannot be a member of itself. This is exactly lemma `mem_not_refl` from Isabelle/ZF `upair.thy`, we put it here for easy reference.

lemma mem_self: shows $x \notin x$ *<proof>*

If we remove an element and put it back we get the set back.

lemma rem_add_eq: assumes $a \in A$ shows $(A \setminus \{a\}) \cup \{a\} = A$ *<proof>*

Applying a transformation to equal values yields equal results.. It is surprising but we do have to use this as a rule in rare cases.

lemma same_constr: assumes $x = y$ shows $P(x) = P(y)$ *<proof>*

If one collection is contained in another, then we can say the same about their unions.

lemma collection_contain: assumes $A \subseteq B$ shows $\bigcup A \subseteq \bigcup B$ *<proof>*

In ZF set theory the zero of natural numbers is the same as the empty set. In the next abbreviation we declare that we want 0 and $\emptyset$ to be synonyms so that we can use $\emptyset$ instead of 0 when appropriate.

abbreviation empty_set ($\emptyset$) where $\emptyset \equiv 0$

If all sets of a nonempty collection are the same, then its union is the same.

lemma ZF1_1_L1: assumes $C \neq \emptyset$ and $\forall y \in C. b(y) = A$ shows $(\bigcup_{y \in C} b(y)) = A$ *<proof>*

The union of all values of a constant meta-function belongs to the same set as the constant.

lemma ZF1_1_L2: assumes $A1: C \neq \emptyset$ and $A2: \forall x \in C. b(x) \in A$ and $A3: \forall x y. x \in C \wedge y \in C \longrightarrow b(x) = b(y)$ shows $(\bigcup_{x \in C} b(x)) \in A$ *<proof>*

If two meta-functions are the same on a cartesian product, then the subsets defined by them are the same. I am surprised Isabelle can not handle this automatically.

lemma ZF1_1_L4: assumes $A1: \forall x \in X. \forall y \in Y. a(x, y) = b(x, y)$

shows $\{a(x,y). \langle x,y \rangle \in X \times Y\} = \{b(x,y). \langle x,y \rangle \in X \times Y\}$
 $\langle proof \rangle$

If two meta-functions are the same on a cartesian product, then the subsets defined by them are the same. This is similar to ZF1_1_L4, except that the set definition varies over $p \in X \times Y$ rather than $\langle x,y \rangle \in X \times Y$.

lemma ZF1_1_L4A: **assumes** A1: $\forall x \in X. \forall y \in Y. a(\langle x,y \rangle) = b(x,y)$
shows $\{a(p). p \in X \times Y\} = \{b(x,y). \langle x,y \rangle \in X \times Y\}$
 $\langle proof \rangle$

A lemma about inclusion in cartesian products. Included here to remember that we need the $U \times V \neq \emptyset$ assumption.

lemma prod_subset: **assumes** $U \times V \neq \emptyset$ $U \times V \subseteq X \times Y$ **shows** $U \subseteq X$ **and** $V \subseteq Y$
 $\langle proof \rangle$

A technical lemma about sections in cartesian products.

lemma section_proj: **assumes** $A \subseteq X \times Y$ **and** $U \times V \subseteq A$ **and** $x \in U$ $y \in V$
shows $U \subseteq \{t \in X. \langle t,y \rangle \in A\}$ **and** $V \subseteq \{t \in Y. \langle x,t \rangle \in A\}$
 $\langle proof \rangle$

If two meta-functions are the same on a set, then they define the same set by separation.

lemma ZF1_1_L4B: **assumes** $\forall x \in X. a(x) = b(x)$
shows $\{a(x). x \in X\} = \{b(x). x \in X\}$
 $\langle proof \rangle$

A set defined by a constant meta-function is a singleton.

lemma ZF1_1_L5: **assumes** $X \neq \emptyset$ **and** $\forall x \in X. b(x) = c$
shows $\{b(x). x \in X\} = \{c\}$ $\langle proof \rangle$

Most of the time, auto does this job, but there are strange cases when the next lemma is needed.

lemma subset_with_property: **assumes** $Y = \{x \in X. b(x)\}$
shows $Y \subseteq X$
 $\langle proof \rangle$

If set A is contained in set B and exist elements x, y of the set A that satisfy a predicate then exist elements of the set B that satisfy the predicate.

lemma exist2_subset: **assumes** $A \subseteq B$ **and** $\exists x \in A. \exists y \in A. \varphi(x,y)$
shows $\exists x \in B. \exists y \in B. \varphi(x,y)$
 $\langle proof \rangle$

We can choose an element from a nonempty set.

lemma nonempty_has_element: **assumes** $X \neq \emptyset$ **shows** $\exists x. x \in X$
 $\langle proof \rangle$

In Isabelle/ZF the intersection of an empty family is empty. This is exactly lemma `Inter_0` from Isabelle's `equalities` theory. We repeat this lemma here as it is very difficult to find. This is one reason we need comments before every theorem: so that we can search for keywords.

lemma `inter_empty_empty`: **shows** $\bigcap \emptyset = \emptyset$ *<proof>*

If an intersection of a collection is not empty, then the collection is not empty. We are (ab)using the fact the the intersection of empty collection is defined to be empty.

lemma `inter_nempty_nempty`: **assumes** $\bigcap A \neq \emptyset$ **shows** $A \neq \emptyset$ *<proof>*

For two collections S, T of sets we define the product collection as the collections of cartesian products $A \times B$, where $A \in S, B \in T$.

definition

$$\text{ProductCollection}(T, S) \equiv \bigcup_{U \in T}. \{U \times V. V \in S\}$$

The union of the product collection of collections S, T is the cartesian product of $\bigcup S$ and $\bigcup T$.

lemma `ZF1_1_L6`: **shows** $\bigcup \text{ProductCollection}(S, T) = \bigcup S \times \bigcup T$ *<proof>*

An intersection of subsets is a subset.

lemma `inter_subsets_subset`: **assumes** $\forall i \in I. P(i) \subseteq X$ **shows** $(\bigcap_{i \in I}. P(i)) \subseteq X$ *<proof>*

Intersection of a collection of subsets of X is a subset of X . Similar to `inter_subsets_subset` but for non-indexed collections.

lemma `inter_subsets_subset1`: **assumes** $M \subseteq \text{Pow}(X)$ **shows** $\bigcap M \subseteq X$ *<proof>*

Intersection of a smaller (but nonempty) collection of sets is larger. Note the assumption that the smaller collection is nonepty in necessary here.

lemma `inter_index_mono`: **assumes** $I \subseteq M$ $I \neq \emptyset$ **shows** $(\bigcap_{i \in M}. P(i)) \subseteq (\bigcap_{i \in I}. P(i))$ *<proof>*

Isabelle/ZF has a "THE" construct that allows to define an element if there is only one such that is satisfies given predicate. In pure ZF we can express something similar using the indentity proven below.

lemma `ZF1_1_L8`: **shows** $\bigcup \{x\} = x$ *<proof>*

Some properties of singletons.

lemma `ZF1_1_L9`: **assumes** $\exists ! x. x \in A \wedge \varphi(x)$

shows
 $\exists a. \{x \in A. \varphi(x)\} = \{a\}$
 $\bigcup \{x \in A. \varphi(x)\} \in A$
 $\varphi(\bigcup \{x \in A. \varphi(x)\})$
 $\exists x \in A. \varphi(x)$
<proof>

A simple version of ZF1_1_L9.

corollary singleton_extract: **assumes** $\exists! x. x \in A$
shows $(\bigcup A) \in A$
<proof>

A criterion for when a set defined by comprehension is a singleton.

lemma singleton_comprehension:
assumes A1: $y \in X$ **and** A2: $\forall x \in X. \forall y \in X. P(x) = P(y)$
shows $(\bigcup \{P(x). x \in X\}) = P(y)$
<proof>

Adding an element of a set to that set does not change the set.

lemma set_elem_add: **assumes** $x \in X$ **shows** $X \cup \{x\} = X$ *<proof>*

Here we define a restriction of a collection of sets to a given set. In romantic math this is typically denoted $X \cap M$ and means $\{X \cap A : A \in M\}$. Note there is also $\text{restrict}(f, A)$ defined for relations in ZF.thy.

definition
RestrictedTo (**infixl** {restricted to} 70) **where**
 $M \{\text{restricted to}\} X \equiv \{X \cap A . A \in M\}$

A lemma on a union of a restriction of a collection to a set.

lemma union_restrict:
shows $\bigcup (M \{\text{restricted to}\} X) = (\bigcup M) \cap X$
<proof>

Next we show a technical identity that is used to prove sufficiency of some condition for a collection of sets to be a base for a topology.

lemma ZF1_1_L10: **assumes** A1: $\forall U \in C. \exists A \in B. U = \bigcup A$
shows $\bigcup \bigcup \{\bigcup \{A \in B. U = \bigcup A\}. U \in C\} = \bigcup C$
<proof>

Standard Isabelle uses a notion of $\text{cons}(A, a)$ that can be thought of as $A \cup \{a\}$.

lemma consdef: **shows** $\text{cons}(a, A) = A \cup \{a\}$
<proof>

If a difference between a set and a singleton is empty, then the set is empty or it is equal to the singleton.

lemma singl_diff_empty: **assumes** $A \setminus \{x\} = \emptyset$

shows $A = \emptyset \vee A = \{x\}$
 $\langle proof \rangle$

If a difference between a set and a singleton is the set, then the only element of the singleton is not in the set.

lemma `singl_diff_eq`: **assumes** $A1: A \setminus \{x\} = A$
shows $x \notin A$
 $\langle proof \rangle$

Simple substitution in membership, has to be used by rule in very rare cases.

lemma `eq_mem`: **assumes** $x \in A$ **and** $y = x$ **shows** $y \in A$
 $\langle proof \rangle$

A basic property of sets defined by comprehension.

lemma `comprehension`: **assumes** $a \in \{x \in X. p(x)\}$
shows $a \in X$ **and** $p(a)$ $\langle proof \rangle$

A basic property of a set defined by another type of comprehension.

lemma `comprehension_repl`: **assumes** $y \in \{p(x). x \in X\}$
shows $\exists x \in X. y = p(x)$ $\langle proof \rangle$

The inverse of the comprehension lemma.

lemma `mem_cond_in_set`: **assumes** $\varphi(c)$ **and** $c \in X$
shows $c \in \{x \in X. \varphi(x)\}$ $\langle proof \rangle$

The image of a set by a greater relation is greater.

lemma `image_rel_mono`: **assumes** $r \subseteq s$ **shows** $r(A) \subseteq s(A)$
 $\langle proof \rangle$

A technical lemma about relations: if x is in its image by a relation U and that image is contained in some set C , then the image of the singleton $\{x\}$ by the relation $U \cup C \times C$ equals C .

lemma `image_greater_rel`:
assumes $x \in U\{x\}$ **and** $U\{x\} \subseteq C$
shows $(U \cup C \times C)\{x\} = C$
 $\langle proof \rangle$

Reformulation of the definition of composition of two relations:

lemma `rel_compdef`:
shows $\langle x, z \rangle \in r \circ s \iff (\exists y. \langle x, y \rangle \in s \wedge \langle y, z \rangle \in r)$
 $\langle proof \rangle$

Domain and range of the relation of the form $\bigcup \{U \times U : U \in P\}$ is $\bigcup P$:

lemma `domain_range_sym`: **shows** $\text{domain}(\bigcup \{U \times U. U \in P\}) = \bigcup P$ **and** $\text{range}(\bigcup \{U \times U. U \in P\}) = \bigcup P$
 $\langle proof \rangle$

The product of converses is equal to in converse of a product

lemma prod_converse: **assumes** $M \subseteq \text{Pow}(X \times X)$
shows $\bigcap \{\text{converse}(A) \mid A \in M\} = \text{converse}(\bigcap M)$
<proof>

An identity for the square (in the sense of composition) of a symmetric relation.

lemma symm_sq_prod_image: **assumes** $\text{converse}(r) = r$
shows $r \circ r = \bigcup \{(r\{x\}) \times (r\{x\}) \mid x \in \text{domain}(r)\}$
<proof>

Composition of reflexive relations is reflexive.

lemma refl_comp_refl:
assumes $U \subseteq X \times X$ $\text{id}(X) \subseteq U$ $V \subseteq X \times X$ $\text{id}(X) \subseteq V$
shows $\text{id}(X) \subseteq U \circ V$ *<proof>*

Square of a reflexive relation contains the relation. Recall that in ZF the identity function on X is the same as the diagonal of $X \times X$, i.e. $\text{id}(X) = \{\langle x, x \rangle \mid x \in X\}$ (see also lemma id_diagonal in func_ZF theory).

lemma refl_square_greater: **assumes** $r \subseteq X \times X$ $\text{id}(X) \subseteq r$
shows $r \subseteq r \circ r$ *<proof>*

The cube of a reflexive relation contains the relation.

lemma refl_cube_greater: **assumes** $r \subseteq X \times X$ $\text{id}(X) \subseteq r$
shows $r \subseteq r \circ r \circ r$ *<proof>*

The cube of a reflexive relation contains the square of the relation.

lemma refl_cube_greater_square: **assumes** $r \subseteq X \times X$ $\text{id}(X) \subseteq r$
shows $r \circ r \subseteq r \circ r \circ r$ *<proof>*

Composition of two reflexive relations is reflexive and greater than each of them. In particular this implies that if $V \subseteq X \times X$ is a reflexive relation then repeatedly composing it with itself forms a sequence of reflexive relations that is increasing in the inclusion order. It's not proven here, but we may prove that one day.

lemma refl_comp_greater_inter:
assumes $U \subseteq X \times X$ $\text{id}(X) \subseteq U$ $V \subseteq X \times X$ $\text{id}(X) \subseteq V$
shows $\text{id}(X) \subseteq U \circ V$ **and** $U \cap V \subseteq U \circ V$
<proof>

A reflexive relation is contained in the union of products of its singleton images.

lemma refl_union_singl_image:
assumes $A \subseteq X \times X$ **and** $\text{id}(X) \subseteq A$ **shows** $A \subseteq \bigcup \{A\{x\} \times A\{x\} \mid x \in X\}$
<proof>

If the cartesian product of the images of x and y by a symmetric relation W has a nonempty intersection with R then x is in relation $W \circ (R \circ W)$ with y .

lemma sym_rel_comp:
assumes $W = \text{converse}(W)$ **and** $(W\{x\}) \times (W\{y\}) \cap R \neq \emptyset$
shows $\langle x, y \rangle \in (W \circ (R \circ W))$
 $\langle \text{proof} \rangle$

Suppose we have two families of sets $\{A(i)\}_{i \in I}$ and $\{B(i)\}_{i \in I}$, indexed by a nonepty set of indices I and such that for every index $i \in I$ we have inclusion $A(i) \circ A(i) \subseteq B(i)$. Then a similar inclusion holds for the products of the families, namely $(\bigcap_{i \in I} A(i)) \circ (\bigcap_{i \in I} A(i)) \subseteq (\bigcap_{i \in I} B(i))$.

lemma square_incl_product: **assumes** $I \neq \emptyset \ \forall i \in I. A(i) \circ A(i) \subseteq B(i)$
shows $(\bigcap_{i \in I} A(i)) \circ (\bigcap_{i \in I} A(i)) \subseteq (\bigcap_{i \in I} B(i))$
 $\langle \text{proof} \rangle$

It's hard to believe but there are cases where we have to reference this rule.

lemma set_mem_eq: **assumes** $x \in A \ A = B$ **shows** $x \in B$ $\langle \text{proof} \rangle$

Given some family $\mathcal{A}$ of subsets of X we can define the family of supersets of $\mathcal{A}$.

definition $\text{Supersets}(X, \mathcal{A}) \equiv \{B \in \text{Pow}(X). \exists A \in \mathcal{A}. A \subseteq B\}$

If A is a member of a collection of sets $\mathcal{A}$ then it is one of supersets of $\mathcal{A}$.

lemma superset_gen: **assumes** $A \subseteq X \ A \in \mathcal{A}$ **shows** $A \in \text{Supersets}(X, \mathcal{A})$
 $\langle \text{proof} \rangle$

The whole space is a superset of any nonempty collection of its subsets.

lemma space_superset: **assumes** $\mathcal{A} \neq \emptyset \ \mathcal{A} \subseteq \text{Pow}(X)$ **shows** $X \in \text{Supersets}(X, \mathcal{A})$
 $\langle \text{proof} \rangle$

The collection of supersets of an empty set is empty. In particular the whole space X is not a superset of an empty set.

lemma supersets_of_empty: **shows** $\text{Supersets}(X, \emptyset) = \emptyset$
 $\langle \text{proof} \rangle$

However, when the space is empty the collection of supersets does not have to be empty - the collection of supersets of the singleton collection containing only the empty set is this collection.

lemma supersets_in_empty: **shows** $\text{Supersets}(\emptyset, \{\emptyset\}) = \{\emptyset\}$
 $\langle \text{proof} \rangle$

This can be done by the auto method, but sometimes takes a long time.

lemma witness_exists: **assumes** $x \in X$ **and** $\varphi(x)$ **shows** $\exists x \in X. \varphi(x)$
 $\langle \text{proof} \rangle$

Another lemma that concludes existence of some set.

```
lemma witness_exists1: assumes  $x \in X$   $\varphi(x)$   $\psi(x)$ 
shows  $\exists x \in X. \varphi(x) \wedge \psi(x)$ 
 $\langle proof \rangle$ 
```

The next lemma has to be used as a rule in some rare cases.

```
lemma exists_in_set: assumes  $\forall x. x \in A \longrightarrow \varphi(x)$  shows  $\forall x \in A. \varphi(x)$ 
 $\langle proof \rangle$ 
```

If x belongs to a set where a property holds, then the property holds for x . This has to be used as rule in rare cases.

```
lemma property_holds: assumes  $\forall t \in X. \varphi(t)$  and  $x \in X$ 
shows  $\varphi(x)$   $\langle proof \rangle$ 
```

Set comprehensions defined by equal expressions are the equal. The second assertion is actually about functions, which are sets of pairs as illustrated in lemma `fun_is_set_of_pairs` in `func1.thy`

```
lemma set_comp_eq: assumes  $\forall x \in X. p(x) = q(x)$ 
shows  $\{p(x). x \in X\} = \{q(x). x \in X\}$  and  $\{\langle x, p(x) \rangle. x \in X\} = \{\langle x, q(x) \rangle. x \in X\}$ 
 $\langle proof \rangle$ 
```

If every element of a non-empty set $X \subseteq Y$ satisfies a condition then the set of elements of Y that satisfy the condition is non-empty.

```
lemma non_empty_cond: assumes  $X \neq \emptyset$   $X \subseteq Y$  and  $\forall x \in X. P(x)$ 
shows  $\{x \in Y. P(x)\} \neq \emptyset$   $\langle proof \rangle$ 
```

If z is a pair, then the cartesian product of the singletons of its elements is the same as the singleton $\{z\}$.

```
lemma pair_prod: assumes  $z = \langle x, y \rangle$  shows  $\{x\} \times \{y\} = \{z\}$ 
 $\langle proof \rangle$ 
```

end

4 Natural numbers in IsarMathLib

```
theory Nat_ZF_IML imports ZF.ArithSimp
```

```
begin
```

The ZF set theory constructs natural numbers from the empty set and the notion of a one-element set. Namely, zero of natural numbers is defined as the empty set. For each natural number n the next natural number is defined as $n \cup \{n\}$. With this definition for every non-zero natural number we get the identity $n = \{0, 1, 2, \dots, n-1\}$. It is good to remember that when

we see an expression like $f : n \rightarrow X$. Also, with this definition the relation "less or equal than" becomes " $\subseteq$ " and the relation "less than" becomes " $\in$ ".

4.1 Induction

The induction lemmas in the standard Isabelle's Nat.thy file like for example `nat_induct` require the induction step to be a higher order statement (the one that uses the $\implies$ sign). I found it difficult to apply from Isar, which is perhaps more of an indication of my Isar skills than anything else. Anyway, here we provide a first order version that is easier to reference in Isar declarative style proofs.

The next theorem is a version of induction on natural numbers that I was thought in school.

```
theorem ind_on_nat:
  assumes A1:  $n \in \text{nat}$  and A2:  $P(0)$  and A3:  $\forall k \in \text{nat}. P(k) \longrightarrow P(\text{succ}(k))$ 
  shows  $P(n)$ 
  <proof>
```

A nonzero natural number has a predecessor.

```
lemma Nat_ZF_1_L3: assumes A1:  $n \in \text{nat}$  and A2:  $n \neq 0$ 
  shows  $\exists k \in \text{nat}. n = \text{succ}(k)$ 
  <proof>
```

What is `succ`, anyway? It's a union with the singleton of the set.

```
lemma succ_explained: shows  $\text{succ}(n) = n \cup \{n\}$ 
  <proof>
```

The singleton containing the empty set is a natural number.

```
lemma one_is_nat: shows  $\{0\} \in \text{nat}$   $\{0\} = \text{succ}(0)$   $\{0\} = 1$ 
  <proof>
```

If k is a member of $\text{succ}(n)$ but is not n , then it must be the member of n .

```
lemma mem_succ_not_eq: assumes  $k \in \text{succ}(n)$   $k \neq n$ 
  shows  $k \in n$  <proof>
```

Empty set is an element of every natural number which is not zero.

```
lemma empty_in_every_succ: assumes A1:  $n \in \text{nat}$ 
  shows  $0 \in \text{succ}(n)$ 
  <proof>
```

Various forms of saying that for natural numbers taking the successor is the same as adding one, also a natural number is a subset of its successor.

```
lemma succ_add_one: assumes  $n \in \text{nat}$ 
  shows
     $n \#+ 1 = \text{succ}(n)$ 
```

```

    n #+ 1 ∈ nat
    {0} #+ n = succ(n)
    n #+ {0} = succ(n)
    succ(n) ∈ nat
    0 ∈ n #+ 1
    n ⊆ n #+ 1
  ⟨proof⟩

```

If a property is true of $n = 0$ and for all natural numbers $n + 1$ then it is true for all natural numbers.

```

lemma zero_pos_all: assumes  $\varphi(0)$  and  $\forall m \in \text{nat}. \varphi(m \#+ 1)$   $n \in \text{nat}$ 
  shows  $\varphi(n)$  ⟨proof⟩

```

A more direct way of stating that empty set is an element of every non-zero natural number:

```

lemma empty_in_non_empty: assumes  $n \in \text{nat}$   $n \neq 0$ 
  shows  $0 \in n$ 
  ⟨proof⟩

```

If one natural number is less than another then their successors are in the same relation.

```

lemma succ_ineq: assumes  $A1: n \in \text{nat}$ 
  shows  $\forall i \in n. \text{succ}(i) \in \text{succ}(n)$ 
  ⟨proof⟩

```

For natural numbers if $k \subseteq n$ the similar holds for their successors.

```

lemma succ_subset: assumes  $A1: k \in \text{nat}$   $n \in \text{nat}$  and  $A2: k \subseteq n$ 
  shows  $\text{succ}(k) \subseteq \text{succ}(n)$ 
  ⟨proof⟩

```

For any two natural numbers one of them is contained in the other.

```

lemma nat_incl_total: assumes  $A1: i \in \text{nat}$   $j \in \text{nat}$ 
  shows  $i \subseteq j \vee j \subseteq i$ 
  ⟨proof⟩

```

For natural numbers i, j if $i \subseteq j$ then $i = j$ or $i \in j$.

```

lemma nat_incl_mem_eq: assumes  $i \in \text{nat}$   $j \in \text{nat}$   $i \subseteq j$ 
  shows  $i \in j \vee i = j$ 
  ⟨proof⟩

```

If two natural numbers are different then one of them is less than the other.

```

lemma nat_mem_total: assumes  $i \in \text{nat}$   $j \in \text{nat}$   $i \neq j$ 
  shows  $i < j \vee j < i$ 
  ⟨proof⟩

```

The set of natural numbers is the union of all successors of natural numbers.

```

lemma nat_union_succ: shows  $\text{nat} = (\bigcup n \in \text{nat}. \text{succ}(n))$ 

```

⟨proof⟩

Successors of natural numbers are subsets of the set of natural numbers.

lemma succnat_subset_nat: **assumes** A1: $n \in \text{nat}$ **shows** $\text{succ}(n) \subseteq \text{nat}$
⟨proof⟩

A natural number is a subset of the set natural numbers. Weird, but true.

lemma nat_subset_nat: **assumes** $n \in \text{nat}$ **shows** $n \subseteq \text{nat}$
⟨proof⟩

Element k of a natural number n is a natural number that is smaller than n .

lemma elem_nat_is_nat: **assumes** A1: $n \in \text{nat}$ **and** A2: $k \in n$
shows $k < n$ $k \in \text{nat}$ $k \leq n$ $\langle k, n \rangle \in \text{Le}$
⟨proof⟩

A version of succ_ineq without a quantifier, with additional assertion using the $n \# + 1$ notation.

lemma succ_ineq1: **assumes** $n \in \text{nat}$ $i \in n$
shows $\text{succ}(i) \in \text{succ}(n)$ $i \# + 1 \in n \# + 1$ $i \in n \# + 1$
⟨proof⟩

For natural numbers membership and inequality are the same and $k \leq n$ is the same as $k \in \text{succ}(n)$. The proof relies on lemmas in the standard Isabelle's Nat and Ordinal theories.

lemma nat_mem_lt: **assumes** $n \in \text{nat}$
shows $k < n \longleftrightarrow k \in n$ **and** $k \leq n \longleftrightarrow k \in \text{succ}(n)$
⟨proof⟩

If n is a natural number and $k < n$ then $k + 1 \leq n$.

lemma nat_less_succ_leq: **assumes** $n \in \text{nat}$ **and** $k < n$
shows $k \# + 1 \leq n$
⟨proof⟩

If n is a natural number and $k \leq n$, then k is a natural number.

lemma leq_nat_is_nat: **assumes** $n \in \text{nat}$ $k \leq n$ **shows** $k \in \text{nat}$
⟨proof⟩

For natural numbers $k \leq n$ is the same as $k \subseteq n$.

lemma nat_leq_subset_iff: **assumes** $k \in \text{nat}$ $n \in \text{nat}$
shows $k \leq n \longleftrightarrow k \subseteq n$ *⟨proof⟩*

The term $k \leq n$ is the same as $k < \text{succ}(n)$.

lemma leq_mem_succ: **shows** $k \leq n \longleftrightarrow k < \text{succ}(n)$ *⟨proof⟩*

A natural number n is smaller than $n + 1$.

lemma nat_less_add_one: assumes $n \in \text{nat}$
 shows $n < n \#+ 1$ and $n \in n \#+ 1$
 $\langle \text{proof} \rangle$

If n and k are natural numbers and $k + 1 \in n$ then also $k < n$, hence $k \in n$.

lemma succ_mem_mem: assumes $n \in \text{nat}$ $k \in \text{nat}$ $k \#+ 1 \in n$
 shows $k < n$ and $k \in n$
 $\langle \text{proof} \rangle$

If the successor of a natural number k is an element of the successor of n then a similar relations holds for the numbers themselves.

lemma succ_mem:
 assumes $n \in \text{nat}$ $\text{succ}(k) \in \text{succ}(n)$
 shows $k \in n$
 $\langle \text{proof} \rangle$

The set of natural numbers is the union of its elements.

lemma nat_union_nat: shows $\text{nat} = \bigcup \text{nat}$
 $\langle \text{proof} \rangle$

Adding natural numbers does not decrease what we add to.

lemma add_nat_le: assumes $A1: n \in \text{nat}$ and $A2: k \in \text{nat}$
 shows
 $n \leq n \#+ k$
 $n \subseteq n \#+ k$
 $n \subseteq k \#+ n$
 $\langle \text{proof} \rangle$

Result of adding an element of k is smaller than of adding k .

lemma add_lt_mono:
 assumes $k \in \text{nat}$ and $j \in k$
 shows
 $(n \#+ j) < (n \#+ k)$
 $(n \#+ j) \in (n \#+ k)$
 $\langle \text{proof} \rangle$

A technical lemma about a decomposition of a sum of two natural numbers: if a number i is from $m + n$ then it is either from m or can be written as a sum of m and a number from n . The proof by induction w.r.t. to m seems to be a bit heavy-handed, but I could not figure out how to do this directly from results from standard Isabelle/ZF.

lemma nat_sum_decomp: assumes $A1: n \in \text{nat}$ and $A2: m \in \text{nat}$
 shows $\forall i \in m \#+ n. i \in m \vee (\exists j \in n. i = m \#+ j)$
 $\langle \text{proof} \rangle$

A variant of induction useful for finite sequences.

lemma fin_nat_ind: assumes $A1: n \in \text{nat}$ and $A2: k \in \text{succ}(n)$

and A3: $P(0)$ and A4: $\forall j \in n. P(j) \longrightarrow P(\text{succ}(j))$
 shows $P(k)$
 $\langle \text{proof} \rangle$

Some properties of positive natural numbers.

lemma succ_plus: assumes $n \in \text{nat}$ $k \in \text{nat}$
 shows
 $\text{succ}(n \#+ j) \in \text{nat}$
 $\text{succ}(n) \#+ \text{succ}(j) = \text{succ}(\text{succ}(n \#+ j))$
 $\langle \text{proof} \rangle$

If k is in the successor of n , then the predecessor of k is in n .

lemma pred_succ_mem: assumes $n \in \text{nat}$ $n \neq 0$ $k \in \text{succ}(n)$ shows $\text{pred}(k) \in n$
 $\langle \text{proof} \rangle$

For non-zero natural numbers $\text{pred}(n) = n - 1$.

lemma pred_minus_one: assumes $n \in \text{nat}$ $n \neq 0$
 shows $n \#- 1 = \text{pred}(n)$ and $n \#- 1 \in n$
 $\langle \text{proof} \rangle$

For natural numbers if $k \in n$ then $k + 1 \subseteq n$.

lemma mem_add_one_subset: assumes $n \in \text{nat}$ $k \in n$ shows $k \#+ 1 \subseteq n$
 $\langle \text{proof} \rangle$

For a natural n if $k \in n + 1$ then $k + 1 \leq n + 1$.

lemma succ_ineq2: assumes $n \in \text{nat}$ $k \in n \#+ 1$
 shows $k \#+ 1 \leq n \#+ 1$ $k \leq n$ $k \in n \vee k = n$
 $\langle \text{proof} \rangle$

A nonzero natural number is of the form $n = m + 1$ for some natural number m . This is very similar to `Nat_ZF_1_L3` except that we use $n + 1$ instead of $\text{succ}(n)$.

lemma nat_not0_succ: assumes $n \in \text{nat}$ $n \neq 0$
 shows $\exists m \in \text{nat}. n = m \#+ 1$
 $\langle \text{proof} \rangle$

A version of induction on natural numbers that uses the $n + 1$ notation instead of $\text{succ}(n)$.

lemma ind_on_nat1:
 assumes $n \in \text{nat}$ and $P(0)$ and $\forall k \in \text{nat}. P(k) \longrightarrow P(k \#+ 1)$
 shows $P(n)$ $\langle \text{proof} \rangle$

A version of induction for finite sequences using the $n + 1$ notation instead of $\text{succ}(n)$:

lemma fin_nat_ind1:
 assumes $n \in \text{nat}$ and $P(0)$ and $\forall j \in n. P(j) \longrightarrow P(j \#+ 1)$
 shows $\forall k \in n \#+ 1. P(k)$ and $P(n)$
 $\langle \text{proof} \rangle$

4.2 Simplification rules for addition and subtraction of natural numbers

This section collects useful simplification rules involving addition and subtraction of natural numbers that we couldn't find in standard Isabelle's `ArithSimp` theory.

Adding and subtracting a natural number cancel each other.

lemma `add_subtract`: **assumes** $m \in \text{nat}$ **shows** $(m \#+ n) \#- n = m$
 $\langle \text{proof} \rangle$

A simplification rule for natural numbers: if $k < n$ then $n - (k + 1) + 1 = n - k$:

lemma `nat_subtr_simpl0`: **assumes** $n \in \text{nat}$ $k \in \text{nat}$
shows $n \#- (k \#+ 1) \#+ 1 = n \#- k$
 $\langle \text{proof} \rangle$

A special case of `nat_subtr_simpl0` when $k = 0$: if n is a non-zero natural number then $(n - 1) + 1 = n$.

lemma `nat_subtr_add1`: **assumes** $n \in \text{nat}$ $n \neq 0$
shows $(n \#- 1) \#+ 1 = n$
 $\langle \text{proof} \rangle$

If k is a natural number then $n + k = n + ((n + k) - n)$.

lemma `nat_subtr_simpl1`: **assumes** $k \in \text{nat}$
shows $n \#+ ((n \#+ k) \#- n) = n \#+ k$
 $\langle \text{proof} \rangle$

4.3 Intervals

In this section we consider intervals of natural numbers i.e. sets of the form $\{n + j : j \in 0..k - 1\}$.

The interval is determined by two parameters: starting point and length.

definition

$$\text{NatInterval}(n, k) \equiv \{n \#+ j. j \in k\}$$

If k is a natural number and i is in the natural interval starting from n of length k then $i - n \in k$ and $i + 1 - n = i - n + 1$. The first assertion may look weird, but note that natural numbers are sets and for natural numbers $i \in k$ is the same as $i < k$.

lemma `inter_diff_in_len`:
assumes $k \in \text{nat}$ **and** $i \in \text{NatInterval}(n, k)$
shows $i \#- n \in k$ **and** $i \#+ 1 \#- n = i \#- n \#+ 1$
 $\langle \text{proof} \rangle$

Intervals don't overlap with their starting point and the union of an interval with its starting point is the sum of the starting point and the length of the interval.

```
lemma length_start_decomp: assumes A1: n ∈ nat k ∈ nat
  shows
    n ∩ NatInterval(n,k) = 0
    n ∪ NatInterval(n,k) = n #+ k
  ⟨proof⟩
```

Natural interval with zero length is empty. Recall that in ZF the empty set is the same as zero of natural numbers.

```
lemma nat_interval_zero_len: shows NatInterval(n,0) = 0
  ⟨proof⟩
```

Natural interval starting with zero is the same as its the length.

```
lemma nat_interval_zero_beg: assumes n ∈ nat shows NatInterval(0,n) =
n
  ⟨proof⟩
```

If i is in the interval starting at $n \in \mathbb{N}$ and length $k \in \mathbb{N}$ then both i and $i + 1$ are in the interval starting at n and of length $k + 1$.

```
lemma interval_incr_len:
  assumes n ∈ nat k ∈ nat i ∈ NatInterval(n,k)
  shows i ∈ NatInterval(n,k #+ 1) and (i #+ 1) ∈ NatInterval(n,k #+
1)
  ⟨proof⟩
```

Some properties of three adjacent intervals.

```
lemma adjacent_intervals3: assumes n ∈ nat k ∈ nat m ∈ nat
  shows
    n #+ k #+ m = (n #+ k) ∪ NatInterval(n #+ k,m)
    n #+ k #+ m = n ∪ NatInterval(n,k #+ m)
    n #+ k #+ m = n ∪ NatInterval(n,k) ∪ NatInterval(n #+ k,m)
  ⟨proof⟩
```

end

5 Order relations - introduction

```
theory Order_ZF imports Fol1 ZF1
```

```
begin
```

This theory file considers various notion related to order. We redefine the notions of a preorder, directed set, total order, linear order and partial order to have the same terminology as Wikipedia (I found it very consistent across different areas of math). We also define and study the notions of intervals

and bounded sets. We show the inclusion relations between the intervals with endpoints being in certain order. We also show that union of bounded sets are bounded. This allows to show in `Finite_ZF.thy` that finite sets are bounded.

5.1 Definitions

In this section we formulate the definitions related to order relations.

A relation r is "total" on a set X if for all elements a, b of X we have a is in relation with b or b is in relation with a . An example is the $\leq$ relation on numbers.

definition

`IsTotal (infixl {is total on} 65) where`
`r {is total on} X  $\equiv$  ( $\forall a \in X. \forall b \in X. \langle a, b \rangle \in r \vee \langle b, a \rangle \in r$ )`

A relation r is a partial order on X if it is reflexive on X (i.e. $\langle x, x \rangle$ for every $x \in X$), antisymmetric (if $\langle x, y \rangle \in r$ and $\langle y, x \rangle \in r$, then $x = y$) and transitive ($\langle x, y \rangle \in r$ and $\langle y, z \rangle \in r$ implies $\langle x, z \rangle \in r$).

definition

`IsPartOrder(X, r)  $\equiv$  refl(X, r)  $\wedge$  antisym(r)  $\wedge$  trans(r)`

A relation that is reflexive and transitive is called a **preorder**.

definition

`IsPreorder(X, r)  $\equiv$  refl(X, r)  $\wedge$  trans(r)`

We say that a relation r up-directs a set if every two-element subset of X has an upper bound.

definition

`UpDirects (_ {up-directs} _ 90)`
`where r {up-directs} X  $\equiv$  X  $\neq$  0  $\wedge$  ( $\forall x \in X. \forall y \in X. \exists z \in X. \langle x, z \rangle \in r \wedge \langle y, z \rangle \in r$ )`

Analogously we say that a relation r down-directs a set if every two-element subset of X has a lower bound.

definition

`DownDirects (_ {down-directs} _ 90)`
`where r {down-directs} X  $\equiv$  X  $\neq$  0  $\wedge$  ( $\forall x \in X. \forall y \in X. \exists z \in X. \langle z, x \rangle \in r \wedge \langle z, y \rangle \in r$ )`

Typically the notion that is actually defined is the notion of a **directed set**, or an **upward directed set**, rather than r down-directs X (or r up-directs X). This is a nonempty set X together with a preorder r such that r up-directs X . We set that up in separate definitions as we sometimes want to use an upward or downward directed set with a partial order rather than a preorder.

definition

$$\text{IsUpDirectedSet}(X, r) \equiv \text{IsPreorder}(X, r) \wedge (r \text{ \{up-directs\} } X)$$

We define the notion of a downward directed set analogously.

definition

$$\text{IsDownDirectedSet}(X, r) \equiv \text{IsPreorder}(X, r) \wedge (r \text{ \{down-directs\} } X)$$

We define a linear order as a binary relation that is antisymmetric, transitive and total. Note that this terminology is different than the one used the standard Isabelle's `Order.thy` file.

definition

$$\text{IsLinOrder}(X, r) \equiv \text{antisym}(r) \wedge \text{trans}(r) \wedge (r \text{ \{is total on\} } X)$$

A set is bounded above if there is that is an upper bound for it, i.e. there are some u such that $\langle x, u \rangle \in r$ for all $x \in A$. In addition, the empty set is defined as bounded.

definition

$$\text{IsBoundedAbove}(A, r) \equiv \exists u. \forall x \in A. \langle x, u \rangle \in r$$

We define sets bounded below analogously.

definition

$$\text{IsBoundedBelow}(A, r) \equiv \exists l. \forall x \in A. \langle l, x \rangle \in r$$

A set is bounded if it is bounded below and above.

definition

$$\text{IsBounded}(A, r) \equiv (\text{IsBoundedAbove}(A, r) \wedge \text{IsBoundedBelow}(A, r))$$

The notation for the definition of an interval may be mysterious for some readers, see lemma `Order_ZF_2_L1` for more intuitive notation.

definition

$$\text{Interval}(r, a, b) \equiv r\{a\} \cap r-\{b\}$$

We also define the maximum (the greater of) two elements in the obvious way.

definition

$$\text{GreaterOf}(r, a, b) \equiv (\text{if } \langle a, b \rangle \in r \text{ then } b \text{ else } a)$$

The definition a a minimum (the smaller of) two elements.

definition

$$\text{SmallerOf}(r, a, b) \equiv (\text{if } \langle a, b \rangle \in r \text{ then } a \text{ else } b)$$

We say that a set has a maximum if it has an element that is not smaller than any other one. We show that under some conditions this element of the set is unique (if exists).

definition

$$\text{HasAmaximum}(r, A) \equiv \exists M \in A. \forall x \in A. \langle x, M \rangle \in r$$

A similar definition what it means that a set has a minimum.

definition

$$\text{HasAminimum}(r, A) \equiv \exists m \in A. \forall x \in A. \langle m, x \rangle \in r$$

Definition of the maximum of a set.

definition

$$\text{Maximum}(r, A) \equiv \text{THE } M. M \in A \wedge (\forall x \in A. \langle x, M \rangle \in r)$$

Definition of a minimum of a set.

definition

$$\text{Minimum}(r, A) \equiv \text{THE } m. m \in A \wedge (\forall x \in A. \langle m, x \rangle \in r)$$

The supremum of a set A is defined as the minimum of the set of upper bounds, i.e. the set $\{u. \forall a \in A. \langle a, u \rangle \in r\} = \bigcap_{a \in A} r\{a\}$.

definition

$$\text{Supremum}(r, A) \equiv \text{Minimum}(r, \bigcap_{a \in A} r\{a\})$$

The notion of "having a supremum" is the same as the set of upper bounds having a minimum, but having it a a separate notion does simplify notation in some cases. The definition is written in terms of images of singletons $\{x\}$ under relation. To understand this formulation note that the set of upper bounds of a set $A \subseteq X$ is $\bigcap_{x \in A} \{y \in X | \langle x, y \rangle \in r\}$, which is the same as $\bigcap_{x \in A} r(\{x\})$, where $r(\{x\})$ is the image of the singleton $\{x\}$ under relation r .

definition

$$\text{HasAsupremum}(r, A) \equiv \text{HasAminimum}(r, \bigcap_{a \in A} r\{a\})$$

The notion of "having an infimum" is the same as the set of lower bounds having a maximum.

definition

$$\text{HasAnInfimum}(r, A) \equiv \text{HasAmaximum}(r, \bigcap_{a \in A} r-\{a\})$$

Infimum is defined analogously. Recall that in Isabelle/ZF $r-(A)$ denotes the inverse image of the set A by relation r (i.e. $r-(A) = \{x : \langle x, y \rangle \in r \text{ for some } y \in A\}$)

definition

$$\text{Infimum}(r, A) \equiv \text{Maximum}(r, \bigcap_{a \in A} r-\{a\})$$

We define a relation to be complete if every nonempty bounded above set has a supremum.

definition

$$\begin{aligned} &\text{IsComplete } (_ \text{ \{is complete\}}) \text{ where} \\ &r \text{ \{is complete\}} \equiv \\ &\forall A. \text{IsBoundedAbove}(A, r) \wedge A \neq 0 \longrightarrow \text{HasAminimum}(r, \bigcap_{a \in A} r\{a\}) \end{aligned}$$

Normally the " $\subseteq$ " does not represent a relation, but it does if we restrict it to some collection of sets X . We will call such relation the $\text{InclusionOn}(X)$.

definition

$$\text{InclusionOn}(X) \equiv \{p \in X \times X. \text{fst}(p) \subseteq \text{snd}(p)\}$$

The way we define `IsBoundedAbove` and `IsBoundedBelow` predicates the empty set is bounded above and below.

lemma `empty_bounded_above_below`:

shows `IsBoundedAbove( $\emptyset$ , r)` `IsBoundedBelow( $\emptyset$ , r)`
<proof>

The most common way of applying the boundedness: if r as a relation on X and a set A is not empty and bounded above then there exist $u \in X$ which is an upper bound of A .

lemma `bounded_above`: **assumes** $r \subseteq X \times X$ $A \neq \emptyset$ `IsBoundedAbove(A, r)`

shows $\exists u \in X. \forall x \in A. \langle x, u \rangle \in r$
<proof>

A dual for `bounded_above`: if r as a relation on X and a set A is not empty and bounded below then there exist $u \in X$ which is a lower bound of A .

lemma `bounded_below`: **assumes** $r \subseteq X \times X$ $A \neq \emptyset$ `IsBoundedBelow(A, r)`

shows $\exists u \in X. \forall x \in A. \langle u, x \rangle \in r$
<proof>

Inclusion relation is a partial order hence a preorder on the powerset of X .

lemma `incl_is_partorder`:

shows `IsPartOrder(X, InclusionOn(X))` **and** `IsPreorder(X, InclusionOn(X))`
<proof>

If a relation down-directs a set, then a larger one does as well.

lemma `down_dir_mono`: **assumes** $r \text{ \{down-directs\} } X$ $r \subseteq R$

shows $R \text{ \{down-directs\} } X$ *<proof>*

If a relation up-directs a set, then a larger one does as well.

lemma `up_dir_mono`: **assumes** $r \text{ \{up-directs\} } X$ $r \subseteq R$

shows $R \text{ \{up-directs\} } X$ *<proof>*

The essential condition to show that a total relation is reflexive.

lemma `Order_ZF_1_L1`: **assumes** $r \text{ \{is total on\} } X$ **and** $a \in X$

shows $\langle a, a \rangle \in r$ *<proof>*

A total relation is reflexive.

lemma `total_is_refl`:

assumes $r \text{ \{is total on\} } X$
shows `refl(X, r)` *<proof>*

A linear order is partial order.

lemma `Order_ZF_1_L2`: **assumes** `IsLinOrder(X, r)`

shows IsPartOrder(X, r)
<proof>

Partial order that is total is linear.

lemma Order_ZF_1_L3:
assumes IsPartOrder(X, r) **and** r {is total on} X
shows IsLinOrder(X, r)
<proof>

Relation that is total on a set is total on any subset.

lemma Order_ZF_1_L4: **assumes** r {is total on} X **and** $A \subseteq X$
shows r {is total on} A
<proof>

We can restrict a partial order relation to the domain.

lemma part_ord_restr: **assumes** IsPartOrder(X, r)
shows IsPartOrder($X, r \cap X \times X$)
<proof>

Partial order on a set implies partial order on a subset.

lemma part_ord_subset: **assumes** IsPartOrder(X, r) **and** $A \subseteq X$
shows IsPartOrder(A, r)
<proof>

We can restrict a total order relation to the domain.

lemma total_ord_restr: **assumes** r {is total on} X
shows $(r \cap X \times X)$ {is total on} X
<proof>

A linear relation is linear on any subset and we can restrict it to any subset.

lemma ord_linear_subset: **assumes** IsLinOrder(X, r) **and** $A \subseteq X$
shows IsLinOrder(A, r) **and** IsLinOrder($A, r \cap A \times A$)
<proof>

If a relation is a partial order on X and it down-directs a subset of X then that is a down-directed set.

lemma down_directs_subset:
assumes r {down-directs} A IsPartOrder(X, r) $A \subseteq X$
shows IsDownDirectedSet(A, r)
<proof>

If the relation is total, then every set is a union of those elements that are nongreater than a given one and nonsmaller than a given one.

lemma Order_ZF_1_L5:
assumes r {is total on} X **and** $A \subseteq X$ **and** $a \in X$
shows $A = \{x \in A. \langle x, a \rangle \in r\} \cup \{x \in A. \langle a, x \rangle \in r\}$
<proof>

A technical fact about reflexive relations.

lemma refl_add_point:
 assumes refl(X,r) and $A \subseteq B \cup \{x\}$ and $B \subseteq X$ and
 $x \in X$ and $\forall y \in B. \langle y, x \rangle \in r$
 shows $\forall a \in A. \langle a, x \rangle \in r$
<proof>

5.2 Intervals

In this section we discuss intervals.

The next lemma explains the notation of the definition of an interval.

lemma Order_ZF_2_L1:
 shows $x \in \text{Interval}(r, a, b) \longleftrightarrow \langle a, x \rangle \in r \wedge \langle x, b \rangle \in r$
<proof>

Since there are some problems with applying the above lemma (seems that simp and auto don't handle equivalence very well), we split Order_ZF_2_L1 into two lemmas.

lemma Order_ZF_2_L1A: assumes $x \in \text{Interval}(r, a, b)$
 shows $\langle a, x \rangle \in r \quad \langle x, b \rangle \in r$
<proof>

Order_ZF_2_L1, implication from right to left.

lemma Order_ZF_2_L1B: assumes $\langle a, x \rangle \in r \quad \langle x, b \rangle \in r$
 shows $x \in \text{Interval}(r, a, b)$
<proof>

If the relation is reflexive, the endpoints belong to the interval.

lemma Order_ZF_2_L2: assumes refl(X,r)
 and $a \in X \quad b \in X$ and $\langle a, b \rangle \in r$
 shows
 $a \in \text{Interval}(r, a, b)$
 $b \in \text{Interval}(r, a, b)$
<proof>

Under the assumptions of Order_ZF_2_L2, the interval is nonempty.

lemma Order_ZF_2_L2A: assumes refl(X,r)
 and $a \in X \quad b \in X$ and $\langle a, b \rangle \in r$
 shows $\text{Interval}(r, a, b) \neq \emptyset$
<proof>

If a, b, c, d are in this order, then $[b, c] \subseteq [a, d]$. We only need transitivity for this to be true.

lemma Order_ZF_2_L3:
 assumes A1: trans(r) and A2: $\langle a, b \rangle \in r \quad \langle b, c \rangle \in r \quad \langle c, d \rangle \in r$
 shows $\text{Interval}(r, b, c) \subseteq \text{Interval}(r, a, d)$

<proof>

For reflexive and antisymmetric relations the interval with equal endpoints consists only of that endpoint.

lemma Order_ZF_2_L4:
 assumes A1: $\text{refl}(X,r)$ **and** A2: $\text{antisym}(r)$ **and** A3: $a \in X$
 shows $\text{Interval}(r,a,a) = \{a\}$
<proof>

For transitive relations the endpoints have to be in the relation for the interval to be nonempty.

lemma Order_ZF_2_L5: **assumes** A1: $\text{trans}(r)$ **and** A2: $\langle a,b \rangle \notin r$
 shows $\text{Interval}(r,a,b) = \emptyset$
<proof>

If a relation is defined on a set, then intervals are subsets of that set.

lemma Order_ZF_2_L6: **assumes** A1: $r \subseteq X \times X$
 shows $\text{Interval}(r,a,b) \subseteq X$
<proof>

5.3 Bounded sets

In this section we consider properties of bounded sets.

For reflexive relations singletons are bounded.

lemma Order_ZF_3_L1: **assumes** $\text{refl}(X,r)$ **and** $a \in X$
 shows $\text{IsBounded}(\{a\},r)$
<proof>

Sets that are bounded above are contained in the domain of the relation.

lemma Order_ZF_3_L1A: **assumes** $r \subseteq X \times X$
 and $\text{IsBoundedAbove}(A,r)$
 shows $A \subseteq X$ *<proof>*

Sets that are bounded below are contained in the domain of the relation.

lemma Order_ZF_3_L1B: **assumes** $r \subseteq X \times X$
 and $\text{IsBoundedBelow}(A,r)$
 shows $A \subseteq X$ *<proof>*

For a total relation, the greater of two elements, as defined above, is indeed greater of any of the two.

lemma Order_ZF_3_L2: **assumes** r {is total on} X
 and $x \in X$ $y \in X$
 shows
 $\langle x, \text{GreaterOf}(r,x,y) \rangle \in r$
 $\langle y, \text{GreaterOf}(r,x,y) \rangle \in r$
 $\langle \text{SmallerOf}(r,x,y), x \rangle \in r$

$\langle \text{SmallerOf}(r, x, y), y \rangle \in r$
 $\langle \text{proof} \rangle$

If A is bounded above by u , B is bounded above by w , then $A \cup B$ is bounded above by the greater of u, w .

lemma Order_ZF_3_L2B:
 assumes A1: r {is total on} X and A2: $\text{trans}(r)$
 and A3: $u \in X$ $w \in X$
 and A4: $\forall x \in A. \langle x, u \rangle \in r \ \forall x \in B. \langle x, w \rangle \in r$
 shows $\forall x \in A \cup B. \langle x, \text{GreaterOf}(r, u, w) \rangle \in r$
 $\langle \text{proof} \rangle$

For total and transitive relation the union of two sets bounded above is bounded above.

lemma Order_ZF_3_L3:
 assumes A1: r {is total on} X and A2: $\text{trans}(r)$
 and A3: $\text{IsBoundedAbove}(A, r) \ \text{IsBoundedAbove}(B, r)$
 and A4: $r \subseteq X \times X$
 shows $\text{IsBoundedAbove}(A \cup B, r)$
 $\langle \text{proof} \rangle$

For total and transitive relations if a set A is bounded above then $A \cup \{a\}$ is bounded above.

lemma Order_ZF_3_L4:
 assumes A1: r {is total on} X and A2: $\text{trans}(r)$
 and A3: $\text{IsBoundedAbove}(A, r)$ and A4: $a \in X$ and A5: $r \subseteq X \times X$
 shows $\text{IsBoundedAbove}(A \cup \{a\}, r)$
 $\langle \text{proof} \rangle$

If A is bounded below by l , B is bounded below by m , then $A \cup B$ is bounded below by the smaller of u, w .

lemma Order_ZF_3_L5B:
 assumes A1: r {is total on} X and A2: $\text{trans}(r)$
 and A3: $l \in X$ $m \in X$
 and A4: $\forall x \in A. \langle l, x \rangle \in r \ \forall x \in B. \langle m, x \rangle \in r$
 shows $\forall x \in A \cup B. \langle \text{SmallerOf}(r, l, m), x \rangle \in r$
 $\langle \text{proof} \rangle$

For total and transitive relation the union of two sets bounded below is bounded below.

lemma Order_ZF_3_L6:
 assumes A1: r {is total on} X and A2: $\text{trans}(r)$
 and A3: $\text{IsBoundedBelow}(A, r) \ \text{IsBoundedBelow}(B, r)$
 and A4: $r \subseteq X \times X$
 shows $\text{IsBoundedBelow}(A \cup B, r)$
 $\langle \text{proof} \rangle$

For total and transitive relations if a set A is bounded below then $A \cup \{a\}$ is bounded below.

lemma Order_ZF_3_L7:
 assumes A1: r {is total on} X and A2: $\text{trans}(r)$
 and A3: $\text{IsBoundedBelow}(A, r)$ and A4: $a \in X$ and A5: $r \subseteq X \times X$
 shows $\text{IsBoundedBelow}(A \cup \{a\}, r)$
<proof>

For total and transitive relations unions of two bounded sets are bounded.

theorem Order_ZF_3_T1:
 assumes r {is total on} X and $\text{trans}(r)$
 and $\text{IsBounded}(A, r)$ $\text{IsBounded}(B, r)$
 and $r \subseteq X \times X$
 shows $\text{IsBounded}(A \cup B, r)$
<proof>

For total and transitive relations if a set A is bounded then $A \cup \{a\}$ is bounded.

lemma Order_ZF_3_L8:
 assumes r {is total on} X and $\text{trans}(r)$
 and $\text{IsBounded}(A, r)$ and $a \in X$ and $r \subseteq X \times X$
 shows $\text{IsBounded}(A \cup \{a\}, r)$
<proof>

A sufficient condition for a set to be bounded below.

lemma Order_ZF_3_L9: assumes A1: $\forall a \in A. \langle 1, a \rangle \in r$
 shows $\text{IsBoundedBelow}(A, r)$
<proof>

A sufficient condition for a set to be bounded above.

lemma Order_ZF_3_L10: assumes A1: $\forall a \in A. \langle a, u \rangle \in r$
 shows $\text{IsBoundedAbove}(A, r)$
<proof>

Intervals are bounded.

lemma Order_ZF_3_L11: shows
 $\text{IsBoundedAbove}(\text{Interval}(r, a, b), r)$
 $\text{IsBoundedBelow}(\text{Interval}(r, a, b), r)$
 $\text{IsBounded}(\text{Interval}(r, a, b), r)$
<proof>

A subset of a set that is bounded below is bounded below.

lemma Order_ZF_3_L12: assumes A1: $\text{IsBoundedBelow}(A, r)$ and A2: $B \subseteq A$
 shows $\text{IsBoundedBelow}(B, r)$
<proof>

A subset of a set that is bounded above is bounded above.

lemma Order_ZF_3_L13: assumes A1: $\text{IsBoundedAbove}(A, r)$ and A2: $B \subseteq A$
 shows $\text{IsBoundedAbove}(B, r)$

<proof>

If for every element of X we can find one in A that is greater, then the A can not be bounded above. Works for relations that are total, transitive and antisymmetric, (i.e. for linear order relations).

```
lemma Order_ZF_3_L14:
  assumes A1: r {is total on} X
  and A2: trans(r) and A3: antisym(r)
  and A4: r  $\subseteq$  X $\times$ X and A5: X $\neq$ 0
  and A6:  $\forall x \in X. \exists a \in A. x \neq a \wedge \langle x, a \rangle \in r$ 
  shows  $\neg$ IsBoundedAbove(A,r)
```

<proof>

The set of elements in a set A that are nongreater than a given element is bounded above.

```
lemma Order_ZF_3_L15: shows IsBoundedAbove( $\{x \in A. \langle x, a \rangle \in r\}$ ,r)
```

<proof>

If A is bounded below, then the set of elements in a set A that are nongreater than a given element is bounded.

```
lemma Order_ZF_3_L16: assumes A1: IsBoundedBelow(A,r)
  shows IsBounded( $\{x \in A. \langle x, a \rangle \in r\}$ ,r)
```

<proof>

end

6 More on order relations

```
theory Order_ZF_1 imports ZF.Order ZF1
```

```
begin
```

In `Order_ZF` we define some notions related to order relations based on the nonstrict orders ($\leq$ type). Some people however prefer to talk about these notions in terms of the strict order relation ($<$ type). This is the case for the standard Isabelle `Order.thy` and also for Metamath. In this theory file we repeat some developments from `Order_ZF` using the strict order relation as a basis. This is mostly useful for Metamath translation, but is also of some general interest. The names of theorems are copied from Metamath.

6.1 Definitions and basic properties

In this section we introduce some definitions taken from Metamath and relate them to the ones used by the standard Isabelle `Order.thy`.

The next definition is the strict version of the linear order. What we write as `R Orders A` is written `ROrdA` in Metamath.

definition

StrictOrder (**infix** **Orders** 65) **where**

$$\begin{aligned} R \text{ Orders } A &\equiv \forall x \ y \ z. (x \in A \wedge y \in A \wedge z \in A) \longrightarrow \\ &(\langle x, y \rangle \in R \longleftrightarrow \neg(x=y \vee \langle y, x \rangle \in R)) \wedge \\ &(\langle x, y \rangle \in R \wedge \langle y, z \rangle \in R \longrightarrow \langle x, z \rangle \in R) \end{aligned}$$

The definition of supremum for a (strict) linear order.

definition

$$\begin{aligned} \text{Sup}(B, A, R) &\equiv \\ &\bigcup \{x \in A. (\forall y \in B. \langle x, y \rangle \notin R) \wedge \\ &(\forall y \in A. \langle y, x \rangle \in R \longrightarrow (\exists z \in B. \langle y, z \rangle \in R))\} \end{aligned}$$

Definition of infimum for a linear order. It is defined in terms of supremum.

definition

$$\text{Infim}(B, A, R) \equiv \text{Sup}(B, A, \text{converse}(R))$$

If relation R orders a set A , (in Metamath sense) then R is irreflexive, transitive and linear therefore is a total order on A (in Isabelle sense).

lemma `orders_imp_tot_ord`: **assumes** $A1$: $R \text{ Orders } A$

shows

`irrefl`(A, R)
`trans`[A](R)
`part_ord`(A, R)
`linear`(A, R)
`tot_ord`(A, R)

<proof>

A converse of `orders_imp_tot_ord`. Together with that theorem this shows that Metamath's notion of an order relation is equivalent to Isabelle's `tot_ord` predicate.

lemma `tot_ord_imp_orders`: **assumes** $A1$: `tot_ord`(A, R)

shows $R \text{ Orders } A$

<proof>

6.2 Properties of (strict) total orders

In this section we discuss the properties of strict order relations. This continues the development contained in the standard Isabelle's `Order.thy` with a view towards using the theorems translated from Metamath.

A relation orders a set iff the converse relation orders a set. Going one way we can use the lemma `tot_od_converse` from the standard Isabelle's `Order.thy`. The other way is a bit more complicated (note that in Isabelle for `converse(converse(r)) = r` one needs r to consist of ordered pairs, which does not follow from the `StrictOrder` definition above).

lemma `cnvso`: **shows** $R \text{ Orders } A \longleftrightarrow \text{converse}(R) \text{ Orders } A$

<proof>

Supremum is unique, if it exists.

lemma `supeu`: **assumes** `A1`: `R Orders A` **and** `A2`: `x ∈ A` **and**
`A3`: $\forall y \in B. \langle x, y \rangle \notin R$ **and** `A4`: $\forall y \in A. \langle y, x \rangle \in R \longrightarrow (\exists z \in B. \langle y, z \rangle \in R)$
shows
 $\exists ! x. x \in A \wedge (\forall y \in B. \langle x, y \rangle \notin R) \wedge (\forall y \in A. \langle y, x \rangle \in R \longrightarrow (\exists z \in B. \langle y, z \rangle \in R))$
`<proof>`

Supremum has expected properties if it exists.

lemma `sup_props`: **assumes** `A1`: `R Orders A` **and**
`A2`: $\exists x \in A. (\forall y \in B. \langle x, y \rangle \notin R) \wedge (\forall y \in A. \langle y, x \rangle \in R \longrightarrow (\exists z \in B. \langle y, z \rangle \in R))$
shows
`Sup(B, A, R) ∈ A`
 $\forall y \in B. \langle \text{Sup}(B, A, R), y \rangle \notin R$
 $\forall y \in A. \langle y, \text{Sup}(B, A, R) \rangle \in R \longrightarrow (\exists z \in B. \langle y, z \rangle \in R)$
`<proof>`

Elements greater or equal than any element of B are greater or equal than supremum of B .

lemma `supnub`: **assumes** `A1`: `R Orders A` **and** `A2`:
 $\exists x \in A. (\forall y \in B. \langle x, y \rangle \notin R) \wedge (\forall y \in A. \langle y, x \rangle \in R \longrightarrow (\exists z \in B. \langle y, z \rangle \in R))$
and `A3`: `c ∈ A` **and** `A4`: $\forall z \in B. \langle c, z \rangle \notin R$
shows $\langle c, \text{Sup}(B, A, R) \rangle \notin R$
`<proof>`

end

7 Even more on order relations

theory `Order_ZF_1a` **imports** `Order_ZF`

begin

This theory is a continuation of `Order_ZF` and talks about maximum and minimum of a set, supremum and infimum and strict (not reflexive) versions of order relations.

7.1 Maximum and minimum of a set

In this section we show that maximum and minimum are unique if they exist. We also show that union of sets that have maxima (minima) has a maximum (minimum). We also show that singletons have maximum and minimum. All this allows to show (in `Finite_ZF`) that every finite set has well-defined maximum and minimum.

A somewhat technical fact that allows to reduce the number of premises in some theorems: the assumption that a set has a maximum implies that it is not empty.

lemma set_max_not_empty: **assumes** HasAmaximum(r,A) **shows** $A \neq \emptyset$
<proof>

If a set has a maximum implies that it is not empty.

lemma set_min_not_empty: **assumes** HasAminimum(r,A) **shows** $A \neq \emptyset$
<proof>

If a set has a supremum then it cannot be empty. We are probably using the fact that $\bigcap \emptyset = \emptyset$, which makes me a bit anxious as this I think is just a convention.

lemma set_sup_not_empty: **assumes** HasAsupremum(r,A) **shows** $A \neq \emptyset$
<proof>

If a set has an infimum then it cannot be empty.

lemma set_inf_not_empty: **assumes** HasAnInfimum(r,A) **shows** $A \neq \emptyset$
<proof>

For antisymmetric relations maximum of a set is unique if it exists.

lemma Order_ZF_4_L1: **assumes** A1: antisym(r) **and** A2: HasAmaximum(r,A) **shows** $\exists! M. M \in A \wedge (\forall x \in A. \langle x, M \rangle \in r)$
<proof>

For antisymmetric relations minimum of a set is unique if it exists.

lemma Order_ZF_4_L2: **assumes** A1: antisym(r) **and** A2: HasAminimum(r,A) **shows** $\exists! m. m \in A \wedge (\forall x \in A. \langle m, x \rangle \in r)$
<proof>

Maximum of a set has desired properties.

lemma Order_ZF_4_L3: **assumes** A1: antisym(r) **and** A2: HasAmaximum(r,A) **shows** $\text{Maximum}(r,A) \in A \wedge \forall x \in A. \langle x, \text{Maximum}(r,A) \rangle \in r$
<proof>

If a set has a maximum then for any subset the set of upper bounds is non-empty.

corollary max_subset_upper_bound:
assumes antisym(r) **and** HasAmaximum(r,A) $B \subseteq A$
shows $\{x \in A. \forall y \in B. \langle y, x \rangle \in r\} \neq \emptyset$
<proof>

Minimum of a set has desired properties.

lemma Order_ZF_4_L4: **assumes** A1: antisym(r) **and** A2: HasAminimum(r,A) **shows** $\text{Minimum}(r,A) \in A \wedge \forall x \in A. \langle \text{Minimum}(r,A), x \rangle \in r$
<proof>

If a set has a minimum then for any subset the set of lower bounds is non-empty.

corollary min_subset_lower_bound:
 assumes antisym(r) and HasAminimum(r,A) $B \subseteq A$
 shows $\{x \in A. \forall y \in B. \langle x, y \rangle \in r\} \neq \emptyset$
<proof>

For total and transitive relations a union of two sets that have maxima has a maximum.

lemma Order_ZF_4_L5:
 assumes A1: r {is total on} (A $\cup$ B) and A2: trans(r)
 and A3: HasAmaximum(r,A) HasAmaximum(r,B)
 shows HasAmaximum(r,A $\cup$ B)
<proof>

For total and transitive relations A union of two sets that have minima has a minimum.

lemma Order_ZF_4_L6:
 assumes A1: r {is total on} (A $\cup$ B) and A2: trans(r)
 and A3: HasAminimum(r,A) HasAminimum(r,B)
 shows HasAminimum(r,A $\cup$ B)
<proof>

Set that has a maximum is bounded above.

lemma Order_ZF_4_L7:
 assumes HasAmaximum(r,A)
 shows IsBoundedAbove(A,r)
<proof>

Set that has a minimum is bounded below.

lemma Order_ZF_4_L8A:
 assumes HasAminimum(r,A)
 shows IsBoundedBelow(A,r)
<proof>

A subset of a set that has a maximum is bounded above.

lemma max_subset_bounded: assumes HasAmaximum(r,A) and $B \subseteq A$
 shows IsBoundedAbove(B,r)
<proof>

A subset of a set that has a minimum is bounded below.

lemma min_subset_bounded: assumes HasAminimum(r,A) and $B \subseteq A$
 shows IsBoundedBelow(B,r)
<proof>

For reflexive relations singletons have a minimum and maximum.

lemma Order_ZF_4_L8: assumes refl(X,r) and $a \in X$

shows HasAmaximum($r, \{a\}$) HasAminimum($r, \{a\}$)
<proof>

If total and transitive relations if we add an element to a set that has a maximum, the set still has a maximum.

lemma Order_ZF_4_L9:
assumes A1: r {is total on} X and A2: $\text{trans}(r)$
and A3: $A \subseteq X$ and A4: $a \in X$ and A5: HasAmaximum(r, A)
shows HasAmaximum($r, A \cup \{a\}$)
<proof>

If a set has a minimum and L is less or equal than all elements of the set, then L is less or equal than the minimum.

lemma Order_ZF_4_L12:
assumes antisym(r) and HasAminimum(r, A) and $\forall a \in A. \langle L, a \rangle \in r$
shows $\langle L, \text{Minimum}(r, A) \rangle \in r$
<proof>

If a set has a maximum and all its elements are less or equal than M , then the maximum of the set is less or equal than M .

lemma Order_ZF_4_L13:
assumes antisym(r) and HasAmaximum(r, A) and $\forall a \in A. \langle a, M \rangle \in r$
shows $\langle \text{Maximum}(r, A), M \rangle \in r$
<proof>

If an element belongs to a set and is greater or equal than all elements of that set, then it is the maximum of that set.

lemma Order_ZF_4_L14:
assumes A1: antisym(r) and A2: $M \in A$ and
A3: $\forall a \in A. \langle a, M \rangle \in r$
shows $\text{Maximum}(r, A) = M$
<proof>

If r is antisymmetric, an element belongs to a set and is less or equal than all elements of that set, then it is the minimum of that set.

lemma Order_ZF_4_L15:
assumes A1: antisym(r) and A2: $m \in A$ and
A3: $\forall a \in A. \langle m, a \rangle \in r$
shows $\text{Minimum}(r, A) = m$
<proof>

If r is an antisymmetric relation on nonempty set X and X is bounded below in that relation, then X has a minimum.

lemma bouded_below_min:
assumes $X \neq \emptyset$ $r \subseteq X \times X$ antisym(r) IsBoundedBelow(X, r)
shows HasAminimum(r, X)
<proof>

If r is an antisymmetric relation on nonempty set X and X is bounded above in that relation, then X has a maximum.

lemma `bouded_above_max`:
 assumes $X \neq \emptyset$ $r \subseteq X \times X$ $\text{antisym}(r)$ $\text{IsBoundedAbove}(X, r)$
 shows $\text{HasAmaximum}(r, X)$
<proof>

For total and transitive relations if we add an element to a set that has a minimum, the set still has a minimum.

lemma `Order_ZF_4_L10`:
 assumes $A1$: r {is total on} X and $A2$: $\text{trans}(r)$
 and $A3$: $A \subseteq X$ and $A4$: $a \in X$ and $A5$: $\text{HasAminimum}(r, A)$
 shows $\text{HasAminimum}(r, A \cup \{a\})$
<proof>

If the order relation has a property that every nonempty bounded set attains a minimum (for example integers are like that), then every nonempty set bounded below attains a minimum.

lemma `Order_ZF_4_L11`:
 assumes $A1$: r {is total on} X and
 $A2$: $\text{trans}(r)$ and
 $A3$: $r \subseteq X \times X$ and
 $A4$: $\forall A. \text{IsBounded}(A, r) \wedge A \neq \emptyset \longrightarrow \text{HasAminimum}(r, A)$ and
 $A5$: $B \neq \emptyset$ and $A6$: $\text{IsBoundedBelow}(B, r)$
 shows $\text{HasAminimum}(r, B)$
<proof>

A dual to `Order_ZF_4_L11`: If the order relation has a property that every nonempty bounded set attains a maximum (for example integers are like that), then every nonempty set bounded above attains a maximum.

lemma `Order_ZF_4_L11A`:
 assumes $A1$: r {is total on} X and
 $A2$: $\text{trans}(r)$ and
 $A3$: $r \subseteq X \times X$ and
 $A4$: $\forall A. \text{IsBounded}(A, r) \wedge A \neq \emptyset \longrightarrow \text{HasAmaximum}(r, A)$ and
 $A5$: $B \neq \emptyset$ and $A6$: $\text{IsBoundedAbove}(B, r)$
 shows $\text{HasAmaximum}(r, B)$
<proof>

If a set does not have a maximum, then for any its element we can find one that is (strictly) greater.

lemma `Order_ZF_4_L16`:
 assumes $A1$: $\text{antisym}(r)$ and $A2$: r {is total on} X and
 $A3$: $A \subseteq X$ and
 $A4$: $\neg \text{HasAmaximum}(r, A)$ and
 $A5$: $x \in A$
 shows $\exists y \in A. \langle x, y \rangle \in r \wedge y \neq x$
<proof>

7.2 Supremum and Infimum

In this section we consider the notions of supremum and infimum a set.

Elements of the set of upper bounds are indeed upper bounds. Isabelle also thinks it is obvious.

lemma Order_ZF_5_L1: **assumes** $u \in (\bigcap a \in A. r\{a\})$ **and** $a \in A$
shows $\langle a, u \rangle \in r$
<proof>

Elements of the set of lower bounds are indeed lower bounds. Isabelle also thinks it is obvious.

lemma Order_ZF_5_L2: **assumes** $l \in (\bigcap a \in A. r-\{a\})$ **and** $a \in A$
shows $\langle l, a \rangle \in r$
<proof>

If the set of upper bounds has a minimum, then the supremum is less or equal than any upper bound. We can probably do away with the assumption that A is not empty, (ab)using the fact that intersection over an empty family is defined in Isabelle to be empty. This lemma is obsolete and will be removed in the future. Use `sup_leq_up_bnd` instead.

lemma Order_ZF_5_L3: **assumes** A1: `antisym(r)` **and** A2: $A \neq 0$ **and**
A3: `HasAminimum(r,  $\bigcap a \in A. r\{a\}$ )` **and**
A4: $\forall a \in A. \langle a, u \rangle \in r$
shows $\langle \text{Supremum}(r, A), u \rangle \in r$
<proof>

Supremum is less or equal than any upper bound.

lemma sup_leq_up_bnd: **assumes** `antisym(r)` `HasAsupremum(r, A)` $\forall a \in A. \langle a, u \rangle \in r$
shows $\langle \text{Supremum}(r, A), u \rangle \in r$
<proof>

If the relation is antisymmetric and complete then the supremum of a nonempty set A with an upper bound u is less or equal u .

lemma compl_sup_leq_up_bnd:
assumes `antisym(r)` `r {is complete}` $A \neq \emptyset$ $\forall a \in A. \langle a, u \rangle \in r$
shows $\langle \text{Supremum}(r, A), u \rangle \in r$
<proof>

Infimum is greater or equal than any lower bound. This lemma is obsolete and will be removed. Use `inf_geq_lo_bnd` instead.

lemma Order_ZF_5_L4: **assumes** A1: `antisym(r)` **and** A2: $A \neq 0$ **and**
A3: `HasAmaximum(r,  $\bigcap a \in A. r-\{a\}$ )` **and**
A4: $\forall a \in A. \langle l, a \rangle \in r$
shows $\langle l, \text{Infimum}(r, A) \rangle \in r$
<proof>

Infimum is greater or equal than any upper bound.

lemma `inf_geq_lo_bnd`: `assumes` `antisym(r)` `HasAnInfimum(r,A)` $\forall a \in A. \langle u, a \rangle \in r$
`shows` $\langle u, \text{Infimum}(r, A) \rangle \in r$
`<proof>`

If z is an upper bound for A and is less or equal than any other upper bound, then z is the supremum of A .

lemma `Order_ZF_5_L5`: `assumes` `A1: antisym(r)` `and` `A2: A ≠ 0` `and`
`A3:  $\forall x \in A. \langle x, z \rangle \in r$` `and`
`A4:  $\forall y. (\forall x \in A. \langle x, y \rangle \in r) \longrightarrow \langle z, y \rangle \in r$`
`shows`
`HasAminimum(r,  $\bigcap a \in A. r\{a\}$ )`
`z = Supremum(r, A)`
`<proof>`

The dual theorem to `Order_ZF_5_L5`: if z is an lower bound for A and is greater or equal than any other lower bound, then z is the infimum of A .

lemma `inf_glb`:
`assumes` `antisym(r)` `A ≠ 0` $\forall x \in A. \langle z, x \rangle \in r$ $\forall y. (\forall x \in A. \langle y, x \rangle \in r) \longrightarrow \langle y, z \rangle \in r$
`shows`
`HasAmaximum(r,  $\bigcap a \in A. r - \{a\}$ )`
`z = Infimum(r, A)`
`<proof>`

Supremum and infimum of a singleton is the element.

lemma `sup_inf_singl`: `assumes` `antisym(r)` `refl(X, r)` $z \in X$
`shows`
`HasAsupremum(r, {z})` `Supremum(r, {z}) = z` `and`
`HasAninfimum(r, {z})` `Infimum(r, {z}) = z`
`<proof>`

If a set has a maximum, then the maximum is the supremum. This lemma is obsolete, use `max_is_sup` instead.

lemma `Order_ZF_5_L6`:
`assumes` `A1: antisym(r)` `and` `A2: A ≠ 0` `and`
`A3: HasAmaximum(r, A)`
`shows`
`HasAminimum(r,  $\bigcap a \in A. r\{a\}$ )`
`Maximum(r, A) = Supremum(r, A)`
`<proof>`

Another version of `Order_ZF_5_L6` that: if a set has a maximum then it has a supremum and the maximum is the supremum.

lemma `max_is_sup`: `assumes` `antisym(r)` `HasAmaximum(r, A)`
`shows` `HasAsupremum(r, A)` `and` `Maximum(r, A) = Supremum(r, A)`

<proof>

Supremum of a set, if exists, is an upper bound for that set.

lemma sup_is_ub: **assumes** antisym(r) HasAsupremum(r,A) $x \in A$
shows $\langle x, \text{Supremum}(r,A) \rangle \in r$
<proof>

Infimum of a set, if exists, is a lower bound for that set.

lemma inf_is_lb: **assumes** antisym(r) HasAnInfimum(r,A) $x \in A$
shows $\langle \text{Infimum}(r,A), x \rangle \in r$
<proof>

If a set has a supremum and that supremum is a member of the set, then it has a maximum and the maximum is that supremum.

lemma sup_is_max:
assumes antisym(r) HasAsupremum(r,A) $\text{Supremum}(r,A) \in A$
shows HasAmaximum(r,A) **and** $\text{Maximum}(r,A) = \text{Supremum}(r,A)$
<proof>

Minimum is the infimum if it exists.

lemma min_is_inf: **assumes** antisym(r) HasAminimum(r,A)
shows HasAnInfimum(r,A) **and** $\text{Minimum}(r,A) = \text{Infimum}(r,A)$
<proof>

If a set has a infimum and that infimum is a member of the set, then it has a minimum and the minimum is that infimum.

lemma inf_is_min:
assumes antisym(r) HasAnInfimum(r,A) $\text{Infimum}(r,A) \in A$
shows HasAminimum(r,A) **and** $\text{Minimum}(r,A) = \text{Infimum}(r,A)$
<proof>

For reflexive and total relations two-element set has a minimum and a maximum.

lemma min_max_two_el: **assumes** r {is total on} X $x \in X$ $y \in X$
shows HasAminimum(r,{x,y}) **and** HasAmaximum(r,{x,y})
<proof>

For antisymmetric, reflexive and total relations two-element set has a supremum and infimum.

lemma inf_sup_two_el: **assumes** antisym(r) r {is total on} X $x \in X$ $y \in X$
shows
HasAnInfimum(r,{x,y})
 $\text{Minimum}(r,{x,y}) = \text{Infimum}(r,{x,y})$
HasAsupremum(r,{x,y})
 $\text{Maximum}(r,{x,y}) = \text{Supremum}(r,{x,y})$
<proof>

A sufficient condition for the supremum to be in the space.

lemma sup_in_space:
 assumes $r \subseteq X \times X$ antisym(r) HasAminimum($r, \bigcap_{a \in A} r\{a\}$)
 shows $\text{Supremum}(r, A) \in X$ and $\forall x \in A. \langle x, \text{Supremum}(r, A) \rangle \in r$
<proof>

A sufficient condition for the infimum to be in the space.

lemma inf_in_space:
 assumes $r \subseteq X \times X$ antisym(r) HasAmaximum($r, \bigcap_{a \in A} r\{a\}$)
 shows $\text{Infimum}(r, A) \in X$ and $\forall x \in A. \langle \text{Infimum}(r, A), x \rangle \in r$
<proof>

In the definition of the HasAnInfimum predicate (see the Order_ZF theory) we use a somewhat unconventional way of describing the set of lower bounds of a set A as $\bigcap_{a \in A} r^{-1}(\{a\})$. The next lemma shows that for relations defined on X and non-empty sets A this is the same as the more standard $\{x \in X : \forall a \in A \langle x, a \rangle \in r\}$. Under the same assumption a similar identity holds between $\bigcap_{a \in A} r(\{a\})$ and $\{x \in X : \forall a \in A \langle a, x \rangle \in r\}$, i.e. for the set of upper bounds. This implies we can formulate the definitions of the HasAsupremum and HasAnInfimum predicates and the expressions for the values of supremum and infimum in terms of this more standard notation.

lemma lower_upper_bounds_alt:
 assumes $r \subseteq X \times X$ $A \neq \emptyset$ shows
 $(\bigcap_{a \in A} r\{a\}) = \{x \in X. \forall a \in A. \langle x, a \rangle \in r\}$
 $(\bigcap_{a \in A} r\{a\}) = \{x \in X. \forall a \in A. \langle a, x \rangle \in r\}$
 $\text{HasAsupremum}(r, A) \longleftrightarrow \text{HasAminimum}(r, \{x \in X. \forall a \in A. \langle a, x \rangle \in r\})$
 $\text{HasAnInfimum}(r, A) \longleftrightarrow \text{HasAmaximum}(r, \{x \in X. \forall a \in A. \langle x, a \rangle \in r\})$
 $\text{Supremum}(r, A) = \text{Minimum}(r, \{x \in X. \forall a \in A. \langle a, x \rangle \in r\})$
 $\text{Infimum}(r, A) = \text{Maximum}(r, \{x \in X. \forall a \in A. \langle x, a \rangle \in r\})$
<proof>

Properties of supremum of a bounded set for complete relations.

lemma Order_ZF_5_L7:
 assumes A1: $r \subseteq X \times X$ and A2: antisym(r) and
 A3: r {is complete} and
 A4: $A \neq \emptyset$ and A5: $\exists x \in X. \forall y \in A. \langle y, x \rangle \in r$
 shows $\text{Supremum}(r, A) \in X$ and $\forall x \in A. \langle x, \text{Supremum}(r, A) \rangle \in r$
<proof>

Similar to Order_ZF_5_L7 with less explicit boundedness condition: if the relation on X is antisymmetric and complete then the supremum of a nonempty bounded set A is an element of X and is an upper bound of A .

lemma compl_bounded_sup_props:
 assumes $r \subseteq X \times X$ antisym(r) r {is complete} and
 $A \neq \emptyset$ IsBoundedAbove(A, r)
 shows $\text{Supremum}(r, A) \in X$ and $\forall x \in A. \langle x, \text{Supremum}(r, A) \rangle \in r$
<proof>

If r is an antisymmetric relation on a set X which has a minimum, and every nonempty subset of X has a supremum then every nonempty subset of X has an infimum. Note that in view of lemma `bouded_below_min` we could assume that X is bounded below instead of assuming that it has a minimum.

lemma `bounded_below_sups_infs`:
assumes $A \subseteq X$ $A \neq \emptyset$ $r \subseteq X \times X$ `antisym(r)` `HasAminimum(r,X)` **and**
 $\forall B \in \text{Pow}(X) \setminus \{\emptyset\}. \text{HasAsupremum}(r,B)$
shows `HasAnInfimum(r,A)` **and**
 $\text{Infimum}(r,A) = \text{Maximum}(r, \{y \in X. \forall x \in A. \langle y, x \rangle \in r\})$
 $\text{Infimum}(r,A) = \text{Supremum}(r, \{y \in X. \forall x \in A. \langle y, x \rangle \in r\})$
 $\langle \text{proof} \rangle$

A theorem dual to `bounded_above_infs_sups`: if a relation is antisymmetric, the whole space has a maximum and every nonempty subset has an infimum, then every nonempty subset has a supremum.

lemma `bounded_above_infs_sups`:
assumes $A \subseteq X$ $A \neq \emptyset$ $r \subseteq X \times X$ `antisym(r)` `HasAmaximum(r,X)` **and**
 $\forall B \in \text{Pow}(X) \setminus \{\emptyset\}. \text{HasAnInfimum}(r,B)$
shows `HasAsupremum(r,A)` **and**
 $\text{Supremum}(r,A) = \text{Minimum}(r, \{y \in X. \forall x \in A. \langle x, y \rangle \in r\})$
 $\text{Supremum}(r,A) = \text{Infimum}(r, \{y \in X. \forall x \in A. \langle x, y \rangle \in r\})$
 $\langle \text{proof} \rangle$

Infimum of the set of infima of a collection of sets is infimum of the union.

lemma `inf_inf`:
assumes
 $r \subseteq X \times X$ `antisym(r)` `trans(r)`
 $\forall T \in \mathcal{T}. \text{HasAnInfimum}(r,T)$
 $\text{HasAnInfimum}(r, \{\text{Infimum}(r,T). T \in \mathcal{T}\})$
shows
 $\text{HasAnInfimum}(r, \bigcup \mathcal{T})$ **and** $\text{Infimum}(r, \{\text{Infimum}(r,T). T \in \mathcal{T}\}) = \text{Infimum}(r, \bigcup \mathcal{T})$
 $\langle \text{proof} \rangle$

Supremum of the set of suprema of a collection of sets is supremum of the union.

lemma `sup_sup`:
assumes
 $r \subseteq X \times X$ `antisym(r)` `trans(r)`
 $\forall T \in \mathcal{T}. \text{HasAsupremum}(r,T)$
 $\text{HasAsupremum}(r, \{\text{Supremum}(r,T). T \in \mathcal{T}\})$
shows
 $\text{HasAsupremum}(r, \bigcup \mathcal{T})$ **and** $\text{Supremum}(r, \{\text{Supremum}(r,T). T \in \mathcal{T}\}) = \text{Supremum}(r, \bigcup \mathcal{T})$
 $\langle \text{proof} \rangle$

If the relation is a linear order then for any element y smaller than the supremum of a set we can find one element of the set that is greater than y .

lemma `Order_ZF_5_L8`:

```

assumes A1:  $r \subseteq X \times X$  and A2:  $\text{IsLinOrder}(X, r)$  and
A3:  $r$  {is complete} and
A4:  $A \subseteq X$   $A \neq 0$  and A5:  $\exists x \in X. \forall y \in A. \langle y, x \rangle \in r$  and
A6:  $\langle y, \text{Supremum}(r, A) \rangle \in r \quad y \neq \text{Supremum}(r, A)$ 
shows  $\exists z \in A. \langle y, z \rangle \in r \wedge y \neq z$ 
<proof>

```

7.3 Strict versions of order relations

One of the problems with translating formalized mathematics from Metamath to IsarMathLib is that Metamath uses strict orders (of the $<$ type) while in IsarMathLib we mostly use nonstrict orders (of the $\leq$ type). This doesn't really make any difference, but is annoying as we have to prove many theorems twice. In this section we prove some theorems to make it easier to translate the statements about strict orders to statements about the corresponding non-strict order and vice versa.

We define a strict version of a relation by removing the $y = x$ line from the relation.

definition

$\text{StrictVersion}(r) \equiv r - \{\langle x, x \rangle. x \in \text{domain}(r)\}$

A reformulation of the definition of a strict version of an order.

lemma def_of_strict_ver: shows

$\langle x, y \rangle \in \text{StrictVersion}(r) \longleftrightarrow \langle x, y \rangle \in r \wedge x \neq y$
 <proof>

The next lemma is about the strict version of an antisymmetric relation.

lemma strict_of_antisym:

assumes A1: $\text{antisym}(r)$ **and** A2: $\langle a, b \rangle \in \text{StrictVersion}(r)$
shows $\langle b, a \rangle \notin \text{StrictVersion}(r)$

<proof>

The strict version of totality.

lemma strict_of_tot:

assumes r {is total on} X **and** $a \in X \quad b \in X \quad a \neq b$
shows $\langle a, b \rangle \in \text{StrictVersion}(r) \vee \langle b, a \rangle \in \text{StrictVersion}(r)$
 <proof>

A trichotomy law for the strict version of a total and antisymmetric relation. It is kind of interesting that one does not need the full linear order for this.

lemma strict_ans_tot_trich:

assumes A1: $\text{antisym}(r)$ **and** A2: r {is total on} X
and A3: $a \in X \quad b \in X$
and A4: $s = \text{StrictVersion}(r)$
shows $\text{Exactly_1_of_3_holds}(\langle a, b \rangle \in s, a = b, \langle b, a \rangle \in s)$

<proof>

A trichotomy law for linear order. This is a special case of `strict_ans_tot_trich`.

corollary `strict_lin_trich`: assumes A1: `IsLinOrder(X,r)` and
 A2: `a∈X b∈X` and
 A3: `s = StrictVersion(r)`
 shows `Exactly_1_of_3_holds(⟨a,b⟩ ∈ s, a=b, ⟨b,a⟩ ∈ s)`
⟨proof⟩

For an antisymmetric relation if a pair is in relation then the reversed pair is not in the strict version of the relation.

lemma `geq_impl_not_less`:
 assumes A1: `antisym(r)` and A2: `⟨a,b⟩ ∈ r`
 shows `⟨b,a⟩ ∉ StrictVersion(r)`
⟨proof⟩

If an antisymmetric relation is transitive, then the strict version is also transitive, an explicit version `strict_of_transB` below.

lemma `strict_of_transA`:
 assumes A1: `trans(r)` and A2: `antisym(r)` and
 A3: `s= StrictVersion(r)` and A4: `⟨a,b⟩ ∈ s ⟨b,c⟩ ∈ s`
 shows `⟨a,c⟩ ∈ s`
⟨proof⟩

If an antisymmetric relation is transitive, then the strict version is also transitive.

lemma `strict_of_transB`:
 assumes A1: `trans(r)` and A2: `antisym(r)`
 shows `trans(StrictVersion(r))`
⟨proof⟩

The next lemma provides a condition that is satisfied by the strict version of a relation if the original relation is a complete linear order.

lemma `strict_of_compl`:
 assumes A1: `r ⊆ X×X` and A2: `IsLinOrder(X,r)` and
 A3: `r {is complete}` and
 A4: `A⊆X A≠0` and A5: `s = StrictVersion(r)` and
 A6: `∃u∈X. ∀y∈A. ⟨y,u⟩ ∈ s`
 shows
 $\exists x \in X. (\forall y \in A. \langle x, y \rangle \notin s) \wedge (\forall y \in X. \langle y, x \rangle \in s \longrightarrow (\exists z \in A. \langle y, z \rangle \in s))$
⟨proof⟩

Strict version of a relation on a set is a relation on that set.

lemma `strict_ver_rel`: assumes A1: `r ⊆ A×A`
 shows `StrictVersion(r) ⊆ A×A`
⟨proof⟩

end

8 Order on natural numbers

theory NatOrder_ZF **imports** Nat_ZF_IML Order_ZF

begin

This theory proves that $\leq$ is a linear order on $\mathbb{N}$. $\leq$ is defined in Isabelle's `Nat` theory, and linear order is defined in `Order_ZF` theory. Contributed by Seo Sanghyeon.

8.1 Order on natural numbers

This is the only section in this theory.

If a, b are natural numbers then a is less or equal b or b is (strictly) less than a . We use a result on ordinals in the proof.

lemma nat_order_2cases: **assumes** $a \in \text{nat}$ **and** $b \in \text{nat}$
shows $a \leq b \vee b < a$
<proof>

A special case of `nat_order_2cases`: If a, b are natural numbers then a is less or equal b or b is less or equal than a .

lemma NatOrder_ZF_1_L1:
assumes $a \in \text{nat}$ **and** $b \in \text{nat}$
shows $a \leq b \vee b \leq a$
<proof>

$\leq$ is antisymmetric, transitive, total, and linear. Proofs by rewrite using definitions.

lemma NatOrder_ZF_1_L2:
shows
 $\text{antisym}(\text{Le})$
 $\text{trans}(\text{Le})$
 $\text{Le } \{\text{is total on}\} \text{ nat}$
 $\text{IsLinOrder}(\text{nat}, \text{Le})$
<proof>

The order on natural numbers is linear on every natural number. Recall that each natural number is a subset of the set of all natural numbers (as well as a member).

lemma natord_lin_on_each_nat:
assumes $A1: n \in \text{nat}$ **shows** $\text{IsLinOrder}(n, \text{Le})$
<proof>

end

9 Functions - introduction

theory func1 imports ZF.func Fol1 ZF1

begin

This theory covers basic properties of function spaces. A set of functions with domain X and values in the set Y is denoted in Isabelle as $X \rightarrow Y$. It just happens that the colon ":" is a synonym of the set membership symbol $\in$ in Isabelle/ZF so we can write $f : X \rightarrow Y$ instead of $f \in X \rightarrow Y$. This is the only case that we use the colon instead of the regular set membership symbol.

9.1 Properties of functions, function spaces and (inverse) images.

Functions in ZF are sets of pairs. This means that if $f : X \rightarrow Y$ then $f \subseteq X \times Y$. This section is mostly about consequences of this understanding of the notion of function.

We define the notion of function that preserves a collection here. Given two collection of sets a function preserves the collections if the inverse image of sets in one collection belongs to the second one. This notion does not have a name in romantic math. It is used to define continuous functions in `Topology_ZF_2` theory. We define it here so that we can use it for other purposes, like defining measurable functions. Recall that $f^{-1}(A)$ means the inverse image of the set A .

definition

$\text{PresColl}(f, S, T) \equiv \forall A \in T. f^{-1}(A) \in S$

A definition that allows to get the first factor of the domain of a binary function $f : X \times Y \rightarrow Z$.

definition

$\text{fst_dom}(f) \equiv \text{domain}(\text{domain}(f))$

If a function maps A into another set, then A is the domain of the function.

lemma func1_1_L1: **assumes** $f : A \rightarrow C$ **shows** $\text{domain}(f) = A$
<proof>

Standard Isabelle defines a `function(f)` predicate. The next lemma shows that our functions satisfy that predicate. It is a special version of Isabelle's `fun_is_function`.

lemma fun_is_fun: **assumes** $f : X \rightarrow Y$ **shows** `function(f)`
<proof>

A lemma explains what `fst\_dom` is for.

lemma fstdomdef: **assumes** A1: $f: X \times Y \rightarrow Z$ **and** A2: $Y \neq \emptyset$
shows $\text{fstdom}(f) = X$
<proof>

A version of the Pi_type lemma from the standard Isabelle/ZF library.

lemma func1_1_L1A: **assumes** A1: $f: X \rightarrow Y$ **and** A2: $\forall x \in X. f(x) \in Z$
shows $f: X \rightarrow Z$
<proof>

A variant of func1_1_L1A.

lemma func1_1_L1B: **assumes** A1: $f: X \rightarrow Y$ **and** A2: $Y \subseteq Z$
shows $f: X \rightarrow Z$
<proof>

There is a value for each argument.

lemma func1_1_L2: **assumes** A1: $f: X \rightarrow Y$ $x \in X$
shows $\exists y \in Y. \langle x, y \rangle \in f$
<proof>

The inverse image is the image of converse. True for relations as well.

lemma vimage_converse: **shows** $r^{-1}(A) = \text{converse}(r)(A)$
<proof>

The image is the inverse image of converse.

lemma image_converse: **shows** $\text{converse}(r)^{-1}(A) = r(A)$
<proof>

The inverse image by a composition is the composition of inverse images.

lemma vimage_comp: **shows** $(r \circ s)^{-1}(A) = s^{-1}(r^{-1}(A))$
<proof>

A version of vimage_comp for three functions.

lemma vimage_comp3: **shows** $(r \circ s \circ t)^{-1}(A) = t^{-1}(s^{-1}(r^{-1}(A)))$
<proof>

Inverse image of any set is contained in the domain.

lemma func1_1_L3: **assumes** A1: $f: X \rightarrow Y$ **shows** $f^{-1}(D) \subseteq X$
<proof>

The inverse image of the codomain is the domain.

lemma func1_1_L4: **assumes** $f: X \rightarrow Y$ **shows** $f^{-1}(Y) = X$
<proof>

The arguments belongs to the domain and values to the codomain.

lemma func1_1_L5:
assumes A1: $\langle x, y \rangle \in f$ **and** A2: $f: X \rightarrow Y$
shows $x \in X \wedge y \in Y$

<proof>

Range of a union is the same as union of ranges. The standard Isabelle/ZF library proves analogous fact for domains (see `domain_Union` in the `equalities` theory).

lemma `range_Union`: **shows** $\text{range}(\bigcup A) = (\bigcup_{x \in A} \text{range}(x))$
<proof>

Function is a subset of cartesian product.

lemma `fun_subset_prod`: **assumes** $A1: f: X \rightarrow Y$ **shows** $f \subseteq X \times Y$
<proof>

The (argument, value) pair belongs to the graph of the function.

lemma `func1_1_L5A`:
 assumes $A1: f: X \rightarrow Y \quad x \in X \quad y = f(x)$
 shows $\langle x, y \rangle \in f \quad y \in \text{range}(f)$
<proof>

The next theorem illustrates the meaning of the concept of function in ZF.

theorem `fun_is_set_of_pairs`: **assumes** $A1: f: X \rightarrow Y$
 shows $f = \{\langle x, f(x) \rangle. x \in X\}$
<proof>

If a pair $\langle x, y \rangle$ is a member of a function f , then x is in the domain and $y = f(x)$.

lemma `pair_fun_member`: **assumes** $f: X \rightarrow Y$ **and** $\langle x, y \rangle \in f$
 shows $x \in X$ **and** $y = f(x)$
<proof>

The range of function that maps X into Y is contained in Y .

lemma `func1_1_L5B`:
 assumes $A1: f: X \rightarrow Y$ **shows** $\text{range}(f) \subseteq Y$
<proof>

The image of any set is contained in the range.

lemma `func1_1_L6`: **assumes** $A1: f: X \rightarrow Y$
 shows $f(B) \subseteq \text{range}(f)$ **and** $f(B) \subseteq Y$
<proof>

The inverse image of any set is contained in the domain.

lemma `func1_1_L6A`: **assumes** $A1: f: X \rightarrow Y$ **shows** $f^{-1}(A) \subseteq X$
<proof>

Image of a greater set is greater.

lemma `func1_1_L8`: **assumes** $A1: A \subseteq B$ **shows** $f(A) \subseteq f(B)$
<proof>

An immediate corollary of `vimage_mono` from the Isabelle/ZF distribution - the inverse image of a greater set is greater. Note we do not require that f is a function, so this is true for relations as well.

lemma `vimage_mono1`: **assumes** $A \subseteq B$ **shows** $f^{-1}(A) \subseteq f^{-1}(B)$
<proof>

A set is contained in the the inverse image of its image. There is similar theorem in `equalities.thy` (`function_image_vimage`) which shows that the image of inverse image of a set is contained in the set.

lemma `func1_1_L9`: **assumes** $A1: f: X \rightarrow Y$ **and** $A2: A \subseteq X$
shows $A \subseteq f^{-1}(f(A))$
<proof>

The inverse image of the image of the domain is the domain.

lemma `inv_im_dom`: **assumes** $A1: f: X \rightarrow Y$ **shows** $f^{-1}(f(X)) = X$
<proof>

A technical lemma needed to make the `func1_1_L11` proof more clear.

lemma `func1_1_L10`:
assumes $A1: f \subseteq X \times Y$ **and** $A2: \exists !y. (y \in Y \wedge \langle x, y \rangle \in f)$
shows $\exists !y. \langle x, y \rangle \in f$
<proof>

If $f \subseteq X \times Y$ and for every $x \in X$ there is exactly one $y \in Y$ such that $\langle x, y \rangle \in f$ then f maps X to Y .

lemma `func1_1_L11`:
assumes $f \subseteq X \times Y$ **and** $\forall x \in X. \exists !y. y \in Y \wedge \langle x, y \rangle \in f$
shows $f: X \rightarrow Y$ *<proof>*

A set defined by a lambda-type expression is a function. There is a similar lemma in `func.thy`, but I had problems with lambda expressions syntax so I could not apply it. This lemma is a workaround for this. Besides, lambda expressions are not readable.

lemma `func1_1_L11A`: **assumes** $A1: \forall x \in X. b(x) \in Y$
shows $\{\langle x, y \rangle \in X \times Y. b(x) = y\} : X \rightarrow Y$
<proof>

The next lemma will replace `func1_1_L11A` one day.

lemma `ZF_fun_from_total`: **assumes** $A1: \forall x \in X. b(x) \in Y$
shows $\{\langle x, b(x) \rangle. x \in X\} : X \rightarrow Y$
<proof>

The value of a function defined by a meta-function is this meta-function (deprecated, use `ZF_fun_from_tot_val(1)` instead).

lemma `func1_1_L11B`:
assumes $A1: f: X \rightarrow Y$ $x \in X$

and A2: $f = \{\langle x, y \rangle \in X \times Y. b(x) = y\}$
shows $f(x) = b(x)$
 $\langle proof \rangle$

The next lemma will replace func1_1_L11B one day.

lemma ZF_fun_from_tot_val:
assumes $f: X \rightarrow Y$ $x \in X$
and $f = \{\langle x, b(x) \rangle. x \in X\}$
shows $f(x) = b(x)$ **and** $b(x) \in Y$
 $\langle proof \rangle$

Identical meaning as ZF_fun_from_tot_val, but phrased a bit differently.

lemma ZF_fun_from_tot_val0:
assumes $f: X \rightarrow Y$ **and** $f = \{\langle x, b(x) \rangle. x \in X\}$
shows $\forall x \in X. f(x) = b(x)$
 $\langle proof \rangle$

Another way of expressing that lambda expression is a function.

lemma lam_is_fun_range: **assumes** $f = \{\langle x, g(x) \rangle. x \in X\}$
shows $f: X \rightarrow \text{range}(f)$
 $\langle proof \rangle$

Yet another way of expressing value of a function.

lemma ZF_fun_from_tot_val1:
assumes $x \in X$ **shows** $\{\langle x, b(x) \rangle. x \in X\}(x) = b(x)$
 $\langle proof \rangle$

An hypotheses-free form of ZF_fun_from_tot_val1: the value of a function $X \ni x \mapsto p(x)$ is $p(x)$ for all $x \in X$.

lemma ZF_fun_from_tot_val2: **shows** $\forall x \in X. \{\langle x, b(x) \rangle. x \in X\}(x) = b(x)$
 $\langle proof \rangle$

The range of a function defined by set comprehension is the set of its values.

lemma range_fun: **shows** $\text{range}(\{\langle x, b(x) \rangle. x \in X\}) = \{b(x). x \in X\}$
 $\langle proof \rangle$

In Isabelle/ZF and Metamath if x is not in the domain of a function f then $f(x)$ is the empty set. This allows us to conclude that if $y \in f(x)$, then x must be an element of the domain of f .

lemma arg_in_domain: **assumes** $f: X \rightarrow Y$ $y \in f(x)$ **shows** $x \in X$
 $\langle proof \rangle$

If x is not in the domain of the function then both the image of the singleton $\{x\}$ and the value of the function are empty. The second of the assertions is also proven by standard Isabelle/ZF apply_0 lemma in the func theory.

lemma arg_not_in_domain: **assumes** $f: X \rightarrow Y$ **and** $x \notin X$

shows $f\{x\} = \emptyset$ **and** $f(x) = \emptyset$
<proof>

A function maps an empty set if and only if it is empty. A well known fact. Yes, an empty set is a function.

lemma `fun_empty_empty`: **shows** $(f:\emptyset\rightarrow X) \longleftrightarrow f=\emptyset$ *<proof>*

We can extend a function by specifying its values on a set disjoint with the domain.

lemma `func1_1_L11C`: **assumes** $A1: f:X\rightarrow Y$ **and** $A2: \forall x\in A. b(x)\in B$
and $A3: X\cap A = \emptyset$ **and** $Dg: g = f \cup \{\langle x, b(x) \rangle. x\in A\}$
shows
 $g : X\cup A \rightarrow Y\cup B$
 $\forall x\in X. g(x) = f(x)$
 $\forall x\in A. g(x) = b(x)$
<proof>

We can extend a function by specifying its value at a point that does not belong to the domain.

lemma `func1_1_L11D`: **assumes** $A1: f:X\rightarrow Y$ **and** $A2: a\notin X$
and $Dg: g = f \cup \{\langle a, b \rangle\}$
shows
 $g : X\cup\{a\} \rightarrow Y\cup\{b\}$
 $\forall x\in X. g(x) = f(x)$
 $g(a) = b$
<proof>

A technical lemma about extending a function both by defining on a set disjoint with the domain and on a point that does not belong to any of those sets.

lemma `func1_1_L11E`:
assumes $A1: f:X\rightarrow Y$ **and**
 $A2: \forall x\in A. b(x)\in B$ **and**
 $A3: X\cap A = \emptyset$ **and** $A4: a \notin X\cup A$
and $Dg: g = f \cup \{\langle x, b(x) \rangle. x\in A\} \cup \{\langle a, c \rangle\}$
shows
 $g : X\cup A\cup\{a\} \rightarrow Y\cup B\cup\{c\}$
 $\forall x\in X. g(x) = f(x)$
 $\forall x\in A. g(x) = b(x)$
 $g(a) = c$
<proof>

A way of defining a function on a union of two possibly overlapping sets. We decompose the union into two differences and the intersection and define a function separately on each part.

lemma `fun_union_overlap`: **assumes** $\forall x\in A\cap B. h(x) \in Y$ $\forall x\in A\setminus B. f(x) \in Y$ $\forall x\in B\setminus A. g(x) \in Y$

shows $\{ \langle x, \text{if } x \in A \setminus B \text{ then } f(x) \text{ else if } x \in B \setminus A \text{ then } g(x) \text{ else } h(x) \rangle . x \in A \cup B \} : A \cup B \rightarrow Y$
<proof>

Inverse image of intersection is the intersection of inverse images.

lemma *invim_inter_inter_invim*: **assumes** $f : X \rightarrow Y$
shows $f^{-1}(A \cap B) = f^{-1}(A) \cap f^{-1}(B)$
<proof>

The inverse image of an intersection of a nonempty collection of sets is the intersection of the inverse images. This generalizes *invim_inter_inter_invim* which is proven for the case of two sets.

lemma *func1_1_L12*:
assumes $A1 : B \subseteq \text{Pow}(Y)$ **and** $A2 : B \neq \emptyset$ **and** $A3 : f : X \rightarrow Y$
shows $f^{-1}(\bigcap B) = (\bigcap_{U \in B} f^{-1}(U))$
<proof>

The inverse image of a set does not change when we intersect the set with the image of the domain.

lemma *inv_im_inter_im*: **assumes** $f : X \rightarrow Y$
shows $f^{-1}(A \cap f(X)) = f^{-1}(A)$
<proof>

If the inverse image of a set is not empty, then the set is not empty. Proof by contradiction.

lemma *func1_1_L13*: **assumes** $A1 : f^{-1}(A) \neq \emptyset$ **shows** $A \neq \emptyset$
<proof>

If a function is not empty (as a set of pairs) then its domain is not empty.

lemma *fun_nempty_domain*: **assumes** $f : X \rightarrow Y$ $p \in f$ **shows** $X \neq \emptyset$
<proof>

If the image of a set is not empty, then the set is not empty. Proof by contradiction.

lemma *func1_1_L13A*: **assumes** $A1 : f(A) \neq \emptyset$ **shows** $A \neq \emptyset$
<proof>

What is the inverse image of a singleton?

lemma *func1_1_L14*: **assumes** $f : X \rightarrow Y$
shows $f^{-1}(\{y\}) = \{x \in X. f(x) = y\}$
<proof>

A lemma that can be used instead *fun_extension_iff* to show that two functions are equal

lemma *func_eq*:
assumes $f : X \rightarrow Y$ $g : X \rightarrow Z$ **and** $\forall x \in X. f(x) = g(x)$

shows $f = g$ $\langle proof \rangle$

An alternative syntax for defining a function: instead of writing $\{\langle x, p(x) \rangle . x \in X\}$ we can write $\lambda x \in X. p(x)$.

lemma `lambda_fun_alt`: **shows** $\{\langle x, p(x) \rangle . x \in X\} = (\lambda x \in X. p(x))$
 $\langle proof \rangle$

If a function is equal to an expression $b(x)$ on X , then it has to be of the form $\{\langle x, b(x) \rangle | x \in X\}$.

lemma `func_eq_set_of_pairs`: **assumes** $f: X \rightarrow Y \ \forall x \in X. f(x) = b(x)$
shows $f = \{\langle x, b(x) \rangle . x \in X\}$
 $\langle proof \rangle$

Function defined on a singleton is a single pair.

lemma `func_singleton_pair`: **assumes** $A1: f : \{a\} \rightarrow X$
shows $f = \{\langle a, f(a) \rangle\}$
 $\langle proof \rangle$

A single pair is a function on a singleton. This is similar to `singleton_fun` from standard Isabelle/ZF.

lemma `pair_func_singleton`: **assumes** $A1: y \in Y$
shows $\{\langle x, y \rangle\} : \{x\} \rightarrow Y$
 $\langle proof \rangle$

The value of a pair on the first element is the second one.

lemma `pair_val`: **shows** $\{\langle x, y \rangle\}(x) = y$
 $\langle proof \rangle$

A more familiar definition of inverse image.

lemma `func1_1_L15`: **assumes** $A1: f: X \rightarrow Y$
shows $f^{-1}(A) = \{x \in X. f(x) \in A\}$
 $\langle proof \rangle$

For symmetric functions inverse images are symmetric.

lemma `symm_vimage_symm`:
assumes $f: X \times X \rightarrow Y$ **and** $\forall x \in X. \forall y \in X. f(x, y) = f(y, x)$
shows $f^{-1}(A) = \text{converse}(f^{-1}(A))$
 $\langle proof \rangle$

A more familiar definition of image.

lemma `func_imagedef`: **assumes** $A1: f: X \rightarrow Y$ **and** $A2: A \subseteq X$
shows $f(A) = \{f(x) . x \in A\}$
 $\langle proof \rangle$

If all elements of a nonempty set map to the same element of the codomain, then the image of this set is a singleton.

lemma `image_constant_singleton`:

assumes $f:X \rightarrow Y$ $A \subseteq X$ $A \neq \emptyset$ $\forall x \in A. f(x) = c$
shows $f(A) = \{c\}$
 $\langle proof \rangle$

A technical lemma about graphs of functions: if we have two disjoint sets A and B then the cartesian product of the inverse image of A and B is disjoint with (the graph of) f .

lemma `vimage_prod_dis_graph`: **assumes** $f:X \rightarrow Y$ $A \cap B = \emptyset$
shows $f^{-1}(A) \times B \cap f = \emptyset$
 $\langle proof \rangle$

For two functions with the same domain X and the codomain Y, Z resp., we can define a third one that maps X to the cartesian product of Y and Z .

lemma `prod_fun_val`:
assumes $\{\langle x, p(x) \rangle. x \in X\}: X \rightarrow Y$ $\{\langle x, q(x) \rangle. x \in X\}: X \rightarrow Z$
defines $h \equiv \{\langle x, \langle p(x), q(x) \rangle \rangle. x \in X\}$
shows $h:X \rightarrow Y \times Z$ **and** $\forall x \in X. h(x) = \langle p(x), q(x) \rangle$
 $\langle proof \rangle$

Suppose we have two functions $f : X \rightarrow Y$ and $g : X \rightarrow Z$ and the third one is defined as $h : X \rightarrow Y \times Z, x \mapsto \langle f(x), g(x) \rangle$. Given two sets U, V we have $h^{-1}(U \times V) = (f^{-1}(U)) \cap (g^{-1}(V))$. We also show that the set where the function f, g are equal is the same as $h^{-1}(\{\langle y, y \rangle : y \in X\})$. It is a bit surprising that we get the last identity without the assumption that $Y = Z$.

lemma `vimage_prod`:
assumes $f:X \rightarrow Y$ $g:X \rightarrow Z$
defines $h \equiv \{\langle x, \langle f(x), g(x) \rangle \rangle. x \in X\}$
shows
 $h:X \rightarrow Y \times Z$
 $\forall x \in X. h(x) = \langle f(x), g(x) \rangle$
 $h^{-1}(U \times V) = f^{-1}(U) \cap g^{-1}(V)$
 $\{x \in X. f(x) = g(x)\} = h^{-1}(\{\langle y, y \rangle. y \in Y\})$
 $\langle proof \rangle$

The image of a set contained in domain under identity is the same set.

lemma `image_id_same`: **assumes** $A \subseteq X$ **shows** $\text{id}(X)(A) = A$
 $\langle proof \rangle$

The inverse image of a set contained in domain under identity is the same set.

lemma `vimage_id_same`: **assumes** $A \subseteq X$ **shows** $\text{id}(X)^{-1}(A) = A$
 $\langle proof \rangle$

What is the image of a singleton?

lemma `singleton_image`:
assumes $f \in X \rightarrow Y$ **and** $x \in X$
shows $f\{x\} = \{f(x)\}$

<proof>

If an element of the domain of a function belongs to a set, then its value belongs to the image of that set.

lemma func1_1_L15D: **assumes** $f:X \rightarrow Y$ $x \in A$ $A \subseteq X$
shows $f(x) \in f(A)$
<proof>

Range is the image of the domain. Isabelle/ZF defines $\text{range}(f)$ as $\text{domain}(\text{converse}(f))$, and that's why we have something to prove here.

lemma range_image_domain:
assumes $A1: f:X \rightarrow Y$ **shows** $f(X) = \text{range}(f)$
<proof>

The difference of images is contained in the image of difference.

lemma diff_image_diff: **assumes** $A1: f: X \rightarrow Y$ **and** $A2: A \subseteq X$
shows $f(X) \setminus f(A) \subseteq f(X \setminus A)$
<proof>

The image of an intersection is contained in the intersection of the images.

lemma image_of_Inter: **assumes** $A1: f:X \rightarrow Y$ **and**
 $A2: I \neq \emptyset$ **and** $A3: \forall i \in I. P(i) \subseteq X$
shows $f(\bigcap_{i \in I} P(i)) \subseteq (\bigcap_{i \in I} f(P(i)))$
<proof>

The image of union is the union of images.

lemma image_of_Union: **assumes** $A1: f:X \rightarrow Y$ **and** $A2: \forall A \in M. A \subseteq X$
shows $f(\bigcup M) = \bigcup \{f(A). A \in M\}$
<proof>

If the domain of a function is nonempty, then the codomain is as well.

lemma codomain_nonempty: **assumes** $f:X \rightarrow Y$ $X \neq \emptyset$ **shows** $Y \neq \emptyset$
<proof>

The image of a nonempty subset of domain is nonempty.

lemma func1_1_L15A:
assumes $A1: f: X \rightarrow Y$ **and** $A2: A \subseteq X$ **and** $A3: A \neq \emptyset$
shows $f(A) \neq \emptyset$
<proof>

The next lemma allows to prove statements about the values in the domain of a function given a statement about values in the range.

lemma func1_1_L15B:
assumes $f:X \rightarrow Y$ **and** $A \subseteq X$ **and** $\forall y \in f(A). P(y)$
shows $\forall x \in A. P(f(x))$
<proof>

An image of an image is the image of a composition.

```
lemma func1_1_L15C: assumes A1: f:X→Y and A2: g:Y→Z
  and A3: A⊆X
  shows
    g(f(A)) = {g(f(x)). x∈A}
    g(f(A)) = (g ∘ f)(A)
  <proof>
```

What is the image of a set defined by a meta-function?

```
lemma func1_1_L17:
  assumes A1: f ∈ X→Y and A2: ∀x∈A. b(x) ∈ X
  shows f({b(x). x∈A}) = {f(b(x)). x∈A}
  <proof>
```

What are the values of composition of three functions?

```
lemma func1_1_L18: assumes A1: f:A→B g:B→C h:C→D
  and A2: x∈A
  shows
    (h ∘ g ∘ f)(x) ∈ D
    (h ∘ g ∘ f)(x) = h(g(f(x)))
  <proof>
```

A composition of functions is a function. This is a slight generalization of standard Isabelle's `comp_fun`.

```
lemma comp_fun_subset:
  assumes A1: g:A→B and A2: f:C→D and A3: B ⊆ C
  shows f ∘ g : A → D
  <proof>
```

This lemma supersedes the lemma `comp_eq_id_iff` in Isabelle/ZF. Contributed by Victor Porton.

```
lemma comp_eq_id_iff1: assumes A1: g: B→A and A2: f: A→C
  shows (∀y∈B. f(g(y)) = y) ⟷ f ∘ g = id(B)
  <proof>
```

A lemma about a value of a function that is a union of some collection of functions.

```
lemma fun_Union_apply: assumes A1: ⋃F : X→Y and
  A2: f∈F and A3: f:A→B and A4: x∈A
  shows (⋃F)(x) = f(x)
  <proof>
```

9.2 Dependent function space

The standard Isabelle/ZF `ZF_Base` theory defines a general notion of a dependent function space $\text{Pi}(X, N)$, where X is any set and N is a collection of sets indexed by X . We rarely use that notion in `IsarMathLib`. The facts

shown in this section provide information on how to interpret the dependent function space notion in terms of a regular space of functions defined on X with values in $Y = \bigcup_{x \in X} N(x)$.

The `Pi_iff_old` lemma from the standard Isabelle/ZF `func` theory shows that if f is a member of the dependent function space $\text{Pi}(X, N)$ and $x \in X$ then there exist exactly one y such that $\langle x, y \rangle \in f$. The next lemma shows that we can slightly strengthen this assertion and claim that there exist exactly one $y \in N(x)$ such that $\langle x, y \rangle \in f$. Consequently, there exists exactly one $y \in \bigcup_{t \in X} N(t)$ such that $\langle x, y \rangle \in f$.

lemma `pi_then`: **assumes** $f \in \text{Pi}(X, N)$ $x \in X$
shows $\exists! y. y \in N(x) \wedge \langle x, y \rangle \in f$ **and** $\exists! y. y \in (\bigcup_{t \in X} N(t)) \wedge \langle x, y \rangle \in f$
<proof>

The next lemma demonstrates a way to understand the dependent function space $\text{Pi}(X, N)$: it is the space of functions that map X into $\bigcup_{t \in X} N(t)$ such that for all $x \in X$ we have $f(x) \in N(x)$.

theorem `pi_fun_space`: **shows** $\text{Pi}(X, N) = \{f \in X \rightarrow (\bigcup_{x \in X} N(x)) . \forall x \in X. f(x) \in N(x)\}$
<proof>

9.3 Functions restricted to a set

Standard Isabelle/ZF defines the notion `restrict(f, A)` of to mean a function (or relation) f restricted to a set. This means that if f is a function defined on X and A is a subset of X then `restrict(f, A)` is a function with the same values as f , but whose domain is A .

Restriction of a restriction to a subset is the same as the direct restriction to that subset.

lemma `restrict_restric_subset`: **assumes** $A \subseteq B$
shows `restrict(restrict(f, B), A) = restrict(f, A)`
<proof>

What is the inverse image of a set under a restricted fuction?

lemma `func1_2_L1`: **assumes** $A1: f: X \rightarrow Y$ **and** $A2: B \subseteq X$
shows `restrict(f, B) - (A) = f - (A) \cap B`
<proof>

A criterion for when one function is a restriction of another. The lemma below provides a result useful in the actual proof of the criterion and applications.

lemma `func1_2_L2`:
assumes $A1: f: X \rightarrow Y$ **and** $A2: g \in A \rightarrow Z$
and $A3: A \subseteq X$ **and** $A4: f \cap A \times Z = g$
shows $\forall x \in A. g(x) = f(x)$
<proof>

Here is the actual criterion.

```
lemma func1_2_L3:
  assumes A1:  $f:X \rightarrow Y$  and A2:  $g:A \rightarrow Z$ 
  and A3:  $A \subseteq X$  and A4:  $f \cap A \times Z = g$ 
  shows  $g = \text{restrict}(f,A)$ 
<proof>
```

Another criterion for when one function is a restriction of another: If a function f maps X to Y and function g maps a subset A of X to Y and $g \subseteq f$ then g is equal to f restricted to A (i.e. $g = f|_A$).

```
lemma fun_restrict_eq:
  assumes  $f:X \rightarrow Y$   $g:A \rightarrow Z$   $A \subseteq X$  and  $g \subseteq f$ 
  shows  $g = \text{restrict}(f,A)$ 
<proof>
```

Yet another criterion for when one function is a restriction of another: If the domain of a function g is contained in the domain of function f and the values of both functions agree on the domain of g , then g is the restriction of f to the domain on g .

```
lemma fun_restrict_eq_vals:
  assumes  $f:X \rightarrow Y$   $g:A \rightarrow Z$   $A \subseteq X$  and  $\forall x \in A. g(x) = f(x)$ 
  shows  $g = \text{restrict}(f,A)$ 
<proof>
```

Which function space a restricted function belongs to?

```
lemma func1_2_L4:
  assumes A1:  $f:X \rightarrow Y$  and A2:  $A \subseteq X$  and A3:  $\forall x \in A. f(x) \in Z$ 
  shows  $\text{restrict}(f,A) : A \rightarrow Z$ 
<proof>
```

A simpler case of func1_2_L4, where the range of the original and restricted function are the same.

```
corollary restrict_fun: assumes A1:  $f:X \rightarrow Y$  and A2:  $A \subseteq X$ 
  shows  $\text{restrict}(f,A) : A \rightarrow Y$ 
<proof>
```

A function restricted to its domain is itself.

```
lemma restrict_domain: assumes  $f:X \rightarrow Y$ 
  shows  $\text{restrict}(f,X) = f$ 
<proof>
```

Suppose a function $f : X \rightarrow Y$ is defined by an expression q , i.e. $f = \{\langle x, y \rangle : x \in X\}$. Then a function that is defined by the same expression, but on a smaller set is the same as the restriction of f to that smaller set.

```
lemma restrict_def_alt: assumes  $A \subseteq X$ 
  shows  $\text{restrict}(\{\langle x, q(x) \rangle. x \in X\}, A) = \{\langle x, q(x) \rangle. x \in A\}$ 
```

<proof>

A composition of two functions is the same as composition with a restriction.

lemma comp_restrict:
 assumes A1: $f : A \rightarrow B$ **and** A2: $g : X \rightarrow C$ **and** A3: $B \subseteq X$
 shows $g \circ f = \text{restrict}(g, B) \circ f$
<proof>

A version of lemma comp_fun_apply from the standard Isabelle/ZF Perm theory but for functions defined as a set of pairs using an expression.

lemma comp_fun_expr_apply: **assumes** $\forall x \in X. a(x) \in Y$ $x \in X$
 shows $(g \circ \{\langle x, a(x) \rangle. x \in X\})(x) = g(a(x))$
<proof>

Value of a composition of functions defined by expressions is given by composition of the expressions.

lemma comp_fun_expr_val:
 assumes $\forall x \in X. a(x) \in Y$ $x \in X$
 shows $(\{\langle y, b(y) \rangle. y \in Y\} \circ \{\langle x, a(x) \rangle. x \in X\})(x) = b(a(x))$
<proof>

Composition of function defined by expressions is a function defined by composition of the expressions.

lemma comp_fun_expr: **assumes** $\forall x \in X. a(x) \in Y$ $\forall y \in Y. b(y) \in Z$
 shows $\{\langle y, b(y) \rangle. y \in Y\} \circ \{\langle x, a(x) \rangle. x \in X\} = \{\langle x, b(a(x)) \rangle. x \in X\}$
<proof>

A way to look at restriction. Contributed by Victor Porton.

lemma right_comp_id_any: **shows** $r \circ \text{id}(C) = \text{restrict}(r, C)$
<proof>

9.4 Constant functions

Constant functions are trivial, but still we need to prove some properties to shorten proofs.

We define constant($= c$) functions on a set X in a natural way as ConstantFunction(X, c).

definition

$\text{ConstantFunction}(X, c) \equiv X \times \{c\}$

Constant function is a function (i.e. belongs to a function space).

lemma func1_3_L1:
 assumes A1: $c \in Y$ **shows** $\text{ConstantFunction}(X, c) : X \rightarrow Y$
<proof>

Constant function is equal to the constant on its domain.

lemma func1_3_L2: **assumes** A1: $x \in X$

shows ConstantFunction(X, c)(x) = c
<proof>

Another way of looking at the constant function - it's a set of pairs $\langle x, c \rangle$ as x ranges over X .

lemma const_fun_def_alt: **shows** ConstantFunction(X, c) = $\{\langle x, c \rangle. x \in X\}$
<proof>

Yet another definition of a constant function: it's a cartesian product of its domain and the singleton of its value.

lemma const_fun_def_alt1: **shows** ConstantFunction(X, c) = $X \times \{c\}$
<proof>

If $c \in A$ then the inverse image of A by the constant function $x \mapsto c$ is the whole domain.

lemma const_vimage_domain: **assumes** $c \in A$
shows ConstantFunction(X, c)-(A) = X
<proof>

If c is not an element of A then the inverse image of A by the constant function $x \mapsto c$ is empty.

lemma const_vimage_empty: **assumes** $c \notin A$
shows ConstantFunction(X, c)-(A) = $\emptyset$
<proof>

9.5 Injections, surjections, bijections etc.

In this section we prove the properties of the spaces of injections, surjections and bijections that we can't find in the standard Isabelle's `Perm.thy`.

For injections the image a difference of two sets is the difference of images

lemma inj_image_dif:
assumes A1: $f \in \text{inj}(A, B)$ and A2: $C \subseteq A$
shows $f(A \setminus C) = f(A) \setminus f(C)$
<proof>

For injections the image of intersection is the intersection of images.

lemma inj_image_inter: **assumes** A1: $f \in \text{inj}(X, Y)$ and A2: $A \subseteq X$ $B \subseteq X$
shows $f(A \cap B) = f(A) \cap f(B)$
<proof>

For surjection from A to B the image of the domain is B .

lemma surj_range_image_domain: **assumes** A1: $f \in \text{surj}(A, B)$
shows $f(A) = B$
<proof>

Surjections are functions that map the domain onto the codomain.

lemma `surj_def_alt`: **shows** $\text{surj}(X,Y) = \{f \in X \rightarrow Y. f(X) = Y\}$
<proof>

Bijections are functions that preserve complements.

lemma `bij_def_alt`:
shows $\text{bij}(X,Y) = \{f \in X \rightarrow Y. \forall A \in \text{Pow}(X). f(X \setminus A) = Y \setminus f(A)\}$
<proof>

For injections the inverse image of an image is the same set.

lemma `inj_vimage_image`: **assumes** $f \in \text{inj}(X,Y)$ **and** $A \subseteq X$
shows $f^{-1}(f(A)) = A$
<proof>

For surjections the image of an inverse image is the same set.

lemma `surj_image_vimage`: **assumes** $A1: f \in \text{surj}(X,Y)$ **and** $A2: A \subseteq Y$
shows $f(f^{-1}(A)) = A$
<proof>

A lemma about how a surjection maps collections of subsets in domain and range.

lemma `surj_subsets`: **assumes** $A1: f \in \text{surj}(X,Y)$ **and** $A2: B \subseteq \text{Pow}(Y)$
shows $\{f(U). U \in \{f^{-1}(V). V \in B\}\} = B$
<proof>

Restriction of an bijection to a set without a point is a a bijection.

lemma `bij_restrict_rem`:
assumes $A1: f \in \text{bij}(A,B)$ **and** $A2: a \in A$
shows $\text{restrict}(f, A \setminus \{a\}) \in \text{bij}(A \setminus \{a\}, B \setminus \{f(a)\})$
<proof>

The domain of a bijection between X and Y is X .

lemma `domain_of_bij`:
assumes $A1: f \in \text{bij}(X,Y)$ **shows** $\text{domain}(f) = X$
<proof>

The value of the inverse of an injection on a point of the image of a set belongs to that set.

lemma `inj_inv_back_in_set`:
assumes $A1: f \in \text{inj}(A,B)$ **and** $A2: C \subseteq A$ **and** $A3: y \in f(C)$
shows
 $\text{converse}(f)(y) \in C$
 $f(\text{converse}(f)(y)) = y$
<proof>

For a bijection between Y and X and a set $A \subseteq X$ an element $y \in Y$ is in the image $f(A)$ if and only if $f^{-1}(y)$ is an element of A . Note this is false with the weakened assumption that f is an injection, for example consider

$f : \{0, 1\} \rightarrow \mathbb{N}, f(n) = n + 1$ and $y = 3$. Then $f^{-1} : \{1, 2\} \rightarrow \{0, 1\}$ and (since 3 is not in the domain of the inverse function) $f^{-1}(3) = \emptyset = 0 \in \{0, 1\}$, but 3 is not in the image $f(\{0, 1\})$.

lemma `bij_val_image_vimage`: `assumes f ∈ bij(X,Y) A ⊆ X y ∈ Y`
`shows y ∈ f(A) ⟷ converse(f)(y) ∈ A`
`<proof>`

For injections if a value at a point belongs to the image of a set, then the point belongs to the set.

lemma `inj_point_of_image`:
`assumes A1: f ∈ inj(A,B) and A2: C ⊆ A and`
`A3: x ∈ A and A4: f(x) ∈ f(C)`
`shows x ∈ C`
`<proof>`

For injections the image of intersection is the intersection of images.

lemma `inj_image_of_Inter`: `assumes A1: f ∈ inj(A,B) and`
`A2: I ≠ ∅ and A3: ∀ i ∈ I. P(i) ⊆ A`
`shows f(⋂ i ∈ I. P(i)) = ( ⋂ i ∈ I. f(P(i)) )`
`<proof>`

An injection is injective onto its range. Suggested by Victor Porton.

lemma `inj_inj_range`: `assumes f ∈ inj(A,B)`
`shows f ∈ inj(A,range(f))`
`<proof>`

An injection is a bijection on its range. Suggested by Victor Porton.

lemma `inj_bij_range`: `assumes f ∈ inj(A,B)`
`shows f ∈ bij(A,range(f))`
`<proof>`

An injection is a bijection between the domain and its image.

lemma `inj_bij_image`: `assumes f ∈ inj(A,B)`
`shows f ∈ bij(A,f(A))`
`<proof>`

A lemma about extending a surjection by one point.

lemma `surj_extend_point`:
`assumes A1: f ∈ surj(X,Y) and A2: a ∉ X and`
`A3: g = f ∪ {⟨a,b⟩}`
`shows g ∈ surj(X ∪ {a}, Y ∪ {b})`
`<proof>`

A lemma about extending an injection by one point. Essentially the same as standard Isabelle's `inj_extend`.

lemma `inj_extend_point`: `assumes f ∈ inj(X,Y) a ∉ X b ∉ Y`

shows $(f \cup \{(a,b)\}) \in \text{inj}(X \cup \{a\}, Y \cup \{b\})$
 $\langle \text{proof} \rangle$

A lemma about extending a bijection by one point.

lemma `bij_extend_point`: **assumes** $f \in \text{bij}(X, Y)$ $a \notin X$ $b \notin Y$
shows $(f \cup \{(a,b)\}) \in \text{bij}(X \cup \{a\}, Y \cup \{b\})$
 $\langle \text{proof} \rangle$

A quite general form of the $a^{-1}b = 1$ implies $a = b$ law.

lemma `comp_inv_id_eq`:
assumes $A1$: $\text{converse}(b) \circ a = \text{id}(A)$ **and**
 $A2$: $a \subseteq A \times B$ $b \in \text{surj}(A, B)$
shows $a = b$
 $\langle \text{proof} \rangle$

A special case of `comp_inv_id_eq` - the $a^{-1}b = 1$ implies $a = b$ law for bijections.

lemma `comp_inv_id_eq_bij`:
assumes $A1$: $a \in \text{bij}(A, B)$ $b \in \text{bij}(A, B)$ **and**
 $A2$: $\text{converse}(b) \circ a = \text{id}(A)$
shows $a = b$
 $\langle \text{proof} \rangle$

Converse of a converse of a bijection is the same bijection. This is a special case of `converse_converse` from standard Isabelle's `equalities` theory where it is proved for relations.

lemma `bij_converse_converse`: **assumes** $a \in \text{bij}(A, B)$
shows $\text{converse}(\text{converse}(a)) = a$
 $\langle \text{proof} \rangle$

If a composition of bijections is identity, then one is the inverse of the other.

lemma `comp_id_conv`: **assumes** $A1$: $a \in \text{bij}(A, B)$ $b \in \text{bij}(B, A)$ **and**
 $A2$: $b \circ a = \text{id}(A)$
shows $a = \text{converse}(b)$ **and** $b = \text{converse}(a)$
 $\langle \text{proof} \rangle$

A version of `comp_id_conv` with weaker assumptions.

lemma `comp_conv_id`: **assumes** $A1$: $a \in \text{bij}(A, B)$ **and** $A2$: $b: B \rightarrow A$ **and**
 $A3$: $\forall x \in A. b(a(x)) = x$
shows $b \in \text{bij}(B, A)$ **and** $a = \text{converse}(b)$ **and** $b = \text{converse}(a)$
 $\langle \text{proof} \rangle$

For a surjection the union of images of singletons is the whole range.

lemma `surj_singleton_image`: **assumes** $A1$: $f \in \text{surj}(X, Y)$
shows $(\bigcup_{x \in X} \{f(x)\}) = Y$
 $\langle \text{proof} \rangle$

9.6 Functions of two variables

In this section we consider functions whose domain is a cartesian product of two sets. Such functions are called functions of two variables (although really in ZF all functions admit only one argument). For every function of two variables we can define families of functions of one variable by fixing the other variable. This section establishes basic definitions and results for this concept.

We can create functions of two variables by combining functions of one variable.

lemma `cart_prod_fun`: `assumes  $f_1:X_1 \rightarrow Y_1$   $f_2:X_2 \rightarrow Y_2$  and`
 `$g = \{ \langle p, \langle f_1(\text{fst}(p)), f_2(\text{snd}(p)) \rangle \rangle . p \in X_1 \times X_2 \}$`
`shows  $g: X_1 \times X_2 \rightarrow Y_1 \times Y_2$  <proof>`

A reformulation of `cart_prod_fun` above in a slightly different notation.

lemma `prod_fun`:
`assumes  $f:X_1 \rightarrow X_2$   $g:X_3 \rightarrow X_4$`
`shows  $\{ \langle \langle x, y \rangle, \langle f(x), g(y) \rangle \rangle . \langle x, y \rangle \in X_1 \times X_3 \} : X_1 \times X_3 \rightarrow X_2 \times X_4$`
`<proof>`

Product of two surjections is a surjection.

theorem `prod_functions_surj`:
`assumes  $f \in \text{surj}(A, B)$   $g \in \text{surj}(C, D)$`
`shows  $\{ \langle \langle a_1, a_2 \rangle, \langle f(a_1), g(a_2) \rangle \rangle . \langle a_1, a_2 \rangle \in A \times C \} \in \text{surj}(A \times C, B \times D)$`
`<proof>`

For a function of two variables created from functions of one variable as in `cart_prod_fun` above, the inverse image of a cartesian product of sets is the cartesian product of inverse images.

lemma `cart_prod_fun_vimage`: `assumes  $f_1:X_1 \rightarrow Y_1$   $f_2:X_2 \rightarrow Y_2$  and`
 `$g = \{ \langle p, \langle f_1(\text{fst}(p)), f_2(\text{snd}(p)) \rangle \rangle . p \in X_1 \times X_2 \}$`
`shows  $g^{-1}(A_1 \times A_2) = f_1^{-1}(A_1) \times f_2^{-1}(A_2)$`
`<proof>`

For a function of two variables defined on $X \times Y$, if we fix an $x \in X$ we obtain a function on Y . Note that if `domain(f)` is $X \times Y$, `range(domain(f))` extracts Y from $X \times Y$.

definition

$$\text{Fix1stVar}(f, x) \equiv \{ \langle y, f(x, y) \rangle . y \in \text{range}(\text{domain}(f)) \}$$

For every $y \in Y$ we can fix the second variable in a binary function $f : X \times Y \rightarrow Z$ to get a function on X .

definition

$$\text{Fix2ndVar}(f, y) \equiv \{ \langle x, f(x, y) \rangle . x \in \text{domain}(\text{domain}(f)) \}$$

We defined `Fix1stVar` and `Fix2ndVar` so that the domain of the function is not listed in the arguments, but is recovered from the function. The next lemma is a technical fact that makes it easier to use this definition.

lemma `fix_var_fun_domain`: **assumes** `A1`: $f : X \times Y \rightarrow Z$
shows
 $x \in X \longrightarrow \text{Fix1stVar}(f, x) = \{\langle y, f \langle x, y \rangle \rangle. y \in Y\}$
 $y \in Y \longrightarrow \text{Fix2ndVar}(f, y) = \{\langle x, f \langle x, y \rangle \rangle. x \in X\}$
<proof>

If we fix the first variable, we get a function of the second variable.

lemma `fix_1st_var_fun`: **assumes** `A1`: $f : X \times Y \rightarrow Z$ **and** `A2`: $x \in X$
shows $\text{Fix1stVar}(f, x) : Y \rightarrow Z$
<proof>

If we fix the second variable, we get a function of the first variable.

lemma `fix_2nd_var_fun`: **assumes** `A1`: $f : X \times Y \rightarrow Z$ **and** `A2`: $y \in Y$
shows $\text{Fix2ndVar}(f, y) : X \rightarrow Z$
<proof>

What is the value of `Fix1stVar`(f, x) at $y \in Y$ and the value of `Fix2ndVar`(f, y) at $x \in X$?

lemma `fix_var_val`:
assumes `A1`: $f : X \times Y \rightarrow Z$ **and** `A2`: $x \in X \quad y \in Y$
shows
 $\text{Fix1stVar}(f, x)(y) = f \langle x, y \rangle$
 $\text{Fix2ndVar}(f, y)(x) = f \langle x, y \rangle$
<proof>

Fixing the second variable commutes with restricting the domain.

lemma `fix_2nd_var_restr_comm`:
assumes `A1`: $f : X \times Y \rightarrow Z$ **and** `A2`: $y \in Y$ **and** `A3`: $X_1 \subseteq X$
shows $\text{Fix2ndVar}(\text{restrict}(f, X_1 \times Y), y) = \text{restrict}(\text{Fix2ndVar}(f, y), X_1)$
<proof>

The next lemma expresses the inverse image of a set by function with fixed first variable in terms of the original function.

lemma `fix_1st_var_vimage`:
assumes `A1`: $f : X \times Y \rightarrow Z$ **and** `A2`: $x \in X$
shows $\text{Fix1stVar}(f, x) \text{-(A)} = \{y \in Y. \langle x, y \rangle \in f \text{-(A)}\}$
<proof>

The next lemma expresses the inverse image of a set by function with fixed second variable in terms of the original function.

lemma `fix_2nd_var_vimage`:
assumes `A1`: $f : X \times Y \rightarrow Z$ **and** `A2`: $y \in Y$
shows $\text{Fix2ndVar}(f, y) \text{-(A)} = \{x \in X. \langle x, y \rangle \in f \text{-(A)}\}$
<proof>

end

10 Binary operations

theory func_ZF **imports** func1

begin

In this theory we consider properties of functions that are binary operations, that is they map $X \times X$ into X .

10.1 Lifting operations to a function space

It happens quite often that we have a binary operation on some set and we need a similar operation that is defined for functions on that set. For example once we know how to add real numbers we also know how to add real-valued functions: for $f, g : X \rightarrow \mathbf{R}$ we define $(f + g)(x) = f(x) + g(x)$. Note that formally the $+$ means something different on the left hand side of this equality than on the right hand side. This section aims at formalizing this process. We will call it "lifting to a function space", if you have a suggestion for a better name, please let me know.

Since we are writing in generic set notation, the definition below is a bit complicated. Here it what it says: Given a set X and another set f (that represents a binary function on X) we are defining f lifted to function space over X as the binary function (a set of pairs) on the space $F = X \rightarrow \text{range}(f)$ such that the value of this function on pair $\langle a, b \rangle$ of functions on X is another function c on X with values defined by $c(x) = f\langle a(x), b(x) \rangle$.

definition

Lift2FcnSpce (**infix** {lifted to function space over} 65) **where**
f {lifted to function space over} $X \equiv$
 $\{ \langle p, \{ \langle x, f(\text{fst}(p)(x), \text{snd}(p)(x)) \rangle \}. x \in X \rangle \}.$
 $p \in (X \rightarrow \text{range}(f)) \times (X \rightarrow \text{range}(f)) \}$

The result of the lift belongs to the function space.

lemma func_ZF_1_L1:

assumes A1: $f : Y \times Y \rightarrow Y$
and A2: $p \in (X \rightarrow \text{range}(f)) \times (X \rightarrow \text{range}(f))$
shows
 $\{ \langle x, f(\text{fst}(p)(x), \text{snd}(p)(x)) \rangle \}. x \in X \} : X \rightarrow \text{range}(f)$
<proof>

The values of the lift are defined by the value of the liftee in a natural way.

lemma func_ZF_1_L2:

assumes A1: $f : Y \times Y \rightarrow Y$

and A2: $p \in (X \rightarrow \text{range}(f)) \times (X \rightarrow \text{range}(f))$ **and** A3: $x \in X$
and A4: $P = \{\langle x, f(\text{fst}(p)(x), \text{snd}(p)(x)) \rangle \mid x \in X\}$
shows $P(x) = f(\text{fst}(p)(x), \text{snd}(p)(x))$
 $\langle \text{proof} \rangle$

Function lifted to a function space results in function space operator.

theorem func_ZF_1_L3:
assumes $f : Y \times Y \rightarrow Y$
and $F = f \text{ \{lifted to function space over\} } X$
shows $F : (X \rightarrow \text{range}(f)) \times (X \rightarrow \text{range}(f)) \rightarrow (X \rightarrow \text{range}(f))$
 $\langle \text{proof} \rangle$

The values of the lift are defined by the values of the liftee in the natural way.

theorem func_ZF_1_L4:
assumes A1: $f : Y \times Y \rightarrow Y$
and A2: $F = f \text{ \{lifted to function space over\} } X$
and A3: $s : X \rightarrow \text{range}(f) \quad r : X \rightarrow \text{range}(f)$
and A4: $x \in X$
shows $(F(s, r))(x) = f(s(x), r(x))$
 $\langle \text{proof} \rangle$

10.2 Associative and commutative operations

In this section we define associative and commutative operations and prove that they remain such when we lift them to a function space.

Typically we say that a binary operation “.” on a set G is “associative” if $(x \cdot y) \cdot z = x \cdot (y \cdot z)$ for all $x, y, z \in G$. Our actual definition below does not use the multiplicative notation so that we can apply it equally to the additive notation $+$ or whatever infix symbol we may want to use. Instead, we use the generic set theory notation and write $P\langle x, y \rangle$ to denote the value of the operation P on a pair $\langle x, y \rangle \in G \times G$.

definition

IsAssociative (infix {is associative on} 65) **where**
 $P \text{ \{is associative on\} } G \equiv P : G \times G \rightarrow G \wedge$
 $(\forall x \in G. \forall y \in G. \forall z \in G.$
 $(P(\langle P(\langle x, y \rangle), z \rangle) = P(\langle x, P(\langle y, z \rangle) \rangle)))$

A binary function $f : X \times X \rightarrow Y$ is commutative if $f\langle x, y \rangle = f\langle y, x \rangle$. Note that in the definition of associativity above we talk about binary “operation” and here we say use the term binary “function”. This is not set in stone, but usually the word “operation” is used when the range is a factor of the domain, while the word “function” allows the range to be a completely unrelated set.

definition

IsCommutative (infix {is commutative on} 65) **where**

$f \text{ \{is commutative on\} } G \equiv \forall x \in G. \forall y \in G. f\langle x, y \rangle = f\langle y, x \rangle$

The lift of a commutative function is commutative.

```
lemma func_ZF_2_L1:
  assumes A1: f : G×G→G
  and A2: F = f {lifted to function space over} X
  and A3: s : X→range(f) r : X→range(f)
  and A4: f {is commutative on} G
  shows F⟨s,r⟩ = F⟨r,s⟩
⟨proof⟩
```

The lift of a commutative function is commutative on the function space.

```
lemma func_ZF_2_L2:
  assumes f : G×G→G
  and f {is commutative on} G
  and F = f {lifted to function space over} X
  shows F {is commutative on} (X→range(f))
⟨proof⟩
```

The lift of an associative function is associative.

```
lemma func_ZF_2_L3:
  assumes A2: F = f {lifted to function space over} X
  and A3: s : X→range(f) r : X→range(f) q : X→range(f)
  and A4: f {is associative on} G
  shows F⟨F⟨s,r⟩,q⟩ = F⟨s,F⟨r,q⟩⟩
⟨proof⟩
```

The lift of an associative function is associative on the function space.

```
lemma func_ZF_2_L4:
  assumes A1: f {is associative on} G
  and A2: F = f {lifted to function space over} X
  shows F {is associative on} (X→range(f))
⟨proof⟩
```

10.3 Restricting operations

In this section we consider conditions under which restriction of the operation to a set inherits properties like commutativity and associativity.

The commutativity is inherited when restricting a function to a set.

```
lemma func_ZF_4_L1:
  assumes A1: f:X×X→Y and A2: A⊆X
  and A3: f {is commutative on} X
  shows restrict(f,A×A) {is commutative on} A
⟨proof⟩
```

Next we define what it means that a set is closed with respect to an operation.

definition

IsOpClosed (**infix** {is closed under} 65) **where**
 $A \text{ \{is closed under\} } f \equiv \forall x \in A. \forall y \in A. f\langle x, y \rangle \in A$

Associative operation restricted to a set that is closed with resp. to this operation is associative.

lemma func_ZF_4_L2: **assumes** A1: $f \text{ \{is associative on\} } X$
and A2: $A \subseteq X$ **and** A3: $A \text{ \{is closed under\} } f$
and A4: $x \in A \ y \in A \ z \in A$
and A5: $g = \text{restrict}(f, A \times A)$
shows $g\langle g\langle x, y \rangle, z \rangle = g\langle x, g\langle y, z \rangle \rangle$
<proof>

An associative operation restricted to a set that is closed with resp. to this operation is associative on the set.

lemma func_ZF_4_L3: **assumes** A1: $f \text{ \{is associative on\} } X$
and A2: $A \subseteq X$ **and** A3: $A \text{ \{is closed under\} } f$
shows $\text{restrict}(f, A \times A) \text{ \{is associative on\} } A$
<proof>

The essential condition to show that if a set A is closed with respect to an operation, then it is closed under this operation restricted to any superset of A .

lemma func_ZF_4_L4: **assumes** $A \text{ \{is closed under\} } f$
and $A \subseteq B$ **and** $x \in A \ y \in A$ **and** $g = \text{restrict}(f, B \times B)$
shows $g\langle x, y \rangle \in A$
<proof>

If a set A is closed under an operation, then it is closed under this operation restricted to any superset of A .

lemma func_ZF_4_L5:
assumes A1: $A \text{ \{is closed under\} } f$
and A2: $A \subseteq B$
shows $A \text{ \{is closed under\} } \text{restrict}(f, B \times B)$
<proof>

The essential condition to show that intersection of sets that are closed with respect to an operation is closed with respect to the operation.

lemma func_ZF_4_L6:
assumes $A \text{ \{is closed under\} } f$
and $B \text{ \{is closed under\} } f$
and $x \in A \cap B \ y \in A \cap B$
shows $f\langle x, y \rangle \in A \cap B$ *<proof>*

Intersection of sets that are closed with respect to an operation is closed under the operation.

lemma func_ZF_4_L7:

```

assumes A {is closed under} f
B {is closed under} f
shows A∩B {is closed under} f
⟨proof⟩

```

10.4 Compositions

For any set X we can consider a binary operation on the set of functions $f : X \rightarrow X$ defined by $C(f, g) = f \circ g$. Composition of functions (or relations) is defined in the standard Isabelle distribution as a higher order function and denoted with the letter $\circ$. In this section we consider the corresponding two-argument ZF-function (binary operation), that is a subset of $((X \rightarrow X) \times (X \rightarrow X)) \times (X \rightarrow X)$.

We define the notion of composition on the set X as the binary operation on the function space $X \rightarrow X$ that takes two functions and creates the their composition.

definition

```

Composition(X)  $\equiv$ 
{⟨p, fst(p)  $\circ$  snd(p)⟩. p  $\in$  (X $\rightarrow$ X)  $\times$  (X $\rightarrow$ X)}

```

Composition operation is a function that maps $(X \rightarrow X) \times (X \rightarrow X)$ into $X \rightarrow X$.

```

lemma func_ZF_5_L1: shows Composition(X) : (X $\rightarrow$ X)  $\times$  (X $\rightarrow$ X)  $\rightarrow$  (X $\rightarrow$ X)
⟨proof⟩

```

The value of the composition operation is the composition of arguments.

```

lemma func_ZF_5_L2: assumes f:X $\rightarrow$ X and g:X $\rightarrow$ X
shows Composition(X)⟨f,g⟩ = f  $\circ$  g
⟨proof⟩

```

What is the value of a composition on an argument?

```

lemma func_ZF_5_L3: assumes f:X $\rightarrow$ X and g:X $\rightarrow$ X and x $\in$ X
shows (Composition(X)⟨f,g⟩)(x) = f(g(x))
⟨proof⟩

```

The essential condition to show that composition is associative.

```

lemma func_ZF_5_L4: assumes A1: f:X $\rightarrow$ X g:X $\rightarrow$ X h:X $\rightarrow$ X
and A2: C = Composition(X)
shows C⟨C⟨f,g⟩,h⟩ = C⟨f,C⟨g,h⟩⟩
⟨proof⟩

```

Composition is an associative operation on $X \rightarrow X$ (the space of functions that map X into itself).

```

lemma func_ZF_5_L5: shows Composition(X) {is associative on} (X $\rightarrow$ X)
⟨proof⟩

```

10.5 Identity function

In this section we show some additional facts about the identity function defined in the standard Isabelle's `Perm` theory. Note there are some lemmas about the identity function in `func1` theory as well.

The identity function on a set is the same as the diagonal of that set.

lemma `id_diagonal`: **shows** $\text{id}(X) = \{\langle x, x \rangle. x \in X\}$
<proof>

A function that maps every point to itself is the identity on its domain.

lemma `identity_fun`: **assumes** $A1: f: X \rightarrow Y$ **and** $A2: \forall x \in X. f(x) = x$
shows $f = \text{id}(X)$
<proof>

Composing a function with identity does not change the function.

lemma `func_ZF_6_L1A`: **assumes** $A1: f : X \rightarrow X$
shows $\text{Composition}(X) \langle f, \text{id}(X) \rangle = f$
 $\text{Composition}(X) \langle \text{id}(X), f \rangle = f$
<proof>

An intuitively clear, but surprisingly nontrivial fact: identity is the only function from a singleton to itself.

lemma `singleton_fun_id`: **shows** $(\{x\} \rightarrow \{x\}) = \{\text{id}(\{x\})\}$
<proof>

Another trivial fact: identity is the only bijection of a singleton with itself.

lemma `single_bij_id`: **shows** $\text{bij}(\{x\}, \{x\}) = \{\text{id}(\{x\})\}$
<proof>

A kind of induction for the identity: if a function f is the identity on a set with a fixpoint of f removed, then it is the identity on the whole set.

lemma `id_fixpoint_rem`: **assumes** $A1: f: X \rightarrow X$ **and**
 $A2: p \in X$ **and** $A3: f(p) = p$ **and**
 $A4: \text{restrict}(f, X - \{p\}) = \text{id}(X - \{p\})$
shows $f = \text{id}(X)$
<proof>

10.6 Lifting to subsets

Suppose we have a binary operation $f : X \times X \rightarrow X$ written additively as $f \langle x, y \rangle = x + y$. Such operation naturally defines another binary operation on the subsets of X that satisfies $A + B = \{x + y : x \in A, y \in B\}$. This new operation which we will call " f lifted to subsets" inherits many properties of f , such as associativity, commutativity and existence of the neutral element. This notion is useful for considering interval arithmetics.

The next definition describes the notion of a binary operation lifted to subsets. It is written in a way that might be a bit unexpected, but really it is the same as the intuitive definition, but shorter. In the definition we take a pair $p \in \text{Pow}(X) \times \text{Pow}(X)$, say $p = \langle A, B \rangle$, where $A, B \subseteq X$. Then we assign this pair of sets the set $\{f\langle x, y \rangle : x \in A, y \in B\} = \{f(x') : x' \in A \times B\}$. The set on the right hand side is the same as the image of $A \times B$ under f . In the definition we don't use A and B symbols, but write $\text{fst}(p)$ and $\text{snd}(p)$, resp. Recall that in Isabelle/ZF $\text{fst}(p)$ and $\text{snd}(p)$ denote the first and second components of an ordered pair p . See the lemma `lift_subsets_explained` for a more intuitive notation.

definition

```
Lift2Subsets (infix {lifted to subsets of} 65) where
  f {lifted to subsets of} X  $\equiv$ 
     $\{\langle p, f(\text{fst}(p) \times \text{snd}(p)) \rangle. p \in \text{Pow}(X) \times \text{Pow}(X)\}$ 
```

The lift to subsets defines a binary operation on the subsets.

```
lemma lift_subsets_binop: assumes A1:  $f : X \times X \rightarrow Y$ 
  shows  $(f \text{ {lifted to subsets of} } X) : \text{Pow}(X) \times \text{Pow}(X) \rightarrow \text{Pow}(Y)$ 
  <proof>
```

The definition of the lift to subsets rewritten in a more intuitive notation. We would like to write the last assertion as $F\langle A, B \rangle = \{f\langle x, y \rangle. x \in A, y \in B\}$, but Isabelle/ZF does not allow such syntax.

```
lemma lift_subsets_explained: assumes A1:  $f : X \times X \rightarrow Y$ 
  and A2:  $A \subseteq X \ B \subseteq X$  and A3:  $F = f \text{ {lifted to subsets of} } X$ 
  shows
     $F\langle A, B \rangle \subseteq Y$  and
     $F\langle A, B \rangle = f(A \times B)$ 
     $F\langle A, B \rangle = \{f\langle p \rangle. p \in A \times B\}$ 
     $F\langle A, B \rangle = \{f\langle x, y \rangle. \langle x, y \rangle \in A \times B\}$ 
  <proof>
```

A sufficient condition for a point to belong to a result of lifting to subsets.

```
lemma lift_subset_suff: assumes A1:  $f : X \times X \rightarrow Y$  and
  A2:  $A \subseteq X \ B \subseteq X$  and A3:  $x \in A \ y \in B$  and
  A4:  $F = f \text{ {lifted to subsets of} } X$ 
  shows  $f\langle x, y \rangle \in F\langle A, B \rangle$ 
  <proof>
```

A kind of converse of `lift_subset_apply`, providing a necessary condition for a point to be in the result of lifting to subsets.

```
lemma lift_subset_nec: assumes A1:  $f : X \times X \rightarrow Y$  and
  A2:  $A \subseteq X \ B \subseteq X$  and
  A3:  $F = f \text{ {lifted to subsets of} } X$  and
  A4:  $z \in F\langle A, B \rangle$ 
  shows  $\exists x \ y. x \in A \wedge y \in B \wedge z = f\langle x, y \rangle$ 
```

<proof>

Lifting to subsets inherits commutativity.

lemma lift_subset_comm: **assumes** A1: $f : X \times X \rightarrow Y$ **and**
 A2: f {is commutative on} X **and**
 A3: $F = f$ {lifted to subsets of} X
shows F {is commutative on} $\text{Pow}(X)$

<proof>

Lifting to subsets inherits associativity. To show that $F\langle\langle A, B \rangle C \rangle = F\langle A, F\langle B, C \rangle \rangle$ we prove two inclusions and the proof of the second inclusion is very similar to the proof of the first one.

lemma lift_subset_assoc: **assumes**
 A1: f {is associative on} X **and** A2: $F = f$ {lifted to subsets of} X
shows F {is associative on} $\text{Pow}(X)$

<proof>

10.7 Distributive operations

In this section we deal with pairs of operations such that one is distributive with respect to the other, that is $a \cdot (b + c) = a \cdot b + a \cdot c$ and $(b + c) \cdot a = b \cdot a + c \cdot a$. We show that this property is preserved under restriction to a set closed with respect to both operations. In *EquivClass1* theory we show that this property is preserved by projections to the quotient space if both operations are congruent with respect to the equivalence relation.

We define distributivity as a statement about three sets. The first set is the set on which the operations act. The second set is the additive operation (a ZF function) and the third is the multiplicative operation.

definition

$\text{IsDistributive}(X, A, M) \equiv (\forall a \in X. \forall b \in X. \forall c \in X.$
 $M\langle a, A\langle b, c \rangle \rangle = A\langle M\langle a, b \rangle, M\langle a, c \rangle \rangle \wedge$
 $M\langle A\langle b, c \rangle, a \rangle = A\langle M\langle b, a \rangle, M\langle c, a \rangle \rangle)$

The essential condition to show that distributivity is preserved by restrictions to sets that are closed with respect to both operations.

lemma func_ZF_7_L1:
assumes A1: $\text{IsDistributive}(X, A, M)$
and A2: $Y \subseteq X$
and A3: Y {is closed under} A Y {is closed under} M
and A4: $A_r = \text{restrict}(A, Y \times Y)$ $M_r = \text{restrict}(M, Y \times Y)$
and A5: $a \in Y$ $b \in Y$ $c \in Y$
shows $M_r\langle a, A_r\langle b, c \rangle \rangle = A_r\langle M_r\langle a, b \rangle, M_r\langle a, c \rangle \rangle \wedge$
 $M_r\langle A_r\langle b, c \rangle, a \rangle = A_r\langle M_r\langle b, a \rangle, M_r\langle c, a \rangle \rangle$

<proof>

Distributivity is preserved by restrictions to sets that are closed with respect to both operations.


```

lemma func_ZF_7_L2:
  assumes IsDistributive(X,A,M)
  and  $Y \subseteq X$ 
  and  $Y$  {is closed under}  $A$ 
   $Y$  {is closed under}  $M$ 
  and  $A_r = \text{restrict}(A, Y \times Y)$   $M_r = \text{restrict}(M, Y \times Y)$ 
  shows IsDistributive( $Y, A_r, M_r$ )
 $\langle proof \rangle$ 

```

end

11 More on functions

```

theory func_ZF_1 imports ZF.Order Order_ZF_1a func_ZF

```

begin

In this theory we consider some properties of functions related to order relations

11.1 Functions and order

This section deals with functions between ordered sets.

If every value of a function on a set is bounded below by a constant, then the image of the set is bounded below.

```

lemma func_ZF_8_L1:
  assumes  $f: X \rightarrow Y$  and  $A \subseteq X$  and  $\forall x \in A. \langle L, f(x) \rangle \in r$ 
  shows IsBoundedBelow( $f(A), r$ )
 $\langle proof \rangle$ 

```

If every value of a function on a set is bounded above by a constant, then the image of the set is bounded above.

```

lemma func_ZF_8_L2:
  assumes  $f: X \rightarrow Y$  and  $A \subseteq X$  and  $\forall x \in A. \langle f(x), U \rangle \in r$ 
  shows IsBoundedAbove( $f(A), r$ )
 $\langle proof \rangle$ 

```

Identity is an order isomorphism.

```

lemma id_ord_iso: shows  $\text{id}(X) \in \text{ord\_iso}(X, r, X, r)$ 
 $\langle proof \rangle$ 

```

Identity is the only order automorphism of a singleton.

```

lemma id_ord_auto_singleton:
  shows  $\text{ord\_iso}(\{x\}, r, \{x\}, r) = \{\text{id}(\{x\})\}$ 
 $\langle proof \rangle$ 

```

The image of a maximum by an order isomorphism is a maximum. Note that from the fact the r is antisymmetric and f is an order isomorphism between (A, r) and (B, R) we can not conclude that R is antisymmetric (we can only show that $R \cap (B \times B)$ is).

lemma max_image_ord_iso:
 assumes A1: antisym(r) and A2: antisym(R) and
 A3: $f \in \text{ord_iso}(A, r, B, R)$ and
 A4: HasAmaximum(r, A)
 shows HasAmaximum(R, B) and Maximum(R, B) = $f(\text{Maximum}(r, A))$
 $\langle \text{proof} \rangle$

Maximum is a fixpoint of order automorphism.

lemma max_auto_fixpoint:
 assumes antisym(r) and $f \in \text{ord_iso}(A, r, A, r)$
 and HasAmaximum(r, A)
 shows Maximum(r, A) = $f(\text{Maximum}(r, A))$
 $\langle \text{proof} \rangle$

If two sets are order isomorphic and we remove x and $f(x)$, respectively, from the sets, then they are still order isomorphic.

lemma ord_iso_rem_point:
 assumes A1: $f \in \text{ord_iso}(A, r, B, R)$ and A2: $a \in A$
 shows restrict($f, A - \{a\}$) $\in \text{ord_iso}(A - \{a\}, r, B - \{f(a)\}, R)$
 $\langle \text{proof} \rangle$

If two sets are order isomorphic and we remove maxima from the sets, then they are still order isomorphic.

corollary ord_iso_rem_max:
 assumes A1: antisym(r) and $f \in \text{ord_iso}(A, r, B, R)$ and
 A4: HasAmaximum(r, A) and A5: $M = \text{Maximum}(r, A)$
 shows restrict($f, A - \{M\}$) $\in \text{ord_iso}(A - \{M\}, r, B - \{f(M)\}, R)$
 $\langle \text{proof} \rangle$

Lemma about extending order isomorphisms by adding one point to the domain.

lemma ord_iso_extend: assumes A1: $f \in \text{ord_iso}(A, r, B, R)$ and
 A2: $M_A \notin A$ $M_B \notin B$ and
 A3: $\forall a \in A. \langle a, M_A \rangle \in r \quad \forall b \in B. \langle b, M_B \rangle \in R$ and
 A4: antisym(r) antisym(R) and
 A5: $\langle M_A, M_A \rangle \in r \iff \langle M_B, M_B \rangle \in R$
 shows $f \cup \{ \langle M_A, M_B \rangle \} \in \text{ord_iso}(A \cup \{M_A\}, r, B \cup \{M_B\}, R)$
 $\langle \text{proof} \rangle$

A kind of converse to ord_iso_rem_max: if two linearly ordered sets are order isomorphic after removing the maxima, then they are order isomorphic.

lemma rem_max_ord_iso:

assumes A1: IsLinOrder(X,r) IsLinOrder(Y,R) **and**
 A2: HasAmaximum(r,X) HasAmaximum(R,Y)
 ord_iso(X - {Maximum(r,X)},r,Y - {Maximum(R,Y)},R) $\neq$ 0
shows ord_iso(X,r,Y,R) $\neq$ 0
<proof>

11.2 Functions in cartesian products

In this section we consider maps arising naturally in cartesian products.

There is a natural bijection etween $X = Y \times \{y\}$ (a "slice") and Y . We will call this the `SliceProjection(Y×{y})`. This is really the ZF equivalent of the meta-function `fst(x)`.

definition

$\text{SliceProjection}(X) \equiv \{\langle p, \text{fst}(p) \rangle \mid p \in X\}$

A slice projection is a bijection between $X \times \{y\}$ and X .

lemma slice_proj_bij: shows

$\text{SliceProjection}(X \times \{y\}) : X \times \{y\} \rightarrow X$
 $\text{domain}(\text{SliceProjection}(X \times \{y\})) = X \times \{y\}$
 $\forall p \in X \times \{y\}. \text{SliceProjection}(X \times \{y\})(p) = \text{fst}(p)$
 $\text{SliceProjection}(X \times \{y\}) \in \text{bij}(X \times \{y\}, X)$
<proof>

Given 2 functions $f : A \rightarrow B$ and $g : C \rightarrow D$, we can consider a function $h : A \times C \rightarrow B \times D$ such that $h(x, y) = \langle f(x), g(y) \rangle$

definition

ProdFunction where
 $\text{ProdFunction}(f, g) \equiv \{\langle z, \langle f(\text{fst}(z)), g(\text{snd}(z)) \rangle \rangle \mid z \in \text{domain}(f) \times \text{domain}(g)\}$

For given functions $f : A \rightarrow B$ and $g : C \rightarrow D$ the function $\text{ProdFunction}(f, g)$ maps $A \times C$ to $B \times D$.

lemma prodFunction:

assumes $f : A \rightarrow B$ $g : C \rightarrow D$
shows $\text{ProdFunction}(f, g) : (A \times C) \rightarrow (B \times D)$
<proof>

For given functions $f : A \rightarrow B$ and $g : C \rightarrow D$ and points $x \in A$, $y \in C$ the value of the function $\text{ProdFunction}(f, g)$ on $\langle x, y \rangle$ is $\langle f(x), g(y) \rangle$.

lemma prodFunctionApp:

assumes $f : A \rightarrow B$ $g : C \rightarrow D$ $x \in A$ $y \in C$
shows $\text{ProdFunction}(f, g)\langle x, y \rangle = \langle f(x), g(y) \rangle$
<proof>

Somewhat technical lemma about inverse image of a set by a $\text{ProdFunction}(f, f)$.

lemma prodFunVimage: assumes $x \in X$ $f : X \rightarrow Y$

shows $\langle x, t \rangle \in \text{ProdFunction}(f, f) - (V) \longleftrightarrow t \in X \wedge \langle fx, ft \rangle \in V$
<proof>

11.3 Induced relations and order isomorphisms

When we have two sets X, Y , function $f : X \rightarrow Y$ and a relation R on Y we can define a relation r on X by saying that $x r y$ if and only if $f(x) R f(y)$. This is especially interesting when f is a bijection as all reasonable properties of R are inherited by r . This section treats mostly the case when R is an order relation and f is a bijection. The standard Isabelle's `Order` theory defines the notion of a space of order isomorphisms between two sets relative to a relation. We expand that material proving that order isomorphisms preserve interesting properties of the relation.

We call the relation created by a relation on Y and a mapping $f : X \rightarrow Y$ the `InducedRelation(f,R)`.

definition

```
InducedRelation(f,R)  $\equiv$ 
  {p  $\in$  domain(f) $\times$ domain(f).  $\langle$ f(fst(p)),f(snd(p)) $\rangle \in R$ }
```

A reformulation of the definition of the relation induced by a function.

lemma `def_of_ind_relA`:

```
  assumes  $\langle$ x,y $\rangle \in$  InducedRelation(f,R)
  shows  $\langle$ f(x),f(y) $\rangle \in R$ 
   $\langle$ proof $\rangle$ 
```

A reformulation of the definition of the relation induced by a function, kind of converse of `def_of_ind_relA`.

lemma `def_of_ind_relB`: **assumes** $f:A \rightarrow B$ **and**

```
  x $\in$ A y $\in$ A and  $\langle$ f(x),f(y) $\rangle \in R$ 
  shows  $\langle$ x,y $\rangle \in$  InducedRelation(f,R)
   $\langle$ proof $\rangle$ 
```

A property of order isomorphisms that is missing from standard Isabelle's `Order.thy`.

lemma `ord_iso_apply_conv`:

```
  assumes  $f \in$  ord_iso(A,r,B,R) and
   $\langle$ f(x),f(y) $\rangle \in R$  and x $\in$ A y $\in$ A
  shows  $\langle$ x,y $\rangle \in r$ 
   $\langle$ proof $\rangle$ 
```

The next lemma tells us where the induced relation is defined

lemma `ind_rel_domain`:

```
  assumes  $R \subseteq B \times B$  and  $f:A \rightarrow B$ 
  shows InducedRelation(f,R)  $\subseteq A \times A$ 
   $\langle$ proof $\rangle$ 
```

A bijection is an order homomorphisms between a relation and the induced one.

lemma `bij_is_ord_iso`: **assumes** $A1: f \in$ `bij(A,B)`

shows $f \in \text{ord_iso}(A, \text{InducedRelation}(f, R), B, R)$
<proof>

An order isomorphism preserves antisymmetry.

lemma `ord_iso_pres_antisym`: **assumes** $A1: f \in \text{ord_iso}(A, r, B, R)$ **and**
 $A2: r \subseteq A \times A$ **and** $A3: \text{antisym}(R)$
shows $\text{antisym}(r)$
<proof>

Order isomorphisms preserve transitivity.

lemma `ord_iso_pres_trans`: **assumes** $A1: f \in \text{ord_iso}(A, r, B, R)$ **and**
 $A2: r \subseteq A \times A$ **and** $A3: \text{trans}(R)$
shows $\text{trans}(r)$
<proof>

Order isomorphisms preserve totality.

lemma `ord_iso_pres_tot`: **assumes** $A1: f \in \text{ord_iso}(A, r, B, R)$ **and**
 $A2: r \subseteq A \times A$ **and** $A3: R \text{ \{is total on\} } B$
shows $r \text{ \{is total on\} } A$
<proof>

Order isomorphisms preserve linearity.

lemma `ord_iso_pres_lin`: **assumes** $f \in \text{ord_iso}(A, r, B, R)$ **and**
 $r \subseteq A \times A$ **and** $\text{IsLinOrder}(B, R)$
shows $\text{IsLinOrder}(A, r)$
<proof>

If a relation is a linear order, then the relation induced on another set by a bijection is also a linear order.

lemma `ind_rel_pres_lin`:
assumes $A1: f \in \text{bij}(A, B)$ **and** $A2: \text{IsLinOrder}(B, R)$
shows $\text{IsLinOrder}(A, \text{InducedRelation}(f, R))$
<proof>

The image by an order isomorphism of a bounded above and nonempty set is bounded above.

lemma `ord_iso_pres_bound_above`:
assumes $A1: f \in \text{ord_iso}(A, r, B, R)$ **and** $A2: r \subseteq A \times A$ **and**
 $A3: \text{IsBoundedAbove}(C, r) \quad C \neq \emptyset$
shows $\text{IsBoundedAbove}(f(C), R) \quad f(C) \neq \emptyset$
<proof>

Order isomorphisms preserve the property of having a minimum.

lemma `ord_iso_pres_has_min`:
assumes $A1: f \in \text{ord_iso}(A, r, B, R)$ **and** $A2: r \subseteq A \times A$ **and**
 $A3: C \subseteq A$ **and** $A4: \text{HasAminum}(R, f(C))$
shows $\text{HasAminum}(r, C)$

<proof>

Order isomorphisms preserve the images of relations. In other words taking the image of a point by a relation commutes with the function.

```
lemma ord_iso_pres_rel_image:
  assumes A1:  $f \in \text{ord\_iso}(A, r, B, R)$  and
  A2:  $r \subseteq A \times A$   $R \subseteq B \times B$  and
  A3:  $a \in A$ 
  shows  $f(r\{a\}) = R\{f(a)\}$ 
```

<proof>

Order isomorphisms preserve collections of upper bounds.

```
lemma ord_iso_pres_up_bounds:
  assumes A1:  $f \in \text{ord\_iso}(A, r, B, R)$  and
  A2:  $r \subseteq A \times A$   $R \subseteq B \times B$  and
  A3:  $C \subseteq A$ 
  shows  $\{f(r\{a\}). a \in C\} = \{R\{b\}. b \in f(C)\}$ 
```

<proof>

The image of the set of upper bounds is the set of upper bounds of the image.

```
lemma ord_iso_pres_min_up_bounds:
  assumes A1:  $f \in \text{ord\_iso}(A, r, B, R)$  and A2:  $r \subseteq A \times A$   $R \subseteq B \times B$  and
  A3:  $C \subseteq A$  and A4:  $C \neq 0$ 
  shows  $f(\bigcap_{a \in C} r\{a\}) = (\bigcap_{b \in f(C)} R\{b\})$ 
```

<proof>

Order isomorphisms preserve completeness.

```
lemma ord_iso_pres_compl:
  assumes A1:  $f \in \text{ord\_iso}(A, r, B, R)$  and
  A2:  $r \subseteq A \times A$   $R \subseteq B \times B$  and A3:  $R$  {is complete}
  shows  $r$  {is complete}
```

<proof>

If the original relation is complete, then the induced one is complete.

```
lemma ind_rel_pres_compl: assumes A1:  $f \in \text{bij}(A, B)$ 
  and A2:  $R \subseteq B \times B$  and A3:  $R$  {is complete}
  shows  $\text{InducedRelation}(f, R)$  {is complete}
```

<proof>

end

12 Finite sets - introduction

```
theory Finite_ZF imports ZF1 Nat_ZF_IML ZF.Cardinal func1
```

begin

Standard Isabelle `Finite.thy` contains a very useful notion of finite powerset: the set of finite subsets of a given set. The definition, however, is specific to Isabelle and based on the notion of "datatype", obviously not something that belongs to ZF set theory. This theory file develops the notion of finite powerset similarly as in `Finite.thy`, but based on standard library's `Cardinal.thy`. This theory file is intended to replace IsarMathLib's `Finite1` and `Finite_ZF_1` theories that are currently derived from the "datatype" approach.

12.1 Definition and basic properties of finite powerset

The goal of this section is to prove an induction theorem about finite powersets: if the empty set has some property and this property is preserved by adding a single element of a set, then this property is true for all finite subsets of this set.

We defined the finite powerset `FinPow(X)` as those elements of the powerset that are finite.

definition

$$\text{FinPow}(X) \equiv \{A \in \text{Pow}(X) . \text{Finite}(A)\}$$

The cardinality of an element of finite powerset is a natural number.

lemma `card_fin_is_nat`: **assumes** $A \in \text{FinPow}(X)$
shows $|A| \in \text{nat}$ **and** $A \approx |A|$
<proof>

The cardinality of a finite set is a natural number.

lemma `card_fin_is_nat1`: **assumes** $\text{Finite}(A)$ **shows** $|A| \in \text{nat}$
<proof>

A reformulation of `card_fin_is_nat`: for a finite set A there is a bijection between $|A|$ and A .

lemma `fin_bij_card`: **assumes** $A1: A \in \text{FinPow}(X)$
shows $\exists b. b \in \text{bij}(|A|, A)$
<proof>

If a set has the same number of elements as $n \in \mathbb{N}$, then its cardinality is n . Recall that in set theory a natural number n is a set that has n elements.

lemma `card_card`: **assumes** $A \approx n$ **and** $n \in \text{nat}$
shows $|A| = n$
<proof>

If we add a point to a finite set, the cardinality increases by one. To understand the second assertion $|A \cup \{a\}| = |A| \cup \{|A|\}$ recall that the cardinality $|A|$ of A is a natural number and for natural numbers we have $n+1 = n \cup \{n\}$.

lemma card_fin_add_one: **assumes** A1: $A \in \text{FinPow}(X)$ **and** A2: $a \in X - A$
shows
 $|A \cup \{a\}| = \text{succ}(|A|)$
 $|A \cup \{a\}| = |A| \cup \{|A|\}$
<proof>

We can decompose the finite powerset into collection of sets of the same natural cardinalities.

lemma finpow_decomp:
shows $\text{FinPow}(X) = (\bigcup n \in \text{nat}. \{A \in \text{Pow}(X). A \approx n\})$
<proof>

Finite powerset is the union of sets of cardinality bounded by natural numbers.

lemma finpow_union_card_nat:
shows $\text{FinPow}(X) = (\bigcup n \in \text{nat}. \{A \in \text{Pow}(X). A \lesssim n\})$
<proof>

A different form of finpow_union_card_nat (see above) - a subset that has not more elements than a given natural number is in the finite powerset.

lemma lepoll_nat_in_finpow:
assumes $n \in \text{nat}$ $A \subseteq X$ $A \lesssim n$
shows $A \in \text{FinPow}(X)$
<proof>

Natural numbers are finite subsets of the set of natural numbers.

lemma nat_finpow_nat: **assumes** $n \in \text{nat}$ **shows** $n \in \text{FinPow}(\text{nat})$
<proof>

A finite subset is a finite subset of itself.

lemma fin_finpow_self: **assumes** $A \in \text{FinPow}(X)$ **shows** $A \in \text{FinPow}(A)$
<proof>

A set is finite iff it is in its finite powerset.

lemma fin_finpow_iff: **shows** $\text{Finite}(A) \longleftrightarrow A \in \text{FinPow}(A)$
<proof>

Induction for finite powerset. This is similar to the standard Isabelle's Fin_induct.

theorem FinPow_induct: **assumes** A1: $P(0)$ **and**
A2: $\forall A \in \text{FinPow}(X). P(A) \longrightarrow (\forall a \in X. P(A \cup \{a\}))$ **and**
A3: $B \in \text{FinPow}(X)$
shows $P(B)$
<proof>

A subset of a finite subset is a finite subset.

lemma subset_finpow: **assumes** $A \in \text{FinPow}(X)$ **and** $B \subseteq A$

shows $B \in \text{FinPow}(X)$
 $\langle \text{proof} \rangle$

If we subtract anything from a finite set, the resulting set is finite.

lemma `diff_finpow`:
assumes $A \in \text{FinPow}(X)$ **shows** $A-B \in \text{FinPow}(X)$
 $\langle \text{proof} \rangle$

If we remove a point from a finites subset, we get a finite subset.

corollary `fin_rem_point_fin`: **assumes** $A \in \text{FinPow}(X)$
shows $A - \{a\} \in \text{FinPow}(X)$
 $\langle \text{proof} \rangle$

Cardinality of a nonempty finite set is a successor of some natural number.

lemma `card_non_empty_succ`:
assumes $A1: A \in \text{FinPow}(X)$ **and** $A2: A \neq 0$
shows $\exists n \in \text{nat}. |A| = \text{succ}(n)$
 $\langle \text{proof} \rangle$

Nonempty set has non-zero cardinality. This is probably true without the assumption that the set is finite, but I couldn't derive it from standard Isabelle theorems.

lemma `card_non_empty_non_zero`:
assumes $A \in \text{FinPow}(X)$ **and** $A \neq 0$
shows $|A| \neq 0$
 $\langle \text{proof} \rangle$

Another variation on the induction theme: If we can show something holds for the empty set and if it holds for all finite sets with at most k elements then it holds for all finite sets with at most $k + 1$ elements, then it holds for all finite sets.

theorem `FinPow_card_ind`: **assumes** $A1: P(0)$ **and**
 $A2: \forall k \in \text{nat}. (\forall A \in \text{FinPow}(X). A \lesssim k \longrightarrow P(A)) \longrightarrow$
 $(\forall A \in \text{FinPow}(X). A \lesssim \text{succ}(k) \longrightarrow P(A))$
and $A3: A \in \text{FinPow}(X)$ **shows** $P(A)$
 $\langle \text{proof} \rangle$

Another type of induction (or, maybe recursion). In the induction step we try to find a point in the set that if we remove it, the fact that the property holds for the smaller set implies that the property holds for the whole set.

lemma `FinPow_ind_rem_one`: **assumes** $A1: P(0)$ **and**
 $A2: \forall A \in \text{FinPow}(X). A \neq 0 \longrightarrow (\exists a \in A. P(A-\{a\}) \longrightarrow P(A))$
and $A3: B \in \text{FinPow}(X)$
shows $P(B)$
 $\langle \text{proof} \rangle$

Yet another induction theorem. This is similar, but slightly more complicated than `FinPow_ind_rem_one`. The difference is in the treatment of the empty set to allow to show properties that are not true for empty set.

lemma `FinPow_rem_ind`: **assumes** `A1`: $\forall A \in \text{FinPow}(X).$
 $A = 0 \vee (\exists a \in A. A = \{a\} \vee P(A - \{a\}) \longrightarrow P(A))$
and `A2`: $A \in \text{FinPow}(X)$ **and** `A3`: $A \neq 0$
shows $P(A)$
<proof>

If a family of sets is closed with respect to taking intersections of two sets then it is closed with respect to taking intersections of any nonempty finite collection.

lemma `inter_two_inter_fin`:
assumes `A1`: $\forall V \in T. \forall W \in T. V \cap W \in T$ **and**
`A2`: $N \neq 0$ **and** `A3`: $N \in \text{FinPow}(T)$
shows $\bigcap N \in T$
<proof>

If a family of sets contains the empty set and is closed with respect to taking unions of two sets then it is closed with respect to taking unions of any finite collection.

lemma `union_two_union_fin`:
assumes `A1`: $\emptyset \in C$ **and** `A2`: $\forall A \in C. \forall B \in C. A \cup B \in C$ **and**
`A3`: $N \in \text{FinPow}(C)$
shows $\bigcup N \in C$
<proof>

Empty set is in finite power set, hence finite power set is never empty.

lemma `empty_in_finpow`: **shows** $\emptyset \in \text{FinPow}(X)$ **and** $\text{FinPow}(X) \neq \emptyset$
<proof>

Singleton is in the finite powerset.

lemma `singleton_in_finpow`: **assumes** $x \in X$
shows $\{x\} \in \text{FinPow}(X)$ *<proof>*

If a set is nonempty then its finite power set contains a nonempty set.

lemma `finpow_nempty_nempty`: **assumes** $X \neq \emptyset$ **shows** $\text{FinPow}(X) \setminus \{\emptyset\} \neq \emptyset$
<proof>

Union of two finite subsets is a finite subset.

lemma `union_finpow`: **assumes** $A \in \text{FinPow}(X)$ **and** $B \in \text{FinPow}(X)$
shows $A \cup B \in \text{FinPow}(X)$
<proof>

The finite power set of a larger set is larger.

lemma `finpow_mono`: **assumes** $A \subseteq B$ **shows** $\text{FinPow}(A) \subseteq \text{FinPow}(B)$

<proof>

Union of finite number of finite sets is finite.

lemma `fin_union_finpow`: **assumes** $M \in \text{FinPow}(\text{FinPow}(X))$
shows $\bigcup M \in \text{FinPow}(X)$
<proof>

If a set is finite after removing one element, then it is finite.

lemma `rem_point_fin_fin`:
assumes $A1: x \in X$ **and** $A2: A - \{x\} \in \text{FinPow}(X)$
shows $A \in \text{FinPow}(X)$
<proof>

An image of a finite set is finite.

lemma `fin_image_fin`: **assumes** $\forall V \in B. K(V) \in C$ **and** $N \in \text{FinPow}(B)$
shows $\{K(V). V \in N\} \in \text{FinPow}(C)$
<proof>

A variant of `fin_image_fin` with a bit weaker first assumption:

lemma `fin_image_fin1`: **assumes** $\forall V \in N. K(V) \in C$ **and** $N \in \text{FinPow}(B)$
shows $\{K(V). V \in N\} \in \text{FinPow}(C)$
<proof>

An image of a nonempty finite set is a nonempty finite set.

lemma `fin_image_fin0`: **assumes** $N \in \text{FinPow}(B) \setminus \{\emptyset\}$ **and** $\forall V \in N. K(V) \in C$
shows $\{K(V). V \in N\} \in \text{FinPow}(C) \setminus \{\emptyset\}$
<proof>

A version of `union_two_union_fin` for nonempty finite subsets of some set X : if a family of sets contains all singletons from a finite subset N of X and is closed with respect to taking unions of two sets then it contains N .

lemma `union_two_union_fin_nempty`:
assumes $N \in \text{FinPow}(X)$ $N \neq \emptyset$ $\forall x \in N. \{x\} \in C$ $\forall A \in C. \forall B \in C. A \cup B \in C$
shows $N \in C$
<proof>

If a set X is finite then the set $\{K(x). x \in X\}$ is also finite. It's basically standard Isabelle/ZF `Finite_RepFun` in nicer notation.

lemma `fin_rep_fin`: **assumes** $\text{Finite}(X)$ **shows** $\text{Finite}(\{K(x). x \in X\})$
<proof>

The image of a singleton by any function is finite. It's of course either empty or has exactly one element, but showing that it's a finite subset of the codomain is good enough for us.

lemma `image_singleton_fin`: **assumes** $f: X \rightarrow Y$ **shows** $f\{x\} \in \text{FinPow}(Y)$
<proof>

Union of a finite indexed family of finite sets is finite.

```

lemma union_fin_list_fin:
  assumes A1:  $n \in \text{nat}$  and A2:  $\forall k \in n. N(k) \in \text{FinPow}(X)$ 
  shows
     $\{N(k). k \in n\} \in \text{FinPow}(\text{FinPow}(X))$  and  $(\bigcup k \in n. N(k)) \in \text{FinPow}(X)$ 
  <proof>
end

```

13 Semilattices and Lattices

```

theory Lattice_ZF imports Order_ZF_1a func1 Finite_ZF

```

```

begin

```

Lattices can be introduced in algebraic way as commutative idempotent $(x \cdot x = x)$ semigroups or as partial orders with some additional properties. These two approaches are equivalent. In this theory we will use the order-theoretic approach.

13.1 Semilattices

We start with a relation r which is a partial order on a set L . Such situation is defined in `Order_ZF` as the predicate `IsPartOrder(L,r)`.

A partially ordered (L, r) set is a join-semilattice if each two-element subset of L has a supremum (i.e. the least upper bound).

definition

```

IsJoinSemilattice(L,r)  $\equiv$ 
   $r \subseteq L \times L \wedge \text{IsPartOrder}(L,r) \wedge (\forall x \in L. \forall y \in L. \text{HasAsupremum}(r, \{x,y\}))$ 

```

A partially ordered (L, r) set is a meet-semilattice if each two-element subset of L has an infimum (i.e. the greatest lower bound).

definition

```

IsMeetSemilattice(L,r)  $\equiv$ 
   $r \subseteq L \times L \wedge \text{IsPartOrder}(L,r) \wedge (\forall x \in L. \forall y \in L. \text{HasAnInfimum}(r, \{x,y\}))$ 

```

Join is a binary operation whose value on a pair $\langle x, y \rangle$ is defined as the supremum of the set $\{x, y\}$.

definition

```

Join(L,r)  $\equiv \{\langle p, \text{Supremum}(r, \{\text{fst}(p), \text{snd}(p)\}) \rangle \mid p \in L \times L\}$ 

```

Meet is a binary operation whose value on a pair $\langle x, y \rangle$ is defined as the infimum of the set $\{x, y\}$.

definition

```

Meet(L,r)  $\equiv \{\langle p, \text{Infimum}(r, \{\text{fst}(p), \text{snd}(p)\}) \rangle \mid p \in L \times L\}$ 

```

In a join-semilattice join is indeed a binary operation.

lemma join_is_binop: **assumes** IsJoinSemilattice(L,r)
shows Join(L,r) : $L \times L \rightarrow L$
<proof>

The value of Join(L,r) on a pair $\langle x, y \rangle$ is the supremum of the set $\{x, y\}$, hence its is greater or equal than both.

lemma join_val:
assumes IsJoinSemilattice(L,r) $x \in L$ $y \in L$
defines $j \equiv \text{Join}(L,r)\langle x, y \rangle$
shows $j \in L$ $j = \text{Supremum}(r, \{x, y\})$ $\langle x, j \rangle \in r$ $\langle y, j \rangle \in r$
<proof>

Join of x, y is less or equal than any upper bound for $\{x, y\}$.

lemma join_least_upbound:
assumes IsJoinSemilattice(L,r) $\langle x, z \rangle \in r$ $\langle y, z \rangle \in r$
shows $\langle \text{Join}(L,r)\langle x, y \rangle, z \rangle \in r$
<proof>

In a meet-semilattice meet is indeed a binary operation.

lemma meet_is_binop: **assumes** IsMeetSemilattice(L,r)
shows Meet(L,r) : $L \times L \rightarrow L$
<proof>

The value of Meet(L,r) on a pair $\langle x, y \rangle$ is the infimum of the set $\{x, y\}$, hence is less or equal than both.

lemma meet_val:
assumes IsMeetSemilattice(L,r) $x \in L$ $y \in L$
defines $m \equiv \text{Meet}(L,r)\langle x, y \rangle$
shows $m \in L$ $m = \text{Infimum}(r, \{x, y\})$ $\langle m, x \rangle \in r$ $\langle m, y \rangle \in r$
<proof>

In a join semilattice a union of two sets that have suprema has a supremum.

lemma join_semiat_sup_two: **assumes** IsJoinSemilattice(L,r)
HasAsupremum(r,A) HasAsupremum(r,B)
shows HasAsupremum(r,A $\cup$ B) **and**
 $\text{Supremum}(r, A \cup B) = \text{Join}(L,r)\langle \text{Supremum}(r,A), \text{Supremum}(r,B) \rangle$
<proof>

In a meet semilattice a union of two sets that have infima has an infimum.

lemma meet_semiat_inf_two: **assumes** IsMeetSemilattice(L,r)
HasAnInfimum(r,A) HasAnInfimum(r,B)
shows HasAnInfimum(r,A $\cup$ B) **and**
 $\text{Infimum}(r, A \cup B) = \text{Meet}(L,r)\langle \text{Infimum}(r,A), \text{Infimum}(r,B) \rangle$
<proof>

In a join semilattice every nonempty finite set has a supremum.

```

lemma join_semilat_fin_sup:
  assumes IsJoinSemilattice(L,r) N∈FinPow(L) N≠∅
  shows HasAsupremum(r,N)
⟨proof⟩

```

In a meet semilattice every nonempty finite set has a infimum.

```

lemma meet_semilat_fin_inf:
  assumes IsMeetSemilattice(L,r) N∈FinPow(L) N≠∅
  shows HasAnInfimum(r,N)
⟨proof⟩

```

In a (nonempty) meet semi-lattice the relation down-directs the set.

```

lemma meet_down_directs: assumes IsMeetSemilattice(L,r) L≠0
  shows r {down-directs} L
⟨proof⟩

```

In a (nonempty) join semi-lattice the relation up-directs the set.

```

lemma join_up_directs: assumes IsJoinSemilattice(L,r) L≠0
  shows r {up-directs} L
⟨proof⟩

```

The next locale defines a notation for join-semilattice. We will use the $\sqcup$ symbol rather than more common $\vee$ to avoid confusion with logical "or".

```

locale join_semilatt =
  fixes L
  fixes r
  assumes joinLatt: IsJoinSemilattice(L,r)
  fixes join (infixl  $\sqcup$  71)
  defines join_def [simp]:  $x \sqcup y \equiv \text{Join}(L,r)\langle x,y \rangle$ 
  fixes sup (sup _ )
  defines sup_def [simp]:  $\text{sup } A \equiv \text{Supremum}(r,A)$ 

```

Join of the elements of the lattice is in the lattice.

```

lemma (in join_semilatt) join_props: assumes x∈L y∈L
  shows  $x \sqcup y \in L$  and  $x \sqcup y = \text{sup } \{x,y\}$ 
⟨proof⟩

```

Join is associative.

```

lemma (in join_semilatt) join_assoc: assumes x∈L y∈L z∈L
  shows  $x \sqcup (y \sqcup z) = x \sqcup y \sqcup z$ 
⟨proof⟩

```

Join is idempotent.

```

lemma (in join_semilatt) join_idempotent: assumes x∈L shows  $x \sqcup x = x$ 
⟨proof⟩

```

The `meet_semilatt` locale is the dual of the join-semilattice locale defined above. We will use the $\sqcap$ symbol to denote join, giving it a bit higher precedence.

```
locale meet_semilatt =
  fixes L
  fixes r
  assumes meetLatt: IsMeetSemilattice(L,r)
  fixes join (infixl  $\sqcap$  72)
  defines join_def [simp]:  $x \sqcap y \equiv \text{Meet}(L,r)\langle x,y \rangle$ 
  fixes sup (infixl  $\sqcup$  72)
  defines sup_def [simp]:  $\inf A \equiv \text{Infimum}(r,A)$ 
```

Meet of the elements of the lattice is in the lattice.

```
lemma (in meet_semilatt) meet_props: assumes  $x \in L \ y \in L$ 
  shows  $x \sqcap y \in L$  and  $x \sqcap y = \inf \{x,y\}$ 
  <proof>
```

Meet is associative.

```
lemma (in meet_semilatt) meet_assoc: assumes  $x \in L \ y \in L \ z \in L$ 
  shows  $x \sqcap (y \sqcap z) = x \sqcap y \sqcap z$ 
  <proof>
```

Meet is idempotent.

```
lemma (in meet_semilatt) meet_idempotent: assumes  $x \in L$  shows  $x \sqcap x = x$ 
  <proof>
```

13.2 Lattices

Here we study lattices, i.e. structures where both meet and join are defined.

A partially ordered (L, r) set is a lattice if it is both join and meet-semilattice, i.e. if every two element set has a supremum (least upper bound) and infimum (greatest lower bound).

definition

```
IsAlattice (infixl {is a lattice on} 90) where
  r {is a lattice on} L  $\equiv$  IsJoinSemilattice(L,r)  $\wedge$  IsMeetSemilattice(L,r)
```

Linear order is a lattice.

```
lemma lin_is_latt: assumes  $r \subseteq L \times L$  and IsLinOrder(L,r)
  shows r {is a lattice on} L
  <proof>
```

13.3 Complete lattices

The definition of a lattice requires that every two-element set has a supremum and infimum. As we show in `join_semilat_fin_sup` and `meet_semilat_fin_inf`

this implies that every finite subset of the lattice has supremum and infimum. In this section we study structures in which every nonempty subset (not necessarily finite) has both supremum and infimum.

Wikipedia defines a complete lattice as a partially ordered set in which every subset has both supremum and infimum. The way we define supremum and infimum only nonempty sets can have infima or suprema so to avoid inconsistency we define a complete lattice as the one in which all nonempty subsets have both infima and suprema. We also add the assumption that the carrier is nonempty so that we don't have to repeat that assumption in every proposition about complete lattices.

definition

$$\text{IsCompleteLattice}(r, L) \equiv L \neq \emptyset \wedge r \subseteq L \times L \wedge \text{IsPartOrder}(L, r) \wedge (\forall A \in \text{Pow}(L) \setminus \{\emptyset\}. (\text{HasASupremum}(r, A) \wedge \text{HasAnInfimum}(r, A)))$$

Complete lattices are bounded and have both maximum and minimum.

lemma `compl_lat_bounded`: **assumes** `IsCompleteLattice(r, L)`
shows `HasAmaximum(r, L) HasAminimum(r, L) IsBounded(L, r)`
<proof>

end

14 Finite sets

theory `Finite1` **imports** `ZF.EquivClass ZF.Finite func1 ZF1`

begin

This theory extends Isabelle standard `Finite` theory. It is obsolete and should not be used for new development. Use the `Finite_ZF` instead.

14.1 Finite powerset

In this section we consider various properties of `Fin` datatype (even though there are no datatypes in ZF set theory).

In `Topology_ZF` theory we consider induced topology that is obtained by taking a subset of a topological space. To show that a topology restricted to a subset is also a topology on that subset we may need a fact that if T is a collection of sets and A is a set then every finite collection $\{V_i\}$ is of the form $V_i = U_i \cap A$, where $\{U_i\}$ is a finite subcollection of T . This is one of those trivial facts that require suprisingly long formal proof. Actually, the need for this fact is avoided by requiring intersection two open sets to be open (rather than intersection of a finite number of open sets). Still, the fact is left here as an example of a proof by induction. We will use `Fin_induct`

lemma from Finite.thy. First we define a property of finite sets that we want to show.

definition

$\text{Prfin}(T, A, M) \equiv (M = 0) \mid (\exists N \in \text{Fin}(T). \forall V \in M. \exists U \in N. (V = U \cap A))$

Now we show the main induction step in a separate lemma. This will make the proof of the theorem FinRestr below look short and nice. The premises of the ind_step lemma are those needed by the main induction step in lemma Fin_induct (see standard Isabelle's Finite.thy).

lemma ind_step: assumes $A: \forall V \in TA. \exists U \in T. V = U \cap A$
 and $A1: W \in TA$ and $A2: M \in \text{Fin}(TA)$
 and $A3: W \notin M$ and $A4: \text{Prfin}(T, A, M)$
 shows $\text{Prfin}(T, A, \text{cons}(W, M))$

<proof>

Now we are ready to prove the statement we need.

theorem FinRestr0: assumes $A: \forall V \in TA. \exists U \in T. V = U \cap A$
 shows $\forall M \in \text{Fin}(TA). \text{Prfin}(T, A, M)$

<proof>

This is a different form of the above theorem:

theorem ZF1FinRestr:

assumes $A1: M \in \text{Fin}(TA)$ and $A2: M \neq 0$
 and $A3: \forall V \in TA. \exists U \in T. V = U \cap A$
 shows $\exists N \in \text{Fin}(T). (\forall V \in M. \exists U \in N. (V = U \cap A)) \wedge N \neq 0$

<proof>

Purely technical lemma used in Topology_ZF_1 to show that if a topology is T_2 , then it is T_1 .

lemma Finite1_L2:

assumes $A: \exists U \forall V. (U \in T \wedge V \in T \wedge x \in U \wedge y \in V \wedge U \cap V = 0)$
 shows $\exists U \in T. (x \in U \wedge y \notin U)$

<proof>

A collection closed with respect to taking a union of two sets is closed under taking finite unions. Proof by induction with the induction step formulated in a separate lemma.

lemma Finite1_L3_IndStep:

assumes $A1: \forall A \ B. ((A \in C \wedge B \in C) \longrightarrow A \cup B \in C)$
 and $A2: A \in C$ and $A3: N \in \text{Fin}(C)$ and $A4: A \notin N$ and $A5: \bigcup N \in C$
 shows $\bigcup \text{cons}(A, N) \in C$

<proof>

The lemma: a collection closed with respect to taking a union of two sets is closed under taking finite unions.

lemma Finite1_L3:

assumes A1: $0 \in C$ **and** A2: $\forall A B. ((A \in C \wedge B \in C) \longrightarrow A \cup B \in C)$ **and**
 A3: $N \in \text{Fin}(C)$
shows $\bigcup N \in C$
<proof>

A collection closed with respect to taking a intersection of two sets is closed under taking finite intersections. Proof by induction with the induction step formulated in a separate lemma. This is slightly more involved than the union case in `Finite1_L3`, because the intersection of empty collection is undefined (or should be treated as such). To simplify notation we define the property to be proven for finite sets as a separate notion.

definition

$\text{IntPr}(T, N) \equiv (N = 0 \mid \bigcap N \in T)$

The induction step.

lemma `Finite1_L4_IndStep`:
assumes A1: $\forall A B. ((A \in T \wedge B \in T) \longrightarrow A \cap B \in T)$
and A2: $A \in T$ **and** A3: $N \in \text{Fin}(T)$ **and** A4: $A \notin N$ **and** A5: $\text{IntPr}(T, N)$
shows $\text{IntPr}(T, \text{cons}(A, N))$
<proof>

The lemma.

lemma `Finite1_L4`:
assumes A1: $\forall A B. A \in T \wedge B \in T \longrightarrow A \cap B \in T$
and A2: $N \in \text{Fin}(T)$
shows $\text{IntPr}(T, N)$
<proof>

Next is a restatement of the above lemma that does not depend on the `IntPr` meta-function.

lemma `Finite1_L5`:
assumes A1: $\forall A B. ((A \in T \wedge B \in T) \longrightarrow A \cap B \in T)$
and A2: $N \neq 0$ **and** A3: $N \in \text{Fin}(T)$
shows $\bigcap N \in T$
<proof>

The images of finite subsets by a meta-function are finite. For example in topology if we have a finite collection of sets, then closing each of them results in a finite collection of closed sets. This is a very useful lemma with many unexpected applications. The proof is by induction. The next lemma is the induction step.

lemma `fin_image_fin_IndStep`:
assumes $\forall V \in B. K(V) \in C$
and $U \in B$ **and** $N \in \text{Fin}(B)$ **and** $U \notin N$ **and** $\{K(V). V \in N\} \in \text{Fin}(C)$
shows $\{K(V). V \in \text{cons}(U, N)\} \in \text{Fin}(C)$
<proof>

The lemma:

lemma `fin_image_fin`:
assumes $A1: \forall V \in B. K(V) \in C$ **and** $A2: N \in \text{Fin}(B)$
shows $\{K(V). V \in N\} \in \text{Fin}(C)$
<proof>

The image of a finite set is finite.

lemma `Finite1_L6A`: **assumes** $A1: f: X \rightarrow Y$ **and** $A2: N \in \text{Fin}(X)$
shows $f(N) \in \text{Fin}(Y)$
<proof>

If the set defined by a meta-function is finite, then every set defined by a composition of this meta function with another one is finite.

lemma `Finite1_L6B`:
assumes $A1: \forall x \in X. a(x) \in Y$ **and** $A2: \{b(y). y \in Y\} \in \text{Fin}(Z)$
shows $\{b(a(x)). x \in X\} \in \text{Fin}(Z)$
<proof>

If the set defined by a meta-function is finite, then every set defined by a composition of this meta function with another one is finite.

lemma `Finite1_L6C`:
assumes $A1: \forall y \in Y. b(y) \in Z$ **and** $A2: \{a(x). x \in X\} \in \text{Fin}(Y)$
shows $\{b(a(x)). x \in X\} \in \text{Fin}(Z)$
<proof>

Cartesian product of finite sets is finite.

lemma `Finite1_L12`: **assumes** $A1: A \in \text{Fin}(A)$ **and** $A2: B \in \text{Fin}(B)$
shows $A \times B \in \text{Fin}(A \times B)$
<proof>

We define the characterisic meta-function that is the identity on a set and assigns a default value everywhere else.

definition

$\text{Characteristic}(A, \text{default}, x) \equiv (\text{if } x \in A \text{ then } x \text{ else default})$

A finite subset is a finite subset of itself.

lemma `Finite1_L13`:
assumes $A1: A \in \text{Fin}(X)$ **shows** $A \in \text{Fin}(A)$
<proof>

Cartesian product of finite subsets is a finite subset of cartesian product.

lemma `Finite1_L14`: **assumes** $A1: A \in \text{Fin}(X)$ $B \in \text{Fin}(Y)$
shows $A \times B \in \text{Fin}(X \times Y)$
<proof>

The next lemma is needed in the `Group_ZF_3` theory in a couple of places.

lemma `Finite1_L15`:

assumes A1: $\{b(x). x \in A\} \in \text{Fin}(B)$ $\{c(x). x \in A\} \in \text{Fin}(C)$
and A2: $f : B \times C \rightarrow E$
shows $\{f \langle b(x), c(x) \rangle. x \in A\} \in \text{Fin}(E)$
 $\langle \text{proof} \rangle$

Singletons are in the finite powerset.

lemma Finite1_L16: **assumes** $x \in X$ **shows** $\{x\} \in \text{Fin}(X)$
 $\langle \text{proof} \rangle$

A special case of Finite1_L15 where the second set is a singleton. In Group_ZF_3 theory this corresponds to the situation where we multiply by a constant.

lemma Finite1_L16AA: **assumes** $\{b(x). x \in A\} \in \text{Fin}(B)$
and $c \in C$ **and** $f : B \times C \rightarrow E$
shows $\{f \langle b(x), c \rangle. x \in A\} \in \text{Fin}(E)$
 $\langle \text{proof} \rangle$

First order version of the induction for the finite powerset.

lemma Finite1_L16B: **assumes** A1: $P(0)$ **and** A2: $B \in \text{Fin}(X)$
and A3: $\forall A \in \text{Fin}(X). \forall x \in X. x \notin A \wedge P(A) \longrightarrow P(A \cup \{x\})$
shows $P(B)$
 $\langle \text{proof} \rangle$

14.2 Finite range functions

In this section we define functions $f : X \rightarrow Y$, with the property that $f(X)$ is a finite subset of Y . Such functions play an important role in the construction of real numbers in the Real_ZF series.

Definition of finite range functions.

definition
 $\text{FinRangeFunctions}(X, Y) \equiv \{f : X \rightarrow Y. f(X) \in \text{Fin}(Y)\}$

Constant functions have finite range.

lemma Finite1_L17: **assumes** $c \in Y$ **and** $X \neq 0$
shows $\text{ConstantFunction}(X, c) \in \text{FinRangeFunctions}(X, Y)$
 $\langle \text{proof} \rangle$

Finite range functions have finite range.

lemma Finite1_L18: **assumes** $f \in \text{FinRangeFunctions}(X, Y)$
shows $\{f(x). x \in X\} \in \text{Fin}(Y)$
 $\langle \text{proof} \rangle$

An alternative form of the definition of finite range functions.

lemma Finite1_L19: **assumes** $f : X \rightarrow Y$
and $\{f(x). x \in X\} \in \text{Fin}(Y)$
shows $f \in \text{FinRangeFunctions}(X, Y)$

<proof>

A composition of a finite range function with another function is a finite range function.

lemma Finite1_L20: **assumes** A1: $f \in \text{FinRangeFunctions}(X,Y)$
 and A2: $g : Y \rightarrow Z$
 shows $g \circ f \in \text{FinRangeFunctions}(X,Z)$
<proof>

Image of any subset of the domain of a finite range function is finite.

lemma Finite1_L21:
 assumes $f \in \text{FinRangeFunctions}(X,Y)$ **and** $A \subseteq X$
 shows $f(A) \in \text{Fin}(Y)$
<proof>

end

15 Finite sets 1

theory Finite_ZF_1 **imports** Finite1 Order_ZF_1a

begin

This theory is based on `Finite1` theory and is obsolete. It contains properties of finite sets related to order relations. See the `FinOrd` theory for a better approach.

15.1 Finite vs. bounded sets

The goal of this section is to show that finite sets are bounded and have maxima and minima.

Finite set has a maximum - induction step.

lemma Finite_ZF_1_1_L1:
 assumes A1: $r \text{ \{is total on\} } X$ **and** A2: $\text{trans}(r)$
 and A3: $A \in \text{Fin}(X)$ **and** A4: $x \in X$ **and** A5: $A = 0 \vee \text{HasAmaximum}(r,A)$
 shows $A \cup \{x\} = 0 \vee \text{HasAmaximum}(r,A \cup \{x\})$
<proof>

For total and transitive relations finite set has a maximum.

theorem Finite_ZF_1_1_T1A:
 assumes A1: $r \text{ \{is total on\} } X$ **and** A2: $\text{trans}(r)$
 and A3: $B \in \text{Fin}(X)$
 shows $B = 0 \vee \text{HasAmaximum}(r,B)$
<proof>

Finite set has a minimum - induction step.

lemma Finite_ZF_1_1_L2:
 assumes A1: r {is total on} X and A2: $\text{trans}(r)$
 and A3: $A \in \text{Fin}(X)$ and A4: $x \in X$ and A5: $A = 0 \vee \text{HasAminum}(r, A)$
 shows $A \cup \{x\} = 0 \vee \text{HasAminum}(r, A \cup \{x\})$
<proof>

For total and transitive relations finite set has a minimum.

theorem Finite_ZF_1_1_T1B:
 assumes A1: r {is total on} X and A2: $\text{trans}(r)$
 and A3: $B \in \text{Fin}(X)$
 shows $B = 0 \vee \text{HasAminum}(r, B)$
<proof>

For transitive and total relations finite sets are bounded.

theorem Finite_ZF_1_T1:
 assumes A1: r {is total on} X and A2: $\text{trans}(r)$
 and A3: $B \in \text{Fin}(X)$
 shows $\text{IsBounded}(B, r)$
<proof>

For linearly ordered finite sets maximum and minimum have desired properties. The reason we need linear order is that we need the order to be total and transitive for the finite sets to have a maximum and minimum and then we also need antisymmetry for the maximum and minimum to be unique.

theorem Finite_ZF_1_T2:
 assumes A1: $\text{IsLinOrder}(X, r)$ and A2: $A \in \text{Fin}(X)$ and A3: $A \neq 0$
 shows
 $\text{Maximum}(r, A) \in A$
 $\text{Minimum}(r, A) \in A$
 $\forall x \in A. \langle x, \text{Maximum}(r, A) \rangle \in r$
 $\forall x \in A. \langle \text{Minimum}(r, A), x \rangle \in r$
<proof>

A special case of Finite_ZF_1_T2 when the set has three elements.

corollary Finite_ZF_1_L2A:
 assumes A1: $\text{IsLinOrder}(X, r)$ and A2: $a \in X \quad b \in X \quad c \in X$
 shows
 $\text{Maximum}(r, \{a, b, c\}) \in \{a, b, c\}$
 $\text{Minimum}(r, \{a, b, c\}) \in \{a, b, c\}$
 $\text{Maximum}(r, \{a, b, c\}) \in X$
 $\text{Minimum}(r, \{a, b, c\}) \in X$
 $\langle a, \text{Maximum}(r, \{a, b, c\}) \rangle \in r$
 $\langle b, \text{Maximum}(r, \{a, b, c\}) \rangle \in r$
 $\langle c, \text{Maximum}(r, \{a, b, c\}) \rangle \in r$
<proof>

If for every element of X we can find one in A that is greater, then the A can not be finite. Works for relations that are total, transitive and antisymmetric.

```

lemma Finite_ZF_1_1_L3:
  assumes A1: r {is total on} X
  and A2: trans(r) and A3: antisym(r)
  and A4: r  $\subseteq$  X $\times$ X and A5: X $\neq$ 0
  and A6:  $\forall x \in X. \exists a \in A. x \neq a \wedge \langle x, a \rangle \in r$ 
  shows A  $\notin$  Fin(X)
 $\langle proof \rangle$ 

end

```

16 Finite sets and order relations

```

theory FinOrd_ZF imports Finite_ZF func_ZF_1 NatOrder_ZF

```

```

begin

```

This theory file contains properties of finite sets related to order relations. Part of this is similar to what is done in `Finite_ZF_1` except that the development is based on the notion of finite powerset defined in `Finite_ZF` rather than the one defined in standard Isabelle `Finite` theory.

16.1 Finite vs. bounded sets

The goal of this section is to show that finite sets are bounded and have maxima and minima.

For total and transitive relations nonempty finite set has a maximum.

```

theorem fin_has_max:
  assumes A1: r {is total on} X and A2: trans(r)
  and A3: B  $\in$  FinPow(X) and A4: B  $\neq$  0
  shows HasAmaximum(r,B)
 $\langle proof \rangle$ 

```

For total and transitive relations nonempty finite set has a minimum.

```

theorem fin_has_min:
  assumes r {is total on} X trans(r) B  $\in$  FinPow(X) B $\neq$ 0
  shows HasAminimum(r,B)
 $\langle proof \rangle$ 

```

For linearly ordered nonempty finite sets the maximum is in the set and indeed it is the greatest element of the set.

```

lemma linord_min_max_props:
  assumes IsLinOrder(X,r) A  $\in$  FinPow(X) A $\neq$  $\emptyset$ 
  shows
    Maximum(r,A)  $\in$  A  $\forall a \in A. \langle a, \text{Maximum}(r,A) \rangle \in r$ 
    Minimum(r,A)  $\in$  A  $\forall a \in A. \langle \text{Minimum}(r,A), a \rangle \in r$ 
    Maximum(r,A)  $\in$  X Minimum(r,A)  $\in$  X

```

<proof>

Suppose we have a linear relation r on X and a subset $B \subseteq X$ with an element $x \in B$ such that there is only finite number of elements y of B for which $\langle y, x \rangle \in r$. Then B has a minimum with the usual properties.

lemma `leq_fin_has_min`:
assumes `IsLinOrder(X,r)` `B⊆X` `x∈B` **and**
 $\{y \in B. \langle y, x \rangle \in r\} \in \text{FinPow}(X)$
shows
`HasAminimum(r,B)`
`Minimum(r,B) ∈ B`
 $\forall y \in B. \langle \text{Minimum}(r,B), y \rangle \in r$

<proof>

Suppose we have a linear relation r on X and a subset $B \subseteq X$ with an element $x \in B$ such that there is only a finite number of elements y of B for which $\langle x, y \rangle \in r$. Then B has a maximum with the usual properties.

lemma `leq_fin_has_max`:
assumes `IsLinOrder(X,r)` `B⊆X` `x∈B` **and**
 $\{y \in B. \langle x, y \rangle \in r\} \in \text{FinPow}(X)$
shows
`HasAmaximum(r,B)`
`Maximum(r,B) ∈ B`
 $\forall y \in B. \langle y, \text{Maximum}(r,B) \rangle \in r$

<proof>

Every nonempty set of natural numbers has a minimum with the usual properties.

theorem `subset_nat_has_min`: **assumes** `B⊆nat` `B≠∅`
shows
`HasAminimum(Le,B)`
`Minimum(Le,B) ∈ B`
 $\forall n \in B. \text{Minimum}(Le,B) \leq n$

<proof>

Every nonempty subset of a natural number has a maximum with expected properties.

lemma `nat_max_props`: **assumes** `n∈nat` `A⊆n` `A≠0`
shows
`Maximum(Le,A) ∈ A`
`Maximum(Le,A) ∈ nat`
 $\forall k \in A. k \leq \text{Maximum}(Le,A)$

<proof>

Yet another version of induction where the induction step is valid only up to $n \in \mathbb{N}$ rather than for all natural numbers. This lemma is redundant as it is easier to prove this assertion using lemma `fin_nat_ind` from `Nat_ZF_I`

which was done in lemma `fin_nat_ind1` there. It is left here for now as an alternative proof based on properties of the maximum of a finite set.

```
lemma ind_on_nat2:
  assumes n∈nat and P(0) and ∀j∈n. P(j)⟶P(j #+ 1)
  shows ∀j∈n #+ 1. P(j) and P(n)
⟨proof⟩
```

16.2 Order isomorphisms of finite sets

In this section we establish that if two linearly ordered finite sets have the same number of elements, then they are order-isomorphic and the isomorphism is unique. This allows us to talk about "enumeration" of a linearly ordered finite set. We define the enumeration as the order isomorphism between the number of elements of the set (which is a natural number $n = \{0, 1, \dots, n-1\}$) and the set.

A really weird corner case - empty set is order isomorphic with itself.

```
lemma empty_ord_iso: shows ord_iso(0,r,0,R) ≠ 0
⟨proof⟩
```

Even weirder than `empty_ord_iso` The order automorphism of the empty set is unique.

```
lemma empty_ord_iso_uniq:
  assumes f ∈ ord_iso(0,r,0,R) g ∈ ord_iso(0,r,0,R)
  shows f = g
⟨proof⟩
```

The empty set is the only order automorphism of itself.

```
lemma empty_ord_iso_empty: shows ord_iso(0,r,0,R) = {0}
⟨proof⟩
```

An induction (or maybe recursion?) scheme for linearly ordered sets. The induction step is that we show that if the property holds when the set is a singleton or for a set with the maximum removed, then it holds for the set. The idea is that since we can build any finite set by adding elements on the right, then if the property holds for the empty set and is invariant with respect to this operation, then it must hold for all finite sets.

```
lemma fin_ord_induction:
  assumes A1: IsLinOrder(X,r) and A2: P(0) and
  A3: ∀A ∈ FinPow(X). A ≠ 0 ⟶ (P(A - {Maximum(r,A)}) ⟶ P(A))
  and A4: B ∈ FinPow(X) shows P(B)
⟨proof⟩
```

A slightly more complicated version of `fin_ord_induction` that allows to prove properties that are not true for the empty set.

```
lemma fin_ord_ind:
```

assumes A1: IsLinOrder(X,r) **and** A2: $\forall A \in \text{FinPow}(X).$
 $A = 0 \vee (A = \{\text{Maximum}(r,A)\} \vee P(A - \{\text{Maximum}(r,A)\}) \longrightarrow P(A))$
and A3: $B \in \text{FinPow}(X)$ **and** A4: $B \neq 0$
shows $P(B)$
<proof>

Yet another induction scheme. We build a linearly ordered set by adding elements that are greater than all elements in the set.

lemma fin_ind_add_max:
assumes A1: IsLinOrder(X,r) **and** A2: $P(0)$ **and** A3: $\forall A \in \text{FinPow}(X).$
 $(\forall x \in X-A. P(A) \wedge (\forall a \in A. \langle a, x \rangle \in r) \longrightarrow P(A \cup \{x\}))$
and A4: $B \in \text{FinPow}(X)$
shows $P(B)$
<proof>

The only order automorphism of a linearly ordered finite set is the identity.

theorem fin_ord_auto_id: **assumes** A1: IsLinOrder(X,r)
and A2: $B \in \text{FinPow}(X)$ **and** A3: $B \neq 0$
shows $\text{ord_iso}(B,r,B,r) = \{\text{id}(B)\}$
<proof>

Every two finite linearly ordered sets are order isomorphic. The statement is formulated to make the proof by induction on the size of the set easier, see fin_ord_iso_ex for an alternative formulation.

lemma fin_order_iso:
assumes A1: IsLinOrder(X,r) IsLinOrder(Y,R) **and**
A2: $n \in \text{nat}$
shows $\forall A \in \text{FinPow}(X). \forall B \in \text{FinPow}(Y).$
 $A \approx n \wedge B \approx n \longrightarrow \text{ord_iso}(A,r,B,R) \neq 0$
<proof>

Every two finite linearly ordered sets are order isomorphic.

lemma fin_ord_iso_ex:
assumes A1: IsLinOrder(X,r) IsLinOrder(Y,R) **and**
A2: $A \in \text{FinPow}(X)$ $B \in \text{FinPow}(Y)$ **and** A3: $B \approx A$
shows $\text{ord_iso}(A,r,B,R) \neq 0$
<proof>

Existence and uniqueness of order isomorphism for two linearly ordered sets with the same number of elements.

theorem fin_ord_iso_ex_uniq:
assumes A1: IsLinOrder(X,r) IsLinOrder(Y,R) **and**
A2: $A \in \text{FinPow}(X)$ $B \in \text{FinPow}(Y)$ **and** A3: $B \approx A$
shows $\exists! f. f \in \text{ord_iso}(A,r,B,R)$
<proof>

end

17 Cardinal numbers

theory Cardinal_ZF **imports** ZF.CardinalArith Finite_ZF func1

begin

This theory file deals with results on cardinal numbers (cardinals). Cardinals are a generalization of the natural numbers, used to measure the cardinality (size) of sets. Contributed by Daniel de la Concepcion.

17.1 Some new ideas on cardinals

All the results of this section are done without assuming the Axiom of Choice. With the Axiom of Choice in play, the proofs become easier and some of the assumptions may be dropped.

Since General Topology Theory is closely related to Set Theory, it is very interesting to make use of all the possibilities of Set Theory to try to classify homeomorphic topological spaces. These ideas are generally used to prove that two topological spaces are not homeomorphic.

There exist cardinals which are the successor of another cardinal, but; as happens with ordinals, there are cardinals which are limit cardinal.

definition

$$\text{LimitC}(i) \equiv \text{Card}(i) \wedge 0 < i \wedge (\forall y. (y < i \wedge \text{Card}(y)) \longrightarrow \text{csucc}(y) < i)$$

Simple fact used a couple of times in proofs.

lemma nat_less_infty: **assumes** $n \in \text{nat}$ **and** $\text{InfCard}(X)$ **shows** $n < X$
<proof>

There are three types of cardinals, the zero one, the successors of other cardinals and the limit cardinals.

lemma Card_cases_disj:
 assumes $\text{Card}(i)$
 shows $i = 0 \mid (\exists j. \text{Card}(j) \wedge i = \text{csucc}(j)) \mid \text{LimitC}(i)$
<proof>

Given an ordinal bounded by a cardinal in ordinal order, we can change to the order of sets.

lemma le_imp_lesspoll:
 assumes $\text{Card}(Q)$
 shows $A \leq Q \implies A \lesssim Q$
<proof>

There are two types of infinite cardinals, the natural numbers and those that have at least one infinite strictly smaller cardinal.

lemma InfCard_cases_disj:

```

    assumes InfCard(Q)
    shows Q=nat  $\vee$  ( $\exists j. \text{csucc}(j) \lesssim Q \wedge \text{InfCard}(j)$ )
  <proof>

```

A more readable version of standard Isabelle/ZF `Ord_linear_lt`

```

lemma Ord_linear_lt_IML: assumes Ord(i) Ord(j)
  shows  $i < j \vee i = j \vee j < i$ 
  <proof>

```

A set is injective and not bijective to the successor of a cardinal if and only if it is injective and possibly bijective to the cardinal.

```

lemma Card_less_csucc_eq_le:
  assumes Card(m)
  shows  $A \prec \text{csucc}(m) \longleftrightarrow A \lesssim m$ 
  <proof>

```

If the successor of a cardinal is infinite, so is the original cardinal.

```

lemma csucc_inf_imp_inf:
  assumes Card(j) and InfCard(csucc(j))
  shows InfCard(j)
  <proof>

```

Since all the cardinals previous to `nat` are finite, it cannot be a successor cardinal; hence it is a `LimitC` cardinal.

```

corollary LimitC_nat:
  shows LimitC(nat)
  <proof>

```

17.2 Main result on cardinals (without the Axiom of Choice)

If two sets are strictly injective to an infinite cardinal, then so is its union. For the case of successor cardinal, this theorem is done in the `isabelle` library in a more general setting; but that theorem is of not use in the case where `LimitC(Q)` and it also makes use of the Axiom of Choice. The mentioned theorem is in the theory file `Cardinal_AC.thy`

Note that if Q is finite and different from 1, let's assume $Q = n$, then the union of A and B is not bounded by Q . Counterexample: two disjoint sets of $n - 1$ elements each have a union of $2n - 2$ elements which are more than n .

Note also that if $Q = 1$ then A and B must be empty and the union is then empty too; and Q cannot be 0 because no set is injective and not bijective to 0.

The proof is divided in two parts, first the case when both sets A and B are finite; and second, the part when at least one of them is infinite. In the first part, it is used the fact that a finite union of finite sets is finite. In the

second part it is used the linear order on cardinals (ordinals). This proof can not be generalized to a setting with an infinite union easily.

lemma `less_less_imp_un_less`:
 assumes $A \prec Q$ and $B \prec Q$ and $\text{InfCard}(Q)$
 shows $A \cup B \prec Q$
<proof>

17.3 Choice axioms

We want to prove some theorems assuming that some version of the Axiom of Choice holds. To avoid introducing it as an axiom we will define an appropriate predicate and put that in the assumptions of the theorems. That way technically we stay inside ZF.

The first predicate we define states that the axiom of Q -choice holds for subsets of K if we can find a choice function for every family of subsets of K whose (that family's) cardinality does not exceed Q .

definition
`AxiomCardinalChoice` ($\{\text{the axiom of}\}_{\text{choice holds for subsets}}_{_}$) **where**
 $\{\text{the axiom of}\} Q \{\text{choice holds for subsets}\} K \equiv \text{Card}(Q) \wedge (\forall M N. (M \lesssim Q \wedge (\forall t \in M. N_t \neq \emptyset \wedge N_t \subseteq K)) \longrightarrow (\exists f. f: \text{Pi}(M, \lambda t. N_t) \wedge (\forall t \in M. f t \in N_t)))$

Next we define a general form of Q choice where we don't require a collection of sets to be included in a set.

definition
`AxiomCardinalChoiceGen` ($\{\text{the axiom of}\}_{\text{choice holds}}_{_}$) **where**
 $\{\text{the axiom of}\} Q \{\text{choice holds}\} \equiv \text{Card}(Q) \wedge (\forall M N. (M \lesssim Q \wedge (\forall t \in M. N_t \neq \emptyset)) \longrightarrow (\exists f. f: \text{Pi}(M, \lambda t. N_t) \wedge (\forall t \in M. f t \in N_t)))$

The axiom of choice holds if and only if the `AxiomCardinalChoice` holds for every couple of a cardinal Q and a set K .

lemma `choice_subset_imp_choice`:
 shows $\{\text{the axiom of}\} Q \{\text{choice holds}\} \longleftrightarrow (\forall K. \{\text{the axiom of}\} Q \{\text{choice holds for subsets}\} K)$
<proof>

A choice axiom for greater cardinality implies one for smaller cardinality

lemma `greater_choice_imp_smaller_choice`:
 assumes $Q \lesssim Q_1$ $\text{Card}(Q)$
 shows $\{\text{the axiom of}\} Q_1 \{\text{choice holds}\} \longrightarrow (\{\text{the axiom of}\} Q \{\text{choice holds}\})$ *<proof>*

If we have a surjective function from a set which is injective to a set of ordinals, then we can find an injection which goes the other way.

lemma `surj_fun_inv`:
 assumes $f \in \text{surj}(A, B)$ $A \subseteq Q$ $\text{Ord}(Q)$

shows $B \lesssim A$
 $\langle proof \rangle$

The difference with the previous result is that in this one A is not a subset of an ordinal, it is only injective with one.

theorem `surj_fun_inv_2`:
assumes $f : \text{surj}(A, B)$ $A \lesssim Q$ $\text{Ord}(Q)$
shows $B \lesssim A$
 $\langle proof \rangle$

17.4 Finite choice

In ZF every finite collection of non-empty sets has a choice function, i.e. a function that selects one element from each set of the collection. In this section we prove various forms of that claim.

The axiom of finite choice always holds.

theorem `finite_choice`:
assumes $n \in \text{nat}$
shows $\{\text{the axiom of}\} n \{\text{choice holds}\}$
 $\langle proof \rangle$

The choice functions of a collection $\mathcal{A}$ are functions f defined on $\mathcal{A}$ and valued in $\bigcup \mathcal{A}$ such that $f(A) \in A$ for every $A \in \mathcal{A}$.

definition

$$\text{ChoiceFunctions}(\mathcal{A}) \equiv \{f \in \mathcal{A} \rightarrow \bigcup \mathcal{A} \mid \forall A \in \mathcal{A}. f(A) \in A\}$$

For finite collections of non-empty sets the set of choice functions is non-empty.

theorem `finite_choice1`: **assumes** $\text{Finite}(\mathcal{A})$ **and** $\forall A \in \mathcal{A}. A \neq \emptyset$
shows $\text{ChoiceFunctions}(\mathcal{A}) \neq \emptyset$
 $\langle proof \rangle$

If a set X is finite and such that for every $x \in X$ we can find $y \in Y$ such that the property $P(x, y)$ holds, then there is a function $f : X \rightarrow Y$ such that $P(x, f(x))$ holds for every $x \in X$.

lemma `finite_choice_fun`: **assumes** $\text{Finite}(X)$ $\forall x \in X. \exists y \in Y. P(x, y)$
shows $\exists f \in X \rightarrow Y. \forall x \in X. P(x, f(x))$
 $\langle proof \rangle$

17.5 Pigeonhole principle

In this section we present various versions of the pigeonhole principle. Simplifying somewhat the pigeonhole principle states that if you have more items than containers, at least one container must hold more than one item. This obvious observation is a surprisingly effective proof technique.

If a function maps a set with larger cardinality to a set with smaller cardinality then we can find two different elements of the domain that map to the same value.

lemma pigeonhole: **assumes** $f:X \rightarrow Y$ $Y < X$
shows $\exists x_1 \in X. \exists x_2 \in X. x_1 \neq x_2 \wedge f(x_1) = f(x_2)$
<proof>

In a list that is longer than its codomain we can find indices $i < j$ where the the list elements repeat.

lemma pigeonhole_list: **assumes** $n \in \text{nat}$ $X < n$ $b:n \rightarrow X$
shows $\exists i \in n. \exists j \in n. i < j \wedge b(i) = b(j)$
<proof>

17.6 Countability vs enumerability

There are different definitions of the notion of countability floating around. In the presence of the Axiom of Choice they are typically equivalent, but without AC they are often not. In this section we define the notions to have a reference what exactly we mean by saying that " X is enumerable" or " X is countable".

We say that a set X is enumerable if it is empty or there is a surjection from the set of natural numbers onto X . We (somewhat arbitrarily) want the empty set to be enumerable, but there are no functions whose domain is the set of natural numbers (or any nonempty set) and whose codomain is empty, so there are no surjections from natural numbers to $\emptyset$. Hence we have to add the empty set as a special case.

definition IsEnumerable ($_ \{is\ enumerable\}$ [90] 91) **where**
 $X \{is\ enumerable\} \equiv X = \emptyset \vee \text{surj}(\text{nat}, X) \neq \emptyset$

A set X is enumerable iff there exist a sequence that includes all elements of X .

lemma enumerable_iff_seq:
shows $X \{is\ enumerable\} \longleftrightarrow (\exists Y. \exists s \in \text{nat} \rightarrow Y. X \subseteq s(\text{nat}))$
<proof>

A set X is countable if there is an injection from X to the natural numbers.

definition IsCountable ($_ \{is\ countable\}$ [90] 91) **where**
 $X \{is\ countable\} \equiv \text{inj}(X, \text{nat}) \neq \emptyset$

If a set is countable then it is enumerable. In ZF the opposite implication cannot be proven.

lemma countable_enumerable:
assumes $X \{is\ countable\}$ **shows** $X \{is\ enumerable\}$
<proof>

end

18 Finite choice and order relations

theory FinOrd_ZF_1 **imports** FinOrd_ZF Cardinal_ZF

begin

In this theory we continue the subject of finite sets and order relation from FinOrd_ZF with some consequences of finite choice for down-directed sets.

18.1 Finte choice and preorders

In the `Order_ZF` theory we define what it means that a relation r down-directs a set X : each two elements of X have a common lower bound in X . If the relation is a preorder (i.e. is reflexive and transitive) and it down-directs X we say that X is a down-directed set (by the relation r).

The next lemma states that each finite subset of a down-directed set has a lower bound in X .

lemma fin_dir_set_bounded:
 assumes IsDownDirectedSet(X, r) **and** $B \in \text{FinPow}(X)$
 shows $\exists x \in X. \forall t \in B. \langle x, t \rangle \in r$
 $\langle \text{proof} \rangle$

Suppose Y is a set down-directed by a (preorder) relation r and f, g are funtions defined on two finite subsets A, B , resp., of X , valued in Y (i.e. $f : A \rightarrow Y$, $f : B \rightarrow Y$ and A, B are finite subsets of X). Then there exist a function $h : A \cup B \rightarrow Y$ that is a lower bound for f on A and for g on B .

lemma two_fun_low_bound:
 assumes IsDownDirectedSet(Y, r) $A \in \text{FinPow}(X)$ $B \in \text{FinPow}(X)$ $f : A \rightarrow Y$ $g : B \rightarrow Y$
 shows $\exists h \in A \cup B \rightarrow Y. (\forall x \in A. \langle h(x), f(x) \rangle \in r) \wedge (\forall x \in B. \langle h(x), g(x) \rangle \in r)$
 $\langle \text{proof} \rangle$

end

19 Equivalence relations

theory EquivClass1 **imports** ZF.EquivClass func_ZF ZF1

begin

In this theory file we extend the work on equivalence relations done in the standard Isabelle's EquivClass theory. That development is very good and all, but we really would prefer an approach contained within the a standard ZF set theory, without extensions specific to Isabelle. That is why this theory is written.

19.1 Congruent functions and projections on the quotient

Suppose we have a set X with a relation $r \subseteq X \times X$ and a function $f : X \rightarrow X$. The function f can be compatible (congruent) with r in the sense that if two elements x, y are related then the values $f(x), f(y)$ are also related. This is especially useful if r is an equivalence relation as it allows to "project" the function to the quotient space X/r (the set of equivalence classes of r) and create a new function F that satisfies the formula $F([x]_r) = [f(x)]_r$. When f is congruent with respect to r such definition of the value of F on the equivalence class $[x]_r$ does not depend on which x we choose to represent the class. In this section we also consider binary operations that are congruent with respect to a relation. These are important in algebra - the congruency condition allows to project the operation to obtain the operation on the quotient space.

First we define the notion of function that maps equivalent elements to equivalent values. We use similar names as in the Isabelle's standard `EquivClass` theory to indicate the conceptual correspondence of the notions.

definition

$$\text{Congruent}(\mathbf{r}, \mathbf{f}) \equiv \\ (\forall x \ y. \langle x, y \rangle \in \mathbf{r} \longrightarrow \langle \mathbf{f}(x), \mathbf{f}(y) \rangle \in \mathbf{r})$$

Now we will define the projection of a function onto the quotient space. In standard math the equivalence class of x with respect to relation r is usually denoted $[x]_r$. Here we reuse notation $r\{x\}$ instead. This means the image of the set $\{x\}$ with respect to the relation, which, for equivalence relations is exactly its equivalence class if you think about it.

definition

$$\text{ProjFun}(\mathbf{A}, \mathbf{r}, \mathbf{f}) \equiv \\ \{\langle c, \bigcup_{x \in c} r\{f(x)\} \rangle. c \in (\mathbf{A}/r)\}$$

Elements of equivalence classes belong to the set.

lemma `EquivClass_1_L1`:

assumes A1: `equiv(A,r)` **and** A2: `C ∈ A//r` **and** A3: `x∈C`
shows `x∈A`
<proof>

The image of a subset of X under projection is a subset of A/r .

lemma `EquivClass_1_L1A`:

assumes `A⊆X` **shows** `{r{x}. x∈A} ⊆ X//r`
<proof>

If an element belongs to an equivalence class, then its image under relation is this equivalence class.

lemma `EquivClass_1_L2`:

assumes A1: `equiv(A,r)` **and** A2: `x∈C`

shows $r\{x\} = C$
<proof>

Elements that belong to the same equivalence class are equivalent.

lemma `EquivClass_1_L2A`:
assumes `equiv(A,r)` $C \in A//r$ $x \in C$ $y \in C$
shows $\langle x, y \rangle \in r$
<proof>

Elements that have the same image under an equivalence relation are equivalent. This is the same as `eq_equiv_class` from standard Isabelle/ZF's `EquivClass` theory, just copied here to be easier to find.

lemma `same_image_equiv`:
assumes `equiv(A,r)` $y \in A$ $r\{x\} = r\{y\}$
shows $\langle x, y \rangle \in r$ *<proof>*

Every x is in the class of y , then they are equivalent.

lemma `EquivClass_1_L2B`:
assumes $A1: \text{equiv}(A,r)$ **and** $A2: y \in A$ **and** $A3: x \in r\{y\}$
shows $\langle x, y \rangle \in r$
<proof>

If a function is congruent then the equivalence classes of the values that come from the arguments from the same class are the same.

lemma `EquivClass_1_L3`:
assumes $A1: \text{equiv}(A,r)$ **and** $A2: \text{Congruent}(r,f)$
and $A3: C \in A//r$ $x \in C$ $y \in C$
shows $r\{f(x)\} = r\{f(y)\}$
<proof>

The values of congruent functions are in the space.

lemma `EquivClass_1_L4`:
assumes $A1: \text{equiv}(A,r)$ **and** $A2: C \in A//r$ $x \in C$
and $A3: \text{Congruent}(r,f)$
shows $f(x) \in A$
<proof>

Equivalence classes are not empty.

lemma `EquivClass_1_L5`:
assumes $A1: \text{refl}(A,r)$ **and** $A2: C \in A//r$
shows $C \neq \emptyset$
<proof>

To avoid using an axiom of choice, we define the projection using the expression $\bigcup_{x \in C} r(\{f(x)\})$. The next lemma shows that for congruent function this is in the quotient space A/r .

lemma `EquivClass_1_L6`:

```

    assumes A1: equiv(A,r) and A2: Congruent(r,f)
    and A3: C ∈ A//r
    shows (⋃x∈C. r{f(x)}) ∈ A//r
  <proof>

```

Congruent functions can be projected.

```

lemma EquivClass_1_T0:
  assumes equiv(A,r) Congruent(r,f)
  shows ProjFun(A,r,f) : A//r → A//r
  <proof>

```

We now define congruent functions of two variables (binary funtions). The predicate `Congruent2` corresponds to `congruent2` in Isabelle's standard `EquivClass` theory, but uses ZF-functions rather than meta-functions.

definition

```

Congruent2(r,f) ≡
  (∀ x1 x2 y1 y2. ⟨x1,x2⟩ ∈ r ∧ ⟨y1,y2⟩ ∈ r →
    ⟨f⟨x1,y1⟩, f⟨x2,y2⟩⟩ ∈ r)

```

Next we define the notion of projecting a binary operation to the quotient space. This is a very important concept that allows to define quotient groups, among other things.

definition

```

ProjFun2(A,r,f) ≡
  {⟨p, ⋃ z ∈ fst(p) × snd(p). r{f(z)}⟩. p ∈ (A//r) × (A//r) }

```

The following lemma is a two-variables equivalent of `EquivClass_1_L3`.

```

lemma EquivClass_1_L7:
  assumes A1: equiv(A,r) and A2: Congruent2(r,f)
  and A3: C1 ∈ A//r C2 ∈ A//r
  and A4: z1 ∈ C1 × C2 z2 ∈ C1 × C2
  shows r{f(z1)} = r{f(z2)}
  <proof>

```

The values of congruent functions of two variables are in the space.

```

lemma EquivClass_1_L8:
  assumes A1: equiv(A,r) and A2: C1 ∈ A//r and A3: C2 ∈ A//r
  and A4: z ∈ C1 × C2 and A5: Congruent2(r,f)
  shows f(z) ∈ A
  <proof>

```

The values of congruent functions are in the space. Note that although this lemma is intended to be used with functions, we don't need to assume that f is a function.

```

lemma EquivClass_1_L8A:
  assumes A1: equiv(A,r) and A2: x∈A y∈A
  and A3: Congruent2(r,f)

```

shows $f\langle x, y \rangle \in A$
 $\langle proof \rangle$

The following lemma is a two-variables equivalent of `EquivClass_1_L6`.

lemma `EquivClass_1_L9`:
assumes `A1: equiv(A,r)` **and** `A2: Congruent2(r,f)`
and `A3: p ∈ (A//r) × (A//r)`
shows $(\bigcup z \in \text{fst}(p) \times \text{snd}(p). r\{f(z)\}) \in A//r$
 $\langle proof \rangle$

Congruent functions of two variables can be projected.

theorem `EquivClass_1_T1`:
assumes `equiv(A,r)` `Congruent2(r,f)`
shows `ProjFun2(A,r,f) : (A//r) × (A//r) → A//r`
 $\langle proof \rangle$

The projection diagram commutes. I wish I knew how to draw this diagram in LaTeX.

lemma `EquivClass_1_L10`:
assumes `A1: equiv(A,r)` **and** `A2: Congruent2(r,f)`
and `A3: x ∈ A y ∈ A`
shows `ProjFun2(A,r,f)⟨r{x},r{y}⟩ = r{f⟨x,y⟩}`
 $\langle proof \rangle$

19.2 Projecting commutative, associative and distributive operations.

In this section we show that if the operations are congruent with respect to an equivalence relation then the projection to the quotient space preserves commutativity, associativity and distributivity.

The projection of commutative operation is commutative.

lemma `EquivClass_2_L1`: **assumes**
`A1: equiv(A,r)` **and** `A2: Congruent2(r,f)`
and `A3: f {is commutative on} A`
and `A4: c1 ∈ A//r c2 ∈ A//r`
shows `ProjFun2(A,r,f)⟨c1,c2⟩ = ProjFun2(A,r,f)⟨c2,c1⟩`
 $\langle proof \rangle$

The projection of commutative operation is commutative.

theorem `EquivClass_2_T1`:
assumes `equiv(A,r)` **and** `Congruent2(r,f)`
and `f {is commutative on} A`
shows `ProjFun2(A,r,f) {is commutative on} A//r`
 $\langle proof \rangle$

The projection of an associative operation is associative.

lemma EquivClass_2_L2:
 assumes A1: equiv(A,r) and A2: Congruent2(r,f)
 and A3: f {is associative on} A
 and A4: $c1 \in A//r$ $c2 \in A//r$ $c3 \in A//r$
 and A5: $g = \text{ProjFun2}(A,r,f)$
 shows $g\langle g\langle c1,c2 \rangle, c3 \rangle = g\langle c1, g\langle c2, c3 \rangle \rangle$
<proof>

The projection of an associative operation is associative on the quotient.

theorem EquivClass_2_T2:
 assumes A1: equiv(A,r) and A2: Congruent2(r,f)
 and A3: f {is associative on} A
 shows $\text{ProjFun2}(A,r,f)$ {is associative on} $A//r$
<proof>

The essential condition to show that distributivity is preserved by projections to quotient spaces, provided both operations are congruent with respect to the equivalence relation.

lemma EquivClass_2_L3:
 assumes A1: IsDistributive(X,A,M)
 and A2: equiv(X,r)
 and A3: Congruent2(r,A) Congruent2(r,M)
 and A4: $a \in X//r$ $b \in X//r$ $c \in X//r$
 and A5: $A_p = \text{ProjFun2}(X,r,A)$ $M_p = \text{ProjFun2}(X,r,M)$
 shows $M_p\langle a, A_p\langle b, c \rangle \rangle = A_p\langle M_p\langle a, b \rangle, M_p\langle a, c \rangle \rangle \wedge$
 $M_p\langle A_p\langle b, c \rangle, a \rangle = A_p\langle M_p\langle b, a \rangle, M_p\langle c, a \rangle \rangle$
<proof>

Distributivity is preserved by projections to quotient spaces, provided both operations are congruent with respect to the equivalence relation.

lemma EquivClass_2_L4: assumes A1: IsDistributive(X,A,M)
 and A2: equiv(X,r)
 and A3: Congruent2(r,A) Congruent2(r,M)
 shows IsDistributive($X//r$, $\text{ProjFun2}(X,r,A)$, $\text{ProjFun2}(X,r,M)$)
<proof>

19.3 Saturated sets

In this section we consider sets that are saturated with respect to an equivalence relation. A set A is saturated with respect to a relation r if $A = r^{-1}(r(A))$. For equivalence relations saturated sets are unions of equivalence classes. This makes them useful as a tool to define subsets of the quotient space using properties of representants. Namely, we often define a set $B \subseteq X/r$ by saying that $[x]_r \in B$ iff $x \in A$. If A is a saturated set, this definition is consistent in the sense that it does not depend on the choice of x to represent $[x]_r$.

The following defines the notion of a saturated set. Recall that in Isabelle $r^{-1}(A)$ is the inverse image of A with respect to relation r . This definition is not specific to equivalence relations.

definition

$\text{IsSaturated}(r,A) \equiv A = r^{-1}(r(A))$

For equivalence relations a set is saturated iff it is an image of itself.

lemma `EquivClass_3_L1`: **assumes** $A1: \text{equiv}(X,r)$

shows $\text{IsSaturated}(r,A) \longleftrightarrow A = r^{-1}(r(A))$

<proof>

For equivalence relations sets are contained in their images.

lemma `EquivClass_3_L2`: **assumes** $A1: \text{equiv}(X,r)$ **and** $A2: A \subseteq X$

shows $A \subseteq r^{-1}(r(A))$

<proof>

The next lemma shows that if " $\sim$ " is an equivalence relation and a set A is such that $a \in A$ and $a \sim b$ implies $b \in A$, then A is saturated with respect to the relation.

lemma `EquivClass_3_L3`: **assumes** $A1: \text{equiv}(X,r)$

and $A2: r \subseteq X \times X$ **and** $A3: A \subseteq X$

and $A4: \forall x \in A. \forall y \in X. \langle x,y \rangle \in r \longrightarrow y \in A$

shows $\text{IsSaturated}(r,A)$

<proof>

If $A \subseteq X$ and A is saturated and $x \sim y$, then $x \in A$ iff $y \in A$. Here we show only one direction.

lemma `EquivClass_3_L4`: **assumes** $A1: \text{equiv}(X,r)$

and $A2: \text{IsSaturated}(r,A)$ **and** $A3: A \subseteq X$

and $A4: \langle x,y \rangle \in r$

and $A5: x \in X \quad y \in A$

shows $x \in A$

<proof>

If $A \subseteq X$ and A is saturated and $x \sim y$, then $x \in A$ iff $y \in A$.

lemma `EquivClass_3_L5`: **assumes** $A1: \text{equiv}(X,r)$

and $A2: \text{IsSaturated}(r,A)$ **and** $A3: A \subseteq X$

and $A4: x \in X \quad y \in X$

and $A5: \langle x,y \rangle \in r$

shows $x \in A \longleftrightarrow y \in A$

<proof>

If A is saturated then $x \in A$ iff its class is in the projection of A .

lemma `EquivClass_3_L6`: **assumes** $A1: \text{equiv}(X,r)$

and $A2: \text{IsSaturated}(r,A)$ **and** $A3: A \subseteq X$ **and** $A4: x \in X$

and $A5: B = \{r\{x\}. x \in A\}$

shows $x \in A \longleftrightarrow r\{x\} \in B$

<proof>

A technical lemma involving a projection of a saturated set and a logical expression with exclusive or. Note that we don't really care what `Xor` is here, this is true for any predicate.

```
lemma EquivClass_3_L7: assumes equiv(X,r)
  and IsSaturated(r,A) and A⊆X
  and x∈X y∈X
  and B = {r{x}. x∈A}
  and (x∈A) Xor (y∈A)
  shows (r{x} ∈ B) Xor (r{y} ∈ B)
<proof>
```

end

20 Finite sequences

```
theory FiniteSeq_ZF imports Nat_ZF_IML func1
```

```
begin
```

This theory treats finite sequences (i.e. maps $n \rightarrow X$, where $n = \{0, 1, \dots, n-1\}$ is a natural number) as lists. It defines and proves the properties of basic operations on lists: concatenation, appending and element etc.

20.1 Lists as finite sequences

A natural way of representing (finite) lists in set theory is through (finite) sequences. In such view a list of elements of a set X is a function that maps the set $\{0, 1, \dots, n-1\}$ into X . Since natural numbers in set theory are defined so that $n = \{0, 1, \dots, n-1\}$, a list of length n can be understood as an element of the function space $n \rightarrow X$.

We define the set of lists with values in set X as `Lists(X)`.

definition

$$\text{Lists}(X) \equiv \bigcup_{n \in \text{nat.}} (n \rightarrow X)$$

The set of nonempty X -value listst will be called `NELists(X)`.

definition

$$\text{NELists}(X) \equiv \bigcup_{n \in \text{nat.}} (\text{succ}(n) \rightarrow X)$$

We first define the shift that moves the second sequence to the domain $\{n, \dots, n+k-1\}$, where n, k are the lengths of the first and the second sequence, resp. To understand the notation in the definitions below recall that in Isabelle/ZF `pred(n)` (predecessor of n is the previous natural number.

definition

$$\text{ShiftedSeq}(b, n) \equiv \{ \langle j, b(j \#- n) \rangle . j \in \text{NatInterval}(n, \text{domain}(b)) \}$$

We define concatenation of two sequences as the union of the first sequence with the shifted second sequence. The result of concatenating lists a and b is called $\text{Concat}(a, b)$.

definition

$$\text{Concat}(a, b) \equiv a \cup \text{ShiftedSeq}(b, \text{domain}(a))$$

For a finite sequence we define the sequence of all elements except the first one. This corresponds to the "tail" function in Haskell. We call it Tail here as well.

definition

$$\text{Tail}(a) \equiv \{ \langle k, a(\text{succ}(k)) \rangle . k \in \text{pred}(\text{domain}(a)) \}$$

A dual notion to Tail is the list of all elements of a list except the last one. Borrowing the terminology from Haskell again, we will call this Init .

definition

$$\text{Init}(a) \equiv \text{restrict}(a, \text{pred}(\text{domain}(a)))$$

Another obvious operation we can talk about is appending an element at the end of a sequence. This is called Append .

definition

$$\text{Append}(a, x) \equiv a \cup \{ \langle \text{domain}(a), x \rangle \}$$

If lists are modeled as finite sequences (i.e. functions on natural intervals $\{0, 1, \dots, n-1\} = n$) it is easy to get the first element of a list as the value of the sequence at 0. The last element is the value at $n-1$. To hide this behind a familiar name we define the Last element of a list.

definition

$$\text{Last}(a) \equiv a(\text{pred}(\text{domain}(a)))$$

A formula for tail of a finite list.

lemma tail_as_set : **assumes** $n \in \text{nat}$ **and** $a: n \#+ 1 \rightarrow X$
shows $\text{Tail}(a) = \{ \langle k, a(k \#+ 1) \rangle . k \in n \}$
<proof>

Formula for the tail of a list defined by an expression:

lemma tail_formula : **assumes** $n \in \text{nat}$ **and** $\forall k \in n \#+ 1. q(k) \in X$
shows $\text{Tail}(\{ \langle k, q(k) \rangle . k \in n \#+ 1 \}) = \{ \langle k, q(k \#+ 1) \rangle . k \in n \}$
<proof>

Codomain of a nonempty list is nonempty.

lemma $\text{nelist_vals_nonempty}$: **assumes** $a: \text{succ}(n) \rightarrow Y$
shows $Y \neq 0$ *<proof>*

Shifted sequence is a function on a the interval of natural numbers.

lemma shifted_seq_props:
 assumes A1: $n \in \text{nat}$ $k \in \text{nat}$ and A2: $b:k \rightarrow X$
 shows
 $\text{ShiftedSeq}(b,n): \text{NatInterval}(n,k) \rightarrow X$
 $\forall i \in \text{NatInterval}(n,k). \text{ShiftedSeq}(b,n)(i) = b(i \#- n)$
 $\forall j \in k. \text{ShiftedSeq}(b,n)(n \#+ j) = b(j)$
<proof>

A list shifted by a 0 is the same list.

lemma seq_shifted_by_zero: assumes $n \in \text{nat}$ $b:n \rightarrow X$
 shows $\text{ShiftedSeq}(b,0) = b$
<proof>

A shifted empty list is still empty.

lemma empty_shifted_empty: shows $\text{ShiftedSeq}(\emptyset, n) = \emptyset$
<proof>

Basis properties of the concatenation of two finite sequences.

theorem concat_props:
 assumes A1: $n \in \text{nat}$ $k \in \text{nat}$ and A2: $a:n \rightarrow X$ $b:k \rightarrow X$
 shows
 $\text{Concat}(a,b): n \#+ k \rightarrow X$
 $\forall i \in n. \text{Concat}(a,b)(i) = a(i)$
 $\forall i \in \text{NatInterval}(n,k). \text{Concat}(a,b)(i) = b(i \#- n)$
 $\forall j \in k. \text{Concat}(a,b)(n \#+ j) = b(j)$
<proof>

Properties of concatenating three lists.

lemma concat_concat_list:
 assumes A1: $n \in \text{nat}$ $k \in \text{nat}$ $m \in \text{nat}$ and
 A2: $a:n \rightarrow X$ $b:k \rightarrow X$ $c:m \rightarrow X$ and
 A3: $d = \text{Concat}(\text{Concat}(a,b), c)$
 shows
 $d : n \#+ k \#+ m \rightarrow X$
 $\forall j \in n. d(j) = a(j)$
 $\forall j \in k. d(n \#+ j) = b(j)$
 $\forall j \in m. d(n \#+ k \#+ j) = c(j)$
<proof>

Properties of concatenating a list with a concatenation of two other lists.

lemma concat_list_concat:
 assumes A1: $n \in \text{nat}$ $k \in \text{nat}$ $m \in \text{nat}$ and
 A2: $a:n \rightarrow X$ $b:k \rightarrow X$ $c:m \rightarrow X$ and
 A3: $e = \text{Concat}(a, \text{Concat}(b,c))$
 shows
 $e : n \#+ k \#+ m \rightarrow X$
 $\forall j \in n. e(j) = a(j)$
 $\forall j \in k. e(n \#+ j) = b(j)$

$\forall j \in m. e(n \# + k \# + j) = c(j)$
<proof>

Concatenation is associative.

theorem concat_assoc:
 assumes A1: $n \in \text{nat}$ $k \in \text{nat}$ $m \in \text{nat}$ and
 A2: $a: n \rightarrow X$ $b: k \rightarrow X$ $c: m \rightarrow X$
 shows $\text{Concat}(\text{Concat}(a, b), c) = \text{Concat}(a, \text{Concat}(b, c))$
<proof>

Properties of Tail.

theorem tail_props:
 assumes A1: $n \in \text{nat}$ and A2: $a: \text{succ}(n) \rightarrow X$
 shows
 $\text{Tail}(a) : n \rightarrow X$
 $\forall k \in n. \text{Tail}(a)(k) = a(\text{succ}(k))$
<proof>

Essentially the second assertion of tail_props but formulated using notation $n + 1$ instead of $\text{succ}(n)$:

lemma tail_props2: assumes $n \in \text{nat}$ $a: n \# + 1 \rightarrow X$ $k \in n$
 shows $\text{Tail}(a)(k) = a(k \# + 1)$
<proof>

A nonempty list can be decomposed into concatenation of its first element and the tail.

lemma first_concat_tail: assumes $n \in \text{nat}$ $a: \text{succ}(n) \rightarrow X$
 shows $a = \text{Concat}(\{0, a(0)\}, \text{Tail}(a))$
<proof>

Properties of Append. It is a bit surprising that the we don't need to assume that n is a natural number.

theorem append_props:
 assumes A1: $a: n \rightarrow X$ and A2: $x \in X$ and A3: $b = \text{Append}(a, x)$
 shows
 $b : \text{succ}(n) \rightarrow X$
 $\forall k \in n. b(k) = a(k)$
 $b(n) = x$
<proof>

A special case of append_props: appending to a nonempty list does not change the head (first element) of the list.

corollary head_of_append:
 assumes $n \in \text{nat}$ and $a: \text{succ}(n) \rightarrow X$ and $x \in X$
 shows $\text{Append}(a, x)(0) = a(0)$
<proof>

Tail commutes with Append.

theorem tail_append_commute:
 assumes A1: $n \in \text{nat}$ and A2: $a: \text{succ}(n) \rightarrow X$ and A3: $x \in X$
 shows $\text{Append}(\text{Tail}(a), x) = \text{Tail}(\text{Append}(a, x))$
<proof>

NELists are nonempty lists

lemma non_zero_List_func_is_NEList:
 shows $\text{NELists}(X) = \{a \in \text{Lists}(X). a \neq 0\}$
<proof>

Properties of Init.

theorem init_props:
 assumes A1: $n \in \text{nat}$ and A2: $a: \text{succ}(n) \rightarrow X$
 shows
 $\text{Init}(a) : n \rightarrow X$
 $\forall k \in n. \text{Init}(a)(k) = a(k)$
 $a = \text{Append}(\text{Init}(a), a(n))$
<proof>

The initial part of a nonempty list is a list, and the domain of the original list is the successor of its initial part.

theorem init_NEList:
 assumes $a \in \text{NELists}(X)$
 shows $\text{Init}(a) \in \text{Lists}(X)$ and $\text{succ}(\text{domain}(\text{Init}(a))) = \text{domain}(a)$
<proof>

If we take init of the result of append, we get back the same list.

lemma init_append: assumes A1: $n \in \text{nat}$ and A2: $a: n \rightarrow X$ and A3: $x \in X$
 shows $\text{Init}(\text{Append}(a, x)) = a$
<proof>

A reformulation of definition of Init.

lemma init_def: assumes $n \in \text{nat}$ and $a: \text{succ}(n) \rightarrow X$
 shows $\text{Init}(a) = \text{restrict}(a, n)$
<proof>

Another reformulation of the definition of Init, starting with the expression defining the list.

lemma init_def_alt: assumes $n \in \text{nat}$ and $\forall k \in n \# + 1. q(k) \in X$
 shows $\text{Init}(\{ \langle k, q(k) \rangle. k \in n \# + 1 \}) = \{ \langle k, q(k) \rangle. k \in n \}$
<proof>

A lemma about extending a finite sequence by one more value. This is just a more explicit version of append_props.

lemma finseq_extend:
 assumes $a: n \rightarrow X$ $y \in X$ $b = a \cup \{ \langle n, y \rangle \}$
 shows

```

b: succ(n) → X
∀k∈n. b(k) = a(k)
b(n) = y
⟨proof⟩

```

The next lemma is a bit displaced as it is mainly about finite sets. It is proven here because it uses the notion of **Append**. Suppose we have a list of element of A is a bijection. Then for every element that does not belong to A we can construct a bijection for the set $A \cup \{x\}$ by appending x . This is just a specialised version of lemma **bij_extend_point** from **func1.thy**.

```

lemma bij_append_point:
  assumes A1: n ∈ nat and A2: b ∈ bij(n,X) and A3: x ∉ X
  shows Append(b,x) ∈ bij(succ(n), X ∪ {x})
⟨proof⟩

```

The next lemma rephrases the definition of **Last**. Recall that in ZF we have $\{0, 1, 2, \dots, n\} = n + 1 = \text{succ}(n)$.

```

lemma last_seq_elem: assumes a: succ(n) → X shows Last(a) = a(n)
⟨proof⟩

```

The last element of a nonempty list valued in X is in X .

```

lemma last_type: assumes a ∈ NELists(X) shows Last(a) ∈ X
⟨proof⟩

```

The last element of a list of length at least 2 is the same as the last element of the tail of that list.

```

lemma last_tail_last: assumes n∈nat a: succ(succ(n)) → X
  shows Last(Tail(a)) = Last(a)
⟨proof⟩

```

If two finite sequences are the same when restricted to domain one shorter than the original and have the same value on the last element, then they are equal.

```

lemma finseq_restr_eq: assumes A1: n ∈ nat and
  A2: a: succ(n) → X b: succ(n) → X and
  A3: restrict(a,n) = restrict(b,n) and
  A4: a(n) = b(n)
  shows a = b
⟨proof⟩

```

Concatenating a list of length 1 is the same as appending its first (and only) element. Recall that in ZF set theory $1 = \{0\}$.

```

lemma append_1elem: assumes A1: n ∈ nat and
  A2: a: n → X and A3: b : 1 → X
  shows Concat(a,b) = Append(a,b(0))
⟨proof⟩

```

If $x \in X$ then the singleton set with the pair $\langle 0, x \rangle$ as the only element is a list of length 1 and hence a nonempty list.

lemma list_len1_singleton: **assumes** $x \in X$
shows $\{\langle 0, x \rangle\} : 1 \rightarrow X$ **and** $\{\langle 0, x \rangle\} \in \text{NELists}(X)$
<proof>

A singleton list is in fact a singleton set with a pair as the only element.

lemma list_singleton_pair: **assumes** $A1: x:1 \rightarrow X$ **shows** $x = \{\langle 0, x(0) \rangle\}$
<proof>

When we append an element to the empty list we get a list with length 1.

lemma empty_append1: **assumes** $A1: x \in X$
shows $\text{Append}(0, x) : 1 \rightarrow X$ **and** $\text{Append}(0, x)(0) = x$
<proof>

Concatenating with an empty list gives the same list. Recall that an empty list is in fact an empty set (see `fun_empty_empty` in `func1` theory).

lemma concat_empty: **assumes** $n \in \text{nat } x: n \rightarrow X$
shows $\text{Concat}(x, \emptyset) = x$ **and** $\text{Concat}(\emptyset, x) = x$
<proof>

Appending an element is the same as concatenating with certain pair.

lemma append_concat_pair:
assumes $n \in \text{nat}$ **and** $a: n \rightarrow X$ **and** $x \in X$
shows $\text{Append}(a, x) = \text{Concat}(a, \{\langle 0, x \rangle\})$
<proof>

An associativity property involving concatenation and appending. For proof we just convert appending to concatenation and use `concat_assoc`.

lemma concat_append_assoc: **assumes** $A1: n \in \text{nat } k \in \text{nat}$ **and**
 $A2: a: n \rightarrow X \quad b: k \rightarrow X$ **and** $A3: x \in X$
shows $\text{Append}(\text{Concat}(a, b), x) = \text{Concat}(a, \text{Append}(b, x))$
<proof>

An identity involving concatenating with `init` and appending the last element.

lemma concat_init_last_elem:
assumes $n \in \text{nat } k \in \text{nat}$ **and**
 $a: n \rightarrow X$ **and** $b: \text{succ}(k) \rightarrow X$
shows $\text{Append}(\text{Concat}(a, \text{Init}(b)), b(k)) = \text{Concat}(a, b)$
<proof>

A lemma about creating lists by composition and how `Append` behaves in such case.

lemma list_compose_append:
assumes $A1: n \in \text{nat}$ **and** $A2: a: n \rightarrow X$ **and**

A3: $x \in X$ and A4: $c : X \rightarrow Y$
shows
 $c \circ \text{Append}(a, x) : \text{succ}(n) \rightarrow Y$
 $c \circ \text{Append}(a, x) = \text{Append}(c \circ a, c(x))$
<proof>

Composition commutes with concatenation i.e. composition with a concatenation is concatenation of compositions.

lemma list_compose_concat:
assumes $n \in \text{nat}$ $m \in \text{nat}$ $a : n \rightarrow X$ $b : m \rightarrow X$ $c : X \rightarrow Y$
shows $c \circ \text{Concat}(a, b) = \text{Concat}(c \circ a, c \circ b)$
<proof>

A lemma about appending an element to a list defined by set comprehension.

lemma set_list_append: **assumes**
A1: $\forall i \in \text{succ}(k). b(i) \in X$ and
A2: $a = \{\langle i, b(i) \rangle. i \in \text{succ}(k)\}$
shows
 $a : \text{succ}(k) \rightarrow X$
 $\{\langle i, b(i) \rangle. i \in k\} : k \rightarrow X$
 $a = \text{Append}(\{\langle i, b(i) \rangle. i \in k\}, b(k))$
<proof>

A version of set_list_append using $n + 1$ instead of $\text{succ}(n)$.

lemma set_list_append1:
assumes $n \in \text{nat}$ and $\forall k \in n \# + 1. q(k) \in X$
defines $a \equiv \{\langle k, q(k) \rangle. k \in n \# + 1\}$
shows
 $a : n \# + 1 \rightarrow X$
 $\{\langle k, q(k) \rangle. k \in n\} : n \rightarrow X$
 $\text{Init}(a) = \{\langle k, q(k) \rangle. k \in n\}$
 $a = \text{Append}(\{\langle k, q(k) \rangle. k \in n\}, q(n))$
 $a = \text{Append}(\text{Init}(a), q(n))$
 $a = \text{Append}(\text{Init}(a), a(n))$
<proof>

An induction theorem for lists.

lemma list_induct: **assumes** A1: $\forall b \in 1 \rightarrow X. P(b)$ and
A2: $\forall b \in \text{NELists}(X). P(b) \longrightarrow (\forall x \in X. P(\text{Append}(b, x)))$ and
A3: $d \in \text{NELists}(X)$
shows $P(d)$
<proof>

A dual notion to Append is Prepend where we add an element to the list at the beginning of the list. We define the value of the list a prepended by an element x as x if index is 0 and $a(k - 1)$ otherwise.

definition

$\text{Prepend}(a, x) \equiv \{\langle k, \text{if } k = 0 \text{ then } x \text{ else } a(k \# - 1) \rangle. k \in \text{domain}(a) \# + 1\}$

If $a : n \rightarrow X$ is a list, then a with prepended $x \in X$ is a list as well and its first element is x .

lemma `prepend_props`:
assumes $n \in \text{nat}$ $a : n \rightarrow X$ $x \in X$
shows $\text{Prepend}(a, x) : (n \# + 1) \rightarrow X$ and $\text{Prepend}(a, x)(0) = x$
<proof>

When prepending an element to a list the values at positive indices do not change.

lemma `prepend_val`: **assumes** $n \in \text{nat}$ $a : n \rightarrow X$ $x \in X$ $k \in n$
shows $\text{Prepend}(a, x)(k \# + 1) = a(k)$
<proof>

The tail of a list prepended by an element is equal to the list.

lemma `tail_prepend`: **assumes** $n \in \text{nat}$ $a : n \rightarrow X$ $x \in X$
shows $\text{Tail}(\text{Prepend}(a, x)) = a$
<proof>

20.2 Lists and cartesian products

Lists of length n of elements of some set X can be thought of as a model of the cartesian product X^n which is more convenient in many applications.

There is a natural bijection between the space $(n + 1) \rightarrow X$ of lists of length $n + 1$ of elements of X and the cartesian product $(n \rightarrow X) \times X$.

lemma `lists_cart_prod`: **assumes** $n \in \text{nat}$
shows $\{\langle x, \langle \text{Init}(x), x(n) \rangle \rangle. x \in \text{succ}(n) \rightarrow X\} \in \text{bij}(\text{succ}(n) \rightarrow X, (n \rightarrow X) \times X)$
<proof>

We can identify a set X with lists of length one of elements of X .

lemma `singleton_list_bij`: **shows** $\{\langle x, x(0) \rangle. x \in 1 \rightarrow X\} \in \text{bij}(1 \rightarrow X, X)$
<proof>

We can identify a set of X -valued lists of length with X .

lemma `list_singleton_bij`: **shows**
 $\{\langle x, \{\langle 0, x \rangle\} \rangle. x \in X\} \in \text{bij}(X, 1 \rightarrow X)$ and
 $\{\langle y, y(0) \rangle. y \in 1 \rightarrow X\} = \text{converse}(\{\langle x, \{\langle 0, x \rangle\} \rangle. x \in X\})$ and
 $\{\langle x, \{\langle 0, x \rangle\} \rangle. x \in X\} = \text{converse}(\{\langle y, y(0) \rangle. y \in 1 \rightarrow X\})$
<proof>

What is the inverse image of a set by the natural bijection between X -valued singleton lists and X ?

lemma `singleton_vimage`: **assumes** $U \subseteq X$ **shows** $\{x \in 1 \rightarrow X. x(0) \in U\} = \{\{\langle 0, y \rangle\}. y \in U\}$
<proof>

A technical lemma about extending a list by values from a set.

lemma list_append_from: assumes A1: $n \in \text{nat}$ and A2: $U \subseteq n \rightarrow X$ and A3: $V \subseteq X$
shows
 $\{x \in \text{succ}(n) \rightarrow X. \text{Init}(x) \in U \wedge x(n) \in V\} = (\bigcup_{y \in V. \{\text{Append}(x, y). x \in U\}})$
<proof>

20.3 Chains

In this section we define chains and operations on them. Chains are essentially lists that are parametrized by the first and last element.

Chains of elements of X connecting x and y are lists (i.e. functions) $c : \{0, 1, \dots, n\} \rightarrow X$ for some $n \in \mathbb{N}$ such that the first element (i.e. $c(0)$) is x and the last (i.e. $c(n)$) element is y . In informal comments we will say that the chain has length n when it consists of $n + 1$ elements of X , i.e. it has n links, see the definition below.

definition Chains(X, n, x, y) $\equiv \{c \in (n \# + 1) \rightarrow X. c(0) = x \wedge c(n) = y\}$

If c is a chain in X of length n connecting x and y then it maps $n + 1$ (i.e. the set $\{0, 1, \dots, n\}$) to X , the first element is x , the n 'th (last) element is y and both x and y are elements of X .

lemma chain_props: assumes $n \in \text{nat}$ $c \in \text{Chains}(X, n, x, y)$
shows $c \in (n \# + 1) \rightarrow X$ $c(0) = x$ $c(n) = y$ $x \in X$ $y \in X$
<proof>

One operation that we can do on a chain $x = c_0, c_1, \dots, c_n = y$ is converting it to a list of pairs $\langle c_i, c_{i+1} \rangle, 0 \leq i < n$. I don't have a good name for this operation, let's call it ChainLinks for now.

definition ChainLinks(c) $\equiv \{\langle i, \langle c(i), c(i \# + 1) \rangle \rangle. i \in (\text{domain}(c) \# - 1)\}$

If c is a chain in X of length n then chain links of c form a list of elements of $X \times X$ of length n .

lemma chain_links_fun: assumes $n \in \text{nat}$ $c \in \text{Chains}(X, n, x, y)$
shows $\text{domain}(c) \# - 1 = n$ and $\text{ChainLinks}(c) : n \rightarrow X \times X$
<proof>

If c is a chain in X connecting x and y of length n and $k < n$ (i.e. $k \in n$) then the value of the k 'th element of the derived chain links is $\langle c(k), c(k+1) \rangle$.

lemma chain_links_val: assumes $n \in \text{nat}$ $c \in \text{Chains}(X, n, x, y)$ $k \in n$
shows $\text{ChainLinks}(c)(k) = \langle c(k), c(k \# + 1) \rangle$
<proof>

If c is a chain in X connecting x and y of a non-zero length n then the first component of the first element of the derived chain links is x and the second component of the last element of the derived chain links is equal to y .

lemma chain_intervals_ends:


```

assumes n∈nat n≠0 c∈Chains(X,n,x,y)
shows
  fst(ChainLinks(c)(0)) = x and
  snd(ChainLinks(c)(n #- 1)) = y
⟨proof⟩

```

If c is a chain in X connecting x and y of length n and k is a natural number such that $k + 1 \in n$ then the second component of the k 'th element of the derived chain intervals is the same as the first component of the $k + 1$ 'th element of the derived chain intervals.

```

lemma chain_links_connected:
  assumes n∈nat c∈Chains(X,n,x,y) k∈nat k #+ 1 ∈ n
  shows snd(ChainLinks(c)(k)) = fst(ChainLinks(c)(k #+ 1))
⟨proof⟩

```

If c_i are chains from x_i to y_i of natural length n_i for $i = 1, 2$ then concatenation c_3 of c_1 and c_2 is a chain from x_1 to y_1 of length $n_1 + n_2 + 1$. The chain links of the concatenation is a list of elements of $X \times X$ of length $n_1 + n_2 + 1$, obtained by appending the pair $\langle c_1(n_1), c_2(0) \rangle$ to the chain links of c_1 and then concatenating that with the chain links of c_2 . The proof is surprisingly long and tedious for such a trivial fact.

```

lemma concat_chains:
  assumes n1∈nat c1∈Chains(X,n1,x1,y1) n2∈nat c2∈Chains(X,n2,x2,y2)
  defines c3 ≡ Concat(c1,c2)
  shows
    c3 ∈ Chains(X,n1 #+ n2 #+ 1,x1,y2)
    ChainLinks(c3): (n1 #+ n2 #+ 1)→X×X
    ChainLinks(c3) = Concat(Append(ChainLinks(c1),⟨c1(n1),c2(0)⟩),ChainLinks(c2))
    ChainLinks(c3) = Concat(Append(ChainLinks(c1),⟨y1,x2⟩),ChainLinks(c2))
⟨proof⟩

```

end

21 Formal languages

```

theory Finite_State_Machines_ZF imports FiniteSeq_ZF Finite1 ZF.CardinalArith

```

```

begin

```

21.1 Introduction

This file deals with finite state machines. The goal is to define regular languages and show that they are closed by finite union, finite intersection, complements and concatenation.

We show that the languages defined by deterministic, non-deterministic and non-deterministic with ϵ moves are equivalent.

First, a transitive closure variation on $r^* = \text{id}(\text{field}(r)) \cup (r \circ r^*)$.

theorem `rtranc1_rev`:
 shows $r^* = \text{id}(\text{field}(r)) \cup (r^* \circ r)$
<proof>

A language is a subset of words.

definition
 IsALanguage ($_$ {is a language with alphabet} $_$) **where**
 $\text{Finite}(\Sigma) \implies L$ {is a language with alphabet} $\Sigma \equiv L \subseteq \text{Lists}(\Sigma)$

The set of all words, and the set of no words are languages.

lemma `full_empty_language`:
 assumes $\text{Finite}(\Sigma)$
 shows $\text{Lists}(\Sigma)$ {is a language with alphabet} Σ
 and $\emptyset$ {is a language with alphabet} Σ
<proof>

21.2 Deterministic Finite Automata

A deterministic finite state automaton is defined as a finite set of states, an initial state, a transition function from state to state based on the word and a set of final states.

definition
 DFSA ($_$ {is an DFSA for alphabet} $_$) **where**
 $\text{Finite}(\Sigma) \implies (S, s_0, t, F)$ {is an DFSA for alphabet} $\Sigma \equiv \text{Finite}(S) \wedge s_0 \in S \wedge F \subseteq S \wedge t: S \times \Sigma \rightarrow S$

A finite automaton defines transitions on pairs of words and states. Two pairs are transition related if the second word is equal to the first except it is missing the last symbol, and the second state is generated by this symbol and the first state by way of the transition function.

definition
 DFSAExecutionRelation ($\{\text{reduce D-relation}\}$ ($_$, $_$) {in alphabet} $_$) **where**
 $\text{Finite}(\Sigma) \implies (S, s_0, t, F)$ {is an DFSA for alphabet} $\Sigma \implies$
 $\{\text{reduce D-relation}\}(S, t)$ {in alphabet} $\Sigma \equiv \{ \langle \langle w, s \rangle, \langle \text{Init}(w), t(s, \text{Last}(w)) \rangle \rangle \mid \langle w, s \rangle \in \text{NELists}(\Sigma) \times S \}$

We define a word to be fully reducible by a finite state automaton if in the transitive closure of the previous relation it is related to the pair of the empty word and a final state.

Since the empty word with the initial state need not be in $\text{field}(\{\text{reduce D-relation}\}(S, t)$ {in alphabet} $\Sigma)$, we add the extra condition that $\langle \langle \emptyset, s_0 \rangle, \emptyset, s_0 \rangle$ is also a valid transition.

definition
 DFSASatisfy ($_$ $\leftarrow$ $_$ {is an DFSA for alphabet} $_$) **where**
 $\text{Finite}(\Sigma) \implies (S, s_0, t, F)$ {is an DFSA for alphabet} $\Sigma \implies i \in \text{Lists}(\Sigma) \implies$

$i \text{ <-D } (S, s_0, t, F) \{ \text{in alphabet} \} \Sigma \equiv (\exists q \in F. \langle \langle i, s_0 \rangle, \langle 0, q \rangle \rangle \in (\{ \text{reduce D-relation} \} (S, t) \{ \text{in alphabet} \} \Sigma)^*) \vee (i = 0 \wedge s_0 \in F)$

We define a locale for better notation

```

locale DetFinStateAuto =
  fixes S and s0 and t and F and Σ
  assumes finite_alphabet: Finite(Σ)

  assumes DFSA: (S, s0, t, F) {is an DFSA for alphabet} Σ

```

We abbreviate the reduce relation to a single symbol within this locale.

```

abbreviation (in DetFinStateAuto) rD where
  rD ≡ {reduce D-relation}(S, t) {in alphabet} Σ

```

We abbreviate the full reduction condition to a single symbol within this locale.

```

abbreviation (in DetFinStateAuto) reduce (_{reduces}) where
  i {reduces} ≡ i <-D (S, s0, t, F) {in alphabet} Σ

```

Destruction lemma about deterministic finite state automata.

```

lemma (in DetFinStateAuto) DFSA_dest:
  shows s0 ∈ S F ⊆ S t: S × Σ → S Finite(S) <proof>

```

The set of words that reduce to final states forms a language. This is by definition.

```

lemma (in DetFinStateAuto) DFSA_language:
  shows {i ∈ Lists(Σ). i <-D (S, s0, t, F) {in alphabet} Σ} {is a language with alphabet} Σ
  <proof>

```

Define this language as an abbreviation to reduce terms

```

abbreviation (in DetFinStateAuto) LanguageDFSA
  where LanguageDFSA ≡ {i ∈ Lists(Σ). i <-D (S, s0, t, F) {in alphabet} Σ}

```

The relation is an actual relation, but even more it is a function (hence the adjective deterministic).

```

lemma (in DetFinStateAuto) reduce_is_relation_function:
  shows relation(rD) function(rD) <proof>

```

The relation, that is actually a function has the following domain and range:

```

lemma (in DetFinStateAuto) reduce_function:
  shows rD: NELists(Σ) × S → Lists(Σ) × S
  <proof>

```

The field of the relation contains all pairs with non-empty words, but we cannot assume that it contains all pairs.

```

corollary (in DetFinStateAuto) reduce_field:

```

shows $\text{field}(r_D) \subseteq \text{Lists}(\Sigma) \times S \text{ NELists}(\Sigma) \times S \subseteq \text{field}(r_D)$
<proof>

If a word is a reduced version of an other, then it can be encoded as a restriction.

lemma (in DetFinStateAuto) seq_is_restriction:
 fixes $w \ s \ u \ v$
 assumes $\langle w, s \rangle, \langle u, v \rangle \in r_D^*$
 shows $\text{restrict}(w, \text{domain}(u)) = u$
<proof>

lemma (in DetFinStateAuto) relation_deteministic:
 assumes $\langle w, s \rangle, \langle u, v \rangle \in r_D^* \ \langle w, s \rangle, \langle u, m \rangle \in r_D^*$
 shows $v = m$
<proof>

Any non-empty word can be reduced to the empty string, but it does not always end in a final state.

lemma (in DetFinStateAuto) endpoint_exists:
 assumes $w \in \text{NELists}(\Sigma)$
 shows $\exists q \in S. \langle w, s_0 \rangle, \langle 0, q \rangle \in r_D^*$
<proof>

Example of Finite Automaton of binary lists starting with 0 and ending with 1

locale ListFrom0To1
begin

Empty state

definition empty **where**
 empty $\equiv 2$

The string starts with 0 state

definition ends0 **where**
 ends0 $\equiv \text{succ}(2)$

The string ends with 1 state

definition starts1 **where**
 starts1 $\equiv 1$

The string ends with 0 state

definition starts0 **where**
 starts0 $\equiv 0$

The states are the previous 4 states. They are encoded as natural numbers to make it easier to reason about them, and as human readable variable names to make it easier to understand.

definition `states` **where**
`states` $\equiv \{\text{empty}, \text{starts0}, \text{starts1}, \text{ends0}\}$

The final state is `starts0`

definition `finalStates` **where**
`finalStates` $\equiv \{\text{starts0}\}$

The transition function is defined as follows:

From the `empty` state, we transition to state `starts1` in case there is a 1 and to state `ends0` in case there is a 0.

From the state `ends0` we stay in it.

From the states `starts1` and `starts0` we transition to `starts0` in case there is a 0, and to `starts1` in case there is a 1.

definition `transFun` **where**
`transFun` $\equiv \{\langle\langle\text{empty}, 1\rangle, \text{starts1}\rangle, \langle\langle\text{empty}, 0\rangle, \text{ends0}\rangle\} \cup$
 $\{\langle\langle\text{ends0}, x\rangle, \text{ends0}\rangle. x \in 2\} \cup$
 $\{\langle\langle\text{starts1}, 0\rangle, \text{starts0}\rangle, \langle\langle\text{starts1}, 1\rangle, \text{starts1}\rangle,$
 $\langle\langle\text{starts0}, 0\rangle, \text{starts0}\rangle, \langle\langle\text{starts0}, 1\rangle, \text{starts1}\rangle\}$

Add lemmas to simplify

lemmas `from0To1[simp]` = `states_def empty_def transFun_def finalStates_def`
`ends0_def starts1_def starts0_def`

Interpret the example as a deterministic finite state automaton

interpretation `dfsaFrom0To1`: `DetFinStateAuto states empty transFun finalStates`
`2` *<proof>*

Abbreviate the relation to something readable.

abbreviation `r0to1` (`r{0.*1}`) **where**
`r{0.*1}` $\equiv \text{dfsaFrom0To1.r}_D$

If a word reaches the state `starts0`, it does not move from it.

lemma `invariant_state_3`:
`fixes w u y`
`assumes` $\langle\langle w, \text{ends0}\rangle, \langle u, y\rangle\rangle \in r\{0.*1\}^*$
`shows` `y = ends0`
<proof>

If the string starts in 0 and has reached states `starts0` or `starts1`; then it reduces to `starts0`.

lemma `invariant_state_0_1`:
`fixes w`
`assumes` `w ∈ NELists(2) w0 = 0`
`shows` $\langle\langle w, \text{starts0}\rangle, \langle 0, \text{starts0}\rangle\rangle \in r\{0.*1\}^*$ $\langle\langle w, \text{starts1}\rangle, \langle 0, \text{starts0}\rangle\rangle \in r\{0.*1\}^*$
<proof>

A more readable reduction statement

abbreviation red ($_ \{ \text{reduces in } 0.^*1 \}$) **where**
 $i \{ \text{reduces in } 0.^*1 \} \equiv \text{dfsaFrom0To1.reduce}(i)$

Any list starting with 0 and ending in 1 reduces.

theorem starts1ends0_DFSA_reduce:
fixes i
assumes $i \in \text{Lists}(2)$ **and** $i_0 = 0$ **and** $\text{Last}(i) = 1$
shows $i \{ \text{reduces in } 0.^*1 \}$
<proof>

Any list that reduces starts with 0 and ends in 1

theorem starts1ends0_DFSA_reduce_rev:
fixes i
assumes $i \in \text{Lists}(2)$ **and** $i \{ \text{reduces in } 0.^*1 \}$
shows $i_0 = 0$ **and** $\text{Last}(i) = 1$
<proof>

We conclude that this example constitutes the language of binary strings starting in 0 and ending in 1

theorem determine_strings:
shows $\text{dfsaFrom0To1.LanguageDFSA} = \{ i \in \text{Lists}(2) . i_0 = 0 \wedge \text{Last}(i) = 1 \}$
<proof>

end

We define the languages determined by a deterministic finite state automaton as **regular**.

definition
 $\text{IsRegularLanguage} (_ \{ \text{is a regular language on } _ \})$ **where**
 $\text{Finite}(\Sigma) \implies L \{ \text{is a regular language on } \Sigma \} \equiv \exists S \ s \ t \ F. ((S, s, t, F) \{ \text{is an DFSA for alphabet } \Sigma \} \wedge L = \text{DetFinStateAuto.LanguageDFSA}(S, s, t, F, \Sigma))$

By definition, the language in the locale is regular.

corollary (in DetFinStateAuto) regular_intersect:
shows $\text{LanguageDFSA} \{ \text{is a regular language on } \Sigma \}$
<proof>

A regular language is a language.

lemma regular_is_language:
assumes $\text{Finite}(\Sigma)$
and $L \{ \text{is a regular language on } \Sigma \}$
shows $L \{ \text{is a language with alphabet } \Sigma \}$ *<proof>*

21.3 Operations on regular languages

The intersection of two regular languages is a regular language.

theorem regular_intersect:

```

    assumes Finite( $\Sigma$ )
    and L1{is a regular language on} $\Sigma$ 
    and L2{is a regular language on} $\Sigma$ 
shows (L1 $\cap$ L2) {is a regular language on} $\Sigma$ 
<proof>

```

The complement of a regular language is a regular language.

```

theorem regular_opp:
  assumes Finite( $\Sigma$ )
  and L{is a regular language on} $\Sigma$ 
  shows (Lists( $\Sigma$ )-L) {is a regular language on} $\Sigma$ 
<proof>

```

The union of two regular languages is a regular language.

```

theorem regular_union:
  assumes Finite( $\Sigma$ )
  and L1{is a regular language on} $\Sigma$ 
  and L2{is a regular language on} $\Sigma$ 
shows (L1 $\cup$ L2) {is a regular language on} $\Sigma$ 
<proof>

```

Another natural operation on words is concatenation, hence we can defined the concatenated language as the set of concatenations of words of one language with words of another.

definition concat where

$$L1 \text{ {is a language with alphabet} } \Sigma \implies L2 \text{ {is a language with alphabet} } \Sigma$$

$$\implies \text{concat}(L1, L2) = \{\text{Concat}(w2, w1) . \langle w1, w2 \rangle \in L1 \times L2\}$$

The result of concatenating two languages is a language.

```

lemma concat_language:
  assumes Finite( $\Sigma$ )
  and L1 {is a language with alphabet} $\Sigma$ 
  and L2 {is a language with alphabet} $\Sigma$ 
shows concat(L1,L2) {is a language with alphabet} $\Sigma$ 
<proof>

```

21.4 Non-deterministic finite state automata

We have reached a point where it is not easy to realize a concatenated language of two regular languages as a regular language. Nevertheless, if we extend our instruments to allow non-determinism it is much easier.

The cost, a priori, is that our class of languages would be larger since our automata are more generic.

The non-determinism is introduced by allowing the transition function to return not just a state, but more than one or even none.

definition

NFSA $(_ ,_ ,_ ,_)$ {is an NFSA for alphabet} $_$ where
 $\text{Finite}(\Sigma) \implies (S, s_0, t, F)$ {is an NFSA for alphabet} $\Sigma \equiv \text{Finite}(S) \wedge s_0 \in S$
 $\wedge F \subseteq S \wedge t: S \times \Sigma \rightarrow \text{Pow}(S)$

The transition relation is then realized by considering all possible steps the transition function returns.

definition

NFSAExecutionRelation $(\{\text{reduce N-relation}\} _ ,_)$ {in alphabet} $_$ where
 $\text{Finite}(\Sigma) \implies (S, s_0, t, F)$ {is an NFSA for alphabet} $\Sigma \implies$
 $\{\text{reduce N-relation}\}(S, t)$ {in alphabet} $\Sigma \equiv \{ \langle \langle w, Q \rangle, \langle \text{Init}(w), \bigcup \{t(s, \text{Last}(w))\} \rangle \in s \in Q \rangle \}. \langle w, Q \rangle \in \text{NELists}(\Sigma) \times \text{Pow}(S) \}$

The full reduction is conceived as one of those possible paths reaching a final state.

definition

NFSA Satisfy $(_ <-N _ ,_ ,_ ,_)$ {in alphabet} $_$ where
 $\text{Finite}(\Sigma) \implies (S, s_0, t, F)$ {is an NFSA for alphabet} $\Sigma \implies i \in \text{Lists}(\Sigma) \implies$
 $i <-N (S, s_0, t, F)$ {in alphabet} $\Sigma \equiv (\exists q \in \text{Pow}(S). (q \cap F \neq \emptyset \wedge \langle \langle i, \{s_0\} \rangle, \langle 0, q \rangle \rangle \in (\{\text{reduce N-relation}\}(S, t)$ {in alphabet} $\Sigma)^*)) \vee (i = \emptyset \wedge s_0 \in F)$

An extra generalization can be consider if we allow the transition relation to go forward without consuming elements from the word. This is implemented as allowing Σ to symbolize an step without the word being touched. We might call it a Σ transition or a ε -transition.

definition

FullNFSA $(_ ,_ ,_ ,_)$ {is an ε -NFSA for alphabet} $_$ where
 $\text{Finite}(\Sigma) \implies (S, s_0, t, F)$ {is an ε -NFSA for alphabet} $\Sigma \equiv \text{Finite}(S) \wedge$
 $s_0 \in S \wedge F \subseteq S \wedge t: S \times \text{succ}(\Sigma) \rightarrow \text{Pow}(S)$

The closure of a set of states can then be viewed as all the states reachable from that set with a transition of type Σ .

definition

EpsilonClosure $(\varepsilon\text{-cl})$ where
 $\text{Finite}(\Sigma) \implies (S, s_0, t, F)$ {is an ε -NFSA for alphabet} $\Sigma \implies E \subseteq S$
 $\implies \varepsilon\text{-cl}(S, t, \Sigma, E) \equiv \bigcup \{P \in \text{Pow}(S). \langle E, P \rangle \in (\{ \langle Q, \{s \in S. \exists q \in Q. s \in t(q, \Sigma) \} \rangle. Q \in \text{Pow}(S) \}^*) \}$

The reduction relation is then extended by considering any such transitions.

definition

FullNFSAExecutionRelation $(\{\text{reduce } \varepsilon\text{-N-relation}\} _ ,_)$ {in alphabet} $_$
where
 $\text{Finite}(\Sigma) \implies (S, s_0, t, F)$ {is an ε -NFSA for alphabet} $\Sigma \implies$
 $\{\text{reduce } \varepsilon\text{-N-relation}\}(S, t)$ {in alphabet} $\Sigma \equiv \{ \langle \langle w, Q \rangle, \langle \text{Init}(w), \varepsilon\text{-cl}(S, t, \Sigma, \bigcup \{t(s, \text{Last}(w))\} \rangle. s \in \varepsilon\text{-cl}(S, t, \Sigma, Q) \} \rangle \rangle. \langle w, Q \rangle \in \text{NELists}(\Sigma) \times \text{Pow}(S) \}$

The full reduction of a word is similar to that of the automata without ε -transitions.

definition

FullNFSASatisfy ($_ <-\varepsilon\text{-N } '(_,_,_,_)' \{ \text{in alphabet} \}__$) **where**
 Finite(Σ) $\implies (S, s_0, t, F) \{ \text{is an } \varepsilon\text{-NFSA for alphabet} \} \Sigma \implies i \in \text{Lists}(\Sigma)$
 $\implies$
 $i <-\varepsilon\text{-N } (S, s_0, t, F) \{ \text{in alphabet} \} \Sigma \equiv (\exists q \in \text{Pow}(S). (\varepsilon\text{-cl}(S, t, \Sigma, q) \cap F \neq \emptyset$
 $\wedge \langle \langle i, \{s_0\} \rangle, \langle 0, q \rangle \rangle \in (\{ \text{reduce } \varepsilon\text{-N-relation} \} (S, t) \{ \text{in alphabet} \} \Sigma)^*) \vee (i$
 $= 0 \wedge \varepsilon\text{-cl}(S, t, \Sigma, \{s_0\}) \cap F \neq \emptyset)$

We define a locale to create some notation

locale NonDetFinStateAuto =
fixes S **and** s_0 **and** t **and** F **and** Σ
assumes finite_alphabet: Finite(Σ)

assumes NFSA: $(S, s_0, t, F) \{ \text{is an NFSA for alphabet} \} \Sigma$

Notation for the transition relation

abbreviation (in NonDetFinStateAuto) nd_rel (r_N)
where $r_N \equiv \{ \text{reduce N-relation} \} (S, t) \{ \text{in alphabet} \} \Sigma$

Notation for the language generated by the non-deterministic automaton

abbreviation (in NonDetFinStateAuto) LanguageNFSA
where LanguageNFSA $\equiv \{ i \in \text{Lists}(\Sigma). i <-\text{N } (S, s_0, t, F) \{ \text{in alphabet} \} \Sigma \}$

21.5 Equivalence of Non-deterministic and Deterministic Finite State Automata

We will show that the non-deterministic automata generate languages that are regular in the sense that there is a deterministic automaton that generates the same language.

The transition function of the deterministic automata we will construct

definition (in NonDetFinStateAuto) tPow **where**
 $tPow \equiv \{ \langle \langle U, u \rangle, (\bigcup v \in U. t \langle v, u \rangle) \rangle. \langle U, u \rangle \in \text{Pow}(S) \times \Sigma \}$

The transition relation of the deterministic automata we will construct

definition (in NonDetFinStateAuto) rPow **where**
 $rPow \equiv \text{DetFinStateAuto.r}_D(\text{Pow}(S), tPow, \Sigma)$

We show that we do have a deterministic automaton

sublocale NonDetFinStateAuto < dfsa:DetFinStateAuto Pow(S) $\{s_0\}$ tPow $\{Q \in \text{Pow}(S). Q \cap F \neq \emptyset\} \Sigma$
<proof>

The two automata have the same relations associated with them.

First, we show that if the non-deterministic automaton produces a reduction step to a word, then the deterministic one we constructed does the same reduction step.

```

lemma (in NonDetFinStateAuto) nd_impl_det:
  assumes  $\langle\langle w, Q \rangle, \langle u, G \rangle\rangle \in r_N$ 
  shows  $\langle\langle w, Q \rangle, \langle u, G \rangle\rangle \in rPow$ 
  <proof>

```

Next, we show that if the deterministic automaton produces a reduction step to a word, then the non-deterministic one we constructed does the same reduction step.

```

lemma (in NonDetFinStateAuto) det_impl_nd:
  assumes  $\langle\langle w, Q \rangle, \langle u, G \rangle\rangle \in rPow$ 
  shows  $\langle\langle w, Q \rangle, \langle u, G \rangle\rangle \in r_N$ 
  <proof>

```

Since both are relations, they are equal

```

corollary (in NonDetFinStateAuto) relation_NFSA_to_DFSA:
  shows  $r_N = rPow$  <proof>

```

As a consequence, by the definition of a language generated by an automaton, both languages are equal.

```

theorem (in NonDetFinStateAuto) language_nfsa:
  shows  $dfs_a.LanguageDFSA = LanguageNFSA$ 
  <proof>

```

The language of a non-deterministic finite state automaton is regular.

```

corollary (in NonDetFinStateAuto) lang_is_regular:
  shows  $LanguageNFSA\{is\ a\ regular\ language\ on\}\Sigma$ 
  <proof>

```

21.6 Epsilon-NFSA languages are regular

We now show that every language recognised by an ε -NFSA is regular. The strategy mirrors the NFSA-to-DFSA construction already in the file: we absorb the ε -transitions into the transition function so as to obtain an ordinary NFSA, and then appeal to $NonDetFinStateAuto(S, s_0, t, F, \Sigma) \implies \{i \in Lists(\Sigma) . i \prec_N (S, s_0, t, F)\{in\ alphabet\}\Sigma\}\{is\ a\ regular\ language\ on\}\Sigma$.

Given an ε -NFSA (S, s_0, t, F) over Σ , define the ε -free transition function by following every ordinary transition with the ε -closure of the resulting set of states.

```

definition EpsilonFreeTransition where
  Finite( $\Sigma$ )  $\implies (S, s_0, t, F)\{is\ an\ \varepsilon\text{-NFSA\ for\ alphabet}\}\Sigma \implies$ 
  EpsilonFreeTransition( $S, t, \Sigma$ )  $\equiv$ 
   $\{\langle\langle s, x \rangle, \bigcup \{\varepsilon\text{-cl}(S, t, \Sigma, t(p, x)) . p \in \varepsilon\text{-cl}(S, t, \Sigma, \{s\})\}\rangle . \langle s, x \rangle \in S \times \Sigma\}$ 

```

The ε -free transition function is a function $S \times \Sigma \rightarrow Pow(S)$.

```

lemma EpsilonFreeTransition_type:

```

```

    assumes fin:Finite( $\Sigma$ )
    and fsa:( $S, s_0, t, F$ ){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
    shows EpsilonFreeTransition( $S, t, \Sigma$ ) :  $S \times \Sigma \rightarrow \text{Pow}(S)$ 
  <proof>

```

The ε -free automaton, where the new final states collect all states whose ε -closure meets F , is an NFSA over Σ .

```

lemma EpsilonFree_is_NFSA:
  assumes fin:Finite( $\Sigma$ )
  and fsa:( $S, s_0, t, F$ ){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
  shows ( $S, s_0, \text{EpsilonFreeTransition}(S, t, \Sigma), \{q \in S. \varepsilon\text{-cl}(S, t, \Sigma, \{q\}) \cap F \neq \emptyset\}$ ){is an NFSA for alphabet} $\Sigma$ 
  <proof>

```

Every set $B \subseteq S$ is contained in its own ε -closure.

```

lemma epsilon_cl_refl_sub:
  assumes fin:Finite( $\Sigma$ )
  and fsa:( $S, s_0, t, F$ ){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
  and bS: $B \subseteq S$ 
  shows  $B \subseteq \varepsilon\text{-cl}(S, t, \Sigma, B)$ 
  <proof>

```

The ε -closure is monotone: if $A \subseteq B \subseteq S$ then $\varepsilon\text{-cl}(S, t, \Sigma, A) \subseteq \varepsilon\text{-cl}(S, t, \Sigma, B)$.

```

lemma epsilon_cl_mono:
  assumes fin:Finite( $\Sigma$ )
  and fsa:( $S, s_0, t, F$ ){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
  and bS: $B \subseteq S$  and sub: $A \subseteq B$ 
  shows  $\varepsilon\text{-cl}(S, t, \Sigma, A) \subseteq \varepsilon\text{-cl}(S, t, \Sigma, B)$ 
  <proof>

```

The ε -closure distributes over arbitrary unions: for $C \subseteq \text{Pow}(S)$ we have $\varepsilon\text{-cl}(S, t, \Sigma, \bigcup C) = \bigcup \{\varepsilon\text{-cl}(S, t, \Sigma, E) \mid E \in C\}$.

```

lemma epsilon_cl_union:
  assumes fin:Finite( $\Sigma$ )
  and fsa:( $S, s_0, t, F$ ){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
  and cS: $C \subseteq \text{Pow}(S)$ 
  shows  $\varepsilon\text{-cl}(S, t, \Sigma, \bigcup C) = \bigcup \{\varepsilon\text{-cl}(S, t, \Sigma, E) . E \in C\}$ 
  <proof>

```

The ε -closure is idempotent: for $C \in \text{Pow}(S)$ we have $\varepsilon\text{-cl}(S, t, \Sigma, \varepsilon\text{-cl}(S, t, \Sigma, C)) = \varepsilon\text{-cl}(S, t, \Sigma, C)$.

```

lemma epsilon_cl_idem:
  assumes fin:Finite( $\Sigma$ )
  and fsa:( $S, s_0, t, F$ ){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
  and cS: $q \in \text{Pow}(S)$ 
  shows  $\varepsilon\text{-cl}(S, t, \Sigma, \varepsilon\text{-cl}(S, t, \Sigma, q)) = \varepsilon\text{-cl}(S, t, \Sigma, q)$ 
  <proof>

```

The value of `EpsilonFreeTransition` at $\langle s, x \rangle$ is the ε -closure of $t^* \langle s, x \rangle$.

```
lemma EpsilonFreeTransition_val:
  assumes fin:Finite( $\Sigma$ )
  and fsa:(S,s0,t,F){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
  and sS:s $\in$ S and xSig:x $\in$  $\Sigma$ 
  shows EpsilonFreeTransition(S,t, $\Sigma$ ) $\langle s, x \rangle$  = ( $\bigcup_{p \in \varepsilon\text{-cl}(S,t,\Sigma,\{s\})} \varepsilon\text{-cl}(S,t,\Sigma,t(p,x))$ )
  <proof>
```

The ε -N execution relation coincides with the N execution relation of the ε -free transition function.

```
lemma epsilon_free_rel_eq:
  assumes fin:Finite( $\Sigma$ )
  and fsa:(S,s0,t,F){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
  shows {reduce  $\varepsilon$ -N-relation}(S,t){in alphabet} $\Sigma$  =
    {reduce N-relation}(S,EpsilonFreeTransition(S,t, $\Sigma$ )){in alphabet} $\Sigma$ 
  <proof>
```

The field of the NFSA relation is equal to its DFSA construction

```
lemma (in NonDetFinStateAuto) eps_nfsa_field:
  shows field({reduce N-relation}(S,t){in alphabet} $\Sigma$ )  $\subseteq$  Lists( $\Sigma$ ) $\times$ Pow(S)
    NELists( $\Sigma$ ) $\times$ Pow(S)  $\subseteq$  field({reduce N-relation}(S,t){in alphabet} $\Sigma$ )
  <proof>
```

The field of the ε relation is equal to its NDFSFA construction

```
lemma eps_nfsa_field:
  assumes fin:Finite( $\Sigma$ )
  and fsa:(S,s0,t,F){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
  shows field({reduce  $\varepsilon$ -N-relation}(S,t){in alphabet} $\Sigma$ )  $\subseteq$  Lists( $\Sigma$ ) $\times$ Pow(S)
    NELists( $\Sigma$ ) $\times$ Pow(S)  $\subseteq$  field({reduce  $\varepsilon$ -N-relation}(S,t){in alphabet} $\Sigma$ )
  <proof>
```

The language accepted by the ε -NFSFA equals the language accepted by its ε -free counterpart. The key observation is that every ε -step in the execution relation is absorbed by the ε -closure in `EpsilonFreeTransition`.

```
lemma EpsilonFree_same_language_1:
  assumes fin:Finite( $\Sigma$ )
  and fsa:(S,s0,t,F){is an  $\varepsilon$ -NFSFA for alphabet} $\Sigma$ 
  and i $\in$ Lists( $\Sigma$ ) i <- $\varepsilon$ -N (S,s0,t,F){in alphabet} $\Sigma$ 
  shows i <-N (S,s0,EpsilonFreeTransition(S,t, $\Sigma$ ),{q $\in$ S.  $\varepsilon\text{-cl}(S,t,\Sigma,\{q\}) \cap F \neq \emptyset$ }){in
alphabet} $\Sigma$ 
  <proof>
```

```
lemma EpsilonFree_same_language_2:
  assumes fin:Finite( $\Sigma$ )
  and fsa:(S,s0,t,F){is an  $\varepsilon$ -NFSFA for alphabet} $\Sigma$ 
  and i $\in$ Lists( $\Sigma$ ) i <-N (S,s0,EpsilonFreeTransition(S,t, $\Sigma$ ),{q $\in$ S.  $\varepsilon\text{-cl}(S,t,\Sigma,\{q\}) \cap F \neq \emptyset$ }){in
alphabet} $\Sigma$ 
  shows i <- $\varepsilon$ -N (S,s0,t,F){in alphabet} $\Sigma$ 
```

<proof>

corollary EpsilonFree_same_language:

```

  assumes fin:Finite( $\Sigma$ )
  and fsa:(S,s0,t,F){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
  shows {i $\in$ Lists( $\Sigma$ ). i  $\leftarrow$ - $\varepsilon$ -N (S,s0,t,F){in alphabet} $\Sigma$ } =
    {i $\in$ Lists( $\Sigma$ ). i  $\leftarrow$ -N (S,s0,EpsilonFreeTransition(S,t, $\Sigma$ ),{q $\in$ S.  $\varepsilon$ -cl(S,t, $\Sigma$ ,{q}) $\cap$ F $\neq$ 0})}{in
alphabet} $\Sigma$ }
<proof>

```

Every language recognised by an ε -NFSA is regular.

theorem epsilonNFSLang_is_regular:

```

  assumes fin:Finite( $\Sigma$ )
  and fsa:(S,s0,t,F){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
  shows {i $\in$ Lists( $\Sigma$ ). i  $\leftarrow$ - $\varepsilon$ -N (S,s0,t,F){in alphabet} $\Sigma$ } {is a regular
language on} $\Sigma$ 
<proof>

```

21.7 Concatenation of regular languages

We now prove the main theorem: the concatenation of two regular languages is regular. The proof constructs an ε -NFSA that first simulates the automaton for L_1 , then makes a free ε -transition to the initial state of the automaton for L_2 upon reaching an accepting state of the first, and finally accepts when the second automaton accepts.

The combined state space for the product ε -NFSA is the disjoint union $S_1 \times \{0\} \cup S_2 \times \{1\}$.

definition concat_eNFSLang_states where

$$\text{concat_eNFSLang_states}(S_1, S_2) \equiv S_1 \times \{0\} \cup S_2 \times \{1\}$$

The transition function of the product ε -NFSA. A state $\langle s, 0 \rangle$ in the first component reads $a \in \Sigma$ by following t_1 ; on the ε -symbol (encoded as Σ) it jumps to $\langle s_0, 1 \rangle$ when $s \in F_1$, and has no ε -move otherwise. A state $\langle s, 1 \rangle$ in the second component reads $a \in \Sigma$ by following t_2 , and ignores ε -steps.

definition concat_eNFSLang_trans where

$$\begin{aligned}
 &\text{Finite}(\Sigma) \implies \\
 &(S_1, s_{01}, t_1, F_1) \{\text{is an DFSA for alphabet}\} \Sigma \implies \\
 &(S_2, s_{02}, t_2, F_2) \{\text{is an DFSA for alphabet}\} \Sigma \implies \\
 &\text{concat_eNFSLang_trans}(S_1, s_{01}, t_1, F_1, S_2, s_{02}, t_2, F_2, \Sigma) \equiv \\
 &\quad \{ \langle \langle s, 0 \rangle, q \rangle, \{ t_1 \langle s, q \rangle \} \times \{0\} \}. \langle s, q \rangle \in S_1 \times \Sigma \} \\
 &\quad \cup \{ \langle \langle s, 0 \rangle, \Sigma \rangle, \{ x \in \{ \langle s_{02}, 1 \rangle \}. s \in F_1 \}. s \in S_1 \} \\
 &\quad \cup \{ \langle \langle s, 1 \rangle, q \rangle, \{ t_2 \langle s, q \rangle \} \times \{1\} \}. \langle s, q \rangle \in S_2 \times \Sigma \} \\
 &\quad \cup \{ \langle \langle s, 1 \rangle, \Sigma \rangle, 0 \}. s \in S_2 \}
 \end{aligned}$$

The product automaton is a valid ε -NFSA.

lemma concat_eNFSLang_valid:

```

assumes fin:Finite( $\Sigma$ )
and A1:(S1,s01,t1,F1){is an DFSA for alphabet} $\Sigma$ 
and A2:(S2,s02,t2,F2){is an DFSA for alphabet} $\Sigma$ 
shows (concat_eNFSA_states(S1,S2),  $\langle s01,0 \rangle$ ,
concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ ),  $F2 \times \{1\}$ 
){is an  $\varepsilon$ -NFSA for alphabet} $\Sigma$ 
<proof>

```

The ε -transition of a component-0 state $\langle s, 0 \rangle$ is $\{\langle s02, 1 \rangle\}$ when $s \in F_1$, and empty otherwise.

```

lemma concat_eNFSA_eps_comp0:
  assumes fin:Finite( $\Sigma$ )
  and A1:(S1,s01,t1,F1){is an DFSA for alphabet} $\Sigma$ 
  and A2:(S2,s02,t2,F2){is an DFSA for alphabet} $\Sigma$ 
  and sS1:s $\in$ S1
  shows concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ ) $\langle \langle s,0 \rangle, \Sigma \rangle$ 
    =  $\{x \in \{\langle s02, 1 \rangle\} . s \in F1\}$ 
<proof>

```

The ε -transition of a component-1 state $\langle s, 1 \rangle$ is empty.

```

lemma concat_eNFSA_eps_comp1:
  assumes fin:Finite( $\Sigma$ )
  and A1:(S1,s01,t1,F1){is an DFSA for alphabet} $\Sigma$ 
  and A2:(S2,s02,t2,F2){is an DFSA for alphabet} $\Sigma$ 
  and sS2:s $\in$ S2
  shows concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ ) $\langle \langle s,1 \rangle, \Sigma \rangle$  = 0
<proof>

```

The normal transition of a component-0 state $\langle s, 0 \rangle$ is a transition of the first DFSA.

```

lemma concat_eNFSA_eps_comp0':
  assumes fin:Finite( $\Sigma$ )
  and A1:(S1,s01,t1,F1){is an DFSA for alphabet} $\Sigma$ 
  and A2:(S2,s02,t2,F2){is an DFSA for alphabet} $\Sigma$ 
  and sS1:s $\in$ S1 and sig:q $\in$  $\Sigma$ 
  shows concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ ) $\langle \langle s,0 \rangle, q \rangle$ 
    =  $\{t1\langle s, q \rangle\} \times \{0\}$ 
<proof>

```

The normal transition of a component-1 state $\langle s, 1 \rangle$ is a normal transition of the second DFSA.

```

lemma concat_eNFSA_eps_comp1':
  assumes fin:Finite( $\Sigma$ )
  and A1:(S1,s01,t1,F1){is an DFSA for alphabet} $\Sigma$ 
  and A2:(S2,s02,t2,F2){is an DFSA for alphabet} $\Sigma$ 
  and sS2:s $\in$ S2 and sig:q $\in$  $\Sigma$ 
  shows concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ ) $\langle \langle s,1 \rangle, q \rangle$  =  $\{t2\langle s, q \rangle\} \times \{1\}$ 
<proof>

```

The normal transition of a component-1 state $\langle s, 1 \rangle$ is a normal transition of the second DFSA.

```

lemma concat_eNFSA_eps_closure:
  fixes S1 S2 s01 s02 t1 t2 F1 F2  $\Sigma$ 
  defines t  $\equiv$  concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ )
  defines S  $\equiv$  concat_eNFSA_states(S1,S2)
  defines s0  $\equiv$   $\langle s01, 0 \rangle$ 
  defines F  $\equiv$  F2 $\times$ {1}
  assumes fin:Finite( $\Sigma$ )
  and A1:(S1,s01,t1,F1){is an DFSA for alphabet} $\Sigma$ 
  and A2:(S2,s02,t2,F2){is an DFSA for alphabet} $\Sigma$ 
  and sig:q $\in$ Pow(S)
  shows  $\varepsilon\text{-cl}(S,t,\Sigma,q) = q \cup \{x \in \{\langle s02, 1 \rangle\} . q \cap (F1 \times 1) \neq \emptyset\}$ 
<proof>

```

Once L2 is reached, the relation is equivalent to its DFSA

```

lemma concat_FSA_apply_L2_step:
  fixes S1 S2 s01 s02 t1 t2 F1 F2  $\Sigma$ 
  defines t  $\equiv$  concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ )
  defines S  $\equiv$  concat_eNFSA_states(S1,S2)
  defines s0  $\equiv$   $\langle s01, 0 \rangle$ 
  defines F  $\equiv$  F2 $\times$ {1}
  assumes fin:Finite( $\Sigma$ )
  and A1:(S1,s01,t1,F1){is an DFSA for alphabet} $\Sigma$ 
  and A2:(S2,s02,t2,F2){is an DFSA for alphabet} $\Sigma$ 
  and j:  $\langle \langle j, p \rangle, \langle r, q \rangle \rangle \in (\{\text{reduce D-relation}\}(S2,t2)\{\text{in alphabet}\}\Sigma)^* j \neq 0$ 
  and states:  $\langle p, 1 \rangle \in \varepsilon\text{-cl}(S,t,\Sigma,P) \ P \in \text{Pow}(S)$ 
  shows  $\exists Q \in \text{Pow}(S) . \langle q, 1 \rangle \in \varepsilon\text{-cl}(S,t,\Sigma,Q) \wedge (\langle \langle j, P \rangle, \langle r, Q \rangle \rangle \in ((\{\text{reduce } \varepsilon\text{-N-relation}\}(S,t)\{\text{in alphabet}\}\Sigma)^*))$ 
<proof>

```

Once the only thing left is a word in L2, it passes if s02 is one of the initial states.

```

lemma concat_FSA_apply_L2:
  fixes S1 S2 s01 s02 t1 t2 F1 F2  $\Sigma$ 
  defines t  $\equiv$  concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ )
  defines S  $\equiv$  concat_eNFSA_states(S1,S2)
  defines s0  $\equiv$   $\langle s01, 0 \rangle$ 
  defines F  $\equiv$  F2 $\times$ {1}
  defines L1  $\equiv$  {i $\in$ Lists( $\Sigma$ ). i  $\leftarrow$  D (S1,s01,t1,F1){in alphabet} $\Sigma$ }
  defines L2  $\equiv$  {i $\in$ Lists( $\Sigma$ ). i  $\leftarrow$  D (S2,s02,t2,F2){in alphabet} $\Sigma$ }
  assumes fin:Finite( $\Sigma$ )
  and A1:(S1,s01,t1,F1){is an DFSA for alphabet} $\Sigma$ 
  and A2:(S2,s02,t2,F2){is an DFSA for alphabet} $\Sigma$ 
  and j: j $\in$ L2  $\langle s02, 1 \rangle \in \varepsilon\text{-cl}(S,t,\Sigma,Q) \ Q \in \text{Pow}(S) \ j \neq 0$ 
  shows  $\exists q \in \text{Pow}(S) . ((\varepsilon\text{-cl}(S,t,\Sigma,q) \cap F \neq \emptyset) \wedge (\langle \langle j, Q \rangle, \langle 0, q \rangle \rangle \in ((\{\text{reduce } \varepsilon\text{-N-relation}\}(S,t)\{\text{in alphabet}\}\Sigma)^*)))$ 
<proof>

```

Concatenating a list on the left of a non-empty list yields a non-empty list.

Step 2 (sub-goal of `concat_FSA_apply_L1_step`): Concat preserves the non-empty list property when the right argument is non-empty.

```
lemma concat_is_list:
  assumes p∈Lists(X) b∈Lists(X)
  shows Concat(p,b)∈Lists(X)
⟨proof⟩
```

```
lemma concat_is_NEList:
  assumes p∈Lists(X) b∈NELists(X)
  shows Concat(p,b)∈NELists(X)
⟨proof⟩
```

```
lemma concat_0_left:
  assumes p∈Lists(X)
  shows Concat(p,0) = p
⟨proof⟩
```

Step 3 (sub-goal of `concat_FSA_apply_L1_step`): The last element of a left-extended non-empty list is the same as the last element of the original non-empty list.

```
lemma concat_last_NEList:
  assumes p∈Lists(X) b∈NELists(X)
  shows Last(Concat(p,b)) = Last(b)
⟨proof⟩
```

Step 4 (sub-goal of `concat_FSA_apply_L1_step`): Init distributes over Concat from the right: init of the concatenation is the concatenation with the init of the right part.

```
lemma concat_init_NEList:
  assumes p∈Lists(X) b∈NELists(X)
  shows Init(Concat(p,b)) = Concat(p,Init(b))
⟨proof⟩
```

Running a word of the form `Concat(v,j)` to `v` in a DFSA is the same as running `j` from the same starting state.

```
lemma (in DetFinStateAuto) dfa_run_suffix:
  assumes vL:v∈Lists(Σ) and jNE:j∈NELists(Σ)
  and run:⟨⟨Concat(v,j),s⟩,⟨v,q⟩⟩∈r_D^*
  shows ⟨⟨j,s⟩,⟨0,q⟩⟩∈r_D^*
⟨proof⟩
```

If a DFSA run reduces word `w` to word `v`, then `v` is an initial prefix of `w`, i.e., there exists a suffix `j` with `w = Concat(v,j)`.

```
lemma (in DetFinStateAuto) list_prefix_split:
  assumes run: ⟨⟨w,s⟩,⟨v,q⟩⟩ ∈ r_D^*
```


shows $\exists j \in \text{Lists}(\Sigma). w = \text{Concat}(v, j)$
<proof>

Once L1 is reached, the relation is equivalent to its DFSA

Step 1: We prove `concat_FSA_apply_L1_step` using the three sub-lemmas above. Note: the statement requires an additional hypothesis that the prefix s is a list over Σ .

lemma `concat_FSA_apply_L1_step`:
fixes $S1\ S2\ s01\ s02\ t1\ t2\ F1\ F2\ \Sigma$
defines $t \equiv \text{concat_eNFSA_trans}(S1, s01, t1, F1, S2, s02, t2, F2, \Sigma)$
defines $S \equiv \text{concat_eNFSA_states}(S1, S2)$
defines $s_0 \equiv \langle s01, 0 \rangle$
defines $F \equiv F2 \times \{1\}$
assumes $\text{fin} : \text{Finite}(\Sigma)$
and $A1 : (S1, s01, t1, F1) \{\text{is an DFSA for alphabet}\} \Sigma$
and $A2 : (S2, s02, t2, F2) \{\text{is an DFSA for alphabet}\} \Sigma$
and $j : \langle \langle j, p \rangle, \langle r, q \rangle \rangle \in (\{\text{reduce D-relation}\}(S1, t1) \{\text{in alphabet}\} \Sigma)^* j \neq 0$
and $\text{states} : \langle p, 0 \rangle \in P\ P \in \text{Pow}(S)$
and $sL : s \in \text{Lists}(\Sigma)$
shows $\exists Q \in \text{Pow}(S). \langle q, 0 \rangle \in Q \wedge ((\langle \text{Concat}(s, j), P \rangle, \langle \text{Concat}(s, r), Q \rangle) \in ((\{\text{reduce } \varepsilon\text{-N-relation}\}(S, t) \{\text{in alphabet}\} \Sigma)^*))$
<proof>

Once the only thing left is a word in L1, it passes if $s01$ is one of the initial states.

lemma `concat_FSA_apply_L1`:
fixes $S1\ S2\ s01\ s02\ t1\ t2\ F1\ F2\ \Sigma$
defines $t \equiv \text{concat_eNFSA_trans}(S1, s01, t1, F1, S2, s02, t2, F2, \Sigma)$
defines $S \equiv \text{concat_eNFSA_states}(S1, S2)$
defines $s_0 \equiv \langle s01, 0 \rangle$
defines $F \equiv F2 \times \{1\}$
defines $L1 \equiv \{i \in \text{Lists}(\Sigma). i \leftarrow_D (S1, s01, t1, F1) \{\text{in alphabet}\} \Sigma\}$
defines $L2 \equiv \{i \in \text{Lists}(\Sigma). i \leftarrow_D (S2, s02, t2, F2) \{\text{in alphabet}\} \Sigma\}$
assumes $\text{fin} : \text{Finite}(\Sigma)$
and $A1 : (S1, s01, t1, F1) \{\text{is an DFSA for alphabet}\} \Sigma$
and $A2 : (S2, s02, t2, F2) \{\text{is an DFSA for alphabet}\} \Sigma$
and $j : j \in L1\ \langle s01, 0 \rangle \in Q\ Q \in \text{Pow}(S)\ j \neq 0$
and $k : u \in \text{Lists}(\Sigma)$
shows $\exists q \in \text{Pow}(S). \langle s02, 1 \rangle \in q \wedge ((\langle \text{Concat}(u, j), Q \rangle, \langle u, q \rangle) \in ((\{\text{reduce } \varepsilon\text{-N-relation}\}(S, t) \{\text{in alphabet}\} \Sigma)^*))$
<proof>

The ε -closure of a set of side-1 states stays within side 1: for $T \subseteq S_2$ we have $\varepsilon\text{-cl}(S, t, \Sigma, T \times \{1\}) = T \times \{1\}$. This holds because the ε -transition from every side-1 state is empty (by `concat_eNFSA_eps_comp1`), so the closure adds nothing.

lemma `epsilon_cl_side1`:

```

fixes S1 S2 s01 s02 t1 t2 F1 F2  $\Sigma$ 
defines t  $\equiv$  concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ )
defines S  $\equiv$  concat_eNFSA_states(S1,S2)
assumes fin:Finite( $\Sigma$ )
and A1:(S1,s01,t1,F1){is an DFSA for alphabet} $\Sigma$ 
and A2:(S2,s02,t2,F2){is an DFSA for alphabet} $\Sigma$ 
and sig:T $\subseteq$ S2
shows  $\varepsilon\text{-cl}(S,t,\Sigma,T\times\{1\}) = T\times\{1\}$ 
<proof>

```

One letter step in the concat ε -NFSA, starting from a state set of the form $\{\langle q_1, 0 \rangle\} \cup Q_2 \times \{1\}$, produces a state set of the same form, with the side-0 singleton updated by the DFA transition of A_1 .

```

lemma exec_step_form:
  fixes S1 S2 s01 s02 t1 t2 F1 F2  $\Sigma$  q1 Q2 ltr
  defines t  $\equiv$  concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ )
  defines S  $\equiv$  concat_eNFSA_states(S1,S2)
  assumes fin:Finite( $\Sigma$ )
  and A1:(S1,s01,t1,F1){is an DFSA for alphabet} $\Sigma$ 
  and A2:(S2,s02,t2,F2){is an DFSA for alphabet} $\Sigma$ 
  and q1S1:q1 $\in$ S1 and Q2S2:Q2 $\subseteq$ S2 and ltrS:ltr $\in$  $\Sigma$ 
  shows  $\exists Q2n \in \text{Pow}(S2). \varepsilon\text{-cl}(S,t,\Sigma, \bigcup \{t\langle ss, ltr \rangle. ss \in \varepsilon\text{-cl}(S,t,\Sigma, \{\langle q_1, 0 \rangle\} \cup Q2 \times \{1\})\})$ 
     $= \{\langle t1\langle q_1, ltr \rangle, 0 \rangle\} \cup Q2n \times \{1\}$ 
<proof>

```

Explicit form of the next state set after one step of the concat ε -NFSA.

```

lemma exec_step_Q2_form:
  fixes S1 S2 s01 s02 t1 t2 F1 F2  $\Sigma$  q1 Q2 ltr
  defines t  $\equiv$  concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ )
  defines S  $\equiv$  concat_eNFSA_states(S1,S2)
  assumes fin:Finite( $\Sigma$ )
  and A1:(S1,s01,t1,F1){is an DFSA for alphabet} $\Sigma$ 
  and A2:(S2,s02,t2,F2){is an DFSA for alphabet} $\Sigma$ 
  and q1S1:q1 $\in$ S1 and Q2S2:Q2 $\subseteq$ S2 and ltrS:ltr $\in$  $\Sigma$ 
  shows  $\varepsilon\text{-cl}(S,t,\Sigma, \bigcup \{t\langle ss, ltr \rangle. ss \in \varepsilon\text{-cl}(S,t,\Sigma, \{\langle q_1, 0 \rangle\} \cup Q2 \times \{1\})\})$ 
     $= \{\langle t1\langle q_1, ltr \rangle, 0 \rangle\} \cup$ 
       $\{\langle t2\langle r, ltr \rangle. r \in Q2\} \cup \{x \in \{t2\langle s02, ltr \rangle\}. q1 \in F1\} \cup$ 
       $\{x \in \{s02\}. t1\langle q1, ltr \rangle \in F1\} \times \{1\}$ 
<proof>

```

If the concat ε -NFSA executes word w (non-empty) from $\{\langle s_{01}, 0 \rangle\}$ and reaches $\langle v, Q \rangle$, then Q has the form $\{\langle q_1, 0 \rangle\} \cup Q_2 \times \{1\}$ with $q_1 \in S_1$, $Q_2 \subseteq S_2$, and A_1 tracks the word: $\langle \langle w, s_{01} \rangle, \langle v, q_1 \rangle \rangle \in r_{D_1}^*$.

```

lemma exec_state_form:
  fixes S1 S2 s01 s02 t1 t2 F1 F2  $\Sigma$  w v Q
  defines t  $\equiv$  concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2, $\Sigma$ )
  defines S  $\equiv$  concat_eNFSA_states(S1,S2)
  assumes fin:Finite( $\Sigma$ )

```

```

and A1: (S1,s01,t1,F1){is an DFSA for alphabet}Σ
and A2: (S2,s02,t2,F2){is an DFSA for alphabet}Σ
and wNE:w∈NELists(Σ)
and run: ⟨⟨w,{s01,0}⟩,⟨v,Q⟩⟩ ∈ (({reduce ε-N-relation}(S,t){in alphabet}Σ)^*)
shows ∃q1∈S1. ∃Q2∈Pow(S2). Q = {⟨q1,0⟩} ∪ Q2×{1} ∧
      ⟨⟨w,s01⟩,⟨v,q1⟩⟩∈(({reduce D-relation}(S1,t1){in alphabet}Σ)^*)
⟨proof⟩

```

For each component-2 state q_2 that appears in the state set reached by the concat ε -NFSA after reading non-empty word w , either (a) there is a non-empty suffix y_{1_k} of w such that A_1 ran from s_{01} to some $f_1 \in F_1$ while consuming the complementary prefix, and A_2 then ran from s_{02} to q_2 while consuming y_{1_k} ; or (b) $q_2 = s_{02}$ and the ε -jump into A_2 happened only at the very end, so A_1 reached some $f_1 \in F_1$ after consuming all of w .

```

lemma exec_A2_component:
  fixes S1 S2 s01 s02 t1 t2 F1 F2 Σ w v q1 Q2 q2
  defines t ≡ concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2,Σ)
  defines S ≡ concat_eNFSA_states(S1,S2)
  assumes fin:Finite(Σ)
  and A1: (S1,s01,t1,F1){is an DFSA for alphabet}Σ
  and A2: (S2,s02,t2,F2){is an DFSA for alphabet}Σ
  and wNE:w∈NELists(Σ)
  and run: ⟨⟨w,{s01,0}⟩,⟨v,{q1,0} ∪ Q2×{1}⟩⟩
           ∈ (({reduce ε-N-relation}(S,t){in alphabet}Σ)^*)
  and q1S1:q1∈S1
  and Q2S2:Q2∈Pow(S2)
  and q2Q2:q2∈Q2
  shows (∃y1_k∈NELists(Σ). ∃f1∈F1.
        ⟨⟨w,s01⟩,⟨y1_k,f1⟩⟩∈(({reduce D-relation}(S1,t1){in alphabet}Σ)^*)
    ∧
        ⟨⟨y1_k,s02⟩,⟨v,q2⟩⟩∈(({reduce D-relation}(S2,t2){in alphabet}Σ)^*)
    ∨
        (q2 = s02 ∧ (∃f1∈F1.
          ⟨⟨w,s01⟩,⟨v,f1⟩⟩∈(({reduce D-relation}(S1,t1){in alphabet}Σ)^*)))
  ⟨proof⟩

```

The language of the product ε -NFSA equals the concatenation of the two component languages.

```

lemma concat_eNFSA_language:
  assumes fin:Finite(Σ)
  and A1: (S1,s01,t1,F1){is an DFSA for alphabet}Σ
  and A2: (S2,s02,t2,F2){is an DFSA for alphabet}Σ
  and L1_def:L1 = {i∈Lists(Σ). i <-D (S1,s01,t1,F1){in alphabet}Σ}
  and L2_def:L2 = {i∈Lists(Σ). i <-D (S2,s02,t2,F2){in alphabet}Σ}
  shows {i∈Lists(Σ). i <-ε-N
        (concat_eNFSA_states(S1,S2),
         ⟨s01,0⟩,
         concat_eNFSA_trans(S1,s01,t1,F1,S2,s02,t2,F2,Σ))}

```

```

      F2×{1}){in alphabet}Σ}
    = concat(L1,L2)
⟨proof⟩

```

The concatenation of two regular languages is regular.

```

theorem concat_language_regular:
  assumes fin:Finite(Σ)
  and L1{is a regular language on}Σ
  and L2{is a regular language on}Σ
  shows concat(L1,L2) {is a regular language on}Σ
⟨proof⟩

end

```

22 Inductive sequences

```

theory InductiveSeq_ZF imports Nat_ZF_IML FiniteSeq_ZF FinOrd_ZF

begin

```

In this theory we discuss sequences defined by conditions of the form $a_0 = x$, $a_{n+1} = f(a_n)$ and similar.

22.1 Sequences defined by induction

One way of defining a sequence (that is a function $a : \mathbb{N} \rightarrow X$) is to provide the first element of the sequence and a function to find the next value when we have the current one. This is usually called "defining a sequence by induction". In this section we set up the notion of a sequence defined by induction and prove the theorems needed to use it.

First we define a helper notion of the sequence defined inductively up to a given natural number n .

```

definition
  InductiveSequenceN(x,f,n) ≡
  THE a. a: succ(n) → domain(f) ∧ a(0) = x ∧ (∀k∈n. a(succ(k)) = f(a(k)))

```

From that we define the inductive sequence on the whole set of natural numbers. Recall that in Isabelle/ZF the set of natural numbers is denoted `nat`.

```

definition
  InductiveSequence(x,f) ≡ ⋃n∈nat. InductiveSequenceN(x,f,n)

```

First we will consider the question of existence and uniqueness of finite inductive sequences. The proof is by induction and the next lemma is the $P(0)$ step. To understand the notation recall that for natural numbers in set theory we have $n = \{0, 1, \dots, n-1\}$ and $\text{succ}(n) = \{0, 1, \dots, n\}$.

lemma indseq_exun0: **assumes** A1: $f: X \rightarrow X$ **and** A2: $x \in X$
shows
 $\exists! a. a: \text{succ}(0) \rightarrow X \wedge a(0) = x \wedge (\forall k \in 0. a(\text{succ}(k)) = f(a(k)))$)
<proof>

A lemma about restricting finite sequences needed for the proof of the inductive step of the existence and uniqueness of finite inductive sequences.

lemma indseq_restrict:
assumes A1: $f: X \rightarrow X$ **and** A2: $x \in X$ **and** A3: $n \in \text{nat}$ **and**
A4: $a: \text{succ}(\text{succ}(n)) \rightarrow X \wedge a(0) = x \wedge (\forall k \in \text{succ}(n). a(\text{succ}(k)) = f(a(k)))$
and A5: $a_r = \text{restrict}(a, \text{succ}(n))$
shows
 $a_r: \text{succ}(n) \rightarrow X \wedge a_r(0) = x \wedge (\forall k \in n. a_r(\text{succ}(k)) = f(a_r(k)))$)
<proof>

Existence and uniqueness of finite inductive sequences. The proof is by induction and the next lemma is the inductive step.

lemma indseq_exun_ind:
assumes A1: $f: X \rightarrow X$ **and** A2: $x \in X$ **and** A3: $n \in \text{nat}$ **and**
A4: $\exists! a. a: \text{succ}(n) \rightarrow X \wedge a(0) = x \wedge (\forall k \in n. a(\text{succ}(k)) = f(a(k)))$
shows
 $\exists! a. a: \text{succ}(\text{succ}(n)) \rightarrow X \wedge a(0) = x \wedge$
 $(\forall k \in \text{succ}(n). a(\text{succ}(k)) = f(a(k)))$)
<proof>

The next lemma combines indseq_exun0 and indseq_exun_ind to show the existence and uniqueness of finite sequences defined by induction.

lemma indseq_exun:
assumes A1: $f: X \rightarrow X$ **and** A2: $x \in X$ **and** A3: $n \in \text{nat}$
shows
 $\exists! a. a: \text{succ}(n) \rightarrow X \wedge a(0) = x \wedge (\forall k \in n. a(\text{succ}(k)) = f(a(k)))$
<proof>

We are now ready to prove the main theorem about finite inductive sequences.

theorem fin_indseq_props:
assumes A1: $f: X \rightarrow X$ **and** A2: $x \in X$ **and** A3: $n \in \text{nat}$ **and**
A4: $a = \text{InductiveSequenceN}(x, f, n)$
shows
 $a: \text{succ}(n) \rightarrow X$
 $a(0) = x$
 $\forall k \in n. a(\text{succ}(k)) = f(a(k))$
<proof>

Since we have uniqueness we can show the inverse of fin_indseq_props: a sequence that satisfies the inductive sequence properties listed there is the inductively defined sequence.

lemma is_fin_indseq:

assumes $n \in \text{nat}$ $f: X \rightarrow X$ $x \in X$ **and**
 $a: \text{succ}(n) \rightarrow X$ $a(0) = x$ $\forall k \in n. a(\text{succ}(k)) = f(a(k))$
shows $a = \text{InductiveSequenceN}(x, f, n)$
<proof>

A corollary about the domain of a finite inductive sequence.

corollary `fin_indseq_domain`:
assumes $A1: f: X \rightarrow X$ **and** $A2: x \in X$ **and** $A3: n \in \text{nat}$
shows $\text{domain}(\text{InductiveSequenceN}(x, f, n)) = \text{succ}(n)$
<proof>

The collection of finite sequences defined by induction is consistent in the sense that the restriction of the sequence defined on a larger set to the smaller set is the same as the sequence defined on the smaller set.

lemma `indseq_consistent`: **assumes** $A1: f: X \rightarrow X$ **and** $A2: x \in X$ **and**
 $A3: i \in \text{nat}$ $j \in \text{nat}$ **and** $A4: i \subseteq j$
shows
 $\text{restrict}(\text{InductiveSequenceN}(x, f, j), \text{succ}(i)) = \text{InductiveSequenceN}(x, f, i)$
<proof>

For any two natural numbers one of the corresponding inductive sequences is contained in the other.

lemma `indseq_subsets`: **assumes** $A1: f: X \rightarrow X$ **and** $A2: x \in X$ **and**
 $A3: i \in \text{nat}$ $j \in \text{nat}$ **and**
 $A4: a = \text{InductiveSequenceN}(x, f, i)$ $b = \text{InductiveSequenceN}(x, f, j)$
shows $a \subseteq b \vee b \subseteq a$
<proof>

The inductive sequence generated by applying a function 0 times is just the singleton list containing the starting point.

lemma `indseq_empty`: **assumes** $f: X \rightarrow X$ $x \in X$
shows
 $\text{InductiveSequenceN}(x, f, 0) : \{0\} \rightarrow X$
 $\text{InductiveSequenceN}(x, f, 0) = \{ \langle 0, x \rangle \}$
<proof>

The tail of an inductive sequence generated by f and started from x is the same as the inductive sequence started from $f(x)$.

lemma `indseq_tail`: **assumes** $n \in \text{nat}$ $f: X \rightarrow X$ $x \in X$
shows $\text{Tail}(\text{InductiveSequenceN}(x, f, \text{succ}(n))) = \text{InductiveSequenceN}(f(x), f, n)$
<proof>

The first theorem about properties of infinite inductive sequences: inductive sequence is indeed a sequence (i.e. a function on the set of natural numbers).

theorem `indseq_seq`: **assumes** $A1: f: X \rightarrow X$ **and** $A2: x \in X$
shows $\text{InductiveSequence}(x, f) : \text{nat} \rightarrow X$

<proof>

Restriction of an inductive sequence to a finite domain is the corresponding finite inductive sequence.

lemma indseq_restr_eq:
 assumes A1: $f: X \rightarrow X$ **and** A2: $x \in X$ **and** A3: $n \in \text{nat}$
 shows
 $\text{restrict}(\text{InductiveSequence}(x, f), \text{succ}(n)) = \text{InductiveSequenceN}(x, f, n)$
<proof>

The first element of the inductive sequence starting at x and generated by f is indeed x .

theorem indseq_valat0: **assumes** A1: $f: X \rightarrow X$ **and** A2: $x \in X$
 shows $\text{InductiveSequence}(x, f)(0) = x$
<proof>

An infinite inductive sequence satisfies the inductive relation that defines it.

theorem indseq_vals:
 assumes A1: $f: X \rightarrow X$ **and** A2: $x \in X$ **and** A3: $n \in \text{nat}$
 shows
 $\text{InductiveSequence}(x, f)(\text{succ}(n)) = f(\text{InductiveSequence}(x, f)(n))$
<proof>

22.2 Images of inductive sequences

In this section we consider the properties of sets that are images of inductive sequences, that is are of the form $\{f^{(n)}(x) : n \in N\}$ for some x in the domain of f , where $f^{(n)}$ denotes the n 'th iteration of the function f . For a function $f: X \rightarrow X$ and a point $x \in X$ such set is sometimes called the orbit of x generated by f .

The basic properties of orbits.

theorem ind_seq_image: **assumes** A1: $f: X \rightarrow X$ **and** A2: $x \in X$ **and**
 A3: $A = \text{InductiveSequence}(x, f)(\text{nat})$
 shows $x \in A$ **and** $\forall y \in A. f(y) \in A$
<proof>

22.3 Subsets generated by a binary operation

In algebra we often talk about sets "generated" by an element, that is sets of the form (in multiplicative notation) $\{a^n | n \in Z\}$. This is related to a general notion of "power" (as in $a^n = a \cdot a \cdot \dots \cdot a$) or multiplicity $n \cdot a = a + a + \dots + a$. The intuitive meaning of such notions is obvious, but we need to do some work to be able to use it in the formalized setting. This section is devoted to sequences that are created by repeatedly applying a binary operation with the second argument fixed to some constant.

Basic properties of sets generated by binary operations.

theorem binop_gen_set:
 assumes A1: $f: X \times Y \rightarrow X$ and A2: $x \in X \quad y \in Y$ and
 A3: $a = \text{InductiveSequence}(x, \text{Fix2ndVar}(f, y))$
 shows
 $a : \text{nat} \rightarrow X$
 $a(\text{nat}) \in \text{Pow}(X)$
 $x \in a(\text{nat})$
 $\forall z \in a(\text{nat}). \text{Fix2ndVar}(f, y)(z) \in a(\text{nat})$
<proof>

A simple corollary to the theorem `binop_gen_set`: a set that contains all iterations of the application of a binary operation exists.

lemma binop_gen_set_ex: assumes A1: $f: X \times Y \rightarrow X$ and A2: $x \in X \quad y \in Y$
 shows $\{A \in \text{Pow}(X). x \in A \wedge (\forall z \in A. f\langle z, y \rangle \in A)\} \neq 0$
<proof>

A more general version of `binop_gen_set` where the generating binary operation acts on a larger set.

theorem binop_gen_set1: assumes A1: $f: X \times Y \rightarrow X$ and
 A2: $X_1 \subseteq X$ and A3: $x \in X_1 \quad y \in Y$ and
 A4: $\forall t \in X_1. f\langle t, y \rangle \in X_1$ and
 A5: $a = \text{InductiveSequence}(x, \text{Fix2ndVar}(\text{restrict}(f, X_1 \times Y), y))$
 shows
 $a : \text{nat} \rightarrow X_1$
 $a(\text{nat}) \in \text{Pow}(X_1)$
 $x \in a(\text{nat})$
 $\forall z \in a(\text{nat}). \text{Fix2ndVar}(f, y)(z) \in a(\text{nat})$
 $\forall z \in a(\text{nat}). f\langle z, y \rangle \in a(\text{nat})$
<proof>

A generalization of `binop_gen_set_ex` that applies when the binary operation acts on a larger set. This is used in our Metamath translation to prove the existence of the set of real natural numbers. Metamath defines the real natural numbers as the smallest set that contains 1 and is closed with respect to operation of adding 1.

lemma binop_gen_set_ex1: assumes A1: $f: X \times Y \rightarrow X$ and
 A2: $X_1 \subseteq X$ and A3: $x \in X_1 \quad y \in Y$ and
 A4: $\forall t \in X_1. f\langle t, y \rangle \in X_1$
 shows $\{A \in \text{Pow}(X_1). x \in A \wedge (\forall z \in A. f\langle z, y \rangle \in A)\} \neq 0$
<proof>

22.4 Inductive sequences with changing generating function

A seemingly more general form of a sequence defined by induction is a sequence generated by the difference equation $x_{n+1} = f_n(x_n)$ where $n \mapsto f_n$ is a given sequence of functions such that each maps X into itself. For

example when $f_n(x) := x + x_n$ then the equation $S_{n+1} = f_n(S_n)$ describes the sequence $n \mapsto S_n = s_0 + \sum_{i=0}^n x_i$, i.e. the sequence of partial sums of the sequence $\{s_0, x_0, x_1, x_2, \dots\}$.

The situation where the function that we iterate changes with n can be derived from the simpler case if we define the generating function appropriately. Namely, we replace the generating function in the definitions of `InductiveSequenceN` by the function $f : X \times n \rightarrow X \times n$, $f\langle x, k \rangle = \langle f_k(x), k + 1 \rangle$ if $k < n$, $\langle f_k(x), k \rangle$ otherwise. The first notion defines the expression we will use to define the generating function. To understand the notation recall that in standard Isabelle/ZF for a pair $s = \langle x, n \rangle$ we have $\text{fst}(s) = x$ and $\text{snd}(s) = n$.

definition

```
StateTransfFunNMeta(F,n,s)  $\equiv$ 
  if (snd(s)  $\in$  n) then  $\langle F(\text{snd}(s))(\text{fst}(s)), \text{succ}(\text{snd}(s)) \rangle$  else s
```

Then we define the actual generating function on sets of pairs from $X \times \{0, 1, \dots, n\}$.

definition

```
StateTransfFunN(X,F,n)  $\equiv$   $\{\langle s, \text{StateTransfFunNMeta}(F,n,s) \rangle. s \in X \times \text{succ}(n)\}$ 
```

Having the generating function we can define the expression that we can use to define the inductive sequence generates.

definition

```
StatesSeq(x,X,F,n)  $\equiv$ 
  InductiveSequenceN( $\langle x, 0 \rangle$ , StateTransfFunN(X,F,n),n)
```

Finally we can define the sequence given by a initial point x , and a sequence F of n functions.

definition

```
InductiveSeqVarFN(x,X,F,n)  $\equiv$   $\{\langle k, \text{fst}(\text{StatesSeq}(x,X,F,n)(k)) \rangle. k \in \text{succ}(n)\}$ 
```

The state transformation function (`StateTransfFunN` is a function that transforms $X \times n$ into itself.

lemma `state_trans_fun`: **assumes** $A1: n \in \text{nat}$ **and** $A2: F: n \rightarrow (X \rightarrow X)$

shows $\text{StateTransfFunN}(X,F,n): X \times \text{succ}(n) \rightarrow X \times \text{succ}(n)$

<proof>

We can apply `fin_indseq_props` to the sequence used in the definition of `InductiveSeqVarFN` to get the properties of the sequence of states generated by the `StateTransfFunN`.

lemma `states_seq_props`:

assumes $A1: n \in \text{nat}$ **and** $A2: F: n \rightarrow (X \rightarrow X)$ **and** $A3: x \in X$ **and**

$A4: b = \text{StatesSeq}(x,X,F,n)$

shows

$b : \text{succ}(n) \rightarrow X \times \text{succ}(n)$

$b(0) = \langle x, 0 \rangle$
 $\forall k \in \text{succ}(n). \text{snd}(b(k)) = k$
 $\forall k \in n. b(\text{succ}(k)) = \langle F(k)(\text{fst}(b(k))), \text{succ}(k) \rangle$
 $\langle \text{proof} \rangle$

Basic properties of finite sequences defined by equation $x_{n+1} = f_n(x_n)$.

theorem fin_indseq_var_f_props:
assumes A1: $n \in \text{nat}$ **and** A2: $x \in X$ **and** A3: $F: n \rightarrow (X \rightarrow X)$ **and**
A4: $a = \text{InductiveSeqVarFN}(x, X, F, n)$
shows
 $a: \text{succ}(n) \rightarrow X$
 $a(0) = x$
 $\forall k \in n. a(\text{succ}(k)) = F(k)(a(k))$
 $\langle \text{proof} \rangle$

Uniqueness lemma for sequences generated by equation $x_{n+1} = f_n(x_n)$:

lemma fin_indseq_var_f_uniq: **assumes** $n \in \text{nat}$ $x \in X$ $F: n \rightarrow (X \rightarrow X)$
and $a: \text{succ}(n) \rightarrow X$ $a(0) = x$ $\forall k \in n. a(\text{succ}(k)) = (F(k))(a(k))$
and $b: \text{succ}(n) \rightarrow X$ $b(0) = x$ $\forall k \in n. b(\text{succ}(k)) = (F(k))(b(k))$
shows $a = b$
 $\langle \text{proof} \rangle$

A sequence that has the properties of sequences generated by equation $x_{n+1} = f_n(x_n)$ must be the one generated by this equation.

theorem is_fin_indseq_var_f: **assumes** $n \in \text{nat}$ $x \in X$ $F: n \rightarrow (X \rightarrow X)$
and $a: \text{succ}(n) \rightarrow X$ $a(0) = x$ $\forall k \in n. a(\text{succ}(k)) = (F(k))(a(k))$
shows $a = \text{InductiveSeqVarFN}(x, X, F, n)$
 $\langle \text{proof} \rangle$

Analogously to the case of a constant generation function we can define infinite sequence generated by a an infinite sequence of generating functions by taking the union of the finite ones. In the definition $x \in X$ is the starting point, X is the common domain of the generating functions and F is a sequence of functions mapping X to itself.

definition

$$\text{InductiveSeqVarF}(x, X, F) \equiv \bigcup_{n \in \text{nat}}. \text{InductiveSeqVarFN}(x, X, \text{restrict}(F, n), n)$$

Suppose we are given a sequence of functions $S = \{f_n\}_{n \in \mathbb{N}}$ such that $f_n : \{0, \dots, n\} \rightarrow X$ and f_n 's are compatible in the sense that for all $n \in \mathbb{N}$ and $k < n$ the restriction of f_n to $\{0, \dots, k\}$ is the same as f_k . Then the union $f = \bigcup_{n \in \mathbb{N}} f_n$ is a sequence that maps $\mathbb{N}$ to X and for $n \in \mathbb{N}$ we have $f(n) = f_n(n)$.

lemma compatible_seq: **assumes** $\forall n \in \text{nat}.$
 $S(n): \text{succ}(n) \rightarrow X \wedge (\forall k \in n. \text{restrict}(S(n), \text{succ}(k)) = S(k))$
defines $f \equiv \bigcup \{S(n). n \in \text{nat}\}$
shows
 $f: \text{nat} \rightarrow X$

$\forall n \in \text{nat}. S(n) = \text{restrict}(f, \text{succ}(n))$
 $\forall n \in \text{nat}. \forall m \in \text{succ}(n). f(m) = (S(n))(m)$
<proof>

A consistency condition: if we make the sequence of generating functions shorter, then we get a shorter inductive sequence with the same values as in the original sequence.

lemma `fin_indseq_var_f_restrict`: **assumes**
 $A1: n \in \text{nat} \quad i \in \text{nat} \quad x \in X \quad F: n \rightarrow (X \rightarrow X) \quad G: i \rightarrow (X \rightarrow X)$
and $A2: i \subseteq n$ **and** $A3: \forall j \in i. G(j) = F(j)$ **and** $A4: k \in \text{succ}(i)$
shows $\text{InductiveSeqVarFN}(x, X, G, i)(k) = \text{InductiveSeqVarFN}(x, X, F, n)(k)$
<proof>

The sequence of finite sequences with values defined by the `InductiveSeqVarFN` and a generating sequence of functions $F: \mathbb{N} \rightarrow (X \rightarrow X)$ satisfies the assumptions of lemma `compatible_seq`.

lemma `inductive_seq_var_compat`:
assumes $n \in \text{nat} \quad x \in X \quad \mathcal{F}: \text{nat} \rightarrow (X \rightarrow X)$
defines $S \equiv \{\langle n, \text{InductiveSeqVarFN}(x, X, \text{restrict}(\mathcal{F}, n), n) \rangle. n \in \text{nat}\}$
shows $S(n): \text{succ}(n) \rightarrow X$ **and** $\forall k \in n. \text{restrict}(S(n), \text{succ}(k)) = S(k)$
<proof>

If $\mathcal{F}: \mathbb{N} \rightarrow (X \rightarrow X)$ is an (infinite) sequence of functions mapping X into itself and $x \in X$ then $s = \text{InductiveSeqVarF}(x, X, \mathcal{F})$ is a well defined sequence of elements of X that satisfies the recursive relation $s_{n+1} = \mathcal{F}_n(s_n)$.

theorem `indseq_var_seq`: **assumes** $x \in X \quad \mathcal{F}: \text{nat} \rightarrow (X \rightarrow X)$
defines $s \equiv \text{InductiveSeqVarF}(x, X, \mathcal{F})$
shows $s: \text{nat} \rightarrow X$ $s(0) = x$ **and** $\forall n \in \text{nat}. s(n \# + 1) = (\mathcal{F}(n))(s(n))$
<proof>

22.5 The Pascal's triangle

One possible application of the inductive sequences is to define the Pascal's triangle. The Pascal's triangle can be defined directly as $P_{n,k} = \binom{n}{k} = \frac{n!}{k!(n-k)!}$ for $n \geq k \geq 0$. Formalizing this definition (or explaining to a 10-years old) is quite difficult as it depends on the definition of factorial and some facts about factorizing natural numbers needed to show that the quotient in $\frac{n!}{k!(n-k)!}$ is always a natural number. Another approach uses induction and the property that each number in the array is the sum of the two numbers directly above it.

To shorten the definition of the function generating the Pascal's triangle we first define expression for the k 'th element in the row following given row r . The rows are represented as lists, i.e. functions $r: n \rightarrow \mathbb{N}$ (recall that for natural numbers we have $n = \{0, 1, 2, \dots, n-1\}$). The value of the next row is 1 at the beginning and equals $r(k-1) + r(k)$ otherwise. A careful

reader might wonder why we do not require the values to be 1 on the right boundary of the Pascal's triangle. We are able to show this as a theorem (see `binom_right_boundary` below) using the fact that in Isabelle/ZF the value of a function on an argument that is outside of the domain is the empty set, which is the same as zero of natural numbers.

definition

`BinomElem(r,k)  $\equiv$  if k=0 then 1 else r(pred(k)) #+ r(k)`

Next we define a function that takes a row in a Pascal's triangle and returns the next row.

definition

`GenBinom  $\equiv$   $\{\langle r, \{ \langle k, \text{BinomElem}(r,k) \rangle. k \in \text{succ}(\text{domain}(r)) \} \rangle. r \in \text{NELists}(\text{nat}) \}$`

The function generating rows of the Pascal's triangle is indeed a function that maps nonempty lists of natural numbers into nonempty lists of natural numbers.

lemma `gen_binom_fun: shows GenBinom: NELists(nat)  $\rightarrow$  NELists(nat)`
<proof>

The value of the function `GenBinom` at a nonempty list r is a list of length one greater than the length of r .

lemma `gen_binom_fun_val: assumes n $\in$ nat r:succ(n) $\rightarrow$ nat`
`shows GenBinom(r):succ(succ(n))  $\rightarrow$  nat`
<proof>

Now we are ready to define the Pascal's triangle as the inductive sequence that starts from a singleton list $0 \mapsto 1$ and is generated by iterations of the `GenBinom` function.

definition

`PascalTriangle  $\equiv$  InductiveSequence( $\{\langle 0, 1 \rangle\}$ , GenBinom)`

The singleton list containing 1 (i.e. the starting point of the inductive sequence that defines the `PascalTriangle`) is a finite list and the `PascalTriangle` is a sequence (an infinite list) of nonempty lists of natural numbers.

lemma `pascal_sequence:`
`shows  $\{\langle 0, 1 \rangle\} \in \text{NELists}(\text{nat})$  and PascalTriangle: nat  $\rightarrow$  NELists(nat)`
<proof>

The `GenBinom` function creates the next row of the Pascal's triangle from the previous one.

lemma `binom_gen: assumes n $\in$ nat`
`shows PascalTriangle(succ(n)) = GenBinom(PascalTriangle(n))`
<proof>

The n 'th row of the Pascal's triangle is a list of $n + 1$ natural numbers.

lemma `pascal_row_list:`

assumes $n \in \text{nat}$ **shows** $\text{PascalTriangle}(n) : \text{succ}(n) \rightarrow \text{nat}$
<proof>

In our approach the Pascal's triangle is a list of lists. The value at index $n \in \mathbb{N}$ is a list of length $n + 1$ (see `pascal_row_list` above). Hence, the largest index in the domain of this list is n . However, we can still show that the value of that list at index $n + 1$ is 0, because in Isabelle/ZF (as well as in Metamath) the value of a function at a point outside of the domain is the empty set, which happens to be the same as the natural number 0.

lemma `pascal_val_beyond`: **assumes** $n \in \text{nat}$
shows $(\text{PascalTriangle}(n))(\text{succ}(n)) = 0$
<proof>

For $n > 0$ the Pascal's triangle values at (n, k) are given by the `BinomElem` expression.

lemma `pascal_row_val`: **assumes** $n \in \text{nat}$ $k \in \text{succ}(\text{succ}(n))$
shows $(\text{PascalTriangle}(\text{succ}(n)))(k) = \text{BinomElem}(\text{PascalTriangle}(n), k)$
<proof>

The notion that will actually be used is the binomial coefficient $\binom{n}{k}$ which we define as the value at the right place of the Pascal's triangle.

definition

$$\text{Binom}(n, k) \equiv (\text{PascalTriangle}(n))(k)$$

Entries in the Pascal's triangle are natural numbers. Since in Isabelle/ZF the value of a function at a point that is outside of the domain is the empty set (which is the same as zero of natural numbers) we do not need any assumption on k .

lemma `binom_in_nat`: **assumes** $n \in \text{nat}$ **shows** $\text{Binom}(n, k) \in \text{nat}$
<proof>

The top of the Pascal's triangle is equal to 1 (i.e. $\binom{0}{0} = 1$). This is an easy fact that it is useful to have handy as it is at the start of a couple of inductive arguments.

lemma `binom_zero_zero`: **shows** $\text{Binom}(0, 0) = 1$
<proof>

The binomial coefficients are 1 on the left boundary of the Pascal's triangle.

theorem `binom_left_boundary`: **assumes** $n \in \text{nat}$ **shows** $\text{Binom}(n, 0) = 1$
<proof>

The main recursive property of binomial coefficients: each number in the $\binom{n}{k}$, $n > 0, 0 \neq k \leq n$ array (i.e. the Pascal's triangle except the top) is the sum of the two numbers directly above it. The statement looks like it has an off-by-one error in the assumptions, but it's ok and needed later.

theorem `binom_prop`: **assumes** $n \in \text{nat}$ $k \leq n$ $\#$ 1 $k \neq 0$

shows $\text{Binom}(n \# + 1, k) = \text{Binom}(n, k \# - 1) \# + \text{Binom}(n, k)$
 $\langle \text{proof} \rangle$

A version `binom_prop` where we write $k + 1$ instead of k .

lemma `binom_prop2`: **assumes** $n \in \text{nat}$ $k \in n \# + 1$
shows $\text{Binom}(n \# + 1, k \# + 1) = \text{Binom}(n, k \# + 1) \# + \text{Binom}(n, k)$
 $\langle \text{proof} \rangle$

A special case of `binom_prop` when $n = k + 1$ that helps with the induction step in the proof that the binomial coefficient are 1 on the right boundary of the Pascal's triangle.

lemma `binom_prop1`: **assumes** $n \in \text{nat}$
shows $\text{Binom}(n \# + 1, n \# + 1) = \text{Binom}(n, n)$
 $\langle \text{proof} \rangle$

The binomial coefficients are 1 on the right boundary of the Pascal's triangle.

theorem `binom_right_boundary`: **assumes** $n \in \text{nat}$ **shows** $\text{Binom}(n, n) = 1$
 $\langle \text{proof} \rangle$

22.6 Monotonic sequences

In this section we consider monotonic sequences, i.e. sequences $\{x\}_n$ such that $x_{n+1} \leq x_n$ for all $n \in \mathbb{N}$ (a decreasing sequence) or $x_n \leq x_{n+1}$ for all $n \in \mathbb{N}$ (an increasing sequence). We will mostly use the raw set theory notation $\langle x_{n+1}, x_n \rangle \in r$ (or $\langle x_n, x_{n+1} \rangle \in r$) instead of the $\leq$ sign to have the relevant relation explicit.

A decreasing sequence is a function $s : \mathbb{N} \rightarrow X$ defined on the set natural numbers such that $\langle s_{n+1}, s_n \rangle \in r$ for all $n \in \mathbb{N}$.

definition

$\text{IsDecreasingSeq}(X, r, s) \equiv s : \text{nat} \rightarrow X \wedge (\forall n \in \text{nat}. \langle s(n \# + 1), s(n) \rangle \in r)$

An increasing sequence is a function $s : \mathbb{N} \rightarrow X$ defined on the set natural numbers such that $\langle s_n, s_{n+1} \rangle \in r$ for all $n \in \mathbb{N}$.

definition

$\text{IsIncreasingSeq}(X, r, s) \equiv s : \text{nat} \rightarrow X \wedge (\forall n \in \text{nat}. \langle s(n), s(n \# + 1) \rangle \in r)$

If r is a preorder relation and s is a decreasing sequence then all terms that come after s_n are less or equal than s_n .

lemma

`decreasing_seq_mono`:

assumes $\text{IsPreorder}(X, r)$ $\text{IsDecreasingSeq}(X, r, s)$ $n \in \text{nat}$ $k \in \text{nat}$
shows $\langle s(n \# + k), s(n) \rangle \in r$
 $\langle \text{proof} \rangle$

If r is a preorder relation and s is an increasing sequence then all terms that come after s_n are greater or equal than s_n .

```

lemma increasing_seq_mono:
  assumes IsPreorder(X,r) IsIncreasingSeq(X,r,s) n∈nat k∈nat
  shows ⟨s(n),s(n #+ k)⟩ ∈ r
  ⟨proof⟩

```

Another formulation of `decreasing_seq_mono`: if a sequence s is decreasing, m is a natural number and $n \leq m$ then $s(m) \leq s(n)$.

```

lemma decreasing_seq_mono1:
  assumes IsPreorder(X,r) IsDecreasingSeq(X,r,s) m∈nat n≤m
  shows ⟨s(m),s(n)⟩ ∈ r
  ⟨proof⟩

```

Another formulation of `increasing_seq_mono`: if a sequence s is increasing, m is a natural number and $n \leq m$ then $s(n) \leq s(m)$.

```

lemma increasing_seq_mono1:
  assumes IsPreorder(X,r) IsIncreasingSeq(X,r,s) m∈nat n≤m
  shows ⟨s(n),s(m)⟩ ∈ r
  ⟨proof⟩

```

If a sequence is decreasing then the preorder is total on the sequence's image.

```

lemma decr_seq_total:
  assumes IsPreorder(X,r) IsDecreasingSeq(X,r,s)
  shows r {is total on} s(nat)
  ⟨proof⟩

```

If a sequence is increasing then the preorder is total on the sequence's image.

```

lemma incr_seq_total:
  assumes IsPreorder(X,r) IsIncreasingSeq(X,r,s)
  shows r {is total on} s(nat)
  ⟨proof⟩

```

end

23 Enumerations

```

theory Enumeration_ZF imports NatOrder_ZF FiniteSeq_ZF FinOrd_ZF

```

```

begin

```

Suppose r is a linear order on a set A that has n elements, where $n \in \mathbb{N}$. In the `FinOrd_ZF` theory we prove a theorem stating that there is a unique order isomorphism between $n = \{0, 1, \dots, n-1\}$ (with natural order) and A . Another way of stating that is that there is a unique way of counting the elements of A in the order increasing according to relation r . Yet another way of stating the same thing is that there is a unique sorted list of elements of A . We will call this list the `Enumeration` of A .

23.1 Enumerations: definition and notation

In this section we introduce the notion of enumeration and define a proof context (a "locale" in Isabelle terms) that sets up the notation for writing about enumerations.

We define enumeration as the only order isomorphism between a set A and the number of its elements. We are using the formula $\bigcup\{x\} = x$ to extract the only element from a singleton. Le is the (natural) order on natural numbers, defined in Nat_ZF theory in the standard Isabelle library.

definition

$$\text{Enumeration}(A,r) \equiv \bigcup \text{ord_iso}(|A|, \text{Le}, A, r)$$

To set up the notation we define a locale `enums`. In this locale we will assume that r is a linear order on some set X . In most applications this set will be just the set of natural numbers. Standard Isabelle uses $\leq$ to denote the "less or equal" relation on natural numbers. We will use the $\leq$ symbol to denote the relation r . Those two symbols usually look the same in the presentation, but they are different in the source. To shorten the notation the enumeration $\text{Enumeration}(A,r)$ will be denoted as $\sigma(A)$. Similarly as in the `Semigroup` theory we will write $a \leftarrow x$ for the result of appending an element x to the finite sequence (list) a . Finally, $a \sqcup b$ will denote the concatenation of the lists a and b .

locale `enums` =

```

fixes X r
assumes linord: IsLinOrder(X,r)

fixes ler (infix  $\leq$  70)
defines ler_def[simp]:  $x \leq y \equiv \langle x,y \rangle \in r$ 

fixes  $\sigma$ 
defines  $\sigma\_def$  [simp]:  $\sigma(A) \equiv \text{Enumeration}(A,r)$ 

fixes append (infix  $\leftarrow$  72)
defines append_def[simp]:  $a \leftarrow x \equiv \text{Append}(a,x)$ 

fixes concat (infixl  $\sqcup$  69)
defines concat_def[simp]:  $a \sqcup b \equiv \text{Concat}(a,b)$ 

```

23.2 Properties of enumerations

In this section we prove basic facts about enumerations.

A special case of the existence and uniqueness of the order isomorphism for finite sets when the first set is a natural number.

lemma (in `enums`) `ord_iso_nat_fin`:


```

assumes A ∈ FinPow(X) and n ∈ nat and A ≈ n
shows ∃!f. f ∈ ord_iso(n,Le,A,r)
⟨proof⟩

```

An enumeration is an order isomorphism, a bijection, and a list.

```

lemma (in enums) enum_props: assumes A ∈ FinPow(X)
shows
  σ(A) ∈ ord_iso(|A|,Le, A,r)
  σ(A) ∈ bij(|A|,A)
  σ(A) : |A| → A
⟨proof⟩

```

A corollary from enum_props. Could have been attached as another assertion, but this slows down verification of some other proofs.

```

lemma (in enums) enum_fun: assumes A ∈ FinPow(X)
shows σ(A) : |A| → X
⟨proof⟩

```

If a list is an order isomorphism then it must be the enumeration.

```

lemma (in enums) ord_iso_enum: assumes A1: A ∈ FinPow(X) and
  A2: n ∈ nat and A3: f ∈ ord_iso(n,Le,A,r)
shows f = σ(A)
⟨proof⟩

```

What is the enumeration of the empty set?

```

lemma (in enums) empty_enum: shows σ(0) = 0
⟨proof⟩

```

Adding a new maximum to a set appends it to the enumeration.

```

lemma (in enums) enum_append:
  assumes A1: A ∈ FinPow(X) and A2: b ∈ X-A and
  A3: ∀a∈A. a≤b
shows σ(A ∪ {b}) = σ(A)↪ b
⟨proof⟩

```

What is the enumeration of a singleton?

```

lemma (in enums) enum_singleton:
  assumes A1: x∈X shows σ({x}): 1 → X and σ({x})(0) = x
⟨proof⟩

```

end

24 Folding in ZF

```

theory Fold_ZF imports InductiveSeq_ZF

```

begin

Suppose we have a binary operation $P : X \times X \rightarrow X$ written multiplicatively as $P\langle x, y \rangle = x \cdot y$. In informal mathematics we can take a sequence $\{x_k\}_{k \in 0..n}$ of elements of X and consider the product $x_0 \cdot x_1 \cdot \dots \cdot x_n$. To do the same thing in formalized mathematics we have to define precisely what is meant by that "...". The definition we want to use is based on the notion of sequence defined by induction discussed in `InductiveSeq_ZF`. We don't really want to derive the terminology for this from the word "product" as that would tie it conceptually to the multiplicative notation. This would be awkward when we want to reuse the same notions to talk about sums like $x_0 + x_1 + \dots + x_n$. In functional programming there is something called "fold". Namely for a function f , initial point a and list $[b, c, d]$ the expression `fold(f, a, [b,c,d])` is defined to be $f(f(f(a,b),c),d)$ (in Haskell something like this is called `foldl`). If we write f in multiplicative notation we get $a \cdot b \cdot c \cdot d$, so this is exactly what we need. The notion of folds in functional programming is actually much more general than what we need here (not that I know anything about that). In this theory file we just make a slight generalization and talk about folding a list with a binary operation $f : X \times Y \rightarrow X$ with X not necessarily the same as Y .

24.1 Folding in ZF

Suppose we have a binary operation $f : X \times Y \rightarrow X$. Then every $y \in Y$ defines a transformation of X defined by $T_y(x) = f\langle x, y \rangle$. In `IsarMathLib` such transformation is called as `Fix2ndVar(f,y)`. Using this notion, given a function $f : X \times Y \rightarrow X$ and a sequence $y = \{y_k\}_{k \in N}$ of elements of Y we can get a sequence of transformations of X . This is defined in `Seq2TransSeq` below. Then we use that sequence of transformations to define the sequence of partial folds (called `FoldSeq`) by means of `InductiveSeqVarFN` (defined in `InductiveSeq_ZF` theory) which implements the inductive sequence determined by a starting point and a sequence of transformations. Finally, we define the fold of a sequence as the last element of the sequence of the partial folds.

Definition that specifies how to convert a sequence a of elements of Y into a sequence of transformations of X , given a binary operation $f : X \times Y \rightarrow X$.

definition

$$\text{Seq2TrSeq}(f,a) \equiv \{\langle k, \text{Fix2ndVar}(f,a(k)) \rangle \mid k \in \text{domain}(a)\}$$

Definition of a sequence of partial folds.

definition

$$\begin{aligned} \text{FoldSeq}(f,x,a) &\equiv \\ &\text{InductiveSeqVarFN}(x, \text{fstDom}(f), \text{Seq2TrSeq}(f,a), \text{domain}(a)) \end{aligned}$$

Definition of a fold.

definition

$\text{Fold}(f, x, a) \equiv \text{Last}(\text{FoldSeq}(f, x, a))$

If X is a set with a binary operation $f : X \times Y \rightarrow X$ then $\text{Seq2TransSeqN}(f, a)$ converts a sequence a of elements of Y into the sequence of corresponding transformations of X .

lemma seq2trans_seq_props:

assumes A1: $n \in \text{nat}$ **and** A2: $f : X \times Y \rightarrow X$ **and** A3: $a : n \rightarrow Y$ **and**
A4: $T = \text{Seq2TrSeq}(f, a)$

shows

$T : n \rightarrow (X \rightarrow X)$ **and**

$\forall k \in n. \forall x \in X. (T(k))(x) = f(x, a(k))$

<proof>

Basic properties of the sequence of partial folds of a sequence $a = \{y_k\}_{k \in \{0, \dots, n\}}$.

theorem fold_seq_props:

assumes A1: $n \in \text{nat}$ **and** A2: $f : X \times Y \rightarrow X$ **and**

A3: $y : n \rightarrow Y$ **and** A4: $x \in X$ **and** A5: $Y \neq 0$ **and**

A6: $F = \text{FoldSeq}(f, x, y)$

shows

$F : \text{succ}(n) \rightarrow X$

$F(0) = x$ **and**

$\forall k \in n. F(\text{succ}(k)) = f(F(k), y(k))$

<proof>

A consistency condition: if we make the list shorter, then we get a shorter sequence of partial folds with the same values as in the original sequence. This can be proven as a special case of `fin_indseq_var_f_restrict` but a proof using `fold_seq_props` and induction turns out to be shorter.

lemma foldseq_restrict: assumes

$n \in \text{nat} \quad k \in \text{succ}(n)$ **and**

$i \in \text{nat} \quad f : X \times Y \rightarrow X \quad a : n \rightarrow Y \quad b : i \rightarrow Y$ **and**

$n \subseteq i \quad \forall j \in n. b(j) = a(j) \quad x \in X \quad Y \neq 0$

shows $\text{FoldSeq}(f, x, b)(k) = \text{FoldSeq}(f, x, a)(k)$

<proof>

A special case of `foldseq_restrict` when the longer sequence is created from the shorter one by appending one element.

corollary fold_seq_append:

assumes $n \in \text{nat} \quad f : X \times Y \rightarrow X \quad a : n \rightarrow Y$ **and**

$x \in X \quad k \in \text{succ}(n) \quad y \in Y$

shows $\text{FoldSeq}(f, x, \text{Append}(a, y))(k) = \text{FoldSeq}(f, x, a)(k)$

<proof>

What we really will be using is the notion of the fold of a sequence, which we define as the last element of (inductively defined) sequence of partial folds. The next theorem lists some properties of the product of the fold operation.

theorem fold_props:
 assumes A1: $n \in \text{nat}$ and
 A2: $f : X \times Y \rightarrow X$ $a : n \rightarrow Y$ $x \in X$ $Y \neq 0$
 shows
 $\text{Fold}(f, x, a) = \text{FoldSeq}(f, x, a)(n)$ and
 $\text{Fold}(f, x, a) \in X$
<proof>

A corner case: what happens when we fold an empty list?

theorem fold_empty: assumes A1: $f : X \times Y \rightarrow X$ and
 A2: $a : 0 \rightarrow Y$ $x \in X$ $Y \neq 0$
 shows $\text{Fold}(f, x, a) = x$
<proof>

The next theorem tells us what happens to the fold of a sequence when we add one more element to it.

theorem fold_append:
 assumes A1: $n \in \text{nat}$ and A2: $f : X \times Y \rightarrow X$ and
 A3: $a : n \rightarrow Y$ and A4: $x \in X$ and A5: $y \in Y$
 shows
 $\text{FoldSeq}(f, x, \text{Append}(a, y))(n) = \text{Fold}(f, x, a)$ and
 $\text{Fold}(f, x, \text{Append}(a, y)) = f(\text{Fold}(f, x, a), y)$
<proof>

Another way of formulating information contained in `fold_append` is to start with a longer sequence $a : n + 1 \rightarrow X$ and then detach the last element from it. This provides an identity between the fold of the longer sequence and the value of the folding function on the fold of the shorter sequence and the last element of the longer one.

lemma fold_detach_last:
 assumes $n \in \text{nat}$ $f : X \times Y \rightarrow X$ $x \in X$ $\forall k \in n \# + 1. q(k) \in Y$
 shows $\text{Fold}(f, x, \{ \langle k, q(k) \rangle. k \in n \# + 1 \}) = f(\text{Fold}(f, x, \{ \langle k, q(k) \rangle. k \in n \}), q(n))$
<proof>

The tail of the sequence of partial folds defined by the folding function f , starting point x and a sequence y is the same as the sequence of partial folds starting from $f(x, y(0))$.

lemma fold_seq_detach_first:
 assumes $n \in \text{nat}$ $f : X \times Y \rightarrow X$ $y : \text{succ}(n) \rightarrow Y$ $x \in X$
 shows $\text{FoldSeq}(f, f(x, y(0)), \text{Tail}(y)) = \text{Tail}(\text{FoldSeq}(f, x, y))$
<proof>

Taking a fold of a sequence y with a function f with the starting point x is the same as the fold starting from $f(x, y(0))$ of the tail of y .

lemma fold_detach_first:
 assumes $n \in \text{nat}$ $f : X \times Y \rightarrow X$ $y : \text{succ}(n) \rightarrow Y$ $x \in X$

```

    shows Fold(f,x,y) = Fold(f,f⟨x,y(0)⟩,Tail(y))
  ⟨proof⟩

```

end

25 Partitions of sets

```

theory Partitions_ZF imports Finite_ZF InductiveSeq_ZF

```

begin

It is a common trick in proofs that we divide a set into non-overlapping subsets. The first case is when we split the set into two nonempty disjoint sets. Here this is modeled as an ordered pair of sets and the set of such divisions of set X is called $\text{Bisections}(X)$. The second variation on this theme is a set-valued function (aren't they all in ZF?) whose values are nonempty and mutually disjoint.

25.1 Bisections

This section is about dividing sets into two non-overlapping subsets.

The set of bisections of a given set A is a set of pairs of nonempty subsets of A that do not overlap and their union is equal to A .

definition

```

Bisections(X) = {p ∈ Pow(X) × Pow(X) .
fst(p) ≠ 0 ∧ snd(p) ≠ 0 ∧ fst(p) ∩ snd(p) = 0 ∧ fst(p) ∪ snd(p) = X}

```

Properties of bisections.

```

lemma bisec_props: assumes ⟨A,B⟩ ∈ Bisections(X) shows
  A ≠ 0 B ≠ 0 A ⊆ X B ⊆ X A ∩ B = 0 A ∪ B = X X ≠ 0
  ⟨proof⟩

```

Kind of inverse of `bisec_props`: a pair of nonempty disjoint sets form a bisection of their union.

lemma is_bisec:

```

  assumes A ≠ 0 B ≠ 0 A ∩ B = 0
  shows ⟨A,B⟩ ∈ Bisections(A ∪ B) ⟨proof⟩

```

Bisection of X is a pair of subsets of X .

```

lemma bisec_is_pair: assumes Q ∈ Bisections(X)
  shows Q = ⟨fst(Q), snd(Q)⟩
  ⟨proof⟩

```

The set of bisections of the empty set is empty.

```

lemma bisec_empty: shows Bisections(0) = 0

```

<proof>

The next lemma shows what can we say about bisections of a set with another element added.

lemma `bisec_add_point`:
 assumes $A1: x \notin X$ **and** $A2: \langle A, B \rangle \in \text{Bisections}(X \cup \{x\})$
 shows $(A = \{x\} \vee B = \{x\}) \vee (\langle A - \{x\}, B - \{x\} \rangle \in \text{Bisections}(X))$
 <proof>

A continuation of the lemma `bisec_add_point` that refines the case when the pair with removed point bisects the original set.

lemma `bisec_add_point_case3`:
 assumes $A1: \langle A, B \rangle \in \text{Bisections}(X \cup \{x\})$
 and $A2: \langle A - \{x\}, B - \{x\} \rangle \in \text{Bisections}(X)$
 shows
 $(\langle A, B - \{x\} \rangle \in \text{Bisections}(X) \wedge x \in B) \vee$
 $(\langle A - \{x\}, B \rangle \in \text{Bisections}(X) \wedge x \in A)$
 <proof>

Another lemma about bisecting a set with an added point.

lemma `point_set_bisec`:
 assumes $A1: x \notin X$ **and** $A2: \langle \{x\}, A \rangle \in \text{Bisections}(X \cup \{x\})$
 shows $A = X$ **and** $X \neq 0$
 <proof>

Yet another lemma about bisecting a set with an added point, very similar to `point_set_bisec` with almost the same proof.

lemma `set_point_bisec`:
 assumes $A1: x \notin X$ **and** $A2: \langle A, \{x\} \rangle \in \text{Bisections}(X \cup \{x\})$
 shows $A = X$ **and** $X \neq 0$
 <proof>

If a pair of sets bisects a finite set, then both elements of the pair are finite.

lemma `bisect_fin`:
 assumes $A1: A \in \text{FinPow}(X)$ **and** $A2: Q \in \text{Bisections}(A)$
 shows $\text{fst}(Q) \in \text{FinPow}(X)$ **and** $\text{snd}(Q) \in \text{FinPow}(X)$
 <proof>

25.2 Partitions

This sections covers the situation when we have an arbitrary number of sets we want to partition into.

We define a notion of a partition as a set valued function such that the values for different arguments are nonempty and disjoint. The name is derived from the fact that such function "partitions" the union of its arguments. Please let me know if you have a better idea for a name for such notion. We would

prefer to say "is a partition", but that reserves the letter "a" as a keyword(?) which causes problems.

definition

Partition ($_$ {is partition} [90] 91) **where**
 P {is partition} $\equiv \forall x \in \text{domain}(P). P(x) \neq 0 \wedge (\forall y \in \text{domain}(P). x \neq y \longrightarrow P(x) \cap P(y) = 0)$

A fact about lists of mutually disjoint sets.

lemma list_partition: **assumes** A1: $n \in \text{nat}$ **and**
 A2: $a : \text{succ}(n) \rightarrow X$ a {is partition}
shows $(\bigcup_{i \in n}. a(i)) \cap a(n) = 0$
<proof>

We can turn every injection into a partition.

lemma inj_partition:
assumes A1: $b \in \text{inj}(X, Y)$
shows
 $\forall x \in X. \{ \langle x, \{b(x)\} \rangle. x \in X \} (x) = \{b(x)\}$ **and**
 $\{ \langle x, \{b(x)\} \rangle. x \in X \}$ {is partition}
<proof>

25.3 Collections of pairwise disjoint sets

The notion of collection of pairwise disjoint sets is similar to a partition except that we don't require the members to be nonempty.

A set valued function X is pairwise disjoint if for any two different indices the corresponding sets have empty intersection.

definition

IsPairwiseDisjoint ($_$ {is pairwise disjoint} [60] 61) **where**
 X {is pairwise disjoint} $\equiv \forall i \in \text{domain}(X). \forall j \in \text{domain}(X). i \neq j \longrightarrow X(i) \cap X(j) = \emptyset$

Suppose that $X : I \rightarrow Y$ is pairwise disjoint collection of sets and we know that x is a member of a set in the collection. Then the index of that set can be expressed as $\bigcup \{j \in I : x \in X(j)\}$.

lemma get_the_one:
assumes $X : I \rightarrow Y$ X {is pairwise disjoint} $i \in I$ $x \in X(i)$
shows $i = \bigcup \{j \in I. x \in X(j)\}$
<proof>

A kind of inverse of **get_the_one**: if $X : I \rightarrow Y$ is pairwise disjoint collection of sets and $x \in \bigcup_{i \in I} X(i)$ then $i := \bigcup \{j \in I : x \in X(j)\}$ is an index and $x \in X(i)$.

lemma one_is_the_one:
assumes $X : I \rightarrow Y$ X {is pairwise disjoint} $x \in (\bigcup_{j \in I}. X(j))$
defines $i \equiv \bigcup \{j \in I. x \in X(j)\}$

shows $i \in I \quad x \in X(i) \quad \langle proof \rangle$

A special case of pairwise disjoint collection is the sequence of set differences of consecutive elements of sequence of sets that is monotonic in the inclusion order. The next lemma shows that if a sequence $\{\mathcal{U}_i\}_{i \in \mathbb{N}}$ of subsets of X is decreasing in the inclusion order on the powerset of X then the sequence $\{\mathcal{U}_i \setminus \mathcal{U}_{i+1}\}_{i \in \mathbb{N}}$ is pairwise disjoint. In addition, the union $\bigcup_{n \in \mathbb{N}} \mathcal{U}_n \setminus \mathcal{U}_{n+1}$ is the same as $\mathcal{U}_0 \setminus \bigcap_{n \in \mathbb{N}} \mathcal{U}_n$.

lemma `decr_pair_disj`:
assumes `IsDecreasingSeq(Pow(X), InclusionOn(Pow(X)),  $\mathcal{U}$ )`
shows
 $\{\langle n, \mathcal{U}(n) \setminus \mathcal{U}(n \# + 1) \rangle. n \in \text{nat}\} \text{ is pairwise disjoint}$
 $(\bigcup_{n \in \text{nat}. \mathcal{U}(n) \setminus \mathcal{U}(n \# + 1)}) = \mathcal{U}(0) \setminus (\bigcap_{n \in \text{nat}. \mathcal{U}(n)})$
 $\langle proof \rangle$

Essentially the same assertions as `decr_pair_disj` but with more explicit assumptions: if $\mathcal{U}$ is a sequence valued in the power set of X and $\mathcal{U}_{n+1} \subseteq \mathcal{U}_n$ for all natural numbers n , then the assumption and hence the assertions of `decr_pair_disj` hold.

lemma `decr_pair_disj1`:
assumes $\mathcal{U} : \text{nat} \rightarrow \text{Pow}(X) \quad \forall n \in \text{nat}. \mathcal{U}(n \# + 1) \subseteq \mathcal{U}(n)$
defines $\mathcal{V} \equiv \{\langle i, \mathcal{U}(i) \setminus \mathcal{U}(i \# + 1) \rangle. i \in \text{nat}\}$
shows
 $\mathcal{V} : \text{nat} \rightarrow \text{Pow}(X)$
`IsDecreasingSeq(Pow(X), InclusionOn(Pow(X)),  $\mathcal{U}$ )`
 $\mathcal{V} \text{ is pairwise disjoint}$
 $(\bigcup_{n \in \text{nat}. \mathcal{U}(n) \setminus \mathcal{U}(n \# + 1)}) = \mathcal{U}(0) \setminus (\bigcap_{n \in \text{nat}. \mathcal{U}(n)})$
 $\langle proof \rangle$

If a sequence $\{\mathcal{U}_i\}_{i \in \mathbb{N}}$ of subsets of X is increasing in the inclusion order on the powerset of X then the sequence $\{\mathcal{U}_{i+1} \setminus \mathcal{U}_i\}_{i \in \mathbb{N}}$ is pairwise disjoint.

lemma `incr_pair_disj`: **assumes** `IsIncreasingSeq(Pow(X), InclusionOn(Pow(X)),  $\mathcal{U}$ )`
shows $\{\langle n, \mathcal{U}(n \# + 1) \setminus \mathcal{U}(n) \rangle. n \in \text{nat}\} \text{ is pairwise disjoint}$
 $\langle proof \rangle$

end

26 Quasigroups

theory `Quasigroup_ZF` **imports** `func1`

begin

A quasigroup is an algebraic structure that that one gets after adding (sort of) divisibility to magma. Quasigroups differ from groups in that they are not necessarily associative and they do not have to have the neutral element.

26.1 Definitions and notation

According to Wikipedia there are at least two approaches to defining a quasigroup. One defines a quasigroup as a set with a binary operation, and the other, from universal algebra, defines a quasigroup as having three primitive operations. We will use the first approach.

A quasigroup operation does not have to have the neutral element. The left division is defined as the only solution to the equation $a \cdot x = b$ (using multiplicative notation). The next definition specifies what does it mean that an operation A has a left division on a set G .

definition

$$\text{HasLeftDiv}(G, A) \equiv \forall a \in G. \forall b \in G. \exists ! x. (x \in G \wedge A\langle a, x \rangle = b)$$

An operation A has the right division if for all elements $a, b \in G$ the equation $x \cdot a = b$ has a unique solution.

definition

$$\text{HasRightDiv}(G, A) \equiv \forall a \in G. \forall b \in G. \exists ! x. (x \in G \wedge A\langle x, a \rangle = b)$$

An operation that has both left and right division is said to have the Latin square property.

definition

$$\text{HasLatinSquareProp} \text{ (infix \{has Latin square property on\} 65) where } \\ A \text{ \{has Latin square property on\} } G \equiv \text{HasLeftDiv}(G, A) \wedge \text{HasRightDiv}(G, A)$$

A quasigroup is a set with a binary operation that has the Latin square property.

definition

$$\text{IsAquasigroup}(G, A) \equiv A : G \times G \rightarrow G \wedge A \text{ \{has Latin square property on\} } G$$

The uniqueness of the left inverse allows us to define the left division as a function. The union expression as the value of the function extracts the only element of the set of solutions of the equation $x \cdot z = y$ for given $\langle x, y \rangle = p \in G \times G$ using the identity $\bigcup \{x\} = x$.

definition

$$\text{LeftDiv}(G, A) \equiv \{ \langle p, \bigcup \{z \in G. A\langle \text{fst}(p), z \rangle = \text{snd}(p) \} \rangle . p \in G \times G \}$$

Similarly the right division is defined as a function on $G \times G$.

definition

$$\text{RightDiv}(G, A) \equiv \{ \langle p, \bigcup \{z \in G. A\langle z, \text{fst}(p) \rangle = \text{snd}(p) \} \rangle . p \in G \times G \}$$

Left and right divisions are binary operations on G .

lemma `lrdv_binop`: `assumes IsAquasigroup(G, A) shows`

$$\text{LeftDiv}(G, A) : G \times G \rightarrow G \text{ and } \text{RightDiv}(G, A) : G \times G \rightarrow G$$

<proof>

We will use multiplicative notation for the quasigroup operation. The right and left division will be denoted a/b and $a \backslash b$, resp.

```

locale quasigroup0 =
  fixes G A
  assumes qgroupassum: IsAQuasigroup(G,A)

  fixes qgproper (infixl · 70)
  defines qgproper_def[simp]: x·y ≡ A⟨x,y⟩

  fixes leftdiv (infixl \ 70)
  defines leftdiv_def[simp]: x\y ≡ LeftDiv(G,A)⟨x,y⟩

  fixes rightdiv (infixl / 70)
  defines rightdiv_def[simp]: x/y ≡ RightDiv(G,A)⟨y,x⟩

```

The quasigroup operation is closed on G .

```

lemma (in quasigroup0) qg_op_closed: assumes x∈G y∈G
shows x·y ∈ G
  ⟨proof⟩

```

A couple of properties of right and left division:

```

lemma (in quasigroup0) lrdiv_props: assumes x∈G y∈G
shows
  ∃!z. z∈G ∧ z·x = y y/x ∈ G (y/x)·x = y and
  ∃!z. z∈G ∧ x·z = y x\y ∈ G x·(x\y) = y
  ⟨proof⟩

```

We can cancel the left element on both sides of an equation.

```

lemma (in quasigroup0) qg_cancel_left:
  assumes x∈G y∈G z∈G and x·y = x·z
shows y=z
  ⟨proof⟩

```

We can cancel the right element on both sides of an equation.

```

lemma (in quasigroup0) qg_cancel_right:
  assumes x∈G y∈G z∈G and y·x = z·x
shows y=z
  ⟨proof⟩

```

Two additional identities for right and left division:

```

lemma (in quasigroup0) lrdiv_ident: assumes x∈G y∈G
shows (y·x)/x = y and x\ (x·y) = y
  ⟨proof⟩

```

end

27 Loops

```
theory Loop_ZF imports Quasigroup_ZF Fold_ZF
```

```
begin
```

This theory specifies the definition and proves basic properties of loops. Loops are very similar to groups, the only property that is missing is associativity of the operation.

27.1 Definitions and notation

In this section we define the notions of identity element and left and right inverse.

A loop is a quasigroup with an identity element.

```
definition IsAloop(G,A)  $\equiv$  IsAQuasigroup(G,A)  $\wedge$  ( $\exists e \in G. \forall x \in G. A\langle e, x \rangle = x$   

 $\wedge A\langle x, e \rangle = x$ )
```

The neutral element for a binary operation $A : G \times G \rightarrow G$ is defined as the only element e of G such that $A\langle x, e \rangle = x$ and $A\langle e, x \rangle = x$ for all $x \in G$. Note that although the loop definition guarantees the existence of (some) such element(s) at this point we do not know if this element is unique. We can define this notion here but it will become usable only after we prove uniqueness.

```
definition
```

```
TheNeutralElement(G,f)  $\equiv$   

  (THE e. e  $\in$  G  $\wedge$  ( $\forall g \in G. f\langle e, g \rangle = g \wedge f\langle g, e \rangle = g$ ))
```

We will reuse the notation defined in the `quasigroup0` locale, just adding the assumption about the existence of a neutral element and notation for it.

```
locale loop0 = quasigroup0 +  

  assumes ex_ident:  $\exists e \in G. \forall x \in G. e \cdot x = x \wedge x \cdot e = x$   
  

  fixes neut (1)  

  defines neut_def[simp]: 1  $\equiv$  TheNeutralElement(G,A)  
  

  fixes listprod ( $\prod$  _ 70)  

  defines listprod_def [simp]:  $\prod s \equiv$  Fold(A,1,s)  
  

  fixes pow  

  defines pow_def [simp]: pow(n,x)  $\equiv$   $\prod \{ \langle k, x \rangle. k \in n \}$ 
```

In the loop context the pair (G, A) forms a loop.

```
lemma (in loop0) is_loop: shows IsAloop(G,A)  

   $\langle proof \rangle$ 
```

If we know that a pair (G, A) forms a loop then the assumptions of the `loop0` locale hold.

lemma `loop_loop0_valid`: **assumes** `IsAloop(G,A)` **shows** `loop0(G,A)`
 $\langle proof \rangle$

The neutral element is unique in the loop.

lemma **(in** `loop0`**)** `neut_uniq_loop`: **shows**
 $\exists ! e. e \in G \wedge (\forall x \in G. e \cdot x = x \wedge x \cdot e = x)$
 $\langle proof \rangle$

The neutral element as defined in the `loop0` locale is indeed neutral.

lemma **(in** `loop0`**)** `neut_props_loop`: **shows** $1 \in G$ **and** $\forall x \in G. 1 \cdot x = x \wedge x \cdot 1 = x$
 $\langle proof \rangle$

A loop is not empty as it contains the neutral element.

corollary **(in** `loop0`**)** `loop_not_empty`: **shows** $G \neq \emptyset$ $\langle proof \rangle$

Every element of a loop has unique left and right inverse (which need not be the same). Here we define the left inverse as a function on G .

definition

$$\text{LeftInv}(G, A) \equiv \{ \langle x, \bigcup \{ y \in G. A \langle y, x \rangle = \text{TheNeutralElement}(G, A) \} \rangle. x \in G \}$$

Definition of the right inverse as a function on G :

definition

$$\text{RightInv}(G, A) \equiv \{ \langle x, \bigcup \{ y \in G. A \langle x, y \rangle = \text{TheNeutralElement}(G, A) \} \rangle. x \in G \}$$

In a loop G right and left inverses are functions on G .

lemma **(in** `loop0`**)** `lr_inv_fun`: **shows** $\text{LeftInv}(G, A) : G \rightarrow G$ $\text{RightInv}(G, A) : G \rightarrow G$
 $\langle proof \rangle$

Right and left inverses have desired properties.

lemma **(in** `loop0`**)** `lr_inv_props`: **assumes** $x \in G$
shows

$$\begin{aligned} \text{LeftInv}(G, A)(x) &\in G \quad (\text{LeftInv}(G, A)(x)) \cdot x = 1 \\ \text{RightInv}(G, A)(x) &\in G \quad x \cdot (\text{RightInv}(G, A)(x)) = 1 \end{aligned}$$

$\langle proof \rangle$

27.2 Product of a list of loop elements

Given a list $s : n \rightarrow L$ of loop elements we can define the product of the elements $s = [x_0, x_1, \dots, x_{n-1}]$ as the fold of loop operation over that list. The `loop0` locale defined a notation for such fold as $\prod s$. That locale also defines a notation for the special case when the list is constant i.e. $x_0 = x_1 = \dots x_{n-1} = x$. In such case $\prod s$ becomes x^{n-1} . There are similar sections on this subject in `Semigroup_ZF`, `Monoid_ZF` and `Group_ZF`.

The product of a list of loop elements is an element of the loop.

lemma (in loop0) list_prod_in_loop: **assumes** $n \in \text{nat}$ $s: n \rightarrow G$
shows $(\prod s) \in G$
<proof>

In particular a natural power of a loop element is an element of the loop.

lemma (in loop0) loop_prod_type: **assumes** $n \in \text{nat}$ $x \in G$ **shows** $\text{pow}(n, x) \in G$
<proof>

For a nonempty list of loop element the last element of the list can be pulled out of the product and put after the product. In other words $\prod k = 0^n s(n) = (\prod k = 0^{n-1} s(n)) \cdot s(n)$.

lemma (in loop0) loop_prod_pull_last: **assumes** $n \in \text{nat}$ $\forall k \in n \# + 1. s(k) \in G$
shows $(\prod \{ \langle k, s(k) \rangle. k \in n \# + 1 \}) = (\prod \{ \langle k, s(k) \rangle. k \in n \}) \cdot s(n)$
<proof>

The way we defined the product notation implies that the product of an empty list is the neutral element of the loop.

lemma (in loop0) loop_prod_empty: **assumes** $s: \emptyset \rightarrow G$ **shows** $(\prod s) = 1$
<proof>

In particular, any loop element raised to the natural zero'th power is the neutral element of the loop. In other words $x^0 = 1$.

corollary (in loop0) loop_zero_pow: **shows** $\text{pow}(0, x) = 1$ *<proof>*

Raising a loop element to a power greater by one corresponds to multiplying the power by that loop element.

lemma (in loop0) loop_pow_mult_one: **assumes** $n \in \text{nat}$ $x \in G$
shows $\text{pow}(n \# + 1, x) = \text{pow}(n, x) \cdot x$
<proof>

end

28 Ordered loops

theory OrderedLoop_ZF **imports** Loop_ZF Fold_ZF

begin

This theory file is about properties of loops (the algebraic structures introduced in IsarMathLib in the Loop_ZF theory) with an additional order relation that is in a way compatible with the loop's binary operation. The oldest reference I have found on the subject is [6].

28.1 Definition, notation and basic properties

An ordered loop (G, A) is a loop with a partial order relation r that is "translation invariant" with respect to the loop operation A .

A triple (G, A, r) is an ordered loop if (G, A) is a loop and r is a relation on G (i.e. a subset of $G \times G$ with is a partial order and for all elements $x, y, z \in G$ the condition $\langle x, y \rangle \in r$ is equivalent to both $\langle A\langle x, z \rangle, A\langle x, z \rangle \rangle \in r$ and $\langle A\langle z, x \rangle, A\langle z, x \rangle \rangle \in r$. This looks a bit awkward in the basic set theory notation, but using the additive notation for the group operation and $x \leq y$ to instead of $\langle x, y \rangle \in r$ this just means that $x \leq y$ if and only if $x + z \leq y + z$ and $x \leq y$ if and only if $z + x \leq z + y$.

definition

```
IsAnOrdLoop(L,A,r) ≡
  IsALoop(L,A) ∧ r ⊆ L×L ∧ IsPartOrder(L,r) ∧ (∀x∈L. ∀y∈L. ∀z∈L.
    ((⟨x,y⟩ ∈ r ↔ ⟨A⟨ x,z⟩,A⟨y,z⟩⟩ ∈ r) ∧ (⟨x,y⟩ ∈ r ↔ ⟨A⟨z,x⟩,A⟨z,y⟩⟩
    ∈ r )))
```

We define the set of nonnegative elements in the obvious way as $L^+ = \{x \in L : 0 \leq x\}$.

definition

```
Nonnegative(L,A,r) ≡ {x∈L. ⟨ TheNeutralElement(L,A),x⟩ ∈ r}
```

The $\text{PositiveSet}(L,A,r)$ is a set similar to $\text{Nonnegative}(L,A,r)$, but without the neutral element. What we call the positive set here is sometimes called "the positive cone" of an ordered loop.

definition

```
PositiveSet(L,A,r) ≡
  {x∈L. ⟨ TheNeutralElement(L,A),x⟩ ∈ r ∧ TheNeutralElement(L,A) ≠ x}
```

We will use the additive notation for ordered loops.

locale loop1 =

fixes L and A and r

assumes ordLoopAssum: IsAnOrdLoop(L,A,r)

fixes neut (0)

defines neut_def[simp]: 0 ≡ TheNeutralElement(L,A)

fixes loopop (infixl + 69)

defines loopop_def[simp]: x + y ≡ A⟨ x,y⟩

fixes lesseq (infix ≤ 68)

defines lesseq_def [simp]: x ≤ y ≡ ⟨x,y⟩ ∈ r

fixes sless (infix < 68)

defines sless_def[simp]: x < y ≡ x≤y ∧ x≠y

```

fixes nonnegative ( $L^+$ )
defines nonnegative_def [simp]:  $L^+ \equiv \text{Nonnegative}(L,A,r)$ 

fixes positive ( $L_+$ )
defines positive_def[simp]:  $L_+ \equiv \text{PositiveSet}(L,A,r)$ 

fixes leftdiv ( $- \_ + \_$ )
defines leftdiv_def[simp]:  $-x+y \equiv \text{LeftDiv}(L,A)\langle x,y \rangle$ 

fixes rightdiv (infixl  $-$  69)
defines rightdiv_def[simp]:  $x-y \equiv \text{RightDiv}(L,A)\langle y,x \rangle$ 

fixes listsum ( $\sum \_$  70)
defines listsum_def[simp]:  $\sum s \equiv \text{Fold}(A,0,s)$ 

fixes nat_mult (infix  $\cdot$  95)
defines nat_mult_def [simp]:  $n \cdot x \equiv \sum \{ \langle k,x \rangle . k \in n \}$ 

```

Theorems proven in the `loop0` locale are valid in the `loop1` locale

```

sublocale loop1 < loop0 L A looper leftdiv rightdiv neut listsum nat_mult
  <proof>

```

The notation $-x+y$ and $x-y$ denotes left and right division, resp. These two operations are closed in a loop, see lemma `lrdv_binop` in the `Quasigroup_ZF` theory. The next lemma reiterates that fact using the notation of the `loop1` context.

```

lemma (in loop1) left_right_sub_closed: assumes  $x \in L$   $y \in L$ 
  shows  $(-x+y) \in L$  and  $x-y \in L$ 
  <proof>

```

In this context $x \leq y$ implies that both x and y belong to L .

```

lemma (in loop1) lsq_members: assumes  $x \leq y$  shows  $x \in L$  and  $y \in L$ 
  <proof>

```

In this context $x < y$ implies that both x and y belong to L .

```

lemma (in loop1) less_members: assumes  $x < y$  shows  $x \in L$  and  $y \in L$ 
  <proof>

```

In an ordered loop the order is translation invariant.

```

lemma (in loop1) ord_trans_inv: assumes  $x \leq y$   $z \in L$ 
  shows  $x+z \leq y+z$  and  $z+x \leq z+y$ 
  <proof>

```

In an ordered loop the strict order is translation invariant.

```

lemma (in loop1) strict_ord_trans_inv: assumes  $x < y$   $z \in L$ 
  shows  $x+z < y+z$  and  $z+x < z+y$ 

```

<proof>

We can cancel an element from both sides of an inequality on the right side.

lemma (in loop1) ineq_cancel_right: **assumes** $x \in L$ $y \in L$ $z \in L$ **and** $x+z \leq y+z$
shows $x \leq y$
<proof>

We can cancel an element from both sides of a strict inequality on the right side.

lemma (in loop1) strict_ineq_cancel_right:
assumes $x \in L$ $y \in L$ $z \in L$ **and** $x+z < y+z$
shows $x < y$
<proof>

We can cancel an element from both sides of an inequality on the left side.

lemma (in loop1) ineq_cancel_left: **assumes** $x \in L$ $y \in L$ $z \in L$ **and** $z+x \leq z+y$
shows $x \leq y$
<proof>

We can cancel an element from both sides of a strict inequality on the left side.

lemma (in loop1) strict_ineq_cancel_left:
assumes $x \in L$ $y \in L$ $z \in L$ **and** $z+x < z+y$
shows $x < y$
<proof>

We can subtract a loop element from both sides of inequality both on the left side and right.

lemma (in loop1) ineq_subtr_from_sides: **assumes** $x \leq y$ $z \in L$
shows $x-z \leq y-z$ **and** $(-z+x) \leq -z+y$
<proof>

We can subtract a loop element from both sides of a strict inequality both on the left side and right.

lemma (in loop1) strict_ineq_subtr: **assumes** $x < y$ $z \in L$
shows $x-z < y-z$ **and** $(-z+x) < -z+y$
<proof>

We can move an element to the other side of a strict inequality with three loop elements - right side variant.

lemma (in loop1) strict_ineq_move_side_right: **assumes** $x \in L$ $y \in L$ $x+y < z$
shows $x < z-y$
<proof>

We can move an element to the other side of an inequality with three loop elements - right side variant.

lemma (in loop1) ineq_move_side_right: assumes $x \in L$ $y \in L$ $x+y \leq z$
shows $x \leq z-y$
 $\langle proof \rangle$

The definition of the nonnegative set in the notation used in the loop1 locale:

lemma (in loop1) nonneg_definition:
shows $x \in L^+ \longleftrightarrow 0 \leq x$ $\langle proof \rangle$

The nonnegative set is contained in the loop.

lemma (in loop1) nonneg_subset: **shows** $L^+ \subseteq L$
 $\langle proof \rangle$

The positive set is contained in the loop.

lemma (in loop1) positive_subset: **shows** $L_+ \subseteq L$
 $\langle proof \rangle$

The definition of the positive set in the notation used in the loop1 locale:

lemma (in loop1) posset_definition:
shows $x \in L_+ \longleftrightarrow (0 \leq x \wedge x \neq 0)$
 $\langle proof \rangle$

Another form of the definition of the positive set in the notation used in the loop1 locale:

lemma (in loop1) posset_definition1:
shows $x \in L_+ \longleftrightarrow 0 < x$
 $\langle proof \rangle$

The order in an ordered loop is antisymmetric.

lemma (in loop1) loop_ord_antisym: assumes $x \leq y$ and $y \leq x$
shows $x=y$
 $\langle proof \rangle$

The loop order is transitive.

lemma (in loop1) loop_ord_trans: assumes $x \leq y$ and $y \leq z$ **shows** $x \leq z$
 $\langle proof \rangle$

The loop order is reflexive.

lemma (in loop1) loop_ord_refl: assumes $x \in L$ **shows** $x \leq x$
 $\langle proof \rangle$

The neutral element is nonnegative.

lemma (in loop1) loop_zero_nonneg: **shows** $0 \in L^+$
 $\langle proof \rangle$

A form of mixed transitivity for the strict order:

lemma (in loop1) loop_strict_ord_trans: assumes $x \leq y$ and $y < z$

shows $x < z$
 $\langle proof \rangle$

Another form of mixed transitivity for the strict order:

lemma (in loop1) loop_strict_ord_trans1: **assumes** $x < y$ **and** $y \leq z$
shows $x < z$
 $\langle proof \rangle$

Yet another form of mixed transitivity for the strict order:

lemma (in loop1) loop_strict_ord_trans2: **assumes** $x < y$ **and** $y < z$
shows $x < z$
 $\langle proof \rangle$

We can move an element to the other side of an inequality. Well, not exactly, but our notation creates an illusion to that effect.

lemma (in loop1) lsq_other_side: **assumes** $x \leq y$
shows $0 \leq -x+y$ $(-x+y) \in L^+$ $0 \leq y-x$ $(y-x) \in L^+$
 $\langle proof \rangle$

We can move an element to the other side of a strict inequality.

lemma (in loop1) ls_other_side: **assumes** $x < y$
shows $0 < -x+y$ $(-x+y) \in L_+$ $0 < y-x$ $(y-x) \in L_+$
 $\langle proof \rangle$

We can add sides of inequalities.

lemma (in loop1) add_ineq: **assumes** $x \leq y$ $z \leq t$
shows $x+z \leq y+t$
 $\langle proof \rangle$

We can add sides of strict inequalities. The proof uses a lemma that relies on the antisymmetry of the order relation.

lemma (in loop1) add_ineq_strict: **assumes** $x < y$ $z < t$
shows $x+z < y+t$
 $\langle proof \rangle$

We can add sides of inequalities one of which is strict.

lemma (in loop1) add_ineq_strict1: **assumes** $x \leq y$ $z < t$
shows $x+z < y+t$ **and** $z+x < t+y$
 $\langle proof \rangle$

Subtracting a positive element decreases the value, while adding a positive element increases the value.

lemma (in loop1) add_subtract_pos: **assumes** $x \in L$ $0 < y$
shows
 $x-y < x$ $(-y+x) < x$ $x < x+y$ $x < y+x$
 $\langle proof \rangle$

In an ordered loop if the order relation down-directs the set of positive elements L_+ then L_+ is a down-directed set (see `Order_ZF` for definitions of those related but different notions).

```
lemma (in loop1) down_directs_directed: assumes r {down-directs} L+
shows IsDownDirectedSet(L+,r)
<proof>
```

28.2 Completeness vs Archimedean property

For ordered fields the completeness of the order implies the Archimedean property: for every two positive elements there is a natural number n such that $y < nx$. In 1891 Otto Stolz "succeeded in proving that the positive cone of a Dedekind complete ordered abelian group is Archimedean". In this section we consider the relation between the completeness of the order and the Archimedean property of the positive set of an ordered loop (not necessarily abelian).

Multiples of a positive element are positive.

```
lemma (in loop1) mults_pos_pos: assumes 0<x n∈nat\{0}
shows 0<n·x
<proof>
```

If the order is complete and x is a positive element of an ordered loop, then the set $\{n \cdot x : n \in \mathbb{N}, n \neq 0\}$ is not bounded above.

```
lemma (in loop1) nat_mult_not_bounded: assumes r {is complete} and 0<x
shows ¬IsBoundedAbove({n·x. n∈nat\{0}},r)
<proof>
```

We say that a nonempty subset A of a ordered loop is Archimedean if for any two elements x, y of A there is a non-zero natural number n such that $n \cdot x$ exceeds y .

```
definition (in loop1)
  IsArchimedean (_ {is Archimedean} [90] 91) where
    B {is Archimedean} ≡ B≠∅ ∧ (∀x∈B. ∀y∈B. ∃n∈nat\{0}. y<n·x)
```

If the order of the loop is complete and total on the positive set, then the positive set is Archimedean.

```
theorem (in loop1) loop_pos_set_archimedean:
assumes L+≠∅ r {is complete} and r {is total on} L+
shows L+ {is Archimedean}
<proof>
```

end

29 Semigroups

```
theory Semigroup_ZF imports Partitions_ZF Fold_ZF Enumeration_ZF
```

begin

It seems that the minimal setup needed to talk about a product of a sequence is a set with a binary operation. Such object is called "magma". However, interesting properties show up when the binary operation is associative and such algebraic structure is called a semigroup. In this theory file we define and study sequences of partial products of sequences of magma and semigroup elements.

29.1 Products of sequences of semigroup elements

Semigroup is a magma in which the binary operation is associative. In this section we mostly study the products of sequences of elements of semigroup. The goal is to establish the fact that taking the product of a sequence is distributive with respect to concatenation of sequences, i.e for two sequences a, b of the semigroup elements we have $\prod(a \sqcup b) = (\prod a) \cdot (\prod b)$, where " $a \sqcup b$ " is concatenation of a and b ($a++b$ in Haskell notation). Less formally, we want to show that we can discard parantheses in expressions of the form $(a_0 \cdot a_1 \cdot \dots \cdot a_n) \cdot (b_0 \cdot \dots \cdot b_k)$.

First we define a notion similar to `Fold`, except that the initial element of the fold is given by the first element of sequence. By analogy with Haskell fold we call that `Fold1`

definition

`Fold1(f,a)  $\equiv$  Fold(f,a(0),Tail(a))`

The definition of the `semigr0` context below introduces notation for writing about finite sequences and semigroup products. In the context we fix the carrier and denote it G . The binary operation on G is called f . All theorems proven in the context `semigr0` will implicitly assume that f is an associative operation on G . We will use multiplicative notation for the semigroup operation. The product of a sequence a is denoted $\prod a$. We will write $a \leftarrow x$ for the result of appending an element x to the finite sequence (list) a . This is a bit nonstandard, but I don't have a better idea for the "append" notation. Finally, $a \sqcup b$ will denote the concatenation of the lists a and b .

locale `semigr0` =

`fixes G f`

`assumes assoc_assum: f {is associative on} G`

`fixes prod (infixl  $\cdot$  72)`

`defines prod_def [simp]:  $x \cdot y \equiv f(x,y)$`

`fixes seqprod ( $\prod$  _ 71)`

```

defines seqprod_def [simp]:  $\prod a \equiv \text{Fold1}(f,a)$ 

fixes append (infix  $\leftarrow$  72)
defines append_def [simp]:  $a \leftarrow x \equiv \text{Append}(a,x)$ 

fixes concat (infixl  $\sqcup$  69)
defines concat_def [simp]:  $a \sqcup b \equiv \text{Concat}(a,b)$ 

```

The next lemma shows our assumption on the associativity of the semigroup operation in the notation defined in in the `semigr0` context.

```

lemma (in semigr0) semigr_assoc: assumes  $x \in G \quad y \in G \quad z \in G$ 
shows  $x \cdot y \cdot z = x \cdot (y \cdot z)$ 
  <proof>

```

In the way we define associativity the assumption that f is associative on G also implies that it is a binary operation on X .

```

lemma (in semigr0) semigr_binop: shows  $f : G \times G \rightarrow G$ 
  <proof>

```

Semigroup operation is closed.

```

lemma (in semigr0) semigr_closed:
assumes  $a \in G \quad b \in G$  shows  $a \cdot b \in G$ 
  <proof>

```

Lemma `append_1elem` written in the notation used in the `semigr0` context.

```

lemma (in semigr0) append_1elem_nice:
assumes  $n \in \text{nat}$  and  $a : n \rightarrow X$  and  $b : 1 \rightarrow X$ 
shows  $a \sqcup b = a \leftarrow b(0)$ 
  <proof>

```

Lemma `concat_init_last_elem` rewritten in the notation used in the `semigr0` context.

```

lemma (in semigr0) concat_init_last:
assumes  $n \in \text{nat} \quad k \in \text{nat}$  and
 $a : n \rightarrow X \quad \text{and} \quad b : \text{succ}(k) \rightarrow X$ 
shows  $(a \sqcup \text{Init}(b)) \leftarrow b(k) = a \sqcup b$ 
  <proof>

```

The product of semigroup (actually, magma – we don't need associativity for this) elements is in the semigroup.

```

lemma (in semigr0) prod_type:
assumes  $n \in \text{nat}$  and  $a : \text{succ}(n) \rightarrow G$ 
shows  $(\prod a) \in G$ 
  <proof>

```

What is the product of one element list?

```

lemma (in semigr0) prod_of_1elem: assumes  $A1: a : 1 \rightarrow G$ 

```

shows $(\prod a) = a(0)$
 $\langle proof \rangle$

What happens to the product of a list when we append an element to the list?

lemma (in semigr0) prod_append: **assumes** A1: $n \in \text{nat}$ **and**
 A2: $a : \text{succ}(n) \rightarrow G$ **and** A3: $x \in G$
shows $(\prod a \leftarrow x) = (\prod a) \cdot x$
 $\langle proof \rangle$

The main theorem of the section: taking the product of a sequence is distributive with respect to concatenation of sequences. The proof is by induction on the length of the second list.

theorem (in semigr0) prod_conc_distr:
assumes A1: $n \in \text{nat}$ $k \in \text{nat}$ **and**
 A2: $a : \text{succ}(n) \rightarrow G$ $b : \text{succ}(k) \rightarrow G$
shows $(\prod a) \cdot (\prod b) = \prod (a \sqcup b)$
 $\langle proof \rangle$

$a \cdot b \cdot (c \cdot d) = a \cdot (b \cdot c) \cdot d$ for semigroup elements $a, b, c, d \in G$. The Commutative semigroups section below contains a couple of rearrangements that need commutativity of the semigroup operation, but this one uses only associativity, so it's here.

lemma (in semigr0) rearr4elem_assoc:
assumes $a \in G$ $b \in G$ $c \in G$ $d \in G$
shows $a \cdot b \cdot (c \cdot d) = a \cdot (b \cdot c) \cdot d$
 $\langle proof \rangle$

29.2 Products over sets of indices

In this section we study the properties of expressions of the form $\prod_{i \in \Lambda} a_i = a_{i_0} \cdot a_{i_1} \cdot \dots \cdot a_{i_{n-1}}$, i.e. what we denote as $\prod(\Lambda, a)$. Λ here is a finite subset of some set X and a is a function defined on X with values in the semigroup G .

Suppose $a : X \rightarrow G$ is an indexed family of elements of a semigroup G and $\Lambda = \{i_0, i_1, \dots, i_{n-1}\} \subseteq \mathbb{N}$ is a finite set of indices. We want to define $\prod_{i \in \Lambda} a_i = a_{i_0} \cdot a_{i_1} \cdot \dots \cdot a_{i_{n-1}}$. To do that we use the notion of **Enumeration** defined in the **Enumeration_ZF** theory file that takes a set of indices and lists them in increasing order, thus converting it to list. Then we use the **Fold1** to multiply the resulting list. Recall that in Isabelle/ZF the capital letter "O" denotes the composition of two functions (or relations).

definition

$\text{SetFold}(f, a, \Lambda, r) = \text{Fold1}(f, a \circ \text{Enumeration}(\Lambda, r))$

For a finite subset Λ of a linearly ordered set X we will write $\sigma(\Lambda)$ to denote the enumeration of the elements of Λ , i.e. the only order isomorphism $|\Lambda| \rightarrow$

Λ , where $|\Lambda| \in \mathbb{N}$ is the number of elements of Λ . We also define notation for taking a product over a set of indices of some sequence of semigroup elements. The product of semigroup elements over some set $\Lambda \subseteq X$ of indices of a sequence $a : X \rightarrow G$ (i.e. $\prod_{i \in \Lambda} a_i$) is denoted $\prod(\Lambda, a)$. In the `semigr1` context we assume that a is a function defined on some linearly ordered set X with values in the semigroup G .

```

locale semigr1 = semigr0 +

  fixes X r
  assumes linord: IsLinOrder(X,r)

  fixes a
  assumes a_is_fun: a : X  $\rightarrow$  G

  fixes  $\sigma$ 
  defines  $\sigma\_def$  [simp]:  $\sigma(\Lambda) \equiv \text{Enumeration}(\Lambda, r)$ 

  fixes setpr ( $\prod$ )
  defines setpr_def [simp]:  $\prod(\Lambda, b) \equiv \text{SetFold}(f, b, \Lambda, r)$ 

```

We can use the `enums` locale in the `semigr0` context.

```

lemma (in semigr1) enums_valid_in_semigr1: shows enums(X,r)
   $\langle proof \rangle$ 

```

Definition of product over a set expressed in notation of the `semigr0` locale.

```

lemma (in semigr1) setproddef:
  shows  $\prod(\Lambda, a) = \prod (a \circ \sigma(\Lambda))$ 
   $\langle proof \rangle$ 

```

A composition of enumeration of a nonempty finite subset of $\mathbb{N}$ with a sequence of elements of G is a nonempty list of elements of G . This implies that a product over set of a finite set of indices belongs to the (carrier of) semigroup.

```

lemma (in semigr1) setprod_type: assumes
  A1:  $\Lambda \in \text{FinPow}(X)$  and A2:  $\Lambda \neq 0$ 
  shows
   $\exists n \in \text{nat} . |\Lambda| = \text{succ}(n) \wedge a \circ \sigma(\Lambda) : \text{succ}(n) \rightarrow G$ 
  and  $\prod(\Lambda, a) \in G$ 
   $\langle proof \rangle$ 

```

The `enum_append` lemma from the `Enumeration` theory specialized for natural numbers.

```

lemma (in semigr1) semigr1_enum_append:
  assumes  $\Lambda \in \text{FinPow}(X)$  and
   $n \in X - \Lambda$  and  $\forall k \in \Lambda. \langle k, n \rangle \in r$ 
  shows  $\sigma(\Lambda \cup \{n\}) = \sigma(\Lambda) \leftarrow n$ 
   $\langle proof \rangle$ 

```

What is product over a singleton?

```
lemma (in semigr1) gen_prod_singleton:
  assumes A1:  $x \in X$ 
  shows  $\prod(\{x\}, a) = a(x)$ 
  <proof>
```

A generalization of `prod_append` to the products over sets of indices.

```
lemma (in semigr1) gen_prod_append:
  assumes
    A1:  $\Lambda \in \text{FinPow}(X)$  and A2:  $\Lambda \neq 0$  and
    A3:  $n \in X - \Lambda$  and
    A4:  $\forall k \in \Lambda. \langle k, n \rangle \in r$ 
  shows  $\prod(\Lambda \cup \{n\}, a) = (\prod(\Lambda, a)) \cdot a(n)$ 
  <proof>
```

Very similar to `gen_prod_append`: a relation between a product over a set of indices and the product over the set with the maximum removed.

```
lemma (in semigr1) gen_product_rem_point:
  assumes A1:  $A \in \text{FinPow}(X)$  and
    A2:  $n \in A$  and A4:  $A - \{n\} \neq 0$  and
    A3:  $\forall k \in A. \langle k, n \rangle \in r$ 
  shows
     $(\prod(A - \{n\}, a)) \cdot a(n) = \prod(A, a)$ 
  <proof>
```

29.3 Commutative semigroups

Commutative semigroups are those whose operation is commutative, i.e. $a \cdot b = b \cdot a$. This implies that for any permutation $s : n \rightarrow n$ we have $\prod_{j=0}^n a_j = \prod_{j=0}^n a_{s(j)}$, or, closer to the notation we are using in the `semigr0` context, $\prod a = \prod(a \circ s)$. Maybe one day we will be able to prove this, but for now the goal is to prove something simpler: that if the semigroup operation is commutative taking the product of a sequence is distributive with respect to the operation: $\prod_{j=0}^n (a_j \cdot b_j) = \left(\prod_{j=0}^n a_j\right) \left(\prod_{j=0}^n b_j\right)$. Many of the rearrangements (namely those that don't use the inverse) proven in the `AbelianGroup_ZF` theory hold in fact in semigroups. Some of them will be reproven in this section.

A rearrangement with 3 elements.

```
lemma (in semigr0) rearr3elems:
  assumes f {is commutative on} G and a ∈ G b ∈ G c ∈ G
  shows a · b · c = a · c · b
  <proof>
```

A rearrangement of four elements.

```
lemma (in semigr0) rearr4elems:
```



```

assumes A1: f {is commutative on} G and
A2: a∈G b∈G c∈G d∈G
shows a·b·(c·d) = a·c·(b·d)
⟨proof⟩

```

We start with a version of `prod_append` that will shorten a bit the proof of the main theorem.

```

lemma (in semigr0) shorter_seq: assumes A1: k ∈ nat and
A2: a ∈ succ(succ(k)) → G
shows (∏ a) = (∏ Init(a)) · a(succ(k))
⟨proof⟩

```

A lemma useful in the induction step of the main theorem.

```

lemma (in semigr0) prod_distr_ind_step:
assumes A1: k ∈ nat and
A2: a : succ(succ(k)) → G and
A3: b : succ(succ(k)) → G and
A4: c : succ(succ(k)) → G and
A5: ∀j∈succ(succ(k)). c(j) = a(j) · b(j)
shows
Init(a) : succ(k) → G
Init(b) : succ(k) → G
Init(c) : succ(k) → G
∀j∈succ(k). Init(c)(j) = Init(a)(j) · Init(b)(j)
⟨proof⟩

```

For commutative operations taking the product of a sequence is distributive with respect to the operation. This version will probably not be used in applications, it is formulated in a way that is easier to prove by induction. For a more convenient formulation see `prod_comm_distrib`. The proof by induction on the length of the sequence.

```

theorem (in semigr0) prod_comm_distr:
assumes A1: f {is commutative on} G and A2: n∈nat
shows ∀ a b c.
(a : succ(n)→G ∧ b : succ(n)→G ∧ c : succ(n)→G ∧
(∀j∈succ(n). c(j) = a(j) · b(j))) →
(∏ c) = (∏ a) · (∏ b)
⟨proof⟩

```

A reformulation of `prod_comm_distr` that is more convenient in applications.

```

theorem (in semigr0) prod_comm_distrib:
assumes f {is commutative on} G and n∈nat and
a : succ(n)→G b : succ(n)→G c : succ(n)→G and
∀j∈succ(n). c(j) = a(j) · b(j)
shows (∏ c) = (∏ a) · (∏ b)
⟨proof⟩

```

A product of two products over disjoint sets of indices is the product over the union.

```

lemma (in semigr1) prod_bisect:
  assumes A1: f {is commutative on} G and A2:  $\Lambda \in \text{FinPow}(X)$ 
  shows
 $\forall P \in \text{Bisections}(\Lambda). \prod(\Lambda, a) = (\prod(\text{fst}(P), a)) \cdot (\prod(\text{snd}(P), a))$ 
  <proof>

```

A better looking reformulation of prod_bisect.

```

theorem (in semigr1) prod_disjoint: assumes
  A1: f {is commutative on} G and
  A2:  $A \in \text{FinPow}(X)$   $A \neq 0$  and
  A3:  $B \in \text{FinPow}(X)$   $B \neq 0$  and
  A4:  $A \cap B = 0$ 
  shows  $\prod(A \cup B, a) = (\prod(A, a)) \cdot (\prod(B, a))$ 
  <proof>

```

A generalization of prod_disjoint.

```

lemma (in semigr1) prod_list_of_lists: assumes
  A1: f {is commutative on} G and A2:  $n \in \text{nat}$ 
  shows  $\forall M \in \text{succ}(n) \rightarrow \text{FinPow}(X).$ 
   $M \{\text{is partition}\} \rightarrow$ 
 $(\prod \{ \langle i, \prod(M(i), a) \rangle. i \in \text{succ}(n) \}) =$ 
 $(\prod(\bigcup i \in \text{succ}(n). M(i), a))$ 
  <proof>

```

A more convenient reformulation of prod_list_of_lists.

```

theorem (in semigr1) prod_list_of_sets:
  assumes A1: f {is commutative on} G and
  A2:  $n \in \text{nat}$   $n \neq 0$  and
  A3:  $M : n \rightarrow \text{FinPow}(X)$   $M \{\text{is partition}\}$ 
  shows
 $(\prod \{ \langle i, \prod(M(i), a) \rangle. i \in n \}) = (\prod(\bigcup i \in n. M(i), a))$ 
  <proof>

```

The definition of the product $\prod(A, a) \equiv \text{SetFold}(f, a, A, r)$ of a some (finite) set of semigroup elements requires that r is a linear order on the set of indices A . This is necessary so that we know in which order we are multiplying the elements. The product over A is defined so that we have $\prod_A a = \prod a \circ \sigma(A)$ where $\sigma : |A| \rightarrow A$ is the enumeration of A (the only order isomorphism between the number of elements in A and A), see lemma `setproddef`. However, if the operation is commutative, the order is irrelevant. The next theorem formalizes that fact stating that we can replace the enumeration $\sigma(A)$ by any bijection between $|A|$ and A . In a way this is a generalization of `setproddef`. The proof is based on application of `prod_list_of_sets` to the finite collection of singletons that comprise A .

```

theorem (in semigr1) prod_order_irr:
  assumes A1: f {is commutative on} G and
  A2:  $A \in \text{FinPow}(X)$   $A \neq 0$  and

```

```

A3: b ∈ bij(|A|,A)
shows (∏ (a 0 b)) = ∏(A,a)
<proof>

```

Another way of expressing the fact that the product dos not depend on the order.

```

corollary (in semigr1) prod_bij_same:
  assumes f {is commutative on} G and
  A ∈ FinPow(X) A ≠ 0 and
  b ∈ bij(|A|,A) c ∈ bij(|A|,A)
  shows (∏ (a 0 b)) = (∏ (a 0 c))
  <proof>

end

```

30 Commutative Semigroups

```

theory CommutativeSemigroup_ZF imports Semigroup_ZF

```

```

begin

```

In the `Semigroup` theory we introduced a notion of `SetFold(f,a,Λ,r)` that represents the sum of values of some function a valued in a semigroup where the arguments of that function vary over some set Λ . Using the additive notation something like this would be expressed as $\sum_{x \in \Lambda} f(x)$ in informal mathematics. This theory considers an alternative to that notion that is more specific to commutative semigroups.

30.1 Sum of a function over a set

The r parameter in the definition of `SetFold(f,a,Λ,r)` (from `Semigroup_ZF`) represents a linear order relation on Λ that is needed to indicate in what order we are summing the values $f(x)$. If the semigroup operation is commutative the order does not matter and the relation r is not needed. In this section we define a notion of summing up values of some function $a : X \rightarrow G$ over a finite set of indices $\Gamma \subseteq X$, without using any order relation on X .

We define the sum of values of a function $a : X \rightarrow G$ over a set Λ as the only element of the set of sums of lists that are bijections between the number of values in Λ (which is a natural number $n = \{0, 1, \dots, n-1\}$ if Λ is finite) and Λ . The notion of `Fold1(f,c)` is defined in `Semigroup_ZF` as the fold (sum) of the list c starting from the first element of that list. The intention is to use the fact that since the result of summing up a list does not depend on the order, the set $\{\text{Fold1}(f,a \ 0 \ b) \mid b \in \text{bij}(|\Lambda|, \Lambda)\}$ is a singleton and we can extract its only value by taking its union.

definition

$$\text{CommSetFold}(f, a, \Lambda) = \bigcup \{ \text{Fold1}(f, a \ 0 \ b) . \ b \in \text{bij}(|\Lambda|, \Lambda) \}$$

the next locale sets up notation for writing about summation in commutative semigroups. We define two kinds of sums. One is the sum of elements of a list (which are just functions defined on a natural number) and the second one represents a more general notion the sum of values of a semigroup valued function over some set of arguments. Since those two types of sums are different notions they are represented by different symbols. However in the presentations they are both intended to be printed as $\sum$.

locale commsemigr =

fixes G f

assumes csgassoc: f {is associative on} G

assumes csgcomm: f {is commutative on} G

fixes csgsum (infixl + 69)

defines csgsum_def[simp]: $x + y \equiv f\langle x, y \rangle$

fixes X a

assumes csgaisfun: $a : X \rightarrow G$

fixes csglistsum ($\sum$ _ 70)

defines csglistsum_def[simp]: $\sum k \equiv \text{Fold1}(f, k)$

fixes csgsetsum ($\sum$)

defines csgsetsum_def[simp]: $\sum(A, h) \equiv \text{CommSetFold}(f, h, A)$

Definition of a sum of function over a set in notation defined in the commsemigr locale.

lemma (in commsemigr) CommSetFolddef:

shows $(\sum(A, a)) = (\bigcup \{ \sum(a \ 0 \ b) . \ b \in \text{bij}(|A|, A) \})$

<proof>

The next lemma states that the result of a sum does not depend on the order we calculate it. This is similar to lemma prod_order_irr in the Semigroup theory, except that the semigr1 locale assumes that the domain of the function we sum up is linearly ordered, while in commsemigr we don't have this assumption.

lemma (in commsemigr) sum_over_set_bij:

assumes A1: $A \in \text{FinPow}(X)$ $A \neq 0$ **and** A2: $b \in \text{bij}(|A|, A)$

shows $(\sum(A, a)) = (\sum(a \ 0 \ b))$

<proof>

The result of a sum is in the semigroup. Also, as the second assertion we show that every semigroup valued function generates a homomorphism

between the finite subsets of a semigroup and the semigroup. Adding an element to a set corresponds to adding a value.

```
lemma (in commsemigr) sum_over_set_add_point:
  assumes A1: A ∈ FinPow(X)  A ≠ 0
  shows  $\sum(A, a) \in G$  and
   $\forall x \in X - A. \sum(A \cup \{x\}, a) = (\sum(A, a)) + a(x)$ 
  <proof>
```

end

31 Monoids

```
theory Monoid_ZF imports func_ZF Loop_ZF Semigroup_ZF
```

begin

This theory provides basic facts about monoids.

31.1 Definition and basic properties

In this section we talk about monoids. The notion of a monoid is similar to the notion of a semigroup except that we require the existence of a neutral element. It is also similar to the notion of group except that we don't require existence of the inverse.

Monoid is a set G with an associative operation and a neutral element. The operation is a function on $G \times G$ with values in G . In the context of ZF set theory this means that it is a set of pairs $\langle x, y \rangle$, where $x \in G \times G$ and $y \in G$. In other words the operation is a certain subset of $(G \times G) \times G$. We express all this by defining a predicate $\text{IsAmonoid}(G, f)$. Here G is the "carrier" of the monoid and f is the binary operation on it.

definition

```
IsAmonoid(G, f)  $\equiv$ 
f {is associative on} G  $\wedge$ 
( $\exists e \in G. (\forall g \in G. (f(\langle e, g \rangle) = g) \wedge (f(\langle g, e \rangle) = g)))$ 
```

The next locale called "monoid0" defines a context for theorems that concern monoids. In this context we assume that the pair (G, f) is a monoid. We will use the $\oplus$ symbol to denote the monoid operation (for no particular reason).

locale monoid0 =

```
fixes G f
assumes monoidAssum: IsAmonoid(G, f)

fixes monoper (infixl  $\oplus$  70)
defines monoper_def [simp]:  $a \oplus b \equiv f(\langle a, b \rangle)$ 
```

Propositions proven in the `semigr0` locale are valid in the `monoid0` locale.

lemma (in `monoid0`) `semigr0_valid_in_monoid0`: **shows** `semigr0(G,f)`
<proof>

The result of the monoid operation is in the monoid (carrier).

lemma (in `monoid0`) `group0_1_L1`:
assumes `a∈G` `b∈G` **shows** `a⊕b ∈ G`
<proof>

There is only one neutral element in a monoid.

lemma (in `monoid0`) `group0_1_L2`: **shows**
 $\exists !e. e \in G \wedge (\forall g \in G. (e \oplus g = g) \wedge g \oplus e = g)$
<proof>

The neutral element is neutral.

lemma (in `monoid0`) `unit_is_neutral`:
assumes `A1: e = TheNeutralElement(G,f)`
shows `e ∈ G` $\wedge$ $(\forall g \in G. e \oplus g = g \wedge g \oplus e = g)$
<proof>

The monoid carrier is not empty.

lemma (in `monoid0`) `group0_1_L3A`: **shows** `G≠0`
<proof>

The monoid operation is a binary function on the carrier with values in the carrier.

lemma (in `monoid0`) `monoid_oper_fun`: **shows** `f:G×G→G`
<proof>

The range of the monoid operation is the whole monoid carrier.

lemma (in `monoid0`) `group0_1_L3B`: **shows** `range(f) = G`
<proof>

Another way to state that the range of the monoid operation is the whole monoid carrier.

lemma (in `monoid0`) `range_carr`: **shows** `f(G×G) = G`
<proof>

In a monoid any neutral element is the neutral element.

lemma (in `monoid0`) `group0_1_L4`:
assumes `A1: e ∈ G` $\wedge$ $(\forall g \in G. e \oplus g = g \wedge g \oplus e = g)$
shows `e = TheNeutralElement(G,f)`
<proof>

The next lemma shows that if the if we restrict the monoid operation to a subset of G that contains the neutral element, then the neutral element of the monoid operation is also neutral with the restricted operation.

```

lemma (in monoid0) group0_1_L5:
  assumes A1:  $\forall x \in H. \forall y \in H. x \oplus y \in H$ 
  and A2:  $H \subseteq G$ 
  and A3:  $e = \text{TheNeutralElement}(G, f)$ 
  and A4:  $g = \text{restrict}(f, H \times H)$ 
  and A5:  $e \in H$ 
  and A6:  $h \in H$ 
  shows  $g\langle e, h \rangle = h \wedge g\langle h, e \rangle = h$ 
<proof>

```

The next theorem shows that if the monoid operation is closed on a subset of G then this set is a (sub)monoid (although we do not define this notion). This fact will be useful when we study subgroups.

```

theorem (in monoid0) group0_1_T1:
  assumes A1:  $H \text{ \{is closed under\} } f$ 
  and A2:  $H \subseteq G$ 
  and A3:  $\text{TheNeutralElement}(G, f) \in H$ 
  shows  $\text{IsAmonoid}(H, \text{restrict}(f, H \times H))$ 
<proof>

```

Under the assumptions of `group0_1_T1` the neutral element of a submonoid is the same as that of the monoid.

```

lemma group0_1_L6:
  assumes A1:  $\text{IsAmonoid}(G, f)$ 
  and A2:  $H \text{ \{is closed under\} } f$ 
  and A3:  $H \subseteq G$ 
  and A4:  $\text{TheNeutralElement}(G, f) \in H$ 
  shows  $\text{TheNeutralElement}(H, \text{restrict}(f, H \times H)) = \text{TheNeutralElement}(G, f)$ 
<proof>

```

If a sum of two elements is not zero, then at least one has to be nonzero.

```

lemma (in monoid0) sum_nonzero_elmnt_nonzero:
  assumes  $a \oplus b \neq \text{TheNeutralElement}(G, f)$ 
  shows  $a \neq \text{TheNeutralElement}(G, f) \vee b \neq \text{TheNeutralElement}(G, f)$ 
<proof>

```

The monoid operation is associative.

```

lemma (in monoid0) sum_associative:
  assumes  $a \in G \ b \in G \ c \in G$ 
  shows  $(a \oplus b) \oplus c = a \oplus (b \oplus c)$ 
<proof>

```

A simple rearrangement of four monoid elements transferred from the `semigr0` locale:

```

lemma (in monoid0) rearr4elem_monoid:
  assumes  $a \in G \ b \in G \ c \in G \ d \in G$ 
  shows  $a \oplus b \oplus (c \oplus d) = a \oplus (b \oplus c) \oplus d$ 
<proof>

```

end

32 Summing lists in a monoid

theory Monoid_ZF_1 **imports** Monoid_ZF FiniteSeq_ZF

begin

This theory consider properties of sums of monoid elements, similar to the ones formalized in the `Semigroup_ZF` theory for sums of semigroup elements. The main difference is that since each monoid has a neutral element it makes sense to define a sum of an empty list of monoid elements. In multiplicative notation the properties considered here can be applied to natural powers of elements ($x^n, n \in \mathbb{N}$) in group or ring theory or, when written additively, to natural multiplicities $n \cdot x, n \in \mathbb{N}$).

32.1 Notation and basic properties of sums of lists of monoid elements

In this section we setup a contex (locale) with notation for sums of lists of monoid elements and prove basic properties of those sums in terms of that notation.

The locale (context) `monoid1` extends the locale `monoid0`, adding the notation for the neutral element as `0` and the sum of a list of monoid elements. It also defines a notation for natural multiple of an element of a monoid, i.e. $n \cdot x = x \oplus x \oplus \dots \oplus x$ (n times).

```
locale monoid1 = monoid0 +
  fixes mzero (0)
  defines mzero_def [simp]: 0  $\equiv$  TheNeutralElement(G,f)

  fixes listsum ( $\sum$  _ 70)
  defines listsum_def [simp]:  $\sum$  s  $\equiv$  Fold(f,0,s)

  fixes nat_mult (infix . 72)
  defines nat_mult_def [simp]: n·x  $\equiv$   $\sum$  {⟨k,x⟩. k∈n}
```

Let's recall that the neutral element of the monoid is an element of the monoid (carrier) G and the monoid operation (f in our notation) is a function that maps $G \times G$ to G .

lemma (in monoid1) zero_monoid_oper: **shows** $0 \in G$ **and** $f: G \times G \rightarrow G$
<proof>

The sum of a list of monoid elements is a monoid element.

lemma (in monoid1) sum_in_mono: **assumes** $n \in \text{nat}$ $\forall k \in n. q(k) \in G$

shows $(\sum \{\langle k, q(k) \rangle. k \in n\}) \in G$
 $\langle proof \rangle$

A different expression of the fact the sum of a list valued in monoid is a monoid element.

lemma (in monoid1) sum_in_monoid: **assumes** $n \in \text{nat}$ $b: n \rightarrow G$
shows $(\sum b) \in G$
 $\langle proof \rangle$

The reason we start from 0 in the definition of the summation sign in the monoid1 locale is that we want to be able to sum the empty list. Such sum of the empty list is 0.

lemma (in monoid1) sum_empty: **assumes** $s: 0 \rightarrow G$ **shows** $(\sum s) = 0$
 $\langle proof \rangle$

For nonempty lists our Σ is the same as Fold1.

lemma (in monoid1) sum_nonempty: **assumes** $n \in \text{nat}$ $s: \text{succ}(n) \rightarrow G$
shows
 $(\sum s) = \text{Fold}(f, s(0), \text{Tail}(s))$
 $(\sum s) = \text{Fold1}(f, s)$
 $\langle proof \rangle$

We can pull the first component of a sum of a nonempty list of monoid elements before the summation sign.

lemma (in monoid1) seq_sum_pull_first0: **assumes** $n \in \text{nat}$ $s: \text{succ}(n) \rightarrow G$
shows $(\sum s) = s(0) \oplus (\sum \text{Tail}(s))$
 $\langle proof \rangle$

The first assertion of the next theorem is similar in content to seq_sum_pull_first0 formulated in terms of the expression defining the list of monoid elements. The second one shows the dual statement: the last element of a sequence can be pulled out of the sequence and put after the summation sign. So, we are showing here that $\sum_{k=0}^n q_k = q_0 \oplus \sum_{k=0}^{n-1} q_{k+1} = (\sum_{k=0}^{n-1} q_k) \oplus q_n$.

theorem (in monoid1) seq_sum_pull_one_elem:
assumes $n \in \text{nat}$ $\forall k \in n \# + 1. q(k) \in G$
shows
 $(\sum \{\langle k, q(k) \rangle. k \in n \# + 1\}) = q(0) \oplus (\sum \{\langle k, q(k \# + 1) \rangle. k \in n\})$
 $(\sum \{\langle k, q(k) \rangle. k \in n \# + 1\}) = (\sum \{\langle k, q(k) \rangle. k \in n\}) \oplus q(n)$
 $\langle proof \rangle$

If we append an element to a list then its sum increases by that element.

lemma (in monoid1) seq_sum_append:
assumes $n \in \text{nat}$ $s: n \rightarrow G$ $x \in G$
shows $(\sum \text{Append}(s, x)) = (\sum s) \oplus x$
 $\langle proof \rangle$

The sum of a nonempty list is the sum of its init plus the last element.

```

lemma (in monoid1) seq_sum_init_last:
  assumes n∈nat s:(n #+ 1)→G
  shows (∑ s) = (∑ Init(s))⊕(s(n))
  <proof>

```

Sum of concatenation of two lists is the sum of sums.

```

lemma (in monoid1) seq_sum_concat:
  assumes n∈nat s:n→G m∈nat q:m→G
  shows (∑ Concat(s,q)) = (∑ s)⊕(∑ q)
  <proof>

```

The sum of a singleton list is its only element,

```

lemma (in monoid1) seq_sum_singleton: assumes q(0) ∈ G
  shows (∑ {<k,q(k)>. k∈1}) = q(0)
  <proof>

```

If the monoid operation is commutative, then the sum of a nonempty sequence added to another sum of a nonempty sequence of the same length is equal to the sum of pointwise sums of the sequence elements. This is the same as the theorem `prod_comm_distrib` from the `Semigroup_ZF` theory, just written in the notation used in the `monoid1` locale.

```

lemma (in monoid1) sum_comm_distrib0:
  assumes f {is commutative on} G n∈nat and
  a : n #+ 1 → G b : n #+ 1 → G c : n #+ 1 → G and
  ∀j∈n #+ 1. c(j) = a(j) ⊕ b(j)
  shows (∑ c) = (∑ a) ⊕ (∑ b)
  <proof>

```

Another version of `sum_comm_distrib0` written in terms of the expressions defining the sequences, shows that for commutative monoids we have $\sum_{k=0}^{n-1} q(k) \oplus p(k) = (\sum_{k=0}^{n-1} p(k)) \oplus (\sum_{k=0}^{n-1} q(k))$.

```

theorem (in monoid1) sum_comm_distrib:
  assumes f {is commutative on} G n∈nat and
  ∀k∈n. p(k) ∈ G ∀k∈n. q(k) ∈ G
  shows
    (∑ {<k,p(k)⊕q(k)>. k∈n}) = (∑ {<k,p(k)>. k∈n}) ⊕ (∑ {<k,q(k)>. k∈n})
  <proof>

```

32.2 Multiplying monoid elements by natural numbers

A special case of summing (or, using more notation-neutral term folding) a list of monoid elements is taking a natural multiple of a single element. This can be applied to various monoids embedded in other algebraic structures. For example a ring is a monoid with addition as the operation, so the notion of natural multiple directly transfers there. Another monoid in a ring is

formed by its multiplication operation. In that case the natural multiple maps into natural powers of a ring element.

Another way of looking at a multiple of a monoid element: it's a sum of the cartesian product of n and the singleton $\{x\}$. This is because the expression $\{\langle k, x \rangle : k \in n\}$ in the definition of the notation for natural multiple i.e. a constant list of the length n is the same as the set $n \times \{x\}$.

lemma (in monoid1) monoid_nat_mult_def_alt: shows $n \cdot x = \sum n \times \{x\}$
<proof>

The zero's multiple of a monoid element is its neutral element.

lemma (in monoid1) nat_mult_zero: shows $0 \cdot x = 0$ *<proof>*

Any multiple of a monoid element is a monoid element.

lemma (in monoid1) nat_mult_type: assumes $n \in \text{nat}$ $x \in G$
 shows $n \cdot x \in G$ *<proof>*

Taking one more multiple of x adds x .

lemma (in monoid1) nat_mult_add_one: assumes $n \in \text{nat}$ $x \in G$
 shows $(n \#+ 1) \cdot x = n \cdot x \oplus x$ and $(n \#+ 1) \cdot x = x \oplus n \cdot x$
<proof>

One element of a monoid is that element.

lemma (in monoid1) nat_mult_one: assumes $x \in G$ shows $1 \cdot x = x$
<proof>

Multiplication of x by a natural number induces a homomorphism between natural numbers with addition and the natural multiples of x .

lemma (in monoid1) nat_mult_add: assumes $n \in \text{nat}$ $m \in \text{nat}$ $x \in G$
 shows $(n \#+ m) \cdot x = n \cdot x \oplus m \cdot x$
<proof>

The neutral element is fixed by this operation.

lemma (in monoid1) nat_mult_neutral: assumes $n \in \text{nat}$ shows $n \cdot 0 = 0$
<proof>

32.3 Chains weighted in a monoid

In the `FiniteSeq_ZF` theory we define the notions of **chains**, which are essentially lists of points of some set X and their **chain links**, which are lists of elements of $X \times X$. Here we consider what happens when the chain intervals are composed with a function valued in a monoid.

For a chain c and a weight function $w : X \times X \rightarrow G$ we define the chain weight as the sum of its intervals composed with w .

definition (in monoid1) ChainWeight where

$$\text{ChainWeight}(w, c) \equiv \sum (w \ 0 \ \text{ChainLinks}(c))$$

If c is a chain in X of a natural length n connecting x and y and if the weight function w maps $X \times X$ to the monoid G , then the weight of the chain c is an element of G .

lemma (in monoid1) chain_weight_type:
 assumes $n \in \text{nat}$ $c \in \text{Chains}(X, n, x, y)$ $w: X \times X \rightarrow G$
 shows $\text{ChainWeight}(w, c) \in G$
<proof>

The weight of the concatenation of chains is equal to the weight of the first chain plus the the weight of the link connecting the chains plus the weight of the second chain.

lemma (in monoid1) chain_concat_weight:
 assumes $n_1 \in \text{nat}$ $c_1 \in \text{Chains}(X, n_1, x_1, y_1)$ $n_2 \in \text{nat}$ $c_2 \in \text{Chains}(X, n_2, x_2, y_2)$
 and $w: X \times X \rightarrow G$
 shows $\text{ChainWeight}(w, \text{Concat}(c_1, c_2)) =$
 $\text{ChainWeight}(w, c_1) \oplus (w \langle y_1, x_2 \rangle) \oplus \text{ChainWeight}(w, c_2)$
<proof>

If, in addition to assumptions of chain_weight_concat the weight function w vanishes on the diagonal and the first chain ends at the beginning of the second one, then the weight of the concatenated chain is the sum of the chain weights.

lemma (in monoid1) chain_concat_weight1:
 assumes $n_1 \in \text{nat}$ $c_1 \in \text{Chains}(X, n_1, x, y)$ $n_2 \in \text{nat}$ $c_2 \in \text{Chains}(X, n_2, y, z)$
 and $w: X \times X \rightarrow G$ $\forall x \in X. w \langle x, x \rangle = 0$
 shows $\text{ChainWeight}(w, \text{Concat}(c_1, c_2)) = \text{ChainWeight}(w, c_1) \oplus \text{ChainWeight}(w, c_2)$
<proof>

end

33 Groups - introduction

theory Group_ZF imports Monoid_ZF_1 Semigroup_ZF

begin

This theory file covers basics of group theory.

33.1 Definition and basic properties of groups

In this section we define the notion of a group and set up the notation for discussing groups. We prove some basic theorems about groups.

To define a group we take a monoid and add a requirement that the right inverse needs to exist for every element of the group.

definition

$\text{IsAgroup}(G, f) \equiv$
 $(\text{IsAmonoid}(G, f) \wedge (\forall g \in G. \exists b \in G. f\langle g, b \rangle = \text{TheNeutralElement}(G, f)))$

We define the group inverse as the set $\{\langle x, y \rangle \in G \times G : x \cdot y = e\}$, where e is the neutral element of the group. This set (which can be written as $(\cdot)^{-1}\{e\}$) is a certain relation on the group (carrier). Since, as we show later, for every $x \in G$ there is exactly one $y \in G$ such that $x \cdot y = e$ this relation is in fact a function from G to G .

definition

$\text{GroupInv}(G, f) \equiv \{\langle x, y \rangle \in G \times G. f\langle x, y \rangle = \text{TheNeutralElement}(G, f)\}$

Next we define the context `group0` in which groups will be discussed. The common assumption for all proposition proven in the context `group0` is that fixed the pair (G, P) is a group. We will use the multiplicative notation for groups. The neutral element is denoted 1 . x^{-1} will denote the inverse of x , i.e. the value of the group inverse operation on x . We define the notation for product of a finite list of elements of G using the notion of `Fold`, defined in the `Fold_ZF` theory. Finally we define the notation `pow(n,x)` which is a product of the list of length n with value x repeated n times. Such list is a function that assigns x to every element of the set $n = \{0, 1, \dots, n-1\}$ and is represented by a set of pairs $\{\langle k, x \rangle : k \in n\}$, which is the same as the set $n \times \{x\}$ (see lemma `fun_def_alt1` in the `func1` theory).

```

locale group0 =
  fixes G
  fixes P
  assumes groupAssum: IsAgroup(G,P)

  fixes neut (1)
  defines neut_def[simp]: 1  $\equiv$  TheNeutralElement(G,P)

  fixes groper (infixl  $\cdot$  70)
  defines groper_def[simp]: a  $\cdot$  b  $\equiv$  P(a,b)

  fixes inv ( $_^{-1}$  [90] 91)
  defines inv_def[simp]:  $x^{-1} \equiv \text{GroupInv}(G,P)(x)$ 

  fixes listprod ( $\prod$  _ 70)
  defines listprod_def [simp]:  $\prod s \equiv \text{Fold}(P,1,s)$ 

  fixes pow
  defines pow_def [simp]: pow(n,x)  $\equiv \prod \{\langle k, x \rangle. k \in n\}$ 

```

First we show a lemma that says that we can use theorems proven in the `monoid0` context (locale).

```

lemma (in group0) group0_2_L1: shows monoid0(G,P)
  <proof>

```

Assumptions of the `monoid1` context are satisfied in the `group0` context.

```
lemma (in group0) monoid1_valid_in_group: shows monoid1(G,P)
  <proof>
```

The theorems proven in the `monoid1` context are valid in the `group0` context.

```
sublocale group0 < monoid: monoid1 G P groper 1 listprod pow
  <proof>
```

In some strange cases Isabelle has difficulties with applying the definition of a group. The next lemma defines a rule to be applied in such cases.

```
lemma definition_of_group: assumes IsAmonoid(G,f)
  and  $\forall g \in G. \exists b \in G. f(g,b) = \text{TheNeutralElement}(G,f)$ 
  shows IsAgroup(G,f)
  <proof>
```

A technical lemma that allows to use 1 as the neutral element of the group without referencing a list of lemmas and definitions.

```
lemma (in group0) group0_2_L2:
  shows  $1 \in G \wedge (\forall g \in G. (1 \cdot g = g \wedge g \cdot 1 = g))$ 
  <proof>
```

A technical lemma that 1 is in the set.

```
lemma (in group0) group0_2_L2_1:
  shows  $1 \in G$ 
  <proof>
```

A technical lemma that 1 is left unital.

```
lemma (in group0) group0_2_L2_2:
  assumes  $g \in G$ 
  shows  $1 \cdot g = g$ 
  <proof>
```

A technical lemma that 1 is right unital.

```
lemma (in group0) group0_2_L2_3:
  assumes  $g \in G$ 
  shows  $g \cdot 1 = g$ 
  <proof>
```

The group is closed under the group operation. Used all the time, useful to have handy.

```
lemma (in group0) group_op_closed: assumes  $a \in G \quad b \in G$ 
  shows  $a \cdot b \in G$  <proof>
```

The group operation is associative. This is another technical lemma that allows to shorten the list of referenced lemmas in some proofs.

```
lemma (in group0) group_oper_assoc:
```

assumes $a \in G \quad b \in G \quad c \in G$ **shows** $a \cdot (b \cdot c) = a \cdot b \cdot c$
<proof>

The group operation maps $G \times G$ into G . It is convenient to have this fact easily accessible in the `group0` context.

lemma (in `group0`) `group_oper_fun`: **shows** $P : G \times G \rightarrow G$
<proof>

The definition of a group requires the existence of the right inverse. We show that this is also the left inverse.

theorem (in `group0`) `group0_2_T1`:
assumes $A1: g \in G$ **and** $A2: b \in G$ **and** $A3: g \cdot b = 1$
shows $b \cdot g = 1$
<proof>

For every element of a group there is only one inverse.

lemma (in `group0`) `group0_2_L4`:
assumes $A1: x \in G$ **shows** $\exists ! y. y \in G \wedge x \cdot y = 1$
<proof>

The group inverse is a function that maps G into G .

theorem `group0_2_T2`:
assumes $A1: \text{IsAgroup}(G, f)$ **shows** $\text{GroupInv}(G, f) : G \rightarrow G$
<proof>

We can think about the group inverse (the function) as the inverse image of the neutral element. Recall that in Isabelle $f^{-1}(A)$ denotes the inverse image of the set A .

theorem (in `group0`) `group0_2_T3`: **shows** $P^{-1}\{1\} = \text{GroupInv}(G, P)$
<proof>

The inverse is in the group.

lemma (in `group0`) `inverse_in_group`: **assumes** $A1: x \in G$ **shows** $x^{-1} \in G$
<proof>

The notation for the inverse means what it is supposed to mean.

lemma (in `group0`) `group0_2_L6`:
assumes $A1: x \in G$ **shows** $x \cdot x^{-1} = 1 \wedge x^{-1} \cdot x = 1$
<proof>

The next two lemmas state that unless we multiply by the neutral element, the result is always different than any of the operands.

lemma (in `group0`) `group0_2_L7`:
assumes $A1: a \in G$ **and** $A2: b \in G$ **and** $A3: a \cdot b = a$
shows $b = 1$
<proof>

See the comment to group0_2_L7.

```
lemma (in group0) group0_2_L8:
  assumes A1: a∈G and A2: b∈G and A3: a·b = b
  shows a=1
<proof>
```

The inverse of the neutral element is the neutral element.

```
lemma (in group0) group_inv_of_one: shows  $1^{-1} = 1$ 
<proof>
```

Dividing by the neutral element does not change the dividend.

```
lemma (in group0) div_by_neutral: assumes x∈G shows  $x \cdot 1^{-1} = x$ 
<proof>
```

if $a^{-1} = 1$, then $a = 1$.

```
lemma (in group0) group0_2_L8A:
  assumes A1: a∈G and A2:  $a^{-1} = 1$ 
  shows a = 1
<proof>
```

If a is not a unit, then its inverse is not a unit either.

```
lemma (in group0) group0_2_L8B:
  assumes a∈G and  $a \neq 1$ 
  shows  $a^{-1} \neq 1$  <proof>
```

If a^{-1} is not a unit, then a is not a unit either.

```
lemma (in group0) group0_2_L8C:
  assumes a∈G and  $a^{-1} \neq 1$ 
  shows  $a \neq 1$ 
<proof>
```

If a product of two elements of a group is equal to the neutral element then they are inverses of each other.

```
lemma (in group0) group0_2_L9:
  assumes A1: a∈G and A2: b∈G and A3: a·b = 1
  shows a =  $b^{-1}$  and b =  $a^{-1}$ 
<proof>
```

It happens quite often that we know what is (have a meta-function for) the right inverse in a group. The next lemma shows that the value of the group inverse (function) is equal to the right inverse (meta-function).

```
lemma (in group0) group0_2_L9A:
  assumes A1:  $\forall g \in G. b(g) \in G \wedge g \cdot b(g) = 1$ 
  shows  $\forall g \in G. b(g) = g^{-1}$ 
<proof>
```

What is the inverse of a product?


```

lemma (in group0) group_inv_of_two:
  assumes A1:  $a \in G$  and A2:  $b \in G$ 
  shows  $b^{-1} \cdot a^{-1} = (a \cdot b)^{-1}$ 
  <proof>

```

What is the inverse of a product of three elements?

```

lemma (in group0) group_inv_of_three:
  assumes A1:  $a \in G$   $b \in G$   $c \in G$ 
  shows
     $(a \cdot b \cdot c)^{-1} = c^{-1} \cdot (a \cdot b)^{-1}$ 
     $(a \cdot b \cdot c)^{-1} = c^{-1} \cdot (b^{-1} \cdot a^{-1})$ 
     $(a \cdot b \cdot c)^{-1} = c^{-1} \cdot b^{-1} \cdot a^{-1}$ 
  <proof>

```

The inverse of the inverse is the element.

```

lemma (in group0) group_inv_of_inv:
  assumes  $a \in G$  shows  $a = (a^{-1})^{-1}$ 
  <proof>

```

Group inverse is nilpotent, therefore a bijection and involution.

```

lemma (in group0) group_inv_bij:
  shows  $\text{GroupInv}(G, P) \cap \text{GroupInv}(G, P) = \text{id}(G)$  and  $\text{GroupInv}(G, P) \in \text{bij}(G, G)$ 
and
   $\text{GroupInv}(G, P) = \text{converse}(\text{GroupInv}(G, P))$ 
  <proof>

```

A set comprehension form of the image of a set under the group inverse.

```

lemma (in group0) ginv_image: assumes  $V \subseteq G$ 
  shows  $\text{GroupInv}(G, P)(V) \subseteq G$  and  $\text{GroupInv}(G, P)(V) = \{g^{-1} \mid g \in V\}$ 
  <proof>

```

Inverse of an element that belongs to the inverse of the set belongs to the set.

```

lemma (in group0) ginv_image_el: assumes  $V \subseteq G$   $g \in \text{GroupInv}(G, P)(V)$ 
  shows  $g^{-1} \in V$ 
  <proof>

```

For the group inverse the image is the same as inverse image.

```

lemma (in group0) inv_image_vimage: shows  $\text{GroupInv}(G, P)(V) = \text{GroupInv}(G, P)^{-1}(V)$ 
  <proof>

```

If the unit is in a set then it is in the inverse of that set.

```

lemma (in group0) neut_inv_neut: assumes  $A \subseteq G$  and  $1 \in A$ 
  shows  $1 \in \text{GroupInv}(G, P)(A)$ 
  <proof>

```

The group inverse is onto.

lemma (in group0) group_inv_surj: shows GroupInv(G,P)(G) = G
 <proof>

If $a^{-1} \cdot b = 1$, then $a = b$.

lemma (in group0) group0_2_L11:
 assumes A1: $a \in G$ $b \in G$ and A2: $a^{-1} \cdot b = 1$
 shows $a = b$
 <proof>

If $a \cdot b^{-1} = 1$, then $a = b$.

lemma (in group0) group0_2_L11A:
 assumes A1: $a \in G$ $b \in G$ and A2: $a \cdot b^{-1} = 1$
 shows $a = b$
 <proof>

Inverses of different group elements are different.

lemma (in group0) el_neq_inv_neq: assumes $a \in G$ $b \in G$ $a \neq b$
 shows $a^{-1} \neq b^{-1}$
 <proof>

If the inverse of b is different than a , then the inverse of a is different than b .

lemma (in group0) group0_2_L11B:
 assumes A1: $a \in G$ and A2: $b^{-1} \neq a$
 shows $a^{-1} \neq b$
 <proof>

What is the inverse of ab^{-1} ?

lemma (in group0) group0_2_L12:
 assumes A1: $a \in G$ $b \in G$
 shows
 $(a \cdot b^{-1})^{-1} = b \cdot a^{-1}$
 $(a^{-1} \cdot b)^{-1} = b^{-1} \cdot a$
 <proof>

A couple useful rearrangements with three elements: we can insert a $b \cdot b^{-1}$ between two group elements (another version) and one about a product of an element and inverse of a product, and two others.

lemma (in group0) group0_2_L14A:
 assumes A1: $a \in G$ $b \in G$ $c \in G$
 shows
 $a \cdot c^{-1} = (a \cdot b^{-1}) \cdot (b \cdot c^{-1})$
 $a^{-1} \cdot c = (a^{-1} \cdot b) \cdot (b^{-1} \cdot c)$
 $a \cdot (b \cdot c)^{-1} = a \cdot c^{-1} \cdot b^{-1}$
 $a \cdot (b \cdot c^{-1}) = a \cdot b \cdot c^{-1}$
 $(a \cdot b^{-1} \cdot c^{-1})^{-1} = c \cdot b \cdot a^{-1}$
 $a \cdot b \cdot c^{-1} \cdot (c \cdot b^{-1}) = a$

$a \cdot (b \cdot c) \cdot c^{-1} = a \cdot b$
 $\langle proof \rangle$

A simple equation to solve

lemma (in group0) simple_equation0:
 assumes $a \in G \quad b \in G \quad c \in G \quad a \cdot b^{-1} = c^{-1}$
 shows $c = b \cdot a^{-1}$
 $\langle proof \rangle$

Another simple equation

lemma (in group0) simple_equation1:
 assumes $a \in G \quad b \in G \quad c \in G \quad a^{-1} \cdot b = c^{-1}$
 shows $c = b^{-1} \cdot a$
 $\langle proof \rangle$

Another lemma about rearranging a product of four group elements.

lemma (in group0) group0_2_L15:
 assumes A1: $a \in G \quad b \in G \quad c \in G \quad d \in G$
 shows $(a \cdot b) \cdot (c \cdot d)^{-1} = a \cdot (b \cdot d^{-1}) \cdot a^{-1} \cdot (a \cdot c^{-1})$
 $\langle proof \rangle$

We can cancel an element with its inverse that is written next to it.

lemma (in group0) inv_cancel_two:
 assumes A1: $a \in G \quad b \in G$
 shows
 $a \cdot b^{-1} \cdot b = a$
 $a \cdot b \cdot b^{-1} = a$
 $a^{-1} \cdot (a \cdot b) = b$
 $a \cdot (a^{-1} \cdot b) = b$
 $\langle proof \rangle$

Another lemma about cancelling with two group elements.

lemma (in group0) group0_2_L16A:
 assumes A1: $a \in G \quad b \in G$
 shows $a \cdot (b \cdot a)^{-1} = b^{-1}$
 $\langle proof \rangle$

Some other identities with three element and cancelling.

lemma (in group0) cancel_middle:
 assumes $a \in G \quad b \in G \quad c \in G$
 shows
 $(a \cdot b)^{-1} \cdot (a \cdot c) = b^{-1} \cdot c$
 $(a \cdot b) \cdot (c \cdot b)^{-1} = a \cdot c^{-1}$
 $a^{-1} \cdot (a \cdot b \cdot c) \cdot c^{-1} = b$
 $a \cdot (b \cdot c^{-1}) \cdot c = a \cdot b$
 $a \cdot b^{-1} \cdot (b \cdot c^{-1}) = a \cdot c^{-1}$
 $\langle proof \rangle$

Adding a neutral element to a set that is closed under the group operation results in a set that is closed under the group operation.

```
lemma (in group0) group0_2_L17:
  assumes  $H \subseteq G$ 
  and  $H$  {is closed under}  $P$ 
  shows  $(H \cup \{1\})$  {is closed under}  $P$ 
  <proof>
```

We can put an element on the other side of an equation.

```
lemma (in group0) group0_2_L18:
  assumes  $A1: a \in G \quad b \in G$ 
  and  $A2: c = a \cdot b$ 
  shows  $c \cdot b^{-1} = a \quad a^{-1} \cdot c = b$ 
  <proof>
```

We can cancel an element on the right from both sides of an equation.

```
lemma (in group0) cancel_right: assumes  $a \in G \quad b \in G \quad c \in G \quad a \cdot b = c \cdot b$ 
  shows  $a = c$ 
  <proof>
```

We can cancel an element on the left from both sides of an equation.

```
lemma (in group0) cancel_left: assumes  $a \in G \quad b \in G \quad c \in G \quad a \cdot b = a \cdot c$ 
  shows  $b = c$ 
  <proof>
```

Multiplying different group elements by the same factor results in different group elements.

```
lemma (in group0) group0_2_L19:
  assumes  $A1: a \in G \quad b \in G \quad c \in G$  and  $A2: a \neq b$ 
  shows  $a \cdot c \neq b \cdot c$  and  $c \cdot a \neq c \cdot b$ 
  <proof>
```

33.2 Subgroups

There are two common ways to define subgroups. One requires that the group operation is closed in the subgroup. The second one defines subgroup as a subset of a group which is itself a group under the group operations. We use the second approach because it results in shorter definition.

The rest of this section is devoted to proving the equivalence of these two definitions of the notion of a subgroup.

A pair (H, P) is a subgroup if H forms a group with the operation P restricted to $H \times H$. It may be surprising that we don't require H to be a subset of G . This however can be inferred from the definition if the pair (G, P) is a group, see lemma group0_3_L2.

definition

$\text{IsSubgroup}(H,P) \equiv \text{IsGroup}(H,\text{restrict}(P,H \times H))$

The group is its own subgroup.

lemma (in group0) group_self_subgroup: shows IsSubgroup(G,P)
<proof>

Formally the group operation in a subgroup is different than in the group as they have different domains. Of course we want to use the original operation with the associated notation in the subgroup. The next couple of lemmas will allow for that.

The next lemma states that the neutral element of a subgroup is in the subgroup and it is both right and left neutral there. The notation is very ugly because we don't want to introduce a separate notation for the subgroup operation.

lemma group0_3_L1:
 assumes A1: IsSubgroup(H,f)
 and A2: n = TheNeutralElement(H,restrict(f,H×H))
 shows $n \in H$
 $\forall h \in H. \text{restrict}(f,H \times H)(n,h) = h$
 $\forall h \in H. \text{restrict}(f,H \times H)(h,n) = h$
<proof>

A subgroup is contained in the group.

lemma (in group0) group0_3_L2:
 assumes A1: IsSubgroup(H,P)
 shows $H \subseteq G$
<proof>

The group's neutral element (denoted 1 in the group0 context) is a neutral element for the subgroup with respect to the group action.

lemma (in group0) group0_3_L3:
 assumes IsSubgroup(H,P)
 shows $\forall h \in H. 1 \cdot h = h \wedge h \cdot 1 = h$
<proof>

The neutral element of a subgroup is the same as that of the group.

lemma (in group0) group0_3_L4: assumes A1: IsSubgroup(H,P)
 shows TheNeutralElement(H,restrict(P,H×H)) = 1
<proof>

The neutral element of the group (denoted 1 in the group0 context) belongs to every subgroup.

lemma (in group0) group0_3_L5: assumes A1: IsSubgroup(H,P)
 shows $1 \in H$
<proof>

Subgroups are closed with respect to the group operation.

```

lemma (in group0) group0_3_L6: assumes A1: IsAsubgroup(H,P)
  and A2: a∈H b∈H
  shows a·b ∈ H
⟨proof⟩

```

A preliminary lemma that we need to show that taking the inverse in the subgroup is the same as taking the inverse in the group.

```

lemma group0_3_L7A:
  assumes A1: IsAgroup(G,f)
  and A2: IsAsubgroup(H,f) and A3: g = restrict(f,H×H)
  shows GroupInv(G,f) ∩ H×H = GroupInv(H,g)
⟨proof⟩

```

Using the lemma above we can show the actual statement: taking the inverse in the subgroup is the same as taking the inverse in the group.

```

theorem (in group0) group0_3_T1:
  assumes A1: IsAsubgroup(H,P)
  and A2: g = restrict(P,H×H)
  shows GroupInv(H,g) = restrict(GroupInv(G,P),H)
⟨proof⟩

```

A slightly weaker, but more convenient in applications, reformulation of the above theorem.

```

theorem (in group0) group0_3_T2:
  assumes IsAsubgroup(H,P)
  and g = restrict(P,H×H)
  shows ∀h∈H. GroupInv(H,g)(h) = h-1
⟨proof⟩

```

Subgroups are closed with respect to taking the group inverse.

```

theorem (in group0) group0_3_T3A:
  assumes A1: IsAsubgroup(H,P) and A2: h∈H
  shows h-1∈ H
⟨proof⟩

```

The next theorem states that a nonempty subset of a group G that is closed under the group operation and taking the inverse is a subgroup of the group.

```

theorem (in group0) group0_3_T3:
  assumes A1: H≠∅
  and A2: H⊆G
  and A3: H {is closed under} P
  and A4: ∀x∈H. x-1 ∈ H
  shows IsAsubgroup(H,P)
⟨proof⟩

```

The singleton with the neutral element is a subgroup.

```

corollary (in group0) unit_singl_subgr:

```

```

shows IsAsubgroup({1},P)
<proof>

```

Intersection of subgroups is a subgroup. This lemma is obsolete and should be replaced by `subgroup_inter`.

```

lemma group0_3_L7:
  assumes A1: IsAgroup(G,f)
  and A2: IsAsubgroup(H1,f)
  and A3: IsAsubgroup(H2,f)
  shows IsAsubgroup(H1∩H2,restrict(f,H1×H1))
<proof>

```

Intersection of subgroups is a subgroup.

```

lemma (in group0) subgroup_inter: assumes  $\mathcal{H} \neq 0$ 
  and  $\forall H \in \mathcal{H}. \text{IsAsubgroup}(H,P)$ 
  shows IsAsubgroup( $\bigcap \mathcal{H}$ ,P)
<proof>

```

The range of the subgroup operation is the whole subgroup.

```

lemma image_subgr_op: assumes A1: IsAsubgroup(H,P)
  shows restrict(P,H×H)(H×H) = H
<proof>

```

If we restrict the inverse to a subgroup, then the restricted inverse is onto the subgroup.

```

lemma (in group0) restr_inv_onto: assumes A1: IsAsubgroup(H,P)
  shows restrict(GroupInv(G,P),H)(H) = H
<proof>

```

A union of two subgroups is a subgroup iff one of the subgroups is a subset of the other subgroup.

```

lemma (in group0) union_subgroups:
  assumes IsAsubgroup(H1,P) and IsAsubgroup(H2,P)
  shows IsAsubgroup(H1∪H2,P)  $\longleftrightarrow$  (H1⊆H2 ∨ H2⊆H1)
<proof>

```

Transitivity for "is a subgroup of" relation. The proof (probably) uses the lemma `restrict_restrict` from standard Isabelle/ZF library which states that `restrict(restrict(f,A),B) = restrict(f,A∩B)`. That lemma is added to the simplifier, so it does not have to be referenced explicitly in the proof below.

```

lemma subgroup_transitive:
  assumes IsAgroup(G3,P) IsAsubgroup(G2,P) IsAsubgroup(G1,restrict(P,G2×G2))
  shows IsAsubgroup(G1,P)
<proof>

```

33.3 Groups vs. loops

We defined groups as monoids with the inverse operation. An alternative way of defining a group is as a loop whose operation is associative.

Groups have left and right division.

lemma (in group0) gr_has_lr_div: shows HasLeftDiv(G,P) and HasRightDiv(G,P)
<proof>

A group is a quasigroup and a loop.

lemma (in group0) group_is_loop: shows IsAQuasigroup(G,P) and IsALoop(G,P)
<proof>

An associative loop is a group.

theorem assoc_loop_is_gr: assumes IsALoop(G,P) and P {is associative
on} G
shows IsAGroup(G,P)
<proof>

For groups the left and right inverse are the same as the group inverse.

lemma (in group0) lr_inv_gr_inv:
shows LeftInv(G,P) = GroupInv(G,P) and RightInv(G,P) = GroupInv(G,P)
<proof>

33.4 Product of a list of group elements

The `group0` context defines notation for a product of a finite list of group elements. This sections shows basic properties of that notation.

The assumptions of the `semigr0` context hold in the `group0` context.

lemma (in group0) semigr0_valid_in_group0: shows semigr0(G,P)
<proof>

Since semigroups do not have a neutral element the product operator in the `semigr0` is defined for nonempty lists only as `Fold1(f,a)`, where f is the semigroup operation and a is a (nonempty) list. This is a bit different from the product of a finite list in the `group0` context. The next lemma helps in translating between these two notations by asserting that in the `group0` context the sum of a finite list s is the same as `Fold1(s1)` where s_1 is the list s with the neutral element of the group prepended to it.

lemma (in group0) list_prod_as_fold1: assumes $n \in \text{nat}$ $s: n \rightarrow G$
shows $(\prod s) = \text{Fold1}(P, \text{Prepend}(s, 1))$
<proof>

For nonempty lists the product is the the same as `Fold1` of the list with respect to the operation.


```

lemma (in group0) nempty_list_prod_as_fold1: assumes  $n \in \text{nat}$   $s : (n \#+ 1) \rightarrow G$ 
shows  $(\prod s) = \text{Fold1}(P, s)$ 
 $\langle \text{proof} \rangle$ 

```

The product of a list is an element of the group.

```

lemma (in group0) list_prod_in_group: assumes  $n \in \text{nat}$   $s : n \rightarrow G$ 
shows  $(\prod s) \in G$ 
 $\langle \text{proof} \rangle$ 

```

Product of a singleton list is its only element.

```

lemma (in group0) prod_singleton: assumes  $s : 1 \rightarrow G$ 
shows  $(\prod s) = s(0)$ 
 $\langle \text{proof} \rangle$ 

```

The `group0` locale defines a notation for a natural power of a group element. The next lemma provides two alternative definitions for that notation: a natural power of a group element x is the product of the constant list $n\{x\}$, i.e. the fold of the group operation starting from the neutral element of $n\{x\}$. It's really the `monoid_nat_mult_def_alt` lemma from `Monoid_ZF_1` theory, just written in the multiplicative notation used in the `group0` context.

```

lemma (in group0) group_nat_pow_def_alt:
shows  $\text{pow}(n, x) = \prod n \times \{x\}$  and  $\text{pow}(n, x) = \text{Fold}(P, 1, n \times \{x\})$ 
 $\langle \text{proof} \rangle$ 

```

A natural power of an element of the group is an element of the group. This is really lemma `nat_mult_type` from `Monoid_ZF_1` theory written in multiplicative notation.

```

lemma (in group0) nat_pow_type: assumes  $n \in \text{nat}$   $x \in G$ 
shows  $\text{pow}(n, x) \in G$   $\langle \text{proof} \rangle$ 

```

x raised to the power $n + 1$ can be written as $x \cdot x^n$ or $(x^n) \cdot x$. This is just lemma `nat_mult_add_one` from `Monoid_ZF_1` theory written in multiplicative notation.

```

lemma (in group0) nat_pow_add_one: assumes  $n \in \text{nat}$   $x \in G$ 
shows  $\text{pow}(n \#+ 1, x) = \text{pow}(n, x) \cdot x$  and  $\text{pow}(n \#+ 1, x) = x \cdot \text{pow}(n, x)$ 
 $\langle \text{proof} \rangle$ 

```

x raised to the power $n + m$ is the product of powers x^n and x^m . This is really lemma `nat_mult_add` from `Monoid_ZF_1` theory written in multiplicative notation.

```

lemma (in group0) nat_pow_sum_exps: assumes  $n \in \text{nat}$   $m \in \text{nat}$   $x \in G$ 
shows  $\text{pow}(n \#+ m, x) = \text{pow}(n, x) \cdot \text{pow}(m, x)$ 
 $\langle \text{proof} \rangle$ 

```

33.5 An alternative definition of groups

In a typical textbook a group is defined as a set G with an associative operation such that two conditions hold:

A: there is an element $e \in G$ such that for all $g \in G$ we have $e \cdot g = g$ and $g \cdot e = g$. We call this element a "unit" or a "neutral element" of the group.
 B: for every $a \in G$ there exists a $b \in G$ such that $a \cdot b = e$, where e is the element of G whose existence is guaranteed by A.

The validity of this definition is rather dubious to me, as condition A does not define any specific element e that can be referred to in condition B - it merely states that a set of such units e is not empty. Of course it does work in the end as we can prove that the set of such neutral elements has exactly one element, but still the definition by itself is not valid. You just can't reference a variable bound by a quantifier outside of the scope of that quantifier.

One way around this is to first use condition A to define the notion of a monoid, then prove the uniqueness of e and then use the condition B to define groups.

Another way is to write conditions A and B together as follows:

$$\exists e \in G (\forall g \in G e \cdot g = g \wedge g \cdot e = g) \wedge (\forall a \in G \exists b \in G a \cdot b = e).$$

This is rather ugly.

What I want to talk about is an amusing way to define groups directly without any reference to the neutral elements. Namely, we can define a group as a non-empty set G with an associative operation " $\cdot$ " such that

C: for every $a, b \in G$ the equations $a \cdot x = b$ and $y \cdot a = b$ can be solved in G .

This theory file aims at proving the equivalence of this alternative definition with the usual definition of the group, as formulated in `Group_ZF.thy`. The informal proofs come from an Aug. 14, 2005 post by buli on the matematyka.org forum.

We will use the multiplicative notation for the group operation. To do this, we define a context (locale) that tells Isabelle to interpret $a \cdot b$ as the value of function P on the pair $\langle a, b \rangle$.

```
locale group2 =
  fixes P
  fixes dot (infixl · 70)
  defines dot_def [simp]: a · b ≡ P⟨a,b⟩
```

The next theorem states that a set G with an associative operation that satisfies condition C is a group, as defined in IsarMathLib `Group_ZF` theory.

```
theorem (in group2) altgroup_is_group:
  assumes A1: G ≠ 0 and A2: P {is associative on} G
  and A3: ∀ a ∈ G. ∀ b ∈ G. ∃ x ∈ G. a · x = b
  and A4: ∀ a ∈ G. ∀ b ∈ G. ∃ y ∈ G. y · a = b
  shows IsAgroup(G,P)
  ⟨proof⟩
```

The converse of `altgroup_is_group`: in every (classically defined) group con-

dition C holds. In informal mathematics we can say "Obviously condition C holds in any group." In formalized mathematics the word "obviously" is not in the language. The next theorem is proven in the context called `group0` defined in the theory `Group_ZF.thy`. Similarly to the `group2` that context defines $a \cdot b$ as $P\langle a, b \rangle$. It also defines notation related to the group inverse and adds an assumption that the pair (G, P) is a group to all its theorems. This is why in the next theorem we don't explicitly assume that (G, P) is a group - this assumption is implicit in the context.

```
theorem (in group0) group_is_altgroup: shows
   $\forall a \in G. \forall b \in G. \exists x \in G. a \cdot x = b$  and  $\forall a \in G. \forall b \in G. \exists y \in G. y \cdot a = b$ 
 $\langle proof \rangle$ 
```

An associative quasigroup is a group. This is a bit weaker than `altgroup_is_group` as the definition of quasigroup requires uniqueness of solutions of $a \cdot x = b$ and $y \cdot a = b$ equations.

```
lemma assoc_quasigroup_group:
  assumes IsAQuasigroup( $G, P$ ) and  $P$  {is associative on}  $G$   $G \neq \emptyset$ 
  shows IsAGroup( $G, P$ )
 $\langle proof \rangle$ 
```

end

34 Groups 1

```
theory Group_ZF_1 imports Group_ZF
```

```
begin
```

In this theory we consider right and left translations and odd functions.

34.1 Translations

In this section we consider translations. Translations are maps $T : G \rightarrow G$ of the form $T_g(a) = g \cdot a$ or $T_g(a) = a \cdot g$. We also consider two-dimensional translations $T_g : G \times G \rightarrow G \times G$, where $T_g(a, b) = (a \cdot g, b \cdot g)$ or $T_g(a, b) = (g \cdot a, g \cdot b)$.

For an element $a \in G$ the right translation is defined a function (set of pairs) such that its value (the second element of a pair) is the value of the group operation on the first element of the pair and g . This looks a bit strange in the raw set notation, when we write a function explicitly as a set of pairs and value of the group operation on the pair $\langle a, b \rangle$ as $P\langle a, b \rangle$ instead of the usual infix $a \cdot b$ or $a + b$.

definition

```
RightTranslation( $G, P, g$ )  $\equiv \{ \langle a, b \rangle \in G \times G. P\langle a, g \rangle = b \}$ 
```

A similar definition of the left translation.

definition

$$\text{LeftTranslation}(G, P, g) \equiv \{\langle a, b \rangle \in G \times G. P\langle g, a \rangle = b\}$$

Translations map G into G . Two dimensional translations map $G \times G$ into itself.

lemma (in group0) group0_5_L1: assumes A1: $g \in G$
 shows $\text{RightTranslation}(G, P, g) : G \rightarrow G$ and $\text{LeftTranslation}(G, P, g) : G \rightarrow G$
<proof>

The values of the translations are what we expect.

lemma (in group0) group0_5_L2: assumes $g \in G$ $a \in G$
 shows
 $\text{RightTranslation}(G, P, g)(a) = a \cdot g$
 $\text{LeftTranslation}(G, P, g)(a) = g \cdot a$
<proof>

Composition of left translations is a left translation by the product.

lemma (in group0) group0_5_L4: assumes A1: $g \in G$ $h \in G$ $a \in G$ and
 A2: $T_g = \text{LeftTranslation}(G, P, g)$ $T_h = \text{LeftTranslation}(G, P, h)$
 shows
 $T_g(T_h(a)) = g \cdot h \cdot a$
 $T_g(T_h(a)) = \text{LeftTranslation}(G, P, g \cdot h)(a)$
<proof>

Composition of right translations is a right translation by the product.

lemma (in group0) group0_5_L5: assumes A1: $g \in G$ $h \in G$ $a \in G$ and
 A2: $T_g = \text{RightTranslation}(G, P, g)$ $T_h = \text{RightTranslation}(G, P, h)$
 shows
 $T_g(T_h(a)) = a \cdot h \cdot g$
 $T_g(T_h(a)) = \text{RightTranslation}(G, P, h \cdot g)(a)$
<proof>

Point free version of group0_5_L4 and group0_5_L5.

lemma (in group0) trans_comp: assumes $g \in G$ $h \in G$ shows
 $\text{RightTranslation}(G, P, g) \circ \text{RightTranslation}(G, P, h) = \text{RightTranslation}(G, P, h \cdot g)$
 $\text{LeftTranslation}(G, P, g) \circ \text{LeftTranslation}(G, P, h) = \text{LeftTranslation}(G, P, g \cdot h)$
<proof>

The image of a set under a composition of translations is the same as the image under translation by a product.

lemma (in group0) trans_comp_image: assumes A1: $g \in G$ $h \in G$ and
 A2: $T_g = \text{LeftTranslation}(G, P, g)$ $T_h = \text{LeftTranslation}(G, P, h)$
 shows $T_g(T_h(A)) = \text{LeftTranslation}(G, P, g \cdot h)(A)$
<proof>

Another form of the image of a set under a composition of translations

```

lemma (in group0) group0_5_L6:
  assumes A1:  $g \in G$   $h \in G$  and A2:  $A \subseteq G$  and
  A3:  $T_g = \text{RightTranslation}(G, P, g)$   $T_h = \text{RightTranslation}(G, P, h)$ 
  shows  $T_g(T_h(A)) = \{a \cdot h \cdot g. a \in A\}$ 
  <proof>

```

The translation by neutral element is the identity on group.

```

lemma (in group0) trans_neutral: shows
   $\text{RightTranslation}(G, P, 1) = \text{id}(G)$  and  $\text{LeftTranslation}(G, P, 1) = \text{id}(G)$ 
  <proof>

```

Translation by neutral element does not move sets.

```

lemma (in group0) trans_neutral_image: assumes  $V \subseteq G$ 
  shows  $\text{RightTranslation}(G, P, 1)(V) = V$  and  $\text{LeftTranslation}(G, P, 1)(V)$ 
   $= V$ 
  <proof>

```

Composition of translations by an element and its inverse is identity.

```

lemma (in group0) trans_comp_id: assumes  $g \in G$  shows
   $\text{RightTranslation}(G, P, g) \circ \text{RightTranslation}(G, P, g^{-1}) = \text{id}(G)$  and
   $\text{RightTranslation}(G, P, g^{-1}) \circ \text{RightTranslation}(G, P, g) = \text{id}(G)$  and
   $\text{LeftTranslation}(G, P, g) \circ \text{LeftTranslation}(G, P, g^{-1}) = \text{id}(G)$  and
   $\text{LeftTranslation}(G, P, g^{-1}) \circ \text{LeftTranslation}(G, P, g) = \text{id}(G)$ 
  <proof>

```

Translations are bijective.

```

lemma (in group0) trans_bij: assumes  $g \in G$  shows
   $\text{RightTranslation}(G, P, g) \in \text{bij}(G, G)$  and  $\text{LeftTranslation}(G, P, g) \in \text{bij}(G, G)$ 
  <proof>

```

Converse of a translation is translation by the inverse.

```

lemma (in group0) trans_conv_inv: assumes  $g \in G$  shows
   $\text{converse}(\text{RightTranslation}(G, P, g)) = \text{RightTranslation}(G, P, g^{-1})$  and
   $\text{converse}(\text{LeftTranslation}(G, P, g)) = \text{LeftTranslation}(G, P, g^{-1})$  and
   $\text{LeftTranslation}(G, P, g) = \text{converse}(\text{LeftTranslation}(G, P, g^{-1}))$  and
   $\text{RightTranslation}(G, P, g) = \text{converse}(\text{RightTranslation}(G, P, g^{-1}))$ 
  <proof>

```

The image of a set by translation is the same as the inverse image by the inverse element translation.

```

lemma (in group0) trans_image_vimage: assumes  $g \in G$  shows
   $\text{LeftTranslation}(G, P, g)(A) = \text{LeftTranslation}(G, P, g^{-1})^{-1}(A)$  and
   $\text{RightTranslation}(G, P, g)(A) = \text{RightTranslation}(G, P, g^{-1})^{-1}(A)$ 
  <proof>

```

Another way of looking at translations is that they are sections of the group operation.

lemma (in group0) trans_eq_section: assumes $g \in G$ shows
 $\text{RightTranslation}(G, P, g) = \text{Fix2ndVar}(P, g)$ and
 $\text{LeftTranslation}(G, P, g) = \text{Fix1stVar}(P, g)$
<proof>

A lemma demonstrating what is the left translation of a set

lemma (in group0) ltrans_image: assumes $A1: V \subseteq G$ and $A2: x \in G$
shows $\text{LeftTranslation}(G, P, x)(V) = \{x \cdot v. v \in V\}$
<proof>

A lemma demonstrating what is the right translation of a set

lemma (in group0) rtrans_image: assumes $A1: V \subseteq G$ and $A2: x \in G$
shows $\text{RightTranslation}(G, P, x)(V) = \{v \cdot x. v \in V\}$
<proof>

Right and left translations of a set are subsets of the group. Interestingly, we do not have to assume the set is a subset of the group.

lemma (in group0) lrtrans_in_group: assumes $x \in G$
shows $\text{LeftTranslation}(G, P, x)(V) \subseteq G$ and $\text{RightTranslation}(G, P, x)(V) \subseteq G$
<proof>

A technical lemma about solving equations with translations.

lemma (in group0) ltrans_inv_in: assumes $A1: V \subseteq G$ and $A2: y \in G$ and
 $A3: x \in \text{LeftTranslation}(G, P, y)(\text{GroupInv}(G, P)(V))$
shows $y \in \text{LeftTranslation}(G, P, x)(V)$
<proof>

We can look at the result of interval arithmetic operation as union of left translated sets.

lemma (in group0) image_ltrans_union: assumes $A \subseteq G$ $B \subseteq G$ shows
 $(P \text{ \{lifted to subsets of\} } G)(A, B) = (\bigcup a \in A. \text{LeftTranslation}(G, P, a)(B))$
<proof>

The right translation version of image_ltrans_union The proof follows the same schema.

lemma (in group0) image_rtrans_union: assumes $A \subseteq G$ $B \subseteq G$ shows
 $(P \text{ \{lifted to subsets of\} } G)(A, B) = (\bigcup b \in B. \text{RightTranslation}(G, P, b)(A))$
<proof>

If the neutral element belongs to a set, then an element of group belongs the translation of that set.

lemma (in group0) neut_trans_elem:
assumes $A1: A \subseteq G$ $g \in G$ and $A2: 1 \in A$
shows $g \in \text{LeftTranslation}(G, P, g)(A)$ $g \in \text{RightTranslation}(G, P, g)(A)$
<proof>

The neutral element belongs to the translation of a set by the inverse of an element that belongs to it.

```
lemma (in group0) elem_trans_neut: assumes A1:  $A \subseteq G$  and A2:  $g \in A$ 
  shows  $1 \in \text{LeftTranslation}(G, P, g^{-1})(A)$   $1 \in \text{RightTranslation}(G, P, g^{-1})(A)$ 
  <proof>
```

34.2 Odd functions

This section is about odd functions.

Odd functions are those that commute with the group inverse: $f(a^{-1}) = (f(a))^{-1}$.

definition

$$\text{IsOdd}(G, P, f) \equiv (\forall a \in G. f(\text{GroupInv}(G, P)(a)) = \text{GroupInv}(G, P)(f(a)))$$

Let's see the definition of an odd function in a more readable notation.

```
lemma (in group0) group0_6_L1:
  shows  $\text{IsOdd}(G, P, p) \longleftrightarrow (\forall a \in G. p(a^{-1}) = (p(a))^{-1})$ 
  <proof>
```

We can express the definition of an odd function in two ways.

```
lemma (in group0) group0_6_L2:
  assumes A1:  $p : G \rightarrow G$ 
  shows
     $(\forall a \in G. p(a^{-1}) = (p(a))^{-1}) \longleftrightarrow (\forall a \in G. (p(a^{-1}))^{-1} = p(a))$ 
  <proof>
```

34.3 Subgroups and interval arithmetic

The section `Binary operations` in the `func_ZF` theory defines the notion of "lifting operation to subsets". In short, every binary operation $f : X \times X \rightarrow X$ on a set X defines an operation on the subsets of X defined by $F(A, B) = \{f(x, y) | x \in A, y \in B\}$. In the group context using multiplicative notation we can write this as $H \cdot K = \{x \cdot y | x \in A, y \in B\}$. Similarly we can define $H^{-1} = \{x^{-1} | x \in H\}$. In this section we study properties of these derived operations and how they relate to the concept of subgroups.

The next locale extends the `groups0` locale with notation related to interval arithmetics.

```
locale group4 = group0 +
  fixes sdot (infixl  $\cdot$  70)
  defines sdot_def [simp]:  $A \cdot B \equiv (P \ \{\text{lifted to subsets of}\} \ G)(A, B)$ 

  fixes sinv ( $_^{-1}$  [90] 91)
  defines sinv_def [simp]:  $A^{-1} \equiv \text{GroupInv}(G, P)(A)$ 
```

The next lemma shows a somewhat more explicit way of defining the product of two subsets of a group.

```
lemma (in group4) interval_prod: assumes  $A \subseteq G$   $B \subseteq G$ 
  shows  $A \cdot B = \{x \cdot y. \langle x, y \rangle \in A \times B\}$ 
  <proof>
```

Product of elements of subsets of the group is in the set product of those subsets

```
lemma (in group4) interval_prod_el: assumes  $A \subseteq G$   $B \subseteq G$   $x \in A$   $y \in B$ 
  shows  $x \cdot y \in A \cdot B$ 
  <proof>
```

An alternative definition of a group inverse of a set.

```
lemma (in group4) interval_inv: assumes  $A \subseteq G$ 
  shows  $A^{-1} = \{x^{-1}. x \in A\}$ 
  <proof>
```

Group inverse of a set is a subset of the group. Interestingly we don't need to assume the set is a subset of the group.

```
lemma (in group4) interval_inv_cl: shows  $A^{-1} \subseteq G$ 
  <proof>
```

The product of two subsets of a group is a subset of the group.

```
lemma (in group4) interval_prod_closed: assumes  $A \subseteq G$   $B \subseteq G$ 
  shows  $A \cdot B \subseteq G$ 
  <proof>
```

The product of sets operation is associative.

```
lemma (in group4) interval_prod_assoc: assumes  $A \subseteq G$   $B \subseteq G$   $C \subseteq G$ 
  shows  $A \cdot B \cdot C = A \cdot (B \cdot C)$ 
  <proof>
```

A simple rearrangement following from associativity of the product of sets operation.

```
lemma (in group4) interval_prod_rearr1: assumes  $A \subseteq G$   $B \subseteq G$   $C \subseteq G$   $D \subseteq G$ 
  shows  $A \cdot B \cdot (C \cdot D) = A \cdot (B \cdot C) \cdot D$ 
  <proof>
```

A subset A of the group is closed with respect to the group operation iff $A \cdot A \subseteq A$.

```
lemma (in group4) subset_gr_op_cl: assumes  $A \subseteq G$ 
  shows  $(A \text{ \{is closed under\} } P) \longleftrightarrow A \cdot A \subseteq A$ 
  <proof>
```

Inverse and square of a subgroup is this subgroup.

```
lemma (in group4) subgroup_inv_sq: assumes IsSubgroup(H,P)
```



```

    shows  $H^{-1} = H$  and  $H \cdot H = H$ 
  <proof>

```

Inverse of a product two sets is a product of inverses with the reversed order.

```

lemma (in group4) interval_prod_inv: assumes  $A \subseteq G$   $B \subseteq G$ 
  shows
     $(A \cdot B)^{-1} = \{(x \cdot y)^{-1} \cdot \langle x, y \rangle \in A \times B\}$ 
     $(A \cdot B)^{-1} = \{y^{-1} \cdot x^{-1} \cdot \langle x, y \rangle \in A \times B\}$ 
     $(A \cdot B)^{-1} = (B^{-1}) \cdot (A^{-1})$ 
  <proof>

```

If H, K are subgroups then $H \cdot K$ is a subgroup iff $H \cdot K = K \cdot H$.

```

theorem (in group4) prod_subgr_subgr:
  assumes IsAsubgroup( $H, P$ ) and IsAsubgroup( $K, P$ )
  shows IsAsubgroup( $H \cdot K, P$ )  $\longleftrightarrow$   $H \cdot K = K \cdot H$ 
  <proof>

```

end

35 Abelian Group

```

theory AbelianGroup_ZF imports Group_ZF

```

```

begin

```

A group is called “abelian” if its operation is commutative, i.e. $P\langle a, b \rangle = P\langle a, b \rangle$ for all group elements a, b , where P is the group operation. It is customary to use the additive notation for abelian groups, so this condition is typically written as $a + b = b + a$. We will be using multiplicative notation though (in which the commutativity condition of the operation is written as $a \cdot b = b \cdot a$), just to avoid the hassle of changing the notation we used for general groups.

35.1 Rearrangement formulae

This section is not interesting and should not be read. Here we will prove formulas in which right hand side uses the same factors as the left hand side, just in different order. These facts are obvious in informal math sense, but Isabelle prover is not able to derive them automatically, so we have to provide explicit proofs.

Proving the facts about associative and commutative operations is quite tedious in formalized mathematics. To a human the thing is simple: we can arrange the elements in any order and put parentheses wherever we want, it is all the same. However, formalizing this statement would be rather difficult (I think). The next lemma attempts a quasi-algorithmic approach

to this type of problem. To prove that two expressions are equal, we first strip one from parentheses, then rearrange the elements in proper order, then put the parentheses where we want them to be. The algorithm for rearrangement is easy to describe: we keep putting the first element (from the right) that is in the wrong place at the left-most position until we get the proper arrangement. As far removing parentheses is concerned Isabelle does its job automatically.

```
lemma (in group0) group0_4_L2:
  assumes A1:P {is commutative on} G
  and A2:a∈G b∈G c∈G d∈G E∈G F∈G
  shows (a·b)·(c·d)·(E·F) = (a·(d·F))·(b·(c·E))
⟨proof⟩
```

Another useful rearrangement.

```
lemma (in group0) group0_4_L3:
  assumes A1:P {is commutative on} G
  and A2: a∈G b∈G and A3: c∈G d∈G E∈G F∈G
  shows a·b·((c·d)-1·(E·F)-1) = (a·(E·c)-1)·(b·(F·d)-1)
⟨proof⟩
```

Some useful rearrangements for two elements of a group.

```
lemma (in group0) group0_4_L4:
  assumes A1:P {is commutative on} G
  and A2: a∈G b∈G
  shows
    b-1·a-1 = a-1·b-1
    (a·b)-1 = a-1·b-1
    (a·b-1)-1 = a-1·b
⟨proof⟩
```

Another bunch of useful rearrangements with three elements.

```
lemma (in group0) group0_4_L4A:
  assumes A1: P {is commutative on} G
  and A2: a∈G b∈G c∈G
  shows
    a·b·c = c·a·b
    a-1·(b-1·c-1)-1 = (a·(b·c)-1)-1
    a·(b·c)-1 = a·b-1·c-1
    a·(b·c-1)-1 = a·b-1·c
    a·b-1·c-1 = a·c-1·b-1
⟨proof⟩
```

Another useful rearrangement.

```
lemma (in group0) group0_4_L4B:
  assumes P {is commutative on} G
  and a∈G b∈G c∈G
  shows a·b-1·(b·c-1) = a·c-1
```

⟨proof⟩

A couple of permutations of order for three elements.

```
lemma (in group0) group0_4_L4C:
  assumes A1: P {is commutative on} G
  and A2: a∈G b∈G c∈G
  shows
    a·b·c = c·a·b
    a·b·c = a·(c·b)
    a·b·c = c·(a·b)
    a·b·c = c·b·a
```

⟨proof⟩

Some rearrangement with three elements and inverse.

```
lemma (in group0) group0_4_L4D:
  assumes A1: P {is commutative on} G
  and A2: a∈G b∈G c∈G
  shows
    a-1·b-1·c = c·a-1·b-1
    b-1·a-1·c = c·a-1·b-1
    (a-1·b·c)-1 = a·b-1·c-1
```

⟨proof⟩

Another rearrangement lemma with three elements and equation.

```
lemma (in group0) group0_4_L5: assumes A1:P {is commutative on} G
  and A2: a∈G b∈G c∈G
  and A3: c = a·b-1
  shows a = b·c
```

⟨proof⟩

In abelian groups we can cancel an element with its inverse even if separated by another element.

```
lemma (in group0) group0_4_L6A: assumes A1: P {is commutative on} G
  and A2: a∈G b∈G
  shows
    a·b·a-1 = b
    a-1·b·a = b
    a-1·(b·a) = b
    a·(b·a-1) = b
```

⟨proof⟩

Another lemma about cancelling with two elements.

```
lemma (in group0) group0_4_L6AA:
  assumes A1: P {is commutative on} G and A2: a∈G b∈G
  shows a·b-1·a-1 = b-1
```

⟨proof⟩

Another lemma about cancelling with two elements.

```

lemma (in group0) group0_4_L6AB:
  assumes A1: P {is commutative on} G and A2: a∈G b∈G
  shows
    a·(a·b)-1 = b-1
    a·(b·a-1) = b
  <proof>

```

Another lemma about cancelling with two elements.

```

lemma (in group0) group0_4_L6AC:
  assumes P {is commutative on} G and a∈G b∈G
  shows a·(a·b-1)-1 = b
  <proof>

```

In abelian groups we can cancel an element with its inverse even if separated by two other elements.

```

lemma (in group0) group0_4_L6B: assumes A1: P {is commutative on} G
  and A2: a∈G b∈G c∈G
  shows
    a·b·c·a-1 = b·c
    a-1·b·c·a = b·c
  <proof>

```

In abelian groups we can cancel an element with its inverse even if separated by three other elements.

```

lemma (in group0) group0_4_L6C: assumes A1: P {is commutative on} G
  and A2: a∈G b∈G c∈G d∈G
  shows a·b·c·d·a-1 = b·c·d
  <proof>

```

Another couple of useful rearrangements of three elements and cancelling.

```

lemma (in group0) group0_4_L6D:
  assumes A1: P {is commutative on} G
  and A2: a∈G b∈G c∈G
  shows
    a·b-1·(a·c-1)-1 = c·b-1
    (a·c)-1·(b·c) = a-1·b
    a·(b·(c·a-1·b-1)) = c
    a·b·c-1·(c·a-1) = b
  <proof>

```

Another useful rearrangement of three elements and cancelling.

```

lemma (in group0) group0_4_L6E:
  assumes A1: P {is commutative on} G
  and A2: a∈G b∈G c∈G
  shows
    a·b·(a·c)-1 = b·c-1
  <proof>

```

A rearrangement with two elements and cancelling, special case of group0_4_L6D when $c = b^{-1}$.

```
lemma (in group0) group0_4_L6F:
  assumes A1: P {is commutative on} G
  and A2: a∈G b∈G
  shows a·b-1·(a·b)-1 = b-1·b-1
⟨proof⟩
```

Some other rearrangements with four elements. The algorithm for proof as in group0_4_L2 works very well here.

```
lemma (in group0) rearr_ab_gr_4_elemA:
  assumes A1: P {is commutative on} G
  and A2: a∈G b∈G c∈G d∈G
  shows
    a·b·c·d = a·d·b·c
    a·b·c·d = a·c·(b·d)
⟨proof⟩
```

Some rearrangements with four elements and inverse that are applications of rearr_ab_gr_4_elem

```
lemma (in group0) rearr_ab_gr_4_elemB:
  assumes A1: P {is commutative on} G
  and A2: a∈G b∈G c∈G d∈G
  shows
    a·b-1·c-1·d-1 = a·d-1·b-1·c-1
    a·b·c·d-1 = a·d-1·b·c
    a·b·c-1·d-1 = a·c-1·(b·d-1)
⟨proof⟩
```

Some rearrangement lemmas with four elements.

```
lemma (in group0) group0_4_L7:
  assumes A1: P {is commutative on} G
  and A2: a∈G b∈G c∈G d∈G
  shows
    a·b·c·d-1 = a·d-1·b·c
    a·d·(b·d·(c·d))-1 = a·(b·c)-1·d-1
    a·(b·c)·d = a·b·d·c
⟨proof⟩
```

Some other rearrangements with four elements.

```
lemma (in group0) group0_4_L8:
  assumes A1: P {is commutative on} G
  and A2: a∈G b∈G c∈G d∈G
  shows
    a·(b·c)-1 = (a·d-1·c-1)·(d·b-1)
    a·b·(c·d) = c·a·(b·d)
    a·b·(c·d) = a·c·(b·d)
    a·(b·c-1)·d = a·b·d·c-1
```

$(a \cdot b) \cdot (c \cdot d)^{-1} \cdot (b \cdot d^{-1})^{-1} = a \cdot c^{-1}$
 $\langle proof \rangle$

Some other rearrangements with four elements.

```
lemma (in group0) group0_4_L8A:
  assumes A1: P {is commutative on} G
  and A2: a ∈ G  b ∈ G  c ∈ G  d ∈ G
  shows
    a · b-1 · (c · d-1) = a · c · (b-1 · d-1)
    a · b-1 · (c · d-1) = a · c · b-1 · d-1
  <proof>
```

Some rearrangements with an equation.

```
lemma (in group0) group0_4_L9:
  assumes A1: P {is commutative on} G
  and A2: a ∈ G  b ∈ G  c ∈ G  d ∈ G
  and A3: a = b · c-1 · d-1
  shows
    d = b · a-1 · c-1
    d = a-1 · b · c-1
    b = a · d · c
  <proof>
```

end

36 Direct product

```
theory DirectProduct_ZF imports func_ZF
```

```
begin
```

This theory considers the direct product of binary operations. Contributed by Seo Sanghyeon.

36.1 Definition

In group theory the notion of direct product provides a natural way of creating a new group from two given groups.

Given $(G, \cdot)$ and $(H, \circ)$ a new operation $(G \times H, \times)$ is defined as $(g, h) \times (g', h') = (g \cdot g', h \circ h')$.

definition

```
DirectProduct(P,Q,G,H) ≡
  {⟨x, ⟨P⟨fst(fst(x)), fst(snd(x))⟩ , Q⟨snd(fst(x)), snd(snd(x))⟩⟩⟩.
  x ∈ (G×H)×(G×H)}
```

We define a context called `direct0` which holds an assumption that P, Q are binary operations on G, H , resp. and denotes R as the direct product of (G, P) and (H, Q) .

```

locale direct0 =
  fixes P Q G H
  assumes Pfun: P : G×G→G
  assumes Qfun: Q : H×H→H
  fixes R
  defines Rdef [simp]: R ≡ DirectProduct(P,Q,G,H)

```

The direct product of binary operations is a binary operation.

```

lemma (in direct0) DirectProduct_ZF_1_L1:
  shows R : (G×H)×(G×H)→G×H
  <proof>

```

And it has the intended value.

```

lemma (in direct0) DirectProduct_ZF_1_L2:
  shows ∀x∈(G×H). ∀y∈(G×H).
  R⟨x,y⟩ = ⟨P⟨fst(x),fst(y)⟩,Q⟨snd(x),snd(y)⟩⟩
  <proof>

```

And the value belongs to the set the operation is defined on.

```

lemma (in direct0) DirectProduct_ZF_1_L3:
  shows ∀x∈(G×H). ∀y∈(G×H). R⟨x,y⟩ ∈ G×H
  <proof>

```

36.2 Associative and commutative operations

If P and Q are both associative or commutative operations, the direct product of P and Q has the same property.

Direct product of commutative operations is commutative.

```

lemma (in direct0) DirectProduct_ZF_2_L1:
  assumes P {is commutative on} G and Q {is commutative on} H
  shows R {is commutative on} G×H
  <proof>

```

Direct product of associative operations is associative.

```

lemma (in direct0) DirectProduct_ZF_2_L2:
  assumes P {is associative on} G and Q {is associative on} H
  shows R {is associative on} G×H
  <proof>

```

end

37 Direct product of groups

```
theory DirectProduct_ZF_1 imports DirectProduct_ZF Group_ZF
```

```
begin
```

This theory proves that the direct product of two groups is a group. The result lives in the `direct0` locale, which already establishes the direct product as a binary operation and proves it inherits associativity and commutativity. Here we add the group structure.

37.1 Neutral element and inverse of the direct product

The neutral element of the direct product is the pair of neutral elements.

```
lemma (in direct0) direct_product_neutral:
  assumes GP: IsAgroup(G,P) and HQ: IsAgroup(H,Q)
  shows TheNeutralElement(G×H,R) =
    ⟨TheNeutralElement(G,P), TheNeutralElement(H,Q)⟩
  ⟨proof⟩
```

37.2 The direct product of groups is a group

The direct product of two groups is a group.

```
theorem (in direct0) direct_product_group:
  assumes GP: IsAgroup(G,P) and HQ: IsAgroup(H,Q)
  shows IsAgroup(G×H, R)
  ⟨proof⟩
```

The group inverse in the direct product is computed componentwise.

```
lemma (in direct0) direct_product_inv:
  assumes GP: IsAgroup(G,P) and HQ: IsAgroup(H,Q)
  and gr: g ∈ G and hr: h ∈ H
  shows GroupInv(G×H,R)⟨g,h⟩ = ⟨GroupInv(G,P)g, GroupInv(H,Q)h⟩
  ⟨proof⟩
```

```
end
```

38 Groups 2

```
theory Group_ZF_2 imports AbelianGroup_ZF func_ZF EquivClass1
```

```
begin
```

This theory continues `Group_ZF` and considers lifting the group structure to function spaces and projecting the group structure to quotient spaces, in particular the quotient group. We also define group homomorphisms and in particular the space $\text{End}(G,P)$ of homomorphisms of a group into itself.

38.1 Lifting groups to function spaces

If we have a monoid (group) G than we get a monoid (group) structure on a space of functions valued in G by defining $(f \cdot g)(x) := f(x) \cdot g(x)$. We call this process "lifting the monoid (group) to function space". This section formalizes this lifting.

The lifted operation is an operation on the function space.

```
lemma (in monoid0) Group_ZF_2_1_L0A:
  assumes A1: F = f {lifted to function space over} X
  shows F : (X→G)×(X→G)→(X→G)
<proof>
```

The result of the lifted operation is in the function space.

```
lemma (in monoid0) Group_ZF_2_1_L0:
  assumes A1: F = f {lifted to function space over} X
  and A2: s:X→G r:X→G
  shows F⟨ s,r⟩ : X→G
<proof>
```

The lifted monoid operation has a neutral element, namely the constant function with the neutral element as the value.

```
lemma (in monoid0) Group_ZF_2_1_L1:
  assumes A1: F = f {lifted to function space over} X
  and A2: E = ConstantFunction(X,TheNeutralElement(G,f))
  shows E : X→G ∧ (∀ s∈X→G. F⟨ E,s⟩ = s ∧ F⟨ s,E⟩ = s)
<proof>
```

Monoids can be lifted to a function space.

```
lemma (in monoid0) Group_ZF_2_1_T1:
  assumes A1: F = f {lifted to function space over} X
  shows IsAmonoid(X→G,F)
<proof>
```

The constant function with the neutral element as the value is the neutral element of the lifted monoid.

```
lemma Group_ZF_2_1_L2:
  assumes A1: IsAmonoid(G,f)
  and A2: F = f {lifted to function space over} X
  and A3: E = ConstantFunction(X,TheNeutralElement(G,f))
  shows E = TheNeutralElement(X→G,F)
<proof>
```

The lifted operation acts on the functions in a natural way defined by the monoid operation.

```
lemma (in monoid0) lifted_val:
  assumes F = f {lifted to function space over} X
```

```

and s:X→G  r:X→G
and x∈X
shows (F⟨s,r⟩)(x) = s(x) ⊕ r(x)
⟨proof⟩

```

The lifted operation acts on the functions in a natural way defined by the group operation. This is the same as `lifted_val`, but in the `group0` context.

```

lemma (in group0) Group_ZF_2_1_L3:
  assumes F = P {lifted to function space over} X
  and s:X→G  r:X→G
  and x∈X
  shows (F⟨s,r⟩)(x) = s(x)·r(x)
  ⟨proof⟩

```

In the `group0` context we can apply theorems proven in `monoid0` context to the lifted monoid.

```

lemma (in group0) Group_ZF_2_1_L4:
  assumes A1: F = P {lifted to function space over} X
  shows monoid0(X→G,F)
  ⟨proof⟩

```

The composition of a function $f : X \rightarrow G$ with the group inverse is a right inverse for the lifted group.

```

lemma (in group0) Group_ZF_2_1_L5:
  assumes A1: F = P {lifted to function space over} X
  and A2: s : X→G
  and A3: i = GroupInv(G,P) 0 s
  shows i: X→G and F⟨s,i⟩ = TheNeutralElement(X→G,F)
  ⟨proof⟩

```

Groups can be lifted to the function space.

```

theorem (in group0) Group_ZF_2_1_T2:
  assumes A1: F = P {lifted to function space over} X
  shows IsAgroup(X→G,F)
  ⟨proof⟩

```

The propositions proven in the `group0` context are valid in the same context when applied to the function space with the lifted group operation.

```

lemma (in group0) group0_valid_fun_space:
  shows group0(X→G,P {lifted to function space over} X)
  ⟨proof⟩

```

What is the group inverse for the lifted group?

```

lemma (in group0) Group_ZF_2_1_L6:
  assumes A1: F = P {lifted to function space over} X
  shows ∀s∈(X→G). GroupInv(X→G,F)(s) = GroupInv(G,P) 0 s
  ⟨proof⟩

```

What is the value of the group inverse for the lifted group?

```
corollary (in group0) lift_gr_inv_val:
  assumes F = P {lifted to function space over} X and
  s : X→G and x∈X
  shows (GroupInv(X→G,F)(s))(x) = (s(x))-1
  <proof>
```

What is the group inverse in a subgroup of the lifted group?

```
lemma (in group0) Group_ZF_2_1_L6A:
  assumes A1: F = P {lifted to function space over} X
  and A2: IsAsubgroup(H,F)
  and A3: g = restrict(F,H×H)
  and A4: s∈H
  shows GroupInv(H,g)(s) = GroupInv(G,P) 0 s
  <proof>
```

The neutral element of a subgroup of the lifted group is the constant function with value equal to the neutral element of the group.

```
lemma (in group0) lift_group_subgr_neut:
  assumes F = P {lifted to function space over} X and IsAsubgroup(H,F)
  shows TheNeutralElement(H,restrict(F,H×H)) = ConstantFunction(X,1)
  <proof>
```

If a group is abelian, then its lift to a function space is also abelian.

```
lemma (in group0) Group_ZF_2_1_L7:
  assumes A1: F = P {lifted to function space over} X
  and A2: P {is commutative on} G
  shows F {is commutative on} (X→G)
  <proof>
```

38.2 Equivalence relations on groups

The goal of this section is to establish that (under some conditions) given an equivalence relation on a group or (monoid) we can project the group (monoid) structure on the quotient and obtain another group.

The neutral element class is neutral in the projection.

```
lemma (in monoid0) Group_ZF_2_2_L1:
  assumes A1: equiv(G,r) and A2: Congruent2(r,f)
  and A3: F = ProjFun2(G,r,f)
  and A4: e = TheNeutralElement(G,f)
  shows r{e} ∈ G//r ∧
  (∀ c ∈ G//r. F⟨ r{e},c⟩ = c ∧ F⟨ c,r{e}⟩ = c)
  <proof>
```

The projected structure is a monoid.

```
theorem (in monoid0) Group_ZF_2_2_T1:
```

```

    assumes A1: equiv(G,r) and A2: Congruent2(r,f)
    and A3: F = ProjFun2(G,r,f)
    shows IsAmonoid(G//r,F)
  <proof>

```

The class of the neutral element is the neutral element of the projected monoid.

```

lemma Group_ZF_2_2_L1:
  assumes A1: IsAmonoid(G,f)
  and A2: equiv(G,r) and A3: Congruent2(r,f)
  and A4: F = ProjFun2(G,r,f)
  and A5: e = TheNeutralElement(G,f)
  shows r{e} = TheNeutralElement(G//r,F)
  <proof>

```

The projected operation can be defined in terms of the group operation on representants in a natural way.

```

lemma (in group0) Group_ZF_2_2_L2:
  assumes A1: equiv(G,r) and A2: Congruent2(r,P)
  and A3: F = ProjFun2(G,r,P)
  and A4: a∈G b∈G
  shows F⟨ r{a},r{b} ⟩ = r{a·b}
  <proof>

```

The class of the inverse is a right inverse of the class.

```

lemma (in group0) Group_ZF_2_2_L3:
  assumes A1: equiv(G,r) and A2: Congruent2(r,P)
  and A3: F = ProjFun2(G,r,P)
  and A4: a∈G
  shows F⟨ r{a},r{a-1} ⟩ = TheNeutralElement(G//r,F)
  <proof>

```

The group structure can be projected to the quotient space.

```

theorem (in group0) Group_ZF_3_T2:
  assumes A1: equiv(G,r) and A2: Congruent2(r,P)
  shows IsAgroup(G//r,ProjFun2(G,r,P))
  <proof>

```

The group inverse (in the projected group) of a class is the class of the inverse.

```

lemma (in group0) Group_ZF_2_2_L4:
  assumes A1: equiv(G,r) and
  A2: Congruent2(r,P) and
  A3: F = ProjFun2(G,r,P) and
  A4: a∈G
  shows r{a-1} = GroupInv(G//r,F)(r{a})
  <proof>

```

38.3 Normal subgroups and quotient groups

If H is a subgroup of G , then for every $a \in G$ we can consider the sets $\{a \cdot h \mid h \in H\}$ and $\{h \cdot a \mid h \in H\}$ (called a left and right "coset of H ", resp.) These sets sometimes form a group, called the "quotient group". This section discusses the notion of quotient groups.

A normal subgroup N of a group G is such that aba^{-1} belongs to N if $a \in G, b \in N$.

definition

$$\text{IsAnormalSubgroup}(G, P, N) \equiv \text{IsASubgroup}(N, P) \wedge \\ (\forall n \in N. \forall g \in G. P(\langle g \cdot n \rangle, \text{GroupInv}(G, P)(g)) \in N)$$

Having a group and a normal subgroup N we can create another group consisting of equivalence classes of the relation $a \sim b \equiv a \cdot b^{-1} \in N$. We will refer to this relation as the quotient group relation. The classes of this relation are in fact cosets of subgroup H .

definition

$$\text{QuotientGroupRel}(G, P, H) \equiv \\ \{ \langle a, b \rangle \in G \times G. P(\langle a, \text{GroupInv}(G, P)(b) \rangle \in H) \}$$

Next we define the operation in the quotient group as the projection of the group operation on the classes of the quotient group relation.

definition

$$\text{QuotientGroupOp}(G, P, H) \equiv \text{ProjFun2}(G, \text{QuotientGroupRel}(G, P, H), P)$$

Definition of a normal subgroup in a more readable notation.

```
lemma (in group0) Group_ZF_2_4_L0:
  assumes IsAnormalSubgroup(G,P,H)
  and g∈G n∈H
  shows g·n·g-1 ∈ H
  <proof>
```

The quotient group relation is reflexive.

```
lemma (in group0) Group_ZF_2_4_L1:
  assumes IsASubgroup(H,P)
  shows refl(G,QuotientGroupRel(G,P,H))
  <proof>
```

The quotient group relation is symmetric.

```
lemma (in group0) Group_ZF_2_4_L2:
  assumes A1:IsASubgroup(H,P)
  shows sym(QuotientGroupRel(G,P,H))
  <proof>
```

The quotient group relation is transitive.

```
lemma (in group0) Group_ZF_2_4_L3A:
```

```

    assumes A1: IsAsubgroup(H,P) and
    A2:  $\langle a,b \rangle \in \text{QuotientGroupRel}(G,P,H)$  and
    A3:  $\langle b,c \rangle \in \text{QuotientGroupRel}(G,P,H)$ 
    shows  $\langle a,c \rangle \in \text{QuotientGroupRel}(G,P,H)$ 
  <proof>

```

The quotient group relation is an equivalence relation. Note we do not need the subgroup to be normal for this to be true.

```

lemma (in group0) Group_ZF_2_4_L3: assumes A1:IsAsubgroup(H,P)
  shows equiv(G,QuotientGroupRel(G,P,H))
  <proof>

```

The next lemma states the essential condition for congruency of the group operation with respect to the quotient group relation.

```

lemma (in group0) Group_ZF_2_4_L4:
  assumes A1: IsAnormalSubgroup(G,P,H)
  and A2:  $\langle a1,a2 \rangle \in \text{QuotientGroupRel}(G,P,H)$ 
  and A3:  $\langle b1,b2 \rangle \in \text{QuotientGroupRel}(G,P,H)$ 
  shows  $\langle a1.b1, a2.b2 \rangle \in \text{QuotientGroupRel}(G,P,H)$ 
  <proof>

```

If the subgroup is normal, the group operation is congruent with respect to the quotient group relation.

```

lemma Group_ZF_2_4_L5A:
  assumes IsAgroup(G,P)
  and IsAnormalSubgroup(G,P,H)
  shows Congruent2(QuotientGroupRel(G,P,H),P)
  <proof>

```

The quotient group is indeed a group.

```

theorem Group_ZF_2_4_T1:
  assumes IsAgroup(G,P) and IsAnormalSubgroup(G,P,H)
  shows
    IsAgroup(G//QuotientGroupRel(G,P,H),QuotientGroupOp(G,P,H))
  <proof>

```

The class (coset) of the neutral element is the neutral element of the quotient group.

```

lemma Group_ZF_2_4_L5B:
  assumes IsAgroup(G,P) and IsAnormalSubgroup(G,P,H)
  and r = QuotientGroupRel(G,P,H)
  and e = TheNeutralElement(G,P)
  shows  $r\{e\} = \text{TheNeutralElement}(G//r,\text{QuotientGroupOp}(G,P,H))$ 
  <proof>

```

A group element is equivalent to the neutral element iff it is in the subgroup we divide the group by.

lemma (in group0) Group_ZF_2_4_L5C: **assumes** $a \in G$
shows $\langle a, 1 \rangle \in \text{QuotientGroupRel}(G, P, H) \longleftrightarrow a \in H$
 $\langle \text{proof} \rangle$

A group element is in H iff its class is the neutral element of G/H .

lemma (in group0) Group_ZF_2_4_L5D:
assumes A1: $\text{IsAnormalSubgroup}(G, P, H)$ **and**
A2: $a \in G$ **and**
A3: $r = \text{QuotientGroupRel}(G, P, H)$ **and**
A4: $\text{TheNeutralElement}(G//r, \text{QuotientGroupOp}(G, P, H)) = e$
shows $r\{a\} = e \longleftrightarrow \langle a, 1 \rangle \in r$
 $\langle \text{proof} \rangle$

The class of $a \in G$ is the neutral element of the quotient G/H iff $a \in H$.

lemma (in group0) Group_ZF_2_4_L5E:
assumes $\text{IsAnormalSubgroup}(G, P, H)$ **and**
 $a \in G$ **and** $r = \text{QuotientGroupRel}(G, P, H)$ **and**
 $\text{TheNeutralElement}(G//r, \text{QuotientGroupOp}(G, P, H)) = e$
shows $r\{a\} = e \longleftrightarrow a \in H$
 $\langle \text{proof} \rangle$

Essential condition to show that every subgroup of an abelian group is normal.

lemma (in group0) Group_ZF_2_4_L5:
assumes A1: $P \text{ \{is commutative on\} } G$
and A2: $\text{IsASubgroup}(H, P)$
and A3: $g \in G \quad h \in H$
shows $g \cdot h \cdot g^{-1} \in H$
 $\langle \text{proof} \rangle$

Every subgroup of an abelian group is normal. Moreover, the quotient group is also abelian.

lemma Group_ZF_2_4_L6:
assumes A1: $\text{IsAGroup}(G, P)$
and A2: $P \text{ \{is commutative on\} } G$
and A3: $\text{IsASubgroup}(H, P)$
shows $\text{IsAnormalSubgroup}(G, P, H)$
 $\text{QuotientGroupOp}(G, P, H) \text{ \{is commutative on\} } (G//\text{QuotientGroupRel}(G, P, H))$
 $\langle \text{proof} \rangle$

The group inverse (in the quotient group) of a class (coset) is the class of the inverse.

lemma (in group0) Group_ZF_2_4_L7:
assumes $\text{IsAnormalSubgroup}(G, P, H)$
and $a \in G$ **and** $r = \text{QuotientGroupRel}(G, P, H)$
and $F = \text{QuotientGroupOp}(G, P, H)$
shows $r\{a^{-1}\} = \text{GroupInv}(G//r, F)(r\{a\})$
 $\langle \text{proof} \rangle$

38.4 Function spaces as monoids

On every space of functions $\{f : X \rightarrow X\}$ we can define a natural monoid structure with composition as the operation. This section explores this fact.

The next lemma states that composition has a neutral element, namely the identity function on X (the one that maps $x \in X$ into itself).

lemma Group_ZF_2_5_L1: **assumes** A1: $F = \text{Composition}(X)$
shows $\exists I \in (X \rightarrow X). \forall f \in (X \rightarrow X). F(I, f) = f \wedge F(f, I) = f$
<proof>

The space of functions that map a set X into itself is a monoid with composition as operation and the identity function as the neutral element.

lemma Group_ZF_2_5_L2: **shows**
 $\text{IsAmonoid}(X \rightarrow X, \text{Composition}(X))$
 $\text{id}(X) = \text{TheNeutralElement}(X \rightarrow X, \text{Composition}(X))$
<proof>

38.5 Homomorphisms

A homomorphism is a function between groups that preserves the group operations.

In general we may have a homomorphism not only between groups, but also between various algebraic structures with one operation like magmas, semi-groups, quasigroups, loops and monoids. In all cases the homomorphism is defined by using the morphism property. In the multiplicative notation we will write that f has a morphism property if $f(x \cdot_G y) = f(x) \cdot_H f(y)$ for all $x, y \in G$. Below we write this definition in raw set theory notation and use the expression `IsMorphism` instead of the possible, but longer `HasMorphismProperty`.

definition
 $\text{IsMorphism}(G, P, F, f) \equiv \forall g_1 \in G. \forall g_2 \in G. f(P(g_1, g_2)) = F(f(g_1), f(g_2))$

A function $f : G \rightarrow H$ between algebraic structures $(G, \cdot_G)$ and $(H, \cdot_H)$ with one operation (each) is a homomorphism if it has the morphism property.

definition
 $\text{Homomor}(f, G, P, H, F) \equiv f : G \rightarrow H \wedge \text{IsMorphism}(G, P, F, f)$

Now a lemma about the definition:

lemma homomor_eq:
assumes $\text{Homomor}(f, G, P, H, F)$ $g_1 \in G$ $g_2 \in G$
shows $f(P(g_1, g_2)) = F(f(g_1), f(g_2))$
<proof>

An endomorphism is a homomorphism from a group to the same group. We define $\text{End}(G, P)$ as the set of endomorphisms for a given group. As we show

later when the group is abelian, the set of endomorphisms with pointwise addition and composition as multiplication forms a ring.

definition

$\text{End}(G,P) \equiv \{f \in G \rightarrow G. \text{Homomor}(f,G,P,G,P)\}$

The defining property of an endomorphism written in notation used in `group0` context:

lemma (in `group0`) `endomorph_eq`: **assumes** $f \in \text{End}(G,P)$ $g_1 \in G$ $g_2 \in G$
shows $f(g_1 \cdot g_2) = f(g_1) \cdot f(g_2)$
 $\langle \text{proof} \rangle$

A function that maps a group G into itself and satisfies $f(g_1 \cdot g_2) = f(g_1) \cdot f(g_2)$ is an endomorphism.

lemma (in `group0`) `eq_endomorph`:
assumes $f: G \rightarrow G$ **and** $\forall g_1 \in G. \forall g_2 \in G. f(g_1 \cdot g_2) = f(g_1) \cdot f(g_2)$
shows $f \in \text{End}(G,P)$
 $\langle \text{proof} \rangle$

The value of the composition function on a pair of endomorphisms is an endomorphism.

lemma (in `group0`) `end_composition`:
assumes $f_1 \in \text{End}(G,P)$ $f_2 \in \text{End}(G,P)$
shows $\text{Composition}(G)(f_1, f_2) \in \text{End}(G,P)$
 $\langle \text{proof} \rangle$

We will use some binary operations that are naturally defined on the function space $G \rightarrow G$, but we consider them restricted to the endomorphisms of G . To shorten the notation in such case we define an abbreviation `InEnd(F,G,P)` which restricts a binary operation F to the set of endomorphisms of G .

abbreviation `InEnd(_ {in End} [_,_])`
where $\text{InEnd}(F,G,P) \equiv \text{restrict}(F, \text{End}(G,P) \times \text{End}(G,P))$

The value of the composition operation restricted to endomorphisms on a pair of endomorphisms is indeed the composition of the elements of the pair.

lemma (in `group0`) `inend_composition_val`:
assumes $f \in \text{End}(G,P)$ $g \in \text{End}(G,P)$
shows $\text{InEnd}(\text{Composition}(G), G, P)(f, g) = (f \circ g)$
 $\langle \text{proof} \rangle$

Endomorphisms of a group form a monoid with composition as the binary operation, and the identity map as the neutral element.

theorem (in `group0`) `end_comp_monoid`:
shows $\text{IsAmonoid}(\text{End}(G,P), \text{InEnd}(\text{Composition}(G), G, P))$
and $\text{TheNeutralElement}(\text{End}(G,P), \text{InEnd}(\text{Composition}(G), G, P)) = \text{id}(G)$
 $\langle \text{proof} \rangle$

end

39 Almost homomorphisms foundation of the real number construction

theory Group_ZF_3 **imports** Group_ZF_2 Finite1

begin

In this theory we consider notions in group theory that are useful for the construction of real numbers in the `Real_ZF_x` series of theories.

39.1 Group valued finite range functions

In this section show that the group valued functions $f : X \rightarrow G$, with the property that $f(X)$ is a finite subset of G , is a group. Such functions play an important role in the construction of real numbers in the `Real_ZF` series.

The following proves the essential condition to show that the set of finite range functions is closed with respect to the lifted group operation.

lemma (in group0) Group_ZF_3_1_L1:
 assumes A1: $F = P$ {lifted to function space over} X
 and
 A2: $s \in \text{FinRangeFunctions}(X, G)$ $r \in \text{FinRangeFunctions}(X, G)$
 shows $F\langle s, r \rangle \in \text{FinRangeFunctions}(X, G)$
<proof>

The set of group valued finite range functions is closed with respect to the lifted group operation.

lemma (in group0) Group_ZF_3_1_L2:
 assumes A1: $F = P$ {lifted to function space over} X
 shows $\text{FinRangeFunctions}(X, G)$ {is closed under} F
<proof>

A composition of a finite range function with the group inverse is a finite range function.

lemma (in group0) Group_ZF_3_1_L3:
 assumes A1: $s \in \text{FinRangeFunctions}(X, G)$
 shows $\text{GroupInv}(G, P) \circ s \in \text{FinRangeFunctions}(X, G)$
<proof>

The set of finite range functions is a subgroup of the lifted group.

theorem Group_ZF_3_1_T1:
 assumes A1: $\text{IsAgroup}(G, P)$
 and A2: $F = P$ {lifted to function space over} X
 and A3: $X \neq 0$
 shows $\text{IsASubgroup}(\text{FinRangeFunctions}(X, G), F)$
<proof>

39.2 Almost homomorphisms

An almost homomorphism is a group valued function defined on a monoid M with the property that the set $\{f(m+n) - f(m) - f(n)\}_{m,n \in M}$ is finite. This term is used by R. D. Arthan in "The Eudoxus Real Numbers". We use this term in the general group context and use the A'Campo's term "slopes" (see his "A natural construction for the real numbers") to mean an almost homomorphism mapping interegers into themselves. We consider almost homomorphisms because we use slopes to define real numbers in the `Real_ZF_x` series.

`HomDiff` is an acronym for "homomorphism difference". This is the expression $s(mn)(s(m)s(n))^{-1}$, or $s(m+n) - s(m) - s(n)$ in the additive notation. It is equal to the neutral element of the group if s is a homomorphism.

definition

```
HomDiff(G,f,s,x) ≡
  f⟨s(f⟨fst(x),snd(x))⟩,
    (GroupInv(G,f)(f⟨s(fst(x)),s(snd(x)))⟩)⟩
```

Almost homomorphisms are defined as those maps $s : G \rightarrow G$ such that the homomorphism difference takes only finite number of values on $G \times G$.

definition

```
AlmostHoms(G,f) ≡
  {s ∈ G→G. {HomDiff(G,f,s,x). x ∈ G×G } ∈ Fin(G)}
```

$\text{AlHomOp1}(G, f)$ is the group operation on almost homomorphisms defined in a natural way by $(s \cdot r)(n) = s(n) \cdot r(n)$. In the terminology defined in `func1.thy` this is the group operation f (on G) lifted to the function space $G \rightarrow G$ and restricted to the set $\text{AlmostHoms}(G, f)$.

definition

```
AlHomOp1(G,f) ≡
  restrict(f {lifted to function space over} G,
    AlmostHoms(G,f) × AlmostHoms(G,f))
```

We also define a composition (binary) operator on almost homomorphisms in a natural way. We call that operator `AlHomOp2` - the second operation on almost homomorphisms. Composition of almost homomorphisms is used to define multiplication of real numbers in `Real_ZF` series.

definition

```
AlHomOp2(G,f) ≡
  restrict(Composition(G), AlmostHoms(G,f) × AlmostHoms(G,f))
```

This lemma provides more readable notation for the `HomDiff` definition. Not really intended to be used in proofs, but just to see the definition in the notation defined in the `group0` locale.

lemma (in `group0`) `HomDiff_notation`:

shows $\text{HomDiff}(G, P, s, \langle m, n \rangle) = s(m \cdot n) \cdot (s(m) \cdot s(n))^{-1}$
<proof>

The next lemma shows the set from the definition of almost homomorphism in a different form.

lemma (in group0) Group_ZF_3_2_L1A: **shows**
 $\{\text{HomDiff}(G, P, s, x) \mid x \in G \times G\} = \{s(m \cdot n) \cdot (s(m) \cdot s(n))^{-1} \mid \langle m, n \rangle \in G \times G\}$
<proof>

Let's define some notation. We inherit the notation and assumptions from the group0 context (locale) and add some. We will use AH to denote the set of almost homomorphisms. $\sim$ is the inverse (negative if the group is the group of integers) of almost homomorphisms, $(\sim p)(n) = p(n)^{-1}$. δ will denote the homomorphism difference specific for the group ($\text{HomDiff}(G, f)$). The notation $s \approx r$ will mean that s, r are almost equal, that is they are in the equivalence relation defined by the group of finite range functions (that is a normal subgroup of almost homomorphisms, if the group is abelian). We show that this is equivalent to the set $\{s(n) \cdot r(n)^{-1} \mid n \in G\}$ being finite. We also add an assumption that the G is abelian as many needed properties do not hold without that.

locale group1 = group0 +
 assumes isAbelian: P {is commutative on} G

 fixes AH
 defines AH_def [simp]: AH $\equiv$ AlmostHoms(G, P)

 fixes Op1
 defines Op1_def [simp]: Op1 $\equiv$ AlHomOp1(G, P)

 fixes Op2
 defines Op2_def [simp]: Op2 $\equiv$ AlHomOp2(G, P)

 fixes FR
 defines FR_def [simp]: FR $\equiv$ FinRangeFunctions(G, G)

 fixes neg ($\sim$ [90] 91)
 defines neg_def [simp]: $\sim s \equiv \text{GroupInv}(G, P) \circ s$

 fixes δ
 defines δ _def [simp]: $\delta(s, x) \equiv \text{HomDiff}(G, P, s, x)$

 fixes AHprod (infix $\cdot$ 69)
 defines AHprod_def [simp]: $s \cdot r \equiv \text{AlHomOp1}(G, P) \langle s, r \rangle$

 fixes AHcomp (infix $\circ$ 70)
 defines AHcomp_def [simp]: $s \circ r \equiv \text{AlHomOp2}(G, P) \langle s, r \rangle$

 fixes AlEq (infix $\cong$ 68)

defines AlEq_def [simp]: $s \cong r \equiv \langle s, r \rangle \in \text{QuotientGroupRel}(\text{AH}, \text{Op1}, \text{FR})$

HomDiff is a homomorphism on the lifted group structure.

lemma (in group1) Group_ZF_3_2_L1:
 assumes A1: $s:G \rightarrow G$ $r:G \rightarrow G$
 and A2: $x \in G \times G$
 and A3: $F = P$ {lifted to function space over} G
 shows $\delta(F\langle s, r \rangle, x) = \delta(s, x) \cdot \delta(r, x)$
<proof>

The group operation lifted to the function space over G preserves almost homomorphisms.

lemma (in group1) Group_ZF_3_2_L2: assumes A1: $s \in \text{AH}$ $r \in \text{AH}$
 and A2: $F = P$ {lifted to function space over} G
 shows $F\langle s, r \rangle \in \text{AH}$
<proof>

The set of almost homomorphisms is closed under the lifted group operation.

lemma (in group1) Group_ZF_3_2_L3:
 assumes $F = P$ {lifted to function space over} G
 shows AH {is closed under} F
<proof>

The terms in the homomorphism difference for a function are in the group.

lemma (in group1) Group_ZF_3_2_L4:
 assumes $s:G \rightarrow G$ and $m \in G$ $n \in G$
 shows
 $m \cdot n \in G$
 $s(m \cdot n) \in G$
 $s(m) \in G$ $s(n) \in G$
 $\delta(s, \langle m, n \rangle) \in G$
 $s(m) \cdot s(n) \in G$
<proof>

It is handy to have a version of Group_ZF_3_2_L4 specifically for almost homomorphisms.

corollary (in group1) Group_ZF_3_2_L4A:
 assumes $s \in \text{AH}$ and $m \in G$ $n \in G$
 shows $m \cdot n \in G$
 $s(m \cdot n) \in G$
 $s(m) \in G$ $s(n) \in G$
 $\delta(s, \langle m, n \rangle) \in G$
 $s(m) \cdot s(n) \in G$
<proof>

The terms in the homomorphism difference are in the group, a different form.

lemma (in group1) Group_ZF_3_2_L4B:

```

assumes A1:s ∈ AH and A2:x∈G×G
shows fst(x)·snd(x) ∈ G
s(fst(x)·snd(x)) ∈ G
s(fst(x)) ∈ G s(snd(x)) ∈ G
δ(s,x) ∈ G
s(fst(x))·s(snd(x)) ∈ G
⟨proof⟩

```

What are the values of the inverse of an almost homomorphism?

```

lemma (in group1) Group_ZF_3_2_L5:
  assumes s ∈ AH and n∈G
  shows (∼s)(n) = (s(n))-1
  ⟨proof⟩

```

Homomorphism difference commutes with the inverse for almost homomorphisms.

```

lemma (in group1) Group_ZF_3_2_L6:
  assumes A1:s ∈ AH and A2:x∈G×G
  shows δ(∼s,x) = (δ(s,x))-1
  ⟨proof⟩

```

The inverse of an almost homomorphism maps the group into itself.

```

lemma (in group1) Group_ZF_3_2_L7:
  assumes s ∈ AH
  shows ∼s : G→G
  ⟨proof⟩

```

The inverse of an almost homomorphism is an almost homomorphism.

```

lemma (in group1) Group_ZF_3_2_L8:
  assumes A1: F = P {lifted to function space over} G
  and A2: s ∈ AH
  shows GroupInv(G→G,F)(s) ∈ AH
  ⟨proof⟩

```

The function that assigns the neutral element everywhere is an almost homomorphism.

```

lemma (in group1) Group_ZF_3_2_L9: shows
  ConstantFunction(G,1) ∈ AH and AH≠0
  ⟨proof⟩

```

If the group is abelian, then almost homomorphisms form a subgroup of the lifted group.

```

lemma Group_ZF_3_2_L10:
  assumes A1: IsAgroup(G,P)
  and A2: P {is commutative on} G
  and A3: F = P {lifted to function space over} G
  shows IsASubgroup(AlmostHoms(G,P),F)

```

<proof>

If the group is abelian, then almost homomorphisms form a group with the first operation, hence we can use theorems proven in `group0` context applied to this group.

```
lemma (in group1) Group_ZF_3_2_L10A:
  shows IsAgroup(AH,Op1) group0(AH,Op1)
  <proof>
```

The group of almost homomorphisms is abelian

```
lemma Group_ZF_3_2_L11: assumes A1: IsAgroup(G,f)
  and A2: f {is commutative on} G
  shows
    IsAgroup(AlmostHoms(G,f),AlHomOp1(G,f))
    AlHomOp1(G,f) {is commutative on} AlmostHoms(G,f)
  <proof>
```

The first operation on homomorphisms acts in a natural way on its operands.

```
lemma (in group1) Group_ZF_3_2_L12:
  assumes s∈AH r∈AH and n∈G
  shows (s·r)(n) = s(n)·r(n)
  <proof>
```

What is the group inverse in the group of almost homomorphisms?

```
lemma (in group1) Group_ZF_3_2_L13:
  assumes A1: s∈AH
  shows
    GroupInv(AH,Op1)(s) = GroupInv(G,P) 0 s
    GroupInv(AH,Op1)(s) ∈ AH
    GroupInv(G,P) 0 s ∈ AH
  <proof>
```

The group inverse in the group of almost homomorphisms acts in a natural way on its operand.

```
lemma (in group1) Group_ZF_3_2_L14:
  assumes s∈AH and n∈G
  shows (GroupInv(AH,Op1)(s))(n) = (s(n))-1
  <proof>
```

The next lemma states that if s, r are almost homomorphisms, then $s \cdot r^{-1}$ is also an almost homomorphism.

```
lemma Group_ZF_3_2_L15: assumes IsAgroup(G,f)
  and f {is commutative on} G
  and AH = AlmostHoms(G,f) Op1 = AlHomOp1(G,f)
  and s ∈ AH r ∈ AH
  shows
    Op1⟨ s,r ⟩ ∈ AH
```

```

GroupInv(AH,Op1)(r) ∈ AH
Op1⟨ s,GroupInv(AH,Op1)(r)⟩ ∈ AH
⟨proof⟩

```

A version of `Group_ZF_3_2_L15` formulated in notation used in `group1` context. States that the product of almost homomorphisms is an almost homomorphism and the the product of an almost homomorphism with a (point-wise) inverse of an almost homomorphism is an almost homomorphism.

```

corollary (in group1) Group_ZF_3_2_L16: assumes s ∈ AH  r ∈ AH
shows s·r ∈ AH  s·(∼r) ∈ AH
⟨proof⟩

```

39.3 The classes of almost homomorphisms

In the `Real_ZF` series we define real numbers as a quotient of the group of integer almost homomorphisms by the integer finite range functions. In this section we setup the background for that in the general group context.

Finite range functions are almost homomorphisms.

```

lemma (in group1) Group_ZF_3_3_L1: shows FR ⊆ AH
⟨proof⟩

```

Finite range functions valued in an abelian group form a normal subgroup of almost homomorphisms.

```

lemma Group_ZF_3_3_L2: assumes A1:IsAgroup(G,f)
and A2:f {is commutative on} G
shows
  IsASubgroup(FinRangeFunctions(G,G),AlHomOp1(G,f))
  IsANormalSubgroup(AlmostHoms(G,f),AlHomOp1(G,f),
    FinRangeFunctions(G,G))
⟨proof⟩

```

The group of almost homomorphisms divided by the subgroup of finite range functions is an abelian group.

```

theorem (in group1) Group_ZF_3_3_T1:
shows
  IsAgroup(AH//QuotientGroupRel(AH,Op1,FR),QuotientGroupOp(AH,Op1,FR))
and
  QuotientGroupOp(AH,Op1,FR) {is commutative on}
  (AH//QuotientGroupRel(AH,Op1,FR))
⟨proof⟩

```

It is useful to have a direct statement that the quotient group relation is an equivalence relation for the group of AH and subgroup FR.

```

lemma (in group1) Group_ZF_3_3_L3: shows
  QuotientGroupRel(AH,Op1,FR) ⊆ AH × AH and
  equiv(AH,QuotientGroupRel(AH,Op1,FR))

```


<proof>

The "almost equal" relation is symmetric.

lemma (in group1) Group_ZF_3_3_L3A: assumes A1: $s \cong r$
 shows $r \cong s$
<proof>

Although we have bypassed this fact when proving that group of almost homomorphisms divided by the subgroup of finite range functions is a group, it is still useful to know directly that the first group operation on AH is congruent with respect to the quotient group relation.

lemma (in group1) Group_ZF_3_3_L4:
 shows $\text{Congruent2}(\text{QuotientGroupRel}(\text{AH}, \text{Op1}, \text{FR}), \text{Op1})$
<proof>

The class of an almost homomorphism s is the neutral element of the quotient group of almost homomorphisms iff s is a finite range function.

lemma (in group1) Group_ZF_3_3_L5: assumes $s \in \text{AH}$ and
 $r = \text{QuotientGroupRel}(\text{AH}, \text{Op1}, \text{FR})$ and
 $\text{TheNeutralElement}(\text{AH}/r, \text{QuotientGroupOp}(\text{AH}, \text{Op1}, \text{FR})) = e$
 shows $r\{s\} = e \longleftrightarrow s \in \text{FR}$
<proof>

The group inverse of a class of an almost homomorphism f is the class of the inverse of f .

lemma (in group1) Group_ZF_3_3_L6:
 assumes A1: $s \in \text{AH}$ and
 $r = \text{QuotientGroupRel}(\text{AH}, \text{Op1}, \text{FR})$ and
 $F = \text{ProjFun2}(\text{AH}, r, \text{Op1})$
 shows $r\{\sim s\} = \text{GroupInv}(\text{AH}/r, F)(r\{s\})$
<proof>

39.4 Compositions of almost homomorphisms

The goal of this section is to establish some facts about composition of almost homomorphisms that are needed for the real numbers construction in Real_ZF_x series. In particular we show that the set of almost homomorphisms is closed under composition and that composition is congruent with respect to the equivalence relation defined by the group of finite range functions (a normal subgroup of almost homomorphisms).

The next formula restates the definition of the homomorphism difference to express the value an almost homomorphism on a product.

lemma (in group1) Group_ZF_3_4_L1:
 assumes $s \in \text{AH}$ and $m \in G$ $n \in G$
 shows $s(m \cdot n) = s(m) \cdot s(n) \cdot \delta(s, \langle m, n \rangle)$

<proof>

What is the value of a composition of almost homomorphisms?

lemma (in group1) Group_ZF_3_4_L2:
 assumes $s \in \text{AH}$ $r \in \text{AH}$ and $m \in G$
 shows $(s \circ r)(m) = s(r(m))$ $s(r(m)) \in G$
<proof>

What is the homomorphism difference of a composition?

lemma (in group1) Group_ZF_3_4_L3:
 assumes A1: $s \in \text{AH}$ $r \in \text{AH}$ and A2: $m \in G$ $n \in G$
 shows $\delta(s \circ r, \langle m, n \rangle) =$
 $\delta(s, \langle r(m), r(n) \rangle) \cdot s(\delta(r, \langle m, n \rangle)) \cdot \delta(s, \langle r(m) \cdot r(n), \delta(r, \langle m, n \rangle) \rangle)$
<proof>

What is the homomorphism difference of a composition (another form)?
 Here we split the homomorphism difference of a composition into a product of three factors. This will help us in proving that the range of homomorphism difference for the composition is finite, as each factor has finite range.

lemma (in group1) Group_ZF_3_4_L4:
 assumes A1: $s \in \text{AH}$ $r \in \text{AH}$ and A2: $x \in G \times G$
 and A3:
 $A = \delta(s, \langle r(\text{fst}(x)), r(\text{snd}(x)) \rangle)$
 $B = s(\delta(r, x))$
 $C = \delta(s, \langle (r(\text{fst}(x)) \cdot r(\text{snd}(x))), \delta(r, x) \rangle)$
 shows $\delta(s \circ r, x) = A \cdot B \cdot C$
<proof>

The range of the homomorphism difference of a composition of two almost homomorphisms is finite. This is the essential condition to show that a composition of almost homomorphisms is an almost homomorphism.

lemma (in group1) Group_ZF_3_4_L5:
 assumes A1: $s \in \text{AH}$ $r \in \text{AH}$
 shows $\{\delta(\text{Composition}(G) \langle s, r \rangle, x) \mid x \in G \times G\} \in \text{Fin}(G)$
<proof>

Composition of almost homomorphisms is an almost homomorphism.

theorem (in group1) Group_ZF_3_4_T1:
 assumes A1: $s \in \text{AH}$ $r \in \text{AH}$
 shows $\text{Composition}(G) \langle s, r \rangle \in \text{AH}$ $s \circ r \in \text{AH}$
<proof>

The set of almost homomorphisms is closed under composition. The second operation on almost homomorphisms is associative.

lemma (in group1) Group_ZF_3_4_L6: shows
 $\text{AH} \text{ \{is closed under\} } \text{Composition}(G)$
 $\text{AHomOp2}(G, P) \text{ \{is associative on\} } \text{AH}$

<proof>

Type information related to the situation of two almost homomorphisms.

lemma (in group1) Group_ZF_3_4_L7:
 assumes A1: $s \in \text{AH}$ $r \in \text{AH}$ and A2: $n \in G$
 shows
 $s(n) \in G$ $(r(n))^{-1} \in G$
 $s(n) \cdot (r(n))^{-1} \in G$ $s(r(n)) \in G$
<proof>

Type information related to the situation of three almost homomorphisms.

lemma (in group1) Group_ZF_3_4_L8:
 assumes A1: $s \in \text{AH}$ $r \in \text{AH}$ $q \in \text{AH}$ and A2: $n \in G$
 shows
 $q(n) \in G$
 $s(r(n)) \in G$
 $r(n) \cdot (q(n))^{-1} \in G$
 $s(r(n) \cdot (q(n))^{-1}) \in G$
 $\delta(s, \langle q(n), r(n) \cdot (q(n))^{-1} \rangle) \in G$
<proof>

A formula useful in showing that the composition of almost homomorphisms is congruent with respect to the quotient group relation.

lemma (in group1) Group_ZF_3_4_L9:
 assumes A1: $s1 \in \text{AH}$ $r1 \in \text{AH}$ $s2 \in \text{AH}$ $r2 \in \text{AH}$
 and A2: $n \in G$
 shows $(s1 \circ r1)(n) \cdot ((s2 \circ r2)(n))^{-1} =$
 $s1(r2(n)) \cdot (s2(r2(n)))^{-1} \cdot s1(r1(n) \cdot (r2(n))^{-1}) \cdot$
 $\delta(s1, \langle r2(n), r1(n) \cdot (r2(n))^{-1} \rangle)$
<proof>

The next lemma shows a formula that translates an expression in terms of the first group operation on almost homomorphisms and the group inverse in the group of almost homomorphisms to an expression using only the underlying group operations.

lemma (in group1) Group_ZF_3_4_L10: assumes A1: $s \in \text{AH}$ $r \in \text{AH}$
 and A2: $n \in G$
 shows $(s \cdot (\text{GroupInv}(\text{AH}, \text{Op1})(r)))(n) = s(n) \cdot (r(n))^{-1}$
<proof>

A necessary condition for two a. h. to be almost equal.

lemma (in group1) Group_ZF_3_4_L11:
 assumes A1: $s \cong r$
 shows $\{s(n) \cdot (r(n))^{-1} \mid n \in G\} \in \text{Fin}(G)$
<proof>

A sufficient condition for two a. h. to be almost equal.

lemma (in group1) Group_ZF_3_4_L12: assumes A1: $s \in \text{AH}$ $r \in \text{AH}$

```

and A2: {s(n)·(r(n))-1. n∈G} ∈ Fin(G)
shows s≅r
⟨proof⟩

```

Another sufficient condition for two a.h. to be almost equal. It is actually just an expansion of the definition of the quotient group relation.

```

lemma (in group1) Group_ZF_3_4_L12A: assumes s∈AH r∈AH
and s·(GroupInv(AH,Op1)(r)) ∈ FR
shows s≅r r≅s
⟨proof⟩

```

Another necessary condition for two a.h. to be almost equal. It is actually just an expansion of the definition of the quotient group relation.

```

lemma (in group1) Group_ZF_3_4_L12B: assumes s≅r
shows s·(GroupInv(AH,Op1)(r)) ∈ FR
⟨proof⟩

```

The next lemma states the essential condition for the composition of a. h. to be congruent with respect to the quotient group relation for the subgroup of finite range functions.

```

lemma (in group1) Group_ZF_3_4_L13:
assumes A1: s1≅s2 r1≅r2
shows (s1or1) ≅ (s2or2)
⟨proof⟩

```

Composition of a. h. to is congruent with respect to the quotient group relation for the subgroup of finite range functions. Recall that if an operation say "o" on X is congruent with respect to an equivalence relation R then we can define the operation on the quotient space X/R by $[s]_R \circ [r]_R := [s \circ r]_R$ and this definition will be correct i.e. it will not depend on the choice of representants for the classes $[x]$ and $[y]$. This is why we want it here.

```

lemma (in group1) Group_ZF_3_4_L13A: shows
  Congruent2(QuotientGroupRel(AH,Op1,FR),Op2)
⟨proof⟩

```

The homomorphism difference for the identity function is equal to the neutral element of the group (denoted e in the group1 context).

```

lemma (in group1) Group_ZF_3_4_L14: assumes A1: x ∈ G×G
shows δ(id(G),x) = 1
⟨proof⟩

```

The identity function ($I(x) = x$) on G is an almost homomorphism.

```

lemma (in group1) Group_ZF_3_4_L15: shows id(G) ∈ AH
⟨proof⟩

```

Almost homomorphisms form a monoid with composition. The identity function on the group is the neutral element there.

```

lemma (in group1) Group_ZF_3_4_L16:
  shows
    IsAmonoid(AH,Op2)
    monoid0(AH,Op2)
    id(G) = TheNeutralElement(AH,Op2)
  <proof>

```

We can project the monoid of almost homomorphisms with composition to the group of almost homomorphisms divided by the subgroup of finite range functions. The class of the identity function is the neutral element of the quotient (monoid).

```

theorem (in group1) Group_ZF_3_4_T2:
  assumes A1: R = QuotientGroupRel(AH,Op1,FR)
  shows
    IsAmonoid(AH//R,ProjFun2(AH,R,Op2))
    R{id(G)} = TheNeutralElement(AH//R,ProjFun2(AH,R,Op2))
  <proof>

```

39.5 Shifting almost homomorphisms

In this this section we consider what happens if we multiply an almost homomorphism by a group element. We show that the resulting function is also an a. h., and almost equal to the original one. This is used only for slopes (integer a.h.) in Int_ZF_2 where we need to correct a positive slopes by adding a constant, so that it is at least 2 on positive integers.

If s is an almost homomorphism and c is some constant from the group, then $s \cdot c$ is an almost homomorphism.

```

lemma (in group1) Group_ZF_3_5_L1:
  assumes A1:  $s \in AH$  and A2:  $c \in G$  and
  A3:  $r = \{ \langle x, s(x) \cdot c \rangle . x \in G \}$ 
  shows
     $\forall x \in G. r(x) = s(x) \cdot c$ 
     $r \in AH$ 
     $s \cong r$ 
  <proof>

```

end

40 Ordered groups - introduction

```

theory OrderedGroup_ZF imports Group_ZF_1 AbelianGroup_ZF Finite_ZF_1
OrderedLoop_ZF

```

begin

This theory file defines and shows the basic properties of (partially or linearly) ordered groups. We show that in linearly ordered groups finite sets

are bounded and provide a sufficient condition for bounded sets to be finite. This allows to show in `Int_ZF_IML.thy` that subsets of integers are bounded iff they are finite. Some theorems proven here are properties of ordered loops rather than groups. However, for now the development is independent from the material in the `OrderedLoop_ZF` theory, we just import the definitions of `NonnegativeSet` and `PositiveSet` from there.

40.1 Ordered groups

This section defines ordered groups and various related notions.

An ordered group is a group equipped with a partial order that is "translation invariant", that is if $a \leq b$ then $a \cdot g \leq b \cdot g$ and $g \cdot a \leq g \cdot b$.

definition

$$\begin{aligned} \text{IsAnOrdGroup}(G, P, r) \equiv & \\ & (\text{IsAGroup}(G, P) \wedge r \subseteq G \times G \wedge \text{IsPartOrder}(G, r) \wedge (\forall g \in G. \forall a \ b. \\ & \langle a, b \rangle \in r \longrightarrow \langle P\langle a, g \rangle, P\langle b, g \rangle \rangle \in r \wedge \langle P\langle g, a \rangle, P\langle g, b \rangle \rangle \in r)) \end{aligned}$$

We also define the absolute value as a ZF-function that is the identity on G^+ and the group inverse on the rest of the group.

definition

$$\begin{aligned} \text{AbsoluteValue}(G, P, r) \equiv & \text{id}(\text{Nonnegative}(G, P, r)) \cup \\ & \text{restrict}(\text{GroupInv}(G, P), G - \text{Nonnegative}(G, P, r)) \end{aligned}$$

The odd functions are defined as those having property $f(a^{-1}) = (f(a))^{-1}$. This looks a bit strange in the multiplicative notation, I have to admit. For linearly ordered groups a function f defined on the set of positive elements uniquely defines an odd function of the whole group. This function is called an odd extension of f .

definition

$$\begin{aligned} \text{OddExtension}(G, P, r, f) \equiv & \\ & (f \cup \{ \langle a, \text{GroupInv}(G, P)(f(\text{GroupInv}(G, P)(a))) \rangle \}. \\ & a \in \text{GroupInv}(G, P)(\text{PositiveSet}(G, P, r)) \} \cup \\ & \{ \langle \text{TheNeutralElement}(G, P), \text{TheNeutralElement}(G, P) \rangle \}) \end{aligned}$$

We will use a similar notation for ordered groups as for the generic groups. G^+ denotes the set of nonnegative elements (that satisfy $1 \leq a$) and G_+ is the set of (strictly) positive elements. $-A$ is the set inverses of elements from A . I hope that using additive notation for this notion is not too shocking here. The symbol f° denotes the odd extension of f . For a function defined on G_+ this is the unique odd function on G that is equal to f on G_+ .

locale group3 =

fixes G and P and r

assumes `ordGroupAssum`: `IsAnOrdGroup`(G, P, r)

```

fixes unit (1)
defines unit_def [simp]: 1  $\equiv$  TheNeutralElement(G,P)

fixes groper (infixl  $\cdot$  70)
defines groper_def [simp]:  $a \cdot b \equiv P\langle a,b \rangle$ 

fixes inv ( $_^{-1}$  [90] 91)
defines inv_def [simp]:  $x^{-1} \equiv \text{GroupInv}(G,P)(x)$ 

fixes lesseq (infix  $\leq$  68)
defines lesseq_def [simp]:  $a \leq b \equiv \langle a,b \rangle \in r$ 

fixes sless (infix  $<$  68)
defines sless_def [simp]:  $a < b \equiv a \leq b \wedge a \neq b$ 

fixes nonnegative ( $G^+$ )
defines nonnegative_def [simp]:  $G^+ \equiv \text{Nonnegative}(G,P,r)$ 

fixes positive ( $G_+$ )
defines positive_def [simp]:  $G_+ \equiv \text{PositiveSet}(G,P,r)$ 

fixes setinv ( $-$  _ 72)
defines setninv_def [simp]:  $-A \equiv \text{GroupInv}(G,P)(A)$ 

fixes abs ( $|$  _  $|$ )
defines abs_def [simp]:  $|a| \equiv \text{AbsoluteValue}(G,P,r)(a)$ 

fixes oddext ( $_^\circ$ )
defines oddext_def [simp]:  $f^\circ \equiv \text{OddExtension}(G,P,r,f)$ 

fixes listprod ( $\prod$  _ 70)
defines listprod_def [simp]:  $\prod s \equiv \text{Fold}(P,1,s)$ 

```

In group3 context we can use the theorems proven in the group0 context.

```

lemma (in group3) OrderedGroup_ZF_1_L1: shows group0(G,P)
  <proof>

```

Ordered group (carrier) is not empty. This is a property of monoids, but it is good to have it handy in the group3 context.

```

lemma (in group3) OrderedGroup_ZF_1_L1A: shows  $G \neq 0$ 
  <proof>

```

The next lemma is just to see the definition of the nonnegative set in our notation.

```

lemma (in group3) OrderedGroup_ZF_1_L2:
  shows  $g \in G^+ \longleftrightarrow 1 \leq g$ 
  <proof>

```

The next lemma is just to see the definition of the positive set in our notation.

```
lemma (in group3) OrderedGroup_ZF_1_L2A:
  shows  $g \in G_+ \iff (1 \leq g \wedge g \neq 1)$ 
  <proof>
```

For total order if g is not in G^+ , then it has to be less or equal the unit.

```
lemma (in group3) OrderedGroup_ZF_1_L2B:
  assumes A1:  $r \text{ {is total on} } G$  and A2:  $a \in G \setminus G^+$ 
  shows  $a \leq 1 \wedge a < 1$ 
  <proof>
```

The group order is reflexive.

```
lemma (in group3) OrderedGroup_ZF_1_L3: assumes  $g \in G$ 
  shows  $g \leq g$ 
  <proof>
```

1 is nonnegative.

```
lemma (in group3) OrderedGroup_ZF_1_L3A: shows  $1 \in G^+$ 
  <proof>
```

In this context $a \leq b$ implies that both a and b belong to G .

```
lemma (in group3) OrderedGroup_ZF_1_L4:
  assumes  $a \leq b$  shows  $a \in G \wedge b \in G$ 
  <proof>
```

Similarly in this context $a \leq b$ implies that both a and b belong to G .

```
lemma (in group3) less_are_members:
  assumes  $a < b$  shows  $a \in G \wedge b \in G$ 
  <proof>
```

It is good to have transitivity handy.

```
lemma (in group3) Group_order_transitive:
  assumes A1:  $a \leq b \wedge b \leq c$  shows  $a \leq c$ 
  <proof>
```

The order in an ordered group is antisymmetric.

```
lemma (in group3) group_order_antisym:
  assumes A1:  $a \leq b \wedge b \leq a$  shows  $a = b$ 
  <proof>
```

Transitivity for the strict order: if $a < b$ and $b \leq c$, then $a < c$.

```
lemma (in group3) OrderedGroup_ZF_1_L4A:
  assumes A1:  $a < b$  and A2:  $b \leq c$ 
  shows  $a < c$ 
  <proof>
```

If $a < b$ then it's not true that $b \leq a$.

lemma (in group3) ls_not_leq: assumes $a < b$ shows $\neg(b \leq a)$
<proof>

Another version of transitivity for the strict order: if $a \leq b$ and $b < c$, then $a < c$.

lemma (in group3) group_strict_ord_transit:
 assumes A1: $a \leq b$ and A2: $b < c$
 shows $a < c$
<proof>

The order is translation invariant.

lemma (in group3) ord_transl_inv: assumes $a \leq b$ $c \in G$
 shows $a \cdot c \leq b \cdot c$ and $c \cdot a \leq c \cdot b$
<proof>

Strict order is preserved by translations.

lemma (in group3) group_strict_ord_transl_inv:
 assumes $a < b$ and $c \in G$
 shows $a \cdot c < b \cdot c$ and $c \cdot a < c \cdot b$
<proof>

If the group order is total, then the group is ordered linearly.

lemma (in group3) group_ord_total_is_lin:
 assumes r {is total on} G
 shows $\text{IsLinOrder}(G, r)$
<proof>

For linearly ordered groups elements in the nonnegative set are greater than those in the complement.

lemma (in group3) OrderedGroup_ZF_1_L4B:
 assumes r {is total on} G
 and $a \in G^+$ and $b \in G - G^+$
 shows $b \leq a$
<proof>

If $a \leq 1$ and $a \neq 1$, then $a \in G \setminus G^+$.

lemma (in group3) OrderedGroup_ZF_1_L4C:
 assumes A1: $a \leq 1$ and A2: $a \neq 1$
 shows $a \in G \setminus G^+$
<proof>

If a is smaller than the neutral element then a is in the complement of the set of nonnegative elements. This is just OrderedGroup_ZF_1_L4C with assumptions in a different form.

corollary (in group3) smaller_one_negative: assumes $a < 1$
 shows $a \in G \setminus G^+$ *<proof>*

An element smaller than an element in $G \setminus G^+$ is in $G \setminus G^+$.

lemma (in group3) OrderedGroup_ZF_1_L4D:
 assumes A1: $a \in G \setminus G^+$ and A2: $b \leq a$
 shows $b \in G \setminus G^+$
<proof>

The nonnegative set is contained in the group.

lemma (in group3) OrderedGroup_ZF_1_L4E: shows $G^+ \subseteq G$
<proof>

The positive set is contained in the nonnegative set, hence in the group.

lemma (in group3) pos_set_in_gr: shows $G_+ \subseteq G^+$ and $G_+ \subseteq G$
<proof>

Taking the inverse on both sides reverses the inequality.

lemma (in group3) OrderedGroup_ZF_1_L5:
 assumes A1: $a \leq b$ shows $b^{-1} \leq a^{-1}$
<proof>

Taking the inverse on both sides reverses the strict inequality.

lemma (in group3) inv_both_strict_ineq:
 assumes $a < b$ shows $b^{-1} < a^{-1}$
<proof>

If an element is less or equal than the unit, then its inverse is nonnegative.

lemma (in group3) OrderedGroup_ZF_1_L5A:
 assumes A1: $a \leq 1$ shows $1 \leq a^{-1}$
<proof>

If an the inverse of an element is greater than the unit, then the element is smaller.

lemma (in group3) OrderedGroup_ZF_1_L5AA:
 assumes A1: $a \in G$ and A2: $1 \leq a^{-1}$
 shows $a \leq 1$
<proof>

If an element is nonnegative, then the inverse is not greater than the unit.
 Also shows that nonnegative elements cannot be negative

lemma (in group3) OrderedGroup_ZF_1_L5AB:
 assumes A1: $1 \leq a$ shows $a^{-1} \leq 1$ and $\neg(a \leq 1 \wedge a \neq 1)$
<proof>

If an element is positive, then its inverse is negative.

lemma (in group3) pos_inv_neg: assumes $1 < a$
 shows $a^{-1} < 1$
<proof>

If two elements are greater or equal than the unit, then the inverse of one is not greater than the other.

```

lemma (in group3) OrderedGroup_ZF_1_L5AC:
  assumes A1:  $1 \leq a$   $1 \leq b$ 
  shows  $a^{-1} \leq b$ 
<proof>

```

40.2 Inequalities

This section develops some simple tools to deal with inequalities.

Taking negative on both sides reverses the inequality, case with an inverse on one side.

```

lemma (in group3) OrderedGroup_ZF_1_L5AD:
  assumes A1:  $b \in G$  and A2:  $a \leq b^{-1}$ 
  shows  $b \leq a^{-1}$ 
<proof>

```

We can cancel the same element on both sides of an inequality.

```

lemma (in group3) OrderedGroup_ZF_1_L5AE:
  assumes A1:  $a \in G$   $b \in G$   $c \in G$  and A2:  $a \cdot b \leq a \cdot c$ 
  shows  $b \leq c$ 
<proof>

```

We can cancel the same element on both sides of an inequality, right side.

```

lemma (in group3) ineq_cancel_right:
  assumes  $a \in G$   $b \in G$   $c \in G$  and  $a \cdot b \leq c \cdot b$ 
  shows  $a \leq c$ 
<proof>

```

We can cancel the same element on both sides of an inequality, a version with an inverse on both sides.

```

lemma (in group3) OrderedGroup_ZF_1_L5AF:
  assumes A1:  $a \in G$   $b \in G$   $c \in G$  and A2:  $a \cdot b^{-1} \leq a \cdot c^{-1}$ 
  shows  $c \leq b$ 
<proof>

```

Taking negative on both sides reverses the inequality, another case with an inverse on one side.

```

lemma (in group3) OrderedGroup_ZF_1_L5AG:
  assumes A1:  $a \in G$  and A2:  $a^{-1} \leq b$ 
  shows  $b^{-1} \leq a$ 
<proof>

```

We can multiply the sides of two inequalities.

```

lemma (in group3) OrderedGroup_ZF_1_L5B:
  assumes A1:  $a \leq b$  and A2:  $c \leq d$ 
  shows  $a \cdot c \leq b \cdot d$ 
<proof>

```

The set of nonnegative elements is closed with respect to the group operation.

lemma (in group3) group_nonnegative_closed: assumes $a \in G^+$ $b \in G^+$
 shows $a \cdot b \in G^+$
<proof>

We can replace first of the factors on one side of an inequality with a greater one.

lemma (in group3) OrderedGroup_ZF_1_L5C:
 assumes A1: $c \in G$ and A2: $a \leq b \cdot c$ and A3: $b \leq b_1$
 shows $a \leq b_1 \cdot c$
<proof>

We can replace second of the factors on one side of an inequality with a greater one.

lemma (in group3) OrderedGroup_ZF_1_L5D:
 assumes A1: $b \in G$ and A2: $a \leq b \cdot c$ and A3: $c \leq c_1$
 shows $a \leq b \cdot c_1$
<proof>

We can replace factors on one side of an inequality with greater ones.

lemma (in group3) OrderedGroup_ZF_1_L5E:
 assumes A1: $a \leq b \cdot c$ and A2: $b \leq b_1$ $c \leq c_1$
 shows $a \leq b_1 \cdot c_1$
<proof>

We don't decrease an element of the group by multiplying by one that is nonnegative.

lemma (in group3) OrderedGroup_ZF_1_L5F:
 assumes A1: $1 \leq a$ and A2: $b \in G$
 shows $b \leq a \cdot b$ $b \leq b \cdot a$
<proof>

We can multiply the right hand side of an inequality by a nonnegative element.

lemma (in group3) OrderedGroup_ZF_1_L5G: assumes A1: $a \leq b$
 and A2: $1 \leq c$ shows $a \leq b \cdot c$ $a \leq c \cdot b$
<proof>

We can put two elements on the other side of inequality, changing their sign.

lemma (in group3) OrderedGroup_ZF_1_L5H:
 assumes A1: $a \in G$ $b \in G$ and A2: $a \cdot b^{-1} \leq c$
 shows
 $a \leq c \cdot b$
 $c^{-1} \cdot a \leq b$
<proof>

We can multiply the sides of one inequality by inverse of another.

```
lemma (in group3) OrderedGroup_ZF_1_L5I:
  assumes a≤b and c≤d
  shows a·d-1 ≤ b·c-1
  <proof>
```

We can put an element on the other side of an inequality changing its sign, version with the inverse.

```
lemma (in group3) OrderedGroup_ZF_1_L5J:
  assumes A1: a∈G b∈G and A2: c ≤ a·b-1
  shows c·b ≤ a
  <proof>
```

We can put an element on the other side of an inequality changing its sign, version with the inverse.

```
lemma (in group3) OrderedGroup_ZF_1_L5JA:
  assumes A1: a∈G b∈G and A2: c ≤ a-1·b
  shows a·c ≤ b
  <proof>
```

A special case of OrderedGroup_ZF_1_L5J where $c = 1$.

```
corollary (in group3) OrderedGroup_ZF_1_L5K:
  assumes A1: a∈G b∈G and A2: 1 ≤ a·b-1
  shows b ≤ a
  <proof>
```

A special case of OrderedGroup_ZF_1_L5JA where $c = 1$.

```
corollary (in group3) OrderedGroup_ZF_1_L5KA:
  assumes A1: a∈G b∈G and A2: 1 ≤ a-1·b
  shows a ≤ b
  <proof>
```

If the order is total, the elements that do not belong to the positive set are negative. We also show here that the group inverse of an element that does not belong to the nonnegative set does belong to the nonnegative set.

```
lemma (in group3) OrderedGroup_ZF_1_L6:
  assumes A1: r {is total on} G and A2: a∈G\G+
  shows a≤1 a-1 ∈ G+ restrict(GroupInv(G,P),G-G+)(a) ∈ G+
  <proof>
```

If a property is invariant with respect to taking the inverse and it is true on the nonnegative set, than it is true on the whole group.

```
lemma (in group3) OrderedGroup_ZF_1_L7:
  assumes A1: r {is total on} G
  and A2: ∀ a∈G+. ∀ b∈G+. Q(a,b)
  and A3: ∀ a∈G. ∀ b∈G. Q(a,b) → Q(a-1,b)
  and A4: ∀ a∈G. ∀ b∈G. Q(a,b) → Q(a,b-1)
```

and A5: $a \in G \ b \in G$
 shows $Q(a, b)$
 $\langle proof \rangle$

If a, b are an elements of an ordered group where the order is total, then
 $a \leq b$ or $b < a$.

lemma (in group3) OrdGroup_2cases: assumes $r \ \{is \ total \ on\} \ G \ a \in G \ b \in G$
 shows $a \leq b \vee b < a$
 $\langle proof \rangle$

If a, b are an elements of an ordered group where the order is total, then
 $a < b$ or $a = b$ or $b \leq a$.

lemma (in group3) OrdGroup_3cases: assumes $r \ \{is \ total \ on\} \ G \ a \in G \ b \in G$
 shows $a < b \vee a = b \vee b < a$
 $\langle proof \rangle$

A lemma about splitting the ordered group "plane" into 6 subsets. Useful
 for proofs by cases.

lemma (in group3) OrdGroup_6cases: assumes A1: $r \ \{is \ total \ on\} \ G$
 and A2: $a \in G \ b \in G$
 shows
 $1 \leq a \wedge 1 \leq b \vee a \leq 1 \wedge b \leq 1 \vee$
 $a \leq 1 \wedge 1 \leq b \wedge 1 \leq a \cdot b \vee a \leq 1 \wedge 1 \leq b \wedge a \cdot b \leq 1 \vee$
 $1 \leq a \wedge b \leq 1 \wedge 1 \leq a \cdot b \vee 1 \leq a \wedge b \leq 1 \wedge a \cdot b \leq 1$
 $\langle proof \rangle$

The next lemma shows what happens when one element of a totally ordered
 group is not greater or equal than another.

lemma (in group3) OrderedGroup_ZF_1_L8:
 assumes A1: $r \ \{is \ total \ on\} \ G$
 and A2: $a \in G \ b \in G$
 and A3: $\neg(a \leq b)$
 shows $b \leq a \ a^{-1} \leq b^{-1} \ a \neq b \ b < a$

$\langle proof \rangle$

If one element is greater or equal and not equal to another, then it is not
 smaller or equal.

lemma (in group3) OrderedGroup_ZF_1_L8AA:
 assumes A1: $a \leq b$ and A2: $a \neq b$
 shows $\neg(b \leq a)$
 $\langle proof \rangle$

A special case of OrderedGroup_ZF_1_L8 when one of the elements is the unit.

corollary (in group3) OrderedGroup_ZF_1_L8A:
 assumes A1: $r \ \{is \ total \ on\} \ G$
 and A2: $a \in G$ and A3: $\neg(1 \leq a)$

shows $1 \leq a^{-1} \quad 1 \neq a \quad a \leq 1$
 $\langle proof \rangle$

A negative element can not be nonnegative.

lemma (in group3) OrderedGroup_ZF_1_L8B:
assumes A1: $a \leq 1$ **and** A2: $a \neq 1$ **shows** $\neg(1 \leq a)$
 $\langle proof \rangle$

An element is greater or equal than another iff the difference is nonpositive.

lemma (in group3) OrderedGroup_ZF_1_L9:
assumes A1: $a \in G \quad b \in G$
shows $a \leq b \iff a \cdot b^{-1} \leq 1$
 $\langle proof \rangle$

We can move an element to the other side of an inequality.

lemma (in group3) OrderedGroup_ZF_1_L9A:
assumes A1: $a \in G \quad b \in G \quad c \in G$
shows $a \cdot b \leq c \iff a \leq c \cdot b^{-1}$
 $\langle proof \rangle$

A one side version of the previous lemma with weaker assumptions.

lemma (in group3) OrderedGroup_ZF_1_L9B:
assumes A1: $a \in G \quad b \in G$ **and** A2: $a \cdot b^{-1} \leq c$
shows $a \leq c \cdot b$
 $\langle proof \rangle$

We can put an element on the other side of inequality, changing its sign.

lemma (in group3) OrderedGroup_ZF_1_L9C:
assumes A1: $a \in G \quad b \in G$ **and** A2: $c \leq a \cdot b$
shows
 $c \cdot b^{-1} \leq a$
 $a^{-1} \cdot c \leq b$
 $\langle proof \rangle$

If an element is greater or equal than another then the difference is nonnegative.

lemma (in group3) OrderedGroup_ZF_1_L9D: **assumes** A1: $a \leq b$
shows $1 \leq b \cdot a^{-1}$
 $\langle proof \rangle$

If an element is greater than another then the difference is positive.

lemma (in group3) OrderedGroup_ZF_1_L9E:
assumes A1: $a \leq b \quad a \neq b$
shows $1 \leq b \cdot a^{-1} \quad 1 \neq b \cdot a^{-1} \quad b \cdot a^{-1} \in G_+$
 $\langle proof \rangle$

If the difference is nonnegative, then $a \leq b$.

```

lemma (in group3) OrderedGroup_ZF_1_L9F:
  assumes A1:  $a \in G$   $b \in G$  and A2:  $1 \leq b \cdot a^{-1}$ 
  shows  $a \leq b$ 
<proof>

```

If we increase the middle term in a product, the whole product increases.

```

lemma (in group3) OrderedGroup_ZF_1_L10:
  assumes  $a \in G$   $b \in G$  and  $c \leq d$ 
  shows  $a \cdot c \cdot b \leq a \cdot d \cdot b$ 
<proof>

```

A product of (strictly) positive elements is not the unit.

```

lemma (in group3) OrderedGroup_ZF_1_L11:
  assumes A1:  $1 \leq a$   $1 \leq b$ 
  and A2:  $1 \neq a$   $1 \neq b$ 
  shows  $1 \neq a \cdot b$ 
<proof>

```

A product of nonnegative elements is nonnegative.

```

lemma (in group3) OrderedGroup_ZF_1_L12:
  assumes A1:  $1 \leq a$   $1 \leq b$ 
  shows  $1 \leq a \cdot b$ 
<proof>

```

If a is not greater than b , then 1 is not greater than $b \cdot a^{-1}$.

```

lemma (in group3) OrderedGroup_ZF_1_L12A:
  assumes A1:  $a \leq b$  shows  $1 \leq b \cdot a^{-1}$ 
<proof>

```

We can move an element to the other side of a strict inequality.

```

lemma (in group3) OrderedGroup_ZF_1_L12B:
  assumes A1:  $a \in G$   $b \in G$  and A2:  $a \cdot b^{-1} < c$ 
  shows  $a < c \cdot b$ 
<proof>

```

We can multiply the sides of two inequalities, first of them strict and we get a strict inequality.

```

lemma (in group3) OrderedGroup_ZF_1_L12C:
  assumes A1:  $a < b$  and A2:  $c \leq d$ 
  shows  $a \cdot c < b \cdot d$ 
<proof>

```

We can multiply the sides of two inequalities, second of them strict and we get a strict inequality.

```

lemma (in group3) OrderedGroup_ZF_1_L12D:
  assumes A1:  $a \leq b$  and A2:  $c < d$ 
  shows  $a \cdot c < b \cdot d$ 

```


<proof>

If two elements of the group are smaller than the neutral element then their product is also smaller.

lemma (in group3) group_less_less: assumes $a < 1$ $b < 1$
 shows $a \cdot b < 1$

<proof>

If the order is total the complement of the set of nonnegative elements is closed with respect to the group operation.

lemma (in group3) group_negative_closed:
 assumes $r \text{ \{is total on\} } G$ $a \in G \setminus G^+$ $b \in G \setminus G^+$
 shows $a \cdot b \in G \setminus G^+$

<proof>

40.3 The set of positive elements

In this section we study G_+ - the set of elements that are (strictly) greater than the unit. The most important result is that every linearly ordered group can be decomposed into $\{1\}$, G_+ and the set of those elements $a \in G$ such that $a^{-1} \in G_+$. Another property of linearly ordered groups that we prove here is that if $G_+ \neq \emptyset$, then it is infinite. This allows to show that nontrivial linearly ordered groups are infinite.

The positive set is closed under the group operation.

lemma (in group3) OrderedGroup_ZF_1_L13: shows G_+ {is closed under}

<proof>

For totally ordered groups every nonunit element is positive or its inverse is positive.

lemma (in group3) OrderedGroup_ZF_1_L14:
 assumes $A1: r \text{ \{is total on\} } G$ and $A2: a \in G$
 shows $a = 1 \vee a \in G_+ \vee a^{-1} \in G_+$

<proof>

If an element belongs to the positive set, then it is not the unit and its inverse does not belong to the positive set.

lemma (in group3) OrderedGroup_ZF_1_L15:
 assumes $A1: a \in G_+$ shows $a \neq 1$ $a^{-1} \notin G_+$

<proof>

If a^{-1} is positive, then a can not be positive or the unit.

lemma (in group3) OrderedGroup_ZF_1_L16:
 assumes $A1: a \in G$ and $A2: a^{-1} \in G_+$ shows $a \neq 1$ $a \notin G_+$

<proof>

For linearly ordered groups each element is either the unit, positive or its inverse is positive.

lemma (in group3) OrdGroup_decomp:
 assumes A1: r {is total on} G and A2: $a \in G$
 shows Exactly_1_of_3_holds ($a=1, a \in G_+, a^{-1} \in G_+$)
<proof>

A if a is a nonunit element that is not positive, then a^{-1} is positive. This is useful for some proofs by cases.

lemma (in group3) OrdGroup_cases:
 assumes A1: r {is total on} G and A2: $a \in G$
 and A3: $a \neq 1$ $a \notin G_+$
 shows $a^{-1} \in G_+$
<proof>

Elements from $G \setminus G_+$ are not greater than the unit.

lemma (in group3) OrderedGroup_ZF_1_L17:
 assumes A1: r {is total on} G and A2: $a \in G - G_+$
 shows $a \leq 1$
<proof>

The next lemma allows to split proofs that something holds for all $a \in G$ into cases $a = 1$, $a \in G_+$, $-a \in G_+$.

lemma (in group3) OrderedGroup_ZF_1_L18:
 assumes A1: r {is total on} G and A2: $b \in G$
 and A3: $Q(1)$ and A4: $\forall a \in G_+. Q(a)$ and A5: $\forall a \in G_+. Q(a^{-1})$
 shows $Q(b)$
<proof>

All elements greater or equal than an element of G_+ belong to G_+ .

lemma (in group3) OrderedGroup_ZF_1_L19:
 assumes A1: $a \in G_+$ and A2: $a \leq b$
 shows $b \in G_+$
<proof>

The inverse of an element of G_+ cannot be in G_+ .

lemma (in group3) OrderedGroup_ZF_1_L20:
 assumes A1: r {is total on} G and A2: $a \in G_+$
 shows $a^{-1} \notin G_+$
<proof>

The set of positive elements of a nontrivial linearly ordered group is not empty.

lemma (in group3) OrderedGroup_ZF_1_L21:
 assumes A1: r {is total on} G and A2: $G \neq \{1\}$
 shows $G_+ \neq \emptyset$
<proof>

If $b \in G_+$, then $a < a \cdot b$. Multiplying a by a positive element increases a .

lemma (in group3) OrderedGroup_ZF_1_L22:

assumes A1: $a \in G$ $b \in G_+$

shows $a \leq a \cdot b$ $a \neq a \cdot b$ $a \cdot b \in G$

<proof>

If G is a nontrivial linearly ordered group, then for every element of G we can find one in G_+ that is greater or equal.

lemma (in group3) OrderedGroup_ZF_1_L23:

assumes A1: r {is total on} G and A2: $G \neq \{1\}$

and A3: $a \in G$

shows $\exists b \in G_+. a \leq b$

<proof>

The G^+ is G_+ plus the unit.

lemma (in group3) OrderedGroup_ZF_1_L24: shows $G^+ = G_+ \cup \{1\}$

<proof>

What is $-G_+$, really?

lemma (in group3) OrderedGroup_ZF_1_L25: shows

$(-G_+) = \{a^{-1}. a \in G_+\}$

$(-G_+) \subseteq G$

<proof>

If the inverse of a is in G_+ , then a is in the inverse of G_+ .

lemma (in group3) OrderedGroup_ZF_1_L26:

assumes A1: $a \in G$ and A2: $a^{-1} \in G_+$

shows $a \in (-G_+)$

<proof>

If a is in the inverse of G_+ , then its inverse is in G_+ .

lemma (in group3) OrderedGroup_ZF_1_L27:

assumes $a \in (-G_+)$

shows $a^{-1} \in G_+$

<proof>

A linearly ordered group can be decomposed into G_+ , $\{1\}$ and $-G_+$

lemma (in group3) OrdGroup_decomp2:

assumes A1: r {is total on} G

shows

$G = G_+ \cup (-G_+) \cup \{1\}$

$G_+ \cap (-G_+) = \emptyset$

$1 \notin G_+ \cup (-G_+)$

<proof>

If $a \cdot b^{-1}$ is nonnegative, then $b \leq a$. This maybe used to recover the order from the set of nonnegative elements and serve as a way to define order by prescribing that set (see the "Alternative definitions" section).

```

lemma (in group3) OrderedGroup_ZF_1_L28:
  assumes A1:  $a \in G$   $b \in G$  and A2:  $a \cdot b^{-1} \in G^+$ 
  shows  $b \leq a$ 
<proof>

```

A special case of OrderedGroup_ZF_1_L28 when $a \cdot b^{-1}$ is positive.

```

corollary (in group3) OrderedGroup_ZF_1_L29:
  assumes A1:  $a \in G$   $b \in G$  and A2:  $a \cdot b^{-1} \in G_+$ 
  shows  $b \leq a$   $b \neq a$ 
<proof>

```

A bit stronger than OrderedGroup_ZF_1_L29, adds case when two elements are equal.

```

lemma (in group3) OrderedGroup_ZF_1_L30:
  assumes  $a \in G$   $b \in G$  and  $a = b \vee b \cdot a^{-1} \in G_+$ 
  shows  $a \leq b$ 
<proof>

```

A different take on decomposition: we can have $a = b$ or $a < b$ or $b < a$.

```

lemma (in group3) OrderedGroup_ZF_1_L31:
  assumes A1:  $r$  {is total on}  $G$  and A2:  $a \in G$   $b \in G$ 
  shows  $a = b \vee (a \leq b \wedge a \neq b) \vee (b \leq a \wedge b \neq a)$ 
<proof>

```

40.4 Intervals and bounded sets

Intervals here are the closed intervals of the form $\{x \in G. a \leq x \leq b\}$.

A bounded set can be translated to put it in G^+ and then it is still bounded above.

```

lemma (in group3) OrderedGroup_ZF_2_L1:
  assumes A1:  $\forall g \in A. L \leq g \wedge g \leq M$ 
  and A2:  $S = \text{RightTranslation}(G, P, L^{-1})$ 
  and A3:  $a \in S(A)$ 
  shows  $a \leq M \cdot L^{-1}$   $1 \leq a$ 
<proof>

```

Every bounded set is an image of a subset of an interval that starts at 1.

```

lemma (in group3) OrderedGroup_ZF_2_L2:
  assumes A1:  $\text{IsBounded}(A, r)$ 
  shows  $\exists B. \exists g \in G^+. \exists T \in G \rightarrow G. A = T(B) \wedge B \subseteq \text{Interval}(r, 1, g)$ 
<proof>

```

If every interval starting at 1 is finite, then every bounded set is finite. I find it interesting that this does not require the group to be linearly ordered (the order to be total).

```

theorem (in group3) OrderedGroup_ZF_2_T1:

```

```

assumes A1:  $\forall g \in G^+. \text{Interval}(r, 1, g) \in \text{Fin}(G)$ 
and A2:  $\text{IsBounded}(A, r)$ 
shows  $A \in \text{Fin}(G)$ 
<proof>

```

In linearly ordered groups finite sets are bounded.

```

theorem (in group3) ord_group_fin_bounded:
  assumes  $r \text{ \{is total on\} } G$  and  $B \in \text{Fin}(G)$ 
  shows  $\text{IsBounded}(B, r)$ 
<proof>

```

For nontrivial linearly ordered groups if for every element G we can find one in A that is greater or equal (not necessarily strictly greater), then A can neither be finite nor bounded above.

```

lemma (in group3) OrderedGroup_ZF_2_L2A:
  assumes A1:  $r \text{ \{is total on\} } G$  and A2:  $G \neq \{1\}$ 
  and A3:  $\forall a \in G. \exists b \in A. a \leq b$ 
  shows
     $\forall a \in G. \exists b \in A. a \neq b \wedge a \leq b$ 
     $\neg \text{IsBoundedAbove}(A, r)$ 
     $A \notin \text{Fin}(G)$ 
<proof>

```

Nontrivial linearly ordered groups are infinite. Recall that $\text{Fin}(A)$ is the collection of finite subsets of A . In this lemma we show that $G \notin \text{Fin}(G)$, that is that G is not a finite subset of itself. This is a way of saying that G is infinite. We also show that for nontrivial linearly ordered groups G_+ is infinite.

```

theorem (in group3) Linord_group_infinite:
  assumes A1:  $r \text{ \{is total on\} } G$  and A2:  $G \neq \{1\}$ 
  shows
     $G_+ \notin \text{Fin}(G)$ 
     $G \notin \text{Fin}(G)$ 
<proof>

```

A property of nonempty subsets of linearly ordered groups that don't have a maximum: for any element in such subset we can find one that is strictly greater.

```

lemma (in group3) OrderedGroup_ZF_2_L2B:
  assumes A1:  $r \text{ \{is total on\} } G$  and A2:  $A \subseteq G$  and
  A3:  $\neg \text{HasAmaximum}(r, A)$  and A4:  $x \in A$ 
  shows  $\exists y \in A. x < y$ 
<proof>

```

In linearly ordered groups $G \setminus G_+$ is bounded above.

```

lemma (in group3) OrderedGroup_ZF_2_L3:
  assumes A1:  $r \text{ \{is total on\} } G$  shows  $\text{IsBoundedAbove}(G - G_+, r)$ 

```

<proof>

In linearly ordered groups if $A \cap G_+$ is finite, then A is bounded above.

```
lemma (in group3) OrderedGroup_ZF_2_L4:
  assumes A1: r {is total on} G and A2:  $A \subseteq G$ 
  and A3:  $A \cap G_+ \in \text{Fin}(G)$ 
  shows IsBoundedAbove(A,r)
<proof>
```

If a set $-A \subseteq G$ is bounded above, then A is bounded below.

```
lemma (in group3) OrderedGroup_ZF_2_L5:
  assumes A1:  $A \subseteq G$  and A2: IsBoundedAbove( $-A$ ,r)
  shows IsBoundedBelow(A,r)
<proof>
```

If $a \leq b$, then the image of the interval $a..b$ by any function is nonempty.

```
lemma (in group3) OrderedGroup_ZF_2_L6:
  assumes  $a \leq b$  and  $f: G \rightarrow G$ 
  shows  $f(\text{Interval}(r,a,b)) \neq \emptyset$ 
<proof>
```

end

41 More on ordered groups

```
theory OrderedGroup_ZF_1 imports OrderedGroup_ZF
```

```
begin
```

In this theory we continue the OrderedGroup_ZF theory development.

41.1 Absolute value and the triangle inequality

The goal of this section is to prove the triangle inequality for ordered groups.

Absolute value maps G into G .

```
lemma (in group3) OrderedGroup_ZF_3_L1:
  shows AbsoluteValue(G,P,r) :  $G \rightarrow G$ 
<proof>
```

If $a \in G^+$, then $|a| = a$.

```
lemma (in group3) OrderedGroup_ZF_3_L2:
  assumes A1:  $a \in G^+$  shows  $|a| = a$ 
<proof>
```

The absolute value of the unit is the unit. In the additive notation that would be $|0| = 0$.

lemma (in group3) OrderedGroup_ZF_3_L2A:
 shows $|1| = 1$ *<proof>*

If a is positive, then $|a| = a$.

lemma (in group3) OrderedGroup_ZF_3_L2B:
 assumes $a \in G_+$ shows $|a| = a$
<proof>

If $a \in G \setminus G^+$, then $|a| = a^{-1}$.

lemma (in group3) OrderedGroup_ZF_3_L3:
 assumes A1: $a \in G \setminus G^+$ shows $|a| = a^{-1}$
<proof>

For elements that not greater than the unit, the absolute value is the inverse.

lemma (in group3) OrderedGroup_ZF_3_L3A:
 assumes A1: $a \leq 1$
 shows $|a| = a^{-1}$
<proof>

In linearly ordered groups the absolute value of any element is in G^+ .

lemma (in group3) OrderedGroup_ZF_3_L3B:
 assumes A1: r {is total on} G and A2: $a \in G$
 shows $|a| \in G^+$
<proof>

For linearly ordered groups (where the order is total), the absolute value maps the group into the positive set.

lemma (in group3) OrderedGroup_ZF_3_L3C:
 assumes A1: r {is total on} G
 shows $\text{AbsoluteValue}(G, P, r) : G \rightarrow G^+$
<proof>

If the absolute value is the unit, then the element is the unit.

lemma (in group3) OrderedGroup_ZF_3_L3D:
 assumes A1: $a \in G$ and A2: $|a| = 1$
 shows $a = 1$
<proof>

In linearly ordered groups the unit is not greater than the absolute value of any element.

lemma (in group3) OrderedGroup_ZF_3_L3E:
 assumes r {is total on} G and $a \in G$
 shows $1 \leq |a|$
<proof>

If b is greater than both a and a^{-1} , then b is greater than $|a|$.

lemma (in group3) OrderedGroup_ZF_3_L4:

assumes A1: $a \leq b$ **and** A2: $a^{-1} \leq b$
shows $|a| \leq b$
 $\langle proof \rangle$

In linearly ordered groups $a \leq |a|$.

lemma (in group3) OrderedGroup_ZF_3_L5:
assumes A1: r {is total on} G **and** A2: $a \in G$
shows $a \leq |a|$
 $\langle proof \rangle$

$a^{-1} \leq |a|$ (in additive notation it would be $-a \leq |a|$).

lemma (in group3) OrderedGroup_ZF_3_L6:
assumes A1: $a \in G$ **shows** $a^{-1} \leq |a|$
 $\langle proof \rangle$

Some inequalities about the product of two elements of a linearly ordered group and its absolute value.

lemma (in group3) OrderedGroup_ZF_3_L6A:
assumes r {is total on} G **and** $a \in G$ $b \in G$
shows
 $a \cdot b \leq |a| \cdot |b|$
 $a \cdot b^{-1} \leq |a| \cdot |b|$
 $a^{-1} \cdot b \leq |a| \cdot |b|$
 $a^{-1} \cdot b^{-1} \leq |a| \cdot |b|$
 $\langle proof \rangle$

$|a^{-1}| \leq |a|$.

lemma (in group3) OrderedGroup_ZF_3_L7:
assumes r {is total on} G **and** $a \in G$
shows $|a^{-1}| \leq |a|$
 $\langle proof \rangle$

$|a^{-1}| = |a|$.

lemma (in group3) OrderedGroup_ZF_3_L7A:
assumes A1: r {is total on} G **and** A2: $a \in G$
shows $|a^{-1}| = |a|$
 $\langle proof \rangle$

$|a \cdot b^{-1}| = |b \cdot a^{-1}|$. It doesn't look so strange in the additive notation:
 $|a - b| = |b - a|$.

lemma (in group3) OrderedGroup_ZF_3_L7B:
assumes A1: r {is total on} G **and** A2: $a \in G$ $b \in G$
shows $|a \cdot b^{-1}| = |b \cdot a^{-1}|$
 $\langle proof \rangle$

Triangle inequality for linearly ordered abelian groups. It would be nice to drop commutativity or give an example that shows we can't do that.


```

theorem (in group3) OrdGroup_triangle_ineq:
  assumes A1: P {is commutative on} G
  and A2: r {is total on} G and A3: a∈G b∈G
  shows |a·b| ≤ |a|·|b|
⟨proof⟩

```

We can multiply the sides of an inequality with absolute value.

```

lemma (in group3) OrderedGroup_ZF_3_L7C:
  assumes P {is commutative on} G
  and r {is total on} G a∈G b∈G
  and |a| ≤ c |b| ≤ d
  shows |a·b| ≤ c·d
⟨proof⟩

```

A version of the OrderedGroup_ZF_3_L7C but with multiplying by the inverse.

```

lemma (in group3) OrderedGroup_ZF_3_L7CA:
  assumes P {is commutative on} G
  and r {is total on} G and a∈G b∈G
  and |a| ≤ c |b| ≤ d
  shows |a·b-1| ≤ c·d
⟨proof⟩

```

Triangle inequality with three integers.

```

lemma (in group3) OrdGroup_triangle_ineq3:
  assumes A1: P {is commutative on} G
  and A2: r {is total on} G and A3: a∈G b∈G c∈G
  shows |a·b·c| ≤ |a|·|b|·|c|
⟨proof⟩

```

Some variants of the triangle inequality.

```

lemma (in group3) OrderedGroup_ZF_3_L7D:
  assumes A1: P {is commutative on} G
  and A2: r {is total on} G and A3: a∈G b∈G
  and A4: |a·b-1| ≤ c
  shows
    |a| ≤ c·|b|
    |a| ≤ |b|·c
    c-1·a ≤ b
    a·c-1 ≤ b
    a ≤ b·c
⟨proof⟩

```

Some more variants of the triangle inequality.

```

lemma (in group3) OrderedGroup_ZF_3_L7E:
  assumes A1: P {is commutative on} G
  and A2: r {is total on} G and A3: a∈G b∈G
  and A4: |a·b-1| ≤ c
  shows b·c-1 ≤ a

```

<proof>

An application of the triangle inequality with four group elements.

lemma (in group3) OrderedGroup_ZF_3_L7F:
 assumes A1: P {is commutative on} G
 and A2: r {is total on} G and
 A3: a∈G b∈G c∈G d∈G
 shows $|a \cdot c^{-1}| \leq |a \cdot b| \cdot |c \cdot d| \cdot |b \cdot d^{-1}|$
<proof>

$|a| \leq L$ implies $L^{-1} \leq a$ (it would be $-L \leq a$ in the additive notation).

lemma (in group3) OrderedGroup_ZF_3_L8:
 assumes A1: a∈G and A2: $|a| \leq L$
 shows
 $L^{-1} \leq a$
<proof>

In linearly ordered groups $|a| \leq L$ implies $a \leq L$ (it would be $a \leq L$ in the additive notation).

lemma (in group3) OrderedGroup_ZF_3_L8A:
 assumes A1: r {is total on} G
 and A2: a∈G and A3: $|a| \leq L$
 shows
 $a \leq L$
 $1 \leq L$
<proof>

A somewhat generalized version of the above lemma.

lemma (in group3) OrderedGroup_ZF_3_L8B:
 assumes A1: a∈G and A2: $|a| \leq L$ and A3: $1 \leq c$
 shows $(L \cdot c)^{-1} \leq a$
<proof>

If b is between a and $a \cdot c$, then $b \cdot a^{-1} \leq c$.

lemma (in group3) OrderedGroup_ZF_3_L8C:
 assumes A1: $a \leq b$ and A2: $c \in G$ and A3: $b \leq c \cdot a$
 shows $|b \cdot a^{-1}| \leq c$
<proof>

For linearly ordered groups if the absolute values of elements in a set are bounded, then the set is bounded.

lemma (in group3) OrderedGroup_ZF_3_L9:
 assumes A1: r {is total on} G
 and A2: $A \subseteq G$ and A3: $\forall a \in A. |a| \leq L$
 shows IsBounded(A, r)
<proof>

A slightly more general version of the previous lemma, stating the same fact for a set defined by separation.

```

lemma (in group3) OrderedGroup_ZF_3_L9A:
  assumes A1: r {is total on} G
  and A2:  $\forall x \in X. b(x) \in G \wedge |b(x)| \leq L$ 
  shows IsBounded( $\{b(x). x \in X\}, r$ )
<proof>

```

A special form of the previous lemma stating a similar fact for an image of a set by a function with values in a linearly ordered group.

```

lemma (in group3) OrderedGroup_ZF_3_L9B:
  assumes A1: r {is total on} G
  and A2:  $f: X \rightarrow G$  and A3:  $A \subseteq X$ 
  and A4:  $\forall x \in A. |f(x)| \leq L$ 
  shows IsBounded( $f(A), r$ )
<proof>

```

For linearly ordered groups if $l \leq a \leq u$ then $|a|$ is smaller than the greater of $|l|, |u|$.

```

lemma (in group3) OrderedGroup_ZF_3_L10:
  assumes A1: r {is total on} G
  and A2:  $l \leq a \leq u$ 
  shows
     $|a| \leq \text{GreaterOf}(r, |l|, |u|)$ 
<proof>

```

For linearly ordered groups if a set is bounded then the absolute values are bounded.

```

lemma (in group3) OrderedGroup_ZF_3_L10A:
  assumes A1: r {is total on} G
  and A2: IsBounded( $A, r$ )
  shows  $\exists L. \forall a \in A. |a| \leq L$ 
<proof>

```

A slightly more general version of the previous lemma, stating the same fact for a set defined by separation.

```

lemma (in group3) OrderedGroup_ZF_3_L11:
  assumes r {is total on} G
  and IsBounded( $\{b(x). x \in X\}, r$ )
  shows  $\exists L. \forall x \in X. |b(x)| \leq L$ 
<proof>

```

Absolute values of elements of a finite image of a nonempty set are bounded by an element of the group.

```

lemma (in group3) OrderedGroup_ZF_3_L11A:
  assumes A1: r {is total on} G
  and A2:  $X \neq 0$  and A3:  $\{b(x). x \in X\} \in \text{Fin}(G)$ 
  shows  $\exists L \in G. \forall x \in X. |b(x)| \leq L$ 
<proof>

```

In totally ordered groups the absolute value of a nonunit element is in G_+ .

```
lemma (in group3) OrderedGroup_ZF_3_L12:
  assumes A1: r {is total on} G
  and A2: a ∈ G and A3: a ≠ 1
  shows |a| ∈ G+
<proof>
```

41.2 Maximum absolute value of a set

Quite often when considering inequalities we prefer to talk about the absolute values instead of raw elements of a set. This section formalizes some material that is useful for that.

If a set has a maximum and minimum, then the greater of the absolute value of the maximum and minimum belongs to the image of the set by the absolute value function.

```
lemma (in group3) OrderedGroup_ZF_4_L1:
  assumes A ⊆ G
  and HasAmaximum(r,A) HasAminimum(r,A)
  and M = GreaterOf(r,|Minimum(r,A)|,|Maximum(r,A)|)
  shows M ∈ AbsoluteValue(G,P,r)(A)
<proof>
```

If a set has a maximum and minimum, then the greater of the absolute value of the maximum and minimum bounds absolute values of all elements of the set.

```
lemma (in group3) OrderedGroup_ZF_4_L2:
  assumes A1: r {is total on} G
  and A2: HasAmaximum(r,A) HasAminimum(r,A)
  and A3: a ∈ A
  shows |a| ≤ GreaterOf(r,|Minimum(r,A)|,|Maximum(r,A)|)
<proof>
```

If a set has a maximum and minimum, then the greater of the absolute value of the maximum and minimum bounds absolute values of all elements of the set. In this lemma the absolute values of elements of a set are represented as the elements of the image of the set by the absolute value function.

```
lemma (in group3) OrderedGroup_ZF_4_L3:
  assumes r {is total on} G and A ⊆ G
  and HasAmaximum(r,A) HasAminimum(r,A)
  and b ∈ AbsoluteValue(G,P,r)(A)
  shows b ≤ GreaterOf(r,|Minimum(r,A)|,|Maximum(r,A)|)
<proof>
```

If a set has a maximum and minimum, then the set of absolute values also has a maximum.

```
lemma (in group3) OrderedGroup_ZF_4_L4:
```

```

    assumes A1: r {is total on} G and A2: A  $\subseteq$  G
    and A3: HasAmaximum(r,A) HasAminimum(r,A)
    shows HasAmaximum(r,AbsoluteValue(G,P,r)(A))
  <proof>

```

If a set has a maximum and a minimum, then all absolute values are bounded by the maximum of the set of absolute values.

```

lemma (in group3) OrderedGroup_ZF_4_L5:
  assumes A1: r {is total on} G and A2: A  $\subseteq$  G
  and A3: HasAmaximum(r,A) HasAminimum(r,A)
  and A4: a $\in$ A
  shows |a|  $\leq$  Maximum(r,AbsoluteValue(G,P,r)(A))
  <proof>

```

41.3 Alternative definitions

Sometimes it is useful to define the order by prescribing the set of positive or nonnegative elements. This section deals with two such definitions. One takes a subset H of G that is closed under the group operation, $1 \notin H$ and for every $a \in H$ we have either $a \in H$ or $a^{-1} \in H$. Then the order is defined as $a \leq b$ iff $a = b$ or $a^{-1}b \in H$. For abelian groups this makes a linearly ordered group. We will refer to order defined this way in the comments as the order defined by a positive set. The context used in this section is the `group0` context defined in `Group_ZF` theory. Recall that `f` in that context denotes the group operation (unlike in the previous sections where the group operation was denoted `P`).

The order defined by a positive set is the same as the order defined by a nonnegative set.

```

lemma (in group0) OrderedGroup_ZF_5_L1:
  assumes A1: r = {p  $\in$  G $\times$ G. fst(p) = snd(p)  $\vee$  fst(p)-1.snd(p)  $\in$  H}
  shows <a,b>  $\in$  r  $\longleftrightarrow$  a $\in$ G  $\wedge$  b $\in$ G  $\wedge$  a-1.b  $\in$  H  $\cup$  {1}
  <proof>

```

The relation defined by a positive set is antisymmetric.

```

lemma (in group0) OrderedGroup_ZF_5_L2:
  assumes A1: r = {p  $\in$  G $\times$ G. fst(p) = snd(p)  $\vee$  fst(p)-1.snd(p)  $\in$  H}
  and A2:  $\forall a \in G. a \neq 1 \longrightarrow (a \in H) \text{ Xor } (a^{-1} \in H)$ 
  shows antisym(r)
  <proof>

```

The relation defined by a positive set is transitive.

```

lemma (in group0) OrderedGroup_ZF_5_L3:
  assumes A1: r = {p  $\in$  G $\times$ G. fst(p) = snd(p)  $\vee$  fst(p)-1.snd(p)  $\in$  H}
  and A2: H $\subseteq$ G H {is closed under} P
  shows trans(r)
  <proof>

```

The relation defined by a positive set is translation invariant. With our definition this step requires the group to be abelian.

lemma (in group0) OrderedGroup_ZF_5_L4:
 assumes A1: $r = \{p \in G \times G. \text{fst}(p) = \text{snd}(p) \vee \text{fst}(p)^{-1} \cdot \text{snd}(p) \in H\}$
 and A2: $P \text{ \{is commutative on\} } G$
 and A3: $\langle a, b \rangle \in r$ and A4: $c \in G$
 shows $\langle a \cdot c, b \cdot c \rangle \in r \wedge \langle c \cdot a, c \cdot b \rangle \in r$
<proof>

If $H \subseteq G$ is closed under the group operation $1 \notin H$ and for every $a \in H$ we have either $a \in H$ or $a^{-1} \in H$, then the relation " $\leq$ " defined by $a \leq b \Leftrightarrow a^{-1}b \in H$ orders the group G . In such order H may be the set of positive or nonnegative elements.

lemma (in group0) OrderedGroup_ZF_5_L5:
 assumes A1: $P \text{ \{is commutative on\} } G$
 and A2: $H \subseteq G$ $H \text{ \{is closed under\} } P$
 and A3: $\forall a \in G. a \neq 1 \longrightarrow (a \in H) \text{ Xor } (a^{-1} \in H)$
 and A4: $r = \{p \in G \times G. \text{fst}(p) = \text{snd}(p) \vee \text{fst}(p)^{-1} \cdot \text{snd}(p) \in H\}$
 shows
 IsAnOrdGroup(G, P, r)
 $r \text{ \{is total on\} } G$
 $\text{Nonnegative}(G, P, r) = \text{PositiveSet}(G, P, r) \cup \{1\}$
<proof>

If the set defined as in OrderedGroup_ZF_5_L4 does not contain the neutral element, then it is the positive set for the resulting order.

lemma (in group0) OrderedGroup_ZF_5_L6:
 assumes $P \text{ \{is commutative on\} } G$
 and $H \subseteq G$ and $1 \notin H$
 and $r = \{p \in G \times G. \text{fst}(p) = \text{snd}(p) \vee \text{fst}(p)^{-1} \cdot \text{snd}(p) \in H\}$
 shows $\text{PositiveSet}(G, P, r) = H$
<proof>

The next definition describes how we construct an order relation from the prescribed set of positive elements.

definition
 $\text{OrderFromPosSet}(G, P, H) \equiv$
 $\{p \in G \times G. \text{fst}(p) = \text{snd}(p) \vee P(\text{GroupInv}(G, P)(\text{fst}(p)), \text{snd}(p)) \in H\}$

The next theorem rephrases lemmas OrderedGroup_ZF_5_L5 and OrderedGroup_ZF_5_L6 using the definition of the order from the positive set OrderFromPosSet. To summarize, this is what it says: Suppose that $H \subseteq G$ is a set closed under that group operation such that $1 \notin H$ and for every nonunit group element a either $a \in H$ or $a^{-1} \in H$. Define the order as $a \leq b$ iff $a = b$ or $a^{-1} \cdot b \in H$. Then this order makes G into a linearly ordered group such H is the set of positive elements (and then of course $H \cup \{1\}$ is the set of nonnegative elements).

```

theorem (in group0) Group_ord_by_positive_set:
  assumes P {is commutative on} G
  and  $H \subseteq G$  H {is closed under} P 1  $\notin$  H
  and  $\forall a \in G. a \neq 1 \longrightarrow (a \in H) \text{ Xor } (a^{-1} \in H)$ 
  shows
    IsAnOrdGroup(G,P,OrderFromPosSet(G,P,H))
    OrderFromPosSet(G,P,H) {is total on} G
    PositiveSet(G,P,OrderFromPosSet(G,P,H)) = H
    Nonnegative(G,P,OrderFromPosSet(G,P,H)) =  $H \cup \{1\}$ 
  <proof>

```

41.4 Odd Extensions

In this section we verify properties of odd extensions of functions defined on G_+ . An odd extension of a function $f : G_+ \rightarrow G$ is a function $f^\circ : G \rightarrow G$ defined by $f^\circ(x) = f(x)$ if $x \in G_+$, $f(1) = 1$ and $f^\circ(x) = (f(x^{-1}))^{-1}$ for $x < 1$. Such function is the unique odd function that is equal to f when restricted to G_+ .

The next lemma is just to see the definition of the odd extension in the notation used in the group1 context.

```

lemma (in group3) OrderedGroup_ZF_6_L1:
  shows  $f^\circ = f \cup \{(a, (f(a^{-1}))^{-1}). a \in -G_+ \cup \{1\}\}$ 
  <proof>

```

A technical lemma that states that from a function defined on G_+ with values in G we have $(f(a^{-1}))^{-1} \in G$.

```

lemma (in group3) OrderedGroup_ZF_6_L2:
  assumes  $f : G_+ \rightarrow G$  and  $a \in -G_+$ 
  shows
     $f(a^{-1}) \in G$ 
     $(f(a^{-1}))^{-1} \in G$ 
  <proof>

```

The main theorem about odd extensions. It basically says that the odd extension of a function is what we want to be.

```

lemma (in group3) odd_ext_props:
  assumes A1:  $r$  {is total on} G and A2:  $f : G_+ \rightarrow G$ 
  shows
     $f^\circ : G \rightarrow G$ 
     $\forall a \in G_+. (f^\circ)(a) = f(a)$ 
     $\forall a \in (-G_+). (f^\circ)(a) = (f(a^{-1}))^{-1}$ 
     $(f^\circ)(1) = 1$ 
  <proof>

```

Odd extensions are odd, of course.

```

lemma (in group3) oddext_is_odd:

```

```

    assumes A1: r {is total on} G and A2: f: G+→G
    and A3: a∈G
    shows (f°)(a-1) = ((f°)(a))-1
  <proof>

```

Another way of saying that odd extensions are odd.

```

lemma (in group3) oddext_is_odd_alt:
  assumes A1: r {is total on} G and A2: f: G+→G
  and A3: a∈G
  shows ((f°)(a-1))-1 = (f°)(a)
  <proof>

```

41.5 Functions with infinite limits

In this section we consider functions $f : G \rightarrow G$ with the property that for $f(x)$ is arbitrarily large for large enough x . More precisely, for every $a \in G$ there exist $b \in G_+$ such that for every $x \geq b$ we have $f(x) \geq a$. In a sense this means that $\lim_{x \rightarrow \infty} f(x) = \infty$, hence the title of this section. We also prove dual statements for functions such that $\lim_{x \rightarrow -\infty} f(x) = -\infty$.

If an image of a set by a function with infinite positive limit is bounded above, then the set itself is bounded above.

```

lemma (in group3) OrderedGroup_ZF_7_L1:
  assumes A1: r {is total on} G and A2: G ≠ {1} and
  A3: f:G→G and
  A4: ∀ a∈G. ∃ b∈G+. ∀ x. b ≤ x → a ≤ f(x) and
  A5: A ⊆ G and
  A6: IsBoundedAbove(f(A), r)
  shows IsBoundedAbove(A, r)
  <proof>

```

If an image of a set defined by separation by a function with infinite positive limit is bounded above, then the set itself is bounded above.

```

lemma (in group3) OrderedGroup_ZF_7_L2:
  assumes A1: r {is total on} G and A2: G ≠ {1} and
  A3: X ≠ 0 and A4: f:G→G and
  A5: ∀ a∈G. ∃ b∈G+. ∀ y. b ≤ y → a ≤ f(y) and
  A6: ∀ x∈X. b(x) ∈ G ∧ f(b(x)) ≤ U
  shows ∃ u. ∀ x∈X. b(x) ≤ u
  <proof>

```

If the image of a set defined by separation by a function with infinite negative limit is bounded below, then the set itself is bounded above. This is dual to OrderedGroup_ZF_7_L2.

```

lemma (in group3) OrderedGroup_ZF_7_L3:
  assumes A1: r {is total on} G and A2: G ≠ {1} and
  A3: X ≠ 0 and A4: f:G→G and

```


A5: $\forall a \in G. \exists b \in G_+. \forall y. b \leq y \longrightarrow f(y^{-1}) \leq a$ **and**
A6: $\forall x \in X. b(x) \in G \wedge L \leq f(b(x))$
shows $\exists 1. \forall x \in X. 1 \leq b(x)$
<proof>

The next lemma combines `OrderedGroup_ZF_7_L2` and `OrderedGroup_ZF_7_L3` to show that if an image of a set defined by separation by a function with infinite limits is bounded, then the set itself is bounded.

lemma (in `group3`) `OrderedGroup_ZF_7_L4`:
assumes **A1:** `r {is total on} G` **and** **A2:** `G  $\neq$  {1}` **and**
A3: `X  $\neq$  0` **and** **A4:** `f:G $\rightarrow$ G` **and**
A5: $\forall a \in G. \exists b \in G_+. \forall y. b \leq y \longrightarrow a \leq f(y)$ **and**
A6: $\forall a \in G. \exists b \in G_+. \forall y. b \leq y \longrightarrow f(y^{-1}) \leq a$ **and**
A7: $\forall x \in X. b(x) \in G \wedge L \leq f(b(x)) \wedge f(b(x)) \leq U$
shows $\exists M. \forall x \in X. |b(x)| \leq M$
<proof>

end

42 Rings - introduction

theory `Ring_ZF` **imports** `AbelianGroup_ZF`

begin

This theory file covers basic facts about rings.

42.1 Definition and basic properties

In this section we define what is a ring and list the basic properties of rings.

We say that three sets (R, A, M) form a ring if (R, A) is an abelian group, (R, M) is a monoid and A is distributive with respect to M on R . A represents the additive operation on R . As such it is a subset of $(R \times R) \times R$ (recall that in ZF set theory functions are sets). Similarly M represents the multiplicative operation on R and is also a subset of $(R \times R) \times R$. We don't require the multiplicative operation to be commutative in the definition of a ring.

definition

$\text{IsAring}(R, A, M) \equiv \text{IsAgroup}(R, A) \wedge (A \text{ \{is commutative on\} } R) \wedge$
 $\text{IsAmonoid}(R, M) \wedge \text{IsDistributive}(R, A, M)$

We also define the notion of having no zero divisors. In standard notation the ring has no zero divisors if for all $a, b \in R$ we have $a \cdot b = 0$ implies $a = 0$ or $b = 0$.

definition

```

HasNoZeroDivs(R,A,M)  $\equiv$  ( $\forall a \in R. \forall b \in R.
M(a,b) = \text{TheNeutralElement}(R,A) \longrightarrow
a = \text{TheNeutralElement}(R,A) \vee b = \text{TheNeutralElement}(R,A)$ )

```

Next we define a locale that will be used when considering rings.

```

locale ring0 =

```

```

  fixes R and A and M

```

```

  assumes ringAssum: IsARing(R,A,M)

```

```

  fixes ringa (infixl + 90)
  defines ringa_def [simp]:  $x+y \equiv A(x,y)$ 

```

```

  fixes ringminus (- _ 89)
  defines ringminus_def [simp]:  $(-x) \equiv \text{GroupInv}(R,A)(x)$ 

```

```

  fixes ringsub (infixl - 90)
  defines ringsub_def [simp]:  $x-y \equiv x+(-y)$ 

```

```

  fixes ringm (infixl · 95)
  defines ringm_def [simp]:  $x \cdot y \equiv M(x,y)$ 

```

```

  fixes ringzero (0)
  defines ringzero_def [simp]:  $0 \equiv \text{TheNeutralElement}(R,A)$ 

```

```

  fixes ringone (1)
  defines ringone_def [simp]:  $1 \equiv \text{TheNeutralElement}(R,M)$ 

```

```

  fixes ringtwo (2)
  defines ringtwo_def [simp]:  $2 \equiv 1+1$ 

```

```

  fixes ringsq (_2 [96] 97)
  defines ringsq_def [simp]:  $x^2 \equiv x \cdot x$ 

```

```

  fixes rlistsum ( $\sum$  _ 70)
  defines rlistsum_def [simp]:  $\sum s \equiv \text{Fold}(A,0,s)$ 

```

```

  fixes rnat_mult (infix · 95)
  defines nat_mult_def [simp]:  $n \cdot x \equiv \sum \{ \langle k,x \rangle. k \in n \}$ 

```

```

  fixes listprod ( $\prod$  _ 70)
  defines listprod_def [simp]:  $\prod s \equiv \text{Fold}(M,1,s)$ 

```

```

  fixes pow
  defines pow_def [simp]:  $\text{pow}(n,x) \equiv \prod \{ \langle k,x \rangle. k \in n \}$ 

```

In the ring0 context we can use theorems proven in some other contexts.

```

lemma (in ring0) Ring_ZF_1_L1: shows

```

```

monoid0(R,M)
monoid1(R,M)
group0(R,A)
A {is commutative on} R
<proof>

```

The theorems proven in in `group0` context (locale) are valid in the `ring0` context when applied to the additive group of the ring.

```

sublocale ring0 < add_group: group0 R A ringzero ringa ringminus rlistsum
rnat_mult
<proof>

```

The theorems proven in the `monoid0` context are valid in the `ring0` context when applied to the multiplicative monoid of the ring.

```

sublocale ring0 < mult_monoid: monoid1 R M ringm ringone listprod pow
<proof>

```

The additive operation in a ring is distributive with respect to the multiplicative operation.

```

lemma (in ring0) ring_oper_distr: assumes A1: a∈R  b∈R  c∈R
  shows
    a·(b+c) = a·b + a·c
    (b+c)·a = b·a + c·a
<proof>

```

Zero and one of the ring are elements of the ring. The negative of zero is zero.

```

lemma (in ring0) Ring_ZF_1_L2:
  shows 0∈R  1∈R  (-0) = 0  2∈R
<proof>

```

The next lemma lists some properties of a ring that require one element of a ring.

```

lemma (in ring0) Ring_ZF_1_L3: assumes a∈R
  shows
    (-a) ∈ R
    (-(-a)) = a
    a+0 = a
    0+a = a
    a·1 = a
    1·a = a
    a-a = 0
    a-0 = a
    2·a = a+a
    (-a)+a = 0
<proof>

```

Properties that require two elements of a ring.

```

lemma (in ring0) Ring_ZF_1_L4: assumes A1:  $a \in R$   $b \in R$ 
shows
 $a+b \in R$ 
 $a-b \in R$ 
 $a \cdot b \in R$ 
 $a+b = b+a$ 
 $\langle proof \rangle$ 

```

Cancellation of an element on both sides of equality. This is a property of groups, written in the (additive) notation we use for the additive operation in rings.

```

lemma (in ring0) ring_cancel_add:
assumes A1:  $a \in R$   $b \in R$  and A2:  $a + b = a$ 
shows  $b = 0$ 
 $\langle proof \rangle$ 

```

Any element of a ring multiplied by zero is zero.

```

lemma (in ring0) Ring_ZF_1_L6:
assumes A1:  $x \in R$  shows  $0 \cdot x = 0$   $x \cdot 0 = 0$ 
 $\langle proof \rangle$ 

```

Negative can be pulled out of a product.

```

lemma (in ring0) Ring_ZF_1_L7:
assumes A1:  $a \in R$   $b \in R$ 
shows
 $(-a) \cdot b = -(a \cdot b)$ 
 $a \cdot (-b) = -(a \cdot b)$ 
 $(-a) \cdot b = a \cdot (-b)$ 
 $\langle proof \rangle$ 

```

Minus times minus is plus.

```

lemma (in ring0) Ring_ZF_1_L7A: assumes  $a \in R$   $b \in R$ 
shows  $(-a) \cdot (-b) = a \cdot b$ 
 $\langle proof \rangle$ 

```

Subtraction is distributive with respect to multiplication.

```

lemma (in ring0) Ring_ZF_1_L8: assumes  $a \in R$   $b \in R$   $c \in R$ 
shows
 $a \cdot (b-c) = a \cdot b - a \cdot c$ 
 $(b-c) \cdot a = b \cdot a - c \cdot a$ 
 $\langle proof \rangle$ 

```

Other basic properties involving two elements of a ring.

```

lemma (in ring0) Ring_ZF_1_L9: assumes  $a \in R$   $b \in R$ 
shows
 $(-b)-a = (-a)-b$ 
 $-(a+b) = (-a)-b$ 

```

```

    
$$-(a-b) = ((-a)+b)$$

    
$$a-(-b) = a+b$$

    <proof>

```

If the difference of two element is zero, then those elements are equal.

```

lemma (in ring0) Ring_ZF_1_L9A:
  assumes A1:  $a \in R$   $b \in R$  and A2:  $a-b = 0$ 
  shows  $a=b$  <proof>

```

Other basic properties involving three elements of a ring.

```

lemma (in ring0) Ring_ZF_1_L10:
  assumes  $a \in R$   $b \in R$   $c \in R$ 
  shows
     $a+(b+c) = a+b+c$ 

     $a-(b+c) = a-b-c$ 
     $a-(b-c) = a-b+c$ 
  <proof>

```

Another property with three elements.

```

lemma (in ring0) Ring_ZF_1_L10A:
  assumes A1:  $a \in R$   $b \in R$   $c \in R$ 
  shows  $a+(b-c) = a+b-c$ 
  <proof>

```

Associativity of addition and multiplication.

```

lemma (in ring0) Ring_ZF_1_L11:
  assumes  $a \in R$   $b \in R$   $c \in R$ 
  shows
     $a+b+c = a+(b+c)$ 
     $a \cdot b \cdot c = a \cdot (b \cdot c)$ 
  <proof>

```

An interpretation of what it means that a ring has no zero divisors.

```

lemma (in ring0) Ring_ZF_1_L12:
  assumes HasNoZeroDivs(R,A,M)
  and  $a \in R$   $a \neq 0$   $b \in R$   $b \neq 0$ 
  shows  $a \cdot b \neq 0$ 
  <proof>

```

If ring has no zero divisors then the set of nonzero elements is closed with respect to multiplication, hence with respect to multiplication restricted to that set.

```

lemma (in ring0) nozerodivs_nonzero_closed:
  assumes HasNoZeroDivs(R,A,M)
  shows  $(R \setminus \{0\})$  {is closed under} M and
     $(R \setminus \{0\})$  {is closed under} restrict(M,  $(R \setminus \{0\}) \times (R \setminus \{0\})$ )

```

<proof>

In rings with no zero divisors we can cancel nonzero factors.

lemma (in ring0) Ring_ZF_1_L12A:
 assumes A1: HasNoZeroDivs(R,A,M) and A2: $a \in R$ $b \in R$ $c \in R$
 and A3: $a \cdot c = b \cdot c$ and A4: $c \neq 0$
 shows $a = b$
<proof>

In rings with no zero divisors if two elements are different, then after multiplying by a nonzero element they are still different.

lemma (in ring0) Ring_ZF_1_L12B:
 assumes A1: HasNoZeroDivs(R,A,M)
 $a \in R$ $b \in R$ $c \in R$ $a \neq b$ $c \neq 0$
 shows $a \cdot c \neq b \cdot c$
<proof>

In rings with no zero divisors multiplying a nonzero element by a nonzero element changes the value.

lemma (in ring0) Ring_ZF_1_L12C:
 assumes A1: HasNoZeroDivs(R,A,M) and
 A2: $a \in R$ $b \in R$ and A3: $0 \neq a$ $1 \neq b$
 shows $a \neq a \cdot b$
<proof>

If a square is nonzero, then the element is nonzero.

lemma (in ring0) Ring_ZF_1_L13:
 assumes $a \in R$ and $a^2 \neq 0$
 shows $a \neq 0$
<proof>

Square of an element and its opposite are the same.

lemma (in ring0) Ring_ZF_1_L14:
 assumes $a \in R$ shows $(-a)^2 = (a^2)$
<proof>

Adding zero to a set that is closed under addition results in a set that is also closed under addition. This is a property of groups.

lemma (in ring0) Ring_ZF_1_L15:
 assumes $H \subseteq R$ and H {is closed under} A
 shows $(H \cup \{0\})$ {is closed under} A
<proof>

Adding zero to a set that is closed under multiplication results in a set that is also closed under multiplication.

lemma (in ring0) Ring_ZF_1_L16:
 assumes A1: $H \subseteq R$ and A2: H {is closed under} M

shows $(H \cup \{0\})$ {is closed under} M
 $\langle proof \rangle$

The ring is trivial iff $0 = 1$.

lemma (in ring0) Ring_ZF_1_L17: **shows** $R = \{0\} \longleftrightarrow 0=1$
 $\langle proof \rangle$

The sets $\{m \cdot x : x \in R\}$ and $\{-m \cdot x : x \in R\}$ are the same.

lemma (in ring0) Ring_ZF_1_L18: **assumes** $A1: m \in R$
shows $\{m \cdot x. x \in R\} = \{(-m) \cdot x. x \in R\}$
 $\langle proof \rangle$

42.2 Rearrangement lemmas

It happens quite often that we want to show a fact like $(a + b)c + d = (ac + d - e) + (bc + e)$ in rings. This is trivial in romantic math and probably there is a way to make it trivial in formalized math. However, I don't know any other way than to tediously prove each such rearrangement when it is needed. This section collects facts of this type.

Rearrangements with two elements of a ring.

lemma (in ring0) Ring_ZF_2_L1: **assumes** $a \in R \ b \in R$
shows $a + b \cdot a = (b + 1) \cdot a$
 $\langle proof \rangle$

Rearrangements with two elements and cancelling.

lemma (in ring0) Ring_ZF_2_L1A: **assumes** $a \in R \ b \in R$
shows
 $a - b + b = a$
 $a + b - a = b$
 $(-a) + b + a = b$
 $(-a) + (b + a) = b$
 $a + (b - a) = b$
 $\langle proof \rangle$

In rings $a - (b + 1)c = (a - d - c) + (d - bc)$ and $a + b + (c + d) = a + (b + c) + d$.

lemma (in ring0) Ring_ZF_2_L2:
assumes $a \in R \ b \in R \ c \in R \ d \in R$
shows
 $a - (b + 1) \cdot c = (a - d - c) + (d - b \cdot c)$
 $a + b + (c + d) = a + b + c + d$
 $a + b + (c + d) = a + (b + c) + d$
 $\langle proof \rangle$

Rearrangement about adding linear functions.

lemma (in ring0) Ring_ZF_2_L3:
assumes $A1: a \in R \ b \in R \ c \in R \ d \in R \ x \in R$

```

    shows (a·x + b) + (c·x + d) = (a+c)·x + (b+d)
  <proof>

```

Rearrangement with three elements

```

lemma (in ring0) Ring_ZF_2_L4:
  assumes M {is commutative on} R
  and a∈R b∈R c∈R
  shows a·(b·c) = a·c·b and a·b·c = a·c·b
  <proof>

```

Some other rearrangements with three elements.

```

lemma (in ring0) ring_rearr_3_elemA:
  assumes A1: M {is commutative on} R and
  A2: a∈R b∈R c∈R
  shows
    a·(a·c) - b·(-b·c) = (a·a + b·b)·c
    a·(-b·c) + b·(a·c) = 0
  <proof>

```

Some rearrangements with four elements. Properties of abelian groups.

```

lemma (in ring0) Ring_ZF_2_L5:
  assumes a∈R b∈R c∈R d∈R
  shows
    a - b - c - d = a - d - b - c
    a + b + c - d = a - d + b + c
    a + b - c - d = a - c + (b - d)
    a + b + c + d = a + c + (b + d)
  <proof>

```

Two big rearrangements with six elements, useful for proving properties of complex addition and multiplication.

```

lemma (in ring0) Ring_ZF_2_L6:
  assumes A1: a∈R b∈R c∈R d∈R e∈R f∈R
  shows
    a·(c·e - d·f) - b·(c·f + d·e) =
      (a·c - b·d)·e - (a·d + b·c)·f
    a·(c·f + d·e) + b·(c·e - d·f) =
      (a·c - b·d)·f + (a·d + b·c)·e
    a·(c+e) - b·(d+f) = a·c - b·d + (a·e - b·f)
    a·(d+f) + b·(c+e) = a·d + b·c + (a·f + b·e)
  <proof>

```

end

43 Binomial theorem

```

theory Ring_Binomial_ZF imports Monoid_ZF_1 Ring_ZF

```


begin

This theory aims at formalizing sufficient background to be able to state and prove the binomial theorem.

43.1 Sums of multiplicities of powers of ring elements and binomial theorem

The binomial theorem asserts that for any two elements of a commutative ring the n -th power of the sum $x + y$ can be written as a sum of certain multiplicities of terms $x^{n-k}y^k$, where $k \in 0..n$. In this section we setup the notation and prove basic properties of such multiplicities and powers of ring elements. We show the binomial theorem as an application.

A ring with addition forms a monoid, hence all propositions proven in the `monoid1` locale (defined in the `Monoid_ZF_1` theory) can be used in the `ring0` locale, applied to the additive operation.

sublocale `ring0` < `add_monoid`: `monoid1` `R` `A` `ringa` `ringzero` `rlistsum` `rnat_mult`

<proof>

The assumptions of the `monoid1` context hold in the `ring0` context

lemma (in `ring0`) `monoid0_valid_in_ring0`: **shows** `monoid1`(`R`,`A`)
<proof>

$0 \cdot x = 0$ and $x^0 = 1$. It is a bit surprising that we do not need to assume that $x \in R$ (i.e. x is an element of the ring). These properties are really proven in the `Monoid_ZF_1` theory where there is no assumption that x is an element of the monoid.

lemma (in `ring0`) `mult_pow_zero`: **shows** $0 \cdot x = 0$ and $\text{pow}(0, x) = 1$
<proof>

Natural multiple and power of a ring element is a ring element.

lemma (in `ring0`) `mult_pow_type`: **assumes** $n \in \text{nat}$ $x \in R$
shows $n \cdot x \in R$ and $\text{pow}(n, x) \in R$
<proof>

The usual properties of multiples and powers: $(n+1)x = nx + x$ and $x^n + 1 = x^n x$. These are just versions of `nat_mult_add_one` from `Monoid_ZF_1` writtent in the notation defined in the `ring0` locale.

lemma (in `ring0`) `nat_mult_pow_add_one`: **assumes** $n \in \text{nat}$ $x \in R$
shows $(n \#+ 1) \cdot x = (n \cdot x) + x$ and $\text{pow}(n \#+ 1, x) = \text{pow}(n, x) \cdot x$
<proof>

Associativity for the multiplication by natural number and the ring multiplication:

lemma (in ring0) nat_mult_assoc: assumes $n \in \text{nat}$ $x \in R$ $y \in R$
 shows $n \cdot x \cdot y = n \cdot (x \cdot y)$
<proof>

Addition of natural numbers is distributive with respect to natural multiple. This is essentially lemma nat_mult_add from Monoid_ZF_1.thy, just transferred to the ring0 locale.

lemma (in ring0) nat_add_mult_distrib: assumes $n \in \text{nat}$ $m \in \text{nat}$ $x \in R$
 shows $(n \#+ m) \cdot x = n \cdot x + m \cdot x$
<proof>

Associativity for the multiplication by natural number and the ring multiplication extended to three elements of the ring:

lemma (in ring0) nat_mult_assoc1: assumes $n \in \text{nat}$ $x \in R$ $y \in R$ $z \in R$
 shows $n \cdot x \cdot y \cdot z = n \cdot (x \cdot y \cdot z)$
<proof>

When we multiply an expression whose value belongs to a ring by a ring element and we get an expression whose value belongs to a ring.

lemma (in ring0) mult_elem_ring_type:
 assumes $n \in \text{nat}$ $x \in R$ and $\forall k \in n. q(k) \in R$
 shows $\forall k \in n. q(k) \cdot x \in R$ and $(\sum \{ \langle k, q(k) \cdot x \rangle. k \in n \}) \in R$
<proof>

The sum of expressions whose values belong to a ring is an expression whose value belongs to a ring.

lemma (in ring0) sum_expr_ring_type:
 assumes $n \in \text{nat}$ $\forall k \in n. q(k) \in R$ $\forall k \in n. p(k) \in R$
 shows $\forall k \in n. q(k) + p(k) \in R$ and $(\sum \{ \langle k, q(k) + p(k) \rangle. k \in n \}) \in R$
<proof>

Combining mult_elem_ring_type and sum_expr_ring_type we obtain that a (kind of) linear combination of expressions whose values belong to a ring belongs to the ring.

lemma (in ring0) lin_comb_expr_ring_type:
 assumes $n \in \text{nat}$ $x \in R$ $y \in R$ $\forall k \in n. q(k) \in R$ $\forall k \in n. p(k) \in R$
 shows $\forall k \in n. q(k) \cdot x + p(k) \cdot y \in R$ and
 $(\sum \{ \langle k, q(k) \cdot x + p(k) \cdot y \rangle. k \in n \}) \in R$
<proof>

A ring0 version of seq_sum_pull_one_elem from Monoid_ZF_1:

lemma (in ring0) rng_seq_sum_pull_one_elem:
 assumes $j \in \text{nat}$ $\forall k \in j \#+ 1. q(k) \in R$
 shows
 $(\sum \{ \langle k, q(k) \rangle. k \in j \#+ 1 \}) = q(0) + (\sum \{ \langle k, q(k \#+ 1) \rangle. k \in j \})$
 $(\sum \{ \langle k, q(k) \rangle. k \in j \#+ 1 \}) = (\sum \{ \langle k, q(k) \rangle. k \in j \}) + q(j)$
<proof>

Distributive laws for finite sums in a ring: $(\sum_{k=0}^{n-1} q(k)) \cdot x = \sum_{k=0}^{n-1} q(k) \cdot x$ and $x \cdot (\sum_{k=0}^{n-1} q(k)) = \sum_{k=0}^{n-1} x \cdot q(k)$.

theorem (in ring0) fin_sum_distrib:

assumes $x \in R \quad n \in \text{nat} \quad \forall k \in n. q(k) \in R$

shows

$$\begin{aligned} (\sum \{ \langle k, q(k) \rangle . k \in n \}) \cdot x &= \sum \{ \langle k, q(k) \cdot x \rangle . k \in n \} \\ x \cdot (\sum \{ \langle k, q(k) \rangle . k \in n \}) &= \sum \{ \langle k, x \cdot q(k) \rangle . k \in n \} \end{aligned}$$

<proof>

In rings we have $\sum_{k=0}^{n-1} q(k) + p(k) = (\sum_{k=0}^{n-1} p(k)) + (\sum_{k=0}^{n-1} q(k))$. This is the same as theorem sum_comm_distrib in Monoid_ZF_1.thy, except that we do not need the assumption about commutativity of the operation as addition in rings is always commutative.

lemma (in ring0) sum_ring_distrib:

assumes $n \in \text{nat}$ **and** $\forall k \in n. p(k) \in R \quad \forall k \in n. q(k) \in R$

shows

$$(\sum \{ \langle k, p(k) + q(k) \rangle . k \in n \}) = (\sum \{ \langle k, p(k) \rangle . k \in n \}) + (\sum \{ \langle k, q(k) \rangle . k \in n \})$$

<proof>

To shorten the notation in the proof of the binomial theorem we give a name to the binomial term $\binom{n}{k} x^{n-k} y^k$.

definition (in ring0) BT where

$$\text{BT}(n, k, x, y) \equiv \text{Binom}(n, k) \cdot \text{pow}(n \# - k, x) \cdot \text{pow}(k, y)$$

If n, k are natural numbers and x, y are ring elements then the binomial term is an element of the ring.

lemma (in ring0) bt_type: **assumes** $n \in \text{nat} \quad k \in \text{nat} \quad x \in R \quad y \in R$

shows $\text{BT}(n, k, x, y) \in R$

<proof>

The binomial term is 1 when the $n = 0$ and $k = 0$. Somehow we do not need the assumption that x, y are ring elements.

lemma (in ring0) bt_at_zero: **shows** $\text{BT}(0, 0, x, y) = 1$

<proof>

The binomial term is x^n when $k = 0$.

lemma (in ring0) bt_at_zero1: **assumes** $n \in \text{nat} \quad x \in R$

shows $\text{BT}(n, 0, x, y) = \text{pow}(n, x)$

<proof>

When $k = 0$ multiplying the binomial term by x is the same as adding one to n .

lemma (in ring0) bt_at_zero2: **assumes** $n \in \text{nat} \quad x \in R$

shows $\text{BT}(n, 0, x, y) \cdot x = \text{BT}(n \# + 1, 0, x, y)$

<proof>

The binomial term is y^n when $k = n$.

```

lemma (in ring0) bt_at_right: assumes n∈nat y∈R
  shows BT(n,n,x,y) = pow(n,y)
  ⟨proof⟩

```

When $k = n$ multiplying the binomial term by x is the same as adding one to n .

```

lemma (in ring0) bt_at_right1: assumes n∈nat y∈R
  shows BT(n,n,x,y)·y = BT(n #+ 1,n #+ 1,x,y)
  ⟨proof⟩

```

A key identity for binomial terms needed for the proof of the binomial theorem:

```

lemma (in ring0) bt_rec_identity:
  assumes M {is commutative on} R j∈nat k∈j x∈R y∈R
  shows
    BT(j,k #+ 1,x,y)·x + BT(j,k,x,y)·y = BT(j #+ 1,k #+ 1,x,y)
  ⟨proof⟩

```

The binomial theorem: if x, y are elements of a commutative ring, $n \in \mathbb{N}$ then $(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k$.

```

theorem (in ring0) binomial_theorem:
  assumes M {is commutative on} R n∈nat x∈R y∈R
  shows
    pow(n,x+y) =  $\sum \{ \langle k, \text{Binom}(n,k) \cdot \text{pow}(n - k, x) \cdot \text{pow}(k, y) \rangle . k \in n \# + 1 \}$ 
  ⟨proof⟩

```

end

44 Ring of almost homomorphisms the ring of integers as a quotient

```

theory Ring_ZF_1 imports Ring_ZF Group_ZF_3

```

```

begin

```

This theory is devoted to the part of ring theory specific the construction of real numbers in the `Real_ZF_x` series of theories. The goal is to show that classes of almost homomorphisms form a ring.

44.1 The ring of classes of almost homomorphisms

Almost homomorphisms do not form a ring as the regular homomorphisms do because the lifted group operation is not distributive with respect to composition – we have $s \circ (r \cdot q) \neq s \circ r \cdot s \circ q$ in general. However, we do have $s \circ (r \cdot q) \approx s \circ r \cdot s \circ q$ in the sense of the equivalence relation defined by the group of finite range functions (that is a normal subgroup of almost

homomorphisms, if the group is abelian). This allows to define a natural ring structure on the classes of almost homomorphisms.

The next lemma provides a formula useful for proving that two sides of the distributive law equation for almost homomorphisms are almost equal.

```
lemma (in group1) Ring_ZF_1_1_L1:
  assumes A1: s∈AH r∈AH q∈AH and A2: n∈G
  shows
    ((s◦(r·q))(n))·(((s◦r)·(s◦q))(n))-1 = δ(s,⟨ r(n),q(n)⟩)
    ((r·q)◦s)(n) = ((r◦s)·(q◦s))(n)
⟨proof⟩
```

The sides of the distributive law equations for almost homomorphisms are almost equal.

```
lemma (in group1) Ring_ZF_1_1_L2:
  assumes A1: s∈AH r∈AH q∈AH
  shows
    s◦(r·q) ≅ (s◦r)·(s◦q)
    (r·q)◦s = (r◦s)·(q◦s)
⟨proof⟩
```

The essential condition to show the distributivity for the operations defined on classes of almost homomorphisms.

```
lemma (in group1) Ring_ZF_1_1_L3:
  assumes A1: R = QuotientGroupRel(AH,Op1,FR)
  and A2: a ∈ AH//R b ∈ AH//R c ∈ AH//R
  and A3: A = ProjFun2(AH,R,Op1) M = ProjFun2(AH,R,Op2)
  shows M⟨a,A⟨ b,c⟩⟩ = A⟨M⟨ a,b⟩,M⟨ a,c⟩⟩ ∧
    M⟨A⟨ b,c⟩,a⟩ = A⟨M⟨ b,a⟩,M⟨ c,a⟩⟩
⟨proof⟩
```

The projection of the first group operation on almost homomorphisms is distributive with respect to the second group operation.

```
lemma (in group1) Ring_ZF_1_1_L4:
  assumes A1: R = QuotientGroupRel(AH,Op1,FR)
  and A2: A = ProjFun2(AH,R,Op1) M = ProjFun2(AH,R,Op2)
  shows IsDistributive(AH//R,A,M)
⟨proof⟩
```

The classes of almost homomorphisms form a ring.

```
theorem (in group1) Ring_ZF_1_1_T1:
  assumes R = QuotientGroupRel(AH,Op1,FR)
  and A = ProjFun2(AH,R,Op1) M = ProjFun2(AH,R,Op2)
  shows IsAring(AH//R,A,M)
⟨proof⟩
```

end

45 Ordered rings

```
theory OrderedRing_ZF imports Ring_ZF OrderedGroup_ZF_1
```

```
begin
```

In this theory file we consider ordered rings.

45.1 Definition and notation

This section defines ordered rings and sets up appropriate notation.

We define ordered ring as a commutative ring with linear order that is preserved by translations and such that the set of nonnegative elements is closed under multiplication. Note that this definition does not guarantee that there are no zero divisors in the ring.

definition

```
IsAnOrdRing(R,A,M,r)  $\equiv$   
( IsAring(R,A,M)  $\wedge$  (M {is commutative on} R)  $\wedge$   
r $\subseteq$ R $\times$ R  $\wedge$  IsLinOrder(R,r)  $\wedge$   
( $\forall a\ b. \forall c\in R. \langle a,b\rangle \in r \longrightarrow \langle A\langle a,c\rangle, A\langle b,c\rangle\rangle \in r$ )  $\wedge$   
(Nonnegative(R,A,r) {is closed under} M))
```

The next context (locale) defines notation used for ordered rings. We do that by extending the notation defined in the `ring0` locale and adding some assumptions to make sure we are talking about ordered rings in this context.

```
locale ring1 = ring0 +
```

```
  assumes mult_commut: M {is commutative on} R
```

```
  fixes r
```

```
  assumes ordincl: r  $\subseteq$  R $\times$ R
```

```
  assumes linord: IsLinOrder(R,r)
```

```
  fixes lesseq (infix  $\leq$  68)
```

```
  defines lesseq_def [simp]: a  $\leq$  b  $\equiv \langle a,b\rangle \in r$ 
```

```
  fixes sless (infix  $<$  68)
```

```
  defines sless_def [simp]: a  $<$  b  $\equiv a\leq b \wedge a\neq b$ 
```

```
  assumes ordgroup:  $\forall a\ b. \forall c\in R. a\leq b \longrightarrow a+c \leq b+c$ 
```

```
  assumes pos_mult_closed: Nonnegative(R,A,r) {is closed under} M
```

```
  fixes abs (| _ |)
```

```
  defines abs_def [simp]: |a|  $\equiv$  AbsoluteValue(R,A,r)(a)
```

```

fixes positiveset (R+)
defines positiveset_def [simp]: R+ ≡ PositiveSet(R,A,r)

```

The next lemma assures us that we are talking about ordered rings in the ring1 context.

```

lemma (in ring1) OrdRing_ZF_1_L1: shows IsAnOrdRing(R,A,M,r)
  ⟨proof⟩

```

We can use theorems proven in the ring1 context whenever we talk about an ordered ring.

```

lemma OrdRing_ZF_1_L2: assumes IsAnOrdRing(R,A,M,r)
  shows ring1(R,A,M,r)
  ⟨proof⟩

```

In the ring1 context $a \leq b$ implies that a, b are elements of the ring.

```

lemma (in ring1) OrdRing_ZF_1_L3: assumes a≤b
  shows a∈R b∈R
  ⟨proof⟩

```

In the ring1 context $a < b$ implies that a, b are elements of the ring.

```

lemma (in ring1) ord_ring_less_members: assumes a<b
  shows a∈R b∈R
  ⟨proof⟩

```

Ordered ring is an ordered group, hence we can use theorems proven in the group3 context.

```

lemma (in ring1) OrdRing_ZF_1_L4: shows
  IsAnOrdGroup(R,A,r)
  r {is total on} R
  A {is commutative on} R
  group3(R,A,r)
  ⟨proof⟩

```

We can express that x is positive by stating that $0 < x$ or by writing that x is an element R_+ .

```

lemma (in ring1) element_pos: shows a∈R+ ↔ 0<a
  ⟨proof⟩

```

The order relation in rings is transitive.

```

lemma (in ring1) ring_ord_transitive: assumes A1: a≤b b≤c
  shows a≤c
  ⟨proof⟩

```

Transitivity for the strict order: if $a < b$ and $b \leq c$, then $a < c$. Property of ordered groups.

```

lemma (in ring1) ring_strict_ord_trans:

```

```

    assumes A1:  $a < b$  and A2:  $b \leq c$ 
    shows  $a < c$ 
  <proof>

```

Another version of transitivity for the strict order: if $a \leq b$ and $b < c$, then $a < c$. Property of ordered groups.

```

lemma (in ring1) ring_strict_ord_transit:
  assumes A1:  $a \leq b$  and A2:  $b < c$ 
  shows  $a < c$ 
  <proof>

```

The ring order is reflexive.

```

lemma (in ring1) ring_ord_refl: assumes  $a \in R$  shows  $a \leq a$ 
  <proof>

```

The next lemma shows what happens when one element of an ordered ring is not greater or equal than another.

```

lemma (in ring1) OrdRing_ZF_1_L4A: assumes A1:  $a \in R$   $b \in R$ 
  and A2:  $\neg(a \leq b)$ 
  shows  $b \leq a$   $(-a) \leq (-b)$   $a \neq b$ 
  <proof>

```

A special case of OrdRing_ZF_1_L4A when one of the constants is 0. This is useful for many proofs by cases.

```

corollary (in ring1) ord_ring_split2: assumes A1:  $a \in R$ 
  shows  $a \leq 0 \vee (0 \leq a \wedge a \neq 0)$ 
  <proof>

```

Taking minus on both sides reverses an inequality.

```

lemma (in ring1) OrdRing_ZF_1_L4B: assumes  $a \leq b$ 
  shows  $(-b) \leq (-a)$ 
  <proof>

```

The next lemma just expands the condition that requires the set of non-negative elements to be closed with respect to multiplication. These are properties of totally ordered groups.

```

lemma (in ring1) OrdRing_ZF_1_L5:
  assumes  $0 \leq a$   $0 \leq b$ 
  shows  $0 \leq a \cdot b$ 
  <proof>

```

Double nonnegative is nonnegative.

```

lemma (in ring1) OrdRing_ZF_1_L5A: assumes A1:  $0 \leq a$ 
  shows  $0 \leq 2 \cdot a$ 
  <proof>

```

A sufficient (somewhat redundant) condition for a structure to be an ordered ring. It says that a commutative ring that is a totally ordered group with

respect to the additive operation such that set of nonnegative elements is closed under multiplication, is an ordered ring.

```
lemma OrdRing_ZF_1_L6:
  assumes
    IsAring(R,A,M)
    M {is commutative on} R
    Nonnegative(R,A,r) {is closed under} M
    IsAnOrdGroup(R,A,r)
    r {is total on} R
  shows IsAnOrdRing(R,A,M,r)
  <proof>
```

$a \leq b$ iff $a - b \leq 0$. This is a fact from `OrderedGroup.thy`, where it is stated in multiplicative notation.

```
lemma (in ring1) OrdRing_ZF_1_L7:
  assumes a∈R b∈R
  shows a≤b ⟷ a-b ≤ 0
  <proof>
```

Negative times positive is negative.

```
lemma (in ring1) OrdRing_ZF_1_L8:
  assumes A1: a≤0 and A2: 0≤b
  shows a·b ≤ 0
  <proof>
```

We can multiply both sides of an inequality by a nonnegative ring element. This property is sometimes (not here) used to define ordered rings.

```
lemma (in ring1) OrdRing_ZF_1_L9:
  assumes A1: a≤b and A2: 0≤c
  shows
    a·c ≤ b·c
    c·a ≤ c·b
  <proof>
```

A special case of `OrdRing_ZF_1_L9`: we can multiply an inequality by a positive ring element.

```
lemma (in ring1) OrdRing_ZF_1_L9A:
  assumes A1: a≤b and A2: c∈R+
  shows
    a·c ≤ b·c
    c·a ≤ c·b
  <proof>
```

A square is nonnegative.

```
lemma (in ring1) OrdRing_ZF_1_L10:
  assumes A1: a∈R shows 0≤(a2)
  <proof>
```

1 is nonnegative.

corollary (in ring1) ordRing_one_is_nonneg: shows $0 \leq 1$
<proof>

In nontrivial rings one is positive.

lemma (in ring1) ordRing_one_is_pos: assumes $0 \neq 1$
shows $1 \in R_+$ $0 < 1$
<proof>

Nonnegative is not negative. Property of ordered groups.

lemma (in ring1) OrdRing_ZF_1_L11: assumes $0 \leq a$
shows $\neg(a \leq 0 \wedge a \neq 0)$
<proof>

A negative element cannot be a square.

lemma (in ring1) OrdRing_ZF_1_L12:
assumes A1: $a \leq 0$ $a \neq 0$
shows $\neg(\exists b \in R. a = (b^2))$
<proof>

If $a \leq b$, then $0 \leq b - a$.

lemma (in ring1) OrdRing_ZF_1_L13: assumes $a \leq b$
shows $0 \leq b - a$
<proof>

If $a < b$, then $0 < b - a$.

lemma (in ring1) OrdRing_ZF_1_L14: assumes $a \leq b$ $a \neq b$
shows
 $0 \leq b - a$ $0 \neq b - a$
 $b - a \in R_+$
<proof>

If the difference is nonnegative, then $a \leq b$.

lemma (in ring1) OrdRing_ZF_1_L15:
assumes $a \in R$ $b \in R$ and $0 \leq b - a$
shows $a \leq b$
<proof>

A nonnegative number is does not decrease when multiplied by a number greater or equal 1.

lemma (in ring1) OrdRing_ZF_1_L16:
assumes A1: $0 \leq a$ and A2: $1 \leq b$
shows $a \leq a \cdot b$
<proof>

We can multiply the right hand side of an inequality between nonnegative ring elements by an element greater or equal 1.

```

lemma (in ring1) OrdRing_ZF_1_L17:
  assumes A1:  $0 \leq a$  and A2:  $a \leq b$  and A3:  $1 \leq c$ 
  shows  $a \leq b \cdot c$ 
  <proof>

```

Strict order is preserved by translations.

```

lemma (in ring1) ring_strict_ord_trans_inv:
  assumes  $a < b$  and  $c \in R$ 
  shows
     $a + c < b + c$ 
     $c + a < c + b$ 
  <proof>

```

We can put an element on the other side of a strict inequality, changing its sign.

```

lemma (in ring1) OrdRing_ZF_1_L18:
  assumes  $a \in R$   $b \in R$  and  $a - b < c$ 
  shows  $a < c + b$ 
  <proof>

```

We can add the sides of two inequalities, the first of them strict, and we get a strict inequality. Property of ordered groups.

```

lemma (in ring1) OrdRing_ZF_1_L19:
  assumes  $a < b$  and  $c \leq d$ 
  shows  $a + c < b + d$ 
  <proof>

```

We can add the sides of two inequalities, the second of them strict and we get a strict inequality. Property of ordered groups.

```

lemma (in ring1) OrdRing_ZF_1_L20:
  assumes  $a \leq b$  and  $c < d$ 
  shows  $a + c < b + d$ 
  <proof>

```

In a non-trivial ring one is less than two.

```

lemma (in ring1) one_less_two: assumes  $0 \neq 1$  shows  $1 < 2$ 
  <proof>

```

In a non-trivial ring two is positive.

```

lemma (in ring1) two_positive: assumes  $0 \neq 1$  shows  $2 \in R_+$   $0 < 2$ 
  <proof>

```

45.2 Absolute value for ordered rings

Absolute value is defined for ordered groups as a function that is the identity on the nonnegative set and the negative of the element (the inverse in the

multiplicative notation) on the rest. In this section we consider properties of absolute value related to multiplication in ordered rings.

Absolute value of a product is the product of absolute values: the case when both elements of the ring are nonnegative.

```
lemma (in ring1) OrdRing_ZF_2_L1:
  assumes  $0 \leq a$   $0 \leq b$ 
  shows  $|a \cdot b| = |a| \cdot |b|$ 
  <proof>
```

The absolute value of an element and its negative are the same.

```
lemma (in ring1) OrdRing_ZF_2_L2: assumes  $a \in R$ 
  shows  $|-a| = |a|$ 
  <proof>
```

The next lemma states that $|a \cdot (-b)| = |(-a) \cdot b| = |(-a) \cdot (-b)| = |a \cdot b|$.

```
lemma (in ring1) OrdRing_ZF_2_L3:
  assumes  $a \in R$   $b \in R$ 
  shows
     $|(-a) \cdot b| = |a \cdot b|$ 
     $|a \cdot (-b)| = |a \cdot b|$ 
     $|(-a) \cdot (-b)| = |a \cdot b|$ 
  <proof>
```

This lemma allows to prove theorems for the case of positive and negative elements of the ring separately.

```
lemma (in ring1) OrdRing_ZF_2_L4: assumes  $a \in R$  and  $\neg(0 \leq a)$ 
  shows  $0 \leq (-a)$   $0 \neq a$ 
  <proof>
```

Absolute value of a product is the product of absolute values.

```
lemma (in ring1) OrdRing_ZF_2_L5:
  assumes A1:  $a \in R$   $b \in R$ 
  shows  $|a \cdot b| = |a| \cdot |b|$ 
  <proof>
```

Triangle inequality. Property of linearly ordered abelian groups.

```
lemma (in ring1) ord_ring_triangle_ineq: assumes  $a \in R$   $b \in R$ 
  shows  $|a+b| \leq |a|+|b|$ 
  <proof>
```

If $a \leq c$ and $b \leq c$, then $a + b \leq 2 \cdot c$.

```
lemma (in ring1) OrdRing_ZF_2_L6:
  assumes  $a \leq c$   $b \leq c$  shows  $a+b \leq 2 \cdot c$ 
  <proof>
```

45.3 Positivity in ordered rings

This section is about properties of the set of positive elements R_+ .

The set of positive elements is a subset of the ring.

lemma (in ring1) ordRing_pos_subset: shows $R_+ \subseteq R$ and $R_+ \subseteq R \setminus \{0\}$
<proof>

The set of positive elements is closed under ring addition. This is a property of ordered groups, we just reference a theorem from `OrderedGroup_ZF` theory in the proof.

lemma (in ring1) OrdRing_ZF_3_L1: shows R_+ {is closed under} A
<proof>

Every element of a ring can be either in the positive set, equal to zero or its opposite (the additive inverse) is in the positive set. This is a property of ordered groups, we just reference a theorem from `OrderedGroup_ZF` theory.

lemma (in ring1) OrdRing_ZF_3_L2: assumes $a \in R$
 shows Exactly_1_of_3_holds ($a=0$, $a \in R_+$, $(-a) \in R_+$)
<proof>

If a ring element $a \neq 0$, and it is not positive, then $-a$ is positive.

lemma (in ring1) OrdRing_ZF_3_L2A: assumes $a \in R$ $a \neq 0$ $a \notin R_+$
 shows $(-a) \in R_+$
<proof>

R_+ is closed under multiplication iff the ring has no zero divisors.

lemma (in ring1) OrdRing_ZF_3_L3:
 shows $(R_+ \text{ {is closed under} } M) \longleftrightarrow \text{HasNoZeroDivs}(R, A, M)$
<proof>

Another (in addition to `OrdRing_ZF_1_L6` sufficient condition that defines order in an ordered ring starting from the positive set.

theorem (in ring0) ring_ord_by_positive_set:
 assumes
 A1: M {is commutative on} R and
 A2: $P \subseteq R$ P {is closed under} A $0 \notin P$ and
 A3: $\forall a \in R. a \neq 0 \longrightarrow (a \in P) \text{ Xor } ((-a) \in P)$ and
 A4: P {is closed under} M and
 A5: $r = \text{OrderFromPosSet}(R, A, P)$
 shows
 IsAnOrdGroup(R, A, r)
 IsAnOrdRing(R, A, M, r)
 r {is total on} R
 $\text{PositiveSet}(R, A, r) = P$
 $\text{Nonnegative}(R, A, r) = P \cup \{0\}$
 $\text{HasNoZeroDivs}(R, A, M)$

<proof>

Nontrivial ordered rings are infinite. More precisely we assume that the neutral element of the additive operation is not equal to the multiplicative neutral element and show that the set of positive elements of the ring is not a finite subset of the ring and the ring is not a finite subset of itself.

theorem (in ring1) ord_ring_infinite: assumes $0 \neq 1$
shows
 $R_+ \notin \text{Fin}(R)$
 $R \notin \text{Fin}(R)$
<proof>

If every element of a nontrivial ordered ring can be dominated by an element from B , then B is not bounded and not finite.

lemma (in ring1) OrdRing_ZF_3_L4:
assumes $0 \neq 1$ and $\forall a \in R. \exists b \in B. a \leq b$
shows
 $\neg \text{IsBoundedAbove}(B, r)$
 $B \notin \text{Fin}(R)$
<proof>

If m is greater or equal the multiplicative unit, then the set $\{m \cdot n : n \in R\}$ is infinite (unless the ring is trivial).

lemma (in ring1) OrdRing_ZF_3_L5: assumes A1: $0 \neq 1$ and A2: $1 \leq m$
shows
 $\{m \cdot x. x \in R_+\} \notin \text{Fin}(R)$
 $\{m \cdot x. x \in R\} \notin \text{Fin}(R)$
 $\{(-m) \cdot x. x \in R\} \notin \text{Fin}(R)$
<proof>

If m is less or equal than the negative of multiplicative unit, then the set $\{m \cdot n : n \in R\}$ is infinite (unless the ring is trivial).

lemma (in ring1) OrdRing_ZF_3_L6: assumes A1: $0 \neq 1$ and A2: $m \leq -1$
shows $\{m \cdot x. x \in R\} \notin \text{Fin}(R)$
<proof>

All elements greater or equal than an element of R_+ belong to R_+ . Property of ordered groups.

lemma (in ring1) OrdRing_ZF_3_L7: assumes A1: $a \in R_+$ and A2: $a \leq b$
shows $b \in R_+$
<proof>

A special case of OrdRing_ZF_3_L7: a ring element greater or equal than 1 is positive.

corollary (in ring1) OrdRing_ZF_3_L8: assumes A1: $0 \neq 1$ and A2: $1 \leq a$
shows $a \in R_+$
<proof>

Adding a positive element to a strictly increases a . Property of ordered groups.

lemma (in ring1) OrdRing_ZF_3_L9: assumes A1: $a \in R$ $b \in R_+$
 shows $a \leq a+b$ $a \neq a+b$
 $\langle proof \rangle$

A special case of OrdRing_ZF_3_L9: in nontrivial rings adding one to a increases a .

corollary (in ring1) OrdRing_ZF_3_L10: assumes A1: $0 \neq 1$ and A2: $a \in R$
 shows $a \leq a+1$ $a \neq a+1$
 $\langle proof \rangle$

If a is not greater than b , then it is strictly less than $b+1$.

lemma (in ring1) OrdRing_ZF_3_L11: assumes A1: $0 \neq 1$ and A2: $a \leq b$
 shows $a < b+1$
 $\langle proof \rangle$

For any ring element a the greater of a and 1 is a positive element that is greater or equal than m . If we add 1 to it we get a positive element that is strictly greater than m . This holds in nontrivial rings.

lemma (in ring1) OrdRing_ZF_3_L12: assumes A1: $0 \neq 1$ and A2: $a \in R$
 shows
 $a \leq \text{GreaterOf}(r, 1, a)$
 $\text{GreaterOf}(r, 1, a) \in R_+$
 $\text{GreaterOf}(r, 1, a) + 1 \in R_+$
 $a \leq \text{GreaterOf}(r, 1, a) + 1$ $a \neq \text{GreaterOf}(r, 1, a) + 1$
 $\langle proof \rangle$

We can multiply strict inequality by a positive element.

lemma (in ring1) OrdRing_ZF_3_L13:
 assumes A1: HasNoZeroDivs(R, A, M) and
 A2: $a < b$ and A3: $c \in R_+$
 shows
 $a \cdot c < b \cdot c$
 $c \cdot a < c \cdot b$
 $\langle proof \rangle$

A sufficient condition for an element to be in the set of positive ring elements.

lemma (in ring1) OrdRing_ZF_3_L14: assumes $0 \leq a$ and $a \neq 0$
 shows $a \in R_+$
 $\langle proof \rangle$

If a ring has no zero divisors, the square of a nonzero element is positive.

lemma (in ring1) OrdRing_ZF_3_L15:
 assumes HasNoZeroDivs(R, A, M) and $a \in R$ $a \neq 0$
 shows $0 \leq a^2$ $a^2 \neq 0$ $a^2 \in R_+$
 $\langle proof \rangle$

In rings with no zero divisors we can (strictly) increase a positive element by multiplying it by an element that is greater than 1.

```
lemma (in ring1) OrdRing_ZF_3_L16:
  assumes HasNoZeroDivs(R,A,M) and a ∈ R+ and 1 ≤ b 1 ≠ b
  shows a ≤ a·b  a ≠ a·b
  <proof>
```

If the right hand side of an inequality is positive we can multiply it by a number that is greater than one.

```
lemma (in ring1) OrdRing_ZF_3_L17:
  assumes A1: HasNoZeroDivs(R,A,M) and A2: b ∈ R+ and
  A3: a ≤ b and A4: 1 < c
  shows a < b·c
  <proof>
```

We can multiply a right hand side of an inequality between positive numbers by a number that is greater than one.

```
lemma (in ring1) OrdRing_ZF_3_L18:
  assumes A1: HasNoZeroDivs(R,A,M) and A2: a ∈ R+ and
  A3: a ≤ b and A4: 1 < c
  shows a < b·c
  <proof>
```

In ordered rings with no zero divisors if at least one of a, b is not zero, then $0 < a^2 + b^2$, in particular $a^2 + b^2 \neq 0$.

```
lemma (in ring1) OrdRing_ZF_3_L19:
  assumes A1: HasNoZeroDivs(R,A,M) and A2: a ∈ R b ∈ R and
  A3: a ≠ 0 ∨ b ≠ 0
  shows 0 < a2 + b2
  <proof>
```

end

46 Normal subgroups, simple groups and Lagrange's theorem

```
theory Group_ZF_4 imports Group_ZF_1 Group_ZF_2 Finite_ZF Cardinal_ZF
```

```
begin
```

This theory file deals with normal subgroup test and some finite group theory. Then we define group homomorphisms and prove that the set of endomorphisms forms a ring with unity and we also prove the first isomorphism theorem.

46.1 Conjugation of subgroups

First we show some properties of conjugation

The conjugate of a subgroup is a subgroup.

```
theorem (in group0) conj_group_is_group:
  assumes IsAsubgroup(H,P) g∈G
  shows IsAsubgroup({g·(h·g-1) . h∈H},P)
⟨proof⟩
```

Every set is equipollent with its conjugates.

```
theorem (in group0) conj_set_is_eqpoll:
  assumes H⊆G g∈G
  shows H≈{g·(h·g-1) . h∈H}
⟨proof⟩
```

Every normal subgroup contains its conjugate subgroups.

```
theorem (in group0) norm_group_cont_conj:
  assumes IsAnormalSubgroup(G,P,H) g∈G
  shows {g·(h·g-1) . h∈H}⊆H
⟨proof⟩
```

If a subgroup contains all its conjugate subgroups, then it is normal.

```
theorem (in group0) cont_conj_is_normal:
  assumes IsAsubgroup(H,P) ∀g∈G. {g·(h·g-1) . h∈H}⊆H
  shows IsAnormalSubgroup(G,P,H)
⟨proof⟩
```

If a group has only one subgroup of a given order, then this subgroup is normal.

```
corollary (in group0) only_one_equipoll_sub:
  assumes IsAsubgroup(H,P) ∀M. IsAsubgroup(M,P) ∧ H≈M ⟶ M=H
  shows IsAnormalSubgroup(G,P,H)
⟨proof⟩
```

The trivial subgroup is then a normal subgroup.

```
corollary (in group0) trivial_normal_subgroup:
  shows IsAnormalSubgroup(G,P,{1})
⟨proof⟩
```

The whole group is normal as a subgroup

```
lemma (in group0) whole_normal_subgroup:
  shows IsAnormalSubgroup(G,P,G)
⟨proof⟩
```

46.2 Simple groups

In this subsection we study the groups that build the rest of the groups: the simple groups.

Since the whole group and the trivial subgroup are always normal, it is natural to define simplicity of groups in the following way:

definition

```
IsSimple ([_,_]{is a simple group} 89)
  where [G,f]{is a simple group}  $\equiv$  IsAgroup(G,f)  $\wedge$  ( $\forall M$ . IsAnormalSubgroup(G,f,M)
 $\longrightarrow$  M=G $\vee$ M={TheNeutralElement(G,f)})
```

From the definition follows that if a group has no subgroups, then it is simple.

```
corollary (in group0) noSubgroup_imp_simple:
  assumes  $\forall H$ . IsASubgroup(H,P)  $\longrightarrow$  H=G $\vee$ H={1}
  shows [G,P]{is a simple group}
  <proof>
```

We add a context for an abelian group

```
locale abelian_group = group0 +
  assumes isAbelian: P {is commutative on} G
```

Since every subgroup is normal in abelian groups, it follows that commutative simple groups do not have subgroups.

```
corollary (in abelian_group) abelian_simple_noSubgroups:
  assumes [G,P]{is a simple group}
  shows  $\forall H$ . IsASubgroup(H,P)  $\longrightarrow$  H=G $\vee$ H={1}
  <proof>
```

46.3 Finite groups

This subsection deals with finite groups and their structure

The subgroup of a finite group is finite.

```
lemma (in group0) finite_subgroup:
  assumes Finite(G) IsASubgroup(H,P)
  shows Finite(H)
  <proof>
```

The space of cosets is also finite. In particular, quotient groups.

```
lemma (in group0) finite_cosets:
  assumes Finite(G) IsASubgroup(H,P)
  defines r  $\equiv$  QuotientGroupRel(G,P,H)
  shows Finite(G//r)
  <proof>
```

All the cosets are equipollent.

```

lemma (in group0) cosets_equipoll:
  assumes IsAsubgroup(H,P) g1∈Gg2∈G
  defines r ≡ QuotientGroupRel(G,P,H)
  shows r{g1} ≈ r{g2}
<proof>

```

The order of a subgroup multiplied by the order of the space of cosets is the order of the group. We only prove the theorem for finite groups.

```

theorem (in group0) Lagrange:
  assumes Finite(G) IsAsubgroup(H,P)
  defines r ≡ QuotientGroupRel(G,P,H)
  shows |G|=|H| ## |G//r|
<proof>

```

46.4 Subgroups generated by sets

In this section we study the minimal subgroup containing a set

Since G is always a group containing the set, we may take the intersection of all subgroups bigger than the set; and hence the result is the subgroup we searched.

```

definition (in group0)
  SubgroupGenerated (<_>_G 80)
  where X⊆G ⇒ <X>_G ≡ ⋂ {H∈Pow(G). X⊆H ∧ IsAsubgroup(H,P)}

```

Every generated subgroup is a subgroup

```

theorem (in group0) subgroupGen_is_subgroup:
  assumes X⊆G
  shows IsAsubgroup(<X>_G,P)
<proof>

```

The generated subgroup contains the original set

```

theorem (in group0) subgroupGen_contains_set:
  assumes X⊆G
  shows X ⊆ <X>_G
<proof>

```

Given a subgroup that contains a set, the generated subgroup from that set is smaller than this subgroup

```

theorem (in group0) subgroupGen_minimal:
  assumes IsAsubgroup(H,P) X⊆H
  shows <X>_G ⊆ H
<proof>

```

end

47 Endomorphisms of abelian groups

theory Group_ZF_5 **imports** Group_ZF_4 Ring_ZF Semigroup_ZF

begin

When the operation P in the group (G, P) is commutative (i.e. the group the abelian) the space $\text{End}(G, P)$ of homomorphisms of a group (G, P) into itself has a nice structure.

47.1 First ring of endomorphisms of an abelian group

In this section we show that for an abelian group (G, P) the space $\text{End}(G, P)$ (defined in the Group_ZF_2 theory) forms a ring.

The set of endomorphisms is closed under pointwise addition (derived from the group operation). This is so because the group is abelian.

theorem (in abelian_group) end_pointwise_addition:
assumes $f \in \text{End}(G, P)$ $g \in \text{End}(G, P)$
defines $F \equiv P \text{ \{lifted to function space over\} } G$
shows $F\langle f, g \rangle \in \text{End}(G, P)$
<proof>

The value of a product of endomorphisms on a group element is the product of values.

lemma (in abelian_group) end_pointwise_add_val:
assumes $f \in \text{End}(G, P)$ $g \in \text{End}(G, P)$ $x \in G$ $F = P \text{ \{lifted to function space over\} } G$
shows $(\text{InEnd}(F, G, P)\langle f, g \rangle)(x) = (f(x)) \cdot (g(x))$
<proof>

The operation of taking the inverse in an abelian group is an endomorphism.

lemma (in abelian_group) end_inverse_group:
shows $\text{GroupInv}(G, P) \in \text{End}(G, P)$
<proof>

The set of homomorphisms of an abelian group is an abelian subgroup of the group of functions from a set to a group, under pointwise addition.

theorem (in abelian_group) end_addition_group:
assumes $F = P \text{ \{lifted to function space over\} } G$
shows $\text{IsAgroup}(\text{End}(G, P), \text{InEnd}(F, G, P))$ **and**
 $\text{InEnd}(F, G, P) \text{ \{is commutative on\} } \text{End}(G, P)$
<proof>

Endomorphisms form a subgroup of the space of functions that map the group to itself.

lemma (in abelian_group) end_addition_subgroup:

```

shows IsSubgroup(End(G,P),P {lifted to function space over} G)
<proof>

```

The neutral element of the group of endomorphisms of a group is the constant function with value equal to the neutral element of the group.

```

lemma (in abelian_group) end_add_neut_elem:
  assumes F = P {lifted to function space over} G
  shows TheNeutralElement(End(G,P),InEnd(F,G,P)) = ConstantFunction(G,1)
  <proof>

```

For the endomorphisms of a group G the group operation lifted to the function space over G is distributive with respect to the composition operation.

```

lemma (in abelian_group) distributive_comp_pointwise:
  assumes F = P {lifted to function space over} G
  shows
    IsDistributive(End(G,P),InEnd(F,G,P),InEnd(Composition(G),G,P))
  <proof>

```

The endomorphisms of an abelian group is in fact a ring with the previous operations.

```

theorem (in abelian_group) end_is_ring:
  assumes F = P {lifted to function space over} G
  shows
    IsAring(End(G,P),InEnd(F,G,P),InEnd(Composition(G),G,P))
  <proof>

```

The theorems proven in the ring0 context are valid in the abelian_group context as applied to the endomorphisms of G .

```

sublocale abelian_group < endo_ring: ring0
  End(G,P)
  InEnd(P {lifted to function space over} G,G,P)
  InEnd(Composition(G),G,P)
  λx b. InEnd(P {lifted to function space over} G,G,P)⟨x,b⟩
  λx. GroupInv(End(G, P), InEnd(P {lifted to function space over} G,G,P))(x)

  λx b. InEnd(P {lifted to function space over} G,G,P)⟨x, GroupInv(End(G,
P), InEnd(P {lifted to function space over} G,G,P))(b)⟩
  λx b. InEnd(Composition(G),G,P)⟨x, b⟩
  TheNeutralElement(End(G, P),InEnd(P {lifted to function space over}
G,G,P))
  TheNeutralElement(End(G, P),InEnd(Composition(G),G,P))
  InEnd(P {lifted to function space over} G,G,P)
  ⟨TheNeutralElement (End(G, P), InEnd(Composition(G),G,P)),
    TheNeutralElement (End(G, P), InEnd(Composition(G),G,P))⟩
  λx. InEnd(Composition(G),G,P)⟨x, x⟩
  λs. Fold(InEnd(P {lifted to function space over} G,G,P), TheNeutralElement(End(G,
P), InEnd(P {lifted to function space over} G,G,P)),s)

```

```

    λn x. Fold(InEnd(P {lifted to function space over} G,G,P), TheNeutralElement(End(G,
P), InEnd(P {lifted to function space over} G,G,P)),{⟨k,x⟩. k∈n})
    λs. Fold(InEnd(Composition(G),G,P), TheNeutralElement(End(G, P), InEnd(Composition(G),G,P
    λn x. Fold(InEnd(Composition(G),G,P), TheNeutralElement(End(G, P), InEnd(Composition(G),G
k∈n}))
    ⟨proof⟩

```

In the context of `group0`, we may use all results of `semigr0`.

```

sublocale group0 < semigroup:semigr0 G P groper λx. Fold1(P,x) Append
Concat
    ⟨proof⟩

end

```

48 Group homomorphisms and kernels

```

theory Group_ZF_6 imports Group_ZF_4

```

```

begin

```

The natural quotient map $q : G \rightarrow G/H$ (for a normal subgroup H) is a group homomorphism. Recall that $\lambda x \in X. p(x)$ is an alternative notation for a function defined as a set of pairs, see lemma `lambda_fun_alt` in `func1.thy`.

```

lemma (in group0) quotient_map:
  assumes IsAnormalSubgroup(G,P,H)
  defines r ≡ QuotientGroupRel(G,P,H) and q ≡ λx∈G. QuotientGroupRel(G,P,H){x}
  shows Homomor(q,G,P,G//r,QuotientGroupOp(G,P,H))
    ⟨proof⟩

```

48.1 The group homomorphism locale

The next locale packages two groups together with a group homomorphism between them. We fix an origin group (G, P) and a target group (H, F) with homomorphism $f : G \rightarrow H$, and introduce subscripted notation for both groups.

```

locale group_homo =
  fixes G P H F f

  assumes origin: IsAgroup(G, P)
    and target: IsAgroup(H, F)
    and hom: Homomor(f, G, P, H, F)

  fixes neut (1G)
  defines neut_def [simp]: 1G ≡ TheNeutralElement(G, P)

  fixes groper (infixl ·G 70)
  defines groper_def [simp]: x ·G y ≡ P⟨x, y⟩

```

```

fixes inv ( $\_^{-1}_G$  [90] 91)
defines inv_def [simp]:  $x^{-1}_G \equiv \text{GroupInv}(G, P)x$ 

fixes listprod ( $\prod_G \_$  70)
defines listprod_def [simp]:  $\prod_G s \equiv \text{Fold}(P, 1_G, s)$ 

fixes pow
defines pow_def [simp]:  $\text{pow}(n, x) \equiv \prod_G \{\langle k, x \rangle. k \in n\}$ 

fixes neutH ( $1_H$ )
defines neutH_def [simp]:  $1_H \equiv \text{TheNeutralElement}(H, F)$ 

fixes groperH (infixl  $\cdot_H$  70)
defines groperH_def [simp]:  $x \cdot_H y \equiv F\langle x, y \rangle$ 

fixes invH ( $\_^{-1}_H$  [90] 91)
defines invH_def [simp]:  $x^{-1}_H \equiv \text{GroupInv}(H, F)x$ 

fixes listprodH ( $\prod_H \_$  70)
defines listprodH_def [simp]:  $\prod_H s \equiv \text{Fold}(F, 1_H, s)$ 

fixes powH
defines powH_def [simp]:  $\text{powH}(n, x) \equiv \prod_H \{\langle k, x \rangle. k \in n\}$ 

```

$\ker$ denotes the kernel of f , i.e. the preimage of the neutral element of the target group.

abbreviation (**in** group_homo) kernel ($\ker$ 90) **where**
 $\ker \equiv f^{-1}\{1_H\}$

The theorems proven in the `group0` context are valid in the `group_homo` context when applied to the origin group G .

sublocale group_homo < org: group0 G P neut groper inv listprod pow
 $\langle \text{proof} \rangle$

The theorems proven in the `group0` context are valid in the `group_homo` context when applied to the target group H .

sublocale group_homo < tgt: group0 H F neutH groperH invH listprodH powH
 $\langle \text{proof} \rangle$

f is a function from G to H .

lemma (**in** group_homo) f_is_fun: **shows** $f: G \rightarrow H$
 $\langle \text{proof} \rangle$

f is a group homomorphism: $f(x \cdot_G y) = f(x) \cdot_H f(y)$.

lemma (**in** group_homo) f_hom:
assumes $x \in G \ y \in G$
shows $f(x \cdot_G y) = fx \cdot_H fy$

⟨proof⟩

f maps the neutral element of G to the neutral element of H .

lemma (in group_homo) f_neutral: shows $f(1_G) = 1_H$
⟨proof⟩

f commutes with the group inverse: $f(x^{-1}) = f(x)^{-1}$.

lemma (in group_homo) f_inv:
 assumes $x \in G$
 shows $f(x^{-1}_G) = (fx)^{-1}_H$
⟨proof⟩

The preimage of a subgroup of H is a subgroup of G .

lemma (in group_homo) preimage_is_subgroup:
 assumes sub: IsAsubgroup(K, F)
 shows IsAsubgroup($f^{-1}(K), P$)
⟨proof⟩

The image of a subgroup of G is a subgroup of H .

lemma (in group_homo) image_is_subgroup:
 assumes sub: IsAsubgroup(K, P)
 shows IsAsubgroup(fK, F)
⟨proof⟩

The image of G under f is a subgroup of H .

lemma (in group_homo) image_is_group:
 shows IsAsubgroup(fG, F)
⟨proof⟩

Membership in the kernel: $x \in G$ belongs to $\ker(f)$ iff $f(x) = 1_H$.

lemma (in group_homo) kernel_mem:
 assumes $x \in G$
 shows $x \in \ker \longleftrightarrow fx = 1_H$
⟨proof⟩

The kernel is a subset of G .

lemma (in group_homo) kernel_in_G: shows $\ker \subseteq G$
⟨proof⟩

The neutral element of G belongs to the kernel.

lemma (in group_homo) neutral_in_kernel: shows $1_G \in \ker$
⟨proof⟩

The preimage of a normal subgroup of H is a normal subgroup of G .

theorem (in group_homo) preimage_normal_subgroup:
 assumes IsAnormalSubgroup(H, F, K)
 shows IsAnormalSubgroup($G, P, f^{-1}(K)$)

<proof>

The kernel is a normal subgroup of G .

theorem (in group_homo) kernel_normal_subgroup:
 shows IsAnormalSubgroup(G , P , $\ker$)
<proof>

In particular, the kernel is a subgroup of G .

corollary (in group_homo) kernel_is_subgroup:
 shows IsAsubgroup($\ker$, P)
<proof>

The inverse of a bijective homomorphism is a homomorphism.

theorem (in group_homo) bijective_hom_inverse:
 assumes $\text{bij}: f \in \text{bij}(G, H)$
 shows $\text{Homomor}(\text{converse}(f), H, F, G, P)$
<proof>

The first isomorphism theorem: f induces a bijective homomorphism $G/\ker(f) \cong f(G)$.

theorem (in group_homo) first_isomorphism_theorem:
 defines $r \equiv \text{QuotientGroupRel}(G, P, \ker)$
 and $\mathcal{P} \equiv \text{QuotientGroupOp}(G, P, \ker)$
 shows $\exists f. \text{Homomor}(f, G//r, \mathcal{P}, fG, \text{restrict}(F, fG \times fG)) \wedge$
 $f \in \text{bij}(G//r, fG)$
<proof>

A group homomorphism is injective if and only if its kernel is trivial.

theorem (in group_homo) kernel_trivial_iff_injective:
 shows $f \in \text{inj}(G, H) \longleftrightarrow \ker = \{1_G\}$
<proof>

48.2 Exact sequences at a term

A pair of composable group homomorphisms $A \xrightarrow{f} B \xrightarrow{g} C$ is *exact at B* when the image of f equals the kernel of g .

definition IsExactAt :: [i, i, i, i, i] $\Rightarrow$ o where
 IsExactAt(f, A, g, C, Q) $\equiv fA = g\text{-}\{\text{TheNeutralElement}(C, Q)\}$

A locale for two composable group homomorphisms $f: A \rightarrow B$ and $g: B \rightarrow C$. Lemmas are proved using the two group_homo instances directly rather than via sublocale inheritance.

locale two_group_homo =
 fixes $A\ P\ B\ F\ C\ Q\ f\ g$
 assumes $\text{hf}: \text{group_homo}(A, P, B, F, f)$
 and $\text{hg}: \text{group_homo}(B, F, C, Q, g)$

abbreviation (in two_group_homo) ExactAtB where

ExactAtB $\equiv$ IsExactAt(f, A, g, C, Q)

```
sublocale two_group_homo < second_homo: group_homo B F C Q g
  TheNeutralElement(B,F)  $\lambda x y. F\langle x,y \rangle$ 
 $\lambda x. \text{GroupInv}(B,F)x \lambda s. \text{Fold}(F, \text{TheNeutralElement}(B, F), s)$ 
 $\lambda n x. \text{Fold}(F, \text{TheNeutralElement}(B, F), \{\langle k, x \rangle . k \in n\})$ 
  TheNeutralElement(C,Q)  $\lambda x y. Q\langle x,y \rangle$ 
 $\lambda x. \text{GroupInv}(C,Q)x \lambda s. \text{Fold}(Q, \text{TheNeutralElement}(C, Q), s)$ 
 $\lambda n x. \text{Fold}(Q, \text{TheNeutralElement}(C, Q), \{\langle k, x \rangle . k \in n\})$ 
  <proof>
```

```
sublocale two_group_homo < first_homo: group_homo A P B F f
  TheNeutralElement(A,P)  $\lambda x y. P\langle x,y \rangle$ 
 $\lambda x. \text{GroupInv}(A,P)x \lambda s. \text{Fold}(P, \text{TheNeutralElement}(A, P), s)$ 
 $\lambda n x. \text{Fold}(P, \text{TheNeutralElement}(A, P), \{\langle k, x \rangle . k \in n\})$ 
  TheNeutralElement(B,F)  $\lambda x y. F\langle x,y \rangle$ 
 $\lambda x. \text{GroupInv}(B,F)x \lambda s. \text{Fold}(F, \text{TheNeutralElement}(B, F), s)$ 
 $\lambda n x. \text{Fold}(F, \text{TheNeutralElement}(B, F), \{\langle k, x \rangle . k \in n\})$ 
  <proof>
```

Exactness at B means the image of f equals the kernel of g .

lemma (in two_group_homo) ExactAtB_unfold:

shows ExactAtB $\longleftrightarrow fA = g\text{-}\{\text{TheNeutralElement}(C, Q)\}$

<proof>

If the sequence is exact at B , the composition $g \circ f$ maps every element of A to the neutral element of C .

lemma (in two_group_homo) exact_comp_trivial:

assumes ExactAtB and $x \in A$

shows $g(fx) = \text{TheNeutralElement}(C, Q)$

<proof>

A sufficient condition for exactness: if $g \circ f$ is trivial and the kernel of g is contained in the image of f , then the sequence is exact at B .

lemma (in two_group_homo) exact_at_B_intro:

assumes $\forall x \in A. g(fx) = \text{TheNeutralElement}(C, Q)$

and $g\text{-}\{\text{TheNeutralElement}(C, Q)\} \subseteq fA$

shows ExactAtB

<proof>

48.3 The trivial group and zero maps

The *trivial group* has carrier $1 = \{0\}$ and operation $\mathbf{0}: 1 \times 1 \rightarrow 1$ defined by $\mathbf{0}(0,0) = 0$.

definition TrivOp :: i where

$\text{TrivOp} \equiv \text{ConstantFunction}(1 \times 1, 0)$

$\mathbf{0}$ is a function $1 \times 1 \rightarrow 1$.

lemma TrivOp_type : $\text{TrivOp} : 1 \times 1 \rightarrow 1$
 $\langle \text{proof} \rangle$

The value of $\mathbf{0}$ at any pair from 1×1 is 0.

lemma TrivOp_val :
 assumes $x \in 1$ $y \in 1$
 shows $\text{TrivOp}(x, y) = 0$
 $\langle \text{proof} \rangle$

The neutral element of $(1, \mathbf{0})$ is 0.

lemma TrivOp_ne : $\text{TheNeutralElement}(1, \text{TrivOp}) = 0$
 $\langle \text{proof} \rangle$

$(1, \mathbf{0})$ is a group.

lemma trivial_group : $\text{IsAgroup}(1, \text{TrivOp})$
 $\langle \text{proof} \rangle$

$(1, \mathbf{0})$ is abelian.

lemma $\text{trivial_group_abelian}$: $\text{TrivOp} \{\text{is commutative on}\} 1$
 $\langle \text{proof} \rangle$

The *zero map* from (G_1, P_1) to (G_2, P_2) sends every element to the neutral element of G_2 .

definition $\text{ZeroMap} :: [i, i, i] \Rightarrow i$ **where**
 $\text{ZeroMap}(G_1, G_2, P_2) \equiv \text{ConstantFunction}(G_1, \text{TheNeutralElement}(G_2, P_2))$

The zero map is a group homomorphism.

lemma ZeroMap_hom :
 assumes $\text{grp1}: \text{IsAgroup}(G_1, P_1)$ and $\text{grp2}: \text{IsAgroup}(G_2, P_2)$
 shows $\text{Homomor}(\text{ZeroMap}(G_1, G_2, P_2), G_1, P_1, G_2, P_2)$
 $\langle \text{proof} \rangle$

The image of the zero map over its domain is the singleton of the neutral element.

lemma ZeroMap_image :
 assumes $\text{grp1}: \text{IsAgroup}(G_1, P_1)$ and $\text{grp2}: \text{IsAgroup}(G_2, P_2)$
 shows $\text{ZeroMap}(G_1, G_2, P_2)G_1 = \{\text{TheNeutralElement}(G_2, P_2)\}$
 $\langle \text{proof} \rangle$

The preimage of the neutral element under the zero map is the whole domain.

lemma $\text{ZeroMap_preimage_full}$:
 assumes $\text{grp1}: \text{IsAgroup}(G_1, P_1)$ and $\text{grp2}: \text{IsAgroup}(G_2, P_2)$
 shows $\text{ZeroMap}(G_1, G_2, P_2)^{-1}\{\text{TheNeutralElement}(G_2, P_2)\} = G_1$
 $\langle \text{proof} \rangle$

48.4 Short exact sequences

A *short exact sequence* $0 \rightarrow A \xrightarrow{f} B \xrightarrow{g} C \rightarrow 0$ is expressed by exactness at all three terms: the sequence is exact at A (i.e. f is injective), at B (i.e. $\text{im}(f) = \ker(g)$), and at C (i.e. g is surjective), each encoded via zero maps to or from the trivial group.

definition `IsAshortExactSequence` :: `[i, i, i, i, i, i, i, i] => o` where
`IsAshortExactSequence(A, P, B, F, C, Q, f, g) ≡`
`group_homo(A, P, B, F, f) ∧`
`group_homo(B, F, C, Q, g) ∧`
`IsExactAt(ZeroMap(1, A, P), 1, f, B, F) ∧`
`IsExactAt(f, A, g, C, Q) ∧`
`IsExactAt(g, B, ZeroMap(C, 1, TrivOp), 1, TrivOp)`

The `short_exact_sequence` locale packages a short exact sequence, expressed by exactness at all three terms.

locale `short_exact_sequence` = `two_group_homo` +
`assumes ses_exact_at_A: IsExactAt(ZeroMap(1, A, P), 1, f, B, F)`
`and ses_exact: ExactAtB`
`and ses_exact_at_C: IsExactAt(g, B, ZeroMap(C, 1, TrivOp), 1, TrivOp)`

The image of f equals the kernel of g .

lemma `(in short_exact_sequence) ses_im_eq_ker:`
`shows fA = g-{TheNeutralElement(C, Q)}`
`<proof>`

f is injective: exactness at A forces the kernel of f to be trivial.

lemma `(in short_exact_sequence) ses_inj:`
`shows f ∈ inj(A, B)`
`<proof>`

g is surjective: exactness at C forces g to hit all of C .

lemma `(in short_exact_sequence) ses_surj:`
`shows gB = C`
`<proof>`

The kernel of f is trivial.

lemma `(in short_exact_sequence) ses_ker_trivial:`
`shows f-{TheNeutralElement(B, F)} = {TheNeutralElement(A, P)}`
`<proof>`

g maps each element of B to an element of C .

lemma `(in short_exact_sequence) ses_g_type:`
`assumes b ∈ B`
`shows gb ∈ C`
`<proof>`

An element of B in the kernel of g is in the image of f .

```
lemma (in short_exact_sequence) ses_ker_in_im:
  assumes b ∈ B and gb = TheNeutralElement(C, Q)
  shows b ∈ fA
  ⟨proof⟩
```

An element in the image of f maps to the neutral element under g .

```
lemma (in short_exact_sequence) ses_im_to_ker:
  assumes b ∈ fA
  shows gb = TheNeutralElement(C, Q)
  ⟨proof⟩
```

First isomorphism theorem for a short exact sequence: the quotient $B/f(A)$ is isomorphic to C via the map induced by g .

```
theorem (in short_exact_sequence) ses_first_iso:
  defines r ≡ QuotientGroupRel(B, F, fA)
  and P ≡ QuotientGroupOp(B, F, fA)
  shows ∃f. Homomor(f, B//r, P, C, Q) ∧ f ∈ bij(B//r, C)
  ⟨proof⟩
```

48.5 The cokernel

A *cokernel* of $f: A \rightarrow B$ is a group homomorphism $q: B \rightarrow C$ satisfying (i) $q \circ f = 0$, and (ii) the universal property: every $h: B \rightarrow D$ with $h \circ f = 0$ factors uniquely through q .

```
definition IsAcokernel :: [i, i, i, i, i, i, i] ⇒ o where
  IsAcokernel(f, A, B, F, q, C, Q) ≡
    group_homo(B, F, C, Q, q) ∧
    (∀x ∈ A. q(fx) = TheNeutralElement(C, Q)) ∧
    (∀D R h. IsAgroup(D, R) ∧ Homomor(h, B, F, D, R) ∧
      (∀x ∈ A. h(fx) = TheNeutralElement(D, R)) ⟶
      (∃!hh. Homomor(hh, C, Q, D, R) ∧ (∀b ∈ B. hh(qb) = hb)))
```

If the target group H is abelian then every subgroup is normal, so in particular $f(G)$ is a normal subgroup of H .

```
lemma (in group_homo) image_normal_if_abelian:
  assumes F {is commutative on} H
  shows IsAnormalSubgroup(H, F, fG)
  ⟨proof⟩
```

The natural quotient map $q: H \rightarrow H/f(G)$ is a cokernel of f whenever H is abelian. The factored map for the universal property is $\bar{h} = \{\langle r, \{b\}, h'b \rangle \mid b \in H\}$, which is exactly the construction used in the first isomorphism theorem.

```
theorem (in group_homo) quotient_is_cokernel:
  assumes abel: F {is commutative on} H
```

```

defines r  $\equiv$  QuotientGroupRel(H, F, fG)
      and q  $\equiv$   $\lambda h \in H. \text{QuotientGroupRel}(H, F, fG)\{h\}$ 
      and Q  $\equiv$  QuotientGroupOp(H, F, fG)
shows IsAcokernel(f, G, H, F, q, H//r, Q)
   $\langle proof \rangle$ 

end

```

49 Rings - Ideals

```

theory Ring_ZF_2 imports Ring_ZF Group_ZF_2 Finite_ZF Finite1 Cardinal_ZF
Semigroup_ZF

```

begin

This section defines the concept of a ring ideal, and defines some basic concepts and types, finishing with the theorem that shows that the quotient of the additive group by the ideal is actually a full ring.

49.1 Ideals

In ring theory ideals are special subsets of a ring that play a similar role as normal subgroups in the group theory.

An ideal is a subgroup of the additive group of the ring, which is closed by left and right multiplication by any ring element.

definition (in ring0) Ideal ($_ \triangleleft R$) **where**
 $I \triangleleft R \equiv (\forall x \in I. \forall y \in R. y \cdot x \in I \wedge x \cdot y \in I) \wedge \text{IsASubgroup}(I, A)$

To write less during proofs, we will write $\mathcal{I}$ to denote the set of ideals of the ring R .

abbreviation (in ring0) ideals ($\mathcal{I}$) **where** $\mathcal{I} \equiv \{J \in \text{Pow}(R). J \triangleleft R\}$

The first examples of ideals are the whole ring and the zero ring:

lemma (in ring0) ring_self_ideal:
shows $R \triangleleft R$
 $\langle proof \rangle$

The singleton containing zero is and ideal.

lemma (in ring0) zero_ideal:
shows $\{0\} \triangleleft R$ $\langle proof \rangle$

An ideal is s subset of the the ring.

lemma (in ring0) ideal_dest_subset:
assumes $I \triangleleft R$
shows $I \subseteq R$ $\langle proof \rangle$

Ideals are closed with respect to the ring addition.

```
lemma (in ring0) ideal_dest_sum:
  assumes I <R x∈I y∈I
  shows x+y ∈I <proof>
```

Ideals are closed with respect to the ring multiplication.

```
lemma (in ring0) ideal_dest_mult:
  assumes I <R x∈I y∈R
  shows x·y ∈I y·x ∈I <proof>
```

Ideals are closed with respect to taking the opposite in the ring.

```
lemma (in ring0) ideal_dest_minus:
  assumes I <R x∈I
  shows (-x) ∈ I
  <proof>
```

Every ideals contains zero.

```
lemma (in ring0) ideal_dest_zero:
  assumes I <R
  shows 0 ∈ I
  <proof>
```

If the rules are satisfied, then we have an ideal

```
theorem (in ring0) ideal_intro:
  assumes ∀x∈I. ∀y∈I. x+y∈I
  ∀x∈I. ∀y∈R. x·y ∈I
  ∀x∈I. ∀y∈R. y·x ∈I
  I ⊆ R I≠0
  shows I<R
  <proof>
```

The simplest way to obtain an ideal from others is the intersection, since the intersection of arbitrary collection of ideals is an ideal.

```
theorem (in ring0) intersection_ideals:
  assumes ∀J∈J. (J <R) J ≠ 0
  shows (⋂ J) <R
  <proof>
```

In particular, intersection of two ideals is an ideal.

```
corollary (in ring0) inter_two_ideals: assumes I<R J<R
  shows (I∩J) <R
  <proof>
```

From any set, we may construct the minimal ideal containing that set

```
definition (in ring0) generatedIdeal (<_>_I)
  where X⊆R ⇒ <X>_I ≡ ⋂ {I∈I. X ⊆ I}
```

The ideal generated by a set is an ideal

corollary (in ring0) generated_ideal_is_ideal:
 assumes $X \subseteq R$ shows $\langle X \rangle_I \triangleleft R$
 $\langle proof \rangle$

The ideal generated by a set is contained in any ideal containing the set.

corollary (in ring0) generated_ideal_small:
 assumes $X \subseteq I$ $I \triangleleft R$
 shows $\langle X \rangle_I \subseteq I$
 $\langle proof \rangle$

The ideal generated by a set contains the set.

corollary (in ring0) generated_ideal_contains_set:
 assumes $X \subseteq R$ shows $X \subseteq \langle X \rangle_I$
 $\langle proof \rangle$

To be able to show properties of an ideal generated by a set, we have the following induction result

lemma (in ring0) induction_generated_ideal:
 assumes
 $X \neq \emptyset$
 $X \subseteq R$
 $\forall y \in R. \forall z \in R. \forall q \in \langle X \rangle_I. P(q) \longrightarrow P(y \cdot q \cdot z)$
 $\forall y \in R. \forall z \in R. P(y) \wedge P(z) \longrightarrow P(y+z)$
 $\forall x \in X. P(x)$
 shows $\forall y \in \langle X \rangle_I. P(y)$
 $\langle proof \rangle$

An ideal is very particular with the elements it may contain. If it contains the neutral element of multiplication then it is in fact the whole ring and not a proper subset.

theorem (in ring0) ideal_with_one:
 assumes $I \triangleleft R$ $1 \in I$ shows $I = R$
 $\langle proof \rangle$

The only ideal containing an invertible element is the whole ring.

theorem (in ring0) ideal_with_unit:
 assumes $I \triangleleft R$ $x \in I$ $\exists y \in R. y \cdot x = 1 \vee x \cdot y = 1$
 shows $I = R$
 $\langle proof \rangle$

The previous result drives us to define what a maximal ideal would be: an ideal such that any bigger ideal is the whole ring:

definition (in ring0) maximalIdeal ($_ \triangleleft_m R$) where
 $I \triangleleft_m R \equiv I \triangleleft R \wedge I \neq R \wedge (\forall J \in \mathcal{I}. I \subseteq J \wedge J \neq R \longrightarrow I = J)$

Before delving into maximal ideals, let's define some operation on ideals that are useful when formulating some proofs. The product ideal of ideals I, J is the smallest ideal containing all products of elements from I and J :

definition (in ring0) productIdeal (infix $\cdot_I$ 90) where
 $I \triangleleft R \implies J \triangleleft R \implies I \cdot_I J \equiv \langle M(I \times J) \rangle_I$

The sum ideal of ideals is the smallest ideal containing both I and J :

definition (in ring0) sumIdeal (infix $+_I$ 90) where
 $I \triangleleft R \implies J \triangleleft R \implies I +_I J \equiv \langle I \cup J \rangle_I$

Sometimes we may need to sum an arbitrary number of ideals, and not just two.

definition(in ring0) sumArbitraryIdeals ($\oplus_I$ 90) where
 $\mathcal{J} \subseteq \mathcal{I} \implies \oplus_I \mathcal{J} \equiv \langle \bigcup \mathcal{J} \rangle_I$

Each component of the sum of ideals is contained in the sum.

lemma (in ring0) comp_in_sum_ideals:
 assumes $I \triangleleft R$ and $J \triangleleft R$
 shows $I \subseteq I +_I J$ and $J \subseteq I +_I J$ and $I \cup J \subseteq I +_I J$
<proof>

Every element in the arbitrary sum of ideals is generated by only a finite subset of those ideals

lemma (in ring0) sum_ideals_finite_sum:
 assumes $\mathcal{J} \subseteq \mathcal{I}$ $s \in (\oplus_I \mathcal{J})$
 shows $\exists \mathcal{T} \in \text{FinPow}(\mathcal{J}). s \in (\oplus_I \mathcal{T})$
<proof>

By definition of product of ideals and of an ideal itself, it follows that the product of ideals is an ideal contained in the intersection

theorem (in ring0) product_in_intersection:
 assumes $I \triangleleft R$ $J \triangleleft R$
 shows $I \cdot_I J \subseteq I \cap J$ and $(I \cdot_I J) \triangleleft R$ and $M(I \times J) \subseteq I \cdot_I J$
<proof>

We will show now that the sum of ideals is no more than the sum of the ideal elements.

lemma (in ring0) sum_elements:
 assumes $I \triangleleft R$ $J \triangleleft R$ $x \in I$ $y \in J$
 shows $x + y \in I +_I J$
<proof>

For two ideals the set containing all sums of their elements is also an ideal.

lemma (in ring0) sum_elements_is_ideal:
 assumes $I \triangleleft R$ $J \triangleleft R$
 shows $\langle I + J \rangle \triangleleft R$

<proof>

The set of all sums of elements of two ideals is their sum ideal i.e. the ideal generated by their union.

corollary (in ring0) sum_ideals_is_sum_elements:
 assumes $I \triangleleft R \ J \triangleleft R$
 shows $(I + J) = I +_I J$
<proof>

The sum ideal of two ideals is indeed an ideal.

corollary (in ring0) sum_ideals_is_ideal:
 assumes $I \triangleleft R \ J \triangleleft R$
 shows $(I +_I J) \triangleleft R$ *<proof>*

The operation of taking the sum of ideals is commutative.

corollary (in ring0) sum_ideals_commute:
 assumes $I \triangleleft R \ J \triangleleft R$
 shows $(I +_I J) = (J +_I I)$
<proof>

Now that we know what the product of ideals is, we are able to define what a prime ideal is:

definition (in ring0) primeIdeal ($_ \triangleleft_p R$) where
 $P \triangleleft_p R \equiv P \triangleleft R \wedge P \neq R \wedge (\forall I \in \mathcal{I}. \forall J \in \mathcal{I}. I \cdot_I J \subseteq P \longrightarrow (I \subseteq P \vee J \subseteq P))$

Any maximal ideal is a prime ideal.

theorem (in ring0) maximal_is_prime:
 assumes $Q \triangleleft_m R$ shows $Q \triangleleft_p R$
<proof>

In case of non-commutative rings, the zero divisor concept is too constrictive. For that we define the following concept of a prime ring. Note that in case that our ring is commutative, this is equivalent to having no zero divisors (there is no of that proof yet).

definition primeRing ($[_,_,_]\{\text{is a prime ring}\}$) where
 $\text{IsAring}(R,A,M) \implies [R,A,M]\{\text{is a prime ring}\} \equiv$
 $(\forall x \in R. \forall y \in R. (\forall z \in R. M\langle x,z \rangle, y) = \text{TheNeutralElement}(R,A)) \longrightarrow$
 $x = \text{TheNeutralElement}(R,A) \vee y = \text{TheNeutralElement}(R,A))$

Prime rings appear when the zero ideal is prime.

lemma (in ring0) prime_ring_zero_prime_ideal:
 assumes $[R,A,M]\{\text{is a prime ring}\} \ R \neq \{0\}$
 shows $\{0\} \triangleleft_p R$
<proof>

If the trivial ideal $\{0\}$ is a prime ideal then the ring is a prime ring.

lemma (in ring0) zero_prime_ideal_prime_ring:

```

    assumes {0} <ₚ R
    shows [R,A,M]{is a prime ring}
  <proof>

```

We can actually use this definition of a prime ring as a condition to check for prime ideals.

```

theorem (in ring0) equivalent_prime_ideal:
  assumes P <ₚ R
  shows  $\forall x \in R. \forall y \in R. (\forall z \in R. x \cdot z \cdot y \in P) \longrightarrow x \in P \vee y \in P$ 
  <proof>

```

The next theorem provides a sufficient condition for a proper ideal P to be a prime ideal: if for all $x, y \in R$ it holds that for all $z \in R$ $xzy \in P$ only when $x \in P$ or $y \in P$ then P is a prime ideal.

```

theorem (in ring0) equivalent_prime_ideal_2:
  assumes  $\forall x \in R. \forall y \in R. (\forall z \in R. x \cdot z \cdot y \in P) \longrightarrow x \in P \vee y \in P$ 
  shows P <ₚ R
  <proof>

```

49.2 Ring quotient

Similar to groups, rings can be quotiented by normal additive subgroups; but to keep the structure of the multiplicative monoid we need extra structure in the normal subgroup. This extra structure is given by the ideal.

Any ideal is a normal subgroup.

```

lemma (in ring0) ideal_normal_add_subgroup:
  assumes I < R
  shows IsANormalSubgroup(R,A,I)
  <proof>

```

Each ring R is a group with respect to its addition operation. By the lemma `ideal_normal_add_subgroup` above an ideal $I \subseteq R$ is a normal subgroup of that group. Therefore we can define the quotient of the ring R by the ideal I using the notion of quotient of a group by its normal subgroup, see section `Normal subgroups and quotient groups` in `Group_ZF_2` theory.

```

definition (in ring0) QuotientBy where
  I < R  $\implies$  QuotientBy(I)  $\equiv$  R // QuotientGroupRel(R,A,I)

```

Any ideal gives rise to an equivalence relation

```

corollary (in ring0) ideal_equiv_rel:
  assumes I < R
  shows equiv(R,QuotientGroupRel(R,A,I))
  <proof>

```

Any quotient by an ideal is an abelian group.

```

lemma (in ring0) quotientBy_add_group:

```

```

assumes I<R
shows IsAgroup(QuotientBy(I), QuotientGroupOp(R, A, I)) and
    QuotientGroupOp(R, A, I) {is commutative on} QuotientBy(I)
<proof>

```

Since every ideal is a normal subgroup of the additive group of the ring it is quite obvious that that addition is congruent with respect to the quotient group relation. The next lemma shows something a little bit less obvious: that the multiplicative ring operation is also congruent with the quotient relation and gives rise to a monoid in the quotient.

```

lemma (in ring0) quotientBy_mul_monoid:
  assumes I<R
  shows Congruent2(QuotientGroupRel(R, A, I),M) and
    IsAmonoid(QuotientBy(I),ProjFun2(R, QuotientGroupRel(R,A,I), M))
<proof>

```

Each ideal defines an equivalence relation on the ring with which both addition and multiplication are congruent. The next couple of definitions set up notation for the operations that result from projecting the ring addition and multiplication on the quotient space. We will write $x +_I y$ to denote the result of the quotient operation (with respect to an ideal I) on classes x and y

```

definition (in ring0) ideal_radd (_{+}_) where
  x{+I}y  $\equiv$  QuotientGroupOp(R, A, I)(x,y)

```

Similarly $x \cdot_I y$ is the value of the projection of the ring's multiplication on the quotient space defined by the an ideal I , which as we know is a normal subgroup of the ring with addition.

```

definition (in ring0) ideal_rmult (_{·}_) where
  x{·I}y  $\equiv$  ProjFun2(R, QuotientGroupRel(R,A,I), M)(x,y)

```

The value of the projection of taking the negative in the ring on the quotient space defined by an ideal I will be denoted $\{-I\}$.

```

definition (in ring0) ideal_rmin ({-}_) where
  {-I}y  $\equiv$  GroupInv(QuotientBy(I),QuotientGroupOp(R, A, I))(y)

```

Subtraction in the quotient space is defined by the $+I$ and $-I$ operations in the obvious way.

```

definition (in ring0) ideal_rsub (_{-}_) where
  x{-I}y  $\equiv$  x{+I}({-I}y)

```

The class of the zero of the ring with respect to the equivalence relation defined by an ideal I will be denoted 0_I .

```

definition (in ring0) ideal_rzero (0_) where
  0_I  $\equiv$  QuotientGroupRel(R,A,I){0}

```

Similarly the class of the neutral element of multiplication in the ring with respect to the equivalence relation defined by an ideal I will be denoted 1_I .

definition (in ring0) ideal_rone (1_) where
 $1_I \equiv \text{QuotientGroupRel}(R, A, I)\{1\}$

The class of the sum of two units of the ring will be denoted 2_I .

definition (in ring0) ideal_rtwo (2_) where
 $2_I \equiv \text{QuotientGroupRel}(R, A, I)\{2\}$

The value of the projection of the ring multiplication onto the the quotient space defined by an ideal I on a pair of the same classes $\langle x, x \rangle$ is denoted x^{2I} .

definition (in ring0) ideal_rsqr (_^2-) where
 $x^{2I} \equiv \text{ProjFun2}(R, \text{QuotientGroupRel}(R, A, I), M)\langle x, x \rangle$

The class of the additive neutral element of the ring (i.e. 0) with respect to the equivalence relation defined by an ideal is the neutral of the projected addition.

lemma (in ring0) neutral_quotient:
 assumes $I \triangleleft R$
 shows
 $\text{QuotientGroupRel}(R, A, I)\{0\} = \text{TheNeutralElement}(\text{QuotientBy}(I), \text{QuotientGroupOp}(R, A, I))$
<proof>

Similarly, the class of the multiplicative neutral element of the ring (i.e. 1) with respect to the equivalence relation defined by an ideal is the neutral of the projected multiplication.

lemma (in ring0) one_quotient:
 assumes $I \triangleleft R$
 defines $r \equiv \text{QuotientGroupRel}(R, A, I)$
 shows $r\{1\} = \text{TheNeutralElement}(\text{QuotientBy}(I), \text{ProjFun2}(R, r, M))$
<proof>

The class of 2 (i.e. $1 + 1$) is the same as the value of the addition projected on the quotient space on the pair of classes of 1.

lemma (in ring0) two_quotient:
 assumes $I \triangleleft R$
 defines $r \equiv \text{QuotientGroupRel}(R, A, I)$
 shows $r\{2\} = \text{QuotientGroupOp}(R, A, I)\langle r\{1\}, r\{1\} \rangle$
<proof>

The class of a square of an element of the ring is the same as the result of the projected multiplication on the pair of classes of the element.

lemma (in ring0) sqrt_quotient:
 assumes $I \triangleleft R$ $x \in R$
 defines $r \equiv \text{QuotientGroupRel}(R, A, I)$

```

shows  $r\{x^2\} = \text{ProjFun2}(R, r, M) \langle r\{x\}, r\{x\} \rangle$ 
 $\langle \text{proof} \rangle$ 

```

The projection of the ring addition is distributive with respect to the projection of the ring multiplication.

```

lemma (in ring0) quotientBy_distributive:
  assumes  $I \triangleleft R$ 
  defines  $r \equiv \text{QuotientGroupRel}(R, A, I)$ 
  shows
     $\text{IsDistributive}(\text{QuotientBy}(I), \text{QuotientGroupOp}(R, A, I), \text{ProjFun2}(R, r, M))$ 
   $\langle \text{proof} \rangle$ 

```

The quotient group is a ring with the quotient multiplication.

```

theorem (in ring0) quotientBy_is_ring:
  assumes  $I \triangleleft R$ 
  defines  $r \equiv \text{QuotientGroupRel}(R, A, I)$ 
  shows  $\text{IsAring}(\text{QuotientBy}(I), \text{QuotientGroupOp}(R, A, I), \text{ProjFun2}(R, r, M))$ 
   $\langle \text{proof} \rangle$ 

```

An important property satisfied by many important rings is being Noetherian: every ideal is finitely generated.

```

definition (in ring0) isFinGen ( $\_ \{ \text{is finitely generated} \}$ ) where
   $I \triangleleft R \implies I \{ \text{is finitely generated} \} \equiv \exists S \in \text{FinPow}(R). I = \langle S \rangle_I$ 

```

For Noetherian rings the arbitrary sum can be reduced to the sum of a finite subset of the initial set of ideals

```

theorem (in ring0) sum_ideals_noetherian:
  assumes  $\forall I \in \mathcal{I}. (I \{ \text{is finitely generated} \}) \mathcal{J} \subseteq \mathcal{I}$ 
  shows  $\exists \mathcal{T} \in \text{FinPow}(\mathcal{J}). (\oplus_I \mathcal{J}) = (\oplus_I \mathcal{T})$ 
   $\langle \text{proof} \rangle$ 

```

end

50 Rings - Ideals of quotient rings

```

theory Ring_ZF_3 imports Ring_ZF_2 Group_ZF_6

```

begin

This section studies the ideals of quotient rings, and defines ring homomorphisms.

50.1 Ring homomorphisms

Morphisms in general are structure preserving functions between algebraic structures. In this section we study ring homomorphisms.

A ring homomorphism is a function between rings which has the morphism property with respect to both addition and multiplication operation, and maps one (the neutral element of multiplication) in the first ring to one in the second ring.

definition

$$\text{IsRingHomomor}(f, R, A, M, S, U, V) \equiv f: R \rightarrow S \wedge \text{IsMorphism}(R, A, U, f) \wedge \text{IsMorphism}(R, M, V, f) \\ \wedge f(\text{TheNeutralElement}(R, M)) = \text{TheNeutralElement}(S, V)$$

The next locale defines notation which we will use in this theory. We assume that we have two rings, one (which we will call the origin ring) defined by the triple (R, A, M) and the second one (which we will call the target ring) by the triple (S, U, V) , and a homomorphism $f: R \rightarrow S$.

```
locale ring_homo =
  fixes R A M S U V f
  assumes origin: IsAring(R,A,M)
    and target: IsAring(S,U,V)
    and homomorphism: IsRingHomomor(f,R,A,M,S,U,V)

  fixes ringa (infixl +R 90)
  defines ringa_def [simp]: x+Ry ≡ A⟨x,y⟩

  fixes ringminus (-R _ 89)
  defines ringminus_def [simp]: (-Rx) ≡ GroupInv(R,A)(x)

  fixes ringsub (infixl -R 90)
  defines ringsub_def [simp]: x-Ry ≡ x+R(-Ry)

  fixes ringm (infixl ·R 95)
  defines ringm_def [simp]: x·Ry ≡ M⟨x,y⟩

  fixes ringzero (0R)
  defines ringzero_def [simp]: 0R ≡ TheNeutralElement(R,A)

  fixes ringone (1R)
  defines ringone_def [simp]: 1R ≡ TheNeutralElement(R,M)

  fixes ringtwo (2R)
  defines ringtwo_def [simp]: 2R ≡ 1R+R1R

  fixes ringsq (_2R [96] 97)
  defines ringsq_def [simp]: x2R ≡ x·Rx

  fixes ringas (infixl +S 90)
  defines ringas_def [simp]: x+Sb ≡ U⟨x,b⟩

  fixes ringminuss (-S _ 89)
  defines ringminuss_def [simp]: (-Sx) ≡ GroupInv(S,U)(x)
```

```

fixes ringsubs (infixl  $-_S$  90)
defines ringsubs_def [simp]:  $x-_S b \equiv x+_S (-_S b)$ 

fixes ringms (infixl  $\cdot_S$  95)
defines ringms_def [simp]:  $x\cdot_S b \equiv V\langle x, b \rangle$ 

fixes ringzeros ( $0_S$ )
defines ringzeros_def [simp]:  $0_S \equiv \text{TheNeutralElement}(S, U)$ 

fixes ringones ( $1_S$ )
defines ringones_def [simp]:  $1_S \equiv \text{TheNeutralElement}(S, V)$ 

fixes ringtwos ( $2_S$ )
defines ringtwos_def [simp]:  $2_S \equiv 1_S+_S 1_S$ 

fixes ringsqs ( $\_{{}^2}^S$  [96] 97)
defines ringsqs_def [simp]:  $x^{{}^2}^S \equiv x\cdot_S x$ 

fixes rlistsum ( $\sum \_$  70)
defines rlistsum_def [simp]:  $\sum s \equiv \text{Fold}(A, 0_R, s)$ 

fixes rnat_mult (infix  $\cdot$  95)
defines nat_mult_def [simp]:  $n\cdot x \equiv \sum \{\langle k, x \rangle. k \in n\}$ 

fixes listprod ( $\prod \_$  70)
defines listprod_def [simp]:  $\prod s \equiv \text{Fold}(M, 1_R, s)$ 

fixes pow
defines pow_def [simp]:  $\text{pow}(n, x) \equiv \prod \{\langle k, x \rangle. k \in n\}$ 

```

We will write $I\triangleleft R_o$ to denote that I is an ideal of the ring R . Note that in this notation the R_o part by itself has no meaning, only the whole $\triangleleft R_o$ serves as postfix operator.

abbreviation (in ring_homo) ideal_origin ($_ \triangleleft R_o$)
 where $I\triangleleft R_o \equiv \text{ring0.Ideal}(R, A, M, I)$

$I\triangleleft R_t$ means that I is an ideal of S .

abbreviation (in ring_homo) ideal_target ($_ \triangleleft R_t$)
 where $I\triangleleft R_t \equiv \text{ring0.Ideal}(S, U, V, I)$

$I\triangleleft_p R_o$ means that I is a prime ideal of R .

abbreviation (in ring_homo) prime_ideal_origin ($_ \triangleleft_p R_o$)
 where $I\triangleleft_p R_o \equiv \text{ring0.primeIdeal}(R, A, M, I)$

We will write $I\triangleleft_p R_t$ to denote that I is a prime ideal of the ring S .

abbreviation (in ring_homo) prime_ideal_target ($_ \triangleleft_p R_t$)
 where $I\triangleleft_p R_t \equiv \text{ring0.primeIdeal}(S, U, V, I)$

`ker` denotes the kernel of f (which is assumed to be a homomorphism between R and S).

abbreviation (in `ring_homo`) `kernel (ker 90)` where
 $\text{ker} \equiv f - \{0_S\}$

The theorems proven in the `ring0` context are valid in the `ring_homo` context when applied to the ring R .

sublocale `ring_homo < origin_ring:ring0`
 $\langle \text{proof} \rangle$

The group homomorphism locale applies to the additive part of a ring homomorphism: (R, A) is the origin group and (S, U) is the target group.

sublocale `ring_homo < add_group_homo: group_homo R A S U f`
`ringzero ringa ringminus rlistsum rnat_mult`
`ringzeros ringas ringminuss`
 $\lambda s. \text{Fold}(U, 0_S, s)$
 $\lambda n x. \text{Fold}(U, 0_S, \{ \langle k, x \rangle. k \in n \})$
 $\langle \text{proof} \rangle$

The theorems proven in the `ring0` context are valid in the `ring_homo` context when applied to the ring S .

sublocale `ring_homo < target_ring:ring0 S U V ringas ringminuss`
`ringsubs ringms ringzeros ringones ringtwos ringsqs`
 $\lambda s. \text{Fold}(U, 0_S, s)$
 $\lambda n x. \text{Fold}(U, 0_S, \{ \langle k, x \rangle. k \in n \})$
 $\lambda s. \text{Fold}(V, 1_S, s)$
 $\lambda n x. \text{Fold}(V, 1_S, \{ \langle k, x \rangle. k \in n \})$
 $\langle \text{proof} \rangle$

A ring homomorphism is a homomorphism both with respect to addition and multiplication.

lemma `ringHomHom: assumes IsRingHomomor(f,R,A,M,S,U,V)`
`shows Homomor(f,R,A,S,U) and Homomor(f,R,M,S,V)`
 $\langle \text{proof} \rangle$

Since in the `ring_homo` locale f is a ring homomorphism it implies that f is a function from R to S .

lemma (in `ring_homo`) `f_is_fun: shows f:R→S`
 $\langle \text{proof} \rangle$

In the `ring_homo` context A is the addition in the first (source) ring M is the multiplication there and U, V are the addition and multiplication resp. in the second (target) ring. The next lemma states the all these are binary operations, a trivial, but frequently used fact.

lemma (in `ring_homo`) `AMUV_are_ops:`
`shows A:R×R→R M:R×R→R U:S×S→S V:S×S→S`

⟨proof⟩

The kernel is a subset of R on which the value of f is zero (of the target ring)

lemma (in ring_homo) kernel_def_alt: shows $\ker = \{r \in R. f(r) = 0_S\}$
⟨proof⟩

the homomorphism f maps each element of the kernel to zero of the target ring.

lemma (in ring_homo) image_kernel:
 assumes $x \in \ker$
 shows $f(x) = 0_S$
⟨proof⟩

As a ring homomorphism f preserves multiplication.

lemma (in ring_homo) homomor_dest_mult:
 assumes $x \in R \ y \in R$
 shows $f(x \cdot_R y) = (f(x)) \cdot_S (f(y))$
⟨proof⟩

As a ring homomorphism f preserves addition.

lemma (in ring_homo) homomor_dest_add:
 assumes $x \in R \ y \in R$
 shows $f(x +_R y) = (f(x)) +_S (f(y))$
⟨proof⟩

For $x \in R$ the value of f is in S .

lemma (in ring_homo) homomor_val: **assumes** $x \in R$
 shows $f(x) \in S$
⟨proof⟩

A ring homomorphism preserves taking negative of an element.

lemma (in ring_homo) homomor_dest_minus:
 assumes $x \in R$
 shows $f(-_R x) = -_S (f(x))$
⟨proof⟩

A ring homomorphism preserves subtraction.

lemma (in ring_homo) homomor_dest_subs:
 assumes $x \in R \ y \in R$
 shows $f(x -_R y) = (f(x)) -_S (f(y))$
⟨proof⟩

A ring homomorphism maps zero to zero.

lemma (in ring_homo) homomor_dest_zero:
 shows $f(0_R) = 0_S$
⟨proof⟩

The kernel of a homomorphism is never empty.

lemma (in ring_homo) kernel_non_empty:
 shows $0_R \in \ker$ and $\ker \neq \emptyset$
<proof>

The image of the kernel by f is the singleton $\{0_S\}$.

corollary (in ring_homo) image_kernel_2: shows $f(\ker) = \{0_S\}$
<proof>

The inverse image of an ideal (in the target ring) is a normal subgroup of the addition group and an ideal in the origin ring. The kernel of the homomorphism is a subset of the inverse of image of every ideal.

lemma (in ring_homo) preimage_ideal:
 assumes $J \triangleleft R_t$
 shows
 $\text{IsANormalSubgroup}(R, A, f^{-1}(J))$
 $(f^{-1}(J)) \triangleleft_{R_o} \ker \subseteq f^{-1}(J)$
<proof>

Kernel of the homomorphism is an ideal.

lemma (in ring_homo) kernel_ideal: shows $\ker \triangleleft_{R_o}$
<proof>

The inverse image of a prime ideal by a homomorphism is not the whole ring. Proof by contradiction.

lemma (in ring_homo) vimage_prime_ideal_not_all:
 assumes $J \triangleleft_p R_t$ shows $f^{-1}(J) \neq R$
<proof>

Even more, if the target ring of the homomorphism is commutative and the ideal is prime then its preimage is also. Note that this is not true in general.

lemma (in ring_homo) preimage_prime_ideal_comm:
 assumes $J \triangleleft_p R_t$ $\vee$ {is commutative on} S
 shows $(f^{-1}(J)) \triangleleft_p R_o$
<proof>

We can replace the assumption that the target ring of the homomorphism is commutative with the assumption that homomorphism is surjective in preimage_prime_ideal_comm above and we can show the same assertion that the preimage of a prime ideal is prime.

lemma (in ring_homo) preimage_prime_ideal_surj:
 assumes $J \triangleleft_p R_t$ $f \in \text{surj}(R, S)$
 shows $(f^{-1}(J)) \triangleleft_p R_o$
<proof>

50.2 Quotient ring with quotient map

The notion of a quotient ring (a.k.a factor ring, difference ring or residue class) is analogous to the notion of quotient group from the group theory.

The next locale `ring2` extends the `ring0` locale (defined in the `Ring_ZF` theory) with the assumption that some fixed set I is an ideal. It also defines some notation related to quotient rings, in particular we define the function (projection) f_I that maps each element r of the ring R to its class $r_I(\{r\}$ where r_I is the quotient group relation defined by I as a (normal) subgroup of R with addition.

```

locale ring2 = ring0 +
  fixes I
  assumes idealAssum:  $I \triangleleft R$ 

  fixes quot ( $R_I$ )
  defines quot_def [simp]:  $R_I \equiv \text{QuotientBy}(I)$ 

  fixes qrel ( $r_I$ )
  defines qrel_def [simp]:  $r_I \equiv \text{QuotientGroupRel}(R, A, I)$ 

  fixes qfun ( $f_I$ )
  defines qfun_def [simp]:  $f_I \equiv \lambda r \in R. r_I\{r\}$ 

  fixes qadd ( $A_I$ )
  defines qadd_def [simp]:  $A_I \equiv \text{QuotientGroupOp}(R, A, I)$ 

  fixes qmul ( $M_I$ )
  defines qmul_def [simp]:  $M_I \equiv \text{ProjFun2}(R, qrel, M)$ 

```

The expression $J \triangleleft R_I$ will mean that J is an ideal of the quotient ring R_I (with the quotient addition and multiplication).

abbreviation (in `ring2`) `qideal ( $\_ \triangleleft R_I$ )` **where**
 $J \triangleleft R_I \equiv \text{ring0.Ideal}(R_I, A_I, M_I, J)$

In the `ring2` The expression $J \triangleleft_p R_I$ means that J is a prime ideal of the quotient ring R_I .

abbreviation (in `ring2`) `qprimeIdeal ( $\_ \triangleleft_p R_I$ )` **where**
 $J \triangleleft_p R_I \equiv \text{ring0.primeIdeal}(R_I, A_I, M_I, J)$

Theorems proven in the `ring0` context can be applied to the quotient ring in the `ring2` context.

```

sublocale ring2 < quotient_ring: ring0 quot qadd qmul
   $\lambda x y. \text{ideal\_radd}(x, I, y) \ \lambda y. \text{ideal\_rmin}(I, y)$ 
   $\lambda x y. \text{ideal\_rsub}(x, I, y) \ \lambda x y. \text{ideal\_rmult}(x, I, y)$ 
   $0_I \ 1_I \ 2_I \ \lambda x. (x^{2I})$ 
   $\lambda s. \text{Fold}(A_I, 0_I, s)$ 

```

```

λn x. Fold(AI, 0I, {⟨k, x⟩ . k ∈ n})
λs. Fold(MI, 1I, s)
λn x. Fold(MI, 1I, {⟨k, x⟩ . k ∈ n})
⟨proof⟩

```

The quotient map is a homomorphism of rings. This is probably one of the most sophisticated facts in IsarMathlib that Isabelle's simp method proves from 10 facts and 5 definitions.

```

theorem (in ring2) quotient_fun_homomor:
  shows IsRingHomomor(fI, R, A, M, RI, AI, MI)
  ⟨proof⟩

```

The quotient map is surjective

```

lemma (in ring2) quot_fun:
  shows fI ∈ surj(R, RI)
  ⟨proof⟩

```

The theorems proven in the `ring_homo` context are valid in the `ring_homo` context when applied to the quotient ring as the second (target) ring and the quotient map as the ring homomorphism.

```

sublocale ring2 < quot_homomorphism: ring_homo R A M quot qadd qmul qfun
  _ _ _ _ _ λx y. ideal_radd(x, I, y) λy. ideal_rmin(I, y)
  λx y. ideal_rsub(x, I, y) λx y. ideal_rmult(x, I, y)
  0I 1I 2I λx. (x2I)
  ⟨proof⟩

```

The ideal we divide by is the kernel of the quotient map.

```

lemma (in ring2) quotient_kernel:
  shows quot_homomorphism.kernel = I
  ⟨proof⟩

```

If an ideal I is a subset of the kernel of the homomorphism then the image of the ideal generated by $I \cup J$, where J is another ideal, is the same as the image of J . Note that $J+_II$ notation means the ideal generated by the union of ideals J and I , see the definitions of `sumIdeal` and `generatedIdeal` in the `Ring_ZF_2` theory, and also corollary `sum_ideals_is_sum_elements` for an alternative definition.

```

theorem (in ring_homo) kernel_empty_image:
  assumes J < R I ⊆ ker I < R
  shows f(J+_II) = f(J) f(I+_IJ) = f(J)
  ⟨proof⟩

```

50.3 Quotient ideals

If we have an ideal J in a ring R , and another ideal I contained in J , then we can form the quotient ideal J/I whose elements are of the form $a + I$ where a is an element of J .

The preimage of an ideal is an ideal, so it applies to the quotient map; but the preimage ideal contains the quotient ideal.

```
lemma (in ring2) ideal_quot_preimage:
  assumes  $J \triangleleft R_I$ 
  shows  $(f_I(J)) \triangleleft R \text{ } I \subseteq f_I(J)$ 
<proof>
```

Since the map is surjective, the image is also an ideal

```
lemma (in ring_homo) image_ideal_surj:
  assumes  $J \triangleleft R_o$   $f \in \text{surj}(R, S)$ 
  shows  $(f(J)) \triangleleft R_t$ 
<proof>
```

If the homomorphism is a surjection and given two ideals in the target ring the inverse image of their product ideal is the sum ideal of the product ideal of their inverse images and the kernel of the homomorphism.

```
corollary (in ring_homo) prime_ideal_quot:
  assumes  $J \triangleleft R_t$   $K \triangleleft R_t$   $f \in \text{surj}(R, S)$ 
  shows  $f(\text{target\_ring.productIdeal}(J, K)) =$ 
     $\text{origin\_ring.sumIdeal}(\text{origin\_ring.productIdeal}((f(J)), (f(K))), \text{ker})$ 
<proof>
```

If the homomorphism is surjective then the product ideal of ideals J, K in the target ring is the image of the product ideal (in the source ring) of the inverse images of J, K .

```
corollary (in ring_homo) prime_ideal_quot_2:
  assumes  $J \triangleleft R_t$   $K \triangleleft R_t$   $f \in \text{surj}(R, S)$ 
  shows  $\text{target\_ring.productIdeal}(J, K) =$ 
     $f(\text{origin\_ring.productIdeal}((f(J)), (f(K))))$ 
<proof>
```

If the homomorphism is surjective and an ideal in the source ring contains the kernel, then the image of that ideal is a prime ideal in the target ring.

```
lemma (in ring_homo) preimage_ideal_prime:
  assumes  $J \triangleleft_p R_o$   $\text{ker} \subseteq J$   $f \in \text{surj}(R, S)$ 
  shows  $(f(J)) \triangleleft_p R_t$ 
<proof>
```

The ideals of the quotient ring are in bijection with the ideals of the original ring that contain the ideal by which we made the quotient.

```
theorem (in ring_homo) ideal_quot_bijection:
  assumes  $f \in \text{surj}(R, S)$ 
  defines  $\text{idealFun} \equiv \lambda J \in \text{target\_ring.ideals}. f(J)$ 
  shows  $\text{idealFun} \in \text{bij}(\text{target\_ring.ideals}, \{K \in \mathcal{I}. \text{ker} \subseteq K\})$ 
<proof>
```

Assume the homomorphism f is surjective and consider the function that maps an ideal J in the target ring to its inverse image $f^{-1}(J)$ (in the source

ring). Then the value of the converse of that function on any ideal containing the kernel of f is the image of that ideal under the homomorphism f .

```
theorem (in ring_homo) quot_converse:
  defines F  $\equiv \lambda J \in \text{target\_ring.ideals. } f\text{-}(J)$ 
  assumes  $J \triangleleft R \text{ } \ker \subseteq J \text{ } f \in \text{surj}(R, S)$ 
  shows  $\text{converse}(F)(J) = f(J)$ 
  <proof>
```

Since the map is surjective, this bijection restricts to prime ideals on both sides.

```
corollary (in ring_homo) prime_ideal_quot_3:
  assumes  $K \triangleleft_p R \text{ } f \in \text{surj}(R, S)$ 
  shows  $K \triangleleft_p R \longleftrightarrow ((f\text{-}(K)) \triangleleft_p R)$ 
  <proof>
```

If the homomorphism is surjective then the function that maps ideals in the target ring to their inverse images (in the source ring) is a bijection between prime ideals in the target ring and the prime ideals containing the kernel in the source ring.

```
corollary (in ring_homo) bij_prime_ideals:
  defines F  $\equiv \lambda J \in \text{target\_ring.ideals. } f\text{-}(J)$ 
  assumes  $f \in \text{surj}(R, S)$ 
  shows  $\text{restrict}(F, \{J \in \text{Pow}(S). J \triangleleft_p R\}) \in$ 
     $\text{bij}(\{J \in \text{Pow}(S). J \triangleleft_p R\}, \{J \in \text{Pow}(R). \ker \subseteq J \wedge (J \triangleleft_p R)\})$ 
  <proof>
```

50.4 Subrings

A subring of a ring (R, A, M) is a subset S which forms a ring under the operations restricted from (R, A, M) .

We define $\text{IsAsubring}(S, A, M)$ to mean that S is a subring of a ring with addition A and multiplication M .

```
definition (in ring0) IsAsubring where
   $S \subseteq R \implies \text{IsAsubring}(S) \equiv \text{IsAring}(S, \text{restrict}(A, S \times S), \text{restrict}(M, S \times S))$ 
```

Every ring is a subring of itself.

```
lemma (in ring0) ring_self_subring:
  shows  $\text{IsAsubring}(R)$ 
  <proof>
```

The image of a ring under a ring homomorphism is a subring of the target ring.

```
lemma (in ring_homo) image_is_subring:
  shows  $\text{target\_ring.IsAsubring}(f(R))$ 
  <proof>
```

The converse of a bijective ring homomorphism is also a ring homomorphism.

```
lemma (in ring_homo) bij_ring_homo_converse:
  assumes bij: f ∈ bij(R,S)
  shows IsRingHomomor(converse(f), S, U, V, R, A, M)
<proof>
```

50.5 First isomorphism theorem

The first isomorphism theorem for rings: any ring homomorphism $f : R \rightarrow S$ induces a ring isomorphism $R/\ker(f) \cong f(R)$. The isomorphism sends each coset $[r]_{\ker(f)}$ to $f(r)$.

If $f : R \rightarrow S$ is a surjective ring homomorphism, then there exists a ring isomorphism $\phi : R/\ker(f) \rightarrow S$.

```
theorem (in ring_homo) first_isomorphism_theorem:
  assumes surj: f ∈ surj(R,S)
  shows ∃φ. IsRingHomomor(φ, origin_ring.QuotientBy(ker),
    QuotientGroupOp(R,A,ker),
    ProjFun2(R,QuotientGroupRel(R,A,ker),M),
    S, U, V) ∧ φ ∈ bij(origin_ring.QuotientBy(ker), S)
<proof>
```

end

51 Rings - Commutative Rings

```
theory Ring_ZF_4 imports Ring_ZF_2 CommutativeSemigroup_ZF
```

begin

This theory develops the theory of commutative rings. We define the `commutative_ring` locale, characterize principal ideals, and study the relationship between prime rings, prime ideals, and the absence of zero divisors.

51.1 Commutative rings

A commutative ring is a ring in which multiplication satisfies the commutativity law. We establish that principal ideals take a simple form in this setting.

A commutative ring is defined as a `ring0` in which multiplication is commutative.

```
locale commutative_ring = ring0 +
  assumes commutative: M{is commutative on}R
```

For any element x of a commutative ring, the set $\{x \cdot y \mid y \in R\}$ is an ideal of the ring.


```

lemma (in commutative_ring) mult_by_elem:
  assumes  $x \in R$ 
  shows  $\{x \cdot y. y \in R\} \triangleleft R$ 
<proof>

```

In a commutative ring the principal ideal generated by an element x equals $\{x \cdot y \mid y \in R\}$.

```

theorem (in commutative_ring) principal_ideal:
  assumes  $x \in R$ 
  shows  $\langle \{x\} \rangle_I = \{x \cdot y. y \in R\}$ 
<proof>

```

51.2 Prime ideals

We characterize prime rings in terms of zero divisors and show that the quotient of a ring by a prime ideal is a prime ring, or equivalently, has no zero divisors in the commutative case.

Commutative prime rings are the same as commutative rings with no zero divisors.

```

lemma (in commutative_ring) prime_ring_zero_divs_1:
  assumes  $[R, A, M] \{\text{is a prime ring}\}$ 
  shows HasNoZeroDivs(R, A, M)
<proof>

```

A commutative ring with no zero divisors is a prime ring.

```

lemma (in commutative_ring) prime_ring_zero_divs_2:
  assumes HasNoZeroDivs(R, A, M)
  shows  $[R, A, M] \{\text{is a prime ring}\}$ 
<proof>

```

The quotient ring of a ring by a prime ideal is a prime ring.

```

theorem (in ring0) prime_ideal_no_zero_divs:
  assumes  $I \triangleleft_p R$ 
  shows  $[\text{QuotientBy}(I), \text{QuotientGroupOp}(R, A, I), \text{ProjFun2}(R, \text{QuotientGroupRel}(R, A, I), M)] \{\text{is a prime ring}\}$ 
<proof>

```

In a commutative ring, the quotient by a prime ideal has no zero divisors. This is a specialization of `prime_ideal_no_zero_divs` using commutativity to convert the prime ring condition directly into the no-zero-divisors property.

```

theorem (in commutative_ring) comm_prime_ideal_no_zero_divs:
  assumes  $I \triangleleft_p R$ 
  shows HasNoZeroDivs(QuotientBy(I), QuotientGroupOp(R, A, I), ProjFun2(R, QuotientGroupRel(R, A, I), M))
<proof>

```

end

52 Rings - Direct Product and Chinese Remainder Theorem

```
theory Ring_ZF_5 imports Ring_ZF_4 Ring_ZF_3 DirectProduct_ZF
```

```
begin
```

This section develops the direct product of two rings and proves the Chinese Remainder Theorem: for comaximal ideals I and J in a commutative ring R (meaning $I + J = R$), there is a ring isomorphism $R/(I \cap J) \cong R/I \times R/J$.

52.1 Direct product of rings

The direct product of two rings is a ring with componentwise operations.

```
lemma direct_product_ring:
  assumes ringR: IsAring(R,A,M) and ringS: IsAring(S,U,V)
  shows IsAring(R×S, DirectProduct(A,U,R,S), DirectProduct(M,V,R,S))
<proof>
```

The additive neutral element of the direct product ring is the pair of additive neutral elements.

```
lemma direct_product_neutral:
  assumes ringR: IsAring(R,A,M) and ringS: IsAring(S,U,V)
  shows TheNeutralElement(R×S, DirectProduct(A,U,R,S)) =
    <TheNeutralElement(R,A), TheNeutralElement(S,U)>
<proof>
```

The multiplicative neutral element of the direct product ring is the pair of multiplicative neutral elements.

```
lemma direct_product_neutral_mult:
  assumes ringR: IsAring(R,A,M) and ringS: IsAring(S,U,V)
  shows TheNeutralElement(R×S, DirectProduct(M,V,R,S)) =
    <TheNeutralElement(R,M), TheNeutralElement(S,V)>
<proof>
```

52.2 Chinese Remainder Theorem

The Chinese Remainder Theorem: if I and J are comaximal ideals of a commutative ring R (i.e. $I + J = R$), then $R/(I \cap J) \cong R/I \times R/J$ as rings. The proof constructs the canonical map $\varphi(r) = ([r]_I, [r]_J)$, shows it is a surjective ring homomorphism with kernel $I \cap J$, and applies the first isomorphism theorem.

```
theorem (in commutative_ring) chinese_remainder:
  assumes idealI: I<R and idealJ: J<R
  and comaximal: I+J = R
  shows ∃φ. IsRingHomomor(φ,
```

```

    QuotientBy(I∩J),
    QuotientGroupOp(R,A,I∩J),
    ProjFun2(R,QuotientGroupRel(R,A,I∩J),M),
    QuotientBy(I)×QuotientBy(J),
    DirectProduct(QuotientGroupOp(R,A,I),QuotientGroupOp(R,A,J),
                  QuotientBy(I),QuotientBy(J)),
    DirectProduct(ProjFun2(R,QuotientGroupRel(R,A,I),M),
                  ProjFun2(R,QuotientGroupRel(R,A,J),M),
                  QuotientBy(I),QuotientBy(J)))
  ∧ φ ∈ bij(QuotientBy(I∩J),
             QuotientBy(I)×QuotientBy(J))
<proof>
end

```

53 Fields - introduction

theory Field_ZF imports Ring_ZF

begin

This theory covers basic facts about fields.

53.1 Definition and basic properties

In this section we define what is a field and list the basic properties of fields.

Field is a nontrivial commutative ring such that all non-zero elements have an inverse. We define the notion of being a field as a statement about three sets. The first set, denoted K is the carrier of the field. The second set, denoted A represents the additive operation on K (recall that in ZF set theory functions are sets). The third set M represents the multiplicative operation on K .

definition

```

IsAfield(K,A,M) ≡
  (IsARing(K,A,M) ∧ (M {is commutative on} K) ∧
   TheNeutralElement(K,A) ≠ TheNeutralElement(K,M) ∧
   (∀ a∈K. a≠TheNeutralElement(K,A) →
    (∃ b∈K. M⟨a,b⟩ = TheNeutralElement(K,M))))

```

The `field0` context extends the `ring0` context adding field-related assumptions and notation related to the multiplicative inverse.

```

locale field0 = ring0 K A M for K A M +
  assumes mult_commute: M {is commutative on} K

  assumes not_triv: 0 ≠ 1

  assumes inv_exists: ∀ x∈K. x≠0 → (∃ y∈K. x·y = 1)

```

```

fixes non_zero (K0)
defines non_zero_def[simp]: K0 ≡ K-{0}

fixes inv (_-1 [96] 97)
defines inv_def[simp]: a-1 ≡ GroupInv(K0,restrict(M,K0×K0))(a)

```

The next lemma assures us that we are talking fields in the `field0` context.

```

lemma (in field0) Field_ZF_1_L1: shows IsAfield(K,A,M)
  ⟨proof⟩

```

We can use theorems proven in the `field0` context whenever we talk about a field.

```

lemma field_field0: assumes IsAfield(K,A,M)
shows field0(K,A,M)
  ⟨proof⟩

```

Let's have an explicit statement that the multiplication in fields is commutative.

```

lemma (in field0) field_mult_comm: assumes a∈K b∈K
shows a·b = b·a
  ⟨proof⟩

```

Fields do not have zero divisors.

```

lemma (in field0) field_has_no_zero_divs: shows HasNoZeroDivs(K,A,M)
  ⟨proof⟩

```

K_0 (the set of nonzero field elements is closed with respect to multiplication.

```

lemma (in field0) Field_ZF_1_L2:
shows K0 {is closed under} M
  ⟨proof⟩

```

Any nonzero element has a right inverse that is nonzero.

```

lemma (in field0) Field_ZF_1_L3: assumes A1: a∈K0
shows ∃b∈K0. a·b = 1
  ⟨proof⟩

```

If we remove zero, the field with multiplication becomes a group and we can use all theorems proven in `group0` context.

```

theorem (in field0) Field_ZF_1_L4: shows
  IsAgroup(K0,restrict(M,K0×K0))
  group0(K0,restrict(M,K0×K0))
  1 = TheNeutralElement(K0,restrict(M,K0×K0))
  ⟨proof⟩

```

The inverse of a nonzero field element is nonzero.

```

lemma (in field0) Field_ZF_1_L5: assumes A1: a∈K a≠0

```

shows $a^{-1} \in K_0 \quad (a^{-1})^2 \in K_0 \quad a^{-1} \in K \quad a^{-1} \neq 0$
<proof>

The inverse is really the inverse.

lemma (in field0) Field_ZF_1_L6: **assumes** A1: $a \in K \quad a \neq 0$
shows $a \cdot a^{-1} = 1 \quad a^{-1} \cdot a = 1$
<proof>

A lemma with two field elements and cancelling.

lemma (in field0) Field_ZF_1_L7: **assumes** $a \in K \quad b \in K \quad b \neq 0$
shows
 $a \cdot b \cdot b^{-1} = a$
 $a \cdot b^{-1} \cdot b = a$
<proof>

53.2 Equations and identities

This section deals with more specialized identities that are true in fields.

$$a/(a^2) = 1/a .$$

lemma (in field0) Field_ZF_2_L1: **assumes** A1: $a \in K \quad a \neq 0$
shows $a \cdot (a^{-1})^2 = a^{-1}$
<proof>

If we multiply two different numbers by a nonzero number, the results will be different.

lemma (in field0) Field_ZF_2_L2:
assumes $a \in K \quad b \in K \quad c \in K \quad a \neq b \quad c \neq 0$
shows $a \cdot c^{-1} \neq b \cdot c^{-1}$
<proof>

We can put a nonzero factor on the other side of non-identity (is this the best way to call it?) changing it to the inverse.

lemma (in field0) Field_ZF_2_L3:
assumes A1: $a \in K \quad b \in K \quad b \neq 0 \quad c \in K$ **and** A2: $a \cdot b \neq c$
shows $a \neq c \cdot b^{-1}$
<proof>

If the inverse of b is different than a , then the inverse of a is different than b .

lemma (in field0) Field_ZF_2_L4:
assumes $a \in K \quad a \neq 0$ **and** $b^{-1} \neq a$
shows $a^{-1} \neq b$
<proof>

An identity with two field elements, one and an inverse.

lemma (in field0) Field_ZF_2_L5:

```

assumes a∈K  b∈K  b≠0
shows (1 + a·b)·b-1 = a + b-1
⟨proof⟩

```

An identity with three field elements, inverse and cancelling.

```

lemma (in field0) Field_ZF_2_L6: assumes A1: a∈K  b∈K  b≠0  c∈K
shows a·b·(c·b-1) = a·c
⟨proof⟩

```

Inverse of an inverse of a non-zero element is the element.

```

lemma (in field0) non_zero_inv_inv: assumes a∈K  a≠0
shows (a-1)-1 = a
⟨proof⟩

```

53.3 1/0=0

In ZF if $f : X \rightarrow Y$ and $x \notin X$ we have $f(x) = \emptyset$. Since $\emptyset$ (the empty set) in ZF is the same as zero of natural numbers we can claim that $1/0 = 0$ in certain sense. In this section we prove a theorem that makes it explicit.

The next locale extends the `field0` locale to introduce notation for division operation.

```

locale fieldd = field0 +
  fixes division
  defines division_def[simp]: division ≡ {⟨p,fst(p)·snd(p)-1⟩. p∈K×K0}

  fixes fdiv (infixl / 95)
  defines fdiv_def[simp]: x/y ≡ division⟨x,y⟩

```

Division is a function on $K \times K_0$ with values in K .

```

lemma (in fieldd) div_fun: shows division: K×K0 → K
⟨proof⟩

```

So, really $1/0 = 0$. The essential lemma is `apply_0` from standard Isabelle's `func.thy`.

```

theorem (in fieldd) one_over_zero: shows 1/0 = 0
⟨proof⟩

```

end

54 Modules

```

theory Module_ZF imports Ring_ZF_3 Field_ZF Group_ZF_5

```

```

begin

```

A module is a generalization of the concept of a vector space in which scalars do not form a field but a ring.

54.1 Definition and basic properties of modules

Let R be a ring and M be an abelian group. The most common definition of a left R -module posits the existence of a scalar multiplication operation $R \times M \rightarrow M$ satisfying certain four properties. Here we take a bit more concise and abstract approach defining a module as a ring action on an abelian group.

We know that endomorphisms of an abelian group $\mathcal{M}$ form a ring with pointwise addition as the additive operation and composition as the ring multiplication. This assertion is a bit imprecise though as pointwise addition is a binary operation on the space of functions $\mathcal{M} \rightarrow \mathcal{M}$ (i.e. its domain is $(\mathcal{M} \rightarrow \mathcal{M}) \times (\mathcal{M} \rightarrow \mathcal{M})$) while we need the space of endomorphisms to be the domain of the ring addition and multiplication. Therefore, to get the actual additive operation we need to restrict the pointwise addition of functions $\mathcal{M} \rightarrow \mathcal{M}$ to the set of endomorphisms of $\mathcal{M}$. Recall from the `Group_ZF_5` that the `InEnd` operator restricts an operation to the set of endomorphisms and see the `func_ZF` theory for definitions of lifting an operation on a set to a function space over that set.

definition `EndAdd( $\mathcal{M}, A$ )` $\equiv$ `InEnd( $A$  {lifted to function space over}  $\mathcal{M}, \mathcal{M}, A$ )`

Similarly we define the multiplication in the ring of endomorphisms as the restriction of compositions to the endomorphisms of $\mathcal{M}$. See the `func_ZF` theory for the definition of the `Composition` operator.

definition `EndMult( $\mathcal{M}, A$ )` $\equiv$ `InEnd( $Composition(\mathcal{M}), \mathcal{M}, A$ )`

We can now reformulate the theorem `end_is_ring` from the `Group_ZF_5` theory in terms of the addition and multiplication of endomorphisms defined above.

lemma `(in abelian_group) end_is_ring1:`
`shows IsAring( $End(G, P), EndAdd(G, P), EndMult(G, P)$ )`
 `$\langle proof \rangle$`

We define an **action** as a homomorphism into a space of endomorphisms (typically of some abelian group). In the definition below S is the set of scalars, A is the addition operation on this set, M is multiplication on the set, $\mathcal{M}$ is the group, A_M is the group operation, and H is the ring homomorphism of the ring of scalars to the ring of endomorphisms of the group. On the right hand side of the definition `End( $\mathcal{M}, A_V$ )` is the set of endomorphisms of the group $\mathcal{M}$ with operation A_M . This definition is only ever used as part of the definition of a module and vector space, it's just convenient to split it off to shorten the main definitions.

definition

$$\text{IsAction}(S, A, M, \mathcal{M}, A_M, H) \equiv \text{IsRingHomomor}(H, S, A, M, \text{End}(\mathcal{M}, A_M), \text{EndAdd}(\mathcal{M}, A_M), \text{EndMult}(\mathcal{M}, A_M))$$

A module is a ring action on an abelian group.

definition $\text{IsLeftModule}(S, A, M, \mathcal{M}, A_M, H) \equiv$

$$\text{IsAring}(S, A, M) \wedge \text{IsAgroup}(\mathcal{M}, A_M) \wedge (A_M \text{ \{is commutative on\} } \mathcal{M}) \wedge \text{IsAction}(S, A, M, \mathcal{M}, A_M, H)$$

If H defines a module then it is a ring action, hence a ring homomorphism, hence a function on that ring.

lemma `module_action_type`: **assumes** $\text{IsLeftModule}(S, A, M, \mathcal{M}, A_M, H)$
shows

$$\text{IsAction}(S, A, M, \mathcal{M}, A_M, H)$$

$$\text{IsRingHomomor}(H, S, A, M, \text{End}(\mathcal{M}, A_M), \text{EndAdd}(\mathcal{M}, A_M), \text{EndMult}(\mathcal{M}, A_M))$$

$$H: S \rightarrow \text{End}(\mathcal{M}, A_M)$$

<proof>

The next locale defines context (i.e. common assumptions and notation) when considering modules. We reuse notation from the `ring0` locale and add notation specific to modules. The addition and multiplication in the ring of scalars is denoted $+$ and $\cdot$, resp. The addition of module elements will be denoted $+_V$. The multiplication (scaling) of scalars by module elements will be denoted $\cdot_S$. Θ is the zero module element, i.e. the neutral element of the abelian group of the module elements.

locale `module0` = `ring0` +

fixes $\mathcal{M} \ A_M \ H$

assumes `mAbGr`: $\text{IsAgroup}(\mathcal{M}, A_M) \wedge (A_M \text{ \{is commutative on\} } \mathcal{M})$

assumes `mAction`: $\text{IsAction}(R, A, M, \mathcal{M}, A_M, H)$

fixes `zero_vec` (Θ)

defines `zero_vec_def` [`simp`]: $\Theta \equiv \text{TheNeutralElement}(\mathcal{M}, A_M)$

fixes `vAdd` (**infixl** $+_V$ 80)

defines `vAdd_def` [`simp`]: $v_1 +_V v_2 \equiv A_M \langle v_1, v_2 \rangle$

fixes `scal` (**infix** $\cdot_S$ 90)

defines `scal_def` [`simp`]: $s \cdot_S v \equiv (H(s))(v)$

fixes `negV` ($-_V$)

defines `negV_def` [`simp`]: $-v \equiv \text{GroupInv}(\mathcal{M}, A_M)(v)$

fixes `vSub` (**infix** $-_V$ 80)

defines `vSub_def` [`simp`]: $v_1 -_V v_2 \equiv v_1 +_V (-v_2)$

We indeed talk about modules in the `module0` context.


```

lemma (in module0) module_in_module0: shows IsLeftModule(R,A,M, $\mathcal{M}$ , $A_M$ ,H)
  <proof>

```

Theorems proven in the `abelian_group` context are valid as applied to the `module0` context as applied to the abelian group of module elements.

```

lemma (in module0) abelian_group_valid_module0:
  shows abelian_group( $\mathcal{M}$ , $A_M$ )
  <proof>

```

Another way to state that theorems proven in the `abelian_group` context can be used in the `module0` context:

```

sublocale module0 < mod_ab_gr: abelian_group  $\mathcal{M}$   $A_M$   $\Theta$  vAdd negV
   $\lambda s$ . Fold( $A_M$ ,  $\Theta$ , s)  $\lambda n$  x. Fold( $A_M$ ,  $\Theta$ , {<k, x> . k  $\in$  n})
  <proof>

```

Theorems proven in the `ring_homo` context are valid in the `module0` context, as applied to ring R and the ring of endomorphisms of the group of module elements.

```

lemma (in module0) ring_homo_valid_module0:
  shows ring_homo(R,A,M,End( $\mathcal{M}$ , $A_M$ ),EndAdd( $\mathcal{M}$ , $A_M$ ),EndMult( $\mathcal{M}$ , $A_M$ ),H)
  <proof>

```

Another way to make theorems proven in the `ring_homo` context available in the `module0` context:

```

sublocale module0 < vec_act_homo: ring_homo R A M
  End( $\mathcal{M}$ , $A_M$ ) EndAdd( $\mathcal{M}$ , $A_M$ ) EndMult( $\mathcal{M}$ , $A_M$ ) H
  ringa
  ringminus
  ringsub
  ringm
  ringzero
  ringone
  ringtwo
  ringsq
   $\lambda x$  y. EndAdd( $\mathcal{M}$ , $A_M$ ) <x, y>
   $\lambda x$ . GroupInv(End( $\mathcal{M}$ , $A_M$ ), EndAdd( $\mathcal{M}$ ,  $A_M$ ))(x)
   $\lambda x$  y. EndAdd( $\mathcal{M}$ , $A_M$ )<x,GroupInv(End( $\mathcal{M}$ ,  $A_M$ ), EndAdd( $\mathcal{M}$ ,  $A_M$ ))(y)>
   $\lambda x$  y. EndMult( $\mathcal{M}$ , $A_M$ )<x, y>
  TheNeutralElement(End( $\mathcal{M}$ ,  $A_M$ ), EndAdd( $\mathcal{M}$ ,  $A_M$ ))
  TheNeutralElement(End( $\mathcal{M}$ ,  $A_M$ ), EndMult( $\mathcal{M}$ ,  $A_M$ ))
  EndAdd( $\mathcal{M}$ , $A_M$ )<TheNeutralElement(End( $\mathcal{M}$ ,  $A_M$ ),EndMult( $\mathcal{M}$ ,  $A_M$ )),TheNeutralElement(End( $\mathcal{M}$ ,
 $A_M$ ), EndMult( $\mathcal{M}$ , $A_M$ ))>
   $\lambda x$ . EndMult( $\mathcal{M}$ , $A_M$ )<x, x>
  <proof>

```

In the ring of endomorphisms of the module the neutral element of the the multiplicative operation is the identity function. The neutral element of the

additive operation is the zero valued constant function, which is also the value of the homomorphism that defines the module at zero.

lemma (in module0) add_mult_neut_elems: shows
 TheNeutralElement(End($\mathcal{M}$, A_M), EndMult($\mathcal{M}, A_M$)) = id($\mathcal{M}$) and
 TheNeutralElement(End($\mathcal{M}, A_M$), EndAdd($\mathcal{M}, A_M$)) = ConstantFunction($\mathcal{M}, \Theta$)
 H(0) = ConstantFunction($\mathcal{M}, \Theta$)
<proof>

The value of the homomorphism defining the module is an endomorphism of the group of module elements and hence a function that maps the module into itself.

lemma (in module0) H_val_type: assumes $r \in R$ shows
 $H(r) \in \text{End}(\mathcal{M}, A_M)$ and $H(r) : \mathcal{M} \rightarrow \mathcal{M}$
<proof>

In the module0 context the neutral element of addition of module elements is denoted Θ . Of course Θ is an element of the module.

lemma (in module0) zero_in_mod: shows $\Theta \in \mathcal{M}$
<proof>

Θ is indeed the neutral element of addition of module elements.

lemma (in module0) zero_neutral: assumes $x \in \mathcal{M}$
 shows $x +_V \Theta = x$ and $\Theta +_V x = x$
<proof>

54.2 Module axioms

A more common definition of a module assumes that R is a ring, V is an abelian group and lists a couple of properties that the multiplications of scalars (elements of R) by the elements of the module V should have. In this section we show that the definition of a module as a ring action on an abelian group V implies these properties.

Θ is fixed by scalar multiplication.

lemma (in module0) zero_fixed: assumes $r \in R$
 shows $r \cdot_S \Theta = \Theta$
<proof>

The scalar multiplication is distributive with respect to the module addition.

lemma (in module0) module_ax1: assumes $r \in R$ $x \in \mathcal{M}$ $y \in \mathcal{M}$
 shows $r \cdot_S (x +_V y) = r \cdot_S x +_V r \cdot_S y$
<proof>

The scalar addition is distributive with respect to scalar multiplication.

lemma (in module0) module_ax2: assumes $r \in R$ $s \in R$ $x \in \mathcal{M}$
 shows $(r+s) \cdot_S x = r \cdot_S x +_V s \cdot_S x$

⟨proof⟩

Multiplication by scalars is associative with multiplication of scalars.

lemma (in module0) module_ax3: **assumes** $r \in R$ $s \in R$ $x \in \mathcal{M}$
shows $(r \cdot s) \cdot_S x = r \cdot_S (s \cdot_S x)$
⟨proof⟩

Scaling a module element by one gives the same module element.

lemma (in module0) module_ax4: **assumes** $x \in \mathcal{M}$ **shows** $1 \cdot_S x = x$
⟨proof⟩

Multiplying by zero is zero.

lemma (in module0) mult_zero:
assumes $g \in \mathcal{M}$ **shows** $0 \cdot_S g = \Theta$
⟨proof⟩

Taking inverses in a module is just multiplying by -1 .

lemma (in module0) inv_module:
assumes $g \in \mathcal{M}$
shows $(-1) \cdot_S g = -g$
⟨proof⟩

end

54.3 Linear Combinations on Modules

theory Module_ZF_1 **imports** Module_ZF CommutativeSemigroup_ZF

begin

Since modules are abelian groups, we can make use of its commutativity to create new elements by adding acted on elements finitely. Consider two ordered collections of ring elements and of group elements (indexed by a finite set); then we can add their actions to obtain a new group element. This is a linear combination.

definition(in module0)
 LinearComb ($\sum [_; \{_, _\}]$ 88)
where $fR: C \rightarrow R \implies fG: C \rightarrow \mathcal{M} \implies D \in \text{FinPow}(C) \implies \text{LinearComb}(D, fR, fG)$
 $\equiv$ if $D \neq \emptyset$ then $\text{CommSetFold}(A_M, \{\langle m, (fRm) \cdot_S (fGm) \rangle. m \in \text{domain}(fR)\}, D)$
 else Θ

The function that for each index element gives us the acted element of the abelian group is a function from the index to the group.

lemma(in module0) coordinate_function:
assumes $AA: C \rightarrow R$ $B: C \rightarrow \mathcal{M}$
shows $\{\langle m, (AAm) \cdot_S (Bm) \rangle. m \in C\}: C \rightarrow \mathcal{M}$
⟨proof⟩

A linear combination results in a group element where the functions and the sets are well defined.

```
theorem(in module0) linComb_is_in_module:
  assumes AA:C→R B:C→M D∈FinPow(C)
  shows (∑ [D;{AA,B}])∈M
  ⟨proof⟩
```

A linear combination of one element functions is just the action of one element onto another.

```
lemma(in module0) linComb_one_element:
  assumes x∈X AA:X→R B:X→M
  shows ∑ [{x};{AA,B}]= (AAx)·S (Bx)
  ⟨proof⟩
```

Since a linear combination is a group element, it makes sense to apply the action onto it. With this result we simplify it to a linear combination.

```
lemma(in module0) linComb_action:
  assumes AA:X→R B:X→M r∈R D∈FinPow(X)
  shows r·S (∑ [D;{AA,B}])=∑ [D;{⟨k,r·(AAk)⟩. k∈X},B}]
  and {⟨m,r·(AAm)⟩. m∈X}:X→R
  ⟨proof⟩
```

A linear combination can always be defined on a cardinal.

```
lemma(in module0) linComb_reorder_terms1:
  assumes AA:X→R B:X→M D∈FinPow(X) g∈bij(|D|,D)
  shows (∑ [D;{AA,B}])=∑ [|D|;{AA 0 g,B 0 g}]
  ⟨proof⟩
```

Actually a linear combination can be defined over any bijective set with the original set.

```
lemma(in module0) linComb_reorder_terms2:
  assumes AA:X→R B:X→M D∈FinPow(X) g∈bij(E,D)
  shows (∑ [D;{AA,B}])=∑ [E;{AA 0 g,B 0 g}]
  ⟨proof⟩
```

Restricting the defining functions to the domain set does nothing to the linear combination

```
corollary(in module0) linComb_restrict_coord:
  assumes AA:X→R B:X→M D∈FinPow(X)
  shows (∑ [D;{AA,B}])=∑ [D;{restrict(AA,D),restrict(B,D)}]
  ⟨proof⟩
```

A linear combination can be defined with a natural number and functions with that number as domain.

```
corollary(in module0) linComb_nat:
  assumes AA:X→R B:X→M D∈FinPow(X)
```

shows $\exists n \in \text{nat}. \exists A1 \in n \rightarrow R. \exists B1 \in n \rightarrow \mathcal{M}. \sum [D; \{AA, B\}] = \sum [n; \{A1, B1\}] \wedge A1n = AAD$
 $\wedge B1n = BD$
<proof>

54.3.1 Adding linear combinations

Adding a linear combination defined over $\emptyset$ leaves it as is

lemma(in module0) linComb_sum_base_induct1:
assumes $AA: X \rightarrow R \ B: X \rightarrow \mathcal{M} \ D \in \text{FinPow}(X) \ AA1: Y \rightarrow R \ B1: Y \rightarrow \mathcal{M}$
shows $(\sum [D; \{AA, B\}]) +_V (\sum [0; \{AA1, B1\}]) = \sum [D; \{AA, B\}]$
<proof>

Applying a product of $1 \times$ to the defining set computes the same linear combination; since they are bijective sets

lemma(in module0) linComb_sum_base_induct2:
assumes $AA: X \rightarrow R \ B: X \rightarrow \mathcal{M} \ D \in \text{FinPow}(X)$
shows $(\sum [D; \{AA, B\}]) = \sum [\{0\} \times D; \{\langle \langle 0, x \rangle, AAx \rangle. x \in X\}, \{\langle \langle 0, x \rangle, Bx \rangle. x \in X\}]]$
and
 $(\sum [D; \{AA, B\}]) = \sum [\{0\} \times D; \{\text{restrict}(\{\langle \langle 0, x \rangle, AAx \rangle. x \in X\}, \{0\} \times D), \text{restrict}(\{\langle \langle 0, x \rangle, Bx \rangle. x \in X\}, \{0\} \times D)\}]$
<proof>

Then, we can model adding a liner combination on the empty set as a linear combination of the disjoint union of sets

lemma(in module0) linComb_sum_base_induct:
assumes $AA: X \rightarrow R \ B: X \rightarrow \mathcal{M} \ D \in \text{FinPow}(X) \ AA1: Y \rightarrow R \ B1: Y \rightarrow \mathcal{M}$
shows $(\sum [D; \{AA, B\}]) +_V (\sum [0; \{AA1, B1\}]) = \sum [D + 0; \{\langle \langle 0, x \rangle, AAx \rangle. x \in X\} \cup \{\langle \langle 1, x \rangle, AA1x \rangle. x \in Y\}, \{\langle \langle 0, x \rangle, Bx \rangle. x \in X\} \cup \{\langle \langle 1, x \rangle, B1x \rangle. x \in Y\}]]$
<proof>

An element of the set for the linear combination can be removed and add it using group addition.

lemma(in module0) sum_one_element:
assumes $AA: X \rightarrow R \ B: X \rightarrow \mathcal{M} \ D \in \text{FinPow}(X) \ t \in D$
shows $(\sum [D; \{AA, B\}]) = (\sum [D - \{t\}; \{AA, B\}]) +_V (\{\langle k, (AAk) \cdot_S (Bk) \rangle. k \in X\}t)$
<proof>

A small technical lemma to proof by induction on finite sets that the addition of linear combinations is a linear combination

lemma(in module0) linComb_sum_ind_step:
assumes $AA: X \rightarrow R \ B: X \rightarrow \mathcal{M} \ D \in \text{FinPow}(X) \ E \in \text{FinPow}(Y) \ AA1: Y \rightarrow R \ B1: Y \rightarrow \mathcal{M} \ t \in E$
 $D \neq 0$
 $(\sum [D; \{AA, B\}]) +_V (\sum [E - \{t\}; \{AA1, B1\}]) = \sum [D + (E - \{t\}); \{\langle \langle 0, x \rangle, AAx \rangle. x \in X\} \cup \{\langle \langle 1, x \rangle, AA1x \rangle. x \in Y\}, \{\langle \langle 0, x \rangle, Bx \rangle. x \in X\} \cup \{\langle \langle 1, x \rangle, B1x \rangle. x \in Y\}]]$
shows $(\sum [D; \{AA, B\}]) +_V (\sum [E; \{AA1, B1\}]) = \sum [D + E; \{\langle \langle 0, x \rangle, AAx \rangle. x \in X\} \cup \{\langle \langle 1, x \rangle, AA1x \rangle. x \in Y\}, \{\langle \langle 0, x \rangle, Bx \rangle. x \in X\} \cup \{\langle \langle 1, x \rangle, B1x \rangle. x \in Y\}]]$
<proof>

The addition of two linear combinations is a linear combination

```

theorem(in module0) linComb_sum:
  assumes AA:X→R AA1:Y→R B:X→M B1:Y→M D≠0 D∈FinPow(X) E∈FinPow(Y)
  shows (∑ [D;{AA,B}]) +V (∑ [E;{AA1,B1}]) = ∑ [D+E; {⟨0,x⟩, AAx}. x∈X ∪ {⟨1,x⟩, AA1x}.
x∈Y}, {⟨0,x⟩, Bx}. x∈X ∪ {⟨1,x⟩, B1x}. x∈Y}]
⟨proof⟩

```

54.3.2 Linear dependency

Now, we have the conditions to define what linear independence means:

```

definition(in module0)
  LinInde (_{is linearly independent} 89)
  where  $\mathcal{T} \subseteq \mathcal{M} \implies \mathcal{T}\{\text{is linearly independent}\} \equiv (\forall X \in \text{nat}. \forall AA \in X \rightarrow R. \forall B \in \text{inj}(X, \mathcal{T}).$ 
 $((\sum [X; \{AA, B\}] = \Theta) \implies (\forall m \in X. AAm = 0))$ 

```

If a set has the zero element, then it is not linearly independent.

```

theorem(in module0) zero_set_dependent:
  assumes  $\Theta \in T \ T \subseteq \mathcal{M} \ R \neq \{0\}$ 
  shows  $\neg(T\{\text{is linearly independent}\})$ 
⟨proof⟩

```

54.4 Submodule

A submodule is a subgroup that is invariant by the action

```

definition(in module0)
  IsAsubmodule
  where  $\text{IsAsubmodule}(\mathcal{N}) \equiv (\forall r \in R. \forall h \in \mathcal{N}. r \cdot_S h \in \mathcal{N}) \wedge \text{IsAsubgroup}(\mathcal{N}, A_M)$ 

```

```

lemma(in module0) submodule_is_subgroup:
  assumes  $\text{IsAsubmodule}(\mathcal{N})$ 
  shows  $\text{IsAsubgroup}(\mathcal{N}, A_M)$ 
⟨proof⟩

```

```

lemma(in module0) submodule_is_subaction:
  assumes  $\text{IsAsubmodule}(\mathcal{N}) \ r \in R \ h \in \mathcal{N}$ 
  shows  $r \cdot_S h \in \mathcal{N}$ 
⟨proof⟩

```

For groups, we need to prove that the inverse function is closed in a set to prove that set to be a subgroup. In module, that is not necessary.

```

lemma(in module0) inverse_in_set:
  assumes  $\forall r \in R. \forall h \in \mathcal{N}. r \cdot_S h \in \mathcal{N} \ \mathcal{N} \subseteq \mathcal{M}$ 
  shows  $\forall h \in \mathcal{N}. (-h) \in \mathcal{N}$ 
⟨proof⟩

```

```

corollary(in module0) submoduleI:
  assumes  $\mathcal{N} \subseteq \mathcal{M} \ \mathcal{N} \neq \emptyset \ \mathcal{N}\{\text{is closed under}\}A_M \ \forall r \in R. \forall h \in \mathcal{N}. r \cdot_S h \in \mathcal{N}$ 

```

shows IsSubmodule($\mathcal{N}$) *<proof>*

Every module has at least two submodules: the whole module and the trivial module.

corollary(in module0) trivial_submodules:
shows IsSubmodule($\mathcal{M}$) **and** IsSubmodule($\{\Theta\}$)
<proof>

The restriction of the action is an action.

lemma(in module0) action_submodule:
assumes IsSubmodule($\mathcal{N}$)
shows $\{\langle r, \text{restrict}(Hr, \mathcal{N}) \rangle. r \in R\} : R \rightarrow \text{End}(\mathcal{N}, \text{restrict}(A_M, \mathcal{N} \times \mathcal{N}))$
<proof>

A submodule is a module with the restricted action.

corollary(in module0) submodule:
assumes IsSubmodule($\mathcal{N}$)
shows IsLeftModule($R, A, M, \mathcal{N}, \text{restrict}(A_M, \mathcal{N} \times \mathcal{N}), \{\langle r, \text{restrict}(Hr, \mathcal{N}) \rangle. r \in R\}$)
<proof>

If we consider linear combinations of elements in a submodule, then the linear combination is also in the submodule.

lemma(in module0) linear_comb_submod:
assumes IsSubmodule($\mathcal{N}$) $D \in \text{FinPow}(X)$ $AA : X \rightarrow R$ $B : X \rightarrow \mathcal{N}$
shows $\sum [D; \{AA, B\}] \in \mathcal{N}$
<proof>

54.4.1 Spans

Since we know linear combinations, we can define the span of a subset of a module as the linear combinations of elements in that subset. We have already proven that the sum can be done only over finite numbers considering a bijection between a finite number and the original finite set, and that the function can be restricted to that finite number.

The terms of a linear combination can be reordered so that they are indexed by the elements of the module.

lemma(in module0) index_module:
assumes $AAA : X \rightarrow R$ $BB : X \rightarrow \mathcal{M}$ $D \in \text{FinPow}(X)$
shows $\exists AA \in \mathcal{M} \rightarrow R. \sum [D; \{AAA, BB\}] = \sum [BBD; \{AA, \text{id}(\mathcal{M})\}] \wedge (\forall x \in \mathcal{M} - BBD. AAx = 0)$
<proof>

A span over a set is the collection over all linear combinations on those elements.

definition(in module0)

```

Span({span of}_)
where  $T \subseteq \mathcal{M} \implies \{\text{span of}\}T \equiv \text{if } T=0 \text{ then } \{\emptyset\} \text{ else } \{\sum [F; \{\text{AA}, \text{id}(T)\}]\}.$ 
 $\langle F, \text{AA} \rangle \in \{ \langle FF, B \rangle \in \text{FinPow}(T) \times (T \rightarrow R) . \forall m \in T - FF. Bm = 0 \}$ 

```

The span of a subset is then a submodule and contains the original set.

```

theorem(in module0) linear_ind_set_comb_submodule:
  assumes  $T \subseteq \mathcal{M}$ 
  shows IsAsubmodule({span of}T)
  and  $T \subseteq \{\text{span of}\}T$ 
<proof>

```

Given a linear combination, it is in the span of the image of the second function.

```

lemma (in module0) linear_comb_span:
  assumes  $\text{AA}: X \rightarrow R \text{ } B: X \rightarrow \mathcal{M} \text{ } D \in \text{FinPow}(X)$ 
  shows  $\sum [D; \{\text{AA}, B\}] \in (\{\text{span of}\}(BD))$ 
<proof>

```

It turns out that the span is the smallest submodule that contains the original set.

```

theorem(in module0) minimal_submodule:
  assumes  $T \subseteq \mathcal{N} \text{ } \text{IsAsubmodule}(\mathcal{N})$ 
  shows  $(\{\text{span of}\}T) \subseteq \mathcal{N}$ 
<proof>

```

end

55 Ideals as modules connecting ring and module theories

```

theory Module_ZF_2 imports Module_ZF_1 Ring_ZF_2

```

```

begin

```

The most basic examples of modules are subsets of the ring itself: since a ring is an abelian group under addition, the ring acts on itself and on its ideals. This file bridges the theory of rings developed in `Ring_ZF` with the module theory of `Module_ZF_1` by showing that every ideal is a submodule of the ring regarded as a module over itself.

55.1 Ideals as Modules

Let's show first that the ring acting on itself is a module; and then we will show that ideals are submodules.

The map that takes every element to its left multiplication map, is a map to endomorphisms.


```

lemma (in ring0) action_regular_map:
  shows {⟨r, {⟨s, r·s⟩. s ∈ R}⟩. r ∈ R} : R → End(R, A) <proof>

```

The previous map respects addition because of distribution

```

lemma (in ring0) action_regular_distrib:
  assumes g1 ∈ R g2 ∈ R
  shows {⟨xa, M ⟨(A ⟨g1, g2)⟩, xa⟩⟩ . xa ∈ R} =
    EndAdd(R, A) ⟨{⟨xa, M ⟨g1, xa⟩⟩ . xa ∈ R}, {⟨xa, M ⟨g2, xa⟩⟩ .
  xa ∈ R}⟩
<proof>

```

The previous map respects multiplication because of associativity

```

lemma (in ring0) action_regular_assoc:
  assumes g1 ∈ R g2 ∈ R
  shows {⟨xa, M ⟨(M ⟨g1, g2)⟩, xa⟩⟩ . xa ∈ R} =
    EndMult(R, A) ⟨{⟨xa, M ⟨g1, xa⟩⟩ . xa ∈ R}, {⟨xa, M ⟨g2, xa⟩⟩
  . xa ∈ R}⟩
<proof>

```

The previous map takes the unit element to the identity map

```

lemma (in ring0) action_regular_neut:
  shows {⟨x, {⟨xa, M ⟨x, xa⟩⟩ . xa ∈ R}⟩ . x ∈ R} 1 = id(R)
<proof>

```

The previous map is an action

```

theorem (in ring0) action_regular:
  shows IsAction(R, A, M, R, A, {⟨r, {⟨s, r·s⟩. s ∈ R}⟩. r ∈ R}) <proof>

```

The action defines the Regular Module

```

theorem (in ring0) reg_module:
  shows module0(R, A, M, R, A, {⟨x, {⟨xa, M ⟨x, xa⟩⟩ . xa ∈ R}⟩ . x ∈ R}) <proof>

```

Every ideal is a submodule of this regular action.

```

corollary (in ring0) ideal_submodule:
  assumes I < R
  shows module0.IsASubmodule(R, A, {⟨x, {⟨xa, M ⟨x, xa⟩⟩ . xa ∈ R}⟩ . x
  ∈ R}, I)
<proof>

```

55.2 Annihilators

An annihilator of a module subset is the set of elements of the ring whose action on that module subset is 0.

```

definition (in module0) ann where
  N ⊆ M ⇒ ann(N) ≡ {r ∈ R. ∀ n ∈ N. r·sn = 0}

```

If the subset is a submodule, then the annihilator is an ideal.

```

lemma (in module0) ann_ideal:
  assumes IsAsubmodule(N)
  shows ann(N) <R <proof>

```

Annihilator is reverse monotonic

```

lemma (in module0) ann_mono:
  assumes  $N \subseteq \mathcal{M}$   $K \subseteq N$ 
  shows  $\text{ann}(N) \subseteq \text{ann}(K)$ 
<proof>

```

If the ring is commutative, the annihilator of a subset shrinks to the annihilator of the generated submodule

```

lemma (in module0) comm_ann_of_ideal:
  assumes  $N \subseteq \mathcal{M}$   $M$  {is commutative on}  $R$ 
  shows  $\text{ann}(N) = \text{ann}(\{\text{span of}\}N)$ 
<proof>

```

Annihilators on commutative rings are ideals

```

corollary (in module0) comm_ann_ideal:
  assumes  $N \subseteq \mathcal{M}$   $M$  {is commutative on}  $R$ 
  shows  $\text{ann}(N) <R$  <proof>

```

end

56 Vector spaces

```

theory VectorSpace_ZF imports Module_ZF

```

begin

Vector spaces have a long history of applications in mathematics and physics. To this collection of applications a new one has been added recently - Large Language Models. It turned out that representing words, phrases and documents as vectors in a high-dimensional vector space provides an effective way to capture semantic relationships and emulate contextual understanding. This theory has nothing to do with LLM's however - it just defines vector space as a mathematical structure as it has been understood from at least the beginning of the XXth century.

56.1 Definition and basic properties of vector spaces

The canonical example of a vector space is $\mathbb{R}^n$ - the set of n -tuples of real numbers. We can add them adding respective coordinates and scale them by multiplying all coordinates by the same number. In a more abstract approach we start with an abelian group (of vectors) and a field (of scalars) and define an operation of multiplying a vector by a scalar so that the

distributive properties $x(v_1 + v_2) = sv_1 + sv_2$ and $(s_1 + s_2)v = s_1v + s_2v$ are satisfied for any scalars s, s_1, s_2 and vectors v, v_1, v_2 .

A vector space is a field action on an abelian group. The notion of an action is defined in `Module_ZF` theory as a ring homomorphism valuesd in the ring of endomorphisms of some (abelian) group. In the definition S is a the ring carrier, A is the set representing the addition operation of the ring, M is the ring multiplication, V is the carrier of the abelian group, A_V represents the group operation, i.e. the vector addition and H is the ring homomorphism defining the action.

definition `IsVectorSpace(S,A,M,V,A_V,H) ≡`
`IsAfield(S,A,M) ∧ IsAgroup(V,A_V) ∧ (A_V {is commutative on} V) ∧ IsAction(S,A,M,V,A_V,H)`

The next locale defines context (i.e. common assumptions and notation) when considering vector spaces. We reuse notation from the `field0` locale adding more similarly to the `module0` locale.

```
locale vector_space0 = field0 +
  fixes V A_V H

  assumes mAbGr: IsAgroup(V,A_V) ∧ (A_V {is commutative on} V)

  assumes mAction: IsAction(K,A,M,V,A_V,H)

  fixes zero_vec (Θ)
  defines zero_vec_def [simp]: Θ ≡ TheNeutralElement(V,A_V)

  fixes vAdd (infixl +_V 80)
  defines vAdd_def [simp]: v1 +V v2 ≡ A_V⟨v1,v2⟩

  fixes scal (infix ·S 90)
  defines scal_def [simp]: s ·S v ≡ (H(s))(v)

  fixes negV (−)
  defines negV_def [simp]: −v ≡ GroupInv(V,A_V)(v)

  fixes vSub (infix −V 80)
  defines vSub_def [simp]: v1 −V v2 ≡ v1 +V (−v2)
```

We indeed talk about vector spaces in the `vector_space0` context.

lemma `(in vector_space0) V_vec_space: shows IsVectorSpace(K,A,M,V,A_V,H)`
`⟨proof⟩`

If a quintuple of sets forms a vector space then the assumptions of the `vector_spce0` hold for those sets.

lemma `vec_spce_vec_spce_ctxt: assumes IsVectorSpace(K,A,M,V,A_V,H)`
`shows vector_space0(K, A, M, V, A_V, H)`
`⟨proof⟩`

The assumptions of `module0` context hold in the `vector_spce0` context.

lemma (in `vector_space0`) `vec_spce_mod`: shows `module0(K, A, M, V, AV, H)`
<proof>

Propositions proven in the `module0` context are valid in the `vector_spce0` context.

sublocale `vector_space0 < vspce_mod`: `module0 K A M`
`ringa ringminus ringsub ringm ringzero ringone ringtwo ringsq`
`λs. Fold(A, 0, s)`
`λ n x. Fold(A, 0, {⟨k, x⟩. k ∈ n})`
`λs. Fold(M, 1, s)`
`λ n x. Fold(M, 1, {⟨k, x⟩. k ∈ n})`
`V AV`
<proof>

56.2 Vector space axioms

In this section we show that the definition of a vector space as a field action on an abelian group implies the vector space axioms as listed on Wikipedia (March 2024). The first four axioms just state that vectors with addition form an abelian group. That is fine of course, but in such case the axioms for scalars being a field should be listed too, and they are not. The entry on modules is more consistent, it states that module elements form an abelian group, scalars form a ring and lists only four properties of multiplication of scalars by vectors as module axioms. The remaining four axioms are just restatement of module axioms and since vector spaces are modules we can prove them by referring to the module axioms proven in the `module0` context

Vector addition is associative.

lemma (in `vector_space0`) `vec_spce_ax1`: assumes `u ∈ V v ∈ V w ∈ V`
 shows `u +V (v +V w) = (u +V v) +V w`
<proof>

Vector addition is commutative.

lemma (in `vector_space0`) `vec_spce_ax2`: assumes `u ∈ V v ∈ V`
 shows `u +V v = v +V u`
<proof>

The zero vector is a vector.

lemma (in `vector_space0`) `vec_spce_ax3a`: shows `0 ∈ V`
<proof>

The zero vector is the neutral element of addition of vectors.

lemma (in `vector_space0`) `vec_spce_ax3b`: assumes `v ∈ V` shows `v +V 0 = v`

<proof>

The additive inverse of a vector is a vector.

lemma (in vector_space0) vec_spce_ax4a: assumes $v \in V$ shows $(-v) \in V$
<proof>

Sum of a vector and its additive inverse is the zero vector.

lemma (in vector_space0) vec_spce_ax4b: assumes $v \in V$
shows $v +_V (-v) = \Theta$
<proof>

Scalar multiplication and field multiplication are "compatible" (as Wikipedia calls it).

lemma (in vector_space0) vec_spce_ax5: assumes $x \in K$ $y \in K$ $v \in V$
shows $x \cdot_S (y \cdot_S v) = (x \cdot y) \cdot_S v$
<proof>

Multiplying the identity element of the field by a vector gives the vector.

lemma (in vector_space0) vec_spce_ax6: assumes $v \in V$ shows $1 \cdot_S v = v$
<proof>

Scalar multiplication is distributive with respect to vector addition.

lemma (in vector_space0) vec_spce_ax7: assumes $x \in K$ $u \in V$ $v \in V$
shows $x \cdot_S (u +_V v) = x \cdot_S u +_V x \cdot_S v$
<proof>

Scalar multiplication is distributive with respect to field addition.

lemma (in vector_space0) vec_spce_ax8: assumes $x \in K$ $y \in K$ $v \in V$
shows $(x + y) \cdot_S v = x \cdot_S v +_V y \cdot_S v$
<proof>

end

57 Ordered fields

theory OrderedField_ZF imports OrderedRing_ZF Field_ZF

begin

This theory covers basic facts about ordered fields.

57.1 Definition and basic properties

Here we define ordered fields and prove their basic properties.

Ordered field is a nontrivial ordered ring such that all non-zero elements have an inverse. We define the notion of being a ordered field as a statement about

four sets. The first set, denoted K is the carrier of the field. The second set, denoted A represents the additive operation on K (recall that in ZF set theory functions are sets). The third set M represents the multiplicative operation on K . The fourth set r is the order relation on K .

definition

$$\begin{aligned} \text{IsAnOrdField}(K,A,M,r) \equiv & (\text{IsAnOrdRing}(K,A,M,r) \wedge \\ & (M \text{ \{is commutative on\} } K) \wedge \\ & \text{TheNeutralElement}(K,A) \neq \text{TheNeutralElement}(K,M) \wedge \\ & (\forall a \in K. a \neq \text{TheNeutralElement}(K,A) \longrightarrow \\ & (\exists b \in K. M(a,b) = \text{TheNeutralElement}(K,M)))) \end{aligned}$$

The next context (`locale`) defines notation used for ordered fields. We do that by extending the notation defined in the `ring1` context that is used for ordered rings and adding some assumptions to make sure we are talking about ordered fields in this context. We should rename the carrier from R used in the `ring1` context to K , more appropriate for fields. Theoretically the Isar `locale` facility supports such renaming, but we experienced difficulties using some lemmas from `ring1` locale after renaming.

locale `field1` = `ring1` +

```

assumes mult_commute:  $M \text{ \{is commutative on\} } R$ 

assumes not_triv:  $0 \neq 1$ 

assumes inv_exists:  $\forall a \in R. a \neq 0 \longrightarrow (\exists b \in R. a \cdot b = 1)$ 

fixes non_zero ( $R_0$ )
defines non_zero_def[simp]:  $R_0 \equiv R \setminus \{0\}$ 

fixes inv ( $_^{-1}$  [96] 97)
defines inv_def[simp]:  $a^{-1} \equiv \text{GroupInv}(R_0, \text{restrict}(M, R_0 \times R_0))(a)$ 

```

The next lemma assures us that we are talking fields in the `field1` context.

lemma (`in field1`) `OrdField_ZF_1_L1`: **shows** `IsAnOrdField`(R,A,M,r)
<proof>

Ordered field is a field, of course.

lemma `OrdField_ZF_1_L1A`: **assumes** `IsAnOrdField`(K,A,M,r)
shows `IsAfield`(K,A,M)
<proof>

Theorems proven in `field0` (about fields) context are valid in the `field1` context (about ordered fields).

lemma (`in field1`) `OrdField_ZF_1_L1B`: **shows** `field0`(R,A,M)
<proof>

We can use theorems proven in the `field1` context whenever we talk about an ordered field.

```

lemma OrdField_ZF_1_L2: assumes IsAnOrdField(K,A,M,r)
  shows field1(K,A,M,r)
  <proof>

```

In ordered rings the existence of a right inverse for all positive elements implies the existence of an inverse for all non zero elements.

```

lemma (in ring1) OrdField_ZF_1_L3:
  assumes A1:  $\forall a \in R_+. \exists b \in R. a \cdot b = 1$  and A2:  $c \in R \quad c \neq 0$ 
  shows  $\exists b \in R. c \cdot b = 1$ 
  <proof>

```

Ordered fields are easier to deal with, because it is sufficient to show the existence of an inverse for the set of positive elements.

```

lemma (in ring1) OrdField_ZF_1_L4:
  assumes  $0 \neq 1$  and M {is commutative on} R
  and  $\forall a \in R_+. \exists b \in R. a \cdot b = 1$ 
  shows IsAnOrdField(R,A,M,r)
  <proof>

```

The set of positive field elements is closed under multiplication.

```

lemma (in field1) OrdField_ZF_1_L5: shows  $R_+$  {is closed under} M
  <proof>

```

The set of positive field elements is closed under multiplication: the explicit version.

```

lemma (in field1) pos_mul_closed:
  assumes A1:  $0 < a \quad 0 < b$ 
  shows  $0 < a \cdot b$ 
  <proof>

```

In fields square of a nonzero element is positive.

```

lemma (in field1) OrdField_ZF_1_L6: assumes  $a \in R \quad a \neq 0$ 
  shows  $a^2 \in R_+$ 
  <proof>

```

The next lemma restates the fact from Field_ZF that our notation for the field inverse means what it is supposed to mean.

```

lemma (in field1) OrdField_ZF_1_L7: assumes  $a \in R \quad a \neq 0$ 
  shows  $a \cdot (a^{-1}) = 1 \quad (a^{-1}) \cdot a = 1$ 
  <proof>

```

A simple lemma about multiplication and cancelling of a positive field element.

```

lemma (in field1) OrdField_ZF_1_L7A:
  assumes A1:  $a \in R \quad b \in R_+$ 
  shows
   $a \cdot b \cdot b^{-1} = a$ 

```

$a \cdot b^{-1} \cdot b = a$
 $\langle proof \rangle$

Some properties of the inverse of a positive element.

lemma (in field1) OrdField_ZF_1_L8: **assumes** A1: $a \in R_+$
shows $a^{-1} \in R_+$ $a \cdot (a^{-1}) = 1$ $(a^{-1}) \cdot a = 1$
 $\langle proof \rangle$

in an ordered field the set of positive elements forms a group under multiplication restricted to it.

lemma (in field1) pos_mul_group:
shows IsAgroup(R_+ , restrict($M, R_+ \times R_+$))
 $\langle proof \rangle$

$\frac{1}{2}$ is a positive member of the field.

lemma (in field1) one_half_pos: **shows** $2^{-1} \in R_+$ $0 < 2^{-1}$
 $\langle proof \rangle$

If a is smaller than b , then $(b - a)^{-1}$ is positive.

lemma (in field1) OrdField_ZF_1_L9: **assumes** $a < b$
shows $(b - a)^{-1} \in R_+$
 $\langle proof \rangle$

In ordered fields if at least one of a, b is not zero, then $a^2 + b^2 > 0$, in particular $a^2 + b^2 \neq 0$ and the (multiplicative) inverse of $a^2 + b^2$ exists.

lemma (in field1) OrdField_ZF_1_L10:
assumes A1: $a \in R$ $b \in R$ **and** A2: $a \neq 0 \vee b \neq 0$
shows $0 < a^2 + b^2$ **and** $\exists c \in R. (a^2 + b^2) \cdot c = 1$
 $\langle proof \rangle$

57.2 Inequalities

In this section we develop tools to deal inequalities in fields.

We can multiply strict inequality by a positive element.

lemma (in field1) OrdField_ZF_2_L1:
assumes $a < b$ **and** $c \in R_+$
shows $a \cdot c < b \cdot c$
 $\langle proof \rangle$

A special case of OrdField_ZF_2_L1 when we multiply an inverse by an element.

lemma (in field1) OrdField_ZF_2_L2:
assumes A1: $a \in R_+$ **and** A2: $a^{-1} < b$
shows $1 < b \cdot a$
 $\langle proof \rangle$

We can multiply an inequality by the inverse of a positive element.

lemma (in field1) OrdField_ZF_2_L3:
 assumes $a \leq b$ and $c \in \mathbb{R}_+$ shows $a \cdot (c^{-1}) \leq b \cdot (c^{-1})$
<proof>

We can multiply a strict inequality by a positive element or its inverse.

lemma (in field1) OrdField_ZF_2_L4:
 assumes $a < b$ and $c \in \mathbb{R}_+$
 shows
 $a \cdot c < b \cdot c$
 $c \cdot a < c \cdot b$
 $a \cdot c^{-1} < b \cdot c^{-1}$
<proof>

We can put a positive factor on the other side of an inequality, changing it to its inverse.

lemma (in field1) OrdField_ZF_2_L5:
 assumes A1: $a \in \mathbb{R}$ $b \in \mathbb{R}_+$ and A2: $a \cdot b \leq c$
 shows $a \leq c \cdot b^{-1}$
<proof>

We can put a positive factor on the other side of an inequality, changing it to its inverse, version with a product initially on the right hand side.

lemma (in field1) OrdField_ZF_2_L5A:
 assumes A1: $b \in \mathbb{R}$ $c \in \mathbb{R}_+$ and A2: $a \leq b \cdot c$
 shows $a \cdot c^{-1} \leq b$
<proof>

We can put a positive factor on the other side of a strict inequality, changing it to its inverse, version with a product initially on the left hand side.

lemma (in field1) OrdField_ZF_2_L6:
 assumes A1: $a \in \mathbb{R}$ $b \in \mathbb{R}_+$ and A2: $a \cdot b < c$
 shows $a < c \cdot b^{-1}$
<proof>

We can put a positive factor on the other side of a strict inequality, changing it to its inverse, version with a product initially on the right hand side.

lemma (in field1) OrdField_ZF_2_L6A:
 assumes A1: $b \in \mathbb{R}$ $c \in \mathbb{R}_+$ and A2: $a < b \cdot c$
 shows $a \cdot c^{-1} < b$
<proof>

Sometimes we can reverse an inequality by taking inverse on both sides.

lemma (in field1) OrdField_ZF_2_L7:
 assumes A1: $a \in \mathbb{R}_+$ and A2: $a^{-1} \leq b$
 shows $b^{-1} \leq a$
<proof>

Sometimes we can reverse a strict inequality by taking inverse on both sides.

```
lemma (in field1) OrdField_ZF_2_L8:
  assumes A1:  $a \in \mathbb{R}_+$  and A2:  $a^{-1} < b$ 
  shows  $b^{-1} < a$ 
  <proof>
```

If the left side of the strict inequality is positive then taking inverses of both sides reverses the inequality.

```
lemma (in field1) poz_elem_inverse_sides: assumes  $0 < a$   $a < b$ 
  shows  $b^{-1} < a^{-1}$ 
  <proof>
```

A technical lemma about solving a strict inequality with three field elements and inverse of a difference.

```
lemma (in field1) OrdField_ZF_2_L9:
  assumes A1:  $a < b$  and A2:  $(b-a)^{-1} < c$ 
  shows  $1 + a \cdot c < b \cdot c$ 
  <proof>
```

One half is field member and sum of two halves is one.

```
lemma (in field1) half_half_one: shows  $2^{-1} \in \mathbb{R}$   $2^{-1} + 2^{-1} = 1$ 
  <proof>
```

57.3 Definition of real numbers

The only purpose of this section is to define what does it mean to be a model of real numbers.

We define model of real numbers as any quadruple of sets (K, A, M, r) such that (K, A, M, r) is an ordered field and the order relation r is complete, that is every set that is nonempty and bounded above in this relation has a supremum.

definition

```
IsAmodelOfReals(K,A,M,r)  $\equiv$  IsAnOrdField(K,A,M,r)  $\wedge$  (r {is complete})
```

end

58 Integers - introduction

```
theory Int_ZF_IML imports OrderedGroup_ZF_1 Finite_ZF_1 ZF.Int Nat_ZF_IML
```

begin

This theory file is an interface between the old-style Isabelle (ZF logic) material on integers and the IsarMathLib project. Here we redefine the meta-level operations on integers (addition and multiplication) to convert

them to ZF-functions and show that integers form a commutative group with respect to addition and commutative monoid with respect to multiplication. Similarly, we redefine the order on integers as a relation, that is a subset of $Z \times Z$. We show that a subset of integers is bounded iff it is finite. As we are forced to use standard Isabelle notation with all these dollar signs, sharps etc. to denote "type coercions" (?) the notation is often ugly and difficult to read.

58.1 Addition and multiplication as ZF-functions.

In this section we provide definitions of addition and multiplication as subsets of $(Z \times Z) \times Z$. We use the (higher order) relation defined in the standard Int theory to define a subset of $Z \times Z$ that constitutes the ZF order relation corresponding to it. We define the set of positive integers using the notion of positive set from the OrderedGroup_ZF theory.

Definition of addition of integers as a binary operation on `int`. Recall that in standard Isabelle/ZF `int` is the set of integers and the sum of integers is denoted by prepending `+` with a dollar sign.

definition

`IntegerAddition` $\equiv \{ \langle x, c \rangle \in (\text{int} \times \text{int}) \times \text{int}. \text{fst}(x) \$+ \text{snd}(x) = c \}$

Definition of multiplication of integers as a binary operation on `int`. In standard Isabelle/ZF product of integers is denoted by prepending the dollar sign to `*`.

definition

`IntegerMultiplication` $\equiv$
 $\{ \langle x, c \rangle \in (\text{int} \times \text{int}) \times \text{int}. \text{fst}(x) \$* \text{snd}(x) = c \}$

Definition of natural order on integers as a relation on `int`. In the standard Isabelle/ZF the inequality relation on integers is denoted $\leq$ prepended with the dollar sign.

definition

`IntegerOrder` $\equiv \{ p \in \text{int} \times \text{int}. \text{fst}(p) \$\leq \text{snd}(p) \}$

This defines the set of positive integers.

definition

`PositiveIntegers` $\equiv \text{PositiveSet}(\text{int}, \text{IntegerAddition}, \text{IntegerOrder})$

`IntegerAddition` and `IntegerMultiplication` are functions on `int` $\times$ `int`.

lemma `Int_ZF_1_L1`: **shows**

`IntegerAddition` : `int` $\times$ `int` $\rightarrow$ `int`
`IntegerMultiplication` : `int` $\times$ `int` $\rightarrow$ `int`

<proof>

The next context (locale) defines notation used for integers. We define 0 to denote the neutral element of addition, 1 as the unit of the multiplicative monoid. We introduce notation $m \leq n$ for integers and write $m..n$ to denote the integer interval with endpoints in m and n . $\text{abs}(m)$ means the absolute value of m . This is a function defined in `OrderedGroup` that assigns x to itself if x is positive and assigns the opposite of x if $x \leq 0$. Unfortunately we cannot use the $|\cdot|$ notation as in the `OrderedGroup` theory as this notation has been hogged by the standard Isabelle's `Int` theory. The notation $-A$ where A is a subset of integers means the set $\{-m : m \in A\}$. The symbol $\text{maxf}(f, M)$ denotes the maximum of function f over the set A . We also introduce a similar notation for the minimum.

`locale int0 =`

```

fixes ints ( $\mathbb{Z}$ )
defines ints_def [simp]:  $\mathbb{Z} \equiv \text{int}$ 

fixes ia (infixl + 69)
defines ia_def [simp]:  $a+b \equiv \text{IntegerAddition}(a,b)$ 

fixes iminus (- _ 72)
defines rminus_def [simp]:  $-a \equiv \text{GroupInv}(\mathbb{Z}, \text{IntegerAddition})(a)$ 

fixes isub (infixl - 69)
defines isub_def [simp]:  $a-b \equiv a+(-b)$ 

fixes imult (infixl · 70)
defines imult_def [simp]:  $a \cdot b \equiv \text{IntegerMultiplication}(a,b)$ 

fixes setneg (- _ 72)
defines setneg_def [simp]:  $-A \equiv \text{GroupInv}(\mathbb{Z}, \text{IntegerAddition})(A)$ 

fixes izero (0)
defines izero_def [simp]:  $0 \equiv \text{TheNeutralElement}(\mathbb{Z}, \text{IntegerAddition})$ 

fixes ione (1)
defines ione_def [simp]:  $1 \equiv \text{TheNeutralElement}(\mathbb{Z}, \text{IntegerMultiplication})$ 

fixes itwo (2)
defines itwo_def [simp]:  $2 \equiv 1+1$ 

fixes ithree (3)
defines ithree_def [simp]:  $3 \equiv 2+1$ 

fixes nonnegative ( $\mathbb{Z}^+$ )
defines nonnegative_def [simp]:
 $\mathbb{Z}^+ \equiv \text{Nonnegative}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerOrder})$ 

fixes positive ( $\mathbb{Z}_+$ )
```

```

defines positive_def [simp]:
 $\mathbb{Z}_+ \equiv \text{PositiveSet}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerOrder})$ 

fixes abs
defines abs_def [simp]:
 $\text{abs}(m) \equiv \text{AbsoluteValue}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerOrder})(m)$ 

fixes lesseq (infix  $\leq$  60)
defines lesseq_def [simp]:  $m \leq n \equiv \langle m, n \rangle \in \text{IntegerOrder}$ 

fixes sless (infix  $<$  68)
defines sless_def [simp]:  $a < b \equiv a \leq b \wedge a \neq b$ 

fixes interval (infix  $..$  70)
defines interval_def [simp]:  $m..n \equiv \text{Interval}(\text{IntegerOrder}, m, n)$ 

fixes maxf
defines maxf_def [simp]:  $\text{maxf}(f, A) \equiv \text{Maximum}(\text{IntegerOrder}, f(A))$ 

fixes minf
defines minf_def [simp]:  $\text{minf}(f, A) \equiv \text{Minimum}(\text{IntegerOrder}, f(A))$ 

fixes oddext ( $_ \circ$ )
defines oddext_def [simp]:  $f^\circ \equiv \text{OddExtension}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerOrder}, f)$ 

fixes listsum ( $\sum$  _ 70)
defines listsum_def [simp]:  $\sum s \equiv \text{Fold}(\text{IntegerAddition}, 0, s)$ 

```

IntegerAddition adds integers and IntegerMultiplication multiplies integers. This states that the ZF functions IntegerAddition and IntegerMultiplication give the same results as the higher-order equivalents defined in the standard Int theory.

```

lemma (in int0) Int_ZF_1_L2: assumes A1:  $a \in \mathbb{Z} \quad b \in \mathbb{Z}$ 
shows
 $a+b = a \ \$+ \ b$ 
 $a \cdot b = a \ \$* \ b$ 
 $\langle \text{proof} \rangle$ 

```

Integer addition and multiplication are associative.

```

lemma (in int0) Int_ZF_1_L3:
assumes  $x \in \mathbb{Z} \quad y \in \mathbb{Z} \quad z \in \mathbb{Z}$ 
shows  $x+y+z = x+(y+z) \quad x \cdot y \cdot z = x \cdot (y \cdot z)$ 
 $\langle \text{proof} \rangle$ 

```

Integer addition and multiplication are commutative.

```

lemma (in int0) Int_ZF_1_L4:
assumes  $x \in \mathbb{Z} \quad y \in \mathbb{Z}$ 
shows  $x+y = y+x \quad x \cdot y = y \cdot x$ 

```

<proof>

Zero is neutral for addition and one for multiplication.

lemma (in int0) Int_ZF_1_L5: **assumes** A1: $x \in \mathbb{Z}$
 shows $(\# 0) + x = x \wedge x + (\# 0) = x$
 $(\# 1) \cdot x = x \wedge x \cdot (\# 1) = x$
<proof>

Zero is neutral for addition and one for multiplication.

lemma (in int0) Int_ZF_1_L6: **shows** $(\# 0) \in \mathbb{Z} \wedge$
 $(\forall x \in \mathbb{Z}. (\# 0) + x = x \wedge x + (\# 0) = x)$
 $(\# 1) \in \mathbb{Z} \wedge$
 $(\forall x \in \mathbb{Z}. (\# 1) \cdot x = x \wedge x \cdot (\# 1) = x)$
<proof>

Integers with addition and integers with multiplication form monoids.

theorem (in int0) Int_ZF_1_T1: **shows**
 IsAmonoid($\mathbb{Z}$, IntegerAddition)
 IsAmonoid($\mathbb{Z}$, IntegerMultiplication)
<proof>

Zero is the neutral element of the integers with addition and one is the neutral element of the integers with multiplication.

lemma (in int0) Int_ZF_1_L8: **shows** $(\# 0) = 0 \quad (\# 1) = 1$
<proof>

0 and 1, as defined in int0 context, are integers.

lemma (in int0) Int_ZF_1_L8A: **shows** $0 \in \mathbb{Z} \quad 1 \in \mathbb{Z}$
<proof>

Zero is not one.

lemma (in int0) int_zero_not_one: **shows** $0 \neq 1$
<proof>

The set of integers is not empty, of course.

lemma (in int0) int_not_empty: **shows** $\mathbb{Z} \neq \emptyset$
<proof>

The set of integers has more than just zero in it.

lemma (in int0) int_not_trivial: **shows** $\mathbb{Z} \neq \{0\}$
<proof>

Each integer has an inverse (in the addition sense).

lemma (in int0) Int_ZF_1_L9: **assumes** A1: $g \in \mathbb{Z}$
 shows $\exists b \in \mathbb{Z}. g + b = 0$
<proof>

Integers with addition form an abelian group. This also shows that we can apply all theorems proven in the proof contexts (locales) that require the assumption that some pair of sets form a group like locale `group0`.

```
theorem Int_ZF_1_T2: shows
  IsAgroup(int,IntegerAddition)
  IntegerAddition {is commutative on} int
  group0(int,IntegerAddition)
  ⟨proof⟩
```

Sum of integers is an integer.

```
lemma (in int0) int_sum_type: assumes m∈ℤ n∈ℤ
shows m+n ∈ ℤ
  ⟨proof⟩
```

Product of integers is an integer.

```
lemma (in int0) int_prod_type: assumes m∈ℤ n∈ℤ
shows m·n ∈ ℤ
  ⟨proof⟩
```

Negative of an integer is an integer.

```
lemma (in int0) int_neg_type: assumes m∈ℤ shows (-m) ∈ ℤ
  ⟨proof⟩
```

Taking a negative twice we get back the same integer.

```
lemma (in int0) neg_neg_noop: assumes m∈ℤ shows (-(-m)) = m
  ⟨proof⟩
```

What is the additive group inverse in the group of integers?

```
lemma (in int0) Int_ZF_1_L9A: assumes A1: m∈ℤ
shows $-m = -m
  ⟨proof⟩
```

Subtracting integers corresponds to adding the negative.

```
lemma (in int0) Int_ZF_1_L10: assumes A1: m∈ℤ n∈ℤ
shows m-n = m $+ $-n m-n = m $- n
  ⟨proof⟩
```

Negative of zero is zero.

```
lemma (in int0) Int_ZF_1_L11: shows (-0) = 0
  ⟨proof⟩
```

A trivial calculation lemma that allows to subtract and add one.

```
lemma Int_ZF_1_L12:
assumes m∈int shows m $- $#1 $+ $#1 = m
  ⟨proof⟩
```

A trivial calculation lemma that allows to subtract and add one, version with ZF-operation.

```
lemma (in int0) Int_ZF_1_L13: assumes m $\in\mathbb{Z}$ 
  shows (m $- $#1) + 1 = m
  <proof>
```

Adding or subtracing one changes integers, but subtracting zero does not.

```
lemma (in int0) Int_ZF_1_L14: assumes A1: m $\in\mathbb{Z}$ 
  shows
    m+1  $\neq$  m
    m-1  $\neq$  m
    m-0 = m
  <proof>
```

If the difference is zero, the integers are equal.

```
lemma (in int0) Int_ZF_1_L15:
  assumes A1: m $\in\mathbb{Z}$  n $\in\mathbb{Z}$  and A2: m-n = 0
  shows m=n
  <proof>
```

58.2 Integers as an ordered group

In this section we define order on integers as a relation, that is a subset of $Z \times Z$ and show that integers form an ordered group.

The next lemma interprets the order definition one way.

```
lemma (in int0) Int_ZF_2_L1:
  assumes A1: m $\in\mathbb{Z}$  n $\in\mathbb{Z}$  and A2: m  $\leq$  n
  shows m  $\leq$  n
  <proof>
```

The next lemma interprets the definition the other way.

```
lemma (in int0) Int_ZF_2_L1A: assumes A1: m  $\leq$  n
  shows m  $\leq$  n m $\in\mathbb{Z}$  n $\in\mathbb{Z}$ 
  <proof>
```

Integer order is a relation on integers.

```
lemma Int_ZF_2_L1B: shows IntegerOrder  $\subseteq$  int $\times$ int
  <proof>
```

The way we define the notion of being bounded below, its sufficient for the relation to be on integers for all bounded below sets to be subsets of integers.

```
lemma (in int0) Int_ZF_2_L1C:
  assumes A1: IsBoundedBelow(A,IntegerOrder)
  shows A $\subseteq\mathbb{Z}$ 
  <proof>
```


The order on integers is reflexive.

```
lemma (in int0) int_ord_is_refl: shows refl( $\mathbb{Z}$ , IntegerOrder)
  <proof>
```

A more explicit version of "integer order is reflexive" claim

```
lemma (in int0) int_ord_is_refl1: assumes  $z \in \mathbb{Z}$ 
  shows  $z \leq z$ 
  <proof>
```

The essential condition to show antisymmetry of the order on integers.

```
lemma (in int0) Int_ZF_2_L3:
  assumes A1:  $m \leq n \quad n \leq m$ 
  shows  $m = n$ 
  <proof>
```

The order on integers is antisymmetric.

```
lemma (in int0) Int_ZF_2_L4: shows antisym(IntegerOrder)
  <proof>
```

The essential condition to show that the order on integers is transitive.

```
lemma Int_ZF_2_L5:
  assumes A1:  $\langle m, n \rangle \in \text{IntegerOrder} \quad \langle n, k \rangle \in \text{IntegerOrder}$ 
  shows  $\langle m, k \rangle \in \text{IntegerOrder}$ 
  <proof>
```

The order on integers is transitive. This version is stated in the `int0` context using notation for integers.

```
lemma (in int0) Int_order_transitive:
  assumes A1:  $m \leq n \quad n \leq k$ 
  shows  $m \leq k$ 
  <proof>
```

The order on integers is transitive.

```
lemma Int_ZF_2_L6: shows trans(IntegerOrder)
  <proof>
```

The order on integers is a partial order.

```
lemma Int_ZF_2_L7: shows IsPartOrder(int, IntegerOrder)
  <proof>
```

The essential condition to show that the order on integers is preserved by translations.

```
lemma (in int0) int_ord_transl_inv:
  assumes A1:  $k \in \mathbb{Z}$  and A2:  $m \leq n$ 
  shows  $m + k \leq n + k \quad k + m \leq k + n$ 
  <proof>
```

Integers form a linearly ordered group. We can apply all theorems proven in `group3` context to integers.

```
theorem (in int0) Int_ZF_2_T1: shows
  IsAnOrdGroup( $\mathbb{Z}$ , IntegerAddition, IntegerOrder)
  IntegerOrder {is total on}  $\mathbb{Z}$ 
  group3( $\mathbb{Z}$ , IntegerAddition, IntegerOrder)
  IsLinOrder( $\mathbb{Z}$ , IntegerOrder)
  <proof>
```

Another way of stating that we can apply theorems proven in the `group3` context (defined `OrderedGroup_ZF` theory) to the ordered group of integers.

```
sublocale int0 < group3 int IntegerAddition IntegerOrder
  0 ia iminus lesseq sless nonnegative positive setneg abs oddext listsum
  <proof>
```

Negative numbers are not nonnegative. This is a special case of `ls_not_leq` from `OrderedGroup_ZF` theory.

```
corollary (in int0) neg_not_nonneg: assumes  $m < 0$  shows  $\neg(0 \leq m)$ 
  <proof>
```

Negative of a positive integer is negative.

```
lemma (in int0) neg_pos_int_neg: assumes  $0 < z$  shows  $(-z) < 0$ 
  <proof>
```

Negative of a negative integer is positive.

```
lemma (in int0) neg_neg_int_pos: assumes  $z < 0$  shows  $0 < (-z)$ 
  <proof>
```

An integer is nonnegative or negative. This is a special case of `OrdGroup_2cases` from `OrderedGroup_ZF` theory and useful for splitting proofs into cases.

```
lemma (in int0) int_nonneg_or_neg: assumes  $z \in \mathbb{Z}$  shows  $0 \leq z \vee z < 0$ 
  <proof>
```

Slightly weaker assertion than `int_nonneg_or_neg` with overlap at zero: an integer is nonnegative or nonpositive.

```
corollary (in int0) int_nonneg_or_nonpos: assumes  $z \in \mathbb{Z}$  shows  $0 \leq z \vee z \leq 0$ 
  <proof>
```

Another variant of splitting into cases: an integer is positive, zero or negative.

```
lemma (in int0) int_neg_zero_pos: assumes  $z \in \mathbb{Z}$  shows  $0 < z \vee z = 0 \vee z < 0$ 
  <proof>
```

If a pair (i, m) belongs to the order relation on integers and $i \neq m$, then i is smaller than m in the sense of defined in the standard Isabelle's `Int.thy`.

```
lemma (in int0) Int_ZF_2_L9: assumes A1:  $i \leq m$  and A2:  $i \neq m$ 
  shows  $i < m$ 
```

<proof>

This shows how Isabelle's $\$<$ operator translates to IsarMathLib notation.

```
lemma (in int0) Int_ZF_2_L9AA: assumes A1:  $m \in \mathbb{Z}$   $n \in \mathbb{Z}$ 
  and A2:  $m \$< n$ 
  shows  $m \leq n$   $m \neq n$   $m < n$ 
<proof>
```

A small technical lemma about putting one on the other side of an inequality.

```
lemma (in int0) Int_ZF_2_L9A:
  assumes A1:  $k \in \mathbb{Z}$  and A2:  $m \leq k$   $\$- (\$ \# 1)$ 
  shows  $m+1 \leq k$ 
<proof>
```

We can put any integer on the other side of an inequality reversing its sign.

```
lemma (in int0) Int_ZF_2_L9B: assumes  $i \in \mathbb{Z}$   $m \in \mathbb{Z}$   $k \in \mathbb{Z}$ 
  shows  $i+m \leq k \iff i \leq k-m$ 
<proof>
```

A special case of Int_ZF_2_L9B with weaker assumptions.

```
lemma (in int0) Int_ZF_2_L9C:
  assumes  $i \in \mathbb{Z}$   $m \in \mathbb{Z}$  and  $i-m \leq k$ 
  shows  $i \leq k+m$ 
<proof>
```

Taking (higher order) minus on both sides of inequality reverses it.

```
lemma (in int0) Int_ZF_2_L10: assumes  $k \leq i$ 
  shows
     $(-i) \leq (-k)$ 
     $\$-i \leq \$-k$ 
<proof>
```

Taking minus on both sides of inequality reverses it, version with a negative on one side.

```
lemma (in int0) Int_ZF_2_L10AA: assumes  $n \in \mathbb{Z}$   $m \leq (-n)$ 
  shows  $n \leq (-m)$ 
<proof>
```

We can cancel the same element on on both sides of an inequality, a version with minus on both sides.

```
lemma (in int0) Int_ZF_2_L10AB:
  assumes  $m \in \mathbb{Z}$   $n \in \mathbb{Z}$   $k \in \mathbb{Z}$  and  $m-n \leq m-k$ 
  shows  $k \leq n$ 
<proof>
```

If an integer is nonpositive, then its opposite is nonnegative.

```
lemma (in int0) Int_ZF_2_L10A: assumes  $k \leq 0$ 
```

shows $0 \leq (-k)$
 $\langle proof \rangle$

If the opposite of an integers is nonnegative, then the integer is nonpositive.

lemma (in int0) Int_ZF_2_L10B:
assumes $k \in \mathbb{Z}$ **and** $0 \leq (-k)$
shows $k \leq 0$
 $\langle proof \rangle$

Adding one to an integer corresponds to taking a successor for a natural number.

lemma (in int0) Int_ZF_2_L11:
shows $i \text{ $+$ } n \text{ $+$ } (1) = i \text{ $+$ } \text{succ}(n)$
 $\langle proof \rangle$

Adding a natural number increases integers.

lemma (in int0) Int_ZF_2_L12: **assumes** $A1: i \in \mathbb{Z}$ **and** $A2: n \in \text{nat}$
shows $i \leq i \text{ $+$ } n$
 $\langle proof \rangle$

Adding one increases integers.

lemma (in int0) Int_ZF_2_L12A: **assumes** $A1: j \leq k$
shows $j \leq k \text{ $+$ } 1 \quad j \leq k+1$
 $\langle proof \rangle$

Adding one increases integers, yet one more version.

lemma (in int0) Int_ZF_2_L12B: **assumes** $A1: m \in \mathbb{Z}$ **shows** $m \leq m+1$
 $\langle proof \rangle$

If $k + 1 = m + n$, where n is a non-zero natural number, then $m \leq k$.

lemma (in int0) Int_ZF_2_L13:
assumes $A1: k \in \mathbb{Z}$ $m \in \mathbb{Z}$ **and** $A2: n \in \text{nat}$
and $A3: k \text{ $+$ } (1) = m \text{ $+$ } \text{succ}(n)$
shows $m \leq k$
 $\langle proof \rangle$

The absolute value of an integer is an integer.

lemma (in int0) Int_ZF_2_L14: **assumes** $A1: m \in \mathbb{Z}$
shows $\text{abs}(m) \in \mathbb{Z}$
 $\langle proof \rangle$

If two integers are nonnegative, then the opposite of one is less or equal than the other and the sum is also nonnegative.

lemma (in int0) Int_ZF_2_L14A:
assumes $0 \leq m$ $0 \leq n$
shows
 $(-m) \leq n$

$0 \leq m + n$
 $\langle proof \rangle$

We can increase components in an estimate.

lemma (in int0) Int_ZF_2_L15:
 assumes $b \leq b_1$ $c \leq c_1$ and $a \leq b+c$
 shows $a \leq b_1+c_1$
 $\langle proof \rangle$

We can add or subtract the sides of two inequalities.

lemma (in int0) int_ineq_add_sides:
 assumes $a \leq b$ and $c \leq d$
 shows
 $a+c \leq b+d$
 $a-d \leq b-c$
 $\langle proof \rangle$

We can increase the second component in an estimate.

lemma (in int0) Int_ZF_2_L15A:
 assumes $b \in \mathbb{Z}$ and $a \leq b+c$ and A3: $c \leq c_1$
 shows $a \leq b+c_1$
 $\langle proof \rangle$

If we increase the second component in a sum of three integers, the whole sum inceases.

lemma (in int0) Int_ZF_2_L15C:
 assumes A1: $m \in \mathbb{Z}$ $n \in \mathbb{Z}$ and A2: $k \leq L$
 shows $m+k+n \leq m+L+n$
 $\langle proof \rangle$

We don't decrease an integer by adding a nonnegative one.

lemma (in int0) Int_ZF_2_L15D:
 assumes $0 \leq n$ $m \in \mathbb{Z}$
 shows $m \leq n+m$
 $\langle proof \rangle$

Some inequalities about the sum of two integers and its absolute value.

lemma (in int0) Int_ZF_2_L15E:
 assumes $m \in \mathbb{Z}$ $n \in \mathbb{Z}$
 shows
 $m+n \leq \text{abs}(m)+\text{abs}(n)$
 $m-n \leq \text{abs}(m)+\text{abs}(n)$
 $(-m)+n \leq \text{abs}(m)+\text{abs}(n)$
 $(-m)-n \leq \text{abs}(m)+\text{abs}(n)$
 $\langle proof \rangle$

We can add a nonnegative integer to the right hand side of an inequality.

lemma (in int0) Int_ZF_2_L15F: assumes $m \leq k$ and $0 \leq n$

shows $m \leq k+n$ $m \leq n+k$
 $\langle proof \rangle$

Triangle inequality for integers.

lemma (in int0) Int_triangle_ineq:
assumes $m \in \mathbb{Z}$ $n \in \mathbb{Z}$
shows $\text{abs}(m+n) \leq \text{abs}(m) + \text{abs}(n)$
 $\langle proof \rangle$

Taking absolute value does not change nonnegative integers.

lemma (in int0) Int_ZF_2_L16:
assumes $0 \leq m$ **shows** $m \in \mathbb{Z}^+$ and $\text{abs}(m) = m$
 $\langle proof \rangle$

$0 \leq 1$, so $|1| = 1$.

lemma (in int0) Int_ZF_2_L16A:
shows $0 \leq 1$ $0 < 1$ $\text{abs}(1) = 1$
 $\langle proof \rangle$

Negative one is smaller than zero.

lemma (in int0) neg_one_less_zero: **shows** $(-1) < 0$
 $\langle proof \rangle$

$1 \leq 2$.

lemma (in int0) Int_ZF_2_L16B: **shows** $1 \leq 2$
 $\langle proof \rangle$

Assume an integer is greater or equal one. Then it is greater or equal than zero, is not equal zero, if we add one to it then it is greater or equal one, two and zero. Two is .

lemma (in int0) Int_ZF_2_L16C:
assumes A1: $1 \leq a$ **shows**
 $0 \leq a$ $a \neq 0$
 $2 \leq a+1$
 $1 \leq a+1$
 $0 \leq a+1$
 $\langle proof \rangle$

If we add one to a nonnegative integer, the result is greater than zero.

lemma (in int0) nneg_add_one: **assumes** $0 \leq a$
shows $0 < a+1$
 $\langle proof \rangle$

Absolute value is the same for an integer and its opposite.

lemma (in int0) Int_ZF_2_L17:
assumes $m \in \mathbb{Z}$ **shows** $\text{abs}(-m) = \text{abs}(m)$
 $\langle proof \rangle$

The absolute value of zero is zero.

```
lemma (in int0) Int_ZF_2_L18: shows  $\text{abs}(0) = 0$ 
   $\langle \text{proof} \rangle$ 
```

A different version of the triangle inequality.

```
lemma (in int0) Int_triangle_ineq1:
  assumes A1:  $m \in \mathbb{Z} \quad n \in \mathbb{Z}$ 
  shows
     $\text{abs}(m-n) \leq \text{abs}(n) + \text{abs}(m)$ 
     $\text{abs}(m-n) \leq \text{abs}(m) + \text{abs}(n)$ 
   $\langle \text{proof} \rangle$ 
```

Another version of the triangle inequality.

```
lemma (in int0) Int_triangle_ineq2:
  assumes  $m \in \mathbb{Z} \quad n \in \mathbb{Z}$ 
  and  $\text{abs}(m-n) \leq k$ 
  shows
     $\text{abs}(m) \leq \text{abs}(n) + k$ 
     $m - k \leq n$ 
     $m \leq n + k$ 
     $n - k \leq m$ 
   $\langle \text{proof} \rangle$ 
```

Triangle inequality with three integers. We could use `OrdGroup_triangle_ineq3`, but since `simp` cannot translate the notation directly, it is simpler to reprove it for integers.

```
lemma (in int0) Int_triangle_ineq3:
  assumes A1:  $m \in \mathbb{Z} \quad n \in \mathbb{Z} \quad k \in \mathbb{Z}$ 
  shows  $\text{abs}(m+n+k) \leq \text{abs}(m) + \text{abs}(n) + \text{abs}(k)$ 
   $\langle \text{proof} \rangle$ 
```

The next lemma shows what happens when one integers is not greater or equal than another.

```
lemma (in int0) Int_ZF_2_L19:
  assumes A1:  $m \in \mathbb{Z} \quad n \in \mathbb{Z}$  and A2:  $\neg(n \leq m)$ 
  shows  $m \leq n \quad (-n) \leq (-m) \quad m \neq n \quad m < n$ 
   $\langle \text{proof} \rangle$ 
```

If one integer is greater or equal and not equal to another, then it is not smaller or equal.

```
lemma (in int0) Int_ZF_2_L19AA: assumes A1:  $m \leq n$  and A2:  $m \neq n$ 
  shows  $\neg(n \leq m)$ 
   $\langle \text{proof} \rangle$ 
```

The next lemma allows to prove theorems for the case of positive and negative integers separately.

```
lemma (in int0) Int_ZF_2_L19A: assumes A1:  $m \in \mathbb{Z}$  and A2:  $\neg(0 \leq m)$ 
```

shows $m \leq 0 \quad 0 \leq (-m) \quad m \neq 0$
 $\langle proof \rangle$

We can prove a theorem about integers by proving that it holds for $m = 0$, $m \in \mathbb{Z}_+$ and $-m \in \mathbb{Z}_+$.

lemma (in int0) Int_ZF_2_L19B:
assumes $m \in \mathbb{Z}$ **and** $Q(0)$ **and** $\forall n \in \mathbb{Z}_+. Q(n)$ **and** $\forall n \in \mathbb{Z}_+. Q(-n)$
shows $Q(m)$
 $\langle proof \rangle$

An integer is not greater than its absolute value.

lemma (in int0) Int_ZF_2_L19C: **assumes** $A1: m \in \mathbb{Z}$
shows
 $m \leq \text{abs}(m)$
 $(-m) \leq \text{abs}(m)$
 $\langle proof \rangle$

$$|m - n| = |n - m|.$$

lemma (in int0) Int_ZF_2_L20: **assumes** $m \in \mathbb{Z} \quad n \in \mathbb{Z}$
shows $\text{abs}(m-n) = \text{abs}(n-m)$
 $\langle proof \rangle$

We can add the sides of inequalities with absolute values.

lemma (in int0) Int_ZF_2_L21:
assumes $A1: m \in \mathbb{Z} \quad n \in \mathbb{Z}$
and $A2: \text{abs}(m) \leq k \quad \text{abs}(n) \leq 1$
shows
 $\text{abs}(m+n) \leq k + 1$
 $\text{abs}(m-n) \leq k + 1$
 $\langle proof \rangle$

Absolute value is nonnegative.

lemma (in int0) int_abs_nonneg: **assumes** $A1: m \in \mathbb{Z}$
shows $\text{abs}(m) \in \mathbb{Z}^+ \quad 0 \leq \text{abs}(m)$
 $\langle proof \rangle$

If a nonnegative integer is less or equal than another, then so is its absolute value.

lemma (in int0) Int_ZF_2_L23:
assumes $0 \leq m \quad m \leq k$
shows $\text{abs}(m) \leq k$
 $\langle proof \rangle$

The standard Isabelle/ZF defined `znegative` predicate on integers. The next lemma expresses that in terms of the order relation on integers.

lemma (in int0) znegative_as_ls_zero: **assumes** $z \in \mathbb{Z}$
shows $\text{znegative}(z) \longleftrightarrow z < 0$

<proof>

A nonnegative integer (i.e. $0 \leq z$) is not negative in the sense of `znegative` predicate. We also use the opportunity to mention that such a nonnegative integer is an integer.

lemma (in int0) nonnegative_not_znegative: assumes $0 \leq z$
shows $\neg \text{znegative}(z)$ and $z \in \mathbb{Z}$

<proof>

A nonnegative integer (i.e. one that belongs to $\mathbb{Z}^+$) is not negative in the sense of `znegative` predicate. We also use the opportunity to mention that a nonnegative integer is an integer.

lemma (in int0) nonnegative_not_znegative1: assumes $z \in \mathbb{Z}^+$
shows $\neg \text{znegative}(z)$ and $z \in \mathbb{Z}$

<proof>

A negative integer is `znegative`.

lemma (in int0) negative_is_znegative: assumes $z \in \mathbb{Z} \setminus \mathbb{Z}^+$ shows `znegative`(z)
<proof>

An integer that is not `znegative` is nonnegative.

corollary (in int0) notzneg_is_nonneg: assumes $z \in \mathbb{Z}$ and $\neg \text{znegative}(z)$
shows $z \in \mathbb{Z}^+$ *<proof>*

An integers that is not `znegative` is greater or equal than zero..

lemma (in int0) notzneg_is_geq_zero: assumes $z \in \mathbb{Z}$ and $\neg \text{znegative}(z)$
shows $0 \leq z$
<proof>

58.3 Induction on integers.

In this section we show some induction lemmas for integers. The basic tools are the induction on natural numbers and the fact that integers can be written as a sum of a smaller integer and a natural number.

An integer can be written a a sum of a smaller integer and a natural number.

lemma (in int0) Int_ZF_3_L2: assumes $A1: i \leq m$
shows $\exists n \in \text{nat}. m = i + n$
<proof>

Induction for integers, the induction step.

lemma (in int0) Int_ZF_3_L6: assumes $A1: i \in \mathbb{Z}$
and $A2: \forall m. i \leq m \wedge Q(m) \longrightarrow Q(m + 1)$
shows $\forall k \in \text{nat}. Q(i + k) \longrightarrow Q(i + \text{succ}(k))$
<proof>

Induction on integers, version with higher-order increment function.

```

lemma (in int0) Int_ZF_3_L7:
  assumes A1:  $i \leq k$  and A2:  $Q(i)$ 
  and A3:  $\forall m. i \leq m \wedge Q(m) \longrightarrow Q(m \$+ (\$# 1))$ 
  shows  $Q(k)$ 
<proof>

```

Induction on integer, implication between two forms of the induction step.

```

lemma (in int0) Int_ZF_3_L7A: assumes
  A1:  $\forall m. i \leq m \wedge Q(m) \longrightarrow Q(m+1)$ 
  shows  $\forall m. i \leq m \wedge Q(m) \longrightarrow Q(m \$+ (\$# 1))$ 
<proof>

```

Induction on integers, version with ZF increment function.

```

theorem (in int0) Induction_on_int:
  assumes A1:  $i \leq k$  and A2:  $Q(i)$ 
  and A3:  $\forall m. i \leq m \wedge Q(m) \longrightarrow Q(m+1)$ 
  shows  $Q(k)$ 
<proof>

```

Another form of induction on integers. This rewrites the basic theorem Int_ZF_3_L7 substituting $P(-k)$ for $Q(k)$.

```

lemma (in int0) Int_ZF_3_L7B: assumes A1:  $i \leq k$  and A2:  $P(\$-i)$ 
  and A3:  $\forall m. i \leq m \wedge P(\$-m) \longrightarrow P(\$-(m \$+ (\$# 1)))$ 
  shows  $P(\$-k)$ 
<proof>

```

Another induction on integers. This rewrites Int_ZF_3_L7 substituting $-k$ for k and $-i$ for i .

```

lemma (in int0) Int_ZF_3_L8: assumes A1:  $k \leq i$  and A2:  $P(i)$ 
  and A3:  $\forall m. \$-i \leq m \wedge P(\$-m) \longrightarrow P(\$-(m \$+ (\$# 1)))$ 
  shows  $P(k)$ 
<proof>

```

An implication between two forms of induction steps.

```

lemma (in int0) Int_ZF_3_L9: assumes A1:  $i \in \mathbb{Z}$ 
  and A2:  $\forall n. n \leq i \wedge P(n) \longrightarrow P(n \$+ \$- (\$# 1))$ 
  shows  $\forall m. \$-i \leq m \wedge P(\$-m) \longrightarrow P(\$-(m \$+ (\$# 1)))$ 
<proof>

```

Backwards induction on integers, version with higher-order decrement function.

```

lemma (in int0) Int_ZF_3_L9A: assumes A1:  $k \leq i$  and A2:  $P(i)$ 
  and A3:  $\forall n. n \leq i \wedge P(n) \longrightarrow P(n \$+ \$- (\$# 1))$ 
  shows  $P(k)$ 
<proof>

```

Induction on integers, implication between two forms of the induction step.

```

lemma (in int0) Int_ZF_3_L10: assumes
  A1:  $\forall n. n \leq i \wedge P(n) \longrightarrow P(n-1)$ 
  shows  $\forall n. n \leq i \wedge P(n) \longrightarrow P(n \text{ \$+ \$-}(\text{\$}\#1))$ 
<proof>

```

Backwards induction on integers.

```

theorem (in int0) Back_induct_on_int:
  assumes A1:  $k \leq i$  and A2:  $P(i)$ 
  and A3:  $\forall n. n \leq i \wedge P(n) \longrightarrow P(n-1)$ 
  shows  $P(k)$ 
<proof>

```

58.4 Bounded vs. finite subsets of integers

The goal of this section is to establish that a subset of integers is bounded is and only is it is finite. The fact that all finite sets are bounded is already shown for all linearly ordered groups in `OrderedGroups_ZF.thy`. To show the other implication we show that all intervals starting at 0 are finite and then use a result from `OrderedGroups_ZF.thy`.

There are no integers between k and $k + 1$.

```

lemma (in int0) Int_ZF_4_L1:
  assumes A1:  $k \in \mathbb{Z}$   $m \in \mathbb{Z}$   $n \in \text{nat}$  and A2:  $k \text{ \$+ } \text{\$}\#1 = m \text{ \$+ } \text{\$}\#n$ 
  shows  $m = k \text{ \$+ } \text{\$}\#1 \vee m \leq k$ 
<proof>

```

A trivial calculation lemma that allows to subtract and add one.

```

lemma Int_ZF_4_L1A:
  assumes  $m \in \text{int}$  shows  $m \text{ \$- } \text{\$}\#1 \text{ \$+ } \text{\$}\#1 = m$ 
<proof>

```

There are no integers between k and $k + 1$, another formulation.

```

lemma (in int0) Int_ZF_4_L1B: assumes A1:  $m \leq L$ 
  shows
   $m = L \vee m+1 \leq L$ 
   $m = L \vee m \leq L-1$ 
<proof>

```

If $j \in m..k + 1$, then $j \in m..n$ or $j = k + 1$.

```

lemma (in int0) Int_ZF_4_L2: assumes A1:  $k \in \mathbb{Z}$ 
  and A2:  $j \in m..(k \text{ \$+ } \text{\$}\#1)$ 
  shows  $j \in m..k \vee j \in \{k \text{ \$+ } \text{\$}\#1\}$ 
<proof>

```

Extending an integer interval by one is the same as adding the new endpoint.

```

lemma (in int0) Int_ZF_4_L3: assumes A1:  $m \leq k$ 
  shows  $m..(k \text{ \$+ } \text{\$}\#1) = m..k \cup \{k \text{ \$+ } \text{\$}\#1\}$ 

```

<proof>

Integer intervals are finite - induction step.

lemma (in int0) Int_ZF_4_L4:
 assumes A1: $i \leq m$ and A2: $i..m \in \text{Fin}(\mathbb{Z})$
 shows $i..(m + 1) \in \text{Fin}(\mathbb{Z})$
 <proof>

Integer intervals are finite.

lemma (in int0) Int_ZF_4_L5: assumes A1: $i \in \mathbb{Z}$ $k \in \mathbb{Z}$
 shows $i..k \in \text{Fin}(\mathbb{Z})$
 <proof>

Bounded integer sets are finite.

lemma (in int0) Int_ZF_4_L6: assumes A1: $\text{IsBounded}(A, \text{IntegerOrder})$
 shows $A \in \text{Fin}(\mathbb{Z})$
 <proof>

A subset of integers is bounded iff it is finite.

theorem (in int0) Int_bounded_iff_fin:
 shows $\text{IsBounded}(A, \text{IntegerOrder}) \longleftrightarrow A \in \text{Fin}(\mathbb{Z})$
 <proof>

The image of an interval by any integer function is finite, hence bounded.

lemma (in int0) Int_ZF_4_L8:
 assumes A1: $i \in \mathbb{Z}$ $k \in \mathbb{Z}$ and A2: $f: \mathbb{Z} \rightarrow \mathbb{Z}$
 shows
 $f(i..k) \in \text{Fin}(\mathbb{Z})$
 $\text{IsBounded}(f(i..k), \text{IntegerOrder})$
 <proof>

If for every integer we can find one in A that is greater or equal, then A is not bounded above, hence infinite.

lemma (in int0) Int_ZF_4_L9: assumes A1: $\forall m \in \mathbb{Z}. \exists k \in A. m \leq k$
 shows
 $\neg \text{IsBoundedAbove}(A, \text{IntegerOrder})$
 $A \notin \text{Fin}(\mathbb{Z})$
 <proof>

58.5 Addition on integers in terms of magnitudes

In standard (informal) mathematics natural numbers form a subset of integers. In ZF that is not true as integers are defined as certain classes of pairs of natural numbers. As a result, addition on natural numbers is not a special case of addition on integers. The standard Isabelle/ZF's Int theory defines the notion of **zmagnitude** i.e. the magnitude of an integer. If z is an integer then **zmagnitude**(z) is its absolute value, interpreted as a natural

number. The goal of this section is to provide facts about `zmagnitude` that are missing from the standard Isabelle/ZF's `Int` library and formulae that express addition of integers in terms of addition of their magnitudes.

The next lemma shows that `zmagnitude` of an integer is the same as `zmagnitude` of its opposite.

```
lemma (in int0) zmag_opposite_same: assumes  $z \in \mathbb{Z}$ 
  shows
     $\text{zmagnitude}(z) = \text{zmagnitude}(-z)$ 
     $\text{zmagnitude}(z) = \text{zmagnitude}(-z)$ 
  <proof>
```

The magnitude of zero (the integer) is zero (the natural number) and the magnitude of one (the integer) is one (the natural number).

```
lemma (in int0) zmag_zero_one: shows  $\text{zmagnitude}(0) = 0$  and  $\text{zmagnitude}(1)$ 
  = 1
  <proof>
```

If z_1, z_2 is a pair of integers then (at least) one of the following six cases holds:

1. Both integers are nonnegative.
2. Both integers are negative.
3. z_1 is nonnegative, z_2 is negative and magnitude of z_2 is less or equal than magnitude of z_1 .
4. z_1 is nonnegative, z_2 is negative and magnitude of z_1 is (strictly) smaller than magnitude of z_2 .
5. z_1 is negative, z_2 is nonnegative and magnitude of z_1 is less or equal than magnitude of z_2 .
6. z_1 is negative, z_2 is nonnegative and magnitude of z_2 is (strictly) less than magnitude of z_1 .

```
lemma (in int0) int_pair_6cases: assumes  $z_1 \in \mathbb{Z}$   $z_2 \in \mathbb{Z}$ 
  shows  $(0 \leq z_1 \wedge 0 \leq z_2) \vee (z_1 < 0 \wedge z_2 < 0) \vee$ 
     $(0 \leq z_1 \wedge z_2 < 0 \wedge \text{zmagnitude}(z_2) \leq \text{zmagnitude}(z_1)) \vee$ 
     $(0 \leq z_1 \wedge z_2 < 0 \wedge \text{zmagnitude}(z_1) < \text{zmagnitude}(z_2)) \vee$ 
     $(z_1 < 0 \wedge 0 \leq z_2 \wedge \text{zmagnitude}(z_1) \leq \text{zmagnitude}(z_2)) \vee$ 
     $(z_1 < 0 \wedge 0 \leq z_2 \wedge \text{zmagnitude}(z_2) < \text{zmagnitude}(z_1))$ 
  <proof>
```

Sum of nonnegative integers is nonnegative. The magnitude of the sum of such integers is the sum of their magnitudes. We can add nonnegative integers by adding their magnitudes and converting the result to an integer. `zmagnitude` (defined in standard Isabelle/ZF's `Int` theory) is the natural number corresponding to the absolute value of an integer number.

```
lemma (in int0) add_nonneg_ints: assumes  $0 \leq z_1$   $0 \leq z_2$ 
  shows
```

```

0 ≤ z1+z2
zmagnitude(z1+z2) = zmagnitude(z1) #+ zmagnitude(z2)
z1+z2 = $# (zmagnitude(z1) #+ zmagnitude(z2))
⟨proof⟩

```

Sum of negative integers is negative. The magnitude of the sum of such integers is the sum of their magnitudes. We can calculate the sum of negative integers by taking the sum of their magnitudes, converting that to an integer and taking negative of the result.

```

lemma (in int0) add_neg_ints: assumes z1<0 z2<0
shows
  z1+z2<0
  zmagnitude(z1+z2) = zmagnitude(z1) #+ zmagnitude(z2)
  z1+z2 = $-($#(zmagnitude(z1) #+ zmagnitude(z2)))
⟨proof⟩

```

If z_1 is a nonnegative integer and z_2 is a negative integer with a less or equal magnitude, then their sum is nonnegative and its magnitude is the difference between magnitudes of z_1 and z_2 .

```

lemma (in int0) add_nonneg_neg1:
  assumes 0 ≤ z1 z2<0 zmagnitude(z2) ≤ zmagnitude(z1)
  shows
    0 ≤ z1+z2
    zmagnitude(z1+z2) = zmagnitude(z1) #- zmagnitude(z2)
    z1+z2 = $#(zmagnitude(z1) #- zmagnitude(z2))
⟨proof⟩

```

If z_1 is a nonnegative integer and z_2 is a negative integer with a greater magnitude, then their sum is negative and its magnitude is the difference between magnitudes of z_2 and z_1 .

```

lemma (in int0) add_nonneg_neg2:
  assumes 0 ≤ z1 z2<0 zmagnitude(z1) < zmagnitude(z2)
  shows
    z1+z2 < 0
    zmagnitude(z1+z2) = zmagnitude(z2) #- zmagnitude(z1)
    z1+z2 = $-($#(zmagnitude(z2) #- zmagnitude(z1)))
⟨proof⟩

```

If z_1 is a negative integer and z_2 is a nonnegative integer with a greater or equal magnitude, then their sum is nonnegative and its magnitude is the difference between magnitudes of z_2 and z_1 . This is essentially `add_nonneg_neg1` with z_1 and z_2 swapped.

```

lemma (in int0) add_neg_nonneg1:
  assumes z1<0 0 ≤ z2 zmagnitude(z1) ≤ zmagnitude(z2)
  shows
    0 ≤ z1+z2
    zmagnitude(z1+z2) = zmagnitude(z2) #- zmagnitude(z1)

```

```

    z1+z2 = $(zmagnitude(z2) #- zmagnitude(z1))
  <proof>

```

If z_1 is a negative integer and z_2 is a nonnegative integer with a smaller magnitude, the their sum is negative and its magnitude is the difference between magnitudes of z_1 and z_2 . This is essentially `add_nonneg_neg2` with z_1 and z_2 swapped.

```

lemma (in int0) add_neg_nonneg2:
  assumes z1<0 0≤z2 zmagnitude(z2) < zmagnitude(z1)
  shows
    z1+z2 < 0
    zmagnitude(z1+z2) = zmagnitude(z1) #- zmagnitude(z2)
    z1+z2 = $-($$(zmagnitude(z1) #- zmagnitude(z2)))
  <proof>

```

zmagnitud respects multiplication

```

lemma (in int0) zmagnitud_mult:
  assumes x∈ℤ y∈ℤ
  shows zmagnitude(x·y) = zmagnitude(x) ** zmagnitude(y)
  <proof>

```

end

59 Integers 1

```

theory Int_ZF_1 imports Int_ZF_IML OrderedRing_ZF

```

```

begin

```

This theory file considers the set of integers as an ordered ring.

59.1 Integers as a ring

In this section we show that integers form a commutative ring.

The next lemma provides the condition to show that addition is distributive with respect to multiplication.

```

lemma (in int0) Int_ZF_1_1_L1: assumes A1: a∈ℤ b∈ℤ c∈ℤ
  shows
    a·(b+c) = a·b + a·c
    (b+c)·a = b·a + c·a
  <proof>

```

Integers form a commutative ring, hence we can use theorems proven in `ring0` context (locale).

```

lemma (in int0) Int_ZF_1_1_L2: shows
  IsAring(ℤ,IntegerAddition,IntegerMultiplication)

```

```

IntegerMultiplication {is commutative on}  $\mathbb{Z}$ 
ring0( $\mathbb{Z}$ , IntegerAddition, IntegerMultiplication)
<proof>

```

Zero and one are integers.

```

lemma (in int0) int_zero_one_are_int: shows  $0 \in \mathbb{Z}$   $1 \in \mathbb{Z}$ 
<proof>

```

Negative of zero is zero.

```

lemma (in int0) int_zero_one_are_intA: shows  $(-0) = 0$ 
<proof>

```

Properties with one integer.

```

lemma (in int0) Int_ZF_1_1_L4: assumes  $A1: a \in \mathbb{Z}$ 
shows
 $a+0 = a$ 
 $0+a = a$ 
 $a \cdot 1 = a$   $1 \cdot a = a$ 
 $0 \cdot a = 0$   $a \cdot 0 = 0$ 
 $(-a) \in \mathbb{Z}$   $-(-a) = a$ 
 $a-a = 0$   $a-0 = a$   $2 \cdot a = a+a$ 
<proof>

```

Properties that require two integers.

```

lemma (in int0) Int_ZF_1_1_L5: assumes  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$ 
shows
 $a+b \in \mathbb{Z}$ 
 $a-b \in \mathbb{Z}$ 
 $a \cdot b \in \mathbb{Z}$ 
 $a+b = b+a$ 
 $a \cdot b = b \cdot a$ 
 $(-b)-a = (-a)-b$ 
 $-(a+b) = (-a)-b$ 
 $-(a-b) = ((-a)+b)$ 
 $(-a) \cdot b = -(a \cdot b)$ 
 $a \cdot (-b) = -(a \cdot b)$ 
 $(-a) \cdot (-b) = a \cdot b$ 
<proof>

```

2 and 3 are integers.

```

lemma (in int0) int_two_three_are_int: shows  $2 \in \mathbb{Z}$   $3 \in \mathbb{Z}$ 
<proof>

```

Another property with two integers.

```

lemma (in int0) Int_ZF_1_1_L5B:
assumes  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$ 
shows  $a-(-b) = a+b$ 
<proof>

```


Properties that require three integers.

```
lemma (in int0) Int_ZF_1_1_L6: assumes a∈ℤ b∈ℤ c∈ℤ
  shows
    a-(b+c) = a-b-c
    a-(b-c) = a-b+c
    a·(b-c) = a·b - a·c
    (b-c)·a = b·a - c·a
  ⟨proof⟩
```

One more property with three integers.

```
lemma (in int0) Int_ZF_1_1_L6A: assumes a∈ℤ b∈ℤ c∈ℤ
  shows a+(b-c) = a+b-c
  ⟨proof⟩
```

Associativity of addition and multiplication.

```
lemma (in int0) Int_ZF_1_1_L7: assumes a∈ℤ b∈ℤ c∈ℤ
  shows
    a+b+c = a+(b+c)
    a·b·c = a·(b·c)
  ⟨proof⟩
```

59.2 Rearrangement lemmas

In this section we collect lemmas about identities related to rearranging the terms in expressions

A formula with a positive integer.

```
lemma (in int0) Int_ZF_1_2_L1: assumes 0≤a
  shows abs(a)+1 = abs(a+1)
  ⟨proof⟩
```

A formula with two integers, one positive.

```
lemma (in int0) Int_ZF_1_2_L2: assumes A1: a∈ℤ and A2: 0≤b
  shows a+(abs(b)+1)·a = (abs(b+1)+1)·a
  ⟨proof⟩
```

A couple of formulae about canceling opposite integers.

```
lemma (in int0) Int_ZF_1_2_L3: assumes A1: a∈ℤ b∈ℤ
  shows
    a+b-a = b
    a+(b-a) = b
    a+b-b = a
    a-b+b = a
    (-a)+(a+b) = b
    a+(b-a) = b
    (-b)+(a+b) = a
    a-(b+a) = -b
```

```

a-(a+b) = -b
a-(a-b) = b
a-b-a = -b
a-b - (a+b) = (-b)-b
⟨proof⟩

```

Subtracting one does not increase integers. This may be moved to a theory about ordered rings one day.

```

lemma (in int0) Int_ZF_1_2_L3A: assumes A1: a≤b
  shows a-1 ≤ b
⟨proof⟩

```

Subtracting one does not increase integers, special case.

```

lemma (in int0) Int_ZF_1_2_L3AA:
  assumes A1: a∈ℤ shows
    a-1 ≤a
    a-1 ≠ a
    ¬(a≤a-1)
    ¬(a+1 ≤a)
    ¬(1+a ≤a)
⟨proof⟩

```

A formula with a nonpositive integer.

```

lemma (in int0) Int_ZF_1_2_L4: assumes a≤0
  shows abs(a)+1 = abs(a-1)
⟨proof⟩

```

A formula with two integers, one negative.

```

lemma (in int0) Int_ZF_1_2_L5: assumes A1: a∈ℤ and A2: b≤0
  shows a+(abs(b)+1)·a = (abs(b-1)+1)·a
⟨proof⟩

```

A rearrangement with four integers.

```

lemma (in int0) Int_ZF_1_2_L6:
  assumes A1: a∈ℤ b∈ℤ c∈ℤ d∈ℤ
  shows
    a-(b-1)·c = (d-b·c)-(d-a-c)
⟨proof⟩

```

Some other rearrangements with two integers.

```

lemma (in int0) Int_ZF_1_2_L7: assumes a∈ℤ b∈ℤ
  shows
    a·b = (a-1)·b+b
    a·(b+1) = a·b+a
    (b+1)·a = b·a+a
    (b+1)·a = a+b·a
⟨proof⟩

```

Another rearrangement with two integers.

```
lemma (in int0) Int_ZF_1_2_L8:
  assumes A1:  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$ 
  shows  $a+1+(b+1) = b+a+2$ 
  <proof>
```

A couple of rearrangement with three integers.

```
lemma (in int0) Int_ZF_1_2_L9:
  assumes  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$   $c \in \mathbb{Z}$ 
  shows
     $(a-b)+(b-c) = a-c$ 
     $(a-b)-(a-c) = c-b$ 
     $a+(b+(c-a-b)) = c$ 
     $(-a)-b+c = c-a-b$ 
     $(-b)-a+c = c-a-b$ 
     $(-((-a)+b+c)) = a-b-c$ 
     $a+b+c-a = b+c$ 
     $a+b-(a+c) = b-c$ 
  <proof>
```

Another couple of rearrangements with three integers.

```
lemma (in int0) Int_ZF_1_2_L9A:
  assumes A1:  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$   $c \in \mathbb{Z}$ 
  shows  $(-(a-b-c)) = c+b-a$ 
  <proof>
```

Another rearrangement with three integers.

```
lemma (in int0) Int_ZF_1_2_L10:
  assumes A1:  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$   $c \in \mathbb{Z}$ 
  shows  $(a+1) \cdot b + (c+1) \cdot b = (c+a+2) \cdot b$ 
  <proof>
```

A technical rearrangement involving inequalities with absolute value.

```
lemma (in int0) Int_ZF_1_2_L10A:
  assumes A1:  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$   $c \in \mathbb{Z}$   $e \in \mathbb{Z}$ 
  and A2:  $\text{abs}(a \cdot b - c) \leq d$   $\text{abs}(b \cdot a - e) \leq f$ 
  shows  $\text{abs}(c - e) \leq f + d$ 
  <proof>
```

Some arithmetics.

```
lemma (in int0) Int_ZF_1_2_L11: assumes A1:  $a \in \mathbb{Z}$ 
  shows
     $a+1+2 = a+3$ 
     $a = 2 \cdot a - a$ 
  <proof>
```

A simple rearrangement with three integers.

```
lemma (in int0) Int_ZF_1_2_L12:
```

```

assumes a $\in\mathbb{Z}$  b $\in\mathbb{Z}$  c $\in\mathbb{Z}$ 
shows
  (b-c)·a = a·b - a·c
  <proof>

```

A big rearrangement with five integers.

```

lemma (in int0) Int_ZF_1_2_L13:
  assumes A1: a $\in\mathbb{Z}$  b $\in\mathbb{Z}$  c $\in\mathbb{Z}$  d $\in\mathbb{Z}$  x $\in\mathbb{Z}$ 
  shows (x+(a·x+b)+c)·d = d·(a+1)·x + (b·d+c·d)
  <proof>

```

Rerrangement about adding linear functions.

```

lemma (in int0) Int_ZF_1_2_L14:
  assumes a $\in\mathbb{Z}$  b $\in\mathbb{Z}$  c $\in\mathbb{Z}$  d $\in\mathbb{Z}$  x $\in\mathbb{Z}$ 
  shows (a·x + b) + (c·x + d) = (a+c)·x + (b+d)
  <proof>

```

A rearrangement with four integers. Again we have to use the generic set notation to use a theorem proven in different context.

```

lemma (in int0) Int_ZF_1_2_L15: assumes A1: a $\in\mathbb{Z}$  b $\in\mathbb{Z}$  c $\in\mathbb{Z}$  d $\in\mathbb{Z}$ 
  and A2: a = b-c-d
  shows
    d = b-a-c
    d = (-a)+b-c
    b = a+d+c
  <proof>

```

A rearrangement with four integers. Property of groups.

```

lemma (in int0) Int_ZF_1_2_L16:
  assumes a $\in\mathbb{Z}$  b $\in\mathbb{Z}$  c $\in\mathbb{Z}$  d $\in\mathbb{Z}$ 
  shows a+(b-c)+d = a+b+d-c
  <proof>

```

Some rearrangements with three integers. Properties of groups.

```

lemma (in int0) Int_ZF_1_2_L17:
  assumes A1: a $\in\mathbb{Z}$  b $\in\mathbb{Z}$  c $\in\mathbb{Z}$ 
  shows
    a+b-c+(c-b) = a
    a+(b+c)-c = a+b
  <proof>

```

Another rearrangement with three integers. Property of abelian groups.

```

lemma (in int0) Int_ZF_1_2_L18:
  assumes A1: a $\in\mathbb{Z}$  b $\in\mathbb{Z}$  c $\in\mathbb{Z}$ 
  shows a+b-c+(c-a) = b
  <proof>

```

59.3 Integers as an ordered ring

We already know from `Int_ZF` that integers with addition form a linearly ordered group. To show that integers form an ordered ring we need the fact that the set of nonnegative integers is closed under multiplication.

We start with the property that a product of nonnegative integers is nonnegative. The proof is by induction and the next lemma is the induction step.

```
lemma (in int0) Int_ZF_1_3_L1: assumes A1: 0 ≤ a  0 ≤ b
  and A3: 0 ≤ a·b
  shows 0 ≤ a·(b+1)
⟨proof⟩
```

Product of nonnegative integers is nonnegative.

```
lemma (in int0) Int_ZF_1_3_L2: assumes A1: 0 ≤ a  0 ≤ b
  shows 0 ≤ a·b
⟨proof⟩
```

The set of nonnegative integers is closed under multiplication.

```
lemma (in int0) Int_ZF_1_3_L2A: shows
  ℤ+ {is closed under} IntegerMultiplication
⟨proof⟩
```

Integers form an ordered ring. All theorems proven in the `ring1` context are valid in `int0` context.

```
theorem (in int0) Int_ZF_1_3_T1: shows
  IsAnOrdRing(ℤ,IntegerAddition,IntegerMultiplication,IntegerOrder)
ring1(ℤ,IntegerAddition,IntegerMultiplication,IntegerOrder)
⟨proof⟩
```

Product of integers that are greater than one is greater than one. The proof is by induction and the next step is the induction step.

```
lemma (in int0) Int_ZF_1_3_L3_indstep:
  assumes A1: 1 ≤ a  1 ≤ b
  and A2: 1 ≤ a·b
  shows 1 ≤ a·(b+1)
⟨proof⟩
```

Product of integers that are greater than one is greater than one.

```
lemma (in int0) Int_ZF_1_3_L3:
  assumes A1: 1 ≤ a  1 ≤ b
  shows 1 ≤ a·b
⟨proof⟩
```

$|a \cdot (-b)| = |(-a) \cdot b| = |(-a) \cdot (-b)| = |a \cdot b|$ This is a property of ordered rings..

```

lemma (in int0) Int_ZF_1_3_L4: assumes  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$ 
  shows
     $\text{abs}((-a) \cdot b) = \text{abs}(a \cdot b)$ 
     $\text{abs}(a \cdot (-b)) = \text{abs}(a \cdot b)$ 
     $\text{abs}((-a) \cdot (-b)) = \text{abs}(a \cdot b)$ 
  <proof>

```

Absolute value of a product is the product of absolute values. Property of ordered rings.

```

lemma (in int0) Int_ZF_1_3_L5:
  assumes A1:  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$ 
  shows  $\text{abs}(a \cdot b) = \text{abs}(a) \cdot \text{abs}(b)$ 
  <proof>

```

Double nonnegative is nonnegative. Property of ordered rings.

```

lemma (in int0) Int_ZF_1_3_L5A: assumes  $0 \leq a$ 
  shows  $0 \leq 2 \cdot a$ 
  <proof>

```

The next lemma shows what happens when one integer is not greater or equal than another.

```

lemma (in int0) Int_ZF_1_3_L6:
  assumes A1:  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$ 
  shows  $\neg(b \leq a) \longleftrightarrow a+1 \leq b$ 
  <proof>

```

Another form of stating that there are no integers between integers m and $m + 1$.

```

corollary (in int0) no_int_between: assumes A1:  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$ 
  shows  $b \leq a \vee a+1 \leq b$ 
  <proof>

```

Another way of saying what it means that one integer is not greater or equal than another.

```

corollary (in int0) Int_ZF_1_3_L6A:
  assumes A1:  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$  and A2:  $\neg(b \leq a)$ 
  shows  $a \leq b-1$ 
  <proof>

```

Yet another form of stating that there are no integers between m and $m + 1$.

```

lemma (in int0) no_int_between1:
  assumes A1:  $a \leq b$  and A2:  $a \neq b$ 
  shows
     $a+1 \leq b$ 
     $a \leq b-1$ 
  <proof>

```

We can decompose proofs into three cases: $a = b$, $a \leq b - 1$ or $a \geq b + 1$.

lemma (in int0) Int_ZF_1_3_L6B: assumes A1: $a \in \mathbb{Z}$ $b \in \mathbb{Z}$
 shows $a=b \vee (a \leq b-1) \vee (b+1 \leq a)$
 $\langle proof \rangle$

A special case of Int_ZF_1_3_L6B when $b = 0$. This allows to split the proofs in cases $a \leq -1$, $a = 0$ and $a \geq 1$.

corollary (in int0) Int_ZF_1_3_L6C: assumes A1: $a \in \mathbb{Z}$
 shows $a=0 \vee (a \leq -1) \vee (1 \leq a)$
 $\langle proof \rangle$

An integer is not less or equal zero iff it is greater or equal one.

lemma (in int0) Int_ZF_1_3_L7: assumes $a \in \mathbb{Z}$
 shows $\neg(a \leq 0) \longleftrightarrow 1 \leq a$
 $\langle proof \rangle$

Product of positive integers is positive.

lemma (in int0) Int_ZF_1_3_L8:
 assumes $a \in \mathbb{Z}$ $b \in \mathbb{Z}$
 and $\neg(a \leq 0)$ $\neg(b \leq 0)$
 shows $\neg((a \cdot b) \leq 0)$
 $\langle proof \rangle$

If $a \cdot b$ is nonnegative and b is positive, then a is nonnegative. Proof by contradiction.

lemma (in int0) Int_ZF_1_3_L9:
 assumes A1: $a \in \mathbb{Z}$ $b \in \mathbb{Z}$
 and A2: $\neg(b \leq 0)$ and A3: $a \cdot b \leq 0$
 shows $a \leq 0$
 $\langle proof \rangle$

One integer is less or equal another iff the difference is nonpositive.

lemma (in int0) Int_ZF_1_3_L10:
 assumes $a \in \mathbb{Z}$ $b \in \mathbb{Z}$
 shows $a \leq b \longleftrightarrow a-b \leq 0$
 $\langle proof \rangle$

Some conclusions from the fact that one integer is less or equal than another.

lemma (in int0) Int_ZF_1_3_L10A: assumes $a \leq b$
 shows $0 \leq b-a$
 $\langle proof \rangle$

We can simplify out a positive element on both sides of an inequality.

lemma (in int0) Int_ineq_simpl_positive:
 assumes A1: $a \in \mathbb{Z}$ $b \in \mathbb{Z}$ $c \in \mathbb{Z}$
 and A2: $a \cdot c \leq b \cdot c$ and A4: $\neg(c \leq 0)$
 shows $a \leq b$
 $\langle proof \rangle$

A technical lemma about conclusion from an inequality between absolute values. This is a property of ordered rings.

```
lemma (in int0) Int_ZF_1_3_L11:
  assumes A1:  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$ 
  and A2:  $\neg(\text{abs}(a) \leq \text{abs}(b))$ 
  shows  $\neg(\text{abs}(a) \leq 0)$ 
<proof>
```

Negative times positive is negative. This a property of ordered rings.

```
lemma (in int0) Int_ZF_1_3_L12:
  assumes  $a \leq 0$  and  $0 \leq b$ 
  shows  $a \cdot b \leq 0$ 
<proof>
```

We can multiply an inequality by a nonnegative number. This is a property of ordered rings.

```
lemma (in int0) Int_ZF_1_3_L13:
  assumes A1:  $a \leq b$  and A2:  $0 \leq c$ 
  shows
     $a \cdot c \leq b \cdot c$ 
     $c \cdot a \leq c \cdot b$ 
<proof>
```

A technical lemma about decreasing a factor in an inequality.

```
lemma (in int0) Int_ZF_1_3_L13A:
  assumes  $1 \leq a$  and  $b \leq c$  and  $(a+1) \cdot c \leq d$ 
  shows  $(a+1) \cdot b \leq d$ 
<proof>
```

We can multiply an inequality by a positive number. This is a property of ordered rings.

```
lemma (in int0) Int_ZF_1_3_L13B:
  assumes A1:  $a \leq b$  and A2:  $c \in \mathbb{Z}_+$ 
  shows
     $a \cdot c \leq b \cdot c$ 
     $c \cdot a \leq c \cdot b$ 
<proof>
```

A rearrangement with four integers and absolute value.

```
lemma (in int0) Int_ZF_1_3_L14:
  assumes A1:  $a \in \mathbb{Z}$   $b \in \mathbb{Z}$   $c \in \mathbb{Z}$   $d \in \mathbb{Z}$ 
  shows  $\text{abs}(a \cdot b) + (\text{abs}(a) + c) \cdot d = (d + \text{abs}(b)) \cdot \text{abs}(a) + c \cdot d$ 
<proof>
```

A technical lemma about what happens when one absolute value is not greater or equal than another.

```
lemma (in int0) Int_ZF_1_3_L15: assumes A1:  $m \in \mathbb{Z}$   $n \in \mathbb{Z}$ 
```


and A2: $\neg(\text{abs}(m) \leq \text{abs}(n))$
shows $n \leq \text{abs}(m)$ $m \neq 0$
<proof>

Negative of a nonnegative is nonpositive.

lemma (in int0) Int_ZF_1_3_L16: **assumes** A1: $0 \leq m$
shows $(-m) \leq 0$
<proof>

Some statements about intervals centered at 0.

lemma (in int0) Int_ZF_1_3_L17: **assumes** A1: $m \in \mathbb{Z}$
shows
 $(-\text{abs}(m)) \leq \text{abs}(m)$
 $(-\text{abs}(m)).. \text{abs}(m) \neq 0$
<proof>

The greater of two integers is indeed greater than both, and the smaller one is smaller than both.

lemma (in int0) Int_ZF_1_3_L18: **assumes** A1: $m \in \mathbb{Z}$ $n \in \mathbb{Z}$
shows
 $m \leq \text{GreaterOf}(\text{IntegerOrder}, m, n)$
 $n \leq \text{GreaterOf}(\text{IntegerOrder}, m, n)$
 $\text{SmallerOf}(\text{IntegerOrder}, m, n) \leq m$
 $\text{SmallerOf}(\text{IntegerOrder}, m, n) \leq n$
<proof>

If $|m| \leq n$, then $m \in -n..n$.

lemma (in int0) Int_ZF_1_3_L19:
assumes A1: $m \in \mathbb{Z}$ **and** A2: $\text{abs}(m) \leq n$
shows
 $(-n) \leq m$ $m \leq n$
 $m \in (-n)..n$
 $0 \leq n$
<proof>

A slight generalization of the above lemma.

lemma (in int0) Int_ZF_1_3_L19A:
assumes A1: $m \in \mathbb{Z}$ **and** A2: $\text{abs}(m) \leq n$ **and** A3: $0 \leq k$
shows $-(n+k) \leq m$
<proof>

Sets of integers that have absolute value bounded are bounded.

lemma (in int0) Int_ZF_1_3_L20:
assumes A1: $\forall x \in X. b(x) \in \mathbb{Z} \wedge \text{abs}(b(x)) \leq L$
shows $\text{IsBounded}(\{b(x). x \in X\}, \text{IntegerOrder})$
<proof>

If a set is bounded, then the absolute values of the elements of that set are bounded.

lemma (in int0) Int_ZF_1_3_L20A: **assumes** IsBounded(A,IntegerOrder)
shows $\exists L. \forall a \in A. \text{abs}(a) \leq L$
<proof>

Absolute values of integers from a finite image of integers are bounded by an integer.

lemma (in int0) Int_ZF_1_3_L20AA:
assumes A1: $\{b(x). x \in \mathbb{Z}\} \in \text{Fin}(\mathbb{Z})$
shows $\exists L \in \mathbb{Z}. \forall x \in \mathbb{Z}. \text{abs}(b(x)) \leq L$
<proof>

If absolute values of values of some integer function are bounded, then the image a set from the domain is a bounded set.

lemma (in int0) Int_ZF_1_3_L20B:
assumes $f: X \rightarrow \mathbb{Z}$ and $A \subseteq X$ and $\forall x \in A. \text{abs}(f(x)) \leq L$
shows IsBounded(f(A),IntegerOrder)
<proof>

A special case of the previous lemma for a function from integers to integers.

corollary (in int0) Int_ZF_1_3_L20C:
assumes $f: \mathbb{Z} \rightarrow \mathbb{Z}$ and $\forall m \in \mathbb{Z}. \text{abs}(f(m)) \leq L$
shows $f(\mathbb{Z}) \in \text{Fin}(\mathbb{Z})$
<proof>

A triangle inequality with three integers. Property of linearly ordered abelian groups.

lemma (in int0) int_triangle_ineq3:
assumes A1: $a \in \mathbb{Z} \quad b \in \mathbb{Z} \quad c \in \mathbb{Z}$
shows $\text{abs}(a-b-c) \leq \text{abs}(a) + \text{abs}(b) + \text{abs}(c)$
<proof>

If $a \leq c$ and $b \leq c$, then $a + b \leq 2 \cdot c$. Property of ordered rings.

lemma (in int0) Int_ZF_1_3_L21:
assumes A1: $a \leq c \quad b \leq c$ **shows** $a+b \leq 2 \cdot c$
<proof>

If an integer a is between b and $b + c$, then $|b - a| \leq c$. Property of ordered groups.

lemma (in int0) Int_ZF_1_3_L22:
assumes $a \leq b$ and $c \in \mathbb{Z}$ and $b \leq c+a$
shows $\text{abs}(b-a) \leq c$
<proof>

An application of the triangle inequality with four integers. Property of linearly ordered abelian groups.

lemma (in int0) Int_ZF_1_3_L22A:
assumes $a \in \mathbb{Z} \quad b \in \mathbb{Z} \quad c \in \mathbb{Z} \quad d \in \mathbb{Z}$

shows $\text{abs}(a-c) \leq \text{abs}(a+b) + \text{abs}(c+d) + \text{abs}(b-d)$
<proof>

If an integer a is between b and $b + c$, then $|b - a| \leq c$. Property of ordered groups. A version of Int_ZF_1_3_L22 with slightly different assumptions.

lemma (in int0) Int_ZF_1_3_L23:
assumes A1: $a \leq b$ and A2: $c \in \mathbb{Z}$ and A3: $b \leq a+c$
shows $\text{abs}(b-a) \leq c$
<proof>

59.4 Maximum and minimum of a set of integers

In this section we provide some sufficient conditions for integer subsets to have extrema (maxima and minima).

Finite nonempty subsets of integers attain maxima and minima.

theorem (in int0) Int_fin_have_max_min:
assumes A1: $A \in \text{Fin}(\mathbb{Z})$ and A2: $A \neq 0$
shows
 HasAmaximum(IntegerOrder,A)
 HasAminimum(IntegerOrder,A)
 $\text{Maximum}(\text{IntegerOrder},A) \in A$
 $\text{Minimum}(\text{IntegerOrder},A) \in A$
 $\forall x \in A. x \leq \text{Maximum}(\text{IntegerOrder},A)$
 $\forall x \in A. \text{Minimum}(\text{IntegerOrder},A) \leq x$
 $\text{Maximum}(\text{IntegerOrder},A) \in \mathbb{Z}$
 $\text{Minimum}(\text{IntegerOrder},A) \in \mathbb{Z}$
<proof>

Bounded nonempty integer subsets attain maximum and minimum.

theorem (in int0) Int_bounded_have_max_min:
assumes IsBounded(A,IntegerOrder) and $A \neq 0$
shows
 HasAmaximum(IntegerOrder,A)
 HasAminimum(IntegerOrder,A)
 $\text{Maximum}(\text{IntegerOrder},A) \in A$
 $\text{Minimum}(\text{IntegerOrder},A) \in A$
 $\forall x \in A. x \leq \text{Maximum}(\text{IntegerOrder},A)$
 $\forall x \in A. \text{Minimum}(\text{IntegerOrder},A) \leq x$
 $\text{Maximum}(\text{IntegerOrder},A) \in \mathbb{Z}$
 $\text{Minimum}(\text{IntegerOrder},A) \in \mathbb{Z}$
<proof>

Nonempty set of integers that is bounded below attains its minimum.

theorem (in int0) int_bounded_below_has_min:
assumes A1: IsBoundedBelow(A,IntegerOrder) and A2: $A \neq 0$
shows
 HasAminimum(IntegerOrder,A)

$\text{Minimum}(\text{IntegerOrder}, A) \in A$

$\forall x \in A. \text{Minimum}(\text{IntegerOrder}, A) \leq x$
<proof>

Nonempty set of integers that is bounded above attains its maximum.

theorem (in int0) int_bounded_above_has_max:
 assumes A1: IsBoundedAbove(A, IntegerOrder) and A2: A $\neq$ 0
 shows
 HasAmaximum(IntegerOrder, A)
 $\text{Maximum}(\text{IntegerOrder}, A) \in A$
 $\text{Maximum}(\text{IntegerOrder}, A) \in \mathbb{Z}$
 $\forall x \in A. x \leq \text{Maximum}(\text{IntegerOrder}, A)$
<proof>

A set defined by separation over a bounded set attains its maximum and minimum.

lemma (in int0) Int_ZF_1_4_L1:
 assumes A1: IsBounded(A, IntegerOrder) and A2: A $\neq$ 0
 and A3: $\forall q \in \mathbb{Z}. F(q) \in \mathbb{Z}$
 and A4: $K = \{F(q). q \in A\}$
 shows
 HasAmaximum(IntegerOrder, K)
 HasAminimum(IntegerOrder, K)
 $\text{Maximum}(\text{IntegerOrder}, K) \in K$
 $\text{Minimum}(\text{IntegerOrder}, K) \in K$
 $\text{Maximum}(\text{IntegerOrder}, K) \in \mathbb{Z}$
 $\text{Minimum}(\text{IntegerOrder}, K) \in \mathbb{Z}$
 $\forall q \in A. F(q) \leq \text{Maximum}(\text{IntegerOrder}, K)$
 $\forall q \in A. \text{Minimum}(\text{IntegerOrder}, K) \leq F(q)$
 IsBounded(K, IntegerOrder)
<proof>

A three element set has a maximum and minimum.

lemma (in int0) Int_ZF_1_4_L1A: assumes A1: $a \in \mathbb{Z}$ $b \in \mathbb{Z}$ $c \in \mathbb{Z}$
 shows
 $\text{Maximum}(\text{IntegerOrder}, \{a, b, c\}) \in \mathbb{Z}$
 $a \leq \text{Maximum}(\text{IntegerOrder}, \{a, b, c\})$
 $b \leq \text{Maximum}(\text{IntegerOrder}, \{a, b, c\})$
 $c \leq \text{Maximum}(\text{IntegerOrder}, \{a, b, c\})$
<proof>

Integer functions attain maxima and minima over intervals.

lemma (in int0) Int_ZF_1_4_L2:
 assumes A1: $f: \mathbb{Z} \rightarrow \mathbb{Z}$ and A2: $a \leq b$
 shows
 $\text{maxf}(f, a..b) \in \mathbb{Z}$
 $\forall c \in a..b. f(c) \leq \text{maxf}(f, a..b)$

```

 $\exists c \in a..b. f(c) = \max(f, a..b)$ 
 $\min(f, a..b) \in \mathbb{Z}$ 
 $\forall c \in a..b. \min(f, a..b) \leq f(c)$ 
 $\exists c \in a..b. f(c) = \min(f, a..b)$ 
<proof>

```

59.5 The set of nonnegative integers

The set of nonnegative integers looks like the set of natural numbers. We explore that in this section. We also rephrase some lemmas about the set of positive integers known from the theory of ordered groups.

The set of positive integers is closed under addition.

```

lemma (in int0) pos_int_closed_add:
  shows  $\mathbb{Z}_+$  {is closed under} IntegerAddition
  <proof>

```

Text expended version of the fact that the set of positive integers is closed under addition

```

lemma (in int0) pos_int_closed_add_unfolded:
  assumes  $a \in \mathbb{Z}_+$   $b \in \mathbb{Z}_+$  shows  $a+b \in \mathbb{Z}_+$ 
  <proof>

```

$\mathbb{Z}^+$ is bounded below.

```

lemma (in int0) Int_ZF_1_5_L1: shows
  IsBoundedBelow( $\mathbb{Z}^+$ , IntegerOrder)
  IsBoundedBelow( $\mathbb{Z}_+$ , IntegerOrder)
  <proof>

```

Subsets of $\mathbb{Z}^+$ are bounded below.

```

lemma (in int0) Int_ZF_1_5_L1A: assumes  $A \subseteq \mathbb{Z}^+$ 
  shows IsBoundedBelow(A, IntegerOrder)
  <proof>

```

Subsets of $\mathbb{Z}_+$ are bounded below.

```

lemma (in int0) Int_ZF_1_5_L1B: assumes A1:  $A \subseteq \mathbb{Z}_+$ 
  shows IsBoundedBelow(A, IntegerOrder)
  <proof>

```

Every nonempty subset of positive integers has a minimum.

```

lemma (in int0) Int_ZF_1_5_L1C: assumes  $A \subseteq \mathbb{Z}_+$  and  $A \neq \emptyset$ 
  shows
    HasAminimum(IntegerOrder, A)
    Minimum(IntegerOrder, A)  $\in A$ 
     $\forall x \in A. \text{Minimum(IntegerOrder, A)} \leq x$ 
  <proof>

```

Infinite subsets of $\mathbb{Z}^+$ do not have a maximum - If $A \subseteq \mathbb{Z}^+$ then for every integer we can find one in the set that is not smaller.

```
lemma (in int0) Int_ZF_1_5_L2:
  assumes A1:  $A \subseteq \mathbb{Z}^+$  and A2:  $A \notin \text{Fin}(\mathbb{Z})$  and A3:  $D \in \mathbb{Z}$ 
  shows  $\exists n \in A. D \leq n$ 
  <proof>
```

Infinite subsets of $\mathbb{Z}_+$ do not have a maximum - If $A \subseteq \mathbb{Z}_+$ then for every integer we can find one in the set that is not smaller. This is very similar to Int_ZF_1_5_L2, except we have $\mathbb{Z}_+$ instead of $\mathbb{Z}^+$ here.

```
lemma (in int0) Int_ZF_1_5_L2A:
  assumes A1:  $A \subseteq \mathbb{Z}_+$  and A2:  $A \notin \text{Fin}(\mathbb{Z})$  and A3:  $D \in \mathbb{Z}$ 
  shows  $\exists n \in A. D \leq n$ 
  <proof>
```

An integer is either positive, zero, or its opposite is positive.

```
lemma (in int0) Int_decomp: assumes  $m \in \mathbb{Z}$ 
  shows Exactly_1_of_3_holds ( $m=0, m \in \mathbb{Z}_+, (-m) \in \mathbb{Z}_+$ )
  <proof>
```

An integer is zero, positive, or it's inverse is positive.

```
lemma (in int0) int_decomp_cases: assumes  $m \in \mathbb{Z}$ 
  shows  $m=0 \vee m \in \mathbb{Z}_+ \vee (-m) \in \mathbb{Z}_+$ 
  <proof>
```

An integer is in the positive set iff it is greater or equal one.

```
lemma (in int0) Int_ZF_1_5_L3: shows  $m \in \mathbb{Z}_+ \longleftrightarrow 1 \leq m$ 
  <proof>
```

The set of positive integers is closed under multiplication. The unfolded form.

```
lemma (in int0) pos_int_closed_mul_unfold:
  assumes  $a \in \mathbb{Z}_+ \quad b \in \mathbb{Z}_+$ 
  shows  $a \cdot b \in \mathbb{Z}_+$ 
  <proof>
```

The set of positive integers is closed under multiplication.

```
lemma (in int0) pos_int_closed_mul: shows
   $\mathbb{Z}_+ \text{ {is closed under} IntegerMultiplication}$ 
  <proof>
```

It is an overkill to prove that the ring of integers has no zero divisors this way, but why not?

```
lemma (in int0) int_has_no_zero_divs:
  shows HasNoZeroDivs( $\mathbb{Z}, \text{IntegerAddition}, \text{IntegerMultiplication}$ )
  <proof>
```

Nonnegative integers are positive ones plus zero.

lemma (in int0) Int_ZF_1_5_L3A: shows $\mathbb{Z}^+ = \mathbb{Z}_+ \cup \{0\}$
<proof>

We can make a function smaller than any constant on a given interval of positive integers by adding another constant.

lemma (in int0) Int_ZF_1_5_L4:
 assumes A1: $f: \mathbb{Z} \rightarrow \mathbb{Z}$ and A2: $K \in \mathbb{Z} \ N \in \mathbb{Z}$
 shows $\exists C \in \mathbb{Z}. \forall n \in \mathbb{Z}_+. K \leq f(n) + C \longrightarrow N \leq n$
<proof>

Absolute value is identity on positive integers.

lemma (in int0) Int_ZF_1_5_L4A:
 assumes $a \in \mathbb{Z}_+$ shows $\text{abs}(a) = a$
<proof>

One and two are in $\mathbb{Z}_+$.

lemma (in int0) int_one_two_are_pos: shows $1 \in \mathbb{Z}_+ \quad 2 \in \mathbb{Z}_+$
<proof>

The image of $\mathbb{Z}_+$ by a function defined on integers is not empty.

lemma (in int0) Int_ZF_1_5_L5: assumes A1: $f: \mathbb{Z} \rightarrow X$
 shows $f(\mathbb{Z}_+) \neq \emptyset$
<proof>

If n is positive, then $n - 1$ is nonnegative.

lemma (in int0) Int_ZF_1_5_L6: assumes A1: $n \in \mathbb{Z}_+$
 shows
 $0 \leq n-1$
 $0 \in 0..(n-1)$
 $0..(n-1) \subseteq \mathbb{Z}$
<proof>

Intgers greater than one in $\mathbb{Z}_+$ belong to $\mathbb{Z}_+$. This is a property of ordered groups and follows from OrderedGroup_ZF_1_L19, but Isabelle's simplifier has problems using that result directly, so we reprove it specifically for integers.

lemma (in int0) Int_ZF_1_5_L7: assumes $a \in \mathbb{Z}_+$ and $a \leq b$
 shows $b \in \mathbb{Z}_+$
<proof>

Adding a positive integer increases integers.

lemma (in int0) Int_ZF_1_5_L7A: assumes $a \in \mathbb{Z} \quad b \in \mathbb{Z}_+$
 shows $a \leq a+b \quad a \neq a+b \quad a+b \in \mathbb{Z}$
<proof>

For any integer m the greater of m and 1 is a positive integer that is greater or equal than m . If we add 1 to it we get a positive integer that is strictly greater than m .

```

lemma (in int0) Int_ZF_1_5_L7B: assumes a $\in\mathbb{Z}$ 
  shows
    a  $\leq$  GreaterOf(IntegerOrder,1,a)
    GreaterOf(IntegerOrder,1,a)  $\in \mathbb{Z}_+$ 
    GreaterOf(IntegerOrder,1,a) + 1  $\in \mathbb{Z}_+$ 
    a  $\leq$  GreaterOf(IntegerOrder,1,a) + 1
    a  $\neq$  GreaterOf(IntegerOrder,1,a) + 1
  <proof>

```

The opposite of an element of $\mathbb{Z}_+$ cannot belong to $\mathbb{Z}_+$.

```

lemma (in int0) Int_ZF_1_5_L8: assumes a  $\in \mathbb{Z}_+$ 
  shows  $(-a) \notin \mathbb{Z}_+$ 
  <proof>

```

For every integer there is one in $\mathbb{Z}_+$ that is greater or equal.

```

lemma (in int0) Int_ZF_1_5_L9: assumes a $\in\mathbb{Z}$ 
  shows  $\exists b \in \mathbb{Z}_+. a \leq b$ 
  <proof>

```

A theorem about odd extensions. Recall from `OrdereGroup_ZF.thy` that the odd extension of an integer function f defined on $\mathbb{Z}_+$ is the odd function on $\mathbb{Z}$ equal to f on $\mathbb{Z}_+$. First we show that the odd extension is defined on $\mathbb{Z}$.

```

lemma (in int0) Int_ZF_1_5_L10: assumes f :  $\mathbb{Z}_+ \rightarrow \mathbb{Z}$ 
  shows OddExtension( $\mathbb{Z}$ , IntegerAddition, IntegerOrder, f) :  $\mathbb{Z} \rightarrow \mathbb{Z}$ 
  <proof>

```

On $\mathbb{Z}_+$, the odd extension of f is the same as f .

```

lemma (in int0) Int_ZF_1_5_L11: assumes f :  $\mathbb{Z}_+ \rightarrow \mathbb{Z}$  and a  $\in \mathbb{Z}_+$  and
  g = OddExtension( $\mathbb{Z}$ , IntegerAddition, IntegerOrder, f)
  shows g(a) = f(a)
  <proof>

```

On $-\mathbb{Z}_+$, the value of the odd extension of f is the negative of $f(-a)$.

```

lemma (in int0) Int_ZF_1_5_L12:
  assumes f :  $\mathbb{Z}_+ \rightarrow \mathbb{Z}$  and a  $\in (-\mathbb{Z}_+)$  and
  g = OddExtension( $\mathbb{Z}$ , IntegerAddition, IntegerOrder, f)
  shows g(a) = -(f(-a))
  <proof>

```

Odd extensions are odd on $\mathbb{Z}$.

```

lemma (in int0) int_oddext_is_odd:
  assumes f :  $\mathbb{Z}_+ \rightarrow \mathbb{Z}$  and a $\in\mathbb{Z}$  and
  g = OddExtension( $\mathbb{Z}$ , IntegerAddition, IntegerOrder, f)
  shows g(-a) = -(g(a))
  <proof>

```

Alternative definition of an odd function.


```

lemma (in int0) Int_ZF_1_5_L13: assumes A1:  $f: \mathbb{Z} \rightarrow \mathbb{Z}$  shows
   $(\forall a \in \mathbb{Z}. f(-a) = (-f(a))) \longleftrightarrow (\forall a \in \mathbb{Z}. -(f(-a)) = f(a))$ 
  <proof>

```

Another way of expressing the fact that odd extensions are odd.

```

lemma (in int0) int_oddext_is_odd_alt:
  assumes  $f: \mathbb{Z}_+ \rightarrow \mathbb{Z}$  and  $a \in \mathbb{Z}$  and
   $g = \text{OddExtension}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerOrder}, f)$ 
  shows  $(-g(-a)) = g(a)$ 
  <proof>

```

59.6 Functions with infinite limits

In this section we consider functions (integer sequences) that have infinite limits. An integer function has infinite positive limit if it is arbitrarily large for large enough arguments. Similarly, a function has infinite negative limit if it is arbitrarily small for small enough arguments. The material in this come mostly from the section in `OrderedGroup_ZF.thy` with the same title. Here we rewrite the theorems from that section in the notation we use for integers and add some results specific for the ordered group of integers.

If an image of a set by a function with infinite positive limit is bounded above, then the set itself is bounded above.

```

lemma (in int0) Int_ZF_1_6_L1: assumes  $f: \mathbb{Z} \rightarrow \mathbb{Z}$  and
   $\forall a \in \mathbb{Z}. \exists b \in \mathbb{Z}_+. \forall x. b \leq x \longrightarrow a \leq f(x)$  and  $A \subseteq \mathbb{Z}$  and
   $\text{IsBoundedAbove}(f(A), \text{IntegerOrder})$ 
  shows  $\text{IsBoundedAbove}(A, \text{IntegerOrder})$ 
  <proof>

```

If an image of a set defined by separation by a function with infinite positive limit is bounded above, then the set itself is bounded above.

```

lemma (in int0) Int_ZF_1_6_L2: assumes A1:  $X \neq 0$  and A2:  $f: \mathbb{Z} \rightarrow \mathbb{Z}$  and
  A3:  $\forall a \in \mathbb{Z}. \exists b \in \mathbb{Z}_+. \forall x. b \leq x \longrightarrow a \leq f(x)$  and
  A4:  $\forall x \in X. b(x) \in \mathbb{Z} \wedge f(b(x)) \leq u$ 
  shows  $\exists u. \forall x \in X. b(x) \leq u$ 
  <proof>

```

If an image of a set defined by separation by a integer function with infinite negative limit is bounded below, then the set itself is bounded above. This is dual to `Int_ZF_1_6_L2`.

```

lemma (in int0) Int_ZF_1_6_L3: assumes A1:  $X \neq 0$  and A2:  $f: \mathbb{Z} \rightarrow \mathbb{Z}$  and
  A3:  $\forall a \in \mathbb{Z}. \exists b \in \mathbb{Z}_+. \forall y. b \leq y \longrightarrow f(-y) \leq a$  and
  A4:  $\forall x \in X. b(x) \in \mathbb{Z} \wedge L \leq f(b(x))$ 
  shows  $\exists l. \forall x \in X. l \leq b(x)$ 
  <proof>

```

The next lemma combines `Int_ZF_1_6_L2` and `Int_ZF_1_6_L3` to show that if the image of a set defined by separation by a function with infinite limits is bounded, then the set itself is bounded. The proof again uses directly a fact from `OrderedGroup_ZF`.

```
lemma (in int0) Int_ZF_1_6_L4:
  assumes A1:  $X \neq 0$  and A2:  $f: \mathbb{Z} \rightarrow \mathbb{Z}$  and
  A3:  $\forall a \in \mathbb{Z}. \exists b \in \mathbb{Z}_+. \forall x. b \leq x \longrightarrow a \leq f(x)$  and
  A4:  $\forall a \in \mathbb{Z}. \exists b \in \mathbb{Z}_+. \forall y. b \leq y \longrightarrow f(-y) \leq a$  and
  A5:  $\forall x \in X. b(x) \in \mathbb{Z} \wedge f(b(x)) \leq U \wedge L \leq f(b(x))$ 
  shows  $\exists M. \forall x \in X. \text{abs}(b(x)) \leq M$ 
  <proof>
```

If a function is larger than some constant for arguments large enough, then the image of a set that is bounded below is bounded below. This is not true for ordered groups in general, but only for those for which bounded sets are finite. This does not require the function to have infinite limit, but such functions do have this property.

```
lemma (in int0) Int_ZF_1_6_L5:
  assumes A1:  $f: \mathbb{Z} \rightarrow \mathbb{Z}$  and A2:  $N \in \mathbb{Z}$  and
  A3:  $\forall m. N \leq m \longrightarrow L \leq f(m)$  and
  A4: IsBoundedBelow(A, IntegerOrder)
  shows IsBoundedBelow(f(A), IntegerOrder)
  <proof>
```

A function that has an infinite limit can be made arbitrarily large on positive integers by adding a constant. This does not actually require the function to have infinite limit, just to be larger than a constant for arguments large enough.

```
lemma (in int0) Int_ZF_1_6_L6: assumes A1:  $N \in \mathbb{Z}$  and
  A2:  $\forall m. N \leq m \longrightarrow L \leq f(m)$  and
  A3:  $f: \mathbb{Z} \rightarrow \mathbb{Z}$  and A4:  $K \in \mathbb{Z}$ 
  shows  $\exists c \in \mathbb{Z}. \forall n \in \mathbb{Z}_+. K \leq f(n) + c$ 
  <proof>
```

If a function has infinite limit, then we can add such constant such that minimum of those arguments for which the function (plus the constant) is larger than another given constant is greater than a third constant. It is not as complicated as it sounds.

```
lemma (in int0) Int_ZF_1_6_L7:
  assumes A1:  $f: \mathbb{Z} \rightarrow \mathbb{Z}$  and A2:  $K \in \mathbb{Z} \ N \in \mathbb{Z}$  and
  A3:  $\forall a \in \mathbb{Z}. \exists b \in \mathbb{Z}_+. \forall x. b \leq x \longrightarrow a \leq f(x)$ 
  shows  $\exists C \in \mathbb{Z}. N \leq \text{Minimum}(\text{IntegerOrder}, \{n \in \mathbb{Z}_+. K \leq f(n) + C\})$ 
  <proof>
```

For any integer m the function $k \mapsto m \cdot k$ has an infinite limit (or negative of that). This is why we put some properties of these functions here, even

though they properly belong to a (yet nonexistent) section on homomorphisms. The next lemma shows that the set $\{a \cdot x : x \in \mathbb{Z}\}$ can be finite only if $a = 0$.

```
lemma (in int0) Int_ZF_1_6_L8:
  assumes A1:  $a \in \mathbb{Z}$  and A2:  $\{a \cdot x. x \in \mathbb{Z}\} \in \text{Fin}(\mathbb{Z})$ 
  shows  $a = 0$ 
  <proof>
```

59.7 Miscellaneous

In this section we put some technical lemmas needed in various other places that are hard to classify.

Suppose we have an integer expression (a meta-function) F such that $F(p)|p|$ is bounded by a linear function of $|p|$, that is for some integers A, B we have $F(p)|p| \leq A|p| + B$. We show that F is then bounded. The proof is easy, we just divide both sides by $|p|$ and take the limit (just kidding).

```
lemma (in int0) Int_ZF_1_7_L1:
  assumes A1:  $\forall q \in \mathbb{Z}. F(q) \in \mathbb{Z}$  and
  A2:  $\forall q \in \mathbb{Z}. F(q) \cdot \text{abs}(q) \leq A \cdot \text{abs}(q) + B$  and
  A3:  $A \in \mathbb{Z} \ B \in \mathbb{Z}$ 
  shows  $\exists L. \forall p \in \mathbb{Z}. F(p) \leq L$ 
  <proof>
```

A lemma about splitting (not really, there is some overlap) the $\mathbb{Z} \times \mathbb{Z}$ into six subsets (cases). The subsets are as follows: first and third quadrant, and second and fourth quadrant further split by the $b = -a$ line.

```
lemma (in int0) int_plane_split_in6: assumes  $a \in \mathbb{Z} \ b \in \mathbb{Z}$ 
  shows
   $0 \leq a \wedge 0 \leq b \vee a \leq 0 \wedge b \leq 0 \vee$ 
   $a \leq 0 \wedge 0 \leq b \wedge 0 \leq a+b \vee a \leq 0 \wedge 0 \leq b \wedge a+b \leq 0 \vee$ 
   $0 \leq a \wedge b \leq 0 \wedge 0 \leq a+b \vee 0 \leq a \wedge b \leq 0 \wedge a+b \leq 0$ 
  <proof>
```

end

60 Division on integers

```
theory IntDiv_ZF_IML imports Int_ZF_1 ZF.IntDiv
```

```
begin
```

This theory translates some results from the Isabelle's `IntDiv.thy` theory to the notation used by `IsarMathLib`.

60.1 Quotient and reminder

For any integers m, n , $n > 0$ there are unique integers q, p such that $0 \leq p < n$ and $m = n \cdot q + p$. Number p in this decomposition is usually called $m \bmod n$. Standard Isabelle denotes numbers q, p as $m \text{ zdiv } n$ and $m \text{ zmod } n$, resp., and we will use the same notation.

The next lemma is sometimes called the "quotient-remainder theorem".

lemma (in int0) IntDiv_ZF_1_L1: **assumes** $m \in \mathbb{Z}$ $n \in \mathbb{Z}$
shows $m = n \cdot (m \text{ zdiv } n) + (m \text{ zmod } n)$
<proof>

If n is greater than 0 then $m \text{ zmod } n$ is between 0 and $n - 1$.

lemma (in int0) IntDiv_ZF_1_L2:
assumes A1: $m \in \mathbb{Z}$ and A2: $0 \leq n$ $n \neq 0$
shows
 $0 \leq m \text{ zmod } n$
 $m \text{ zmod } n \leq n$ $m \text{ zmod } n \neq n$
 $m \text{ zmod } n \leq n-1$
<proof>

$(m \cdot k) \text{ div } k = m$.

lemma (in int0) IntDiv_ZF_1_L3:
assumes $m \in \mathbb{Z}$ $k \in \mathbb{Z}$ and $k \neq 0$
shows
 $(m \cdot k) \text{ zdiv } k = m$
 $(k \cdot m) \text{ zdiv } k = m$
<proof>

The next lemma essentially translates `zdiv_mono1` from standard Isabelle to our notation.

lemma (in int0) IntDiv_ZF_1_L4:
assumes A1: $m \leq k$ and A2: $0 \leq n$ $n \neq 0$
shows $m \text{ zdiv } n \leq k \text{ zdiv } n$
<proof>

A quotient-remainder theorem about integers greater than a given product.

lemma (in int0) IntDiv_ZF_1_L5:
assumes A1: $n \in \mathbb{Z}_+$ and A2: $n \leq k$ and A3: $k \cdot n \leq m$
shows
 $m = n \cdot (m \text{ zdiv } n) + (m \text{ zmod } n)$
 $m = (m \text{ zdiv } n) \cdot n + (m \text{ zmod } n)$
 $(m \text{ zmod } n) \in 0..(n-1)$
 $k \leq (m \text{ zdiv } n)$
 $m \text{ zdiv } n \in \mathbb{Z}_+$
<proof>

end

61 Integer division: three as a modulus

theory IntDiv_ZF_1 **imports** IntDiv_ZF_IML

begin

This theory collects arithmetic facts about integer division and remainders that are needed for indexing constructions, in particular the properties of 3 as a modulus used to interleave three families of groups into a single $\mathbb{Z}$ -indexed family (the long exact sequence in homology).

The left summand can be reduced modulo p without changing the remainder of the sum: $(n + m) \bmod p = ((n \bmod p) + m) \bmod p$.

lemma (in int0) zmod_add_left:
 assumes $n \in \mathbb{Z}$ $m \in \mathbb{Z}$ $p \in \mathbb{Z}$
 shows $(n+m) \bmod p = ((n \bmod p) + m) \bmod p$
<proof>

61.1 Properties of three as a modulus

Three is a positive integer.

lemma (in int0) three_pos: **shows** $3 \in \mathbb{Z}_+$
<proof>

Three satisfies the positivity preconditions needed for zmod and zdiv.

lemma (in int0) three_zmod_prec: **shows** $0 \leq 3$ $3 \neq 0$
<proof>

For any integer m , $m \bmod 3$ is either 0, 1, or 2.

lemma (in int0) zmod_three_cases:
 assumes $m \in \mathbb{Z}$
 shows $m \bmod 3 = 0 \vee m \bmod 3 = 1 \vee m \bmod 3 = 2$
<proof>

If $m \bmod 3 = 0$ then $(m - 1) \bmod 3 = 2$.

corollary (in int0) zmod_0_minus_one:
 assumes $m \in \mathbb{Z}$
 and $m \bmod 3 = 0$
 shows $(m-1) \bmod 3 = 2$
<proof>

If $m \bmod 3 = 1$ then $(m - 1) \bmod 3 = 0$.

corollary (in int0) zmod_1_minus_one:
 assumes $m \in \mathbb{Z}$
 and $m \bmod 3 = 1$
 shows $(m-1) \bmod 3 = 0$
<proof>

If $m \bmod 3 = 2$ then $(m - 1) \bmod 3 = 1$.

```
corollary (in int0) zmod_2_minus_one:
  assumes m ∈ ℤ
  and m zmod 3 = 2
  shows (m-1) zmod 3 = 1
⟨proof⟩
```

If $m \bmod 3 = 2$ then $(m + 1) \bmod 3 = 0$.

```
corollary (in int0) zmod_2_plus_one:
  assumes m ∈ ℤ
  and m zmod 3 = 2
  shows (m + 1) zmod 3 = 0
⟨proof⟩
```

If $m \bmod 3 = 0$ then $(m + 1) \bmod 3 = 1$.

```
corollary (in int0) zmod_0_plus_one:
  assumes m ∈ ℤ
  and m zmod 3 = 0
  shows (m + 1) zmod 3 = 1
⟨proof⟩
```

Adding three and dividing by three increments the quotient by one.

```
corollary (in int0) zdiv_add_three:
  assumes m ∈ ℤ
  shows (m + 3) zdiv 3 = m zdiv 3 + 1
⟨proof⟩
```

Dividing $m+2$ by three and subtracting one gives the same result as dividing $m - 1$ by three.

```
corollary (in int0) zdiv_minus_one:
  assumes m ∈ ℤ
  shows ((m+2) zdiv 3) -1 = (m-1) zdiv 3
⟨proof⟩
```

end

62 Chain complexes of abelian groups

```
theory ChainComplex_ZF imports Group_ZF_6 IntDiv_ZF_IML
```

```
begin
```

A *chain complex* of abelian groups consists of a family of abelian groups $(C_n)_{n \in \mathbb{Z}}$ together with group homomorphisms $d_n: C_n \rightarrow C_{n-1}$, satisfying $d_{n-1} \circ d_n = 0$ for every $n \in \mathbb{Z}$ (i.e. the image of d_n is contained in the kernel of d_{n-1}). Here C and P are Isabelle-level functions $\mathbb{Z} \rightarrow \mathbf{Set}$ giving

the carriers and operations, and d is an Isabelle-level function giving the boundary maps.

definition (in int0) IsAchainComplex :: $[i \Rightarrow i, i \Rightarrow i, i \Rightarrow i] \Rightarrow o$ **where**

$$\text{IsAchainComplex}(C, P, d) \equiv$$

$$(\forall n \in \text{int}. \text{IsAgroup}(C(n), P(n)) \wedge (P(n)) \{\text{is commutative on}\} (C(n)))$$

$$\wedge$$

$$(\forall n \in \text{int}. \text{Homomor}(d(n), C(n), P(n), C(n - 1), P(n - 1))) \wedge$$

$$(\forall n \in \text{int}. (d(n))(C(n)) \subseteq$$

$$(d(n - 1)) - \{\text{TheNeutralElement}(C(n - 1 - 1), P(n - 1 - 1))\})$$

The locale `chain_complex` packages a fixed chain complex.

locale chain_complex = int0 +
fixes C :: $i \Rightarrow i$ **and** P :: $i \Rightarrow i$ **and** d :: $i \Rightarrow i$
assumes cplx: IsAchainComplex(C, P, d)

abbreviation (in chain_complex) zero_C
where zero_C $\equiv \lambda n. \text{TheNeutralElement}(C(n), P(n))$

Additive notation for the group operation at degree n : $x +_n y$ stands for $P_n(x, y)$.

abbreviation (in chain_complex) cgrp_add :: $[i, i, i] \Rightarrow i$
 $(_ +_ _ [65, 65, 66] 65)$ **where**
 $x +_n y \equiv P(n) \langle x, y \rangle$

Each C_n is a group.

lemma (in chain_complex) cplx_group:
assumes n $\in$ ints
shows IsAgroup(C(n), P(n))
 $\langle \text{proof} \rangle$

Each C_n is abelian.

lemma (in chain_complex) cplx_abelian:
assumes n $\in$ ints
shows (P(n)) {is commutative on} (C(n))
 $\langle \text{proof} \rangle$

The boundary map d_n is a group homomorphism from C_n to C_{n-1} .

lemma (in chain_complex) cplx_hom:
assumes n $\in$ ints
shows Homomor(d(n), C(n), P(n), C(n - 1), P(n - 1))
 $\langle \text{proof} \rangle$

The image of d_n is contained in the kernel of d_{n-1} : the boundary-squared-zero condition.

lemma (in chain_complex) bnd_sq_zero:
assumes n $\in$ ints
shows (d(n))(C(n)) $\subseteq$

$(d(n-1)) - \{zero_C(n-1-1)\}$
 $\langle proof \rangle$

For any $x \in C_n$, applying d_{n-1} to $d_n(x)$ gives the neutral element of C_{n-2} .

lemma (in chain_complex) bnd_sq_zero_elem:
 assumes $n \in \text{ints}$ and $x \in C(n)$
 shows $d(n-1)(d(n)x) = zero_C(n-1-1)$
 $\langle proof \rangle$

The composition $d_{n-1} \circ d_n$ is the constant zero map on C_n .

lemma (in chain_complex) bnd_sq_zero_2:
 assumes $n \in \text{int}$
 shows $d(n-1) \circ d(n) =$
 $\text{ConstantFunction}(C(n), zero_C(n-1-1))$
 $\langle proof \rangle$

62.1 Chain maps

A *chain map* between two chain complexes is a family of group homomorphisms $f_n: C_n \rightarrow C'_n$ that commute with the boundary maps: $d'_n \circ f_n = f_{n-1} \circ d_n$ for every $n \in \mathbb{Z}$.

definition (in int0) IsAchainMap ::
 $[i \Rightarrow i, i \Rightarrow i, i \Rightarrow i, i \Rightarrow i, i \Rightarrow i, i \Rightarrow i, i \Rightarrow i] \Rightarrow o$ **where**
 $\text{IsAchainMap}(f, C, P, d, C', P', d') \equiv$
 $(\forall n \in \text{int}. \text{Homomor}(f(n), C(n), P(n), C'(n), P'(n))) \wedge$
 $(\forall n \in \text{int}. d'(n) \circ f(n) = f(n-1) \circ d(n))$

The locale `chain_map` packages two fixed chain complexes and a chain map between them.

locale chain_map = int0 +
 fixes $C :: i \Rightarrow i$ and $P :: i \Rightarrow i$ and $d :: i \Rightarrow i$
 and $C' :: i \Rightarrow i$ and $P' :: i \Rightarrow i$ and $d' :: i \Rightarrow i$
 and $f :: i \Rightarrow i$
 assumes $\text{src_cplx}: \text{IsAchainComplex}(C, P, d)$
 and $\text{tgt_cplx}: \text{IsAchainComplex}(C', P', d')$
 and $\text{cmap}: \text{IsAchainMap}(f, C, P, d, C', P', d')$

sublocale chain_map $\subseteq$ src: chain_complex
 $\langle proof \rangle$

sublocale chain_map $\subseteq$ tgt: chain_complex **where** $C=C'$ and $P=P'$ and $d=d'$
 $\langle proof \rangle$

Each component f_n is a group homomorphism.

lemma (in chain_map) cmap_hom:
 assumes $n \in \text{ints}$
 shows $\text{Homomor}(f(n), C(n), P(n), C'(n), P'(n))$

<proof>

The chain map commutes with the boundary maps.

```
lemma (in chain_map) cmap_comm:
  assumes n ∈ ints
  shows d'(n) ∘ f(n) = f(n - 1) ∘ d(n)
  <proof>
```

end

63 Homology groups

```
theory Homology_ZF imports ChainComplex_ZF
```

```
begin
```

The *cycles* at degree n are the kernel of the boundary map d_n , i.e. the elements of C_n that are sent to zero by d_n . The *boundaries* at degree n are the image of the boundary map d_{n+1} , i.e. elements of C_n of the form $d_{n+1}(x)$ for some $x \in C_{n+1}$. The chain complex condition $d_n \circ d_{n+1} = 0$ implies that every boundary is a cycle. The n -th *homology group* is the quotient Z_n/B_n of cycles by boundaries.

```
abbreviation (in chain_complex) Cycles :: i ⇒ i where
  Cycles(n) ≡ (d(n))-{zero_C(n - 1)}
```

```
abbreviation (in chain_complex) Boundaries :: i ⇒ i where
  Boundaries(n) ≡ (d(n + 1))(C(n + 1))
```

We denote the restriction of the group operation $P(n)$ to the cycles by $\text{CycleOp}(n)$. This is the operation that makes Z_n a group in its own right.

```
abbreviation (in chain_complex) CycleOp :: i ⇒ i where
  CycleOp(n) ≡ restrict(P(n), Cycles(n) × Cycles(n))
```

The carrier of the n -th homology group is the quotient set $Z_n/B_n = Z_n/\sim$, where $x \sim y$ iff $x - y \in B_n$.

```
definition (in chain_complex) Hn :: i ⇒ i where
  Hn(n) ≡ Cycles(n) // QuotientGroupRel(Cycles(n), CycleOp(n), Boundaries(n))
```

```
definition (in chain_complex) HnOp :: i ⇒ i where
  HnOp(n) ≡ QuotientGroupOp(Cycles(n), CycleOp(n), Boundaries(n))
```

The key structural fact: every boundary is a cycle. This is a direct consequence of the boundary-squared-zero condition in the chain complex.

```
lemma (in chain_complex) boundaries_in_cycles:
  assumes n ∈ ℤ
```

shows Boundaries(n) $\subseteq$ Cycles(n)
<proof>

Cycles at degree n form a subgroup of C_n .

lemma (in chain_complex) cycles_is_subgroup:
assumes $n \in \mathbb{Z}$
shows IsSubgroup(Cycles(n), P(n))
<proof>

Cycles at degree n form a group (with the restricted operation).

lemma (in chain_complex) cycles_is_group:
assumes $n \in \mathbb{Z}$
shows IsAGroup(Cycles(n), CycleOp(n))
<proof>

The cycle group at degree n is abelian, inheriting commutativity from C_n .

lemma (in chain_complex) cycles_is_abelian:
assumes $n \in \mathbb{Z}$
shows CycleOp(n) {is commutative on} Cycles(n)
<proof>

Boundaries at degree n form a subgroup of the ambient group C_n .

lemma (in chain_complex) boundaries_subgroup_ambient:
assumes $n \in \mathbb{Z}$
shows IsSubgroup(Boundaries(n), P(n))
<proof>

Boundaries at degree n form a subgroup of the cycle group Z_n .

lemma (in chain_complex) boundaries_subgroup_cycles:
assumes $n \in \mathbb{Z}$
shows IsSubgroup(Boundaries(n), CycleOp(n))
<proof>

63.1 The n -th homology group

The n -th homology group $H_n = Z_n/B_n$ is a group.

theorem (in chain_complex) Hn_is_group:
assumes $n \in \mathbb{Z}$
shows IsAGroup(Hn(n), HnOp(n))
<proof>

The n -th homology group is abelian, as a quotient of the abelian cycle group.

theorem (in chain_complex) Hn_is_abelian:
assumes $n \in \mathbb{Z}$
shows HnOp(n) {is commutative on} Hn(n)
<proof>

63.2 Chain maps induce homomorphisms on homology

A chain map sends cycles to cycles: $d_n(z) = 0$ implies $d'_n(f_n(z)) = 0$.

```
lemma (in chain_map) cmap_maps_cycles:
  assumes n ∈ ints and z ∈ src.Cycles(n)
  shows f(n)z ∈ tgt.Cycles(n)
⟨proof⟩
```

A chain map sends boundaries to boundaries: the image of d_{n+1} maps into the image of d'_{n+1} .

```
lemma (in chain_map) cmap_maps_boundaries:
  assumes n ∈ ints and z ∈ src.Boundaries(n)
  shows f(n)z ∈ tgt.Boundaries(n)
⟨proof⟩
```

The map on homology induced by a chain map f : it sends the class $[z]$ in $H_n(C)$ to the class $[f_n(z)]$ in $H_n(C')$.

```
definition (in chain_map) Hn_map :: i ⇒ i where
  Hn_map(n) ≡
    {⟨QuotientGroupRel(src.Cycles(n), src.CycleOp(n), src.Boundaries(n)){z},
      QuotientGroupRel(tgt.Cycles(n), tgt.CycleOp(n), tgt.Boundaries(n)){f(n)z}⟩.
      z ∈ src.Cycles(n)}
```

The map induced by a chain map is a group homomorphism $H_n(C) \rightarrow H_n(C')$.

```
theorem (in chain_map) Hn_map_is_hom:
  assumes n ∈ ints
  shows Homomor(Hn_map(n), src.Hn(n), src.HnOp(n), tgt.Hn(n), tgt.HnOp(n))
⟨proof⟩
```

end

64 Exact sequences

```
theory ExactSequence_ZF imports Homology_ZF IntDiv_ZF_1
```

```
begin
```

A chain complex is a *long exact sequence* when boundaries equal cycles at every degree, i.e. $B_n = Z_n$ for all n . Since $B_n \subseteq Z_n$ always holds (boundary-squared-zero), this is equivalent to requiring $Z_n \subseteq B_n$, which in turn means every cycle is a boundary, i.e. $H_n = 0$ for all n .

```
definition (in chain_complex) IsLongExactSequence :: o where
  IsLongExactSequence ≡ ∀ n ∈ ℤ. Boundaries(n) = Cycles(n)
```

64.1 Short exact sequences of chain complexes

A *short exact sequence of chain complexes* is a pair of chain maps $f: A \rightarrow B$ and $g: B \rightarrow C$ between chain complexes such that at each degree n the sequence $0 \rightarrow A_n \rightarrow B_n \rightarrow C_n \rightarrow 0$ is a short exact sequence of abelian groups.

definition (in int0) IsASESofChainComplexes ::

$[i \Rightarrow i, i \Rightarrow i, i \Rightarrow i, i \Rightarrow i, i \Rightarrow i, i \Rightarrow i, i \Rightarrow i, i \Rightarrow i, i \Rightarrow i, i \Rightarrow i] \Rightarrow$

o where

IsASESofChainComplexes(A, PA, dA, f, B, PB, dB, g, C, PC, dC) $\equiv$
 IsAchainComplex(A, PA, dA) $\wedge$
 IsAchainComplex(B, PB, dB) $\wedge$
 IsAchainComplex(C, PC, dC) $\wedge$
 IsAchainMap(f, A, PA, dA, B, PB, dB) $\wedge$
 IsAchainMap(g, B, PB, dB, C, PC, dC) $\wedge$
 $(\forall n \in \text{int}. \text{IsAshortExactSequence}(A(n), PA(n), B(n), PB(n), C(n), PC(n), f(n), g(n)))$

The locale `ses_chain_complex` packages a short exact sequence of chain complexes $A \xrightarrow{f} B \xrightarrow{g} C$.

locale ses_chain_complex = int0 +
 fixes A :: $i \Rightarrow i$ and PA :: $i \Rightarrow i$ and dA :: $i \Rightarrow i$
 and B :: $i \Rightarrow i$ and PB :: $i \Rightarrow i$ and dB :: $i \Rightarrow i$
 and C :: $i \Rightarrow i$ and PC :: $i \Rightarrow i$ and dC :: $i \Rightarrow i$
 and f :: $i \Rightarrow i$ and g :: $i \Rightarrow i$
 assumes ses_cplx_A: IsAchainComplex(A, PA, dA)
 and ses_cplx_B: IsAchainComplex(B, PB, dB)
 and ses_cplx_C: IsAchainComplex(C, PC, dC)
 and ses_map_f: IsAchainMap(f, A, PA, dA, B, PB, dB)
 and ses_map_g: IsAchainMap(g, B, PB, dB, C, PC, dC)
 and ses_exact: $\forall n \in \text{int}. \text{IsAshortExactSequence}(A(n), PA(n), B(n), PB(n), C(n), PC(n), f(n), g(n))$

sublocale ses_chain_complex < ab: chain_map
 where C=A and P=PA and d=dA and C'=B and P'=PB and d'=dB and f=f
 $\langle \text{proof} \rangle$

sublocale ses_chain_complex < bc: chain_map
 where C=B and P=PB and d=dB and C'=C and P'=PC and d'=dC and f=g
 $\langle \text{proof} \rangle$

At each degree n , the groups and maps form a short exact sequence.

lemma (in ses_chain_complex) deg_ses:

assumes $n \in \text{ints}$
 shows IsAshortExactSequence(A(n), PA(n), B(n), PB(n), C(n), PC(n), f(n), g(n))
 $\langle \text{proof} \rangle$

Additive notation for the three complexes: $x +_n^A y, x +_n^B y, x +_n^C y$.

abbreviation (in ses_chain_complex) ses_add_A :: [i, i, i] $\Rightarrow$ i
 ($_ +^A _ _$ [65,65,66] 65) **where**
 $x +^A_n y \equiv \text{PA}(n)\langle x, y \rangle$

abbreviation (in ses_chain_complex) ses_add_B :: [i, i, i] $\Rightarrow$ i
 ($_ +^B _ _$ [65,65,66] 65) **where**
 $x +^B_n y \equiv \text{PB}(n)\langle x, y \rangle$

abbreviation (in ses_chain_complex) ses_add_C :: [i, i, i] $\Rightarrow$ i
 ($_ +^C _ _$ [65,65,66] 65) **where**
 $x +^C_n y \equiv \text{PC}(n)\langle x, y \rangle$

Additive inverse notation for the three complexes: $-^A_n x$, $-^B_n x$, $-^C_n x$.

abbreviation (in ses_chain_complex) ses_inv_A :: [i, i] $\Rightarrow$ i
 ($_ -^A _ _$ [65, 90] 80) **where**
 $-^A_n x \equiv \text{GroupInv}(A(n), \text{PA}(n))x$

abbreviation (in ses_chain_complex) ses_inv_B :: [i, i] $\Rightarrow$ i
 ($_ -^B _ _$ [65, 90] 80) **where**
 $-^B_n x \equiv \text{GroupInv}(B(n), \text{PB}(n))x$

abbreviation (in ses_chain_complex) ses_inv_C :: [i, i] $\Rightarrow$ i
 ($_ -^C _ _$ [65, 90] 80) **where**
 $-^C_n x \equiv \text{GroupInv}(C(n), \text{PC}(n))x$

64.2 The connecting homomorphism

For $b \in B_n$ with $g_n(b)$ a cycle in C_n , the element $d_n^B(b)$ lies in $\ker(g_{n-1}) = \text{im}(f_{n-1})$, so there is a unique $a \in A_{n-1}$ with $f_{n-1}(a) = d_n^B(b)$. This is the *raw lift* $\delta_n^{\text{raw}}(b)$.

definition (in ses_chain_complex) conn_lift :: [i, i] $\Rightarrow$ i **where**
 $\text{conn_lift}(n, b) \equiv \text{THE } x. x \in A(n-1) \wedge f(n-1)x = d_n^B(b)$

Helper: extract the degree-wise exactness and injectivity facts.

lemma (in ses_chain_complex) ses_hf:
 assumes $n \in \text{ints}$
 shows $\text{group_homo}(A(n), \text{PA}(n), B(n), \text{PB}(n), f(n))$
 $\langle \text{proof} \rangle$

lemma (in ses_chain_complex) ses_hg:
 assumes $n \in \text{ints}$
 shows $\text{group_homo}(B(n), \text{PB}(n), C(n), \text{PC}(n), g(n))$
 $\langle \text{proof} \rangle$

lemma (in ses_chain_complex) ses_f_inj:
 assumes $n \in \text{ints}$
 shows $f(n) \in \text{inj}(A(n), B(n))$
 $\langle \text{proof} \rangle$

```

lemma (in ses_chain_complex) ses_im_eq_ker:
  assumes n ∈ ints
  shows f(n)A(n) = g(n)-{TheNeutralElement(C(n), PC(n))}
  ⟨proof⟩

```

The raw lift exists and is unique.

```

lemma (in ses_chain_complex) conn_lift_exists:
  assumes n: n ∈ ints and bB: b ∈ B(n) and cyc: g(n)b ∈ bc.tgt.Cycles(n)
  shows ∃!x. x ∈ A(n - 1) ∧ f(n - 1)x = dB(n)b
  ⟨proof⟩

```

```

lemma (in ses_chain_complex) conn_lift_maps:
  assumes n ∈ ints b ∈ B(n) g(n)b ∈ bc.tgt.Cycles(n)
  shows f(n - 1)(conn_lift(n, b)) = dB(n)b
  ⟨proof⟩

```

```

lemma (in ses_chain_complex) conn_lift_in_A:
  assumes n ∈ ints b ∈ B(n) g(n)b ∈ bc.tgt.Cycles(n)
  shows conn_lift(n, b) ∈ A(n - 1)
  ⟨proof⟩

```

The raw lift is a cycle: $d_{n-1}^A(\text{conn_lift}(n, b)) = 0$.

```

lemma (in ses_chain_complex) conn_lift_in_cycles:
  assumes n: n ∈ ints and bB: b ∈ B(n) and cyc: g(n)b ∈ bc.tgt.Cycles(n)
  shows conn_lift(n, b) ∈ ab.src.Cycles(n - 1)
  ⟨proof⟩

```

Independence of lift: if two elements of B_n have the same image under g_n , their raw lifts represent the same homology class.

```

lemma (in ses_chain_complex) conn_lift_indep:
  assumes n: n ∈ ints and b1: b1 ∈ B(n) and b2: b2 ∈ B(n)
  and cyc: g(n)b1 ∈ bc.tgt.Cycles(n) and eq: g(n)b1 = g(n)b2
  shows ⟨conn_lift(n, b1), conn_lift(n, b2)⟩ ∈
    QuotientGroupRel(ab.src.Cycles(n - 1), ab.src.CycleOp(n - 1), ab.src.Boundaries(n - 1))
  ⟨proof⟩

```

Boundaries map to boundaries: if $c \in B_n(C)$ and b is a lift of c , then $\text{conn_lift}(n, b) \in B_{n-1}(A)$.

```

lemma (in ses_chain_complex) conn_lift_bnd_to_bnd:
  assumes n: n ∈ ints and bB: b ∈ B(n)
  and bnd: g(n)b ∈ bc.tgt.Boundaries(n)
  shows conn_lift(n, b) ∈ ab.src.Boundaries(n - 1)
  ⟨proof⟩

```

The connecting homomorphism $\delta_n: H_n(C) \rightarrow H_{n-1}(A)$ is defined by choosing any lift $b \in B_n$ with $g_n(b) = c$ and mapping $[c]$ to $[\text{conn_lift}(n, b)]$. By independence of lift and the boundary condition, this is well-defined.

definition (in ses_chain_complex) delta :: i $\Rightarrow$ i where

```
delta(n)  $\equiv$ 
  {⟨QuotientGroupRel(bc.tgt.Cycles(n), bc.tgt.CycleOp(n), bc.tgt.Boundaries(n)) {g(n)b},
    QuotientGroupRel(ab.src.Cycles(n - 1), ab.src.CycleOp(n - 1),
ab.src.Boundaries(n - 1))
    {conn_lift(n, b)}⟩.
  b  $\in$  {x  $\in$  B(n). g(n)x  $\in$  bc.tgt.Cycles(n)}}}
```

The connecting map is a well-defined function from $H_n(C)$ to $H_{n-1}(A)$.

lemma (in ses_chain_complex) delta_fun:

```
assumes n: n  $\in$  ints
shows delta(n) : bc.tgt.Hn(n)  $\rightarrow$  ab.src.Hn(n - 1)
⟨proof⟩
```

The connecting homomorphism $\delta_n: H_n(C) \rightarrow H_{n-1}(A)$ is a group homomorphism.

theorem (in ses_chain_complex) delta_is_hom:

```
assumes n: n  $\in$  ints
shows Homomor(delta(n), bc.tgt.Hn(n), bc.tgt.HnOp(n),
ab.src.Hn(n - 1), ab.src.HnOp(n - 1))
⟨proof⟩
```

The map induced by f on H_{n-1} evaluates on a coset representative: for $z \in Z_{n-1}(A)$ it sends $[z]$ to $[f_{n-1}(z)]$ in $H_{n-1}(B)$.

lemma (in ses_chain_complex) ab_hn_map_apply:

```
assumes n: n  $\in$  ints and z: z  $\in$  ab.src.Cycles(n - 1)
shows ab.Hn_map(n - 1)
  (QuotientGroupRel(ab.src.Cycles(n - 1), ab.src.CycleOp(n - 1), ab.src.Boundaries(n
- 1)) {z})
  = QuotientGroupRel(ab.tgt.Cycles(n - 1), ab.tgt.CycleOp(n - 1), ab.tgt.Boundaries(n
- 1)) {f(n - 1)z}
⟨proof⟩
```

The boundary $d_n^B(b)$ represents the zero class in $H_{n-1}(B)$ for any $b \in B_n$.

lemma (in ses_chain_complex) dBb_class_zero:

```
assumes n: n  $\in$  ints and b: b  $\in$  B(n)
shows QuotientGroupRel(ab.tgt.Cycles(n - 1), ab.tgt.CycleOp(n - 1),
ab.tgt.Boundaries(n - 1)) {dB(n)b}
  = TheNeutralElement(ab.tgt.Hn(n - 1), ab.tgt.HnOp(n - 1))
⟨proof⟩
```

Exactness at $H_{n-1}(A)$: the image of δ_n equals the kernel of $f_*: H_{n-1}(A) \rightarrow H_{n-1}(B)$.

theorem (in ses_chain_complex) les_exact_at_HnA:

```
assumes n: n  $\in$  ints
shows IsExactAt(delta(n), bc.tgt.Hn(n), ab.Hn_map(n - 1),
ab.tgt.Hn(n - 1), ab.tgt.HnOp(n - 1))
⟨proof⟩
```

Exactness at $H_n(B)$: the image of f_* equals the kernel of g_* .

```

theorem (in ses_chain_complex) les_exact_at_HnB:
  assumes n: n ∈ ints
  shows IsExactAt(ab.Hn_map(n), ab.src.Hn(n), bc.Hn_map(n),
                  bc.tgt.Hn(n), bc.tgt.HnOp(n))
<proof>

```

Exactness at $H_n(C)$: the image of g_* equals the kernel of δ_n .

```

theorem (in ses_chain_complex) les_exact_at_HnC:
  assumes n: n ∈ ints
  shows IsExactAt(bc.Hn_map(n), ab.tgt.Hn(n), delta(n),
                  ab.src.Hn(n - 1), ab.src.HnOp(n - 1))
<proof>

```

The long exact sequence in homology: for each degree $n \in \mathbb{Z}$, the sequence $\dots \rightarrow H_n(A) \xrightarrow{f_*} H_n(B) \xrightarrow{g_*} H_n(C) \xrightarrow{\delta_n} H_{n-1}(A) \rightarrow \dots$ is exact at every term.

```

theorem (in ses_chain_complex) long_exact_sequence:
  assumes n: n ∈ ints
  shows
    IsExactAt(delta(n), bc.tgt.Hn(n), ab.Hn_map(n - 1),
              ab.tgt.Hn(n - 1), ab.tgt.HnOp(n - 1)) ∧
    IsExactAt(ab.Hn_map(n), ab.src.Hn(n), bc.Hn_map(n),
              bc.tgt.Hn(n), bc.tgt.HnOp(n)) ∧
    IsExactAt(bc.Hn_map(n), ab.tgt.Hn(n), delta(n),
              ab.src.Hn(n - 1), ab.src.HnOp(n - 1))
<proof>

```

64.3 The long exact sequence as a chain complex

We encode the long exact sequence in homology as a single chain complex (the *interleaved complex*), indexed by $\mathbb{Z}$ via the identification

$$G(3k) = H_k(A), \quad G(3k-1) = H_k(B), \quad G(3k-2) = H_k(C),$$

with boundary maps $d(3k) = f_{k,*}$, $d(3k-1) = g_{k,*}$, $d(3k-2) = \delta_k$. The three cases are detected by $m \bmod 3$.

```

definition (in ses_chain_complex) les_G :: i ⇒ i where
  les_G(m) ≡
    if m zmod 3 = 0
    then ab.src.Hn(m zdiv 3)
    else if m zmod 3 = 2
    then ab.tgt.Hn((m + 1) zdiv 3)
    else bc.tgt.Hn((m + 2) zdiv 3)

```

```

definition (in ses_chain_complex) les_opG :: i ⇒ i where
  les_opG(m) ≡

```



```

    if m zmod 3 = 0
    then ab.src.HnOp(m zdiv 3)
    else if m zmod 3 = 2
    then ab.tgt.HnOp((m + 1) zdiv 3)
    else bc.tgt.HnOp((m + 2) zdiv 3)

```

definition (in ses_chain_complex) les_d :: i ⇒ i where

```

les_d(m) ≡
  if m zmod 3 = 0
  then ab.Hn_map(m zdiv 3)
  else if m zmod 3 = 2
  then bc.Hn_map((m + 1) zdiv 3)
  else delta((m + 2) zdiv 3)

```

When $m \bmod 3 = 0$ the interleaved boundary satisfies $d_{m-1} \circ d_m = 0$: $d_m = f_{k,*}$ and $d_{m-1} = g_{k,*}$ (where $k = m \operatorname{div} 3$), and $g_{k,*} \circ f_{k,*} = 0$ by exactness at $H_k(B)$.

lemma (in ses_chain_complex) les_dd_zero_mod0:

```

  assumes n: n ∈ ints
    and nmod: n zmod 3 = 0
    and X: X ∈ les_G(n)
  shows les_d(n - 1)(les_d(n)X) =
    TheNeutralElement(les_G(n - 1 - 1), les_opG(n - 1 - 1))

```

<proof>

When $m \bmod 3 = 2$ the interleaved boundary satisfies $d_{m-1} \circ d_m = 0$: $d_m = g_{k,*}$ and $d_{m-1} = \delta_k$ (where $k = (m + 1) \operatorname{div} 3$), and $\delta_k \circ g_{k,*} = 0$ by exactness at $H_k(C)$.

lemma (in ses_chain_complex) les_dd_zero_mod2:

```

  assumes n: n ∈ ints
    and nmod: n zmod 3 = 2
    and X: X ∈ les_G(n)
  shows les_d(n - 1)(les_d(n)X) =
    TheNeutralElement(les_G(n - 1 - 1), les_opG(n - 1 - 1))

```

<proof>

When $m \bmod 3 = 1$ the interleaved boundary satisfies $d_{m-1} \circ d_m = 0$: $d_m = \delta_k$ and $d_{m-1} = f_{k-1,*}$ (where $k = (m + 2) \operatorname{div} 3$), and $f_{k-1,*} \circ \delta_k = 0$ by exactness at $H_{k-1}(A)$.

lemma (in ses_chain_complex) les_dd_zero_mod1:

```

  assumes n: n ∈ ints
    and nmod: n zmod 3 = 1
    and X: X ∈ les_G(n)
  shows les_d(n - 1)(les_d(n)X) =
    TheNeutralElement(les_G(n - 1 - 1), les_opG(n - 1 - 1))

```

<proof>

The n -th group in the long exact sequence is a group.

```

lemma (in ses_chain_complex) les_G_is_group:
  assumes n: n ∈ ints
  shows IsAgroup(les_G(n), les_opG(n))
⟨proof⟩

```

The n -th group in the long exact sequence is abelian.

```

lemma (in ses_chain_complex) les_G_is_abelian:
  assumes n: n ∈ ints
  shows les_opG(n) {is commutative on} les_G(n)
⟨proof⟩

```

The n -th boundary map in the long exact sequence is a group homomorphism.

```

lemma (in ses_chain_complex) les_d_is_hom:
  assumes n: n ∈ ints
  shows Homom(les_d(n), les_G(n), les_opG(n), les_G(n - 1), les_opG(n - 1))
⟨proof⟩

```

The interleaved long exact sequence data forms a chain complex.

```

theorem (in ses_chain_complex) les_is_chain_complex:
  shows IsAchainComplex(les_G, les_opG, les_d)
⟨proof⟩

```

```

sublocale ses_chain_complex ⊆ les: chain_complex where C=les_G and P=les_opG
and d=les_d
⟨proof⟩

```

```

corollary (in ses_chain_complex) les_is_long_exact_sequence:
  shows les.IsLongExactSequence
⟨proof⟩

```

64.4 The snake lemma

A *morphism of short exact sequences* is a commutative diagram

$$0 \rightarrow A \xrightarrow{f} B \xrightarrow{g} C \rightarrow 0$$

on top and

$$0 \rightarrow A' \xrightarrow{f'} B' \xrightarrow{g'} C' \rightarrow 0$$

on the bottom, connected by vertical group homomorphisms $\alpha: A \rightarrow A'$, $\beta: B \rightarrow B'$, $\gamma: C \rightarrow C'$ such that $f' \circ \alpha = \beta \circ f$ and $g' \circ \beta = \gamma \circ g$.

```

locale ses_morphism =
  fixes A  :: i and PA :: i and f  :: i
    and B  :: i and PB :: i and g  :: i
    and C  :: i and PC :: i

```

```

    and A' :: i and PA':: i and f' :: i
    and B' :: i and PB':: i and g' :: i
    and C' :: i and PC':: i
    and  $\alpha$  :: i and  $\beta$  :: i and  $\gamma$  :: i
  assumes row1:      IsAshortExactSequence(A, PA, B, PB, C, PC, f,
g)
    and row2:      IsAshortExactSequence(A', PA', B', PB', C', PC',
f', g')
    and hom_ $\alpha$ :    Homomor( $\alpha$ , A, PA, A', PA')
    and hom_ $\beta$ :    Homomor( $\beta$ , B, PB, B', PB')
    and hom_ $\gamma$ :    Homomor( $\gamma$ , C, PC, C', PC')
    and comm_left:   f' 0  $\alpha$  =  $\beta$  0 f
    and comm_right:  g' 0  $\beta$  =  $\gamma$  0 g
    and comm_A':     PA' {is commutative on} A'
    and comm_B':     PB' {is commutative on} B'
    and comm_C':     PC' {is commutative on} C'

```

Kernel of each vertical map.

```

abbreviation (in ses_morphism) ker_ $\alpha$  :: i where
  ker_ $\alpha$   $\equiv$   $\alpha$ -{TheNeutralElement(A', PA')}

```

```

abbreviation (in ses_morphism) ker_ $\beta$  :: i where
  ker_ $\beta$   $\equiv$   $\beta$ -{TheNeutralElement(B', PB')}

```

```

abbreviation (in ses_morphism) ker_ $\gamma$  :: i where
  ker_ $\gamma$   $\equiv$   $\gamma$ -{TheNeutralElement(C', PC')}

```

Cokernel carrier and operation for each vertical map.

```

abbreviation (in ses_morphism) coker_ $\alpha$  :: i where
  coker_ $\alpha$   $\equiv$  A' // QuotientGroupRel(A', PA',  $\alpha$ A)

```

```

abbreviation (in ses_morphism) cokerOp_ $\alpha$  :: i where
  cokerOp_ $\alpha$   $\equiv$  QuotientGroupOp(A', PA',  $\alpha$ A)

```

```

abbreviation (in ses_morphism) coker_ $\beta$  :: i where
  coker_ $\beta$   $\equiv$  B' // QuotientGroupRel(B', PB',  $\beta$ B)

```

```

abbreviation (in ses_morphism) cokerOp_ $\beta$  :: i where
  cokerOp_ $\beta$   $\equiv$  QuotientGroupOp(B', PB',  $\beta$ B)

```

```

abbreviation (in ses_morphism) coker_ $\gamma$  :: i where
  coker_ $\gamma$   $\equiv$  C' // QuotientGroupRel(C', PC',  $\gamma$ C)

```

```

abbreviation (in ses_morphism) cokerOp_ $\gamma$  :: i where
  cokerOp_ $\gamma$   $\equiv$  QuotientGroupOp(C', PC',  $\gamma$ C)

```

Basic structural lemmas: group types extracted from the SES rows.

```

lemma (in ses_morphism) grpA: IsAgroup(A, PA)
  <proof>

```

lemma (in ses_morphism) grpB: IsAgroup(B, PB)
 $\langle proof \rangle$

lemma (in ses_morphism) grpC: IsAgroup(C, PC)
 $\langle proof \rangle$

lemma (in ses_morphism) grpA': IsAgroup(A', PA')
 $\langle proof \rangle$

lemma (in ses_morphism) grpB': IsAgroup(B', PB')
 $\langle proof \rangle$

lemma (in ses_morphism) grpC': IsAgroup(C', PC')
 $\langle proof \rangle$

The maps as typed functions.

lemma (in ses_morphism) f_fun: $f : A \rightarrow B$
 $\langle proof \rangle$

lemma (in ses_morphism) g_fun: $g : B \rightarrow C$
 $\langle proof \rangle$

lemma (in ses_morphism) f'_fun: $f' : A' \rightarrow B'$
 $\langle proof \rangle$

lemma (in ses_morphism) g'_fun: $g' : B' \rightarrow C'$
 $\langle proof \rangle$

lemma (in ses_morphism) α _fun: $\alpha : A \rightarrow A'$
 $\langle proof \rangle$

lemma (in ses_morphism) β _fun: $\beta : B \rightarrow B'$
 $\langle proof \rangle$

lemma (in ses_morphism) γ _fun: $\gamma : C \rightarrow C'$
 $\langle proof \rangle$

Image of f equals kernel of g (exactness at B).

lemma (in ses_morphism) row1_im_eq_ker:
 $fA = g\text{-}\{\text{TheNeutralElement}(C, PC)\}$
 $\langle proof \rangle$

Image of f' equals kernel of g' (exactness at B').

lemma (in ses_morphism) row2_im_eq_ker:
 $f'A' = g'\text{-}\{\text{TheNeutralElement}(C', PC')\}$
 $\langle proof \rangle$

f is injective.

lemma (in ses_morphism) f_inj: $f \in \text{inj}(A, B)$
 <proof>

f' is injective.

lemma (in ses_morphism) f'_inj: $f' \in \text{inj}(A', B')$
 <proof>

g is surjective.

lemma (in ses_morphism) g_surj: $gB = C$
 <proof>

g' is surjective.

lemma (in ses_morphism) g'_surj: $g'B' = C'$
 <proof>

Given $b \in B$ such that $\beta(b) \in \text{im}(f')$, the unique preimage of $\beta(b)$ under f' in A' .

definition (in ses_morphism) snake_a_of_b :: $i \Rightarrow i$ where
 snake_a_of_b(b) $\equiv$ THE a'. a' $\in A' \wedge f'a' = \beta b$

The snake connecting map. For $c \in \ker(\gamma)$, we lift c to some $b \in B$ with $g(b) = c$; then $\beta(b) \in \ker(g')$ by commutativity, so $\beta(b) = f'(a')$ for a unique $a' \in A'$, and $\delta(c) = [a']$ in $\text{coker}(\alpha)$. We define it as a replacement indexed by (c, b) -pairs with $g(b) = c$; we prove below that the cokernel class is independent of the choice of lift b , so this is a well-defined function.

definition (in ses_morphism) snake_delta :: i where
 snake_delta $\equiv$
 $\{\langle \text{fst}(p), \text{QuotientGroupRel}(A', PA', \alpha A)\{\text{snake_a_of_b}(\text{snd}(p))\} \rangle\}.$
 $p \in \{p \in \ker_\gamma \times B. g(\text{snd}(p)) = \text{fst}(p)\}$

64.5 The snake connecting homomorphism

The image of α is a normal subgroup of (A', PA') .

lemma (in ses_morphism) alpha_img_normal:
 shows IsAnormalSubgroup($A', PA', \alpha A$)
 <proof>

The kernel of γ is a subgroup of (C, PC) .

lemma (in ses_morphism) ker_gamma_subgroup:
 shows IsASubgroup($\ker_\gamma, PC$)
 <proof>

For $b \in B$ with $g(b) \in \ker(\gamma)$, $\beta(b)$ lies in $\text{im}(f')$.

lemma (in ses_morphism) beta_b_in_im_f':
 assumes bB: $b \in B$ and gb_ker: $gb \in \ker_\gamma$
 shows $\beta b \in f'A'$

<proof>

The preimage a' of $\beta(b)$ under f' exists uniquely when $g(b) \in \ker(\gamma)$.

lemma (in ses_morphism) snake_a_of_b_exists_unique:
 assumes bB: $b \in B$ and gb_ker: $gb \in \ker_\gamma$
 shows $\exists! a'. a' \in A' \wedge f'a' = \beta b$

<proof>

$\text{snake_a_of_b}(b) \in A'$ when $g(b) \in \ker(\gamma)$.

lemma (in ses_morphism) snake_a_of_b_in_A':
 assumes bB: $b \in B$ and gb_ker: $gb \in \ker_\gamma$
 shows $\text{snake_a_of_b}(b) \in A'$

<proof>

$f'(\text{snake_a_of_b}(b)) = \beta(b)$ when $g(b) \in \ker(\gamma)$.

lemma (in ses_morphism) snake_a_of_b_maps:
 assumes bB: $b \in B$ and gb_ker: $gb \in \ker_\gamma$
 shows $f'(\text{snake_a_of_b}(b)) = \beta b$

<proof>

The cokernel class of snake_a_of_b is independent of the choice of lift b : if $g(b_1) = g(b_2)$ and both lie in $\ker(\gamma)$, then $[\text{snake_a_of_b}(b_1)] = [\text{snake_a_of_b}(b_2)]$ in $\text{coker}(\alpha)$.

lemma (in ses_morphism) snake_class_independent:
 assumes b1B: $b_1 \in B$ and b2B: $b_2 \in B$
 and gb_eq: $gb_1 = gb_2$ and gb_ker: $gb_1 \in \ker_\gamma$
 shows $\text{QuotientGroupRel}(A', PA', \alpha A)\{\text{snake_a_of_b}(b_1)\} =$
 $\text{QuotientGroupRel}(A', PA', \alpha A)\{\text{snake_a_of_b}(b_2)\}$

<proof>

snake_delta is a function from $\ker(\gamma)$ to $\text{coker}(\alpha)$.

lemma (in ses_morphism) snake_delta_is_fun:
 shows $\text{snake_delta} : \ker_\gamma \rightarrow \text{coker}_\alpha$

<proof>

The snake connecting map is a group homomorphism from $\ker(\gamma)$ to $\text{coker}(\alpha)$.

theorem (in ses_morphism) snake_delta_is_hom:
 shows $\text{Homomor}(\text{snake_delta}, \ker_\gamma, \text{restrict}(\text{PC}, \ker_\gamma \times \ker_\gamma), \text{coker}_\alpha,$
 $\text{cokerOp}_\alpha)$

<proof>

end

65 Integers 2

theory Int_ZF_2 imports func_ZF_1 Int_ZF_1 IntDiv_ZF_IML Group_ZF_3

begin

In this theory file we consider the properties of integers that are needed for the real numbers construction in `Real_ZF` series.

65.1 Slopes

In this section we study basic properties of slopes - the integer almost homomorphisms. The general definition of an almost homomorphism f on a group G written in additive notation requires the set $\{f(m+n) - f(m) - f(n) : m, n \in G\}$ to be finite. In this section we establish a definition that is equivalent for integers: that for all integer m, n we have $|f(m+n) - f(m) - f(n)| \leq L$ for some L .

First we extend the standard notation for integers with notation related to slopes. We define slopes as almost homomorphisms on the additive group of integers. The set of slopes is denoted $\mathcal{S}$. We also define "positive" slopes as those that take infinite number of positive values on positive integers. We write $\delta(s, m, n)$ to denote the homomorphism difference of s at m, n (i.e. the expression $s(m+n) - s(m) - s(n)$). We denote $\max \delta(s)$ the maximum absolute value of homomorphism difference of s as m, n range over integers. If s is a slope, then the set of homomorphism differences is finite and this maximum exists. In `Group_ZF_3` we define the equivalence relation on almost homomorphisms using the notion of a quotient group relation and use " $\approx$ " to denote it. As here this symbol seems to be hogged by the standard Isabelle, we will use " $\sim$ " instead " $\approx$ ". We show in this section that $s \sim r$ iff for some L we have $|s(m) - r(m)| \leq L$ for all integer m . The " $+$ " denotes the first operation on almost homomorphisms. For slopes this is addition of functions defined in the natural way. The " $\circ$ " symbol denotes the second operation on almost homomorphisms (see `Group_ZF_3` for definition), defined for the group of integers. In short " $\circ$ " is the composition of slopes. The " $^{-1}$ " symbol acts as an infix operator that assigns the value $\min\{n \in \mathbb{Z}_+ : p \leq f(n)\}$ to a pair (of sets) f and p . In application f represents a function defined on $\mathbb{Z}_+$ and p is a positive integer. We choose this notation because we use it to construct the right inverse in the ring of classes of slopes and show that this ring is in fact a field. To study the homomorphism difference of the function defined by $p \mapsto f^{-1}(p)$ we introduce the symbol ε defined as $\varepsilon(f, \langle m, n \rangle) = f^{-1}(m+n) - f^{-1}(m) - f^{-1}(n)$. Of course the intention is to use the fact that $\varepsilon(f, \langle m, n \rangle)$ is the homomorphism difference of the function g defined as $g(m) = f^{-1}(m)$. We also define $\gamma(s, m, n)$ as the expression $\delta(f, m, -n) + s(0) - \delta(f, n, -n)$. This is useful because of the identity $f(m-n) = \gamma(m, n) + f(m) - f(n)$ that allows to obtain bounds on the value of a slope at the difference of two integers. For every integer m we introduce notation m^S defined by $m^E(n) = m \cdot n$. The mapping $q \mapsto q^S$

embeds integers into $\mathcal{S}$ preserving the order, (that is, maps positive integers into $\mathcal{S}_+$).

locale int1 = int0 +

```

fixes slopes ( $\mathcal{S}$  )
defines slopes_def[simp]:  $\mathcal{S} \equiv \text{AlmostHoms}(\mathbb{Z}, \text{IntegerAddition})$ 

fixes posslopes ( $\mathcal{S}_+$ )
defines posslopes_def[simp]:  $\mathcal{S}_+ \equiv \{s \in \mathcal{S}. s(\mathbb{Z}_+) \cap \mathbb{Z}_+ \notin \text{Fin}(\mathbb{Z})\}$ 

fixes  $\delta$ 
defines  $\delta\_def[simp]$ :  $\delta(s, m, n) \equiv s(m+n) - s(m) - s(n)$ 

fixes maxhomdiff ( $\text{max}\delta$  )
defines maxhomdiff_def[simp]:
 $\text{max}\delta(s) \equiv \text{Maximum}(\text{IntegerOrder}, \{\text{abs}(\delta(s, m, n)). \langle m, n \rangle \in \mathbb{Z} \times \mathbb{Z}\})$ 

fixes AlEqRel
defines AlEqRel_def[simp]:
 $\text{AlEqRel} \equiv \text{QuotientGroupRel}(\mathcal{S}, \text{AlHomOp1}(\mathbb{Z}, \text{IntegerAddition}), \text{FinRangeFunctions}(\mathbb{Z}, \mathbb{Z}))$ 

fixes AlEq (infix  $\sim$  68)
defines AlEq_def[simp]:  $s \sim r \equiv \langle s, r \rangle \in \text{AlEqRel}$ 

fixes slope_add (infix + 70)
defines slope_add_def[simp]:  $s + r \equiv \text{AlHomOp1}(\mathbb{Z}, \text{IntegerAddition}) \langle s, r \rangle$ 

fixes slope_comp (infix  $\circ$  70)
defines slope_comp_def[simp]:  $s \circ r \equiv \text{AlHomOp2}(\mathbb{Z}, \text{IntegerAddition}) \langle s, r \rangle$ 

fixes neg (- [90] 91)
defines neg_def[simp]:  $-s \equiv \text{GroupInv}(\mathbb{Z}, \text{IntegerAddition}) 0 s$ 

fixes slope_inv (infix  $^{-1}$  71)
defines slope_inv_def[simp]:
 $f^{-1}(p) \equiv \text{Minimum}(\text{IntegerOrder}, \{n \in \mathbb{Z}_+. p \leq f(n)\})$ 
fixes  $\varepsilon$ 
defines  $\varepsilon\_def[simp]$ :
 $\varepsilon(f, p) \equiv f^{-1}(\text{fst}(p) + \text{snd}(p)) - f^{-1}(\text{fst}(p)) - f^{-1}(\text{snd}(p))$ 

fixes  $\gamma$ 
defines  $\gamma\_def[simp]$ :
 $\gamma(s, m, n) \equiv \delta(s, m, -n) - \delta(s, n, -n) + s(0)$ 

fixes intembed ( $^{\mathcal{S}}$ )
defines intembed_def[simp]:  $m^{\mathcal{S}} \equiv \{\langle n, m \cdot n \rangle. n \in \mathbb{Z}\}$ 

```

We can use theorems proven in the group1 context.

lemma (in int1) Int_ZF_2_1_L1: shows group1($\mathbb{Z}$, IntegerAddition)
 $\langle proof \rangle$

Type information related to the homomorphism difference expression.

lemma (in int1) Int_ZF_2_1_L2: assumes $f \in S$ and $n \in \mathbb{Z}$ $m \in \mathbb{Z}$
 shows
 $m+n \in \mathbb{Z}$
 $f(m+n) \in \mathbb{Z}$
 $f(m) \in \mathbb{Z} \quad f(n) \in \mathbb{Z}$
 $f(m) + f(n) \in \mathbb{Z}$
 $\text{HomDiff}(\mathbb{Z}, \text{IntegerAddition}, f, \langle m, n \rangle) \in \mathbb{Z}$
 $\langle proof \rangle$

Type information related to the homomorphism difference expression.

lemma (in int1) Int_ZF_2_1_L2A:
 assumes $f: \mathbb{Z} \rightarrow \mathbb{Z}$ and $n \in \mathbb{Z} \quad m \in \mathbb{Z}$
 shows
 $m+n \in \mathbb{Z}$
 $f(m+n) \in \mathbb{Z} \quad f(m) \in \mathbb{Z} \quad f(n) \in \mathbb{Z}$
 $f(m) + f(n) \in \mathbb{Z}$
 $\text{HomDiff}(\mathbb{Z}, \text{IntegerAddition}, f, \langle m, n \rangle) \in \mathbb{Z}$
 $\langle proof \rangle$

Slopes map integers into integers.

lemma (in int1) Int_ZF_2_1_L2B:
 assumes A1: $f \in S$ and A2: $m \in \mathbb{Z}$
 shows $f(m) \in \mathbb{Z}$
 $\langle proof \rangle$

The homomorphism difference in multiplicative notation is defined as the expression $s(m \cdot n) \cdot (s(m) \cdot s(n))^{-1}$. The next lemma shows that in the additive notation used for integers the homomorphism difference is $f(m+n) - f(m) - f(n)$ which we denote as $\delta(f, m, n)$.

lemma (in int1) Int_ZF_2_1_L3:
 assumes $f: \mathbb{Z} \rightarrow \mathbb{Z}$ and $m \in \mathbb{Z} \quad n \in \mathbb{Z}$
 shows $\text{HomDiff}(\mathbb{Z}, \text{IntegerAddition}, f, \langle m, n \rangle) = \delta(f, m, n)$
 $\langle proof \rangle$

The next formula restates the definition of the homomorphism difference to express the value an almost homomorphism on a sum.

lemma (in int1) Int_ZF_2_1_L3A:
 assumes A1: $f \in S$ and A2: $m \in \mathbb{Z} \quad n \in \mathbb{Z}$
 shows
 $f(m+n) = f(m) + (f(n) + \delta(f, m, n))$
 $\langle proof \rangle$

The homomorphism difference of any integer function is integer.

lemma (in int1) Int_ZF_2_1_L3B:

assumes $f:\mathbb{Z}\rightarrow\mathbb{Z}$ **and** $m\in\mathbb{Z}$ $n\in\mathbb{Z}$
shows $\delta(f,m,n) \in \mathbb{Z}$
 $\langle proof \rangle$

The value of an integer function at a sum expressed in terms of δ .

lemma (in int1) Int_ZF_2_1_L3C: **assumes** A1: $f:\mathbb{Z}\rightarrow\mathbb{Z}$ **and** A2: $m\in\mathbb{Z}$ $n\in\mathbb{Z}$
shows $f(m+n) = \delta(f,m,n) + f(n) + f(m)$
 $\langle proof \rangle$

The next lemma presents two ways the set of homomorphism differences can be written.

lemma (in int1) Int_ZF_2_1_L4: **assumes** A1: $f:\mathbb{Z}\rightarrow\mathbb{Z}$
shows $\{\text{abs}(\text{HomDiff}(\mathbb{Z}, \text{IntegerAddition}, f, x)) \mid x \in \mathbb{Z} \times \mathbb{Z}\} =$
 $\{\text{abs}(\delta(f,m,n)) \mid m,n \in \mathbb{Z}\}$
 $\langle proof \rangle$

If f maps integers into integers and for all $m, n \in \mathbb{Z}$ we have $|f(m+n) - f(m) - f(n)| \leq L$ for some L , then f is a slope.

lemma (in int1) Int_ZF_2_1_L5: **assumes** A1: $f:\mathbb{Z}\rightarrow\mathbb{Z}$
and A2: $\forall m\in\mathbb{Z}. \forall n\in\mathbb{Z}. \text{abs}(\delta(f,m,n)) \leq L$
shows $f \in \mathcal{S}$
 $\langle proof \rangle$

The absolute value of homomorphism difference of a slope s does not exceed $\max\delta(s)$.

lemma (in int1) Int_ZF_2_1_L7:
assumes A1: $s \in \mathcal{S}$ **and** A2: $n \in \mathbb{Z}$ $m \in \mathbb{Z}$
shows
 $\text{abs}(\delta(s,m,n)) \leq \max\delta(s)$
 $\delta(s,m,n) \in \mathbb{Z}$ $\max\delta(s) \in \mathbb{Z}$
 $(-\max\delta(s)) \leq \delta(s,m,n)$
 $\langle proof \rangle$

A useful estimate for the value of a slope at 0, plus some type information for slopes.

lemma (in int1) Int_ZF_2_1_L8: **assumes** A1: $s \in \mathcal{S}$
shows
 $\text{abs}(s(0)) \leq \max\delta(s)$
 $0 \leq \max\delta(s)$
 $\text{abs}(s(0)) \in \mathbb{Z}$ $\max\delta(s) \in \mathbb{Z}$
 $\text{abs}(s(0)) + \max\delta(s) \in \mathbb{Z}$
 $\langle proof \rangle$

In `Group_ZF_3.thy` we show that finite range functions valued in an abelian group form a normal subgroup of almost homomorphisms. This allows to define the equivalence relation between almost homomorphisms as the relation resulting from dividing by that normal subgroup. Then we show in

Group_ZF_3_4_L12 that if the difference of f and g has finite range (actually $f(n) \cdot g(n)^{-1}$ as we use multiplicative notation in Group_ZF_3.thy), then f and g are equivalent. The next lemma translates that fact into the notation used in int1 context.

```
lemma (in int1) Int_ZF_2_1_L9: assumes A1: s ∈ S   r ∈ S
  and A2: ∀ m ∈ ℤ. abs(s(m)-r(m)) ≤ L
  shows s ~ r
⟨proof⟩
```

A necessary condition for two slopes to be almost equal. For slopes the definition postulates the set $\{f(m) - g(m) : m \in \mathbb{Z}\}$ to be finite. This lemma shows that this implies that $|f(m) - g(m)|$ is bounded (by some integer) as m varies over integers. We also mention here that in this context $s \sim r$ implies that both s and r are slopes.

```
lemma (in int1) Int_ZF_2_1_L9A: assumes s ~ r
  shows
    ∃ L ∈ ℤ. ∀ m ∈ ℤ. abs(s(m)-r(m)) ≤ L
    s ∈ S   r ∈ S
⟨proof⟩
```

Let's recall that the relation of almost equality is an equivalence relation on the set of slopes.

```
lemma (in int1) Int_ZF_2_1_L9B: shows
  A1EqRel ⊆ S × S
  equiv(S, A1EqRel)
⟨proof⟩
```

Another version of sufficient condition for two slopes to be almost equal: if the difference of two slopes is a finite range function, then they are almost equal.

```
lemma (in int1) Int_ZF_2_1_L9C: assumes s ∈ S   r ∈ S and
  s + (-r) ∈ FinRangeFunctions(ℤ, ℤ)
  shows
    s ~ r
    r ~ s
⟨proof⟩
```

If two slopes are almost equal, then the difference has finite range. This is the inverse of Int_ZF_2_1_L9C.

```
lemma (in int1) Int_ZF_2_1_L9D: assumes A1: s ~ r
  shows s + (-r) ∈ FinRangeFunctions(ℤ, ℤ)
⟨proof⟩
```

What is the value of a composition of slopes?

```
lemma (in int1) Int_ZF_2_1_L10:
  assumes s ∈ S   r ∈ S and m ∈ ℤ
```

shows $(s \circ r)(m) = s(r(m)) \quad s(r(m)) \in \mathbb{Z}$
 $\langle proof \rangle$

Composition of slopes is a slope.

lemma (in int1) Int_ZF_2_1_L11:
assumes $s \in \mathcal{S} \quad r \in \mathcal{S}$
shows $s \circ r \in \mathcal{S}$
 $\langle proof \rangle$

Negative of a slope is a slope.

lemma (in int1) Int_ZF_2_1_L12: **assumes** $s \in \mathcal{S}$ **shows** $-s \in \mathcal{S}$
 $\langle proof \rangle$

What is the value of a negative of a slope?

lemma (in int1) Int_ZF_2_1_L12A:
assumes $s \in \mathcal{S}$ **and** $m \in \mathbb{Z}$ **shows** $(-s)(m) = -(s(m))$
 $\langle proof \rangle$

What are the values of a sum of slopes?

lemma (in int1) Int_ZF_2_1_L12B: **assumes** $s \in \mathcal{S} \quad r \in \mathcal{S}$ **and** $m \in \mathbb{Z}$
shows $(s+r)(m) = s(m) + r(m)$
 $\langle proof \rangle$

Sum of slopes is a slope.

lemma (in int1) Int_ZF_2_1_L12C: **assumes** $s \in \mathcal{S} \quad r \in \mathcal{S}$
shows $s+r \in \mathcal{S}$
 $\langle proof \rangle$

A simple but useful identity.

lemma (in int1) Int_ZF_2_1_L13:
assumes $s \in \mathcal{S}$ **and** $n \in \mathbb{Z} \quad m \in \mathbb{Z}$
shows $s(n \cdot m) + (s(m) + \delta(s, n \cdot m, m)) = s((n+1) \cdot m)$
 $\langle proof \rangle$

Some estimates for the absolute value of a slope at the opposite integer.

lemma (in int1) Int_ZF_2_1_L14: **assumes** A1: $s \in \mathcal{S}$ **and** A2: $m \in \mathbb{Z}$
shows
 $s(-m) = s(0) - \delta(s, m, -m) - s(m)$
 $\text{abs}(s(m) + s(-m)) \leq 2 \cdot \max \delta(s)$
 $\text{abs}(s(-m)) \leq 2 \cdot \max \delta(s) + \text{abs}(s(m))$
 $s(-m) \leq \text{abs}(s(0)) + \max \delta(s) - s(m)$
 $\langle proof \rangle$

An identity that expresses the value of an integer function at the opposite integer in terms of the value of that function at the integer, zero, and the homomorphism difference. We have a similar identity in Int_ZF_2_1_L14, but over there we assume that f is a slope.

lemma (in int1) Int_ZF_2_1_L14A: **assumes** A1: $f:\mathbb{Z}\rightarrow\mathbb{Z}$ **and** A2: $m\in\mathbb{Z}$
shows $f(-m) = (-\delta(f,m,-m)) + f(0) - f(m)$
 $\langle proof \rangle$

The next lemma allows to use the expression $\max f(f, 0..M-1)$. Recall that $\max f(f,A)$ is the maximum of (function) f on (the set) A .

lemma (in int1) Int_ZF_2_1_L15:
assumes $s\in\mathcal{S}$ **and** $M\in\mathbb{Z}_+$
shows
 $\max f(s, 0..(M-1)) \in \mathbb{Z}$
 $\forall n \in 0..(M-1). s(n) \leq \max f(s, 0..(M-1))$
 $\min f(s, 0..(M-1)) \in \mathbb{Z}$
 $\forall n \in 0..(M-1). \min f(s, 0..(M-1)) \leq s(n)$
 $\langle proof \rangle$

A lower estimate for the value of a slope at $nM + k$.

lemma (in int1) Int_ZF_2_1_L16:
assumes A1: $s\in\mathcal{S}$ **and** A2: $m\in\mathbb{Z}$ **and** A3: $M\in\mathbb{Z}_+$ **and** A4: $k\in 0..(M-1)$
shows $s(m\cdot M) + (\min f(s, 0..(M-1)) - \max \delta(s)) \leq s(m\cdot M + k)$
 $\langle proof \rangle$

Identity is a slope.

lemma (in int1) Int_ZF_2_1_L17: **shows** $\text{id}(\mathbb{Z}) \in \mathcal{S}$
 $\langle proof \rangle$

Simple identities about (absolute value of) homomorphism differences.

lemma (in int1) Int_ZF_2_1_L18:
assumes A1: $f:\mathbb{Z}\rightarrow\mathbb{Z}$ **and** A2: $m\in\mathbb{Z} \quad n\in\mathbb{Z}$
shows
 $\text{abs}(f(n) + f(m) - f(m+n)) = \text{abs}(\delta(f,m,n))$
 $\text{abs}(f(m) + f(n) - f(m+n)) = \text{abs}(\delta(f,m,n))$
 $(-f(m)) - f(n) + f(m+n) = \delta(f,m,n)$
 $(-f(n)) - f(m) + f(m+n) = \delta(f,m,n)$
 $\text{abs}((-f(m+n)) + f(m) + f(n)) = \text{abs}(\delta(f,m,n))$
 $\langle proof \rangle$

Some identities about the homomorphism difference of odd functions.

lemma (in int1) Int_ZF_2_1_L19:
assumes A1: $f:\mathbb{Z}\rightarrow\mathbb{Z}$ **and** A2: $\forall x\in\mathbb{Z}. (-f(-x)) = f(x)$
and A3: $m\in\mathbb{Z} \quad n\in\mathbb{Z}$
shows
 $\text{abs}(\delta(f,-m,m+n)) = \text{abs}(\delta(f,m,n))$
 $\text{abs}(\delta(f,-n,m+n)) = \text{abs}(\delta(f,m,n))$
 $\delta(f,n,-(m+n)) = \delta(f,m,n)$
 $\delta(f,m,-(m+n)) = \delta(f,m,n)$
 $\text{abs}(\delta(f,-m,-n)) = \text{abs}(\delta(f,m,n))$
 $\langle proof \rangle$

Recall that f is a slope iff $f(m+n) - f(m) - f(n)$ is bounded as m, n ranges over integers. The next lemma is the first step in showing that we only need to check this condition as m, n ranges over positive integers. Namely we show that if the condition holds for positive integers, then it holds if one integer is positive and the second one is nonnegative.

lemma (in int1) Int_ZF_2_1_L20: assumes A1: $f: \mathbb{Z} \rightarrow \mathbb{Z}$ and
 A2: $\forall a \in \mathbb{Z}_+. \forall b \in \mathbb{Z}_+. \text{abs}(\delta(f, a, b)) \leq L$ and
 A3: $m \in \mathbb{Z}^+ \quad n \in \mathbb{Z}_+$
 shows
 $0 \leq L$
 $\text{abs}(\delta(f, m, n)) \leq L + \text{abs}(f(0))$
<proof>

If the slope condition holds for all pairs of integers such that one integer is positive and the second one is nonnegative, then it holds when both integers are nonnegative.

lemma (in int1) Int_ZF_2_1_L21: assumes A1: $f: \mathbb{Z} \rightarrow \mathbb{Z}$ and
 A2: $\forall a \in \mathbb{Z}^+. \forall b \in \mathbb{Z}_+. \text{abs}(\delta(f, a, b)) \leq L$ and
 A3: $n \in \mathbb{Z}^+ \quad m \in \mathbb{Z}^+$
 shows $\text{abs}(\delta(f, m, n)) \leq L + \text{abs}(f(0))$
<proof>

If the homomorphism difference is bounded on $\mathbb{Z}_+ \times \mathbb{Z}_+$, then it is bounded on $\mathbb{Z}^+ \times \mathbb{Z}^+$.

lemma (in int1) Int_ZF_2_1_L22: assumes A1: $f: \mathbb{Z} \rightarrow \mathbb{Z}$ and
 A2: $\forall a \in \mathbb{Z}_+. \forall b \in \mathbb{Z}_+. \text{abs}(\delta(f, a, b)) \leq L$
 shows $\exists M. \forall m \in \mathbb{Z}^+. \forall n \in \mathbb{Z}^+. \text{abs}(\delta(f, m, n)) \leq M$
<proof>

For odd functions we can do better than in Int_ZF_2_1_L22: if the homomorphism difference of f is bounded on $\mathbb{Z}^+ \times \mathbb{Z}^+$, then it is bounded on $\mathbb{Z} \times \mathbb{Z}$, hence f is a slope. Loong proof by splitting the $\mathbb{Z} \times \mathbb{Z}$ into six subsets.

lemma (in int1) Int_ZF_2_1_L23: assumes A1: $f: \mathbb{Z} \rightarrow \mathbb{Z}$ and
 A2: $\forall a \in \mathbb{Z}_+. \forall b \in \mathbb{Z}_+. \text{abs}(\delta(f, a, b)) \leq L$
 and A3: $\forall x \in \mathbb{Z}. (-f(-x)) = f(x)$
 shows $f \in \mathcal{S}$
<proof>

If the homomorphism difference of a function defined on positive integers is bounded, then the odd extension of this function is a slope.

lemma (in int1) Int_ZF_2_1_L24:
 assumes A1: $f: \mathbb{Z}_+ \rightarrow \mathbb{Z}$ and A2: $\forall a \in \mathbb{Z}_+. \forall b \in \mathbb{Z}_+. \text{abs}(\delta(f, a, b)) \leq L$
 shows $\text{OddExtension}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerOrder}, f) \in \mathcal{S}$
<proof>

Type information related to γ .

lemma (in int1) Int_ZF_2_1_L25:

assumes A1: $f:\mathbb{Z}\rightarrow\mathbb{Z}$ and A2: $m\in\mathbb{Z} \quad n\in\mathbb{Z}$
shows
 $\delta(f,m,-n) \in \mathbb{Z}$
 $\delta(f,n,-n) \in \mathbb{Z}$
 $(-\delta(f,n,-n)) \in \mathbb{Z}$
 $f(0) \in \mathbb{Z}$
 $\gamma(f,m,n) \in \mathbb{Z}$
<proof>

A couple of formulae involving $f(m-n)$ and $\gamma(f,m,n)$.

lemma (in int1) Int_ZF_2_1_L26:
assumes A1: $f:\mathbb{Z}\rightarrow\mathbb{Z}$ and A2: $m\in\mathbb{Z} \quad n\in\mathbb{Z}$
shows
 $f(m-n) = \gamma(f,m,n) + f(m) - f(n)$
 $f(m-n) = \gamma(f,m,n) + (f(m) - f(n))$
 $f(m-n) + (f(n) - \gamma(f,m,n)) = f(m)$
<proof>

A formula expressing the difference between $f(m-n-k)$ and $f(m)-f(n)-f(k)$ in terms of γ .

lemma (in int1) Int_ZF_2_1_L26A:
assumes A1: $f:\mathbb{Z}\rightarrow\mathbb{Z}$ and A2: $m\in\mathbb{Z} \quad n\in\mathbb{Z} \quad k\in\mathbb{Z}$
shows
 $f(m-n-k) - (f(m) - f(n) - f(k)) = \gamma(f,m-n,k) + \gamma(f,m,n)$
<proof>

If s is a slope, then $\gamma(s,m,n)$ is uniformly bounded.

lemma (in int1) Int_ZF_2_1_L27: **assumes** A1: $s\in\mathcal{S}$
shows $\exists L\in\mathbb{Z}. \forall m\in\mathbb{Z}. \forall n\in\mathbb{Z}. \text{abs}(\gamma(s,m,n)) \leq L$
<proof>

If s is a slope, then $s(m) \leq s(m-1) + M$, where L does not depend on m .

lemma (in int1) Int_ZF_2_1_L28: **assumes** A1: $s\in\mathcal{S}$
shows $\exists M\in\mathbb{Z}. \forall m\in\mathbb{Z}. s(m) \leq s(m-1) + M$
<proof>

If s is a slope, then the difference between $s(m-n-k)$ and $s(m)-s(n)-s(k)$ is uniformly bounded.

lemma (in int1) Int_ZF_2_1_L29: **assumes** A1: $s\in\mathcal{S}$
shows
 $\exists M\in\mathbb{Z}. \forall m\in\mathbb{Z}. \forall n\in\mathbb{Z}. \forall k\in\mathbb{Z}. \text{abs}(s(m-n-k) - (s(m)-s(n)-s(k))) \leq M$
<proof>

If s is a slope, then we can find integers M, K such that $s(m-n-k) \leq s(m)-s(n)-s(k) + M$ and $s(m)-s(n)-s(k) + K \leq s(m-n-k)$, for all integer m, n, k .

lemma (in int1) Int_ZF_2_1_L30: **assumes** A1: $s\in\mathcal{S}$

shows
 $\exists M \in \mathbb{Z}. \forall m \in \mathbb{Z}. \forall n \in \mathbb{Z}. \forall k \in \mathbb{Z}. s(m-n-k) \leq s(m) - s(n) - s(k) + M$
 $\exists K \in \mathbb{Z}. \forall m \in \mathbb{Z}. \forall n \in \mathbb{Z}. \forall k \in \mathbb{Z}. s(m) - s(n) - s(k) + K \leq s(m-n-k)$
<proof>

By definition functions f, g are almost equal if $f - g^*$ is bounded. In the next lemma we show it is sufficient to check the boundedness on positive integers.

lemma (in int1) Int_ZF_2_1_L31: **assumes** A1: $s \in \mathcal{S} \quad r \in \mathcal{S}$
and A2: $\forall m \in \mathbb{Z}_+. \text{abs}(s(m) - r(m)) \leq L$
shows $s \sim r$
<proof>

A sufficient condition for an odd slope to be almost equal to identity: If for all positive integers the value of the slope at m is between m and m plus some constant independent of m , then the slope is almost identity.

lemma (in int1) Int_ZF_2_1_L32: **assumes** A1: $s \in \mathcal{S} \quad M \in \mathbb{Z}$
and A2: $\forall m \in \mathbb{Z}_+. m \leq s(m) \wedge s(m) \leq m + M$
shows $s \sim \text{id}(\mathbb{Z})$
<proof>

A lemma about adding a constant to slopes. This is actually proven in Group_ZF_3_5_L1, in Group_ZF_3.thy here we just refer to that lemma to show it in notation used for integers. Unfortunately we have to use raw set notation in the proof.

lemma (in int1) Int_ZF_2_1_L33:
assumes A1: $s \in \mathcal{S}$ **and** A2: $c \in \mathbb{Z}$ **and**
A3: $r = \{ \langle m, s(m) + c \rangle. m \in \mathbb{Z} \}$
shows
 $\forall m \in \mathbb{Z}. r(m) = s(m) + c$
 $r \in \mathcal{S}$
 $s \sim r$
<proof>

65.2 Composing slopes

Composition of slopes is not commutative. However, as we show in this section if f and g are slopes then the range of $f \circ g - g \circ f$ is bounded. This allows to show that the multiplication of real numbers is commutative.

Two useful estimates.

lemma (in int1) Int_ZF_2_2_L1:
assumes A1: $f: \mathbb{Z} \rightarrow \mathbb{Z}$ **and** A2: $p \in \mathbb{Z} \quad q \in \mathbb{Z}$
shows
 $\text{abs}(f((p+1) \cdot q) - (p+1) \cdot f(q)) \leq \text{abs}(\delta(f, p \cdot q, q)) + \text{abs}(f(p \cdot q) - p \cdot f(q))$
 $\text{abs}(f((p-1) \cdot q) - (p-1) \cdot f(q)) \leq \text{abs}(\delta(f, (p-1) \cdot q, q)) + \text{abs}(f(p \cdot q) - p \cdot f(q))$
<proof>

If f is a slope, then $|f(p \cdot q) - p \cdot f(q)| \leq (|p| + 1) \cdot \max \delta(f)$. The proof is by induction on p and the next lemma is the induction step for the case when $0 \leq p$.

```
lemma (in int1) Int_ZF_2_2_L2:
  assumes A1:  $f \in \mathcal{S}$  and A2:  $0 \leq p \quad q \in \mathbb{Z}$ 
  and A3:  $\text{abs}(f(p \cdot q) - p \cdot f(q)) \leq (\text{abs}(p) + 1) \cdot \max \delta(f)$ 
  shows
     $\text{abs}(f((p+1) \cdot q) - (p+1) \cdot f(q)) \leq (\text{abs}(p+1) + 1) \cdot \max \delta(f)$ 
  <proof>
```

If f is a slope, then $|f(p \cdot q) - p \cdot f(q)| \leq (|p| + 1) \cdot \max \delta$. The proof is by induction on p and the next lemma is the induction step for the case when $p \leq 0$.

```
lemma (in int1) Int_ZF_2_2_L3:
  assumes A1:  $f \in \mathcal{S}$  and A2:  $p \leq 0 \quad q \in \mathbb{Z}$ 
  and A3:  $\text{abs}(f(p \cdot q) - p \cdot f(q)) \leq (\text{abs}(p) + 1) \cdot \max \delta(f)$ 
  shows  $\text{abs}(f((p-1) \cdot q) - (p-1) \cdot f(q)) \leq (\text{abs}(p-1) + 1) \cdot \max \delta(f)$ 
  <proof>
```

If f is a slope, then $|f(p \cdot q) - p \cdot f(q)| \leq (|p| + 1) \cdot \max \delta(f)$. Proof by cases on $0 \leq p$.

```
lemma (in int1) Int_ZF_2_2_L4:
  assumes A1:  $f \in \mathcal{S}$  and A2:  $p \in \mathbb{Z} \quad q \in \mathbb{Z}$ 
  shows  $\text{abs}(f(p \cdot q) - p \cdot f(q)) \leq (\text{abs}(p) + 1) \cdot \max \delta(f)$ 
  <proof>
```

The next elegant result is Lemma 7 in the Arthan's paper [2].

```
lemma (in int1) Arthan_Lem_7:
  assumes A1:  $f \in \mathcal{S}$  and A2:  $p \in \mathbb{Z} \quad q \in \mathbb{Z}$ 
  shows  $\text{abs}(q \cdot f(p) - p \cdot f(q)) \leq (\text{abs}(p) + \text{abs}(q) + 2) \cdot \max \delta(f)$ 
  <proof>
```

This is Lemma 8 in the Arthan's paper.

```
lemma (in int1) Arthan_Lem_8: assumes A1:  $f \in \mathcal{S}$ 
  shows  $\exists A \ B. A \in \mathbb{Z} \wedge B \in \mathbb{Z} \wedge (\forall p \in \mathbb{Z}. \text{abs}(f(p)) \leq A \cdot \text{abs}(p) + B)$ 
  <proof>
```

If f and g are slopes, then $f \circ g$ is equivalent (almost equal) to $g \circ f$. This is Theorem 9 in Arthan's paper [2].

```
theorem (in int1) Arthan_Th_9: assumes A1:  $f \in \mathcal{S} \quad g \in \mathcal{S}$ 
  shows  $f \circ g \sim g \circ f$ 
  <proof>
```

end

66 Integers 3

theory Int_ZF_3 **imports** Int_ZF_2

begin

This theory is a continuation of Int_ZF_2. We consider here the properties of slopes (almost homomorphisms on integers) that allow to define the order relation and multiplicative inverse on real numbers. We also prove theorems that allow to show completeness of the order relation of real numbers we define in Real_ZF.

66.1 Positive slopes

This section provides background material for defining the order relation on real numbers.

Positive slopes are functions (of course.)

lemma (in int1) Int_ZF_2_3_L1: **assumes** A1: $f \in \mathcal{S}_+$ **shows** $f: \mathbb{Z} \rightarrow \mathbb{Z}$
<proof>

A small technical lemma to simplify the proof of the next theorem.

lemma (in int1) Int_ZF_2_3_L1A:
assumes A1: $f \in \mathcal{S}_+$ **and** A2: $\exists n \in f(\mathbb{Z}_+) \cap \mathbb{Z}_+. a \leq n$
shows $\exists M \in \mathbb{Z}_+. a \leq f(M)$
<proof>

The next lemma is Lemma 3 in the Arthan's paper.

lemma (in int1) Arthan_Lem_3:
assumes A1: $f \in \mathcal{S}_+$ **and** A2: $D \in \mathbb{Z}_+$
shows $\exists M \in \mathbb{Z}_+. \forall m \in \mathbb{Z}_+. (m+1) \cdot D \leq f(m \cdot M)$
<proof>

A special case of Arthan_Lem_3 when $D = 1$.

corollary (in int1) Arthan_L_3_spec: **assumes** A1: $f \in \mathcal{S}_+$
shows $\exists M \in \mathbb{Z}_+. \forall n \in \mathbb{Z}_+. n+1 \leq f(n \cdot M)$
<proof>

We know from Group_ZF_3.thy that finite range functions are almost homomorphisms. Besides reminding that fact for slopes the next lemma shows that finite range functions do not belong to $\mathcal{S}_+$. This is important, because the projection of the set of finite range functions defines zero in the real number construction in Real_ZF_x.thy series, while the projection of $\mathcal{S}_+$ becomes the set of (strictly) positive reals. We don't want zero to be positive, do we? The next lemma is a part of Lemma 5 in the Arthan's paper [2].

lemma (in int1) Int_ZF_2_3_L1B:

assumes A1: $f \in \text{FinRangeFunctions}(\mathbb{Z}, \mathbb{Z})$
shows $f \in \mathcal{S} \quad f \notin \mathcal{S}_+$
<proof>

We want to show that if f is a slope and neither f nor $-f$ are in $\mathcal{S}_+$, then f is bounded. The next lemma is the first step towards that goal and shows that if slope is not in $\mathcal{S}_+$ then $f(\mathbb{Z}_+)$ is bounded above.

lemma (in int1) Int_ZF_2_3_L2: **assumes** A1: $f \in \mathcal{S}$ **and** A2: $f \notin \mathcal{S}_+$
shows $\text{IsBoundedAbove}(f(\mathbb{Z}_+), \text{IntegerOrder})$
<proof>

If f is a slope and $-f \notin \mathcal{S}_+$, then $f(\mathbb{Z}_+)$ is bounded below.

lemma (in int1) Int_ZF_2_3_L3: **assumes** A1: $f \in \mathcal{S}$ **and** A2: $-f \notin \mathcal{S}_+$
shows $\text{IsBoundedBelow}(f(\mathbb{Z}_+), \text{IntegerOrder})$
<proof>

A slope that is bounded on $\mathbb{Z}_+$ is bounded everywhere.

lemma (in int1) Int_ZF_2_3_L4:
assumes A1: $f \in \mathcal{S}$ **and** A2: $m \in \mathbb{Z}$
and A3: $\forall n \in \mathbb{Z}_+. \text{abs}(f(n)) \leq L$
shows $\text{abs}(f(m)) \leq 2 \cdot \max \delta(f) + L$
<proof>

A slope whose image of the set of positive integers is bounded is a finite range function.

lemma (in int1) Int_ZF_2_3_L4A:
assumes A1: $f \in \mathcal{S}$ **and** A2: $\text{IsBounded}(f(\mathbb{Z}_+), \text{IntegerOrder})$
shows $f \in \text{FinRangeFunctions}(\mathbb{Z}, \mathbb{Z})$
<proof>

A slope whose image of the set of positive integers is bounded below is a finite range function or a positive slope.

lemma (in int1) Int_ZF_2_3_L4B:
assumes $f \in \mathcal{S}$ **and** $\text{IsBoundedBelow}(f(\mathbb{Z}_+), \text{IntegerOrder})$
shows $f \in \text{FinRangeFunctions}(\mathbb{Z}, \mathbb{Z}) \vee f \in \mathcal{S}_+$
<proof>

If one slope is not greater than another on positive integers, then they are almost equal or the difference is a positive slope.

lemma (in int1) Int_ZF_2_3_L4C: **assumes** A1: $f \in \mathcal{S} \quad g \in \mathcal{S}$ **and**
A2: $\forall n \in \mathbb{Z}_+. f(n) \leq g(n)$
shows $f \sim g \vee g + (-f) \in \mathcal{S}_+$
<proof>

Positive slopes are arbitrarily large for large enough arguments.

lemma (in int1) Int_ZF_2_3_L5:
assumes A1: $f \in \mathcal{S}_+$ **and** A2: $K \in \mathbb{Z}$

shows $\exists N \in \mathbb{Z}_+. \forall m. N \leq m \longrightarrow K \leq f(m)$
<proof>

Positive slopes are arbitrarily small for small enough arguments. Kind of dual to Int_ZF_2_3_L5.

lemma (in int1) Int_ZF_2_3_L5A: **assumes** A1: $f \in \mathcal{S}_+$ **and** A2: $K \in \mathbb{Z}$
shows $\exists N \in \mathbb{Z}_+. \forall m. N \leq m \longrightarrow f(-m) \leq K$
<proof>

A special case of Int_ZF_2_3_L5 where $K = 1$.

corollary (in int1) Int_ZF_2_3_L6: **assumes** $f \in \mathcal{S}_+$
shows $\exists N \in \mathbb{Z}_+. \forall m. N \leq m \longrightarrow f(m) \in \mathbb{Z}_+$
<proof>

A special case of Int_ZF_2_3_L5 where $m = N$.

corollary (in int1) Int_ZF_2_3_L6A: **assumes** $f \in \mathcal{S}_+$ **and** $K \in \mathbb{Z}$
shows $\exists N \in \mathbb{Z}_+. K \leq f(N)$
<proof>

If values of a slope are not bounded above, then the slope is positive.

lemma (in int1) Int_ZF_2_3_L7: **assumes** A1: $f \in \mathcal{S}$
and A2: $\forall K \in \mathbb{Z}. \exists n \in \mathbb{Z}_+. K \leq f(n)$
shows $f \in \mathcal{S}_+$
<proof>

For unbounded slope f either $f \in \mathcal{S}_+$ or $-f \in \mathcal{S}_+$.

theorem (in int1) Int_ZF_2_3_L8:
assumes A1: $f \in \mathcal{S}$ **and** A2: $f \notin \text{FinRangeFunctions}(\mathbb{Z}, \mathbb{Z})$
shows $(f \in \mathcal{S}_+) \text{ Xor } ((-f) \in \mathcal{S}_+)$
<proof>

The sum of positive slopes is a positive slope.

theorem (in int1) sum_of_pos_sls_is_pos_sl:
assumes A1: $f \in \mathcal{S}_+$ $g \in \mathcal{S}_+$
shows $f+g \in \mathcal{S}_+$
<proof>

The composition of positive slopes is a positive slope.

theorem (in int1) comp_of_pos_sls_is_pos_sl:
assumes A1: $f \in \mathcal{S}_+$ $g \in \mathcal{S}_+$
shows $f \circ g \in \mathcal{S}_+$
<proof>

A slope equivalent to a positive one is positive.

lemma (in int1) Int_ZF_2_3_L9:
assumes A1: $f \in \mathcal{S}_+$ **and** A2: $\langle f, g \rangle \in \text{A1EqRel}$ **shows** $g \in \mathcal{S}_+$
<proof>

The set of positive slopes is saturated with respect to the relation of equivalence of slopes.

lemma (in int1) pos_slopes_saturated: shows IsSaturated(AlEqRel, S_+)
<proof>

A technical lemma involving a projection of the set of positive slopes and a logical expression with exclusive or.

lemma (in int1) Int_ZF_2_3_L10:
 assumes A1: $f \in S$ $g \in S$
 and A2: $R = \{ \text{AlEqRel}\{s\}. s \in S_+ \}$
 and A3: $(f \in S_+) \text{ Xor } (g \in S_+)$
 shows $(\text{AlEqRel}\{f\} \in R) \text{ Xor } (\text{AlEqRel}\{g\} \in R)$
<proof>

Identity function is a positive slope.

lemma (in int1) Int_ZF_2_3_L11: shows $\text{id}(\mathbb{Z}) \in S_+$
<proof>

The identity function is not almost equal to any bounded function.

lemma (in int1) Int_ZF_2_3_L12: assumes A1: $f \in \text{FinRangeFunctions}(\mathbb{Z}, \mathbb{Z})$
 shows $\neg(\text{id}(\mathbb{Z}) \sim f)$
<proof>

66.2 Inverting slopes

Not every slope is a 1:1 function. However, we can still invert slopes in the sense that if f is a slope, then we can find a slope g such that $f \circ g$ is almost equal to the identity function. The goal of this section is to establish this fact for positive slopes.

If f is a positive slope, then for every positive integer p the set $\{n \in \mathbb{Z}_+ : p \leq f(n)\}$ is a nonempty subset of positive integers. Recall that $f^{-1}(p)$ is the notation for the smallest element of this set.

lemma (in int1) Int_ZF_2_4_L1:
 assumes A1: $f \in S_+$ and A2: $p \in \mathbb{Z}_+$ and A3: $A = \{n \in \mathbb{Z}_+. p \leq f(n)\}$
 shows
 $A \subseteq \mathbb{Z}_+$
 $A \neq 0$
 $f^{-1}(p) \in A$
 $\forall m \in A. f^{-1}(p) \leq m$
<proof>

If f is a positive slope and p is a positive integer p , then $f^{-1}(p)$ (defined as the minimum of the set $\{n \in \mathbb{Z}_+ : p \leq f(n)\}$) is a (well defined) positive integer.

lemma (in int1) Int_ZF_2_4_L2:

assumes $f \in \mathcal{S}_+$ **and** $p \in \mathbb{Z}_+$
shows
 $f^{-1}(p) \in \mathbb{Z}_+$
 $p \leq f(f^{-1}(p))$
 $\langle proof \rangle$

If f is a positive slope and p is a positive integer such that $n \leq f(p)$, then $f^{-1}(n) \leq p$.

lemma (in int1) Int_ZF_2_4_L3:
assumes $f \in \mathcal{S}_+$ **and** $m \in \mathbb{Z}_+$ $p \in \mathbb{Z}_+$ **and** $m \leq f(p)$
shows $f^{-1}(m) \leq p$
 $\langle proof \rangle$

An upper bound $f(f^{-1}(m) - 1)$ for positive slopes.

lemma (in int1) Int_ZF_2_4_L4:
assumes A1: $f \in \mathcal{S}_+$ **and** A2: $m \in \mathbb{Z}_+$ **and** A3: $f^{-1}(m) - 1 \in \mathbb{Z}_+$
shows $f(f^{-1}(m) - 1) \leq m$ $f(f^{-1}(m) - 1) \neq m$
 $\langle proof \rangle$

The (candidate for) the inverse of a positive slope is nondecreasing.

lemma (in int1) Int_ZF_2_4_L5:
assumes A1: $f \in \mathcal{S}_+$ **and** A2: $m \in \mathbb{Z}_+$ **and** A3: $m \leq n$
shows $f^{-1}(m) \leq f^{-1}(n)$
 $\langle proof \rangle$

If $f^{-1}(m)$ is positive and n is a positive integer, then, then $f^{-1}(m + n) - 1$ is positive.

lemma (in int1) Int_ZF_2_4_L6:
assumes A1: $f \in \mathcal{S}_+$ **and** A2: $m \in \mathbb{Z}_+$ $n \in \mathbb{Z}_+$ **and**
A3: $f^{-1}(m) - 1 \in \mathbb{Z}_+$
shows $f^{-1}(m+n) - 1 \in \mathbb{Z}_+$
 $\langle proof \rangle$

If f is a slope, then $f(f^{-1}(m + n) - f^{-1}(m) - f^{-1}(n))$ is uniformly bounded above and below. Will it be the messiest IsarMathLib proof ever? Only time will tell.

lemma (in int1) Int_ZF_2_4_L7: **assumes** A1: $f \in \mathcal{S}_+$ **and**
A2: $\forall m \in \mathbb{Z}_+. f^{-1}(m) - 1 \in \mathbb{Z}_+$
shows
 $\exists U \in \mathbb{Z}. \forall m \in \mathbb{Z}_+. \forall n \in \mathbb{Z}_+. f(f^{-1}(m+n) - f^{-1}(m) - f^{-1}(n)) \leq U$
 $\exists N \in \mathbb{Z}. \forall m \in \mathbb{Z}_+. \forall n \in \mathbb{Z}_+. N \leq f(f^{-1}(m+n) - f^{-1}(m) - f^{-1}(n))$
 $\langle proof \rangle$

The expression $f^{-1}(m + n) - f^{-1}(m) - f^{-1}(n)$ is uniformly bounded for all pairs $\langle m, n \rangle \in \mathbb{Z}_+ \times \mathbb{Z}_+$. Recall that in the int1 context $\varepsilon(f, x)$ is defined so that $\varepsilon(f, \langle m, n \rangle) = f^{-1}(m + n) - f^{-1}(m) - f^{-1}(n)$.

lemma (in int1) Int_ZF_2_4_L8: **assumes** A1: $f \in \mathcal{S}_+$ **and**

A2: $\forall m \in \mathbb{Z}_+. f^{-1}(m) - 1 \in \mathbb{Z}_+$
shows $\exists M. \forall x \in \mathbb{Z}_+ \times \mathbb{Z}_+. \text{abs}(\varepsilon(f, x)) \leq M$
 $\langle \text{proof} \rangle$

The (candidate for) inverse of a positive slope is a (well defined) function on $\mathbb{Z}_+$.

lemma (in int1) Int_ZF_2_4_L9:
assumes A1: $f \in \mathcal{S}_+$ **and** A2: $g = \{\langle p, f^{-1}(p) \rangle. p \in \mathbb{Z}_+\}$
shows
 $g : \mathbb{Z}_+ \rightarrow \mathbb{Z}_+$
 $g : \mathbb{Z}_+ \rightarrow \mathbb{Z}$
 $\langle \text{proof} \rangle$

What are the values of the (candidate for) the inverse of a positive slope?

lemma (in int1) Int_ZF_2_4_L10:
assumes A1: $f \in \mathcal{S}_+$ **and** A2: $g = \{\langle p, f^{-1}(p) \rangle. p \in \mathbb{Z}_+\}$ **and** A3: $p \in \mathbb{Z}_+$
shows $g(p) = f^{-1}(p)$
 $\langle \text{proof} \rangle$

The (candidate for) the inverse of a positive slope is a slope.

lemma (in int1) Int_ZF_2_4_L11: **assumes** A1: $f \in \mathcal{S}_+$ **and**
A2: $\forall m \in \mathbb{Z}_+. f^{-1}(m) - 1 \in \mathbb{Z}_+$ **and**
A3: $g = \{\langle p, f^{-1}(p) \rangle. p \in \mathbb{Z}_+\}$
shows $\text{OddExtension}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerOrder}, g) \in \mathcal{S}$
 $\langle \text{proof} \rangle$

Every positive slope that is at least 2 on positive integers almost has an inverse.

lemma (in int1) Int_ZF_2_4_L12: **assumes** A1: $f \in \mathcal{S}_+$ **and**
A2: $\forall m \in \mathbb{Z}_+. f^{-1}(m) - 1 \in \mathbb{Z}_+$
shows $\exists h \in \mathcal{S}. f \circ h \sim \text{id}(\mathbb{Z})$
 $\langle \text{proof} \rangle$

Int_ZF_2_4_L12 is almost what we need, except that it has an assumption that the values of the slope that we get the inverse for are not smaller than 2 on positive integers. The Arthan's proof of Theorem 11 has a mistake where he says "note that for all but finitely many $m, n \in N$ $p = g(m)$ and $q = g(n)$ are both positive". Of course there may be infinitely many pairs $\langle m, n \rangle$ such that p, q are not both positive. This is however easy to workaround: we just modify the slope by adding a constant so that the slope is large enough on positive integers and then look for the inverse.

theorem (in int1) pos_slope_has_inv: **assumes** A1: $f \in \mathcal{S}_+$
shows $\exists g \in \mathcal{S}. f \sim g \wedge (\exists h \in \mathcal{S}. g \circ h \sim \text{id}(\mathbb{Z}))$
 $\langle \text{proof} \rangle$

66.3 Completeness

In this section we consider properties of slopes that are needed for the proof of completeness of real numbers constructed in `Real_ZF_1.thy`. In particular we consider properties of embedding of integers into the set of slopes by the mapping $m \mapsto m^S$, where m^S is defined by $m^S(n) = m \cdot n$.

If m is an integer, then m^S is a slope whose value is $m \cdot n$ for every integer.

lemma (in `int1`) `Int_ZF_2_5_L1: assumes A1:  $m \in \mathbb{Z}$`
shows
 $\forall n \in \mathbb{Z}. (m^S)(n) = m \cdot n$
 $m^S \in \mathcal{S}$
 $\langle proof \rangle$

For any slope f there is an integer m such that there is some slope g that is almost equal to m^S and dominates f in the sense that $f \leq g$ on positive integers (which implies that either g is almost equal to f or $g - f$ is a positive slope. This will be used in `Real_ZF_1.thy` to show that for any real number there is an integer that (whose real embedding) is greater or equal.

lemma (in `int1`) `Int_ZF_2_5_L2: assumes A1:  $f \in \mathcal{S}$`
shows $\exists m \in \mathbb{Z}. \exists g \in \mathcal{S}. (m^S \sim g \wedge (f \sim g \vee g + (-f) \in \mathcal{S}_+))$
 $\langle proof \rangle$

The negative of an integer embeds in slopes as a negative of the original embedding.

lemma (in `int1`) `Int_ZF_2_5_L3: assumes A1:  $m \in \mathbb{Z}$`
shows $(-m)^S = -(m^S)$
 $\langle proof \rangle$

The sum of embeddings is the embedding of the sum.

lemma (in `int1`) `Int_ZF_2_5_L3A: assumes A1:  $m \in \mathbb{Z} \ k \in \mathbb{Z}$`
shows $(m^S) + (k^S) = ((m+k)^S)$
 $\langle proof \rangle$

The composition of embeddings is the embedding of the product.

lemma (in `int1`) `Int_ZF_2_5_L3B: assumes A1:  $m \in \mathbb{Z} \ k \in \mathbb{Z}$`
shows $(m^S) \circ (k^S) = ((m \cdot k)^S)$
 $\langle proof \rangle$

Embedding integers in slopes preserves order.

lemma (in `int1`) `Int_ZF_2_5_L4: assumes A1:  $m \leq n$`
shows $(m^S) \sim (n^S) \vee (n^S) + (-m^S) \in \mathcal{S}_+$
 $\langle proof \rangle$

We aim at showing that $m \mapsto m^S$ is an injection modulo the relation of almost equality. To do that we first show that if m^S has finite range, then $m = 0$.


```

lemma (in int1) Int_ZF_2_5_L5:
  assumes m $\in\mathbb{Z}$  and mS  $\in$  FinRangeFunctions( $\mathbb{Z}$ , $\mathbb{Z}$ )
  shows m=0
  <proof>

```

Embeddings of two integers are almost equal only if the integers are equal.

```

lemma (in int1) Int_ZF_2_5_L6:
  assumes A1: m $\in\mathbb{Z}$  k $\in\mathbb{Z}$  and A2: (mS)  $\sim$  (kS)
  shows m=k
  <proof>

```

Embedding of 1 is the identity slope and embedding of zero is a finite range function.

```

lemma (in int1) Int_ZF_2_5_L7: shows
  1S = id( $\mathbb{Z}$ )
  0S  $\in$  FinRangeFunctions( $\mathbb{Z}$ , $\mathbb{Z}$ )
  <proof>

```

A somewhat technical condition for a embedding of an integer to be "less or equal" (in the sense appropriate for slopes) than the composition of a slope and another integer (embedding).

```

lemma (in int1) Int_ZF_2_5_L8:
  assumes A1: f  $\in S$  and A2: N  $\in \mathbb{Z}$  M  $\in \mathbb{Z}$  and
  A3:  $\forall n \in \mathbb{Z}_+. M \cdot n \leq f(N \cdot n)$ 
  shows MS  $\sim$  f $\circ$ (NS)  $\vee$  (f $\circ$ (NS)) + (- (MS))  $\in S_+$ 
  <proof>

```

Another technical condition for the composition of a slope and an integer (embedding) to be "less or equal" (in the sense appropriate for slopes) than embedding of another integer.

```

lemma (in int1) Int_ZF_2_5_L9:
  assumes A1: f  $\in S$  and A2: N  $\in \mathbb{Z}$  M  $\in \mathbb{Z}$  and
  A3:  $\forall n \in \mathbb{Z}_+. f(N \cdot n) \leq M \cdot n$ 
  shows f $\circ$ (NS)  $\sim$  (MS)  $\vee$  (MS) + (- (f $\circ$ (NS)))  $\in S_+$ 
  <proof>

```

end

67 Integer powers of group elements

```

theory IntGroup_ZF imports Group_ZF_2 Int_ZF_1 Cardinal_ZF

```

```

begin

```

In the Monoid_ZF_1 theory we consider multiplicities of $n \cdot x$ of monoid elements, i.e. special cases of expressions of the form $x_1 \oplus x_2 \oplus \cdots \oplus x_n$ where

$x_i = x$ for $i = 1..n$. In the group context where we usually use multiplicative notation this translates to the "power" x^n where $n \in \mathbb{N}$, see also section "Product of a list of group elements" in the `Group_ZF` theory. In the group setting the notion of raising an element to natural power can be naturally generalized to the notion of raising an element to an integer power.

67.1 Properties of natural powers of an element and its inverse

The integer power is defined in terms of natural powers of an element and its inverse. In this section we study properties of expressions $(x^n) \cdot (x^{-1})^k$, where x is a group element and n, k are natural numbers.

The natural power of an element's inverse is the inverse of the power of the element.

lemma (in group0) nat_pow_inverse: assumes $n \in \text{nat}$ $x \in G$
 shows $\text{pow}(n, x^{-1}) = (\text{pow}(n, x))^{-1}$
<proof>

The natural power of x multiplied by the same power of x^{-1} cancel out to give the neutral element of the group.

corollary (in group0) nat_pow_inv_cancel: assumes $n \in \text{nat}$ $x \in G$
 shows $\text{pow}(n, x) \cdot \text{pow}(n, x^{-1}) = 1$ $\text{pow}(n, x^{-1}) \cdot \text{pow}(n, x) = 1$
<proof>

If $k \leq n$ are natural numbers and x an element of the group, then $x^n \cdot (x^{-1})^k = x^{n-k}$.

lemma (in group0) nat_pow_cancel_less: assumes $n \in \text{nat}$ $k \leq n$ $x \in G$
 shows $\text{pow}(n, x) \cdot \text{pow}(k, x^{-1}) = \text{pow}(n \#- k, x)$
<proof>

If $k \leq n$ are natural numbers and x an element of the group, then $(x^{-1})^n \cdot x^k = (x^{-1})^{(k-n)}$.

lemma (in group0) nat_pow_cancel_less1: assumes $n \in \text{nat}$ $k \leq n$ $x \in G$
 shows $\text{pow}(n, x^{-1}) \cdot \text{pow}(k, x) = \text{pow}(n \#- k, x^{-1})$
<proof>

If $k \leq n$ are natural numbers and x an element of the group, then $x^k \cdot (x^{-1})^n = (x^{-1})^{(k-n)}$.

lemma (in group0) nat_pow_cancel_more: assumes $n \in \text{nat}$ $k \leq n$ $x \in G$
 shows $\text{pow}(k, x^{-1}) \cdot \text{pow}(n, x) = \text{pow}(n \#- k, x)$
<proof>

If $k \leq n$ are natural numbers and x an element of the group, then $x^k \cdot (x^{-1})^n = x^{n-k}$.

lemma (in group0) nat_pow_cancel_more1: assumes $n \in \text{nat}$ $k \leq n$ $x \in G$

```

    shows pow(k,x).pow(n,x-1) = pow(n #- k,x-1)
  <proof>

```

The power to a product is the power of the power.

```

lemma (in group0) nat_pow_mult:
  assumes z1 ∈ nat z2 ∈ nat g ∈ G
  shows pow(z1 ** z2,g) = pow(z2,pow(z1,g))
  <proof>

```

An application of pigeonhole principle to finite groups: if a group is finite then every element has a non-zero power that is equal to the neutral element.

```

lemma (in group0) finite_fin_order: assumes Finite(G) x ∈ G
  shows ∃ n ∈ nat \ {0}. pow(n,x) = 1
  <proof>

```

67.2 Integer powers

In this section we introduce notation basic properties of integer power in group context. The goal is to show that the power homomorphism property: if x is an element of the group and n, m are integers then $x^{n+m} = x^n \cdot x^m$.

We extend the `group0` context with some notation from `int0` context. Since we inherit the multiplicative notation from the `group0` context the integer "one" is denoted 1_Z rather than just 1 (which is the group's neutral element).

```

locale group_int0 = group0 +

```

```

  fixes ints (Z)
  defines ints_def [simp]: Z ≡ int

  fixes ia (infixl + 69)
  defines ia_def [simp]: a+b ≡ IntegerAddition⟨ a,b⟩

  fixes iminus (- _ 72)
  defines rminus_def [simp]: -a ≡ GroupInv(Z,IntegerAddition)(a)

  fixes isub (infixl - 69)
  defines isub_def [simp]: a-b ≡ a+ (- b)

  fixes imul (infixl ·Z 69)
  defines imul_def [simp]: a·Zb ≡ IntegerMultiplication⟨a,b⟩

  fixes setneg (- _ 72)
  defines setneg_def [simp]: -A ≡ GroupInv(Z,IntegerAddition)(A)

  fixes izero (0)
  defines izero_def [simp]: 0 ≡ TheNeutralElement(Z,IntegerAddition)

  fixes ione (1Z)

```

```

defines ione_def [simp]:  $1_Z \equiv \text{TheNeutralElement}(\mathbb{Z}, \text{IntegerMultiplication})$ 

fixes itwo (2)
defines itwo_def [simp]:  $2 \equiv 1_Z + 1_Z$ 

fixes ithree (3)
defines ithree_def [simp]:  $3 \equiv 2 + 1_Z$ 

fixes nonnegative ( $\mathbb{Z}^+$ )
defines nonnegative_def [simp]:
 $\mathbb{Z}^+ \equiv \text{Nonnegative}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerOrder})$ 

fixes positive ( $\mathbb{Z}_+$ )
defines positive_def [simp]:
 $\mathbb{Z}_+ \equiv \text{PositiveSet}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerOrder})$ 

fixes abs
defines abs_def [simp]:
 $\text{abs}(m) \equiv \text{AbsoluteValue}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerOrder})(m)$ 

fixes lesseq (infix  $\leq$  60)
defines lesseq_def [simp]:  $m \leq n \equiv \langle m, n \rangle \in \text{IntegerOrder}$ 

fixes sless (infix  $<$  68)
defines sless_def [simp]:  $a < b \equiv a \leq b \wedge a \neq b$ 

fixes interval (infix  $\dots$  70)
defines interval_def [simp]:  $m..n \equiv \text{Interval}(\text{IntegerOrder}, m, n)$ 

fixes maxf
defines maxf_def [simp]:  $\text{maxf}(f, A) \equiv \text{Maximum}(\text{IntegerOrder}, f(A))$ 

fixes minf
defines minf_def [simp]:  $\text{minf}(f, A) \equiv \text{Minimum}(\text{IntegerOrder}, f(A))$ 

fixes oddext ( $_^\circ$ )
defines oddext_def [simp]:  $f^\circ \equiv \text{OddExtension}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerOrder}, f)$ 

fixes listsum ( $\sum$  _ 70)
defines listsum_def [simp]:  $\sum s \equiv \text{Fold}(\text{IntegerAddition}, 0, s)$ 

```

Next define notation for the integer power $\text{powz}(z, x)$. The difficulty here is that in ZF set theory nonnegative integers and natural numbers are different things. So, we use the notion of **zmagnitude** defined in the standard Isabelle/ZF Int theory. For an integer number z , **zmagnitude**(z) is like absolute value of z but interpreted as a natural number. Hence, we define the integer power $\text{powz}(z, x)$ as x raised to the magnitude of z if z is nonnegative or x^{-1} raised to the same natural power otherwise.

definition (in group_int0) powz where
 powz(z,x) $\equiv$ pow(zmagnitudo(z),if 0 $\leq$ z then x else x⁻¹)

We bring in all the results about integers in int0 with the notation included in group_int0.

sublocale group_int0 < ints:int0 $\mathbb{Z}$ ia iminus isub imul setneg izero ione
 itwo
 ithree nonnegative positive abs lesseq sless interval maxf minf oddext
 listsum
<proof>

An integer power of a group element is in the group.

lemma (in group_int0) powz_type: assumes z $\in\mathbb{Z}$ x $\in$ G shows powz(z,x) $\in$ G
<proof>

A group element raised to (integer) zero'th power is equal to the group's neutral element. An element raised to (integer) power one is the same element.

lemma (in group_int0) int_power_zero_one: assumes x $\in$ G
 shows powz(0,x) = 1 and powz(1_Z,x) = x
<proof>

The neutral raised to any (integer) power is equal to the group's neutral element.

lemma (in group_int0) int_power_neutral: assumes z $\in\mathbb{Z}$
 shows powz(z,1) = 1
<proof>

If x is an element of the group and z_1, z_2 are nonnegative integers then $x^{z_1+z_2} = x^{z_1} \cdot x^{z_2}$, i.e. the power homomorphism property holds.

lemma (in group_int0) powz_hom_nneg_nneg: assumes 0 $\leq$ z₁ 0 $\leq$ z₂ x $\in$ G
 shows powz(z₁+z₂,x) = powz(z₁,x)·powz(z₂,x)
<proof>

If x is an element of the group and z_1, z_2 are negative integers then the power homomorphism property holds.

lemma (in group_int0) powz_hom_neg_neg:
 assumes z₁<0 z₂<0 x $\in$ G
 shows powz(z₁+z₂,x) = powz(z₁,x)·powz(z₂,x)
<proof>

When the integers are of different signs we further split into cases depending on which magnitude is greater. If x is an element of the group and z_1 is nonnegative, z_2 is negative and $|z_2| \leq |z_1|$ then the power homomorphism property holds. The proof of this lemma is presented with more detail than

necessary, to show the schema of the proofs of the remaining lemmas that we let Isabelle prove automatically.

lemma (in group_int0) powz_hom_nneg_neg1:
 assumes $0 \leq z_1 \quad z_2 < 0 \quad \text{zmagnitude}(z_2) \leq \text{zmagnitude}(z_1) \quad x \in G$
 shows $\text{powz}(z_1 + z_2, x) = \text{powz}(z_1, x) \cdot \text{powz}(z_2, x)$
<proof>

If x is an element of the group and z_1 is nonnegative, z_2 is negative and $|z_1| < |z_2|$ then the power homomorphism property holds.

lemma (in group_int0) powz_hom_nneg_neg2:
 assumes $0 \leq z_1 \quad z_2 < 0 \quad \text{zmagnitude}(z_1) < \text{zmagnitude}(z_2) \quad x \in G$
 shows $\text{powz}(z_1 + z_2, x) = \text{powz}(z_1, x) \cdot \text{powz}(z_2, x)$
<proof>

If x is an element of the group and z_1 is negative, z_2 is nonnegative and $|z_1| \leq |z_2|$ then the power homomorphism property holds.

lemma (in group_int0) powz_hom_neg_nneg1:
 assumes $z_1 < 0 \quad 0 \leq z_2 \quad \text{zmagnitude}(z_1) \leq \text{zmagnitude}(z_2) \quad x \in G$
 shows $\text{powz}(z_1 + z_2, x) = \text{powz}(z_1, x) \cdot \text{powz}(z_2, x)$
<proof>

If x is an element of the group and z_1 is negative, z_2 is nonnegative and $|z_2| < |z_1|$ then the power homomorphism property holds.

lemma (in group_int0) powz_hom_neg_nneg2:
 assumes $z_1 < 0 \quad 0 \leq z_2 \quad \text{zmagnitude}(z_2) < \text{zmagnitude}(z_1) \quad x \in G$
 shows $\text{powz}(z_1 + z_2, x) = \text{powz}(z_1, x) \cdot \text{powz}(z_2, x)$
<proof>

The next theorem collects the results from the above lemmas to show the power homomorphism property holds for any pair of integer numbers and any group element.

theorem (in group_int0) powz_hom_prop: assumes $z_1 \in \mathbb{Z} \quad z_2 \in \mathbb{Z} \quad x \in G$
 shows $\text{powz}(z_1 + z_2, x) = \text{powz}(z_1, x) \cdot \text{powz}(z_2, x)$
<proof>

If x is an element of the group and z_1, z_2 are integers then $x^{z_1 z_2} = (x^{z_2})^{z_1}$.

lemma (in group_int0) powz_mult: assumes $z_1 \in \mathbb{Z} \quad z_2 \in \mathbb{Z} \quad x \in G$
 shows $\text{powz}(z_1 \cdot z_2, x) = \text{powz}(z_2, \text{powz}(z_1, x))$
<proof>

A group element raised to power -1 is the inverse of that group element.

lemma (in group_int0) inpt_power_neg_one: assumes $x \in G$
 shows $\text{powz}(-1_Z, x) = x^{-1}$
<proof>

Increasing the (integer) power by one is the same as multiplying by the group element.

```

lemma (in group_int0) int_power_add_one: assumes  $z \in \mathbb{Z}$   $x \in G$ 
  shows  $\text{powz}(z+1, x) = \text{powz}(z, x) \cdot x$ 
  <proof>

```

For integer power taking a negative of the exponent is the same as taking inverse of the group element.

```

lemma (in group_int0) minus_exp_inv_base: assumes  $z \in \mathbb{Z}$   $x \in G$ 
  shows  $\text{powz}(-z, x) = \text{powz}(z, x^{-1})$ 
  <proof>

```

Integer power of a group element is the same as the inverse of the element raised to negative of the exponent.

```

lemma (in group_int0) minus_exp_inv_base1: assumes  $z \in \mathbb{Z}$   $x \in G$ 
  shows  $\text{powz}(z, x) = \text{powz}(-z, x^{-1})$ 
  <proof>

```

The next context is like `group_int0` but adds the assumption that the group operation is commutative (i.e. the group is abelian).

```

locale abgroup_int0 = group_int0 +
  assumes isAbelian: P {is commutative on} G

```

In abelian groups taking a nonnegative integer power commutes with the group operation. Unfortunately we have to drop to raw set theory notation in the proof to be able to use `int0.Induction_on_int` from the `abgroup_int0` context.

```

lemma (in abgroup_int0) powz_groupop_commute0: assumes  $0 \leq k$   $x \in G$   $y \in G$ 
  shows  $\text{powz}(k, x \cdot y) = \text{powz}(k, x) \cdot \text{powz}(k, y)$ 
  <proof>

```

In abelian groups taking a nonpositive integer power commutes with the group operation. We could use backwards induction in the proof but it is shorter to use the nonnegative case from `powz_groupop_commute0`.

```

lemma (in abgroup_int0) powz_groupop_commute1: assumes  $k \leq 0$   $x \in G$   $y \in G$ 
  shows  $\text{powz}(k, x \cdot y) = \text{powz}(k, x) \cdot \text{powz}(k, y)$ 
  <proof>

```

In abelian groups taking an integer power commutes with the group operation.

```

theorem (in abgroup_int0) powz_groupop_commute: assumes  $z \in \mathbb{Z}$   $x \in G$   $y \in G$ 
  shows  $\text{powz}(z, x \cdot y) = \text{powz}(z, x) \cdot \text{powz}(z, y)$ 
  <proof>

```

For any integer n the mapping $x \mapsto x^n$ maps G into G and is a homomorphism hence an endomorphism of (G, P) .

```

theorem (in abgroup_int0) powz_end: assumes  $n \in \mathbb{Z}$ 
  defines  $h \equiv \{ \langle x, \text{powz}(n, x) \rangle . x \in G \}$ 

```

```

  shows h:G→G  Homomor(h,G,P,G,P)  h ∈ End(G,P)
<proof>

```

The mapping $\mathbb{Z} \ni n \mapsto (x \mapsto x^n \in G)$ maps integers to endomorphisms of (G, P) .

```

theorem (in abgroup_int0) powz_maps_int_End:
  shows {<n,{x,powz(n,x)}. x∈G>}. n∈ℤ} : ℤ→End(G,P)
<proof>

```

end

68 $\mathbb{Z}$ modules

```

theory IntModule_ZF imports Module_ZF IntGroup_ZF

```

begin

In this section we show that the integers, as a ring, have only one module structure on each abelian group. We will show that the module structure is unique, but we will also show which action is the one that defines that module structure.

68.1 Integer powers as the only integer actions on abelian groups

As we show in theorem `powz_maps_int_End` in the `IntGroup` theory the mapping $\mathbb{Z} \ni n \mapsto (G \ni x \mapsto x^n)$ maps integers to endomorphisms of the abelian group G . We will show here that this mapping is the unique action of integers on that group.

When $\mathbb{Z}$ acts on a group, that action is unique.

```

lemma (in int0) action_unique:
  assumes IsLeftModule(int,IntegerAddition,IntegerMultiplication,G,f,S1)
  and
    IsLeftModule(int,IntegerAddition,IntegerMultiplication,G,f,S2)
  shows S1 = S2
<proof>

```

We can use theorems proven in the `abelian_group` locale in the `abgroup_int0` locale.

```

sublocale abgroup_int0 < abgroup: abelian_group
<proof>

```

The rest of the propositions in this section concern the mapping $\mathbb{Z} \ni n \mapsto (G \ni x \mapsto x^n)$. To simplify the statements and proofs we will denote this mapping as $\mathcal{S}$.


```

locale abgroup_int1 = abgroup_int0 +
  fixes S
  defines S_def[simp]:  $S \equiv \{ \langle z, \{ \langle x, \text{powz}(z, x) \rangle . x \in G \} \rangle . z \in \mathbb{Z} \}$ 

```

The mapping S is a group homomorphism between $(\mathbb{Z}, +)$ and (G, P)

```

lemma (in abgroup_int1) group_action_int_add_morphism:
  assumes  $r \in \mathbb{Z} \ s \in \mathbb{Z} \ g \in G$ 
  shows  $(S(r+s))(g) = ((S(r))(g)) \cdot ((S(s))(g))$ 
  <proof>

```

Same as in lemma group_action_int_add_morphism above, but not pointwise:

```

lemma (in abgroup_int1) group_action_int_add_morphism_fun:
  assumes  $r \in \mathbb{Z} \ s \in \mathbb{Z}$ 
  shows  $S(r+s) = \text{EndAdd}(G, P) \langle S(r), S(s) \rangle$ 
  <proof>

```

The mapping S is a homomorphism between $(\mathbb{Z}, \cdot)$ and $(G \rightarrow G, \circ)$

```

lemma (in abgroup_int1) group_action_int_mult_morphism:
  assumes  $r \in \mathbb{Z} \ s \in \mathbb{Z}$ 
  shows  $S(r \cdot_Z s) = \text{EndMult}(G, P) \langle S(r), S(s) \rangle$ 
  <proof>

```

The action maps the integer 1 to the identity, i.e. the neutral element of the composition of endomorphisms (which is the multiplication operation of the ring of endomorphisms of the group).

```

lemma (in abgroup_int1) group_action_int_unit:
  shows  $S(1_Z) = \text{TheNeutralElement}(\text{End}(G, P), \text{EndMult}(G, P))$ 
  <proof>

```

The function from integers to endomorphisms of G defined by $z \mapsto (x \mapsto x^z)$ is a module.

```

theorem (in abgroup_int1) group_action_int:
  shows  $\text{IsLeftModule}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerMultiplication}, G, P, S)$ 
  <proof>

```

If there is a $\mathbb{Z}$ -module on an abelian group, it is the one found in the previous result.

```

theorem (in abgroup_int1) group_action_int_rev0:
  assumes  $\text{IsLeftModule}(\mathbb{Z}, \text{IntegerAddition}, \text{IntegerMultiplication}, G, P, S)$ 
  shows  $S=S$ 
  <proof>

```

end

69 Construction real numbers - the generic part

```

theory Real_ZF imports Int_ZF_IML Ring_ZF_1

```

begin

The goal of the `Real_ZF` series of theory files is to provide a construction of the set of real numbers. There are several ways to construct real numbers. Most common start from the rational numbers and use Dedekind cuts or Cauchy sequences. `Real_ZF_x.thy` series formalizes an alternative approach that constructs real numbers directly from the group of integers. Our formalization is mostly based on [2]. Different variants of this construction are also described in [1] and [3]. I recommend to read these papers, but for the impatient here is a short description: we take a set of maps $s : Z \rightarrow Z$ such that the set $\{s(m+n) - s(m) - s(n)\}_{n,m \in Z}$ is finite (Z means the integers here). We call these maps slopes. Slopes form a group with the natural addition $(s+r)(n) = s(n) + r(n)$. The maps such that the set $s(Z)$ is finite (finite range functions) form a subgroup of slopes. The additive group of real numbers is defined as the quotient group of slopes by the (sub)group of finite range functions. The multiplication is defined as the projection of the composition of slopes into the resulting quotient (coset) space.

69.1 The definition of real numbers

This section contains the construction of the ring of real numbers as classes of slopes - integer almost homomorphisms. The real definitions are in `Group_ZF_2` theory, here we just specialize the definitions of almost homomorphisms, their equivalence and operations to the additive group of integers from the general case of abelian groups considered in `Group_ZF_2`.

The set of slopes is defined as the set of almost homomorphisms on the additive group of integers.

definition

`Slopes` $\equiv$ `AlmostHoms(int,IntegerAddition)`

The first operation on slopes (pointwise addition) is a special case of the first operation on almost homomorphisms.

definition

`SlopeOp1` $\equiv$ `AlHomOp1(int,IntegerAddition)`

The second operation on slopes (composition) is a special case of the second operation on almost homomorphisms.

definition

`SlopeOp2` $\equiv$ `AlHomOp2(int,IntegerAddition)`

Bounded integer maps are functions from integers to integers that have finite range. They play a role of zero in the set of real numbers we are constructing.

definition

`BoundedIntMaps` $\equiv$ `FinRangeFunctions(int,int)`

Bounded integer maps form a normal subgroup of slopes. The equivalence relation on slopes is the (group) quotient relation defined by this subgroup.

definition

`SlopeEquivalenceRel` $\equiv$ `QuotientGroupRel(Slopes,SlopeOp1,BoundedIntMaps)`

The set of real numbers is the set of equivalence classes of slopes.

definition

`RealNumbers` $\equiv$ `Slopes//SlopeEquivalenceRel`

The addition on real numbers is defined as the projection of pointwise addition of slopes on the quotient. This means that the additive group of real numbers is the quotient group: the group of slopes (with pointwise addition) defined by the normal subgroup of bounded integer maps.

definition

`RealAddition` $\equiv$ `ProjFun2(Slopes,SlopeEquivalenceRel,SlopeOp1)`

Multiplication is defined as the projection of composition of slopes on the quotient. The fact that it works is probably the most surprising part of the construction.

definition

`RealMultiplication` $\equiv$ `ProjFun2(Slopes,SlopeEquivalenceRel,SlopeOp2)`

We first show that we can use theorems proven in some proof contexts (locales). The locale `group1` requires assumption that we deal with an abelian group. The next lemma allows to use all theorems proven in the context called `group1`.

lemma `Real_ZF_1_L1: shows group1(int,IntegerAddition)`

<proof>

Real numbers form a ring. This is a special case of the theorem proven in `Ring_ZF_1.thy`, where we show the same in general for almost homomorphisms rather than slopes.

theorem `Real_ZF_1_T1: shows IsAring(RealNumbers,RealAddition,RealMultiplication)`

<proof>

We can use theorems proven in `group0` and `group1` contexts applied to the group of real numbers.

lemma `Real_ZF_1_L2: shows`

`group0(RealNumbers,RealAddition)`
`RealAddition {is commutative on} RealNumbers`
`group1(RealNumbers,RealAddition)`

<proof>

Let's define some notation.

```

locale real0 =

  fixes real ( $\mathbb{R}$ )
  defines real_def [simp]:  $\mathbb{R} \equiv \text{RealNumbers}$ 

  fixes ra (infixl + 69)
  defines ra_def [simp]:  $a + b \equiv \text{RealAddition}\langle a, b \rangle$ 

  fixes rminus (- _ 72)
  defines rminus_def [simp]:  $-a \equiv \text{GroupInv}(\mathbb{R}, \text{RealAddition})(a)$ 

  fixes rsub (infixl - 69)
  defines rsub_def [simp]:  $a - b \equiv a + (-b)$ 

  fixes rm (infixl  $\cdot$  70)
  defines rm_def [simp]:  $a \cdot b \equiv \text{RealMultiplication}\langle a, b \rangle$ 

  fixes rzero (0)
  defines rzero_def [simp]:
  0  $\equiv \text{TheNeutralElement}(\text{RealNumbers}, \text{RealAddition})$ 

  fixes rone (1)
  defines rone_def [simp]:
  1  $\equiv \text{TheNeutralElement}(\text{RealNumbers}, \text{RealMultiplication})$ 

  fixes rtwo (2)
  defines rtwo_def [simp]: 2  $\equiv 1 + 1$ 

  fixes non_zero ( $\mathbb{R}_0$ )
  defines non_zero_def [simp]:  $\mathbb{R}_0 \equiv \mathbb{R} - \{0\}$ 

  fixes inv (_-1 [90] 91)
  defines inv_def [simp]:
   $a^{-1} \equiv \text{GroupInv}(\mathbb{R}_0, \text{restrict}(\text{RealMultiplication}, \mathbb{R}_0 \times \mathbb{R}_0))(a)$ 

```

In real0 context all theorems proven in the ring0, context are valid.

```

lemma (in real0) Real_ZF_1_L3: shows
  ring0( $\mathbb{R}$ , RealAddition, RealMultiplication)
  <proof>

```

Lets try out our notation to see that zero and one are real numbers.

```

lemma (in real0) Real_ZF_1_L4: shows 0 $\in\mathbb{R}$  1 $\in\mathbb{R}$ 
  <proof>

```

The lemma below lists some properties that require one real number to state.

```

lemma (in real0) Real_ZF_1_L5: assumes A1:  $a \in \mathbb{R}$ 
shows
   $(-a) \in \mathbb{R}$ 
   $(-(-a)) = a$ 

```

```

a+0 = a
0+a = a
a·1 = a
1·a = a
a-a = 0
a-0 = a
⟨proof⟩

```

The lemma below lists some properties that require two real numbers to state.

```

lemma (in real0) Real_ZF_1_L6: assumes a∈ℝ  b∈ℝ
  shows
    a+b ∈ ℝ
    a-b ∈ ℝ
    a·b ∈ ℝ
    a+b = b+a
    (-a)·b = -(a·b)
    a·(-b) = -(a·b)
  ⟨proof⟩

```

Multiplication of reals is associative.

```

lemma (in real0) Real_ZF_1_L6A: assumes a∈ℝ  b∈ℝ  c∈ℝ
  shows a·(b·c) = (a·b)·c
  ⟨proof⟩

```

Addition is distributive with respect to multiplication.

```

lemma (in real0) Real_ZF_1_L7: assumes a∈ℝ  b∈ℝ  c∈ℝ
  shows
    a·(b+c) = a·b + a·c
    (b+c)·a = b·a + c·a
    a·(b-c) = a·b - a·c
    (b-c)·a = b·a - c·a
  ⟨proof⟩

```

A simple rearrangement with four real numbers.

```

lemma (in real0) Real_ZF_1_L7A:
  assumes a∈ℝ  b∈ℝ  c∈ℝ  d∈ℝ
  shows a-b + (c-d) = a+c-b-d
  ⟨proof⟩

```

RealAddition is defined as the projection of the first operation on slopes (that is, slope addition) on the quotient (slopes divided by the "almost equal" relation). The next lemma plays with definitions to show that this is the same as the operation induced on the appropriate quotient group. The names AH, Op1 and FR are used in `group1` context to denote almost homomorphisms, the first operation on AH and finite range functions resp.

```

lemma Real_ZF_1_L8: assumes

```

```

AH = AlmostHoms(int,IntegerAddition) and
Op1 = AlHomOp1(int,IntegerAddition) and
FR = FinRangeFunctions(int,int)
shows RealAddition = QuotientGroupOp(AH,Op1,FR)
<proof>

```

The symbol **0** in the `real0` context is defined as the neutral element of real addition. The next lemma shows that this is the same as the neutral element of the appropriate quotient group.

```

lemma (in real0) Real_ZF_1_L9: assumes
  AH = AlmostHoms(int,IntegerAddition) and
  Op1 = AlHomOp1(int,IntegerAddition) and
  FR = FinRangeFunctions(int,int) and
  r = QuotientGroupRel(AH,Op1,FR)
shows
  TheNeutralElement(AH//r,QuotientGroupOp(AH,Op1,FR)) = 0
  SlopeEquivalenceRel = r
<proof>

```

Zero is the class of any finite range function.

```

lemma (in real0) Real_ZF_1_L10:
  assumes A1: s ∈ Slopes
  shows SlopeEquivalenceRel{s} = 0 ⟷ s ∈ BoundedIntMaps
<proof>

```

We will need a couple of results from `Group_ZF_3.thy`. The first two that state that the definition of addition and multiplication of real numbers are consistent, that is the result does not depend on the choice of the slopes representing the numbers. The second one implies that what we call `SlopeEquivalenceRel` is actually an equivalence relation on the set of slopes. We also show that the neutral element of the multiplicative operation on reals (in short number 1) is the class of the identity function on integers.

```

lemma Real_ZF_1_L11: shows
  Congruent2(SlopeEquivalenceRel,SlopeOp1)
  Congruent2(SlopeEquivalenceRel,SlopeOp2)
  SlopeEquivalenceRel ⊆ Slopes × Slopes
  equiv(Slopes, SlopeEquivalenceRel)
  SlopeEquivalenceRel{id(int)} =
  TheNeutralElement(RealNumbers,RealMultiplication)
  BoundedIntMaps ⊆ Slopes
<proof>

```

A one-side implication of the equivalence from `Real_ZF_1_L10`: the class of a bounded integer map is the real zero.

```

lemma (in real0) Real_ZF_1_L11A: assumes s ∈ BoundedIntMaps
  shows SlopeEquivalenceRel{s} = 0
<proof>

```

The next lemma is rephrases the result from `Group_ZF_3.thy` that says that the negative (the group inverse with respect to real addition) of the class of a slope is the class of that slope composed with the integer additive group inverse. The result and proof is not very readable as we use mostly generic set theory notation with long names here. `Real_ZF_1.thy` contains the same statement written in a more readable notation: $[-s] = -[s]$.

```
lemma (in real0) Real_ZF_1_L12: assumes A1: s ∈ Slopes and
  Dr: r = QuotientGroupRel(Slopes,SlopeOp1,BoundedIntMaps)
  shows r{GroupInv(int,IntegerAddition) 0 s} = -(r{s})
  <proof>
```

Two classes are equal iff the slopes that represent them are almost equal.

```
lemma Real_ZF_1_L13: assumes s ∈ Slopes p ∈ Slopes
  and r = SlopeEquivalenceRel
  shows r{s} = r{p} ⟷ ⟨s,p⟩ ∈ r
  <proof>
```

Identity function on integers is a slope. This lemma concludes the easy part of the construction that follows from the fact that slope equivalence classes form a ring. It is easy to see that multiplication of classes of almost homomorphisms is not commutative in general. The remaining properties of real numbers, like commutativity of multiplication and the existence of multiplicative inverses have to be proven using properties of the group of integers, rather than in general setting of abelian groups. This is done in the `Real_ZF_1` theory.

```
lemma Real_ZF_1_L14: shows id(int) ∈ Slopes
  <proof>
```

end

70 Construction of real numbers

```
theory Real_ZF_1 imports Real_ZF Int_ZF_3 OrderedField_ZF
```

begin

In this theory file we continue the construction of real numbers started in `Real_ZF` to a successful conclusion. We put here those parts of the construction that can not be done in the general settings of abelian groups and require integers.

70.1 Definitions and notation

In this section we define notions and notation needed for the rest of the construction.

We define positive slopes as those that take an infinite number of positive values on the positive integers (see `Int_ZF_2` for properties of positive slopes).

definition

$\text{PositiveSlopes} \equiv \{s \in \text{Slopes}.$
 $s(\text{PositiveIntegers}) \cap \text{PositiveIntegers} \notin \text{Fin}(\text{int})\}$

The order on the set of real numbers is constructed by specifying the set of positive reals. This set is defined as the projection of the set of positive slopes.

definition

$\text{PositiveReals} \equiv \{\text{SlopeEquivalenceRel}\{s\}. s \in \text{PositiveSlopes}\}$

The order relation on real numbers is constructed from the set of positive elements in a standard way (see section "Alternative definitions" in `OrderedGroup_ZF`.)

definition

$\text{OrderOnReals} \equiv \text{OrderFromPosSet}(\text{RealNumbers}, \text{RealAddition}, \text{PositiveReals})$

The next locale extends the locale `real0` to define notation specific to the construction of real numbers. The notation follows the one defined in `Int_ZF_2.thy`. If m is an integer, then the real number which is the class of the slope $n \mapsto m \cdot n$ is denoted m^R . For a real number a notation $\lfloor a \rfloor$ means the largest integer m such that the real version of it (that is, m^R) is not greater than a . For an integer m and a subset of reals S the expression $\Gamma(S, m)$ is defined as $\max\{\lfloor p^R \cdot x \rfloor : x \in S\}$. This plays a role in the proof of completeness of real numbers. We also reuse some notation defined in the `int0` context, like $\mathbb{Z}_+$ (the set of positive integers) and $\text{abs}(m)$ (the absolute value of an integer, and some defined in the `int1` context, like the addition ($+$) and composition ($\circ$) of slopes.

locale `real1` = `real0` +

```

fixes AlEq (infix  $\sim$  68)
defines AlEq_def[simp]:  $s \sim r \equiv \langle s, r \rangle \in \text{SlopeEquivalenceRel}$ 

fixes slope_add (infix  $+$  70)
defines slope_add_def[simp]:
 $s + r \equiv \text{SlopeOp1}\langle s, r \rangle$ 

fixes slope_comp (infix  $\circ$  71)
defines slope_comp_def[simp]:  $s \circ r \equiv \text{SlopeOp2}\langle s, r \rangle$ 

fixes slopes ( $S$ )
defines slopes_def[simp]:  $S \equiv \text{AlmostHoms}(\text{int}, \text{IntegerAddition})$ 

fixes posslopes ( $S_+$ )
defines posslopes_def[simp]:  $S_+ \equiv \text{PositiveSlopes}$ 

```



```

fixes slope_class ([ _ ])
defines slope_class_def[simp]: [f]  $\equiv$  SlopeEquivalenceRel{f}

fixes slope_neg (-_ [90] 91)
defines slope_neg_def[simp]: -s  $\equiv$  GroupInv(int,IntegerAddition) 0 s

fixes lesseq (infix  $\leq$  60)
defines lesseq_def[simp]: a  $\leq$  b  $\equiv$   $\langle a,b \rangle \in$  OrderOnReals

fixes sless (infix < 60)
defines sless_def[simp]: a < b  $\equiv$  a $\leq$ b  $\wedge$  a $\neq$ b

fixes positivereals ( $\mathbb{R}_+$ )
defines positivereals_def[simp]:  $\mathbb{R}_+ \equiv$  PositiveSet( $\mathbb{R}$ ,RealAddition,OrderOnReals)

fixes intembed ( $_^R$  [90] 91)
defines intembed_def[simp]:
 $m^R \equiv \{ \langle n, \text{IntegerMultiplication}(m,n) \rangle . n \in \text{int} \}$ 

fixes floor ([_ _])
defines floor_def[simp]:
[a]  $\equiv$  Maximum(IntegerOrder, {m  $\in$  int.  $m^R \leq a$ })

fixes  $\Gamma$ 
defines  $\Gamma$ _def[simp]:  $\Gamma(S,p) \equiv$  Maximum(IntegerOrder, { $\lfloor p^R \cdot x \rfloor . x \in S$ })

fixes ia (infixl + 69)
defines ia_def[simp]: a+b  $\equiv$  IntegerAddition( $\langle a,b \rangle$ )

fixes iminus (- _ 72)
defines iminus_def[simp]: -a  $\equiv$  GroupInv(int,IntegerAddition)(a)

fixes isub (infixl - 69)
defines isub_def[simp]: a-b  $\equiv$  a+ (- b)

fixes intpositives ( $\mathbb{Z}_+$ )
defines intpositives_def[simp]:
 $\mathbb{Z}_+ \equiv$  PositiveSet(int,IntegerAddition,IntegerOrder)

fixes zlesseq (infix  $\leq$  60)
defines zlesseq_def[simp]: m  $\leq$  n  $\equiv$   $\langle m,n \rangle \in$  IntegerOrder

fixes imult (infixl  $\cdot$  70)
defines imult_def[simp]: a $\equiv$  IntegerMultiplication( $\langle a,b \rangle$ )

fixes izero ( $0_{\mathbb{Z}}$ )
defines izero_def[simp]:  $0_{\mathbb{Z}} \equiv$  TheNeutralElement(int,IntegerAddition)

```

```

fixes ione ( $1_Z$ )
defines ione_def[simp]:  $1_Z \equiv \text{TheNeutralElement}(\text{int}, \text{IntegerMultiplication})$ 

fixes itwo ( $2_Z$ )
defines itwo_def[simp]:  $2_Z \equiv 1_Z + 1_Z$ 

fixes abs
defines abs_def[simp]:
abs(m)  $\equiv \text{AbsoluteValue}(\text{int}, \text{IntegerAddition}, \text{IntegerOrder})(m)$ 

fixes  $\delta$ 
defines  $\delta$ _def[simp]:  $\delta(s, m, n) \equiv s(m+n) - s(m) - s(n)$ 

```

70.2 Multiplication of real numbers

Multiplication of real numbers is defined as a projection of composition of slopes onto the space of equivalence classes of slopes. Thus, the product of the real numbers given as classes of slopes s and r is defined as the class of $s \circ r$. The goal of this section is to show that multiplication defined this way is commutative.

Let's recall a theorem from `Int_ZF_2.thy` that states that if f, g are slopes, then $f \circ g$ is equivalent to $g \circ f$. Here we conclude from that that the classes of $f \circ g$ and $g \circ f$ are the same.

```

lemma (in real1) Real_ZF_1_1_L2: assumes A1:  $f \in \mathcal{S} \quad g \in \mathcal{S}$ 
shows  $[f \circ g] = [g \circ f]$ 
 $\langle \text{proof} \rangle$ 

```

Classes of slopes are real numbers.

```

lemma (in real1) Real_ZF_1_1_L3: assumes A1:  $f \in \mathcal{S}$ 
shows  $[f] \in \mathbb{R}$ 
 $\langle \text{proof} \rangle$ 

```

Each real number is a class of a slope.

```

lemma (in real1) Real_ZF_1_1_L3A: assumes A1:  $a \in \mathbb{R}$ 
shows  $\exists f \in \mathcal{S} . a = [f]$ 
 $\langle \text{proof} \rangle$ 

```

It is useful to have the definition of addition and multiplication in the `real1` context notation.

```

lemma (in real1) Real_ZF_1_1_L4:
assumes A1:  $f \in \mathcal{S} \quad g \in \mathcal{S}$ 
shows
 $[f] + [g] = [f+g]$ 
 $[f] \cdot [g] = [f \circ g]$ 
 $\langle \text{proof} \rangle$ 

```

The next lemma is essentially the same as `Real_ZF_1_L12`, but written in the notation defined in the `real1` context. It states that if f is a slope, then $-[f] = [-f]$.

```
lemma (in real1) Real_ZF_1_1_L4A: assumes  $f \in \mathcal{S}$ 
shows  $[-f] = -[f]$ 
 $\langle proof \rangle$ 
```

Subtracting real numbers corresponds to adding the opposite slope.

```
lemma (in real1) Real_ZF_1_1_L4B: assumes A1:  $f \in \mathcal{S}$   $g \in \mathcal{S}$ 
shows  $[f] - [g] = [f + (-g)]$ 
 $\langle proof \rangle$ 
```

Multiplication of real numbers is commutative.

```
theorem (in real1) real_mult_commute: assumes A1:  $a \in \mathbb{R}$   $b \in \mathbb{R}$ 
shows  $a \cdot b = b \cdot a$ 
 $\langle proof \rangle$ 
```

Multiplication is commutative on reals.

```
lemma real_mult_commutative: shows
  RealMultiplication {is commutative on} RealNumbers
 $\langle proof \rangle$ 
```

The neutral element of multiplication of reals (denoted as **1** in the `real1` context) is the class of identity function on integers. This is really shown in `Real_ZF_1_L11`, here we only rewrite it in the notation used in the `real1` context.

```
lemma (in real1) real_one_cl_identity: shows  $[id(int)] = 1$ 
 $\langle proof \rangle$ 
```

If f is bounded, then its class is the neutral element of additive operation on reals (denoted as **0** in the `real1` context).

```
lemma (in real1) real_zero_cl_bounded_map:
assumes  $f \in \text{BoundedIntMaps}$  shows  $[f] = 0$ 
 $\langle proof \rangle$ 
```

Two real numbers are equal iff the slopes that represent them are almost equal. This is proven in `Real_ZF_1_L13`, here we just rewrite it in the notation used in the `real1` context.

```
lemma (in real1) Real_ZF_1_1_L5:
assumes  $f \in \mathcal{S}$   $g \in \mathcal{S}$ 
shows  $[f] = [g] \longleftrightarrow f \sim g$ 
 $\langle proof \rangle$ 
```

If the pair of function belongs to the slope equivalence relation, then their classes are equal. This is convenient, because we don't need to assume that f, g are slopes (follows from the fact that $f \sim g$).

```

lemma (in real1) Real_ZF_1_1_L5A: assumes  $f \sim g$ 
shows  $[f] = [g]$ 
 $\langle proof \rangle$ 

```

Identity function on integers is a slope. This is proven in Real_ZF_1_L13, here we just rewrite it in the notation used in the real1 context.

```

lemma (in real1) id_on_int_is_slope: shows  $\text{id}(\text{int}) \in \mathcal{S}$ 
 $\langle proof \rangle$ 

```

A result from Int_ZF_2.thy: the identity function on integers is not almost equal to any bounded function.

```

lemma (in real1) Real_ZF_1_1_L7:
assumes A1:  $f \in \text{BoundedIntMaps}$ 
shows  $\neg(\text{id}(\text{int}) \sim f)$ 
 $\langle proof \rangle$ 

```

Zero is not one.

```

lemma (in real1) real_zero_not_one: shows  $1 \neq 0$ 
 $\langle proof \rangle$ 

```

Negative of a real number is a real number. Property of groups.

```

lemma (in real1) Real_ZF_1_1_L8: assumes  $a \in \mathbb{R}$  shows  $(-a) \in \mathbb{R}$ 
 $\langle proof \rangle$ 

```

An identity with three real numbers.

```

lemma (in real1) Real_ZF_1_1_L9: assumes  $a \in \mathbb{R}$   $b \in \mathbb{R}$   $c \in \mathbb{R}$ 
shows  $a \cdot (b \cdot c) = a \cdot c \cdot b$ 
 $\langle proof \rangle$ 

```

70.3 The order on reals

In this section we show that the order relation defined by prescribing the set of positive reals as the projection of the set of positive slopes makes the ring of real numbers into an ordered ring. We also collect the facts about ordered groups and rings that we use in the construction.

Positive slopes are slopes and positive reals are real.

```

lemma Real_ZF_1_2_L1: shows
  PositiveSlopes  $\subseteq$  Slopes
  PositiveReals  $\subseteq$  RealNumbers
 $\langle proof \rangle$ 

```

Positive reals are the same as classes of a positive slopes.

```

lemma (in real1) Real_ZF_1_2_L2:
shows  $a \in \text{PositiveReals} \longleftrightarrow (\exists f \in \mathcal{S}_+. a = [f])$ 
 $\langle proof \rangle$ 

```

Let's recall from `Int_ZF_2.thy` that the sum and composition of positive slopes is a positive slope.

```
lemma (in real1) Real_ZF_1_2_L3:
  assumes f ∈ S+  g ∈ S+
  shows
    f+g ∈ S+
    f◦g ∈ S+
  <proof>
```

Bounded integer maps are not positive slopes.

```
lemma (in real1) Real_ZF_1_2_L5:
  assumes f ∈ BoundedIntMaps
  shows f ∉ S+
  <proof>
```

The set of positive reals is closed under addition and multiplication. Zero (the neutral element of addition) is not a positive number.

```
lemma (in real1) Real_ZF_1_2_L6: shows
  PositiveReals {is closed under} RealAddition
  PositiveReals {is closed under} RealMultiplication
  0 ∉ PositiveReals
  <proof>
```

If a class of a slope f is not zero, then either f is a positive slope or $-f$ is a positive slope. The real proof is in `Int_ZF_2.thy`.

```
lemma (in real1) Real_ZF_1_2_L7:
  assumes A1: f ∈ S and A2: [f] ≠ 0
  shows (f ∈ S+) Xor ((-f) ∈ S+)
  <proof>
```

The next lemma rephrases `Int_ZF_2_3_L10` in the notation used in `real1` context.

```
lemma (in real1) Real_ZF_1_2_L8:
  assumes A1: f ∈ S  g ∈ S
  and A2: (f ∈ S+) Xor (g ∈ S+)
  shows ([f] ∈ PositiveReals) Xor ([g] ∈ PositiveReals)
  <proof>
```

The trichotomy law for the (potential) order on reals: if $a \neq 0$, then either a is positive or $-a$ is positive.

```
lemma (in real1) Real_ZF_1_2_L9:
  assumes A1: a ∈ R and A2: a ≠ 0
  shows (a ∈ PositiveReals) Xor ((-a) ∈ PositiveReals)
  <proof>
```

Finally we are ready to prove that real numbers form an ordered ring with no zero divisors.

```

theorem reals_are_ord_ring: shows
  IsAnOrdRing(RealNumbers,RealAddition,RealMultiplication,OrderOnReals)
  OrderOnReals {is total on} RealNumbers
  PositiveSet(RealNumbers,RealAddition,OrderOnReals) = PositiveReals
  HasNoZeroDivs(RealNumbers,RealAddition,RealMultiplication)
<proof>

```

All theorems proven in the `ring1` (about ordered rings), `group3` (about ordered groups) and `group1` (about groups) contexts are valid as applied to ordered real numbers with addition and (real) order.

```

lemma Real_ZF_1_2_L10: shows
  ring1(RealNumbers,RealAddition,RealMultiplication,OrderOnReals)
  IsAnOrdGroup(RealNumbers,RealAddition,OrderOnReals)
  group3(RealNumbers,RealAddition,OrderOnReals)
  OrderOnReals {is total on} RealNumbers
<proof>

```

If $a = b$ or $b - a$ is positive, then a is less or equal b .

```

lemma (in real1) Real_ZF_1_2_L11: assumes A1:  $a \in \mathbb{R}$   $b \in \mathbb{R}$  and
  A3:  $a=b \vee b-a \in \text{PositiveReals}$ 
shows  $a \leq b$ 
<proof>

```

A sufficient condition for two classes to be in the real order.

```

lemma (in real1) Real_ZF_1_2_L12: assumes A1:  $f \in \mathcal{S}$   $g \in \mathcal{S}$  and
  A2:  $f \sim g \vee (g + (-f)) \in \mathcal{S}_+$ 
shows  $[f] \leq [g]$ 
<proof>

```

Taking negative on both sides reverses the inequality, a case with an inverse on one side. Property of ordered groups.

```

lemma (in real1) Real_ZF_1_2_L13:
  assumes A1:  $a \in \mathbb{R}$  and A2:  $(-a) \leq b$ 
shows  $(-b) \leq a$ 
<proof>

```

Real order is antisymmetric.

```

lemma (in real1) real_ord_antisym:
  assumes A1:  $a \leq b$   $b \leq a$  shows  $a=b$ 
<proof>

```

Real order is transitive.

```

lemma (in real1) real_ord_transitive: assumes A1:  $a \leq b$   $b \leq c$ 
shows  $a \leq c$ 
<proof>

```

We can multiply both sides of an inequality by a nonnegative real number.

```

lemma (in real1) Real_ZF_1_2_L14:
  assumes  $a \leq b$  and  $0 \leq c$ 
  shows
     $a \cdot c \leq b \cdot c$ 
     $c \cdot a \leq c \cdot b$ 
  <proof>

```

A special case of Real_ZF_1_2_L14: we can multiply an inequality by a real number.

```

lemma (in real1) Real_ZF_1_2_L14A:
  assumes A1:  $a \leq b$  and A2:  $c \in \mathbb{R}_+$ 
  shows  $c \cdot a \leq c \cdot b$ 
  <proof>

```

In the real1 context notation $a \leq b$ implies that a and b are real numbers.

```

lemma (in real1) Real_ZF_1_2_L15: assumes  $a \leq b$  shows  $a \in \mathbb{R}$   $b \in \mathbb{R}$ 
  <proof>

```

$a \leq b$ implies that $0 \leq b - a$.

```

lemma (in real1) Real_ZF_1_2_L16: assumes  $a \leq b$ 
  shows  $0 \leq b - a$ 
  <proof>

```

A sum of nonnegative elements is nonnegative.

```

lemma (in real1) Real_ZF_1_2_L17: assumes  $0 \leq a$   $0 \leq b$ 
  shows  $0 \leq a + b$ 
  <proof>

```

We can add sides of two inequalities

```

lemma (in real1) Real_ZF_1_2_L18: assumes  $a \leq b$   $c \leq d$ 
  shows  $a + c \leq b + d$ 
  <proof>

```

The order on real is reflexive.

```

lemma (in real1) real_ord_refl: assumes  $a \in \mathbb{R}$  shows  $a \leq a$ 
  <proof>

```

We can add a real number to both sides of an inequality.

```

lemma (in real1) add_num_to_ineq: assumes  $a \leq b$  and  $c \in \mathbb{R}$ 
  shows  $a + c \leq b + c$ 
  <proof>

```

We can put a number on the other side of an inequality, changing its sign.

```

lemma (in real1) Real_ZF_1_2_L19:
  assumes  $a \in \mathbb{R}$   $b \in \mathbb{R}$  and  $c \leq a + b$ 
  shows  $c - b \leq a$ 
  <proof>

```

What happens when one real number is not greater or equal than another?

```
lemma (in real1) Real_ZF_1_2_L20: assumes a∈ℝ b∈ℝ and ¬(a≤b)
  shows b < a
⟨proof⟩
```

We can put a number on the other side of an inequality, changing its sign, version with a minus.

```
lemma (in real1) Real_ZF_1_2_L21:
  assumes a∈ℝ b∈ℝ and c ≤ a-b
  shows c+b ≤ a
⟨proof⟩
```

The order on reals is a relation on reals.

```
lemma (in real1) Real_ZF_1_2_L22: shows OrderOnReals ⊆ ℝ×ℝ
⟨proof⟩
```

A set that is bounded above in the sense defined by order on reals is a subset of real numbers.

```
lemma (in real1) Real_ZF_1_2_L23:
  assumes A1: IsBoundedAbove(A,OrderOnReals)
  shows A ⊆ ℝ
⟨proof⟩
```

Properties of the maximum of three real numbers.

```
lemma (in real1) Real_ZF_1_2_L24:
  assumes A1: a∈ℝ b∈ℝ c∈ℝ
  shows
    Maximum(OrderOnReals,{a,b,c}) ∈ {a,b,c}
    Maximum(OrderOnReals,{a,b,c}) ∈ ℝ
    a ≤ Maximum(OrderOnReals,{a,b,c})
    b ≤ Maximum(OrderOnReals,{a,b,c})
    c ≤ Maximum(OrderOnReals,{a,b,c})
⟨proof⟩
```

A form of transitivity for the order on reals.

```
lemma (in real1) real_strict_ord_transit:
  assumes A1: a≤b and A2: b<c
  shows a<c
⟨proof⟩
```

We can multiply a right hand side of an inequality between positive real numbers by a number that is greater than one.

```
lemma (in real1) Real_ZF_1_2_L25:
  assumes b ∈ ℝ+ and a≤b and 1<c
  shows a<b·c
⟨proof⟩
```


We can move a real number to the other side of a strict inequality, changing its sign.

```
lemma (in real1) Real_ZF_1_2_L26:
  assumes a∈ℝ b∈ℝ and a-b < c
  shows a < c+b
  ⟨proof⟩
```

Real order is translation invariant.

```
lemma (in real1) real_ord_transl_inv:
  assumes a≤b and c∈ℝ
  shows c+a ≤ c+b
  ⟨proof⟩
```

It is convenient to have the transitivity of the order on integers in the notation specific to `real1` context. This may be confusing for the presentation readers: even though $\leq$ and $\leq$ are printed in the same way, they are different symbols in the source. In the `real1` context the former denotes inequality between integers, and the latter denotes inequality between real numbers (classes of slopes). The next lemma is about transitivity of the order relation on integers.

```
lemma (in real1) int_order_transitive:
  assumes A1: a≤b b≤c
  shows a≤c
  ⟨proof⟩
```

A property of nonempty subsets of real numbers that don't have a maximum: for any element we can find one that is (strictly) greater.

```
lemma (in real1) Real_ZF_1_2_L27:
  assumes A⊆ℝ and ¬HasAmaximum(OrderOnReals,A) and x∈A
  shows ∃y∈A. x<y
  ⟨proof⟩
```

The next lemma shows what happens when one real number is not greater or equal than another.

```
lemma (in real1) Real_ZF_1_2_L28:
  assumes a∈ℝ b∈ℝ and ¬(a≤b)
  shows b<a
  ⟨proof⟩
```

If a real number is less than another, then the second one can not be less or equal than the first.

```
lemma (in real1) Real_ZF_1_2_L29:
  assumes a<b shows ¬(b≤a)
  ⟨proof⟩
```

70.4 Inverting reals

In this section we tackle the issue of existence of (multiplicative) inverses of real numbers and show that real numbers form an ordered field. We also restate here some facts specific to ordered fields that we need for the construction. The actual proofs of most of these facts can be found in `Field_ZF.thy` and `OrderedField_ZF.thy`

We rewrite the theorem from `Int_ZF_2.thy` that shows that for every positive slope we can find one that is almost equal and has an inverse.

```
lemma (in real1) pos_slopes_have_inv: assumes f ∈ S+
  shows ∃g∈S. f~g ∧ (∃h∈S. g·h ~ id(int))
  <proof>
```

The set of real numbers we are constructing is an ordered field.

```
theorem (in real1) reals_are_ord_field: shows
  IsAnOrdField(RealNumbers,RealAddition,RealMultiplication,OrderOnReals)
  <proof>
```

Reals form a field.

```
lemma reals_are_field:
  shows IsAfield(RealNumbers,RealAddition,RealMultiplication)
  <proof>
```

Theorem proven in `field0` and `field1` contexts are valid as applied to real numbers.

```
lemma field_cntxts_ok: shows
  field0(RealNumbers,RealAddition,RealMultiplication)
  field1(RealNumbers,RealAddition,RealMultiplication,OrderOnReals)
  <proof>
```

If a is positive, then a^{-1} is also positive.

```
lemma (in real1) Real_ZF_1_3_L1: assumes a ∈ ℝ+
  shows a-1 ∈ ℝ+    a-1 ∈ ℝ
  <proof>
```

A technical fact about multiplying strict inequality by the inverse of one of the sides.

```
lemma (in real1) Real_ZF_1_3_L2:
  assumes a ∈ ℝ+ and a-1 < b
  shows 1 < b·a
  <proof>
```

If a is smaller than b , then $(b - a)^{-1}$ is positive.

```
lemma (in real1) Real_ZF_1_3_L3: assumes a < b
  shows (b-a)-1 ∈ ℝ+
  <proof>
```

We can put a positive factor on the other side of a strict inequality, changing it to its inverse.

lemma (in real1) Real_ZF_1_3_L4:
assumes A1: $a \in \mathbb{R}$ $b \in \mathbb{R}_+$ **and** A2: $a \cdot b < c$
shows $a < c \cdot b^{-1}$
<proof>

We can put a positive factor on the other side of a strict inequality, changing it to its inverse, version with the product initially on the right hand side.

lemma (in real1) Real_ZF_1_3_L4A:
assumes A1: $b \in \mathbb{R}$ $c \in \mathbb{R}_+$ **and** A2: $a < b \cdot c$
shows $a \cdot c^{-1} < b$
<proof>

We can put a positive factor on the other side of an inequality, changing it to its inverse, version with the product initially on the right hand side.

lemma (in real1) Real_ZF_1_3_L4B:
assumes A1: $b \in \mathbb{R}$ $c \in \mathbb{R}_+$ **and** A2: $a \leq b \cdot c$
shows $a \cdot c^{-1} \leq b$
<proof>

We can put a positive factor on the other side of an inequality, changing it to its inverse, version with the product initially on the left hand side.

lemma (in real1) Real_ZF_1_3_L4C:
assumes A1: $a \in \mathbb{R}$ $b \in \mathbb{R}_+$ **and** A2: $a \cdot b \leq c$
shows $a \leq c \cdot b^{-1}$
<proof>

A technical lemma about solving a strict inequality with three real numbers and inverse of a difference.

lemma (in real1) Real_ZF_1_3_L5:
assumes $a < b$ **and** $(b - a)^{-1} < c$
shows $1 + a \cdot c < b \cdot c$
<proof>

We can multiply an inequality by the inverse of a positive number.

lemma (in real1) Real_ZF_1_3_L6:
assumes $a \leq b$ **and** $c \in \mathbb{R}_+$ **shows** $a \cdot c^{-1} \leq b \cdot c^{-1}$
<proof>

We can multiply a strict inequality by a positive number or its inverse.

lemma (in real1) Real_ZF_1_3_L7:
assumes $a < b$ **and** $c \in \mathbb{R}_+$ **shows**
 $a \cdot c < b \cdot c$
 $c \cdot a < c \cdot b$
 $a \cdot c^{-1} < b \cdot c^{-1}$
<proof>

An identity with three real numbers, inverse and cancelling.

lemma (in real1) Real_ZF_1_3_L8: assumes $a \in \mathbb{R}$ $b \in \mathbb{R}$ $b \neq 0$ $c \in \mathbb{R}$
 shows $a \cdot b \cdot (c \cdot b^{-1}) = a \cdot c$
<proof>

70.5 Completeness

This goal of this section is to show that the order on real numbers is complete, that is every subset of reals that is bounded above has a smallest upper bound.

If m is an integer, then m^R is a real number. Recall that in **real1** context m^R denotes the class of the slope $n \mapsto m \cdot n$.

lemma (in real1) real_int_is_real: assumes $m \in \text{int}$
 shows $m^R \in \mathbb{R}$
<proof>

The negative of the real embedding of an integer is the embedding of the negative of the integer.

lemma (in real1) Real_ZF_1_4_L1: assumes $m \in \text{int}$
 shows $(-m)^R = -(m^R)$
<proof>

The embedding of sum of integers is the sum of embeddings.

lemma (in real1) Real_ZF_1_4_L1A: assumes $m \in \text{int}$ $k \in \text{int}$
 shows $m^R + k^R = (m+k)^R$
<proof>

The embedding of a difference of integers is the difference of embeddings.

lemma (in real1) Real_ZF_1_4_L1B: assumes A1: $m \in \text{int}$ $k \in \text{int}$
 shows $m^R - k^R = (m-k)^R$
<proof>

The embedding of the product of integers is the product of embeddings.

lemma (in real1) Real_ZF_1_4_L1C: assumes $m \in \text{int}$ $k \in \text{int}$
 shows $m^R \cdot k^R = (m \cdot k)^R$
<proof>

For any real numbers there is an integer whose real version is greater or equal.

lemma (in real1) Real_ZF_1_4_L2: assumes A1: $a \in \mathbb{R}$
 shows $\exists m \in \text{int}. a \leq m^R$
<proof>

For any real numbers there is an integer whose real version (embedding) is less or equal.

lemma (in real1) Real_ZF_1_4_L3: assumes A1: $a \in \mathbb{R}$
 shows $\{m \in \text{int}. m^R \leq a\} \neq \emptyset$
<proof>

Embeddings of two integers are equal only if the integers are equal.

lemma (in real1) Real_ZF_1_4_L4:
 assumes A1: $m \in \text{int}$ $k \in \text{int}$ and A2: $m^R = k^R$
 shows $m = k$
<proof>

The embedding of integers preserves the order.

lemma (in real1) Real_ZF_1_4_L5: assumes A1: $m \leq k$
 shows $m^R \leq k^R$
<proof>

The embedding of integers preserves the strict order.

lemma (in real1) Real_ZF_1_4_L5A: assumes A1: $m \leq k$ $m \neq k$
 shows $m^R < k^R$
<proof>

For any real number there is a positive integer whose real version is (strictly) greater. This is Lemma 14 i) in [2].

lemma (in real1) Arthan_Lemma14i: assumes A1: $a \in \mathbb{R}$
 shows $\exists n \in \mathbb{Z}_+. a < n^R$
<proof>

If one embedding is less or equal than another, then the integers are also less or equal.

lemma (in real1) Real_ZF_1_4_L6:
 assumes A1: $k \in \text{int}$ $m \in \text{int}$ and A2: $m^R \leq k^R$
 shows $m \leq k$
<proof>

The floor function is well defined and has expected properties.

lemma (in real1) Real_ZF_1_4_L7: assumes A1: $a \in \mathbb{R}$
 shows
 IsBoundedAbove($\{m \in \text{int}. m^R \leq a\}$, IntegerOrder)
 $\{m \in \text{int}. m^R \leq a\} \neq \emptyset$
 $\lfloor a \rfloor \in \text{int}$
 $\lfloor a \rfloor^R \leq a$
<proof>

Every integer whose embedding is less or equal a real number a is less or equal than the floor of a .

lemma (in real1) Real_ZF_1_4_L8:
 assumes A1: $m \in \text{int}$ and A2: $m^R \leq a$
 shows $m \leq \lfloor a \rfloor$

<proof>

Integer zero and one embed as real zero and one.

lemma (in real1) int_0_1_are_real_zero_one:
 shows $0_Z^R = 0$ $1_Z^R = 1$
 <proof>

Integer two embeds as the real two.

lemma (in real1) int_two_is_real_two: **shows** $2_Z^R = 2$
 <proof>

A positive integer embeds as a positive (hence nonnegative) real.

lemma (in real1) int_pos_is_real_pos: **assumes** A1: $p \in \mathbb{Z}_+$
 shows
 $p^R \in \mathbb{R}$
 $0 \leq p^R$
 $p^R \in \mathbb{R}_+$
 <proof>

The ordered field of reals we are constructing is archimedean, i.e., if x, y are its elements with y positive, then there is a positive integer M such that x is smaller than $M^R y$. This is Lemma 14 ii) in [2].

lemma (in real1) Arthan_Lemma14ii: **assumes** A1: $x \in \mathbb{R}$ $y \in \mathbb{R}_+$
 shows $\exists M \in \mathbb{Z}_+. x < M^R \cdot y$
 <proof>

Taking the floor function preserves the order.

lemma (in real1) Real_ZF_1_4_L9: **assumes** A1: $a \leq b$
 shows $\lfloor a \rfloor \leq \lfloor b \rfloor$
 <proof>

If S is bounded above and p is a positive integer, then $\Gamma(S, p)$ is well defined.

lemma (in real1) Real_ZF_1_4_L10:
 assumes A1: IsBoundedAbove(S , OrderOnReals) $S \neq 0$ and A2: $p \in \mathbb{Z}_+$
 shows
 IsBoundedAbove($\{\lfloor p^R \cdot x \rfloor. x \in S\}$, IntegerOrder)
 $\Gamma(S, p) \in \{\lfloor p^R \cdot x \rfloor. x \in S\}$
 $\Gamma(S, p) \in \text{int}$
 <proof>

If p is a positive integer, then for all $s \in S$ the floor of $p \cdot x$ is not greater than $\Gamma(S, p)$.

lemma (in real1) Real_ZF_1_4_L11:
 assumes A1: IsBoundedAbove(S , OrderOnReals) and A2: $x \in S$ and A3: $p \in \mathbb{Z}_+$
 shows $\lfloor p^R \cdot x \rfloor \leq \Gamma(S, p)$
 <proof>

The candidate for supremum is an integer mapping with values given by Γ .

```
lemma (in real1) Real_ZF_1_4_L12:
  assumes A1: IsBoundedAbove(S,OrderOnReals)  S≠0 and
  A2: g = {⟨p,Γ(S,p)⟩. p∈ℤ+}
  shows
  g : ℤ+→int
  ∀n∈ℤ+. g(n) = Γ(S,n)
⟨proof⟩
```

Every integer is equal to the floor of its embedding.

```
lemma (in real1) Real_ZF_1_4_L14: assumes A1: m ∈ int
  shows ⌊mR⌋ = m
⟨proof⟩
```

Floor of (real) zero is (integer) zero.

```
lemma (in real1) floor_01_is_zero_one: shows
  ⌊0⌋ = 0Z   ⌊1⌋ = 1Z
⟨proof⟩
```

Floor of (real) two is (integer) two.

```
lemma (in real1) floor_2_is_two: shows ⌊2⌋ = 2Z
⟨proof⟩
```

Floor of a product of embeddings of integers is equal to the product of integers.

```
lemma (in real1) Real_ZF_1_4_L14A: assumes A1: m ∈ int  k ∈ int
  shows ⌊mR·kR⌋ = m·k
⟨proof⟩
```

Floor of the sum of a number and the embedding of an integer is the floor of the number plus the integer.

```
lemma (in real1) Real_ZF_1_4_L15: assumes A1: x∈ℝ and A2: p ∈ int
  shows ⌊x + pR⌋ = ⌊x⌋ + p
⟨proof⟩
```

Floor of the difference of a number and the embedding of an integer is the floor of the number minus the integer.

```
lemma (in real1) Real_ZF_1_4_L16: assumes A1: x∈ℝ and A2: p ∈ int
  shows ⌊x - pR⌋ = ⌊x⌋ - p
⟨proof⟩
```

The floor of sum of embeddings is the sum of the integers.

```
lemma (in real1) Real_ZF_1_4_L17: assumes m ∈ int  n ∈ int
  shows ⌊mR + nR⌋ = m + n
⟨proof⟩
```

A lemma about adding one to floor.

lemma (in real1) Real_ZF_1_4_L17A: assumes A1: $a \in \mathbb{R}$
 shows $1 + \lfloor a \rfloor^R = (1_Z + \lfloor a \rfloor)^R$
<proof>

The difference between the a number and the embedding of its floor is (strictly) less than one.

lemma (in real1) Real_ZF_1_4_L17B: assumes A1: $a \in \mathbb{R}$
 shows
 $a - \lfloor a \rfloor^R < 1$
 $a < (1_Z + \lfloor a \rfloor)^R$
<proof>

The next lemma corresponds to Lemma 14 iii) in [2]. It says that we can find a rational number between any two different real numbers.

lemma (in real1) Arthan_Lemma14iii: assumes A1: $x < y$
 shows $\exists M \in \text{int}. \exists N \in \mathbb{Z}_+. x \cdot N^R < M^R \wedge M^R < y \cdot N^R$
<proof>

Some estimates for the homomorphism difference of the floor function.

lemma (in real1) Real_ZF_1_4_L18: assumes A1: $x \in \mathbb{R} \quad y \in \mathbb{R}$
 shows
 $\text{abs}(\lfloor x+y \rfloor - \lfloor x \rfloor - \lfloor y \rfloor) \leq 2_Z$
<proof>

Suppose $S \neq \emptyset$ is bounded above and $\Gamma(S, m) = \lfloor m^R \cdot x \rfloor$ for some positive integer m and $x \in S$. Then if $y \in S, x \leq y$ we also have $\Gamma(S, m) = \lfloor m^R \cdot y \rfloor$.

lemma (in real1) Real_ZF_1_4_L20:
 assumes A1: IsBoundedAbove(S, OrderOnReals) $S \neq 0$ and
 A2: $n \in \mathbb{Z}_+ \quad x \in S$ and
 A3: $\Gamma(S, n) = \lfloor n^R \cdot x \rfloor$ and
 A4: $y \in S \quad x \leq y$
 shows $\Gamma(S, n) = \lfloor n^R \cdot y \rfloor$
<proof>

The homomorphism difference of $n \mapsto \Gamma(S, n)$ is bounded by 2 on positive integers.

lemma (in real1) Real_ZF_1_4_L21:
 assumes A1: IsBoundedAbove(S, OrderOnReals) $S \neq 0$ and
 A2: $m \in \mathbb{Z}_+ \quad n \in \mathbb{Z}_+$
 shows $\text{abs}(\Gamma(S, m+n) - \Gamma(S, m) - \Gamma(S, n)) \leq 2_Z$
<proof>

The next lemma provides sufficient condition for an odd function to be an almost homomorphism. It says for odd functions we only need to check that the homomorphism difference (denoted δ in the real1 context) is bounded on positive integers. This is really proven in Int_ZF_2.thy, but we restate it here for convenience. Recall from Group_ZF_3.thy that OddExtension of a

function defined on the set of positive elements (of an ordered group) is the only odd function that is equal to the given one when restricted to positive elements.

lemma (in real1) Real_ZF_1_4_L21A:
 assumes A1: $f: \mathbb{Z}_+ \rightarrow \text{int}$ $\forall a \in \mathbb{Z}_+. \forall b \in \mathbb{Z}_+. \text{abs}(\delta(f, a, b)) \leq L$
 shows OddExtension(int, IntegerAddition, IntegerOrder, f) $\in \mathcal{S}$
<proof>

The candidate for (a representant of) the supremum of a nonempty bounded above set is a slope.

lemma (in real1) Real_ZF_1_4_L22:
 assumes A1: IsBoundedAbove(S, OrderOnReals) $S \neq 0$ and
 A2: $g = \{ \langle p, \Gamma(S, p) \rangle. p \in \mathbb{Z}_+ \}$
 shows OddExtension(int, IntegerAddition, IntegerOrder, g) $\in \mathcal{S}$
<proof>

A technical lemma used in the proof that all elements of S are less or equal than the candidate for supremum of S .

lemma (in real1) Real_ZF_1_4_L23:
 assumes A1: $f \in \mathcal{S}$ and A2: $N \in \text{int}$ $M \in \text{int}$ and
 A3: $\forall n \in \mathbb{Z}_+. M \cdot n \leq f(N \cdot n)$
 shows $M^R \leq [f] \cdot (N^R)$
<proof>

A technical lemma aimed used in the proof the candidate for supremum of S is less or equal than any upper bound for S .

lemma (in real1) Real_ZF_1_4_L23A:
 assumes A1: $f \in \mathcal{S}$ and A2: $N \in \text{int}$ $M \in \text{int}$ and
 A3: $\forall n \in \mathbb{Z}_+. f(N \cdot n) \leq M \cdot n$
 shows $[f] \cdot (N^R) \leq M^R$
<proof>

The essential condition to claim that the candidate for supremum of S is greater or equal than all elements of S .

lemma (in real1) Real_ZF_1_4_L24:
 assumes A1: IsBoundedAbove(S, OrderOnReals) and
 A2: $x < y$ $y \in S$ and
 A4: $N \in \mathbb{Z}_+$ $M \in \text{int}$ and
 A5: $M^R < y \cdot N^R$ and A6: $p \in \mathbb{Z}_+$
 shows $p \cdot M \leq \Gamma(S, p \cdot N)$
<proof>

An obvious fact about odd extension of a function $p \mapsto \Gamma(s, p)$ that is used a couple of times in proofs.

lemma (in real1) Real_ZF_1_4_L24A:
 assumes A1: IsBoundedAbove(S, OrderOnReals) $S \neq 0$ and A2: $p \in \mathbb{Z}_+$
 and A3:

```

    h = OddExtension(int,IntegerAddition,IntegerOrder,{⟨p,Γ(S,p)⟩. p∈ℤ+})
    shows h(p) = Γ(S,p)
  ⟨proof⟩

```

The candidate for the supremum of S is not smaller than any element of S .

```

lemma (in real1) Real_ZF_1_4_L25:
  assumes A1: IsBoundedAbove(S,OrderOnReals) and
    A2: ¬HasAmaximum(OrderOnReals,S) and
    A3: x∈S and A4:
    h = OddExtension(int,IntegerAddition,IntegerOrder,{⟨p,Γ(S,p)⟩. p∈ℤ+})
  shows x ≤ [h]
  ⟨proof⟩

```

The essential condition to claim that the candidate for supremum of S is less or equal than any upper bound of S .

```

lemma (in real1) Real_ZF_1_4_L26:
  assumes A1: IsBoundedAbove(S,OrderOnReals) and
    A2: x≤y x∈S and
    A4: N ∈ ℤ+ M ∈ int and
    A5: y·NR < MR and A6: p ∈ ℤ+
  shows ⌊(N·p)R·x⌋ ≤ M·p
  ⟨proof⟩

```

A piece of the proof of the fact that the candidate for the supremum of S is not greater than any upper bound of S , done separately for clarity (of mind).

```

lemma (in real1) Real_ZF_1_4_L27:
  assumes IsBoundedAbove(S,OrderOnReals) S≠0 and
    h = OddExtension(int,IntegerAddition,IntegerOrder,{⟨p,Γ(S,p)⟩. p∈ℤ+})
  and p ∈ ℤ+
  shows ∃x∈S. h(p) = ⌊pR·x⌋
  ⟨proof⟩

```

The candidate for the supremum of S is not greater than any upper bound of S .

```

lemma (in real1) Real_ZF_1_4_L28:
  assumes A1: IsBoundedAbove(S,OrderOnReals) S≠0
  and A2: ∀x∈S. x≤y and A3:
    h = OddExtension(int,IntegerAddition,IntegerOrder,{⟨p,Γ(S,p)⟩. p∈ℤ+})
  shows [h] ≤ y
  ⟨proof⟩

```

Now we can prove that every nonempty subset of reals that is bounded above has a supremum. Proof by considering two cases: when the set has a maximum and when it does not.

```

lemma (in real1) real_order_complete:
  assumes A1: IsBoundedAbove(S,OrderOnReals) S≠0

```

```

    shows HasAminimum(OrderOnReals,  $\bigcap a \in S. \text{OrderOnReals}\{a\}$ )
  <proof>

```

Finally, we are ready to formulate the main result: that the construction of real numbers from the additive group of integers results in a complete ordered field. This theorem completes the construction. It was fun.

```

theorem eudoxus_reals_are_reals: shows
  IsAmodelOfReals(RealNumbers, RealAddition, RealMultiplication, OrderOnReals)
  <proof>

```

```

end

```

71 Topology - introduction

```

theory Topology_ZF imports ZF1 Finite_ZF Fol1

```

```

begin

```

This theory file provides basic definitions and properties of topology, open and closed sets, closure and boundary.

71.1 Basic definitions and properties

A typical textbook defines a topology on a set X as a collection T of subsets of X such that $X \in T$, $\emptyset \in T$ and T is closed with respect to arbitrary unions and intersection of two sets. One can notice here that since we always have $\bigcup T = X$, the set on which the topology is defined (the "carrier" of the topology) can always be constructed from the topology itself and is superfluous in the definition. Moreover, as Marnix Klooster pointed out to me, the fact that the empty set is open can also be proven from other axioms. Hence, we define a topology as a collection of sets that is closed under arbitrary unions and intersections of two sets, without any mention of the set on which the topology is defined. Recall that $\text{Pow}(T)$ is the powerset of T , so that if $M \in \text{Pow}(T)$ then M is a subset of T . The sets that belong to a topology T will be sometimes called "open in" T or just "open" if the topology is clear from the context.

Topology is a collection of sets that is closed under arbitrary unions and intersections of two sets.

definition

```

  IsATopology (_ {is a topology} [90] 91) where
  T {is a topology}  $\equiv ( \forall M \in \text{Pow}(T). \bigcup M \in T ) \wedge$ 
  (  $\forall U \in T. \forall V \in T. U \cup V \in T$  )

```

We define interior of a set A as the union of all open sets contained in A . We use $\text{Interior}(A, T)$ to denote the interior of A .

definition

$$\text{Interior}(A, T) \equiv \bigcup \{U \in T. U \subseteq A\}$$

A set is closed if it is contained in the carrier of topology and its complement is open.

definition

$$\begin{array}{l} \text{IsClosed (infixl \{is closed in\} 90) where} \\ D \text{ \{is closed in\} } T \equiv (D \subseteq \bigcup T \wedge (\bigcup T) \setminus D \in T) \end{array}$$

To prove various properties of closure we will often use the collection of closed sets that contain a given set A . Such collection does not have a separate name in informal math. We will call it $\text{ClosedCovers}(A, T)$.

definition

$$\text{ClosedCovers}(A, T) \equiv \{D \in \text{Pow}(\bigcup T). D \text{ \{is closed in\} } T \wedge A \subseteq D\}$$

The closure of a set A is defined as the intersection of the collection of closed sets that contain A .

definition

$$\text{Closure}(A, T) \equiv \bigcap \text{ClosedCovers}(A, T)$$

We also define boundary of a set as the intersection of its closure with the closure of the complement (with respect to the carrier).

definition

$$\text{Boundary}(A, T) \equiv \text{Closure}(A, T) \cap \text{Closure}(\bigcup T - A, T)$$

A set K is compact if for every collection of open sets that covers K we can choose a finite one that still covers the set. Recall that $\text{FinPow}(M)$ is the collection of finite subsets of M (finite powerset of M), defined in IsarMathLib's `Finite_ZF` theory.

definition

$$\begin{array}{l} \text{IsCompact (infixl \{is compact in\} 90) where} \\ K \text{ \{is compact in\} } T \equiv (K \subseteq \bigcup T \wedge \\ (\forall M \in \text{Pow}(T). K \subseteq \bigcup M \longrightarrow (\exists N \in \text{FinPow}(M). K \subseteq \bigcup N))) \end{array}$$

A basic example of a topology: the powerset of any set is a topology.

lemma `Pow_is_top`: `shows Pow(X) \{is a topology\}`
`\<proof>`

Empty set is open.

lemma `empty_open`:

`assumes T \{is a topology\} shows \emptyset \in T`
`\<proof>`

The carrier is open.

lemma `carr_open`: `assumes T \{is a topology\} shows (\bigcup T) \in T`
`\<proof>`

Union of a collection of open sets is open.

lemma union_open: assumes T {is a topology} and $\forall A \in \mathcal{A}. A \in T$
 shows $(\bigcup \mathcal{A}) \in T$ *<proof>*

Union of a indexed family of open sets is open.

lemma union_indexed_open: assumes $A1$: T {is a topology} and $A2$: $\forall i \in I. P(i) \in T$
 shows $(\bigcup_{i \in I} P(i)) \in T$ *<proof>*

The complement of an open set is closed.

lemma compl_open_closed: assumes $U \in T$ shows $((\bigcup T) \setminus U)$ {is closed in} T
<proof>

The intersection of any nonempty collection of topologies on a set X is a topology.

lemma Inter_tops_is_top:
 assumes $A1$: $\mathcal{M} \neq 0$ and $A2$: $\forall T \in \mathcal{M}. T$ {is a topology}
 shows $(\bigcap \mathcal{M})$ {is a topology}
<proof>

Singletons are compact. Interestingly we do not have to assume that T is a topology for this. Note singletons do not have to be closed, we need the the space to be T_1 for that (see Topology_ZF_1).

lemma singl_compact:
 assumes $x \in \bigcup T$ shows $\{x\}$ {is compact in} T
<proof>

We will now introduce some notation. In Isar, this is done by definining a "locale". Locale is kind of a context that holds some assumptions and notation used in all theorems proven in it. In the locale (context) below called `topology0` we assume that T is a topology. The interior of the set A (with respect to the topology in the context) is denoted $\text{int}(A)$. The closure of a set $A \subseteq \bigcup T$ is denoted $\text{cl}(A)$ and the boundary is ∂A .

```
locale topology0 =
  fixes T
  assumes topSpaceAssum: T {is a topology}

  fixes int
  defines int_def [simp]: int(A)  $\equiv$  Interior(A,T)

  fixes cl
  defines cl_def [simp]: cl(A)  $\equiv$  Closure(A,T)

  fixes boundary ( $\partial$ _ [91] 92)
  defines boundary_def [simp]:  $\partial A \equiv$  Boundary(A,T)
```

Intersection of a finite nonempty collection of open sets is open.

```
lemma (in topology0) fin_inter_open_open: assumes N≠0 N ∈ FinPow(T)
shows  $\bigcap N \in T$ 
  <proof>
```

Having a topology T and a set X we can define the induced topology as the one consisting of the intersections of X with sets from T . The notion of a collection restricted to a set is defined in ZF1.thy.

```
lemma (in topology0) Top_1_L4:
shows (T {restricted to} X) {is a topology}
  <proof>
```

71.2 Interior of a set

In this section we show basic properties of the interior of a set.

Interior of a set A is contained in A .

```
lemma (in topology0) Top_2_L1: shows  $\text{int}(A) \subseteq A$ 
  <proof>
```

Interior is open.

```
lemma (in topology0) Top_2_L2: shows  $\text{int}(A) \in T$ 
  <proof>
```

A set is open iff it is equal to its interior.

```
lemma (in topology0) Top_2_L3: shows  $U \in T \longleftrightarrow \text{int}(U) = U$ 
  <proof>
```

Interior of the interior is the interior.

```
lemma (in topology0) Top_2_L4: shows  $\text{int}(\text{int}(A)) = \text{int}(A)$ 
  <proof>
```

Interior of a bigger set is bigger.

```
lemma (in topology0) interior_mono:
assumes A1:  $A \subseteq B$  shows  $\text{int}(A) \subseteq \text{int}(B)$ 
  <proof>
```

An open subset of any set is a subset of the interior of that set.

```
lemma (in topology0) Top_2_L5: assumes  $U \subseteq A$  and  $U \in T$ 
shows  $U \subseteq \text{int}(A)$ 
  <proof>
```

If a point of a set has an open neighborhood contained in the set, then the point belongs to the interior of the set.

```
lemma (in topology0) Top_2_L6: assumes  $\exists U \in T. (x \in U \wedge U \subseteq A)$ 
shows  $x \in \text{int}(A)$ 
```

⟨proof⟩

A set is open iff its every point has a an open neighbourhood contained in the set. We will formulate this statement as two lemmas (implication one way and the other way). The lemma below shows that if a set is open then every point has a an open neighbourhood contained in the set.

lemma (in topology0) open_open_neigh:
assumes A1: $\forall x \in T$
shows $\forall x \in V. \exists U \in T. (x \in U \wedge U \subseteq V)$
⟨proof⟩

If every point of a set has a an open neighbourhood contained in the set then the set is open.

lemma (in topology0) open_neigh_open:
assumes A1: $\forall x \in V. \exists U \in T. (x \in U \wedge U \subseteq V)$
shows $V \in T$
⟨proof⟩

The intersection of interiors is a equal to the interior of intersections.

lemma (in topology0) int_inter_int: **shows** $\text{int}(A) \cap \text{int}(B) = \text{int}(A \cap B)$
⟨proof⟩

71.3 Closed sets, closure, boundary.

This section is devoted to closed sets and properties of the closure and boundary operators.

The carrier of the space is closed.

lemma (in topology0) Top_3_L1: **shows** $(\bigcup T) \{\text{is closed in}\} T$
⟨proof⟩

Empty set is closed.

lemma (in topology0) Top_3_L2: **shows** $0 \{\text{is closed in}\} T$
⟨proof⟩

The collection of closed covers of a subset of the carrier of topology is never empty. This is good to know, as we want to intersect this collection to get the closure.

lemma (in topology0) Top_3_L3:
assumes A1: $A \subseteq \bigcup T$ **shows** $\text{ClosedCovers}(A, T) \neq 0$
⟨proof⟩

Intersection of a nonempty family of closed sets is closed.

lemma (in topology0) Top_3_L4: **assumes** A1: $K \neq 0$ and
A2: $\forall D \in K. D \{\text{is closed in}\} T$
shows $(\bigcap K) \{\text{is closed in}\} T$

<proof>

The union and intersection of two closed sets are closed.

lemma (in topology0) Top_3_L5:
 assumes A1: D_1 {is closed in} T D_2 {is closed in} T
 shows
 $(D_1 \cap D_2)$ {is closed in} T
 $(D_1 \cup D_2)$ {is closed in} T
<proof>

Finite union of closed sets is closed. To understand the proof recall that $D \in \text{Pow}(\bigcup T)$ means that D is a subset of the carrier of the topology.

lemma (in topology0) fin_union_cl_is_cl:
 assumes
 A1: $N \in \text{FinPow}(\{D \in \text{Pow}(\bigcup T). D \text{ {is closed in} } T\})$
 shows $(\bigcup N)$ {is closed in} T
<proof>

Closure of a set is closed, hence the complement of the closure is open.

lemma (in topology0) cl_is_closed: assumes $A \subseteq \bigcup T$
 shows $\text{cl}(A)$ {is closed in} T and $\bigcup T - \text{cl}(A) \in T$
<proof>

Closure of a bigger sets is bigger.

lemma (in topology0) top_closure_mono:
 assumes A1: $B \subseteq \bigcup T$ and A2: $A \subseteq B$
 shows $\text{cl}(A) \subseteq \text{cl}(B)$
<proof>

Boundary of a set is closed.

lemma (in topology0) boundary_closed:
 assumes A1: $A \subseteq \bigcup T$ shows ∂A {is closed in} T
<proof>

A set is closed iff it is equal to its closure.

lemma (in topology0) Top_3_L8: assumes A1: $A \subseteq \bigcup T$
 shows A {is closed in} $T \longleftrightarrow \text{cl}(A) = A$
<proof>

Complement of an open set is closed.

lemma (in topology0) Top_3_L9: assumes A1: $A \in T$
 shows $(\bigcup T - A)$ {is closed in} T
<proof>

A set is contained in its closure.

lemma (in topology0) cl_contains_set: assumes $A \subseteq \bigcup T$ shows $A \subseteq \text{cl}(A)$
<proof>

Closure of a subset of the carrier is a subset of the carrier and closure of the complement is the complement of the interior.

```
lemma (in topology0) Top_3_L11: assumes A1:  $A \subseteq \bigcup T$ 
  shows
     $\text{cl}(A) \subseteq \bigcup T$ 
     $\text{cl}(\bigcup T \setminus A) = \bigcup T \setminus \text{int}(A)$ 
  <proof>
```

Boundary of a set is the closure of the set minus the interior of the set.

```
lemma (in topology0) Top_3_L12: assumes A1:  $A \subseteq \bigcup T$ 
  shows  $\partial A = \text{cl}(A) - \text{int}(A)$ 
  <proof>
```

If a set A is contained in a closed set B , then the closure of A is contained in B .

```
lemma (in topology0) Top_3_L13:
  assumes A1:  $B \{\text{is closed in}\} T$     $A \subseteq B$ 
  shows  $\text{cl}(A) \subseteq B$ 
  <proof>
```

If a set is disjoint with an open set, then we can close it and it will still be disjoint.

```
lemma (in topology0) disj_open_cl_disj:
  assumes A1:  $A \subseteq \bigcup T$     $\forall U \in T \text{ and } A2: A \cap U = 0$ 
  shows  $\text{cl}(A) \cap U = 0$ 
  <proof>
```

A reformulation of disj_open_cl_disj: If a point belongs to the closure of a set, then we can find a point from the set in any open neighborhood of the point.

```
lemma (in topology0) cl_inter_neigh:
  assumes A1:  $A \subseteq \bigcup T$  and  $U \in T$  and  $x \in \text{cl}(A) \cap U$ 
  shows  $A \cap U \neq 0$  <proof>
```

A reverse of cl_inter_neigh: if every open neighborhood of a point has a nonempty intersection with a set, then that point belongs to the closure of the set.

```
lemma (in topology0) inter_neigh_cl:
  assumes A1:  $A \subseteq \bigcup T$  and A2:  $x \in \bigcup T$  and A3:  $\forall U \in T. x \in U \longrightarrow U \cap A \neq 0$ 
  shows  $x \in \text{cl}(A)$ 
  <proof>
```

end

72 Topology 1

```
theory Topology_ZF_1 imports Topology_ZF
```

begin

In this theory file we study separation axioms and the notion of base and subbase. Using the products of open sets as a subbase we define a natural topology on a product of two topological spaces.

72.1 Separation axioms

Topological spaces can be classified according to certain properties called "separation axioms". In this section we define what it means that a topological space is T_0 , T_1 or T_2 .

A topology on X is T_0 if for every pair of distinct points of X there is an open set that contains only one of them.

definition

isT0 ($_$ {is T_0 } [90] 91) **where**
 T {is T_0 } $\equiv \forall x y. ((x \in \bigcup T \wedge y \in \bigcup T \wedge x \neq y) \longrightarrow$
 $(\exists U \in T. (x \in U \wedge y \notin U) \vee (y \in U \wedge x \notin U)))$

A topology is T_1 if for every such pair there exist an open set that contains the first point but not the second.

definition

isT1 ($_$ {is T_1 } [90] 91) **where**
 T {is T_1 } $\equiv \forall x y. ((x \in \bigcup T \wedge y \in \bigcup T \wedge x \neq y) \longrightarrow$
 $(\exists U \in T. (x \in U \wedge y \notin U)))$

T_1 topological spaces are exactly those in which all singletons are closed.

lemma (in topology0) **t1_def_alt:**

shows T {is T_1 } $\longleftrightarrow (\forall x \in \bigcup T. \{x\} \text{ is closed in } T)$
<proof>

A topology is T_2 (Hausdorff) if for every pair of points there exist a pair of disjoint open sets each containing one of the points. This is an important class of topological spaces. In particular, metric spaces are Hausdorff.

definition

isT2 ($_$ {is T_2 } [90] 91) **where**
 T {is T_2 } $\equiv \forall x y. ((x \in \bigcup T \wedge y \in \bigcup T \wedge x \neq y) \longrightarrow$
 $(\exists U \in T. \exists V \in T. x \in U \wedge y \in V \wedge U \cap V = \emptyset))$

A topology is regular if every closed set can be separated from a point in its complement by (disjoint) opens sets.

definition

IsRegular ($_$ {is regular} 90)
where T {is regular} $\equiv \forall D. D \text{ is closed in } T \longrightarrow (\forall x \in \bigcup T - D. \exists U \in T. \exists V \in T.$
 $D \subseteq U \wedge x \in V \wedge U \cap V = \emptyset)$

Some sources (e.g. Metamath) use a different definition of regularity: any open neighborhood has a closed subneighborhood. The next lemma shows the equivalence of this with our definition.

lemma is_regular_def_alt: **assumes** T {is a topology}
shows T {is regular} $\longleftrightarrow (\forall W \in T. \forall x \in W. \exists V \in T. x \in V \wedge \text{Closure}(V, T) \subseteq W)$
<proof>

If a topology is T_1 then it is T_0 . We don't really assume here that T is a topology on X . Instead, we prove the relation between $\text{is}T_0$ condition and $\text{is}T_1$.

lemma T1_is_T0: **assumes** $A1$: T {is T_1 } **shows** T {is T_0 }
<proof>

If a topology is T_2 then it is T_1 .

lemma T2_is_T1: **assumes** $A1$: T {is T_2 } **shows** T {is T_1 }
<proof>

In a T_0 space two points that can not be separated by an open set are equal. Proof by contradiction.

lemma Top_1_1_L1: **assumes** $A1$: T {is T_0 } **and** $A2$: $x \in \bigcup T$ $y \in \bigcup T$
and $A3$: $\forall U \in T. (x \in U \longleftrightarrow y \in U)$
shows $x = y$
<proof>

A topology is T_3 if it is regular and T_0 . T_3 spaces are called "regular Hausdorff", which is a bit confusing as the definition requires the space to be T_0 rather than T_2 . It is ok though as we will show that T_3 as defined here implies T_2 so indeed T_3 spaces are regular and Hausdorff. In some older sources the definitions of a regular and a T_3 space are swapped. We follow the terminology from Wikipedia's "Separation axiom" entry, where T_3 implies "regular".

definition

$\text{is}T_3$ ($_$ {is T_3 } [90] 91) **where**
 T {is T_3 } $\equiv (T$ {is regular}) $\wedge T$ {is T_0 }

If a topology is T_3 then it is T_2 . It's interesting that even here we do not have to assume that T is a topology.

lemma T3_is_T2: **assumes** T {is T_3 } **shows** T {is T_2 }
<proof>

Sometimes T_3 space is defined as one that is regular and T_1 (rather than T_0). The next lemma shows that this definition is equivalent to the standard one.

lemma T3_def_alt: **shows** T {is T_3 } $\longleftrightarrow (T$ {is regular}) $\wedge T$ {is T_1 }
<proof>

72.2 Bases and subbases

Sometimes it is convenient to talk about topologies in terms of their bases and subbases. These are certain collections of open sets that define the whole topology.

A base of topology is a collection of open sets such that every open set is a union of the sets from the base.

definition

IsABaseFor (infixl {is a base for} 65) **where**
 $B \text{ {is a base for} } T \equiv B \subseteq T \wedge T = \{\bigcup A. A \in \text{Pow}(B)\}$

A subbase is a collection of open sets such that finite intersection of those sets form a base.

definition

IsASubBaseFor (infixl {is a subbase for} 65) **where**
 $B \text{ {is a subbase for} } T \equiv$
 $B \subseteq T \wedge \{\bigcap A. A \in \text{FinPow}(B)\} \text{ {is a base for} } T$

Below we formulate a condition that we will prove to be necessary and sufficient for a collection B of open sets to form a base. It says that for any two sets U, V from the collection B we can find a point $x \in U \cap V$ with a neighborhood from B contained in $U \cap V$.

definition

SatisfiesBaseCondition ($_$ {satisfies the base condition} [50] 50)
where
 $B \text{ {satisfies the base condition} } \equiv$
 $\forall U V. ((U \in B \wedge V \in B) \longrightarrow (\exists x \in U \cap V. \exists W \in B. x \in W \wedge W \subseteq U \cap V))$

A collection that is closed with respect to intersection satisfies the base condition.

lemma `inter_closed_base`: **assumes** $\forall U \in B. (\forall V \in B. U \cap V \in B)$
shows $B \text{ {satisfies the base condition} }$
 $\langle \text{proof} \rangle$

Each open set is a union of some sets from the base.

lemma `Top_1_2_L1`: **assumes** $B \text{ {is a base for} } T$ **and** $U \in T$
shows $\exists A \in \text{Pow}(B). U = \bigcup A$
 $\langle \text{proof} \rangle$

Elements of base are open.

lemma `base_sets_open`:
assumes $B \text{ {is a base for} } T$ **and** $U \in B$
shows $U \in T$
 $\langle \text{proof} \rangle$

A base defines topology uniquely.

```

lemma same_base_same_top:
  assumes B {is a base for} T and B {is a base for} S
  shows T = S
  <proof>

```

Every point from an open set has a neighborhood from the base that is contained in the set.

```

lemma point_open_base_neigh:
  assumes A1: B {is a base for} T and A2:  $U \in T$  and A3:  $x \in U$ 
  shows  $\exists V \in B. V \subseteq U \wedge x \in V$ 
  <proof>

```

A criterion for a collection to be a base for a topology that is a slight reformulation of the definition. The only thing different that in the definition is that we assume only that every open set is a union of some sets from the base. The definition requires also the opposite inclusion that every union of the sets from the base is open, but that we can prove if we assume that T is a topology.

```

lemma is_a_base_criterion: assumes A1:  $T$  {is a topology}
  and A2:  $B \subseteq T$  and A3:  $\forall V \in T. \exists A \in \text{Pow}(B). V = \bigcup A$ 
  shows B {is a base for} T
  <proof>

```

A necessary condition for a collection of sets to be a base for some topology : every point in the intersection of two sets in the base has a neighborhood from the base contained in the intersection.

```

lemma Top_1_2_L2:
  assumes A1:  $\exists T. T$  {is a topology}  $\wedge B$  {is a base for} T
  and A2:  $\forall B \in B \ W \in B$ 
  shows  $\forall x \in V \cap W. \exists U \in B. x \in U \wedge U \subseteq V \cap W$ 
  <proof>

```

We will construct a topology as the collection of unions of (would-be) base. First we prove that if the collection of sets satisfies the condition we want to show to be sufficient, the the intersection belongs to what we will define as topology (am I clear here?). Having this fact ready simplifies the proof of the next lemma. There is not much topology here, just some set theory.

```

lemma Top_1_2_L3:
  assumes A1:  $\forall x \in V \cap W. \exists U \in B. x \in U \wedge U \subseteq V \cap W$ 
  shows  $V \cap W \in \{\bigcup A. A \in \text{Pow}(B)\}$ 
  <proof>

```

The next lemma is needed when proving that the would-be topology is closed with respect to taking intersections. We show here that intersection of two sets from this (would-be) topology can be written as union of sets from the topology.

lemma Top_1_2_L4:
 assumes A1: $U_1 \in \{\bigcup A. A \in \text{Pow}(B)\}$ $U_2 \in \{\bigcup A. A \in \text{Pow}(B)\}$
 and A2: B {satisfies the base condition}
 shows $\exists C. C \subseteq \{\bigcup A. A \in \text{Pow}(B)\} \wedge U_1 \cap U_2 = \bigcup C$
<proof>

If B satisfies the base condition, then the collection of unions of sets from B is a topology and B is a base for this topology.

theorem Top_1_2_T1:
 assumes A1: B {satisfies the base condition}
 and A2: $T = \{\bigcup A. A \in \text{Pow}(B)\}$
 shows T {is a topology} and B {is a base for} T
<proof>

The carrier of the base and topology are the same.

lemma Top_1_2_L5: assumes B {is a base for} T
 shows $\bigcup T = \bigcup B$
<proof>

If B is a base for T , then T is the smallest topology containing B .

lemma base_smallest_top:
 assumes A1: B {is a base for} T and A2: S {is a topology} and A3: $B \subseteq S$
 shows $T \subseteq S$
<proof>

If B is a base for T and B is a topology, then $B = T$.

lemma base_topology: assumes B {is a topology} and B {is a base for} T
 shows $B = T$ *<proof>*

72.3 Product topology

In this section we consider a topology defined on a product of two sets.

Given two topological spaces we can define a topology on the product of the carriers such that the cartesian products of the sets of the topologies are a base for the product topology. Recall that for two collections S, T of sets the product collection is defined (in `ZF1.thy`) as the collections of cartesian products $A \times B$, where $A \in S, B \in T$. The $T \times_t S$ notation is defined as an alternative to the verbose `ProductTopology(T, S)`.

definition ProductTopology (infixl $\times_t$ 65) where
 $T \times_t S \equiv \{\bigcup W. W \in \text{Pow}(\text{ProductCollection}(T, S))\}$

The product collection satisfies the base condition.

lemma Top_1_4_L1:
 assumes A1: T {is a topology} S {is a topology}

and A2: $A \in \text{ProductCollection}(T, S)$ $B \in \text{ProductCollection}(T, S)$
shows $\forall x \in (A \cap B). \exists W \in \text{ProductCollection}(T, S). (x \in W \wedge W \subseteq A \cap B)$
<proof>

The product topology is indeed a topology on the product.

theorem Top_1_4_T1: **assumes** A1: T {is a topology} S {is a topology}
shows
 $(T \times_t S)$ {is a topology}
 $\text{ProductCollection}(T, S)$ {is a base for} $(T \times_t S)$
 $\bigcup (T \times_t S) = \bigcup T \times \bigcup S$
<proof>

Each point of a set open in the product topology has a neighborhood which is a cartesian product of open sets.

lemma prod_top_point_neighb:
assumes A1: T {is a topology} S {is a topology} **and**
A2: $U \in \text{ProductTopology}(T, S)$ **and** A3: $x \in U$
shows $\exists V \ W. \forall T \ \wedge \ W \in S \ \wedge \ V \times W \subseteq U \ \wedge \ x \in V \times W$
<proof>

Products of open sets are open in the product topology.

lemma prod_open_open_prod:
assumes A1: T {is a topology} S {is a topology} **and**
A2: $U \in T \ V \in S$
shows $U \times V \in \text{ProductTopology}(T, S)$
<proof>

Sets that are open in the product topology are contained in the product of the carrier.

lemma prod_open_type: **assumes** A1: T {is a topology} S {is a topology}
and
A2: $V \in \text{ProductTopology}(T, S)$
shows $V \subseteq \bigcup T \times \bigcup S$
<proof>

A reverse of prod_top_point_neighb: if each point of set has an neighborhood in the set that is a cartesian product of open sets, then the set is open.

lemma point_neighb_prod_top:
assumes T {is a topology} S {is a topology}
and $\forall p \in V. \exists U \in T. \exists W \in S. p \in U \times W \ \wedge \ U \times W \subseteq V$
shows $V \in \text{ProductTopology}(T, S)$
<proof>

Suppose we have subsets $A \subseteq X, B \subseteq Y$, where X, Y are topological spaces with topologies T, S . We can consider relative topologies on T_A, S_B on sets A, B and the collection of cartesian products of sets open in T_A, S_B , (namely $\{U \times V : U \in T_A, V \in S_B\}$). The next lemma states that this

collection is a base of the product topology on $X \times Y$ restricted to the product $A \times B$.

```
lemma prod_restr_base_restr:
  assumes A1: T {is a topology} S {is a topology}
  shows
    ProductCollection(T {restricted to} A, S {restricted to} B)
    {is a base for} (ProductTopology(T,S) {restricted to} A×B)
  <proof>
```

We can commute taking restriction (relative topology) and product topology. The reason the two topologies are the same is that they have the same base.

```
lemma prod_top_restr_comm:
  assumes A1: T {is a topology} S {is a topology}
  shows
    ProductTopology(T {restricted to} A, S {restricted to} B) =
    ProductTopology(T,S) {restricted to} (A×B)
  <proof>
```

Projection of a section of an open set is open.

```
lemma prod_sec_open1: assumes A1: T {is a topology} S {is a topology}
and
  A2:  $V \in \text{ProductTopology}(T,S)$  and A3:  $x \in \bigcup T$ 
  shows  $\{y \in \bigcup S. \langle x,y \rangle \in V\} \in S$ 
  <proof>
```

Projection of a section of an open set is open. This is dual of `prod_sec_open1` with a very similar proof.

```
lemma prod_sec_open2: assumes A1: T {is a topology} S {is a topology}
and
  A2:  $V \in \text{ProductTopology}(T,S)$  and A3:  $y \in \bigcup S$ 
  shows  $\{x \in \bigcup T. \langle x,y \rangle \in V\} \in T$ 
  <proof>
```

72.4 Hausdorff spaces

In this section we study properties of Hausdorff spaces (sometimes called separated spaces) These are topological spaces that are T_2 as defined above.

A space is Hausdorff if and only if the diagonal $\Delta = \{\langle x,x \rangle : x \in X\}$ is closed in the product topology on $X \times X$.

```
theorem t2_iff_diag_closed: assumes T {is a topology}
  shows  $T \text{ is } T_2 \iff \{\langle x,x \rangle. x \in \bigcup T\} \text{ is closed in } \text{ProductTopology}(T,T)$ 
  <proof>
```

72.5 Compact sets are closed in Hausdorff spaces

One of the facts demonstrated in every class on General Topology is that in a T_2 (Hausdorff) topological space compact sets are closed. Formalizing the

proof of this fact gave me an interesting insight into the role of the Axiom of Choice (AC) in many informal proofs.

A typical informal proof of this fact goes like this: we want to show that the complement of K is open. To do this, choose an arbitrary point $y \in K^c$. Since X is T_2 , for every point $x \in K$ we can find an open set U_x such that $y \notin \overline{U_x}$. Obviously $\{U_x\}_{x \in K}$ covers K , so select a finite subcollection that covers K , and so on. I had never realized that such reasoning requires the Axiom of Choice. Namely, suppose we have a lemma that states "In T_2 spaces, if $x \neq y$, then there is an open set U such that $x \in U$ and $y \notin \overline{U}$ " (like our lemma `T2_cl_open_sep` below). This only states that the set of such open sets U is not empty. To get the collection $\{U_x\}_{x \in K}$ in this proof we have to select one such set among many for every $x \in K$ and this is where we use the Axiom of Choice. Probably in 99/100 cases when an informal calculus proof states something like $\forall \varepsilon \exists \delta_\varepsilon \dots$ the proof uses AC. Most of the time the use of AC in such proofs can be avoided. This is also the case for the fact that in a T_2 space compact sets are closed.

First we prove a lemma that in a T_2 space two points can be separated by the closure of an open set.

```
lemma (in topology0) T2_cl_open_sep:
  assumes T {is T2} and x ∈ ⋃ T y ∈ ⋃ T x ≠ y
  shows ∃ U ∈ T. (x ∈ U ∧ y ∉ cl(U))
⟨proof⟩
```

AC-free proof that in a Hausdorff space compact sets are closed. To understand the notation recall that in Isabelle/ZF `Pow(A)` is the powerset (the set of subsets) of A and `FinPow(A)` denotes the set of finite subsets of A in `IsarMathLib`.

```
theorem (in topology0) in_t2_compact_is_cl:
  assumes A1: T {is T2} and A2: K {is compact in} T
  shows K {is closed in} T
⟨proof⟩
```

end

73 Topology 2

```
theory Topology_ZF_2 imports Topology_ZF_1 func1 Fol1
```

```
begin
```

This theory continues the series on general topology and covers the definition and basic properties of continuous functions. We also introduce the notion of homeomorphism and prove the pasting lemma.

73.1 Continuous functions.

In this section we define continuous functions and prove that certain conditions are equivalent to a function being continuous.

In standard math we say that a function is continuous with respect to two topologies τ_1, τ_2 if the inverse image of sets from topology τ_2 are in τ_1 . Here we define a predicate that is supposed to reflect that definition, with a difference that we don't require in the definition that τ_1, τ_2 are topologies. This means for example that when we define measurable functions, the definition will be the same.

The notation $f^{-1}(A)$ means the inverse image of (a set) A with respect to (a function) f .

definition

$\text{IsContinuous}(\tau_1, \tau_2, f) \equiv (\forall U \in \tau_2. f^{-1}(U) \in \tau_1)$

The space of continuous functions mapping $X = \bigcup \tau_1$ to $Y = \bigcup \tau_2$ will be denoted $\text{Cont}(\tau_1, \tau_2)$.

definition

$\text{Cont}(\tau_1, \tau_2) \equiv \{f \in (\bigcup \tau_1) \rightarrow (\bigcup \tau_2). \text{IsContinuous}(\tau_1, \tau_2, f)\}$

A trivial example of a continuous function - identity is continuous.

lemma `id_cont`: **shows** $\text{IsContinuous}(\tau, \tau, \text{id}(\bigcup \tau))$
<proof>

Identity is in the space of continuous functions from $\bigcup \tau$ to itself.

lemma `id_cont_sp`: **shows** $\{\langle x, x \rangle. x \in \bigcup \tau\} \in \text{Cont}(\tau, \tau)$
<proof>

A constant function is continuous.

lemma `const_cont`: **assumes** $T \text{ \{is a topology\}}$
shows $\text{IsContinuous}(T, \tau, \text{ConstantFunction}(\bigcup T, c))$
<proof>

If $c \in Y = \bigcup S$, then the constant function defined on $X = \bigcup T$ that is equal to c is in the the space of continuous functions from X to Y .

lemma `const_cont_sp`: **assumes** $T \text{ \{is a topology\}}$ $c \in \bigcup S$
shows $\{\langle x, c \rangle. x \in \bigcup T\} \in \text{Cont}(T, S)$
<proof>

We will work with a pair of topological spaces. The following locale sets up our context that consists of two topologies τ_1, τ_2 and a continuous function $f : X_1 \rightarrow X_2$, where X_i is defined as $\bigcup \tau_i$ for $i = 1, 2$. We also define notation $\text{cl}_1(A)$ and $\text{cl}_2(A)$ for closure of a set A in topologies τ_1 and τ_2 , respectively.

locale `two_top_spaces0` =

```

fixes  $\tau_1$ 
assumes tau1_is_top:  $\tau_1$  {is a topology}

fixes  $\tau_2$ 
assumes tau2_is_top:  $\tau_2$  {is a topology}

fixes  $X_1$ 
defines X1_def [simp]:  $X_1 \equiv \bigcup \tau_1$ 

fixes  $X_2$ 
defines X2_def [simp]:  $X_2 \equiv \bigcup \tau_2$ 

fixes f
assumes fmapAssum:  $f: X_1 \rightarrow X_2$ 

fixes isContinuous ( $\_$  {is continuous} [50] 50)
defines isContinuous_def [simp]:  $g$  {is continuous}  $\equiv$  IsContinuous( $\tau_1, \tau_2, g$ )

fixes  $cl_1$ 
defines cl1_def [simp]:  $cl_1(A) \equiv \text{Closure}(A, \tau_1)$ 

fixes  $cl_2$ 
defines cl2_def [simp]:  $cl_2(A) \equiv \text{Closure}(A, \tau_2)$ 

```

First we show that theorems proven in locale topology0 are valid when applied to topologies τ_1 and τ_2 .

```

lemma (in two_top_spaces0) topol_cntxs_valid:
  shows topology0( $\tau_1$ ) and topology0( $\tau_2$ )
  <proof>

```

For continuous functions the inverse image of a closed set is closed.

```

lemma (in two_top_spaces0) TopZF_2_1_L1:
  assumes A1:  $f$  {is continuous} and A2:  $D$  {is closed in}  $\tau_2$ 
  shows  $f^{-1}(D)$  {is closed in}  $\tau_1$ 
  <proof>

```

If the inverse image of every closed set is closed, then the image of a closure is contained in the closure of the image.

```

lemma (in two_top_spaces0) Top_ZF_2_1_L2:
  assumes A1:  $\forall D. ((D \text{ {is closed in} } \tau_2) \longrightarrow f^{-1}(D) \text{ {is closed in} } \tau_1)$ 
  and A2:  $A \subseteq X_1$ 
  shows  $f(cl_1(A)) \subseteq cl_2(f(A))$ 
  <proof>

```

If $f(\overline{A}) \subseteq \overline{f(A)}$ (the image of the closure is contained in the closure of the image), then $\overline{f^{-1}(B)} \subseteq f^{-1}(\overline{B})$ (the inverse image of the closure contains the closure of the inverse image).

lemma (in two_top_spaces0) Top_ZF_2_1_L3:
 assumes A1: $\forall A. (A \subseteq X_1 \longrightarrow f(\text{cl}_1(A)) \subseteq \text{cl}_2(f(A)))$
 shows $\forall B. (B \subseteq X_2 \longrightarrow \text{cl}_1(f^{-1}(B)) \subseteq f^{-1}(\text{cl}_2(B)))$
<proof>

If $\overline{f^{-1}(B)} \subseteq f^{-1}(\overline{B})$ (the inverse image of a closure contains the closure of the inverse image), then the function is continuous. This lemma closes a series of implications in lemmas Top_ZF_2_1_L1, Top_ZF_2_1_L2 and Top_ZF_2_1_L3 showing equivalence of four definitions of continuity.

lemma (in two_top_spaces0) Top_ZF_2_1_L4:
 assumes A1: $\forall B. (B \subseteq X_2 \longrightarrow \text{cl}_1(f^{-1}(B)) \subseteq f^{-1}(\text{cl}_2(B)))$
 shows f {is continuous}
<proof>

For continuous functions the closure of the inverse image is contained in the inverse image of the closure. This is a shortcut through a series of implications provided by Top_ZF_2_1_L1, Top_ZF_2_1_L2 and Top_ZF_2_1_L3.

corollary (in two_top_spaces0) im_cl_in_cl_im:
 assumes f {is continuous} and $B \subseteq X_2$
 shows $\text{cl}_1(f^{-1}(B)) \subseteq f^{-1}(\text{cl}_2(B))$
<proof>

Another condition for continuity: it is sufficient to check if the inverse image of every set in a base is open.

lemma (in two_top_spaces0) Top_ZF_2_1_L5:
 assumes A1: B {is a base for} τ_2 and A2: $\forall U \in B. f^{-1}(U) \in \tau_1$
 shows f {is continuous}
<proof>

We can strengthen the previous lemma: it is sufficient to check if the inverse image of every set in a subbase is open. The proof is rather awkward, as usual when we deal with general intersections. We have to keep track of the case when the collection is empty.

lemma (in two_top_spaces0) Top_ZF_2_1_L6:
 assumes A1: B {is a subbase for} τ_2 and A2: $\forall U \in B. f^{-1}(U) \in \tau_1$
 shows f {is continuous}
<proof>

A dual of Top_ZF_2_1_L5: a function that maps base sets to open sets is open.

lemma (in two_top_spaces0) base_image_open:
 assumes A1: $\mathcal{B}$ {is a base for} τ_1 and A2: $\forall B \in \mathcal{B}. f(B) \in \tau_2$ and A3:
 $U \in \tau_1$
 shows $f(U) \in \tau_2$
<proof>

A composition of two continuous functions is continuous.

lemma comp_cont: assumes IsContinuous(T,S,f) and IsContinuous(S,R,g)

```

shows IsContinuous(T,R,g 0 f)
<proof>

```

A composition of three continuous functions is continuous.

```

lemma comp_cont3:
  assumes IsContinuous(T,S,f) and IsContinuous(S,R,g) and IsContinuous(R,P,h)
  shows IsContinuous(T,P,h 0 g 0 f)
<proof>

```

The graph of a continuous function into a Hausdorff topological space is closed in the product topology. Recall that in ZF a function is the same as its graph.

```

lemma (in two_top_spaces0) into_T2_graph_closed:
  assumes f {is continuous}  $\tau_2$  {is  $T_2$ }
  shows f {is closed in} ProductTopology( $\tau_1, \tau_2$ )
<proof>

```

73.2 Homeomorphisms

This section studies "homeomorphisms" - continuous bijections whose inverses are also continuous. Notions that are preserved by (commute with) homeomorphisms are called "topological invariants".

Homeomorphism is a bijection that preserves open sets.

```

definition IsAhomeomorphism(T,S,f)  $\equiv$ 
  f  $\in$  bij( $\bigcup T, \bigcup S$ )  $\wedge$  IsContinuous(T,S,f)  $\wedge$  IsContinuous(S,T,converse(f))

```

Inverse (converse) of a homeomorphism is a homeomorphism.

```

lemma homeo_inv: assumes IsAhomeomorphism(T,S,f)
  shows IsAhomeomorphism(S,T,converse(f))
<proof>

```

Homeomorphisms are open maps.

```

lemma homeo_open: assumes IsAhomeomorphism(T,S,f) and U $\in$ T
  shows f(U)  $\in$  S
<proof>

```

A continuous bijection that is an open map is a homeomorphism.

```

lemma bij_cont_open_homeo:
  assumes f  $\in$  bij( $\bigcup T, \bigcup S$ ) and IsContinuous(T,S,f) and  $\forall U \in T. f(U) \in S$ 
  shows IsAhomeomorphism(T,S,f)
<proof>

```

A continuous bijection that maps base to open sets is a homeomorphism.

```

lemma (in two_top_spaces0) bij_base_open_homeo:

```

```

    assumes A1:  $f \in \text{bij}(X_1, X_2)$  and A2:  $\mathcal{B}$  {is a base for}  $\tau_1$  and A3:  $\mathcal{C}$ 
    {is a base for}  $\tau_2$  and
    A4:  $\forall U \in \mathcal{C}. f^{-1}(U) \in \tau_1$  and A5:  $\forall V \in \mathcal{B}. f(V) \in \tau_2$ 
    shows  $\text{IsAhomeomorphism}(\tau_1, \tau_2, f)$ 
    <proof>

```

A bijection that maps base to base is a homeomorphism.

```

lemma (in two_top_spaces0) bij_base_homeo:
    assumes A1:  $f \in \text{bij}(X_1, X_2)$  and A2:  $\mathcal{B}$  {is a base for}  $\tau_1$  and
    A3:  $\{f(B). B \in \mathcal{B}\}$  {is a base for}  $\tau_2$ 
    shows  $\text{IsAhomeomorphism}(\tau_1, \tau_2, f)$ 
    <proof>

```

Interior is a topological invariant.

```

theorem int_top_invariant: assumes A1:  $A \subseteq \bigcup T$  and A2:  $\text{IsAhomeomorphism}(T, S, f)$ 
    shows  $f(\text{Interior}(A, T)) = \text{Interior}(f(A), S)$ 
    <proof>

```

73.3 Topologies induced by mappings

In this section we consider various ways a topology may be defined on a set that is the range (or the domain) of a function whose domain (or range) is a topological space.

A bijection from a topological space induces a topology on the range.

```

theorem bij_induced_top: assumes A1:  $T$  {is a topology} and A2:  $f \in \text{bij}(\bigcup T, Y)$ 
    shows
    { $f(U). U \in T$ } {is a topology} and
    {  $\{f(x). x \in U\}. U \in T$ } {is a topology} and
     $(\bigcup \{f(U). U \in T\}) = Y$  and
     $\text{IsAhomeomorphism}(T, \{f(U). U \in T\}, f)$ 
    <proof>

```

73.4 Partial functions and continuity

Suppose we have two topologies τ_1, τ_2 on sets $X_i = \bigcup \tau_i, i = 1, 2$. Consider some function $f : A \rightarrow X_2$, where $A \subseteq X_1$ (we will call such function "partial"). In such situation we have two natural possibilities for the pairs of topologies with respect to which this function may be continuous. One is obviously the original τ_1, τ_2 and in the second one the first element of the pair is the topology relative to the domain of the function: $\{A \cap U | U \in \tau_1\}$. These two possibilities are not exactly the same and the goal of this section is to explore the differences.

If a function is continuous, then its restriction is continuous in relative topology.

```

lemma (in two_top_spaces0) restr_cont:

```

```

    assumes A1:  $A \subseteq X_1$  and A2:  $f$  {is continuous}
    shows IsContinuous( $\tau_1$  {restricted to}  $A$ ,  $\tau_2$ , restrict( $f, A$ ))
  <proof>

```

If a function is continuous, then it is continuous when we restrict the topology on the range to the image of the domain.

```

lemma (in two_top_spaces0) restr_image_cont:
  assumes A1:  $f$  {is continuous}
  shows IsContinuous( $\tau_1$ ,  $\tau_2$  {restricted to}  $f(X_1)$ ,  $f$ )
  <proof>

```

A combination of restr_cont and restr_image_cont.

```

lemma (in two_top_spaces0) restr_restr_image_cont:
  assumes A1:  $A \subseteq X_1$  and A2:  $f$  {is continuous} and
  A3:  $g = \text{restrict}(f, A)$  and
  A4:  $\tau_3 = \tau_1$  {restricted to}  $A$ 
  shows IsContinuous( $\tau_3$ ,  $\tau_2$  {restricted to}  $g(A)$ ,  $g$ )
  <proof>

```

We need a context similar to two_top_spaces0 but without the global function $f : X_1 \rightarrow X_2$.

```

locale two_top_spaces1 =

```

```

  fixes  $\tau_1$ 
  assumes tau1_is_top:  $\tau_1$  {is a topology}

```

```

  fixes  $\tau_2$ 
  assumes tau2_is_top:  $\tau_2$  {is a topology}

```

```

  fixes  $X_1$ 
  defines X1_def [simp]:  $X_1 \equiv \bigcup \tau_1$ 

```

```

  fixes  $X_2$ 
  defines X2_def [simp]:  $X_2 \equiv \bigcup \tau_2$ 

```

If a partial function $g : X_1 \supseteq A \rightarrow X_2$ is continuous with respect to (τ_1, τ_2) , then A is open (in τ_1) and the function is continuous in the relative topology.

```

lemma (in two_top_spaces1) partial_fun_cont:
  assumes A1:  $g : A \rightarrow X_2$  and A2: IsContinuous( $\tau_1, \tau_2, g$ )
  shows  $A \in \tau_1$  and IsContinuous( $\tau_1$  {restricted to}  $A$ ,  $\tau_2$ ,  $g$ )
  <proof>

```

For partial function defined on open sets continuity in the whole and relative topologies are the same.

```

lemma (in two_top_spaces1) part_fun_on_open_cont:
  assumes A1:  $g : A \rightarrow X_2$  and A2:  $A \in \tau_1$ 
  shows IsContinuous( $\tau_1, \tau_2, g$ )  $\longleftrightarrow$ 
    IsContinuous( $\tau_1$  {restricted to}  $A$ ,  $\tau_2$ ,  $g$ )
  <proof>

```

73.5 Product topology and continuity

We start with three topological spaces (τ_1, X_1) , (τ_2, X_2) and (τ_3, X_3) and a function $f : X_1 \times X_2 \rightarrow X_3$. We will study the properties of f with respect to the product topology $\tau_1 \times \tau_2$ and τ_3 . This situation is similar as in locale `two_top_spaces0` but the first topological space is assumed to be a product of two topological spaces.

First we define a locale with three topological spaces.

```

locale prod_top_spaces0 =

  fixes  $\tau_1$ 
  assumes tau1_is_top:  $\tau_1$  {is a topology}

  fixes  $\tau_2$ 
  assumes tau2_is_top:  $\tau_2$  {is a topology}

  fixes  $\tau_3$ 
  assumes tau3_is_top:  $\tau_3$  {is a topology}

  fixes  $X_1$ 
  defines X1_def [simp]:  $X_1 \equiv \bigcup_{\tau_1}$ 

  fixes  $X_2$ 
  defines X2_def [simp]:  $X_2 \equiv \bigcup_{\tau_2}$ 

  fixes  $X_3$ 
  defines X3_def [simp]:  $X_3 \equiv \bigcup_{\tau_3}$ 

  fixes  $\eta$ 
  defines eta_def [simp]:  $\eta \equiv \text{ProductTopology}(\tau_1, \tau_2)$ 

```

Fixing the first variable in a two-variable continuous function results in a continuous function.

```

lemma (in prod_top_spaces0) fix_1st_var_cont:
  assumes f:  $X_1 \times X_2 \rightarrow X_3$  and IsContinuous( $\eta, \tau_3, f$ )
  and x $\in X_1$ 
  shows IsContinuous( $\tau_2, \tau_3, \text{Fix1stVar}(f, x)$ )
  <proof>

```

Fixing the second variable in a two-variable continuous function results in a continuous function.

```

lemma (in prod_top_spaces0) fix_2nd_var_cont:
  assumes f:  $X_1 \times X_2 \rightarrow X_3$  and IsContinuous( $\eta, \tau_3, f$ )
  and y $\in X_2$ 
  shows IsContinuous( $\tau_1, \tau_3, \text{Fix2ndVar}(f, y)$ )
  <proof>

```


Having two continuous mappings we can construct a third one on the cartesian product of the domains.

```

lemma cart_prod_cont:
  assumes A1:  $\tau_1$  {is a topology}  $\tau_2$  {is a topology} and
    A2:  $\eta_1$  {is a topology}  $\eta_2$  {is a topology} and
    A3a:  $f_1: \bigcup \tau_1 \rightarrow \bigcup \eta_1$  and A3b:  $f_2: \bigcup \tau_2 \rightarrow \bigcup \eta_2$  and
    A4:  $\text{IsContinuous}(\tau_1, \eta_1, f_1)$   $\text{IsContinuous}(\tau_2, \eta_2, f_2)$  and
    A5:  $g = \{ \langle p, \langle f_1(\text{fst}(p)), f_2(\text{snd}(p)) \rangle \rangle . p \in \bigcup \tau_1 \times \bigcup \tau_2 \}$ 
  shows  $\text{IsContinuous}(\text{ProductTopology}(\tau_1, \tau_2), \text{ProductTopology}(\eta_1, \eta_2), g)$ 
<proof>

```

A reformulation of the `cart_prod_cont` lemma above in slightly different notation.

```

theorem (in two_top_spaces0) product_cont_functions:
  assumes  $f: X_1 \rightarrow X_2$   $g: \bigcup \tau_3 \rightarrow \bigcup \tau_4$ 
     $\text{IsContinuous}(\tau_1, \tau_2, f)$   $\text{IsContinuous}(\tau_3, \tau_4, g)$ 
     $\tau_4$  {is a topology}  $\tau_3$  {is a topology}
  shows  $\text{IsContinuous}(\text{ProductTopology}(\tau_1, \tau_3), \text{ProductTopology}(\tau_2, \tau_4), \{ \langle \langle x, y \rangle, \langle fx, gy \rangle \rangle .$ 
 $\langle x, y \rangle \in X_1 \times \bigcup \tau_3 \})$ 
<proof>

```

A special case of `cart_prod_cont` when the function acting on the second axis is the identity.

```

lemma cart_prod_cont1:
  assumes A1:  $\tau_1$  {is a topology} and A1a:  $\tau_2$  {is a topology} and
    A2:  $\eta_1$  {is a topology} and
    A3:  $f_1: \bigcup \tau_1 \rightarrow \bigcup \eta_1$  and A4:  $\text{IsContinuous}(\tau_1, \eta_1, f_1)$  and
    A5:  $g = \{ \langle p, \langle f_1(\text{fst}(p)), \text{snd}(p) \rangle \rangle . p \in \bigcup \tau_1 \times \bigcup \tau_2 \}$ 
  shows  $\text{IsContinuous}(\text{ProductTopology}(\tau_1, \tau_2), \text{ProductTopology}(\eta_1, \tau_2), g)$ 
<proof>

```

Having two continuous mappings f, g we can construct a third one with values in the cartesian product of the codomains of f, g , defined by $x \mapsto \langle f(x), g(x) \rangle$.

```

lemma (in prod_top_spaces0) cont_funcs_prod:
  assumes  $f: X_1 \rightarrow X_2$   $g: X_1 \rightarrow X_3$   $\text{IsContinuous}(\tau_1, \tau_2, f)$   $\text{IsContinuous}(\tau_1, \tau_3, g)$ 
  defines  $h \equiv \{ \langle x, \langle f(x), g(x) \rangle \rangle . x \in X_1 \}$ 
  shows  $\text{IsContinuous}(\tau_1, \text{ProductTopology}(\tau_2, \tau_3), h)$ 
<proof>

```

Having two continuous mappings f, g we can construct a third one with values in the cartesian product of the codomains of f, g , defined by $x \mapsto \langle f(x), g(x) \rangle$. This is essentially the same as `cont_funcs_prod` but formulated in a way that is sometimes easier to apply. Recall that $\tau_2 \times_t \tau_3$ is a notation for the product topology of τ_1 and τ_2 .

```

lemma cont_funcs_prod1:
  assumes  $\tau_1$  {is a topology}  $\tau_2$  {is a topology}  $\tau_3$  {is a topology} and

```

$\{\langle x, p(x) \rangle. x \in \bigcup \tau_1\} \in \text{Cont}(\tau_1, \tau_2) \quad \{\langle x, q(x) \rangle. x \in \bigcup \tau_1\} \in \text{Cont}(\tau_1, \tau_3)$
shows $\{\langle x, \langle p(x), q(x) \rangle \rangle. x \in \bigcup \tau_1\} \in \text{Cont}(\tau_1, \tau_2 \times_t \tau_3)$
<proof>

Two continuous functions into a Hausdorff space are equal on a closed set. Note that in the lemma below f is assumed to map X_1 to X_2 in the locale, we only need to add a similar assumption for the second function.

lemma (in two_top_spaces0) two_fun_eq_closed:
assumes $g: X_1 \rightarrow X_2$ f {is continuous} g {is continuous} τ_2 {is T_2 }
shows $\{x \in X_1. f(x) = g(x)\}$ {is closed in} τ_1
<proof>

Closure of an image of a singleton by a relation in $X \times Y$ is contained in the image of this singleton by the closure of the relation (in the product topology). Compare the proof of Metamath's theorem with the same name.

lemma imasncls:
assumes T {is a topology} S {is a topology} $R \subseteq (\bigcup T) \times (\bigcup S)$ $x \in \bigcup T$
shows $\text{Closure}(R\{x\}, S) \subseteq \text{Closure}(R, T \times_t S)\{x\}$
<proof>

73.6 Pasting lemma

The classical pasting lemma states that if U_1, U_2 are both open (or closed) and a function is continuous when restricted to both U_1 and U_2 then it is continuous when restricted to $U_1 \cup U_2$. In this section we prove a generalization statement stating that the set $\{U \in \tau_1 | f|_U \text{ is continuous}\}$ is a topology.

A typical statement of the pasting lemma uses the notion of a function restricted to a set being continuous without specifying the topologies with respect to which this continuity holds. In two_top_spaces0 context the notation g {is continuous} means continuity with respect to topologies τ_1, τ_2 . The next lemma is a special case of partial_fun_cont and states that if for some set $A \subseteq X_1 = \bigcup \tau_1$ the function $f|_A$ is continuous (with respect to (τ_1, τ_2)), then A has to be open. This clears up terminology and indicates why we need to pay attention to the issue of which topologies we talk about when we say that the restricted (to some closed set for example) function is continuous.

lemma (in two_top_spaces0) restriction_continuous1:
assumes $A1: A \subseteq X_1$ and $A2: \text{restrict}(f, A)$ {is continuous}
shows $A \in \tau_1$
<proof>

If a function is continuous on each set of a collection of open sets, then it is continuous on the union of them. We could use continuity with respect to the relative topology here, but we know that on open sets this is the same as the original topology.

```

lemma (in two_top_spaces0) pasting_lemma1:
  assumes A1:  $M \subseteq \tau_1$  and A2:  $\forall U \in M. \text{restrict}(f, U) \text{ \{is continuous\}}$ 
  shows  $\text{restrict}(f, \bigcup M) \text{ \{is continuous\}}$ 
<proof>

```

If a function is continuous on two sets, then it is continuous on intersection.

```

lemma (in two_top_spaces0) cont_inter_cont:
  assumes A1:  $A \subseteq X_1$   $B \subseteq X_1$  and
  A2:  $\text{restrict}(f, A) \text{ \{is continuous\}}$   $\text{restrict}(f, B) \text{ \{is continuous\}}$ 
  shows  $\text{restrict}(f, A \cap B) \text{ \{is continuous\}}$ 
<proof>

```

The collection of open sets U such that f restricted to U is continuous, is a topology.

```

theorem (in two_top_spaces0) pasting_theorem:
  shows  $\{U \in \tau_1. \text{restrict}(f, U) \text{ \{is continuous\}}\} \text{ \{is a topology\}}$ 
<proof>

```

0 is continuous.

```

corollary (in two_top_spaces0) zero_continuous: shows 0 \{is continuous\}
<proof>

```

end

74 Topology 4

```

theory Topology_ZF_4 imports Topology_ZF_1 Order_ZF func1 NatOrder_ZF
begin

```

This theory deals with convergence in topological spaces. Contributed by Daniel de la Concepcion.

74.1 Nets

Nets are a generalization of sequences. It is known that sequences do not determine the behavior of the topological spaces that are not first countable; i.e., have a countable neighborhood base for each point. To solve this problem, nets were defined so that the behavior of any topological space can be thought in terms of convergence of nets.

We say that a relation r *directs* a set X if the relation is reflexive, transitive on X and for every two elements x, y of X there is some element z such that both x and y are in the relation with z . Note that this naming is a bit inconsistent with what is defined in `Order_ZF` where we define what it means that r *up-directs* X (the third condition in the definition below) or

r down-directs X . This naming inconsistency will be fixed in the future (maybe).

definition

```
IsDirectedSet (_ directs _ 90)
  where r directs X  $\equiv$  refl(X,r)  $\wedge$  trans(r)  $\wedge$  ( $\forall x \in X. \forall y \in X. \exists z \in X. \langle x, z \rangle \in r$ 
 $\wedge \langle y, z \rangle \in r$ )
```

Any linear order is a directed set; in particular $(\mathbb{N}, \leq)$.

lemma linorder_imp_directed:

```
  assumes IsLinOrder(X,r)
  shows r directs X
<proof>
```

Natural numbers are a directed set.

corollary Le_directs_nat:

```
  shows IsLinOrder(nat,Le) Le directs nat
<proof>
```

We are able to define the concept of net, now that we now what a directed set is.

definition

```
IsNet (_ {is a net on} _ 90)
  where N {is a net on} X  $\equiv$  fst(N):domain(fst(N)) $\rightarrow$ X  $\wedge$  (snd(N) directs
domain(fst(N)))  $\wedge$  domain(fst(N)) $\neq \emptyset$ 
```

Provided a topology and a net directed on its underlying set, we can talk about convergence of the net in the topology.

definition (in topology0)

```
NetConverges (_  $\rightarrow_N$  _ 90)
  where N {is a net on}  $\bigcup T \implies N \rightarrow_N x \equiv$ 
  ( $x \in \bigcup T$ )  $\wedge$  ( $\forall U \in \text{Pow}(\bigcup T). (x \in \text{int}(U) \longrightarrow (\exists t \in \text{domain}(\text{fst}(N)). \forall m \in \text{domain}(\text{fst}(N)).$ 
  ( $\langle t, m \rangle \in \text{snd}(N) \longrightarrow \text{fst}(N)m \in U$ ))))
```

One of the most important directed sets, is the neighborhoods of a point.

theorem (in topology0) directedset_neighborhoods:

```
  assumes  $x \in \bigcup T$ 
  defines Neigh  $\equiv \{U \in \text{Pow}(\bigcup T). x \in \text{int}(U)\}$ 
  defines  $r \equiv \{\langle U, V \rangle \in (\text{Neigh} \times \text{Neigh}). V \subseteq U\}$ 
  shows r directs Neigh
<proof>
```

There can be nets directed by the neighborhoods that converge to the point; if there is a choice function.

theorem (in topology0) net_direct_neigh_converg:

```
  assumes  $x \in \bigcup T$ 
  defines Neigh  $\equiv \{U \in \text{Pow}(\bigcup T). x \in \text{int}(U)\}$ 
```

```

defines r $\equiv$ { $\langle U, V \rangle \in (\text{Neigh} \times \text{Neigh})$ .  $V \subseteq U$ }
assumes f: $\text{Neigh} \rightarrow \bigcup T$   $\forall U \in \text{Neigh}$ .  $f(U) \in U$ 
shows  $\langle f, r \rangle \rightarrow_N x$ 
 $\langle \text{proof} \rangle$ 

```

74.2 Filters

Nets are a generalization of sequences that can make us see that not all topological spaces can be described by sequences. Nevertheless, nets are not always the tool used to deal with convergence. The reason is that they make use of directed sets which are completely unrelated with the topology.

The topological tools to deal with convergence are what is called filters.

definition

```

IsFilter ( _ {is a filter on} _ 90)
where  $\mathcal{F}$  {is a filter on}  $X \equiv (\emptyset \notin \mathcal{F}) \wedge (X \in \mathcal{F}) \wedge \mathcal{F} \subseteq \text{Pow}(X) \wedge$ 
 $(\forall A \in \mathcal{F}. \forall B \in \mathcal{F}. A \cap B \in \mathcal{F}) \wedge (\forall B \in \mathcal{F}. \forall C \in \text{Pow}(X). B \subseteq C \longrightarrow C \in \mathcal{F})$ 

```

The next lemma splits the the definition of a filter into four conditions to make it easier to reference each one separately in proofs.

```

lemma is_filter_def_split: assumes  $\mathcal{F}$  {is a filter on}  $X$ 
shows  $\emptyset \notin \mathcal{F}$   $X \in \mathcal{F}$   $\mathcal{F} \subseteq \text{Pow}(X)$ 
 $\forall A \in \mathcal{F}. \forall B \in \mathcal{F}. A \cap B \in \mathcal{F}$  and  $\forall B \in \mathcal{F}. \forall C \in \text{Pow}(X). B \subseteq C \longrightarrow C \in \mathcal{F}$ 
 $\langle \text{proof} \rangle$ 

```

Filters are closed with respect to taking finite intersections.

```

lemma filter_fin_inter_closed:
assumes  $\mathcal{F}$  {is a filter on}  $X$   $M \in \text{FinPow}(\mathcal{F}) \setminus \{\emptyset\}$ 
shows  $\bigcap M \in \mathcal{F}$ 
 $\langle \text{proof} \rangle$ 

```

Filters are closed with respect to taking supersets.

```

lemma filter_superset_closed:
assumes  $\mathcal{F}$  {is a filter on}  $X$   $\mathcal{A} \subseteq \mathcal{F}$ 
shows  $\text{Supersets}(X, \mathcal{A}) \subseteq \mathcal{F}$ 
 $\langle \text{proof} \rangle$ 

```

Not all the sets of a filter are needed to be consider at all times; as it happens with a topology we can consider bases.

definition

```

IsBaseFilter ( _ {is a base filter} _ 90)
where  $\mathcal{C}$  {is a base filter}  $\mathcal{F} \equiv \mathcal{C} \subseteq \mathcal{F} \wedge \mathcal{F} = \{A \in \text{Pow}(\bigcup \mathcal{F}). (\exists D \in \mathcal{C}. D \subseteq A)\}$ 

```

Not every set is a base for a filter, as it happens with topologies, there is a condition to be satisfied.

definition

```

SatisfiesFilterBase ( _ {satisfies the filter base condition} 90)

```

where C {satisfies the filter base condition} $\equiv (\forall A \in C. \forall B \in C. \exists D \in C. D \subseteq A \cap B) \wedge C \neq \emptyset \wedge \emptyset \notin C$

Every set of a filter contains a set from the filter's base.

lemma basic_element_filter:
assumes $A \in \mathcal{F}$ **and** C {is a base filter} $\mathcal{F}$
shows $\exists D \in C. D \subseteq A$
<proof>

The following two results state that the filter base condition is necessary and sufficient for the filter generated by a base, to be an actual filter. The third result, rewrites the previous two.

theorem basic_filter_1:
assumes C {is a base filter} $\mathcal{F}$ **and** C {satisfies the filter base condition}
shows $\mathcal{F}$ {is a filter on} $\bigcup \mathcal{F}$
<proof>

A base filter satisfies the filter base condition.

theorem basic_filter_2:
assumes C {is a base filter} $\mathcal{F}$ **and** $\mathcal{F}$ {is a filter on} $\bigcup \mathcal{F}$
shows C {satisfies the filter base condition}
<proof>

A base filter for a collection satisfies the filter base condition iff that collection is in fact a filter.

theorem basic_filter:
assumes C {is a base filter} $\mathcal{F}$
shows $(C$ {satisfies the filter base condition}) $\longleftrightarrow (\mathcal{F}$ {is a filter on} $\bigcup \mathcal{F})$
<proof>

A base for a filter determines a filter up to the underlying set.

theorem base_unique_filter:
assumes C {is a base filter} $\mathcal{F}_1$ **and** C {is a base filter} $\mathcal{F}_2$
shows $\mathcal{F}_1 = \mathcal{F}_2 \longleftrightarrow \bigcup \mathcal{F}_1 = \bigcup \mathcal{F}_2$
<proof>

Suppose that we take any nonempty collection C of subsets of some set X . Then this collection is a base filter for the collection of all supersets (in X) of sets from C .

theorem base_unique_filter_set1:
assumes $C \subseteq \text{Pow}(X)$ **and** $C \neq \emptyset$
shows C {is a base filter} $\{A \in \text{Pow}(X). \exists D \in C. D \subseteq A\}$ **and** $\bigcup \{A \in \text{Pow}(X). \exists D \in C. D \subseteq A\} = X$
<proof>

A collection C that satisfies the filter base condition is a base filter for some other collection $\mathcal{F}$ iff $\mathcal{F}$ is the collection of supersets of C .

theorem base_unique_filter_set2:
 assumes $C \subseteq \text{Pow}(X)$ and C {satisfies the filter base condition}
 shows $((C \text{ {is a base filter}} \mathfrak{F}) \wedge \bigcup \mathfrak{F} = X) \longleftrightarrow \mathfrak{F} = \{A \in \text{Pow}(X). \exists D \in C. D \subseteq A\}$
<proof>

A simple corollary from the previous lemma.

corollary base_unique_filter_set3:
 assumes $C \subseteq \text{Pow}(X)$ and C {satisfies the filter base condition}
 shows $C \text{ {is a base filter}} \{A \in \text{Pow}(X). \exists D \in C. D \subseteq A\}$ and $\bigcup \{A \in \text{Pow}(X). \exists D \in C. D \subseteq A\} = X$
<proof>

Every filter can be expanded by a set

lemma extend_filter:
 assumes $A \in \text{Pow}(X)$ $\mathfrak{F}$ {is a filter on} X $A \neq \emptyset$ $A \notin \mathfrak{F}$ $X \setminus A \notin \mathfrak{F}$
 shows $\exists \mathfrak{G}. (\mathfrak{G} \text{ {is a filter on}} X) \wedge A \in \mathfrak{G} \wedge \mathfrak{F} \subseteq \mathfrak{G}$
<proof>

The convergence for filters is much easier concept to write. Given a topology and a filter on the same underlying set, we can define convergence as containing all the neighborhoods of the point.

definition (in topology0)
 FilterConverges $(_ \rightarrow_F _ 50)$ where
 $\mathfrak{F} \text{ {is a filter on}} \bigcup T \implies \mathfrak{F} \rightarrow_F x \equiv$
 $x \in \bigcup T \wedge (\{U \in \text{Pow}(\bigcup T). x \in \text{int}(U)\} \subseteq \mathfrak{F})$

The neighborhoods of a point form a filter that converges to that point.

lemma (in topology0) neigh_filter:
 assumes $x \in \bigcup T$
 defines $\text{Neigh} \equiv \{U \in \text{Pow}(\bigcup T). x \in \text{int}(U)\}$
 shows $\text{Neigh} \text{ {is a filter on}} \bigcup T$ and $\text{Neigh} \rightarrow_F x$
<proof>

Note that with the net we built in a previous result, it wasn't clear that we could construct an actual net that converged to the given point without the axiom of choice. With filters, there is no problem.

Another positive point of filters is due to the existence of filter basis. If we have a basis for a filter, then the filter converges to a point iff every neighborhood of that point contains a basic filter element.

theorem (in topology0) convergence_filter_base1:
 assumes $\mathfrak{F}$ {is a filter on} $\bigcup T$ and C {is a base filter} $\mathfrak{F}$ and $\mathfrak{F} \rightarrow_F x$
 shows $\forall U \in \text{Pow}(\bigcup T). x \in \text{int}(U) \longrightarrow (\exists D \in C. D \subseteq U)$ and $x \in \bigcup T$
<proof>

A sufficient condition for a filter to converge to a point.

theorem (in topology0) convergence_filter_base2:

```

    assumes  $\mathcal{F}$  {is a filter on}  $\bigcup T$  and  $C$  {is a base filter}  $\mathcal{F}$ 
    and  $\forall U \in \text{Pow}(\bigcup T). x \in \text{int}(U) \longrightarrow (\exists D \in C. D \subseteq U)$  and  $x \in \bigcup T$ 
    shows  $\mathcal{F} \rightarrow_F x$ 
  <proof>

```

A necessary and sufficient condition for a filter to converge to a point.

```

theorem (in topology0) convergence_filter_base_eq:
  assumes  $\mathcal{F}$  {is a filter on}  $\bigcup T$  and  $C$  {is a base filter}  $\mathcal{F}$ 
  shows  $(\mathcal{F} \rightarrow_F x) \longleftrightarrow ((\forall U \in \text{Pow}(\bigcup T). x \in \text{int}(U) \longrightarrow (\exists D \in C. D \subseteq U)) \wedge x \in \bigcup T)$ 
  <proof>

```

end

75 Topology and neighborhoods

```

theory Topology_ZF_4a imports Topology_ZF_4
begin

```

This theory considers the relations between topology and systems of neighborhood filters.

75.1 Neighborhood systems

The standard way of defining a topological space is by specifying a collection of sets that we consider "open" (see the `Topology_ZF` theory). An alternative of this approach is to define a collection of neighborhoods for each point of the space.

We define a neighborhood system as a function that takes each point $x \in X$ and assigns it a collection of subsets of X which is called the neighborhoods of x . The neighborhoods of a point x form a filter that satisfies an additional axiom that for every neighborhood N of x we can find another one U such that N is a neighborhood of every point of U .

definition

```

  IsNeighSystem ( _ {is a neighborhood system on} _ 90)
  where  $\mathcal{M}$  {is a neighborhood system on}  $X \equiv (\mathcal{M} : X \rightarrow \text{Pow}(\text{Pow}(X))) \wedge$ 
     $(\forall x \in X. (\mathcal{M}(x) \text{ {is a filter on} } X) \wedge (\forall N \in \mathcal{M}(x). x \in N \wedge (\exists U \in \mathcal{M}(x). \forall y \in U. (N \in \mathcal{M}(y))$ 
  ) ) )

```

A neighborhood system on X consists of collections of subsets of X .

lemma neighborhood_subset:

```

  assumes  $\mathcal{M}$  {is a neighborhood system on}  $X$  and  $x \in X$  and  $N \in \mathcal{M}(x)$ 
  shows  $N \subseteq X$  and  $x \in N$ 
  <proof>

```

Some sources (like Wikipedia) use a bit different definition of neighborhood systems where the U is required to be contained in N . The next lemma shows that this stronger version can be recovered from our definition.


```

lemma neigh_def_stronger:
  assumes  $\mathcal{M}$  {is a neighborhood system on}  $X$  and  $x \in X$  and  $N \in \mathcal{M}(x)$ 
  shows  $\exists U \in \mathcal{M}(x). U \subseteq N \wedge (\forall y \in U. (N \in \mathcal{M}(y)))$ 
<proof>

```

75.2 From a neighborhood system to topology

Given a neighborhood system $\{\mathcal{M}_x\}_{x \in X}$ we can define a topology on X . Namely, we consider a subset of X open if $U \in \mathcal{M}_x$ for every element x of U .

The collection of sets defined as above is indeed a topology.

```

theorem topology_from_neighs:
  assumes  $\mathcal{M}$  {is a neighborhood system on}  $X$ 
  defines Tdef:  $T \equiv \{U \in \text{Pow}(X). \forall x \in U. U \in \mathcal{M}(x)\}$ 
  shows  $T$  {is a topology} and  $\bigcup T = X$ 
<proof>

```

Some sources (like Wikipedia) define the open sets generated by a neighborhood system "as those sets containing a neighborhood of each of their points". The next lemma shows that this definition is equivalent to the one we are using.

```

lemma topology_from_neighs1:
  assumes  $\mathcal{M}$  {is a neighborhood system on}  $X$ 
  shows  $\{U \in \text{Pow}(X). \forall x \in U. U \in \mathcal{M}(x)\} = \{U \in \text{Pow}(X). \forall x \in U. \exists V \in \mathcal{M}(x). V \subseteq U\}$ 
<proof>

```

75.3 From a topology to a neighborhood system

Once we have a topology T we can define a natural neighborhood system on $X = \bigcup T$. In this section we define such neighborhood system and prove its basic properties.

For a topology T we define a neighborhood system of T as a function that takes an $x \in X = \bigcup T$ and assigns it the collection of supersets of open sets containing x . We call that the "neighborhood system of T "

```

definition
  NeighSystem ({neighborhood system of} _ 91)
  where {neighborhood system of}  $T \equiv \{ \langle x, \{N \in \text{Pow}(\bigcup T). \exists U \in T. (x \in U \wedge U \subseteq N)\} \rangle. x \in \bigcup T \}$ 

```

The way we defined the neighborhood system of T means that it is a function on $\bigcup T$.

```

lemma neigh_fun: shows ({neighborhood system of}  $T$ ):  $\bigcup T \rightarrow \text{Pow}(\text{Pow}(\bigcup T))$ 
<proof>

```

The value of the neighborhood system of T at $x \in \bigcup T$ is the collection of supersets of open sets containing x .

lemma *neigh_val*: **assumes** $x \in \bigcup T$
shows $(\{\text{neighborhood system of}\} T)(x) = \{N \in \text{Pow}(\bigcup T). \exists U \in T. (x \in U \wedge U \subseteq N)\}$
<proof>

The next lemma shows that open sets are members of (what we will prove later to be) the natural neighborhood system on $X = \bigcup T$.

lemma *open_are_neighs*:
assumes $U \in T$ $x \in U$
shows $x \in \bigcup T$ **and** $U \in \{V \in \text{Pow}(\bigcup T). \exists U \in T. (x \in U \wedge U \subseteq V)\}$
<proof>

Another fact we will need is that for every $x \in X = \bigcup T$ the neighborhoods of x form a filter

lemma *neighs_is_filter*:
assumes T {is a topology} **and** $x \in \bigcup T$
defines $M \text{def: } \mathcal{M} \equiv \{\text{neighborhood system of}\} T$
shows $\mathcal{M}(x)$ {is a filter on} $(\bigcup T)$
<proof>

The next theorem states that the natural neighborhood system on $X = \bigcup T$ indeed is a neighborhood system.

theorem *neigh_from_topology*:
assumes T {is a topology}
shows $(\{\text{neighborhood system of}\} T)$ {is a neighborhood system on} $(\bigcup T)$
<proof>

Any neighborhood of an element of the closure of a subset intersects the subset.

lemma *neigh_inter_nempty*:
assumes T {is a topology} $A \subseteq \bigcup T$ $x \in \text{Closure}(A, T)$ **and**
 $N \in (\{\text{neighborhood system of}\} T)(x)$
shows $N \cap A \neq \emptyset$
<proof>

75.4 Neighborhood systems are 1:1 with topologies

We can create a topology from a neighborhood system and neighborhood system from topology. The question is then if we start from a neighborhood system, create a topology from it then create the topology's natural neighborhood system, do we get back the neighborhood system we started from? Similarly, if we start from a topology, create its neighborhood system and then create a topology from that, do we get the original topology? This section provides the affirmative answer (for now only for the first question). This means that there is a one-to-one correspondence between the set of topologies on a set and the set of abstract neighborhood systems on the set.

Each abstract neighborhood of x contains an open neighborhood of x .

lemma open_nei_in_nei:
 assumes $\mathcal{M}$ {is a neighborhood system on} X $x \in X$ $N \in \mathcal{M}(x)$
 defines Tdef: $T \equiv \{U \in \text{Pow}(X). \forall x \in U. U \in \mathcal{M}(x)\}$
 shows $N \in \text{Pow}(X)$ and $\exists U \in T. (x \in U \wedge U \subseteq N)$
<proof>

In the the next theorem we show that if we start from a neighborhood system, create a topology from it, then create it's natural neighborhood system, we get back the original neighborhood system.

theorem nei_top_nei_round_trip:
 assumes $\mathcal{M}$ {is a neighborhood system on} X
 defines Tdef: $T \equiv \{U \in \text{Pow}(X). \forall x \in U. U \in \mathcal{M}(x)\}$
 shows $(\{\text{neighborhood system of}\} T) = \mathcal{M}$
<proof>

75.5 Set neighborhoods

Some sources (like Metamath) take a somewhat different approach where instead of defining the collection of neighborhoods of a point $x \in X$ they define a collection of neighborhoods of a subset of X (where X is the carrier of a topology T (i.e. $X = \bigcup T$). In this approach a neighborhood system is a function whose domain is the powerset of X , i.e. the set of subsets of X . The two approaches are equivalent in a sense as having a neighborhood system we can create a set neighborhood system and vice versa.

We define a set neighborhood system as a function that takes a subset A of the carrier of the topology and assigns it the collection of supersets of all open sets that contain A .

definition
 SetNeighSystem ({set neighborhood system of} _ 91)
 where {set neighborhood system of} T
 $\equiv \{ \langle A, \{N \in \text{Pow}(\bigcup T). \exists U \in T. (A \subseteq U \wedge U \subseteq N)\} \rangle. A \in \text{Pow}(\bigcup T) \}$

Given a set neighborhood system we can recover the (standard) neighborhood system by taking the values of the set neighborhood system at singletons $\{x\}$ where $x \in X = \bigcup T$.

lemma neigh_from_nei: assumes $x \in \bigcup T$
 shows $(\{\text{neighborhood system of}\} T)(x) = (\{\text{set neighborhood system of}\} T)\{x\}$
<proof>

The set neighborhood system of T is a function mapping subsets of $\bigcup T$ to collections of subsets of $\bigcup T$.

lemma nei_fun:
 shows $(\{\text{set neighborhood system of}\} T) : \text{Pow}(\bigcup T) \rightarrow \text{Pow}(\text{Pow}(\bigcup T))$

<proof>

The value of the set neighborhood system of T at subset A of $\bigcup T$ is the collection of subsets N of $\bigcup T$ for which exists an open subset $U \subseteq N$ that contains A .

lemma *nei_val*: **assumes** $A \subseteq \bigcup T$
shows
 $(\{\text{set neighborhood system of}\} T)(A) = \{N \in \text{Pow}(\bigcup T). \exists U \in T. (A \subseteq U \wedge U \subseteq N)\}$
<proof>

A member of the value of the set neighborhood system of T at A is a subset of $\bigcup T$. The interesting part is that we can show it without any assumption on A .

lemma *nei_val_subset*:
assumes $N \in (\{\text{set neighborhood system of}\} T)(A)$
shows $A \subseteq \bigcup T$ **and** $N \subseteq \bigcup T$
<proof>

If T is a topology, then every subset of its carrier (i.e. $\bigcup T$) is a (set) neighborhood of the empty set.

lemma *nei_empty*: **assumes** T {is a topology} $N \subseteq \bigcup T$
shows $N \in (\{\text{set neighborhood system of}\} T)(\emptyset)$
<proof>

If T is a topology, then the (set) neighborhoods of a nonempty subset of $\bigcup T$ form a filter on $X = \bigcup T$.

theorem *nei_filter*: **assumes** T {is a topology} $D \subseteq (\bigcup T)$ $D \neq \emptyset$
shows $(\{\text{set neighborhood system of}\} T)(D)$ {is a filter on} $(\bigcup T)$
<proof>

If N is a (set) neighborhood of A in T , then exist an open set U such that N contains U which contains A . This is similar to the Metamath's theorem with the same name, except that here we do not need assume that T is a topology (which is a bit worrying).

lemma *neii2*: **assumes** $N \in (\{\text{set neighborhood system of}\} T)(A)$
shows $\exists U \in T. (A \subseteq U \wedge U \subseteq N)$
<proof>

An open set U covering a set A is a set neighborhood of A .

lemma *open_superset_nei*: **assumes** $V \in T$ $A \subseteq V$
shows $V \in (\{\text{set neighborhood system of}\} T)(A)$
<proof>

An open set is a set neighborhood of itself.

corollary *open_is_nei*: **assumes** $V \in T$
shows $V \in (\{\text{set neighborhood system of}\} T)(V)$

<proof>

An open neighborhood of x is a set neighborhood of $\{x\}$.

corollary `open_nei_singl`: **assumes** $V \in T \ x \in V$
shows $V \in (\{\text{set neighborhood system of } T\} \{x\})$
<proof>

The Cartesian product of two neighborhoods is a neighborhood in the product topology. Similar to the Metamath's theorem with the same name.

lemma `neitx`:
assumes $T \{\text{is a topology}\} S \{\text{is a topology}\}$ **and**
 $A \in (\{\text{set neighborhood system of } T\} (C))$ **and**
 $B \in (\{\text{set neighborhood system of } S\} (D))$
shows $A \times B \in (\{\text{set neighborhood system of } (T \times_t S)\} (C \times D))$
<proof>

Any neighborhood of an element of the closure of a subset intersects the subset. This is practically the same as `neigh_inter_empty`, just formulated in terms of set neighborhoods of singletons. Compare with Metamath's theorem with the same name.

lemma `neindisj`: **assumes** $T \{\text{is a topology}\} A \subseteq \bigcup T \ x \in \text{Closure}(A, T)$ **and**
 $N \in (\{\text{set neighborhood system of } T\} \{x\})$
shows $N \cap A \neq \emptyset$
<proof>

end

76 Uniform spaces

theory `UniformSpace_ZF` **imports** `Topology_ZF_2` `Topology_ZF_4a` `func_ZF_1`
begin

This theory defines uniform spaces and proves their basic properties.

76.1 Entourages and neighborhoods

Just like a topological space constitutes the minimal setting in which one can speak of continuous functions, the notion of uniform spaces (commonly attributed to André Weil) captures the minimal setting in which one can speak of uniformly continuous functions. In some sense this is a generalization of the notion of metric (or metrizable) spaces and topological groups.

There are several definitions of uniform spaces. The fact that these definitions are equivalent is far from obvious (some people call such phenomenon cryptomorphism). We will use the definition of the uniform structure (or "uniformity") based on entourages. This was the original definition by Weil

and it seems to be the most commonly used. A uniformity consists of entourages that are binary relations between points of space X that satisfy a certain collection of conditions, specified below.

definition

IsUniformity ($_ \{ \text{is a uniformity on} \} _ 90$) **where**
 $\Phi \{ \text{is a uniformity on} \} X \equiv (\Phi \{ \text{is a filter on} \} (X \times X))$
 $\wedge (\forall U \in \Phi. \text{id}(X) \subseteq U \wedge (\exists V \in \Phi. V \cap V \subseteq U) \wedge \text{converse}(U) \in \Phi)$

Since the whole $X \times X$ is in a uniformity, a uniformity is never empty.

lemma `uniformity_non_empty`: **assumes** $\Phi \{ \text{is a uniformity on} \} X$
shows $\Phi \neq \emptyset$
 $\langle \text{proof} \rangle$

If Φ is a uniformity on X , then the every element V of Φ is a certain relation on X (a subset of $X \times X$) and is called an "entourage". For an $x \in X$ we call $V\{x\}$ a neighborhood of x . The first useful fact we will show is that neighborhoods are non-empty.

lemma `neigh_not_empty`:
assumes $\Phi \{ \text{is a uniformity on} \} X$ $W \in \Phi$ **and** $x \in X$
shows $W\{x\} \neq \emptyset$ **and** $x \in W\{x\}$
 $\langle \text{proof} \rangle$

The filter part of the definition of uniformity for easier reference:

lemma `unif_filter`: **assumes** $\Phi \{ \text{is a uniformity on} \} X$
shows $\Phi \{ \text{is a filter on} \} (X \times X)$
 $\langle \text{proof} \rangle$

If X is not empty then the singleton $\{X \times X\}$ is a (trivial) example of a uniformity on X .

lemma `min_uniformity`: **assumes** $X \neq \emptyset$ **shows** $\{X \times X\} \{ \text{is a uniformity on} \} X$
 $\langle \text{proof} \rangle$

$X \times X$ is an entourage of every uniformity on X .

lemma `min_uniformity1`:
assumes $\Phi \{ \text{is a uniformity on} \} X$ **shows** $(X \times X) \in \Phi$
 $\langle \text{proof} \rangle$

On the other side of the spectrum is the collection of sets containing the diagonal, that is also a uniformity.

lemma `max_uniformity`: **assumes** $X \neq \emptyset$
shows $\{U \in \text{Pow}(X \times X). \text{id}(X) \subseteq U\} \{ \text{is a uniformity on} \} X$
 $\langle \text{proof} \rangle$

The second part of the definition of uniformity for easy reference:

lemma `entourage_props`:

assumes Φ {is a uniformity on} X and $A \in \Phi$
shows
 $A \subseteq X \times X$
 $\text{id}(X) \subseteq A$
 $\exists V \in \Phi. V \circ V \subseteq A$
 $\text{converse}(A) \in \Phi$
<proof>

The definition of uniformity states (among other things) that for every member U of uniformity Φ there is another one, say V such that $V \circ V \subseteq U$. Sometimes such V is said to be half the size of U . The next lemma states that V can be taken to be symmetric.

lemma half_size_sym: **assumes** Φ {is a uniformity on} X $W \in \Phi$
shows $\exists V \in \Phi. V \circ V \subseteq W \wedge V = \text{converse}(V)$
<proof>

Inside every member W of the uniformity Φ we can find one that is symmetric and smaller than a third of size W . Compare with the Metamath's theorem with the same name.

lemma ustex3sym: **assumes** Φ {is a uniformity on} X $A \in \Phi$
shows $\exists B \in \Phi. B \circ (B \circ B) \subseteq A \wedge B = \text{converse}(B)$
<proof>

If Φ is a uniformity on X then every element of Φ is a subset of $X \times X$ whose domain is X .

lemma uni_domain:
assumes Φ {is a uniformity on} X $W \in \Phi$
shows $W \subseteq X \times X$ and $\text{domain}(W) = X$
<proof>

If Φ is a uniformity on X and $W \in \Phi$ then for every $x \in X$ the image of the singleton $\{x\}$ by W is contained in X . Compare the Metamath's theorem with the same name.

lemma ustimasn:
assumes Φ {is a uniformity on} X $W \in \Phi$ and $x \in X$
shows $W\{x\} \subseteq X$
<proof>

Uniformity Φ defines a natural topology on its space X via the neighborhood system that assigns the collection $\{V(\{x\}) : V \in \Phi\}$ to every point $x \in X$. In the next lemma we show that if we define a function this way the values of that function are what they should be. This is only a technical fact which is useful to shorten the remaining proofs, usually treated as obvious in standard mathematics.

lemma neigh_filt_fun:
assumes Φ {is a uniformity on} X

defines $\mathcal{M} \equiv \{\langle x, \{V\{x\}.V \in \Phi\} \rangle . x \in X\}$
shows $\mathcal{M}: X \rightarrow \text{Pow}(\text{Pow}(X))$ **and** $\forall x \in X. \mathcal{M}(x) = \{V\{x\}.V \in \Phi\}$
<proof>

In the next lemma we show that the collection defined in lemma `neigh_filt_fun` is a filter on X . The proof is kind of long, but it just checks that all filter conditions hold.

lemma `filter_from_uniformity`:
assumes Φ {is a uniformity on} X **and** $x \in X$
defines $\mathcal{M} \equiv \{\langle x, \{V\{x\}.V \in \Phi\} \rangle . x \in X\}$
shows $\mathcal{M}(x)$ {is a filter on} X
<proof>

A rephrasing of `filter_from_uniformity`: if Φ is a uniformity on X , then $\{V(\{x\}) | V \in \Phi\}$ is a filter on X for every $x \in X$.

lemma `unif_filter_at_point`:
assumes Φ {is a uniformity on} X **and** $x \in X$
shows $\{V\{x\}.V \in \Phi\}$ {is a filter on} X
<proof>

A frequently used property of filters is that they are "upward closed" i.e. supersets of a filter element are also in the filter. The next lemma makes this explicit for easy reference as applied to the natural filter created from a uniformity.

corollary `unif_filter_up_closed`:
assumes Φ {is a uniformity on} X $x \in X$ $U \in \{V\{x\}.V \in \Phi\}$ $W \subseteq X$ $U \subseteq W$
shows $W \in \{V\{x\}.V \in \Phi\}$
<proof>

The function defined in the premises of lemma `neigh_filt_fun` (or `filter_from_uniformity`) is a neighborhood system. The proof uses the existence of the "half-the-size" neighborhood condition ($\exists V \in \Phi. V \cap V \subseteq U$) of the uniformity definition, but not the `converse(U) \in \Phi` part.

theorem `neigh_from_uniformity`:
assumes Φ {is a uniformity on} X
shows $\{\langle x, \{V\{x\}.V \in \Phi\} \rangle . x \in X\}$ {is a neighborhood system on} X
<proof>

When we have a uniformity Φ on X we can define a topology on X in a (relatively) natural way. We will call that topology the `UniformTopology( $\Phi$ )`. We could probably reformulate the definition to skip the X parameter because if Φ is a uniformity on X then X can be recovered from (is determined by) Φ .

definition
`UniformTopology( $\Phi, X$ )` $\equiv \{U \in \text{Pow}(X). \forall x \in U. U \in \{V\{x\}.V \in \Phi\}\}$

An identity showing how the definition of uniform topology is constructed. Here, the $M = \{\langle t, \{V\{t\} : V \in \Phi\} \rangle : t \in X\}$ is the neighborhood system (a function on X) created from uniformity Φ . Then for each $x \in X$, $M(x) = \{V\{t\} : V \in \Phi\}$ is the set of neighborhoods of x .

lemma uniftop_def_alt:

shows $\text{UniformTopology}(\Phi, X) = \{U \in \text{Pow}(X). \forall x \in U. U \in \{\langle t, \{V\{t\}. V \in \Phi\} \rangle. t \in X\}(x)\}$
<proof>

The collection of sets constructed in the `UniformTopology` definition is indeed a topology on X .

theorem uniform_top_is_top:

assumes Φ {is a uniformity on} X
shows
 $\text{UniformTopology}(\Phi, X)$ {is a topology} and $\bigcup \text{UniformTopology}(\Phi, X) = X$
<proof>

If we have a uniformity Φ we can create a neighborhood system from it in two ways. We can create a neighborhood system directly from Φ using the formula $X \ni x \mapsto \{V\{x\} | V \in \Phi\}$ (see theorem `neigh_from_uniformity`). Alternatively we can construct a topology from Φ as in theorem `uniform_top_is_top` and then create a neighborhood system from this topology as in theorem `neigh_from_topology`. The next theorem states that these two ways give the same result.

theorem neigh_unif_same: **assumes** Φ {is a uniformity on} X

shows
 $\{\langle x, \{V\{x\}. V \in \Phi\} \rangle. x \in X\} = \{\text{neighborhood system of}\} \text{UniformTopology}(\Phi, X)$
<proof>

Another form of the definition of topology generated from a uniformity.

lemma uniftop_def_alt1: **assumes** Φ {is a uniformity on} X

shows $\text{UniformTopology}(\Phi, X) = \{U \in \text{Pow}(X). \forall x \in U. \exists W \in \Phi. W\{x\} \subseteq U\}$
<proof>

Images of singletons by entourages are neighborhoods of those singletons.

lemma image_singleton_ent_nei:

assumes Φ {is a uniformity on} X $V \in \Phi$ $x \in X$
defines $\mathcal{M} \equiv \{\text{neighborhood system of}\} \text{UniformTopology}(\Phi, X)$
shows $V\{x\} \in \mathcal{M}(x)$
<proof>

The set neighborhoods of a singleton $\{x\}$ where $x \in X$ consist of images of the singleton by the entourages $W \in \Phi$. See also the Metamath's theorem with the same name.

lemma utopsnneip: **assumes** Φ {is a uniformity on} X $x \in X$

defines $\mathcal{S} \equiv \{\text{set neighborhood system of}\} \text{UniformTopology}(\Phi, X)$

shows $\mathcal{S}\{x\} = \{W\{x\}. W \in \Phi\}$
<proof>

Images of singletons by entourages are set neighborhoods of those singletons.
 See also the Metamath theorem with the same name.

corollary utopsnei: **assumes** Φ {is a uniformity on} X $W \in \Phi$ $x \in X$
defines $\mathcal{S} \equiv \{\text{set neighborhood system of}\}$ UniformTopology(Φ, X)
shows $W\{x\} \in \mathcal{S}\{x\}$ *<proof>*

If Φ is a uniformity on X that generates a topology T , R is any relation on X (i.e. $R \subseteq X \times X$), W is a symmetric entourage (i.e. $W \in \Phi$, and W is symmetric (i.e. equal to its converse)), then the closure of R in the product topology is contained the the composition $V \circ (M \circ V)$. Metamath has a similar theorem with the same name.

lemma utop3cls:
assumes Φ {is a uniformity on} X $R \subseteq X \times X$ $W \in \Phi$ $W = \text{converse}(W)$
defines $J \equiv \text{UniformTopology}(\Phi, X)$
shows $\text{Closure}(R, J \times_t J) \subseteq W \circ (R \circ W)$
<proof>

Uniform spaces are regular. Note that is not the same as T_3 , see Topology_ZF_1 for separation axioms definitions. In some sources the definitions of "regular" and T_3 are swapped. In IsarMathLib we adopt the terminology as on the "Separation axiom" page on Wikipedia.

theorem utopreg:
assumes Φ {is a uniformity on} X
shows UniformTopology(Φ, X) {is regular}
<proof>

If the topological space generated by a uniformity Φ on X is T_1 then the intersection $\bigcap \Phi$ is contained in the diagonal $\{\langle x, x \rangle : x \in X\}$. Note the opposite inclusion is true for every uniformity.

lemma unif_t1_inter_diag:
assumes Φ {is a uniformity on} X and UniformTopology(Φ, X) {is T_1 }
shows $\bigcap \Phi \subseteq \{\langle x, x \rangle. x \in X\}$
<proof>

If the intersection of a uniformity is contained in the the diagonal $\{\langle x, x \rangle : x \in X\}$ then the topological space generated by this uniformity is T_1 .

lemma unif_inter_diag_t1:
assumes Φ {is a uniformity on} X and $\bigcap \Phi \subseteq \{\langle x, x \rangle. x \in X\}$
shows UniformTopology(Φ, X) {is T_1 }
<proof>

If Φ is a uniformity on X then the intersection of Φ is contained in diagonal of X if and only if $\bigcup \Phi$ is equal to that diagonal. Some people call such uniform space "separating".

theorem unif_inter_diag: **assumes** Φ {is a uniformity on} X
shows $\bigcap \Phi \subseteq \{\langle x, x \rangle. x \in X\} \longleftrightarrow \bigcap \Phi = \{\langle x, x \rangle. x \in X\}$
<proof>

The next theorem collects the information we have to show that if Φ is a uniformity on X , with the induced topology T then conditions T is T_0 , T is T_1 , T is T_2 T is T_3 are all equivalent to the intersection of Φ being contained in the diagonal (which is equivalent to the intersection of Φ being equal to the diagonal, see unif_inter_diag above).

theorem unif_sep_axioms_diag: **assumes** Φ {is a uniformity on} X
defines $T \equiv \text{UniformTopology}(\Phi, X)$
shows
 $\bigcap \Phi \subseteq \{\langle x, x \rangle. x \in X\} \longleftrightarrow T$ {is T_0 }
 $\bigcap \Phi \subseteq \{\langle x, x \rangle. x \in X\} \longleftrightarrow T$ {is T_1 }
 $\bigcap \Phi \subseteq \{\langle x, x \rangle. x \in X\} \longleftrightarrow T$ {is T_2 }
 $\bigcap \Phi \subseteq \{\langle x, x \rangle. x \in X\} \longleftrightarrow T$ {is T_3 }
<proof>

76.2 Base of a uniformity

A base or a fundamental system of entourages of a uniformity Φ is a subset of Φ that is sufficient to uniquely determine it. This is analogous to the notion of a base of a topology (see Topology_ZF_1 or a base of a filter (see Topology_ZF_4).

A base of a uniformity Φ is any subset $\mathfrak{B} \subseteq \Phi$ such that every entourage in Φ contains (at least) one from $\mathfrak{B}$. The phrase is a base for is already defined to mean a base for a topology, so we use the phrase is a uniform base of here.

definition

IsUniformityBase ($_$ {is a uniform base of} $_$ 90) **where**
 $\mathfrak{B}$ {is a uniform base of} $\Phi \equiv \mathfrak{B} \subseteq \Phi \wedge (\forall U \in \Phi. \exists B \in \mathfrak{B}. B \subseteq U)$

Symmetric entourages form a base of the uniformity.

lemma symm_are_base: **assumes** Φ {is a uniformity on} X
shows $\{V \in \Phi. V = \text{converse}(V)\}$ {is a uniform base of} Φ
<proof>

Given a base of a uniformity we can recover the uniformity taking the supersets. The Supersets constructor is defined in ZF1.

lemma uniformity_from_base:

assumes Φ {is a uniformity on} X $\mathfrak{B}$ {is a uniform base of} Φ
shows $\Phi = \text{Supersets}(X \times X, \mathfrak{B})$
<proof>

Analogous to the predicate "satisfies base condition" (defined in Topology_ZF_1) and "is a base filter" (defined in Topology_ZF_4) we can specify conditions

for a collection $\mathfrak{B}$ of subsets of $X \times X$ to be a base of some uniformity on X . Namely, the following conditions are necessary and sufficient:

1. Intersection of two sets of $\mathfrak{B}$ contains a set of $\mathfrak{B}$.
2. Every set of $\mathfrak{B}$ contains the diagonal of $X \times X$.
3. For each set $B_1 \in \mathfrak{B}$ we can find a set $B_2 \in \mathfrak{B}$ such that $B_2 \subseteq B_1^{-1}$.
4. For each set $B_1 \in \mathfrak{B}$ we can find a set $B_2 \in \mathfrak{B}$ such that $B_2 \circ B_2 \subseteq B_1$.

The conditions in the definition below are taken from N. Bourbaki "Elements of Mathematics, General Topology", Chapter II.1., except for the last two that are missing there.

definition

`IsUniformityBaseOn ( _ {is a uniform base on} _ 90) where`
`$\mathfrak{B}$  {is a uniform base on}  $X \equiv$`
`$(\forall B_1 \in \mathfrak{B}. \forall B_2 \in \mathfrak{B}. \exists B_3 \in \mathfrak{B}. B_3 \subseteq B_1 \cap B_2) \wedge (\forall B \in \mathfrak{B}. \text{id}(X) \subseteq B) \wedge$`
`$(\forall B_1 \in \mathfrak{B}. \exists B_2 \in \mathfrak{B}. B_2 \subseteq \text{converse}(B_1)) \wedge (\forall B_1 \in \mathfrak{B}. \exists B_2 \in \mathfrak{B}. B_2 \circ B_2 \subseteq B_1) \wedge$`
`$\mathfrak{B} \subseteq \text{Pow}(X \times X) \wedge \mathfrak{B} \neq \emptyset$`

The next lemma splits the definition of `IsUniformityBaseOn` into four conditions to enable more precise references in proofs.

lemma `uniformity_base_props: assumes  $\mathfrak{B}$  {is a uniform base on}  $X$`
`shows`
`$\forall B_1 \in \mathfrak{B}. \forall B_2 \in \mathfrak{B}. \exists B_3 \in \mathfrak{B}. B_3 \subseteq B_1 \cap B_2$`
`$\forall B \in \mathfrak{B}. \text{id}(X) \subseteq B$`
`$\forall B_1 \in \mathfrak{B}. \exists B_2 \in \mathfrak{B}. B_2 \subseteq \text{converse}(B_1)$`
`$\forall B_1 \in \mathfrak{B}. \exists B_2 \in \mathfrak{B}. B_2 \circ B_2 \subseteq B_1$`
`$\mathfrak{B} \subseteq \text{Pow}(X \times X)$  and  $\mathfrak{B} \neq \emptyset$`
`<proof>`

If supersets of some collection of subsets of $X \times X$ form a uniformity, then this collection satisfies the conditions in the definition of `IsUniformityBaseOn`.

theorem `base_is_uniform_base:`

`assumes $\mathfrak{B} \subseteq \text{Pow}(X \times X)$ and Supersets($X \times X, \mathfrak{B}$) {is a uniformity on} X
 shows $\mathfrak{B}$ {is a uniform base on} X
 <proof>`

if a nonempty collection of subsets of $X \times X$ satisfies conditions in the definition of `IsUniformityBaseOn` then the supersets of that collection form a uniformity on X .

theorem `uniformity_base_is_base:`

`assumes  $X \neq \emptyset$  and  $\mathfrak{B}$  {is a uniform base on}  $X$`
`shows Supersets( $X \times X, \mathfrak{B}$ ) {is a uniformity on}  $X$`
`<proof>`

A fundamental system of entourages is a uniform base of its supersets.

lemma `uniform_base_supersets:`

```

    assumes  $X \neq \emptyset$  and  $\mathcal{B}$  {is a uniform base on}  $X$ 
    shows  $\mathcal{B}$  {is a uniform base of} (Supersets( $X \times X, \mathcal{B}$ ))
  <proof>

```

A uniform base of a uniformity is a fundamental system of entourages.

```

lemma uniform_base_of_is_on:
  assumes  $\Phi$  {is a uniformity on}  $X$   $\mathcal{B}$  {is a uniform base of}  $\Phi$ 
  shows  $\mathcal{B}$  {is a uniform base on}  $X$ 
  <proof>

```

The assumption that X is not empty in `uniform_base_is_base` above is necessary as the assertion is false if X is empty.

```

lemma uniform_space_empty: assumes  $\mathcal{B}$  {is a uniform base on}  $\emptyset$ 
  shows  $\neg(\text{Supersets}(\emptyset \times \emptyset, \mathcal{B}) \text{ {is a uniformity on} } \emptyset)$ 
  <proof>

```

76.3 Uniformly continuous functions

Just as the most general setting for continuity of functions is that of topological spaces, uniform spaces are the most general setting for the study of uniform continuity.

A map between 2 uniformities is uniformly continuous if it preserves the entourages:

```

definition
  IsUniformlyCont ( $_$  {is uniformly continuous between}  $_$  {and}  $_$  90) where
     $f: X \rightarrow Y \implies \Phi$  {is a uniformity on}  $X \implies \Gamma$  {is a uniformity on}  $Y \implies$ 
     $f$  {is uniformly continuous between}  $\Phi$  {and}  $\Gamma \equiv \forall V \in \Gamma. (\text{ProdFunction}(f, f) - V) \in \Phi$ 

```

Any uniformly continuous function is continuous when considering the topologies on the uniformities.

```

lemma uniformly_cont_is_cont:
  assumes  $f: X \rightarrow Y$   $\Phi$  {is a uniformity on}  $X$   $\Gamma$  {is a uniformity on}  $Y$ 
     $f$  {is uniformly continuous between}  $\Phi$  {and}  $\Gamma$ 
  shows IsContinuous(UniformTopology( $\Phi, X$ ), UniformTopology( $\Gamma, Y$ ),  $f$ )
  <proof>

```

end

77 Metric spaces

```

theory MetricSpace_ZF imports Topology_ZF_1 OrderedLoop_ZF Lattice_ZF
UniformSpace_ZF
begin

```

A metric space is a set on which a distance between points is defined as a function $d : X \times X \rightarrow [0, \infty)$. With this definition each metric space is a

topological space which is paracompact and Hausdorff (T_2), hence normal (in fact even perfectly normal).

77.1 Pseudometric - definition and basic properties

A metric on X is usually defined as a function $d : X \times X \rightarrow [0, \infty)$ that satisfies the conditions $d(x, x) = 0$, $d(x, y) = 0 \Rightarrow x = y$ (identity of indiscernibles), $d(x, y) = d(y, x)$ (symmetry) and $d(x, y) \leq d(x, z) + d(z, y)$ (triangle inequality) for all $x, y \in X$. Here we are going to be a bit more general and define metric and pseudo-metric as a function valued in an ordered loop.

First we define a pseudo-metric, which has the axioms of a metric, but without the second part of the identity of indiscernibles. In our definition `IsApseudoMetric` is a predicate on five sets: the function d , the set X on which the metric is defined, the loop carrier G , the loop operation A and the order r on G .

definition

$$\begin{aligned} \text{IsApseudoMetric}(d, X, G, A, r) &\equiv d : X \times X \rightarrow \text{Nonnegative}(G, A, r) \\ &\wedge (\forall x \in X. d\langle x, x \rangle = \text{TheNeutralElement}(G, A)) \\ &\wedge (\forall x \in X. \forall y \in X. d\langle x, y \rangle = d\langle y, x \rangle) \\ &\wedge (\forall x \in X. \forall y \in X. \forall z \in X. \langle d\langle x, z \rangle, A\langle d\langle x, y \rangle, d\langle y, z \rangle \rangle \in r) \end{aligned}$$

We add the full axiom of identity of indiscernibles to the definition of a pseudometric to get the definition of metric.

definition

$$\begin{aligned} \text{IsAMetric}(d, X, G, A, r) &\equiv \\ &\text{IsApseudoMetric}(d, X, G, A, r) \wedge (\forall x \in X. \forall y \in X. d\langle x, y \rangle = \text{TheNeutralElement}(G, A) \\ &\longrightarrow x=y) \end{aligned}$$

A disk is defined as set of points located less than the radius from the center.

definition $\text{Disk}(X, d, r, c, R) \equiv \{x \in X. \langle d\langle c, x \rangle, R \rangle \in \text{StrictVersion}(r)\}$

We define metric topology as consisting of unions of open disks. Note the same definition is used for the topology generated by a pseudometric.

definition

$$\text{MetricTopology}(X, L, A, r, d) \equiv \{\bigcup \mathcal{A}. \mathcal{A} \in \text{Pow}(\bigcup_{c \in X. \{ \text{Disk}(X, d, r, c, R) \mid R \in \text{PositiveSet}(L, A, r) \}})\}$$

Next we define notation for metric spaces. We will reuse the additive notation defined in the `loop1` locale adding only the assumption about d being a pseudometric and notation for a disk centered at c with radius R . Since for many theorems it is sufficient to assume the pseudometric axioms we will assume in this context that the sets d, X, L, A, r form a pseudometric rather than a metric. In the `pmetric_space` context τ denotes the topology defined

by the metric d . Analogously to the notation defined in the `topology0` context $\text{int}(A)$, $\text{cl}(A)$, ∂A will denote the interior, closure and boundary of the set A with respect to the metric topology.

```

locale pmetric_space = loop1 +
  fixes d and X
  assumes pmetricAssum: IsApseudoMetric(d,X,L,A,r)
  fixes disk
  defines disk_def [simp]: disk(c,R)  $\equiv$  Disk(X,d,r,c,R)
  fixes pmettop ( $\tau$ )
  defines pmettop [simp]:  $\tau \equiv$  MetricTopology(X,L,A,r,d)
  fixes interior (int)
  defines interior_def [simp]: int(D)  $\equiv$  Interior(D, $\tau$ )
  fixes cl
  defines cl_def [simp]: cl(D)  $\equiv$  Closure(D, $\tau$ )

```

The next lemma shows the definition of the pseudometric in the notation used in the `pmetric_space` context.

```

lemma (in pmetric_space) pmetric_properties: shows
  d: X×X  $\rightarrow$  L+
   $\forall x \in X. d\langle x, x \rangle = 0$ 
   $\forall x \in X. \forall y \in X. d\langle x, y \rangle = d\langle y, x \rangle$ 
   $\forall x \in X. \forall y \in X. \forall z \in X. d\langle x, z \rangle \leq d\langle x, y \rangle + d\langle y, z \rangle$ 
  <proof>

```

The values of the metric are in the in the nonnegative set of the loop, hence in the loop.

```

lemma (in pmetric_space) pmetric_loop_valued: assumes x∈X y∈X
  shows d⟨x,y⟩ ∈ L+ d⟨x,y⟩ ∈ L
  <proof>

```

The definition of the disk in the notation used in the `pmetric_space` context:

```

lemma (in pmetric_space) disk_definition: shows disk(c,R) = {x∈X. d⟨c,x⟩
  < R}
  <proof>

```

If the radius is positive then the center is in disk.

```

lemma (in pmetric_space) center_in_disk: assumes c∈X and R∈L+ shows
  c ∈ disk(c,R)
  <proof>

```

A technical lemma that allows us to shorten some proofs: if c is an element of X and x is in disk with center c and radius R then R is a positive element of L and $-d(x, c) + R$ is in the set of positive elements of the loop.

```

lemma (in pmetric_space) radius_in_loop: assumes c∈X and x ∈ disk(c,R)
  shows R∈L 0<R R∈L+ (-d⟨c,x⟩ + R) ∈ L+
  <proof>

```

If a point x is inside a disk B and $m \leq -d\langle c, x \rangle + R$ then the disk centered at the point x and with radius m is contained in the disk B .

lemma (in pmetric_space) disk_in_disk:
 assumes $c \in X$ and $x \in \text{disk}(c, R)$ and $m \leq (-d\langle c, x \rangle + R)$
 shows $\text{disk}(x, m) \subseteq \text{disk}(c, R)$
<proof>

A special case of disk_in_disk where we set $m = -d\langle c, x \rangle + R$: if x is an element of a disk with center $c \in X$ and radius R then this disk contains the disk centered at x and with radius $-d\langle c, x \rangle + R$.

lemma (in pmetric_space) disk_in_disk1:
 assumes $c \in X$ and $x \in \text{disk}(c, R)$
 shows $\text{disk}(x, -d\langle c, x \rangle + R) \subseteq \text{disk}(c, R)$
<proof>

Assuming that two disks have the same center, closed disk with smaller radius is contained in the (open) disk with a larger radius.

lemma (in pmetric_space) disk_radius_strict_mono:
 assumes $r_1 < r_2$
 shows $\{y \in X. d\langle x, y \rangle \leq r_1\} \subseteq \text{disk}(x, r_2)$
<proof>

If we assume that the loop's order relation down-directs L_+ then the collection of disks centered at points of the space and with radii in the positive set of the loop satisfies the base condition. The property that an order relation "down-directs" a set is defined in Order_ZF and means that every two-element subset of the set has a lower bound in that set.

lemma (in pmetric_space) disks_form_base:
 assumes $r \text{ down-directs } L_+$
 defines $B \equiv \bigcup_{c \in X. \{ \text{disk}(c, R). R \in L_+ \}}$
 shows $B \text{ satisfies the base condition}$
<proof>

Disks centered at points farther away than the sum of radii do not overlap.

lemma (in pmetric_space) far_disks:
 assumes $x \in X$ $y \in X$ $r_x + r_y \leq d\langle x, y \rangle$
 shows $\text{disk}(x, r_x) \cap \text{disk}(y, r_y) = \emptyset$
<proof>

If we have a loop element that is smaller than the distance between two points, then we can separate these points with disks.

lemma (in pmetric_space) disjoint_disks:
 assumes $x \in X$ $y \in X$ $r_x < d\langle x, y \rangle$
 shows $(-r_x + (d\langle x, y \rangle)) \in L_+$ and $\text{disk}(x, r_x) \cap \text{disk}(y, -r_x + (d\langle x, y \rangle)) = \emptyset$
<proof>

The definition of metric topology written in notation of `pmetric_space` context:

```
lemma (in pmetric_space) metric_top_def_alt:
  defines B  $\equiv \bigcup_{c \in X. \{ \text{disk}(c, R) . R \in L_+ \}}$ 
  shows  $\tau = \{ \bigcup A . A \in \text{Pow}(B) \}$ 
  <proof>
```

If the order of the loop down-directs its set of positive elements then the metric topology defined as collection of unions of (open) disks is indeed a topology. Recall that in the `pmetric_space` context τ denotes the metric topology.

```
theorem (in pmetric_space) pmetric_is_top:
  assumes r {down-directs} L+
  shows  $\tau$  {is a topology}
  <proof>
```

If r down-directs L_+ then the collection of open disks is a base for the metric topology.

```
theorem (in pmetric_space) disks_are_base:
  assumes r {down-directs} L+
  defines B  $\equiv \bigcup_{c \in X. \{ \text{disk}(c, R) . R \in L_+ \}}$ 
  shows B {is a base for}  $\tau$ 
  <proof>
```

If r down-directs L_+ then X is the carrier of metric topology.

```
theorem (in pmetric_space) metric_top_carrier:
  assumes r {down-directs} L+ shows  $\bigcup \tau = X$ 
  <proof>
```

Under the assumption that r down-directs L_+ the propositions proven in the `topology0` context can be used in the `pmetric_space` context.

```
lemma (in pmetric_space) topology0_valid_in_pmetric_space:
  assumes r {down-directs} L+
  shows topology0( $\tau$ )
  <proof>
```

If r down-directs L_+ then disks are open in the metric topology.

```
lemma (in pmetric_space) disks_open:
  assumes c  $\in X$  R  $\in L_+$  r {down-directs} L+
  shows  $\text{disk}(c, R) \in \tau$ 
  <proof>
```

If r down-directs L_+ and x is an element of an open set U then there exist radius $R \in L_+$ such that the disk with center x and radius R is contained in U .

```
lemma (in pmetric_space) point_open_disk:
```

```

assumes r {down-directs} L+ U ∈ τ x ∈ U
shows ∃ R ∈ L+. disk(x, R) ⊆ U
⟨proof⟩

```

If r down-directs L_+ then the generated topology cannot distinguish two points if their distance is zero. "Cannot distinguish" here means that if one is in an open set then the second one is in that set too.

```

lemma (in pmetric_space) zero_dist_same_open:
  assumes r {down-directs} L+ U ∈ τ x ∈ U y ∈ X d⟨x,y⟩=0
  shows y ∈ U
⟨proof⟩

```

A pseudometric that induces a T_0 topology is a metric.

```

theorem (in pmetric_space) pmetric_t0_metric:
  assumes r {down-directs} L+ and τ {is T0}
  shows IsAmetric(d, X, L, A, r)
⟨proof⟩

```

To define the `metric_space` locale we take the `pmetric_space` and add the assumption of identity of indiscernibles.

```

locale metric_space = pmetric_space +
  assumes ident_indisc: ∀ x ∈ X. ∀ y ∈ X. d⟨x,y⟩=0 ⟶ x=y

```

In the `metric_space` locale d is a metric.

```

lemma (in metric_space) d_metric: shows IsAmetric(d, X, L, A, r)
⟨proof⟩

```

Distance of different points is greater than zero.

```

lemma (in metric_space) dist_pos: assumes x ∈ X y ∈ X x ≠ y
  shows 0 < d⟨x,y⟩ d⟨x,y⟩ ∈ L+
⟨proof⟩

```

If r down-directs L_+ then the ordered loop valued metric space is T_2 (i.e. Hausdorff).

```

theorem (in metric_space) metric_space_T2:
  assumes r {down-directs} L+
  shows τ {is T2}
⟨proof⟩

```

77.2 Uniform structures on (pseudo-)metric spaces

Each pseudometric space with pseudometric $d : X \times X \rightarrow L^+$ supports a natural uniform structure, defined as supersets of the collection of inverse images $U_c = d^{-1}([0, c])$, where $c > 0$.

In the following definition X is the underlying space, L is the loop (carrier), A is the loop operation, r is an order relation compatible with A , and d is a

pseudometric on X , valued in the ordered loop L . With this we define the uniform gauge as the collection of inverse images of the closed intervals $[0, c]$ as c varies of the set of positive elements of L . See `uniform_gauge_def_alt` for this definition in a more readable notation.

definition

`UniformGauge(X,L,A,r,d)  $\equiv$  {d-({c $\in$ Nonnegative(L,A,r). <c,b>  $\in$  r}). b $\in$ PositiveSet(L,A,r)}`

In the `pmetric_space` context we will write `UniformGauge(X,L,A,r,d)` as $\mathfrak{B}$.

abbreviation (in `pmetric_space`) `gauge` ($\mathfrak{B}$) **where** $\mathfrak{B} \equiv \text{UniformGauge}(X,L,A,r,d)$

In notation defined in the `pmetric_space` context we can write the uniform gauge as $\{d^{-1}(\{c \in L^+ : c \leq b\}) : b \in L_+\}$.

lemma (in `pmetric_space`) `uniform_gauge_def_alt`:

shows $\mathfrak{B} = \{d^{-1}(\{c \in L^+ . c \leq b\}) . b \in L_+\}$

<proof>

Members of the uniform gauge are subsets of $X \times X$ i.e. relations on X .

lemma (in `pmetric_space`) `uniform_gauge_relations`:

assumes $B \in \mathfrak{B}$ **shows** $B \subseteq X \times X$

<proof>

If the distance between two points of X is less or equal b , then this pair of points is in $d^{-1}([0, b])$.

lemma (in `pmetric_space`) `gauge_members`:

assumes $x \in X$ $y \in X$ $d\langle x, y \rangle \leq b$

shows $\langle x, y \rangle \in d^{-1}(\{c \in L^+ . c \leq b\})$

<proof>

Suppose $b \in L_+$ (i.e. b is an element of the loop that is greater than the neutral element) and $x \in X$. Then the image of the singleton set $\{x\}$ by the relation $B = \{d^{-1}(\{c \in L^+ : c \leq b\})$ is the set $\{y \in X : d\langle x, y \rangle \leq b\}$, i.e. the closed disk with center x and radius b . Hence the the image $B\{x\}$ contains the open disk with center x and radius b .

lemma (in `pmetric_space`) `disk_in_gauge`:

assumes $b \in L_+$ $x \in X$

defines $B \equiv d^{-1}(\{c \in L^+ . c \leq b\})$

shows $B\{x\} = \{y \in X . d\langle x, y \rangle \leq b\}$ **and** $\text{disk}(x, b) \subseteq B\{x\}$

<proof>

Gauges corresponding to larger elements of the loop are larger.

lemma (in `pmetric_space`) `uniform_gauge_mono`:

assumes $b_1 \leq b_2$ **shows** $d^{-1}(\{c \in L^+ . c \leq b_1\}) \subseteq d^{-1}(\{c \in L^+ . c \leq b_2\})$

<proof>

For any two sets of the form $d^{-1}([0, b])$ we can find a third one that is contained in both.

lemma (in pmetric_space) gauge_1st_cond:
 assumes $r \{ \text{down-directs} \} L_+ \ B_1 \in \mathfrak{B} \ B_2 \in \mathfrak{B}$
 shows $\exists B_3 \in \mathfrak{B}. \ B_3 \subseteq B_1 \cap B_2$
<proof>

Sets of the form $d^{-1}([0, b])$ contain the diagonal.

lemma (in pmetric_space) gauge_2nd_cond: assumes $B \in \mathfrak{B}$ shows $\text{id}(X) \subseteq B$
<proof>

Sets of the form $d^{-1}([0, b])$ are symmetric.

lemma (in pmetric_space) gauge_symmetric:
 assumes $B \in \mathfrak{B}$ shows $B = \text{converse}(B)$
<proof>

A set of the form $d^{-1}([0, b])$ contains a symmetric set of this form.

corollary (in pmetric_space) gauge_3rd_cond:
 assumes $B_1 \in \mathfrak{B}$ shows $\exists B_2 \in \mathfrak{B}. \ B_2 \subseteq \text{converse}(B_1)$
<proof>

The collection of sets of the form $d^{-1}([0, b])$ for $b \in L_+$ is contained of the powerset of $X \times X$.

lemma (in pmetric_space) gauge_5thCond: shows $\mathfrak{B} \subseteq \text{Pow}(X \times X)$
<proof>

If the set of positive values is non-empty, then there are sets of the form $d^{-1}([0, b])$ for $b > 0$.

lemma (in pmetric_space) gauge_6thCond:
 assumes $L_+ \neq \emptyset$ shows $\mathfrak{B} \neq \emptyset$ *<proof>*

The remaining 4th condition for the sets of the form $d^{-1}([0, b])$ to be a uniform base (or a fundamental system of entourages) cannot be proven without additional assumptions in the context of ordered loop valued metrics. To see that consider the example of natural numbers with the metric $d\langle x, y \rangle = |x - y|$, where we think of d as valued in the nonnegative set of ordered group of integers. Now take the set $B_1 = d^{-1}([0, 1]) = d^{-1}(\{0, 1\})$. Then the set $B_1 \circ B_1$ is strictly larger than B_1 , but there is no smaller set B_2 we can take so that $B_2 \circ B_2 \subseteq B_1$. One condition that is sufficient is that for every $b_1 > 0$ there is a $b_2 > 0$ such that $b_2 + b_2 \leq b_1$. I have not found a standard name for this property, for now we will use the name `IsHalfable`.

definition

$\text{IsHalfable}(L, A, r) \equiv \forall b_1 \in \text{PositiveSet}(L, A, r). \ \exists b_2 \in \text{PositiveSet}(L, A, r). \ \langle A\langle b_2, b_2 \rangle, b_1 \rangle \in r$

The property of halfability written in the notation used in the `pmetric_space` context.

lemma (in pmetric_space) is_halfable_def_alt:

assumes IsHalfable(L,A,r) $b_1 \in L_+$
shows $\exists b_2 \in L_+. b_2 + b_2 \leq b_1$
 $\langle proof \rangle$

If $B_i = d^{-1}(\{c \in L_+ : c \leq b_i\})$ for $i = 1, 2$ and $b_2 + b_2 \leq b_1$ then $B_2 \circ B_2 \subseteq B_1$. The proof uses the triangle inequality so it's not really a property of ordered loops only.

lemma (in pmetric_space) half_vimage_square:
assumes $b_2 \in L_+$ **and** $b_2 + b_2 \leq b_1$
defines $B_1 \equiv d^{-1}(\{c \in L_+ : c \leq b_1\})$ **and** $B_2 \equiv d^{-1}(\{c \in L_+ : c \leq b_2\})$
shows $B_2 \circ B_2 \subseteq B_1$
 $\langle proof \rangle$

If the loop order is halfable then for every set B_1 of the form $d^{-1}([0, b_1])$ for some $b_1 > 0$ we can find another one $B_2 = d^{-1}([0, b_2])$ such that B_2 composed with itself is contained in B_1 .

lemma (in pmetric_space) gauge_4thCond:
assumes IsHalfable(L,A,r) $B_1 \in \mathfrak{B}$ **shows** $\exists B_2 \in \mathfrak{B}. B_2 \circ B_2 \subseteq B_1$
 $\langle proof \rangle$

If X and L_+ are not empty, the order relation r down-directs L_+ , and the loop order is halfable, then $\mathfrak{B}$ (which in the pmetric_space context is an abbreviation for $\{d^{-1}(\{c \in L_+ : c \leq b\}) : b \in L_+\}$) is a fundamental system of entourages, hence its supersets form a uniformity on X and hence those supersets define a topology on X .

theorem (in pmetric_space) metric_gauge_base:
assumes $X \neq \emptyset$ $L_+ \neq \emptyset$ r {down-directs} L_+ IsHalfable(L,A,r)
shows
 $\mathfrak{B}$ {is a uniform base on} X
Supersets($X \times X, \mathfrak{B}$) {is a uniformity on} X
UniformTopology(Supersets($X \times X, \mathfrak{B}$), X) {is a topology}
 $\bigcup \text{UniformTopology}(\text{Supersets}(X \times X, \mathfrak{B}), X) = X$
 $\langle proof \rangle$

At this point we know that a pseudometric induces two topologies: one consisting of unions of open disks (the metric topology) and second one being the uniform topology derived from the uniformity generated the fundamental system of entourages (the base uniformity) of the sets of the form $d^{-1}([0, b])$ for $b > 0$. The next theorem states that if X and L_+ are not empty, r down-directs L_+ , and the loop order is halfable, then these two topologies are in fact the same. Recall that in the pmetric_space context τ denotes the metric topology.

theorem (in pmetric_space) metric_top_is_uniform_top:
assumes $X \neq \emptyset$ $L_+ \neq \emptyset$ r {down-directs} L_+ IsHalfable(L,A,r)
shows $\tau = \text{UniformTopology}(\text{Supersets}(X \times X, \mathfrak{B}), X)$
 $\langle proof \rangle$

end

78 Basic properties of real numbers

```
theory Real_ZF_2 imports OrderedField_ZF MetricSpace_ZF
begin
```

Isabelle/ZF and IsarMathLib do not have a set of real numbers built-in. The `Real_ZF` and `Real_ZF_1` theories provide a construction but here we do not use it in any way and we just assume that we have a model of real numbers (i.e. a completely ordered field) as defined in the `Ordered_Field` theory. The construction only assures us that objects with the desired properties exist in the ZF world.

78.1 Basic notation for real numbers

In this section we define notation that we will use whenever real numbers play a role, i.e. most of mathematics.

The next locale sets up notation for contexts where real numbers are used. Note we define the (real) natural numbers $\mathbb{N}$ as starting from one.

```
locale reals =
  fixes Reals( $\mathbb{R}$ ) and Add and Mul and ROrd
  assumes R_are_reals: IsAmodelOfReals( $\mathbb{R}$ ,Add,Mul, ROrd)

  fixes zero (0)
  defines zero_def[simp]: 0  $\equiv$  TheNeutralElement( $\mathbb{R}$ ,Add)

  fixes one (1)
  defines one_def[simp]: 1  $\equiv$  TheNeutralElement( $\mathbb{R}$ ,Mul)

  fixes realmul (infixl  $\cdot$  71)
  defines realmul_def[simp]:  $x \cdot y \equiv \text{Mul}\langle x,y \rangle$ 

  fixes realadd (infixl  $+$  69)
  defines realadd_def[simp]:  $x + y \equiv \text{Add}\langle x,y \rangle$ 

  fixes realminus(- _ 89)
  defines realminus_def[simp]:  $(-x) \equiv \text{GroupInv}(\mathbb{R},\text{Add})(x)$ 

  fixes realsub (infixl  $-$  90)
  defines realsub_def [simp]:  $x-y \equiv x+(-y)$ 

  fixes lesseq (infix  $\leq$  68)
  defines lesseq_def [simp]:  $x \leq y \equiv \langle x,y \rangle \in \text{ROrd}$ 
```

```

fixes sless (infix < 68)
defines sless_def [simp]:  $x < y \equiv x \leq y \wedge x \neq y$ 

fixes nonnegative ( $\mathbb{R}^+$ )
defines nonnegative_def [simp]:  $\mathbb{R}^+ \equiv \text{Nonnegative}(\mathbb{R}, \text{Add}, \text{ROrd})$ 

fixes positiveset ( $\mathbb{R}_+$ )
defines positiveset_def [simp]:  $\mathbb{R}_+ \equiv \text{PositiveSet}(\mathbb{R}, \text{Add}, \text{ROrd})$ 

fixes setinv (- _ 72)
defines setninv_def [simp]:  $-A \equiv \text{GroupInv}(\mathbb{R}, \text{Add})(A)$ 

fixes non_zero ( $\mathbb{R}_0$ )
defines non_zero_def [simp]:  $\mathbb{R}_0 \equiv \mathbb{R} - \{0\}$ 

fixes abs (| _ |)
defines abs_def [simp]:  $|x| \equiv \text{AbsoluteValue}(\mathbb{R}, \text{Add}, \text{ROrd})(x)$ 

fixes dist
defines dist_def [simp]:  $\text{dist} \equiv \{\langle p, |\text{fst}(p) - \text{snd}(p)| \rangle \mid p \in \mathbb{R} \times \mathbb{R}\}$ 

fixes two (2)
defines two_def [simp]:  $2 \equiv 1 + 1$ 

fixes inv (_-1 [96] 97)
defines inv_def [simp]:
   $x^{-1} \equiv \text{GroupInv}(\mathbb{R}_0, \text{restrict}(\text{Mul}, \mathbb{R}_0 \times \mathbb{R}_0))(x)$ 

fixes half ( $\frac{1}{2}$ )
defines half_def [simp]:  $\frac{1}{2} \equiv 2^{-1}$ 

fixes listsum ( $\sum$  _ 70)
defines listsum_def [simp]:  $\sum s \equiv \text{Fold}(\text{Add}, 0, s)$ 

fixes nat_mult (infix · 95)
defines nat_mult_def [simp]:  $n \cdot x \equiv \sum \{\langle k, x \rangle \mid k \in n\}$ 

fixes realsq (_2 [96] 97)
defines realsq_def [simp]:  $x^2 \equiv x \cdot x$ 

fixes oddext (_°)
defines oddext_def [simp]:  $f^\circ \equiv \text{OddExtension}(\mathbb{R}, \text{Add}, \text{ROrd}, f)$ 

fixes disk
defines disk_def [simp]:  $\text{disk}(c, r) \equiv \text{Disk}(\mathbb{R}, \text{dist}, \text{ROrd}, c, r)$ 

fixes rxn ( $\mathbb{N}$ )
defines rxn_def [simp]:  $\mathbb{N} \equiv \bigcap \{N \in \text{Pow}(\mathbb{R}) \mid 1 \in N \wedge (\forall n. n \in N \longrightarrow n+1 \in N)\}$ 

```

```

fixes listprod ( $\prod$  _ 70)
defines listprod_def [simp]:  $\prod s \equiv \text{Fold}(\text{Mul}, 1, s)$ 

fixes pow
defines pow_def [simp]:  $\text{pow}(n, x) \equiv \prod \{ \langle k, x \rangle . k \in n \}$ 

fixes has_supremum (_ {has a supremum} [90] 91)
defines has_supremum_def [simp]:  $A \{ \text{has a supremum} \} \equiv \text{HasAsupremum}(\text{ROrd}, A)$ 

fixes has_infimum (_ {has an infimum} [90] 91)
defines has_infimum_def [simp]:  $A \{ \text{has an infimum} \} \equiv \text{HasAnInfimum}(\text{ROrd}, A)$ 

fixes sup
defines sup_def [simp]:  $\text{sup}(A) \equiv \text{Supremum}(\text{ROrd}, A)$ 

fixes inf
defines inf_def [simp]:  $\text{inf}(A) \equiv \text{Infimum}(\text{ROrd}, A)$ 

```

The assumptions of the `field1` locale (that sets the context for ordered fields) hold in the `reals` locale

```

lemma (in reals) field1_is_valid: shows field1( $\mathbb{R}$ , Add, Mul, ROrd)
  <proof>

```

We can use theorems proven in the `field1` locale in the `reals` locale. Note that since the `field1` locale is an extension of the `ring1` locale, which is an extension of `ring0` locale, this makes available also the theorems proven in the `ring1` and `ring0` locales.

```

sublocale reals < field1 Reals Add Mul realadd realminus realsub realmul

```

```

  zero one two realsq listsum nat_mult listprod pow ROrd
  <proof>

```

The `group3` locale from the `OrderedGroup_ZF` theory defines context for theorems about ordered groups. We can use theorems proven in there in the `reals` locale as real numbers with addition form an ordered group.

```

sublocale reals < group3 Reals Add ROrd zero realadd realminus lesseq
  sless
  nonnegative positiveset setinv abs oddext listsum
  <proof>

```

Since real numbers with addition form a group we can use the theorems proven in the `group0` locale defined in the `Group_ZF` theory in the `reals` locale.

```

sublocale reals < group0 Reals Add zero realadd realminus listsum nat_mult
  <proof>

```

Let's recall basic properties of the real line.

lemma (in reals) basic_props: shows ROrd {is total on} R and Add {is commutative on} R
 <proof>

The distance function `dist` defined in the `reals` locale is a metric.

lemma (in reals) dist_is_metric: shows
 $\text{dist} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}^+$
 $\forall x \in \mathbb{R}. \forall y \in \mathbb{R}. \text{dist}(x, y) = |x - y|$
 $\forall x \in \mathbb{R}. \text{dist}(x, x) = 0$
 $\forall x \in \mathbb{R}. \forall y \in \mathbb{R}. \text{dist}(x, y) = \text{dist}(y, x)$
 $\forall x \in \mathbb{R}. \forall y \in \mathbb{R}. \forall z \in \mathbb{R}. |x - z| \leq |x - y| + |y - z|$
 $\forall x \in \mathbb{R}. \forall y \in \mathbb{R}. \forall z \in \mathbb{R}. \text{dist}(x, z) \leq \text{dist}(x, y) + \text{dist}(y, z)$
 $\forall x \in \mathbb{R}. \forall y \in \mathbb{R}. \text{dist}(x, y) = 0 \longrightarrow x = y$
 IsApseudoMetric(`dist`, $\mathbb{R}$, $\mathbb{R}$, Add, ROrd)
 IsAmetric(`dist`, $\mathbb{R}$, $\mathbb{R}$, Add, ROrd)
 <proof>

Real numbers form an ordered loop.

lemma (in reals) reals_loop: shows IsAnOrdLoop($\mathbb{R}$, Add, ROrd)
 <proof>

The assumptions of the `pmetric_space` locale hold in the `reals` locale.

lemma (in reals) pmetric_space_valid: shows pmetric_space($\mathbb{R}$, Add, ROrd, `dist`, $\mathbb{R}$)
 <proof>

The assumptions of the `metric_space` locale hold in the `reals` locale.

lemma (in reals) metric_space_valid: shows metric_space($\mathbb{R}$, Add, ROrd, `dist`, $\mathbb{R}$)
 <proof>

Some properties of the order relation on reals:

lemma (in reals) pos_is_lattice: shows
 IsLinOrder($\mathbb{R}$, ROrd)
 IsLinOrder($\mathbb{R}_+$, ROrd $\cap$ $\mathbb{R}_+ \times \mathbb{R}_+$)
 (ROrd $\cap$ $\mathbb{R}_+ \times \mathbb{R}_+$) {is a lattice on} $\mathbb{R}_+$
 <proof>

Of course the set of positive real numbers is nonempty as one is there.

lemma (in reals) pos_non_empty: shows $\mathbb{R}_+ \neq 0$
 <proof>

We say that a relation r *down-directs* a set R if every two-element subset of R has a lower bound. The next lemma states that the natural order relation on real numbers down-directs the set of positive reals.

lemma (in reals) rord_down_directs: shows ROrd {down-directs} $\mathbb{R}_+$
 <proof>

We define the topology on reals as the metric topology coming from the `dist` metric (i.e. consisting of the unions of open disks).

```
definition (in reals) RealTopology ( $\tau_{\mathbb{R}}$ )
  where  $\tau_{\mathbb{R}} \equiv \text{MetricTopology}(\mathbb{R}, \mathbb{R}, \text{Add}, \text{ROrd}, \text{dist})$ 
```

A more explicit definition of the real topology in notation used in the `reals` context.

```
lemma (in reals) real_topology_def_alt:
  shows  $\tau_{\mathbb{R}} = \{\bigcup A. A \in \text{Pow}(\bigcup c \in \mathbb{R}. \{\text{disk}(c, r). r \in \mathbb{R}_+\})\}$ 
  <proof>
```

Real numbers form a Hausdorff topological space with topology generated by open disks.

```
theorem (in reals) reals_is_top:
  shows  $\tau_{\mathbb{R}} \{\text{is a topology}\} \bigcup \tau_{\mathbb{R}} = \mathbb{R} \ \tau_{\mathbb{R}} \{\text{is } T_2\}$ 
  <proof>
```

end

79 Complex numbers

```
theory Complex_ZF imports func_ZF_1 OrderedField_ZF
```

begin

The goal of this theory is to define complex numbers and prove that the Metamath complex numbers axioms hold.

79.1 From complete ordered fields to complex numbers

This section consists mostly of definitions and a proof context for talking about complex numbers. Suppose we have a set R with binary operations A and M and a relation r such that the quadruple (R, A, M, r) forms a complete ordered field. The next definitions take (R, A, M, r) and construct the sets that represent the structure of complex numbers: the carrier ($\mathbb{C} = R \times R$), binary operations of addition and multiplication of complex numbers and the order relation on $\mathbb{R} = R \times 0$. The `ImCxAdd`, `ReCxAdd`, `ImCxMul`, `ReCxMul` are helper meta-functions representing the imaginary part of a sum of complex numbers, the real part of a sum of real numbers, the imaginary part of a product of complex numbers and the real part of a product of real numbers, respectively. The actual operations (subsets of $(R \times R) \times R$ are named `CplxAdd` and `CplxMul`.

When R is an ordered field, it comes with an order relation. This induces a natural strict order relation on $\{\langle x, 0 \rangle : x \in R\} \subseteq R \times R$. We call the set $\{\langle x, 0 \rangle : x \in R\}$ `ComplexReals(R,A)` and the strict order relation

$\text{CplxROrder}(R, A, r)$. The order on the real axis of complex numbers is defined as the relation induced on it by the canonical projection on the first coordinate and the order we have on the real numbers. OK, lets repeat this slower. We start with the order relation r on a (model of) real numbers R . We want to define an order relation on a subset of complex numbers, namely on $R \times \{0\}$. To do that we use the notion of a relation induced by a mapping. The mapping here is $f : R \times \{0\} \rightarrow R, f\langle x, 0 \rangle = x$ which is defined under a name of `SliceProjection` in `func_ZF.thy`. This defines a relation r_1 (called `InducedRelation(f,r)`, see `func_ZF`) on $R \times \{0\}$ such that $\langle \langle x, 0 \rangle, \langle y, 0 \rangle \in r_1$ iff $\langle x, y \rangle \in r$. This way we get what we call $\text{CplxROrder}(R, A, r)$. However, this is not the end of the story, because Metamath uses strict inequalities in its axioms, rather than weak ones like `IsarMathLib` (mostly). So we need to take the strict version of this order relation. This is done in the syntax definition of $<_{\mathbb{R}}$ in the definition of `complex0` context. Since Metamath proves a lot of theorems about the real numbers extended with $+\infty$ and $-\infty$, we define the notation for inequalities on the extended real line as well.

A helper expression representing the real part of the sum of two complex numbers.

definition

$$\text{ReCxAdd}(R, A, a, b) \equiv A\langle \text{fst}(a), \text{fst}(b) \rangle$$

An expression representing the imaginary part of the sum of two complex numbers.

definition

$$\text{ImCxAdd}(R, A, a, b) \equiv A\langle \text{snd}(a), \text{snd}(b) \rangle$$

The set (function) that is the binary operation that adds complex numbers.

definition

$$\begin{aligned} \text{CplxAdd}(R, A) \equiv \\ \{ \langle p, \langle \text{ReCxAdd}(R, A, \text{fst}(p), \text{snd}(p)), \text{ImCxAdd}(R, A, \text{fst}(p), \text{snd}(p)) \rangle \rangle \mid \\ p \in (R \times R) \times (R \times R) \} \end{aligned}$$

The expression representing the imaginary part of the product of complex numbers.

definition

$$\text{ImCxMul}(R, A, M, a, b) \equiv A\langle M\langle \text{fst}(a), \text{snd}(b) \rangle, M\langle \text{snd}(a), \text{fst}(b) \rangle \rangle$$

The expression representing the real part of the product of complex numbers.

definition

$$\begin{aligned} \text{ReCxMul}(R, A, M, a, b) \equiv \\ A\langle M\langle \text{fst}(a), \text{fst}(b) \rangle, \text{GroupInv}(R, A)(M\langle \text{snd}(a), \text{snd}(b) \rangle) \rangle \end{aligned}$$

The function (set) that represents the binary operation of multiplication of complex numbers.

definition

```
CplxMul(R,A,M) ≡
{ ⟨p, ⟨ReCxMul(R,A,M,fst(p),snd(p)),ImCxMul(R,A,M,fst(p),snd(p))⟩ ⟩ }.
```

```
p ∈ (R×R)×(R×R)}
```

The definition real numbers embedded in the complex plane.

definition

```
ComplexReals(R,A) ≡ R×{TheNeutralElement(R,A)}
```

Definition of order relation on the real line.

definition

```
CplxROrder(R,A,r) ≡
InducedRelation(SliceProjection(ComplexReals(R,A)),r)
```

The next locale defines proof context and notation that will be used for complex numbers.

locale complex0 =

```
fixes R and A and M and r
assumes R_are_reals: IsAmodelOfReals(R,A,M,r)
```

```
fixes complex (C)
defines complex_def[simp]: C ≡ R×R
```

```
fixes rone (1R)
defines rone_def[simp]: 1R ≡ TheNeutralElement(R,M)
```

```
fixes rzero (0R)
defines rzero_def[simp]: 0R ≡ TheNeutralElement(R,A)
```

```
fixes one (1)
defines one_def[simp]: 1 ≡ ⟨1R, 0R⟩
```

```
fixes zero (0)
defines zero_def[simp]: 0 ≡ ⟨0R, 0R⟩
```

```
fixes iunit (i)
defines iunit_def[simp]: i ≡ ⟨0R, 1R⟩
```

```
fixes creal (R)
defines creal_def[simp]: R ≡ {⟨r, 0R⟩. r∈R}
```

```
fixes rmul (infixl · 71)
defines rmul_def[simp]: a · b ≡ M⟨a,b⟩
```

```
fixes radd (infixl + 69)
defines radd_def[simp]: a + b ≡ A⟨a,b⟩
```

```
fixes rneg (- _ 70)
defines rneg_def[simp]: - a ≡ GroupInv(R,A)(a)
```

```

fixes ca (infixl + 69)
defines ca_def[simp]: a + b  $\equiv$  CplxAdd(R,A)⟨a,b⟩

fixes cm (infixl · 71)
defines cm_def[simp]: a · b  $\equiv$  CplxMul(R,A,M)⟨a,b⟩

fixes cdiv (infixl / 70)
defines cdiv_def[simp]: a / b  $\equiv \bigcup \{ x \in \mathbb{C}. b \cdot x = a \}$ 

fixes sub (infixl - 69)
defines sub_def[simp]: a - b  $\equiv \bigcup \{ x \in \mathbb{C}. b + x = a \}$ 

fixes cneg (-_ 95)
defines cneg_def[simp]: - a  $\equiv \mathbf{0} - a$ 

fixes lessr (infix <ℝ 68)
defines lessr_def[simp]:
a <ℝ b  $\equiv \langle a,b \rangle \in \text{StrictVersion}(\text{CplxROrder}(R,A,r))$ 

fixes cpnf (+∞)
defines cpnf_def[simp]: +∞  $\equiv \mathbb{C}$ 

fixes cmnf (-∞)
defines cmnf_def[simp]: -∞  $\equiv \{\mathbb{C}\}$ 

fixes cxr (ℝ*)
defines cxr_def[simp]: ℝ*  $\equiv \mathbb{R} \cup \{+\infty, -\infty\}$ 

fixes cxn (ℕ)
defines cxn_def[simp]:
ℕ  $\equiv \bigcap \{ N \in \text{Pow}(\mathbb{R}). \mathbf{1} \in N \wedge (\forall n. n \in N \longrightarrow n+1 \in N) \}$ 

fixes cltrrset (<)
defines cltrrset_def[simp]:
<  $\equiv \text{StrictVersion}(\text{CplxROrder}(R,A,r)) \cap \mathbb{R} \times \mathbb{R} \cup$ 
 $\{ \langle -\infty, +\infty \rangle \} \cup (\mathbb{R} \times \{+\infty\}) \cup (\{-\infty\} \times \mathbb{R})$ 

fixes cltrr (infix < 68)
defines cltrr_def[simp]: a < b  $\equiv \langle a,b \rangle \in <$ 

fixes lsq (infix ≤ 68)
defines lsq_def[simp]: a ≤ b  $\equiv \neg (b < a)$ 

fixes two (2)
defines two_def[simp]: 2  $\equiv 1 + 1$ 

fixes three (3)
defines three_def[simp]: 3  $\equiv 2+1$ 

```

```

fixes four (4)
defines four_def[simp]:  $4 \equiv 3+1$ 

fixes five (5)
defines five_def[simp]:  $5 \equiv 4+1$ 

fixes six (6)
defines six_def[simp]:  $6 \equiv 5+1$ 

fixes seven (7)
defines seven_def[simp]:  $7 \equiv 6+1$ 

fixes eight (8)
defines eight_def[simp]:  $8 \equiv 7+1$ 

fixes nine (9)
defines nine_def[simp]:  $9 \equiv 8+1$ 

```

79.2 Axioms of complex numbers

In this section we will prove that all Metamath's axioms of complex numbers hold in the `complex0` context.

The next lemma lists some contexts that are valid in the `complex0` context.

```

lemma (in complex0) valid_cntxts: shows
  field1(R,A,M,r)
  field0(R,A,M)
  ring1(R,A,M,r)
  group3(R,A,r)
  ring0(R,A,M)
  M {is commutative on} R
  group0(R,A)
  <proof>

```

The next lemma shows the definition of real and imaginary part of complex sum and product in a more readable form using notation defined in `complex0` locale.

```

lemma (in complex0) cplx_mul_add_defs: shows
  ReCxAdd(R,A,<a,b>,<c,d>) =  $a + c$ 
  ImCxAdd(R,A,<a,b>,<c,d>) =  $b + d$ 
  ImCxMul(R,A,M,<a,b>,<c,d>) =  $a \cdot d + b \cdot c$ 
  ReCxMul(R,A,M,<a,b>,<c,d>) =  $a \cdot c + (-b \cdot d)$ 
  <proof>

```

Real and imaginary parts of sums and products of complex numbers are real.

```

lemma (in complex0) cplx_mul_add_types:

```

```

assumes A1:  $z_1 \in \mathbb{C} \quad z_2 \in \mathbb{C}$ 
shows
   $\text{ReCxAdd}(R, A, z_1, z_2) \in R$ 
   $\text{ImCxAdd}(R, A, z_1, z_2) \in R$ 
   $\text{ImCxMul}(R, A, M, z_1, z_2) \in R$ 
   $\text{ReCxMul}(R, A, M, z_1, z_2) \in R$ 
 $\langle \text{proof} \rangle$ 

```

Complex reals are complex. Recall the definition of R in the `complex0` locale.

```

lemma (in complex0) axresscn: shows  $R \subseteq \mathbb{C}$ 
 $\langle \text{proof} \rangle$ 

```

Complex 1 is not complex 0.

```

lemma (in complex0) ax1ne0: shows  $1 \neq 0$ 
 $\langle \text{proof} \rangle$ 

```

Complex addition is a complex valued binary operation on complex numbers.

```

lemma (in complex0) axaddopr: shows  $\text{CplxAdd}(R, A): \mathbb{C} \times \mathbb{C} \rightarrow \mathbb{C}$ 
 $\langle \text{proof} \rangle$ 

```

Complex multiplication is a complex valued binary operation on complex numbers.

```

lemma (in complex0) axmulopr: shows  $\text{CplxMul}(R, A, M): \mathbb{C} \times \mathbb{C} \rightarrow \mathbb{C}$ 
 $\langle \text{proof} \rangle$ 

```

What are the values of complex addition and multiplication in terms of their real and imaginary parts?

```

lemma (in complex0) cplx_mul_add_vals:
  assumes A1:  $a \in R \quad b \in R \quad c \in R \quad d \in R$ 
  shows
     $\langle a, b \rangle + \langle c, d \rangle = \langle a + c, b + d \rangle$ 
     $\langle a, b \rangle \cdot \langle c, d \rangle = \langle a \cdot c + (-b \cdot d), a \cdot d + b \cdot c \rangle$ 
 $\langle \text{proof} \rangle$ 

```

Complex multiplication is commutative.

```

lemma (in complex0) axmulcom: assumes A1:  $a \in \mathbb{C} \quad b \in \mathbb{C}$ 
  shows  $a \cdot b = b \cdot a$ 
 $\langle \text{proof} \rangle$ 

```

A sum of complex numbers is complex.

```

lemma (in complex0) axaddcl: assumes  $a \in \mathbb{C} \quad b \in \mathbb{C}$ 
  shows  $a + b \in \mathbb{C}$ 
 $\langle \text{proof} \rangle$ 

```

A product of complex numbers is complex.

```

lemma (in complex0) axmulcl: assumes  $a \in \mathbb{C} \quad b \in \mathbb{C}$ 
  shows  $a \cdot b \in \mathbb{C}$ 

```

⟨proof⟩

Multiplication is distributive with respect to addition.

lemma (in complex0) axdistr:
 assumes A1: $a \in \mathbb{C}$ $b \in \mathbb{C}$ $c \in \mathbb{C}$
 shows $a \cdot (b + c) = a \cdot b + a \cdot c$
⟨proof⟩

Complex addition is commutative.

lemma (in complex0) axaddcom: assumes $a \in \mathbb{C}$ $b \in \mathbb{C}$
 shows $a+b = b+a$
⟨proof⟩

Complex addition is associative.

lemma (in complex0) axaddass: assumes A1: $a \in \mathbb{C}$ $b \in \mathbb{C}$ $c \in \mathbb{C}$
 shows $a + b + c = a + (b + c)$
⟨proof⟩

Complex multiplication is associative.

lemma (in complex0) axmulass: assumes A1: $a \in \mathbb{C}$ $b \in \mathbb{C}$ $c \in \mathbb{C}$
 shows $a \cdot b \cdot c = a \cdot (b \cdot c)$
⟨proof⟩

Complex 1 is real. This really means that the pair $\langle 1, 0 \rangle$ is on the real axis.

lemma (in complex0) ax1re: shows $1 \in \mathbb{R}$
⟨proof⟩

The imaginary unit is a "square root" of -1 (that is, $i^2 + 1 = 0$).

lemma (in complex0) axi2m1: shows $i \cdot i + 1 = 0$
⟨proof⟩

0 is the neutral element of complex addition.

lemma (in complex0) ax0id: assumes $a \in \mathbb{C}$
 shows $a + 0 = a$
⟨proof⟩

The imaginary unit is a complex number.

lemma (in complex0) axicn: shows $i \in \mathbb{C}$
⟨proof⟩

All complex numbers have additive inverses.

lemma (in complex0) axnegex: assumes A1: $a \in \mathbb{C}$
 shows $\exists x \in \mathbb{C}. a + x = 0$
⟨proof⟩

A non-zero complex number has a multiplicative inverse.

lemma (in complex0) axrecex: assumes A1: $a \in \mathbb{C}$ and A2: $a \neq 0$

shows $\exists x \in \mathbb{C}. a \cdot x = 1$
 $\langle proof \rangle$

Complex 1 is a right neutral element for multiplication.

lemma (in complex0) ax1id: **assumes** A1: $a \in \mathbb{C}$
shows $a \cdot 1 = a$
 $\langle proof \rangle$

A formula for sum of (complex) real numbers.

lemma (in complex0) sum_of_reals: **assumes** $a \in \mathbb{R} \quad b \in \mathbb{R}$
shows
 $a + b = \langle \text{fst}(a) + \text{fst}(b), 0_R \rangle$
 $\langle proof \rangle$

The sum of real numbers is real.

lemma (in complex0) axaddrcl: **assumes** A1: $a \in \mathbb{R} \quad b \in \mathbb{R}$
shows $a + b \in \mathbb{R}$
 $\langle proof \rangle$

The formula for the product of (complex) real numbers.

lemma (in complex0) prod_of_reals: **assumes** A1: $a \in \mathbb{R} \quad b \in \mathbb{R}$
shows $a \cdot b = \langle \text{fst}(a) \cdot \text{fst}(b), 0_R \rangle$
 $\langle proof \rangle$

The product of (complex) real numbers is real.

lemma (in complex0) axmulrcl: **assumes** $a \in \mathbb{R} \quad b \in \mathbb{R}$
shows $a \cdot b \in \mathbb{R}$
 $\langle proof \rangle$

The existence of a real negative of a real number.

lemma (in complex0) axrnegex: **assumes** A1: $a \in \mathbb{R}$
shows $\exists x \in \mathbb{R}. a + x = 0$
 $\langle proof \rangle$

Each nonzero real number has a real inverse

lemma (in complex0) axrrecex:
assumes A1: $a \in \mathbb{R} \quad a \neq 0$
shows $\exists x \in \mathbb{R}. a \cdot x = 1$
 $\langle proof \rangle$

Our $\mathbb{R}$ symbol is the real axis on the complex plane.

lemma (in complex0) real_means_real_axis: **shows** $\mathbb{R} = \text{ComplexReals}(R, A)$
 $\langle proof \rangle$

The CplxROrder thing is a relation on the complex reals.

lemma (in complex0) cplx_ord_on_cplx_reals:
shows $\text{CplxROrder}(R, A, r) \subseteq \mathbb{R} \times \mathbb{R}$

⟨proof⟩

The strict version of the complex relation is a relation on complex reals.

lemma (in complex0) cplx_strict_ord_on_cplx_reals:
 shows StrictVersion(CplxROrder(R,A,r)) $\subseteq \mathbb{R} \times \mathbb{R}$
⟨proof⟩

The CplxROrder thing is a relation on the complex reals. Here this is formulated as a statement that in complex0 context $a < b$ implies that a, b are complex reals

lemma (in complex0) strict_cplx_ord_type: assumes $a <_{\mathbb{R}} b$
 shows $a \in \mathbb{R} \quad b \in \mathbb{R}$
⟨proof⟩

A more readable version of the definition of the strict order relation on the real axis. Recall that in the complex0 context r denotes the (non-strict) order relation on the underlying model of real numbers.

lemma (in complex0) def_of_real_axis_order: shows
 $\langle x, 0_R \rangle <_{\mathbb{R}} \langle y, 0_R \rangle \longleftrightarrow \langle x, y \rangle \in r \wedge x \neq y$
⟨proof⟩

The (non strict) order on complex reals is antisymmetric, transitive and total.

lemma (in complex0) cplx_ord_antsym_trans_tot: shows
 antisym(CplxROrder(R,A,r))
 trans(CplxROrder(R,A,r))
 CplxROrder(R,A,r) {is total on} $\mathbb{R}$
⟨proof⟩

The trichotomy law for the strict order on the complex reals.

lemma (in complex0) cplx_strict_ord_trich:
 assumes $a \in \mathbb{R} \quad b \in \mathbb{R}$
 shows Exactly_1_of_3_holds($a <_{\mathbb{R}} b$, $a = b$, $b <_{\mathbb{R}} a$)
⟨proof⟩

The strict order on the complex reals is kind of antisymmetric.

lemma (in complex0) pre_axlttri: assumes A1: $a \in \mathbb{R} \quad b \in \mathbb{R}$
 shows $a <_{\mathbb{R}} b \longleftrightarrow \neg(a = b \vee b <_{\mathbb{R}} a)$
⟨proof⟩

The strict order on complex reals is transitive.

lemma (in complex0) cplx_strict_ord_trans:
 shows trans(StrictVersion(CplxROrder(R,A,r)))
⟨proof⟩

The strict order on complex reals is transitive - the explicit version of cplx_strict_ord_trans.

```

lemma (in complex0) pre_axlttrn:
  assumes A1:  $a <_{\mathbb{R}} b$      $b <_{\mathbb{R}} c$ 
  shows  $a <_{\mathbb{R}} c$ 
<proof>

```

The strict order on complex reals is preserved by translations.

```

lemma (in complex0) pre_axltadd:
  assumes A1:  $a <_{\mathbb{R}} b$  and A2:  $c \in \mathbb{R}$ 
  shows  $c+a <_{\mathbb{R}} c+b$ 
<proof>

```

The set of positive complex reals is closed with respect to multiplication.

```

lemma (in complex0) pre_axmulgt0: assumes A1:  $0 <_{\mathbb{R}} a$      $0 <_{\mathbb{R}} b$ 
  shows  $0 <_{\mathbb{R}} a \cdot b$ 
<proof>

```

The order on complex reals is linear and complete.

```

lemma (in complex0) cmplx_reals_ord_lin_compl: shows
  CplxROrder(R,A,r) {is complete}
  IsLinOrder( $\mathbb{R}$ ,CplxROrder(R,A,r))
<proof>

```

The property of the strict order on complex reals that corresponds to completeness.

```

lemma (in complex0) pre_axsup: assumes A1:  $X \subseteq \mathbb{R}$      $X \neq 0$  and
  A2:  $\exists x \in \mathbb{R}. \forall y \in X. y <_{\mathbb{R}} x$ 
  shows
     $\exists x \in \mathbb{R}. (\forall y \in X. \neg(x <_{\mathbb{R}} y)) \wedge (\forall y \in \mathbb{R}. (y <_{\mathbb{R}} x \longrightarrow (\exists z \in X. y <_{\mathbb{R}} z)))$ 
<proof>

```

end

80 Rings - Zariski Topology

```

theory Ring_Zariski_ZF imports Ring_ZF_2 Topology_ZF Topology_ZF_1

```

begin

This file deals with the definition of the topology on the set of prime ideals. It defines the topology, computes the closed sets and the closure and interior operators.

80.1 Spectrum

The prime spectrum of a ring is the set of its prime ideals, which serves as the carrier set of the Zariski topology.

We define the spectrum as the collection of prime ideals of the ring.

definition (in ring0) Spec where
 $\text{Spec} \equiv \{I \in \mathcal{I}. I \triangleleft_p R\}$

80.2 Zariski topology

The Zariski topology on the spectrum is generated by D-sets, one for each ideal of the ring.

The basic set that defines the topology is given by the D operator

definition (in ring0) openBasic (D) where
 $S \subseteq R \implies D(S) \equiv \{I \in \text{Spec}. \neg(S \subseteq I)\}$

The D operator preserves subsets

lemma (in ring0) D_operator_preserve_subset:
 assumes $S \subseteq T \quad T \subseteq R$
 shows $D(S) \subseteq D(T)$
<proof>

The D operator values can be obtained by considering only ideals. This is useful as we have operations on ideals that we do not have on subsets.

lemma (in ring0) D_operator_only_ideals:
 assumes $T \subseteq R$
 shows $D(T) = D(\langle T \rangle_I)$
<proof>

The intersection of two D-sets is the D-set on the product of ideals

lemma (in ring0) intersection_open_basic:
 assumes $I \triangleleft R \quad J \triangleleft R$
 shows $D(I) \cap D(J) = D(I \cdot_I J)$
<proof>

The union of D-sets is the D-set of the sum of the ideals

lemma (in ring0) union_open_basic:
 assumes $\mathcal{J} \subseteq \mathcal{I}$
 shows $\bigcup \{D(I). I \in \mathcal{J}\} = D(\oplus_I \mathcal{J})$
<proof>

From the previous results on intersection and union, we conclude that the D-sets we computed form a topology

corollary (in ring0) zariski_top:
 shows $\{D(J). J \in \mathcal{I}\}$ is a topology
<proof>

We include all the results of topology0 into ring0 under the namespace "zariski"

abbreviation (in ring0) ZarInt (int) where
 $\text{int}(U) \equiv \text{Interior}(U, \{D(J) \mid J \in \mathcal{I}\})$

We define the Zariski closure operator.

abbreviation (in ring0) ZarCl (cl) where
 $\text{cl}(U) \equiv \text{Closure}(U, \{D(J) \mid J \in \mathcal{I}\})$

We define the Zariski boundary operator.

abbreviation (in ring0) ZarBound (∂ _) where
 $\partial U \equiv \text{Boundary}(U, \{D(J) \mid J \in \mathcal{I}\})$

The ring locale specializes the topology0 locale under the Zariski topology.

sublocale ring0 < zariski:topology0 {D(J) . J ∈ $\mathcal{I}$ }

$\lambda U. \text{Interior}(U, \{D(J) \mid J \in \mathcal{I}\})$
 $\lambda U. \text{Closure}(U, \{D(J) \mid J \in \mathcal{I}\})$
 $\lambda U. \text{Boundary}(U, \{D(J) \mid J \in \mathcal{I}\})$
<proof>

The interior of a proper subset is given by the D-set of the intersection of all the prime ideals not in that subset

lemma (in ring0) interior_zariski:
 $\text{assumes } U \subseteq \text{Spec } R$
 $\text{shows } \text{int}(U) = D(\bigcap (\text{Spec} - U))$
<proof>

80.3 Closed sets and boundary

We characterize the closed sets and study the closure operator in the Zariski topology.

The whole space is the D-set of the ring as an ideal of itself

lemma (in ring0) openBasic_total:
 $\text{shows } D(R) = \text{Spec } R$
<proof>

The union of all D-sets equals the whole spectrum.

corollary (in ring0) total_spec:
 $\text{shows } \bigcup \{D(J) \mid J \in \mathcal{I}\} = \text{Spec } R$
<proof>

The empty set is the D-set of the zero ideal

lemma (in ring0) openBasic_empty:
 $\text{shows } D(\{0\}) = \emptyset$
<proof>

A closed set is a set of primes containing a given ideal

lemma (in ring0) closeBasic:

assumes $U\{\text{is closed in}\}\{D(J). J \in \mathcal{I}\}$
obtains J **where** $J \in \mathcal{I}$ **and** $U = \{K \in \text{Spec}. J \subseteq K\}$
 $\langle \text{proof} \rangle$

We define the closed sets as V-sets

definition (in ring0) closeBasic (V) **where**
 $S \subseteq R \implies V(S) = \{K \in \text{Spec}. S \subseteq K\}$

V-sets and D-sets are complementary

lemma (in ring0) V_is_closed:
assumes $J \in \mathcal{I}$
shows $\text{Spec} - V(J) = D(J)$ **and** $V(J)\{\text{is closed in}\}\{D(J). J \in \mathcal{I}\}$
 $\langle \text{proof} \rangle$

As with D-sets, by De Morgan's Laws we get the same result for unions and intersections on V-sets

lemma (in ring0) V_union:
assumes $J \in \mathcal{I} \ K \in \mathcal{I}$
shows $V(J) \cup V(K) = V(J \cdot_I K)$
 $\langle \text{proof} \rangle$

The intersection of a family of V-sets is the V-set of the sum of the corresponding ideals.

lemma (in ring0) V_intersect:
assumes $\mathcal{J} \subseteq \mathcal{I} \ \mathcal{J} \neq \emptyset$
shows $\bigcap \{V(I). I \in \mathcal{J}\} = V(\bigoplus_I \mathcal{J})$
 $\langle \text{proof} \rangle$

The closure of a set is the V-set of the intersection of all its points.

lemma (in ring0) closure_zariski:
assumes $U \subseteq \text{Spec} \ U \neq \emptyset$
shows $\text{cl}(U) = V(\bigcap U)$
 $\langle \text{proof} \rangle$

80.4 Properties of Spec

In this section we study the topological properties of Spec related to the ring structure.

All Spec topologies are T_0 .

theorem (in ring0) zariski_t0:
shows $\{D(I). I \in \mathcal{I}\}\{\text{is } T_0\}$
 $\langle \text{proof} \rangle$

Noetherian rings have compact Zariski topology

theorem (in ring0) zariski_compact:
assumes $\forall I \in \mathcal{I}. (I\{\text{is finitely generated}\})$

```

    shows Spec {is compact in} {D(I). I ∈ I}
  <proof>

```

```

end

```

81 Ring homomorphisms as continuous maps on Spec

```

theory Ring_Zariski_ZF_1 imports Ring_Zariski_ZF Ring_ZF_3 Topology_ZF_2

```

```

begin

```

Every ring homomorphism $f : R \rightarrow S$ induces a map $f^* : \text{Spec}(S) \rightarrow \text{Spec}(R)$ on prime spectra by pulling back prime ideals. This file shows that f^* is continuous with respect to the Zariski topologies on $\text{Spec}(R)$ and $\text{Spec}(S)$, establishing Spec as a contravariant functor from commutative rings to topological spaces.

81.1 Induced maps on spectra

A surjective ring homomorphism induces a map between the prime spectra that lands in the closed set defined by the kernel.

```

lemma (in ring_homo) spectrum_surj:
  defines g ≡ λu∈target_ring.Spec. f-u
  assumes f∈surj(R,S)
  shows g: target_ring.Spec → V(ker)
  <proof>

```

The induced map on spectra is a bijection onto the V-set of the kernel.

```

lemma (in ring_homo) spectrum_surj_bij:
  defines g ≡ λu∈target_ring.Spec. f-u
  assumes f∈surj(R,S)
  shows g∈bij(target_ring.Spec, V(ker))
  <proof>

```

81.2 Continuity of the induced map

We define the Zariski topologies on origin and target spectra, then show the induced map is continuous and open.

The Zariski topology on the origin ring's spectrum.

```

definition (in ring_homo) top_origin (τo) where
  top_origin ≡ {origin_ring.openBasic(J) . J ∈ origin_ring.ideals}

```

The Zariski topology on the target ring's spectrum.

```

definition (in ring_homo) top_target (τt) where

```

$\text{top_target} \equiv \{\text{target_ring.openBasic}(J) \mid J \in \text{target_ring.ideals}\}$

Continuity of a map between spectra with respect to the Zariski topologies.

definition (in ring_homo) spec_cont where
 $\text{spec_cont}(h) \equiv \text{IsContinuous}(\tau_t, \tau_o, h)$

The induced map on spectra is continuous with respect to the Zariski topologies.

lemma (in ring_homo) spectrum_surj_cont:
 defines $g \equiv \lambda u \in \text{target_ring.Spec}. f \cdot u$
 assumes $f \in \text{surj}(R, S)$
 shows $\text{IsContinuous}(\tau_t, \tau_o \text{ {restricted to}} V(\ker)), g)$
<proof>

The induced map on spectra sends open sets to open sets of the restricted topology.

lemma (in ring_homo) spectrum_surj_open:
 defines $g \equiv \lambda u \in \text{target_ring.Spec}. f \cdot u$
 assumes $f \in \text{surj}(R, S)$
 shows $\forall U \in \tau_t. gU \in \tau_o \text{ {restricted to}} V(\ker)$
<proof>

A quotient ring has a spectrum homeomorphic to a closed subspace of the spectrum of the base ring. Specifically, the closed subspace associated to the ideal by which we quotient.

corollary (in ring_homo) surj_homeomorphism:
 assumes $f \in \text{surj}(R, S)$
 defines $g \equiv \lambda u \in \text{target_ring.Spec}. f \cdot u$
 shows $\text{IsAhomeomorphism}(\tau_t, \tau_o \text{ {restricted to}} V(\ker), g)$
<proof>

end

82 Topology 3

theory Topology_ZF_3 imports Topology_ZF_2 FiniteSeq_ZF

begin

Topology_ZF_1 theory describes how we can define a topology on a product of two topological spaces. One way to generalize that is to construct topology for a cartesian product of n topological spaces. The cartesian product approach is somewhat inconvenient though. Another way to approach product topology on X^n is to model cartesian product as sets of sequences (of length n) of elements of X . This means that having a topology on X we want to define a topology on the space $n \rightarrow X$, where n is a natural number (recall that $n = \{0, 1, \dots, n-1\}$ in ZF). However, this in turn can be done

more generally by defining a topology on any function space $I \rightarrow X$, where I is any set of indices. This is what we do in this theory.

82.1 The base of the product topology

In this section we define the base of the product topology.

Suppose $\mathcal{X} = I \rightarrow \bigcup T$ is a space of functions from some index set I to the carrier of a topology T . Then take a finite collection of open sets $W : N \rightarrow T$ indexed by $N \subseteq I$. We can define a subset of $\mathcal{X}$ that models the cartesian product of W .

definition

$$\text{FinProd}(\mathcal{X}, W) \equiv \{x \in \mathcal{X}. \forall i \in \text{domain}(W). x(i) \in W(i)\}$$

Now we define the base of the product topology as the collection of all finite products (in the sense defined above) of open sets.

definition

$$\text{ProductTopBase}(I, T) \equiv \bigcup_{N \in \text{FinPow}(I)} \{\text{FinProd}(I \rightarrow \bigcup T, W). W \in N \rightarrow T\}$$

Finally, we define the product topology on sequences. We use the "Seq" prefix although the definition is good for any index sets, not only natural numbers.

definition

$$\text{SeqProductTopology}(I, T) \equiv \{\bigcup B. B \in \text{Pow}(\text{ProductTopBase}(I, T))\}$$

Product topology base is closed with respect to intersections.

lemma prod_top_base_inter:

assumes A1: T {is a topology} **and**
A2: $U \in \text{ProductTopBase}(I, T) \quad V \in \text{ProductTopBase}(I, T)$
shows $U \cap V \in \text{ProductTopBase}(I, T)$

<proof>

In the next theorem we show the collection of sets defined above as $\text{ProductTopBase}(\mathcal{X}, T)$ satisfies the base condition. This is a condition, defined in `Topology_ZF_1` that allows to claim that this collection is a base for some topology.

theorem prod_top_base_is_base: **assumes** T {is a topology}

shows $\text{ProductTopBase}(I, T)$ {satisfies the base condition}

<proof>

The (sequence) product topology is indeed a topology on the space of sequences. In the proof we are using the fact that $(\emptyset \rightarrow X) = \{\emptyset\}$.

theorem seq_prod_top_is_top: **assumes** T {is a topology}

shows

$\text{SeqProductTopology}(I, T)$ {is a topology} **and**

$\text{ProductTopBase}(I, T)$ {is a base for} $\text{SeqProductTopology}(I, T)$ **and**

$\bigcup \text{SeqProductTopology}(I, T) = (I \rightarrow \bigcup T)$

<proof>

82.2 Finite product of topologies

As a special case of the space of functions $I \rightarrow X$ we can consider space of lists of elements of X , i.e. space $n \rightarrow X$, where n is a natural number (recall that in ZF set theory $n = \{0, 1, \dots, n-1\}$). Such spaces model finite cartesian products X^n but are easier to deal with in formalized way (than the said products). This section discusses natural topology defined on $n \rightarrow X$ where X is a topological space.

When the index set is finite, the definition of `ProductTopBase(I,T)` can be simplified.

```
lemma fin_prod_def_nat: assumes A1: n∈nat and A2: T {is a topology}
  shows ProductTopBase(n,T) = {FinProd(n→⋃T,W). W∈n→T}
  <proof>
```

A technical lemma providing a formula for finite product on one topological space.

```
lemma single_top_prod: assumes A1: W:1→τ
  shows FinProd(1→⋃τ,W) = { {⟨0,y⟩}. y ∈ W(0) }
  <proof>
```

Intuitively, the topological space of singleton lists valued in X is the same as X . However, each element of this space is a list of length one, i.e a set consisting of a pair $\langle 0, x \rangle$ where x is an element of X . The next lemma provides a formula for the product topology in the corner case when we have only one factor and shows that the product topology of one space is essentially the same as the space.

```
lemma singleton_prod_top: assumes A1: τ {is a topology}
  shows
    SeqProductTopology(1,τ) = { { {⟨0,y⟩}. y∈U }. U∈τ } and
    IsAhomeomorphism(τ,SeqProductTopology(1,τ),{⟨y,{⟨0,y⟩}⟩.y ∈ ⋃τ})
  <proof>
```

A special corner case of `finite_top_prod_homeo`: a space X is homeomorphic to the space of one element lists of X .

```
theorem singleton_prod_top1: assumes A1: τ {is a topology}
  shows IsAhomeomorphism(SeqProductTopology(1,τ),τ,{⟨x,x(0)⟩. x∈1→⋃τ})
  <proof>
```

A technical lemma describing the carrier of a (cartesian) product topology of the (sequence) product topology of n copies of topology τ and another copy of τ .

```
lemma finite_prod_top: assumes τ {is a topology} and T = SeqProductTopology(n,τ)
  shows (⋃ProductTopology(T,τ)) = (n→⋃τ)×⋃τ
  <proof>
```

If U is a set from the base of X^n and V is open in X , then $U \times V$ is in the base of X^{n+1} . The next lemma is an analogue of this fact for the function space approach.

lemma `finite_prod_succ_base`: **assumes** $A1$: τ {is a topology} **and** $A2$: $n \in \text{nat}$ **and**
 $A3$: $U \in \text{ProductTopBase}(n, \tau)$ **and** $A4$: $V \in \tau$
shows $\{x \in \text{succ}(n) \rightarrow \bigcup \tau. \text{Init}(x) \in U \wedge x(n) \in V\} \in \text{ProductTopBase}(\text{succ}(n), \tau)$
<proof>

If U is open in X^n and V is open in X , then $U \times V$ is open in X^{n+1} . The next lemma is an analogue of this fact for the function space approach.

lemma `finite_prod_succ`: **assumes** $A1$: τ {is a topology} **and** $A2$: $n \in \text{nat}$ **and**
 $A3$: $U \in \text{SeqProductTopology}(n, \tau)$ **and** $A4$: $V \in \tau$
shows $\{x \in \text{succ}(n) \rightarrow \bigcup \tau. \text{Init}(x) \in U \wedge x(n) \in V\} \in \text{SeqProductTopology}(\text{succ}(n), \tau)$
<proof>

In the `Topology_ZF_2` theory we define product topology of two topological spaces. The next lemma explains in what sense the topology on finite lists of length n of elements of topological space X can be thought as a model of the product topology on the cartesian product of n copies of that space. Namely, we show that the space of lists of length $n + 1$ of elements of X is homeomorphic to the product topology (as defined in `Topology_ZF_2`) of two spaces: the space of lists of length n and X . Recall that if $\mathcal{B}$ is a base (i.e. satisfies the base condition), then the collection $\{\bigcup B \mid B \in \text{Pow}(\mathcal{B})\}$ is a topology (generated by $\mathcal{B}$).

theorem `finite_top_prod_homeo`: **assumes** $A1$: τ {is a topology} **and** $A2$: $n \in \text{nat}$ **and**
 $A3$: $f = \{\langle x, \langle \text{Init}(x), x(n) \rangle \rangle. x \in \text{succ}(n) \rightarrow \bigcup \tau\}$ **and**
 $A4$: $T = \text{SeqProductTopology}(n, \tau)$ **and**
 $A5$: $S = \text{SeqProductTopology}(\text{succ}(n), \tau)$
shows $\text{IsAhomeomorphism}(S, \text{ProductTopology}(T, \tau), f)$
<proof>

end

83 Relation between nets and filters

theory `Topology_ZF_4b` **imports** `Topology_ZF_4`
begin

This theory continues `Topology_ZF_4` with results relating the net and filter convergence notions introduced there.

83.1 Relation between nets and filters

In this section we show that filters do not generalize nets, but still nets and filters are in a way equivalent as far as convergence is considered.

Let's build now a net from a filter, such that both converge to the same points.

definition

`NetOfFilter (Net(_) 40) where`
`$\mathcal{F}$  {is a filter on}  $\bigcup \mathcal{F} \implies \text{Net}(\mathcal{F}) \equiv$`
`$\langle \{ \langle A, \text{fst}(A) \rangle. A \in \{ \langle x, F \rangle \in (\bigcup \mathcal{F}) \times \mathcal{F}. x \in F \} \}, \{ \langle A, B \rangle \in \{ \langle x, F \rangle \in (\bigcup \mathcal{F}) \times \mathcal{F}. x \in F \} \times \{ \langle x, F \rangle \in (\bigcup \mathcal{F}) \times \mathcal{F}. x \in F \}. \text{snd}(B) \subseteq \text{snd}(A) \} \}$`

Net of a filter is indeed a net.

theorem `net_of_filter_is_net:`
`assumes  $\mathcal{F}$  {is a filter on}  $X$`
`shows  $(\text{Net}(\mathcal{F}))$  {is a net on}  $X$`
`$\langle \text{proof} \rangle$`

If a filter converges to some point then its net converges to the same point.

theorem `(in topology0) filter_conver_net_of_filter_conver:`
`assumes  $\mathcal{F}$  {is a filter on}  $\bigcup T$  and  $\mathcal{F} \rightarrow_F x$`
`shows  $(\text{Net}(\mathcal{F})) \rightarrow_N x$`
`$\langle \text{proof} \rangle$`

If a net converges to a point, then a filter also converges to a point.

theorem `(in topology0) net_of_filter_conver_filter_conver:`
`assumes  $\mathcal{F}$  {is a filter on}  $\bigcup T$  and  $(\text{Net}(\mathcal{F})) \rightarrow_N x$`
`shows  $\mathcal{F} \rightarrow_F x$`
`$\langle \text{proof} \rangle$`

A filter converges to a point if and only if its net converges to the point.

theorem `(in topology0) filter_conver_iff_net_of_filter_conver:`
`assumes  $\mathcal{F}$  {is a filter on}  $\bigcup T$`
`shows  $(\mathcal{F} \rightarrow_F x) \longleftrightarrow ((\text{Net}(\mathcal{F})) \rightarrow_N x)$`
`$\langle \text{proof} \rangle$`

The previous result states that, when considering convergence, the filters do not generalize nets. When considering a filter, there is always a net that converges to the same points of the original filter.

Now we see that with nets, results come naturally applying the axiom of choice; but with filters, the results come, may be less natural, but with no choice. The reason is that $\text{Net}(\mathcal{F})$ is a net that doesn't come into our attention as a first choice; maybe because we restrict ourselves to the anti-symmetry property of orders without realizing that a directed set is not an order.

The following results will state that filters are not just a subclass of nets, but that nets and filters are equivalent on convergence: for every filter there is a net converging to the same points, and also, for every net there is a filter converging to the same points.

definition

FilterOfNet (Filter (_ .. _) 40) **where**
 $(N \text{ \{is a net on\} } X) \implies \text{Filter } N..X \equiv \{A \in \text{Pow}(X). \exists D \in \{\{fst(N)snd(s). s \in \{s \in \text{domain}(fst(N)) \times \text{domain}(fst(N)). s \in snd(N) \wedge fst(s)=t0\}\}. t0 \in \text{domain}(fst(N))\}. D \subseteq A\}$

Filter of a net is indeed a filter

theorem filter_of_net_is_filter:

assumes $N \text{ \{is a net on\} } X$
shows $(\text{Filter } N..X) \text{ \{is a filter on\} } X$ **and**
 $\{\{fst(N)snd(s). s \in \{s \in \text{domain}(fst(N)) \times \text{domain}(fst(N)). s \in snd(N) \wedge fst(s)=t0\}\}. t0 \in \text{domain}(fst(N))\} \text{ \{is a base filter\} } (\text{Filter } N..X)$
<proof>

Convergence of a net implies the convergence of the corresponding filter.

theorem (in topology0) net_conver_filter_of_net_conver:

assumes $N \text{ \{is a net on\} } \bigcup T$ **and** $N \rightarrow_N x$
shows $(\text{Filter } N..(\bigcup T)) \rightarrow_F x$
<proof>

Convergence of a filter corresponding to a net implies convergence of the net.

theorem (in topology0) filter_of_net_conver_net_conver:

assumes $N \text{ \{is a net on\} } \bigcup T$ **and** $(\text{Filter } N..(\bigcup T)) \rightarrow_F x$
shows $N \rightarrow_N x$
<proof>

Filter of net converges to a point x if and only the net converges to x .

theorem (in topology0) filter_of_net_conv_iff_net_conv:

assumes $N \text{ \{is a net on\} } \bigcup T$
shows $((\text{Filter } N..(\bigcup T)) \rightarrow_F x) \iff (N \rightarrow_N x)$
<proof>

We know now that filters and nets are the same thing, when working convergence of topological spaces. Sometimes, the nature of filters makes it easier to generalize them as follows.

Instead of considering all subsets of some set X , we can consider only open sets (we get an open filter) or closed sets (we get a closed filter). There are many more useful examples that characterize topological properties.

This type of generalization cannot be done with nets.

Also a filter can give us a topology in the following way:

theorem top_of_filter:

```

    assumes  $\mathcal{F}$  {is a filter on}  $\bigcup \mathcal{F}$ 
    shows  $(\mathcal{F} \cup \{0\})$  {is a topology}
  <proof>

```

We can use `topology0` locale with filters.

```

lemma topology0_filter:
  assumes  $\mathcal{F}$  {is a filter on}  $\bigcup \mathcal{F}$ 
  shows topology0( $\mathcal{F} \cup \{0\}$ )
  <proof>

```

The next abbreviation introduces notation where we want to specify the space where the filter convergence takes place.

```

abbreviation FilConvTop( $\_ \rightarrow_F \_ \{in\} \_$ )
  where  $\mathcal{F} \rightarrow_F x \{in\} T \equiv \text{topology0.FilterConverges}(T, \mathcal{F}, x)$ 

```

The next abbreviation introduces notation where we want to specify the space where the net convergence takes place.

```

abbreviation NetConvTop( $\_ \rightarrow_N \_ \{in\} \_$ )
  where  $N \rightarrow_N x \{in\} T \equiv \text{topology0.NetConverges}(T, N, x)$ 

```

Each point of the union of a filter is a limit of that filter.

```

lemma lim_filter_top_of_filter:
  assumes  $\mathcal{F}$  {is a filter on}  $\bigcup \mathcal{F}$  and  $x \in \bigcup \mathcal{F}$ 
  shows  $\mathcal{F} \rightarrow_F x \{in\} (\mathcal{F} \cup \{0\})$ 
  <proof>

```

end

84 Topology - examples

```

theory Topology_ZF_examples imports Topology_ZF Cardinal_ZF

```

```

begin

```

This theory deals with some concrete examples of topologies.

84.1 CoCardinal Topology

In this section we define and prove the basic properties of the co-cardinal topology on a set X .

The collection of subsets of a set whose complement is strictly bounded by a cardinal is a topology given some assumptions on the cardinal.

definition

```

CoCardinal( $X, T$ )  $\equiv \{F \in \text{Pow}(X) . X - F \prec T\} \cup \{0\}$ 

```

For any set and any infinite cardinal we prove that $\text{CoCardinal}(X, Q)$ forms a topology. The proof is done with an infinite cardinal, but it is obvious that the set Q can be any set equipollent with an infinite cardinal. It is a topology also if the set where the topology is defined is too small or the cardinal too large; in this case, as it is later proved the topology is a discrete topology. And the last case corresponds with $Q=1$ which translates in the indiscrete topology.

```
lemma CoCar_is_topology:
  assumes InfCard (Q)
  shows CoCardinal(X,Q) {is a topology}
  <proof>
```

We can use theorems proven in `topology0` context for the co-cardinal topology.

```
theorem topology0_CoCardinal:
  assumes InfCard(T)
  shows topology0(CoCardinal(X,T))
  <proof>
```

It can also be proven that if $\text{CoCardinal}(X, T)$ is a topology, $X \neq 0$, $\text{Card}(T)$ and $T \neq 0$; then T is an infinite cardinal, $X \prec T$ or $T=1$. It follows from the fact that the union of two closed sets is closed. Choosing the appropriate cardinals, the cofinite and the cocountable topologies are obtained.

The cofinite topology is a very special topology because it is closely related to the separation axiom T_1 . It also appears naturally in algebraic geometry.

```
definition
  Cofinite (CoFinite _ 90) where
  CoFinite X  $\equiv$  CoCardinal(X,nat)
```

Cocountable topology in fact consists of the empty set and all cocountable subsets of X .

```
definition
  Cocountable (CoCountable _ 90) where
  CoCountable X  $\equiv$  CoCardinal(X,csucc(nat))
```

84.2 Total set, Closed sets, Interior, Closure and Boundary

There are several assertions that can be done to the $\text{CoCardinal}(X, T)$ topology. In each case, we will not assume sufficient conditions for $\text{CoCardinal}(X, T)$ to be a topology, but they will be enough to do the calculations in every possible case.

The topology is defined in the set X

```
lemma union_cocardinal:
  assumes  $T \neq 0$ 
```

shows $\bigcup \text{CoCardinal}(X,T) = X$
<proof>

The closed sets are the small subsets of X and X itself.

lemma `closed_sets_cocardinal`:
assumes $T \neq 0$
shows $D \text{ \{is closed in\} CoCardinal}(X,T) \longleftrightarrow (D \in \text{Pow}(X) \wedge D \prec T) \vee D = X$
<proof>

The interior of a set is itself if it is open or 0 if it isn't open.

lemma `interior_set_cocardinal`:
assumes $\text{noC}: T \neq 0 \text{ and } A \subseteq X$
shows $\text{Interior}(A, \text{CoCardinal}(X,T)) = (\text{if } ((X-A) \prec T) \text{ then } A \text{ else } 0)$
<proof>

X is a closed set that contains A . This lemma is necessary because we cannot use the lemmas proven in the `topology0` context since $T \neq 0$ is too weak for $\text{CoCardinal}(X,T)$ to be a topology.

lemma `X_closedcov_cocardinal`:
assumes $T \neq 0 \text{ and } A \subseteq X$
shows $X \in \text{ClosedCovers}(A, \text{CoCardinal}(X,T))$ *<proof>*

The closure of a set is itself if it is closed or X if it isn't closed.

lemma `closure_set_cocardinal`:
assumes $T \neq 0 \text{ and } A \subseteq X$
shows $\text{Closure}(A, \text{CoCardinal}(X,T)) = (\text{if } (A \prec T) \text{ then } A \text{ else } X)$
<proof>

The boundary of a set is empty if A and $X - A$ are closed, X if not A neither $X - A$ are closed and; if only one is closed, then the closed one is its boundary.

lemma `boundary_cocardinal`:
assumes $T \neq 0 \text{ and } A \subseteq X$
shows $\text{Boundary}(A, \text{CoCardinal}(X,T)) = (\text{if } A \prec T \text{ then } (\text{if } (X-A) \prec T \text{ then } 0 \text{ else } A) \text{ else } (\text{if } (X-A) \prec T \text{ then } X-A \text{ else } X))$
<proof>

If the set is too small or the cardinal too large, then the topology is just the discrete topology.

lemma `discrete_cocardinal`:
assumes $X \prec T$
shows $\text{CoCardinal}(X,T) = \text{Pow}(X)$
<proof>

If the cardinal is taken as $T=1$ then the topology is indiscrete.

lemma `indiscrete_cocardinal`:
shows $\text{CoCardinal}(X,1) = \{0, X\}$

<proof>

The topological subspaces of the $\text{CoCardinal}(X, T)$ topology are also CoCardinal topologies.

lemma `subspace_cocardinal:`

`shows CoCardinal(X, T) {restricted to} Y = CoCardinal(Y ∩ X, T)`

<proof>

84.3 Excluded Set Topology

In this section, we consider all the subsets of a set which have empty intersection with a fixed set.

The excluded set topology consists of subsets of X that are disjoint with a fixed set U .

definition $\text{ExcludedSet}(X, U) \equiv \{F \in \text{Pow}(X) . U \cap F = \emptyset\} \cup \{X\}$

For any set; we prove that $\text{ExcludedSet}(X, Q)$ forms a topology.

theorem `excludedset_is_topology:`

`shows ExcludedSet(X, Q) {is a topology}`

<proof>

We can use `topology0` when discussing excluded set topology.

theorem `topology0_excludedset:`

`shows topology0(ExcludedSet(X, T))`

<proof>

Choosing a singleton set, it is considered a point in excluded topology.

definition

$\text{ExcludedPoint}(X, p) \equiv \text{ExcludedSet}(X, \{p\})$

84.4 Total set, closed sets, interior, closure and boundary

Here we discuss what are closed sets, interior, closure and boundary in excluded set topology.

The topology is defined in the set X

lemma `union_excludedset:`

`shows  $\bigcup \text{ExcludedSet}(X, T) = X$`

<proof>

The closed sets are those which contain the set $(X \cap T)$ and $\emptyset$.

lemma `closed_sets_excludedset:`

`shows  $D \text{ {is closed in} } \text{ExcludedSet}(X, T) \longleftrightarrow (D \in \text{Pow}(X) \wedge (X \cap T) \subseteq D) \vee D = \emptyset$`

<proof>

The interior of a set is itself if it is X or the difference with the set T

lemma interior_set_excludedset:
 assumes $A \subseteq X$
 shows $\text{Interior}(A, \text{ExcludedSet}(X, T)) = (\text{if } A=X \text{ then } X \text{ else } A-T)$
<proof>

The closure of a set is itself if it is $\emptyset$ or the union with T .

lemma closure_set_excludedset:
 assumes $A \subseteq X$
 shows $\text{Closure}(A, \text{ExcludedSet}(X, T)) = (\text{if } A=\emptyset \text{ then } \emptyset \text{ else } A \cup (X \cap T))$
<proof>

The boundary of a set is $\emptyset$ if A is X or $\emptyset$, and $X \cap T$ in other case.

lemma boundary_excludedset:
 assumes $A \subseteq X$
 shows $\text{Boundary}(A, \text{ExcludedSet}(X, T)) = (\text{if } A=\emptyset \vee A=X \text{ then } \emptyset \text{ else } X \cap T)$
<proof>

84.5 Special cases and subspaces

This section provides some miscellaneous facts about excluded set topologies.

The excluded set topology is equal in the sets T and $X \cap T$.

lemma smaller_excludedset:
 shows $\text{ExcludedSet}(X, T) = \text{ExcludedSet}(X, (X \cap T))$
<proof>

If the set which is excluded is disjoint with X , then the topology is discrete.

lemma empty_excludedset:
 assumes $T \cap X = \emptyset$
 shows $\text{ExcludedSet}(X, T) = \text{Pow}(X)$
<proof>

The topological subspaces of the ExcludedSet X T topology are also ExcludedSet topologies.

lemma subspace_excludedset:
 shows $\text{ExcludedSet}(X, T) \text{ \{restricted to\} } Y = \text{ExcludedSet}(Y \cap X, T)$
<proof>

84.6 Included Set Topology

In this section we consider the subsets of a set which contain a fixed set. The family defined in this section and the one in the previous section are dual; meaning that the closed set of one are the open sets of the other.

We define the included set topology as the collection of supersets of some fixed subset of the space X .

definition

$$\text{IncludedSet}(X, U) \equiv \{F \in \text{Pow}(X) \mid U \subseteq F\} \cup \{0\}$$

In the next theorem we prove that $\text{IncludedSet } X \ Q$ forms a topology.

theorem includedset_is_topology:
$$\text{shows } \text{IncludedSet}(X, Q) \text{ \{is a topology\}}$$

<proof>

We can reference the theorems proven in the `topology0` context when discussing the included set topology.

theorem topology0_includedset:
$$\text{shows } \text{topology0}(\text{IncludedSet}(X, T))$$

<proof>

Choosing a singleton set, it is considered a point excluded topology. In the following lemmas and theorems, when necessary it will be considered that $T \neq 0$ and $T \subseteq X$. These cases will appear in the special cases section.

definition

$$\text{IncludedPoint } (\text{IncludedPoint } _ _ 90) \text{ where}$$

$$\text{IncludedPoint } X \ p \equiv \text{IncludedSet}(X, \{p\})$$

84.7 Basic topological notions in included set topology

This section discusses total set, closed sets, interior, closure and boundary for included set topology.

The topology is defined in the set X .

lemma union_includedset:
$$\text{assumes } T \subseteq X$$

$$\text{shows } \bigcup \text{IncludedSet}(X, T) = X$$

<proof>

The closed sets are those which are disjoint with T and X .

lemma closed_sets_includedset:
$$\text{assumes } T \subseteq X$$

$$\text{shows } D \text{ \{is closed in\} } \text{IncludedSet}(X, T) \longleftrightarrow (D \in \text{Pow}(X) \wedge (D \cap T) = 0) \vee$$

$$D = X$$

<proof>

The interior of a set is itself if it is open or the empty set if it isn't.

lemma interior_set_includedset:
$$\text{assumes } A \subseteq X$$

$$\text{shows } \text{Interior}(A, \text{IncludedSet}(X, T)) = (\text{if } T \subseteq A \text{ then } A \text{ else } 0)$$

<proof>

The closure of a set is itself if it is closed or the whole space if it is not.

lemma closure_set_includedset:

```

assumes  $A \subseteq X$   $T \subseteq X$ 
shows  $\text{Closure}(A, \text{IncludedSet}(X, T)) = (\text{if } T \cap A = 0 \text{ then } A \text{ else } X)$ 
<proof>

```

The boundary of a set is $X-A$ if A contains T completely, is A if $X-A$ contains T completely and X if T is divided between the two sets. The case where $T=0$ is considered as a special case.

```

lemma boundary_includedset:
  assumes  $A \subseteq X$   $T \subseteq X$   $T \neq 0$ 
  shows  $\text{Boundary}(A, \text{IncludedSet}(X, T)) = (\text{if } T \subseteq A \text{ then } X-A \text{ else } (\text{if } T \cap A = 0$ 
  then  $A$  else  $X))$ 
<proof>

```

84.8 Special cases and subspaces

In this section we discuss some corner cases when some parameters in our definitions are empty and provide some facts about subspaces in included set topologies.

The topology is discrete if $T=0$

```

lemma smaller_includedset:
  shows  $\text{IncludedSet}(X, 0) = \text{Pow}(X)$ 
<proof>

```

If the set which is included is not a subset of X , then the topology is trivial.

```

lemma empty_includedset:
  assumes  $\sim(T \subseteq X)$ 
  shows  $\text{IncludedSet}(X, T) = \{0\}$ 
<proof>

```

The topological subspaces of the $\text{IncludedSet}(X, T)$ topology are also IncludedSet topologies. The trivial case does not fit the idea in the demonstration because if $Y \subseteq X$ then $\text{IncludedSet}(Y \cap X, Y \cap T)$ is never trivial. There is no need for a separate proof because the only subspace of the trivial topology is itself.

```

lemma subspace_includedset:
  assumes  $T \subseteq X$ 
  shows  $\text{IncludedSet}(X, T) \text{ \{restricted to\} } Y = \text{IncludedSet}(Y \cap X, Y \cap T)$ 
<proof>

```

end

85 More examples in topology

```

theory Topology_ZF_examples_1
imports Topology_ZF_1 Order_ZF

```

begin

In this theory file we reformulate the concepts related to a topology in relation with a base of the topology and we give examples of topologies defined by bases or subbases.

85.1 New ideas using a base for a topology

85.2 The topology of a base

Given a family of subsets satisfying the base condition, it is possible to construct a topology where that family is a base of. Even more, it is the only topology with such characteristics.

definition

TopologyWithBase (TopologyBase _ 50) **where**
U {satisfies the base condition} $\implies$ TopologyBase U $\equiv$ THE T. U {is a base for} T

If a collection U of sets satisfies the base condition then the topology constructed from it is indeed a topology and U is a base for this topology.

theorem Base_topology_is_a_topology:

assumes U {satisfies the base condition}
shows (TopologyBase U) {is a topology} and U {is a base for} (TopologyBase U)
<proof>

A base doesn't need the empty set.

lemma base_no_0:

shows B{is a base for}T $\longleftrightarrow$ (B-{0}){is a base for}T
<proof>

The interior of a set is the union of all the sets of the base which are fully contained by it.

lemma interior_set_base_topology:

assumes U {is a base for} T T{is a topology}
shows Interior(A,T) = $\bigcup \{T \in U. T \subseteq A\}$
<proof>

In the following, we offer another lemma about the closure of a set given a basis for a topology. This lemma is based on `cl_inter_neigh` and `inter_neigh_cl`. It states that it is only necessary to check the sets of the base, not all the open sets.

lemma closure_set_base_topology:

assumes U {is a base for} Q Q{is a topology} $A \subseteq \bigcup Q$
shows Closure(A,Q) = $\{x \in \bigcup Q. \forall T \in U. x \in T \longrightarrow A \cap T \neq \emptyset\}$
<proof>

The restriction of a base is a base for the restriction.

lemma subspace_base_topology:
 assumes B {is a base for} T
 shows (B {restricted to} Y) {is a base for} (T {restricted to} Y)
<proof>

If the base of a topology is contained in the base of another topology, then the topologies maintain the same relation.

theorem base_subset:
 assumes B{is a base for}TB2{is a base for}T2B $\subseteq$ B2
 shows T $\subseteq$ T2
<proof>

85.3 Dual Base for Closed Sets

A dual base for closed sets is the collection of complements of sets of a base for the topology.

definition
 DualBase (DualBase _ _ 80) where
 B{is a base for}T $\implies$ DualBase B T $\equiv$ { $\bigcup$ T-U. U $\in$ B} $\cup$ { $\bigcup$ T}

lemma closed_inter_dual_base:
 assumes D{is closed in}TB{is a base for}T
 obtains M where M $\subseteq$ DualBase B TD= $\bigcap$ M
<proof>

We have already seen for a base that whenever there is a union of open sets, we can consider only basic open sets due to the fact that any open set is a union of basic open sets. What we should expect now is that when there is an intersection of closed sets, we can consider only dual basic closed sets.

lemma closure_dual_base:
 assumes U {is a base for} QQ{is a topology}A $\subseteq$ $\bigcup$ Q
 shows Closure(A,Q)= $\bigcap$ {T $\in$ DualBase U Q. A $\subseteq$ T}
<proof>

85.4 Partition topology

In the theory file Partitions_ZF.thy; there is a definition to work with partitions. In this setting is much easier to work with a family of subsets.

definition
 IsAPartition (_{is a partition of}_ 90) where
 (U {is a partition of} X) $\equiv$ ($\bigcup$ U=X $\wedge$ ($\forall$ A $\in$ U. $\forall$ B $\in$ U. A=B $\vee$ A $\cap$ B=0) $\wedge$ 0 $\notin$ U)

A subcollection of a partition is a partition of its union.

lemma subpartition:

```

assumes U {is a partition of} X  $V \subseteq U$ 
shows V {is a partition of}  $\bigcup V$ 
 $\langle proof \rangle$ 

```

A restriction of a partition is a partition. If the empty set appears it has to be removed.

```

lemma restriction_partition:
  assumes U {is a partition of} X
  shows ((U {restricted to} Y) - {0}) {is a partition of} (X ∩ Y)
   $\langle proof \rangle$ 

```

Given a partition, the complement of a union of a subfamily is a union of a subfamily.

```

lemma diff_union_is_union_diff:
  assumes  $R \subseteq P$  P {is a partition of} X
  shows  $X - \bigcup R = \bigcup (P - R)$ 
   $\langle proof \rangle$ 

```

85.5 Partition topology is a topology.

A partition satisfies the base condition.

```

lemma partition_base_condition:
  assumes P {is a partition of} X
  shows P {satisfies the base condition}
   $\langle proof \rangle$ 

```

Since a partition is a base of a topology, and this topology is uniquely determined; we can built it. In the definition we have to make sure that we have a partition.

definition

```

PartitionTopology (PTopology _ _ 50) where
  (U {is a partition of} X)  $\implies$  PTopology X U  $\equiv$  TopologyBase U

```

```

theorem Ptopology_is_a_topology:
  assumes U {is a partition of} X
  shows (PTopology X U) {is a topology} and U {is a base for} (PTopology X U)
   $\langle proof \rangle$ 

```

```

lemma topology0_ptopology:
  assumes U {is a partition of} X
  shows topology0 (PTopology X U)
   $\langle proof \rangle$ 

```

85.6 Total set, Closed sets, Interior, Closure and Boundary

The topology is defined in the set X

```

lemma union_ptopology:
  assumes U {is a partition of} X
  shows  $\bigcup (\text{PTopology } X \text{ } U) = X$ 
  <proof>

```

The closed sets are the open sets.

```

lemma closed_sets_ptopology:
  assumes T {is a partition of} X
  shows  $D \text{ {is closed in} } (\text{PTopology } X \text{ } T) \iff D \in (\text{PTopology } X \text{ } T)$ 
  <proof>

```

There is a formula for the interior given by an intersection of sets of the dual base. Is the intersection of all the closed sets of the dual basis such that they do not complement A to X . Since the interior of X must be inside X , we have to enter X as one of the sets to be intersected.

```

lemma interior_set_ptopology:
  assumes U {is a partition of}  $XA \subseteq X$ 
  shows  $\text{Interior}(A, (\text{PTopology } X \text{ } U)) = \bigcap \{T \in \text{DualBase } U \mid (\text{PTopology } X \text{ } U) .$ 
 $T = X \vee T \cup A \neq X\}$ 
  <proof>

```

The closure of a set is the union of all the sets of the partition which intersect with A .

```

lemma closure_set_ptopology:
  assumes U {is a partition of}  $XA \subseteq X$ 
  shows  $\text{Closure}(A, (\text{PTopology } X \text{ } U)) = \bigcup \{T \in U. T \cap A \neq \emptyset\}$ 
  <proof>

```

The boundary of a set is given by the union of the sets of the partition which have non empty intersection with the set but that are not fully contained in it. Another equivalent statement would be: the union of the sets of the partition which have non empty intersection with the set and its complement.

```

lemma boundary_set_ptopology:
  assumes U {is a partition of}  $XA \subseteq X$ 
  shows  $\text{Boundary}(A, (\text{PTopology } X \text{ } U)) = \bigcup \{T \in U. T \cap A \neq \emptyset \wedge \sim(T \subseteq A)\}$ 
  <proof>

```

85.7 Special cases and subspaces

The discrete and the indiscrete topologies appear as special cases of this partition topologies.

```

lemma discrete_partition:
  shows  $\{\{x\}. x \in X\} \text{ {is a partition of} } X$ 
  <proof>

```

```

lemma indiscrete_partition:
  assumes  $X \neq \emptyset$ 

```



```

shows {X} {is a partition of} X
<proof>

theorem discrete_ptopology:
  shows (PTopology X {{x}.x∈X})=Pow(X)
<proof>

theorem indiscrete_ptopology:
  assumes X≠0
  shows (PTopology X {X})={0,X}
<proof>

```

The topological subspaces of the (PTopology X U) are partition topologies.

```

lemma subspace_ptopology:
  assumes U{is a partition of}X
  shows (PTopology X U) {restricted to} Y=(PTopology (X∩Y) ((U {restricted
to} Y)-{0}))
<proof>

```

85.8 Order topologies

85.9 Order topology is a topology

Given a totally ordered set, several topologies can be defined using the order relation. First we define an open interval, notice that the set defined as Interval is a closed interval; and open rays.

```

definition
  IntervalX where
    IntervalX(X,r,b,c)≡(Interval(r,b,c)∩X)-{b,c}
definition
  LeftRayX where
    LeftRayX(X,r,b)≡{c∈X. ⟨c,b⟩∈r}-{b}
definition
  RightRayX where
    RightRayX(X,r,b)≡{c∈X. ⟨b,c⟩∈r}-{b}

```

Intersections of intervals and rays.

```

lemma inter_two_intervals:
  assumes bu∈Xbv∈Xcu∈Xcv∈XIsLinOrder(X,r)
  shows IntervalX(X,r,bu,cu)∩IntervalX(X,r,bv,cv)=IntervalX(X,r,GreaterOf(r,bu,bv),SmallerOf(r,bu,bv))
<proof>

lemma inter_rray_interval:
  assumes bv∈Xbu∈Xcv∈XIsLinOrder(X,r)
  shows RightRayX(X,r,bu)∩IntervalX(X,r,bv,cv)=IntervalX(X,r,GreaterOf(r,bu,bv),cv)
<proof>

```

lemma inter_lray_interval:
 assumes $bv \in X$ $cu \in X$ $cv \in X$ $\text{IsLinOrder}(X, r)$
 shows $\text{LeftRayX}(X, r, cu) \cap \text{IntervalX}(X, r, bv, cv) = \text{IntervalX}(X, r, bv, \text{SmallerOf}(r, cu, cv))$
<proof>

lemma inter_lray_rray:
 assumes $bu \in X$ $cv \in X$ $\text{IsLinOrder}(X, r)$
 shows $\text{LeftRayX}(X, r, bu) \cap \text{RightRayX}(X, r, cv) = \text{IntervalX}(X, r, cv, bu)$
<proof>

lemma inter_lray_lray:
 assumes $bu \in X$ $cv \in X$ $\text{IsLinOrder}(X, r)$
 shows $\text{LeftRayX}(X, r, bu) \cap \text{LeftRayX}(X, r, cv) = \text{LeftRayX}(X, r, \text{SmallerOf}(r, bu, cv))$
<proof>

lemma inter_rray_rray:
 assumes $bu \in X$ $cv \in X$ $\text{IsLinOrder}(X, r)$
 shows $\text{RightRayX}(X, r, bu) \cap \text{RightRayX}(X, r, cv) = \text{RightRayX}(X, r, \text{GreaterOf}(r, bu, cv))$
<proof>

The open intervals and rays satisfy the base condition.

lemma intervals_rays_base_condition:
 assumes $\text{IsLinOrder}(X, r)$
 shows $\{\text{IntervalX}(X, r, b, c) \mid \langle b, c \rangle \in X \times X\} \cup \{\text{LeftRayX}(X, r, b) \mid b \in X\} \cup \{\text{RightRayX}(X, r, b) \mid b \in X\}$ {satisfies the base condition}
<proof>

Since the intervals and rays form a base of a topology, and this topology is uniquely determined; we can built it. In the definition we have to make sure that we have a totally ordered set.

definition

OrderTopology (OrdTopology _ _ 50) where
 $\text{IsLinOrder}(X, r) \implies \text{OrdTopology } X \ r \equiv \text{TopologyBase } \{\text{IntervalX}(X, r, b, c) \mid \langle b, c \rangle \in X \times X\} \cup \{\text{LeftRayX}(X, r, b) \mid b \in X\} \cup \{\text{RightRayX}(X, r, b) \mid b \in X\}$

theorem Ordtopology_is_a_topology:
 assumes $\text{IsLinOrder}(X, r)$
 shows $(\text{OrdTopology } X \ r)$ {is a topology} and $\{\text{IntervalX}(X, r, b, c) \mid \langle b, c \rangle \in X \times X\} \cup \{\text{LeftRayX}(X, r, b) \mid b \in X\} \cup \{\text{RightRayX}(X, r, b) \mid b \in X\}$ {is a base for} $(\text{OrdTopology } X \ r)$
<proof>

lemma topology0_ordtopology:
 assumes $\text{IsLinOrder}(X, r)$
 shows $\text{topology0}(\text{OrdTopology } X \ r)$
<proof>

85.10 Total set

The topology is defined in the set X , when X has more than one point

```

lemma union_ordtopology:
  assumes IsLinOrder(X,r)  $\exists x y. x \neq y \wedge x \in X \wedge y \in X$ 
  shows  $\bigcup (\text{OrdTopology } X \ r) = X$ 
  <proof>

```

The interior, closure and boundary can be calculated using the formulas proved in the section that deals with the base.

The subspace of an order topology doesn't have to be an order topology.

85.11 Right order and Left order topologies.

Notice that the left and right rays are closed under intersection, hence they form a base of a topology. They are called right order topology and left order topology respectively.

If the order in X has a minimal or a maximal element, is necessary to consider X as an element of the base or that limit point wouldn't be in any basic open set.

85.11.1 Right and Left Order topologies are topologies

```

lemma leftrays_base_condition:
  assumes IsLinOrder(X,r)
  shows {LeftRayX(X,r,b).  $b \in X \cup \{X\}$ } {satisfies the base condition}
  <proof>

```

```

lemma rightrays_base_condition:
  assumes IsLinOrder(X,r)
  shows {RightRayX(X,r,b).  $b \in X \cup \{X\}$ } {satisfies the base condition}
  <proof>

```

definition

```

  LeftOrderTopology (LOrdTopology _ _ 50) where
    IsLinOrder(X,r)  $\implies$  LOrdTopology X r  $\equiv$  TopologyBase {LeftRayX(X,r,b).
     $b \in X \cup \{X\}$ }

```

definition

```

  RightOrderTopology (ROrdTopology _ _ 50) where
    IsLinOrder(X,r)  $\implies$  ROrdTopology X r  $\equiv$  TopologyBase {RightRayX(X,r,b).
     $b \in X \cup \{X\}$ }

```

theorem LOrdtopology_ROrdtopology_are_topologies:

```

  assumes IsLinOrder(X,r)
  shows (LOrdTopology X r) {is a topology} and {LeftRayX(X,r,b).  $b \in X \cup \{X\}$ }
  {is a base for} (LOrdTopology X r)
  and (ROrdTopology X r) {is a topology} and {RightRayX(X,r,b).  $b \in X \cup \{X\}$ }
  {is a base for} (ROrdTopology X r)

```

<proof>

```

lemma topology0_lordtopology_rordtopology:
  assumes IsLinOrder(X,r)
  shows topology0(L0rdTopology X r) and topology0(R0rdTopology X r)
<proof>

```

85.11.2 Total set

The topology is defined on the set X

```

lemma union_lordtopology_rordtopology:
  assumes IsLinOrder(X,r)
  shows  $\bigcup (L0rdTopology X r)=X$  and  $\bigcup (R0rdTopology X r)=X$ 
<proof>

```

85.12 Union of Topologies

The union of two topologies is not a topology. A way to overcome this fact is to define the following topology:

definition

```

  joinT (joinT _ 90) where
     $(\forall T \in M. T \text{ is a topology} \wedge (\forall Q \in M. \bigcup Q = \bigcup T)) \implies (\text{joinT } M \equiv \text{THE } T. (\bigcup M) \text{ is a subbase for } T)$ 

```

First let's proof that given a family of sets, then it is a subbase for a topology.

The first result states that from any family of sets we get a base using finite intersections of them. The second one states that any family of sets is a subbase of some topology.

theorem subset_as_subbase:

```

  shows  $\{\bigcap A. A \in \text{FinPow}(B)\}$  {satisfies the base condition}
<proof>

```

theorem Top_subbase:

```

  assumes  $T = \{\bigcup A. A \in \text{Pow}(\{\bigcap A. A \in \text{FinPow}(B)\})\}$ 
  shows  $T$  {is a topology} and  $B$  {is a subbase for}  $T$ 
<proof>

```

A subbase defines a unique topology.

theorem same_subbase_same_top:

```

  assumes  $B$  {is a subbase for}  $T$  and  $B$  {is a subbase for}  $S$ 
  shows  $T = S$ 
<proof>

```

end

86 Properties in Topology

theory Topology_ZF_properties **imports** Topology_ZF_examples Topology_ZF_examples_1

begin

This theory deals with topological properties which make use of cardinals.

86.1 Properties of compactness

It is already defined what is a compact topological space, but there is a generalization which may be useful sometimes.

definition

```
IsCompactOfCard (_{is compact of cardinal}_ {in}_ 90)
  where K{is compact of cardinal} Q{in}T  $\equiv$  (Card(Q)  $\wedge$   $K \subseteq \bigcup T$   $\wedge$ 
    ( $\forall M \in \text{Pow}(T). K \subseteq \bigcup M \longrightarrow (\exists N \in \text{Pow}(M). K \subseteq \bigcup N \wedge N \prec Q)$ ))
```

The usual compact property is the one defined over the cardinal of the natural numbers.

lemma Compact_is_card_nat:

```
  shows K{is compact in}T  $\longleftrightarrow$  (K{is compact of cardinal} nat {in}T)
  <proof>
```

Another property of this kind widely used is the Lindelöf property; it is the one on the successor of the natural numbers.

definition

```
IsLindelöf (_{is lindelöf in}_ 90) where
  K {is lindelöf in} T  $\equiv$  K{is compact of cardinal} csucc(nat){in}T
```

It would be natural to think that every countable set with any topology is Lindelöf; but this statement is not provable in ZF. The reason is that to build a subcover, most of the time we need to *choose* sets from an infinite collection which cannot be done in ZF. Additional axioms are needed, but strictly weaker than the axiom of choice.

However, if the topology has not many open sets, then the topological space is indeed compact.

theorem card_top_comp:

```
  assumes Card(Q) T  $\prec$  Q  $K \subseteq \bigcup T$ 
  shows (K){is compact of cardinal}Q{in}T
  <proof>
```

The union of two compact sets, is compact; of any cardinality.

theorem union_compact:

```
  assumes K{is compact of cardinal}Q{in}T K1{is compact of cardinal}Q{in}T
  InfCard(Q)
  shows (K  $\cup$  K1){is compact of cardinal}Q{in}T <proof>
```

If a set is compact of cardinality Q for some topology, it is compact of cardinality Q for every coarser topology.

theorem compact_coarser:

assumes $T_1 \subseteq T$ and $\bigcup T_1 = \bigcup T$ and $(K) \{ \text{is compact of cardinality } Q \} \{ \text{in } T \}$
 shows $(K) \{ \text{is compact of cardinality } Q \} \{ \text{in } T_1 \}$

<proof>

If some set is compact for some cardinal, it is compact for any greater cardinal.

theorem compact_greater_card:

assumes $Q \leq Q_1$ and $(K) \{ \text{is compact of cardinality } Q \} \{ \text{in } T \}$ and $\text{Card}(Q_1)$
 shows $(K) \{ \text{is compact of cardinality } Q_1 \} \{ \text{in } T \}$

<proof>

A closed subspace of a compact space of any cardinality, is also compact of the same cardinality.

theorem compact_closed:

assumes $K \{ \text{is compact of cardinality } Q \} \{ \text{in } T \}$
 and $R \{ \text{is closed in } T \}$
 shows $(K \cap R) \{ \text{is compact of cardinality } Q \} \{ \text{in } T \}$

<proof>

86.2 Properties of numerability

The properties of numerability deal with cardinals of some sets built from the topology. The properties which are normally used are the ones related to the cardinal of the natural numbers or its successor.

definition

$\text{IsFirstOfCard } (_ \{ \text{is of first type of cardinality } _ \} \text{ 90})$ where
 $(T \{ \text{is of first type of cardinality } Q \}) \equiv \forall x \in \bigcup T. (\exists B. (B \{ \text{is a base for } T \}) \wedge (\{ b \in B. x \in b \} \prec Q))$

definition

$\text{IsSecondOfCard } (_ \{ \text{is of second type of cardinality } _ \} \text{ 90})$ where
 $(T \{ \text{is of second type of cardinality } Q \}) \equiv (\exists B. (B \{ \text{is a base for } T \}) \wedge (B \prec Q))$

definition

$\text{IsSeparableOfCard } (_ \{ \text{is separable of cardinality } _ \} \text{ 90})$ where
 $T \{ \text{is separable of cardinality } Q \} \equiv \exists U \in \text{Pow}(\bigcup T). \text{Closure}(U, T) = \bigcup T \wedge U \prec Q$

definition

$\text{IsFirstCountable } (_ \{ \text{is first countable} \} \text{ 90})$ where
 $(T \{ \text{is first countable} \}) \equiv T \{ \text{is of first type of cardinality } \text{csucc}(\text{nat}) \}$

definition

$\text{IsSecondCountable } (_ \{ \text{is second countable} \} \text{ 90})$ where

$(T \text{ \{is second countable\}}) \equiv (T \text{ \{is of second type of cardinal\}csucc(nat)})$

definition

IsSeparable ($_ \{ \text{is separable} \}$ 90) **where**
 $T \{ \text{is separable} \} \equiv T \{ \text{is separable of cardinal} \} \text{csucc(nat)}$

If a set is of second type of cardinal Q , then it is of first type of that same cardinal.

theorem second_imp_first:

assumes $T \{ \text{is of second type of cardinal} \} Q$
shows $T \{ \text{is of first type of cardinal} \} Q$

<proof>

A set is dense iff it intersects all non-empty, open sets of the topology.

lemma dense_int_open:

assumes $T \{ \text{is a topology} \}$ **and** $A \subseteq \bigcup T$
shows $\text{Closure}(A, T) = \bigcup T \iff (\forall U \in T. U \neq \emptyset \longrightarrow A \cap U \neq \emptyset)$

<proof>

86.3 Relations between numerability properties and choice principles

It is known that some statements in topology aren't just derived from choice axioms, but also equivalent to them. Here is an example

The following are equivalent:

- Every topological space of second cardinality $\text{csucc}(Q)$ is separable of cardinality $\text{csucc}(Q)$.
- The axiom of Q choice.

In the article [4] there is a proof of this statement for $Q = \mathbb{N}$, with more equivalences.

If a topology is of second type of cardinal $\text{csucc}(Q)$, then it is separable of the same cardinal. This result makes use of the axiom of choice for the cardinal Q on subsets of $\bigcup T$.

theorem $Q_choice_imp_second_imp_separable$:

assumes $T \{ \text{is of second type of cardinal} \} \text{csucc}(Q)$
and $\{ \text{the axiom of} \} Q \{ \text{choice holds for subsets} \} \bigcup T$
and $T \{ \text{is a topology} \}$
shows $T \{ \text{is separable of cardinal} \} \text{csucc}(Q)$

<proof>

The next theorem resolves that the axiom of Q choice for subsets of $\bigcup T$ is necessary for second type spaces to be separable of the same cardinal $\text{csucc}(Q)$.

theorem second_imp_separable_imp_Q_choice:
 assumes $\forall T. (T\{\text{is a topology}\} \wedge (T\{\text{is of second type of cardinal}\}\text{csucc}(Q)))$
 $\longrightarrow (T\{\text{is separable of cardinal}\}\text{csucc}(Q))$
 and $\text{Card}(Q)$
 shows $\{\text{the axiom of}\} Q \{\text{choice holds}\}$
<proof>

Here is the equivalence from the two previous results.

theorem Q_choice_eq_secon_imp_sepa:
 assumes $\text{Card}(Q)$
 shows $(\forall T. (T\{\text{is a topology}\} \wedge (T\{\text{is of second type of cardinal}\}\text{csucc}(Q)))$
 $\longrightarrow (T\{\text{is separable of cardinal}\}\text{csucc}(Q))$
 $\longleftrightarrow (\{\text{the axiom of}\} Q \{\text{choice holds}\})$
<proof>

Given a base injective with a set, then we can find a base whose elements are indexed by that set.

lemma base_to_indexed_base:
 assumes $B \lesssim_Q B \{\text{is a base for}\} T$
 shows $\exists N. \{N_i. i \in Q\} \{\text{is a base for}\} T$
<proof>

86.4 Relation between numerability and compactness

If the axiom of Q choice holds, then any topology of second type of cardinal $\text{csucc}(Q)$ is compact of cardinal $\text{csucc}(Q)$

theorem compact_of_cardinal_Q:
 assumes $\{\text{the axiom of}\} Q \{\text{choice holds for subsets}\} (\text{Pow}(Q))$
 $T\{\text{is of second type of cardinal}\}\text{csucc}(Q)$
 $T\{\text{is a topology}\}$
 shows $((\bigcup T)\{\text{is compact of cardinal}\}\text{csucc}(Q)\{\text{in}\}T)$
<proof>

In the following proof, we have chosen an infinite cardinal to be able to apply the equation $Q \times Q \approx Q$. For finite cardinals; both, the assumption and the axiom of choice, are always true.

theorem second_imp_compact_imp_Q_choice_PowQ:
 assumes $\forall T. (T\{\text{is a topology}\} \wedge (T\{\text{is of second type of cardinal}\}\text{csucc}(Q)))$
 $\longrightarrow ((\bigcup T)\{\text{is compact of cardinal}\}\text{csucc}(Q)\{\text{in}\}T)$
 and $\text{InfCard}(Q)$
 shows $\{\text{the axiom of}\} Q \{\text{choice holds for subsets}\} (\text{Pow}(Q))$
<proof>

The two previous results, state the following equivalence:

theorem Q_choice_Pow_eq_secon_imp_comp:
 assumes $\text{InfCard}(Q)$
 shows $(\forall T. (T\{\text{is a topology}\} \wedge (T\{\text{is of second type of cardinal}\}\text{csucc}(Q)))$
 $\longrightarrow ((\bigcup T)\{\text{is compact of cardinal}\}\text{csucc}(Q)\{\text{in}\}T))$

$\longleftrightarrow (\{\text{the axiom of } Q \text{ choice holds for subsets} \} (Pow(Q)))$
 $\langle proof \rangle$

In the next result we will prove that if the space $(\kappa, Pow(\kappa))$, for κ an infinite cardinal, is compact of its successor cardinal; then all topological spaces which are of second type of the successor cardinal of κ are also compact of that cardinal.

theorem `Q_csuccQ_comp_eq_Q_choice_Pow`:
assumes `InfCard(Q)` `(Q){is compact of cardinal}csucc(Q){in}Pow(Q)`
shows $\forall T. (T\{\text{is a topology}\} \wedge (T\{\text{is of second type of cardinal}\}csucc(Q)))$
 $\longrightarrow ((\bigcup T)\{\text{is compact of cardinal}\}csucc(Q)\{\text{in}\}T)$
 $\langle proof \rangle$

theorem `Q_disc_is_second_card_csuccQ`:
assumes `InfCard(Q)`
shows `Pow(Q){is of second type of cardinal}csucc(Q)`
 $\langle proof \rangle$

This previous results give us another equivalence of the axiom of Q choice that is apparently weaker (easier to check) to the previous one.

theorem `Q_disc_comp_csuccQ_eq_Q_choice_csuccQ`:
assumes `InfCard(Q)`
shows $(Q\{\text{is compact of cardinal}\}csucc(Q)\{\text{in}\}(Pow(Q))) \longleftrightarrow (\{\text{the axiom of}\}Q\{\text{choice holds for subsets}\}(Pow(Q)))$
 $\langle proof \rangle$

end

87 Topology 5

theory `Topology_ZF_5` **imports** `Topology_ZF_properties` `Topology_ZF_examples_1`
`Topology_ZF_4b`
begin

87.1 Some results for separation axioms

First we will give a global characterization of T_1 -spaces; which is interesting because it involves the cardinal $\aleph$.

lemma `(in topology0) T1_cocardinal_coarser`:
shows $(T \{\text{is } T_1\}) \longleftrightarrow (CoFinite (\bigcup T)) \subseteq T$
 $\langle proof \rangle$

In the previous proof, it is obvious that we don't need to check if ever cofinite set is open. It is enough to check if every singleton is closed.

corollary `(in topology0) T1_iff_singleton_closed`:
shows $(T \{\text{is } T_1\}) \longleftrightarrow (\forall x \in \bigcup T. \{x\} \{\text{is closed in}\} T)$

<proof>

Secondly, let's show that the $\text{CoCardinal } X \text{ } Q$ topologies for different sets Q are all ordered as the partial order of sets. (The order is linear when considering only cardinals)

lemma `order_cocardinal_top:`
 fixes X
 assumes $Q1 \lesssim Q2$
 shows $\text{CoCardinal}(X, Q1) \subseteq \text{CoCardinal}(X, Q2)$

<proof>

corollary `cocardinal_is_T1:`
 fixes $X \ K$
 assumes $\text{InfCard}(K)$
 shows $\text{CoCardinal}(X, K) \ \{\text{is } T_1\}$

<proof>

In T_2 -spaces, filters and nets have at most one limit point.

lemma `(in topology0) T2_imp_unique_limit_filter:`
 assumes $T \ \{\text{is } T_2\} \ \mathfrak{F} \ \{\text{is a filter on}\} \bigcup T \ \mathfrak{F} \rightarrow_F x \ \mathfrak{F} \rightarrow_F y$
 shows $x=y$

<proof>

lemma `(in topology0) T2_imp_unique_limit_net:`
 assumes $T \ \{\text{is } T_2\} \ N \ \{\text{is a net on}\} \bigcup T \ N \rightarrow_N x \ N \rightarrow_N y$
 shows $x=y$

<proof>

In fact, T_2 -spaces are characterized by this property. For this proof we build a filter containing the union of two filters.

lemma `(in topology0) unique_limit_filter_imp_T2:`
 assumes $\forall x \in \bigcup T. \forall y \in \bigcup T. \forall \mathfrak{F}. ((\mathfrak{F} \ \{\text{is a filter on}\} \bigcup T) \wedge (\mathfrak{F} \rightarrow_F x) \wedge (\mathfrak{F} \rightarrow_F y)) \longrightarrow x=y$
 shows $T \ \{\text{is } T_2\}$

<proof>

lemma `(in topology0) unique_limit_net_imp_T2:`
 assumes $\forall x \in \bigcup T. \forall y \in \bigcup T. \forall N. ((N \ \{\text{is a net on}\} \bigcup T) \wedge (N \rightarrow_N x) \wedge (N \rightarrow_N y)) \longrightarrow x=y$
 shows $T \ \{\text{is } T_2\}$

<proof>

This results make easy to check if a space is T_2 .

The topology which comes from a filter as in $\mathfrak{F} \ \{\text{is a filter on}\} \bigcup \mathfrak{F} \implies (\mathfrak{F} \cup \text{cons}(\emptyset, \emptyset)) \ \{\text{is a topology}\}$ is not T_2 generally. We will see in this file later on, that the exceptions are a consequence of the spectrum.

corollary `filter_T2_imp_card1:`

assumes $(\mathcal{F} \cup \{0\}) \text{ is } T_2$ $\mathcal{F} \text{ is a filter on } \bigcup \mathcal{F}$ $x \in \bigcup \mathcal{F}$
shows $\bigcup \mathcal{F} = \{x\}$
<proof>

There are more separation axioms that just T_0 , T_1 or T_2

definition

IsNormal ($_ \text{is normal}$) 90)
where $T \text{ is normal} \equiv \forall A. A \text{ is closed in } T \longrightarrow (\forall B. B \text{ is closed in } T$
 $\wedge A \cap B = 0 \longrightarrow$
 $(\exists U \in T. \exists V \in T. A \subseteq U \wedge B \subseteq V \wedge U \cap V = 0))$

definition

isT4 ($_ \text{is } T_4$) 90)
where $T \text{ is } T_4 \equiv (T \text{ is } T_1) \wedge (T \text{ is normal})$

lemma (in topology0) T4_is_T3:

assumes $T \text{ is } T_4$ **shows** $T \text{ is } T_3$
<proof>

Regularity can be rewritten in terms of existence of certain neighborhoods.

lemma (in topology0) regular_imp_exist_clos_neig:

assumes $T \text{ is regular}$ **and** $U \in T$ **and** $x \in U$
shows $\exists V \in T. x \in V \wedge \text{cl}(V) \subseteq U$
<proof>

lemma (in topology0) exist_clos_neig_imp_regular:

assumes $\forall x \in \bigcup T. \forall U \in T. x \in U \longrightarrow (\exists V \in T. x \in V \wedge \text{cl}(V) \subseteq U)$
shows $T \text{ is regular}$
<proof>

lemma (in topology0) regular_eq:

shows $T \text{ is regular} \longleftrightarrow (\forall x \in \bigcup T. \forall U \in T. x \in U \longrightarrow (\exists V \in T. x \in V \wedge \text{cl}(V) \subseteq U))$
<proof>

A Hausdorff space separates compact spaces from points.

theorem (in topology0) T2_compact_point:

assumes $T \text{ is } T_2$ $A \text{ is compact in } T$ $x \in \bigcup T$ $x \notin A$
shows $\exists U \in T. \exists V \in T. A \subseteq U \wedge x \in V \wedge U \cap V = 0$
<proof>

A Hausdorff space separates compact spaces from other compact spaces.

theorem (in topology0) T2_compact_compact:

assumes $T \text{ is } T_2$ $A \text{ is compact in } T$ $B \text{ is compact in } T$ $A \cap B = 0$
shows $\exists U \in T. \exists V \in T. A \subseteq U \wedge B \subseteq V \wedge U \cap V = 0$
<proof>

A compact Hausdorff space is normal.

corollary (in topology0) T2_compact_is_normal:

assumes $T\{\text{is } T_2\} (\bigcup T)\{\text{is compact in}\}T$
 shows $T\{\text{is normal}\} \langle \text{proof} \rangle$

87.2 Hereditability

A topological property is hereditary if whenever a space has it, every subspace also has it.

definition $\text{IsHer } (_ \{\text{is hereditary}\} 90)$

where $P \{\text{is hereditary}\} \equiv \forall T. T\{\text{is a topology}\} \wedge P(T) \longrightarrow (\forall A \in \text{Pow}(\bigcup T). P(T\{\text{restricted to}\}A))$

lemma $\text{subspace_of_subspace}:$

assumes $A \subseteq B \subseteq \bigcup T$

shows $T\{\text{restricted to}\}A = (T\{\text{restricted to}\}B)\{\text{restricted to}\}A$

$\langle \text{proof} \rangle$

The separation properties T_0 , T_1 , T_2 y T_3 are hereditary.

theorem $\text{regular_here}:$

assumes $T\{\text{is regular}\} A \in \text{Pow}(\bigcup T)$ shows $(T\{\text{restricted to}\}A)\{\text{is regular}\}$

$\langle \text{proof} \rangle$

corollary $\text{here_regular}:$

shows $\text{IsRegular} \{\text{is hereditary}\} \langle \text{proof} \rangle$

theorem $\text{T1_here}:$

assumes $T\{\text{is } T_1\} A \in \text{Pow}(\bigcup T)$ shows $(T\{\text{restricted to}\}A)\{\text{is } T_1\}$

$\langle \text{proof} \rangle$

corollary $\text{here_T1}:$

shows $\text{isT1} \{\text{is hereditary}\} \langle \text{proof} \rangle$

lemma $\text{here_and}:$

assumes $P \{\text{is hereditary}\} Q \{\text{is hereditary}\}$

shows $(\lambda T. P(T) \wedge Q(T)) \{\text{is hereditary}\} \langle \text{proof} \rangle$

corollary $\text{here_T3}:$

shows $\text{isT3} \{\text{is hereditary}\} \langle \text{proof} \rangle$

lemma $\text{T2_here}:$

assumes $T\{\text{is } T_2\} A \in \text{Pow}(\bigcup T)$ shows $(T\{\text{restricted to}\}A)\{\text{is } T_2\}$

$\langle \text{proof} \rangle$

corollary $\text{here_T2}:$

shows $\text{isT2} \{\text{is hereditary}\} \langle \text{proof} \rangle$

lemma $\text{T0_here}:$

assumes $T\{\text{is } T_0\} A \in \text{Pow}(\bigcup T)$ shows $(T\{\text{restricted to}\}A)\{\text{is } T_0\}$

$\langle \text{proof} \rangle$

corollary here_T0:
 shows isT0 {is hereditary} *<proof>*

87.3 Spectrum and anti-properties

The spectrum of a topological property is a class of sets such that all topologies defined over that set have that property.

The spectrum of a property gives us the list of sets for which the property doesn't give any topological information. Being in the spectrum of a topological property is an invariant in the category of sets and function; meaning that equipollent sets are in the same spectra.

definition Spec ($_ \{is\ in\ the\ spectrum\ of\} _ \ 99$)
 where Spec(K,P) $\equiv \forall T. ((T\{is\ a\ topology\} \wedge \bigcup T \approx K) \longrightarrow P(T))$

lemma equipollent_spect:
 assumes $A \approx B$ B {is in the spectrum of} P
 shows A {is in the spectrum of} P
<proof>

theorem eqpoll_iff_spec:
 assumes $A \approx B$
 shows (B {is in the spectrum of} P) $\longleftrightarrow$ (A {is in the spectrum of} P)
<proof>

From the previous statement, we see that the spectrum could be formed only by representative of classes of sets. If AC holds, this means that the spectrum can be taken as a set or class of cardinal numbers.

Here is an example of the spectrum. The proof lies in the indiscrete filter $\{A\}$ that can be build for any set. In this proof, we see that without choice, there is no way to define the spectrum of a property with cardinals because if a set is not comparable with any ordinal, its cardinal is defined as 0 without the set being empty.

theorem T4_spectrum:
 shows (A {is in the spectrum of} isT4) $\longleftrightarrow A \lesssim 1$
<proof>

If the topological properties are related, then so are the spectra.

lemma P_imp_Q_spec_inv:
 assumes $\forall T. T\{is\ a\ topology\} \longrightarrow (Q(T) \longrightarrow P(T))$ A {is in the spectrum of} Q
 shows A {is in the spectrum of} P
<proof>

Since we already know the spectrum of T_4 ; if we know the spectrum of T_0 , it should be easier to compute the spectrum of T_1 , T_2 and T_3 .

theorem T0_spectrum:

shows (A {is in the spectrum of} isT0) $\longleftrightarrow$ $A \lesssim 1$
<proof>

theorem T1_spectrum:

shows (A {is in the spectrum of} isT1) $\longleftrightarrow$ $A \lesssim 1$
<proof>

theorem T2_spectrum:

shows (A {is in the spectrum of} isT2) $\longleftrightarrow$ $A \lesssim 1$
<proof>

theorem T3_spectrum:

shows (A {is in the spectrum of} isT3) $\longleftrightarrow$ $A \lesssim 1$
<proof>

theorem compact_spectrum:

shows (A {is in the spectrum of} ($\lambda T. (\bigcup T)$ {is compact in}T)) $\longleftrightarrow$ Finite(A)
<proof>

It is, at least for some people, surprising that the spectrum of some properties cannot be completely determined in ZF .

theorem compactK_spectrum:

assumes {the axiom of}K{choice holds for subsets}(Pow(K)) Card(K)
 shows (A {is in the spectrum of} ($\lambda T. ((\bigcup T)$ {is compact of cardinal} csucc(K){in}T))) $\longleftrightarrow$ ($A \lesssim K$)
<proof>

theorem compactK_spectrum_reverse:

assumes $\forall A. (A$ {is in the spectrum of} ($\lambda T. ((\bigcup T)$ {is compact of cardinal} csucc(K){in}T))) $\longleftrightarrow$ ($A \lesssim K$) InfCard(K)
 shows {the axiom of}K{choice holds for subsets}(Pow(K))
<proof>

This last theorem states that if one of the forms of the axiom of choice related to this compactness property fails, then the spectrum will be different. Notice that even for Lindelöf spaces that will happen.

The spectrum gives us the possibility to define what an anti-property means. A space is anti-P if the only subspaces which have the property are the ones in the spectrum of P. This concept tries to put together spaces that are completely opposite to spaces where P(T).

definition

antiProperty (_{is anti-}_ 50)

where $T\{\text{is anti-}\}P \equiv \forall A \in \text{Pow}(\bigcup T). P(T\{\text{restricted to}\}A) \longrightarrow (A \{\text{is in the spectrum of}\} P)$

abbreviation

$\text{ANTI}(P) \equiv \lambda T. (T\{\text{is anti-}\}P)$

A first, very simple, but very useful result is the following: when the properties are related and the spectra are equal, then the anti-properties are related in the opposite direction.

theorem (in topology0) eq_spect_rev_imp_anti:
 assumes $\forall T. T\{\text{is a topology}\} \longrightarrow P(T) \longrightarrow Q(T) \forall A. (A\{\text{is in the spectrum of}\}Q) \longrightarrow (A\{\text{is in the spectrum of}\}P)$
 and $T\{\text{is anti-}\}Q$
 shows $T\{\text{is anti-}\}P$
<proof>

If a space can be $P(T) \wedge Q(T)$ only in case the underlying set is in the spectrum of P ; then $Q(T) \longrightarrow \text{ANTI}(P, T)$ when Q is hereditary.

theorem Q_P_imp_Spec:
 assumes $\forall T. ((T\{\text{is a topology}\} \wedge P(T) \wedge Q(T)) \longrightarrow ((\bigcup T)\{\text{is in the spectrum of}\}P))$
 and $Q\{\text{is hereditary}\}$
 shows $\forall T. T\{\text{is a topology}\} \longrightarrow (Q(T) \longrightarrow (T\{\text{is anti-}\}P))$
<proof>

If a topological space has an hereditary property, then it has its double-anti property.

theorem (in topology0) her_P_imp_anti2P:
 assumes $P\{\text{is hereditary}\} P(T)$
 shows $T\{\text{is anti-}\}\text{ANTI}(P)$
<proof>

The anti-properties are always hereditary

theorem anti_here:
 shows $\text{ANTI}(P)\{\text{is hereditary}\}$
<proof>

corollary (in topology0) anti_imp_anti3:
 assumes $T\{\text{is anti-}\}P$
 shows $T\{\text{is anti-}\}\text{ANTI}(\text{ANTI}(P))$
<proof>

In the article [5], we can find some results on anti-properties.

theorem (in topology0) anti_T0:
 shows $(T\{\text{is anti-}\}\text{isT0}) \longleftrightarrow T = \{0, \bigcup T\}$
<proof>

lemma indiscrete_spectrum:

shows (A {is in the spectrum of}($\lambda T. T = \{0, \bigcup T\}$)) $\longleftrightarrow A \lesssim 1$
<proof>

theorem (in topology0) anti_indiscrete:
shows (T{is anti-}($\lambda T. T = \{0, \bigcup T\}$)) $\longleftrightarrow T \text{is } T_0$
<proof>

The conclusion is that being T_0 is just the opposite to being indiscrete.

Next, let's compute the anti- T_i for $i = 1, 2, 3$ or 4 . Surprisingly, they are all the same. Meaning, that the total negation of T_1 is enough to negate all of these axioms.

theorem anti_T1:
shows (T{is anti-}isT1) $\longleftrightarrow$ (IsLinOrder($T, \{\langle U, V \rangle \in \text{Pow}(\bigcup T) \times \text{Pow}(\bigcup T) . U \subseteq V\}$))
<proof>

corollary linordtop_here:
shows ($\lambda T. \text{IsLinOrder}(T, \{\langle U, V \rangle \in \text{Pow}(\bigcup T) \times \text{Pow}(\bigcup T) . U \subseteq V\})$){is hereditary}
<proof>

theorem (in topology0) anti_T4:
shows (T{is anti-}isT4) $\longleftrightarrow$ (IsLinOrder($T, \{\langle U, V \rangle \in \text{Pow}(\bigcup T) \times \text{Pow}(\bigcup T) . U \subseteq V\}$))
<proof>

theorem (in topology0) anti_T3:
shows (T{is anti-}isT3) $\longleftrightarrow$ (IsLinOrder($T, \{\langle U, V \rangle \in \text{Pow}(\bigcup T) \times \text{Pow}(\bigcup T) . U \subseteq V\}$))
<proof>

theorem (in topology0) anti_T2:
shows (T{is anti-}isT2) $\longleftrightarrow$ (IsLinOrder($T, \{\langle U, V \rangle \in \text{Pow}(\bigcup T) \times \text{Pow}(\bigcup T) . U \subseteq V\}$))
<proof>

lemma linord_spectrum:
shows (A{is in the spectrum of}($\lambda T. \text{IsLinOrder}(T, \{\langle U, V \rangle \in \text{Pow}(\bigcup T) \times \text{Pow}(\bigcup T) . U \subseteq V\})$)) $\longleftrightarrow A \lesssim 1$
<proof>

theorem (in topology0) anti_linord:
shows (T{is anti-}($\lambda T. \text{IsLinOrder}(T, \{\langle U, V \rangle \in \text{Pow}(\bigcup T) \times \text{Pow}(\bigcup T) . U \subseteq V\})$))
 $\longleftrightarrow T \text{is } T_1$
<proof>

In conclusion, T_1 is also an anti-property.

Let's define some anti-properties that we'll use in the future.

definition

```
IsAntiComp (_{is anti-compact})
where T{is anti-compact}  $\equiv$  T{is anti-}( $\lambda T. (\bigcup T)\{is compact in\}T$ )
```

definition

```
IsAntiLin (_{is anti-lindeloeff})
where T{is anti-lindeloeff}  $\equiv$  T{is anti-}( $\lambda T. ((\bigcup T)\{is lindeloeff in\}T)$ )
```

Anti-compact spaces are also called pseudo-finite spaces in literature before the concept of anti-property was defined.

end

88 Topology 6

```
theory Topology_ZF_6 imports Topology_ZF_4b Topology_ZF_2 Topology_ZF_1
```

begin

This theory deals with the relations between continuous functions and convergence of filters. At the end of the file there some results about the building of functions in cartesian products.

88.1 Image filter

First of all, we will define the appropriate tools to work with functions and filters together.

We define the image filter as the collections of supersets of images of sets from a filter.

definition

```
ImageFilter (_[_].._ 98)
where  $\mathfrak{F}$  {is a filter on}  $X \implies f:X \rightarrow Y \implies f[\mathfrak{F}]..Y \equiv \{A \in \text{Pow}(Y). \exists D \in \{f(B) . B \in \mathfrak{F}\}. D \subseteq A\}$ 
```

Note that in the previous definition, it is necessary to state Y as the final set because f is also a function to every superset of its range. X can be changed by $\text{domain}(f)$ without any change in the definition.

lemma base_image_filter:

```
assumes  $\mathfrak{F}$  {is a filter on}  $X$   $f:X \rightarrow Y$ 
shows  $\{fB . B \in \mathfrak{F}\}$  {is a base filter}  $(f[\mathfrak{F}]..Y)$  and  $(f[\mathfrak{F}]..Y)$  {is a filter on}  $Y$ 
<proof>
```

88.2 Continuous at a point vs. globally continuous

In this section we show that continuity of a function implies local continuity (at a point) and that local continuity at all points implies (global) continuity.

If a function is continuous, then it is continuous at every point.

```
lemma cont_global_imp_continuous_x:
  assumes  $x \in \bigcup \tau_1$  IsContinuous( $\tau_1, \tau_2, f$ )  $f: (\bigcup \tau_1) \rightarrow (\bigcup \tau_2)$   $x \in \bigcup \tau_1$ 
  shows  $\forall U \in \tau_2. f(x) \in U \longrightarrow (\exists V \in \tau_1. x \in V \wedge f(V) \subseteq U)$ 
<proof>
```

A function that is continuous at every point of its domain is continuous.

```
lemma ccontinuous_all_x_imp_cont_global:
  assumes  $\forall x \in \bigcup \tau_1. \forall U \in \tau_2. fx \in U \longrightarrow (\exists V \in \tau_1. x \in V \wedge fV \subseteq U)$   $f \in (\bigcup \tau_1) \rightarrow (\bigcup \tau_2)$ 
  and
     $\tau_1$  {is a topology}
  shows IsContinuous( $\tau_1, \tau_2, f$ )
<proof>
```

88.3 Continuous functions and filters

In this section we consider the relations between filters and continuity.

If the function is continuous then if the filter converges to a point the image filter converges to the image point.

```
lemma (in two_top_spaces0) cont_imp_filter_conver_preserved:
  assumes  $\mathfrak{F}$  {is a filter on}  $X_1$   $f$  {is continuous}  $\mathfrak{F} \rightarrow_F x$  {in}  $\tau_1$ 
  shows  $(f[\mathfrak{F}]..X_2) \rightarrow_F (f(x))$  {in}  $\tau_2$ 
<proof>
```

Continuity in filter at every point of the domain implies global continuity.

```
lemma (in two_top_spaces0) filter_conver_preserved_imp_cont:
  assumes  $\forall x \in \bigcup \tau_1. \forall \mathfrak{F}. ((\mathfrak{F} \text{ {is a filter on} } X_1) \wedge (\mathfrak{F} \rightarrow_F x \text{ {in} } \tau_1))$ 
 $\longrightarrow ((f[\mathfrak{F}]..X_2) \rightarrow_F (fx) \text{ {in} } \tau_2)$ 
  shows  $f$  {is continuous}
<proof>
```

end

89 Topology 7

```
theory Topology_ZF_7 imports Topology_ZF_5
begin
```

89.1 Connection Properties

Another type of topological properties are the connection properties. These properties establish if the space is formed of several pieces or just one.

A space is connected iff there is no clopen set other than the empty set and the total set.

```
definition IsConnected (_{is connected} 70)
```

where $T \text{ \{is connected\}} \equiv \forall U. (U \in T \wedge (U \text{ \{is closed in\} } T)) \longrightarrow U = \emptyset \vee U = \bigcup T$

lemma indiscrete_connected:
 shows $\{0, X\} \text{ \{is connected\}}$
<proof>

The anti-property of connectedness is called total-diconnectedness.

definition IsTotDis ($_ \text{ \{is totally-disconnected\} } 70$)
 where $\text{IsTotDis} \equiv \text{ANTI}(\text{IsConnected})$

lemma conn_spectrum:
 shows $(A \text{ \{is in the spectrum of\} } \text{IsConnected}) \longleftrightarrow A \lesssim 1$
<proof>

The discrete space is a first example of totally-disconnected space.

lemma discrete_tot_dis:
 shows $\text{Pow}(X) \text{ \{is totally-disconnected\}}$
<proof>

An space is hyperconnected iff every two non-empty open sets meet.

definition IsHConnected ($_ \text{ \{is hyperconnected\} } 90$)
 where $T \text{ \{is hyperconnected\}} \equiv \forall U \ V. U \in T \wedge V \in T \wedge U \cap V = \emptyset \longrightarrow U = \emptyset \vee V = \emptyset$

Every hyperconnected space is connected.

lemma HConn_imp_Conn:
 assumes $T \text{ \{is hyperconnected\}}$
 shows $T \text{ \{is connected\}}$
<proof>

lemma Indiscrete_HConn:
 shows $\{0, X\} \text{ \{is hyperconnected\}}$
<proof>

A first example of an hyperconnected space but not indiscrete, is the cofinite topology on the natural numbers.

lemma Cofinite_nat_HConn:
 assumes $\neg(X < \text{nat})$
 shows $(\text{CoFinite } X) \text{ \{is hyperconnected\}}$
<proof>

lemma HConn_spectrum:
 shows $(A \text{ \{is in the spectrum of\} } \text{IsHConnected}) \longleftrightarrow A \lesssim 1$
<proof>

In the following results we will show that anti-hyperconnectedness is a separation property between T_1 and T_2 . We will show also that both implications are proper.

First, the closure of a point in every topological space is always hyperconnected. This is the reason why every anti-hyperconnected space must be T_1 : every singleton must be closed.

```
lemma (in topology0) cl_point_imp_HConn:
  assumes  $x \in \bigcup T$ 
  shows  $(T \text{ restricted to } \text{Closure}(\{x\}, T)) \text{ is hyperconnected}$ 
  <proof>
```

A consequence is that every totally-disconnected space is T_1 .

```
lemma (in topology0) tot_dis_imp_T1:
  assumes  $T \text{ is totally-disconnected}$ 
  shows  $T \text{ is } T_1$ 
  <proof>
```

In the literature, there exists a class of spaces called sober spaces; where the only non-empty closed hyperconnected subspaces are the closures of points and closures of different singletons are different.

```
definition IsSober ( $\_ \text{ is sober}$ )90
  where  $T \text{ is sober} \equiv \forall A \in \text{Pow}(\bigcup T) - \{0\}. (A \text{ is closed in } T \wedge ((T \text{ restricted to } A) \text{ is hyperconnected})) \longrightarrow (\exists x \in \bigcup T. A = \text{Closure}(\{x\}, T) \wedge (\forall y \in \bigcup T. A = \text{Closure}(\{y\}, T) \longrightarrow y = x))$ 
```

Being sober is weaker than being anti-hyperconnected.

```
theorem (in topology0) anti_HConn_imp_sober:
  assumes  $T \text{ is anti-} \text{IsHConnected}$ 
  shows  $T \text{ is sober}$ 
  <proof>
```

Every sober space is T_0 .

```
lemma (in topology0) sober_imp_T0:
  assumes  $T \text{ is sober}$ 
  shows  $T \text{ is } T_0$ 
  <proof>
```

Every T_2 space is anti-hyperconnected.

```
theorem (in topology0) T2_imp_anti_HConn:
  assumes  $T \text{ is } T_2$ 
  shows  $T \text{ is anti-} \text{IsHConnected}$ 
  <proof>
```

Every anti-hyperconnected space is T_1 .

```
theorem anti_HConn_imp_T1:
  assumes  $T \text{ is anti-} \text{IsHConnected}$ 
  shows  $T \text{ is } T_1$ 
  <proof>
```

There is at least one topological space that is T_1 , but not anti-hyperconnected. This space is the cofinite topology on the natural numbers.

lemma Cofinite_not_anti_HConn:
 shows $\neg((\text{CoFinite nat})\{\text{is anti-}\}\text{IsHConnected})$ and $(\text{CoFinite nat})\{\text{is } T_1\}$
<proof>

The join-topology build from the cofinite topology on the natural numbers, and the excluded set topology on the natural numbers excluding $\{0,1\}$; is just the union of both.

lemma join_top_cofinite_excluded_set:
 shows $(\text{joinT } \{\text{CoFinite nat}, \text{ExcludedSet}(\text{nat}, \{0,1\})\}) = (\text{CoFinite nat}) \cup \text{ExcludedSet}(\text{nat}, \{0,1\})$
<proof>

The previous topology is not T_2 , but is anti-hyperconnected.

theorem join_Cofinite_ExclPoint_not_T2:
 shows
 $\neg((\text{joinT } \{\text{CoFinite nat}, \text{ExcludedSet}(\text{nat}, \{0,1\})\})\{\text{is } T_2\})$ and
 $(\text{joinT } \{\text{CoFinite nat}, \text{ExcludedSet}(\text{nat}, \{0,1\})\})\{\text{is anti-}\} \text{IsHConnected}$
<proof>

Let's show that anti-hyperconnected is in fact T_1 and sober. The trick of the proof lies in the fact that if a subset is hyperconnected, its closure is so too (the closure of a point is then always hyperconnected because singletons are in the spectrum); since the closure is closed, we can apply the sober property on it.

theorem (in topology0) T1_sober_imp_anti_HConn:
 assumes $T\{\text{is } T_1\}$ and $T\{\text{is sober}\}$
 shows $T\{\text{is anti-}\}\text{IsHConnected}$
<proof>

theorem (in topology0) anti_HConn_iff_T1_sober:
 shows $(T\{\text{is anti-}\}\text{IsHConnected}) \longleftrightarrow (T\{\text{is sober}\} \wedge T\{\text{is } T_1\})$
<proof>

A space is ultraconnected iff every two non-empty closed sets meet.

definition IsUConnected ($_ \{\text{is ultraconnected}\}80$)
 where $T\{\text{is ultraconnected}\} \equiv \forall A B. A\{\text{is closed in}\}T \wedge B\{\text{is closed in}\}T \wedge A \cap B = 0 \longrightarrow A = 0 \vee B = 0$

Every ultraconnected space is trivially normal.

lemma (in topology0) UConn_imp_normal:
 assumes $T\{\text{is ultraconnected}\}$
 shows $T\{\text{is normal}\}$
<proof>

Every ultraconnected space is connected.

lemma UConn_imp_Conn:

```

    assumes T{is ultraconnected}
    shows T{is connected}
  <proof>

```

```

lemma UConn_spectrum:
  shows (A{is in the spectrum of}IsUConnected)  $\longleftrightarrow$   $A \lesssim 1$ 
  <proof>

```

This time, anti-ultraconnected is an old property.

```

theorem (in topology0) anti_UConn:
  shows (T{is anti-}IsUConnected)  $\longleftrightarrow$  T{is  $T_1$ }
  <proof>

```

Is is natural that separation axioms and connection axioms are anti-properties of each other; as the concepts of connectedness and separation are opposite.

To end this section, let's try to characterize anti-sober spaces.

```

lemma sober_spectrum:
  shows (A{is in the spectrum of}IsSober)  $\longleftrightarrow$   $A \lesssim 1$ 
  <proof>

```

```

theorem (in topology0)anti_sober:
  shows (T{is anti-}IsSober)  $\longleftrightarrow$   $T = \{0, \bigcup T\}$ 
  <proof>

```

end

90 Topology 8

```

theory Topology_ZF_8 imports Topology_ZF_6 EquivClass1
begin

```

Suppose T is a topology, r is an equivalence relation on $X = \bigcup T$ and $P_r : X \rightarrow X/r$ maps an element of X to its equivalence class $r\{x\}$. Then we can define a topology (on X/r) by taking the collection of those subsets V of X/r for which the inverse image by the projection P_r is in T . This is the weakest topology on X/r such that P_r is continuous. In this theory we consider a seemingly more general situation where we start with a topology T on $X = \bigcup T$ and a surjection $f : X \rightarrow Y$ and define a topology on Y by taking those subsets V of Y for which the inverse image by the mapping f is in T . Turns out that this construction is in a way equivalent to the previous one as the topology defined this way is homeomorphic to the topology defined by the equivalence relation r_f on X that relates two elements of X if f has the same value on them.

90.1 Definition of quotient topology

In this section we define the quotient topology generated by a topology T and a surjection $f : \bigcup T \rightarrow Y$, and show its basic properties.

For a topological space $X = \bigcup T$ and a surjection $f : X \rightarrow Y$ we define $\{\text{quotient topology in } Y \text{ by } f\}$ as the collection of subsets of Y whose inverse images by f are open.

definition (in topology0)

```
QuotientTop ({quotient topology in}_by_ 80)
  where f ∈ surj(⋃ T, Y) ⇒ {quotient topology in} Y {by} f ≡
    {U ∈ Pow(Y). f- U ∈ T}
```

Outside of the topology0 context we will indicate also the generating topology and write $\{\text{quotient topology in } Y \text{ by } f \text{ from } X\}$.

abbreviation QuotientTopTop ({quotient topology in}_by_{from}_)

```
  where {quotient topology in} Y {by} f {from} T ≡ topology0.QuotientTop(T, Y, f)
```

The quotient topology is indeed a topology.

theorem (in topology0) quotientTop_is_top:

```
  assumes f ∈ surj(⋃ T, Y)
  shows ({quotient topology in} Y {by} f) {is a topology}
  ⟨proof⟩
```

The quotient function is continuous.

lemma (in topology0) quotient_func_cont:

```
  assumes f ∈ surj(⋃ T, Y)
  shows IsContinuous(T, ({quotient topology in} Y {by} f), f)
  ⟨proof⟩
```

One of the important properties of this topology, is that a function from the quotient space is continuous iff the composition with the quotient function is continuous.

theorem (in two_top_spaces0) cont_quotient_top:

```
  assumes h ∈ surj(⋃ τ1, Y) g: Y → ⋃ τ2 IsContinuous(τ1, τ2, g ∘ h)
  shows IsContinuous(({quotient topology in} Y {by} h {from} τ1), τ2, g)
  ⟨proof⟩
```

The underlying set of the quotient topology is Y .

lemma (in topology0) total_quo_func:

```
  assumes f ∈ surj(⋃ T, Y)
  shows (⋃ ({quotient topology in} Y {by} f)) = Y
  ⟨proof⟩
```

90.2 Quotient topologies from equivalence relations

In this section we will show that the quotient topologies come from an equivalence relation.

The quotient projection $b \mapsto r\{b\}$ is a function that maps the domain of the relation to the quotient. Note we do not need to assume that r is an equivalence relation.

lemma quotient_proj_fun:
 shows $\{\langle b, r\{b\} \rangle. b \in A\} : A \rightarrow A // r$ *<proof>*

The quotient projection is a surjection. Again r does not need to be an equivalence relation here

lemma quotient_proj_surj:
 shows $\{\langle b, r\{b\} \rangle. b \in A\} \in \text{surj}(A, A // r)$
<proof>

The inverse image of a subset U of the quotient by the quotient projection is the union of U . Note since U is a subset of A/r it is a collection of equivalence classes.

lemma preim_equi_proj:
 assumes $U \subseteq A // r$ equiv(A,r)
 shows $\{\langle b, r\{b\} \rangle. b \in A\} - (U) = \bigcup U$
<proof>

Now we define what a quotient topology from an equivalence relation is:

definition (in topology0)
 EquivQuo ($\{\text{quotient by}\} _ 70$)
 where $\text{equiv}(\bigcup T, r) \implies (\{\text{quotient by}\} r) \equiv \{\text{quotient topology in}\} (\bigcup T) // r$
 $\{\text{by}\} \{\langle b, r\{b\} \rangle. b \in \bigcup T\}$

Outside of the topology0 context we need to indicate the original topology.

abbreviation EquivQuoTop ($_ \{\text{quotient by}\} _$)
 where $T \{\text{quotient by}\} r \equiv \text{topology0.EquivQuo}(T, r)$

First, another description of the topology (more intuitive):

theorem (in topology0) quotient_equiv_rel:
 assumes $\text{equiv}(\bigcup T, r)$
 shows $(\{\text{quotient by}\} r) = \{U \in \text{Pow}((\bigcup T) // r). \bigcup U \in T\}$
<proof>

We apply previous results to this topology.

theorem (in topology0) total_quo_equi:
 assumes $\text{equiv}(\bigcup T, r)$
 shows $\bigcup (\{\text{quotient by}\} r) = (\bigcup T) // r$
<proof>

The quotient by an equivalence relation is indeed a topology.

theorem (in topology0) equiv_quo_is_top:
 assumes $\text{equiv}(\bigcup T, r)$
 shows $(\{\text{quotient by}\} r) \{\text{is a topology}\}$

$\langle proof \rangle$

The next theorem is the main result of this section: all quotient topologies arise from an equivalence relation given by the quotient function $f : X \rightarrow Y$. This means that any quotient topology is homeomorphic to a topology given by an equivalence relation quotient.

theorem (in topology0) equiv_quotient_top:
 assumes $f \in \text{surj}(\bigcup T, Y)$
 defines $r \equiv \{ \langle x, y \rangle \in \bigcup T \times \bigcup T. f(x) = f(y) \}$
 defines $g \equiv \{ \langle y, f^{-1}\{y\} \rangle. y \in Y \}$
 shows $\text{equiv}(\bigcup T, r)$ and
 $\text{IsAhomeomorphism}((\{\text{quotient topology in}\} Y \text{ by } f), (\{\text{quotient by}\} r), g)$
 $\langle proof \rangle$

The mapping $\langle b, c \rangle \mapsto \langle r\{a\}, r\{b\} \rangle$ is a function that maps the product of the carrier by itself to the product of the quotients. Note r does not have to be an equivalence relation.

lemma product_equiv_rel_fun:
 shows $\{ \langle \langle b, c \rangle, \langle r\{b\}, r\{c\} \rangle \rangle. \langle b, c \rangle \in \bigcup T \times \bigcup T \} : (\bigcup T \times \bigcup T) \rightarrow ((\bigcup T) // r \times (\bigcup T) // r)$
 $\langle proof \rangle$

The mapping $\langle b, c \rangle \mapsto \langle r\{a\}, r\{b\} \rangle$ is a surjection of the product of the carrier by itself onto the carrier of the product topology. Again r does not have to be an equivalence relation for this.

lemma (in topology0) prod_equiv_rel_surj:
 shows $\{ \langle \langle b, c \rangle, \langle r\{b\}, r\{c\} \rangle \rangle. \langle b, c \rangle \in \bigcup T \times \bigcup T \} \in \text{surj}(\bigcup (T \times_t T), ((\bigcup T) // r \times (\bigcup T) // r))$
 $\langle proof \rangle$

The product quotient projection (i.e. the mapping the mapping $\langle b, c \rangle \mapsto \langle r\{a\}, r\{b\} \rangle$ is continuous.

lemma (in topology0) product_quo_fun:
 assumes $\text{equiv}(\bigcup T, r)$
 shows
 $\text{IsContinuous}(T \times_t T, (\{\text{quotient by}\} r) \times_t (\{\text{quotient by}\} r), \{ \langle \langle b, c \rangle, \langle r\{b\}, r\{c\} \rangle \rangle. \langle b, c \rangle \in \bigcup T \times \bigcup T \})$
 $\langle proof \rangle$

The product of quotient topologies is a quotient topology given that the quotient map is open. This isn't true in general.

theorem (in topology0) prod_quotient:
 assumes $\text{equiv}(\bigcup T, r) \ \forall A \in T. \{ \langle b, r\{b\} \rangle. b \in \bigcup T \} (A) \in (\{\text{quotient by}\} r)$
 shows $((\{\text{quotient by}\} r) \times_t \{\text{quotient by}\} r) =$
 $(\{\text{quotient topology in}\} ((\bigcup T) // r) \times ((\bigcup T) // r)) \ \{\text{by}\} \{ \langle \langle b, c \rangle, \langle r\{b\}, r\{c\} \rangle \rangle. \langle b, c \rangle \in \bigcup T \times \bigcup T \} \ \{\text{from}\} (T \times_t T))$
 $\langle proof \rangle$

end

91 Homeomorphisms as a group connecting topology and group theory

```
theory Topology_ZF_9
imports Topology_ZF_2 Group_ZF_2 Topology_ZF_7 Topology_ZF_8
begin
```

The collection of all homeomorphisms of a topological space onto itself forms a group under composition. This file develops that group structure, connecting the theory of topological spaces with the algebraic theory of groups.

91.1 Group of homeomorphisms

We show that the homeomorphisms of a topological space onto itself form a group under composition, and compute this group for several concrete topologies.

First, we define the set of homeomorphisms.

definition

$$\text{HomeoG}(T) \equiv \{f: \bigcup T \rightarrow \bigcup T. \text{IsAhomeomorphism}(T, T, f)\}$$

The homeomorphisms are closed by composition.

```
lemma (in topology0) homeo_composition:
  assumes f∈HomeoG(T) g∈HomeoG(T)
  shows Composition( $\bigcup T$ )⟨f, g⟩∈HomeoG(T)
⟨proof⟩
```

The identity function is a homeomorphism.

```
lemma (in topology0) homeo_id:
  shows id( $\bigcup T$ )∈HomeoG(T)
⟨proof⟩
```

The homeomorphisms form a monoid and its neutral element is the identity.

```
theorem (in topology0) homeo_submonoid:
  shows IsAmonoid(HomeoG(T), restrict(Composition( $\bigcup T$ ), HomeoG(T) × HomeoG(T)))
```

```
  TheNeutralElement(HomeoG(T), restrict(Composition( $\bigcup T$ ), HomeoG(T) × HomeoG(T)))=id( $\bigcup T$ )
⟨proof⟩
```

The homeomorphisms form a group, with the composition.

```
theorem (in topology0) homeo_group:
  shows IsAgroup(HomeoG(T), restrict(Composition( $\bigcup T$ ), HomeoG(T) × HomeoG(T)))
⟨proof⟩
```

91.2 Examples computed

We compute HomeoG for several concrete topologies: the cocardinal topology, the excluded set topology, the included set topology, and the order topology.

For the cocardinal topology every bijection is a homeomorphism, so HomeoG equals $\text{bij}(X, X)$.

theorem homeo_cocardinal:
assumes InfCard(Q)
shows HomeoG(CoCardinal(X,Q))= $\text{bij}(X, X)$
<proof>

The homeomorphisms of the excluded set topology are exactly the bijections of X that map $X \setminus T$ to itself.

theorem homeo_excluded:
shows HomeoG(ExcludedSet(X,T))= $\{f \in \text{bij}(X, X) . f(X-T) = (X-T)\}$
<proof>

Continuity in the included set topology implies continuity in the excluded set topology.

lemma cont_in_cont_ex:
assumes IsContinuous(IncludedSet(X,T), IncludedSet(X,T), f) $f: X \rightarrow X$ $T \subseteq X$
shows IsContinuous(ExcludedSet(X,T), ExcludedSet(X,T), f)
<proof>

Continuity in the excluded set topology implies continuity in the included set topology.

lemma cont_ex_cont_in:
assumes IsContinuous(ExcludedSet(X,T), ExcludedSet(X,T), f) $f: X \rightarrow X$ $T \subseteq X$
shows IsContinuous(IncludedSet(X,T), IncludedSet(X,T), f)
<proof>

By the previous two lemmas, the homeomorphism groups of the included and excluded set topologies on X coincide.

lemma homeo_included:
assumes $T \subseteq X$
shows HomeoG(IncludedSet(X,T))= $\{f \in \text{bij}(X, X) . f(X - T) = X - T\}$
<proof>

Every order isomorphism of a linearly ordered set is a homeomorphism of the induced order topology.

lemma homeo_order:
assumes IsLinOrder(X,r) $\exists x y. x \neq y \wedge x \in X \wedge y \in X$
shows $\text{ord_iso}(X, r, X, r) \subseteq \text{HomeoG}(\text{OrdTopology } X \ r)$
<proof>

91.3 Properties preserved by functions

Continuous surjections preserve connectedness and compactness. As consequences, quotient spaces of connected or compact spaces inherit those properties.

The continuous surjective image of a connected space is connected.

```
theorem (in two_top_spaces0) cont_image_conn:
  assumes IsContinuous( $\tau_1, \tau_2, f$ )  $f \in \text{surj}(X_1, X_2)$   $\tau_1\{\text{is connected}\}$ 
  shows  $\tau_2\{\text{is connected}\}$ 
  <proof>
```

A continuous function from a connected space to a totally-disconnected space must be constant.

```
corollary(in two_top_spaces0) cont_conn_tot_disc:
  assumes IsContinuous( $\tau_1, \tau_2, f$ )  $\tau_1\{\text{is connected}\}$   $\tau_2\{\text{is totally-disconnected}\}$ 
   $f: X_1 \rightarrow X_2$   $X_1 \neq 0$ 
  shows  $\exists q \in X_2. \forall w \in X_1. f(w) = q$ 
  <proof>
```

The continuous image of a compact space is compact.

```
theorem (in two_top_spaces0) cont_image_com:
  assumes IsContinuous( $\tau_1, \tau_2, f$ )  $f \in \text{surj}(X_1, X_2)$   $X_1\{\text{is compact of cardinal } K\}$   $\tau_1$ 
  shows  $X_2\{\text{is compact of cardinal } K\}$   $\tau_2$ 
  <proof>
```

Analogously to the connected case, the continuous image of a compact space in an anti-compact space is finite.

```
corollary (in two_top_spaces0) cont_comp_anti_comp:
  assumes IsContinuous( $\tau_1, \tau_2, f$ )  $X_1\{\text{is compact in } \tau_1\}$   $\tau_2\{\text{is anti-compact}\}$ 
   $f: X_1 \rightarrow X_2$   $X_1 \neq 0$ 
  shows Finite(range( $f$ )) and range( $f$ )  $\neq 0$ 
  <proof>
```

A quotient of a compact space is compact.

```
corollary(in topology0) compQuot:
  assumes  $(\bigcup T)\{\text{is compact in } T \text{ equiv } (\bigcup T, r)\}$ 
  shows  $(\bigcup T) // r\{\text{is compact in } (\{\text{quotient by } r\})\}$ 
  <proof>
```

A quotient of a connected space is connected.

```
corollary(in topology0) ConnQuot:
  assumes  $T\{\text{is connected}\}$  equiv  $(\bigcup T, r)$ 
  shows  $(\{\text{quotient by } r\})\{\text{is connected}\}$ 
  <proof>
```

end

92 Topology 10

```
theory Topology_ZF_10
imports Topology_ZF_7
begin
```

This file deals with properties of product spaces. We consider only the product of two spaces, but most of these results extend to finite products.

92.1 Closure and closed sets in product space

This section shows that closure and closed sets interact well with the product topology: the closure of a product is the product of the closures, and a product of closed sets is closed.

The closure of $A \times B$ in the product topology equals $\overline{A} \times \overline{B}$.

```
lemma cl_product:
  assumes T{is a topology} S{is a topology} A $\subseteq$  $\bigcup$ T B $\subseteq$  $\bigcup$ S
  shows Closure(A $\times$ B,ProductTopology(T,S))=Closure(A,T) $\times$ Closure(B,S)
<proof>
```

The product of closed sets is closed in the product topology.

```
corollary closed_product:
  assumes T{is a topology} S{is a topology} A{is closed in}T B{is closed
in}S
  shows (A $\times$ B) {is closed in}ProductTopology(T,S)
<proof>
```

92.2 Separation properties in product space

Products preserve the separation axioms T_0 , T_1 , T_2 , and regularity.

The product of T_0 spaces is T_0 .

```
theorem T0_product:
  assumes T{is a topology}S{is a topology}T{is T0}S{is T0}
  shows ProductTopology(T,S){is T0}
<proof>
```

The product of T_1 spaces is T_1 .

```
theorem T1_product:
  assumes T{is a topology}S{is a topology}T{is T1}S{is T1}
  shows ProductTopology(T,S){is T1}
<proof>
```

The product of T_2 spaces is T_2 .

```
theorem T2_product:
  assumes T{is a topology}S{is a topology}T{is T2}S{is T2}
```

```

    shows ProductTopology(T,S){is T2}
  <proof>

```

The product of regular spaces is regular.

```

theorem regular_product:
  assumes T{is a topology} S{is a topology} T{is regular} S{is regular}
  shows ProductTopology(T,S){is regular}
  <proof>

```

92.3 Connection properties in product space

We prove that the product of two connected spaces is connected. The key lemmas show that both coordinate projections map product-open sets to open sets.

An open set in the product topology projects to an open set in T .

```

lemma projection_open:
  assumes T{is a topology}S{is a topology}B∈ProductTopology(T,S)
  shows {y∈⋃T. ∃x∈⋃S. ⟨y,x⟩∈B}∈T
  <proof>

```

An open set in the product topology projects to an open set in S .

```

lemma projection_open2:
  assumes T{is a topology}S{is a topology}B∈ProductTopology(T,S)
  shows {y∈⋃S. ∃x∈⋃T. ⟨x,y⟩∈B}∈S
  <proof>

```

The product of connected spaces is connected.

```

theorem compact_product:
  assumes T{is a topology}S{is a topology}T{is connected}S{is connected}
  shows ProductTopology(T,S){is connected}
  <proof>

```

end

93 Order topology connecting order theory and topology

```

theory Topology_ZF_11 imports Topology_ZF_7 Finite_ZF_1

```

```

begin

```

This file deals with order topologies. The order topology is already defined in `Topology_ZF_examples_1.thy`.

93.1 Order topologies

We will assume most of the time that the ordered set has more than one point. It is natural to think that the topological properties can be translated to properties of the order; since every order rises one and only one topology in a set.

93.2 Separation properties

Order topologies have a lot of separation properties.

Every order topology is Hausdorff.

theorem order_top_T2:

assumes IsLinOrder(X,r) $\exists x y. x \neq y \wedge x \in X \wedge y \in X$
shows (OrdTopology X r){is T₂}

<proof>

Every order topology is T_4 , but the proof needs lots of machinery. At the end of the file, we will prove that every order topology is normal; sooner or later.

93.3 Connectedness properties

Connectedness is related to two properties of orders: completeness and density

Some order-dense properties:

definition

IsDenseSub (_ {is dense in}_ {with respect to}_) where
A {is dense in}X{with respect to}r $\equiv$
 $\forall x \in X. \forall y \in X. \langle x, y \rangle \in r \wedge x \neq y \rightarrow (\exists z \in A - \{x, y\}. \langle x, z \rangle \in r \wedge \langle z, y \rangle \in r)$

definition

IsDenseUnp (_ {is not-properly dense in}_ {with respect to}_) where
A {is not-properly dense in}X{with respect to}r $\equiv$
 $\forall x \in X. \forall y \in X. \langle x, y \rangle \in r \wedge x \neq y \rightarrow (\exists z \in A. \langle x, z \rangle \in r \wedge \langle z, y \rangle \in r)$

definition

IsWeaklyDenseSub (_ {is weakly dense in}_ {with respect to}_) where
A {is weakly dense in}X{with respect to}r $\equiv$
 $\forall x \in X. \forall y \in X. \langle x, y \rangle \in r \wedge x \neq y \rightarrow ((\exists z \in A - \{x, y\}. \langle x, z \rangle \in r \wedge \langle z, y \rangle \in r) \vee \text{Interval}X(X, r, x, y) = 0)$

definition

IsDense (_ {is dense with respect to}_) where
X {is dense with respect to}r $\equiv$
 $\forall x \in X. \forall y \in X. \langle x, y \rangle \in r \wedge x \neq y \rightarrow (\exists z \in X - \{x, y\}. \langle x, z \rangle \in r \wedge \langle z, y \rangle \in r)$

lemma dense_sub:
 shows $(X \text{ \{is dense with respect to\} } r) \longleftrightarrow (X \text{ \{is dense in\} } X \text{ \{with respect to\} } r)$
<proof>

lemma not_prop_dense_sub:
 shows $(A \text{ \{is dense in\} } X \text{ \{with respect to\} } r) \longrightarrow (A \text{ \{is not-properly dense in\} } X \text{ \{with respect to\} } r)$
<proof>

In densely ordered sets, intervals are infinite.

theorem dense_order_inf_intervals:
 assumes IsLinOrder(X,r) IntervalX(X, r, b, c) $\neq 0$ $b \in X$ $c \in X$ X\{is dense with respect to\}r
 shows $\neg \text{Finite}(\text{IntervalX}(X, r, b, c))$
<proof>

Left rays are infinite.

theorem dense_order_inf_lrays:
 assumes IsLinOrder(X,r) LeftRayX(X,r,c) $\neq 0$ $c \in X$ X\{is dense with respect to\}r
 shows $\neg \text{Finite}(\text{LeftRayX}(X, r, c))$
<proof>

Right rays are infinite.

theorem dense_order_inf_rrays:
 assumes IsLinOrder(X,r) RightRayX(X,r,b) $\neq 0$ $b \in X$ X\{is dense with respect to\}r
 shows $\neg \text{Finite}(\text{RightRayX}(X, r, b))$
<proof>

The whole space in a densely ordered set is infinite.

corollary dense_order_infinite:
 assumes IsLinOrder(X,r) X\{is dense with respect to\}r
 $\exists x y. x \neq y \wedge x \in X \wedge y \in X$
 shows $\neg (X \prec \text{nat})$
<proof>

If an order topology is connected, then the order is complete. It is equivalent to assume that $r \subseteq X \times X$ or prove that $r \cap X \times X$ is complete.

theorem conn_imp_complete:
 assumes IsLinOrder(X,r) $\exists x y. x \neq y \wedge x \in X \wedge y \in X$ $r \subseteq X \times X$
 (OrdTopology X r)\{is connected\}
 shows $r \text{ \{is complete\} }$
<proof>

If an order topology is connected, then the order is dense.

theorem conn_imp_dense:


```

assumes IsLinOrder(X,r)  $\exists x y. x \neq y \wedge x \in X \wedge y \in X$ 
      (OrdTopology X r){is connected}
shows X {is dense with respect to}r
<proof>

```

Actually a connected order topology is one that comes from a dense and complete order.

First a lemma. In a complete ordered set, every non-empty set bounded from below has a maximum lower bound.

```

lemma complete_order_bounded_below:
  assumes r{is complete} IsBoundedBelow(A,r)  $A \neq \emptyset$   $r \subseteq X \times X$ 
  shows HasAmaximum( $r, \bigcap c \in A. r - \{c\}$ )
<proof>

```

```

theorem comp_dense_imp_conn:
  assumes IsLinOrder(X,r)  $\exists x y. x \neq y \wedge x \in X \wedge y \in X$   $r \subseteq X \times X$ 
      X {is dense with respect to}r r{is complete}
  shows (OrdTopology X r){is connected}
<proof>

```

93.4 Numerability axioms

A κ -separable order topology is in relation with order density.

If an order topology has a subset A which is topologically dense, then that subset is weakly order-dense in X .

```

lemma dense_top_imp_Wdense_ord:
  assumes IsLinOrder(X,r)  $\text{Closure}(A, \text{OrdTopology } X \text{ } r) = X$   $A \subseteq X$   $\exists x y. x \neq y \wedge x \in X \wedge y \in X$ 
  shows A{is weakly dense in}X{with respect to}r
<proof>

```

Conversely, a weakly order-dense set is topologically dense if it is also considered that: if there is a maximum or a minimum elements whose singletons are open, this points have to be in A . In conclusion, weakly order-density is a property closed to topological density.

Another way to see this: Consider a weakly order-dense set A :

- If X has a maximum and a minimum and $\{min, max\}$ is open: A is topologically dense in $X \setminus \{min, max\}$, where min is the minimum in X and max is the maximum in X .
- If X has a maximum, $\{max\}$ is open and X has no minimum or $\{min\}$ isn't open: A is topologically dense in $X \setminus \{max\}$, where max is the maximum in X .

- If X has a minimum, $\{min\}$ is open and X has no maximum or $\{max\}$ isn't open A is topologically dense in $X \setminus \{min\}$, where min is the minimum in X .
- If X has no minimum or maximum, or $\{min, max\}$ has no proper open sets: A is topologically dense in X .

lemma `Wdense_ord_imp_dense_top`:
assumes `IsLinOrder(X,r)` A {is weakly dense in} X {with respect to} r $A \subseteq X$
 $\exists x y. x \neq y \wedge x \in X \wedge y \in X$
`HasAminimum(r,X)  $\longrightarrow$  {Minimum(r,X)}  $\in$  (OrdTopology X r)  $\longrightarrow$  Minimum(r,X)  $\in A$`
`HasAmaximum(r,X)  $\longrightarrow$  {Maximum(r,X)}  $\in$  (OrdTopology X r)  $\longrightarrow$  Maximum(r,X)  $\in A$`
shows `Closure(A,OrdTopology X r)=X`
<proof>

The conclusion is that an order topology is κ -separable iff there is a set A with cardinality strictly less than κ which is weakly-dense in X .

theorem `separable_imp_wdense`:
assumes `(OrdTopology X r){is separable of cardinal} $\}Q$   $\exists x y. x \neq y \wedge x \in X \wedge y \in X$`
`IsLinOrder(X,r)`
shows $\exists A \in \text{Pow}(X). A \prec Q \wedge (A \text{ is weakly dense in } X \text{ with respect to } r)$
<proof>

theorem `wdense_imp_separable`:
assumes $\exists x y. x \neq y \wedge x \in X \wedge y \in X$ $(A \text{ is weakly dense in } X \text{ with respect to } r)$
`IsLinOrder(X,r)` $A \prec Q$ `InfCard(Q)` $A \subseteq X$
shows `(OrdTopology X r){is separable of cardinal} $\}Q$`
<proof>

end

94 Properties in topology 2

theory `Topology_ZF_properties_2` **imports** `Topology_ZF_7` `Topology_ZF_1`
`Finite_ZF_1` `Topology_ZF_11`

begin

94.1 Local properties.

This theory file deals with local topological properties; and applies local compactness to the one point compactification.

We will say that a topological space is locally @term"P" iff every point has a neighbourhood basis of subsets that have the property @term"P" as subspaces.

definition

```
IsLocally (_{is locally}_ 90)
  where T{is a topology}  $\implies$  T{is locally}P  $\equiv (\forall x \in \bigcup T. \forall b \in T. x \in b \longrightarrow$ 
   $(\exists c \in \text{Pow}(b). x \in \text{Interior}(c, T) \wedge P(c, T)))$ 
```

94.2 First examples

Our first examples deal with the locally finite property. Finiteness is a property of sets, and hence it is preserved by homeomorphisms; which are in particular bijective.

The discrete topology is locally finite.

```
lemma discrete_locally_finite:
  shows Pow(A){is locally}( $\lambda A. (\lambda B. \text{Finite}(A))$ )
  <proof>
```

The included set topology is locally finite when the set is finite.

```
lemma included_finite_locally_finite:
  assumes Finite(A) and  $A \subseteq X$ 
  shows (IncludedSet(X, A)){is locally}( $\lambda A. (\lambda B. \text{Finite}(A))$ )
  <proof>
```

94.3 Local compactness

definition

```
IsLocallyComp (_{is locally-compact}_ 70)
  where T{is locally-compact} $\equiv$ T{is locally}( $\lambda B. \lambda T. B\{\text{is compact in}\}T$ )
```

We center ourselves in local compactness, because it is a very important tool in topological groups and compactifications.

If a subset is compact of some cardinal for a topological space, it is compact of the same cardinal in the subspace topology.

```
lemma compact_imp_compact_subspace:
  assumes A{is compact of cardinal}K{in}T  $A \subseteq B$ 
  shows A{is compact of cardinal}K{in}(T{restricted to}B) <proof>
```

The converse of the previous result is not always true. For compactness, it holds because the axiom of finite choice always holds.

```
lemma compact_subspace_imp_compact:
  assumes A{is compact in}(T{restricted to}B)  $A \subseteq B$ 
  shows A{is compact in}T <proof>
```

If the axiom of choice holds for some cardinal, then we can drop the compact sets of that cardinal are compact of the same cardinal as subspaces of every superspace.

```
lemma Kcompact_subspace_imp_Kcompact:
```

assumes $A\{\text{is compact of cardinal}\}Q\{\text{in}\}(T\{\text{restricted to}\}B) \ A \subseteq B \ (\{\text{the axiom of}\} \ Q \ \{\text{choice holds}\})$
shows $A\{\text{is compact of cardinal}\}Q\{\text{in}\}T$
<proof>

Every set, with the cofinite topology is compact.

lemma `cofinite_compact`:
shows $X \ \{\text{is compact in}\}(\text{CoFinite } X) \ \langle\text{proof}\rangle$

A corollary is then that the cofinite topology is locally compact; since every subspace of a cofinite space is cofinite.

corollary `cofinite_locally_compact`:
shows $(\text{CoFinite } X)\{\text{is locally-compact}\}$
<proof>

In every locally compact space, by definition, every point has a compact neighbourhood.

theorem $(\text{in topology0}) \ \text{locally_compact_exist_compact_neig}$:
assumes $T\{\text{is locally-compact}\}$
shows $\forall x \in \bigcup T. \ \exists A \in \text{Pow}(\bigcup T). \ A\{\text{is compact in}\}T \wedge x \in \text{int}(A)$
<proof>

In Hausdorff spaces, the previous result is an equivalence.

theorem $(\text{in topology0}) \ \text{exist_compact_neig_T2_imp_locally_compact}$:
assumes $\forall x \in \bigcup T. \ \exists A \in \text{Pow}(\bigcup T). \ x \in \text{int}(A) \wedge A\{\text{is compact in}\}T \ T\{\text{is } T_2\}$
shows $T\{\text{is locally-compact}\}$
<proof>

94.4 Compactification by one point

Given a topological space, we can always add one point to the space and get a new compact topology; as we will check in this section.

definition
 $\text{OPCompactification } (\{\text{one-point compactification of}\}_90)$
where $\{\text{one-point compactification of}\}T \equiv T \cup \{(\bigcup T) - K\}. \ K \in \{B \in \text{Pow}(\bigcup T). \ B\{\text{is compact in}\}T \wedge B\{\text{is closed in}\}T\}$

Firstly, we check that what we defined is indeed a topology.

theorem $(\text{in topology0}) \ \text{op_comp_is_top}$:
shows $(\{\text{one-point compactification of}\}T)\{\text{is a topology}\} \ \langle\text{proof}\rangle$

The original topology is an open subspace of the new topology.

theorem $(\text{in topology0}) \ \text{open_subspace}$:
shows $\bigcup T \in \{\text{one-point compactification of}\}T \ \text{and} \ (\{\text{one-point compactification of}\}T)\{\text{restricted to}\}\bigcup T = T$
<proof>

We added only one new point to the space.

```
lemma (in topology0) op_compact_total:
  shows  $\bigcup (\{\text{one-point compactification of } T\}) = \{\bigcup T\} \cup (\bigcup T)$ 
  <proof>
```

The one point compactification, gives indeed a compact topological space.

```
theorem (in topology0) compact_op:
  shows  $(\{\bigcup T\} \cup (\bigcup T)) \{\text{is compact in}\} (\{\text{one-point compactification of } T\})$ 
  <proof>
```

The one point compactification is Hausdorff iff the original space is also Hausdorff and locally compact.

```
lemma (in topology0) op_compact_T2_1:
  assumes  $(\{\text{one-point compactification of } T\}) \{\text{is } T_2\}$ 
  shows  $T \{\text{is } T_2\}$ 
  <proof>
```

```
lemma (in topology0) op_compact_T2_2:
  assumes  $(\{\text{one-point compactification of } T\}) \{\text{is } T_2\}$ 
  shows  $T \{\text{is locally-compact}\}$ 
  <proof>
```

```
lemma (in topology0) op_compact_T2_3:
  assumes  $T \{\text{is locally-compact}\} \ T \{\text{is } T_2\}$ 
  shows  $(\{\text{one-point compactification of } T\}) \{\text{is } T_2\}$ 
  <proof>
```

In conclusion, every locally compact Hausdorff topological space is regular; since this property is hereditary.

```
corollary (in topology0) locally_compact_T2_imp_regular:
  assumes  $T \{\text{is locally-compact}\} \ T \{\text{is } T_2\}$ 
  shows  $T \{\text{is regular}\}$ 
  <proof>
```

This last corollary has an explanation: In Hausdorff spaces, compact sets are closed and regular spaces are exactly the "locally closed spaces" (those which have a neighbourhood basis of closed sets). So the neighbourhood basis of compact sets also works as the neighbourhood basis of closed sets we needed to find.

```
definition
  IsLocallyClosed (_{is locally-closed})
  where  $T \{\text{is locally-closed}\} \equiv T \{\text{is locally}\} (\lambda B \ TT. \ B \{\text{is closed in}\} TT)$ 
```

```
lemma (in topology0) regular_locally_closed:
  shows  $T \{\text{is regular}\} \longleftrightarrow (T \{\text{is locally-closed}\})$ 
  <proof>
```

94.5 Hereditary properties and local properties

In this section, we prove a relation between a property and its local property for hereditary properties. Then we apply it to locally-Hausdorff or locally- T_2 . We also prove the relation between locally- T_2 and another property that appeared when considering anti-properties, the anti-hyperconnectedness.

If a property is hereditary in open sets, then local properties are equivalent to find just one open neighbourhood with that property instead of a whole local basis.

lemma (in topology0) her_P_is_loc_P:
 assumes $\forall TT. \forall B \in \text{Pow}(\bigcup TT). \forall A \in TT. TT\{\text{is a topology}\} \wedge P(B, TT) \longrightarrow P(B \cap A, TT)$
 shows $(T\{\text{is locally}\}P) \longleftrightarrow (\forall x \in \bigcup T. \exists A \in T. x \in A \wedge P(A, T))$
<proof>

definition

IsLocallyT2 ($_ \{\text{is locally-}T_2\}$ 70)
 where $T\{\text{is locally-}T_2\} \equiv T\{\text{is locally}\}(\lambda B. \lambda T. (T\{\text{restricted to}\}B)\{\text{is } T_2\})$

Since T_2 is an hereditary property, we can apply the previous lemma.

corollary (in topology0) loc_T2:
 shows $(T\{\text{is locally-}T_2\}) \longleftrightarrow (\forall x \in \bigcup T. \exists A \in T. x \in A \wedge (T\{\text{restricted to}\}A)\{\text{is } T_2\})$
<proof>

First, we prove that a locally- T_2 space is anti-hyperconnected.

Before starting, let's prove that an open subspace of an hyperconnected space is hyperconnected.

lemma(in topology0) open_subspace_hyperconn:
 assumes $T\{\text{is hyperconnected}\} U \in T$
 shows $(T\{\text{restricted to}\}U)\{\text{is hyperconnected}\}$
<proof>

lemma(in topology0) locally_T2_is_antiHConn:
 assumes $T\{\text{is locally-}T_2\}$
 shows $T\{\text{is anti-}\}\text{IsHConnected}$
<proof>

Now we find a counter-example for: Every anti-hyperconnected space is locally-Hausdorff.

The example we are going to consider is the following. Put in X an anti-hyperconnected topology, where an infinite number of points don't have finite sets as neighbourhoods. Then add a new point to the set, $p \notin X$. Consider the open sets on $X \cup p$ as the anti-hyperconnected topology and the open sets that contain p are $p \cup A$ where $X \setminus A$ is finite.

This construction equals the one-point compactification iff X is anti-compact; i.e., the only compact sets are the finite ones. In general this topology is contained in the one-point compactification topology, making it compact too.

It is easy to check that any open set containing p meets infinite other non-empty open set. The question is if such a topology exists.

```
theorem (in topology0) COF_comp_is_top:
  assumes  $T\{is\ T_1\} \neg (\bigcup T \prec nat)$ 
  shows  $((\{one\text{-}point\ compactification\ of\}(CoFinite\ (\bigcup T))) - \{\{\bigcup T\}\}) \cup T$ 
 $\{is\ a\ topology\}$ 
 $\langle proof \rangle$ 
```

The previous construction preserves anti-hyperconnectedness.

```
theorem (in topology0) COF_comp_antiHConn:
  assumes  $T\{is\ anti\text{-}\}IsHConnected\ \neg (\bigcup T \prec nat)$ 
  shows  $((\{one\text{-}point\ compactification\ of\}(CoFinite\ (\bigcup T))) - \{\{\bigcup T\}\}) \cup T$ 
 $\{is\ anti\text{-}\}IsHConnected$ 
 $\langle proof \rangle$ 
```

The previous construction, applied to a densely ordered topology, gives the desired counterexample. What happens is that every neighbourhood of $\bigcup T$ is dense; because there are no finite open sets, and hence meets every non-empty open set. In conclusion, $\bigcup T$ cannot be separated from other points by disjoint open sets.

Every open set that contains $\bigcup T$ is dense, when considering the order topology in a densely ordered set with more than two points.

```
theorem neigh_infPoint_dense:
  fixes  $T\ X\ r$ 
  defines  $T\_def: T \equiv (OrdTopology\ X\ r)$ 
  assumes  $IsLinOrder(X,r)\ X\{is\ dense\ with\ respect\ to\}r$ 
   $\exists x\ y. x \neq y \wedge x \in X \wedge y \in X\ U \in ((\{one\text{-}point\ compactification\ of\}(CoFinite\ (\bigcup T))) - \{\{\bigcup T\}\}) \cup T$ 
 $\bigcup T \in U$ 
   $V \in ((\{one\text{-}point\ compactification\ of\}(CoFinite\ (\bigcup T))) - \{\{\bigcup T\}\}) \cup T\ V \neq \emptyset$ 
  shows  $U \cap V \neq \emptyset$ 
 $\langle proof \rangle$ 
```

A densely ordered set with more than one point gives an order topology. Applying the previous construction to this topology we get a non locally-Hausdorff space.

```
theorem OPComp_cofinite_dense_order_not_loc_T2:
  fixes  $T\ X\ r$ 
  defines  $T\_def: T \equiv (OrdTopology\ X\ r)$ 
  assumes  $IsLinOrder(X,r)\ X\{is\ dense\ with\ respect\ to\}r$ 
   $\exists x\ y. x \neq y \wedge x \in X \wedge y \in X$ 
  shows  $\neg(((\{one\text{-}point\ compactification\ of\}(CoFinite\ (\bigcup T))) - \{\{\bigcup T\}\}) \cup T)\{is\ locally\text{-}T_2\}$ 
```

<proof>

This topology, from the previous result, gives a counter-example for anti-hyperconnected implies locally- T_2 .

```

theorem antiHConn_not_imp_loc_T2:
  fixes T X r
  defines T_def:T  $\equiv$  (OrdTopology X r)
  assumes IsLinOrder(X,r) X{is dense with respect to}r
     $\exists x y. x \neq y \wedge x \in X \wedge y \in X$ 
  shows  $\neg(((\text{one-point compactification of})(\text{CoFinite } (\bigcup T))) - \{\bigcup T\} \cup T) \text{ is locally-}T_2)$ 
    and  $((\text{one-point compactification of})(\text{CoFinite } (\bigcup T))) - \{\bigcup T\} \cup T \text{ is anti-IsHConnected}$ 
<proof>

```

Let's prove that T_2 spaces are locally- T_2 , but that there are locally- T_2 spaces which aren't T_2 . In conclusion $T_2 \Rightarrow$ locally- $T_2 \Rightarrow$ anti-hyperconnected; all implications proper.

```

theorem(in topology0) T2_imp_loc_T2:
  assumes T{is  $T_2$ }
  shows T{is locally- $T_2$ }
<proof>

```

If there is a closed singleton, then we can consider a topology that makes this point double.

```

theorem(in topology0) doble_point_top:
  assumes {m}{is closed in}T
  shows  $(T \cup \{(U - \{m\}) \cup \bigcup T\} \cup W. \langle U, W \rangle \in \{V \in T. m \in V\} \times T) \text{ is a topology}$ 
<proof>

```

The previous topology is defined over a set with one more point.

```

lemma(in topology0) union_doublepoint_top:
  assumes {m}{is closed in}T
  shows  $\bigcup (T \cup \{(U - \{m\}) \cup \bigcup T\} \cup W. \langle U, W \rangle \in \{V \in T. m \in V\} \times T) = \bigcup T \cup \{\bigcup T\}$ 
<proof>

```

In this topology, the previous topological space is an open subspace.

```

theorem(in topology0) open_subspace_double_point:
  assumes {m}{is closed in}T
  shows  $(T \cup \{(U - \{m\}) \cup \bigcup T\} \cup W. \langle U, W \rangle \in \{V \in T. m \in V\} \times T) \text{ restricted to } \bigcup T = T$ 
  and  $\bigcup T \in (T \cup \{(U - \{m\}) \cup \bigcup T\} \cup W. \langle U, W \rangle \in \{V \in T. m \in V\} \times T)$ 
<proof>

```

The previous topology construction applied to a T_2 non-discrete space topology, gives a counter-example to: Every locally- T_2 space is T_2 .

If there is a singleton which is not open, but closed; then the construction on that point is not T_2 .


```

theorem(in topology0) loc_T2_imp_T2_counter_1:
  assumes {m}∉T {m}{is closed in}T
  shows ¬((T∪{(U-{m})}∪{∪T}∪W. ⟨U,W⟩∈{V∈T. m∈V}×T)) {is T2})
  <proof>

```

This topology is locally- T_2 .

```

theorem(in topology0) loc_T2_imp_T2_counter_2:
  assumes {m}∉T m∈∪T T{is T2}
  shows (T∪{(U-{m})}∪{∪T}∪W. ⟨U,W⟩∈{V∈T. m∈V}×T)) {is locally-T2}
  <proof>

```

There can be considered many more local properties, which; as happens with locally- T_2 ; can distinguish between spaces other properties cannot.

94.6 Locally-regular spaces

A topological space is locally-regular if every point has a base of neighbourhoods whose subspace topology is regular.

definition

```

  IsLocallyRegular (_{is locally-regular} 70)
  where T{is locally-regular} ≡ T{is locally}(λB. λTT. (TT{restricted
to}B){is regular})

```

Since regularity is a hereditary property (it passes to every subspace), we can apply the general lemma `her_P_is_loc_P`: a space is locally-regular iff every point merely has *some* open neighbourhood whose subspace topology is regular (rather than a full neighbourhood basis of them).

```

corollary (in topology0) loc_regular:
  shows (T{is locally-regular}) ↔ (∀x∈∪T. ∃A∈T. x∈A ∧ (T{restricted
to}A){is regular})
  <proof>

```

Every regular space is locally-regular: the whole space is itself an open neighbourhood whose subspace topology is regular, for each of its points.

```

lemma (in topology0) regular_imp_locally_regular:
  assumes T{is regular}
  shows T{is locally-regular}
  <proof>

```

end

95 Properties in Topology 3

```

theory Topology_ZF_properties_3 imports Topology_ZF_7 Finite_ZF_1 Topology_ZF_1
Topology_ZF_9
  Topology_ZF_properties_2 FinOrd_ZF

```

begin

This theory file deals with more topological properties and the relation with the previous ones in other theory files.

95.1 More anti-properties

In this section we study more anti-properties.

95.2 First examples

A first example of an anti-compact space is the discrete space.

```
lemma pow_compact_imp_finite:
  assumes B{is compact in}Pow(A)
  shows Finite(B)
<proof>
```

```
theorem pow_anti_compact:
  shows Pow(A){is anti-compact}
<proof>
```

In a previous file, Topology_ZF_5.thy, we proved that the spectrum of the lindelöf property depends on the axiom of countable choice on subsets of the power set of the natural number.

In this context, the examples depend on whether this choice principle holds or not. This is the reason that the examples of anti-lindelöf topologies are left for the next section.

95.3 Structural results

We first differentiate the spectrum of the lindelöf property depending on some axiom of choice.

```
lemma lindelöf_spec1:
  assumes {the axiom of} nat {choice holds for subsets}(Pow(nat))
  shows (A {is in the spectrum of} ( $\lambda T. ((\bigcup T)\{is\ lindelöf\ in\}T))) \longleftrightarrow$ 
  ( $A \lesssim_{nat}$ )
<proof>
```

```
lemma lindelöf_spec2:
  assumes  $\neg(\{the\ axiom\ of\} nat \{choice\ holds\ for\ subsets\}(Pow(nat)))$ 
  shows (A {is in the spectrum of} ( $\lambda T. ((\bigcup T)\{is\ lindelöf\ in\}T))) \longleftrightarrow$ 
  Finite(A)
<proof>
```

If the axiom of countable choice on subsets of the pow of the natural numbers doesn't hold, then anti-lindelöf spaces are anti-compact.

```

theorem(in topology0) no_choice_imp_anti_lindelof_is_anti_comp:
  assumes  $\neg$ ({the axiom of} nat {choice holds for subsets}(Pow(nat))) )
  T{is anti-lindelof}
  shows T{is anti-compact}
  <proof>

```

If the axiom of countable choice holds for subsets of the power set of the natural numbers, then there exists a topological space that is anti-lindelof but no anti-compact.

```

theorem no_choice_imp_anti_lindelof_is_anti_comp:
  assumes ({the axiom of} nat {choice holds for subsets}(Pow(nat)))
  shows ({one-point compactification of}Pow(nat)){is anti-lindelof}
  <proof>

```

```

theorem op_comp_pow_nat_no_anti_comp:
  shows  $\neg$ (({one-point compactification of}Pow(nat)){is anti-compact})
  <proof>

```

In conclusion, we reached another equivalence of this choice principle.

The axiom of countable choice holds for subsets of the power set of the natural numbers if and only if there exists a topological space which is anti-lindelof but not anti-compact; this space can be chosen as the one-point compactification of the discrete topology on $\mathbb{N}$.

```

theorem acc_pow_nat_equiv1:
  shows ({the axiom of} nat {choice holds for subsets}(Pow(nat)))  $\longleftrightarrow$ 
  (({one-point compactification of}Pow(nat)){is anti-lindelof})
  <proof>

```

```

theorem acc_pow_nat_equiv2:
  shows ({the axiom of} nat {choice holds for subsets}(Pow(nat)))  $\longleftrightarrow$ 
  ( $\exists$ T. T{is a topology}
   $\wedge$  (T{is anti-lindelof})  $\wedge$   $\neg$ (T{is anti-compact}))
  <proof>

```

In the file `Topology_ZF_properties.thy`, it is proven that $\mathbb{N}$ is lindelof if and only if the axiom of countable choice holds for subsets of $Pow(\mathbb{N})$. Now we check that, in ZF, this space is always anti-lindelof.

```

theorem nat_anti_lindelof:
  shows Pow(nat){is anti-lindelof}
  <proof>

```

This result is interesting because depending on the different axioms we add to ZF, it means two different things:

- Every subspace of $\mathbb{N}$ is Lindelof.
- Only the compact subspaces of $\mathbb{N}$ are Lindelof.

Now, we could wonder if the class of compact spaces and the class of lindeloef spaces being equal is consistent in ZF. Let's find a topological space which is lindeloef and no compact without assuming any axiom of choice or any negation of one. This will prove that the class of lindeloef spaces and the class of compact spaces cannot be equal in any model of ZF.

theorem lord_nat:

shows (L0rdTopology nat Le)={LeftRayX(nat,Le,n). n∈nat} ∪ {nat} ∪ {0}
<proof>

lemma countable_lord_nat:

shows {LeftRayX(nat,Le,n). n∈nat} ∪ {nat} ∪ {0} <csucc(nat)
<proof>

corollary lindeloef_lord_nat:

shows nat{is lindeloef in}(L0rdTopology nat Le)
<proof>

theorem not_comp_lord_nat:

shows ¬(nat{is compact in}(L0rdTopology nat Le))
<proof>

95.4 More Separation properties

In this section we study more separation properties.

95.5 Definitions

We start with a property that has already appeared in `Topology_ZF_1.thy`. A KC-space is a space where compact sets are closed.

definition

IsKC (_ {is KC}) where
 $T\{is\ KC\} \equiv \forall A \in Pow(\bigcup T). A\{is\ compact\ in\}T \longrightarrow A\{is\ closed\ in\}T$

Another type of space is an US-space; those where sequences have at most one limit.

definition

IsUS (_ {is US}) where
 $T\{is\ US\} \equiv \forall N\ x\ y. (N:nat \rightarrow \bigcup T) \wedge NetConvTop(\langle N, Le \rangle, x, T) \wedge NetConvTop(\langle N, Le \rangle, y, T) \longrightarrow y=x$

95.6 First results

The proof in `Topology_ZF_1.thy` shows that a Hausdorff space is KC.

corollary(in `topology0`) T2_imp_KC:

assumes T{is T₂}
 shows T{is KC}

<proof>

From the spectrum of compactness, it follows that any KC-space is T_1 .

```
lemma(in topology0) KC_imp_T1:
  assumes T{is KC}
  shows T{is T1}
<proof>
```

Even more, if a space is KC, then it is US. We already know that for T_2 spaces, any net or filter has at most one limit; and that this property is equivalent with T_2 . The US property is much weaker because we don't know what happens with other nets that are not directed by the order on the natural numbers.

```
theorem(in topology0) KC_imp_US:
  assumes T{is KC}
  shows T{is US}
<proof>
```

US spaces are also T_1 .

```
theorem (in topology0) US_imp_T1:
  assumes T{is US}
  shows T{is T1}
<proof>
```

95.7 Counter-examples

We need to find counter-examples that prove that these properties are new ones.

We know that $T_2 \Rightarrow loc.T_2 \Rightarrow \text{anti-hyperconnected} \Rightarrow T_1$ and $T_2 \Rightarrow KC \Rightarrow US \Rightarrow T_1$. The question is: What is the relation between KC or US and, $loc.T_2$ or anti-hyperconnected?

In the file `Topology_ZF_properties_2.thy` we built a topological space which is locally- T_2 but not T_2 . It happens actually that this space is not even US given the appropriate topology T .

```
lemma (in topology0) locT2_not_US_1:
  assumes {m}∉T {m}{is closed in}T ∃N∈nat→⋃T. (⟨N,Le⟩→N m) ∧ m∉Nnat

  shows ∃N∈nat→⋃ (T∪{(U-{m})∪{⋃T}∪W. ⟨U,W⟩∈{V∈T. m∈V}×T}). (⟨N,Le⟩→N
  ⋃T {in} (T∪{(U-{m})∪{⋃T}∪W. ⟨U,W⟩∈{V∈T. m∈V}×T}))
  ∧ (⟨N,Le⟩→N m {in} (T∪{(U-{m})∪{⋃T}∪W. ⟨U,W⟩∈{V∈T. m∈V}×T}))
<proof>
```

```
corollary (in topology0) locT2_not_US_2:
  assumes {m}∉T {m}{is closed in}T ∃N∈nat→⋃T. (⟨N,Le⟩→N m) ∧ m∉Nnat
  shows ¬((T∪{(U-{m})∪{⋃T}∪W. ⟨U,W⟩∈{V∈T. m∈V}×T}){is US})
```

<proof>

In particular, we also know that a locally- T_2 space doesn't need to be KC; since $KC \Rightarrow US$. Also we know that anti-hyperconnected spaces don't need to be KC or US, since $\text{locally-}T_2 \Rightarrow \text{anti-hyperconnected}$.

Let's find a KC space that is not T_2 , an US space which is not KC and a T_1 space which is not US.

First, let's prove some lemmas about what relation is there between this properties under the influence of other ones. This will help us to find counter-examples.

Anti-compactness erases the differences between several properties.

lemma (in topology0) anticompact_KC_equiv_T1:
 assumes $T\{\text{is anti-compact}\}$
 shows $T\{\text{is KC}\} \longleftrightarrow T\{\text{is } T_1\}$

<proof>

Then if we find an anti-compact and T_1 but no T_2 space, there is a counter-example for $KC \Rightarrow T_2$. A counter-example for US doesn't need to be KC mustn't be anti-compact.

The cocountable topology on $\text{csucc}(\text{nat})$ is such a topology.

The cocountable topology on $\mathbb{N}^+$ is hyperconnected.

lemma cocountable_in_csucc_nat_HConn:
 shows $(\text{CoCountable } \text{csucc}(\text{nat}))\{\text{is hyperconnected}\}$

<proof>

The cocountable topology on $\mathbb{N}^+$ is not anti-hyperconnected.

corollary cocountable_in_csucc_nat_notAntiHConn:
 shows $\neg((\text{CoCountable } \text{csucc}(\text{nat}))\{\text{is anti-}\}\text{IsHConnected})$

<proof>

The cocountable topology on $\mathbb{N}^+$ is not T_2 .

theorem cocountable_in_csucc_nat_noT2:
 shows $\neg(\text{CoCountable } \text{csucc}(\text{nat}))\{\text{is } T_2\}$

<proof>

The cocountable topology on $\mathbb{N}^+$ is T_1 .

theorem cocountable_in_csucc_nat_T1:
 shows $(\text{CoCountable } \text{csucc}(\text{nat}))\{\text{is } T_1\}$

<proof>

The cocountable topology on $\mathbb{N}^+$ is anti-compact.

theorem cocountable_in_csucc_nat_antiCompact:
 shows $(\text{CoCountable } \text{csucc}(\text{nat}))\{\text{is anti-compact}\}$

<proof>

In conclusion, the cocountable topology defined on $\text{csucc}(\text{nat})$ is KC but not T_2 . Also note that is KC but not anti-hyperconnected, hence KC or US spaces need not to be sober.

The cofinite topology on the natural numbers is T_1 , but not US.

theorem `cofinite_not_US:`
 shows $\neg((\text{CoFinite } \text{nat})\{\text{is US}\})$
<proof>

To end, we need a space which is US but no KC. This example comes from the one point compactification of a T_2 , anti-compact and non discrete space. This T_2 , anti-compact and non discrete space comes from a construction over the cardinal $\mathbb{N}^+$ or $\text{csucc}(\text{nat})$.

theorem `extension_pow_top:`
 shows $(\text{Pow}(\text{csucc}(\text{nat})) \cup \{\{\text{csucc}(\text{nat})\}\})\text{US. } S \in (\text{CoCountable } \text{csucc}(\text{nat})) - \{0\}\{\text{is a topology}\}$
<proof>

This topology is defined over $\mathbb{N}^+ \cup \{\mathbb{N}^+\}$ or $\text{csucc}(\text{nat}) \cup \{\text{csucc}(\text{nat})\}$.

lemma `extension_pow_union:`
 shows $\bigcup (\text{Pow}(\text{csucc}(\text{nat})) \cup \{\{\text{csucc}(\text{nat})\}\})\text{US. } S \in (\text{CoCountable } \text{csucc}(\text{nat})) - \{0\} = \text{csucc}(\text{nat}) \cup \{\text{csucc}(\text{nat})\}$
<proof>

This topology has a discrete open subspace.

lemma `extension_pow_subspace:`
 shows $(\text{Pow}(\text{csucc}(\text{nat})) \cup \{\{\text{csucc}(\text{nat})\}\})\text{US. } S \in (\text{CoCountable } \text{csucc}(\text{nat})) - \{0\}\{\text{restricted to } \text{csucc}(\text{nat}) = \text{Pow}(\text{csucc}(\text{nat}))\}$
 and $\text{csucc}(\text{nat}) \in (\text{Pow}(\text{csucc}(\text{nat})) \cup \{\{\text{csucc}(\text{nat})\}\})\text{US. } S \in (\text{CoCountable } \text{csucc}(\text{nat})) - \{0\}$
<proof>

This topology is Hausdorff.

theorem `extension_pow_T2:`
 shows $(\text{Pow}(\text{csucc}(\text{nat})) \cup \{\{\text{csucc}(\text{nat})\}\})\text{US. } S \in (\text{CoCountable } \text{csucc}(\text{nat})) - \{0\}\{\text{is } T_2\}$
<proof>

The topology we built is not discrete; i.e., not every set is open.

theorem `extension_pow_notDiscrete:`
 shows $\{\text{csucc}(\text{nat})\} \notin (\text{Pow}(\text{csucc}(\text{nat})) \cup \{\{\text{csucc}(\text{nat})\}\})\text{US. } S \in (\text{CoCountable } \text{csucc}(\text{nat})) - \{0\}$
<proof>

The topology we built is anti-compact.

theorem `extension_pow_antiCompact:`

```

    shows (Pow(csucc(nat)) ∪ {{csucc(nat)}}US. S∈(CoCountable csucc(nat))-{0}){is
anti-compact}
⟨proof⟩

```

If a topological space is KC, then its one-point compactification is US.

```

theorem (in topology0) KC_imp_OP_comp_is_US:
  assumes T{is KC}
  shows ({one-point compactification of}T){is US}
⟨proof⟩

```

In the one-point compactification of an anti-compact space, ever subspace that contains the infinite point is compact.

```

theorem (in topology0) anti_comp_imp_OP_inf_comp:
  assumes T{is anti-compact} A⊆∪ ({one-point compactification of}T) ∪ T∈A
  shows A{is compact in}({one-point compactification of}T)
⟨proof⟩

```

As a last result in this section, the one-point compactification of our topology is not a KC space.

```

theorem extension_pow_OP_not_KC:
  shows ¬((({one-point compactification of}(Pow(csucc(nat)) ∪ {{csucc(nat)}}US.
S∈(CoCountable csucc(nat))-{0}))) {is KC})
⟨proof⟩

```

In conclusion, $US \not\equiv KC$.

95.8 Other types of properties

In this section we will define new properties that aren't defined as anti-properties and that are not separation axioms. In some cases we will consider their anti-properties.

95.9 Definitions

A space is called perfect if it has no isolated points. This definition may vary in the literature to similar, but not equivalent definitions.

```

definition
  IsPerf (_ {is perfect}) where
  T{is perfect} ≡ ∀x∈∪T. {x}∉T

```

An anti-perfect space is called scattered.

```

definition
  IsScatt (_ {is scattered}) where
  T{is scattered} ≡ T{is anti-}IsPerf

```

A topological space with two disjoint dense subspaces is called resolvable.

definition

IsRes ($_$ {is resolvable}) **where**
 $T\{\text{is resolvable}\} \equiv \exists U \in \text{Pow}(\bigcup T). \exists V \in \text{Pow}(\bigcup T). \text{Closure}(U, T) = \bigcup T \wedge \text{Closure}(V, T) = \bigcup T \wedge U \cap V = \emptyset$

A topological space where every dense subset is open is called submaximal.

definition

IsSubMax ($_$ {is submaximal}) **where**
 $T\{\text{is submaximal}\} \equiv \forall U \in \text{Pow}(\bigcup T). \text{Closure}(U, T) = \bigcup T \longrightarrow U \in T$

A subset of a topological space is nowhere-dense if the interior of its closure is empty.

definition

IsNowhereDense ($_$ {is nowhere dense in} $_$) **where**
 $A\{\text{is nowhere dense in}\} T \equiv A \subseteq \bigcup T \wedge \text{Interior}(\text{Closure}(A, T), T) = \emptyset$

A topological space is then a Luzin space if every nowhere-dense subset is countable.

definition

IsLuzin ($_$ {is luzin}) **where**
 $T\{\text{is luzin}\} \equiv \forall A \in \text{Pow}(\bigcup T). (A\{\text{is nowhere dense in}\} T) \longrightarrow A \lesssim_{\text{nat}}$

An also useful property is local-connexion.

definition

IsLocConn ($_$ {is locally-connected}) **where**
 $T\{\text{is locally-connected}\} \equiv T\{\text{is locally}\}(\lambda T. \lambda B. ((T\{\text{restricted to}\} B)\{\text{is connected}\}))$

An SI-space is an anti-resolvable perfect space.

definition

IsAntiRes ($_$ {is anti-resolvable}) **where**
 $T\{\text{is anti-resolvable}\} \equiv T\{\text{is anti-}\} \text{IsRes}$

definition

IsSI ($_$ {is Strongly Irresolvable}) **where**
 $T\{\text{is Strongly Irresolvable}\} \equiv (T\{\text{is anti-resolvable}\}) \wedge (T\{\text{is perfect}\})$

95.10 First examples

Firstly, we need to compute the spectrum of the being perfect.

lemma spectrum_perfect:

shows $(A\{\text{is in the spectrum of}\} \text{IsPerf}) \longleftrightarrow A = 0$
<proof>

The discrete space is clearly scattered:

lemma pow_is_scattered:

shows $\text{Pow}(A)\{\text{is scattered}\}$

<proof>

The trivial topology is perfect, if it is defined over a set with more than one point.

```
lemma trivial_is_perfect:
  assumes  $\exists x y. x \in X \wedge y \in X \wedge x \neq y$ 
  shows  $\{0, X\}$ {is perfect}
<proof>
```

The trivial topology is resolvable, if it is defined over a set with more than one point.

```
lemma trivial_is_resolvable:
  assumes  $\exists x y. x \in X \wedge y \in X \wedge x \neq y$ 
  shows  $\{0, X\}$ {is resolvable}
<proof>
```

The spectrum of Luzin spaces is the class of countable sets, so there are lots of examples of Luzin spaces.

```
lemma spectrum_Luzin:
  shows  $(A \text{ is in the spectrum of } \text{IsLuzin}) \longleftrightarrow A \lesssim \text{nat}$ 
<proof>
```

95.11 Structural results

Every resolvable space is also perfect.

```
theorem (in topology0) resolvable_imp_perfect:
  assumes  $T$ {is resolvable}
  shows  $T$ {is perfect}
<proof>
```

The spectrum of being resolvable follows:

```
corollary spectrum_resolvable:
  shows  $(A \text{ is in the spectrum of } \text{IsRes}) \longleftrightarrow A = 0$ 
<proof>
```

The cofinite space over $\mathbb{N}$ is a T_1 , perfect and luzin space.

```
theorem cofinite_nat_perfect:
  shows  $(\text{CoFinite nat})$ {is perfect}
<proof>
```

```
theorem cofinite_nat_luzin:
  shows  $(\text{CoFinite nat})$ {is luzin}
<proof>
```

The cocountable topology on $\mathbb{N}^+$ or $\text{csucc}(\text{nat})$ is also T_1 , perfect and luzin; but defined on a set not in the spectrum.

```
theorem cocountable_csucc_nat_perfect:
```

```

    shows (CoCountable csucc(nat)){is perfect}
  <proof>

```

```

theorem cocountable_csucc_nat_luzin:
  shows (CoCountable csucc(nat)){is luzin}
  <proof>

```

The existence of T_2 , uncountable, perfect and luzin spaces is unprovable in *ZFC*. It is related to the *CH* and Martin's axiom.

end

96 Ultrafilters

```

theory Ultrafilter_ZF imports Topology_ZF_4 Topology_ZF_examples
begin

```

This theory deals with ultrafilters; which is a maximal filter.

```

definition IsUltrafilter (_{is an ultrafilter on}_ 80)
  where  $\mathfrak{F}$ {is an ultrafilter on} $X \equiv (\mathfrak{F}$ {is a filter on} $X) \wedge (\forall \mathfrak{G}. (\mathfrak{G}$ {is
a filter on} $X) \longrightarrow (\mathfrak{F} \subseteq \mathfrak{G} \longrightarrow \mathfrak{G} = \mathfrak{F}))$ 

```

Every set or its opposite is in an ultrafilter

```

lemma set_ultrafilter:
  assumes  $A \in \text{Pow}(X)$   $\mathfrak{F}$ {is an ultrafilter on} $X$ 
  shows  $A \in \mathfrak{F} \vee (X-A) \in \mathfrak{F}$ 
  <proof>

```

It contains only one of them

```

lemma only_set_or_opposite:
  assumes  $A \in \text{Pow}(X)$   $\mathfrak{F}$ {is a filter on} $X$   $A \in \mathfrak{F}$ 
  shows  $(X-A) \notin \mathfrak{F}$ 
  <proof>

```

If a filter contains a set or its opposite, it is in an ultrafilter

```

lemma set_ultrafilter_equiv:
  assumes  $\mathfrak{F}$ {is a filter on} $X$   $\forall A \in \text{Pow}(X). A \in \mathfrak{F} \vee (X-A) \in \mathfrak{F}$ 
  shows  $\mathfrak{F}$ {is an ultrafilter on} $X$  <proof>

```

Filters of containing a point, are ultrafilters

```

lemma include_point_ultrafilter:
  assumes  $x \in X$ 
  shows  $\{A \in \text{Pow}(X). x \in A\}$ {is an ultrafilter on} $X$ 
  <proof>

```

An ultrafilter that has no singleton sets, does not have finite sets

```

lemma ultrafilter_finite_set:
  assumes  $\mathfrak{F}$ {is an ultrafilter on} $X$   $\forall x \in X. \{x\} \notin \mathfrak{F}$ 
  shows  $A \in \text{FinPow}(X) \longrightarrow A \notin \mathfrak{F}$ 

```

shows $A \notin \mathcal{F}$
 $\langle proof \rangle$

The cofinite filter plays an important role in ultrafilters.

corollary ultrafilter_contains_cofinite:
 assumes $\mathcal{F}$ {is an ultrafilter on} $X \ \forall x \in X. \{x\} \notin \mathcal{F}$
 shows $(\text{CoFinite}(X)) - \{0\} \subseteq \mathcal{F}$
 $\langle proof \rangle$

An ultrafilter is free or principal.

corollary ultrafilter_principal_or_free:
 assumes $\mathcal{F}$ {is an ultrafilter on} X
 shows $\bigcap \mathcal{F} = 0 \vee (\exists x \in X. \bigcap \mathcal{F} = \{x\})$
 $\langle proof \rangle$

end

97 Ultraproduct construction

theory UltraConstruction_ZF imports Ultrafilter_ZF EquivClass1
begin

This theory deals with the ultra product construction, internal entities and some basic properties of them.

locale ultra =
 fixes $\mathcal{F}$ and I and X
 assumes ultraFilter: $\mathcal{F}$ {is an ultrafilter on} I and nonEmpty: $\forall i \in I. X(i) \neq 0$

begin

We define an equivalence relation

definition hyper_rel **where**
 $\text{hyper_rel} \equiv \{ \langle f, g \rangle \in \text{Pi}(I, X) \times \text{Pi}(I, X) . \{ n \in I. f n = g n \} \in \mathcal{F} \}$

The relation is reflexive

lemma hyper_refl:
 shows $\text{refl}(\text{Pi}(I, X), \text{hyper_rel}) \ \langle proof \rangle$

The relation is symmetric

lemma hyper_sym:
 shows $\text{sym}(\text{hyper_rel}) \ \langle proof \rangle$

The relation is transitive

lemma hyper_trans:
 shows $\text{trans}(\text{hyper_rel}) \ \langle proof \rangle$

The relation is an equivalence

lemma `hyper_equiv`:
 shows `equiv(Pi(I,X), hyper_rel) <proof>`

definition `hyper_set` where
`hyper_set` $\equiv$ `Pi(I,X) // hyper_rel`

lemma `incl_inj`:
 fixes `Y`
 assumes $\forall i \in I. X(i) = Y$
 defines `incl` $\equiv$ $\{\langle x, \text{hyper_rel}\{\text{ConstantFunction}(I,x)\} \rangle. x \in Y\}$
 shows `incl` $\in$ `inj(Y, hyper_set)` *<proof>*

lemma `hyper_set_structure`:
 assumes $\forall i \in I. P(i): X(i) \times X(i) \rightarrow X(i)$
 defines `Q` $\equiv$ $\{\{\langle q1, q2 \rangle, \{\langle i, P(i)\langle q1i, q2i \rangle \rangle. i \in I\} \rangle. \langle q1, q2 \rangle \in \text{Pi}(I, X) \times \text{Pi}(I, X)\}$
 shows `Congruent2(hyper_rel, Q)` $Q: \text{Pi}(I, X) \times \text{Pi}(I, X) \rightarrow \text{Pi}(I, X)$
 $\forall i \in I. \forall x \in \text{Pi}(I, X). \forall y \in \text{Pi}(I, X). Q\langle x, y \rangle i = P(i)\langle xi, yi \rangle$
 <proof>

97.1 Internal sets

definition `internal_set` where
`S: Pi(I,  $\lambda i. \text{Pow}(X(i))$ )  $\implies$  internal_set(S)  $\equiv$   $\{\text{hyper\_rel}\{x\}. x \in \{x \in \text{Pi}(I, X). \{n \in I. xn \in S_n\} \in \mathfrak{F}\}\}$`

lemma `internal_subset`:
 assumes `S: Pi(I,  $\lambda i. \text{Pow}(X(i))$ )`
 shows `internal_set(S)  $\subseteq$  hyper_set`
 <proof>

lemma `internal_sub`:
 assumes `S1: Pi(I,  $\lambda i. \text{Pow}(X(i))$ )` `S2: Pi(I,  $\lambda i. \text{Pow}(X(i))$ )` $\{n \in I. S1n \subseteq S2n\} \in \mathfrak{F}$
 shows `internal_set(S1)  $\subseteq$  internal_set(S2)`
 <proof>

lemma `internal_sub_rev`:
 assumes `S1: Pi(I,  $\lambda i. \text{Pow}(X(i))$ )` `S2: Pi(I,  $\lambda i. \text{Pow}(X(i))$ )`
 and `internal_set(S1)  $\subseteq$  internal_set(S2)`
 and `Pi( $\{i \in I. \neg(S1i \subseteq S2i)\}, \lambda j. S1j - S2j) \neq 0$`
 and `Pi(I, X)  $\neq 0$`
 shows $\{n \in I. S1n \subseteq S2n\} \in \mathfrak{F}$
 <proof>

corollary `internal_eq`:
 assumes `S1: Pi(I,  $\lambda i. \text{Pow}(X(i))$ )` `S2: Pi(I,  $\lambda i. \text{Pow}(X(i))$ )` $\{n \in I. S1n = S2n\} \in \mathfrak{F}$
 shows `internal_set(S1) = internal_set(S2)`

<proof>

lemma internal_total_set:
 shows internal_set($\{\langle i, X(i) \rangle. i \in I\}$) = hyper_set
<proof>

definition isInternal **where**
isInternal(Y) $\equiv \exists S \in \text{Pi}(I, \lambda i. \text{Pow}(X(i))). Y = \text{internal_set}(S)$

corollary total_inter:
 shows isInternal(hyper_set)
<proof>

lemma internal_inter:
 assumes isInternal(A) isInternal(B)
 shows isInternal(A $\cap$ B)
<proof>

The complement of an internal set is internal

lemma complement_internal:
 assumes isInternal(A)
 shows isInternal(hyper_set - A)
<proof>

lemma finite_internal:
 assumes A $\in \text{FinPow}(\text{hyper_set})$
 shows isInternal(A) *<proof>*

lemma internal_union:
 assumes isInternal(A) isInternal(B)
 shows isInternal(A $\cup$ B)
<proof>

definition elevatedProp ($\wedge_{-}^{90}$) **where**
 $\wedge^P \equiv \lambda t. \exists y \in \text{Pi}(I, X). \{i \in I. P(yi)\} \in \mathfrak{F} \wedge \text{hyper_rel}\{y\} = t$

lemma internal_subset_internal:
 assumes isInternal(A)
 shows isInternal($\{x \in A. (\wedge^P)(x)\}$)
<proof>

definition internal_rel **where**
 $S : \text{Pi}(I, \lambda i. \text{Pow}(X(i) \times X(i))) \implies \text{internal_rel}(S) \equiv \{\langle \text{hyper_rel}\{x\}, \text{hyper_rel}\{y\} \rangle. \langle x, y \rangle \in \{ \langle p, q \rangle \in \text{Pi}(I, X) \times \text{Pi}(I, X). \{n \in I. \langle pn, qn \rangle \in S_n \} \in \mathfrak{F} \} \}$

lemma internal_rel_subset:
 assumes S : $\text{Pi}(I, \lambda i. \text{Pow}(X(i) \times X(i)))$
 shows internal_rel(S) $\subseteq \text{hyper_set} \times \text{hyper_set}$

<proof>

lemma converse_internal:

assumes $S: \Pi(I, \lambda i. \text{Pow}(X(i) \times X(i)))$

shows $\text{converse}(\text{internal_rel}(S)) = \text{internal_rel}(\{\langle i, \text{converse}(S i) \rangle. i \in I\})$

<proof>

lemma internal_rel_total:

shows $\text{internal_rel}(\{\langle i, X(i) \times X(i) \rangle. i \in I\}) = \text{hyper_set} \times \text{hyper_set}$

<proof>

end

end

98 Internal functions in ultraproduct construction

theory UltraConstruction_ZF_1 imports UltraConstruction_ZF **begin**

This theory continues UltraConstruction_ZF with results about internal functions whose fibres are elements of an internal family.

context ultra **begin**

98.1 Internal functions

definition internal_fun where

$S1: \Pi(I, \lambda i. \text{Pow}(X(i))) \implies S2: \Pi(I, \lambda i. \text{Pow}(X(i))) \implies S: \Pi(I, \lambda i. S1 i \rightarrow S2 i)$

$\implies \text{internal_fun}(S) \equiv \text{internal_rel}(S)$

lemma internal_fun_is_fun:

assumes $S1: \Pi(I, \lambda i. \text{Pow}(X(i)))$ $S2: \Pi(I, \lambda i. \text{Pow}(X(i)))$ $S: \Pi(I, \lambda i. S1 i \rightarrow S2 i)$

shows $\text{internal_fun}(S): \text{internal_set}(S1) \rightarrow \text{internal_set}(S2)$ $S: \Pi(I, \lambda i. \text{Pow}(X(i) \times X(i)))$ *<proof>*

lemma internal_fun_apply:

assumes $S1 \in (\prod_{i \in I. \text{Pow}(X(i)))$ $S2 \in (\prod_{i \in I. \text{Pow}(X(i)))$ $S \in (\prod_{i \in I. S1 i \rightarrow S2 i)$

and $x \in \Pi(I, X)$ $\{i: I. x i \in S1 i\} \in \mathfrak{F}$

shows $\text{internal_fun}(S)(\text{hyper_rel}\{x\}) = \text{hyper_rel}\{\{\langle i, \text{if } x i \in S1 i \text{ then } S i (x i) \text{ else } x i \rangle. i \in I\}\}$

and $\{\langle i, \text{if } x i \in S1 i \text{ then } S i (x i) \text{ else } x i \rangle. i \in I\}: \Pi(I, X)$

<proof>

lemma internal_fun_apply_2:

assumes $S1 \in (\prod_{i \in I. \text{Pow}(X(i)))$ $S2 \in (\prod_{i \in I. \text{Pow}(X(i)))$ $S \in (\prod_{i \in I. S1 i \rightarrow S2 i)$

and $\text{hyper_rel}\{x\} \in \text{internal_set}(S1)$
 shows $\text{internal_fun}(S)(\text{hyper_rel}\{x\}) = \text{hyper_rel}\{\langle i, \text{if } x \ i \in S1$
 $i \text{ then } S \ i \ (x \ i) \text{ else } x \ i \rangle . i \in I \rangle\}$
 and $\{\langle i, \text{if } x \ i \in S1 \ i \text{ then } S \ i \ (x \ i) \text{ else } x \ i \rangle . i \in I\} : \text{Pi}(I, X)$
 $\langle \text{proof} \rangle$

lemma internal_fun_inj:
 assumes $S1 \in (\prod_{i \in I}. \text{Pow}(X(i)))$ $S2 \in (\prod_{i \in I}. \text{Pow}(X(i)))$ $S \in (\prod_{i \in I}. S1 \ i \rightarrow S2 \ i)$
 and $\{i : I. Si : \text{inj}(S1i, S2i)\} \in \mathfrak{F}$
 shows $\text{internal_fun}(S) \in \text{inj}(\text{internal_set}(S1), \text{internal_set}(S2))$
 $\langle \text{proof} \rangle$

lemma internal_inj_fun:
 assumes $S1 \in (\prod_{i \in I}. \text{Pow}(X(i)))$ $S2 \in (\prod_{i \in I}. \text{Pow}(X(i)))$ $S \in (\prod_{i \in I}. S1 \ i \rightarrow S2 \ i)$
 and $\text{internal_fun}(S) \in \text{inj}(\text{internal_set}(S1), \text{internal_set}(S2))$
 and $\text{Pi}(\{n \in I. S_n \notin \text{inj}(S1n, S2n)\}, \lambda n. \{\langle q1, q2 \rangle \in S1n \times S1n. (S_n)q1 = (S_n)q2 \wedge q1 \neq q2\}) \neq 0$
 and $\text{Pi}(I, X) \neq 0$
 shows $\{n \in I. S_n \in \text{inj}(S1n, S2n)\} \in \mathfrak{F}$
 $\langle \text{proof} \rangle$

lemma internal_fun_converse:
 assumes $S1 \in (\prod_{i \in I}. \text{Pow}(X(i)))$ $S2 \in (\prod_{i \in I}. \text{Pow}(X(i)))$ $S \in (\prod_{i \in I}. S1 \ i \rightarrow S2 \ i)$
 and $\text{internal_fun}(S) \in \text{bij}(\text{internal_set}(S1), B)$ and $(\prod_{n \in \{n \in I . S \ n \notin \text{inj}(S1 \ n, S2 \ n)\}}. S \ n \ q1 = S \ n \ q2 \wedge q1 \neq q2) \neq 0$
 defines $SS \equiv \{\langle i, \text{if } Si \in \text{inj}(S1i, S2i) \text{ then } \text{converse}(Si) \text{ else } 0 \rangle . i \in I\}$
 shows $\text{converse}(\text{internal_fun}(S)) = \text{internal_fun}(SS)$
 $\langle \text{proof} \rangle$

definition isHyperFinite where
 $H \in \text{Pow}(\text{hyper_set}) \implies \text{isHyperFinite}(H) \equiv \exists S \in \text{Pi}(I, \lambda i. \text{FinPow}(X(i))). H = \text{internal_set}(S)$

lemma hyperfinite_internal:
 assumes $H \in \text{Pow}(\text{hyper_set})$ $\text{isHyperFinite}(H)$
 shows $\text{isInternal}(H)$
 $\langle \text{proof} \rangle$

end

end

99 Hypernatural numbers

```
theory HyperNatural_ZF imports UltraConstruction_ZF UltraConstruction_ZF_1
ZF.Cardinal Finite1
begin
```

The goal of this file is to show that hyperfinite sets are the sets of hypernatural numbers internally bijective with an open left ray of hypernatural.

99.1 Auxiliary results on finite sets

The following results about finite sets and cardinalities are needed in the construction and study of the hypernatural numbers.

The cardinality of a non-trivial set difference is strictly smaller for finite sets

```
lemma finite_diff_strict_smaller: assumes Finite(A) A≠0 A∩B≠0
  shows |A-B| < |A|
⟨proof⟩
```

This applies to strict subsets

```
corollary strict_subset_smaller: assumes Finite(A) B ⊆ A B≠A
  shows |B| < |A|
⟨proof⟩
```

It can be further applied to solve the relation between injective and bijective functions on finite sets.

```
corollary inj_function_same_card_bij: assumes Finite(B) A ≈ B f∈inj(A,B)
  shows f∈bij(A,B)
⟨proof⟩
```

This locale deals with hyper numbers.

```
locale hyperNatural = ultra _ nat λ_. nat +
  assumes non_principal_filter: ⋂ ⋈ = 0
```

```
begin
```

99.2 Ultraproduct of the naturals

The hypernatural numbers $\mathbb{N}^*$ are constructed as an ultraproduct of copies of $\mathbb{N}$ indexed by $\mathbb{N}$ itself, modulo a non-principal ultrafilter $\mathcal{F}$. Elements of $\mathbb{N}^*$ are equivalence classes $[x]$ of sequences $x : \mathbb{N} \rightarrow \mathbb{N}$, with $[x] = [y]$ iff $\{n \in \mathbb{N} \mid x(n) = y(n)\} \in \mathcal{F}$. This section defines the basic structure and the order relation on $\mathbb{N}^*$.

```
abbreviation hyperNat (*N) where
*N ≡ hyper_set
```

definition seq_class ([_]) **where**
 $x \in \text{nat} \rightarrow \text{nat} \implies [x] \equiv \text{hyper_rel}\{x\}$

definition omega (ω) **where**
 $\omega = [\text{id}(\text{nat})]$

definition incl (*_ 70) **where**
 $x \in \text{nat} \implies *x \equiv [\text{ConstantFunction}(\text{nat}, x)]$

lemma incl_inj_nat:
 shows $\{\langle x, *x \rangle. x \in \text{nat}\} \in \text{inj}(\text{nat}, *N) \langle \text{proof} \rangle$

lemma omega_not_nat:
 shows $x \in \text{nat} \longrightarrow (*x) \neq \omega \text{ and } \omega : *N$
 $\langle \text{proof} \rangle$

We define the order relation as the internal relation created by a constant relation of order in natural numbers

definition rel_star (*_ 80) **where**
 $R \subseteq \text{nat} \times \text{nat} \implies *R \equiv \text{internal_rel}(\text{ConstantFunction}(\text{nat}, R))$

definition lessEq_star (infix $* \leq$ 80) **where**
 $x * y \equiv \langle x, y \rangle \in (*Le*)$

definition less_star (infix $* <$ 80) **where**
 $x * y \equiv \langle x, y \rangle \in (*Lt*)$

Two hypernaturals are ordered iff where their representative sequences are ordered is a set in the filter

lemma star_rel_seq:
 assumes $x : \text{nat} \rightarrow \text{nat} \ y : \text{nat} \rightarrow \text{nat} \ R \subseteq \text{nat} \times \text{nat}$
 shows $\langle [x], [y] \rangle \in *R \iff \{i \in \text{nat}. \langle x_i, y_i \rangle \in R\} \in \mathfrak{F}$
 $\langle \text{proof} \rangle$

corollary star_rel_imp_seq:
 assumes $x : \text{nat} \rightarrow \text{nat} \ y : \text{nat} \rightarrow \text{nat} \ R1 \subseteq R2 \ R2 \subseteq \text{nat} \times \text{nat}$
 shows $\langle [x], [y] \rangle \in *R1 \implies \langle [x], [y] \rangle \in *R2$
 $\langle \text{proof} \rangle$

corollary star_rel_intersect_seq:
 assumes $x : \text{nat} \rightarrow \text{nat} \ y : \text{nat} \rightarrow \text{nat} \ R1 = R2 \cap R3 \ R2 \subseteq \text{nat} \times \text{nat} \ R3 \subseteq \text{nat} \times \text{nat}$
 shows $\langle [x], [y] \rangle \in *R1 \iff \langle [x], [y] \rangle \in *R2 \wedge \langle [x], [y] \rangle \in *R3$
 $\langle \text{proof} \rangle$

corollary less_than_seq:
 assumes $x : \text{nat} \rightarrow \text{nat} \ y : \text{nat} \rightarrow \text{nat}$
 shows $[x] * \leq [y] \iff \{i \in \text{nat}. x_i \leq y_i\} \in \mathfrak{F}$
 $\langle \text{proof} \rangle$

```

corollary less_seq:
  assumes x:nat→nat y:nat→nat
  shows [x] *< [y]  $\longleftrightarrow$  {i∈nat. xi < yi}∈ $\mathfrak{F}$ 
  <proof>

```

```

corollary less_imp_less_eq:
  assumes x:nat→nat y:nat→nat
  shows [x] *< [y]  $\longrightarrow$  [x] *≤ [y]
  <proof>

```

99.3 Transfer principle instances

Properties of the natural numbers that hold on a filter-large set of indices transfer to corresponding properties of the hypernatural numbers. This section establishes the key instances: ω is larger than every standard natural number, every function $f : \mathbb{N} \rightarrow \mathbb{N}$ extends to a hyperfunction on $\mathbb{N}^*$, every subset of the standard naturals is an internal set, and every non-empty internal set has a least element.

There is a hypernatural bigger than all natural numbers

```

lemma omega_bigger_naturals:
  assumes x:nat
  shows (*x) *≤  $\omega$ 
  <proof>

```

Every function on the natural numbers can be extended to a function on the hypernaturals

```

definition hyper_fun (*_*) where
f:nat→nat  $\implies$  *f_*  $\equiv$  internal_fun(ConstantFunction(nat,f))

```

```

lemma hyper_fun_on_nat:
  assumes f∈nat→nat x∈nat
  shows *f_*(*x) = *(fx)
  <proof>

```

Applying a function extended from the natural numbers to the class of certain sequence gives out the sequence of the composition

```

lemma hyper_fun_on_nat_2:
  assumes f∈nat→nat x∈nat→nat
  shows *f_*([x]) = [f 0 x]
  <proof>

```

Every subset of an internal set of the natural numbers is internal

```

lemma standard_internal_set:
  assumes isInternal(A) A  $\subseteq$  ({⟨x,*x⟩. x∈nat}nat) X  $\subseteq$  A
  shows isInternal(X)
  <proof>

```

Every non empty internal set has a minimum element

theorem `internal_set_has_minimum:`
 assumes `isInternal(A)` `A≠0`
 shows $\exists t \in A. \forall q \in A. t \leq q$
<proof>

99.4 Arithmetic of hypernatural numbers

Hypernatural arithmetic mirrors natural number arithmetic via the transfer principle. Every non-zero hypernatural has a predecessor, the non-standard part $\mathbb{N}^* \setminus \mathbb{N}$ has no minimum element, and the standard copy of $\mathbb{N}$ is not an internal set. The section culminates in the characterisation of hyperfinite sets as exactly those internally bijective with a bounded initial segment of $\mathbb{N}^*$.

Every hypernatural that is not zero, is a successor

theorem `succ_hyper_nat:`
 assumes `n ∈ *N` `n ≠ 0`
 shows $\exists t \in *N. * \{ \langle i, \text{succ}(i) \rangle. i \in \text{nat} \}_* t = n$
<proof>

The set of hypernaturals is not well ordered, since we can only prove that internal sets have minimums. Here is an example of a set without a minimum:

corollary `not_well_ordered:`
 defines `NonNatural` $\equiv *N - \{ \langle x, *x \rangle. x \in \text{nat} \}$
 shows $\forall t \in \text{NonNatural}. \exists q \in \text{NonNatural}. q < t$
<proof>

The natural numbers is not an internal set

corollary `nat_not_internal:`
 shows $\neg \text{isInternal}(\{ \langle x, *x \rangle. x \in \text{nat} \})$
<proof>

lemma `bij_finite:`
 assumes `Finite(A)` `Finite(B)`
 shows `Finite(A→B)` *<proof>*

lemma `hyperfinite_bijective_left_ray:`
 assumes `H ∈ Pow(*N)` `isHyperFinite(H)`
 shows $\exists N \in *N. \exists S \in \text{nat} \rightarrow \text{Pow}(\text{nat}). \exists g \in \text{Pi}(\text{nat}, \lambda i. S_i \rightarrow \text{nat}). \text{internal_fun}(g) : \text{bij}(H, \{ i \in *N. i < N \})$
<proof>

lemma `internal_inj_fun_nat:`
 assumes `S1: nat → Pow(nat)` `S2 ∈ nat → Pow(nat)` `S ∈ (∏i ∈ nat. S1 i → S2 i)`

```

    and internal_fun(S) ∈ inj(internal_set(S1), internal_set(S2))
  shows {n ∈ nat. Sn ∈ inj(S1n, S2n)} ∈  $\mathfrak{F}$  <proof>

lemma hyperfinite_bijective_left_ray_rev:
  assumes  $\exists N \in \mathbb{N}. \exists S \in \text{nat} \rightarrow \text{Pow}(\text{nat}). \exists g \in \text{Pi}(\text{nat}, \lambda i. S_i \rightarrow \text{nat}). \text{internal\_fun}(g): \text{bij}(H, \{i \in \mathbb{N} \mid i \leq N\})$ 
  shows isHyperFinite(H)
<proof>

end
end

```

100 Alternative definitions of uniformity

```

theory UniformSpace_ZF_1 imports UniformSpace_ZF
begin

```

The UniformSpace_ZF theory defines uniform spaces based on entourages (also called surroundings sometimes). In this theory we consider an alternative definition based on the notion of uniform covers.

100.1 Uniform covers

Given a set X we can consider collections of subsets of X whose unions are equal to X . Any such collection is called a cover of X . We can define relation on the set of covers of X , called "star refinement" (definition below). A collection of covers is a "family of uniform covers" if it is a filter with respect to the star refinement ordering. A member of such family is called a "uniform cover", but one has to remember that this notion has meaning only in the contexts a the whole family of uniform covers. Looking at a specific cover in isolation we can not say whether it is a uniform cover or not.

The set of all covers of X is called $\text{Covers}(X)$.

definition

$$\text{Covers}(X) \equiv \{P \in \text{Pow}(\text{Pow}(X)) \mid \bigcup P = X\}$$

A cover of a nonempty set must have a nonempty member.

```

lemma cover_nonempty: assumes  $X \neq 0 \mid P \in \text{Covers}(X)$ 

```

```

  shows  $\exists U \in P. U \neq 0$ 

```

<proof>

A "star" of R with respect to $\mathcal{R}$ is the union of all $S \in \mathcal{R}$ that intersect R .

definition

$$\text{Star}(R, \mathcal{R}) \equiv \bigcup \{S \in \mathcal{R} \mid S \cap R \neq 0\}$$

An element of $\mathcal{R}$ is a subset of its star with respect to $\mathcal{R}$.

lemma element_subset_star: assumes $U \in P$ shows $U \subseteq \text{Star}(U, P)$
<proof>

An alternative formula for star of a singleton.

lemma star_singleton: shows $(\bigcup \{V \times V. V \in P\})\{x\} = \text{Star}(\{x\}, P)$
<proof>

Star of a larger set is larger.

lemma star_mono: assumes $U \subseteq V$ shows $\text{Star}(U, P) \subseteq \text{Star}(V, P)$
<proof>

In particular, star of a set is larger than star of any singleton in that set.

corollary star_single_mono: assumes $x \in U$ shows $\text{Star}(\{x\}, P) \subseteq \text{Star}(U, P)$
<proof>

A cover $\mathcal{R}$ (of X) is said to be a "barycentric refinement" of a cover $\mathcal{C}$ iff for every $x \in X$ the star of $\{x\}$ in $\mathcal{R}$ is contained in some $C \in \mathcal{C}$.

definition

IsBarycentricRefinement ($_ <^B _$ 90)
where $P <^B Q \equiv \forall x \in \bigcup P. \exists U \in Q. \text{Star}(\{x\}, P) \subseteq U$

A cover is a barycentric refinement of the collection of stars of the singletons $\{x\}$ as x ranges over X .

lemma singl_star_bary:
assumes $P \in \text{Covers}(X)$ **shows** $P <^B \{\text{Star}(\{x\}, P). x \in X\}$
<proof>

A cover $\mathcal{R}$ is a "star refinement" of a cover $\mathcal{C}$ iff for each $R \in \mathcal{R}$ there is a $C \in \mathcal{C}$ such that the star of R with respect to $\mathcal{R}$ is contained in C .

definition

IsStarRefinement ($_ <^* _$ 90)
where $P <^* Q \equiv \forall U \in P. \exists V \in Q. \text{Star}(U, P) \subseteq V$

Every cover star-refines the trivial cover $\{X\}$.

lemma cover_stref_triv: **assumes** $P \in \text{Covers}(X)$ **shows** $P <^* \{X\}$
<proof>

Star refinement implies barycentric refinement.

lemma star_is_bary: **assumes** $Q \in \text{Covers}(X)$ **and** $Q <^* P$
shows $Q <^B P$
<proof>

Barycentric refinement of a barycentric refinement is a star refinement.

lemma bary_bary_star:
assumes $P \in \text{Covers}(X)$ $Q \in \text{Covers}(X)$ $R \in \text{Covers}(X)$ $P <^B Q$ $Q <^B R$ $X \neq \emptyset$
shows $P <^* R$

<proof>

The notion of a filter defined in `Topology_ZF_4` is not sufficiently general to use it to define uniform covers, so we write the conditions directly. A nonempty collection Θ of covers of X is a family of uniform covers if

- a) if $\mathcal{R} \in \Theta$ and $\mathcal{C}$ is any cover of X such that $\mathcal{R}$ is a star refinement of $\mathcal{C}$, then $\mathcal{C} \in \Theta$
- b) for any $\mathcal{C}, \mathcal{D} \in \Theta$ there is some $\mathcal{R} \in \Theta$ such that $\mathcal{R}$ is a star refinement of both $\mathcal{C}$ and $\mathcal{D}$.

This departs slightly from the definition in Wikipedia that requires that Θ contains the trivial cover $\{X\}$. As we show in lemma `unicov_contains_trivial` below we don't lose anything by weakening the definition this way.

definition

```
AreUniformCovers ( _ {are uniform covers of} _ 90)
where  $\Theta$  {are uniform covers of}  $X \equiv \Theta \subseteq \text{Covers}(X) \wedge \Theta \neq \emptyset \wedge$ 
 $(\forall \mathcal{R} \in \Theta. \forall \mathcal{C} \in \text{Covers}(X). ((\mathcal{R} <^* \mathcal{C}) \longrightarrow \mathcal{C} \in \Theta)) \wedge$ 
 $(\forall \mathcal{C} \in \Theta. \forall \mathcal{D} \in \Theta. \exists \mathcal{R} \in \Theta. (\mathcal{R} <^* \mathcal{C}) \wedge (\mathcal{R} <^* \mathcal{D}))$ 
```

A family of uniform covers contain the trivial cover $\{X\}$.

lemma `unicov_contains_triv`: **assumes** Θ {are uniform covers of} X
shows $\{X\} \in \Theta$

<proof>

If Θ are uniform covers of X then we can recover X from Θ by taking $\bigcup \bigcup \Theta$.

lemma `space_from_unicov`: **assumes** Θ {are uniform covers of} X **shows**
 $X = \bigcup \bigcup \Theta$

<proof>

Every uniform cover has a star refinement.

lemma `unicov_has_star_ref`:
assumes Θ {are uniform covers of} X **and** $P \in \Theta$
shows $\exists Q \in \Theta. (Q <^* P)$
<proof>

In particular, every uniform cover has a barycentric refinement.

corollary `unicov_has_bar_ref`:
assumes Θ {are uniform covers of} X **and** $P \in \Theta$
shows $\exists Q \in \Theta. (Q <^B P)$
<proof>

From the definition of uniform covers we know that if a uniform cover P is a star-refinement of a cover Q then Q is in a uniform cover. The next lemma shows that in order for Q to be a uniform cover it is sufficient that P is a barycentric refinement of Q .

lemma `unicov_bary_cov`:

assumes Θ {are uniform covers of} X $P \in \Theta$ $Q \in \text{Covers}(X)$ $P <^B Q$ **and**
 $X \neq 0$
shows $Q \in \Theta$
<proof>

A technical lemma to simplify proof of the `uniformity_from_unicov` theorem.

lemma `star_ref_mem`: **assumes** $U \in P$ $P <^* Q$ **and** $\bigcup \{W \times W. W \in Q\} \subseteq A$
shows $U \times U \subseteq A$
<proof>

An identity related to square (in the sense of composition) of a relation of the form $\bigcup \{U \times U : U \in P\}$. I am amazed that Isabelle can see that this is true without an explicit proof, I can't.

lemma `rel_square_starr`: **shows**
 $(\bigcup \{U \times U. U \in P\}) \circ (\bigcup \{U \times U. U \in P\}) = \bigcup \{U \times \text{Star}(U, P). U \in P\}$
<proof>

An identity similar to `rel_square_starr` but with `Star` on the left side of the Cartesian product:

lemma `rel_square_starl`: **shows**
 $(\bigcup \{U \times U. U \in P\}) \circ (\bigcup \{U \times U. U \in P\}) = \bigcup \{\text{Star}(U, P) \times U. U \in P\}$
<proof>

A somewhat technical identity about the square of a symmetric relation:

lemma `rel_sq_image`:
assumes $W = \text{converse}(W)$ $\text{domain}(W) \subseteq X$
shows $\text{Star}(\{x\}, \{W\{t\}. t \in X\}) = (W \circ W)\{x\}$
<proof>

Given a family of uniform covers of X we can create a uniformity on X by taking the supersets of $\bigcup \{A \times A : A \in P\}$ as P ranges over the uniform covers. The next definition specifies the operation creating entourages from uniform covers.

definition
 $\text{UniformityFromUniCov}(X, \Theta) \equiv \text{Supersets}(X \times X, \{\bigcup \{U \times U. U \in P\}. P \in \Theta\})$

For any member P of a cover Θ the set $\bigcup \{U \times U : U \in P\}$ is a member of $\text{UniformityFromUniCov}(X, \Theta)$.

lemma `basic_unif`: **assumes** $\Theta \subseteq \text{Covers}(X)$ $P \in \Theta$
shows $\bigcup \{U \times U. U \in P\} \in \text{UniformityFromUniCov}(X, \Theta)$
<proof>

If Θ is a family of uniform covers of X then $\text{UniformityFromUniCov}(X, \Theta)$ is a uniformity on X

theorem `uniformity_from_unicov`:
assumes Θ {are uniform covers of} X $X \neq 0$

shows $\text{UniformityFromUniCov}(X, \Theta)$ {is a uniformity on} X
<proof>

Given a uniformity Φ on X we can create a family of uniform covers by taking the collection of covers P for which there exist an entourage $U \in \Phi$ such that for each $x \in X$, there is an $A \in P$ such that $U(\{x\}) \subseteq A$. The next definition specifies the operation of creating a family of uniform covers from a uniformity.

definition

$\text{UniCovFromUniformity}(X, \Phi) \equiv \{P \in \text{Covers}(X) \mid \exists U \in \Phi. \forall x \in X. \exists A \in P. U(\{x\}) \subseteq A\}$

When we convert the quantifiers into unions and intersections in the definition of $\text{UniCovFromUniformity}$ we get an alternative definition of the operation that creates a family of uniform covers from a uniformity. Just a curiosity, not used anywhere.

lemma $\text{UniCovFromUniformityDef}$: **assumes** $X \neq 0$

shows $\text{UniCovFromUniformity}(X, \Phi) = (\bigcup U \in \Phi. \bigcap x \in X. \{P \in \text{Covers}(X) \mid \exists A \in P. U(\{x\}) \subseteq A\})$
<proof>

If Φ is a (diagonal) uniformity on X , then covers of the form $\{W\{x\} : x \in X\}$ are members of $\text{UniCovFromUniformity}(X, \Phi)$.

lemma cover_image :

assumes Φ {is a uniformity on} X $W \in \Phi$
shows $\{W\{x\} \mid x \in X\} \in \text{UniCovFromUniformity}(X, \Phi)$
<proof>

If Φ is a (diagonal) uniformity on X , then every two elements of $\text{UniCovFromUniformity}(X, \Phi)$ have a common barycentric refinement.

lemma $\text{common_bar_refinemnt}$:

assumes
 Φ {is a uniformity on} X
 $\Theta = \text{UniCovFromUniformity}(X, \Phi)$
 $\mathcal{C} \in \Theta$ $\mathcal{D} \in \Theta$
shows $\exists \mathcal{R} \in \Theta. (\mathcal{R} <^B \mathcal{C}) \wedge (\mathcal{R} <^B \mathcal{D})$
<proof>

If Φ is a (diagonal) uniformity on X , then every element of $\text{UniCovFromUniformity}(X, \Phi)$ has a barycentric refinement there.

corollary bar_refinement_ex :

assumes Φ {is a uniformity on} X $\Theta = \text{UniCovFromUniformity}(X, \Phi)$ $\mathcal{C} \in \Theta$
shows $\exists \mathcal{R} \in \Theta. (\mathcal{R} <^B \mathcal{C})$
<proof>

If Φ is a (diagonal) uniformity on X , then $\text{UniCovFromUniformity}(X, \Phi)$ is a family of uniform covers.

theorem unicov_from_uniformity: **assumes** Φ {is a uniformity on} X **and** $X \neq 0$
shows $\text{UniCovFromUniformity}(X, \Phi)$ {are uniform covers of} X
<proof>

The $\text{UniCovFromUniformity}$ operation is the inverse of $\text{UniformityFromUniCov}$.

theorem unicov_from_unif_inv: **assumes** Θ {are uniform covers of} X $X \neq 0$
shows $\text{UniCovFromUniformity}(X, \text{UniformityFromUniCov}(X, \Theta)) = \Theta$
<proof>

The $\text{UniformityFromUniCov}$ operation is the inverse of $\text{UniCovFromUniformity}$.

theorem unif_from_unicov_inv: **assumes** Φ {is a uniformity on} X $X \neq 0$
shows $\text{UniformityFromUniCov}(X, \text{UniCovFromUniformity}(X, \Phi)) = \Phi$
<proof>

end

101 Set of uniformities as a complete lattice

theory UniformityLattice_ZF **imports** Cardinal_ZF UniformSpace_ZF Lattice_ZF

begin

In this theory we consider the collection of all uniformities on a arbitrary set X . Since uniformities are sets of subsets of $X \times X$ such set of uniformities is naturally ordered by the inclusion relation. Specifically, for two uniformities $\mathcal{U}_1$ and $\mathcal{U}_2$ on a set X if $\mathcal{U}_1 \subseteq \mathcal{U}_2$ we say that $\mathcal{U}_2$ is finer than $\mathcal{U}_1$ or that $\mathcal{U}_1$ is coarser than $\mathcal{U}_2$. In this theory we show that with this order the collection of uniformities forms a complete lattice.

101.1 Uniformities on a fixed set as a partially ordered set

In this section we introduce the definitions of the set of uniformities on X and show that the inclusion relation is on this set makes it a partially ordered set (a poset) with a maximum and minimum.

We define $\text{Uniformities}(X)$ as the set of all uniformities on X .

definition

$\text{Uniformities}(X) \equiv \{\Phi \in \text{Pow}(\text{Pow}(X \times X)) \mid \Phi \text{ {is a uniformity on} } X\}$

If Φ is a uniformity on X , then Φ is a collection of subsets of $X \times X$, hence it's a member of $\text{Uniformities}(X)$.

lemma unif_in_unifs: **assumes** Φ {is a uniformity on} X
shows $\Phi \in \text{Uniformities}(X)$
<proof>

For nonempty sets the set of uniformities is not empty as well.

lemma `uniformities_exist`: `assumes  $X \neq \emptyset$  shows  $\text{Uniformities}(X) \neq \emptyset$`
`<proof>`

Uniformities on a set X are naturally ordered by inclusion, we call the resulting order relation `OrderOnUniformities`.

definition

`OrderOnUniformities(X)  $\equiv$  InclusionOn(Uniformities(X))`

For the order on uniformities two uniformities are in relation iff one is contained in the other.

lemma `order_unif_iff`:

`assumes  $\Phi \in \text{Uniformities}(X)$   $\Psi \in \text{Uniformities}(X)$`
`shows  $\langle \Phi, \Psi \rangle \in \text{OrderOnUniformities}(X) \longleftrightarrow \Phi \subseteq \Psi$`
`<proof>`

The order defined by inclusion on uniformities is a partial order.

lemma `ord_unif_partial_ord`:

`shows  $\text{IsPartOrder}(\text{Uniformities}(X), \text{OrderOnUniformities}(X))$`
`<proof>`

In particular, the order defined by inclusion on uniformities is antisymmetric. Having this as a separate fact is handy as we reference some lemmas proven for antisymmetric (not necessarily partial order) relations.

corollary `ord_unif_antisymm`: `shows  $\text{antisym}(\text{OrderOnUniformities}(X))$`
`<proof>`

If X is not empty then the singleton $\{X \times X\}$ is the minimal element of the set of uniformities on X ordered by inclusion and the collection of subsets of $X \times X$ that contain the diagonal is the maximal element.

theorem `uniformities_min_max`: `assumes  $X \neq \emptyset$  shows`

`HasAminimum( $\text{OrderOnUniformities}(X), \text{Uniformities}(X)$ )`
`Minimum( $\text{OrderOnUniformities}(X), \text{Uniformities}(X)$ ) =  $\{X \times X\}$`
`HasAmaximum( $\text{OrderOnUniformities}(X), \text{Uniformities}(X)$ )`
`Maximum( $\text{OrderOnUniformities}(X), \text{Uniformities}(X)$ ) =  $\{U \in \text{Pow}(X \times X) . \text{id}(X) \subseteq U\}$`
`<proof>`

101.2 Least upper bound of a set of uniformities

In this section we show that inclusion order on the collection of uniformities on a fixed set is (Dedekind) complete i.e. every nonempty set of uniformities has a least upper bound.

Given a set of uniformities $\mathcal{U}$ on X we define a collection of subsets of X called `LUB_UnifBase` (the least upper bound base in comments) as the set of all products of nonempty finite subsets of $\bigcup \mathcal{U}$. The "least upper bound base" term is not justified at this point, but we will show later that this set is actually a uniform base (i.e. a fundamental system of entourages) on X and

hence the supersets of it form a uniformity on X , which is the supremum (i.e. the least upper bound) of $\mathcal{U}$.

definition $\text{LUB_UnifBase}(\mathcal{U}) = \{\bigcap M. M \in \text{FinPow}(\bigcup \mathcal{U}) \setminus \{\emptyset\}\}$

For any two sets in the least upper bound base there is a third one contained in both.

lemma `lub_unif_base_1st_cond:`
assumes $\mathcal{U} \subseteq \text{Uniformities}(X)$ $U_1 \in \text{LUB_UnifBase}(\mathcal{U})$ $U_2 \in \text{LUB_UnifBase}(\mathcal{U})$
shows $\exists U_3 \in \text{LUB_UnifBase}(\mathcal{U}). U_3 \subseteq U_1 \cap U_2$
<proof>

Each set in the least upper bound base contains the diagonal of $X \times X$.

lemma `lub_unif_base_2nd_cond:`
assumes $\mathcal{U} \subseteq \text{Uniformities}(X)$ $U \in \text{LUB_UnifBase}(\mathcal{U})$
shows $\text{id}(X) \subseteq U$
<proof>

The converse of each set from the least upper bound base contains a set from it.

lemma `lub_unif_base_3rd_cond:`
assumes $\mathcal{U} \subseteq \text{Uniformities}(X)$ $U_1 \in \text{LUB_UnifBase}(\mathcal{U})$
shows $\exists U_2 \in \text{LUB_UnifBase}(\mathcal{U}). U_2 \subseteq \text{converse}(U_1)$
<proof>

For each set (relation) U_1 from the least upper bound base there is another one U_2 such that U_2 composed with itself is contained in U_1 .

lemma `lub_unif_base_4th_cond:`
assumes $\mathcal{U} \subseteq \text{Uniformities}(X)$ $U_1 \in \text{LUB_UnifBase}(\mathcal{U})$
shows $\exists U_2 \in \text{LUB_UnifBase}(\mathcal{U}). U_2 \circ U_2 \subseteq U_1$
<proof>

The least upper bound base is a collection of relations on X .

lemma `lub_unif_base_5th_cond:`
assumes $\mathcal{U} \subseteq \text{Uniformities}(X)$ **shows** $\text{LUB_UnifBase}(\mathcal{U}) \subseteq \text{Pow}(X \times X)$
<proof>

If a collection of uniformities is nonempty, then the least upper bound base is non-empty as well.

lemma `lub_unif_base_6th_cond:` **assumes** $\mathcal{U} \subseteq \text{Uniformities}(X)$ $\mathcal{U} \neq \emptyset$
shows $\text{LUB_UnifBase}(\mathcal{U}) \neq \emptyset$
<proof>

If a collection of uniformities $\mathcal{U}$ is nonempty, $\mathcal{B}$ denotes the least upper bound base for $\mathcal{U}$, then $\mathcal{B}$ is a uniform base on X , hence its supersets form a uniformity on X and the uniform topology generated by that uniformity is indeed a topology on X .

theorem lub_unif_base_base:
 assumes $X \neq \emptyset$ $\mathcal{U} \subseteq \text{Uniformities}(X)$ $\mathcal{U} \neq \emptyset$
 defines $\mathfrak{B} \equiv \text{LUB_UnifBase}(\mathcal{U})$
 shows
 $\mathfrak{B}$ {is a uniform base on} X
 $\text{Supersets}(X \times X, \mathfrak{B})$ {is a uniformity on} X
 $\text{UniformTopology}(\text{Supersets}(X \times X, \mathfrak{B}), X)$ {is a topology}
 $\bigcup \text{UniformTopology}(\text{Supersets}(X \times X, \mathfrak{B}), X) = X$
<proof>

At this point we know that supersets with respect to $X \times X$ of the least upper bound base for a collection of uniformities $\mathcal{U}$ form a uniformity. To shorten the notation we will call this uniformity $\text{LUB_Unif}(X, \mathcal{U})$.

definition

$\text{LUB_Unif}(X, \mathcal{U}) \equiv \text{Supersets}(X \times X, \text{LUB_UnifBase}(\mathcal{U}))$

With this definition we can rewrite some assertions of theorem lub_unif_base_base in a bit shorter form:

corollary lub_unif_base_base1: assumes $X \neq \emptyset$ $\mathcal{U} \subseteq \text{Uniformities}(X)$ $\mathcal{U} \neq \emptyset$
 shows
 $\text{LUB_Unif}(X, \mathcal{U})$ {is a uniformity on} X
 $\text{UniformTopology}(\text{LUB_Unif}(X, \mathcal{U}), X)$ {is a topology}
 $\bigcup \text{UniformTopology}(\text{LUB_Unif}(X, \mathcal{U}), X) = X$
<proof>

For any collection of uniformities $\mathcal{U}$ on a nonempty set X the $\text{LUB_Unif}(X, \mathcal{U})$ collection defined above is indeed an upper bound of $\mathcal{U}$ in the order defined by the inclusion relation.

lemma lub_unif_upper_bound:

 assumes $X \neq \emptyset$ $\mathcal{U} \subseteq \text{Uniformities}(X)$ $\Phi \in \mathcal{U}$
 shows $\langle \Phi, \text{LUB_Unif}(X, \mathcal{U}) \rangle \in \text{OrderOnUniformities}(X)$
<proof>

Any upper bound (in the order defined by inclusion relation) of a nonempty collection of uniformities $\mathcal{U}$ on a nonempty set X is greater or equal (in that order) than $\text{LUB_Unif}(X, \mathcal{U})$. Together with lub_unif_upper_bound it means that $\text{LUB_Unif}(X, \mathcal{U})$ is indeed the least upper bound of $\mathcal{U}$.

lemma lub_unif_lub:

 assumes $X \neq \emptyset$ $\mathcal{U} \subseteq \text{Uniformities}(X)$ $\mathcal{U} \neq \emptyset$ and
 $\forall \Phi \in \mathcal{U}. \langle \Phi, \Psi \rangle \in \text{OrderOnUniformities}(X)$
 shows $\langle \text{LUB_Unif}(X, \mathcal{U}), \Psi \rangle \in \text{OrderOnUniformities}(X)$
<proof>

A nonempty collection $\mathcal{U}$ of uniformities on X has a supremum (i.e. the least upper bound) which is a uniformity.

theorem lub_unif_sup: assumes $X \neq \emptyset$ $\mathcal{U} \subseteq \text{Uniformities}(X)$ $\mathcal{U} \neq \emptyset$
 shows

```

    HasAsupremum(OrderOnUniformities(X),  $\mathcal{U}$ )
    LUB_Unif(X,  $\mathcal{U}$ ) = Supremum(OrderOnUniformities(X),  $\mathcal{U}$ )
    Supremum(OrderOnUniformities(X),  $\mathcal{U}$ ) {is a uniformity on} X
  <proof>

```

The order on uniformities derived from inclusion is complete.

```

theorem uniformities_complete: assumes  $X \neq \emptyset$ 
  shows OrderOnUniformities(X) {is complete}
  <proof>

```

Typically the union of a collection of uniformities is not a uniformity. However, if it is then that union is the supremum of that collection.

```

lemma union_unif_sup:
  assumes  $X \neq \emptyset$   $\mathcal{U} \subseteq \text{Uniformities}(X)$   $\mathcal{U} \neq \emptyset$   $(\bigcup \mathcal{U})$  {is a uniformity on} X
  shows  $\bigcup \mathcal{U} = \text{Supremum}(\text{OrderOnUniformities}(X), \mathcal{U})$ 
  <proof>

```

101.3 Greatest lower bound of a set of uniformities

In this this section we show that every set of uniformities on a fixed set X has a greatest lower bound, i.e. an infimum. The approach taken in the previous section to show the existence of the least upper bound of a collection of uniformities does not work for the greatest lower bound. The collection defined as the set of all products of nonempty finite subsets of $\bigcap \mathcal{U}$ in general is not a fundamental system of entourages. Even though the first three conditions hold for such collection, the fourth one does not. The approach that works is to show the the supremum of the collection of lower bounds is a lower bound, hence the maximum of the set of lower bounds. We present this approach below in theorem `unif_inf_alt`. The simplest approach though is to use the general property of partially ordered sets shown in lemma `bounded_below_sups_infs` in the `Order_ZF_1a` theory: if all nonempty subsets of such set have suprema and the whole set has a minimum then all nonempty subsets have infima.

To shorten the proofs we introduce the concept of the supremum of the set of lower bounds of some collection of uniformities $\mathcal{U}$. We know from the previous section that such supremum exists. Later in this section we show that this supremum is itself a lower bound of $\mathcal{U}$, so it's the greatest lower bound i.e. infimum of $\mathcal{U}$.

definition

```

SLB_Unif(X,  $\mathcal{U}$ )  $\equiv$  Supremum(OrderOnUniformities(X),  $\{\Psi \in \text{Uniformities}(X) . \Psi \subseteq \bigcap \mathcal{U}\}$ )

```

Let $\mathfrak{U}$ denote the collection of all uniformities on a nonempty set X , ordered by inclusion. Then every collection nonempty of uniformities $\mathcal{U} \subseteq \mathfrak{U}$ has an

infimum (the greatest lower bound) which is equal to the supremum of the collection of lower bounds.

```
theorem unif_inf: assumes  $X \neq \emptyset$   $\mathcal{U} \subseteq \text{Uniformities}(X)$   $\mathcal{U} \neq \emptyset$ 
  shows  $\text{HasAnInfimum}(\text{OrderOnUniformities}(X), \mathcal{U})$  and
   $\text{Infimum}(\text{OrderOnUniformities}(X), \mathcal{U}) = \text{SLB\_Unif}(X, \mathcal{U})$ 
 $\langle \text{proof} \rangle$ 
```

Uniformities on a nonempty set, ordered by inclusion form a complete lattice.

```
theorem uniformities_compl_latt: assumes  $X \neq \emptyset$ 
  shows  $\text{IsCompleteLattice}(\text{OrderOnUniformities}(X), \text{Uniformities}(X))$ 
 $\langle \text{proof} \rangle$ 
```

The rest of the propositions in this section presents an alternative, more explicit approach to proving the existence of infimum of a set of uniformities that does not use theorem `bounded_below_sups_infs` from the `Order_ZF_1a` theory. It's here mostly as a curiosity, since I have formalised this approach first and I did not want to remove it as it may be of some interest to somebody. The next lemma shows that the set of lower bounds of a nonempty set of uniformities is nonempty.

```
lemma lb_nempty_nempty: assumes  $X \neq \emptyset$   $\mathcal{U} \subseteq \text{Uniformities}(X)$   $\mathcal{U} \neq \emptyset$ 
  shows  $\{\Psi \in \text{Uniformities}(X). \Psi \subseteq \bigcap \mathcal{U}\} \neq \emptyset$ 
 $\langle \text{proof} \rangle$ 
```

The supremum of the set of lower bounds of a collection of uniformities is a lower bound of that collection.

```
lemma unif_slb_is_lb: assumes  $X \neq \emptyset$   $\mathcal{U} \subseteq \text{Uniformities}(X)$   $\mathcal{U} \neq \emptyset$ 
  shows  $\text{SLB\_Unif}(X, \mathcal{U}) \subseteq \bigcap \mathcal{U}$ 
 $\langle \text{proof} \rangle$ 
```

The next theorem states the same assertion as `unif_inf` with a different proof that does not rely on the `bounded_below_sups_infs` lemma from the `Order_ZF_1a` theory.

```
theorem unif_inf_alt: assumes  $X \neq \emptyset$   $\mathcal{U} \subseteq \text{Uniformities}(X)$   $\mathcal{U} \neq \emptyset$ 
  shows  $\text{HasAnInfimum}(\text{OrderOnUniformities}(X), \mathcal{U})$  and
   $\text{Infimum}(\text{OrderOnUniformities}(X), \mathcal{U}) = \text{SLB\_Unif}(X, \mathcal{U})$ 
 $\langle \text{proof} \rangle$ 
```

end

102 Real valued metric spaces

```
theory MetricSpace_ZF_1 imports Real_ZF_2
begin
```

The development of metric spaces in `IsarMathLib` is different from the usual treatment of the subject because the notion of a metric (or a pseudometric)

is defined in the `MetricSpace_ZF` theory more generally as a function valued in an ordered loop. This theory file brings the subject closer to the standard way by specializing that general definition to the usual special case where the value of the metric are nonnegative real numbers.

102.1 Real valued metric spaces: context and notation

The `reals` context (locale) defined in the `Real_ZF_2` theory fixes a model of reals (i.e. a complete ordered field) and defines notation for things like zero, one, the set of positive numbers, absolute value etc. For metric spaces we reuse the notation defined there.

The `rpmetric_space` locale extends the `reals` locale, adding the carrier X of the metric space and the metric $\lceil$ to the context, together with the assumption that $\lceil : X \times X \rightarrow \mathbb{R}^+$ is a pseudo metric. We choose to denote the disk in X with center c and radius r as `ball(c,r)`. As in the `pmetric_space` locale we define the τ to be the metric topology, i.e. the topology induced by the (real valued) pseudometric $\lceil$. An alternative would be to define the `rpmetric_space` as an extension of the `rpmetric_space` context, but that is in turn an extension of the `loop1` locale that defines notation for left and right division which which do not want in the context of real numbers.

```
locale rpmetric_space = reals +
  fixes X and d
  assumes pmetricAssum: IsApseudoMetric(d,X, $\mathbb{R}$ ,Add,ROrd)
  fixes ball
  defines ball_def [simp]: ball(c,r)  $\equiv$  Disk(X,d,ROrd,c,r)
  fixes pmettop ( $\tau$ )
  defines pmettop [simp]:  $\tau \equiv$  MetricTopology(X, $\mathbb{R}$ ,Add,ROrd,d)
  fixes interior (int)
  defines interior_def [simp]: int(D)  $\equiv$  Interior(D, $\tau$ )
  fixes cl
  defines cl_def [simp]: cl(D)  $\equiv$  Closure(D, $\tau$ )
```

The propositions proven in the `pmetric_space` context defined in `Metric_Space_ZF` theory are valid in the `rpmetric_space` context.

```
lemma (in rpmetric_space) pmetric_space_rpmetric_space_valid:
  shows pmetric_space( $\mathbb{R}$ ,Add,ROrd,d,X)
  <proof>
```

The context `rpmetric_space` is a special case of context `pmetric_space` where the fixed objects in `pmetric_space` map to (in the order defined in `pmetric_space`) the set of real numbers, real addition, the order relation on reals, the strict order relation on reals, the set of non-negative reals and the set of positive reals. The metrics d maps to the real metrics d , the carrier of the metric space X is still X , and the disks from `pmetric_space` are now called

balls in `rpmetric_space`. The notation for right and left division from `rpmetric_space` is not used in `pmetric_space`.

```

sublocale rpmetric_space < pmetric_space
  R Add R0rd 0 realadd lesseq sless nonnegative positiveset
  λx y. LeftDiv(R,Add)⟨x,y⟩
  λx y. RightDiv(R,Add)⟨y,x⟩
  listsum nat_mult d X ball
  ⟨proof⟩

```

The `rmetric_space` locale (context) specializes the the `rpmetric_space` context by adding the assumption of identity of indiscernibles.

```

locale rmetric_space = rpmetric_space +
  assumes ident_indisc: ∀x∈X. ∀y∈X. d⟨x,y⟩ = 0 ⟶ x=y

```

The propositions proven in the `metric_space` context defined in `Metric_Space_ZF` theory are valid in the `rmetric_space` context.

```

lemma (in rmetric_space) metric_space_rmetric_space_valid:
  shows metric_space(R,Add,R0rd,d,X)
  ⟨proof⟩

```

The `rmetric_space` context is a special case of the `metric_space` context, with fixed objects mapping the same as in the mapping between `rpmetric_space` and `pmetric_space` above.

```

sublocale rmetric_space < metric_space
  R Add R0rd 0 realadd lesseq sless nonnegative positiveset
  λx y. LeftDiv(R,Add)⟨x,y⟩
  λx y. RightDiv(R,Add)⟨y,x⟩
  listsum nat_mult d X ball
  ⟨proof⟩

```

102.2 Real valued metric spaces are Hausdorff as topological spaces

The usual (real-valued) metric spaces are a special case of ordered loop valued metric spaces defined in the `MetricSpace_ZF` theory, hence they are T_2 as topological spaces. Below we repeat the major theorems of `MetricSpace_ZF` theory specialized the standard setting of real valued metrics.

Since in the `rpmetric_space` context $\mathfrak{D}$ is a pseudometrics the (real valued) metric topology indeed a topology.

```

theorem (in rpmetric_space) rpmetric_is_top:
  shows τ {is a topology}
  ⟨proof⟩

```

The collection of open disks (caled balls in the `rpmetric_space` context is a base for the (real valued) metric topology.

theorem (in `rpmetric_space`) `rdisks_are_base`:
shows $(\bigcup_{c \in X. \{ \text{ball}(c, R). R \in \mathbb{R}_+ \}})$ {is a base for} τ
 $\langle \text{proof} \rangle$

X is the carrier of the (real valued) metric topology.

theorem (in `rpmetric_space`) `rmetric_top_carrier`: **shows** $\bigcup \tau = X$
 $\langle \text{proof} \rangle$

The topology generated by a (real valued) metric is Hausdorff (i.e. T_2).

theorem (in `rmetric_space`) `rmetric_space_T2`: **shows** τ {is T_2 }
 $\langle \text{proof} \rangle$

102.3 Real valued (pseudo)metric spaces as uniform spaces

The ordered loop valued pseudometric spaces are uniform spaces. In this section we specialize major propositions from that context to the real valued pseudometric.

In the `MetricSpace_ZF` theory we define a property `IsHalfable` of an ordered loop that states that for every positive element b_1 of the loop there is another (positive) one b_2 such that $b_2 + b_2 \leq b_1$. This property is needed for the ordered loop valued pseudometric space to be a uniform space. In the next lemma we show that real numbers satisfy this property.

lemma (in `reals`) `pos_reals_halfable`: **shows** `IsHalfable`($\mathbb{R}$, `Add`, `ROrd`)
 $\langle \text{proof} \rangle$

In the `rpmetric_space` we will write `UniformGauge`($X, \mathbb{R}, \text{Add}, \text{ROrd}, d$) i.e. $\{ \lceil^{-1}([0, b] : b \in \mathbb{R}_+ \}$ as $\mathcal{U}$.

abbreviation (in `rpmetric_space`) `rgauge` ($\mathcal{U}$) **where** $\mathcal{U} \equiv \text{UniformGauge}(X, \mathbb{R}, \text{Add}, \text{ROrd}, d)$

$\mathcal{U}$ is a fundamental system of entourages, hence its supersets form a uniformity on X and hence those supersets define a topology on X . This is a special case of the theorem `metric_gauge_base` from the `Metric_Space_ZF` theory but instead an ordered loop we have real numbers, so all the premises are automatically satisfied, except for the one of X being nonempty.

theorem (in `rpmetric_space`) `metric_gauge_base`: **assumes** $X \neq \emptyset$
shows
 $\mathcal{U}$ {is a uniform base on} X
`Supersets`($X \times X, \mathcal{U}$) {is a uniformity on} X
`UniformTopology`(`Supersets`($X \times X, \mathcal{U}$), X) {is a topology}
 $\bigcup \text{UniformTopology}(\text{Supersets}(X \times X, \mathcal{U}), X) = X$
 $\langle \text{proof} \rangle$

The topology generated by open disks is the same as the one coming from the the uniformity consisting of supersets of sets in $\mathcal{U}$.

theorem (in `rpmetric_space`) `rmetric_top_is_uniform_top`: **assumes** $X \neq \emptyset$

```

shows  $\tau = \text{UniformTopology}(\text{Supersets}(X \times X, \mathcal{U}), X)$ 
  <proof>

end

```

103 Uniformity defined by a collection of pseudometrics

```

theory MetricUniform_ZF imports FinOrd_ZF_1 MetricSpace_ZF UniformityLattice_ZF

begin

```

In the `MetricSpace_ZF` we show how a single (ordered loop valued) pseudometric defines a uniformity. In this theory we extend this to the situation where we have an arbitrary collection of pseudometrics, all defined on the same set X and valued in an ordered loop L . Since real numbers form an ordered loop all results proven in this theory are true for the standard real-valued pseudometrics.

103.1 Uniformity defined by a collection of pseudometrics

Suppose $\mathcal{M}$ is a collection of (an ordered loop valued) pseudometrics on X , i.e. $d : X \times X \rightarrow L^+$ is a pseudometric for every $d \in \mathcal{M}$. Then, for each $d \in \mathcal{M}$ the collection of sets $\mathfrak{B} = \{d^{-1}(\{c \in L^+ : c \leq b\}) : b \in L_+\}$ forms a fundamental system of entourages. Consequently, by taking supersets of $\mathfrak{B}$ we can get a uniformity defined by the pseudometric (see theorem `metric_gauge_base` in the `MetricSpace_ZF` theory. Repeating this process for each $d \in \mathcal{M}$ we obtain a collection of uniformities on X . Since all uniformities on X ordered by inclusion form a complete lattice (see theorem `uniformities_compl_latt` in the `UniformityLattice_ZF`) there exists the smallest (coarsest) uniformity that contains all uniformities defined by the pseudometrics in $\mathcal{M}$.

To shorten the main definition let's define the uniformity derived from a single pseudometric as `UnifFromPseudometric`. In the definition below X is the underlying set, L, A, r are the carrier, binary operation and the order relation, resp., of the ordered loop where the pseudometrics takes its values and d is the pseudometrics.

definition

```

UnifFromPseudometric(X,L,A,r,d)  $\equiv$  Supersets( $X \times X$ ,UniformGauge( $X,L,A,r,d$ ))

```

We construct the uniformity from a collection of pseudometrics $\mathcal{M}$ by taking the supremum (i.e. the least upper bound) of the collection of uniformities defined by each pseudometric from $\mathcal{M}$.

definition

$\text{UnifFromPseudometrics}(X, L, A, r, \mathcal{M}) \equiv \text{LUB_Unif}(X, \{\text{UnifFromPseudometric}(X, L, A, r, d) \mid d \in \mathcal{M}\})$

The context `multiple_pmetric` is very similar to the `pmetric_space` context except that rather than fixing a single pseudometric d we fix a nonempty collection of pseudometrics $\mathcal{M}$. That forces the notation for `disk`, topology, interior and closure to depend on the pseudometric d . The $\mathcal{U}$ symbol will denote the collection of uniformities generated by $\mathcal{M}$. We also assume that the positive cone L_+ of the loop is nonempty, r down-directs L_+ (see `Order_ZF` for definition) and that the (positive cone of the) ordered loop is halfable (see `MetricSpace_ZF`). In short, we assume what we need for ordered loop valued pseudometrics to generate uniformities.

```

locale multiple_pmetric = loop1 +
  fixes  $\mathcal{M}$  and  $X$ 
  assumes emptyX:  $X \neq \emptyset$ 
  assumes emptyLp:  $L_+ \neq \emptyset$ 
  assumes downdir:  $r$  {down-directs}  $L_+$ 
  assumes halfable: IsHalfable( $L, A, r$ )
  assumes emptyM:  $\mathcal{M} \neq \emptyset$ 
  assumes mpmetricAssm:  $\forall d \in \mathcal{M}. \text{IsApseudoMetric}(d, X, L, A, r)$ 
  fixes  $\mathcal{U}$ 
  defines unifs_def [simp]:  $\mathcal{U} \equiv \{\text{UnifFromPseudometric}(X, L, A, r, d) \mid d \in \mathcal{M}\}$ 
  fixes disk
  defines disk_def [simp]: disk( $d, c, R$ )  $\equiv \text{Disk}(X, d, r, c, R)$ 
  fixes pmettop ( $\tau$ )
  defines pmettop [simp]:  $\tau(d) \equiv \text{MetricTopology}(X, L, A, r, d)$ 
  fixes interior (int)
  defines interior_def [simp]: int( $d, D$ )  $\equiv \text{Interior}(D, \tau(d))$ 
  fixes cl
  defines cl_def [simp]: cl( $d, D$ )  $\equiv \text{Closure}(D, \tau(d))$ 

```

If d is one of the pseudometrics from $\mathcal{M}$ then theorems proven in the `pmetric_space` context are valid as applied to d .

```

lemma (in multiple_pmetric) pmetric_space_valid_in_mpm:
  assumes  $d \in \mathcal{M}$  shows pmetric_space( $L, A, r, d, X$ )
  <proof>

```

In the `multiple_pmetric` context each element of $\mathcal{U}$ is a uniformity.

```

lemma (in multiple_pmetric) each_gen_unif_unif: shows  $\mathcal{U} \subseteq \text{Uniformities}(X)$ 
  <proof>

```

The uniformity generated by a family of pseudometrics $\mathcal{M}$ is indeed a uniformity which is the supremum of uniformities in $\mathcal{U}$.

```

lemma (in multiple_pmetric) gen_unif_unif:
  shows
     $\text{UnifFromPseudometrics}(X, L, A, r, \mathcal{M})$  {is a uniformity on}  $X$ 
     $\text{UnifFromPseudometrics}(X, L, A, r, \mathcal{M}) = \text{Supremum}(\text{OrderOnUniformities}(X), \mathcal{U})$ 

```

<proof>

The uniformity generated by a family of pseudometrics contains all uniformities generated by the pseudometrics in $\mathcal{M}$.

lemma (in `multiple_pmetric`) `gen_unif_contains_unifs`: **assumes** $\Phi \in \mathcal{U}$
shows $\Phi \subseteq \text{UnifFromPseudometrics}(X, L, A, r, \mathcal{M})$
<proof>

If a uniformity contains all uniformities generated by the pseudometrics in $\mathcal{M}$ then it contains the uniformity generated by that family of pseudometrics.

lemma (in `multiple_pmetric`) `gen_unif_LUB`:
assumes Ψ {is a uniformity on} X and $\forall \Phi \in \mathcal{U}. \Phi \subseteq \Psi$
shows $\text{UnifFromPseudometrics}(X, L, A, r, \mathcal{M}) \subseteq \Psi$
<proof>

The uniformity generated by the collection of pseudometrics $\mathcal{M}$ contains all inverse images of the initial segments of the positive cone of L , (i.e. sets of the form $d^{-1}(\{c \in L : 0 \leq c \leq y\})$ for $d \in \mathcal{M}$ and y from the positive cone L_+ of L).

lemma (in `multiple_pmetric`) `gen_unif_contains_gauges`: **assumes** $d \in \mathcal{M}$ $y \in L_+$
shows $d^{-1}(\{c \in L^+ : \langle c, y \rangle \in r\}) \in \text{UnifFromPseudometrics}(X, L, A, r, \mathcal{M})$
<proof>

The uniformity generated by the collection of pseudometrics $\mathcal{M}$ is the coarsest uniformity that contains all inverse images of the initial segments of the positive cone of L with respect to all $d \in \mathcal{M}$.

theorem (in `multiple_pmetric`) `gen_unif_corsest`:
assumes Ψ {is a uniformity on} X and $\forall d \in \mathcal{M}. \forall y \in L_+. d^{-1}(\{c \in L^+ : \langle c, y \rangle \in r\}) \in \Psi$
shows $\text{UnifFromPseudometrics}(X, L, A, r, \mathcal{M}) \subseteq \Psi$
<proof>

103.2 Uniformity defined by a collection of pseudometrics - an alternative approach

This section presents an alternative, more direct approach to defining a uniformity generated by a collection of pseudometric. This approach is probably equivalent to the one presented in the previous section, but more complicated and hence obsolete.

The next two definitions describe the way a common fundamental system of entourages for $\mathcal{M}$ is constructed. First we take finite subset M of $\mathcal{M}$. Then we choose $f : M \rightarrow L_+$. This way for each $d \in M$ the value $f(d)$ is a positive element of L and $\{d^{-1}(\{c \in L^+ : c \leq f(d)\}) : d \in M\}$ is a finite collection of subsets of $X \times X$. Then we take intersections of such

finite collections as M varies over $\mathcal{M}$ and f varies over all possible functions mapping M to L_+ . To simplify notation for this construction we split it into two steps. In the first step we define a collection of finite intersections resulting from choosing a finite set of pseudometrics M , $f : M \rightarrow L_+$ and varying the selector function f over the space of functions mapping M to the set of positive elements of L .

definition

$\text{UniformGaugeSets}(X, L, A, r, M) \equiv$
 $\{(\bigcap_{d \in M} d^{-1}(\{c \in \text{Nonnegative}(L, A, r) \mid \langle c, f(d) \rangle \in r\})). f \in M \rightarrow \text{PositiveSet}(L, A, r)\}$

In the second step we collect all uniform gauge sets defined above as parameter M vary over all nonempty finite subsets of $\mathcal{M}$. This is the collection of sets that we will show forms a fundamental system of entourages.

definition

$\text{UniformGauges}(X, L, A, r, \mathcal{M}) \equiv \bigcup_{M \in \text{FinPow}(\mathcal{M}) \setminus \{\emptyset\}} \text{UniformGaugeSets}(X, L, A, r, M)$

Analogously what is done in the `pmetric_space` context we will write $\text{UniformGauges}(X, L, A, r, \mathcal{M})$ as $\mathfrak{B}$ in the `multiple_pmetric` context.

abbreviation (in `multiple_pmetric`) `mgauge` ($\mathfrak{B}$) where $\mathfrak{B} \equiv \text{UniformGauges}(X, L, A, r, \mathcal{M})$

The next lemma just shows the definition of $\mathfrak{B}$ in notation used in the `multiple_pmetric`.

lemma (in `multiple_pmetric`) `mgauge_def_alt`: **shows**

$\mathfrak{B} = (\bigcup_{M \in \text{FinPow}(\mathcal{M}) \setminus \{\emptyset\}} \{(\bigcap_{d \in M} d^{-1}(\{c \in L^+ \mid c \leq f(d)\})). f \in M \rightarrow L_+\})$
 $\langle \text{proof} \rangle$

$\mathfrak{B}$ consists of (finite) intersections of sets of the form $d^{-1}(\{c \in L^+ \mid c \leq f(d)\})$ where $f : M \rightarrow L_+$ some finite subset $M \subseteq \mathcal{M}$. More precisely, if M is a nonempty finite subset of $\mathcal{M}$ and f maps M to the positive set of the loop L , then the set $\bigcap_{d \in M} d^{-1}(\{c \in L^+ \mid c \leq f(d)\})$ is in $\mathfrak{B}$.

lemma (in `multiple_pmetric`) `mgauge_finset_fun`:

assumes $M \in \text{FinPow}(\mathcal{M})$ $M \neq \emptyset$ $f : M \rightarrow L_+$
shows $(\bigcap_{d \in M} d^{-1}(\{c \in L^+ \mid c \leq f(d)\})) \in \mathfrak{B}$
 $\langle \text{proof} \rangle$

If d is member of any finite subset of $\mathcal{M}$ then d maps $X \times X$ to the nonnegative set of the ordered loop L .

lemma (in `multiple_pmetric`) `each_pmetric_map`:

assumes $M \in \text{FinPow}(\mathcal{M})$ **and** $d \in M$ **shows** $d : X \times X \rightarrow L^+$
 $\langle \text{proof} \rangle$

Members of the uniform gauge defined by multiple pseudometrics are subsets of $X \times X$ i.e. relations on X .

lemma (in `multiple_pmetric`) `muniform_gauge_relations`:

assumes $B \in \mathfrak{B}$ **shows** $B \subseteq X \times X$

<proof>

Suppose M_1 is a subset of M and $\mathcal{M}$. f_1, f map M_1 and M , resp. to L_+ and $f \leq f_1$ on M_1 . Then the set $\bigcap_{d \in M} d^{-1}(\{y \in L_+ : y \leq f(d)\})$ is included in the set $\bigcap_{d \in M_1} d^{-1}(\{y \in L_+ : y \leq f_1(d)\})$.

lemma (in `multiple_pmetric`) `fun_inter_vimage_mono`:

assumes $M_1 \subseteq \mathcal{M}$ $M_1 \subseteq M$ $M_1 \neq \emptyset$ $f_1 : M_1 \rightarrow L_+$ $f : M \rightarrow L_+$ **and**

$\forall d \in M_1. f(d) \leq f_1(d)$

shows

$(\bigcap_{d \in M. d - (\{c \in L^+. c \leq f(d)\})) \subseteq (\bigcap_{d \in M_1. d - (\{c \in L^+. c \leq f_1(d)\}))$

<proof>

For any two sets B_1, B_2 in $\mathfrak{B}$ there exist a third one that is contained in both.

lemma (in `multiple_pmetric`) `mgauge_1st_cond`:

assumes $B_1 \in \mathfrak{B}$ $B_2 \in \mathfrak{B}$

shows $\exists B \in \mathfrak{B}. B \subseteq B_1 \cap B_2$

<proof>

Sets in $\mathfrak{B}$ contain the diagonal and are symmetric, hence contain a symmetric subset from $\mathfrak{B}$.

lemma (in `multiple_pmetric`) `mgauge_2nd_and_3rd_cond`: **assumes** $B \in \mathfrak{B}$

shows $\text{id}(X) \subseteq B$ $B = \text{converse}(B)$ $\exists B_2 \in \mathfrak{B}. B_2 \subseteq \text{converse}(B)$

<proof>

$\mathfrak{B}$ is a subset of the power set of $X \times X$.

lemma (in `multiple_pmetric`) `mgauge_5thCond`: **shows** $\mathfrak{B} \subseteq \text{Pow}(X \times X)$

<proof>

If $\mathcal{M}$ and L_+ are nonempty then $\mathfrak{B}$ is also nonempty.

lemma (in `multiple_pmetric`) `mgauge_6thCond`: **shows** $\mathfrak{B} \neq \emptyset$

<proof>

If the loop order is halfable then for every set $B_1 \in \mathfrak{B}$ there is another set $B_2 \in \mathfrak{B}$ such that $B_2 \circ B_2 \subseteq B_1$.

lemma (in `multiple_pmetric`) `mgauge_4thCond`:

assumes $B_1 \in \mathfrak{B}$ **shows** $\exists B_2 \in \mathfrak{B}. B_2 \circ B_2 \subseteq B_1$

<proof>

If $\mathcal{M}$ is a nonempty collection of pseudometrics on a nonempty set X valued in loop L partially ordered by relation r such that the set of positive elements L_+ is nonempty, r down directs L_+ and r is halfable on L , then $\mathfrak{B}$ is a fundamental system of entourages in X hence its supersets form a uniformity on X and hence those supersets define a topology on X

lemma (in `multiple_pmetric`) `mmetric_gauge_base`:

shows

```

 $\mathfrak{B}$  {is a uniform base on}  $X$ 
Supersets( $X \times X, \mathfrak{B}$ ) {is a uniformity on}  $X$ 
UniformTopology(Supersets( $X \times X, \mathfrak{B}$ ),  $X$ ) {is a topology}
 $\bigcup \text{UniformTopology}(\text{Supersets}(X \times X, \mathfrak{B}), X) = X$ 
<proof>

```

103.3 Halving sequences of entourages

The notion of uniformity can be defined in several ways. Our primary definition in the `UniformSpace_ZF` theory is based on the concept of entourages. In `UniformSpace_ZF_1` we consider an alternative definition based on uniform covers. The third possible definition is based on families of pseudometrics. "The significance of defining a uniformity by means of a family of pseudometrics lies in the fact that all uniformities can be so obtained." (from Bourbaki: General Topology Chapter IX par. 1.4). In this section we formalize a part of the material that is needed to prove this statement that can be done without the Axiom of Choice and in the general setting of ordered-loop valued pseudometrics.

The definition of a uniformity requires that for each entourage U there is another one V such that $V \circ V \subseteq U$. Furthermore, lemma `half_size_symm` shows that we can assume V is symmetric. In some propositions in this section we will assume that we are given a function say h that provides those half-size entourages, i.e. maps a uniformity Φ into itself and for each entourage $U \in \Phi$ we have $h(U) \circ h(U) \subseteq U$.

The existence of such function follows from the Axiom of Choice, but we don't want to use AC in this theory.

The next definition lists the desired properties of such halving function for shorter references in theorem assumptions.

definition

```

IsHalvingFunction ( _ {is a halving function for} _ ) where
  h {is a halving function for}  $\Phi \equiv$ 
     $h: \Phi \rightarrow \Phi \wedge (\forall U \in \Phi. h(U) = \text{converse}(h(U)) \wedge h(U) \circ h(U) \subseteq U)$ 

```

Let Φ be a uniformity on X , $U \in \Phi$ and h is a halving function for Φ . This function then defines inductively a sequence $\{H_n\}_{n \in \mathbb{N}}$ of entourages such that $H_0 = U$ and $H_{n+1} \circ H_{n+1} \subseteq H_n$. The next lemma shows that H is a Φ -valued sequence which starts at U .

lemma halving_seq_start:

```

assumes  $U \in \Phi$  h {is a halving function for}  $\Phi$ 
defines  $H \equiv \text{InductiveSequence}(U, h)$ 
shows  $H: \text{nat} \rightarrow \Phi$  and  $H(0) = U$ 
<proof>

```

For the inductively defined sequence of halving entourages H starting from U and a natural number n we indeed have $H_{n+1} \circ H_{n+1} \subseteq H_n$.


```

lemma halving_seq_halves: assumes  $U \in \Phi$   $h$  {is a halving function for}
 $\Phi$   $n \in \text{nat}$ 
  defines  $H \equiv \text{InductiveSequence}(U, h)$ 
  shows  $H(n \# + 1) = \text{converse}(H(n \# + 1))$  and  $H(n \# + 1) \cap H(n \# + 1) \subseteq H(n)$ 
<proof>

```

A halving sequence in Φ is decreasing in the inclusion order on Φ , hence the inclusion order on Φ is total on the sequence's image of the natural numbers, hence total on the sequence's image of positive natural numbers. We need that fact about positive natural numbers because we plan to prove that the image of the sequence on the positive natural numbers forms a fundamental system of symmetric entourages. and the first element of the sequence (at index 0) does not have to be symmetric.

```

lemma halving_seq_decr:
  assumes  $\Phi$  {is a uniformity on}  $X$   $U \in \Phi$   $h$  {is a halving function for}
 $\Phi$ 
  defines  $H \equiv \text{InductiveSequence}(U, h)$ 
  shows
    IsDecreasingSeq( $\Phi$ , InclusionOn( $\Phi$ ),  $H$ )
    InclusionOn( $\Phi$ ) {is total on} ( $H(\text{nat})$ )
    InclusionOn( $\Phi$ ) {is total on} ( $H(\text{nat} \setminus \{0\})$ )
<proof>

```

We aim at showing that a halving sequence image is a fundamental system of entourages. The next lemma shows the first condition in the uniform base definition: for two sets in a halving sequence image there is a third one that is contained in both.

```

lemma halving_seq_base_1st_cond:
  assumes  $\Phi$  {is a uniformity on}  $X$   $U \in \Phi$   $h$  {is a halving function for}
 $\Phi$ 
  defines  $H \equiv \text{InductiveSequence}(U, h)$ 
  assumes  $B_1 \in H(\text{nat} \setminus \{0\})$   $B_2 \in H(\text{nat} \setminus \{0\})$ 
  shows  $\exists B \in H(\text{nat} \setminus \{0\}). B \subseteq B_1 \cap B_2$ 
<proof>

```

Sets in the halving sequence image of positive naturals contain the diagonal, are symmetric and for every such set B there is another one contained in the converse of the first one and another one B_2 such that $B_2 \circ B_2 \subseteq B_1$. Note the symmetry of the sets is not required in the definition of a fundamental system of entourages, but it's good to have as we plan to construct a pseudometric that generates that halving sequence image of positive naturals.

```

lemma halving_seq_base_2_3_4_conds:
  assumes  $\Phi$  {is a uniformity on}  $X$   $U \in \Phi$   $h$  {is a halving function for}
 $\Phi$ 
  defines  $H \equiv \text{InductiveSequence}(U, h)$ 
  assumes  $B \in H(\text{nat} \setminus \{0\})$ 
  shows

```

$\text{id}(X) \subseteq B \text{ } B = \text{converse}(B)$
 $\exists B_1 \in H(\text{nat} \setminus \{0\}). B_1 \subseteq \text{converse}(B)$
 $\exists B_2 \in H(\text{nat} \setminus \{0\}). B_2 \cap B_2 \subseteq B$
<proof>

The halving sequence image of positive naturals is contained in the powerset of $X \times X$ and is not empty.

lemma `halving_seq_base_5_and_6th_cond`:
assumes Φ {is a uniformity on} X $U \in \Phi$ h {is a halving function for}
 Φ
defines $H \equiv \text{InductiveSequence}(U, h)$
shows $H(\text{nat} \setminus \{0\}) \subseteq \text{Pow}(X \times X)$ and $H(\text{nat} \setminus \{0\}) \neq \emptyset$
<proof>

If Φ is a uniformity, h is a halving function for it and $U \in \Phi$ then the image of the halving sequence defined inductively as $H_0 = U, H_{n+1} = h(H(n))$ on the positive naturals is a fundamental system of entourages.

theorem `halving_seq_base`:
assumes Φ {is a uniformity on} X $U \in \Phi$ h {is a halving function for}
 Φ
defines $H \equiv \text{InductiveSequence}(U, h)$
shows $(H(\text{nat} \setminus \{0\}))$ {is a uniform base on} X
<proof>

Theorem `halving_seq_base` shows that given a halving function h for a uniformity Φ each entourage $U \in \Phi$ defines a uniformity that has a countable base consisting of symmetric entourages and (as we show later) contains that entourage. The next definition summarizes the construction of this coarser uniformity.

definition
 $\text{CountBaseUnif}(X, h, U) \equiv \text{Supersets}(X \times X, \text{InductiveSequence}(U, h)(\text{nat} \setminus \{0\}))$

If Φ is a uniformity, h is a halving function for it and $U \in \Phi$ then $\text{CountBaseUnif}(X, h, U)$ is a uniformity on X that contains U (as a member) and is contained in Φ .

lemma `unif_count_base_unif`:
assumes $X \neq \emptyset$ Φ {is a uniformity on} X $U \in \Phi$ h {is a halving function for} Φ
shows $\text{CountBaseUnif}(X, h, U)$ {is a uniformity on} X and
 $U \in \text{CountBaseUnif}(X, h, U) \text{ } \text{CountBaseUnif}(X, h, U) \subseteq \Phi$
<proof>

If Φ is a uniformity on a nonempty set X and admits a halving function h then Φ is the supremum of the collection of uniformities $\{\Psi_U : U \in \Phi\}$ in the inclusion order relation, where $\Psi_U = \text{CountBaseUnif}(X, h, U)$. Since Ψ_U has a enumerable base this shows that every uniformity (that admits a halving function) is a union and a supremum of some collection of uniformities each of which has an enumerable base. We are using the word "enumerable" in

the sense defined in `Countable vs enumerable` section of the `Cardinal_ZF`, note in ZF this is not the same as "countable".

theorem `sup_count_base_unifs`:

assumes $X \neq \emptyset$ Φ {is a uniformity on} X h {is a halving function for} Φ

shows

$\Phi = \bigcup \{\text{CountBaseUnif}(X, h, U) \mid U \in \Phi\}$

$\Phi = \text{Supremum}(\text{OrderOnUniformities}(X), \{\text{CountBaseUnif}(X, h, U) \mid U \in \Phi\})$

<proof>

The next goal we want to achieve is to show that if a uniformity has an enumerable base consisting of symmetric entourages $\{V_n\}_{n \in \mathbb{N}}$ then it can be represented by a pseudometric. In the Bourbaki's proof the first step is to inductively define a sequence $\{V_n\}_{n \in \mathbb{N}}$ of entourages with the property that $U_1 \subseteq V_1$ and for all natural $n \geq 1$ we have $U_{n+1} \circ U_{n+1} \circ U_{n+1} \subseteq U_n \cap V_n$. The existence of such sequence for every uniformity with countable base follows from theorem `ustex3sym` in `UniformSpace_ZF` theory and the Axiom of Choice. Here we construct this sequence from an analogue of the halving function but with the property the $t(U) \circ t(u) \circ t(U) \subseteq U$ using the notion of inductive sequence with changing generation function. The `IsDiv3Function`(Φ, t) predicate defines an analogue of the halving function but with the property the $t(U) \circ t(u) \circ t(U) \subseteq U$ for all $U \in \Phi$.

definition `IsDiv3Function`(Φ, t) $\equiv t: \Phi \rightarrow \Phi \wedge$

$(\forall U \in \Phi. t(U) = \text{converse}(t(U)) \wedge t(U) \circ (t(U) \circ t(U)) \subseteq U)$

Given a function t that maps a uniformity Φ into itself and a sequence of entourages $\mathcal{V} : \mathbb{N} \rightarrow \Phi$ we define a sequence of functions called `SeqInter` such that for every $n \in \mathbb{N}$ the `SeqInter`(n) maps Φ into itself and `SeqInter`(n)(U) = $t(U \cap \mathcal{V}_n)$. In actual theorems we will assume that the function t satisfies the `IsDiv3Function` predicate and the `SeqInter` will be a generating sequence of functions in an inductive definition.

definition

`SeqInter`($\Phi, \mathcal{V}, t$) $\equiv \{ \langle n, \{U, t(U \cap \mathcal{V}(n))\} \rangle \mid U \in \Phi \}. n \in \text{nat} \}$

Suppose Φ is a uniformity on X , t maps Φ into itself and $\mathcal{V} : \mathbb{N} \rightarrow \Phi$ is a sequence of entourages of Φ . Then `SeqInter`($\Phi, \mathcal{V}, t$) is a sequence of functions mapping Φ into itself.

lemma `seqinter_is_var_seq`:

assumes Φ {is a uniformity on} X $\mathcal{V} : \text{nat} \rightarrow \Phi$ $t : \Phi \rightarrow \Phi$

shows `SeqInter`($\Phi, \mathcal{V}, t$) : $\text{nat} \rightarrow (\Phi \rightarrow \Phi)$

<proof>

The value of the n 'th element of `SeqInter`($\Phi, \mathcal{V}, t$) at $U \in \Phi$ is equal to $t(U \cap \mathcal{V}_n)$.

lemma `seqinter_val`: **assumes** $n \in \text{nat}$ $U \in \Phi$

shows $(\text{SeqInter}(\Phi, \mathcal{V}, t)(n))(U) = t(U \cap \mathcal{V}(n))$
<proof>

Suppose Φ is a uniformity on X , a function t satisfies the $\text{IsDiv3Function}(\Phi, t)$ predicate and $\mathcal{V} : \mathbb{N} \rightarrow \Phi$ is a sequence of entourages of Φ . Then the sequence $\mathcal{U}$ starting at $X \times X$ and generated inductively by the sequence of functions $\text{SeqInter}(\Phi, \mathcal{V}, t)$ is a sequence of symmetric elements (entourages) of Φ such that $(\mathcal{U}_{n+1})^3 \subseteq U_n \cap V_n$ for all $n \in \mathbb{N}$.

lemma `seq_div_cube`:

assumes Φ {is a uniformity on} X $\text{IsDiv3Function}(\Phi, t)$ $\mathcal{V} : \text{nat} \rightarrow \Phi$

defines $\mathcal{U} \equiv \text{InductiveSeqVarF}(X \times X, \Phi, \text{SeqInter}(\Phi, \mathcal{V}, t))$

shows $\mathcal{U} : \text{nat} \rightarrow \Phi$ $\mathcal{U}(0) = X \times X$ and $\forall n \in \text{nat}.$

$\mathcal{U}(n) = \text{converse}(\mathcal{U}(n)) \wedge \mathcal{U}(n \# + 1) \circ \mathcal{U}(n \# + 1) \circ \mathcal{U}(n \# + 1) \subseteq \mathcal{U}(n) \cap \mathcal{V}(n)$

<proof>

To shorten the notation we will call the sequence denoted as $\mathcal{U}$ in lemma `seq_div_third_pow` `Div3Seq` in the formal part and the "thirding sequence" in the informal comments. These are not good names, but I can't think about anything better. Anyway, all we care about is that its existence follows from the existence of $t : \Phi \rightarrow \Phi$ such that $t(U)$ is symmetric and $t(U) \circ t(U) \circ t(U) \subseteq U$ for all $U \in \Phi$ and it has the properties listed in `seq_div_third_pow`.

definition $\text{Div3Seq}(X, \Phi, \mathcal{V}, t) \equiv \text{InductiveSeqVarF}(X \times X, \Phi, \text{SeqInter}(\Phi, \mathcal{V}, t))$

A couple of other properties of the thirding sequence in addition to what is shown in `seq_div_cube`: Under the same assumptions and additionally that n is a natural number and denoting the thirding sequence as T the composition cube of the $(n + 1)$ 'th element of the thirding sequence and the $(n + 1)$ 'th element itself are contained in the n 'th element, and also the $(n + 1)$ 'th element is contained in $\mathcal{V}_n$.

lemma `div3_seq_props`:

assumes Φ {is a uniformity on} X $\text{IsDiv3Function}(\Phi, t)$ $\mathcal{V} : \text{nat} \rightarrow \Phi$ $n \in \text{nat}$

defines $T \equiv \text{Div3Seq}(X, \Phi, \mathcal{V}, t)$

shows $T(n \# + 1) \subseteq T(n)$ $T(n \# + 1) \subseteq \mathcal{V}(n)$ and

$T(n \# + 1) \circ T(n \# + 1) \circ T(n \# + 1) \subseteq T(n)$

<proof>

If Φ is a uniformity on X , t maps Φ into itself so that $t(U)$ is symmetric and $t(U) \circ t(U) \circ t(U) \subseteq U$ for all $U \in \Phi$, and $(V) : \mathbb{N} \rightarrow \Phi$ is a sequence of entourages whose image is a fundamental system of entourages of Φ , then the image of the thirding sequence $T : \mathbb{N} \rightarrow \Phi$ defined by t and $\mathcal{V}$ is a fundamental system of entourages (a uniform base) of Φ .

theorem `div3_seq_base_of`:

assumes Φ {is a uniformity on} X $\text{IsDiv3Function}(\Phi, t)$ and

$\mathcal{V} : \text{nat} \rightarrow \Phi$ $(\mathcal{V}(\text{nat}))$ {is a uniform base of} Φ

defines $T \equiv \text{Div3Seq}(X, \Phi, \mathcal{V}, t)$

shows $(T(\text{nat}))$ {is a uniform base of} Φ
<proof>

The rest of this section aims at showing that With the assumptions of `seq_div_cube` (i.e. when $\mathcal{V}$ is any sequence of entourages of Φ) the image of the thirding sequence is a fundamental system of entourages. This is a mildly interesting fact proven by a mistake that is unlikely to be ever used, but will stay here as a curiosity. We start with the claim that the thirding sequence is decreasing in the inclusion order on Φ hence the inclusion order on Φ is total on the sequence's image of the natural numbers.

lemma `div3_seq_decr`:
assumes Φ {is a uniformity on} X `IsDiv3Function`(Φ, t) $\mathcal{V}:\text{nat} \rightarrow \Phi$
shows
 $\text{IsDecreasingSeq}(\Phi, \text{InclusionOn}(\Phi), \text{Div3Seq}(X, \Phi, \mathcal{V}, t))$
 $\text{InclusionOn}(\Phi)$ {is total on} $(\text{Div3Seq}(X, \Phi, \mathcal{V}, t)(\text{nat}))$
<proof>

The next lemma shows the first condition we need: for two sets in a thirding sequence image there is a third one that is contained in both.

lemma `div3_seq_base_1st_cond`:
assumes Φ {is a uniformity on} X `IsDiv3Function`(Φ, t) $\mathcal{V}:\text{nat} \rightarrow \Phi$
defines $T \equiv \text{Div3Seq}(X, \Phi, \mathcal{V}, t)$
assumes $B_1 \in T(\text{nat})$ $B_2 \in T(\text{nat})$
shows $\exists B \in T(\text{nat}). B \subseteq B_1 \cap B_2$
<proof>

Sets in the thirding sequence image of natural numbers contain the diagonal, are symmetric and for every such set B there is another one contained in the converse of the first one and another one B_2 such that $B_2 \circ B_2 \subseteq B_1$.

lemma `div3_seq_base_2_3_4_conds`:
assumes Φ {is a uniformity on} X `IsDiv3Function`(Φ, t) $\mathcal{V}:\text{nat} \rightarrow \Phi$
defines $T \equiv \text{Div3Seq}(X, \Phi, \mathcal{V}, t)$
assumes $B \in T(\text{nat})$
shows
 $\text{id}(X) \subseteq B$ $B = \text{converse}(B)$
 $\exists B_1 \in T(\text{nat}). B_1 \subseteq \text{converse}(B)$
 $\exists B_2 \in T(\text{nat}). B_2 \circ B_2 \subseteq B$
<proof>

The thirding sequence image of natural numbers is contained in the powerset of $X \times X$ and is not empty.

lemma `div3_seq_base_5_and_6th_cond`:
assumes Φ {is a uniformity on} X `IsDiv3Function`(Φ, t) $\mathcal{V}:\text{nat} \rightarrow \Phi$
defines $T \equiv \text{Div3Seq}(X, \Phi, \mathcal{V}, t)$
shows $T(\text{nat}) \subseteq \text{Pow}(X \times X)$ and $T(\text{nat}) \neq \emptyset$
<proof>

If Φ is a uniformity on X , t maps Φ into itself so that $t(U)$ is symmetric and $t(U) \circ t(U) \circ T(U) \subseteq U$ for all $U \in \Phi$, and $(V) : \mathbb{N} \rightarrow \Phi$ is any sequence of entourages from Φ then the image of the sequence defined inductively by $T_0 = X \times X$, $T_{n+1} = t(T_n \cap V_n)$ is a fundamental system of entourages.

```

theorem div3_seq_base:
  assumes  $\Phi$  {is a uniformity on}  $X$  IsDiv3Function( $\Phi, t$ )  $V : \text{nat} \rightarrow \Phi$ 
  defines  $T \equiv \text{Div3Seq}(X, \Phi, V, t)$ 
  shows  $(T(\text{nat}))$  {is a uniform base on}  $X$ 
   $\langle \text{proof} \rangle$ 
end

```

104 Topological groups - introduction

```

theory TopologicalGroup_ZF imports Topology_ZF_3 Group_ZF_1 Semigroup_ZF

```

```

begin

```

This theory is about the first subject of algebraic topology: topological groups.

104.1 Topological group: definition and notation

Topological group is a group that is a topological space at the same time. This means that a topological group is a triple of sets, say (G, f, T) such that T is a topology on G , f is a group operation on G and both f and the operation of taking inverse in G are continuous. Since IsarMathLib defines topology without using the carrier, (see `Topology_ZF`), in our setup we just use $\bigcup T$ instead of G and say that the pair of sets $(\bigcup T, f)$ is a group. This way our definition of being a topological group is a statement about two sets: the topology T and the group operation f on $G = \bigcup T$. Since the domain of the group operation is $G \times G$, the pair of topologies in which f is supposed to be continuous is T and the product topology on $G \times G$ (which we will call τ below).

This way we arrive at the following definition of a predicate that states that pair of sets is a topological group.

definition

```

  IsAtopologicalGroup( $T, f$ )  $\equiv$  ( $T$  {is a topology})  $\wedge$  IsAgroup( $\bigcup T, f$ )  $\wedge$ 
  IsContinuous(ProductTopology( $T, T$ ),  $T, f$ )  $\wedge$ 
  IsContinuous( $T, T$ , GroupInv( $\bigcup T, f$ ))

```

We will inherit notation from the `topology0` locale. That locale assumes that T is a topology. For convenience we will denote $G = \bigcup T$ and τ to be the product topology on $G \times G$. To that we add some notation specific to groups. We will use additive notation for the group operation, even though

we don't assume that the group is abelian. The notation $g + A$ will mean the left translation of the set A by element g , i.e. $g + A = \{g + a | a \in A\}$. The group operation G induces a natural operation on the subsets of G defined as $\langle A, B \rangle \mapsto \{x + y | x \in A, y \in B\}$. Such operation has been considered in `func_ZF` and called f "lifted to subsets of" G . We will denote the value of such operation on sets A, B as $A + B$. The set of neighborhoods of zero (denoted $\mathcal{N}_0$) is the collection of (not necessarily open) sets whose interior contains the neutral element of the group.

```
locale topgroup = topology0 +
```

```

  fixes G
  defines G_def [simp]: G  $\equiv$   $\bigcup$  T

  fixes prodtop ( $\tau$ )
  defines prodtop_def [simp]:  $\tau \equiv$  ProductTopology(T,T)

  fixes f

  assumes Ggroup: IsAgroup(G,f)

  assumes fcon: IsContinuous( $\tau$ ,T,f)

  assumes inv_cont: IsContinuous(T,T,GroupInv(G,f))

  fixes grop (infixl + 90)
  defines grop_def [simp]:  $x+y \equiv f\langle x,y \rangle$ 

  fixes grinv (- _ 89)
  defines grinv_def [simp]:  $(-x) \equiv$  GroupInv(G,f)(x)

  fixes grsub (infixl - 90)
  defines grsub_def [simp]:  $x-y \equiv x+(-y)$ 

  fixes setinv (- _ 72)
  defines setninv_def [simp]:  $-A \equiv$  GroupInv(G,f)(A)

  fixes ltrans (infix + 73)
  defines ltrans_def [simp]:  $x + A \equiv$  LeftTranslation(G,f,x)(A)

  fixes rtrans (infix + 73)
  defines rtrans_def [simp]:  $A + x \equiv$  RightTranslation(G,f,x)(A)

  fixes setadd (infixl + 71)
  defines setadd_def [simp]:  $A+B \equiv (f \text{ {lifted to subsets of} } G)\langle A,B \rangle$ 

  fixes gzero (0)
  defines gzero_def [simp]:  $0 \equiv$  TheNeutralElement(G,f)

```

```

fixes zerohoods ( $\mathcal{N}_0$ )
defines zerohoods_def [simp]:  $\mathcal{N}_0 \equiv \{A \in \text{Pow}(G). \mathbf{0} \in \text{int}(A)\}$ 

fixes listsum ( $\sum \_ 70$ )
defines listsum_def[simp]:  $\sum s \equiv \text{Fold}(f, \mathbf{0}, s)$ 

fixes nat_mult (infix  $\cdot$  95)
defines nat_mult_def [simp]:  $n \cdot x \equiv \sum \{\langle k, x \rangle. k \in n\}$ 

```

The first lemma states that we indeed talk about topological group in the context of `topgroup` locale.

```

lemma (in topgroup) topGroup: shows IsAtopologicalGroup( $T, f$ )
  <proof>

```

If a pair of sets (T, f) forms a topological group, then all theorems proven in the `topgroup` context are valid as applied to (T, f) .

```

lemma topGroupLocale: assumes IsAtopologicalGroup( $T, f$ )
  shows topgroup( $T, f$ )
  <proof>

```

We can use the `group0` locale in the context of `topgroup`.

```

lemma (in topgroup) group0_valid_in_tgroup: shows group0( $G, f$ )
  <proof>

```

We can use the `group0` locale in the context of `topgroup`.

```

sublocale topgroup < group0  $G$   $f$  gzero grop grinv listsum nat_mult
  <proof>

```

We can use `semigr0` locale in the context of `topgroup`.

```

lemma (in topgroup) semigr0_valid_in_tgroup: shows semigr0( $G, f$ )
  <proof>

```

We can use the `prod_top_spaces0` locale in the context of `topgroup`.

```

lemma (in topgroup) prod_top_spaces0_valid: shows prod_top_spaces0( $T, T, T$ )
  <proof>

```

Negative of a group element is in group.

```

lemma (in topgroup) neg_in_tgroup: assumes  $g \in G$  shows  $(-g) \in G$ 
  <proof>

```

Sum of two group elements is in the group.

```

lemma (in topgroup) group_op_closed_add: assumes  $x_1 \in G$   $x_2 \in G$ 
  shows  $x_1 + x_2 \in G$ 
  <proof>

```

Zero is in the group.

```

lemma (in topgroup) zero_in_tgroup: shows  $\mathbf{0} \in G$ 

```


⟨proof⟩

Another lemma about canceling with two group elements written in additive notation

```
lemma (in topgroup) inv_cancel_two_add:
  assumes  $x_1 \in G$   $x_2 \in G$ 
  shows
     $x_1 + (-x_2) + x_2 = x_1$ 
     $x_1 + x_2 + (-x_2) = x_1$ 
     $(-x_1) + (x_1 + x_2) = x_2$ 
     $x_1 + ((-x_1) + x_2) = x_2$ 
⟨proof⟩
```

Useful identities proven in the Group_ZF theory, rewritten here in additive notation. Note since the group operation notation is left associative we don't really need the first set of parentheses in some cases.

```
lemma (in topgroup) cancel_middle_add: assumes  $x_1 \in G$   $x_2 \in G$   $x_3 \in G$ 
  shows
     $(x_1 + (-x_2)) + (x_2 + (-x_3)) = x_1 + (-x_3)$ 
     $((-x_1) + x_2) + ((-x_2) + x_3) = (-x_1) + x_3$ 
     $(- (x_1 + x_2)) + (x_1 + x_3) = (-x_2) + x_3$ 
     $(x_1 + x_2) + (- (x_3 + x_2)) = x_1 + (-x_3)$ 
     $(-x_1) + (x_1 + x_2 + x_3) + (-x_3) = x_2$ 
⟨proof⟩
```

We can cancel an element on the right from both sides of an equation.

```
lemma (in topgroup) cancel_right_add:
  assumes  $x_1 \in G$   $x_2 \in G$   $x_3 \in G$   $x_1 + x_2 = x_3 + x_2$ 
  shows  $x_1 = x_3$ 
⟨proof⟩
```

We can cancel an element on the left from both sides of an equation.

```
lemma (in topgroup) cancel_left_add:
  assumes  $x_1 \in G$   $x_2 \in G$   $x_3 \in G$   $x_1 + x_2 = x_1 + x_3$ 
  shows  $x_2 = x_3$ 
⟨proof⟩
```

We can put an element on the other side of an equation.

```
lemma (in topgroup) put_on_the_other_side:
  assumes  $x_1 \in G$   $x_2 \in G$   $x_3 = x_1 + x_2$ 
  shows  $x_3 + (-x_2) = x_1$  and  $(-x_1) + x_3 = x_2$ 
⟨proof⟩
```

A simple equation from lemma simple_equation0 in Group_ZF in additive notation

```
lemma (in topgroup) simple_equation0_add:
```

```

assumes  $x_1 \in G \quad x_2 \in G \quad x_3 \in G \quad x_1 + (-x_2) = (-x_3)$ 
shows  $x_3 = x_2 + (-x_1)$ 
 $\langle proof \rangle$ 

```

A simple equation from lemma `simple_equation1` in `Group_ZF` in additive notation

```

lemma (in topgroup) simple_equation1_add:
  assumes  $x_1 \in G \quad x_2 \in G \quad x_3 \in G \quad (-x_1) + x_2 = (-x_3)$ 
  shows  $x_3 = (-x_2) + x_1$ 
   $\langle proof \rangle$ 

```

The set comprehension form of negative of a set. The proof uses the `ginv_image` lemma from `Group_ZF` theory which states the same thing in multiplicative notation.

```

lemma (in topgroup) ginv_image_add: assumes  $V \subseteq G$ 
  shows  $(-V) \subseteq G$  and  $(-V) = \{-x. x \in V\}$ 
   $\langle proof \rangle$ 

```

The additive notation version of `ginv_image_el` lemma from `Group_ZF` theory

```

lemma (in topgroup) ginv_image_el_add: assumes  $V \subseteq G \quad x \in (-V)$ 
  shows  $(-x) \in V$ 
   $\langle proof \rangle$ 

```

Of course the product topology is a topology (on $G \times G$).

```

lemma (in topgroup) prod_top_on_G:
  shows  $\tau$  {is a topology} and  $\bigcup \tau = G \times G$ 
   $\langle proof \rangle$ 

```

Let's recall that f is a binary operation on G in this context.

```

lemma (in topgroup) topgroup_f_binop: shows  $f : G \times G \rightarrow G$ 
   $\langle proof \rangle$ 

```

A subgroup of a topological group is a topological group with relative topology and restricted operation. Relative topology is the same as `T {restricted to} H` which is defined to be $\{V \cap H : V \in T\}$ in `ZF1` theory.

```

lemma (in topgroup) top_subgroup: assumes  $A1: \text{IsAsubgroup}(H, f)$ 
  shows  $\text{IsAtopologicalGroup}(T \text{ {restricted to} } H, \text{restrict}(f, H \times H))$ 
   $\langle proof \rangle$ 

```

104.2 Interval arithmetic, translations and inverse of set

In this section we list some properties of operations of translating a set and reflecting it around the neutral element of the group. Many of the results are proven in other theories, here we just collect them and rewrite in notation specific to the `topgroup` context.

Different ways of looking at adding sets.

```

lemma (in topgroup) interval_add: assumes  $A \subseteq G$   $B \subseteq G$  shows
   $A+B \subseteq G$ 
   $A+B = f(A \times B)$ 
   $A+B = (\bigcup x \in A. x+B)$ 
   $A+B = \{x+y. \langle x,y \rangle \in A \times B\}$ 
  <proof>

```

If the neutral element is in a set, then it is in the sum of the sets.

```

lemma (in topgroup) interval_add_zero: assumes  $A \subseteq G$   $0 \in A$ 
shows  $0 \in A+A$ 
<proof>

```

Some lemmas from Group_ZF_1 about images of set by translations written in additive notation

```

lemma (in topgroup) lrtrans_image: assumes  $V \subseteq G$   $x \in G$ 
shows
   $x+V = \{x+v. v \in V\}$ 
   $V+x = \{v+x. v \in V\}$ 
  <proof>

```

Right and left translations of a set are subsets of the group. This is of course typically applied to the subsets of the group, but formally we don't need to assume that.

```

lemma (in topgroup) lrtrans_in_group_add: assumes  $x \in G$ 
shows  $x+V \subseteq G$  and  $V+x \subseteq G$ 
<proof>

```

A corollary from interval_add

```

corollary (in topgroup) elements_in_set_sum: assumes  $A \subseteq G$   $B \subseteq G$ 
   $t \in A+B$  shows  $\exists s \in A. \exists q \in B. t=s+q$ 
<proof>

```

A corollary from lrtrans_image

```

corollary (in topgroup) elements_in_ltrans:
assumes  $B \subseteq G$   $g \in G$   $t \in g+B$ 
shows  $\exists q \in B. t=g+q$ 
<proof>

```

Another corollary of lrtrans_image

```

corollary (in topgroup) elements_in_rtrans:
assumes  $B \subseteq G$   $g \in G$   $t \in B+g$  shows  $\exists q \in B. t=q+g$ 
<proof>

```

Another corollary from interval_add

```

corollary (in topgroup) elements_in_set_sum_inv:
assumes  $A \subseteq G$   $B \subseteq G$   $t=s+q$   $s \in A$   $q \in B$ 
shows  $t \in A+B$ 

```

⟨proof⟩

Another corollary of `lrtrans_image`

corollary (in topgroup) `elements_in_ltrans_inv`: assumes $B \subseteq G$ $g \in G$ $q \in B$ $t = g + q$
shows $t \in g + B$
⟨proof⟩

Another corollary of `rtrans_image_add`

lemma (in topgroup) `elements_in_rtrans_inv`:
assumes $B \subseteq G$ $g \in G$ $q \in B$ $t = q + g$
shows $t \in B + g$
⟨proof⟩

Right and left translations are continuous.

lemma (in topgroup) `trans_cont`: assumes $g \in G$ shows
`IsContinuous`($T, T, \text{RightTranslation}(G, f, g)$) and
`IsContinuous`($T, T, \text{LeftTranslation}(G, f, g)$)
⟨proof⟩

Left and right translations of an open set are open.

lemma (in topgroup) `open_tr_open`: assumes $g \in G$ and $V \in T$
shows $g + V \in T$ and $V + g \in T$
⟨proof⟩

Right and left translations are homeomorphisms.

lemma (in topgroup) `tr_homeo`: assumes $g \in G$ shows
`IsAhomeomorphism`($T, T, \text{RightTranslation}(G, f, g)$) and
`IsAhomeomorphism`($T, T, \text{LeftTranslation}(G, f, g)$)
⟨proof⟩

Left translations preserve interior.

lemma (in topgroup) `ltrans_interior`: assumes $A1$: $g \in G$ and $A2$: $A \subseteq G$
shows $g + \text{int}(A) = \text{int}(g + A)$
⟨proof⟩

Right translations preserve interior.

lemma (in topgroup) `rtrans_interior`: assumes $A1$: $g \in G$ and $A2$: $A \subseteq G$
shows $\text{int}(A) + g = \text{int}(A + g)$
⟨proof⟩

Translating by an inverse and then by an element cancels out.

lemma (in topgroup) `trans_inverse_elem`: assumes $g \in G$ and $A \subseteq G$
shows $g + ((-g) + A) = A$
⟨proof⟩

Inverse of an open set is open.

lemma (in topgroup) `open_inv_open`: assumes $V \in T$ shows $(-V) \in T$

<proof>

Inverse is a homeomorphism.

lemma (in topgroup) inv_homeo: shows IsAhomeomorphism(T,T,GroupInv(G,f))
<proof>

Taking negative preserves interior.

lemma (in topgroup) int_inv_inv_int: assumes $A \subseteq G$
 shows $\text{int}(-A) = -(\text{int}(A))$
<proof>

104.3 Neighborhoods of zero

Zero neighborhoods are (not necessarily open) sets whose interior contains the neutral element of the group. In the topgroup locale the collection of neighborhoods of zero is denoted $\mathcal{N}_0$.

The whole space is a neighborhood of zero.

lemma (in topgroup) zneigh_not_empty: shows $G \in \mathcal{N}_0$
<proof>

Any element that belongs to a subset of the group belongs to that subset with the interior of a neighborhood of zero added.

lemma (in topgroup) elem_in_int_sad: assumes $A \subseteq G$ $g \in A$ $H \in \mathcal{N}_0$
 shows $g \in A + \text{int}(H)$
<proof>

Any element belongs to the interior of any neighborhood of zero left translated by that element.

lemma (in topgroup) elem_in_int_ltrans:
 assumes $g \in G$ and $H \in \mathcal{N}_0$
 shows $g \in \text{int}(g+H)$ and $g \in \text{int}(g+H) + \text{int}(H)$
<proof>

Any element belongs to the interior of any neighborhood of zero right translated by that element.

lemma (in topgroup) elem_in_int_rtrans:
 assumes A1: $g \in G$ and A2: $H \in \mathcal{N}_0$
 shows $g \in \text{int}(H+g)$ and $g \in \text{int}(H+g) + \text{int}(H)$
<proof>

Negative of a neighborhood of zero is a neighborhood of zero.

lemma (in topgroup) neg_neigh_neigh: assumes $H \in \mathcal{N}_0$
 shows $(-H) \in \mathcal{N}_0$
<proof>

Left translating an open set by a negative of a point that belongs to it makes it a neighborhood of zero.

lemma (in topgroup) open_trans_neigh: assumes A1: $U \in \mathcal{T}$ and $g \in U$
 shows $(-g) + U \in \mathcal{N}_0$
<proof>

Right translating an open set by a negative of a point that belongs to it makes it a neighborhood of zero.

lemma (in topgroup) open_trans_neigh_2: assumes A1: $U \in \mathcal{T}$ and $g \in U$
 shows $U + (-g) \in \mathcal{N}_0$
<proof>

Right and left translating an neighborhood of zero by a point and its negative makes it back a neighborhood of zero.

lemma (in topgroup) lrtrans_neigh: assumes $W \in \mathcal{N}_0$ and $x \in G$
 shows $x + (W + (-x)) \in \mathcal{N}_0$ and $(x + W) + (-x) \in \mathcal{N}_0$
<proof>

If A is a subset of B translated by $-x$ then its translation by x is a subset of B .

lemma (in topgroup) trans_subset:
 assumes $A \subseteq ((-x) + B)$ $x \in G$ $B \subseteq G$
 shows $x + A \subseteq B$
<proof>

Every neighborhood of zero has a symmetric subset that is a neighborhood of zero.

theorem (in topgroup) exists_sym_zerohood:
 assumes $U \in \mathcal{N}_0$
 shows $\exists V \in \mathcal{N}_0. (V \subseteq U \wedge (-V) = V)$
<proof>

We can say even more than in exists_sym_zerohood: every neighborhood of zero U has a symmetric subset that is a neighborhood of zero and its set double is contained in U .

theorem (in topgroup) exists_procls_zerohood:
 assumes $U \in \mathcal{N}_0$
 shows $\exists V \in \mathcal{N}_0. (V \subseteq U \wedge (V + V) \subseteq U \wedge (-V) = V)$
<proof>

104.4 Closure in topological groups

This section is devoted to a characterization of closure in topological groups.

Closure of a set is contained in the sum of the set and any neighborhood of zero.

lemma (in topgroup) cl_contains_zneigh:
 assumes A1: $A \subseteq G$ and A2: $H \in \mathcal{N}_0$
 shows $\text{cl}(A) \subseteq A + H$

<proof>

The next theorem provides a characterization of closure in topological groups in terms of neighborhoods of zero.

theorem (in topgroup) cl_topgroup:
 assumes $A \subseteq G$ shows $\text{cl}(A) = (\bigcap_{H \in \mathcal{N}_0} A + H)$
<proof>

104.5 Sums of sequences of elements and subsets

In this section we consider properties of the function $G^n \rightarrow G, x = (x_0, x_1, \dots, x_{n-1}) \mapsto \sum_{i=0}^{n-1} x_i$. We will model the cartesian product G^n by the space of sequences $n \rightarrow G$, where $n = \{0, 1, \dots, n-1\}$ is a natural number. This space is equipped with a natural product topology defined in `Topology_ZF_3`.

Let's recall first that the sum of elements of a group is an element of the group.

lemma (in topgroup) sum_list_in_group:
 assumes $n \in \text{nat}$ and $x: \text{succ}(n) \rightarrow G$
 shows $(\sum x) \in G$
<proof>

In this context $x+y$ is the same as the value of the group operation on the elements x and y . Normally we shouldn't need to state this as a separate lemma.

lemma (in topgroup) grop_def1: shows $f\langle x, y \rangle = x+y$ *<proof>*

Another theorem from `Semigroup_ZF` theory that is useful to have in the additive notation.

lemma (in topgroup) shorter_set_add:
 assumes $n \in \text{nat}$ and $x: \text{succ}(\text{succ}(n)) \rightarrow G$
 shows $(\sum x) = (\sum \text{Init}(x)) + (x(\text{succ}(n)))$
<proof>

Sum is a continuous function in the product topology.

theorem (in topgroup) sum_continuous: assumes $n \in \text{nat}$
 shows $\text{IsContinuous}(\text{SeqProductTopology}(\text{succ}(n), T), T, \{\langle x, \sum x \rangle . x \in \text{succ}(n) \rightarrow G\})$
<proof>
end

105 Topological groups 1

theory TopologicalGroup_ZF_1 **imports** TopologicalGroup_ZF Topology_ZF_properties_2
begin

This theory deals with some topological properties of topological groups.

105.1 Separation properties of topological groups

The topological groups have very specific properties. For instance, G is T_0 iff it is T_3 .

```
theorem(in topgroup) cl_point:
  assumes  $x \in G$ 
  shows  $\text{cl}(\{x\}) = (\bigcap H \in \mathcal{N}_0. x \in H)$ 
<proof>
```

We prove the equivalence between T_0 and T_1 first.

```
theorem (in topgroup) neu_closed_imp_T1:
  assumes  $\{0\}$  is closed in  $T$ 
  shows  $T$  is  $T_1$ 
<proof>
```

```
theorem (in topgroup) T0_imp_neu_closed:
  assumes  $T$  is  $T_0$ 
  shows  $\{0\}$  is closed in  $T$ 
<proof>
```

105.2 Existence of nice neighbourhoods.

```
lemma (in topgroup) exist_basehoods_closed:
  assumes  $U \in \mathcal{N}_0$ 
  shows  $\exists V \in \mathcal{N}_0. \text{cl}(V) \subseteq U$ 
<proof>
```

105.3 Rest of separation axioms

```
theorem(in topgroup) T1_imp_T2:
  assumes  $T$  is  $T_1$ 
  shows  $T$  is  $T_2$ 
<proof>
```

Here follow some auxiliary lemmas.

```
lemma (in topgroup) trans_closure:
  assumes  $x \in G$   $A \subseteq G$ 
  shows  $\text{cl}(x+A) = x + \text{cl}(A)$ 
<proof>
```

```
lemma (in topgroup) trans_interior2: assumes  $A1: g \in G$  and  $A2: A \subseteq G$ 
  shows  $\text{int}(A)+g = \text{int}(A+g)$ 
<proof>
```

```
lemma (in topgroup) trans_closure2:
  assumes  $x \in G$   $A \subseteq G$ 
  shows  $\text{cl}(A+x) = \text{cl}(A) + x$ 
<proof>
```



```

lemma (in topgroup) trans_subset:
  assumes  $A \subseteq ((-x)+B)$   $x \in GA \subseteq GB \subseteq G$ 
  shows  $x+A \subseteq B$ 
<proof>

```

Every topological group is regular, and hence T_3 . The proof is in the next section, since it uses local properties.

105.4 Local properties

In a topological group, all local properties depend only on the neighbourhoods of the neutral element; when considering topological properties. The next result of regularity, will use this idea, since translations preserve closed sets.

```

lemma (in topgroup) local_iff_neutral:
  assumes  $\forall U \in \mathcal{T} \cap \mathcal{N}_0. \exists N \in \mathcal{N}_0. N \subseteq U \wedge P(N, T) \quad \forall N \in \text{Pow}(G). \forall x \in G. P(N, T) \longrightarrow$ 
 $P(x+N, T)$ 
  shows  $T\{\text{is locally}\}P$ 
<proof>

```

```

lemma (in topgroup) trans_closed:
  assumes  $A\{\text{is closed in}\}Tx \in G$ 
  shows  $(x+A)\{\text{is closed in}\}T$ 
<proof>

```

As it is written in the previous section, every topological group is regular.

```

theorem (in topgroup) topgroup_reg:
  shows  $T\{\text{is regular}\}$ 
<proof>

```

The promised corollary follows:

```

corollary (in topgroup) T2_imp_T3:
  assumes  $T\{\text{is } T_2\}$ 
  shows  $T\{\text{is } T_3\}$  <proof>

```

end

106 Topological groups 2

```

theory TopologicalGroup_ZF_2 imports Topology_ZF_10 Topology_ZF_8 Group_ZF_2
TopologicalGroup_ZF_1
Group_ZF_4

```

begin

This theory deals with topological properties of subgroups, quotient groups and relations between group-theoretical properties and topological properties.

106.1 Subgroups topologies

The closure of a subgroup is a subgroup.

```
theorem (in topgroup) closure_subgroup:  
  assumes IsAsubgroup(H,f)  
  shows IsAsubgroup(cl(H),f)  
<proof>
```

The closure of a normal subgroup is normal.

```
theorem (in topgroup) normal_sub:  
  assumes IsAnormalSubgroup(G,f,H)  
  shows IsAnormalSubgroup(G,f,cl(H))  
<proof>
```

Every open subgroup is also closed.

```
theorem (in topgroup) open_subgroup_closed:  
  assumes IsAsubgroup(H,f) H∈T  
  shows H{is closed in}T  
<proof>
```

Any subgroup with non-empty interior is open.

```
theorem (in topgroup) clopen_or_emptyInt:  
  assumes IsAsubgroup(H,f) int(H)≠0  
  shows H∈T  
<proof>
```

In conclusion, a subgroup is either open or has empty interior.

```
corollary(in topgroup) emptyInterior_xor_op:  
  assumes IsAsubgroup(H,f)  
  shows (int(H)=0) Xor (H∈T)  
<proof>
```

Then no connected topological groups has proper subgroups with non-empty interior.

```
corollary(in topgroup) connected_emptyInterior:  
  assumes IsAsubgroup(H,f) T{is connected}  
  shows (int(H)=0) Xor (H=G)  
<proof>
```

Every locally-compact subgroup of a T_0 group is closed.

```
theorem (in topgroup) loc_compact_T0_closed:  
  assumes IsAsubgroup(H,f) (T{restricted to}H){is locally-compact} T{is  
T0}  
  shows H{is closed in}T  
<proof>
```

106.2 Quotient topological groups

The quotient topology given by the quotient group equivalent relation, has an open quotient map.

```
theorem(in topgroup) quotient_map_topgroup_open:
  assumes IsSubgroup(H,f) A∈T
  defines r ≡ QuotientGroupRel(G,f,H)
  shows {⟨b,r{b}⟩. b∈⋃T}A∈(T{quotient by}r)
  <proof>
```

A quotient of a topological group is just a quotient group with an appropriate topology that makes product and inverse continuous.

```
theorem (in topgroup) quotient_top_group_F_cont:
  assumes IsAnormalSubgroup(G,f,H)
  defines r ≡ QuotientGroupRel(G,f,H)
  defines F ≡ QuotientGroupOp(G,f,H)
  shows IsContinuous(ProductTopology(T{quotient by}r,T{quotient by}r),T{quotient
by}r,F)
  <proof>
```

```
lemma (in group0) Group_ZF_2_4_L8:
  assumes IsAnormalSubgroup(G,P,H)
  defines r ≡ QuotientGroupRel(G,P,H)
  and F ≡ QuotientGroupOp(G,P,H)
  shows GroupInv(G//r,F):G//r→G//r
  <proof>
```

```
theorem (in topgroup) quotient_top_group_INV_cont:
  assumes IsAnormalSubgroup(G,f,H)
  defines r ≡ QuotientGroupRel(G,f,H)
  defines F ≡ QuotientGroupOp(G,f,H)
  shows IsContinuous(T{quotient by}r,T{quotient by}r,GroupInv(G//r,F))
  <proof>
```

Finally we can prove that quotient groups of topological groups are topological groups.

```
theorem(in topgroup) quotient_top_group:
  assumes IsAnormalSubgroup(G,f,H)
  defines r ≡ QuotientGroupRel(G,f,H)
  defines F ≡ QuotientGroupOp(G,f,H)
  shows IsAtopologicalGroup({quotient by}r,F)
  <proof>
```

We can always consider a factor group which is T_2 .

```
theorem(in topgroup) factor_haus:
  shows (T{quotient by}QuotientGroupRel(G,f,cl({0})))is T2
  <proof>
```

end

107 Uniform structures on topological groups

theory TopologicalGroup_Uniformity_ZF **imports** TopologicalGroup_ZF UniformSpace_ZF

begin

Each topological group is a uniform space. This theory is about the uniformities that are naturally defined by a topological group structure.

107.1 Natural uniformities in topological groups: definitions and notation

There are two basic uniformities that can be defined on a topological group.

Definition of left uniformity

definition (in topgroup) leftUniformity
where leftUniformity $\equiv \{V \in \text{Pow}(G \times G). \exists U \in \mathcal{N}_0. \{\langle s, t \rangle \in G \times G. (-s) + t \in U\} \subseteq V\}$

Definition of right uniformity

definition (in topgroup) rightUniformity
where rightUniformity $\equiv \{V \in \text{Pow}(G \times G). \exists U \in \mathcal{N}_0. \{\langle s, t \rangle \in G \times G. s + (-t) \in U\} \subseteq V\}$

Right and left uniformities are indeed uniformities.

lemma (in topgroup) side_uniformities:
shows leftUniformity {is a uniformity on} G **and** rightUniformity {is a uniformity on} G
<proof>

The topologies generated by the right and left uniformities are the original group topology.

lemma (in topgroup) top_generated_side_uniformities:
shows UniformTopology(leftUniformity, G) = T **and** UniformTopology(rightUniformity, G) = T
<proof>

The side uniformities are called this way because of how they affect left and right translations. In the next lemma we show that left translations are uniformly continuous with respect to the left uniformity.

lemma (in topgroup) left_mult_uniformity: **assumes** $x \in G$
shows
LeftTranslation(G, f, x) {is uniformly continuous between} leftUniformity {and} leftUniformity
<proof>

Right translations are uniformly continuous with respect to the right uniformity.

```

lemma (in topgroup) right_mult_uniformity: assumes x∈G
  shows
    RightTranslation(G,f,x) {is uniformly continuous between} rightUniformity
    {and} rightUniformity
  <proof>

```

The third uniformity important on topological groups is called the uniformity of Roelcke.

```

definition(in topgroup) roelckeUniformity
  where roelckeUniformity ≡ {V∈Pow(G×G). ∃U∈  $\mathcal{N}_0$ . {<s,t>∈G×G. t ∈(U+s)+U}⊆
  V}

```

The Roelcke uniformity is indeed a uniformity on the group.

```

lemma (in topgroup) roelcke_uniformity:
  shows roelckeUniformity {is a uniformity on} G
  <proof>

```

The topology given by the roelcke uniformity is the original topology

```

lemma (in topgroup) top_generated_roelcke_uniformity:
  shows UniformTopology(roelckeUniformity,G) = T
  <proof>

```

The inverse map is uniformly continuous in the Roelcke uniformity

```

theorem (in topgroup) inv_uniform_roelcke:
  shows
    GroupInv(G,f) {is uniformly continuous between} roelckeUniformity
    {and} roelckeUniformity
  <proof>

```

end

108 Two versions of Tarski's Axiom

```

theory Tarski_ZF imports ZF.Cardinal

```

```

begin

```

A small theory on the Tarski's axiom, inspired by a question on mathoverflow.org.

108.1 Two versions of the Tarski's axiom

There are two versions of the Tarski's axiom, one adapted from Tarski 1939 according to Wikipedia's page on Tarski-Grothendieck set theory:

$\forall x. \exists y. x \in y \wedge (\forall z \in y. \mathcal{P}(z) \in \mathcal{P}(y) \wedge \mathcal{P}(z) \in y) \wedge (\forall z \in \mathcal{P}(y). z \approx y \vee z \in y))$,
and another one from Metamath, adapted from that page:

$\forall x. \exists y. x \in y \wedge (\forall z \in y. \mathcal{P}(z) \in \mathcal{P}(y) \wedge (\exists w \in y. \mathcal{P}(z) \in \mathcal{P}(w))) \wedge (\forall z. z \in \mathcal{P}(y) \longrightarrow z \approx y \vee z \in y).$

In this section we show that these two versions are equivalent.

The two versions of Tarski's axiom are equivalent.

theorem Tarski_axioms: shows

$(\forall x. \exists y. x \in y \wedge (\forall z \in y. \text{Pow}(z) \in \text{Pow}(y) \wedge \text{Pow}(z) \in y) \wedge (\forall z \in \text{Pow}(y). z \approx y \vee z \in y))$
 $\longleftrightarrow$
 $(\forall x. \exists y. x \in y \wedge (\forall z \in y. \text{Pow}(z) \in \text{Pow}(y) \wedge (\exists w \in y. \text{Pow}(z) \in \text{Pow}(w)))) \wedge (\forall z. z \in \text{Pow}(y) \longrightarrow z \approx y \vee z \in y))$
<proof>

end

109 Metamath introduction

theory MMI_prelude imports Order_ZF_1

begin

Metamath's set.mm features a large (over 8000) collection of theorems proven in the ZFC set theory. This theory is part of an attempt to translate those theorems to Isar so that they are available for Isabelle/ZF users. A total of about 1200 assertions have been translated, 600 of that with proofs (the rest was proven automatically by Isabelle). The translation was done with the support of the mmisar tool, whose source is included in the IsarMathLib distributions prior to version 1.6.4. The translation tool was doing about 99 percent of work involved, with the rest mostly related to the difference between Isabelle/ZF and Metamath metalogics. Metamath uses Tarski-Megill metalogic that does not have a notion of bound variables (see http://planetx.cc.vt.edu/AsteroidMeta/Distinctors_vs_binders for details and discussion). The translation project is closed now as I decided that it was too boring and tedious even with the support of mmisar software. Also, the translated proofs are not as readable as native Isar proofs which goes against IsarMathLib philosophy.

109.1 Importing from Metamath - how is it done

We are interested in importing the theorems about complex numbers that start from the "recnt" theorem on. This is done mostly automatically by the mmisar tool that is included in the IsarMathLib distributions prior to version 1.6.4. The tool works as follows:

First it reads the list of (Metamath) names of theorems that are already imported to IsarMathlib ("known theorems") and the list of theorems that

are intended to be imported in this session ("new theorems"). The new theorems are consecutive theorems about complex numbers as they appear in the Metamath database. Then `mmisar` creates a "Metamath script" that contains Metamath commands that open a log file and put the statements and proofs of the new theorems in that file in a readable format. The tool writes this script to a disk file and executes `metamath` with standard input redirected from that file. Then the log file is read and its contents converted to the Isar format. In Metamath, the proofs of theorems about complex numbers depend only on 28 axioms of complex numbers and some basic logic and set theory theorems. The tool finds which of these dependencies are not known yet and repeats the process of getting their statements from Metamath as with the new theorems. As a result of this process `mmisar` creates files `new_theorems.thy`, `new_deps.thy` and `new_known_theorems.txt`. The file `new_theorems.thy` contains the theorems (with proofs) imported from Metamath in this session. These theorems are added (by hand) to the current `MMI_Complex_ZF_x.thy` file. The file `new_deps.thy` contains the statements of new dependencies with generic proofs "by auto". These are added to the `MMI_logic_and_sets.thy`. Most of the dependencies can be proven automatically by Isabelle. However, some manual work has to be done for the dependencies that Isabelle can not prove by itself and to correct problems related to the fact that Metamath uses a metalogic based on distinct variable constraints (Tarski-Megill metalogic), rather than an explicit notion of free and bound variables.

The old list of known theorems is replaced by the new list and `mmisar` is ready to convert the next batch of new theorems. Of course this rarely works in practice without tweaking the `mmisar` source files every time a new batch is processed.

109.2 The context for Metamath theorems

We list the Metamath's axioms of complex numbers and define notation here.

The next definition is what Metamath $X \in V$ is translated to. I am not sure why it works, probably because Isabelle does a type inference and the "=" sign indicates that both sides are sets.

definition

```
IsASet :: i⇒o ( _ isASet [90] 90) where
```

```
IsASet_def[simp]: X isASet ≡ X = X
```

The next locale sets up the context to which Metamath theorems about complex numbers are imported. It assumes the axioms of complex numbers and defines the notation used for complex numbers.

One of the problems with importing theorems from Metamath is that Meta-

math allows direct infix notation for binary operations so that the notation $a f b$ is allowed where f is a function (that is, a set of pairs). To my knowledge, Isar allows only notation $f\langle a, b \rangle$ with a possibility of defining a syntax say $a + b$ to mean the same as $f\langle a, b \rangle$ (please correct me if I am wrong here). This is why we have two objects for addition: one called `caddset` that represents the binary function, and the second one called `ca` which defines the $a + b$ notation for `caddset` $\langle a, b \rangle$. The same applies to multiplication of real numbers.

Another difficulty is that Metamath allows to define sets with syntax $\{x|p\}$ where p is some formula that (usually) depends on x . Isabelle allows the set comprehension like this only as a subset of another set i.e. $\{x \in A.p(x)\}$. This forces us to have a slightly different definition of (complex) natural numbers, requiring explicitly that natural numbers is a subset of reals. Because of that, the proofs of Metamath theorems that reference the definition directly can not be imported.

```

locale MMIsar0 =
  fixes real ( $\mathbb{R}$ )
  fixes complex ( $\mathbb{C}$ )
  fixes one (1)
  fixes zero (0)
  fixes iunit (i)
  fixes caddset (+)
  fixes cmulset ( $\cdot$ )
  fixes lessrrel ( $<_{\mathbb{R}}$ )

  fixes ca (infixl + 69)
  defines ca_def:  $a + b \equiv +\langle a, b \rangle$ 
  fixes cm (infixl  $\cdot$  71)
  defines cm_def:  $a \cdot b \equiv \cdot\langle a, b \rangle$ 
  fixes sub (infixl - 69)
  defines sub_def:  $a - b \equiv \bigcup \{ x \in \mathbb{C}. b + x = a \}$ 
  fixes cneg (-_ 95)
  defines cneg_def:  $- a \equiv 0 - a$ 
  fixes cdiv (infixl / 70)
  defines cdiv_def:  $a / b \equiv \bigcup \{ x \in \mathbb{C}. b \cdot x = a \}$ 
  fixes cpnf ( $+\infty$ )
  defines cpnf_def:  $+\infty \equiv \mathbb{C}$ 
  fixes cmnf ( $-\infty$ )
  defines cmnf_def:  $-\infty \equiv \{\mathbb{C}\}$ 
  fixes cxr ( $\mathbb{R}^*$ )
  defines cxr_def:  $\mathbb{R}^* \equiv \mathbb{R} \cup \{+\infty, -\infty\}$ 
  fixes cxn ( $\mathbb{N}$ )
  defines cxn_def:  $\mathbb{N} \equiv \bigcap \{ N \in \text{Pow}(\mathbb{R}). 1 \in N \wedge (\forall n. n \in N \longrightarrow n+1 \in N) \}$ 
  fixes lessr (infix  $<_{\mathbb{R}}$  68)
  defines lessr_def:  $a <_{\mathbb{R}} b \equiv \langle a, b \rangle \in <_{\mathbb{R}}$ 
  fixes cltrrset (<)
  defines cltrrset_def:

```



```

< ≡ (<ℝ ∩ ℝ×ℝ) ∪ {⟨-∞,+∞⟩} ∪
(ℝ×{+∞}) ∪ ({-∞}×ℝ)
fixes cltrr (infix < 68)
defines cltrr_def: a < b ≡ ⟨a,b⟩ ∈ <
fixes convcltrr (infix > 68)
defines convcltrr_def: a > b ≡ ⟨a,b⟩ ∈ converse(<)
fixes lsq (infix ≤ 68)
defines lsq_def: a ≤ b ≡ ¬ (b < a)
fixes two (2)
defines two_def: 2 ≡ 1+1
fixes three (3)
defines three_def: 3 ≡ 2+1
fixes four (4)
defines four_def: 4 ≡ 3+1
fixes five (5)
defines five_def: 5 ≡ 4+1
fixes six (6)
defines six_def: 6 ≡ 5+1
fixes seven (7)
defines seven_def: 7 ≡ 6+1
fixes eight (8)
defines eight_def: 8 ≡ 7+1
fixes nine (9)
defines nine_def: 9 ≡ 8+1

assumes MMI_pre_axlttri:
A ∈ ℝ ∧ B ∈ ℝ → (A <ℝ B ↔ ¬(A=B ∨ B <ℝ A))
assumes MMI_pre_axlttrn:
A ∈ ℝ ∧ B ∈ ℝ ∧ C ∈ ℝ → ((A <ℝ B ∧ B <ℝ C) → A <ℝ C)
assumes MMI_pre_axltadd:
A ∈ ℝ ∧ B ∈ ℝ ∧ C ∈ ℝ → (A <ℝ B → C+A <ℝ C+B)
assumes MMI_pre_axmulgt0:
A ∈ ℝ ∧ B ∈ ℝ → (0 <ℝ A ∧ 0 <ℝ B → 0 <ℝ A·B)
assumes MMI_pre_axsup:
A ⊆ ℝ ∧ A ≠ 0 ∧ (∃x∈ℝ. ∀y∈A. y <ℝ x) →
(∃x∈ℝ. (∀y∈A. ¬(x <ℝ y)) ∧ (∀y∈ℝ. (y <ℝ x → (∃z∈A. y <ℝ z))))
assumes MMI_axresscn: ℝ ⊆ ℂ
assumes MMI_ax1ne0: 1 ≠ 0
assumes MMI_axcnex: ℂ isASet
assumes MMI_axaddopr: + : (ℂ × ℂ) → ℂ
assumes MMI_axmulopr: · : (ℂ × ℂ) → ℂ
assumes MMI_axmulcom: A ∈ ℂ ∧ B ∈ ℂ → A · B = B · A
assumes MMI_axaddcl: A ∈ ℂ ∧ B ∈ ℂ → A + B ∈ ℂ
assumes MMI_axmulcl: A ∈ ℂ ∧ B ∈ ℂ → A · B ∈ ℂ
assumes MMI_axdistr:
A ∈ ℂ ∧ B ∈ ℂ ∧ C ∈ ℂ → A·(B + C) = A·B + A·C
assumes MMI_axaddcom: A ∈ ℂ ∧ B ∈ ℂ → A + B = B + A
assumes MMI_axaddass:
A ∈ ℂ ∧ B ∈ ℂ ∧ C ∈ ℂ → A + B + C = A + (B + C)

```

```

assumes MMI_axmulass:
 $A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C} \longrightarrow A \cdot B \cdot C = A \cdot (B \cdot C)$ 
assumes MMI_ax1re:  $1 \in \mathbb{R}$ 
assumes MMI_axi2m1:  $i \cdot i + 1 = 0$ 
assumes MMI_ax0id:  $A \in \mathbb{C} \longrightarrow A + 0 = A$ 
assumes MMI_axicn:  $i \in \mathbb{C}$ 
assumes MMI_axnegex:  $A \in \mathbb{C} \longrightarrow (\exists x \in \mathbb{C}. (A + x) = 0)$ 
assumes MMI_axrecex:  $A \in \mathbb{C} \wedge A \neq 0 \longrightarrow (\exists x \in \mathbb{C}. A \cdot x = 1)$ 
assumes MMI_ax1id:  $A \in \mathbb{C} \longrightarrow A \cdot 1 = A$ 
assumes MMI_axaddrcl:  $A \in \mathbb{R} \wedge B \in \mathbb{R} \longrightarrow A + B \in \mathbb{R}$ 
assumes MMI_axmulrcl:  $A \in \mathbb{R} \wedge B \in \mathbb{R} \longrightarrow A \cdot B \in \mathbb{R}$ 
assumes MMI_axrnegex:  $A \in \mathbb{R} \longrightarrow (\exists x \in \mathbb{R}. A + x = 0)$ 
assumes MMI_axrrecex:  $A \in \mathbb{R} \wedge A \neq 0 \longrightarrow (\exists x \in \mathbb{R}. A \cdot x = 1)$ 

```

end

110 Logic and sets in Metamatah

```
theory MMI_logic_and_sets imports MMI_prelude
```

```
begin
```

110.1 Basic Metamath theorems

This section contains Metamath theorems that the more advanced theorems from `MMIsar.thy` depend on. Most of these theorems are proven automatically by Isabelle, some have to be proven by hand and some have to be modified to convert from Tarski-Megill metalogic used by Metamath to one based on explicit notion of free and bound variables.

```
lemma MMI_ax_mp: assumes  $\varphi$  and  $\varphi \longrightarrow \psi$  shows  $\psi$ 
  <proof>
```

```
lemma MMI_sseli: assumes A1:  $A \subseteq B$ 
  shows  $C \in A \longrightarrow C \in B$ 
  <proof>
```

```
lemma MMI_sselii: assumes A1:  $A \subseteq B$  and
  A2:  $C \in A$ 
shows  $C \in B$ 
  <proof>
```

```
lemma MMI_syl: assumes A1:  $\varphi \longrightarrow \text{ps}$  and
  A2:  $\text{ps} \longrightarrow \text{ch}$ 
shows  $\varphi \longrightarrow \text{ch}$ 
  <proof>
```

lemma MMI_elimhyp: assumes A1: $A = \text{if } (\varphi, A, B) \longrightarrow (\varphi \longleftrightarrow \psi)$
and
A2: $B = \text{if } (\varphi, A, B) \longrightarrow (\text{ch} \longleftrightarrow \psi)$ and
A3: ch
shows ψ
 $\langle \text{proof} \rangle$

lemma MMI_neeq1:
shows $A = B \longrightarrow (A \neq C \longleftrightarrow B \neq C)$
 $\langle \text{proof} \rangle$

lemma MMI_mp2: assumes A1: φ and
A2: ψ and
A3: $\varphi \longrightarrow (\psi \longrightarrow \text{chi})$
shows chi
 $\langle \text{proof} \rangle$

lemma MMI_xpex: assumes A1: $A \text{ isASet}$ and
A2: $B \text{ isASet}$
shows $(A \times B) \text{ isASet}$
 $\langle \text{proof} \rangle$

lemma MMI_fex:
shows
 $A \in C \longrightarrow (F : A \rightarrow B \longrightarrow F \text{ isASet})$
 $A \text{ isASet} \longrightarrow (F : A \rightarrow B \longrightarrow F \text{ isASet})$
 $\langle \text{proof} \rangle$

lemma MMI_3eqtr4d: assumes A1: $\varphi \longrightarrow A = B$ and
A2: $\varphi \longrightarrow C = A$ and
A3: $\varphi \longrightarrow D = B$
shows $\varphi \longrightarrow C = D$
 $\langle \text{proof} \rangle$

lemma MMI_3com1: assumes A1: $(\varphi \wedge \psi \wedge \text{chi}) \longrightarrow \text{th}$
shows $(\psi \wedge \text{chi} \wedge \varphi) \longrightarrow \text{th}$
 $\langle \text{proof} \rangle$

lemma MMI_syln: assumes A1: $(\varphi \wedge \psi) \longrightarrow \text{chi}$ and
A2: $\text{th} \longrightarrow \varphi$
shows $(\text{th} \wedge \psi) \longrightarrow \text{chi}$
 $\langle \text{proof} \rangle$

lemma MMI_3impa: assumes A1: $((\varphi \wedge \psi) \wedge \text{chi}) \longrightarrow \text{th}$
shows $(\varphi \wedge \psi \wedge \text{chi}) \longrightarrow \text{th}$
 $\langle \text{proof} \rangle$

lemma MMI_3adant2: assumes A1: $(\varphi \wedge \psi) \longrightarrow \text{chi}$
shows $(\varphi \wedge \text{th} \wedge \psi) \longrightarrow \text{chi}$

<proof>

lemma MMI_3adant1: **assumes** A1: $(\varphi \wedge \psi) \longrightarrow \text{chi}$
shows $(\text{th} \wedge \varphi \wedge \psi) \longrightarrow \text{chi}$
<proof>

lemma (in MMIsar0) MMI_opreq12d: **assumes** A1: $\varphi \longrightarrow A = B$ **and**
A2: $\varphi \longrightarrow C = D$
shows
 $\varphi \longrightarrow (A + C) = (B + D)$
 $\varphi \longrightarrow (A \cdot C) = (B \cdot D)$
 $\varphi \longrightarrow (A - C) = (B - D)$
 $\varphi \longrightarrow (A / C) = (B / D)$
<proof>

lemma MMI_mp2an: **assumes** A1: φ **and**
A2: ψ **and**
A3: $(\varphi \wedge \psi) \longrightarrow \text{chi}$
shows chi
<proof>

lemma MMI_mp3an: **assumes** A1: φ **and**
A2: ψ **and**
A3: ch **and**
A4: $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$
shows ϑ
<proof>

lemma MMI_eqeltrr: **assumes** A1: $A = B$ **and**
A2: $A \in C$
shows $B \in C$
<proof>

lemma MMI_eqtr: **assumes** A1: $A = B$ **and**
A2: $B = C$
shows $A = C$
<proof>

lemma MMI_impbi: **assumes** A1: $\varphi \longrightarrow \psi$ **and**
A2: $\psi \longrightarrow \varphi$
shows $\varphi \longleftrightarrow \psi$
<proof>

lemma MMI_mp3an3: **assumes** A1: ch **and**
A2: $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$
shows $(\varphi \wedge \psi) \longrightarrow \vartheta$
<proof>

lemma MMI_eqeq12d: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $\varphi \longrightarrow C = D$
shows $\varphi \longrightarrow (A = C \longleftrightarrow B = D)$
 $\langle proof \rangle$

lemma MMI_mpan2: assumes A1: ψ and
 A2: $(\varphi \wedge \psi) \longrightarrow ch$
shows $\varphi \longrightarrow ch$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_opreq2:
shows
 $A = B \longrightarrow (C + A) = (C + B)$
 $A = B \longrightarrow (C \cdot A) = (C \cdot B)$
 $A = B \longrightarrow (C - A) = (C - B)$
 $A = B \longrightarrow (C / A) = (C / B)$
 $\langle proof \rangle$

lemma MMI_syl5bir: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ and
 A2: $\vartheta \longrightarrow ch$
shows $\varphi \longrightarrow (\vartheta \longrightarrow \psi)$
 $\langle proof \rangle$

lemma MMI_adantr: assumes A1: $\varphi \longrightarrow \psi$
shows $(\varphi \wedge ch) \longrightarrow \psi$
 $\langle proof \rangle$

lemma MMI_mpan: assumes A1: φ and
 A2: $(\varphi \wedge \psi) \longrightarrow ch$
shows $\psi \longrightarrow ch$
 $\langle proof \rangle$

lemma MMI_eqeq1d: assumes A1: $\varphi \longrightarrow A = B$
shows $\varphi \longrightarrow (A = C \longleftrightarrow B = C)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_opreq1:
shows
 $A = B \longrightarrow (A \cdot C) = (B \cdot C)$
 $A = B \longrightarrow (A + C) = (B + C)$
 $A = B \longrightarrow (A - C) = (B - C)$
 $A = B \longrightarrow (A / C) = (B / C)$
 $\langle proof \rangle$

lemma MMI_syl6eq: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $B = C$
shows $\varphi \longrightarrow A = C$
 $\langle proof \rangle$

lemma MMI_syl6bi: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ and
 A2: $\text{ch} \longrightarrow \vartheta$
shows $\varphi \longrightarrow (\psi \longrightarrow \vartheta)$
<proof>

lemma MMI_imp: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow \text{ch})$
shows $(\varphi \wedge \psi) \longrightarrow \text{ch}$
<proof>

lemma MMI_sylibd: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow \text{ch})$ and
 A2: $\varphi \longrightarrow (\text{ch} \longleftrightarrow \vartheta)$
shows $\varphi \longrightarrow (\psi \longrightarrow \vartheta)$
<proof>

lemma MMI_ex: assumes A1: $(\varphi \wedge \psi) \longrightarrow \text{ch}$
shows $\varphi \longrightarrow (\psi \longrightarrow \text{ch})$
<proof>

lemma MMI_r19_23aiv: assumes A1: $\forall x. (x \in A \longrightarrow (\varphi(x) \longrightarrow \psi))$
shows $(\exists x \in A. \varphi(x)) \longrightarrow \psi$
<proof>

lemma MMI_bitr: assumes A1: $\varphi \longleftrightarrow \psi$ and
 A2: $\psi \longleftrightarrow \text{ch}$
shows $\varphi \longleftrightarrow \text{ch}$
<proof>

lemma MMI_epeq12i: assumes A1: $A = B$ and
 A2: $C = D$
shows $A = C \longleftrightarrow B = D$
<proof>

lemma MMI_dedth3h:
 assumes A1: $A = \text{if } (\varphi, A, D) \longrightarrow (\vartheta \longleftrightarrow \text{ta})$ and
 A2: $B = \text{if } (\psi, B, R) \longrightarrow (\text{ta} \longleftrightarrow \text{et})$ and
 A3: $C = \text{if } (\text{ch}, C, S) \longrightarrow (\text{et} \longleftrightarrow \text{ze})$ and
 A4: ze
shows $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$
<proof>

lemma MMI_bibi1d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$
shows $\varphi \longrightarrow ((\psi \longleftrightarrow \vartheta) \longleftrightarrow (\text{ch} \longleftrightarrow \vartheta))$
<proof>

lemma MMI_epeq1:
shows $A = B \longrightarrow (A = C \longleftrightarrow B = C)$
<proof>

lemma MMI_bibi12d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ and
 A2: $\varphi \longrightarrow (\vartheta \longleftrightarrow \text{ta})$
 shows $\varphi \longrightarrow ((\psi \longleftrightarrow \vartheta) \longleftrightarrow (\text{ch} \longleftrightarrow \text{ta}))$
 $\langle \text{proof} \rangle$

lemma MMI_eqeq2d: assumes A1: $\varphi \longrightarrow A = B$
 shows $\varphi \longrightarrow (C = A \longleftrightarrow C = B)$
 $\langle \text{proof} \rangle$

lemma MMI_eqeq2:
 shows $A = B \longrightarrow (C = A \longleftrightarrow C = B)$
 $\langle \text{proof} \rangle$

lemma MMI_elim1: assumes A1: $B \in C$
 shows if $(A \in C, A, B) \in C$
 $\langle \text{proof} \rangle$

lemma MMI_3adant3: assumes A1: $(\varphi \wedge \psi) \longrightarrow \text{ch}$
 shows $(\varphi \wedge \psi \wedge \vartheta) \longrightarrow \text{ch}$
 $\langle \text{proof} \rangle$

lemma MMI_bitr3d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ and
 A2: $\varphi \longrightarrow (\psi \longleftrightarrow \vartheta)$
 shows $\varphi \longrightarrow (\text{ch} \longleftrightarrow \vartheta)$
 $\langle \text{proof} \rangle$

lemma MMI_3eqtr3d: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $\varphi \longrightarrow A = C$ and
 A3: $\varphi \longrightarrow B = D$
 shows $\varphi \longrightarrow C = D$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_opreq1d: assumes A1: $\varphi \longrightarrow A = B$
 shows
 $\varphi \longrightarrow (A + C) = (B + C)$
 $\varphi \longrightarrow (A - C) = (B - C)$
 $\varphi \longrightarrow (A \cdot C) = (B \cdot C)$
 $\varphi \longrightarrow (A / C) = (B / C)$
 $\langle \text{proof} \rangle$

lemma MMI_3com12: assumes A1: $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$
 shows $(\psi \wedge \varphi \wedge \text{ch}) \longrightarrow \vartheta$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_opreq2d: assumes A1: $\varphi \longrightarrow A = B$
 shows
 $\varphi \longrightarrow (C + A) = (C + B)$

$\varphi \longrightarrow (C - A) = (C - B)$
 $\varphi \longrightarrow (C \cdot A) = (C \cdot B)$
 $\varphi \longrightarrow (C / A) = (C / B)$
 $\langle proof \rangle$

lemma MMI_3com23: assumes A1: $(\varphi \wedge \psi \wedge ch) \longrightarrow \vartheta$
 shows $(\varphi \wedge ch \wedge \psi) \longrightarrow \vartheta$
 $\langle proof \rangle$

lemma MMI_3expa: assumes A1: $(\varphi \wedge \psi \wedge ch) \longrightarrow \vartheta$
 shows $((\varphi \wedge \psi) \wedge ch) \longrightarrow \vartheta$
 $\langle proof \rangle$

lemma MMI_adantrr: assumes A1: $(\varphi \wedge \psi) \longrightarrow ch$
 shows $(\varphi \wedge (\psi \wedge \vartheta)) \longrightarrow ch$
 $\langle proof \rangle$

lemma MMI_3expb: assumes A1: $(\varphi \wedge \psi \wedge ch) \longrightarrow \vartheta$
 shows $(\varphi \wedge (\psi \wedge ch)) \longrightarrow \vartheta$
 $\langle proof \rangle$

lemma MMI_an4s: assumes A1: $((\varphi \wedge \psi) \wedge (ch \wedge \vartheta)) \longrightarrow \tau$
 shows $((\varphi \wedge ch) \wedge (\psi \wedge \vartheta)) \longrightarrow \tau$
 $\langle proof \rangle$

lemma MMI_eqtrd: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $\varphi \longrightarrow B = C$
 shows $\varphi \longrightarrow A = C$
 $\langle proof \rangle$

lemma MMI_ad2ant2l: assumes A1: $(\varphi \wedge \psi) \longrightarrow ch$
 shows $((\vartheta \wedge \varphi) \wedge (\tau \wedge \psi)) \longrightarrow ch$
 $\langle proof \rangle$

lemma MMI_pm3_2i: assumes A1: φ and
 A2: ψ
 shows $\varphi \wedge \psi$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_opreq2i: assumes A1: $A = B$
 shows
 $(C + A) = (C + B)$
 $(C - A) = (C - B)$
 $(C \cdot A) = (C \cdot B)$
 $\langle proof \rangle$

lemma MMI_mpbir2an: assumes A1: $\varphi \longleftrightarrow (\psi \wedge ch)$ and

A2: ψ and
 A3: ch
 shows φ
 $\langle proof \rangle$

lemma MMI_reu4: assumes A1: $\forall x y. x = y \longrightarrow (\varphi(x) \longleftrightarrow \psi(y))$
 shows $(\exists! x . x \in A \wedge \varphi(x)) \longleftrightarrow$
 $((\exists x \in A . \varphi(x)) \wedge (\forall x \in A . \forall y \in A .$
 $(\varphi(x) \wedge \psi(y)) \longrightarrow x = y)))$
 $\langle proof \rangle$

lemma MMI_risset:
 shows $A \in B \longleftrightarrow (\exists x \in B . x = A)$
 $\langle proof \rangle$

lemma MMI_sylib: assumes A1: $\varphi \longrightarrow \psi$ and
 A2: $\psi \longleftrightarrow ch$
 shows $\varphi \longrightarrow ch$
 $\langle proof \rangle$

lemma MMI_mp3an13: assumes A1: φ and
 A2: ch and
 A3: $(\varphi \wedge \psi \wedge ch) \longrightarrow \vartheta$
 shows $\psi \longrightarrow \vartheta$
 $\langle proof \rangle$

lemma MMI_eqcomd: assumes A1: $\varphi \longrightarrow A = B$
 shows $\varphi \longrightarrow B = A$
 $\langle proof \rangle$

lemma MMI_sylan9eqr: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $\psi \longrightarrow B = C$
 shows $(\psi \wedge \varphi) \longrightarrow A = C$
 $\langle proof \rangle$

lemma MMI_exp32: assumes A1: $(\varphi \wedge (\psi \wedge ch)) \longrightarrow \vartheta$
 shows $\varphi \longrightarrow (\psi \longrightarrow (ch \longrightarrow \vartheta))$
 $\langle proof \rangle$

lemma MMI_impcom: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow ch)$
 shows $(\psi \wedge \varphi) \longrightarrow ch$
 $\langle proof \rangle$

lemma MMI_a1d: assumes A1: $\varphi \longrightarrow \psi$
 shows $\varphi \longrightarrow (ch \longrightarrow \psi)$
 $\langle proof \rangle$

lemma MMI_r19_21aiv: assumes A1: $\forall x. \varphi \longrightarrow (x \in A \longrightarrow \psi(x))$
 shows $\varphi \longrightarrow (\forall x \in A . \psi(x))$

$\langle proof \rangle$

lemma MMI_r19_22:

shows $(\forall x \in A . (\varphi(x) \longrightarrow \psi(x))) \longrightarrow$
 $((\exists x \in A . \varphi(x)) \longrightarrow (\exists x \in A . \psi(x)))$
 $\langle proof \rangle$

lemma MMI_syl6: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow ch)$ and

A2: $ch \longrightarrow \vartheta$

shows $\varphi \longrightarrow (\psi \longrightarrow \vartheta)$

$\langle proof \rangle$

lemma MMI_mpid: assumes A1: $\varphi \longrightarrow ch$ and

A2: $\varphi \longrightarrow (\psi \longrightarrow (ch \longrightarrow \vartheta))$

shows $\varphi \longrightarrow (\psi \longrightarrow \vartheta)$

$\langle proof \rangle$

lemma MMI_eqtr3t:

shows $(A = C \wedge B = C) \longrightarrow A = B$

$\langle proof \rangle$

lemma MMI_syl5bi: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ and

A2: $\vartheta \longrightarrow \psi$

shows $\varphi \longrightarrow (\vartheta \longrightarrow ch)$

$\langle proof \rangle$

lemma MMI_mp3an1: assumes A1: φ and

A2: $(\varphi \wedge \psi \wedge ch) \longrightarrow \vartheta$

shows $(\psi \wedge ch) \longrightarrow \vartheta$

$\langle proof \rangle$

lemma MMI_rgen2: assumes A1: $\forall x y . (x \in A \wedge y \in A) \longrightarrow \varphi(x,y)$

shows $\forall x \in A . \forall y \in A . \varphi(x,y)$

$\langle proof \rangle$

lemma MMI_ax_17: shows $\varphi \longrightarrow (\forall x . \varphi)$ $\langle proof \rangle$

lemma MMI_3eqtr4g: assumes A1: $\varphi \longrightarrow A = B$ and

A2: $C = A$ and

A3: $D = B$

shows $\varphi \longrightarrow C = D$

$\langle proof \rangle$

lemma MMI_3imtr4: assumes A1: $\varphi \longrightarrow \psi$ and

A2: $ch \longleftrightarrow \varphi$ and
 A3: $\vartheta \longleftrightarrow \psi$
 shows $ch \longrightarrow \vartheta$
<proof>

lemma MMI_eleq2i: assumes A1: $A = B$
 shows $C \in A \longleftrightarrow C \in B$
<proof>

lemma MMI_albii: assumes A1: $\varphi \longleftrightarrow \psi$
 shows $(\forall x. \varphi) \longleftrightarrow (\forall x. \psi)$
<proof>

lemma MMI_reucl:
 shows $(\exists! x. x \in A \wedge \varphi(x)) \longrightarrow \bigcup \{x \in A. \varphi(x)\} \in A$
<proof>

lemma MMI_dedth2h: assumes A1: $A = \text{if } (\varphi, A, C) \longrightarrow (ch \longleftrightarrow \vartheta)$
) and
 A2: $B = \text{if } (\psi, B, D) \longrightarrow (\vartheta \longleftrightarrow \tau)$ and
 A3: τ
 shows $(\varphi \wedge \psi) \longrightarrow ch$
<proof>

lemma MMI_eleq1d: assumes A1: $\varphi \longrightarrow A = B$
 shows $\varphi \longrightarrow (A \in C \longleftrightarrow B \in C)$
<proof>

lemma MMI_syl5eqel: assumes A1: $\varphi \longrightarrow A \in B$ and
 A2: $C = A$
 shows $\varphi \longrightarrow C \in B$
<proof>

lemma IML_eeuni: assumes A1: $x \in A$ and A2: $\exists! t. t \in A \wedge \varphi(t)$
 shows $\varphi(x) \longleftrightarrow \bigcup \{x \in A. \varphi(x)\} = x$
<proof>

lemma MMI_reuuni1:
 shows $(x \in A \wedge (\exists! x. x \in A \wedge \varphi(x))) \longrightarrow$
 $(\varphi(x) \longleftrightarrow \bigcup \{x \in A. \varphi(x)\} = x)$
<proof>

lemma MMI_epeq1i: assumes A1: $A = B$
 shows $A = C \longleftrightarrow B = C$

<proof>

lemma MMI_syl6rbbr: assumes A1: $\forall x. \varphi(x) \longrightarrow (\psi(x) \longleftrightarrow \text{ch}(x))$ and
A2: $\forall x. \vartheta(x) \longleftrightarrow \text{ch}(x)$
shows $\forall x. \varphi(x) \longrightarrow (\vartheta(x) \longleftrightarrow \psi(x))$
<proof>

lemma MMI_syl6rbbrA: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ and
A2: $\vartheta \longleftrightarrow \text{ch}$
shows $\varphi \longrightarrow (\vartheta \longleftrightarrow \psi)$
<proof>

lemma MMI_vtoclga: assumes A1: $\forall x. x = A \longrightarrow (\varphi(x) \longleftrightarrow \psi)$ and
A2: $\forall x. x \in B \longrightarrow \varphi(x)$
shows $A \in B \longrightarrow \psi$
<proof>

lemma MMI_3bitr4: assumes A1: $\varphi \longleftrightarrow \psi$ and
A2: $\text{ch} \longleftrightarrow \varphi$ and
A3: $\vartheta \longleftrightarrow \psi$
shows $\text{ch} \longleftrightarrow \vartheta$
<proof>

lemma MMI_mpbii: assumes Amin: ψ and
Amaj: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$
shows $\varphi \longrightarrow \text{ch}$
<proof>

lemma MMI_eqid:
shows $A = A$
<proof>

lemma MMI_pm3_27:
shows $(\varphi \wedge \psi) \longrightarrow \psi$
<proof>

lemma MMI_pm3_26:
shows $(\varphi \wedge \psi) \longrightarrow \varphi$
<proof>

lemma MMI_ancoms: assumes A1: $(\varphi \wedge \psi) \longrightarrow \text{ch}$
shows $(\psi \wedge \varphi) \longrightarrow \text{ch}$
<proof>

lemma MMI_syl3anc: assumes A1: $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$ and

A2: $\tau \longrightarrow \varphi$ and
 A3: $\tau \longrightarrow \psi$ and
 A4: $\tau \longrightarrow \text{ch}$
 shows $\tau \longrightarrow \vartheta$
<proof>

lemma MMI_syl5eq: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $C = A$
 shows $\varphi \longrightarrow C = B$
<proof>

lemma MMI_eqcomi: assumes A1: $A = B$
 shows $B = A$
<proof>

lemma MMI_3eqtr: assumes A1: $A = B$ and
 A2: $B = C$ and
 A3: $C = D$
 shows $A = D$
<proof>

lemma MMI_mpbir: assumes Amin: ψ and
 Amaj: $\varphi \longleftrightarrow \psi$
 shows φ
<proof>

lemma MMI_syl3an3: assumes A1: $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$ and
 A2: $\tau \longrightarrow \text{ch}$
 shows $(\varphi \wedge \psi \wedge \tau) \longrightarrow \vartheta$
<proof>

lemma MMI_3eqtrd: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $\varphi \longrightarrow B = C$ and
 A3: $\varphi \longrightarrow C = D$
 shows $\varphi \longrightarrow A = D$
<proof>

lemma MMI_syl5: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow \text{ch})$ and
 A2: $\vartheta \longrightarrow \psi$
 shows $\varphi \longrightarrow (\vartheta \longrightarrow \text{ch})$
<proof>

lemma MMI_exp3a: assumes A1: $\varphi \longrightarrow ((\psi \wedge \text{ch}) \longrightarrow \vartheta)$
 shows $\varphi \longrightarrow (\psi \longrightarrow (\text{ch} \longrightarrow \vartheta))$
<proof>

lemma MMI_com12: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow \text{ch})$
 shows $\psi \longrightarrow (\varphi \longrightarrow \text{ch})$
<proof>

lemma MMI_3imp: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow (\text{ch} \longrightarrow \vartheta))$
 shows $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$
 $\langle \text{proof} \rangle$

lemma MMI_3eqtr3: assumes A1: $A = B$ and
 A2: $A = C$ and
 A3: $B = D$
 shows $C = D$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) **MMI_opreq1i:** assumes A1: $A = B$
 shows
 $(A + C) = (B + C)$
 $(A - C) = (B - C)$
 $(A / C) = (B / C)$
 $(A \cdot C) = (B \cdot C)$
 $\langle \text{proof} \rangle$

lemma MMI_eqtr3: assumes A1: $A = B$ and
 A2: $A = C$
 shows $B = C$
 $\langle \text{proof} \rangle$

lemma MMI_dedth: assumes A1: $A = \text{if } (\varphi , A , B) \longrightarrow (\psi \longleftrightarrow \text{ch})$
 and
 A2: ch
 shows $\varphi \longrightarrow \psi$
 $\langle \text{proof} \rangle$

lemma MMI_id:
 shows $\varphi \longrightarrow \varphi$
 $\langle \text{proof} \rangle$

lemma MMI_eqtr3d: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $\varphi \longrightarrow A = C$
 shows $\varphi \longrightarrow B = C$
 $\langle \text{proof} \rangle$

lemma MMI_sylan2: assumes A1: $(\varphi \wedge \psi) \longrightarrow \text{ch}$ and
 A2: $\vartheta \longrightarrow \psi$
 shows $(\varphi \wedge \vartheta) \longrightarrow \text{ch}$
 $\langle \text{proof} \rangle$

lemma MMI_adant1: assumes A1: $\varphi \longrightarrow \psi$
 shows $(\text{ch} \wedge \varphi) \longrightarrow \psi$
 $\langle \text{proof} \rangle$

```

lemma (in MMIisar0) MMI_opreq12:
  shows
    ( A = B  $\wedge$  C = D )  $\longrightarrow$  ( A + C ) = ( B + D )
    ( A = B  $\wedge$  C = D )  $\longrightarrow$  ( A - C ) = ( B - D )
    ( A = B  $\wedge$  C = D )  $\longrightarrow$  ( A  $\cdot$  C ) = ( B  $\cdot$  D )
    ( A = B  $\wedge$  C = D )  $\longrightarrow$  ( A / C ) = ( B / D )
  <proof>

lemma MMI_anidms: assumes A1: (  $\varphi \wedge \varphi$  )  $\longrightarrow \psi$ 
  shows  $\varphi \longrightarrow \psi$ 
  <proof>

lemma MMI_anabsan2: assumes A1: (  $\varphi \wedge ( \psi \wedge \psi )$  )  $\longrightarrow$  ch
  shows (  $\varphi \wedge \psi$  )  $\longrightarrow$  ch
  <proof>

lemma MMI_3simp2:
  shows (  $\varphi \wedge \psi \wedge$  ch )  $\longrightarrow \psi$ 
  <proof>

lemma MMI_3simp3:
  shows (  $\varphi \wedge \psi \wedge$  ch )  $\longrightarrow$  ch
  <proof>

lemma MMI_sylbir: assumes A1:  $\psi \longleftrightarrow \varphi$  and
  A2:  $\psi \longrightarrow$  ch
  shows  $\varphi \longrightarrow$  ch
  <proof>

lemma MMI_3eqtr3g: assumes A1:  $\varphi \longrightarrow$  A = B and
  A2: A = C and
  A3: B = D
  shows  $\varphi \longrightarrow$  C = D
  <proof>

lemma MMI_3bitr: assumes A1:  $\varphi \longleftrightarrow \psi$  and
  A2:  $\psi \longleftrightarrow$  ch and
  A3: ch  $\longleftrightarrow \vartheta$ 
  shows  $\varphi \longleftrightarrow \vartheta$ 
  <proof>

lemma MMI_3bitr3: assumes A1:  $\varphi \longleftrightarrow \psi$  and
  A2:  $\varphi \longleftrightarrow$  ch and
  A3:  $\psi \longleftrightarrow \vartheta$ 

```

shows $ch \longleftrightarrow \vartheta$
⟨proof⟩

lemma MMI_eqcom:
shows $A = B \longleftrightarrow B = A$
⟨proof⟩

lemma MMI_syl6bb: **assumes** A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ **and**
A2: $ch \longleftrightarrow \vartheta$
shows $\varphi \longrightarrow (\psi \longleftrightarrow \vartheta)$
⟨proof⟩

lemma MMI_3bitr3d: **assumes** A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ **and**
A2: $\varphi \longrightarrow (\psi \longleftrightarrow \vartheta)$ **and**
A3: $\varphi \longrightarrow (ch \longleftrightarrow \tau)$
shows $\varphi \longrightarrow (\vartheta \longleftrightarrow \tau)$
⟨proof⟩

lemma MMI_syl3an2: **assumes** A1: $(\varphi \wedge \psi \wedge ch) \longrightarrow \vartheta$ **and**
A2: $\tau \longrightarrow \psi$
shows $(\varphi \wedge \tau \wedge ch) \longrightarrow \vartheta$
⟨proof⟩

lemma MMI_df_rex:
shows $(\exists x \in A . \varphi(x)) \longleftrightarrow (\exists x . (x \in A \wedge \varphi(x)))$
⟨proof⟩

lemma MMI_mpbi: **assumes** Amin: φ **and**
Amaj: $\varphi \longleftrightarrow \psi$
shows ψ
⟨proof⟩

lemma MMI_mp3an12: **assumes** A1: φ **and**
A2: ψ **and**
A3: $(\varphi \wedge \psi \wedge ch) \longrightarrow \vartheta$
shows $ch \longrightarrow \vartheta$
⟨proof⟩

lemma MMI_syl5bb: **assumes** A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ **and**
A2: $\vartheta \longleftrightarrow \psi$
shows $\varphi \longrightarrow (\vartheta \longleftrightarrow ch)$
⟨proof⟩

lemma MMI_eleq1a:
shows $A \in B \longrightarrow (C = A \longrightarrow C \in B)$
⟨proof⟩

lemma MMI_sylbird: assumes A1: $\varphi \longrightarrow (ch \longleftrightarrow \psi)$ and
 A2: $\varphi \longrightarrow (ch \longrightarrow \vartheta)$
 shows $\varphi \longrightarrow (\psi \longrightarrow \vartheta)$
<proof>

lemma MMI_19_23aiv: assumes A1: $\forall x. \varphi(x) \longrightarrow \psi$
 shows $(\exists x. \varphi(x)) \longrightarrow \psi$
<proof>

lemma MMI_eqeltrrd: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $\varphi \longrightarrow A \in C$
 shows $\varphi \longrightarrow B \in C$
<proof>

lemma MMI_syl2an: assumes A1: $(\varphi \wedge \psi) \longrightarrow ch$ and
 A2: $\vartheta \longrightarrow \varphi$ and
 A3: $\tau \longrightarrow \psi$
 shows $(\vartheta \wedge \tau) \longrightarrow ch$
<proof>

lemma MMI_adantrl: assumes A1: $(\varphi \wedge \psi) \longrightarrow ch$
 shows $(\varphi \wedge (\vartheta \wedge \psi)) \longrightarrow ch$
<proof>

lemma MMI_ad2ant2r: assumes A1: $(\varphi \wedge \psi) \longrightarrow ch$
 shows $((\varphi \wedge \vartheta) \wedge (\psi \wedge \tau)) \longrightarrow ch$
<proof>

lemma MMI_adantll: assumes A1: $(\varphi \wedge \psi) \longrightarrow ch$
 shows $((\vartheta \wedge \varphi) \wedge \psi) \longrightarrow ch$
<proof>

lemma MMI_anandirs: assumes A1: $((\varphi \wedge ch) \wedge (\psi \wedge ch)) \longrightarrow \tau$
 shows $((\varphi \wedge \psi) \wedge ch) \longrightarrow \tau$
<proof>

lemma MMI_adantlr: assumes A1: $(\varphi \wedge \psi) \longrightarrow ch$
 shows $((\varphi \wedge \vartheta) \wedge \psi) \longrightarrow ch$
<proof>

lemma MMI_an42s: assumes A1: $((\varphi \wedge \psi) \wedge (ch \wedge \vartheta)) \longrightarrow \tau$
 shows $((\varphi \wedge ch) \wedge (\vartheta \wedge \psi)) \longrightarrow \tau$
<proof>

lemma MMI_mp3an2: assumes A1: ψ and

A2: $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$
shows $(\varphi \wedge \text{ch}) \longrightarrow \vartheta$
<proof>

lemma MMI_3simp1:
shows $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \varphi$
<proof>

lemma MMI_3impb: **assumes** A1: $(\varphi \wedge (\psi \wedge \text{ch})) \longrightarrow \vartheta$
shows $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$
<proof>

lemma MMI_mpbird: **assumes** Amin: $\varphi \longrightarrow \text{ch}$ **and**
Amaj: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$
shows $\varphi \longrightarrow \psi$
<proof>

lemma (in MMIsar0) MMI_opreq12i: **assumes** A1: $A = B$ **and**
A2: $C = D$
shows
 $(A + C) = (B + D)$
 $(A \cdot C) = (B \cdot D)$
 $(A - C) = (B - D)$
<proof>

lemma MMI_3eqtr4: **assumes** A1: $A = B$ **and**
A2: $C = A$ **and**
A3: $D = B$
shows $C = D$
<proof>

lemma MMI_eqtr4d: **assumes** A1: $\varphi \longrightarrow A = B$ **and**
A2: $\varphi \longrightarrow C = B$
shows $\varphi \longrightarrow A = C$
<proof>

lemma MMI_3eqtr3rd: **assumes** A1: $\varphi \longrightarrow A = B$ **and**
A2: $\varphi \longrightarrow A = C$ **and**
A3: $\varphi \longrightarrow B = D$
shows $\varphi \longrightarrow D = C$
<proof>

lemma MMI_syland: **assumes** A1: $(\varphi \wedge \psi) \longrightarrow \text{ch}$ **and**

A2: $\vartheta \longrightarrow \varphi$ **and**
A3: $\vartheta \longrightarrow \psi$
shows $\vartheta \longrightarrow \text{ch}$
<proof>

lemma MMI_anim12i: **assumes** **A1:** $\varphi \longrightarrow \psi$ **and**
A2: $\text{ch} \longrightarrow \vartheta$
shows $(\varphi \wedge \text{ch}) \longrightarrow (\psi \wedge \vartheta)$
<proof>

lemma (in MMIsar0) MMI_opreqan12d: **assumes** **A1:** $\varphi \longrightarrow A = B$ **and**
A2: $\psi \longrightarrow C = D$
shows
 $(\varphi \wedge \psi) \longrightarrow (A + C) = (B + D)$
 $(\varphi \wedge \psi) \longrightarrow (A - C) = (B - D)$
 $(\varphi \wedge \psi) \longrightarrow (A \cdot C) = (B \cdot D)$
<proof>

lemma MMI_sylanr2: **assumes** **A1:** $(\varphi \wedge (\psi \wedge \text{ch})) \longrightarrow \vartheta$ **and**
A2: $\tau \longrightarrow \text{ch}$
shows $(\varphi \wedge (\psi \wedge \tau)) \longrightarrow \vartheta$
<proof>

lemma MMI_sylanl2: **assumes** **A1:** $((\varphi \wedge \psi) \wedge \text{ch}) \longrightarrow \vartheta$ **and**
A2: $\tau \longrightarrow \psi$
shows $((\varphi \wedge \tau) \wedge \text{ch}) \longrightarrow \vartheta$
<proof>

lemma MMI_ancom2s: **assumes** **A1:** $(\varphi \wedge (\psi \wedge \text{ch})) \longrightarrow \vartheta$
shows $(\varphi \wedge (\text{ch} \wedge \psi)) \longrightarrow \vartheta$
<proof>

lemma MMI_anandis: **assumes** **A1:** $((\varphi \wedge \psi) \wedge (\varphi \wedge \text{ch})) \longrightarrow \tau$
shows $(\varphi \wedge (\psi \wedge \text{ch})) \longrightarrow \tau$
<proof>

lemma MMI_sylan9eq: **assumes** **A1:** $\varphi \longrightarrow A = B$ **and**
A2: $\psi \longrightarrow B = C$
shows $(\varphi \wedge \psi) \longrightarrow A = C$
<proof>

lemma MMI_keephyp: **assumes** **A1:** $A = \text{if } (\varphi, A, B) \longrightarrow (\psi \longleftrightarrow \vartheta)$
and
A2: $B = \text{if } (\varphi, A, B) \longrightarrow (\text{ch} \longleftrightarrow \vartheta)$ **and**
A3: ψ **and**

A4: ch
 shows ϑ
 $\langle proof \rangle$

lemma MMI_eleq1:
 shows $A = B \longrightarrow (A \in C \longleftrightarrow B \in C)$
 $\langle proof \rangle$

lemma MMI_pm4_2i:
 shows $\varphi \longrightarrow (\psi \longleftrightarrow \psi)$
 $\langle proof \rangle$

lemma MMI_3anbi123d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ and
 A2: $\varphi \longrightarrow (\vartheta \longleftrightarrow \tau)$ and
 A3: $\varphi \longrightarrow (\eta \longleftrightarrow \zeta)$
 shows $\varphi \longrightarrow ((\psi \wedge \vartheta \wedge \eta) \longleftrightarrow (ch \wedge \tau \wedge \zeta))$
 $\langle proof \rangle$

lemma MMI_imbi12d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ and
 A2: $\varphi \longrightarrow (\vartheta \longleftrightarrow \tau)$
 shows $\varphi \longrightarrow ((\psi \longrightarrow \vartheta) \longleftrightarrow (ch \longrightarrow \tau))$
 $\langle proof \rangle$

lemma MMI_bitrd: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ and
 A2: $\varphi \longrightarrow (ch \longleftrightarrow \vartheta)$
 shows $\varphi \longrightarrow (\psi \longleftrightarrow \vartheta)$
 $\langle proof \rangle$

lemma MMI_df_ne:
 shows $(A \neq B \longleftrightarrow \neg (A = B))$
 $\langle proof \rangle$

lemma MMI_3pm3_2i: assumes A1: φ and
 A2: ψ and
 A3: ch
 shows $\varphi \wedge \psi \wedge ch$
 $\langle proof \rangle$

lemma MMI_eqeq2i: assumes A1: $A = B$
 shows $C = A \longleftrightarrow C = B$
 $\langle proof \rangle$

lemma MMI_syl5bbr: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ and
 A2: $\psi \longleftrightarrow \vartheta$
 shows $\varphi \longrightarrow (\vartheta \longleftrightarrow ch)$
 $\langle proof \rangle$

lemma MMI_biimpd: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$
 shows $\varphi \longrightarrow (\psi \longrightarrow ch)$

<proof>

lemma MMI_orrd: assumes A1: $\varphi \longrightarrow (\neg (\psi) \longrightarrow \text{ch})$
shows $\varphi \longrightarrow (\psi \vee \text{ch})$
<proof>

lemma MMI_jaoi: assumes A1: $\varphi \longrightarrow \psi$ and
A2: $\text{ch} \longrightarrow \psi$
shows $(\varphi \vee \text{ch}) \longrightarrow \psi$
<proof>

lemma MMI_oridm:
shows $(\varphi \vee \varphi) \longleftrightarrow \varphi$
<proof>

lemma MMI_orbi1d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$
shows $\varphi \longrightarrow ((\psi \vee \vartheta) \longleftrightarrow (\text{ch} \vee \vartheta))$
<proof>

lemma MMI_orbi2d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$
shows $\varphi \longrightarrow ((\vartheta \vee \psi) \longleftrightarrow (\vartheta \vee \text{ch}))$
<proof>

lemma MMI_3bitr4g: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ and
A2: $\vartheta \longleftrightarrow \psi$ and
A3: $\tau \longleftrightarrow \text{ch}$
shows $\varphi \longrightarrow (\vartheta \longleftrightarrow \tau)$
<proof>

lemma MMI_negbid: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$
shows $\varphi \longrightarrow (\neg (\psi) \longleftrightarrow \neg (\text{ch}))$
<proof>

lemma MMI_ioran:
shows $\neg ((\varphi \vee \psi)) \longleftrightarrow$
 $(\neg (\varphi) \wedge \neg (\psi))$
<proof>

lemma MMI_syl6rbb: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ and
A2: $\text{ch} \longleftrightarrow \vartheta$
shows $\varphi \longrightarrow (\vartheta \longleftrightarrow \psi)$
<proof>

lemma MMI_anbi12i: assumes A1: $\varphi \longleftrightarrow \psi$ and
A2: $\text{ch} \longleftrightarrow \vartheta$
shows $(\varphi \wedge \text{ch}) \longleftrightarrow (\psi \wedge \vartheta)$
<proof>

lemma MMI_keepe1: assumes A1: $A \in C$ and
 A2: $B \in C$
 shows if $(\varphi, A, B) \in C$
 $\langle proof \rangle$

lemma MMI_imbi2d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$
 shows $\varphi \longrightarrow ((\vartheta \longrightarrow \psi) \longleftrightarrow (\vartheta \longrightarrow ch))$
 $\langle proof \rangle$

lemma MMI_eqeltr: assumes $A = B$ and $B \in C$
 shows $A \in C$ $\langle proof \rangle$

lemma MMI_3impia: assumes A1: $(\varphi \wedge \psi) \longrightarrow (ch \longrightarrow \vartheta)$
 shows $(\varphi \wedge \psi \wedge ch) \longrightarrow \vartheta$
 $\langle proof \rangle$

lemma MMI_eqneqd: assumes A1: $\varphi \longrightarrow (A = B \longleftrightarrow C = D)$
 shows $\varphi \longrightarrow (A \neq B \longleftrightarrow C \neq D)$
 $\langle proof \rangle$

lemma MMI_3ad2ant2: assumes A1: $\varphi \longrightarrow ch$
 shows $(\psi \wedge \varphi \wedge \vartheta) \longrightarrow ch$
 $\langle proof \rangle$

lemma MMI_mp3anl3: assumes A1: ch and
 A2: $((\varphi \wedge \psi \wedge ch) \wedge \vartheta) \longrightarrow \tau$
 shows $((\varphi \wedge \psi) \wedge \vartheta) \longrightarrow \tau$
 $\langle proof \rangle$

lemma MMI_bitr4d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ and
 A2: $\varphi \longrightarrow (\vartheta \longleftrightarrow ch)$
 shows $\varphi \longrightarrow (\psi \longleftrightarrow \vartheta)$
 $\langle proof \rangle$

lemma MMI_neeq1d: assumes A1: $\varphi \longrightarrow A = B$
 shows $\varphi \longrightarrow (A \neq C \longleftrightarrow B \neq C)$
 $\langle proof \rangle$

lemma MMI_3anim123i: assumes A1: $\varphi \longrightarrow \psi$ and
 A2: $\text{ch} \longrightarrow \vartheta$ and
 A3: $\tau \longrightarrow \eta$
shows $(\varphi \wedge \text{ch} \wedge \tau) \longrightarrow (\psi \wedge \vartheta \wedge \eta)$
<proof>

lemma MMI_3exp: assumes A1: $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$
shows $\varphi \longrightarrow (\psi \longrightarrow (\text{ch} \longrightarrow \vartheta))$
<proof>

lemma MMI_exp4a: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow ((\text{ch} \wedge \vartheta) \longrightarrow \tau))$
shows $\varphi \longrightarrow (\psi \longrightarrow (\text{ch} \longrightarrow (\vartheta \longrightarrow \tau)))$
<proof>

lemma MMI_3imp1: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow (\text{ch} \longrightarrow (\vartheta \longrightarrow \tau)))$

shows $((\varphi \wedge \psi \wedge \text{ch}) \wedge \vartheta) \longrightarrow \tau$
<proof>

lemma MMI_anim1i: assumes A1: $\varphi \longrightarrow \psi$
shows $(\varphi \wedge \text{ch}) \longrightarrow (\psi \wedge \text{ch})$
<proof>

lemma MMI_3adantl1: assumes A1: $((\varphi \wedge \psi) \wedge \text{ch}) \longrightarrow \vartheta$
shows $(\tau \wedge \varphi \wedge \psi) \wedge \text{ch} \longrightarrow \vartheta$
<proof>

lemma MMI_3adantl2: assumes A1: $((\varphi \wedge \psi) \wedge \text{ch}) \longrightarrow \vartheta$
shows $(\varphi \wedge \tau \wedge \psi) \wedge \text{ch} \longrightarrow \vartheta$
<proof>

lemma MMI_3comr: assumes A1: $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$
shows $\text{ch} \wedge \varphi \wedge \psi \longrightarrow \vartheta$
<proof>

lemma MMI_bitr3: assumes A1: $\psi \longleftrightarrow \varphi$ and
 A2: $\psi \longleftrightarrow \text{ch}$
shows $\varphi \longleftrightarrow \text{ch}$
<proof>

lemma MMI_anbi12d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ and
 A2: $\varphi \longrightarrow (\vartheta \longleftrightarrow \tau)$
shows $\varphi \longrightarrow ((\psi \wedge \vartheta) \longleftrightarrow (\text{ch} \wedge \tau))$
<proof>

lemma MMI_pm3_26i: assumes A1: $\varphi \wedge \psi$

shows φ
⟨proof⟩

lemma MMI_pm3_27i: **assumes** A1: $\varphi \wedge \psi$
shows ψ
⟨proof⟩

lemma MMI_anabsan: **assumes** A1: $((\varphi \wedge \varphi) \wedge \psi) \longrightarrow \text{ch}$
shows $(\varphi \wedge \psi) \longrightarrow \text{ch}$
⟨proof⟩

lemma MMI_3eqtr4rd: **assumes** A1: $\varphi \longrightarrow A = B$ **and**
A2: $\varphi \longrightarrow C = A$ **and**
A3: $\varphi \longrightarrow D = B$
shows $\varphi \longrightarrow D = C$
⟨proof⟩

lemma MMI_syl3an1: **assumes** A1: $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$ **and**
A2: $\tau \longrightarrow \varphi$
shows $(\tau \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$
⟨proof⟩

lemma MMI_syl3anl2: **assumes** A1: $((\varphi \wedge \psi \wedge \text{ch}) \wedge \vartheta) \longrightarrow \tau$ **and**
A2: $\eta \longrightarrow \psi$
shows $((\varphi \wedge \eta \wedge \text{ch}) \wedge \vartheta) \longrightarrow \tau$
⟨proof⟩

lemma MMI_jca: **assumes** A1: $\varphi \longrightarrow \psi$ **and**
A2: $\varphi \longrightarrow \text{ch}$
shows $\varphi \longrightarrow (\psi \wedge \text{ch})$
⟨proof⟩

lemma MMI_3ad2ant3: **assumes** A1: $\varphi \longrightarrow \text{ch}$
shows $(\psi \wedge \vartheta \wedge \varphi) \longrightarrow \text{ch}$
⟨proof⟩

lemma MMI_anim2i: **assumes** A1: $\varphi \longrightarrow \psi$
shows $(\text{ch} \wedge \varphi) \longrightarrow (\text{ch} \wedge \psi)$
⟨proof⟩

lemma MMI_ancom:
shows $(\varphi \wedge \psi) \longleftrightarrow (\psi \wedge \varphi)$
⟨proof⟩

lemma MMI_anbi1i: assumes Aaa: $\varphi \longleftrightarrow \psi$
 shows $(\varphi \wedge \text{ch}) \longleftrightarrow (\psi \wedge \text{ch})$
 $\langle \text{proof} \rangle$

lemma MMI_an42:
 shows $((\varphi \wedge \psi) \wedge (\text{ch} \wedge \vartheta)) \longleftrightarrow$
 $((\varphi \wedge \text{ch}) \wedge (\vartheta \wedge \psi))$
 $\langle \text{proof} \rangle$

lemma MMI_sylanb: assumes A1: $(\varphi \wedge \psi) \longrightarrow \text{ch}$ and
 A2: $\vartheta \longleftrightarrow \varphi$
 shows $(\vartheta \wedge \psi) \longrightarrow \text{ch}$
 $\langle \text{proof} \rangle$

lemma MMI_an4:
 shows $((\varphi \wedge \psi) \wedge (\text{ch} \wedge \vartheta)) \longleftrightarrow$
 $((\varphi \wedge \text{ch}) \wedge (\psi \wedge \vartheta))$
 $\langle \text{proof} \rangle$

lemma MMI_syl2anb: assumes A1: $(\varphi \wedge \psi) \longrightarrow \text{ch}$ and
 A2: $\vartheta \longleftrightarrow \varphi$ and
 A3: $\tau \longleftrightarrow \psi$
 shows $(\vartheta \wedge \tau) \longrightarrow \text{ch}$
 $\langle \text{proof} \rangle$

lemma MMI_eqtr2d: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $\varphi \longrightarrow B = C$
 shows $\varphi \longrightarrow C = A$
 $\langle \text{proof} \rangle$

lemma MMI_sylbid: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ and
 A2: $\varphi \longrightarrow (\text{ch} \longrightarrow \vartheta)$
 shows $\varphi \longrightarrow (\psi \longrightarrow \vartheta)$
 $\langle \text{proof} \rangle$

lemma MMI_sylanl1: assumes A1: $((\varphi \wedge \psi) \wedge \text{ch}) \longrightarrow \vartheta$ and
 A2: $\tau \longrightarrow \varphi$
 shows $((\tau \wedge \psi) \wedge \text{ch}) \longrightarrow \vartheta$
 $\langle \text{proof} \rangle$

lemma MMI_sylan2b: assumes A1: $(\varphi \wedge \psi) \longrightarrow \text{ch}$ and
 A2: $\vartheta \longleftrightarrow \psi$
 shows $(\varphi \wedge \vartheta) \longrightarrow \text{ch}$
 $\langle \text{proof} \rangle$

lemma MMI_pm3_22:
 shows $(\varphi \wedge \psi) \longrightarrow (\psi \wedge \varphi)$
 $\langle \text{proof} \rangle$

lemma MMI_ancli: assumes A1: $\varphi \longrightarrow \psi$
 shows $\varphi \longrightarrow (\varphi \wedge \psi)$
<proof>

lemma MMI_ad2antlr: assumes A1: $\varphi \longrightarrow \psi$
 shows $((\text{ch} \wedge \varphi) \wedge \vartheta) \longrightarrow \psi$
<proof>

lemma MMI_biimpa: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$
 shows $(\varphi \wedge \psi) \longrightarrow \text{ch}$
<proof>

lemma MMI_sylan2i: assumes A1: $\varphi \longrightarrow ((\psi \wedge \text{ch}) \longrightarrow \vartheta)$ and
 A2: $\tau \longrightarrow \text{ch}$
 shows $\varphi \longrightarrow ((\psi \wedge \tau) \longrightarrow \vartheta)$
<proof>

lemma MMI_3jca: assumes A1: $\varphi \longrightarrow \psi$ and
 A2: $\varphi \longrightarrow \text{ch}$ and
 A3: $\varphi \longrightarrow \vartheta$
 shows $\varphi \longrightarrow (\psi \wedge \text{ch} \wedge \vartheta)$
<proof>

lemma MMI_com34: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow (\text{ch} \longrightarrow (\vartheta \longrightarrow \tau)))$

 shows $\varphi \longrightarrow (\psi \longrightarrow (\vartheta \longrightarrow (\text{ch} \longrightarrow \tau)))$
<proof>

lemma MMI_imp43: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow (\text{ch} \longrightarrow (\vartheta \longrightarrow \tau)))$

 shows $((\varphi \wedge \psi) \wedge (\text{ch} \wedge \vartheta)) \longrightarrow \tau$
<proof>

lemma MMI_3anass:
 shows $(\varphi \wedge \psi \wedge \text{ch}) \longleftrightarrow (\varphi \wedge (\psi \wedge \text{ch}))$
<proof>

lemma MMI_3eqtr4r: assumes A1: $A = B$ and
 A2: $C = A$ and
 A3: $D = B$
 shows $D = C$
<proof>

lemma MMI_jctl: assumes A1: ψ
 shows $\varphi \longrightarrow (\psi \wedge \varphi)$
<proof>

lemma MMI_sylibr: assumes A1: $\varphi \longrightarrow \psi$ and
 A2: $\text{ch} \longleftrightarrow \psi$
shows $\varphi \longrightarrow \text{ch}$
<proof>

lemma MMI_mpanl1: assumes A1: φ and
 A2: $((\varphi \wedge \psi) \wedge \text{ch}) \longrightarrow \vartheta$
shows $(\psi \wedge \text{ch}) \longrightarrow \vartheta$
<proof>

lemma MMI_a1i: assumes A1: φ
shows $\psi \longrightarrow \varphi$
<proof>

lemma (in MMIsar0) MMI_opreqan12rd: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $\psi \longrightarrow C = D$
shows
 $(\psi \wedge \varphi) \longrightarrow (A + C) = (B + D)$
 $(\psi \wedge \varphi) \longrightarrow (A \cdot C) = (B \cdot D)$
 $(\psi \wedge \varphi) \longrightarrow (A - C) = (B - D)$
 $(\psi \wedge \varphi) \longrightarrow (A / C) = (B / D)$
<proof>

lemma MMI_3adantl3: assumes A1: $((\varphi \wedge \psi) \wedge \text{ch}) \longrightarrow \vartheta$
shows $((\varphi \wedge \psi \wedge \tau) \wedge \text{ch}) \longrightarrow \vartheta$
<proof>

lemma MMI_sylbi: assumes A1: $\varphi \longleftrightarrow \psi$ and
 A2: $\psi \longrightarrow \text{ch}$
shows $\varphi \longrightarrow \text{ch}$
<proof>

lemma MMI_eirr:
shows $\neg (A \in A)$
<proof>

lemma MMI_eleq1i: assumes A1: $A = B$
shows $A \in C \longleftrightarrow B \in C$
<proof>

lemma MMI_mtbir: assumes A1: $\neg (\psi)$ and
 A2: $\varphi \longleftrightarrow \psi$
shows $\neg (\varphi)$
<proof>

lemma MMI_mto: assumes A1: $\neg (\psi)$ and
 A2: $\varphi \longrightarrow \psi$

shows $\neg (\varphi)$
 $\langle proof \rangle$

lemma MMI_df_nel:
shows $(A \notin B \longleftrightarrow \neg (A \in B))$
 $\langle proof \rangle$

lemma MMI_snid: **assumes** A1: A isASet
shows $A \in \{ A \}$
 $\langle proof \rangle$

lemma MMI_en2lp:
shows $\neg (A \in B \wedge B \in A)$
 $\langle proof \rangle$

lemma MMI_imnan:
shows $(\varphi \longrightarrow \neg (\psi)) \longleftrightarrow \neg ((\varphi \wedge \psi))$
 $\langle proof \rangle$

lemma MMI_sseqtr4: **assumes** A1: $A \subseteq B$ and
A2: $C = B$
shows $A \subseteq C$
 $\langle proof \rangle$

lemma MMI_ssun1:
shows $A \subseteq (A \cup B)$
 $\langle proof \rangle$

lemma MMI_ibar:
shows $\varphi \longrightarrow (\psi \longleftrightarrow (\varphi \wedge \psi))$
 $\langle proof \rangle$

lemma MMI_mtбири: **assumes** Amin: $\neg (ch)$ and
Amaj: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$
shows $\varphi \longrightarrow \neg (\psi)$
 $\langle proof \rangle$

lemma MMI_con2i: **assumes** Aa: $\varphi \longrightarrow \neg (\psi)$
shows $\psi \longrightarrow \neg (\varphi)$
 $\langle proof \rangle$

lemma MMI_intnand: **assumes** A1: $\varphi \longrightarrow \neg (\psi)$
shows $\varphi \longrightarrow \neg ((ch \wedge \psi))$
 $\langle proof \rangle$

lemma MMI_intnanrd: **assumes** A1: $\varphi \longrightarrow \neg (\psi)$
shows $\varphi \longrightarrow \neg ((\psi \wedge ch))$

$\langle proof \rangle$

lemma MMI_biorf:

shows $\neg (\varphi) \longrightarrow (\psi \longleftrightarrow (\varphi \vee \psi))$

$\langle proof \rangle$

lemma MMI_bitr2d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ **and**

A2: $\varphi \longrightarrow (\text{ch} \longleftrightarrow \vartheta)$

shows $\varphi \longrightarrow (\vartheta \longleftrightarrow \psi)$

$\langle proof \rangle$

lemma MMI_orass:

shows $((\varphi \vee \psi) \vee \text{ch}) \longleftrightarrow (\varphi \vee (\psi \vee \text{ch}))$

$\langle proof \rangle$

lemma MMI_orcom:

shows $(\varphi \vee \psi) \longleftrightarrow (\psi \vee \varphi)$

$\langle proof \rangle$

lemma MMI_3bitr4d: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ **and**

A2: $\varphi \longrightarrow (\vartheta \longleftrightarrow \psi)$ **and**

A3: $\varphi \longrightarrow (\tau \longleftrightarrow \text{ch})$

shows $\varphi \longrightarrow (\vartheta \longleftrightarrow \tau)$

$\langle proof \rangle$

lemma MMI_3imtr4d: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow \text{ch})$ **and**

A2: $\varphi \longrightarrow (\vartheta \longleftrightarrow \psi)$ **and**

A3: $\varphi \longrightarrow (\tau \longleftrightarrow \text{ch})$

shows $\varphi \longrightarrow (\vartheta \longrightarrow \tau)$

$\langle proof \rangle$

lemma MMI_3impdi: assumes A1: $((\varphi \wedge \psi) \wedge (\varphi \wedge \text{ch})) \longrightarrow \vartheta$

shows $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$

$\langle proof \rangle$

lemma MMI_bi2anan9: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ **and**

A2: $\vartheta \longrightarrow (\tau \longleftrightarrow \eta)$

shows $(\varphi \wedge \vartheta) \longrightarrow ((\psi \wedge \tau) \longleftrightarrow (\text{ch} \wedge \eta))$

$\langle proof \rangle$

lemma MMI_ssel2:

shows $((A \subseteq B \wedge C \in A) \longrightarrow C \in B)$

$\langle proof \rangle$

lemma MMI_an1rs: assumes A1: $((\varphi \wedge \psi) \wedge \text{ch}) \longrightarrow \vartheta$
 shows $((\varphi \wedge \text{ch}) \wedge \psi) \longrightarrow \vartheta$
 $\langle \text{proof} \rangle$

lemma MMI_ralbidva: assumes A1: $\forall x. (\varphi \wedge x \in A) \longrightarrow (\psi(x) \longleftrightarrow \text{ch}(x))$
 shows $\varphi \longrightarrow ((\forall x \in A . \psi(x)) \longleftrightarrow (\forall x \in A . \text{ch}(x)))$
 $\langle \text{proof} \rangle$

lemma MMI_rexbidva: assumes A1: $\forall x. (\varphi \wedge x \in A) \longrightarrow (\psi(x) \longleftrightarrow \text{ch}(x))$
 shows $\varphi \longrightarrow ((\exists x \in A . \psi(x)) \longleftrightarrow (\exists x \in A . \text{ch}(x)))$
 $\langle \text{proof} \rangle$

lemma MMI_con2bid: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \neg (\text{ch}))$
 shows $\varphi \longrightarrow (\text{ch} \longleftrightarrow \neg (\psi))$
 $\langle \text{proof} \rangle$

lemma MMI_so: assumes
 A1: $\forall x y z. (x \in A \wedge y \in A \wedge z \in A) \longrightarrow$
 $((\langle x, y \rangle \in R \longleftrightarrow \neg ((x = y \vee \langle y, x \rangle \in R))) \wedge$
 $((\langle x, y \rangle \in R \wedge \langle y, z \rangle \in R) \longrightarrow \langle x, z \rangle \in R))$
 shows R Orders A
 $\langle \text{proof} \rangle$

lemma MMI_con1bid: assumes A1: $\varphi \longrightarrow (\neg (\psi) \longleftrightarrow \text{ch})$
 shows $\varphi \longrightarrow (\neg (\text{ch}) \longleftrightarrow \psi)$
 $\langle \text{proof} \rangle$

lemma MMI_sotrieq:
 shows $((R \text{ Orders } A) \wedge (B \in A \wedge C \in A)) \longrightarrow$
 $(B = C \longleftrightarrow \neg ((\langle B, C \rangle \in R \vee \langle C, B \rangle \in R)))$
 $\langle \text{proof} \rangle$

lemma MMI_bicomd: assumes A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$
 shows $\varphi \longrightarrow (\text{ch} \longleftrightarrow \psi)$
 $\langle \text{proof} \rangle$

lemma MMI_sotrieq2:
 shows $(R \text{ Orders } A \wedge (B \in A \wedge C \in A)) \longrightarrow$
 $(B = C \longleftrightarrow (\neg (\langle B, C \rangle \in R) \wedge \neg (\langle C, B \rangle \in R)))$
 $\langle \text{proof} \rangle$

lemma MMI_orc:

shows $\varphi \longrightarrow (\varphi \vee \psi)$
 $\langle proof \rangle$

lemma MMI_syl6bbr: **assumes** A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ **and**
 A2: $\vartheta \longleftrightarrow ch$
shows $\varphi \longrightarrow (\psi \longleftrightarrow \vartheta)$
 $\langle proof \rangle$

lemma MMI_orbi1i: **assumes** A1: $\varphi \longleftrightarrow \psi$
shows $(\varphi \vee ch) \longleftrightarrow (\psi \vee ch)$
 $\langle proof \rangle$

lemma MMI_syl5rbbr: **assumes** A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$ **and**
 A2: $\psi \longleftrightarrow \vartheta$
shows $\varphi \longrightarrow (ch \longleftrightarrow \vartheta)$
 $\langle proof \rangle$

lemma MMI_anbi2d: **assumes** A1: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$
shows $\varphi \longrightarrow ((\vartheta \wedge \psi) \longleftrightarrow (\vartheta \wedge ch))$
 $\langle proof \rangle$

lemma MMI_ord: **assumes** A1: $\varphi \longrightarrow (\psi \vee ch)$
shows $\varphi \longrightarrow (\neg (\psi) \longrightarrow ch)$
 $\langle proof \rangle$

lemma MMI_impbid: **assumes** A1: $\varphi \longrightarrow (\psi \longrightarrow ch)$ **and**
 A2: $\varphi \longrightarrow (ch \longrightarrow \psi)$
shows $\varphi \longrightarrow (\psi \longleftrightarrow ch)$
 $\langle proof \rangle$

lemma MMI_jcad: **assumes** A1: $\varphi \longrightarrow (\psi \longrightarrow ch)$ **and**
 A2: $\varphi \longrightarrow (\psi \longrightarrow \vartheta)$
shows $\varphi \longrightarrow (\psi \longrightarrow (ch \wedge \vartheta))$
 $\langle proof \rangle$

lemma MMI_ax_1:
shows $\varphi \longrightarrow (\psi \longrightarrow \varphi)$
 $\langle proof \rangle$

lemma MMI_pm2_24:
shows $\varphi \longrightarrow (\neg (\varphi) \longrightarrow \psi)$
 $\langle proof \rangle$

lemma MMI_imp3a: **assumes** A1: $\varphi \longrightarrow (\psi \longrightarrow (ch \longrightarrow \vartheta))$
shows $\varphi \longrightarrow ((\psi \wedge ch) \longrightarrow \vartheta)$
 $\langle proof \rangle$

```

lemma (in MMIIsar0) MMI_breq1:
  shows
     $A = B \longrightarrow (A \leq C \longleftrightarrow B \leq C)$ 
     $A = B \longrightarrow (A < C \longleftrightarrow B < C)$ 
  <proof>

lemma MMI_biimprd: assumes A1:  $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ 
  shows  $\varphi \longrightarrow (\text{ch} \longrightarrow \psi)$ 
  <proof>

lemma MMI_jaod: assumes A1:  $\varphi \longrightarrow (\psi \longrightarrow \text{ch})$  and
  A2:  $\varphi \longrightarrow (\vartheta \longrightarrow \text{ch})$ 
  shows  $\varphi \longrightarrow ((\psi \vee \vartheta) \longrightarrow \text{ch})$ 
  <proof>

lemma MMI_com23: assumes A1:  $\varphi \longrightarrow (\psi \longrightarrow (\text{ch} \longrightarrow \vartheta))$ 
  shows  $\varphi \longrightarrow (\text{ch} \longrightarrow (\psi \longrightarrow \vartheta))$ 
  <proof>

lemma (in MMIIsar0) MMI_breq2:
  shows
     $A = B \longrightarrow (C \leq A \longleftrightarrow C \leq B)$ 
     $A = B \longrightarrow (C < A \longleftrightarrow C < B)$ 
  <proof>

lemma MMI_syld: assumes A1:  $\varphi \longrightarrow (\psi \longrightarrow \text{ch})$  and
  A2:  $\varphi \longrightarrow (\text{ch} \longrightarrow \vartheta)$ 
  shows  $\varphi \longrightarrow (\psi \longrightarrow \vartheta)$ 
  <proof>

lemma MMI_biimpcd: assumes A1:  $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ 
  shows  $\psi \longrightarrow (\varphi \longrightarrow \text{ch})$ 
  <proof>

lemma MMI_mp2and: assumes A1:  $\varphi \longrightarrow \psi$  and
  A2:  $\varphi \longrightarrow \text{ch}$  and
  A3:  $\varphi \longrightarrow ((\psi \wedge \text{ch}) \longrightarrow \vartheta)$ 
  shows  $\varphi \longrightarrow \vartheta$ 
  <proof>

lemma MMI_sonr:
  shows  $(R \text{ Orders } A \wedge B \in A) \longrightarrow \neg (\langle B, B \rangle \in R)$ 
  <proof>

lemma MMI_orri: assumes A1:  $\neg (\varphi) \longrightarrow \psi$ 
  shows  $\varphi \vee \psi$ 
  <proof>

```


lemma MMI_mpbiri: assumes Amin: ch and
 Amaj: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$
shows $\varphi \longrightarrow \psi$
 <proof>

lemma MMI_pm2_46:
shows $\neg ((\varphi \vee \psi)) \longrightarrow \neg (\psi)$
 <proof>

lemma MMI_elun:
shows $A \in (B \cup C) \longleftrightarrow (A \in B \vee A \in C)$
 <proof>

lemma (in MMIsar0) MMI_pnfxr:
shows $+\infty \in \mathbb{R}^*$
 <proof>

lemma MMI_elisseti: assumes A1: $A \in B$
shows A isASet
 <proof>

lemma (in MMIsar0) MMI_mnfxr:
shows $-\infty \in \mathbb{R}^*$
 <proof>

lemma MMI_elpr2: assumes A1: B isASet and
 A2: C isASet
shows $A \in \{B, C\} \longleftrightarrow (A = B \vee A = C)$
 <proof>

lemma MMI_orbi2i: assumes A1: $\varphi \longleftrightarrow \psi$
shows $(\text{ch} \vee \varphi) \longleftrightarrow (\text{ch} \vee \psi)$
 <proof>

lemma MMI_3orass:
shows $(\varphi \vee \psi \vee \text{ch}) \longleftrightarrow (\varphi \vee (\psi \vee \text{ch}))$
 <proof>

lemma MMI_bitr4: assumes A1: $\varphi \longleftrightarrow \psi$ and
 A2: $\text{ch} \longleftrightarrow \psi$
shows $\varphi \longleftrightarrow \text{ch}$
 <proof>

lemma MMI_eleq2:
shows $A = B \longrightarrow (C \in A \longleftrightarrow C \in B)$
 <proof>

lemma MMI_nelneq:

shows $(A \in C \wedge \neg (B \in C)) \longrightarrow \neg (A = B)$
 $\langle proof \rangle$

lemma MMI_df_pr:
shows $\{A, B\} = (\{A\} \cup \{B\})$
 $\langle proof \rangle$

lemma MMI_ineq2i: **assumes** A1: $A = B$
shows $(C \cap A) = (C \cap B)$
 $\langle proof \rangle$

lemma MMI_mt2: **assumes** A1: ψ and
A2: $\varphi \longrightarrow \neg (\psi)$
shows $\neg (\varphi)$
 $\langle proof \rangle$

lemma MMI_disjsn:
shows $(A \cap \{B\}) = 0 \longleftrightarrow \neg (B \in A)$
 $\langle proof \rangle$

lemma MMI_undisj2:
shows $((A \cap B) = 0 \wedge (A \cap C) = 0) \longleftrightarrow (A \cap (B \cup C)) = 0$
 $\langle proof \rangle$

lemma MMI_disjssun:
shows $((A \cap B) = 0 \longrightarrow (A \subseteq (B \cup C) \longleftrightarrow A \subseteq C))$
 $\langle proof \rangle$

lemma MMI_uncom:
shows $(A \cup B) = (B \cup A)$
 $\langle proof \rangle$

lemma MMI_sseq2i: **assumes** A1: $A = B$
shows $(C \subseteq A \longleftrightarrow C \subseteq B)$
 $\langle proof \rangle$

lemma MMI_disj:
shows $(A \cap B) = 0 \longleftrightarrow (\forall x \in A. \neg (x \in B))$
 $\langle proof \rangle$

lemma MMI_syl5ibr: **assumes** A1: $\varphi \longrightarrow (\psi \longrightarrow ch)$ and
A2: $\psi \longleftrightarrow \vartheta$
shows $\varphi \longrightarrow (\vartheta \longrightarrow ch)$
 $\langle proof \rangle$

lemma MMI_con3d: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow \text{ch})$
 shows $\varphi \longrightarrow (\neg (\text{ch}) \longrightarrow \neg (\psi))$
 $\langle \text{proof} \rangle$

lemma MMI_dfrex2:
 shows $(\exists x \in A . \varphi(x)) \longleftrightarrow \neg (\neg (\forall x \in A . \neg \varphi(x)))$
 $\langle \text{proof} \rangle$

lemma MMI_visset:
 shows $x \text{ isASet}$
 $\langle \text{proof} \rangle$

lemma MMI_elpr: assumes A1: $A \text{ isASet}$
 shows $A \in \{B, C\} \longleftrightarrow (A = B \vee A = C)$
 $\langle \text{proof} \rangle$

lemma MMI_rexbii: assumes A1: $\forall x. \varphi(x) \longleftrightarrow \psi(x)$
 shows $(\exists x \in A . \varphi(x)) \longleftrightarrow (\exists x \in A . \psi(x))$
 $\langle \text{proof} \rangle$

lemma MMI_r19_43:
 shows $(\exists x \in A . (\varphi(x) \vee \psi(x))) \longleftrightarrow$
 $(\neg (\neg (\exists x \in A . \varphi(x)) \vee \neg (\exists x \in A . \psi(x))))$
 $\langle \text{proof} \rangle$

lemma MMI_exancom:
 shows $(\exists x . (\varphi(x) \wedge \psi(x))) \longleftrightarrow$
 $(\exists x . (\psi(x) \wedge \varphi(x)))$
 $\langle \text{proof} \rangle$

lemma MMI_ceqsexv: assumes A1: $A \text{ isASet}$ and
 A2: $\forall x. x = A \longrightarrow (\varphi(x) \longleftrightarrow \psi(x))$
 shows $(\exists x . (x = A \wedge \varphi(x))) \longleftrightarrow \psi(A)$
 $\langle \text{proof} \rangle$

lemma MMI_orbi12i_orig: assumes A1: $\varphi \longleftrightarrow \psi$ and
 A2: $\text{ch} \longleftrightarrow \vartheta$
 shows $(\varphi \vee \text{ch}) \longleftrightarrow (\psi \vee \vartheta)$
 $\langle \text{proof} \rangle$

lemma MMI_orbi12i: assumes A1: $(\exists x. \varphi(x)) \longleftrightarrow \psi$ and
 A2: $(\exists x. \text{ch}(x)) \longleftrightarrow \vartheta$
 shows $(\exists x. \varphi(x)) \vee (\exists x. \text{ch}(x)) \longleftrightarrow (\psi \vee \vartheta)$
 $\langle \text{proof} \rangle$

lemma MMI_syl6ib: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow \text{ch})$ and

```

    A2: ch  $\longleftrightarrow$   $\vartheta$ 
  shows  $\varphi \longrightarrow ( \psi \longrightarrow \vartheta )$ 
  <proof>

lemma MMI_intnan: assumes A1:  $\neg ( \varphi )$ 
  shows  $\neg ( ( \psi \wedge \varphi ) )$ 
  <proof>

lemma MMI_intnanr: assumes A1:  $\neg ( \varphi )$ 
  shows  $\neg ( ( \varphi \wedge \psi ) )$ 
  <proof>

lemma MMI_pm3_2ni: assumes A1:  $\neg ( \varphi )$  and
  A2:  $\neg ( \psi )$ 
  shows  $\neg ( ( \varphi \vee \psi ) )$ 
  <proof>

lemma (in MMIisar0) MMI_breq12:
  shows
    (  $A = B \wedge C = D$  )  $\longrightarrow$  (  $A < C \longleftrightarrow B < D$  )
    (  $A = B \wedge C = D$  )  $\longrightarrow$  (  $A \leq C \longleftrightarrow B \leq D$  )
  <proof>

lemma MMI_necom:
  shows  $A \neq B \longleftrightarrow B \neq A$ 
  <proof>

lemma MMI_3jaoi: assumes A1:  $\varphi \longrightarrow \psi$  and
  A2:  $ch \longrightarrow \psi$  and
  A3:  $\vartheta \longrightarrow \psi$ 
  shows (  $\varphi \vee ch \vee \vartheta$  )  $\longrightarrow \psi$ 
  <proof>

lemma MMI_jctr: assumes A1:  $\psi$ 
  shows  $\varphi \longrightarrow ( \varphi \wedge \psi )$ 
  <proof>

lemma MMI_olc:
  shows  $\varphi \longrightarrow ( \psi \vee \varphi )$ 
  <proof>

lemma MMI_3syl: assumes A1:  $\varphi \longrightarrow \psi$  and
  A2:  $\psi \longrightarrow ch$  and
  A3:  $ch \longrightarrow \vartheta$ 
  shows  $\varphi \longrightarrow \vartheta$ 
  <proof>

```

lemma MMI_mtbird: assumes Amin: $\varphi \longrightarrow \neg (ch)$ and
 Amaj: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$
 shows $\varphi \longrightarrow \neg (\psi)$
<proof>

lemma MMI_pm2_21d: assumes A1: $\varphi \longrightarrow \neg (\psi)$
 shows $\varphi \longrightarrow (\psi \longrightarrow ch)$
<proof>

lemma MMI_3jaodan: assumes A1: $(\varphi \wedge \psi) \longrightarrow ch$ and
 A2: $(\varphi \wedge \vartheta) \longrightarrow ch$ and
 A3: $(\varphi \wedge \tau) \longrightarrow ch$
 shows $(\varphi \wedge (\psi \vee \vartheta \vee \tau)) \longrightarrow ch$
<proof>

lemma MMI_sylan2br: assumes A1: $(\varphi \wedge \psi) \longrightarrow ch$ and
 A2: $\psi \longleftrightarrow \vartheta$
 shows $(\varphi \wedge \vartheta) \longrightarrow ch$
<proof>

lemma MMI_3jaoian: assumes A1: $(\varphi \wedge \psi) \longrightarrow ch$ and
 A2: $(\vartheta \wedge \psi) \longrightarrow ch$ and
 A3: $(\tau \wedge \psi) \longrightarrow ch$
 shows $((\varphi \vee \vartheta \vee \tau) \wedge \psi) \longrightarrow ch$
<proof>

lemma MMI_mtbid: assumes Amin: $\varphi \longrightarrow \neg (\psi)$ and
 Amaj: $\varphi \longrightarrow (\psi \longleftrightarrow ch)$
 shows $\varphi \longrightarrow \neg (ch)$
<proof>

lemma MMI_con1d: assumes A1: $\varphi \longrightarrow (\neg (\psi) \longrightarrow ch)$
 shows $\varphi \longrightarrow (\neg (ch) \longrightarrow \psi)$
<proof>

lemma MMI_pm2_21nd: assumes A1: $\varphi \longrightarrow \psi$
 shows $\varphi \longrightarrow (\neg (\psi) \longrightarrow ch)$
<proof>

lemma MMI_syl3an1b: assumes A1: $(\varphi \wedge \psi \wedge ch) \longrightarrow \vartheta$ and
 A2: $\tau \longleftrightarrow \varphi$
 shows $(\tau \wedge \psi \wedge ch) \longrightarrow \vartheta$
<proof>

lemma MMI_adantld: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow ch)$
 shows $\varphi \longrightarrow ((\vartheta \wedge \psi) \longrightarrow ch)$
<proof>

lemma MMI_adantrd: assumes A1: $\varphi \longrightarrow (\psi \longrightarrow ch)$

shows $\varphi \longrightarrow ((\psi \wedge \vartheta) \longrightarrow \text{ch})$
<proof>

lemma MMI_anasss: **assumes** A1: $((\varphi \wedge \psi) \wedge \text{ch}) \longrightarrow \vartheta$
shows $(\varphi \wedge (\psi \wedge \text{ch})) \longrightarrow \vartheta$
<proof>

lemma MMI_syl3an3b: **assumes** A1: $(\varphi \wedge \psi \wedge \text{ch}) \longrightarrow \vartheta$ **and**
A2: $\tau \longleftrightarrow \text{ch}$
shows $(\varphi \wedge \psi \wedge \tau) \longrightarrow \vartheta$
<proof>

lemma MMI_mpbid: **assumes** Amin: $\varphi \longrightarrow \psi$ **and**
Amaj: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$
shows $\varphi \longrightarrow \text{ch}$
<proof>

lemma MMI_orbi12d: **assumes** A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$ **and**
A2: $\varphi \longrightarrow (\vartheta \longleftrightarrow \tau)$
shows $\varphi \longrightarrow ((\psi \vee \vartheta) \longleftrightarrow (\text{ch} \vee \tau))$
<proof>

lemma MMI_ianor:
shows $\neg (\varphi \wedge \psi) \longleftrightarrow \neg \varphi \vee \neg \psi$
<proof>

lemma MMI_bitr2: **assumes** A1: $\varphi \longleftrightarrow \psi$ **and**
A2: $\psi \longleftrightarrow \text{ch}$
shows $\text{ch} \longleftrightarrow \varphi$
<proof>

lemma MMI_biimp: **assumes** A1: $\varphi \longleftrightarrow \psi$
shows $\varphi \longrightarrow \psi$
<proof>

lemma MMI_mpan2d: **assumes** A1: $\varphi \longrightarrow \text{ch}$ **and**
A2: $\varphi \longrightarrow ((\psi \wedge \text{ch}) \longrightarrow \vartheta)$
shows $\varphi \longrightarrow (\psi \longrightarrow \vartheta)$
<proof>

lemma MMI_ad2antrr: **assumes** A1: $\varphi \longrightarrow \psi$
shows $((\varphi \wedge \text{ch}) \wedge \vartheta) \longrightarrow \psi$
<proof>

lemma MMI_biimpac: **assumes** A1: $\varphi \longrightarrow (\psi \longleftrightarrow \text{ch})$
shows $(\psi \wedge \varphi) \longrightarrow \text{ch}$

<proof>

lemma MMI_con2bii: **assumes** A1: $\varphi \longleftrightarrow \neg (\psi)$
shows $\psi \longleftrightarrow \neg (\varphi)$
<proof>

lemma MMI_pm3_26bd: **assumes** A1: $\varphi \longleftrightarrow (\psi \wedge \text{ch})$
shows $\varphi \longrightarrow \psi$
<proof>

lemma MMI_biimpr: **assumes** A1: $\varphi \longleftrightarrow \psi$
shows $\psi \longrightarrow \varphi$
<proof>

lemma (in MMIsar0) MMI_3brtr3g: **assumes** A1: $\varphi \longrightarrow A < B$ **and**
A2: $A = C$ **and**
A3: $B = D$
shows $\varphi \longrightarrow C < D$
<proof>

lemma (in MMIsar0) MMI_breq12i: **assumes** A1: $A = B$ **and**
A2: $C = D$
shows
 $A < C \longleftrightarrow B < D$
 $A \leq C \longleftrightarrow B \leq D$
<proof>

lemma MMI_negbii: **assumes** Aa: $\varphi \longleftrightarrow \psi$
shows $\neg\varphi \longleftrightarrow \neg\psi$
<proof>

lemma (in MMIsar0) MMI_breq1i: **assumes** A1: $A = B$
shows
 $A < C \longleftrightarrow B < C$
 $A \leq C \longleftrightarrow B \leq C$
<proof>

lemma MMI_syl5eqr: **assumes** A1: $\varphi \longrightarrow A = B$ **and**
A2: $A = C$
shows $\varphi \longrightarrow C = B$
<proof>

lemma (in MMIisar0) MMI_breq2d: **assumes** A1: $\varphi \longrightarrow A = B$
shows
 $\varphi \longrightarrow C < A \longleftrightarrow C < B$
 $\varphi \longrightarrow C \leq A \longleftrightarrow C \leq B$
<proof>

lemma MMI_ccase: **assumes** A1: $\varphi \wedge \psi \longrightarrow \tau$ **and**
A2: $\text{ch} \wedge \psi \longrightarrow \tau$ **and**
A3: $\varphi \wedge \vartheta \longrightarrow \tau$ **and**
A4: $\text{ch} \wedge \vartheta \longrightarrow \tau$
shows $(\varphi \vee \text{ch}) \wedge (\psi \vee \vartheta) \longrightarrow \tau$
<proof>

lemma MMI_pm3_27bd: **assumes** A1: $\varphi \longleftrightarrow \psi \wedge \text{ch}$
shows $\varphi \longrightarrow \text{ch}$
<proof>

lemma MMI_nsyl3: **assumes** A1: $\varphi \longrightarrow \neg\psi$ **and**
A2: $\text{ch} \longrightarrow \psi$
shows $\text{ch} \longrightarrow \neg\varphi$
<proof>

lemma MMI_jctild: **assumes** A1: $\varphi \longrightarrow \psi \longrightarrow \text{ch}$ **and**
A2: $\varphi \longrightarrow \vartheta$
shows $\varphi \longrightarrow$
 $\psi \longrightarrow \vartheta \wedge \text{ch}$
<proof>

lemma MMI_jctird: **assumes** A1: $\varphi \longrightarrow \psi \longrightarrow \text{ch}$ **and**
A2: $\varphi \longrightarrow \vartheta$
shows $\varphi \longrightarrow$
 $\psi \longrightarrow \text{ch} \wedge \vartheta$
<proof>

lemma MMI_ccase2: **assumes** A1: $\varphi \wedge \psi \longrightarrow \tau$ **and**
A2: $\text{ch} \longrightarrow \tau$ **and**
A3: $\vartheta \longrightarrow \tau$
shows $(\varphi \vee \text{ch}) \wedge (\psi \vee \vartheta) \longrightarrow \tau$
<proof>

lemma MMI_3bitr3r: **assumes** A1: $\varphi \longleftrightarrow \psi$ **and**
A2: $\varphi \longleftrightarrow \text{ch}$ **and**
A3: $\psi \longleftrightarrow \vartheta$
shows $\vartheta \longleftrightarrow \text{ch}$
<proof>

lemma (in MMIisar0) MMI_syl6breq: **assumes** A1: $\varphi \longrightarrow A < B$ **and**
 A2: $B = C$
shows
 $\varphi \longrightarrow A < C$
 $\langle proof \rangle$

lemma MMI_pm2_61i: **assumes** A1: $\varphi \longrightarrow \psi$ **and**
 A2: $\neg\varphi \longrightarrow \psi$
shows ψ
 $\langle proof \rangle$

lemma MMI_syl6req: **assumes** A1: $\varphi \longrightarrow A = B$ **and**
 A2: $B = C$
shows $\varphi \longrightarrow C = A$
 $\langle proof \rangle$

lemma MMI_pm2_61d: **assumes** A1: $\varphi \longrightarrow \psi \longrightarrow \text{ch}$ **and**
 A2: $\varphi \longrightarrow$
 $\neg\psi \longrightarrow \text{ch}$
shows $\varphi \longrightarrow \text{ch}$
 $\langle proof \rangle$

lemma MMI_orim1d: **assumes** A1: $\varphi \longrightarrow \psi \longrightarrow \text{ch}$
shows $\varphi \longrightarrow$
 $\psi \vee \vartheta \longrightarrow \text{ch} \vee \vartheta$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_breq1d: **assumes** A1: $\varphi \longrightarrow A = B$
shows
 $\varphi \longrightarrow A < C \longleftrightarrow B < C$
 $\varphi \longrightarrow A \leq C \longleftrightarrow B \leq C$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_breq12d: **assumes** A1: $\varphi \longrightarrow A = B$ **and**
 A2: $\varphi \longrightarrow C = D$
shows
 $\varphi \longrightarrow A < C \longleftrightarrow B < D$
 $\varphi \longrightarrow A \leq C \longleftrightarrow B \leq D$
 $\langle proof \rangle$

lemma MMI_bibi2d: **assumes** A1: $\varphi \longrightarrow$
 $\psi \longleftrightarrow \text{ch}$
shows $\varphi \longrightarrow$
 $(\vartheta \longleftrightarrow \psi) \longleftrightarrow$
 $\vartheta \longleftrightarrow \text{ch}$
 $\langle proof \rangle$

lemma MMI_con4bid: **assumes** A1: $\varphi \longrightarrow$
 $\neg\psi \longleftrightarrow \neg\text{ch}$
shows $\varphi \longrightarrow$
 $\psi \longleftrightarrow \text{ch}$
 $\langle\text{proof}\rangle$

lemma MMI_3com13: **assumes** A1: $\varphi \wedge \psi \wedge \text{ch} \longrightarrow \vartheta$
shows $\text{ch} \wedge \psi \wedge \varphi \longrightarrow \vartheta$
 $\langle\text{proof}\rangle$

lemma MMI_3bitr3rd: **assumes** A1: $\varphi \longrightarrow$
 $\psi \longleftrightarrow \text{ch}$ **and**
A2: $\varphi \longrightarrow$
 $\psi \longleftrightarrow \vartheta$ **and**
A3: $\varphi \longrightarrow$
 $\text{ch} \longleftrightarrow \tau$
shows $\varphi \longrightarrow$
 $\tau \longleftrightarrow \vartheta$
 $\langle\text{proof}\rangle$

lemma MMI_3imtr4g: **assumes** A1: $\varphi \longrightarrow \psi \longrightarrow \text{ch}$ **and**
A2: $\vartheta \longleftrightarrow \psi$ **and**
A3: $\tau \longleftrightarrow \text{ch}$
shows $\varphi \longrightarrow$
 $\vartheta \longrightarrow \tau$
 $\langle\text{proof}\rangle$

lemma MMI_expcom: **assumes** A1: $\varphi \wedge \psi \longrightarrow \text{ch}$
shows $\psi \longrightarrow \varphi \longrightarrow \text{ch}$
 $\langle\text{proof}\rangle$

lemma (in MMIisar0) MMI_breq2i: **assumes** A1: $A = B$
shows
 $C < A \longleftrightarrow C < B$
 $C \leq A \longleftrightarrow C \leq B$
 $\langle\text{proof}\rangle$

lemma MMI_3bitr2r: **assumes** A1: $\varphi \longleftrightarrow \psi$ **and**
A2: $\text{ch} \longleftrightarrow \psi$ **and**
A3: $\text{ch} \longleftrightarrow \vartheta$
shows $\vartheta \longleftrightarrow \varphi$
 $\langle\text{proof}\rangle$

lemma MMI_dedth4h: **assumes** A1: $A = \text{if}(\varphi, A, R) \longrightarrow$
 $\tau \longleftrightarrow \eta$ **and**
A2: $B = \text{if}(\psi, B, S) \longrightarrow$

$\eta \longleftrightarrow \zeta$ **and**
 A3: $C = \text{if}(\text{ch}, C, F) \longrightarrow$
 $\zeta \longleftrightarrow \text{si}$ **and**
 A4: $D = \text{if}(\vartheta, D, G) \longrightarrow \text{si} \longleftrightarrow \text{rh}$ **and**
 A5: rh
shows $(\varphi \wedge \psi) \wedge \text{ch} \wedge \vartheta \longrightarrow \tau$
<proof>

lemma MMI_anbi1d: **assumes** A1: $\varphi \longrightarrow$
 $\psi \longleftrightarrow \text{ch}$
shows $\varphi \longrightarrow$
 $\psi \wedge \vartheta \longleftrightarrow \text{ch} \wedge \vartheta$
<proof>

lemma (in MMIsar0) MMI_breqtrrd: **assumes** A1: $\varphi \longrightarrow A < B$ **and**
 A2: $\varphi \longrightarrow C = B$
shows $\varphi \longrightarrow A < C$
<proof>

lemma MMI_syl3an: **assumes** A1: $\varphi \wedge \psi \wedge \text{ch} \longrightarrow \vartheta$ **and**
 A2: $\tau \longrightarrow \varphi$ **and**
 A3: $\eta \longrightarrow \psi$ **and**
 A4: $\zeta \longrightarrow \text{ch}$
shows $\tau \wedge \eta \wedge \zeta \longrightarrow \vartheta$
<proof>

lemma MMI_3bitrd: **assumes** A1: $\varphi \longrightarrow$
 $\psi \longleftrightarrow \text{ch}$ **and**
 A2: $\varphi \longrightarrow$
 $\text{ch} \longleftrightarrow \vartheta$ **and**
 A3: $\varphi \longrightarrow$
 $\vartheta \longleftrightarrow \tau$
shows $\varphi \longrightarrow$
 $\psi \longleftrightarrow \tau$
<proof>

lemma (in MMIsar0) MMI_breqtr: **assumes** A1: $A < B$ **and**
 A2: $B = C$
shows $A < C$
<proof>

lemma MMI_mpi: assumes A1: ψ and
 A2: $\varphi \longrightarrow \psi \longrightarrow \text{ch}$
shows $\varphi \longrightarrow \text{ch}$
 $\langle \text{proof} \rangle$

lemma MMI_eqtr2: assumes A1: $A = B$ and
 A2: $B = C$
shows $C = A$
 $\langle \text{proof} \rangle$

lemma MMI_eqneqi: assumes A1: $A = B \longleftrightarrow C = D$
shows $A \neq B \longleftrightarrow C \neq D$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_eqbrtrrd: assumes A1: $\varphi \longrightarrow A = B$ and
 A2: $\varphi \longrightarrow A < C$
shows $\varphi \longrightarrow B < C$
 $\langle \text{proof} \rangle$

lemma MMI_mpd: assumes A1: $\varphi \longrightarrow \psi$ and
 A2: $\varphi \longrightarrow \psi \longrightarrow \text{ch}$
shows $\varphi \longrightarrow \text{ch}$
 $\langle \text{proof} \rangle$

lemma MMI_mpdan: assumes A1: $\varphi \longrightarrow \psi$ and
 A2: $\varphi \wedge \psi \longrightarrow \text{ch}$
shows $\varphi \longrightarrow \text{ch}$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_breqtrd: assumes A1: $\varphi \longrightarrow A < B$ and
 A2: $\varphi \longrightarrow B = C$
shows $\varphi \longrightarrow A < C$
 $\langle \text{proof} \rangle$

lemma MMI_mpand: assumes A1: $\varphi \longrightarrow \psi$ and
 A2: $\varphi \longrightarrow$
 $\psi \wedge \text{ch} \longrightarrow \vartheta$
shows $\varphi \longrightarrow \text{ch} \longrightarrow \vartheta$
 $\langle \text{proof} \rangle$

lemma MMI_imbi1d: assumes A1: $\varphi \longrightarrow$
 $\psi \longleftrightarrow \text{ch}$
shows $\varphi \longrightarrow$
 $(\psi \longrightarrow \vartheta) \longleftrightarrow$
 $(\text{ch} \longrightarrow \vartheta)$
 $\langle \text{proof} \rangle$

lemma MMI_mtbii: assumes Amin: $\neg\psi$ and

Amaj: $\varphi \longrightarrow$

$\psi \longleftrightarrow \text{ch}$

shows $\varphi \longrightarrow \neg\text{ch}$

<proof>

lemma MMI_sylan2d: assumes A1: $\varphi \longrightarrow$

$\psi \wedge \text{ch} \longrightarrow \vartheta$ and

A2: $\varphi \longrightarrow \tau \longrightarrow \text{ch}$

shows $\varphi \longrightarrow$

$\psi \wedge \tau \longrightarrow \vartheta$

<proof>

lemma MMI_imp32: assumes A1: $\varphi \longrightarrow$

$\psi \longrightarrow \text{ch} \longrightarrow \vartheta$

shows $\varphi \wedge \psi \wedge \text{ch} \longrightarrow \vartheta$

<proof>

lemma (in MMIsar0) MMI_breqan12d: assumes A1: $\varphi \longrightarrow A = B$ and

A2: $\psi \longrightarrow C = D$

shows

$\varphi \wedge \psi \longrightarrow A < C \longleftrightarrow B < D$

$\varphi \wedge \psi \longrightarrow A \leq C \longleftrightarrow B \leq D$

<proof>

lemma MMI_aidd: assumes A1: $\varphi \longrightarrow \psi \longrightarrow \text{ch}$

shows $\varphi \longrightarrow$

$\psi \longrightarrow \vartheta \longrightarrow \text{ch}$

<proof>

lemma (in MMIsar0) MMI_3brtr3d: assumes A1: $\varphi \longrightarrow A \leq B$ and

A2: $\varphi \longrightarrow A = C$ and

A3: $\varphi \longrightarrow B = D$

shows $\varphi \longrightarrow C \leq D$

<proof>

lemma MMI_ad2antl1: assumes A1: $\varphi \longrightarrow \psi$

shows $\text{ch} \wedge \vartheta \wedge \varphi \longrightarrow \psi$

<proof>

lemma MMI_adantrrl: assumes A1: $\varphi \wedge \psi \wedge \text{ch} \longrightarrow \vartheta$

shows $\varphi \wedge \psi \wedge \tau \wedge \text{ch} \longrightarrow \vartheta$

<proof>

lemma MMI_syl2ani: assumes A1: $\varphi \longrightarrow$

$\psi \wedge \text{ch} \longrightarrow \vartheta$ and
 A2: $\tau \longrightarrow \psi$ and
 A3: $\eta \longrightarrow \text{ch}$
 shows $\varphi \longrightarrow$
 $\tau \wedge \eta \longrightarrow \vartheta$
 $\langle \text{proof} \rangle$

lemma MMI_im2anan9: assumes A1: $\varphi \longrightarrow \psi \longrightarrow \text{ch}$ and
 A2: $\vartheta \longrightarrow$
 $\tau \longrightarrow \eta$
 shows $\varphi \wedge \vartheta \longrightarrow$
 $\psi \wedge \tau \longrightarrow \text{ch} \wedge \eta$
 $\langle \text{proof} \rangle$

lemma MMI_ancomsd: assumes A1: $\varphi \longrightarrow$
 $\psi \wedge \text{ch} \longrightarrow \vartheta$
 shows $\varphi \longrightarrow$
 $\text{ch} \wedge \psi \longrightarrow \vartheta$
 $\langle \text{proof} \rangle$

lemma MMI_mpani: assumes A1: ψ and
 A2: $\varphi \longrightarrow$
 $\psi \wedge \text{ch} \longrightarrow \vartheta$
 shows $\varphi \longrightarrow \text{ch} \longrightarrow \vartheta$
 $\langle \text{proof} \rangle$

lemma MMI_syldan: assumes A1: $\varphi \wedge \psi \longrightarrow \text{ch}$ and
 A2: $\varphi \wedge \text{ch} \longrightarrow \vartheta$
 shows $\varphi \wedge \psi \longrightarrow \vartheta$
 $\langle \text{proof} \rangle$

lemma MMI_mp3anl1: assumes A1: φ and
 A2: $(\varphi \wedge \psi \wedge \text{ch}) \wedge \vartheta \longrightarrow \tau$
 shows $(\psi \wedge \text{ch}) \wedge \vartheta \longrightarrow \tau$
 $\langle \text{proof} \rangle$

lemma MMI_3ad2ant1: assumes A1: $\varphi \longrightarrow \text{ch}$
 shows $\varphi \wedge \psi \wedge \vartheta \longrightarrow \text{ch}$
 $\langle \text{proof} \rangle$

lemma MMI_pm3_2:
 shows $\varphi \longrightarrow$
 $\psi \longrightarrow \varphi \wedge \psi$
 $\langle \text{proof} \rangle$

lemma MMI_pm2_43i: assumes A1: $\varphi \longrightarrow$
 $\varphi \longrightarrow \psi$

shows $\varphi \longrightarrow \psi$
 $\langle proof \rangle$

lemma MMI_jctil: **assumes** A1: $\varphi \longrightarrow \psi$ **and**
 A2: ch
shows $\varphi \longrightarrow ch \wedge \psi$
 $\langle proof \rangle$

lemma MMI_mpanl12: **assumes** A1: φ **and**
 A2: ψ **and**
 A3: $(\varphi \wedge \psi) \wedge ch \longrightarrow \vartheta$
shows $ch \longrightarrow \vartheta$
 $\langle proof \rangle$

lemma MMI_mpanr1: **assumes** A1: ψ **and**
 A2: $\varphi \wedge \psi \wedge ch \longrightarrow \vartheta$
shows $\varphi \wedge ch \longrightarrow \vartheta$
 $\langle proof \rangle$

lemma MMI_ad2antrl: **assumes** A1: $\varphi \longrightarrow \psi$
shows $ch \wedge \varphi \wedge \vartheta \longrightarrow \psi$
 $\langle proof \rangle$

lemma MMI_3adant3r: **assumes** A1: $\varphi \wedge \psi \wedge ch \longrightarrow \vartheta$
shows $\varphi \wedge \psi \wedge ch \wedge \tau \longrightarrow \vartheta$
 $\langle proof \rangle$

lemma MMI_3adant1l: **assumes** A1: $\varphi \wedge \psi \wedge ch \longrightarrow \vartheta$
shows $(\tau \wedge \varphi) \wedge \psi \wedge ch \longrightarrow \vartheta$
 $\langle proof \rangle$

lemma MMI_3adant2r: **assumes** A1: $\varphi \wedge \psi \wedge ch \longrightarrow \vartheta$
shows $\varphi \wedge (\psi \wedge \tau) \wedge ch \longrightarrow \vartheta$
 $\langle proof \rangle$

lemma MMI_3bitr4rd: **assumes** A1: $\varphi \longrightarrow$
 $\psi \longleftrightarrow ch$ **and**
 A2: $\varphi \longrightarrow$
 $\vartheta \longleftrightarrow \psi$ **and**
 A3: $\varphi \longrightarrow$
 $\tau \longleftrightarrow ch$
shows $\varphi \longrightarrow$
 $\tau \longleftrightarrow \vartheta$
 $\langle proof \rangle$

lemma MMI_3anrev:

shows $\varphi \wedge \psi \wedge \text{ch} \longleftrightarrow \text{ch} \wedge \psi \wedge \varphi$

<proof>

lemma MMI_eqtr4: assumes A1: $A = B$ and

A2: $C = B$

shows $A = C$

<proof>

lemma MMI_anidm:

shows $\varphi \wedge \varphi \longleftrightarrow \varphi$

<proof>

lemma MMI_bi2anan9r: assumes A1: $\varphi \longrightarrow$

$\psi \longleftrightarrow \text{ch}$ and

A2: $\vartheta \longrightarrow$

$\tau \longleftrightarrow \eta$

shows $\vartheta \wedge \varphi \longrightarrow$

$\psi \wedge \tau \longleftrightarrow \text{ch} \wedge \eta$

<proof>

lemma MMI_3imtr3g: assumes A1: $\varphi \longrightarrow \psi \longrightarrow \text{ch}$ and

A2: $\psi \longleftrightarrow \vartheta$ and

A3: $\text{ch} \longleftrightarrow \tau$

shows $\varphi \longrightarrow$

$\vartheta \longrightarrow \tau$

<proof>

lemma MMI_a3d: assumes A1: $\varphi \longrightarrow$

$\neg\psi \longrightarrow \neg\text{ch}$

shows $\varphi \longrightarrow \text{ch} \longrightarrow \psi$

<proof>

lemma MMI_sylan9bbr: assumes A1: $\varphi \longrightarrow$

$\psi \longleftrightarrow \text{ch}$ and

A2: $\vartheta \longrightarrow$

$\text{ch} \longleftrightarrow \tau$

shows $\vartheta \wedge \varphi \longrightarrow$

$\psi \longleftrightarrow \tau$

<proof>

lemma MMI_sylan9bb: assumes A1: $\varphi \longrightarrow$

$\psi \longleftrightarrow \text{ch}$ and

A2: $\vartheta \longrightarrow$

$\text{ch} \longleftrightarrow \tau$

shows $\varphi \wedge \vartheta \longrightarrow$

$\psi \longleftrightarrow \tau$

<proof>

lemma MMI_3bitr3g: **assumes** A1: $\varphi \longrightarrow$
 $\psi \longleftrightarrow \text{ch}$ **and**
A2: $\psi \longleftrightarrow \vartheta$ **and**
A3: $\text{ch} \longleftrightarrow \tau$
shows $\varphi \longrightarrow$
 $\vartheta \longleftrightarrow \tau$
 $\langle \text{proof} \rangle$

lemma MMI_pm5_21:
shows $\neg\varphi \wedge \neg\psi \longrightarrow$
 $\varphi \longleftrightarrow \psi$
 $\langle \text{proof} \rangle$

lemma MMI_an6:
shows $(\varphi \wedge \psi \wedge \text{ch}) \wedge \vartheta \wedge \tau \wedge \eta \longleftrightarrow$
 $(\varphi \wedge \vartheta) \wedge (\psi \wedge \tau) \wedge \text{ch} \wedge \eta$
 $\langle \text{proof} \rangle$

lemma MMI_syl3anl1: **assumes** A1: $(\varphi \wedge \psi \wedge \text{ch}) \wedge \vartheta \longrightarrow \tau$ **and**
A2: $\eta \longrightarrow \varphi$
shows $(\eta \wedge \psi \wedge \text{ch}) \wedge \vartheta \longrightarrow \tau$
 $\langle \text{proof} \rangle$

lemma MMI_imp4a: **assumes** A1: $\varphi \longrightarrow$
 $\psi \longrightarrow$
 $\text{ch} \longrightarrow$
 $\vartheta \longrightarrow \tau$
shows $\varphi \longrightarrow$
 $\psi \longrightarrow$
 $\text{ch} \wedge \vartheta \longrightarrow \tau$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_breqan12rd: **assumes** A1: $\varphi \longrightarrow A = B$ **and**
A2: $\psi \longrightarrow C = D$
shows
 $\psi \wedge \varphi \longrightarrow A < C \longleftrightarrow B < D$
 $\psi \wedge \varphi \longrightarrow A \leq C \longleftrightarrow B \leq D$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_3brtr4d: **assumes** A1: $\varphi \longrightarrow A < B$ **and**
A2: $\varphi \longrightarrow C = A$ **and**
A3: $\varphi \longrightarrow D = B$
shows $\varphi \longrightarrow C < D$
 $\langle \text{proof} \rangle$

lemma MMI_adantrrr: assumes A1: $\varphi \wedge \psi \wedge \text{ch} \longrightarrow \vartheta$
 shows $\varphi \wedge \psi \wedge \text{ch} \wedge \tau \longrightarrow \vartheta$
<proof>

lemma MMI_adantrlr: assumes A1: $\varphi \wedge \psi \wedge \text{ch} \longrightarrow \vartheta$
 shows $\varphi \wedge (\psi \wedge \tau) \wedge \text{ch} \longrightarrow \vartheta$
<proof>

lemma MMI_imdistani: assumes A1: $\varphi \longrightarrow \psi \longrightarrow \text{ch}$
 shows $\varphi \wedge \psi \longrightarrow \varphi \wedge \text{ch}$
<proof>

lemma MMI_anabss3: assumes A1: $(\varphi \wedge \psi) \wedge \psi \longrightarrow \text{ch}$
 shows $\varphi \wedge \psi \longrightarrow \text{ch}$
<proof>

lemma MMI_mp3anl2: assumes A1: ψ and
 A2: $(\varphi \wedge \psi \wedge \text{ch}) \wedge \vartheta \longrightarrow \tau$
 shows $(\varphi \wedge \text{ch}) \wedge \vartheta \longrightarrow \tau$
<proof>

lemma MMI_mpanl2: assumes A1: ψ and
 A2: $(\varphi \wedge \psi) \wedge \text{ch} \longrightarrow \vartheta$
 shows $\varphi \wedge \text{ch} \longrightarrow \vartheta$
<proof>

lemma MMI_mpancom: assumes A1: $\psi \longrightarrow \varphi$ and
 A2: $\varphi \wedge \psi \longrightarrow \text{ch}$
 shows $\psi \longrightarrow \text{ch}$
<proof>

lemma MMI_or12:
 shows $\varphi \vee \psi \vee \text{ch} \longleftrightarrow \psi \vee \varphi \vee \text{ch}$
<proof>

lemma MMI_rcla4ev: assumes A1: $\forall x. x = A \longrightarrow \varphi(x) \longleftrightarrow \psi$
 shows $A \in B \wedge \psi \longrightarrow (\exists x \in B. \varphi(x))$
<proof>

lemma MMI_jctir: assumes A1: $\varphi \longrightarrow \psi$ and
 A2: ch
 shows $\varphi \longrightarrow \psi \wedge \text{ch}$
<proof>

lemma MMI_iffalse:
 shows $\neg \varphi \longrightarrow \text{if}(\varphi, A, B) = B$
<proof>

lemma MMI_iftrue:
 shows $\varphi \longrightarrow \text{if}(\varphi, A, B) = A$
<proof>

lemma MMI_pm2_61d2: assumes A1: $\varphi \longrightarrow$
 $\neg\psi \longrightarrow \text{ch}$ and
 A2: $\psi \longrightarrow \text{ch}$
 shows $\varphi \longrightarrow \text{ch}$
<proof>

lemma MMI_pm2_61dan: assumes A1: $\varphi \wedge \psi \longrightarrow \text{ch}$ and
 A2: $\varphi \wedge \neg\psi \longrightarrow \text{ch}$
 shows $\varphi \longrightarrow \text{ch}$
<proof>

lemma MMI_orcanai: assumes A1: $\varphi \longrightarrow \psi \vee \text{ch}$
 shows $\varphi \wedge \neg\psi \longrightarrow \text{ch}$
<proof>

lemma MMI_ifcl:
 shows $A \in C \wedge B \in C \longrightarrow \text{if}(\varphi, A, B) \in C$
<proof>

lemma MMI_imim2i: assumes A1: $\varphi \longrightarrow \psi$
 shows $(\text{ch} \longrightarrow \varphi) \longrightarrow \text{ch} \longrightarrow \psi$
<proof>

lemma MMI_com13: assumes A1: $\varphi \longrightarrow$
 $\psi \longrightarrow \text{ch} \longrightarrow \vartheta$
 shows $\text{ch} \longrightarrow$
 $\psi \longrightarrow$
 $\varphi \longrightarrow \vartheta$
<proof>

lemma MMI_rcla4v: assumes A1: $\forall x. x = A \longrightarrow \varphi(x) \longleftrightarrow \psi$
 shows $A \in B \longrightarrow (\forall x \in B. \varphi(x)) \longrightarrow \psi$
<proof>

lemma MMI_syl5d: assumes A1: $\varphi \longrightarrow$
 $\psi \longrightarrow \text{ch} \longrightarrow \vartheta$ and
 A2: $\varphi \longrightarrow \tau \longrightarrow \text{ch}$
 shows $\varphi \longrightarrow$
 $\psi \longrightarrow$
 $\tau \longrightarrow \vartheta$
<proof>

lemma MMI_eqcoms: assumes A1: $A = B \longrightarrow \varphi$
 shows $B = A \longrightarrow \varphi$

<proof>

lemma MMI_rgen: assumes A1: $\forall x. x \in A \longrightarrow \varphi(x)$
shows $\forall x \in A. \varphi(x)$
<proof>

lemma (in MMIsar0) MMI_reex:
shows $\mathbb{R} = \mathbb{R}$
<proof>

lemma MMI_sstri: assumes A1: $A \subseteq B$ and
A2: $B \subseteq C$
shows $A \subseteq C$
<proof>

lemma MMI_ssexi: assumes A1: $B = B$ and
A2: $A \subseteq B$
shows $A = A$
<proof>

end

111 Complex numbers in Metamatah - introduction

theory MMI_Complex_ZF imports MMI_logic_and_sets

begin

This theory contains theorems (with proofs) about complex numbers imported from the Metamath's set.mm database. The original Metamath proofs were mostly written by Norman Megill, see the Metamath Proof Explorer pages for full attribution. This theory contains about 200 theorems from "recnt" to "div11t".

lemma (in MMIsar0) MMI_recnt:
shows $A \in \mathbb{R} \longrightarrow A \in \mathbb{C}$
<proof>

lemma (in MMIsar0) MMI_recn: assumes A1: $A \in \mathbb{R}$
shows $A \in \mathbb{C}$
<proof>

lemma (in MMIsar0) MMI_recnd: assumes A1: $\varphi \longrightarrow A \in \mathbb{R}$
shows $\varphi \longrightarrow A \in \mathbb{C}$
<proof>

```

lemma (in MMIisar0) MMI_elimne0:
  shows if ( A  $\neq$  0 , A , 1 )  $\neq$  0
  <proof>

lemma (in MMIisar0) MMI_addex:
  shows + isASet
  <proof>

lemma (in MMIisar0) MMI_mulex:
  shows  $\cdot$  isASet
  <proof>

lemma (in MMIisar0) MMI_adddirt:
  shows ( A  $\in$   $\mathbb{C}$   $\wedge$  B  $\in$   $\mathbb{C}$   $\wedge$  C  $\in$   $\mathbb{C}$  )  $\longrightarrow$ 
  ( ( A + B )  $\cdot$  C ) = ( ( A  $\cdot$  C ) + ( B  $\cdot$  C ) )
  <proof>

lemma (in MMIisar0) MMI_addcl: assumes A1: A  $\in$   $\mathbb{C}$  and
  A2: B  $\in$   $\mathbb{C}$ 
  shows ( A + B )  $\in$   $\mathbb{C}$ 
  <proof>

lemma (in MMIisar0) MMI_mulcl: assumes A1: A  $\in$   $\mathbb{C}$  and
  A2: B  $\in$   $\mathbb{C}$ 
  shows ( A  $\cdot$  B )  $\in$   $\mathbb{C}$ 
  <proof>

lemma (in MMIisar0) MMI_addcom: assumes A1: A  $\in$   $\mathbb{C}$  and
  A2: B  $\in$   $\mathbb{C}$ 
  shows ( A + B ) = ( B + A )
  <proof>

lemma (in MMIisar0) MMI_mulcom: assumes A1: A  $\in$   $\mathbb{C}$  and
  A2: B  $\in$   $\mathbb{C}$ 
  shows ( A  $\cdot$  B ) = ( B  $\cdot$  A )
  <proof>

lemma (in MMIisar0) MMI_addass: assumes A1: A  $\in$   $\mathbb{C}$  and
  A2: B  $\in$   $\mathbb{C}$  and
  A3: C  $\in$   $\mathbb{C}$ 
  shows ( ( A + B ) + C ) = ( A + ( B + C ) )
  <proof>

lemma (in MMIisar0) MMI_mulass: assumes A1: A  $\in$   $\mathbb{C}$  and
  A2: B  $\in$   $\mathbb{C}$  and
  A3: C  $\in$   $\mathbb{C}$ 
  shows ( ( A  $\cdot$  B )  $\cdot$  C ) = ( A  $\cdot$  ( B  $\cdot$  C ) )
  <proof>

```

lemma (in MMIisar0) MMI_adddi: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $(A \cdot (B + C)) = ((A \cdot B) + (A \cdot C))$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_adddir: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $((A + B) \cdot C) = ((A \cdot C) + (B \cdot C))$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_1cn:
 shows $1 \in \mathbb{C}$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_0cn:
 shows $0 \in \mathbb{C}$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_addid1: assumes A1: $A \in \mathbb{C}$
 shows $(A + 0) = A$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_addid2: assumes A1: $A \in \mathbb{C}$
 shows $(0 + A) = A$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_mulid1: assumes A1: $A \in \mathbb{C}$
 shows $(A \cdot 1) = A$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_mulid2: assumes A1: $A \in \mathbb{C}$
 shows $(1 \cdot A) = A$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_negex: assumes A1: $A \in \mathbb{C}$
 shows $\exists x \in \mathbb{C} . (A + x) = 0$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_recex: assumes A1: $A \in \mathbb{C}$ and
 A2: $A \neq 0$
 shows $\exists x \in \mathbb{C} . (A \cdot x) = 1$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_readdcl: assumes A1: $A \in \mathbb{R}$ and
 A2: $B \in \mathbb{R}$
 shows $(A + B) \in \mathbb{R}$
<proof>

lemma (in MMIsar0) MMI_remulcl: assumes A1: $A \in \mathbb{R}$ and
 A2: $B \in \mathbb{R}$
 shows $(A \cdot B) \in \mathbb{R}$
<proof>

lemma (in MMIsar0) MMI_addcan: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $(A + B) = (A + C) \longleftrightarrow B = C$
<proof>

lemma (in MMIsar0) MMI_addcan2: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $(A + C) = (B + C) \longleftrightarrow A = B$
<proof>

lemma (in MMIsar0) MMI_addcant:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A + B) = (A + C) \longleftrightarrow B = C)$
<proof>

lemma (in MMIsar0) MMI_addcan2t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow ((A + C) = (B + C) \longleftrightarrow$
 $A = B)$
<proof>

lemma (in MMIsar0) MMI_add12t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow (A + (B + C)) =$
 $(B + (A + C))$
<proof>

lemma (in MMIsar0) MMI_add23t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow ((A + B) + C) =$
 $(A + (B + C))$
<proof>

lemma (in MMIsar0) MMI_add4t:
 shows $((A \in \mathbb{C} \wedge B \in \mathbb{C}) \wedge (C \in \mathbb{C} \wedge D \in \mathbb{C})) \longrightarrow$
 $((A + B) + (C + D)) = ((A + C) + (B + D))$
<proof>

lemma (in MMIsar0) MMI_add42t:
 shows $((A \in \mathbb{C} \wedge B \in \mathbb{C}) \wedge (C \in \mathbb{C} \wedge D \in \mathbb{C})) \longrightarrow$
 $((A + B) + (C + D)) = ((A + C) + (D + B))$
<proof>

lemma (in MMIsar0) MMI_add12: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $(A + (B + C)) = (B + (A + C))$
<proof>

lemma (in MMIsar0) MMI_add23: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $((A + B) + C) = ((A + C) + B)$
<proof>

lemma (in MMIsar0) MMI_add4: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$ and
 A4: $D \in \mathbb{C}$
 shows $((A + B) + (C + D)) =$
 $((A + C) + (B + D))$
<proof>

lemma (in MMIsar0) MMI_add42: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$ and
 A4: $D \in \mathbb{C}$
 shows $((A + B) + (C + D)) =$
 $((A + C) + (D + B))$
<proof>

lemma (in MMIsar0) MMI_addid2t:
 shows $A \in \mathbb{C} \longrightarrow (0 + A) = A$
<proof>

lemma (in MMIsar0) MMI_peano2cn:
 shows $A \in \mathbb{C} \longrightarrow (A + 1) \in \mathbb{C}$
<proof>

lemma (in MMIsar0) MMI_peano2re:
 shows $A \in \mathbb{R} \longrightarrow (A + 1) \in \mathbb{R}$
<proof>

lemma (in MMIsar0) MMI_negeu: assumes A1: $A \in \mathbb{C}$ and

A2: $B \in \mathbb{C}$
shows $\exists! x . x \in \mathbb{C} \wedge (A + x) = B$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_subval: **assumes** $A \in \mathbb{C} \ B \in \mathbb{C}$
shows $A - B = \bigcup \{ x \in \mathbb{C} . B + x = A \}$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_df_neg: **shows** $(- A) = \mathbf{0} - A$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_negeq:
shows $A = B \longrightarrow (-A) = (- B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_negeqi: **assumes** $A1: A = B$
shows $(- A) = (-B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_negeqd: **assumes** $A1: \varphi \longrightarrow A = B$
shows $\varphi \longrightarrow (-A) = (-B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_hbneg: **assumes** $A1: y \in A \longrightarrow (\forall x . y \in A)$
shows $y \in ((- A)) \longrightarrow (\forall x . (y \in ((- A))))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_minusex:
shows $((- A)) \text{ isASet } \langle proof \rangle$

lemma (in MMIsar0) MMI_subcl: **assumes** $A1: A \in \mathbb{C} \text{ and } A2: B \in \mathbb{C}$
shows $(A - B) \in \mathbb{C}$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_subclt:
shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow (A - B) \in \mathbb{C}$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_negclt:
 shows $A \in \mathbb{C} \longrightarrow ((- A)) \in \mathbb{C}$
<proof>

lemma (in MMIsar0) MMI_negcl: assumes A1: $A \in \mathbb{C}$
 shows $((- A)) \in \mathbb{C}$
<proof>

lemma (in MMIsar0) MMI_subadd: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $(A - B) = C \longleftrightarrow (B + C) = A$
<proof>

lemma (in MMIsar0) MMI_subsub23: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $(A - B) = C \longleftrightarrow (A - C) = B$
<proof>

lemma (in MMIsar0) MMI_subaddt:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow ((A - B) = C \longleftrightarrow (B + C) = A)$
<proof>

lemma (in MMIsar0) MMI_pncan3t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow (A + (B - A)) = B$
<proof>

lemma (in MMIsar0) MMI_pncan3: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows $(A + (B - A)) = B$
<proof>

lemma (in MMIsar0) MMI_negidt:
 shows $A \in \mathbb{C} \longrightarrow (A + ((- A))) = 0$
<proof>

lemma (in MMIsar0) MMI_negid: assumes A1: $A \in \mathbb{C}$
 shows $(A + ((- A))) = 0$
<proof>

lemma (in MMIsar0) MMI_negsub: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows $(A + ((- B))) = (A - B)$

<proof>

lemma (in MMIsar0) MMI_negsubbt:

shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow (A + (-B)) = (A - B)$

<proof>

lemma (in MMIsar0) MMI_addsubasst:

shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow ((A + B) - C) = (A + (B - C))$

<proof>

lemma (in MMIsar0) MMI_addsubbt:

shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow ((A + B) - C) = ((A - C) + B)$

<proof>

lemma (in MMIsar0) MMI_addsub12t:

shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow (A + (B - C)) = (B + (A - C))$

<proof>

lemma (in MMIsar0) MMI_addsubass: **assumes** A1: $A \in \mathbb{C}$ **and**

A2: $B \in \mathbb{C}$ **and**

A3: $C \in \mathbb{C}$

shows $((A + B) - C) = (A + (B - C))$

<proof>

lemma (in MMIsar0) MMI_addsub: **assumes** A1: $A \in \mathbb{C}$ **and**

A2: $B \in \mathbb{C}$ **and**

A3: $C \in \mathbb{C}$

shows $((A + B) - C) = ((A - C) + B)$

<proof>

lemma (in MMIsar0) MMI_2addsubbt:

shows $((A \in \mathbb{C} \wedge B \in \mathbb{C}) \wedge (C \in \mathbb{C} \wedge D \in \mathbb{C})) \longrightarrow ((A + B) + C) - D = ((A + C) - D) + B$

<proof>

lemma (in MMIsar0) MMI_negneg: **assumes** A1: $A \in \mathbb{C}$

shows $-(-A) = A$

<proof>

lemma (in MMIsar0) MMI_subid: **assumes** A1: $A \in \mathbb{C}$

shows $A - A = 0$

<proof>

lemma (in MMIsar0) MMI_subid1: **assumes** A1: $A \in \mathbb{C}$

$\text{shows } (A - 0) = A$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_negnegt:
 $\text{shows } A \in \mathbb{C} \longrightarrow (-(- A)) = A$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_subnegt:
 $\text{shows } (A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow (A - (- B)) = (A + B)$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_subidt:
 $\text{shows } A \in \mathbb{C} \longrightarrow (A - A) = 0$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_subidt:
 $\text{shows } A \in \mathbb{C} \longrightarrow (A - 0) = A$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_pncant:
 $\text{shows } (A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow ((A + B) - B) = A$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_pncan2t:
 $\text{shows } (A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow ((A + B) - A) = B$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_npcant:
 $\text{shows } (A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow ((A - B) + B) = A$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_npcant:
 $\text{shows } (A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A - B) + (B - C)) = (A - C)$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_nppcant:
 $\text{shows } (A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A - B) + C) + B = (A + C)$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_subneg: assumes A1: $A \in \mathbb{C}$ and
A2: $B \in \mathbb{C}$
 $\text{shows } (A - (- B)) = (A + B)$
 $\langle \text{proof} \rangle$

lemma (in MMIsar0) MMI_subeq0: assumes A1: $A \in \mathbb{C}$ and
A2: $B \in \mathbb{C}$

shows ($A - B$) = 0 $\longleftrightarrow$ $A = B$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_neg11: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows ($(- A)$) = ($(- B)$) $\longleftrightarrow$ $A = B$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_negcon1: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows ($(- A)$) = $B \longleftrightarrow$ ($(- B)$) = A
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_negcon2: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows $A =$ ($(- B)$) $\longleftrightarrow$ $B =$ ($(- A)$)
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_neg11t:
 shows ($A \in \mathbb{C} \wedge B \in \mathbb{C}$) $\longrightarrow$ (($(- A)$) = ($(- B)$) $\longleftrightarrow$ $A = B$)
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_negcon1t:
 shows ($A \in \mathbb{C} \wedge B \in \mathbb{C}$) $\longrightarrow$ (($(- A)$) = $B \longleftrightarrow$ ($(- B)$) = A)
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_negcon2t:
 shows ($A \in \mathbb{C} \wedge B \in \mathbb{C}$) $\longrightarrow$ ($A =$ ($(- B)$) $\longleftrightarrow$ $B =$ ($(- A)$))
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_subcant:
 shows ($A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}$) $\longrightarrow$ (($A - B$) =
 ($A - C$) $\longleftrightarrow$ $B = C$)
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_subcan2t:
 shows ($A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}$) $\longrightarrow$
 (($A - C$) = ($B - C$) $\longleftrightarrow$ $A = B$)
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_subcan: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows ($A - B$) = ($A - C$) $\longleftrightarrow$ $B = C$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_subcan2: assumes A1: $A \in \mathbb{C}$ and

```

      A2:  $B \in \mathbb{C}$  and
      A3:  $C \in \mathbb{C}$ 
    shows  $(A - C) = (B - C) \longleftrightarrow A = B$ 
  <proof>

lemma (in MMIisar0) MMI_subeq0t:
  shows  $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow ((A - B) = 0 \longleftrightarrow A = B)$ 
  <proof>

lemma (in MMIisar0) MMI_neg0:
  shows  $(- 0) = 0$ 
  <proof>

lemma (in MMIisar0) MMI_renegcl: assumes A1:  $A \in \mathbb{R}$ 
  shows  $(- A) \in \mathbb{R}$ 
  <proof>

lemma (in MMIisar0) MMI_renegclt:
  shows  $A \in \mathbb{R} \longrightarrow (- A) \in \mathbb{R}$ 
  <proof>

lemma (in MMIisar0) MMI_resubclt:
  shows  $(A \in \mathbb{R} \wedge B \in \mathbb{R}) \longrightarrow (A - B) \in \mathbb{R}$ 
  <proof>

lemma (in MMIisar0) MMI_resubcl: assumes A1:  $A \in \mathbb{R}$  and
      A2:  $B \in \mathbb{R}$ 
  shows  $(A - B) \in \mathbb{R}$ 
  <proof>

lemma (in MMIisar0) MMI_0re:
  shows  $0 \in \mathbb{R}$ 
  <proof>

lemma (in MMIisar0) MMI_mulid2t:
  shows  $A \in \mathbb{C} \longrightarrow (1 \cdot A) = A$ 
  <proof>

lemma (in MMIisar0) MMI_mul12t:
  shows  $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow (A \cdot (B \cdot C)) =$ 
     $(B \cdot (A \cdot C))$ 
  <proof>

lemma (in MMIisar0) MMI_mul23t:
  shows  $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow ((A \cdot B) \cdot C) =$ 

```

$((A \cdot C) \cdot B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mul4t:
 shows $((A \in \mathbb{C} \wedge B \in \mathbb{C}) \wedge (C \in \mathbb{C} \wedge D \in \mathbb{C})) \longrightarrow$
 $((A \cdot B) \cdot (C \cdot D)) = ((A \cdot C) \cdot (B \cdot D))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_muladdt:
 shows $((A \in \mathbb{C} \wedge B \in \mathbb{C}) \wedge (C \in \mathbb{C} \wedge D \in \mathbb{C})) \longrightarrow$
 $((A + B) \cdot (C + D)) =$
 $((A \cdot C) + (D \cdot B)) + ((A \cdot D) + (C \cdot B))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_muladd11t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow ((1 + A) \cdot (1 + B)) =$
 $((1 + A) + (B + (A \cdot B)))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mul12: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $(A \cdot (B \cdot C)) = (B \cdot (A \cdot C))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mul23: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $((A \cdot B) \cdot C) = ((A \cdot C) \cdot B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mul4: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$ and
 A4: $D \in \mathbb{C}$
 shows $((A \cdot B) \cdot (C \cdot D)) = ((A \cdot C) \cdot (B \cdot D))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_muladd: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$ and
 A4: $D \in \mathbb{C}$
 shows $((A + B) \cdot (C + D)) =$
 $((A \cdot C) + (D \cdot B)) + ((A \cdot D) + (C \cdot B))$
 $\langle proof \rangle$

```

lemma (in MMIsar0) MMI_subdit:
  shows (  $A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}$  )  $\longrightarrow$ 
    (  $A \cdot (B - C)$  ) = ( (  $A \cdot B$  ) - (  $A \cdot C$  ) )
  <proof>

lemma (in MMIsar0) MMI_subdirt:
  shows (  $A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}$  )  $\longrightarrow$ 
    ( (  $A - B$  )  $\cdot C$  ) = ( (  $A \cdot C$  ) - (  $B \cdot C$  ) )
  <proof>

lemma (in MMIsar0) MMI_subdi: assumes A1:  $A \in \mathbb{C}$  and
  A2:  $B \in \mathbb{C}$  and
  A3:  $C \in \mathbb{C}$ 
  shows (  $A \cdot (B - C)$  ) = ( (  $A \cdot B$  ) - (  $A \cdot C$  ) )
  <proof>

lemma (in MMIsar0) MMI_subdir: assumes A1:  $A \in \mathbb{C}$  and
  A2:  $B \in \mathbb{C}$  and
  A3:  $C \in \mathbb{C}$ 
  shows ( (  $A - B$  )  $\cdot C$  ) = ( (  $A \cdot C$  ) - (  $B \cdot C$  ) )
  <proof>

lemma (in MMIsar0) MMI_mul01: assumes A1:  $A \in \mathbb{C}$ 
  shows (  $A \cdot 0$  ) = 0
  <proof>

lemma (in MMIsar0) MMI_mul02: assumes A1:  $A \in \mathbb{C}$ 
  shows (  $0 \cdot A$  ) = 0
  <proof>

lemma (in MMIsar0) MMI_1ptimes: assumes A1:  $A \in \mathbb{C}$ 
  shows ( (  $1 + 1$  )  $\cdot A$  ) = (  $A + A$  )
  <proof>

lemma (in MMIsar0) MMI_mul01t:
  shows  $A \in \mathbb{C} \longrightarrow (A \cdot 0) = 0$ 
  <proof>

lemma (in MMIsar0) MMI_mul02t:
  shows  $A \in \mathbb{C} \longrightarrow (0 \cdot A) = 0$ 
  <proof>

lemma (in MMIsar0) MMI_mulneg1: assumes A1:  $A \in \mathbb{C}$  and
  A2:  $B \in \mathbb{C}$ 
  shows ( (  $- A$  )  $\cdot B$  ) = ( - (  $A \cdot B$  ) )
  <proof>

```


lemma (in MMIsar0) MMI_mulneg2: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows $(A \cdot (-B)) =$
 $(-(A \cdot B))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mul2neg: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows $((-A) \cdot (-B)) =$
 $(A \cdot B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_negdi: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows $(-(A + B)) =$
 $((-A) + (-B))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_negsubdi: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows $(-(A - B)) =$
 $((-A) + B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_negsubdi2: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows $(-(A - B)) = (B - A)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mulneg1t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow$
 $((-A) \cdot B) =$
 $(-(A \cdot B))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mulneg2t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow$
 $(A \cdot (-B)) =$
 $(-(A \cdot B))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mulneg12t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow$
 $((-A) \cdot B) =$
 $(A \cdot (-B))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mul2negt:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow$

$$((- A) \cdot (- B)) = (A \cdot B)$$
<proof>

lemma (in MMIisar0) MMI_negdit:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow$
 $(- (A + B)) =$
 $((- A) + (- B))$
<proof>

lemma (in MMIisar0) MMI_negdi2t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow$
 $(- (A + B)) = ((- A) - B)$
<proof>

lemma (in MMIisar0) MMI_negsubdit:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow$
 $(- (A - B)) = ((- A) + B)$
<proof>

lemma (in MMIisar0) MMI_negsubdi2t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow$
 $(- (A - B)) = (B - A)$
<proof>

lemma (in MMIisar0) MMI_subsub2t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $(A - (B - C)) = (A + (C - B))$
<proof>

lemma (in MMIisar0) MMI_subsubt:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $(A - (B - C)) = ((A - B) + C)$
<proof>

lemma (in MMIisar0) MMI_subsub3t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $(A - (B - C)) = ((A + C) - B)$
<proof>

lemma (in MMIisar0) MMI_subsub4t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A - B) - C) = (A - (B + C))$
<proof>

lemma (in MMIisar0) MMI_sub23t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$

$((A - B) - C) = ((A - C) - B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_nnncant:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A - (B - C)) - C) = (A - B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_nnncan1t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A - B) - (A - C)) = (C - B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_nnncan2t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A - C) - (B - C)) = (A - B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_nncant:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C}) \longrightarrow$
 $(A - (A - B)) = B$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_nppcan2t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A - (B + C)) + C) = (A - B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mulm1t:
 shows $A \in \mathbb{C} \longrightarrow ((-1) \cdot A) = (-A)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mulm1: assumes A1: $A \in \mathbb{C}$
 shows $((-1) \cdot A) = (-A)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_sub4t:
 shows $((A \in \mathbb{C} \wedge B \in \mathbb{C}) \wedge (C \in \mathbb{C} \wedge D \in \mathbb{C})) \longrightarrow$
 $((A + B) - (C + D)) =$
 $((A - C) + (B - D))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_sub4: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$ and
 A4: $D \in \mathbb{C}$
 shows $((A + B) - (C + D)) =$

$((A - C) + (B - D))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mulsbt:
 shows $((A \in \mathbb{C} \wedge B \in \mathbb{C}) \wedge (C \in \mathbb{C} \wedge D \in \mathbb{C})) \longrightarrow$
 $((A - B) \cdot (C - D)) =$
 $((A \cdot C) + (D \cdot B)) - ((A \cdot D) + (C \cdot B))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_pnpccant:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A + B) - (A + C)) = (B - C)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_pnpccan2t:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A + C) - (B + C)) = (A - B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_pnnccant:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A + B) - (A - C)) = (B + C)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_ppnccant:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A + B) + (C - B)) = (A + C)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_pnnccan: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $((A + B) - (A - C)) = (B + C)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mulcan: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$ and
 A4: $A \neq 0$
 shows $(A \cdot B) = (A \cdot C) \longleftrightarrow B = C$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mulcant2: assumes A1: $A \neq 0$
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \longrightarrow$
 $((A \cdot B) = (A \cdot C) \longleftrightarrow B = C)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_mulcant:

shows ((A ∈ $\mathbb{C}$ ∧ B ∈ $\mathbb{C}$ ∧ C ∈ $\mathbb{C}$) ∧ A ≠ 0) →
 ((A · B) = (A · C) ↔ B = C)
<proof>

lemma (in MMIsar0) MMI_mulcan2t:
 shows ((A ∈ $\mathbb{C}$ ∧ B ∈ $\mathbb{C}$ ∧ C ∈ $\mathbb{C}$) ∧ C ≠ 0) →
 ((A · C) = (B · C) ↔ A = B)
<proof>

lemma (in MMIsar0) MMI_mul0or: assumes A1: A ∈ $\mathbb{C}$ and
 A2: B ∈ $\mathbb{C}$
 shows (A · B) = 0 ↔ (A = 0 ∨ B = 0)
<proof>

lemma (in MMIsar0) MMI_msq0: assumes A1: A ∈ $\mathbb{C}$
 shows (A · A) = 0 ↔ A = 0
<proof>

lemma (in MMIsar0) MMI_mul0ort:
 shows (A ∈ $\mathbb{C}$ ∧ B ∈ $\mathbb{C}$) →
 ((A · B) = 0 ↔ (A = 0 ∨ B = 0))
<proof>

lemma (in MMIsar0) MMI_muln0bt:
 shows (A ∈ $\mathbb{C}$ ∧ B ∈ $\mathbb{C}$) →
 ((A ≠ 0 ∧ B ≠ 0) ↔ (A · B) ≠ 0)
<proof>

lemma (in MMIsar0) MMI_muln0: assumes A1: A ∈ $\mathbb{C}$ and
 A2: B ∈ $\mathbb{C}$ and
 A3: A ≠ 0 and
 A4: B ≠ 0
 shows (A · B) ≠ 0
<proof>

lemma (in MMIsar0) MMI_receu: assumes A1: A ∈ $\mathbb{C}$ and
 A2: B ∈ $\mathbb{C}$ and
 A3: A ≠ 0
 shows ∃! x . x ∈ $\mathbb{C}$ ∧ (A · x) = B
<proof>

lemma (in MMIsar0) MMI_divval: assumes A ∈ $\mathbb{C}$ B ∈ $\mathbb{C}$ B ≠ 0
 shows A / B = ∪ { x ∈ $\mathbb{C}$. B · x = A }
<proof>

lemma (in MMIisar0) MMI_divmul: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$ and
 A4: $B \neq 0$
 shows $(A / B) = C \iff (B \cdot C) = A$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_divmulz: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $B \neq 0 \implies$
 $((A / B) = C \iff (B \cdot C) = A)$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_divmult:
 shows $((A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \wedge B \neq 0) \implies$
 $((A / B) = C \iff (B \cdot C) = A)$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_divmul2t:
 shows $((A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \wedge B \neq 0) \implies$
 $((A / B) = C \iff A = (B \cdot C))$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_divmul3t:
 shows $((A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \wedge B \neq 0) \implies$
 $((A / B) = C \iff A = (C \cdot B))$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_divcl: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $B \neq 0$
 shows $(A / B) \in \mathbb{C}$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_divclz: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows $B \neq 0 \implies (A / B) \in \mathbb{C}$
 $\langle proof \rangle$

lemma (in MMIisar0) MMI_divclt:
 shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge B \neq 0) \implies$
 $(A / B) \in \mathbb{C}$

<proof>

lemma (in MMIsar0) MMI_reccl: assumes A1: $A \in \mathbb{C}$ and
A2: $A \neq 0$
shows $(1 / A) \in \mathbb{C}$
<proof>

lemma (in MMIsar0) MMI_recclz: assumes A1: $A \in \mathbb{C}$
shows $A \neq 0 \longrightarrow (1 / A) \in \mathbb{C}$
<proof>

lemma (in MMIsar0) MMI_recclt:
shows $(A \in \mathbb{C} \wedge A \neq 0) \longrightarrow (1 / A) \in \mathbb{C}$
<proof>

lemma (in MMIsar0) MMI_divcan2: assumes A1: $A \in \mathbb{C}$ and
A2: $B \in \mathbb{C}$ and
A3: $A \neq 0$
shows $(A \cdot (B / A)) = B$
<proof>

lemma (in MMIsar0) MMI_divcan1: assumes A1: $A \in \mathbb{C}$ and
A2: $B \in \mathbb{C}$ and
A3: $A \neq 0$
shows $((B / A) \cdot A) = B$
<proof>

lemma (in MMIsar0) MMI_divcan1z: assumes A1: $A \in \mathbb{C}$ and
A2: $B \in \mathbb{C}$
shows $A \neq 0 \longrightarrow ((B / A) \cdot A) = B$
<proof>

lemma (in MMIsar0) MMI_divcan2z: assumes A1: $A \in \mathbb{C}$ and
A2: $B \in \mathbb{C}$
shows $A \neq 0 \longrightarrow (A \cdot (B / A)) = B$
<proof>

lemma (in MMIsar0) MMI_divcan1t:
shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge A \neq 0) \longrightarrow$
 $((B / A) \cdot A) = B$
<proof>

lemma (in MMIsar0) MMI_divcan2t:
shows $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge A \neq 0) \longrightarrow$
 $(A \cdot (B / A)) = B$
<proof>

```

lemma (in MMIisar0) MMI_divne0bt:
  shows (  $A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge B \neq 0$  )  $\longrightarrow$ 
    (  $A \neq 0 \longleftrightarrow (A / B) \neq 0$  )
  <proof>

lemma (in MMIisar0) MMI_divne0: assumes A1:  $A \in \mathbb{C}$  and
  A2:  $B \in \mathbb{C}$  and
  A3:  $A \neq 0$  and
  A4:  $B \neq 0$ 
  shows (  $A / B$  )  $\neq 0$ 
  <proof>

lemma (in MMIisar0) MMI_recne0z: assumes A1:  $A \in \mathbb{C}$ 
  shows  $A \neq 0 \longrightarrow (1 / A) \neq 0$ 
  <proof>

lemma (in MMIisar0) MMI_recne0t:
  shows (  $A \in \mathbb{C} \wedge A \neq 0$  )  $\longrightarrow (1 / A) \neq 0$ 
  <proof>

lemma (in MMIisar0) MMI_recid: assumes A1:  $A \in \mathbb{C}$  and
  A2:  $A \neq 0$ 
  shows (  $A \cdot (1 / A)$  ) = 1
  <proof>

lemma (in MMIisar0) MMI_recidz: assumes A1:  $A \in \mathbb{C}$ 
  shows  $A \neq 0 \longrightarrow (A \cdot (1 / A)) = 1$ 
  <proof>

lemma (in MMIisar0) MMI_recidt:
  shows (  $A \in \mathbb{C} \wedge A \neq 0$  )  $\longrightarrow$ 
    (  $A \cdot (1 / A)$  ) = 1
  <proof>

lemma (in MMIisar0) MMI_recid2t:
  shows (  $A \in \mathbb{C} \wedge A \neq 0$  )  $\longrightarrow$ 
    ( (  $1 / A$  )  $\cdot A$  ) = 1
  <proof>

lemma (in MMIisar0) MMI_divrec: assumes A1:  $A \in \mathbb{C}$  and
  A2:  $B \in \mathbb{C}$  and
  A3:  $B \neq 0$ 
  shows (  $A / B$  ) = (  $A \cdot (1 / B)$  )
  <proof>

lemma (in MMIisar0) MMI_divrecz: assumes A1:  $A \in \mathbb{C}$  and
  A2:  $B \in \mathbb{C}$ 
  shows  $B \neq 0 \longrightarrow (A / B) = (A \cdot (1 / B))$ 
  <proof>

```



```

lemma (in MMIisar0) MMI_divirect:
  shows (  $A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge B \neq 0$  )  $\longrightarrow$ 
    (  $A / B$  ) = (  $A \cdot (1 / B)$  )
  <proof>

lemma (in MMIisar0) MMI_divrec2t:
  shows (  $A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge B \neq 0$  )  $\longrightarrow$ 
    (  $A / B$  ) = ( (  $1 / B$  )  $\cdot A$  )
  <proof>

lemma (in MMIisar0) MMI_divasst:
  shows ( (  $A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}$  )  $\wedge C \neq 0$  )  $\longrightarrow$ 
    ( (  $A \cdot B$  ) /  $C$  ) = (  $A \cdot (B / C)$  )
  <proof>

lemma (in MMIisar0) MMI_div23t:
  shows ( (  $A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}$  )  $\wedge C \neq 0$  )  $\longrightarrow$ 
    ( (  $A \cdot B$  ) /  $C$  ) = ( (  $A / C$  )  $\cdot B$  )
  <proof>

lemma (in MMIisar0) MMI_div13t:
  shows ( (  $A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}$  )  $\wedge B \neq 0$  )  $\longrightarrow$ 
    ( (  $A / B$  )  $\cdot C$  ) = ( (  $C / B$  )  $\cdot A$  )
  <proof>

lemma (in MMIisar0) MMI_div12t:
  shows ( (  $A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}$  )  $\wedge C \neq 0$  )  $\longrightarrow$ 
    (  $A \cdot (B / C)$  ) = (  $B \cdot (A / C)$  )
  <proof>

lemma (in MMIisar0) MMI_divassz: assumes A1:  $A \in \mathbb{C}$  and
  A2:  $B \in \mathbb{C}$  and
  A3:  $C \in \mathbb{C}$ 
  shows  $C \neq 0 \longrightarrow$ 
    ( (  $A \cdot B$  ) /  $C$  ) = (  $A \cdot (B / C)$  )
  <proof>

lemma (in MMIisar0) MMI_divass: assumes A1:  $A \in \mathbb{C}$  and
  A2:  $B \in \mathbb{C}$  and
  A3:  $C \in \mathbb{C}$  and
  A4:  $C \neq 0$ 
  shows ( (  $A \cdot B$  ) /  $C$  ) = (  $A \cdot (B / C)$  )
  <proof>

lemma (in MMIisar0) MMI_divdir: assumes A1:  $A \in \mathbb{C}$  and
  A2:  $B \in \mathbb{C}$  and

```

A3: $C \in \mathbb{C}$ and
 A4: $C \neq 0$
 shows $((A + B) / C) =$
 $((A / C) + (B / C))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_div23: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$ and
 A4: $C \neq 0$
 shows $((A \cdot B) / C) = ((A / C) \cdot B)$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_divdirz: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $C \in \mathbb{C}$
 shows $C \neq 0 \longrightarrow$
 $((A + B) / C) =$
 $((A / C) + (B / C))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_divdirt:
 shows $((A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge C \in \mathbb{C}) \wedge C \neq 0) \longrightarrow$
 $((A + B) / C) =$
 $((A / C) + (B / C))$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_divcan3: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $A \neq 0$
 shows $((A \cdot B) / A) = B$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_divcan4: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$ and
 A3: $A \neq 0$
 shows $((B \cdot A) / A) = B$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_divcan3z: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$
 shows $A \neq 0 \longrightarrow ((A \cdot B) / A) = B$
 $\langle proof \rangle$

lemma (in MMIsar0) MMI_divcan4z: assumes A1: $A \in \mathbb{C}$ and
 A2: $B \in \mathbb{C}$

```

    shows  $A \neq 0 \longrightarrow ((B \cdot A) / A) = B$ 
  <proof>

lemma (in MMIisar0) MMI_divcan3t:
  shows  $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge A \neq 0) \longrightarrow$ 
     $((A \cdot B) / A) = B$ 
  <proof>

lemma (in MMIisar0) MMI_divcan4t:
  shows  $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge A \neq 0) \longrightarrow$ 
     $((B \cdot A) / A) = B$ 
  <proof>

lemma (in MMIisar0) MMI_div11: assumes A1:  $A \in \mathbb{C}$  and
  A2:  $B \in \mathbb{C}$  and
  A3:  $C \in \mathbb{C}$  and
  A4:  $C \neq 0$ 
  shows  $(A / C) = (B / C) \longleftrightarrow A = B$ 
  <proof>

lemma (in MMIisar0) MMI_div11t:
  shows  $(A \in \mathbb{C} \wedge B \in \mathbb{C} \wedge (C \in \mathbb{C} \wedge C \neq 0)) \longrightarrow$ 
     $((A / C) = (B / C) \longleftrightarrow A = B)$ 
  <proof>

end

```

112 Metamath examples

```
theory MMI_examples imports MMI_Complex_ZF
```

```
begin
```

This theory contains 10 theorems translated from Metamath (with proofs). It is included in the proof document as an illustration of how a translated Metamath proof looks like. The "known_theorems.txt" file included in the IsarMathLib distribution provides a list of all translated facts.

```

lemma (in MMIisar0) MMI_dividt:
  shows  $(A \in \mathbb{C} \wedge A \neq 0) \longrightarrow (A / A) = 1$ 
  <proof>

lemma (in MMIisar0) MMI_div0t:
  shows  $(A \in \mathbb{C} \wedge A \neq 0) \longrightarrow (0 / A) = 0$ 
  <proof>

lemma (in MMIisar0) MMI_diveq0t:
  shows  $(A \in \mathbb{C} \wedge C \in \mathbb{C} \wedge C \neq 0) \longrightarrow$ 

```

```

    ( ( A / C ) = 0  $\longleftrightarrow$  A = 0 )
  <proof>

lemma (in MMIisar0) MMI_recrec: assumes A1: A  $\in$   $\mathbb{C}$  and
    A2: A  $\neq$  0
  shows ( 1 / ( 1 / A ) ) = A
  <proof>

lemma (in MMIisar0) MMI_divid: assumes A1: A  $\in$   $\mathbb{C}$  and
    A2: A  $\neq$  0
  shows ( A / A ) = 1
  <proof>

lemma (in MMIisar0) MMI_div0: assumes A1: A  $\in$   $\mathbb{C}$  and
    A2: A  $\neq$  0
  shows ( 0 / A ) = 0
  <proof>

lemma (in MMIisar0) MMI_div1: assumes A1: A  $\in$   $\mathbb{C}$ 
  shows ( A / 1 ) = A
  <proof>

lemma (in MMIisar0) MMI_div1t:
  shows A  $\in$   $\mathbb{C} \longrightarrow$  ( A / 1 ) = A
  <proof>

lemma (in MMIisar0) MMI_divnegt:
  shows ( A  $\in$   $\mathbb{C} \wedge$  B  $\in$   $\mathbb{C} \wedge$  B  $\neq$  0 )  $\longrightarrow$ 
    ( - ( A / B ) ) = ( ( - A ) / B )
  <proof>

lemma (in MMIisar0) MMI_divsubdirt:
  shows ( ( A  $\in$   $\mathbb{C} \wedge$  B  $\in$   $\mathbb{C} \wedge$  C  $\in$   $\mathbb{C}$  )  $\wedge$  C  $\neq$  0 )  $\longrightarrow$ 
    ( ( A - B ) / C ) =
    ( ( A / C ) - ( B / C ) )
  <proof>

end

```

113 Metamath interface

```
theory Metamath_Interface imports Complex_ZF MMI_prelude
```

```
begin
```

This theory contains some lemmas that make it possible to use the theorems translated from Metamath in a the `complex0` context.

113.1 MMIsar0 and complex0 contexts.

In the section we show a lemma that the assumptions in `complex0` context imply the assumptions of the `MMIsar0` context. The `Metamath_sampler` theory provides examples how this lemma can be used.

The next lemma states that we can use the theorems proven in the `MMIsar0` context in the `complex0` context. Unfortunately we have to use low level Isabelle methods "rule" and "unfold" in the proof, simp and blast fail on the order axioms.

```
lemma (in complex0) MMIsar_valid:
  shows MMIsar0(R,C,1,0,i,CplxAdd(R,A),CplxMul(R,A,M),
    StrictVersion(CplxROrder(R,A,r)))
  <proof>
```

end

114 Metamath sampler

```
theory Metamath_Sampler imports Metamath_Interface MMI_Complex_ZF_2
```

begin

The theorems translated from Metamath reside in the `MMI_Complex_ZF`, `MMI_Complex_ZF_1` and `MMI_Complex_ZF_2` theories. The proofs of these theorems are very verbose and for this reason the theories are not shown in the proof document or the FormaMath.org site. This theory file contains some examples of theorems translated from Metamath and formulated in the `complex0` context. This serves two purposes: to give an overview of the material covered in the translated theorems and to provide examples of how to take a translated theorem (proven in the `MMIsar0` context) and transfer it to the `complex0` context. The typical procedure for moving a theorem from `MMIsar0` to `complex0` is as follows: First we define certain aliases that map names defined in the `complex0` to their corresponding names in the `MMIsar0` context. This makes it easy to copy and paste the statement of the theorem as displayed with ProofGeneral. Then we run the Isabelle from ProofGeneral up to the theorem we want to move. When the theorem is verified ProofGeneral displays the statement in the raw set theory notation, stripped from any notation defined in the `MMIsar0` locale. This is what we copy to the proof in the `complex0` locale. After that we just can write "then have ?thesis by simp" and the simplifier translates the raw set theory notation to the one used in `complex0`.

114.1 Extended reals and order

In this section we import a couple of theorems about the extended real line and the linear order on it.

Metamath uses the set of real numbers extended with $+\infty$ and $-\infty$. The $+\infty$ and $-\infty$ symbols are defined quite arbitrarily as $\mathbb{C}$ and $\{\mathbb{C}\}$, respectively. The next lemma that corresponds to Metamath's `renfdisj` states that $+\infty$ and $-\infty$ are not elements of $\mathbb{R}$.

lemma (in complex0) `renfdisj`: shows $\mathbb{R} \cap \{+\infty, -\infty\} = \emptyset$
<proof>

The order relation used most often in Metamath is defined on the set of complex reals extended with $+\infty$ and $-\infty$. The next lemma allows to use Metamath's `xrltso` that states that the $<$ relations is a strict linear order on the extended set.

lemma (in complex0) `xrltso`: shows $<$ Orders $\mathbb{R}^*$
<proof>

Metamath defines the usual $<$ and $\leq$ ordering relations for the extended real line, including $+\infty$ and $-\infty$.

lemma (in complex0) `xrrebndt`: assumes A1: $x \in \mathbb{R}^*$
 shows $x \in \mathbb{R} \iff (-\infty < x \wedge x < +\infty)$
<proof>

A quite involved inequality.

lemma (in complex0) `lt2mul2divt`:
 assumes A1: $a \in \mathbb{R} \quad b \in \mathbb{R} \quad c \in \mathbb{R} \quad d \in \mathbb{R}$ and
 A2: $0 < b \quad 0 < d$
 shows $a \cdot b < c \cdot d \iff a/d < c/b$
<proof>

A real number is smaller than its half iff it is positive.

lemma (in complex0) `halfpos`: assumes A1: $a \in \mathbb{R}$
 shows $0 < a \iff a/2 < a$
<proof>

One more inequality.

lemma (in complex0) `ledivp1t`:
 assumes A1: $a \in \mathbb{R} \quad b \in \mathbb{R}$ and
 A2: $0 \leq a \quad 0 \leq b$
 shows $(a/(b + 1)) \cdot b \leq a$
<proof>

114.2 Natural real numbers

In standard mathematics natural numbers are treated as a subset of real numbers. From the set theory point of view however those are quite differ-

ent objects. In this section we talk about "real natural" numbers i.e. the counterpart of natural numbers that is a subset of the reals.

Two ways of saying that there are no natural numbers between n and $n + 1$.

lemma (in complex0) no_nats_between:

assumes A1: $n \in \mathbb{N}$ $k \in \mathbb{N}$

shows

$n \leq k \longleftrightarrow n < k+1$

$n < k \longleftrightarrow n + 1 \leq k$

<proof>

Metamath has some very complicated and general version of induction on (complex) natural numbers that I can't even understand. As an exercise I derived a more standard version that is imported to the complex0 context below.

lemma (in complex0) cplx_nat_ind: assumes A1: $\psi(1)$ and

A2: $\forall k \in \mathbb{N}. \psi(k) \longrightarrow \psi(k+1)$ and

A3: $n \in \mathbb{N}$

shows $\psi(n)$

<proof>

Some simple arithmetics.

lemma (in complex0) arith: shows

$2 + 2 = 4$

$2 \cdot 2 = 4$

$3 \cdot 2 = 6$

$3 \cdot 3 = 9$

<proof>

114.3 Infimum and supremum in real numbers

Real numbers form a complete ordered field. Here we import a couple of Metamath theorems about supremu and infimum.

If a set S has a smallest element, then the infimum of S belongs to it.

lemma (in complex0) lbinfmcl: assumes A1: $S \subseteq \mathbb{R}$ and

A2: $\exists x \in S. \forall y \in S. x \leq y$

shows $\text{Infim}(S, \mathbb{R}, <) \in S$

<proof>

Supremum of any subset of reals that is bounded above is real.

lemma (in complex0) sup_is_real:

assumes $A \subseteq \mathbb{R}$ and $A \neq \emptyset$ and $\exists x \in \mathbb{R}. \forall y \in A. y \leq x$

shows $\text{Sup}(A, \mathbb{R}, <) \in \mathbb{R}$

<proof>

If a real number is smaller than the supremum of A , then we can find an element of A greater than it.

```

lemma (in complex0) suprlub:
  assumes  $A \subseteq \mathbb{R}$  and  $A \neq 0$  and  $\exists x \in \mathbb{R}. \forall y \in A. y \leq x$ 
  and  $B \in \mathbb{R}$  and  $B < \text{Sup}(A, \mathbb{R}, <)$ 
  shows  $\exists z \in A. B < z$ 
<proof>

```

Something a bit more interesting: infimum of a set that is bounded below is real and equal to the minus supremum of the set flipped around zero.

```

lemma (in complex0) infmsup:
  assumes  $A \subseteq \mathbb{R}$  and  $A \neq 0$  and  $\exists x \in \mathbb{R}. \forall y \in A. x \leq y$ 
  shows
     $\text{Infim}(A, \mathbb{R}, <) \in \mathbb{R}$ 
     $\text{Infim}(A, \mathbb{R}, <) = ( -\text{Sup}(\{z \in \mathbb{R}. (-z) \in A\}, \mathbb{R}, <) )$ 
<proof>

end

```

References

- [1] N. A’Campo. A natural construction for the real numbers. 2003.
- [2] R. D. Arthan. The Eudoxus Real Numbers. 2004.
- [3] R. Street et al. The Efficient Real Numbers. 2003.
- [4] Strecker G.E. Herrlich H. When is $\mathbb{N}$ lindelöf? *Comment. Math. Univ. Carolinae*, 1997.
- [5] I. L. Reilly and M. K. Vamanamurthy. Some topological anti-properties. *Illinois J. Math.*, 24:382–389, 1980.
- [6] D. Zelinski. On Ordered Loops. *Amer. J. Math.*, 70(4):681–697, Oct. 1948.